Mapa completo do PINNeAPPle
Volumes
Visão geral
Como este documento foi feito

Volume 1 — Núcleo, porta de entrada e catálogo
Sumário
1. pinneapple
2. pinneapple_core
3. pinneapple_catalog
4. pinneapple_registry
5. pinneapple_orchestration
6. pinneapple_models
7. pinneapple_train
8. pinneapple_solvers

Volume 2 — Física: problemas, equações e ciência
Sumário
1. pinneapple_physics
2. pinneapple_presets
3. pinneapple_problemdesign
4. pinneapple_science
5. pinneapple_condense
6. pinneapple_query

Volume 3 — Simulação numérica
Sumário
1. pinneapple_simulation

Volume 4 — Aprendizado de máquina e dados
Sumário
1. pinneapple_neural
2. pinneapple_data
3. pinneapple_pdb
4. pinneapple_adaptation
5. pinneapple_arena
6. pinneapple_worldmodel
7. pinneapple_perception
8. pinneapple_quantum

Volume 5 — Confiança, validação e decisão
Sumário
1. pinneapple_analysis
2. pinneapple_check
3. pinneapple_veriphysics
4. pinneapple_security
5. pinneapple_decision
6. pinneapple_reason

Volume 6 — Projeto e sistemas
Sumário
1. pinneapple_design
2. pinneapple_systems
3. pinneapple_twin3d

Volume 7 — Ferramentas, orquestração e produtos
Sumário
1. pinneapple_tools
2. pinneapple_blender
3. pinneapple_flow
4. pinneapple_hub
5. pinneapple_lab
6. pinneapple_llm
7. pinneapple_mcp
8. pinneapple_autolab
9. pinneapple_app

Volume 8 — Apps, exemplos, reproduções, scripts e CI
1. Apps (`apps/`)
2. Exemplos (`examples/`)
3. Reproduções de papers
4. Scripts (`scripts/`)
5. Benchmarks (`benchmarks/`)
6. Integração contínua (`.github/workflows/`)
7. Documentação existente (`docs/`)

Volume 9 — Índice alfabético, glossário e apêndice numérico
1. Índice alfabético
2. Glossário
3. Apêndice: números do inventário

Mapa completo do PINNeAPPle

Documento de referência, em português, de todo o repositório: 41 pacotes, 2.142 módulos, 2.757 classes e 6.130 funções públicas. Gerado a partir do inventário automático do commit `e53cdcfb04`.

Como o texto passa de 150 páginas, ele está dividido em volumes por camada. Este arquivo é o volume 0: visão geral, arquitetura e sumário.

Volumes

volume	conteúdo	pacotes
Volume 1 — Núcleo, porta de entrada e catálogo	A porta de entrada `pp.*`, os primitivos do núcleo, o catálogo de métodos, o registro de artefatos e os shims de compatibilidade.	`pinneapple`, `pinneapple_core`, `pinneapple_catalog`, `pinneapple_registry`, `pinneapple_orchestration`, `pinneapple_models`, `pinneapple_train`, `pinneapple_solvers`
Volume 2 — Física: problemas, equações e ciência	Como um problema físico é descrito e resolvido (`PhysicalProblem`, `pp.solve`), as equações, os presets industriais, o desenho de problemas, os experimentos por área, a condensação e as consultas físicas.	`pinneapple_physics`, `pinneapple_presets`, `pinneapple_problemdesign`, `pinneapple_science`, `pinneapple_condense`, `pinneapple_query`
Volume 3 — Simulação numérica	Solvers clássicos (FDM, FEM, FVM, espectral, SPH, LBM), pontes para solvers externos, eletromagnetismo, voo, multicorpos, veículos e rovers.	`pinneapple_simulation`
Volume 4 — Aprendizado de máquina e dados	Arquiteturas e treino de redes (PINN, operadores neurais, grafos), dados e amostragem, adaptação, modelos de mundo, percepção, computação quântica e a arena de benchmarks.	`pinneapple_neural`, `pinneapple_data`, `pinneapple_pdb`, `pinneapple_adaptation`, `pinneapple_arena`, `pinneapple_worldmodel`, `pinneapple_perception`, `pinneapple_quantum`
Volume 5 — Confiança, validação e decisão	A camada que confere cada resultado contra a física: validação, incerteza, inversão, o linter `pinneapple check`, registros de evidência, segurança, decisão e a cadeia de porquês.	`pinneapple_analysis`, `pinneapple_check`, `pinneapple_veriphysics`, `pinneapple_security`, `pinneapple_decision`, `pinneapple_reason`

volume	conteúdo	pacotes
Volume 6 — Projeto e sistemas	Geometria, otimização de projeto, projeto a partir de intenção, sistemas dinâmicos, séries temporais, co-simulação e gêmeos digitais.	`pinneapple_design`, `pinneapple_systems`, `pinneapple_twin3d`
Volume 7 — Ferramentas, orquestração e produtos	Ferramentas (benchmarks, exportação, visualização, automação de GUIs de engenharia, NVIDIA), o Flow, o Hub, o Lab, os LLMs, o servidor MCP, o laboratório autônomo e o app web.	`pinneapple_tools`, `pinneapple_blender`, `pinneapple_flow`, `pinneapple_hub`, `pinneapple_lab`, `pinneapple_llm`, `pinneapple_mcp`, `pinneapple_autolab`, `pinneapple_app`
Volume 8 — Apps, exemplos, reproduções, scripts e CI	64 apps, 88 entradas de exemplos, reproduções de papers, scripts, benchmarks e 12 workflows de CI.	—
Volume 9 — Índice alfabético, glossário e apêndice numérico	Índice de todas as classes e funções públicas, glossário e números.	—

Visão geral

O que é o PINNeAPPle

O PINNeAPPle é uma biblioteca Python de IA para física. A ideia central tem duas partes:

1. **Uma descrição, muitos métodos.** O problema físico é descrito uma vez (`pp.PhysicalProblem`, ou o nome de um preset). Depois ele é resolvido por redes informadas pela física (PINN), operadores neurais, redes de grafos ou solvers clássicos (diferenças finitas, elementos finitos, pontes para OpenFOAM e CalculiX), sempre pela mesma chamada: `pp.solve(problema, método)`.
2. **Uma camada de confiança.** Antes de alguém usar um resultado, ele é conferido contra a física: resíduo da equação, conservação, condições de contorno, comparação com referências e domínio de validade. A saída é uma decisão explícita (`APPROVED` / `REVIEW` / `REJECT` no `pp.trust_card`; `pass` / `warn` / `fail` no `pp.validate`).

Em volta desse núcleo há camadas de projeto (geometria, otimização, projeto a partir de um texto), sistemas (séries temporais, co-simulação, gêmeos digitais), orquestração (Flow, Lab, laboratório autônomo, LLMs, servidor MCP) e produtos (Hub, apps web). O pacote de topo `pinneapple` (importado como `pp`) é uma camada fina: cada nome dele delega para um dos pacotes `pinneapple_*`.

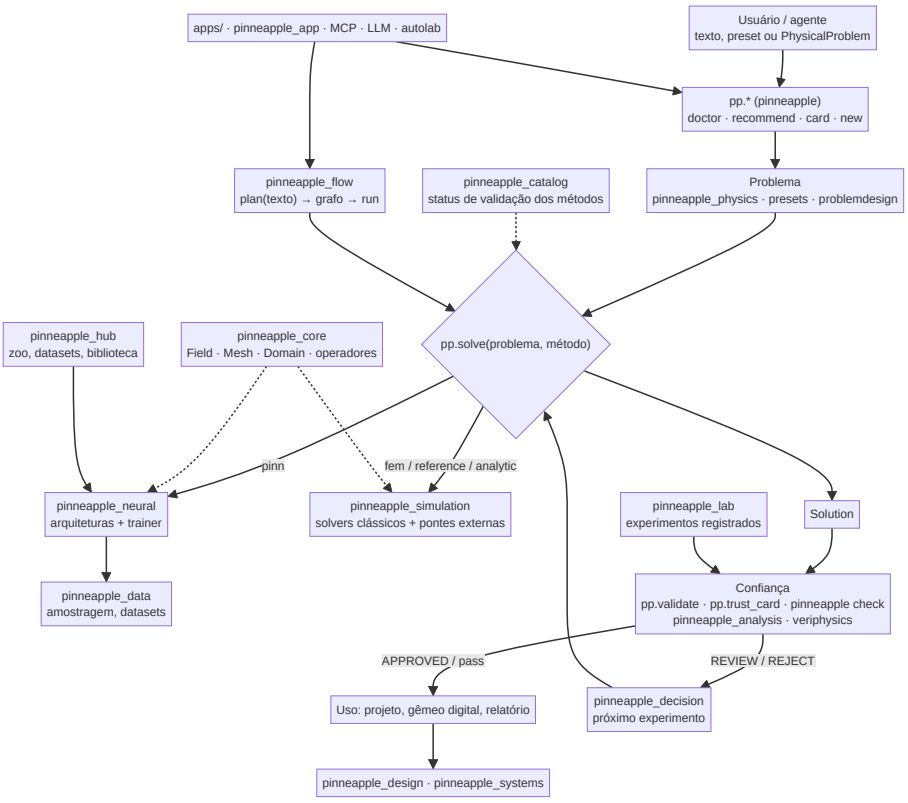
Arquitetura em camadas

A tabela resume as camadas na ordem em que um problema passa por elas. Cada camada depende só da interface da anterior (um `ProblemSpec`, coordenadas amostradas, a função de perda, um modelo treinado), e não de herança ou de um objeto compartilhado. Por isso dá para trocar uma peça sem mexer nas outras.

camada	pacotes principais	papel
Porta de entrada	`pinneapple` (`pp.*`), `pinneapple_core.onboarding`	`pp.doctor`, `pp.recommend`, `pp.card`, `pp.new`, `pp.solve`, `pp.compare`, `pp.validate`
Primitivos	`pinneapple_core`	`Field`, `Mesh`, `Domain`, `Geometry`, operadores diferenciais, motor de workflows
Problema	`pinneapple_physics` (`physical_problem`, `pde_environment`, `symbolic_pde`), `pinneapple_presets`, `pinneapple_problemdesign`	descrever o problema: equações, contornos, presets, desenho guiado
Solução clássica	`pinneapple_simulation`, `pinneapple_condense`	FDM, FEM, FVM, SPH, LBM, pontes externas, eletromagnetismo, voo, veículos, condensação
Aprendizado	`pinneapple_neural`, `pinneapple_data`, `pinneapple_adaptation`, `pinneapple_worldmodel`, `pinneapple_quantum`	arquiteturas, treino, dados, transferência, modelos de mundo
Confiança	`pinneapple_analysis`, `pinneapple_check`, `pinneapple_veriphysics`, `pinneapple_security`	validação, incerteza, inversão, linter de física em CI, registros de evidência, assinaturas
Decisão e raciocínio	`pinneapple_decision`, `pinneapple_reason`, `pinneapple_query`	qual experimento rodar a seguir, cadeia de porquês, consultas físicas com planejador
Projeto e sistemas	`pinneapple_design`, `pinneapple_systems`, `pinneapple_twin3d`	geometria, otimização, projeto por intenção, séries temporais, gêmeos digitais
Catálogo e registro	`pinneapple_catalog`, `pinneapple_registry`, `pinneapple_hub`	métodos com status de validação, recursos com licença, artefatos, zoo de modelos
Orquestração	`pinneapple_flow`, `pinneapple_lab`, `pinneapple_autolab`, `pinneapple_llm`, `pinneapple_mcp`, `pinneapple_orchestration`	grafos de nós, experimentos registrados, missões autônomas, agentes
Ferramentas e produtos	`pinneapple_tools`, `pinneapple_arena`, `pinneapple_blender`, `pinneapple_app`, `apps/`	benchmarks, exportação, visualização, automação de GUIs, apps web

Os pacotes `pinneapple_models`, `pinneapple_train` e `pinneapple_solvers` são shims de compatibilidade: só reexportam `pinneapple_neural.architectures`, `pinneapple_neural.trainer` e `pinneapple_simulation.numerical_solvers`, para que caminhos de import antigos continuem funcionando.

Como as peças se conectam



Em palavras:

- **`pp.*`** é a porta de entrada. `pp.doctor` diz o que falta instalar, `pp.recommend` sugere preset, método e modelo a partir de uma frase, `pp.card` mostra o que um preset recebe e devolve, e `pp.new` cria um projeto do usuário fora do repositório.
- **`pp.solve` / `pp.compare`** (em `pinneapple_physics.solving`) recebem um `PhysicalProblem`, um `ProblemSpec` ou o nome de um preset e despacham para o método pedido (`"pinn"`, `"fem"`, `"analytic"`, `"reference"`, `"exact"`, `"external"`; `pp.list_methods()` lista os registrados). O resultado é um `Solution`.
- **Confiança**: `pp.validate` (em `pinneapple_analysis.validation.validate`) e `pp.trust_card` (em `pinneapple_analysis.trust.field_card`) conferem resultados e modelos; `pinneapple check` roda as mesmas verificações em CI.
- **`pinneapple_flow`** transforma um texto num grafo de nós tipados (`plan`), e o executa sem interface ou no Flow Studio (`run`).
- **`pinneapple_hub`** guarda e entrega modelos treinados e verificados (`pp.zoo`), datasets (`pp.datasets`) e uma biblioteca pesquisável de ferramentas, geometrias, fluxos e exemplos.
- **`pinneapple_catalog`** lista cada solver, método de treino, equação e problema com o código, as referências e o status de validação calculado a partir dos testes (`validated` / `tested` / `untested`).
- **Apps** (`apps/` e `pinneapple_app`) são interfaces web e ferramentas de demonstração montadas sobre a biblioteca; o servidor MCP (`pinneapple_mcp`) expõe as mesmas funções para agentes de código.

A porta de entrada `pp.*`

`import pinneapple as pp` expõe 297 nomes (versão `0.7.0`). A tabela diz de onde cada um vem e o que faz.

nome	como é carregado	origem	o que faz
`pp.ActiveLearningConfig`	import direto	`pinneapple_data.ActiveLearningConfig`	
`pp.AdaptiveCollocationTrainer`	import direto	`pinneapple_data.AdaptiveCollocationTrainer`	
`pp.AFNO`	import direto	`pinneapple_neural.architectures.AFNO`	
`pp.annulus`	import direto	`pinneapple_design.geometry.annulus`	
`pp.arena`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_arena`	pinneapple_arena — YAML/JSON-driven multi-model physics benchmark runner.
`pp.as_tensor`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_neural.utils.as_tensor`	Return the prediction Tensor of a model output.
`pp.assess_drift`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_analysis.uncertainty.drift.assess_drift`	Check a batch of surrogate predictions for accuracy loss, miscalibration, input shift and residual trend.
`pp.assimilation`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_analysis.data_assimilation`	
`pp.astro`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_physics.astro`	
`pp.auto_residual`	import direto	`pinneapple_physics.symbolic_pde.auto_residual`	
`pp.Backend`	import direto	`pinneapple_tools.compute_backends.Backend`	
`pp.backend`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_tools.compute_backends`	
`pp.batched_inference`	import direto	`pinneapple_neural.trainer.batched_inference`	

nome	como é carregado	origem	o que faz
`pp.benchmark`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_analysis.benchmark`	
`pp.BenchmarkConfig`	import direto	`pinneapple_tools.benchmark_suite.BenchmarkConfig`	
`pp.BenchmarkResult`	import direto	`pinneapple_tools.benchmark_suite.BenchmarkResult`	
`pp.BenchmarkTaskBase`	import direto	`pinneapple_tools.benchmark_suite.BenchmarkTaskBase`	
`pp.best_device`	import direto	`pinneapple_neural.trainer.best_device`	
`pp.best_method`	import direto	`pinneapple_core.onboarding.catalog.best_method`	O método mais barato que roda hoje para o preset: analítico, senão solver de referência, senão rede treinada.
`pp.bodies`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_design.geometry.bodies`	
`pp.BoxConstraint`	import direto	`pinneapple_design.design_optimizer.BoxConstraint`	
`pp.build_digital_twin`	import direto	`pinneapple_systems.digital_twin.build_digital_twin`	
`pp.build_model`	definido em `pinneapple/__init__.py`	`pinneapple.build_model`	
`pp.CADToCFDPipeline`	import direto	`pinneapple_simulation.numerical_solvers.CADToCFDPipeline`	
`pp.cae`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_data.cae`	
`pp.card`	import direto	`pinneapple_core.onboarding.card`	
`pp.CauchyMisfit`	import direto	`pinneapple_analysis.inverse_problems.CauchyMisfit`	

nome	como é carregado	origem	o que faz
pp.CausalPINNTrainer	import direto	pinneapple_neural.trainer.CausalPINNTrainer	
pp.CausalWeightScheduler	import direto	pinneapple_neural.trainer.CausalWeightScheduler	
pp.cfd	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.numerical_solvers.external_flow	
pp.CFDMesh	import direto	pinneapple_simulation.numerical_solvers.CFDMesh	
pp.channel_with_hole	import direto	pinneapple_design.geometry.channel_with_hole	
pp.ChannelDomain2D	import direto	pinneapple_design.geometry.ChannelDomain2D	
pp.ChannelWithObstacleDomain2D	import direto	pinneapple_design.geometry.ChannelWithObstacleDomain2D	
pp.circle	import direto	pinneapple_design.geometry.circle	
pp.closed_form	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.closed_form	
pp.CollocationConfig	import direto	pinneapple_data.CollocationConfig	
pp.CollocationSampler	import direto	pinneapple_data.CollocationSampler	
pp.compare	import direto	pinneapple_physics.solving.compare	Solve or reuse each method and score it against `reference` at the same points.
pp.Comparison	import direto	pinneapple_physics.solving.Comparison	
pp.compile	preguiçoso (_LAZY_ATTRS)	pinneapple_physics.compile_api.compile	Prepare a problem (a `PhysicalProblem`, a `ProblemSpec` or a preset name) for faster solving.
pp.ComposedObsOperator	import direto	pinneapple_analysis.inverse_problems.ComposedObsOperator	
pp.CompositeObjective	import direto	pinneapple_design.design_optimizer.CompositeObjective	

nome	como é carregado	origem	o que faz
pp.CompositeRegularizer	import direto	pinneapple_analysis.inverse_problems.CompositeRegularizer	
pp.compute_isosurface	import direto	pinneapple_neural.predictor.compute_isosurface	
pp.compute_pareto_front	import direto	pinneapple_design.design_optimizer.compute_pareto_front	
pp.compute_streamlines	import direto	pinneapple_neural.predictor.compute_streamlines	
pp.ComputeBudget	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.cost.equal_budget.ComputeBudget	A training budget: any combination of wall-clock seconds, FLOPs and optimiser steps (None = no limit).
pp.conformal_study	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.uncertainty.conformal_study.conformal_study	Calibrate on (y_cal, pred_cal) and measure the coverage on (y_test, pred_test).
pp.ConformalPredictor	import direto	pinneapple_analysis.uncertainty.ConformalPredictor	
pp.ConstraintSet	import direto	pinneapple_design.design_optimizer.ConstraintSet	
pp.ContinuousAdjointSolver	import direto	pinneapple_design.design_optimizer.ContinuousAdjointSolver	
pp.core	submódulo preguiçoso (_SUBMODULES)	pinneapple_core	Ponto de entrada do pacote: exporta Domain, Geometry, Mesh, Field, FunctionField, Grid, PointCloud e os operadores, e carrega func, fem, loss, optim, data, transforms etc. preguiçosamente no primeiro acesso.
pp.count_gpus	import direto	pinneapple_neural.trainer.count_gpus	
pp.csg_annulus	import direto	pinneapple_design.geometry.csg_annulus	
pp.CSGCircle	import direto	pinneapple_design.geometry.CSGCircle	
pp.CSGDifference	import direto	pinneapple_design.geometry.CSGDifference	
pp.CSGEllipse	import direto	pinneapple_design.geometry.CSGEllipse	
pp.CSGIntersection	import direto	pinneapple_design.geometry.CSGIntersection	
pp.CSGPolygon	import direto	pinneapple_design.geometry.CSGPolygon	

nome	como é carregado	origem	o que faz
pp.CSGRectangle	import direto	pinneapple_design.geometry.CSGRectangle	
pp.CSGUnion	import direto	pinneapple_design.geometry.CSGUnion	
pp.curl	preguiçoso (_LAZY_ATTRS)	pinneapple_core.operators.curl	Rotacional: vetor em 3D, escalar (componente z) em 2D.
pp.dashboard	submódulo preguiçoso (_SUBMODULES)	pinneapple_tools.visualization.dashboard	
pp.data	submódulo preguiçoso (_SUBMODULES)	pinneapple_data	
pp.datasets	submódulo preguiçoso (_SUBMODULES)	pinneapple_hub.zoo.datasets	
pp.DDPPINNTrainer	import direto	pinneapple_neural.trainer.DDPPINNTrainer	
pp.DDPTrainerConfig	import direto	pinneapple_neural.trainer.DDPTrainerConfig	
pp.decide	preguiçoso (_LAZY_ATTRS)	pinneapple_decision.api.decide	Decide which experiment to run next. problem may be a ProblemSpec or a free-form dict (see adapter.py). Facts that could not be determined are listed in decision.diagnostics["problem_facts"]["unknown"].
pp.decision	submódulo preguiçoso (_SUBMODULES)	pinneapple_decision	pinneapple_decision -- probabilistic decision layer over the Physics AI pipeline.
pp.decision_engine	preguiçoso (_LAZY_ATTRS)	pinneapple_decision.api.decision_engine	Return the default engine. Passing any argument (or reset=True) replaces it with a new engine built from those arguments (evidence is not carried over).
pp.DEFAULT_MODELS	import direto	pinneapple_tools.benchmark_suite.DEFAULT_MODELS	
pp.design	preguiçoso (_LAZY_ATTRS)	pinneapple_design.intent.design	
pp.design_opt	submódulo preguiçoso (_SUBMODULES)	pinneapple_design.design_optimizer	Ponto de entrada do otimizador de projeto: reexporta o laço fechado (espaço de parâmetros, surrogate, otimizador, refinamento por PINN, Pareto) e a camada de estudos de projeto.

nome	como é carregado	origem	o que faz
pp.DesignOptConfig	import direto	pinneapple_design.design_optimizer.DesignOptConfig	
pp.DesignOptimizerConfig	import direto	pinneapple_design.design_optimizer.DesignOptimizerConfig	
pp.DesignOptLoop	import direto	pinneapple_design.design_optimizer.DesignOptLoop	
pp.DesignOptResult	import direto	pinneapple_design.design_optimizer.DesignOptResult	
pp.DigitalTwin	import direto	pinneapple_systems.digital_twin.DigitalTwin	
pp.DigitalTwinConfig	import direto	pinneapple_systems.digital_twin.DigitalTwinConfig	
pp.DirichletBC	import direto	pinneapple_physics.symbolic_pde.DirichletBC	
pp.div	preguiçoso (_LAZY_ATTRS)	pinneapple_core.operators.div	Divergência de um campo vetorial.
pp.doctor	import direto	pinneapple_core.onboarding.doctor	pp.doctor: verifica a instalação contra um perfil (start, cfd, research, enterprise, all) e imprime um comando de correção por falha.
pp.dominant_frequency	preguiçoso (_LAZY_ATTRS)	pinneapple_physics.physics_audit.regime.dominant_frequency	Frequency (1 / time unit of dt) of the strongest spectral peak of a sampled signal, e.g. the vortex-shedding frequency of a wake probe or a lift history, for :func:strouhal.
pp.DoMINO	import direto	pinneapple_physics.pinn_solver.DoMINO	
pp.DragAdjointObjective	import direto	pinneapple_design.design_optimizer.DragAdjointObjective	
pp.DragObjective	import direto	pinneapple_design.design_optimizer.DragObjective	
pp.dt	submódulo preguiçoso (_SUBMODULES)	pinneapple_systems.digital_twin	
pp.dynamics	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.particle_dynamics	Simulação dinâmica diferenciável em PyTorch puro: corpos rígidos, MPM (MLS-MPM) e SPH; re-exporta também os parâmetros de solo/roda de Bekker-Wong.
pp.EKIConfig	import direto	pinneapple_analysis.inverse_problems.EKIConfig	

nome	como é carregado	origem	o que faz
pp.ellipse	import direto	pinneapple_design.geometry.ellipse	
pp.em	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.electromagnetics	
pp.ensemble	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.ensemble	
pp.EnsembleKalmanInversion	import direto	pinneapple_analysis.inverse_problems.EnsembleKalmanInversion	
pp.EnsembleUQ	import direto	pinneapple_analysis.uncertainty.EnsembleUQ	
pp.env	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.pde_environment	
pp.excitation	submódulo preguiçoso (_SUBMODULES)	pinneapple_systems.excitation	
pp.execute	preguiçoso (_LAZY_ATTRS)	pinneapple_decision.api.execute	Execute a decision issued by the same engine (constraints are re-checked first).
pp.Experiment	import direto	pinneapple_physics.experiment.Experiment	A method applied to a problem, with a seed, options and an optional reference to score against.
pp.ExperimentResult	import direto	pinneapple_physics.experiment.ExperimentResult	
pp.export	submódulo preguiçoso (_SUBMODULES)	pinneapple_tools.model_export	
pp.export_csv	import direto	pinneapple_tools.model_export.export_csv	Export coordinate + field arrays to a CSV file.
pp.export_npz	import direto	pinneapple_tools.model_export.export_npz	Export coordinate + field arrays to a compressed NumPy `.npz` archive.
pp.export_onnx	import direto	pinneapple_tools.model_export.export_onnx	Export any nn.Module to ONNX.
pp.export_torchscript	import direto	pinneapple_tools.model_export.export_torchscript	Export any nn.Module to TorchScript.

nome	como é carregado	origem	o que faz
`pp.fea`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_simulation.external_solvers.calculix.study`	Estudos CalculiX em poucas linhas: malha (hexaedros, tetraedros gmsh ou deck existente), conjuntos por geometria, um passo de análise (estático, frequência, flambagem, calor), execução e resultados em arrays.
`pp.FlowVisualizer`	import direto	`pinneapple_neural.predictor.FlowVisualizer`	
`pp.flux`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.operators.flux`	Fluxo de um campo vetorial pela fronteira, `integral f . n dS`.
`pp.FourierFeatureEmbedding`	import direto	`pinneapple_neural.architectures.FourierFeatureEmbedding`	
`pp.func`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_core.func`	Transformações de função (`grad`, `jacrev`, `jacfwd`, `jacobian`, `hessian`, `vmap`) sobre `torch.func` com `wrt` por nome ou posição, e `implicit_solve`, que diferencia através de um solver por uma solução adjunta.
`pp.GaussianMisfit`	import direto	`pinneapple_analysis.inverse_problems.GaussianMisfit`	
`pp.geom`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_design.geometry`	
`pp.geometries`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_design.geometry_library`	Fachada da biblioteca de geometrias (exposta como `pp.geometries`): listar, construir, exportar, renderizar miniaturas, carregar arquivos pré-gerados e exportar a galeria web.
`pp.geophysics`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_simulation.geophysics`	
`pp.get_backend`	import direto	`pinneapple_tools.compute_backends.get_backend`	
`pp.get_pde_family`	import direto	`pinneapple_physics.pde_environment.get_pde_family`	
`pp.get_physics_backend`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.backend.get_physics_backend`	Devolve o objeto backend pelo nome ou o backend ativo.
`pp.get_preset`	import direto	`pinneapple_physics.pde_environment.get_preset`	
`pp.get_rank`	import direto	`pinneapple_neural.trainer.get_rank`	

nome	como é carregado	origem	o que faz
`pp.get_rans_preset`	import direto	`pinneapple_physics.pde_environment.get_rans_preset`	
`pp.get_world_size`	import direto	`pinneapple_neural.trainer.get_world_size`	
`pp.GlobalSensitivity`	import direto	`pinneapple_analysis.inverse_problems.GlobalSensitivity`	
`pp.gpu_info`	import direto	`pinneapple_neural.trainer.gpu_info`	
`pp.grad`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.operators.grad`	Gradiente: `(N, d)` para escalar, `(N, c, d)` para `c` componentes.
`pp.GradNormBalancer`	import direto	`pinneapple_neural.trainer.GradNormBalancer`	
`pp.HardBC`	import direto	`pinneapple_physics.symbolic_pde.HardBC`	
`pp.HashGridEncoding`	import direto	`pinneapple_neural.architectures.HashGridEncoding`	
`pp.HashGridMLP`	import direto	`pinneapple_neural.architectures.HashGridMLP`	
`pp.hessian`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.operators.hessian`	Hessiana `(N, d, d)` de um campo escalar.
`pp.HeteroscedasticMisfit`	import direto	`pinneapple_analysis.inverse_problems.HeteroscedasticMisfit`	
`pp.HuberMisfit`	import direto	`pinneapple_analysis.inverse_problems.HuberMisfit`	
`pp.Hybrid`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.module.Hybrid`	Solver mais correção neural (por exemplo modo `residual`), treinável através do solver.
`pp.IdentifiabilityAnalyzer`	import direto	`pinneapple_analysis.inverse_problems.IdentifiabilityAnalyzer`	
`pp.identify_pde`	import direto	`pinneapple_physics.pde_environment.identify_pde`	
`pp.infer_on_grid_1d`	import direto	`pinneapple_neural.predictor.infer_on_grid_1d`	
`pp.infer_on_grid_2d`	import direto	`pinneapple_neural.predictor.infer_on_grid_2d`	

nome	como é carregado	origem	o que faz
`pp.inference`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_neural.predictor`	
`pp.info`	definido em `pinneapple/__init__.py`	`pinneapple.info`	Print library version, device info, and available presets/models.
`pp.IntegralObsOperator`	import direto	`pinneapple_analysis.inverse_problems.IntegralObsOperator`	
`pp.integrate`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.operators.integrate`	Integral sobre o suporte (trapézio na grade, P1 exato na malha, quadratura ou Monte Carlo na nuvem).
`pp.inverse`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_analysis.inverse_problems`	Reexporta o kit de problemas inversos (modelos de ruído, regularizadores, operadores de observação, EKI, sensibilidade, termos faltantes, `InverseProblemSolver`, e o núcleo de identificabilidade/calibração/studio). O docstring ainda usa o nome antigo `pinneapple_inverse`.
`pp.InverseProblemSolver`	import direto	`pinneapple_analysis.inverse_problems.InverseProblemSolver`	
`pp.InverseSolverConfig`	import direto	`pinneapple_analysis.inverse_problems.InverseSolverConfig`	
`pp.is_distributed`	import direto	`pinneapple_neural.trainer.is_distributed`	
`pp.IteratedEKI`	import direto	`pinneapple_analysis.inverse_problems.IteratedEKI`	
`pp.jacobian`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.operators.jacobian`	Jacobiana `(N, c, d)` de um campo.
`pp.JAXBackend`	import direto	`pinneapple_tools.compute_backends.JAXBackend`	
`pp.jit_pinn`	import direto	`pinneapple_tools.compute_backends.jit_pinn`	
`pp.KOmegaSSTResiduals`	import direto	`pinneapple_physics.pde_environment.KOmegaSSTResiduals`	
`pp.laplacian`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.operators.laplacian`	Soma das segundas derivadas, por componente em campos vetoriais.
`pp.LCurveSelector`	import direto	`pinneapple_analysis.inverse_problems.LCurveSelector`	

nome	como é carregado	origem	o que faz
`pp.LidDrivenCavityDomain2D`	import direto	`pinneapple_design.geometry.LidDrivenCavityDomain2D`	
`pp.LinearObsOperator`	import direto	`pinneapple_analysis.inverse_problems.LinearObsOperator`	
`pp.list_methods`	import direto	`pinneapple_physics.solving.list_methods`	Registered method names, optionally only those of one `kind`.
`pp.list_models`	definido em `pinneapple/__init__.py`	`pinneapple.list_models`	
`pp.list_pde_families`	import direto	`pinneapple_physics.pde_environment.list_pde_families`	
`pp.list_presets`	import direto	`pinneapple_physics.pde_environment.list_presets`	
`pp.LocalSensitivity`	import direto	`pinneapple_analysis.inverse_problems.LocalSensitivity`	
`pp.loss`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_core.loss`	Termos de perda como funções simples (`pde`, `boundary`, `supervised`, `conservation`, `energy`, `symmetry`, `inverse`), soma ponderada (`combine`) e o `Balancer` de pesos dinâmicos escolhido por nome.
`pp.LossRatioBalancer`	import direto	`pinneapple_neural.trainer.LossRatioBalancer`	
`pp.lshape`	import direto	`pinneapple_design.geometry.lshape`	
`pp.MassConservationConstraint`	import direto	`pinneapple_design.design_optimizer.MassConservationConstraint`	
`pp.maxwell`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_physics.maxwell_checks`	
`pp.maybe_compile`	import direto	`pinneapple_neural.trainer.maybe_compile`	
`pp.MCDropoutWrapper`	import direto	`pinneapple_analysis.uncertainty.MCDropoutWrapper`	
`pp.MeshGraphNet`	import direto	`pinneapple_neural.architectures.MeshGraphNet`	

nome	como é carregado	origem	o que faz
`pp.meta`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_adaptation.meta_learning`	Reexporta configurações, amostrador e treinadores de meta-learning e oferece `meta_train` (constrói e treina) e `meta_adapt` (adapta a uma tarefa nova).
`pp.MetaBenchmarkConfig`	import direto	`pinneapple_tools.benchmark_suite.MetaBenchmarkConfig`	
`pp.MetaBenchmarkFamily`	import direto	`pinneapple_tools.benchmark_suite.MetaBenchmarkFamily`	
`pp.MetaBenchmarkPipeline`	import direto	`pinneapple_tools.benchmark_suite.MetaBenchmarkPipeline`	
`pp.MetaBenchmarkResult`	import direto	`pinneapple_tools.benchmark_suite.MetaBenchmarkResult`	
`pp.MethodNotAvailable`	import direto	`pinneapple_physics.solving.MethodNotAvailable`	The requested method cannot solve this problem (missing solver output, missing option, ...).
`pp.metrics`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_physics.metrics`	
`pp.ModelRegistry`	import direto	`pinneapple_neural.architectures.ModelRegistry`	
`pp.models`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_models`	[shim] Reexporta registro, catálogo e arquiteturas de `pinneapple_neural.architectures` para código legado; marcado como obsoleto (use `pinneapple_neural.architectures`). [sem teste direto]
`pp.ModelSpec`	import direto	`pinneapple_tools.benchmark_suite.ModelSpec`	
`pp.ModifiedMLP`	import direto	`pinneapple_neural.architectures.ModifiedMLP`	
`pp.molecular`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_tools.visualization.molecular`	
`pp.MPMSimulator`	import direto	`pinneapple_simulation.particle_dynamics.MPMSimulator`	
`pp.MPMState`	import direto	`pinneapple_simulation.particle_dynamics.MPMState`	

nome	como é carregado	origem	o que faz
pp.multibody	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.multibody	Dinâmica de corpos rígidos em coordenadas reduzidas: uma base flutuante com árvores de juntas de revolução; reexporta a árvore e os utilitários de braço.
pp.naca_parametric	import direto	pinneapple_design.design_optimizer.naca_parametric	
pp.NeumannBC	import direto	pinneapple_physics.symbolic_pde.NeumannBC	
pp.new	definido em pinneapple/__init__.py	pinneapple.new	
pp.new_project	import direto	pinneapple_core.onboarding.new_project	
pp.non_hermitian	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.non_hermitian	
pp.NSFlowSolver	import direto	pinneapple_simulation.numerical_solvers.NSFlowSolver	
pp.NTKWeightBalancer	import direto	pinneapple_neural.trainer.NTKWeightBalancer	
pp.Parameter	import direto	pinneapple_physics.physical_problem.Parameter	A named physical parameter.
pp.ParetoFront	import direto	pinneapple_design.design_optimizer.ParetoFront	
pp.ParticleSystem	import direto	pinneapple_simulation.particle_dynamics.ParticleSystem	
pp.pde_from_sympy	import direto	pinneapple_physics.symbolic_pde.pde_from_sympy	
pp.PeriodicBC	import direto	pinneapple_physics.symbolic_pde.PeriodicBC	
pp.PhysicalProblem	import direto	pinneapple_physics.physical_problem.PhysicalProblem	A complete physics problem: geometry, physics, conditions, parameters, solver choice and intent.
pp.physics_audit	submódulo preguiçoso (_SUBMODULES)	pinneapple_physics.physics_audit	

nome	como é carregado	origem	o que faz
`pp.PhysicsDomain2D`	import direto	`pinneapple_design.geometry.PhysicsDomain2D`	
`pp.PhysicsModule`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.module.PhysicsModule`	Classe base de modelos e solvers com `kind`, `describe()`, `num_parameters()` e um `fit` simples.
`pp.PhysicsOptimizer`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.optim.PhysicsOptimizer`	Parâmetros com limites, objetivo e restrições, minimizados pelo método escolhido em `minimize`.
`pp.PhysicsScenario`	import direto	`pinneapple_worldmodel.PhysicsScenario`	
`pp.PhysicsSurrogate`	import direto	`pinneapple_design.design_optimizer.PhysicsSurrogate`	
`pp.PhysicsValidator`	import direto	`pinneapple_analysis.validation.PhysicsValidator`	
`pp.PhysicsWorldModel`	import direto	`pinneapple_worldmodel.PhysicsWorldModel`	
`pp.pinn`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_physics.pinn_solver`	
`pp.PINNArenaBenchmark`	import direto	`pinneapple_tools.benchmark_suite.PINNArenaBenchmark`	
`pp.PINNRefinement`	import direto	`pinneapple_design.design_optimizer.PINNRefinement`	
`pp.plot_isosurface_3d`	import direto	`pinneapple_neural.predictor.plot_isosurface_3d`	
`pp.plot_streamlines_2d_from_model`	import direto	`pinneapple_neural.predictor.plot_streamlines_2d_from_model`	
`pp.plot_volume_slice`	import direto	`pinneapple_neural.predictor.plot_volume_slice`	
`pp.PointObsOperator`	import direto	`pinneapple_analysis.inverse_problems.PointObsOperator`	
`pp.presets`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_presets`	Industrial presets: engineering problems already set up, solved and validated with PINNeAPPle.
`pp.profiles`	import direto	`pinneapple_core.onboarding.profiles`	

nome	como é carregado	origem	o que faz
pp.propagate_tolerances	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.uncertainty.tolerance.propagate	Propagate independent input tolerances through fn by Monte-Carlo sampling (JCGM 101:2008).
pp.propagate_uncertainty	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.uncertainty.tolerance.propagate_space	Propagate independent inputs with any distribution (see :mod:pinneapple_analysis.uncertainty.distributions) through fn.
pp.qualitative	submódulo preguiçoso (_SUBMODULES)	pinneapple_design.qualitative	
pp.Quantity	import direto	pinneapple_physics.physical_problem.Quantity	A quantity of interest: what the user actually wants out of the solution (a field, a coefficient, a peak value). field names the model output it derives from, when there is one.
pp.quantum	submódulo preguiçoso (_SUBMODULES)	pinneapple_quantum	pinneapple_quantum — PINNeAPPle Quantum Module (PQM).
pp.quickstart	definido em pinneapple/__init__.py	pinneapple.quickstart	Interactive quickstart that summarises a problem preset and its fields.
pp.recommend	import direto	pinneapple_core.onboarding.recommend	pp.recommend: transforma uma frase em preset, método e modelo, com o motivo e os termos que casaram; é pontuação por palavras-chave e pode errar. [sem teste direto]
pp.rectangle	import direto	pinneapple_design.geometry.rectangle	
pp.register_method	import direto	pinneapple_physics.solving.register_method	Decorator registering fn(problem, **options) -> Solution under name for solve/compare. kind labels the family of the method (classical, neural, ...) in every Solution it returns.
pp.register_preset	import direto	pinneapple_physics.pde_environment.register_preset	
pp.reporting	submódulo preguiçoso (_SUBMODULES)	pinneapple_tools.reporting	
pp.ResidualBasedAL	import direto	pinneapple_data.ResidualBasedAL	
pp.RigidBody	import direto	pinneapple_simulation.particle_dynamics.RigidBody	
pp.RigidBodyState	import direto	pinneapple_simulation.particle_dynamics.RigidBodyState	

nome	como é carregado	origem	o que faz
pp.RigidBodySystem	import direto	pinneapple_simulation.particle_dynamics.RigidBodySystem	
pp.run_parallel_sweep	import direto	pinneapple_neural.trainer.run_parallel_sweep	
pp.run_tree	preguiçoso (_LAZY_ATTRS)	pinneapple_decision.api.run_tree	Walk the physics-AI decision tree (tree.physics_ai_tree() by default).
pp.science	submódulo preguiçoso (_SUBMODULES)	pinneapple_science	PINNeAPPle science (pp.science): experiments by area of science, at academic and research level, theory and practice (a virtual bench), in 0-D (lumped), 1-D, 2-D, 3-D and 3-D + time; and the atlas of every area.
pp.score_against_reference	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.verification.physics_confidence_score.score_against_reference	Score several models' predictions against one held-out reference dataset.
pp.security	submódulo preguiçoso (_SUBMODULES)	pinneapple_security	pinneapple_security — data and process security for scientific and industrial Physics AI.
pp.SelfAdaptiveWeights	import direto	pinneapple_neural.trainer.SelfAdaptiveWeights	
pp.sensitivity	submódulo preguiçoso (_SUBMODULES)	pinneapple_analysis.sensitivity	
pp.Sequential	preguiçoso (_LAZY_ATTRS)	pinneapple_core.module.Sequential	Encadeia componentes (módulos, solvers ou funções), a saída de um é a entrada do próximo.
pp.set_backend	import direto	pinneapple_tools.compute_backends.set_backend	
pp.ShapeParametrization	import direto	pinneapple_design.design_optimizer.ShapeParametrization	
pp.SineLayer	import direto	pinneapple_neural.architectures.SineLayer	
pp.SIREN	import direto	pinneapple_neural.architectures.SIREN	
pp.sobol_indices	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.sensitivity.sobol.sobol_indices	First-order and total Sobol indices (and optionally second order) of fn over space.
pp.Solution	import direto	pinneapple_physics.solving.Solution	

nome	como é carregado	origem	o que faz
`pp.solve`	import direto	`pinneapple_physics.solving.solve`	Solve `problem` with `method` (a registered name or a callable `(problem, **options) -> Solution`).
`pp.SolverModule`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.module.SolverModule`	Um solver como componente, com parâmetros treináveis opcionais; `from_method` usa um backend de `pp.solve` (não diferenciável).
`pp.solvers`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_solvers`	[shim] Pacote de compatibilidade (obsoleto) que reexporta os solvers de `pinneapple_simulation.numerical_solvers`; os solvers opcionais viram `None` se o import falhar. [sem teste direto do `__init__`]
`pp.SpalartAllmarasResiduals`	import direto	`pinneapple_physics.pde_environment.SpalartAllmarasResiduals`	
`pp.SparsityRegularizer`	import direto	`pinneapple_analysis.inverse_problems.SparsityRegularizer`	
`pp.SPHParticles`	import direto	`pinneapple_simulation.particle_dynamics.SPHParticles`	
`pp.StructuralObjective`	import direto	`pinneapple_design.design_optimizer.StructuralObjective`	
`pp.StudentTMisfit`	import direto	`pinneapple_analysis.inverse_problems.StudentTMisfit`	
`pp.StudyModel`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_analysis.study_models.StudyModel`	
`pp.Subdomain`	import direto	`pinneapple_physics.pinn_solver.Subdomain`	
`pp.SubdomainPINN`	import direto	`pinneapple_physics.pinn_solver.SubdomainPINN`	
`pp.suggest_problem_spec`	import direto	`pinneapple_physics.pde_environment.suggest_problem_spec`	
`pp.SurrogateConfig`	import direto	`pinneapple_design.design_optimizer.SurrogateConfig`	
`pp.SweepConfig`	import direto	`pinneapple_neural.trainer.SweepConfig`	
`pp.symbolic`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_physics.symbolic_pde`	
`pp.SymbolicPDE`	import direto	`pinneapple_physics.symbolic_pde.SymbolicPDE`	

nome	como é carregado	origem	o que faz
`pp.t_junction`	import direto	`pinneapple_design.geometry.t_junction`	
`pp.terramechanics`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_simulation.terramechanics`	Terramecânica de rodas de rovers planetários: solos com proveniência, modelo de roda de Wong-Reece e extensões, terreno deformável com sulcos, bancada de roda única com dados publicados e substitutos PINN do contato.
`pp.ThermalEfficiencyObjective`	import direto	`pinneapple_design.design_optimizer.ThermalEfficiencyObjective`	
`pp.ThroughputMonitor`	import direto	`pinneapple_neural.trainer.ThroughputMonitor`	
`pp.TikhonovRegularizer`	import direto	`pinneapple_analysis.inverse_problems.TikhonovRegularizer`	
`pp.TimeMarchingTrainer`	import direto	`pinneapple_neural.trainer.TimeMarchingTrainer`	
`pp.to_jsonable`	preguiçoso (`_LAZY_ATTRS`)	`pinneapple_core.jsonable.to_jsonable`	Devolve uma cópia de um objeto que `json.dumps` aceita, com opção de arredondar e tratar não-finitos.
`pp.TotalVariationRegularizer`	import direto	`pinneapple_analysis.inverse_problems.TotalVariationRegularizer`	
`pp.train`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_train`	[shim] Pacote de compatibilidade (marcado como obsoleto) que reexporta o treinador, callbacks e métricas de `pinneapple_neural.trainer`; nomes opcionais viram `None` se o import falhar. [sem teste direto do `__init__`]
`pp.TrainConfig`	import direto	`pinneapple_neural.trainer.TrainConfig`	
`pp.Trainer`	import direto	`pinneapple_neural.trainer.Trainer`	
`pp.transfer`	submódulo preguiçoso (`_SUBMODULES`)	`pinneapple_adaptation.transfer_learning`	Reexporta `TransferConfig`, `TransferTrainer`, utilitários de congelamento, `ParametricFamilyTransfer` e `PhysicsTransferAdapter`. [sem teste direto do `__init__`]
`pp.TransferBenchmarkConfig`	import direto	`pinneapple_tools.benchmark_suite.TransferBenchmarkConfig`	
`pp.TransferBenchmarkPipeline`	import direto	`pinneapple_tools.benchmark_suite.TransferBenchmarkPipeline`	
`pp.TransferBenchmarkResult`	import direto	`pinneapple_tools.benchmark_suite.TransferBenchmarkResult`	

nome	como é carregado	origem	o que faz
pp.TransferScenario	import direto	pinneapple_tools.benchmark_suite.TransferScenario	
pp.transforms	submódulo preguiçoso (_SUBMODULES)	pinneapple_core.transforms	Transformações invertíveis e componíveis (>>) sobre tabelas, PhysicsDataset e, para as de escala, PhysicalProblem, registrando proveniência a cada apply; as não-escalares levantam NotImplementedError em nível de problema.
pp.trust_card	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.trust.field_card.trust_card	Run field_checks and turn them into a trust card: the deploy decision (TrustReport: a failing critical check rejects, any other failure or low coverage asks for review) and the six questions of the lab trust card. kwargs go to field_checks.
pp.TwoPhaseConfig	import direto	pinneapple_neural.trainer.TwoPhaseConfig	
pp.TwoPhaseHistory	import direto	pinneapple_neural.trainer.TwoPhaseHistory	
pp.TwoPhaseTrainer	import direto	pinneapple_neural.trainer.TwoPhaseTrainer	
pp.uncertainty	submódulo preguiçoso (_SUBMODULES)	pinneapple_analysis.uncertainty	
pp.UnnormModel	import direto	pinneapple_neural.trainer.UnnormModel	
pp.uq	submódulo preguiçoso (_SUBMODULES)	pinneapple_analysis.uncertainty	
pp.uq_predict	import direto	pinneapple_analysis.uncertainty.uq_predict	
pp.use_backend	preguiçoso (_LAZY_ATTRS)	pinneapple_core.backend.use_backend	Gerenciador de contexto que roda um bloco num backend e restaura o anterior.
pp.validate	preguiçoso (_LAZY_ATTRS)	pinneapple_analysis.validation.validate.pp_validate	
pp.validate_model	import direto	pinneapple_analysis.validation.validate_model	
pp.vehicle	submódulo preguiçoso (_SUBMODULES)	pinneapple_simulation.vehicle	Veículos planetários com rodas: descrição (chassi + módulos de canto) com CAD, modelo multicorpo construído a partir dela e simulador em terreno deformável com rodas de Wong-Reece.
pp.verify	preguiçoso (_LAZY_ATTRS)	pinneapple_decision.api.verify	Verify a result; only verified results become evidence for the next decision.

nome	como é carregado	origem	o que faz
pp.veriphysics	submódulo preguiçoso (_SUBMODULES)	pinneapple_veriphysics	pinneapple_veriphysics — the verification and evidence layer of Veriphysics, as part of PINNeAPPle.
pp.viz	submódulo preguiçoso (_SUBMODULES)	pinneapple_tools.visualization.studio	
pp.vmap_residual	import direto	pinneapple_tools.compute_backends.vmap_residual	
pp.WeightMinimizationObjective	import direto	pinneapple_design.design_optimizer.WeightMinimizationObjective	
pp.WeightScheduler	import direto	pinneapple_neural.trainer.WeightScheduler	
pp.WeightSchedulerConfig	import direto	pinneapple_neural.trainer.WeightSchedulerConfig	
pp.worldmodel	submódulo preguiçoso (_SUBMODULES)	pinneapple_worldmodel	pinneapple_worldmodel — Generalist Physics AI World Model.
pp.WorldModelConfig	import direto	pinneapple_worldmodel.WorldModelConfig	
pp.WorldModelDataset	import direto	pinneapple_worldmodel.WorldModelDataset	
pp.WorldModelTrainer	import direto	pinneapple_worldmodel.WorldModelTrainer	
pp.zoo	submódulo preguiçoso (_SUBMODULES)	pinneapple_hub.zoo	

Como este documento foi feito

1. `docs/pt/_mapa/inventario.py` percorre o repositório por AST e grava `inventario.json`.
2. O texto em português de cada pacote, módulo e símbolo foi escrito lendo o código e está em `docs/pt/_mapa/prosa/*.json`; `docs/pt/_mapa/checar_prosa.py` confere que nada ficou sem texto.
3. `docs/pt/_mapa/gerar.py` monta os volumes. Quando falta texto em português para um símbolo, a tabela mostra a primeira linha da docstring original (em inglês).
4. `docs/pt/_mapa/pdf.js` gera o PDF.

Volume 1 — Núcleo, porta de entrada e catálogo

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

A porta de entrada `pp.*`, os primitivos do núcleo, o catálogo de métodos, o registro de artefatos e os shims de compatibilidade.

Sumário

1. pinneapple
2. pinneapple_core
3. pinneapple_catalog
4. pinneapple_registry
5. pinneapple_orchestration
6. pinneapple_models
7. pinneapple_train
8. pinneapple_solvers

1. `pinneapple`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
3	866	0	2	431 (39 arquivos)	2

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
check		2	0	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple`

pinneapple — Physics-Informed Neural Networks for Engineering and Research.

`pinneapple/__init__.py` · 854 linhas · testes: 431

nome	tipo	assinatura	o que faz
quickstart	função	`(problem_id='burgers_1d', **problem_kwargs)`	Interactive quickstart that summarises a problem preset and its fields.

nome	tipo	assinatura	o que faz
`info`	função	`()`	Print library version, device info, and available presets/models.

`pinneapple.check`

pinneapple.check: the physics linter (see :mod:pinneapple_check). `python -m pinneapple.check [paths]`.

`pinneapple/check/__init__.py` · 4 linhas · **sem teste direto**

`pinneapple.check.__main__`

`python -m pinneapple.check [paths]` is `pinneapple check [paths]`.

`pinneapple/check/__main__.py` · 8 linhas · **sem teste direto**

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 1 de 3.
- Funções de teste em arquivos que importam algum módulo do pacote: 431.

Limitações conhecidas

- *Nenhuma registrada.*

2. `pinneapple_core`

Objetivo

O `pinneapple_core` reúne as primitivas compartilhadas da biblioteca: `Domain` (a região do problema), `Geometry` (um domínio com fronteiras nomeadas e um gerador de malha), `Mesh` (malha simplicial 1D/2D/3D com cálculo P1 exato) e `Field`/`FunctionField` (valores de uma grandeza numa grade, malha, nuvem de pontos ou numa função torch, como uma PINN). Todo campo responde `gradient()`, `divergence()`, `interpolate(x)` e `integrate()`, seja qual for a discretização. O módulo `operators` oferece uma função por operador (`grad`, `div`, `curl`, `laplacian`, `jacobian`, `hessian`, `integrate`, `flux`) que funciona igual em todas essas representações.

Sobre essas primitivas, o pacote traz as peças de treino e otimização usadas pelo resto do projeto: `pp.func` (transformações de `torch.func` com `wrt` por nome e diferenciação implícita através de solvers), `pp.loss` (termos de perda funcionais e um `Balancer` com estratégias como GradNorm, NTK e ReLoBRaLo), `pp.transforms` (transformações invertíveis com proveniência, inclusive adimensionalização de um `PhysicalProblem`), `data` (`PhysicsDataset`, `DataLoader` e amostradores de colocação, fronteira, malha, trajetória, adaptativo e ativo), `optim` (`PhysicsOptimizer` com SLSQP, penalidade, Lagrangiano aumentado, evolução diferencial etc.), `module` (`PhysicsModule`, `Sequential`, `SolverModule` e `Hybrid` para compor redes e solvers) e `fem` (elementos finitos P1 diferenciáveis em torch, para gradientes de forma).

O pacote `pinneapple` (o `pp.*`) reexporta boa parte disso de forma preguiçosa: `pp.loss`, `pp.func`, `pp.transforms`, `pp.PhysicsOptimizer`, `pp.PhysicsModule`, `pp.grad`/`pp.div`/`pp.laplacian`, `pp.use_backend` e o `PhysicsBackend` (mesma API sobre torch e jax). O `__init__` carrega os submódulos que dependem de torch só no primeiro uso.

Três subpacotes atendem o resto do ecossistema. `onboarding` é a primeira hora do usuário: `pp.doctor` (checa a instalação por perfil), `pp.recommend` (frase em inglês ou português vira preset, método e snippet, por pontuação de palavras-chave), `card`/catálogo de presets de `pinneapple_physics` e `pp.new` (gera um projeto independente). `service` dá às apps em `apps/*` e ao servidor MCP fila de jobs em SQLite, chaves de API, medição de uso e uploads, só com a biblioteca padrão. `workflows` orquestra simulações como um DAG de passos (adaptadores `command`, `python`, `openfoam`, `calculix`; executores `local`, `docker`, `slurm`), com retomada, cache por hash de conteúdo e geração de datasets a partir das execuções.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
34	8.747	71	92	233 (20 arquivos)	15

Pasta de testes dedicada: `tests/pinneapple_core` (132 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`backend`	`PhysicsBackend`: uma API de arrays, derivadas e solves sobre torch, jax ou outro motor registrado.	1	7	5
`data`	API de dados física: `PhysicsDataset`, `DataLoader` e amostradores (colocação, fronteira, malha, trajetória, adaptativo, ativo).	1	10	25
`domain`	`Domain`: caixa alinhada aos eixos, opcionalmente cortada por uma função distância com sinal.	1	1	0
`fem`	Elementos finitos P1 densos e diferenciáveis em torch (rigidez, massa, Poisson) para gradientes de forma.	1	5	25
`field`	`Field`, `Grid`, `PointCloud` e `FunctionField`: valores numa grade, malha, nuvem ou função, com gradiente, integral e interpolação.	1	4	0
`func`	`pp.func`: `grad`/`jacobian`/`hessian`/`vmap` com `wrt` por nome, e diferenciação implícita (`implicit_solve`).	1	8	10
`geometry`	`Geometry`: um `Domain` com fronteiras nomeadas e um gerador de malha.	1	1	0
`jsonable`	Conversão de resultados para JSON seguro (NumPy para Python, NaN/Inf para `None`, arredondamento).	1	2	8
`loss`	`pp.loss`: termos de perda funcionais e o `Balancer` de pesos escolhido por nome.	1	11	22
`mesh`	`Mesh`: malhas simpliciais (intervalos, triângulos, tetraedros) com cálculo P1.	1	1	0
`module`	`PhysicsModule` e composição de redes e solvers (`Sequential`, `SolverModule`, `Hybrid`).	1	5	7
`onboarding`	A primeira hora: `doctor`, `recommend`, catálogo de presets (`card`) e `new_project`, com CLI.	6	32	34

sub-módulo	o que faz	módulos	classes + funções	testes *
`operators`	Operadores físicos (`grad`, `div`, `curl`, `laplacian`, `jacobian`, `hessian`, `integrate`, `flux`) para todas as representações.	1	8	0
`optim`	`PhysicsOptimizer`: parâmetros, objetivo e restrições escritos uma vez e minimizados por várias famílias de otimizador.	1	6	8
`service`	Peças de runtime para servidores de apps (fila de jobs, chaves de API, medição de uso, uploads), só com a biblioteca padrão.	6	18	30
`transforms`	`pp.transforms`: transformações invertíveis e componíveis para tabelas, datasets e problemas, com proveniência.	1	10	13
`workflows`	Workflows de simulação como DAG de passos, com executores e adaptadores plugáveis, execuções rastreáveis e datasets a partir das execuções.	7	34	64

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_core`

Ponto de entrada do pacote: exporta `Domain`, `Geometry`, `Mesh`, `Field`, `FunctionField`, `Grid`, `PointCloud` e os operadores, e carrega `func`, `fem`, `loss`, `optim`, `data`, `transforms` etc. preguiçosamente no primeiro acesso.

`pinneapple_core/__init__.py` · 49 linhas · testes: 79

`pinneapple_core.backend`

Define `PhysicsBackend`, interface única de arrays, derivadas e solves, com implementações para torch e jax (float64 por padrão) e um registro para novos motores; a classe base levanta `NotImplementedError` nos métodos abstratos.

`pinneapple_core/backend.py` · 233 linhas · testes: 5 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`PhysicsBackend`	classe	`(name, xp)`	Interface base que todo motor implementa (arrays, `solve`, `integrate`, `grad`, `jacobian`, `vmap`, `jit`); os métodos abstratos levantam `NotImplementedError`.
`TorchPhysicsBackend`	classe	`() -> None`	Implementação de `PhysicsBackend` sobre torch.
`JaxPhysicsBackend`	classe	`() -> None`	Implementação de `PhysicsBackend` sobre jax, com `jax_enable_x64` ligado para casar com os outros motores.
`register_backend`	função	`(name, cls) -> None`	Registra uma classe de backend sob um nome para `set_backend`/`use_backend`; instanciada sob demanda, uma vez.
`list_backends`	função	`() -> list`	Lista os nomes de backends registrados.
`get_physics_backend`	função	`(name=None) -> PhysicsBackend`	Devolve o objeto backend pelo nome ou o backend ativo.

nome	tipo	assinatura	o que faz
`use_backend`	função	`(name)`	Gerenciador de contexto que roda um bloco num backend e restaura o anterior.

pinneapple_core.data

API de dados para PINN, operador neural e problema inverso: um `DataLoader` recebe datasets e amostradores e gera um `Batch` por passo, marcando `requires_grad` nos pontos de colocação.

`pinneapple_core/data.py` · 432 linhas · testes: 25 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Batch`	classe	`(groups, step, meta)`	Dados de um passo de treino, um grupo por fonte (`batch["col"]["x"]`), com atalhos x/y quando há uma só fonte.
`PhysicsDataset`	classe	`(inputs, targets=None, *, coords=None, fields=None, domain=None, name='', provenance=None…)`	Dataset rotulado `inputs[i] -> targets[i]` com nomes de coordenadas e campos, domínio e proveniência; suporta `split`.
`Sampler`	classe	`(coords)`	Base dos amostradores que geram pontos sob demanda; subclasses implementam `sample` (a base levanta `NotImplementedError`).
`CollocationSampler`	classe	`(domain, strategy='uniform', seed=0) -> None`	Pontos interiores para o resíduo da PDE, com estratégia `uniform`, `lhs` ou `sobol`.
`BoundarySampler`	classe	`(domain, part=None, seed=0) -> None`	Pontos de fronteira com normais unitárias externas, na fronteira toda ou numa parte nomeada de uma `Geometry`.
`MeshSampler`	classe	`(mesh, mode='nodes', seed=0) -> None`	Pontos ligados a uma malha: nós, centros de células ou pontos aleatórios dentro das células.
`TrajectorySampler`	classe	`(states, n_in=1, n_out=1, stride=1, seed=0) -> None`	Janelas de trajetórias `(B, T, d)`: `n_in` estados como entrada e os `n_out` seguintes como alvo.
`AdaptiveSampler`	classe	`(base, residual_fn, pool_size=20000, exponent=1.0, uniform_fraction=0.2, seed=0) -> None`	Amostragem adaptativa pelo resíduo: sorteia de um conjunto de candidatos com probabilidade crescente no resíduo, mais uma fração uniforme.
`ActiveSampler`	classe	`(base, score_fn, n_candidates=2000) -> None`	Seleção por aquisição: sorteia candidatos e devolve os de maior `score_fn`.
`DataLoader`	classe	`(sources, batch_size=256, steps=None, shuffle=True, drop_last=False, seed=0, physics_awar…)`	Gera lotes de datasets e amostradores; com `physics_aware` liga `requires_grad`, ajusta dtype/dispositivo e anexa nomes de coordenadas.

pinneapple_core.domain

`Domain` como caixa em qualquer dimensão ou caixa cortada por SDF (formas CSG 2D de `pinneapple_design`): pertinência, volume e amostragem de interior e fronteira; fronteira de domínios não-caixa só em 2D (`NotImplementedError`). [sem teste direto]

`pinneapple_core/domain.py` · 190 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Domain`	classe	`(lo, hi, *, sdf=None, shape=None) -> None`	Região de R^d: caixa ou caixa cortada por SDF, com pertinência, volume e amostragem de interior e fronteira.

pinneapple_core.fem

Elementos finitos P1 em torch com matrizes densas, de modo que a solução é diferenciável em relação às coordenadas dos vértices; pensado para malhas de até alguns milhares de nós.

`pinneapple_core/fem.py` · 110 linhas · testes: 25

nome	tipo	assinatura	o que faz
`cell_geometry`	função	`(points, cells) -> tuple`	Volumes e gradientes baricêntricos de cada célula, diferenciáveis nas coordenadas dos pontos.
`stiffness`	função	`(points, cells, kappa=1.0) -> Tensor`	Matriz de rigidez densa `(N, N)` com coeficiente `kappa`.
`mass`	função	`(points, cells) -> Tensor`	Matriz de massa consistente densa `(N, N)`.
`integrate_p1`	função	`(points, cells, values) -> Tensor`	Integral do interpolante P1 de valores nodais, diferenciável nos pontos e nos valores.
`solve_poisson`	função	`(points, cells, source, dirichlet_nodes, dirichlet_values=0.0, kappa=1.0) -> Tensor`	Resolve `-div(kappa grad u) = source` com Dirichlet em nós dados, diferenciável pela solução linear.

pinneapple_core.field

`Field` liga valores ao suporte (`Grid`, `Mesh` ou `PointCloud`) e oferece gradiente, divergência, interpolação e integral; `FunctionField` dá a mesma interface para uma função torch via autograd. [sem teste direto]

`pinneapple_core/field.py` · 407 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Grid`	classe	`(axes) -> None`	Grade produto tensorial definida por eixos 1D estritamente crescentes.
`PointCloud`	classe	`(points, *, domain=None, weights=None, k_neighbors=None) -> None`	Pontos dispersos com pesos de quadratura opcionais e vizinhos para derivadas.
`Field`	classe	`(values, support, name=None) -> None`	Valores de uma grandeza numa `Grid`, `Mesh` ou `PointCloud`, com gradiente, divergência, interpolação e integral.
`FunctionField`	classe	`(fn, domain=None, dim=None) -> None`	Campo definido por uma função torch (expressão analítica ou rede, como numa PINN), com derivadas por autograd.

pinneapple_core.func

Transformações de função (`grad`, `jacrev`, `jacfwd`, `jacobian`, `hessian`, `vmap`) sobre `torch.func` com `wrt` por nome ou posição, e `implicit_solve`, que diferencia através de um solver por uma solução adjunta.

pinneapple_core/func.py · 180 linhas · testes: 10

nome	tipo	assinatura	o que faz
grad	função	(fn, wrt=0, *, has_aux=False) -> Callable	Gradiente de uma função escalar em relação ao(s) argumento(s) wrt, por nome ou posição.
jacrev	função	(fn, wrt=0, *, has_aux=False) -> Callable	Jacobiana por modo reverso (barata com poucas saídas).
jacfwd	função	(fn, wrt=0, *, has_aux=False) -> Callable	Jacobiana por modo direto (barata com poucas entradas).
jacobian	função	(fn, wrt=0, *, mode='auto') -> Callable	Jacobiana com mode rev, fwd ou auto.
hessian	função	(fn, wrt=0) -> Callable	Hessiana de uma função escalar em relação a wrt.
vmap	função	(fn, in_dims=0, out_dims=0, **kwargs) -> Callable	Vetoriza uma função sobre uma dimensão de lote (torch.func.vmap).
newton_solve	função	(residual, x0, *params, tol=1e-10, max_iter=50) -> Tensor	Iteração de Newton amortecida para residual(x, *params) = 0 com Jacobiana densa.
implicit_solve	função	(residual, x0, *params, solver=None, tol=1e-10, max_iter=50) -> Tensor	Resolve residual(x, *params) = 0 e diferencia pela solução com o teorema da função implícita (uma solução adjunta no backward).

pinneapple_core.geometry

Geometry acrescenta fronteiras nomeadas (x_min, x_max, ...) e geração de malha a um Domain; malhar domínios não-caixa só funciona em 2D (NotImplementedError). [sem teste direto]

pinneapple_core/geometry.py · 114 linhas · **sem teste direto** · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Geometry	classe	(domain, boundaries=None) -> None	Um Domain com partes de fronteira nomeadas e um método mesh.

pinneapple_core.jsonable

Implementação única da conversão de resultados para JSON usada por apps, servidor MCP, lab e ferramentas de dados: escalares e arrays NumPy viram Python, NaN/Inf viram None.

pinneapple_core/jsonable.py · 62 linhas · testes: 8

nome	tipo	assinatura	o que faz
round_sig	função	(x, digits=6, small=0.001) -> float	Arredonda por casas decimais para valores grandes e por dígitos significativos para valores pequenos.
to_jsonable	função	(obj, *, digits=None, nonfinite=None, drop_private=False, unknown='keep') -> Any	Devolve uma cópia de um objeto que json.dumps aceita, com opção de arredondar e tratar não-finitos.

pinneapple_core.loss

Termos de perda como funções simples (`pde`, `boundary`, `supervised`, `conservation`, `energy`, `symmetry`, `inverse`), soma ponderada (`combine`) e o `Balancer` de pesos dinâmicos escolhido por nome.

`pinneapple_core/loss.py` · 266 linhas · testes: 22

nome	tipo	assinatura	o que faz
`Balancer`	classe	`(strategy='fixed', names=None, model=None, weights=None, update_every=100, *, schedule=No…)`	Ponderação de perdas escolhida por nome de estratégia (fixa, aprendida, GradNorm, NTK, ReLoBRaLo, SoftAdapt, currículo etc.).
`causal_weights`	função	`(residual, t, n_chunks=32, epsilon=1.0) -> Tensor`	Pesos causais por ponto, que decaem com o resíduo acumulado de fatias de tempo anteriores.
`pde`	função	`(residual, *, weights=None, t=None, causal_epsilon=None, n_chunks=32) -> Tensor`	Resíduo quadrático médio, com pesos opcionais e versão causal no tempo.
`boundary`	função	`(pred, target=0.0) -> Tensor`	Erro quadrático médio em relação aos valores de contorno ou inicial.
`supervised`	função	`(pred, target, kind='mse', *, delta=1.0) -> Tensor`	Ajuste a dados com `mse`, `mae`, `huber` ou `relative_l2`.
`conservation`	função	`(quantity, target=0.0, scale=None) -> Tensor`	Desvio quadrático de uma grandeza conservada em relação ao alvo, normalizado por escala.
`energy`	função	`(e, reference=None, mode='conserve') -> Tensor`	Penalidade sobre a energia ao longo do tempo: conservada ou não crescente.
`symmetry`	função	`(model, x, transform, output_transform=None) -> Tensor`	Penalidade por quebrar uma simetria: `mean((f(T x) - S f(x))^2)`.
`inverse`	função	`(pred, observed, params=None, prior=None, reg=0.0) -> Tensor`	Desajuste às observações mais regularização dos parâmetros desconhecidos em direção a um prior.
`combine`	função	`(terms, weights=None) -> Tensor`	Soma ponderada de termos nomeados; peso para termo ausente levanta `KeyError`.
`list_strategies`	função	`() -> List[str]`	Lista os nomes de estratégias aceitos pelo `Balancer`.

pinneapple_core.mesh

`Mesh` com pontos `(N, d)` e células `(M, d+1)`; campos nodais P1, integral exata para dados P1 e gradiente exato para dados lineares. [sem teste direto]

`pinneapple_core/mesh.py` · 291 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Mesh`	classe	`(points, cells) -> None`	Malha simplicial não estruturada em R^d (d = 1, 2 ou 3) com construtores estruturados e cálculo P1.

pinneapple_core.module

`PhysicsModule` é o `nn.Module` da física: redes, solvers e híbridos se compõem com `Sequential`, `SolverModule` e `Hybrid`, e têm um `fit` simples.

pinneapple_core/module.py · 234 linhas · testes: 7

nome	tipo	assinatura	o que faz
PhysicsModule	classe	(kind)	Classe base de modelos e solvers com kind, describe(), num_parameters() e um fit simples.
Lambda	classe	(fn) -> None	Envolve uma função simples sem parâmetros como componente.
Sequential	classe	(*parts) -> None	Encadeia componentes (módulos, solvers ou funções), a saída de um é a entrada do próximo.
SolverModule	classe	(fn, params=None, trainable=(), name=None) -> None	Um solver como componente, com parâmetros treináveis opcionais; from_method usa um backend de pp.solve (não diferenciável).
Hybrid	classe	(solver, corrector, mode='residual', corrector_input=None, detach_solver=False) -> None	Solver mais correção neural (por exemplo modo residual), treinável através do solver.

pinneapple_core.onboarding

Subpacote da primeira hora com a biblioteca: reexporta doctor, recommend, card e new_project.

pinneapple_core/onboarding/__init__.py · 38 linhas · testes: 34

pinneapple_core.onboarding.catalog

Catálogo de presets: completa o registro de pinneapple_physics.pde_environment.presets com descrição, tags, método que roda hoje e um snippet executável; a busca é por palavras-chave (inglês ou português), não um modelo.

pinneapple_core/onboarding/catalog.py · 817 linhas · testes: 34

nome	tipo	assinatura	o que faz
Card	classe	(name, domain, description, tags, coordinates, fields, spatial_dim, transient, domain_bou…)	Descrição de um preset: domínio, coordenadas, campos, condições, solver, método, snippet, presets relacionados e disponibilidade.
best_method	função	(name) -> str	O método mais barato que roda hoje para o preset: analítico, senão solver de referência, senão rede treinada.
needs_geometry_tags	função	(name) -> bool	Diz se o preset liga condições de contorno a tags de geometria nomeadas.
tag_names	função	(name) -> Tuple[str, ...]	Tags de geometria citadas pelas condições do preset.
default_geometry_note	função	(name) -> str	Nota sobre qual geometria canônica o preset fornece para as fronteiras com tag, ou string vazia.
trainable	função	(name) -> bool	Diz se dá para treinar uma rede neste preset só pelo nome.
probe	função	(name) -> Tuple[bool, str]	Tenta uma vez o método mais barato do preset e informa se funcionou, com o motivo.

nome	tipo	assinatura	o que faz
`runnable`	função	`(name) -> bool`	Diz se o preset produz um campo nesta máquina agora, sem geometria extra nem solver a instalar.
`working_method`	função	`(name) -> str`	O método que funciona aqui para o preset (o sondado após fallback, senão o preferido).
`solver_available`	função	`(solver) -> bool`	Diz se esta máquina consegue rodar o solver de referência nomeado.
`solver_requirement`	função	`(solver) -> str`	Comando de instalação de um solver de referência ausente, ou string vazia.
`snippet`	função	`(name, method=None, *, train=False) -> str`	Menor programa correto para um preset, executável como impresso.
`standalone_snippet`	função	`(name) -> str`	Programa que resolve a mesma física do preset descrita do zero.
`card`	função	`(name, *, with_related=True, check=True) -> Card`	Monta o `Card` de um preset; nome desconhecido levanta `KeyError` com sugestões próximas.
`describe`	função	`(name) -> Dict[str, Any]`	O `card` de um preset como dicionário pronto para JSON.
`catalogue`	função	`() -> List[Card]`	Todos os presets registrados como cards, ordenados por domínio e nome.
`domains`	função	`() -> Dict[str, List[str]]`	Nomes de presets agrupados por domínio.
`format_card`	função	`(c, width=78) -> str`	Formata um card como texto simples, sem cor nem caixas.
`search`	função	`(text, top=5, *, min_score=0.0) -> List[Dict[str, Any]]`	Ordena presets contra um texto livre (inglês ou português) por pontuação de palavras-chave.

`pinneapple_core.onboarding.cli`

Comandos `pinneapple doctor | recommend | card | new`, todos offline e com saída de texto simples.

`pinneapple_core/onboarding/cli.py` · 153 linhas · testes: 34

nome	tipo	assinatura	o que faz
`doctor_main`	função	`(argv) -> int`	Implementa o subcomando `pinneapple doctor`.
`recommend_main`	função	`(argv) -> int`	Implementa o subcomando `pinneapple recommend`.
`card_main`	função	`(argv) -> int`	Implementa o subcomando `pinneapple card`.
`new_main`	função	`(argv) -> int`	Implementa o subcomando `pinneapple new`.
`main`	função	`(argv=None) -> int`	Despacha `pinneapple <doctor|recommend|card|new> ...`.

`pinneapple_core.onboarding.doctor`

`pp.doctor`: verifica a instalação contra um perfil (`start`, `cfd`, `research`, `enterprise`, `all`) e imprime um comando de correção por falha.

`pinneapple_core/onboarding/doctor.py` · 319 linhas · testes: 34

nome	tipo	assinatura	o que faz
`Check`	classe	`(name, ok, detail, fix, required)`	Um item verificado, com resultado (`None` se não pôde rodar), detalhe, correção e se é obrigatório.
`DoctorReport`	classe	`(profile, checks)`	Relatório do `doctor` para um perfil, com a lista de checagens e as falhas.
`profiles`	função	`() -> Dict[str, str]`	Perfis aceitos pelo `doctor` e a frase que descreve cada um.
`doctor`	função	`(profile='start', *, verbose=True) -> DoctorReport`	Verifica a instalação contra um perfil e, com `verbose`, imprime o relatório.

`pinneapple_core.onboarding.newproject`

`pp.new` / `pinneapple new`: escreve um projeto independente deste repositório, com modelos `plain` (só YAML), `research` e `cfd`.

`pinneapple_core/onboarding/newproject.py` · 842 linhas · testes: 34

nome	tipo	assinatura	o que faz
`plan`	função	`(name, template='plain', preset='burgers_1d') -> Dict[str, Any]`	Mostra o que `new_project` escreveria, sem escrever (ensaio).
`new_project`	função	`(name, *, dir=None, template='plain', preset='burgers_1d', verbose=True) -> str`	Escreve um projeto independente a partir de um template e preset e devolve a pasta.

`pinneapple_core.onboarding.recommend`

`pp.recommend`: transforma uma frase em preset, método e modelo, com o motivo e os termos que casaram; é pontuação por palavras-chave e pode errar. [sem teste direto]

`pinneapple_core/onboarding/recommend.py` · 299 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Recommendation`	classe	`(query, preset, method, model, confidence, reason, snippet, matched, alternatives, next_s…)`	Resultado do `recommend`: preset, método, modelo, confiança, motivo, snippet, termos casados, alternativas e próximos passos.
`recommend`	função	`(text, *, top=3) -> Recommendation`	Transforma a descrição de um problema em preset, método e modelo, com os motivos.

`pinneapple_core.operators`

Operadores físicos com a mesma chamada em campo contínuo (autograd), grade, malha simplicial e nuvem de pontos; em `Field` devolvem `Field`, em funções devolvem tensores com grafo. [sem teste direto]

`pinneapple_core/operators.py` · 275 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`grad`	função	`(f, x=None)`	Gradiente: `(N, d)` para escalar, `(N, c, d)` para `c` componentes.
`div`	função	`(f, x=None)`	Divergência de um campo vetorial.

nome	tipo	assinatura	o que faz
jacobian	função	(f, x=None)	Jacobiana (N, c, d) de um campo.
curl	função	(f, x=None)	Rotacional: vetor em 3D, escalar (componente z) em 2D.
hessian	função	(f, x=None)	Hessiana (N, d, d) de um campo escalar.
laplacian	função	(f, x=None)	Soma das segundas derivadas, por componente em campos vetoriais.
integrate	função	(f, **kwargs)	Integral sobre o suporte (trapézio na grade, P1 exato na malha, quadratura ou Monte Carlo na nuvem).
flux	função	(f, where=None, *, domain=None, n=20000, seed=0)	Fluxo de um campo vetorial pela fronteira, integral f . n dS.

pinneapple_core.optim

PhysicsOptimizer: o problema (parâmetros com limites, objetivo, restrições de igualdade e desigualdade) é escrito uma vez e minimize(method) escolhe a família (SLSQP, adjunto, penalidade, Lagrangiano aumentado, evolução diferencial etc.).

pinneapple_core/optim.py · 395 linhas · testes: 8

nome	tipo	assinatura	o que faz
Constraint	classe	(fn, kind, name)	Restrição fn(x) <= 0 (ineq) ou fn(x) == 0 (eq).
OptimizeResult	classe	(x, params, fun, violation, feasible, n_evals, method, family, success, message, wall_tim…)	Resultado de minimize: ponto, objetivo, violação, viabilidade, avaliações, método, tempo e histórico.
PhysicsOptimizer	classe	(parameters, objective, constraints=(), *, backend='torch', argument='vector', tol=0.001)…)	Parâmetros com limites, objetivo e restrições, minimizados pelo método escolhido em minimize.
ineq	função	(fn, name='') -> Constraint	Cria uma restrição de desigualdade fn(x) <= 0.
eq	função	(fn, name='') -> Constraint	Cria uma restrição de igualdade fn(x) == 0.
list_methods	função	() -> List[str]	Lista os métodos aceitos por PhysicsOptimizer.minimize.

pinneapple_core.service

Peças de runtime para servidores (apps, servidor MCP, serviços próprios) só com a biblioteca padrão; as rotas FastAPI ficam em apps/_shared/appkit.py.

pinneapple_core/service/__init__.py · 45 linhas · testes: 30

nome	tipo	assinatura	o que faz
data_dir	função	(app, prefix=None, environ=None) -> Path	Diretório de estado de uma app: <PREFIX>_DATA_DIR, senão $PINNEAPPLE_APP_DATA/<app>, senão /data/<app>.

pinneapple_core.service.apikeys

Chaves de API criadas uma vez (a chave em texto só aparece na criação), guardadas como SHA-256 e revogáveis; clientes enviam `X-API-Key`. [sem teste direto]

`pinneapple_core/service/apikeys.py` · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`APIKeyStore`	classe	`(db_path, app='')`	Store SQLite de chaves de API: criar, listar, verificar e revogar.
`hash_key`	função	`(key) -> str`	SHA-256 de uma chave, usado para guardar e procurar.

pinneapple_core.service.db

Utilitários SQLite dos stores de serviço: uma conexão por thread, WAL e transações `BEGIN IMMEDIATE`. [sem teste direto]

`pinneapple_core/service/db.py` · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LocalDB`	classe	`(db_path, schema='')`	Uma conexão SQLite por thread para o mesmo arquivo, com esquema aplicado.
`connect`	função	`(db_path) -> sqlite3.Connection`	Abre conexão em autocommit com WAL e timeout de espera.
`transaction`	função	`(conn)`	Gerenciador de contexto de transação `BEGIN IMMEDIATE` com rollback em erro.

pinneapple_core.service.jobs

Fila de jobs persistente em SQLite com workers em thread ou processo, progresso, logs, cancelamento cooperativo, novas tentativas e retomada após reinício. [sem teste direto]

`pinneapple_core/service/jobs.py` · 510 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`JobCancelled`	classe	`()`	Levantada dentro de um job por `ctx.progress`/`ctx.check_cancel` após pedido de cancelamento.
`JobTimeout`	classe	`()`	Levantada dentro de um job em thread que passou do limite de tempo.
`JobContext`	classe	`(db_path, job_id, params, workdir, attempt, deadline=None)`	O que um job em execução vê: id, parâmetros, diretório de trabalho, progresso e log.
`JobQueue`	classe	`(db_path, *, app='', max_concurrent=2, executor='thread', workdir=None, poll_interval=0.2…)`	Fila persistente em SQLite com registro de funções, `submit`, `wait`, cancelamento, novas tentativas e limite de concorrência entre workers.

pinneapple_core.service.metering

Medição de uso por app, principal e unidade, com limites gratuitos por mês (UTC) vindos do código e do ambiente; só mede, não cobra. [sem teste direto]

`pinneapple_core/service/metering.py` · 110 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UsageLimitExceeded`	classe	`(unit, used, amount, limit, period)`	Erro de uso acima do limite gratuito do período para uma unidade.
`UsageMeter`	classe	`(db_path, app, limits=None)`	Registra e verifica uso por principal e unidade (`consume`, `usage`, `totals`).
`current_period`	função	`(ts=None) -> str`	Período corrente em UTC no formato ano-mês.
`limits_from_env`	função	`(prefix, defaults, environ=None) -> dict[str, float | None]`	Limites gratuitos por unidade, com padrões sobrescritos por `<PREFIX>_FREE_<UNIT>`.

`pinneapple_core.service.uploads`

Armazena uploads com limite de tamanho, sufixos permitidos, nomes seguros e SHA-256 em `<root>/<id>/`. [sem teste direto]

`pinneapple_core/service/uploads.py` · 106 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UploadTooLarge`	classe	`()`	Erro de upload acima do tamanho máximo.
`UploadRejected`	classe	`()`	Erro de upload recusado (por exemplo sufixo não permitido).
`UploadStore`	classe	`(root, max_bytes=50 * 2 ** 20, allowed_suffixes=None)`	Guarda uploads em disco com metadados, e permite ler, listar, apagar e podar por idade.
`safe_name`	função	`(name) -> str`	Reduz um nome de arquivo ao nome base com caracteres seguros, nunca vazio nem oculto.

`pinneapple_core.transforms`

Transformações invertíveis e componíveis (`>>`) sobre tabelas, `PhysicsDataset` e, para as de escala, `PhysicalProblem`, registrando proveniência a cada `apply`; as não-escalares levantam `NotImplementedError` em nível de problema.

`pinneapple_core/transforms.py` · 636 linhas · testes: 13 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`NotInvertible`	classe	`()`	Erro de transformação (ou passo de composição) sem inversa.
`Transform`	classe	`()`	Base das transformações: `forward`/`inverse` em tabelas e `params()` para proveniência.
`Compose`	classe	`(steps) -> None`	Sequência de transformações; `inverse` roda na ordem reversa.
`Scale`	classe	`(scale, shift=None) -> None`	Escala afim de colunas nomeadas; num `PhysicalProblem` reescala coordenadas, domínio e parâmetros.
`Nondimensionalize`	classe	`(problem=None, *, length=None, time=None, field=None, scale=None, shift=None) -> None`	Leva um problema a variáveis adimensionais e permite trazer predições de volta às unidades originais.
`Coordinate`	classe	`(kind, inputs, outputs, center=None) -> None`	Mudança de coordenadas em colunas: polar, cilíndrica, esférica ou log.

nome	tipo	assinatura	o que faz
`Symmetry`	classe	`(kind, coords, vectors=(), *, axis=None, angle=0.0, center=None) -> None`	Operação de simetria (reflexão, rotação, com campos vetoriais) para aumento de dados ou teste de equivariância.
`Periodic`	classe	`(coord, period, harmonics=1, lo=0.0) -> None`	Troca uma coordenada periódica por `cos`/`sin` da fase, tornando a rede periódica por construção.
`FourierFeatures`	classe	`(coords, n_features=16, scale=1.0, seed=0, prefix='ff') -> None`	Features de Fourier aleatórias de algumas colunas.
`list_pde_rules`	função	`() -> List[str]`	Tipos de PDE cujos parâmetros `Scale`/`Nondimensionalize` sabem reescalar.

`pinneapple_core.workflows`

Orquestra ferramentas existentes (solvers, Docker, Slurm) como DAG de passos; reexporta `Workflow`, `Step`, `WorkflowRunner`, executores, adaptadores, templates e `dataset_from_run`.

`pinneapple_core/workflows/__init__.py` · 79 linhas · testes: 64

`pinneapple_core.workflows.adapters`

Adaptadores por `kind` de passo (`command`, `python`, `openfoam`, `calculix`): preparam o diretório, rodam comandos pelo executor e devolvem saídas em JSON.

`pinneapple_core/workflows/adapters.py` · 285 linhas · testes: 20

nome	tipo	assinatura	o que faz
`StepFailed`	classe	`()`	Erro de passo que rodou e falhou (saída não zero, erro do solver, saída ausente).
`StepContext`	classe	`(run_id, step, workdir, executor, upstream, log, should_stop, attempt)`	Contexto de um passo: id da execução, passo, diretório, executor, saídas a montante, log e tentativa.
`StepCancelled`	classe	`()`	Erro de passo cancelado.
`Adapter`	classe	`(kind)`	Base abstrata de um tipo de passo (`kind`) com verificação de disponibilidade e `run`.
`CommandAdapter`	classe	`()`	Passo `command`: roda `argv` com `{param}` preenchidos e lê `outputs.json` se existir.
`PythonAdapter`	classe	`()`	Passo `python`: chama `module:fn` no processo (só executor local) ou roda um script com o executor.
`OpenFOAMAdapter`	classe	`()`	Passo `openfoam`: copia um caso, aplica edições, roda blockMesh e o solver e lê a saúde do log.
`CalculiXAdapter`	classe	`()`	Passo `calculix`: copia um `.inp` e roda `ccx`.
`resolve_function`	função	`(ref) -> Callable`	Importa e devolve a função de uma referência `module:function`.
`register_adapter`	função	`(adapter) -> None`	Registra um adaptador para tornar executáveis passos do seu `kind`.

nome	tipo	assinatura	o que faz
`get_adapter`	função	`(kind) -> Adapter`	Devolve o adaptador registrado para um `kind`.

pinneapple_core.workflows.dag

Definição do workflow como DAG (`Step`, `Workflow`): validação de dependências e ciclos, ordem, JSON e expansão por tabela de casos com preenchimento de `{param}`. [sem teste direto]

`pinneapple_core/workflows/dag.py` · 208 linhas · **sem teste direto** · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`WorkflowError`	classe	`()`	Erro de workflow inválido (passo desconhecido, id duplicado, ciclo).
`Step`	classe	`(id, kind, params, needs, executor, executor_options, retries, timeout, inputs, group)`	Um passo do DAG: id, `kind`, parâmetros, dependências, executor, tentativas, timeout e arquivos de entrada.
`Workflow`	classe	`(name, steps, fail_fast, max_parallel, metadata)`	Conjunto de passos com validação, ordem topológica, `fail_fast`, paralelismo máximo e salvar/carregar JSON.
`fill`	função	`(obj, values) -> Any`	Preenche `{name}` em strings, recursivamente; uma string que é só um placeholder recebe o valor tal qual.
`workflow_from_cases`	função	`(name, cases, template, *, id_column=None, final=(), **kw) -> Workflow`	Copia os passos de um template uma vez por caso de uma tabela, com ids `<caso>.<passo>`.

pinneapple_core.workflows.datasets

Leva uma tabela de casos a um workflow e as execuções terminadas a uma tabela de treino com `DatasetSpec` (marcado `partial` se faltam casos). [sem teste direto]

`pinneapple_core/workflows/datasets.py` · 115 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`read_cases`	função	`(path)`	Lê uma tabela de casos (CSV, Parquet, HDF5, JSON) e o contrato anexado, se houver.
`id_column_of`	função	`(df, contract=None) -> str | None`	Descobre a coluna de id dos casos pelo contrato ou por nomes comuns.
`dataset_from_run`	função	`(runner, run_id, cases, *, output_step, path, id_column=None, outputs=None, skeleton=None…)`	Grava a tabela de dataset de uma execução (parâmetros + saídas numéricas do passo) e devolve o `DatasetSpec`.

pinneapple_core.workflows.engine

`WorkflowRunner`: executa workflows com estado e log por passo em SQLite, novas tentativas, retomada e pulo de passos já feitos por hash de conteúdo. [sem teste direto]

`pinneapple_core/workflows/engine.py` · 367 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WorkflowRunner`	classe	`(root, *, cache=True, input_root=None)`	Executa e retoma workflows, guarda estado, logs e cache por hash em SQLite sob uma raiz.
`sha256_file`	função	`(path) -> str`	SHA-256 do conteúdo de um arquivo.
`step_hash`	função	`(step, upstream_hashes, root=None) -> str`	Hash de conteúdo de um passo (tipo, parâmetros, opções, arquivos de entrada, hashes a montante).

`pinneapple_core.workflows.executors`

Executores plugáveis que rodam o comando de um passo: `local` (subprocesso), `docker` (sem rede por padrão) e `slurm` (script sbatch, só escrito por padrão com `dry_run=True`). [sem teste direto]

`pinneapple_core/workflows/executors.py` · 236 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ExecResult`	classe	`(returncode, seconds, command, dry_run, timed_out, cancelled, details)`	Resultado da execução de um comando: código de saída, tempo, comando, ensaio, timeout, cancelamento.
`Executor`	classe	`(**options)`	Base de executores: rodam um comando de um passo; subclasses definem `name` e `run`.
`LocalExecutor`	classe	`()`	Executor por subprocesso local, com timeout e parada sob pedido.
`DockerExecutor`	classe	`()`	Executor `docker run --rm` com o diretório do passo em /work, sem rede por padrão e com limites.
`SlurmExecutor`	classe	`()`	Executor que escreve um script sbatch por passo (submete só fora do modo de ensaio).
`register_executor`	função	`(name, cls) -> None`	Registra uma classe de executor sob um nome usável em `executor=`.
`get_executor`	função	`(name, **options) -> Executor`	Instancia o executor registrado com as opções dadas.

`pinneapple_core.workflows.templates`

Templates de workflow seguros para servidor público (`slab_heat`, `beam_calculix`, `cavity_openfoam`), que validam cada caso contra o espaço de parâmetros.

`pinneapple_core/workflows/templates.py` · 213 linhas · testes: 20

nome	tipo	assinatura	o que faz
`Template`	classe	`(id, title, description, space, steps, output_step, final, outputs, solver)`	Template de workflow: espaço de parâmetros, passos por caso, passo de saída e solver necessário.
`slab_heat_solve`	função	`(kw, ctx) -> dict`	Perfil de temperatura da placa em `t_end` pela série exata; escreve `profile.csv`.
`collect_outputs`	função	`(kw, ctx) -> dict`	Resumo sobre todos os passos a montante (contagem, mínimo e máximo de cada saída numérica); escreve `summary.csv`.

nome	tipo	assinatura	o que faz
`cantilever_deck`	função	`(kw, ctx) -> dict`	Gera o `job.inp` de uma viga em balanço (hexaedros C3D8I) e a deflexão de ponta de Euler-Bernoulli.
`cantilever_compare`	função	`(kw, ctx) -> dict`	Compara a deflexão máxima do CalculiX com a de Euler-Bernoulli e devolve a razão.

Exemplos de uso reais do repositório

PINN de Poisson sobre as primitivas — `examples/core_primitives/01_poisson_pinn.py`

```
        from pinneapple_core import Field, FunctionField, Geometry

        torch.manual_seed(0)
        geom = Geometry.box([0, 0], [1, 1])
        net = torch.nn.Sequential(
            torch.nn.Linear(2, 48), torch.nn.Tanh(),
            torch.nn.Linear(48, 48), torch.nn.Tanh(),
            torch.nn.Linear(48, 1),
        )
        u = FunctionField(net, geom.domain)

        source = lambda x: 2 * np.pi**2 * torch.sin(np.pi * x[:, 0]) *
torch.sin(np.pi * x[:, 1])
        x_int = torch.tensor(geom.sample_interior(2000, seed=0),
dtype=torch.float32)
        x_bnd = torch.tensor(geom.sample_boundary(400, seed=1),
dtype=torch.float32)
```

DataLoader com amostradores e pp.loss — `examples/data_api/01_pinn_operator_inverse.py`

```
        geom = Geometry.box([0.0], [1.0])
        net = mlp(1, 1)
        loader = DataLoader({"col": CollocationSampler(geom, "lhs"), "bc":
BoundarySampler(geom)}, batch_size={"col": 256, "bc": 2}, steps=1500)
        opt = torch.optim.Adam(net.parameters(), lr=3e-3)
        for batch in loader:
            x = batch["col"]["x"]
# already requires_grad
            r = -second(net(x), x) - PI**2 * torch.sin(PI * x)
            loss = L.combine({"pde": L.pde(r), "bc":
L.boundary(net(batch["bc"]["x"]))}, {"bc": 10.0})
            opt.zero_grad(); loss.backward(); opt.step()
```

Workflow com template e dataset — `docs/guides/workflow_runner.md`

```
        from pinneapple_core.workflows import Step, Workflow,
WorkflowRunner, TEMPLATES, dataset_from_run

        runner = WorkflowRunner("work/runner")
        t = TEMPLATES["slab_heat"]
        cases = t.example_cases(20)                       # or
read_cases("campaign.parquet")
        run = runner.run(t.workflow(cases, max_parallel=4))
        runner.run(run_id=run["id"])                      # resume (no-op
when everything finished)
        spec, path = dataset_from_run(runner, run["id"], cases,
output_step="solve", path="slab.parquet")
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 19 de 34.
- Funções de teste em arquivos que importam algum módulo do pacote: 233.

Limitações conhecidas

- `fem` usa matrizes densas: o docstring diz que serve para malhas de até alguns milhares de nós (casos de referência, protótipos de otimização de forma, testes).
- Domínios e geometrias não-caixa só são suportados em 2D (formas CSG de `pinneapple_design`); medir ou malhar a fronteira em outras dimensões levanta `NotImplementedError`.
- `PhysicsBackend` base e `Sampler` base levantam `NotImplementedError`; o backend jax exige jax instalado e liga float64 globalmente.
- Em `transforms`, só `Scale` e `Nondimensionalize` têm forma em nível de problema; as demais levantam `NotImplementedError` ao serem aplicadas a um `PhysicalProblem`.
- `recommend` e `search` do onboarding são pontuação por palavras-chave sobre um catálogo curado, não um modelo; podem errar e dizem quais termos casaram.
- Os adaptadores `openfoam` e `calculix` e os templates `beam_calculix`/`cavity_openfoam` precisam de OpenFOAM ou `ccx` instalados; o executor `slurm` só escreve o script por padrão (`dry_run=True`).
- `service` mede uso mas não cobra; o cancelamento de jobs em modo thread é cooperativo (só vale quando o job chama `ctx.progress`/`ctx.check_cancel`).
- Vários módulos não têm teste que os importe diretamente (`domain`, `field`, `mesh`, `geometry`, `operators`, `onboarding.recommend`, os stores de `service` e partes de `workflows`); são exercitados pelos testes que importam o pacote ou o subpacote.

3. `pinneapple_catalog`

Objetivo

`pinneapple_catalog` é o catálogo do que o PINNeAPPle conhece, sempre com a fonte. Ele tem duas partes. `resources` lista recursos públicos de Physics AI (datasets, conjuntos de CAD/geometria, modelos pré-treinados e benchmarks hospedados), cada um com licença conferida, classificação de uso e, quando houver, o repositório no Hugging Face. `methods` lista cada solver, método de treino, equação física e problema (preset) da biblioteca, com o caminho do código, as referências bibliográficas e o status de validação.

O problema que o pacote resolve é de rastreabilidade e de licença. Um componente que quer um recurso público pergunta primeiro ao catálogo: `require_allowed` aplica a mesma trava de licença de `pinneapple_neural._licencas` e recusa recursos sem uso comercial (ou com licença não verificada) quando `PINNEAPPLE_COMMERCIAL_MODE=1`. `fetch` baixa o recurso do Hugging Face Hub só depois dessa trava.

No lado dos métodos, o status de validação não é escrito à mão. O script `scripts/build_method_status.py` varre `tests/` e grava `pinneapple_catalog/method_status.json`. Um item é `validated` quando algum teste o compara com uma referência independente (forma fechada, valor publicado ou solução manufaturada), `tested` quando há testes sem essa comparação, e `untested` caso contrário. `validated_methods()` devolve só os validados; a página pública `docs/validation_status.md` é conferida contra essa varredura por teste. Os caminhos de código apontam para os outros pacotes (`pinneapple_simulation`, `pinneapple_neural`, `pinneapple_physics`, `pinneapple_systems`) e os problemas apontam para os presets de `pinneapple_physics/pde_environment/presets/`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
3	1.024	2	8	16 (3 arquivos)	1

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`methods`	Inventário de solvers, métodos de treino, equações e problemas, com código, referências e status de validação lido de `method_status.json`.	1	5	8
`resources`	Catálogo de recursos públicos com licença conferida, filtros, trava de licença e download pelo Hugging Face Hub.	1	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_catalog`

Raiz do pacote: reexporta a API de `resources` (`Resource`, `list_resources`, `get_resource`, `require_allowed`, `fetch`); `methods` é importado à parte.

`pinneapple_catalog/__init__.py` · 11 linhas · testes: 8

`pinneapple_catalog.methods`

Lista fixa de `Method` (IDs S = solver, T = treino, E = equação, P = problema) com caminho do código, referências citadas no código e referências acrescentadas, e funções para consultar o status de validação.

`pinneapple_catalog/methods.py` · 569 linhas · testes: 8

nome	tipo	assinatura	o que faz
`Method`	classe	`(id, name, code, probes, refs_in_code, refs_added, equations)`	Registro imutável de um item do catálogo: id, nome, caminhos de código, termos de busca nos testes (`probes`), referências e, para problemas, as equações usadas; expõe `category` e `references`.

nome	tipo	assinatura	o que faz
`get_method`	função	`(method_id) -> Method`	Devolve o `Method` de um id ou levanta `KeyError` se o id não existe.
`list_methods`	função	`(category=None) -> List[Method]`	Lista todos os métodos, ou só os de uma categoria (`solver`, `training`, `equation`, `problem`).
`method_status`	função	`() -> Dict[str, Dict]`	Lê `method_status.json` e devolve, por id, o status e os testes de referência e outros testes; devolve dict vazio se o arquivo não existe.
`validated_methods`	função	`(category=None) -> List[Method]`	Lista oficial: só os métodos cujo status na última varredura é `validated`.

`pinneapple_catalog.resources`

Lista fixa de `Resource` com licença, uso comercial (`yes`/`no`/`unverified`), classificação (`use_now`, `product`, `reference`, `skip`), filtros, trava de licença e download. [sem teste direto pelo inventário; é exercitado via `tests/test_catalog_resources.py`, que importa pela raiz]

`pinneapple_catalog/resources.py` · 444 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Resource`	classe	`(id, name, kind, domain, summary, license, commercial, url, rating, paper, size, hf_repo,…)`	Registro imutável de um recurso público (tipo, domínio, licença, uso comercial, URL, classificação, artigo, tamanho, repositório HF, notas, tags); `research_only` é verdadeiro sempre que o uso comercial não é `yes`.
`list_resources`	função	`(kind=None, domain=None, rating=None, commercial_only=False) -> List[Resource]`	Filtra os recursos por tipo, domínio e classificação; `commercial_only` mantém só os de uso comercial `yes`.
`get_resource`	função	`(resource_id) -> Resource`	Devolve o `Resource` de um id ou levanta `KeyError` listando os ids conhecidos.
`require_allowed`	função	`(resource_id) -> Resource`	Trava de licença: avisa para recursos só de pesquisa e os recusa em modo comercial, via `require_research_use` de `pinneapple_neural._licencas`.
`fetch`	função	`(resource_id, *, allow_patterns=None, local_dir=None, token=None, revision=None) -> str`	Passa pela trava de licença e baixa o recurso com `huggingface_hub.snapshot_download`, devolvendo o caminho local; aceita `allow_patterns` para baixar só parte.

Exemplos de uso reais do repositório

Recursos sem licença comercial contam como só pesquisa — `tests/test_catalog_resources.py`

```
        from pinneapple_catalog import fetch, get_resource, list_resources,
require_allowed
        from pinneapple_neural._licencas import ResearchOnlyError,
ResearchOnlyWarning


        def test_unknown_or_missing_license_counts_as_research_only():
            assert get_resource("pressnet").research_only  # no LICENSE file
            assert get_resource("drivaernet").research_only  # CC BY-NC
```

```
            assert not get_resource("airfrans").research_only  # ODbL allows
commercial use


        def test_commercial_only_filter_excludes_research_only():
            assert all(not r.research_only for r in
list_resources(commercial_only=True))
```

Status de validação cobre todo o catálogo de métodos — `tests/test_catalog_methods.py`

```
        def test_status_covers_every_item_and_validated_is_a_subset():
            status = method_status()
            assert {m.id for m in list_methods()} <= set(status)
            for m in validated_methods():
                assert status[m.id]["status"] == "validated"
                assert status[m.id]["reference_tests"], m.id
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 2 de 3.
- Funções de teste em arquivos que importam algum módulo do pacote: 16.

Limitações conhecidas

- As listas de recursos e de métodos são fixas no código; incluir um item novo exige editar `resources.py` ou `methods.py`.
- O status de validação depende de rodar `scripts/build_method_status.py`; se `method_status.json` estiver desatualizado, `validated_methods()` também estará. Sem o arquivo, `method_status()` devolve vazio.
- A detecção de testes é por presença de strings (`probes`) nos arquivos de teste, não por execução.
- `fetch` exige o extra `hub` (`huggingface_hub`) e só funciona para recursos com `hf_repo`; os demais levantam `ValueError` apontando a URL de origem.
- Licença não verificada (`unverified`) é tratada como não comercial; em `PINNEAPPLE_COMMERCIAL_MODE=1` esses recursos são recusados.
- Alguns caminhos de código e docstrings citam o repositório irmão `PINNeAPPle-CFD`, que pode não estar presente; o teste pula esses caminhos nesse caso.
- `pinneapple_catalog.methods` não é reexportado pela raiz do pacote; é preciso importar `pinneapple_catalog.methods` diretamente.

4. `pinneapple_registry`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
10	1.699	10	3	92 (10 arquivos)	3

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
dataset_store		1	1	0
experiment_memory		1	2	19
experiment_store		1	1	10
gateway		1	2	10
gateway_api		1	2	10
gateway_client		1	2	10
model_store		1	1	13
problem_store		1	1	0
registry		1	1	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_registry`

pinneapple_registry — a local, self-hosted artifact registry: versioned model storage, dataset storage, experiment tracking, and problem-spec history, plus Triton Inference Server export.

`pinneapple_registry/__init__.py` · 45 linhas · testes: 50

`pinneapple_registry.dataset_store`

pinneapple_registry.dataset_store — `DatasetStore`: local, per-problem/scenario storage for simulation datasets (coords + field arrays), with auto-computed summary statistics.

`pinneapple_registry/dataset_store.py` · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
DatasetStore	classe	(root)	

`pinneapple_registry.experiment_memory`

pinneapple_registry.experiment_memory -- `ExperimentMemory`: persistent cross-run memory for an (separately built, not part of this repo) autonomous research loop.

`pinneapple_registry/experiment_memory.py` · 252 linhas · testes: 19

nome	tipo	assinatura	o que faz
ExperimentRecord	classe	(problem_description, physics_domain, approach_summary, outcome, confidence_score, metric…)	One logged attempt at solving a physics problem.
ExperimentMemory	classe	(store=None, *, db_path=None)	Persistent memory of past experiment attempts, backed by the real ExperimentStore.

`pinneapple_registry.experiment_store`

pinneapple_registry.experiment_store — `ExperimentStore`: a SQLite-backed run tracker. Closes the one gap `pinneapple_hub` genuinely has no answer for: it publishes finished models to the Hugging Face Hub,

but nothing in the repo durably records *runs* (start/metrics-per-epoch/finish) …

`pinneapple_registry/experiment_store.py` · 144 linhas · testes: 10

nome	tipo	assinatura	o que faz
`ExperimentStore`	classe	`(db_path)`	

`pinneapple_registry.gateway`

`ModelGateway`: publish physical models as versioned endpoints, run them, and record every run.

`pinneapple_registry/gateway.py` · 457 linhas · testes: 10 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`ModelGateway`	classe	`(root, *, cache_size=8)`	
`ood_verdict`	função	`(flag, policy='warn') -> str`	What `predict` does with one assessed row under `policy`: `"in range"` (predicted), `"warn"` (predicted and flagged as an extrapolation) or `"refuse"` (no prediction).

`pinneapple_registry.gateway_api`

The gateway's REST shape (FastAPI), shared by every app that serves models: `/v1/models/...`.

`pinneapple_registry/gateway_api.py` · 144 linhas · testes: 10

nome	tipo	assinatura	o que faz
`count_rows`	função	`(inputs) -> int`	Rows a request asks for, before the model is loaded.
`gateway_router`	função	`(gateway, *, prefix='/v1', on_rows=None, caller=None, max_rows=10000, tags=('models',))`	

`pinneapple_registry.gateway_client`

`GatewayClient`: call any server that exposes the gateway API (`/v1/models/...`) from Python.

`pinneapple_registry/gateway_client.py` · 83 linhas · testes: 10

nome	tipo	assinatura	o que faz
`GatewayError`	classe	`(status, message)`	
`GatewayClient`	classe	`(base_url, *, api_key=None, timeout=30.0, headers=None)`	

`pinneapple_registry.model_store`

pinneapple_registry.model_store — `ModelStore`: a local, filesystem- backed, versioned store for trained models, paralleling (not replacing) `pinneapple_hub` — see the package docstring in `pinneapple_registry/__init__.py` for why both exist.

`pinneapple_registry/model_store.py` · 246 linhas · testes: 13

nome	tipo	assinatura	o que faz
`ModelStore`	classe	`(root)`	

`pinneapple_registry.problem_store`

pinneapple_registry.problem_store — `ProblemStore`: versioned problem definitions with history.

`pinneapple_registry/problem_store.py` · 98 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ProblemStore`	classe	`(root)`	

`pinneapple_registry.registry`

pinneapple_registry.registry — `ArtifactRegistry`: unifies `ModelStore`/`DatasetStore`/`ExperimentStore`/`ProblemStore` under one root, plus a Triton Inference Server model-repository generator.

`pinneapple_registry/registry.py` · 131 linhas · **sem teste direto**

nome	tipo	assina-tura	o que faz
`ArtifactRegistry`	classe	`(root)`	One root directory, four sub-stores, one experiment-tracking DB::

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 7 de 10.
- Funções de teste em arquivos que importam algum módulo do pacote: 92.

Limitações conhecidas

- *Nenhuma registrada.*

5. `pinneapple_orchestration`

Objetivo

`pinneapple_orchestration` é uma camada opcional de pipelines no estilo Prefect. Ela não traz lógica nova: só encadeia o que já existe em outros pacotes. Os dados vão para o `DatasetStore` e os modelos para o registro de artefatos de `pinneapple_registry`. Os componentes treináveis vêm de `pinneapple_systems.component_library` e o gêmeo digital de `pinneapple_systems.digital_twin`.

O pacote é pequeno de propósito: são cinco flows. `flow_data_generation` roda N vezes uma função de amostragem dada pelo usuário e grava cada cenário. `flow_training` monta um componente registrado, treina e registra a execução num `ExperimentStore`. `flow_benchmark` ranqueia todas as arquiteturas de um `component_type`. `flow_registry_publish` promove um modelo de estágio (`development`

para `staging`/`production`), sempre com confirmação explícita. `flow_digital_twin_prep` monta um `DigitalTwin` a partir de um modelo do registro, só se ele estiver no estágio exigido.

Cada flow chama uma task fina em `tasks/`, e cada task delega ao pacote dono da lógica (por exemplo `ComponentModel.fit`, que usa o `Trainer` comum). Todo flow roda dentro de `flow_run_manifest`, que grava um JSON por execução com parâmetros, SHA do git, log das tasks e artefatos.

O Prefect não é obrigatório. Com o extra `pinneapple[orchestration]` instalado, os decoradores `flow`/`task` são os do Prefect (retries, agendamento, UI). Sem ele, `_prefect_compat` troca os decoradores por funções que não fazem nada, e cada flow vira uma função Python comum, chamável e testável.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
16	591	1	11	6 (1 arquivos)	14

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`_prefect_compat`	Shim que usa os decoradores `flow`/`task` do Prefect quando instalado e, senão, decoradores que devolvem a função sem mudança.	1	0	0
`common`	Utilitários comuns aos flows; hoje só o manifesto de proveniência de cada execução.	2	2	0
`flows`	Os cinco flows públicos (geração de dados, treino, benchmark, publicação no registro, preparo de gêmeo digital).	6	5	6
`tasks`	As tasks finas por trás de cada flow, que delegam a `pinneapple_registry` e `pinneapple_systems`.	6	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_orchestration`

Ponto de entrada do pacote: reexporta `PREFECT_AVAILABLE`, os subpacotes `tasks` e `flows`, `flow_run_manifest` e `FlowRunManifest`.

`pinneapple_orchestration/__init__.py` · 44 linhas · testes: 6

`pinneapple_orchestration._prefect_compat`

[shim] Importa `flow` e `task` do Prefect se existir; senão define decoradores de passagem que aceitam `@flow` e `@flow(...)` e ignoram os kwargs. Define `PREFECT_AVAILABLE`. [sem teste direto]

`pinneapple_orchestration/_prefect_compat.py` · 36 linhas · **sem teste direto** · sinais no código: shim ou obsoleto

pinneapple_orchestration.common

Subpacote que reexporta `flow_run_manifest` e `FlowRunManifest`. [sem teste direto]

`pinneapple_orchestration/common/__init__.py` · 6 linhas · **sem teste direto**

pinneapple_orchestration.common.provenance

Gerenciador de contexto que grava `<manifest_dir>/<flow_name>/<run_id>.json` ao fim de cada flow (sucesso ou falha), com parâmetros, SHA do git, log de tasks, artefatos e duração. [sem teste direto]

`pinneapple_orchestration/common/provenance.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FlowRunManifest`	classe	`(flow_name, run_id, parameters, git_sha, started_at, finished_at, status, task_log, artif…)`	Registro de uma execução de flow: nome, id, parâmetros, SHA do git, horários, status, log de tasks, artefatos e erro.
`flow_run_manifest`	função	`(flow_name, parameters=None, *, manifest_dir='data/manifests', repo_dir=None) -> Iterator…)`	Gerenciador de contexto que cria um `FlowRunManifest`, marca sucesso ou falha e grava o JSON no fim.

pinneapple_orchestration.flows

Subpacote que reexporta os cinco flows.

`pinneapple_orchestration/flows/__init__.py` · 16 linhas · testes: 6

pinneapple_orchestration.flows.flow_01_data_generation

Flow que roda `sample_fn` N vezes e grava cada resultado como cenário num `DatasetStore`. [sem teste direto; coberto por `tests/test_orchestration.py` via `flows`]

`pinneapple_orchestration/flows/flow_01_data_generation.py` · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`flow_data_generation`	função	`(dataset_store, problem_id, sample_fn, n_scenarios, *, manifest_dir='data/manifests') -> …)`	Gera `n_scenarios` cenários com `sample_fn`, grava no `DatasetStore` e devolve os ids e o `run_id`.

pinneapple_orchestration.flows.flow_02_training

Flow que monta e treina um componente registrado e registra a execução num `ExperimentStore` opcional.

`pinneapple_orchestration/flows/flow_02_training.py` · 36 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`flow_training`	função	`(component_name, train_loader, val_loader=None, *, experiment_store=None, epochs=100, lr=…)`	Treina o componente `component_name` com os loaders dados e devolve o modelo, o resultado do `fit` e o `run_id`.

pinneapple_orchestration.flows.flow_03_benchmark

Flow que ranqueia todas as arquiteturas registradas para um `component_type`.

`pinneapple_orchestration/flows/flow_03_benchmark.py` · 29 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`flow_benchmark`	função	`(component_type, train_loader, val_loader=None, *, epochs=50, lr=0.001, manifest_dir='dat…)`	Ranqueia as arquiteturas de um `component_type` e guarda o ranking no manifesto.

pinneapple_orchestration.flows.flow_04_registry_publish

Flow terminal que salva um modelo no registro e o promove de estágio só com `confirm=True`.

`pinneapple_orchestration/flows/flow_04_registry_publish.py` · 32 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`flow_registry_publish`	função	`(registry, problem_id, model, *, metadata=None, stage='staging', confirm=False, manifest_…)`	Publica um modelo no registro e devolve a versão; levanta `PermissionError` se o estágio não é `development` e `confirm` não é `True`.

pinneapple_orchestration.flows.flow_05_digital_twin_prep

Flow que monta um `DigitalTwin` a partir de um modelo do registro, exigindo por padrão o estágio `production`.

`pinneapple_orchestration/flows/flow_05_digital_twin_prep.py` · 36 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`flow_digital_twin_prep`	função	`(registry, problem_id, field_names, coord_names=None, *, version=None, require_stage='pro…)`	Devolve um `DigitalTwin` montado com o modelo do registro, checando o estágio exigido.

pinneapple_orchestration.tasks

Subpacote que reexporta as cinco tasks. [sem teste direto]

`pinneapple_orchestration/tasks/__init__.py` · 16 linhas · **sem teste direto**

pinneapple_orchestration.tasks.benchmark_tasks

Task que só chama `pinneapple_systems.component_library.benchmark_component_type`; não há lógica de benchmark aqui. [sem teste direto]

`pinneapple_orchestration/tasks/benchmark_tasks.py` · 25 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`benchmark_component_type_task`	função	`(component_type, train_loader, val_loader=None, *, epochs=50, lr=0.001) -> ComponentBench…)`	Repassa a chamada para `benchmark_component_type` e devolve um `ComponentBenchmarkResult`.

`pinneapple_orchestration.tasks.data_generation_tasks`

Task genérica de geração de dados: roda um thunk `sample_fn() -> (coords, fields)` dado pelo usuário e salva cada execução no `DatasetStore`. [sem teste direto]

`pinneapple_orchestration/tasks/data_generation_tasks.py` · 43 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_scenarios_task`	função	`(dataset_store, problem_id, sample_fn, n_scenarios, *, scenario_prefix='scenario') -> Lis…)`	Roda `sample_fn()` `n_scenarios` vezes, salva cada resultado como cenário `<prefixo>_NNNN` e devolve os ids.

`pinneapple_orchestration.tasks.digital_twin_tasks`

Task que carrega um modelo do registro e monta um `DigitalTwin`, recusando com `PermissionError` modelos fora do estágio exigido. [sem teste direto]

`pinneapple_orchestration/tasks/digital_twin_tasks.py` · 48 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`prepare_digital_twin_task`	função	`(registry, problem_id, field_names, coord_names=None, *, version=None, require_stage='pro…)`	Escolhe a versão (a mais recente por padrão), confere o estágio, carrega o modelo e devolve um `DigitalTwin`.

`pinneapple_orchestration.tasks.registry_tasks`

Task que salva o modelo em `development` e só promove para outro estágio com `confirm=True`; um `trust_report` REJECT bloqueia a promoção salvo `override_reason`. [sem teste direto]

`pinneapple_orchestration/tasks/registry_tasks.py` · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`publish_model_task`	função	`(registry, problem_id, model, *, metadata=None, stage='staging', confirm=False, trust_rep…)`	Salva o modelo em `development` e promove para `stage` só com `confirm=True`; devolve a versão.

`pinneapple_orchestration.tasks.training_tasks`

Task fina que monta um componente via `ComponentRegistry`, chama `ComponentModel.fit()` e registra métricas num `ExperimentStore`. [sem teste direto]

`pinneapple_orchestration/tasks/training_tasks.py` · 51 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`train_component_task`	função	`(component_name, train_loader, val_loader=None, *, experiment_store=None, epochs=100, lr=…)`	Monta o componente, treina com `fit`, registra `best_val` no `ExperimentStore` (se dado) e devolve modelo, resultado e `run_id`.

Exemplos de uso reais do repositório

Gerar cenários com uma função de amostragem — `tests/test_orchestration.py`

```
        def test_flow_data_generation(tmp_path):
            reg = _registry()

            def sample_fn():
                return np.random.rand(20, 2).astype(np.float32), {"u":
np.random.rand(20, 1).astype(np.float32)}

            result = flow_data_generation(reg.datasets, "pipe_flow",
sample_fn, 3, manifest_dir=str(tmp_path / "manifests"))
            assert len(result["scenarios"]) == 3
            assert reg.datasets.list_scenarios("pipe_flow") ==
sorted(result["scenarios"])
```

Promoção de estágio exige confirm=True — `tests/test_orchestration.py`

```
            try:
                flow_registry_publish(reg, "valve_demo", model,
stage="production", confirm=False,
                                      manifest_dir=str(tmp_path /
"manifests"))
                assert False, "expected PermissionError"
            except PermissionError:
                pass

            result = flow_registry_publish(reg, "valve_demo", model,
stage="production", confirm=True,
                                           manifest_dir=str(tmp_path /
"manifests"))
            assert reg.models.metadata("valve_demo", result["version"])
["stage"] == "production"
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 2 de 16.
- Funções de teste em arquivos que importam algum módulo do pacote: 6.

Limitações conhecidas

- Prefect é opcional (extra `pinneapple[orchestration]`); sem ele os flows rodam como funções comuns, sem retries, agendamento nem UI.
- Sem Prefect, os kwargs dos decoradores (por exemplo `retries`) são ignorados em silêncio.
- `flow_run_manifest` grava por padrão em `data/manifests` relativo ao diretório atual e chama `git rev-parse HEAD`; fora de um repositório git o SHA fica `None`.

- A geração de dados não configura solvers sozinha: o usuário precisa passar um thunk `sample_fn` sem argumentos.
- `flow_registry_publish` não repassa `trust_report` nem `override_reason` para `publish_model_task`; para usar esse portão é preciso chamar a task direto.
- Os testes (`tests/test_orchestration.py`) importam só `flows`; tasks, `common` e `_prefect_compat` não têm teste direto.
- Depende de `pinneapple_registry` e `pinneapple_systems` (importados no topo dos módulos de flows e tasks).

6. `pinneapple_models`

Objetivo

`pinneapple_models` é um shim de compatibilidade. Ele não implementa nenhum modelo: só reexporta, de `pinneapple_neural.architectures`, o registro (`ModelRegistry`), o catálogo (`ModelCatalog`), a classe base `BaseModel` e algumas arquiteturas (`VanillaPINN`, `SIREN`, `ModifiedMLP`, `FourierNeuralOperator`, `FNO2d`, `DeepONet`, `MeshGraphNet`, `GraphBatch`, `GraphOutput`).

O problema que resolve é manter código antigo funcionando. Exemplos e scripts escritos com `from pinneapple_models import ...` continuam rodando depois que as arquiteturas foram para `pinneapple_neural`. O próprio docstring diz que código novo deve importar direto de `pinneapple_neural.architectures`.

Além dos nomes planos, o pacote cria pseudo-namespaces antigos (`pinns`, `neural_operators`, `graphnn`), um alias `catalog = ModelCatalog`, uma função `registry()` e tenta importar `register_all` do catálogo de `pinneapple_neural` (se falhar, vira uma função vazia). O submódulo `pinneapple_models.registry` existe só para que o caminho `pinneapple_models.registry.ModelRegistry` funcione, como em `pinneapple_train` e `pinneapple_solvers`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
2	74	0	1	1 (1 arquivos)	1

Pasta de testes dedicada: `tests/pinneapple_models` (1 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`registry`	Submódulo de compatibilidade que reexporta `ModelRegistry` de `pinneapple_neural.architectures.registry`.	1	0	1

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_models`

[shim] Reexporta registro, catálogo e arquiteturas de `pinneapple_neural.architectures` para código legado; marcado como obsoleto (use `pinneapple_neural.architectures`). [sem teste direto]

`pinneapple_models/__init__.py` · 64 linhas · **sem teste direto** · sinais no código: shim ou obsoleto

nome	tipo	assinatura	o que faz
`registry`	função	`()`	Função legada que devolve a classe `ModelRegistry` de `pinneapple_neural` (não cria registro novo).

`pinneapple_models.registry`

[shim] Expõe `ModelRegistry` no caminho antigo `pinneapple_models.registry`, criado porque `tests/pinneapple_models/test_registry.py` não coletava sem ele.

`pinneapple_models/registry.py` · 10 linhas · testes: 1

Exemplos de uso reais do repositório

Teste de fumaça do registro pelo caminho antigo — `tests/pinneapple_models/test_registry.py`

```
def test_models_registry_smoke():
    from pinneapple_models.registry import ModelRegistry

    try:
        from pinneapple_models.register_all import register_all
        register_all()
    except Exception:
        pass

    names = ModelRegistry.list()
    assert isinstance(names, list)
```

Tour do registro via imports legados — `examples/architectures/00_registry_tour.py`

```
from pinneapple_models import ModelCatalog
from pinneapple_models import register_all
from pinneapple_models import ModelRegistry


def _short(s: str, n: int = 90) -> str:
    s = (s or "").strip().replace("\n", " ")
    return s if len(s) <= n else s[: n - 1] + "…"


def main() -> None:
    register_all()

    print("\n=== Families ===")
    print(ModelRegistry.families())
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 1 de 2.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.

Limitações conhecidas

- É só um shim: todo o comportamento vem de `pinneapple_neural.architectures`; qualquer mudança lá afeta este pacote.
- Marcado como obsoleto no docstring; código novo deve importar de `pinneapple_neural.architectures`.
- Se `pinneapple_neural.architectures._catalog.register_all` não puder ser importado, `register_all` vira uma função que não faz nada, em silêncio.
- `XTFC` no namespace `pinns` fica `None` se o import de `pinneapple_neural.architectures.pinns.xtfc` falhar.
- O teste existente tenta importar `pinneapple_models.register_all` como módulo, que não existe; o erro é engolido por `try/except`.
- O `__init__` não tem teste direto; só o submódulo `registry` é coberto, por um teste de fumaça.

7. `pinneapple_train`

Objetivo

`pinneapple_train` é um shim de compatibilidade. Ele não tem código próprio: só reexporta o treinador que hoje mora em `pinneapple_neural.trainer`. Existe para que código antigo que faz `from pinneapple_train import Trainer, TrainConfig` (ou importa `pinneapple_train.trainer`, `.losses`, `.metrics`) continue funcionando.

O `__init__` reexporta `Trainer`, `TrainConfig`, `EarlyStopping`, `ModelCheckpoint`, `Metrics`, `Metric`, `MetricBundle` e `default_metrics`. Nomes opcionais (`TimeMarchingTrainer`, `RunLogger`, `Checkpoint`, `save_checkpoint`, `SweepConfig`, `run_parallel_sweep`, `AdaptiveSweepConfig`, `run_adaptive_sweep`) vêm dentro de `try/except` e viram `None` se o import falhar.

Os submódulos `trainer.py`, `losses.py` e `metrics.py` foram criados porque exemplos (`examples/time_series/*`, `examples/trainer/01_audited_training.py`), ferramentas de benchmark e a base de conhecimento de `pinneapple_problemdesign` importavam esses caminhos, que antes não existiam. Código novo deve importar direto de `pinneapple_neural.trainer`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
4	109	0	0	1 (1 arquivos)	1

Pasta de testes dedicada: `tests/pinneapple_train` (1 funções de teste).

Submódulos

sub-módulo	o que faz	módulos	classes + funções	testes *
`losses`	Reexporta `SupervisedLoss`, `PhysicsLossHook`, `CombinedLoss` e `build_loss` de `pinneapple_neural.trainer.losses`.	1	0	1
`metrics`	Reexporta as métricas (`MSE`, `MAE`, `RMSE`, `R2`, `RelL2`, `MaxError`, `default_metrics` etc.) de `pinneapple_neural.trainer.metrics`.	1	0	1
`trainer`	Reexporta `Trainer` e `TrainConfig` de `pinneapple_neural.trainer.trainer`.	1	0	1

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_train`

[shim] Pacote de compatibilidade (marcado como obsoleto) que reexporta o treinador, callbacks e métricas de `pinneapple_neural.trainer`; nomes opcionais viram `None` se o import falhar. [sem teste direto do `__init__`]

`pinneapple_train/__init__.py` · 65 linhas · **sem teste direto** · sinais no código: shim ou obsoleto

`pinneapple_train.losses`

[shim] Reexporta as perdas (`SupervisedLoss`, `PhysicsLossHook`, `CombinedLoss`, `build_loss`) de `pinneapple_neural.trainer.losses`.

`pinneapple_train/losses.py` · 8 linhas · testes: 1

`pinneapple_train.metrics`

[shim] Reexporta as métricas de regressão e `default_metrics` de `pinneapple_neural.trainer.metrics`.

`pinneapple_train/metrics.py` · 14 linhas · testes: 1

`pinneapple_train.trainer`

[shim] Reexporta `Trainer` e `TrainConfig` de `pinneapple_neural.trainer.trainer`, para o caminho `pinneapple_train.trainer` que exemplos e ferramentas já usavam.

`pinneapple_train/trainer.py` · 22 linhas · testes: 1

Exemplos de uso reais do repositório

Teste mínimo do treinador pelo caminho antigo — `tests/pinneapple_train/test_trainer_minimal.py`

```
from pinneapple_train.trainer import Trainer, TrainConfig
from pinneapple_train.losses import CombinedLoss, SupervisedLoss
from pinneapple_train.metrics import default_metrics

...
    combined = CombinedLoss(supervised=SupervisedLoss("mse"),
```

```
physics=None)

            def loss_fn(model_, y_hat, batch):
                return combined(model_, y_hat, batch)

            tr = Trainer(model=model, loss_fn=loss_fn,
metrics=default_metrics())
```

Exemplo de séries temporais que usa o shim — `examples/time_series/01_FNO.py`

```
        from pinneapple_train.trainer import Trainer, TrainConfig
        from pinneapple_train.losses import CombinedLoss, SupervisedLoss
        from pinneapple_train.metrics import default_metrics
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 3 de 4.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.

Limitações conhecidas

- É só um shim: todo o comportamento está em `pinneapple_neural.trainer`; o pacote é marcado como obsoleto.
- Vários nomes (`TimeMarchingTrainer`, `RunLogger`, `Checkpoint`, `SweepConfig`, `run_parallel_sweep`, `AdaptiveSweepConfig` etc.) são importados com `except Exception` e viram `None` em silêncio se o módulo de origem falhar.
- O `__init__` não tem teste direto; só os submódulos são exercitados por `tests/pinneapple_train/test_trainer_minimal.py`.
- Os submódulos reexportam só uma parte de `pinneapple_neural.trainer`; nomes fora das listas `__all__` não estão disponíveis por este caminho.

8. `pinneapple_solvers`

Objetivo

`pinneapple_solvers` é um shim de compatibilidade. Não tem código próprio: reexporta os solvers numéricos que hoje vivem em `pinneapple_simulation.numerical_solvers`, para que código antigo com `from pinneapple_solvers import ...` continue funcionando.

O `__init__` traz `SolverBase`, `SolverOutput`, `SolverRegistry` e `FDMSolver` diretamente. `FEMSolver`, `FVMSolver`, `LBMSolver`, `SpectralSolver`, `SPHSolver` e `FFTSolver` vêm dentro de `try/except` e viram `None` se o import falhar. `registry` é apenas um apelido para a classe `SolverRegistry`.

O submódulo `fft.py` foi criado porque `tests/pinneapple_solvers/test_fft.py` importava `pinneapple_solvers.fft`, que não existia. O comentário no `__init__` registra outro bug corrigido: antes ele importava um nome inexistente (`FFTProcessor`), e o erro era engolido pelo `except`. Código novo deve importar de `pinneapple_simulation.numerical_solvers`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
2	71	0	0	1 (1 arquivos)	1

Pasta de testes dedicada: `tests/pinneapple_solvers` (4 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`fft`	Reexporta `FFTSolver` de `pinneapple_simulation.numerical_solvers.fft`.	1	0	1

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_solvers`

[shim] Pacote de compatibilidade (obsoleto) que reexporta os solvers de `pinneapple_simulation.numerical_solvers`; os solvers opcionais viram `None` se o import falhar. [sem teste direto do `__init__`]

`pinneapple_solvers/__init__.py` · 61 linhas · **sem teste direto** · sinais no código: shim ou obsoleto

`pinneapple_solvers.fft`

[shim] Reexporta `FFTSolver` de `pinneapple_simulation.numerical_solvers.fft`, para o caminho usado em `tests/pinneapple_solvers/test_fft.py`.

`pinneapple_solvers/fft.py` · 10 linhas · testes: 1

Exemplos de uso reais do repositório

Teste de fumaça do FFTSolver pelo caminho antigo — `tests/pinneapple_solvers/test_fft.py`

```python
import torch
from pinneapple_solvers.fft import FFTSolver

def test_fft_solver_smoke():
    solver = FFTSolver()
    x = torch.randn(4, 128)
    out = solver(x)
    assert hasattr(out, "result")
    assert out.result.shape[0] == 4
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 1 de 2.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.

Limitações conhecidas

- É só um shim obsoleto: todo o comportamento está em `pinneapple_simulation.numerical_solvers`.
- `FEMSolver`, `FVMSolver`, `LBMSolver`, `SpectralSolver`, `SPHSolver` e `FFTSolver` são importados com `except Exception` e viram `None` em silêncio se o módulo de origem falhar.
- READMEs de exemplos (`examples/use_cases/TEMPLATE/README.md`, `examples/use_cases/drill_pipe_surrogate/README.md`) ainda citam `FEnicsBridge`, `OpenFOAMBridge` e `pinneapple_solvers.openfoam_bridge`, que este shim não exporta.
- O `__init__` não tem teste direto; `tests/pinneapple_solvers/` testa `pinneapple_solvers.fft` e o `FDMSolver` pelo caminho novo.

Volume 2 — Física: problemas, equações e ciência

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Como um problema físico é descrito e resolvido (`PhysicalProblem`, `pp.solve`), as equações, os presets industriais, o desenho de problemas, os experimentos por área, a condensação e as consultas físicas.

Sumário

1. pinneapple_physics
2. pinneapple_presets
3. pinneapple_problemdesign
4. pinneapple_science
5. pinneapple_condense
6. pinneapple_query

1. `pinneapple_physics`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
166	43.959	169	763	1.075 (105 arquivos)	52

Pasta de testes dedicada: `tests/pinneapple_physics` (48 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`advection_diffusion_1d`		1	1	13
`astro`		24	216	71
`blackhole`		12	60	19
`closed_form`		19	91	103
`codegen`		3	4	13
`compile_api`		1	3	5
`conservation`		1	3	6
`darcy_checks`		1	31	26
`deepxde_bridge`		2	4	9
`ensemble`		1	9	68
`ensemble_viz`		1	1	3
`experiment`		1	3	0

submódulo	o que faz	módulos	classes + funções	testes*
maxwell_checks		1	19	51
metrics		1	11	36
multiscale		11	76	45
non_hermitian		4	19	8
online_learning		1	4	0
paper_problems		1	7	5
paper_problems_xtfc		1	4	10
pcb		3	10	16
pde_environment		28	154	154
physical_problem		1	3	12
physics_audit		8	65	37
pinn_solver		19	41	121
solid		2	6	8
solving		1	12	71
symbolic_pde		6	24	17
tracking		1	3	3
tribology		2	3	20
weather		7	38	8

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_physics`

pinneapple_physics — Physics problem definition and PINN solving.

`pinneapple_physics/__init__.py` · 604 linhas · testes: 206

nome	tipo	assinatura	o que faz
TagConditionsUnresolved	classe	`()`	Raised by :func:`solve_pde` when *spec* has one or more `selector_type="tag"` conditions that are not covered by explicit caller-supplied input (`x_bc`/`x_ic`/`x_data` plus a matching `mask_<condition_name>`).
PhysicsPipeline	classe	`(model, spec, history)`	Result of :func:`pipeline` -- a trained model bound to the `ProblemSpec` it was trained on, with a `.predict()` convenience method. `.model`/`.spec`/`.history` are the same objects `get_preset`/`solve_pde` already produce -- this is a thin wrapper, not a new abstraction to learn.
compile_physics	função	`(spec, **kwargs)`	Compile a ProblemSpec into weighted PINN loss functions.
identify	função	`(description)`	Identify PDE family from a natural-language description.
define_problem	função	`(preset=None, **spec_kwargs) -> 'ProblemSpec'`	Quick ProblemSpec builder.
solve_pde	função	`(spec, model, *, epochs=5000, device='cpu', lr=0.001, n_collocation=2048, seed=0, weights…)`	One-shot: compile physics losses and train *model* on a ProblemSpec.

nome	tipo	assinatura	o que faz
`pipeline`	função	`(task, *, architecture='modified_mlp', hidden_dim=64, n_layers=4, epochs=2000, device='cp…)`	One call, named-task physics AI pipeline: preset -> model -> trained solution -> `.predict()`-ready object.

`pinneapple_physics.advection_diffusion_1d`

Periodic 1-D advection-diffusion u_t + c u_x = nu u_xx on [0, 2 pi): exact solution, classical finite-difference schemes and the PDE residual of any predicted space-time field.

`pinneapple_physics/advection_diffusion_1d.py` · 80 linhas · testes: 13

nome	tipo	assinatura	o que faz
`AdvectionDiffusion1D`	classe	`(nx, nt, t_end)`	

`pinneapple_physics.astro`

Positional astronomy and astrophysics for the PINNeAPPle observatory.

`pinneapple_physics/astro/__init__.py` · 54 linhas · testes: 71

`pinneapple_physics.astro.astrometry`

Apparent places of the Sun, Moon and planets (and catalogue stars) for an observer on Earth.

`pinneapple_physics/astro/astrometry.py` · 194 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Observer`	classe	`(lat, lon, elevation_m, pressure_mbar, temperature_c, name)`	A place on Earth: geodetic latitude/longitude (deg, east positive), height (m), and the weather used by the refraction model.
`Apparent`	classe	`(body, ra, dec, ra_astrometric, dec_astrometric, distance_km, light_time_s, range_rate_km…)`	
`refraction_deg`	função	`(alt_true_deg, pressure_mbar=1010.0, temperature_c=10.0)`	Atmospheric refraction (deg) for a true altitude, Saemundsson (1986) with the usual P/T scaling. Zero below -1 deg, where the formula is not meaningful.
`airmass`	função	`(alt_apparent_deg)`	Relative air mass (Kasten & Young 1989), finite down to the horizon; `inf` below it.
`extinction_mag`	função	`(alt_apparent_deg, k_v=0.25)`	Atmospheric extinction (mag) at V for an extinction coefficient `k_v` (mag/airmass; 0.2-0.3 typical).
`apparent`	função	`(body, t, observer=None, eph=None, *, deflection=True, refraction=True) -> Apparent`	Apparent place of a solar-system `body` at `t` (scalar or array), geocentric if `observer` is None.

nome	tipo	assinatura	o que faz
`star_apparent`	função	`(ra_j2000, dec_j2000, t, observer=None, *, pm_ra_mas=0.0, pm_dec_mas=0.0, parallax_mas=0.…)`	Apparent places of catalogue stars (ICRS J2000 epoch 2000.0) at a single instant `t`.

pinneapple_physics.astro.conditions

Observing conditions for tonight from open numerical weather prediction (Open-Meteo, CC BY 4.0).

`pinneapple_physics/astro/conditions.py` · 126 linhas · testes: 25

nome	tipo	assinatura	o que faz
`fetch_ensemble`	função	`(lat, lon, model='icon_seamless', hours=48, timeout=15.0)`	
`members`	função	`(resp, var) -> np.ndarray`	(n_members, n_hours) array for `var` (the control run and every `var_memberNN` column).
`summarize`	função	`(resp, clear_threshold=30.0) -> dict`	
`calibrate`	função	`(p_forecast, observed_clear, bins=10) -> dict`	Reliability of past clear-sky probabilities against what was observed, and an isotonic-style correction map: for each probability bin, the observed frequency (needs a history of forecasts and observations).
`fetch_cloud_forecast`	função	`(lat, lon, days=3, timeout=8.0) -> dict`	Hourly total cloud cover (%) for the next `days` from Open-Meteo's forecast API (no key; CC BY 4.0): {"time": [ISO UTC], "cloud_cover": [%], "source": …}. Raises on network errors (callers fall back).
`cloud_at`	função	`(forecast, iso_utc) -> float | None`	Cloud cover (%) at a time, linearly interpolated in the hourly forecast; None outside it.
`sky_view`	função	`(mode, forecast, iso_utc) -> dict`	What the sky lets through for an observing session. `mode` "perfect" removes the weather (any target, any night); "real" uses the cloud-cover forecast and hides the view at or above `CLOUDY_PCT`. Without a forecast "real" falls back to "perfect" and says so.

pinneapple_physics.astro.conjunction

Conjunction screening: closest approach in Clohessy-Wiltshire relative motion, collision probability, J2 orbits.

`pinneapple_physics/astro/conjunction.py` · 160 linhas · testes: 8

nome	tipo	assinatura	o que faz
`ClosestApproach`	classe	`(t_tca_s, position_km, velocity_km_s, miss_distance_km, relative_speed_km_s, perpendicula…)`	
`cw_mean_motion`	função	`(a_km, mu_km3_s2=398600.4418) -> float`	Mean motion n = sqrt(mu / a^3) of the (circular) reference orbit, rad/s.

nome	tipo	assinatura	o que faz
`cw_closest_approach`	função	`(n, state0, duration_s, n_samples=2000) -> ClosestApproach`	Closest approach over [0, duration_s] for the CW deputy state `state0 = (x0, y0, z0, vx0, vy0, vz0)` (km, km/s) about a reference orbit of mean motion `n` (rad/s).
`collision_probability_2d`	função	`(xm, zm, sigma_x, sigma_z, r_c) -> float`	Probability mass of N((xm, zm), diag(sigma_x^2, sigma_z^2)) inside the disk x^2 + z^2 <= r_c^2.
`collision_probability_centered`	função	`(r_c, sigma) -> float`	Closed form for a centred isotropic Gaussian: 1 - exp(-r_c^2 / (2 sigma^2)) (Rayleigh CDF).
`screen_cw_conjunction`	função	`(n, state0, *, duration_s=None, n_samples=2000, hard_body_radius_km=0.01, sigma_km=1.0) -…)`	Closest approach + collision probability for one CW encounter (one reference period by default).
`two_body_j2_rhs`	função	`(mu, J2, Re)`	Right-hand side f(t, [x, y, z, vx, vy, vz]) of two-body motion plus the J2 acceleration.
`propagate_two_body_j2`	função	`(state0, t_eval, *, mu=398600.4418, J2=0.00108262668, Re=6378.137, rtol=1e-11, atol=1e-08…)`	States (len(t_eval), 6) of two-body + J2 motion from `state0` (km, km/s), DOP853.

`pinneapple_physics.astro.decay`

Orbit decay of low-Earth-orbit satellites by atmospheric drag, with a physics-informed model and its checks.

`pinneapple_physics/astro/decay.py` · 258 linhas · testes: 16

nome	tipo	assinatura	o que faz
`DensityTable`	classe	`(h_km=None, **kw)`	log-density on an altitude grid, interpolated (NumPy and PyTorch) so models can be evaluated fast.
`exponential_density`	função	`(h_km)`	
`msis_available`	função	`() -> bool`	
`orbit_mean_density`	função	`(h_km, date='2026-10-13T00:00', f107=150.0, f107a=150.0, ap=15.0, inclination_deg=51.6, n…)`	Orbit-averaged NRLMSIS 2.0 total mass density (kg/m^3) at altitudes `h_km`: mean over latitudes sampled along the orbit (|lat| <= inclination, weighted by the time spent there) and over all local times.
`ballistic_from_bstar`	função	`(bstar_per_er) -> float`	B = C_D A / m (m^2/kg) implied by an SGP4 B* (1/Earth radii): B* = B rho0 / 2.
`reference_decay`	função	`(h0_km, B, days, table, n=200, reentry_km=150.0)`	Altitude (km) vs time (days) from an adaptive Runge-Kutta integration of the decay ODE.

nome	tipo	assinatura	o que faz
envelope	função	`(h0_km, B, days, *, f107=150.0, f107a=None, ap=15.0, date='2026-10-13T00:00', inclination…)`	Monte-Carlo spread of the reference decay over the uncertain inputs: solar flux (log-normal, 20 %), geomagnetic activity (Ap x 0.5..2) and the ballistic coefficient (log-normal, 15 %). Returns t, median, 5 %, 95 % altitude curves.
train_decay_pinn	função	`(h0_km=415.0, B=0.005, days=60.0, *, f107=150.0, ap=15.0, date='2026-10-13T00:00', inclin…)`	Train the decay PINN and return its report: prediction, reference, envelope, checks and verdict.
sgp4_decay	função	`(line1, line2, days=30.0, step_min=10.0)`	Mean altitude (km) vs time (days) predicted by SGP4 from a TLE: the geocentric radius averaged over each orbit (removes the eccentricity and J2 oscillations), minus the equatorial radius.
compare_with_sgp4	função	`(line1, line2, days=30.0, f107=70.0, ap=8.0, B=ISS_B) -> dict`	Decay predicted by this model (NRLMSIS + a physical ballistic coefficient B) next to SGP4's own prediction from the same TLE. SGP4's B* is a fitted drag term for its own power-law atmosphere, not a physical ballistic coefficient (converted with B* = B rho0 / 2 it is typically an order of magnitude …

pinneapple_physics.astro.ephemeris

Solar-system ephemeris on top of a JPL DE kernel (public domain), read by :mod:`.spk`.

`pinneapple_physics/astro/ephemeris.py` · 77 linhas · testes: 71

nome	tipo	assinatura	o que faz
Ephemeris	classe	`(path=None)`	
find_kernel	função	`() -> str`	
default_ephemeris	função	`(path=None) -> Ephemeris`	

pinneapple_physics.astro.facts

Facts and curiosities about the objects in the sky, for a general audience, in English and Portuguese.

`pinneapple_physics/astro/facts.py` · 746 linhas · testes: 30

nome	tipo	assinatura	o que faz
sources	função	`() -> dict`	Every source a number or a curiosity can cite: {key: {title, publisher, url, licence}}.
ids	função	`() -> dict`	Every object with a card: {id: kind}. Ids: Solar System bodies by name ('saturn'), stars 'star- ', Messier objects 'M31', constellations 'con-' ('con-Ser1', 'con-Ser2' for Serpens), satellites 'sat-' ('sat-iss').
resolve	função	`(ref) -> str`	The card id for an id or an observatory reference: 'saturn', 'body:saturn', 'star:2061', 'HR 2061', 'M 31', 'sat:25544', 'con:Ori'. Raises KeyError if there is no card.
names	função	`(oid) -> dict`	{en, pt} display names of a card.

nome	tipo	assinatura	o que faz
`numbers`	função	`(oid) -> dict`	The key numbers of a card: {key: {value, unit, source}}.
`comparison`	função	`(key, number, lang='en') -> str | None`	An everyday comparison for a number, computed from it (source: computed).
`quiz`	função	`(oid, lang='en', seed=0, n=3) -> list[dict]`	A short multiple-choice quiz from the card's numbers: [{question, options, answer (index), source}].
`card`	função	`(ref, lang='en') -> dict`	The info card of an object in `lang`: name, kind, summary, numbers (value, unit, label, comparison, source), three to five curiosities with sources, a short history, missions, a quiz and the sources cited.
`list_cards`	função	`(kind=None, lang='en') -> list[dict]`	[{id, kind, name}] of every card (or of one kind: 'planet', 'star', 'constellation', 'messier', ...).
`search`	função	`(query, lang=None, limit=12) -> list[dict]`	Cards whose name (in English or Portuguese), id or designation matches `query` (accents and case ignored); prefix matches first.
`did_you_know`	função	`(lang='en', seed=None) -> dict`	A random curiosity from the hand-written cards, for a start-up tip: {id, name, text, source}.
`validate`	função	`() -> list[str]`	Problems in the dataset (an empty list means every card has a source per number, text in both languages, three to five curiosities, a history and known sources).
`position`	função	`(ref)`	J2000 (ICRS) RA/Dec in degrees of a fixed object (star, Messier object, constellation label point), or None for Solar System bodies and satellites, whose positions come from the ephemeris / TLE.
`tonight`	função	`(ref, observer, t, hours=24.0, step_min=10.0, eph=None) -> dict | None`	How to see an object from `observer` in the `hours` after `t`: altitude now, rise, culmination (highest point) and set (the next rise only if it is below the horizon now), the highest altitude while the sky is dark (Sun below -12 deg) and when, and whether it never rises or never sets. None ...

`pinneapple_physics.astro.frames`

Reference-frame rotations: frame bias, precession (IAU 2006), nutation (IAU 1980, terms above 5 mas), geodetic observer positions (WGS84), horizon (alt-az) and ecliptic coordinates.

`pinneapple_physics/astro/frames.py` · 202 linhas · testes: 16

nome	tipo	assinatura	o que faz
`fundamental_args`	função	`(T)`	Delaunay arguments D, M, M', F, Omega (rad) for Julian centuries TT since J2000 (Meeus ch. 22).
`nutation`	função	`(jd_tt)`	Nutation in longitude and obliquity (rad).
`mean_obliquity`	função	`(jd_tt)`	
`rot_x`	função	`(a)`	
`rot_y`	função	`(a)`	

nome	tipo	assinatura	o que faz
`rot_z`	função	`(a)`	
`precession_matrix`	função	`(jd_tt)`	Mean J2000 -> mean equator/equinox of date: IAU 2006 (P03) angles, IERS Conventions 2010 eq. 5.40.
`nutation_matrix`	função	`(jd_tt)`	Mean of date -> true equator/equinox of date.
`npb_matrix`	função	`(jd_tt)`	GCRS -> true equator and equinox of date (N P B).
`ecliptic_matrix`	função	`(jd_tt=None)`	GCRS -> ecliptic. `None` gives the J2000 mean ecliptic (IAU 2006 obliquity 84381.406").
`geodetic_to_itrs`	função	`(lat_deg, lon_deg, height_m=0.0)`	Observer position (km) in the terrestrial frame.
`itrs_to_geodetic`	função	`(r)`	Inverse of :func:`geodetic_to_itrs` (Bowring iteration); returns lat, lon (deg), height (km).
`radec`	função	`(v)`	Unit or full vectors (..., 3) -> RA (deg, 0..360), Dec (deg), norm.
`from_radec`	função	`(ra_deg, dec_deg)`	
`equatorial_to_horizontal`	função	`(ha_rad, dec_rad, lat_rad)`	Hour angle, declination, latitude (rad) -> altitude, azimuth (deg; azimuth from north through east).
`horizontal_to_equatorial`	função	`(alt_deg, az_deg, lat_deg)`	Altitude/azimuth -> hour angle (deg) and declination (deg).
`angular_separation`	função	`(ra1, dec1, ra2, dec2)`	Great-circle separation (deg), Vincenty form (stable at all angles).
`position_angle`	função	`(ra1, dec1, ra2, dec2)`	Position angle (deg, north through east) of point 2 seen from point 1.

`pinneapple_physics.astro.journey`

A journey from your place on Earth to a planet, a moon, the Sun or a star: the geometry of the trip.

`pinneapple_physics/astro/journey.py` · 193 linhas · testes: 19

nome	tipo	assinatura	o que faz
`galactic`	função	`(v)`	Galactic (l, b) in degrees of an ICRS (J2000) vector.
`body_position`	função	`(body, t, eph=None) -> np.ndarray`	Heliocentric ICRS position (km) of a Solar System body at t (geometric, no light time).
`observer_position`	função	`(observer, t, eph=None) -> np.ndarray`	Heliocentric ICRS position (km) of a place on Earth at t (Earth from the ephemeris + the site in GCRS).
`star_position`	função	`(ra_deg, dec_deg, parallax_mas) -> np.ndarray`	Heliocentric ICRS position (km) of a star from its J2000 direction and parallax.
`travel_times`	função	`(distance_km) -> dict`	Seconds needed to cover a distance at the speeds in `SPEEDS`.
`apparent_magnitude_star`	função	`(abs_mag, distance_km)`	m = M + 5 log10(d / 10 pc).

nome	tipo	assinatura	o que faz
`angular_diameter_deg`	função	`(radius_km, distance_km)`	
`plan`	função	`(observer, target, t, standoff=None, n_profile=60, eph=None) -> dict`	The trip from `observer` to `target` (body name, 'star: ---------- ', 'HR ' or a facts id) on date `t`.
`scene`	função	`(t, observer=None, max_parallax_err=0.2, eph=None) -> dict`	Everything a 3-D journey draws, heliocentric ICRS km: the planets, Sun and Moon at t (with radii), and the bright stars with a usable parallax (Hipparcos 2007 for the brightest, BSC5 otherwise) with absolute magnitude and B-V; stars without a parallax are listed by direction only.

`pinneapple_physics.astro.lensing`

Gravitational lensing demos on the real sky: a Schwarzschild black hole placed in front of the stars you are looking at, rendered with the geodesic ray tracer of :mod:`pinneapple_physics.blackhole.raytrace`, plus the closed forms (weak-field deflection, Einstein radius, shadow diameter) checked ...

`pinneapple_physics/astro/lensing.py` · 366 linhas · testes: 32

nome	tipo	assinatura	o que faz
`deflection_angle_rad`	função	`(mass_kg, impact_parameter_m)`	Weak-field light deflection 4GM / (c^2 b).
`einstein_radius_rad`	função	`(mass_kg, d_lens_m, d_source_m)`	
`shadow_diameter_rad`	função	`(mass_kg, distance_m)`	Angular diameter of a Schwarzschild shadow, 2 sqrt(27) GM / (c^2 D).
`eht_comparison`	função	`() -> list[dict]`	
`sun_deflection_arcsec`	função	`(elongation_deg)`	Deflection of starlight by the Sun at an angular distance from its centre (Eddington 1919: 1.75" at the limb), using b = 1 AU sin(elongation).
`star_splat`	função	`(v, mag_limit=6.5, sigma_deg=0.1, mag_white=1.5)`	Display footprint of a star of visual magnitude v: Gaussian width (deg) and peak brightness.
`catalog_sky`	função	`(width=4096, height=2048, mag_limit=6.5, milky_way=0.32, star_sigma_deg=0.1) -> np.ndarray`	Equirectangular RGB image (x = RA / 360 * width, Dec up) of the bright-star catalogue over the Milky Way glow: stars drawn as Gaussian splats with black-body colours, sized and scaled with magnitude (`star_splat`).

nome	tipo	assinatura	o que faz
`tone_map`	função	`(img, exposure=1.6, gamma=0.75) -> np.ndarray`	Linear sky radiance -> display values in [0, 1]: a soft shoulder 1 - exp(-exposure x) (bright cores of stars saturate smoothly instead of clipping) and a display gamma that lifts the faint band.
`bright_stars_in_field`	função	`(ra_deg, dec_deg, radius_deg, mag_limit=3.5) -> int`	Number of catalogue stars brighter than `mag_limit` within `radius_deg` of (ra, dec).
`lensed_sky`	função	`(ra_deg, dec_deg, fov_deg=30.0, width=480, height=270, r_cam=30.0, sky=None, supersample=…)`	Render the real sky around (ra, dec) as seen through a Schwarzschild black hole at that position.
`trace_thin_disc`	função	`(width=640, height=360, inclination_deg=80.0, r_cam=60.0, fov_deg=32.0, max_crossings=3, …)`	Exact backward ray tracing for an infinitely thin equatorial disc around a Schwarzschild black hole (G = M = c = 1).
`disc_image`	função	`(tr, r_in=6.0, r_out=20.0, doppler=True, t_peak=3900.0, exposure=2.4, gamma=0.6, opacity=…)`	Colour and intensity of the disc for traced rays.
`accretion_view`	função	`(ra_deg, dec_deg, width=640, height=360, inclination_deg=86.0, r_cam=60.0, fov_deg=42.0, …)`	A Schwarzschild black hole with a thin accretion disc placed in front of the real sky around (ra, dec): exact null geodesics (`trace_thin_disc`), Doppler beaming and gravitational redshift (`disc_image`), the lensed background from the bright-star catalogue and the Milky Way, then a light bloom.
`lensing_view`	função	`(ra_deg, dec_deg, fov_deg=60.0, width=480, height=270, r_cam=120.0, sky=None, disc='auto'…)`	Display-ready view of a black hole in front of the sky around (ra, dec).

`pinneapple_physics.astro.nbody`

Classical N-body integrators for the Sun-planets(-Moon) problem, with conservation diagnostics.

`pinneapple_physics/astro/nbody.py` · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`initial_state`	função	`(jd_tdb, bodies=SOLAR_SYSTEM, eph=None)`	
`accelerations`	função	`(r, mu)`	
`energy`	função	`(r, v, mu)`	Total energy per unit G (km^2/s^2 * km^3/s^2 / ... consistent units: sum mu_i v_i^2/2 - sum mu_i mu_j/r_ij).

nome	tipo	assinatura	o que faz
`angular_momentum`	função	`(r, v, mu)`	
`integrate`	função	`(r0, v0, mu, t_end_s, dt_s, method='yoshida4', n_out=200)`	Integrate from t=0 to `t_end_s`; returns dict with times (s), r (n_out, N, 3), v, and the relative energy error and angular-momentum drift histories.

pinneapple_physics.astro.neural_propagator

A learned propagator for near-Earth asteroids: one network for many bodies.

`pinneapple_physics/astro/neural_propagator.py` · 240 linhas · testes: 16

nome	tipo	assinatura	o que faz
`NeuralPropagator`	classe	`(net=None, xm=None, xs=None, ys=None, info=None)`	
`sample_population`	função	`(n, seed=0)`	NEO-like osculating elements: a 0.7-2.6 AU, e 0.02-0.65 (perihelion > 0.2 AU), i 0-35 deg, random angles.
`kepler_positions`	função	`(body, jd)`	
`features`	função	`(body, tau)`	Inputs: a, e, sin/cos i, angles as sin/cos, tau = t / span, and the Kepler mean anomaly at t (sin/cos).
`rtn_basis`	função	`(rk, vk)`	Radial / transverse (along-track) / normal unit vectors of the Kepler orbit, (n, 3, 3) rows R, T, N.
`make_dataset`	função	`(n_bodies=160, n_t=60, seed=0, cache=None)`	Features and targets: the deviation from the Kepler orbit in its R/T/N frame (milli-AU).
`tisserand`	função	`(r_km, v_km_s, a_jup_au=5.2026)`	Tisserand parameter w.r.t. Jupiter from heliocentric states (ICRF axes; inclination to the ecliptic).
`evaluate`	função	`(model, bodies, n_t=60)`	Errors of the learned propagator and of Kepler alone against the full integrator, per body, plus timing.
`trust_report`	função	`(folder=MODEL_DIR, real=None) -> dict`	Verdict of the shipped network from its held-out evaluation: to be useful it must beat Kepler by 5x on the median of the per-body maximum error over unseen bodies (and not be worse than Kepler at the 90th percentile).

pinneapple_physics.astro.observing

An observing session, with honest physics: plan a night, set up a telescope, slew to a target and track it.

`pinneapple_physics/astro/observing.py` · 356 linhas · testes: 21

nome	tipo	assinatura	o que faz
`Telescope`	classe	`(name, aperture_mm, focal_mm, kind, central_obstruction)`	
`Eyepiece`	classe	`(name, focal_mm, afov_deg)`	
`Camera`	classe	`(name, width_mm, height_mm, pixel_um)`	
`target_altaz`	função	`(target, t, observer, refraction=True, eph=None)`	Apparent (alt, az, ra_of_date, dec_of_date) in degrees of a body name or a J2000 (ra, dec) pair at the times `t` (scalar or array).
`rise_transit_set`	função	`(target, observer, jd_start, days=1.0, eph=None) -> dict`	Rise, upper transit and set (UTC Julian dates) in [jd_start, jd_start + days). Bodies: the almanac definition (centre 34' + semi-diameter below the horizon). Stars and deep-sky objects: centre 34' below the geometric horizon (standard refraction), transit when the hour angle is zero.
`extinction`	função	`(alt_apparent_deg, k=None) -> dict`	Extinction (mag) per band at an apparent altitude: k * X with X the Kasten-Young air mass.
`nl_to_mag`	função	`(b_nl)`	Surface brightness: nanolamberts -> V mag/arcsec^2 (Krisciunas & Schaefer 1991, eq. 1 inverted).
`mag_to_nl`	função	`(m)`	
`moon_brightness_nl`	função	`(moon_phase_angle_deg, moon_sep_deg, moon_alt_deg, target_alt_deg, k=0.172)`	Sky brightness (nanolamberts, V) added by moonlight at the target, Krisciunas & Schaefer (1991, PASP 103, 1033): B = f(rho) I*(alpha) 10^(-0.4 k X(Z_moon)) (1 - 10^(-0.4 k X(Z))), with the Moon's illuminance I* = 10^(-0.4 (3.84 + 0.026|alpha| + 4e-9 alpha^4)), the scattering function f(rho) = ...
`dark_sky_nl`	função	`(target_alt_deg, k=0.172, b_zenith_nl=79.0)`	The natural night sky at the target (KS91 eq. 2): B_zen 10^(-0.4 k (X - 1)) X; 79 nL ~ 21.6 V mag/arcsec^2.
`twilight_mag`	função	`(sun_alt_deg)`	Approximate zenith sky brightness in twilight, V mag/arcsec^2, from the Sun's altitude: about 11 at sunset's civil limit (-6 deg), 16.5 at -12 deg and the dark-sky level beyond -18 deg (a linear fit to typical twilight photometry; good to ~1 mag, enough to decide what can be seen).
`sky_brightness`	função	`(sun_alt_deg, moon_phase_angle_deg, moon_sep_deg, moon_alt_deg, target_alt_deg, k=0.172, …)`	Sky brightness at the target, V mag/arcsec^2: dark sky + moonlight (Krisciunas & Schaefer 1991) + twilight.
`visibility`	função	`(target, observer, t, hours=24.0, step_min=5.0, min_alt=30.0, eph=None) -> dict`	The night for a target from `observer` over `hours` after `t`: samples of altitude, azimuth, air mass, Sun and Moon altitude, Moon separation and sky brightness; rise / transit / set; astronomical twilight; the Moon's phase; the best window (dark sky, Sun < -18 deg, target above `min_alt`, ...

nome	tipo	assinatura	o que faz
optics	função	(telescope, eyepiece=None, camera=None, naked_eye_limit=6.0, wavelength_nm=550.0) -> dict	What a telescope shows. Standard formulas: magnification M = F / f_e; true field = AFOV / M; exit pupil D / M; focal ratio F / D; Dawes limit 116 / D(mm) arcsec; Rayleigh limit 1.22 lambda / D; limiting magnitude m = m_eye + 5 log10(D / 7 mm) (the light grasp over a 7 mm dark-adapted pupil); light …
axis_time	função	(delta_deg, rate_deg_s, accel_deg_s2) -> float	Time to move one axis by delta_deg with a trapezoidal velocity profile (accelerate, coast, decelerate); a triangular profile when the move is too short to reach the slew rate.
hour_angle_dec	função	(alt_deg, az_deg, lat_deg)	(hour angle, declination) in degrees from (alt, az).
slew	função	(start_altaz, target_altaz, lat_deg, mount='altaz', rate_deg_s=4.0, accel_deg_s2=2.0, set…)	A GoTo slew: both axes move at once, each with a trapezoidal profile; the slew takes the slower axis plus a settling time. Refused below min_alt. An equatorial (German) mount slews in hour angle and declination and must do a meridian flip when the target is on the other side of the meridian …
parallactic_angle	função	(lat_deg, ha_deg, dec_deg)	Parallactic angle q (deg): the angle at the target between the directions to the zenith and to the pole, tan q = sin H / (tan phi cos delta - sin delta cos H).
field_rotation_rate	função	(lat_deg, alt_deg, az_deg)	Field rotation of an alt-azimuth mount (deg per second of time): the rate of change of the parallactic angle, dq/dt = -omega cos(phi) cos(A) / cos(h), with A the azimuth from north through east; its size is the classical omega cos(phi) cos(A) / cos(h), infinite at the zenith. The image in an alt-az …
survey_url	função	(ra_deg, dec_deg, fov_deg, width=512, height=512, rotation_deg=0.0, survey=SURVEY) -> str	URL of a JPEG cut-out of a HiPS survey around (ra, dec) from CDS hips2fits (gnomonic projection, north up).
survey_cutout	função	(ra_deg, dec_deg, fov_deg, width=512, height=512, cache_dir=None, timeout=8.0, survey=SUR…)	JPEG bytes of a survey cut-out, cached on disk (rounded position and field, so nearby requests share a file). Raises OSError when the service cannot be reached and nothing is cached.

pinneapple_physics.astro.orbits

Two-body orbits: osculating elements from a state vector, Kepler's equation, propagation, and the heliocentric orbits of the planets from the JPL ephemeris.

pinneapple_physics/astro/orbits.py · 121 linhas · testes: 27

nome	tipo	assinatura	o que faz
solve_kepler	função	(M, e, tol=1e-14, maxiter=50)	Eccentric anomaly E from mean anomaly M (rad) for 0 <= e < 1 (Newton, Danby starter).
elements_from_state	função	(r, v, mu=GM_SUN) -> dict	Osculating Keplerian elements (angles in degrees) of a bound orbit from position (km) / velocity (km/s).

nome	tipo	assinatura	o que faz
`state_from_elements`	função	`(el, mu=GM_SUN, dt_s=0.0)`	Position/velocity (km, km/s) after `dt_s` seconds of pure Keplerian motion.
`heliocentric_state`	função	`(body, jd_tdb, frame='ecliptic', eph=None)`	Heliocentric position (km) and velocity (km/s) of a body, J2000 ecliptic (default) or ICRF axes.
`planet_elements`	função	`(body, jd_tdb, eph=None) -> dict`	Osculating heliocentric elements (J2000 ecliptic) of a planet; Moon: geocentric elements.
`orbit_polyline`	função	`(body, jd_tdb, n=256, eph=None)`	One revolution of the osculating orbit, heliocentric J2000 ecliptic, in AU (n, 3).
`light_time_and_doppler`	função	`(range_km, range_rate_km_s, rest_wavelength_nm=656.281)`	One-way light time and the relativistic Doppler shift of a spectral line (H-alpha by default).

`pinneapple_physics.astro.phenomena`

Almanac phenomena: rise / transit / set, twilight, Moon phase, planet magnitudes and elongations, oppositions and conjunctions, equinoxes and solstices, lunar and solar eclipses, transits of Mercury and Venus.

`pinneapple_physics/astro/phenomena.py` · 435 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`find_roots`	função	`(f, jd0, jd1, step, tol=0.5 * SECOND)`	All sign changes of `f(jd_array)` in [jd0, jd1], each refined to `tol` days. Returns a list of (jd, direction) with direction +1 for a rising crossing.
`find_minima`	função	`(f, jd0, jd1, step, tol=1.0 * SECOND)`	Local minima of `f` (golden-section refinement). Returns a list of (jd, value).
`horizon_altitude`	função	`(body, distance_km=None) -> float`	Geometric altitude of the body's centre at rise/set: -(34' refraction + semi-diameter).
`rise_transit_set`	função	`(body, observer, jd_start, days=1.0, eph=None)`	Rise, upper transit and set times (UTC Julian dates) of a solar-system body in [jd_start, +days).
`twilight_events`	função	`(observer, jd_start, days=1.0, eph=None)`	Sunrise/sunset and the start/end of civil, nautical and astronomical twilight.
`sky_state`	função	`(sun_alt_deg) -> str`	
`phase_angle`	função	`(body, t, eph=None)`	Sun-body-Earth angle (deg) and the Sun-body distance (km), geocentric.
`illuminated_fraction`	função	`(body, t, eph=None)`	
`moon_phase`	função	`(t, eph=None) -> dict`	Illuminated fraction, phase angle, elongation, age-like phase longitude (0 new, 90 first quarter, 180 full) and the position angle of the bright limb (deg, from north through east).
`magnitude`	função	`(body, t, eph=None)`	Visual magnitude of the Sun, Moon and planets (Meeus ch. 41 / Explanatory Supplement 1992 formulas; Saturn without the ring term, so up to ~0.8 mag too faint when the rings are open).

nome	tipo	assinatura	o que faz
`planet_events`	função	`(body, jd0, jd1, eph=None)`	Oppositions and conjunctions (geocentric ecliptic longitude of date equal to the Sun's or opposite), and for Mercury / Venus the greatest elongations; with elongation and magnitude at each event.
`lunar_phase_events`	função	`(jd0, jd1, eph=None)`	New Moon, first quarter, full Moon, last quarter between two dates.
`seasons`	função	`(jd0, jd1, eph=None)`	Equinoxes and solstices: apparent geocentric longitude of the Sun = 0, 90, 180, 270 deg.
`lunar_eclipses`	função	`(jd0, jd1, eph=None)`	Lunar eclipses: shadow geometry at each full Moon with the Danjon 2% enlargement of the Earth's shadow. Returns greatest-eclipse time, type and umbral / penumbral magnitudes.
`solar_eclipses`	função	`(jd0, jd1, observer=None, eph=None)`	Solar eclipses anywhere on Earth (geocentric criterion), with an approximate type, and the local circumstances (maximum obscuration and its time) for `observer` if given.
`planetary_transits`	função	`(jd0, jd1, eph=None)`	Transits of Mercury and Venus across the Sun (geocentric): inferior conjunctions with the planet's minimum separation from the Sun's centre below the solar radius plus the planet's.

`pinneapple_physics.astro.satellites`

Earth satellites from public two-line elements (CelesTrak GP data): SGP4 propagation (the `sgp4` package, MIT, Vallado et al. 2006 reference implementation), TEME -> terrestrial frame, topocentric look angles, Earth-shadow test, standard-magnitude brightness model and visible-pass prediction.

`pinneapple_physics/astro/satellites.py` · 246 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Satellite`	classe	`(name, line1, line2)`	
`gmst82`	função	`(t)`	GMST (rad), IAU 1982 expression used by the SGP4 TEME frame.
`parse_tle`	função	`(text) -> list[Satellite]`	
`fetch_tles`	função	`(group='stations', cache_dir=None, max_age_hours=6.0, timeout=15.0) -> tuple[list[Satelli…)`	Current TLEs of a CelesTrak group, cached on disk for `max_age_hours` (CelesTrak asks clients not to poll more than once per update cycle). Returns (satellites, info) where info has `source`, `fetched_utc` and `error` when the network is unavailable and no cache exists.
`sun_itrs`	função	`(t, eph=None)`	Geocentric Sun position (km) in the terrestrial frame.
`look`	função	`(sat, observer, t, eph=None) -> dict`	Topocentric altitude/azimuth/range, sub-satellite point, illumination and magnitude, vectorised over t.
`magnitude`	função	`(std_mag, range_km, phase_rad, sunlit=1.0)`	Diffuse-sphere brightness: m = m_std + 5 log10(d / 1000 km) - 2.5 log10(pi F(phase)), with F = (sin p + (pi - p) cos p) / pi (equals 1/pi at 90 deg, where m = m_std).

nome	tipo	assinatura	o que faz
find_passes	função	(sat, observer, t0, days=1.0, min_alt=10.0, step_s=20.0, eph=None) -> list[dict]	Passes above the horizon in [t0, t0 + days): rise / culmination / set (UTC), max altitude, and visibility (satellite sunlit, observer's Sun below -6 deg, altitude >= min_alt) with the brightest magnitude while visible. Times are refined to ~0.1 s.

pinneapple_physics.astro.smallbodies

Asteroids and comets: JPL Small-Body Database elements, a high-accuracy propagator in the field of the Sun, the planets and the Moon from the DE421 ephemeris (with the Sun's relativistic term), close approaches to the Earth, and a learned (neural) propagator checked against it.

pinneapple_physics/astro/smallbodies.py · 222 linhas · testes: 16

nome	tipo	assinatura	o que faz
SmallBody	classe	(name, epoch_jd, a_au, e, i_deg, node_deg, peri_deg, M_deg, kind, neo, pha, H, diameter_k…)	
Trajectory	classe	(body, jd0, jd1, eph=None, rtol=1e-11, gr=True)	One adaptive DOP853 integration from the epoch over [jd0, jd1] with dense output: positions and velocities anywhere in the span cost an interpolation, not a new integration.
load_body	função	(key) -> SmallBody	
available_bodies	função	()	Keys of the bundled objects, the near-Earth asteroids first.
propagate	função	(body, jd_out, eph=None, rtol=1e-11, gr=True)	Heliocentric ICRF positions (km) and velocities (km/s) at TDB Julian dates jd_out.
close_approaches	função	(body, jd0, jd1, target='earth', max_dist_au=0.2, step_days=0.25, eph=None, traj=None)	Minima of the distance to target (geocentre by default), refined to ~1 s by golden section on the dense trajectory.
horizons_vectors	função	(path=None)	Parse a JPL Horizons VECTORS table (CSV, AU-D, ecliptic J2000): jd, positions (AU), velocities (AU/d).
cad_reference	função	(body, year)	The JPL close-approach record (body = Earth) for a given year, with its 3-sigma distance range.
validation_summary	função	(eph=None) -> dict	The propagator against independent references: Ceres vs JPL Horizons vectors (2022, from 2018 elements) and the Apophis flyby of 2029-04-13 vs the JPL close-approach table (distance with its 3-sigma range, time, speed).

pinneapple_physics.astro.spaceweather

Space weather for observers: a Kp forecast with calibrated uncertainty and the probability of seeing the aurora.

pinneapple_physics/astro/spaceweather.py · 193 linhas · testes: 16

nome	tipo	assinatura	o que faz
`load_history`	função	`()`	
`geomagnetic_latitude`	função	`(lat_deg, lon_deg)`	Dipole (centred, IGRF-13 2020) geomagnetic latitude.
`oval_boundary`	função	`(kp)`	Equatorward geomagnetic latitude of the visible aurora for a Kp (NOAA SWPC table: 66.5 deg at Kp 0 to 48.1 deg at Kp 9, ~2.04 deg per Kp).
`kp_needed`	função	`(maglat_deg, horizon_margin_deg=0.0)`	
`build_xy`	função	`(kp, f107, horizon)`	
`train_forecaster`	função	`(horizon=1, seed=0, epochs=300)`	Quantile network for Kp `horizon` 3-hour steps ahead, conformalised; evaluated on the last two months.
`forecast`	função	`(model, kp_hist, f107_now)`	Forecast (lo, median, hi) for Kp `horizon` steps after the last value of `kp_hist` (needs >= 27 days).
`aurora_probability`	função	`(lo, med, hi, maglat_deg)`	P(Kp >= needed) from the forecast quantiles (piecewise-linear CDF through q10 / q50 / q90, clipped 0..9), overhead and low on the poleward horizon (~6 deg more reach).
`fetch_live_kp`	função	`(timeout=10.0)`	Last ~7 days of 3-hourly planetary Kp from NOAA SWPC (JSON). Returns (iso times, kp) or raises.

`pinneapple_physics.astro.spk`

A small reader for JPL SPK kernels (DAF files, segment types 2 and 3: Chebyshev position / position+velocity).

`pinneapple_physics/astro/spk.py` · 124 linhas · testes: 16

nome	tipo	assinatura	o que faz
`Segment`	classe	`(target, center, frame, kind, start_et, end_et, init, intlen, rsize, n, coeffs)`	
`SPK`	classe	`(path)`	Memory-mapped SPK kernel. `state(target, center, et)` chains segments through common centres.

`pinneapple_physics.astro.stars`

Bright-star catalogue (Yale BSC5) and stellar astrophysics: distances, absolute magnitudes, effective temperature from colour, bolometric corrections from a black-body V-band integral, luminosity, radius, Planck spectra and black-body colours.

`pinneapple_physics/astro/stars.py` · 198 linhas · testes: 23

nome	tipo	assinatura	o que faz
`StarCatalog`	classe	`(hr, name, desig, con, ra, dec, pmra, pmdec, v, bv, sp, plx, plx_dyn, rv)`	

nome	tipo	assinatura	o que faz
load_bright_stars	função	() -> StarCatalog	
load_constellations	função	() -> list	IAU constellation figures with a key (the IAU abbreviation; Serpens is split into Ser1 = Caput and Ser2 = Cauda), the Latin name, the names per language ({"en": ..., "pt": ...}) and a label point.
constellation_names	função	(lang=None) -> dict	Constellation names by key (IAU abbreviation, Ser1/Ser2 for the two halves of Serpens): the IAU Latin name (en) and the Brazilian Portuguese name (pt), e.g. Cru -> Crux / Cruzeiro do Sul. With lang, a {key: name} map in that language. Sources: constellation_names.json.
planck_lambda	função	(wavelength_m, temperature_k)	Spectral radiance B_lambda (W m^-2 sr^-1 m^-1).
wien_peak_m	função	(temperature_k)	
blackbody_rgb	função	(temperature_k)	sRGB colour of a black body (reuses the black-hole renderer's fit, :mod:pinneapple_physics.blackhole).
blackbody_spectrum	função	(temperature_k, n=300, lam_min_nm=100.0, lam_max_nm=3000.0)	
teff_from_bv	função	(bv)	Effective temperature (K) from B-V, Ballesteros (2012, EPL 97, 34008), black-body based; good to ~5% for main-sequence and giant stars between B-V = -0.3 and 1.8.
bolometric_correction	função	(teff)	BC_V for a black body of temperature teff, zero-pointed so that BC_V(Sun, 5772 K) = M_bol,sun - M_V,sun = -0.07. A physically transparent approximation (real stellar atmospheres differ by ~0.1-0.3 mag for cool M stars and hot O stars).
star_properties	função	(cat, i, extinction_av=0.0) -> dict	Distance, absolute magnitude, effective temperature, luminosity and radius of catalogue star i.
hr_diagram	função	(cat, max_rel_parallax_error=None, min_parallax_mas=5.0)	Points (B-V, M_V) for stars with a usable trigonometric parallax (>= min_parallax_mas).

pinneapple_physics.astro.stars_ml

Machine learning on the bright-star catalogue, checked against the catalogue's own spectral classification.

pinneapple_physics/astro/stars_ml.py · 154 linhas · testes: 16

nome	tipo	assinatura	o que faz
luminosity_group	função	(sp)	
spectral_class	função	(sp)	
dataset	função	(cat=None, min_parallax_mas=5.0)	

nome	tipo	assinatura	o que faz
gmm_fit	função	(X, k=3, iters=200, seed=0)	Gaussian mixture by expectation-maximisation (full covariances), k-means++-style initialisation.
cross_validate	função	(X, y, k=15, folds=5, seed=0)	
analyse	função	(seed=0) -> dict	

pinneapple_physics.astro.surrogate

A physics-informed neural surrogate for orbit propagation, checked against the classical integrator.

pinneapple_physics/astro/surrogate.py · 268 linhas · testes: 32

nome	tipo	assinatura	o que faz
reference_trajectory	função	(body, jd_tdb, days, n=400, eph=None)	Heliocentric positions (AU) and velocities (AU/day) of every body from the Yoshida-4 N-body integration.
train_orbit_pinn	função	(body='mars', jd_tdb=2461327.0, days=365.0, **kwargs) -> dict	Train the PINN orbit surrogate (thread-safe; see _train_orbit_pinn).

pinneapple_physics.astro.timescales

Time scales for positional astronomy: UTC, TAI, TT, TDB, UT1 and sidereal time.

pinneapple_physics/astro/timescales.py · 216 linhas · testes: 9

nome	tipo	assinatura	o que faz
Time	classe	(utc, dut1)	An instant (or array of instants) on the UTC scale with the derived scales as Julian dates.
tai_minus_utc	função	(jd_utc)	TAI-UTC in seconds. Before 1972 returns nan (UTC then was rubber seconds; use :func:delta_t).
delta_t_model	função	(year)	TT - UT1 (s) from the Espenak & Meeus (2006) polynomials, 1900-2150. Used before 1972 and as a fallback.
dut1	função	(jd_utc)	
tdb_minus_tt	função	(jd_tt)	TDB - TT (s), the 2-term periodic approximation (error < 30 us, Explanatory Supplement eq. 10.2).
jd_from_datetime	função	(dt) -> float	
datetime_from_jd	função	(jd) -> datetime	
julian_centuries	função	(jd)	
era	função	(jd_ut1)	Earth rotation angle (rad), IAU 2000.
gmst_rad	função	(t)	Greenwich mean sidereal time (rad), IAU 2006 (ERA-based, Capitaine et al. 2005).
gast_rad	função	(t)	Greenwich apparent sidereal time (rad): GMST + equation of the equinoxes (Delta-psi cos eps).
gast_hours	função	(t)	

pinneapple_physics.astro.validation

Accuracy checks of the observatory's methods against independent references, as data.

`pinneapple_physics/astro/validation.py` · 198 linhas · testes: 16

nome	tipo	assinatura	o que faz
`check_novas`	função	`()`	
`check_horizons`	função	`()`	
`check_usno_almanac`	função	`()`	
`check_eclipses`	função	`()`	
`check_iss_passes`	função	`()`	
`check_skyfield_live`	função	`(n=24, seed=1)`	Random epochs 1972-2049 (the UTC era) and observers: topocentric apparent alt/az of all bodies vs skyfield (optional).
`check_integrators`	função	`()`	
`run_all`	função	`(include_skyfield=True) -> list[dict]`	

pinneapple_physics.blackhole

Black-hole accretion: simulations and "black hole weather" forecasting with a trust horizon.

`pinneapple_physics/blackhole/__init__.py` · 20 linhas · testes: 19

pinneapple_physics.blackhole.capture

Record a Twin3D scene to PNG frames / GIF with headless Chromium (Playwright), for docs and posts.

`pinneapple_physics/blackhole/capture.py` · 79 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`capture_twin`	função	`(folder, steps, out_dir=None, *, field='', cmap='turbo', view='', zoom=1.0, size=(1280, 7…)`	Screenshot the exported scene in `folder` at each time step; returns the PNG paths.

pinneapple_physics.blackhole.convergence

Resolution study of the black-hole accretion simulations: is the forecaster's training data converged?

`pinneapple_physics/blackhole/convergence.py` · 262 linhas · testes: 19

nome	tipo	assinatura	o que faz
`window_mask`	função	`(grid) -> np.ndarray`	Cells of any grid whose centre lies in the physical forecast window.

nome	tipo	assinatura	o que faz
`cell_volumes`	função	`(grid) -> np.ndarray`	2 pi r^2 dr d(cos theta) of every cell.
`remap`	função	`(a, faces_f, faces_c) -> np.ndarray`	Conservative (volume-weighted) remap of cell averages (..., nr, nth) between two spherical grids covering the same domain; the grids need not nest. The volume r^2 dr sin(theta) dtheta is separable, so the remap is one overlap matrix per axis.
`remap_r`	função	`(a, rf_f, rf_c) -> np.ndarray`	Remap of a radial profile (shell averages) by overlap length in log r.
`psd_slope`	função	`(x, dt, f_lo, f_hi) -> float`	Slope of the Welch power spectrum of x in log-log between f_lo and f_hi.
`run_statistics`	função	`(folder, t0, t1, log_rho_range=LOG_RHO_RANGE) -> dict`	Time-window statistics of one run on its own grid: scalars as time series, profiles and fields as time means.
`resolution_study`	função	`(runs, *, t0=2000.0, t1=10000.0, log=print) -> dict`	`runs`: {nr: [run folders, one per seed]} with ntheta = nr / 2 and a constant refinement ratio.

`pinneapple_physics.blackhole.dataset`

The black-hole weather dataset: density frames of whole simulated trajectories, packed for the model zoo.

`pinneapple_physics/blackhole/dataset.py` · 170 linhas · testes: 19

nome	tipo	assinatura	o que faz
`encode`	função	`(log10_rho) -> np.ndarray`	log10 density -> uint16 (raises outside [Q_LO, Q_HI]).
`decode`	função	`(q) -> np.ndarray`	uint16 column `log10_rho_q` -> log10 density (float32).
`deterministic_npz`	função	`(arrays) -> bytes`	npz with fixed zip timestamps and order, so the SHA-256 does not change between identical builds.
`build_arrays`	função	`(runs, conv=None, *, splits=SPLITS, t0=T0, t1=T1) -> dict[str, np.ndarray]`	Arrays of the zoo dataset (`<split>__<column>`) from the simulate_run folders under `runs` and the resolution-study report `conv` (its scalars per grid and seed become the `convergence` split).
`write_npz`	função	`(arrays, path) -> Path`	All splits in one file (`<split>__<column>` arrays).
`write_split_files`	função	`(arrays, folder) -> dict[str, Path]`	One `<split>.npz` per split in `folder` (the zoo layout for large datasets: a user downloads only the split they load, and every file stays far below the 100 MB a git hosting allows). Returns {file name: path}.
`load_trajectories`	função	`(split, path=None) -> dict[int, dict]`	Whole trajectories of a split, by seed, frames in time order. `path`: a local data.npz (default: the zoo).

nome	tipo	assinatura	o que faz
`window_volumes`	função	`(path=None) -> np.ndarray`	2 pi r^2 dr d(cos theta) of the window cells.

`pinneapple_physics.blackhole.figures`

Figures for the black-hole weather study: skill against lead time and the resolution study.

`pinneapple_physics/blackhole/figures.py` · 121 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_skill`	função	`(report, path, model=None, title='') -> str`	Left: MAE against lead for the model and the baselines. Right: skill score against persistence with its 95 % interval and the skill horizon.
`plot_convergence`	função	`(report, path) -> str`	Time-mean shell density and specific angular momentum on every grid, and the mean accretion rate per grid with its 95 % interval.

`pinneapple_physics.blackhole.forecast`

Black-hole weather forecasting: the U-Net of Duarte, Nemmen & Navarro (2022), a trust horizon and a physics check.

`pinneapple_physics/blackhole/forecast.py` · 473 linhas · testes: 19

nome	tipo	assinatura	o que faz
`DensityCodec`	classe	`(log_min, log_max, r_cells, theta_trim)`	Log-normalisation and crop shared by training and inference (stored with the checkpoint).
`DuarteUNet`	classe	`(filters=32, frames=5, residual=False, padding='zeros')`	The U-Net of Duarte et al. (2022) (`create_auto_encoder` in the original `src/models.py`).
`TrainConfig`	classe	`(filters, frames, residual, padding, epochs, batch_size, lr, alpha, clip, loss, rollout_b…)`	
`MassEnvelope`	classe	`(rate_max, margin)`	Bound on how fast the mass in the forecast window can change, learned from the training simulations.
`MassProjection`	classe	`(codec, cell_volume, envelope, frame_dt)`	Project forecast frames onto the mass budget seen in training: the window mass may change between frames by at most `envelope.rate_max` (relative, per unit time), so a frame that gains or loses more is rescaled (uniformly in density, i.e. a shift in normalised log density) to the nearest allowed …
`make_blocks`	função	`(x, k=5, stride=1, out_blocks=1) -> tuple[np.ndarray, np.ndarray]`	Frames (T, H, W) -> inputs (N, k, H, W) = frames i..i+k-1 and targets = frames i+k..i+(1+out_blocks)k-1 (`out_blocks` > 1 gives the targets of a multi-step rollout loss).
`duarte_loss`	função	`(pred, target, alpha=8.0)`	MAE over the window + alpha * MAE over the high-density pixels (target > 0.5).

nome	tipo	assinatura	o que faz
`duarte_regional_loss`	função	`(pred, target, regions=None)`	MSE over the window + sum of weight * MAE over the paper's four regions (fractions of the crop).
`train_forecaster`	função	`(X, Y, Xv, Yv, cfg, codec, out_dir, *, log=print, cell_volume=None, init=None) -> dict[st…)`	Adam + the Duarte loss; keeps the weights with the best validation loss in `out_dir/forecaster.pt`.
`train_on_trajectories`	função	`(train, val, cfg, out_dir, *, cell_volume=None, val_blocks=1, val_stride=1, init=None, lo…)`	Train on whole trajectories of log10 density frames (T, H, W), already cropped to the forecast window and `frame_dt` apart: blocks never cross trajectories, and validation trajectories are separate ones.
`evaluate_loss`	função	`(model, X, Y, cfg, batch=32) -> float`	Loss of the forecast unrolled over all the target blocks in `Y` (one block: the plain loss).
`save_forecaster`	função	`(path, model, cfg, codec, meta) -> None`	
`load_forecaster`	função	`(path) -> tuple[DuarteUNet, DensityCodec, dict]`	
`rollout`	função	`(model, x0, n_blocks, *, clip=False, mass_projection=None) -> np.ndarray`	Iterative forecast: feed each predicted block back in. x0 (k, H, W) -> (n_blocks * k, H, W).
`lead_time_scores`	função	`(pred, truth, last_input, mean_state, weights=None) -> dict[str, np.ndarray]`	Per lead (frame) on normalised log-density: MAE of the forecast, of persistence (the last input frame) and of the time-mean flow; the anomaly correlation with the truth, anomalies taken from the time-mean flow (the ACC of weather forecasting); and the **tendency correlation**, between the forecast …
`trust_horizon`	função	`(scores, frame_dt, acc_min=0.6, block=1) -> dict[str, float]`	Lead time up to which the forecast stays useful: before its anomaly correlation first drops below `acc_min` (the weather convention), and before its error first exceeds persistence / the time mean.
`window_mass`	função	`(rho, cell_volume) -> np.ndarray`	Mass in the (cropped) window for density frames (T, H, W) and cell volumes (H, W).

`pinneapple_physics.blackhole.hydro`

Viscous hydrodynamics of accretion around a Schwarzschild black hole, axisymmetric (r, theta), PyTorch.

`pinneapple_physics/blackhole/hydro.py` · 506 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RIAFConfig`	classe	`(nr, ntheta, r_in, r_out, gamma, alpha, viscosity, cfl, rho_floor, p_floor, v_max, t_cap,…)`	

nome	tipo	assinatura	o que faz
`AccretionFlow`	classe	`(cfg)`	2.5-D (axisymmetric, with v_phi) viscous accretion flow; `run` saves snapshots of the primitives.
`torus_state`	função	`(flow) -> np.ndarray`	Equilibrium torus with l(R) = l_c (R/R_c)^a (pressure maximum at R_c, inner edge at R_edge) in an atmosphere.
`bondi_pw`	função	`(r, *, cs_inf, gamma=1.4, rho_inf=1.0)`	Transonic Bondi accretion in the Paczynski-Wiita potential (exact, by root finding).

`pinneapple_physics.blackhole.hydro_numba`

Numba kernels for :class:`~pinneapple_physics.blackhole.hydro.AccretionFlow` (`RIAFConfig(backend="numba")`).

`pinneapple_physics/blackhole/hydro_numba.py` · 321 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NumbaStepper`	classe	`(flow)`	Holds the numpy geometry of an AccretionFlow and advances its state with the numba kernels.
`rhs`	função	`(W, r, th, rf, thf, area_r, area_t, vol, inv_r, geo_t, Rc, gamma, alpha, visc, dU, fluxes)`	dU on interior cells (5, nr, nth) and boundary fluxes [mass_in, mass_out, angmom_in, angmom_out].
`prim_to_cons`	função	`(W, Rc, gamma, U)`	Interior primitives of W (with ghosts) -> conserved U (5, nr, nth).
`cons_to_prim`	função	`(U, W, Rc, r, gamma, rho_floor, p_floor, v_max, t_cap)`	Conserved U -> interior primitives of W, with floors and caps; returns the number of capped cells.
`boundaries`	função	`(W, outer, has_outer)`	
`time_step`	função	`(W, r, dr, dth, gamma, alpha, visc, cfl)`	

`pinneapple_physics.blackhole.raytrace`

General-relativistic ray tracing of an accretion flow around a Schwarzschild black hole ("Interstellar" view).

`pinneapple_physics/blackhole/raytrace.py` · 316 linhas · testes: 19

nome	tipo	assinatura	o que faz
`RayPaths`	classe	`(ray, ir, ith, ds, kdotv, lapse, captured, escape_dir, shape)`	Where each ray crosses the emitting region, in camera-to-far order (flattened over rays).
`Camera`	classe	`(width, height, r_cam, inclination_deg, fov_deg, roll_deg, dphi, ds_in, ds_out, max_phi)`	
`blackbody_rgb`	função	`(T) -> np.ndarray`	Approximate sRGB colour (0..1) of a blackbody at temperature T [K] (Tanner Helland fit), (..., 3).
`star_field`	função	`(width=4096, height=2048, n=9000, seed=3) -> np.ndarray`	Equirectangular star map (H, W, 3) with a faint Milky-Way-like band, deterministic.

nome	tipo	assinatura	o que faz
`bloom`	função	`(img, strength=0.35, sigma=6.0) -> np.ndarray`	Cinematic glow: add a blurred copy of the bright parts.
`render`	função	`(paths, rho, temperature, velocity=None, *, doppler=True, kappa=0.0, exposure=1.0, scale=…)`	Render one frame.

`pinneapple_physics.blackhole.runs`

Generate and read the accretion-flow simulations behind the black-hole weather forecaster (issue #399).

`pinneapple_physics/blackhole/runs.py` · 89 linhas · testes: 19

nome	tipo	assinatura	o que faz
`load_run`	função	`(folder) -> dict`	-> dict(t, frames (T, 5, nr, ntheta), mdot, grid, config).
`simulate_run`	função	`(cfg, folder, *, t_end=10000.0, every=10.0, chunk=50, name=None, log=print) -> str`	Run (or resume) one simulation into `folder`; returns the folder.

`pinneapple_physics.blackhole.skill`

Skill of the black-hole weather forecaster against persistence, climatology and DMD, with physics checks.

`pinneapple_physics/blackhole/skill.py` · 178 linhas · testes: 19

nome	tipo	assinatura	o que faz
`DMDBaseline`	classe	`(trajectories, rank)`	Rank-r DMD of frame sequences (`pinneapple_neural` `DynamicModeDecomposition`, mean-centred), iterated from one frame; the component of that frame outside the DMD subspace is kept frozen (persistence).
`window_mass`	função	`(x, codec, vol) -> np.ndarray`	Mass in the window for normalised frames (..., H, W).
`implied_inflow`	função	`(x, codec, vol, dt, shells) -> np.ndarray`	Net inward mass flow through the shells (radial indices) implied by the change of the enclosed mass, -dM(<r)/dt between consecutive frames: (frames - 1, len(shells)). Ignores flow through the polar edges.
`evaluate_skill`	função	`(models, train, val, test, *, frame_dt, cell_volume, blocks=20, start_every=5, block_M=10…)`	

`pinneapple_physics.blackhole.twin`

3D digital-twin view of an axisymmetric accretion flow with PINNeAPPle-Twin3D.

`pinneapple_physics/blackhole/twin.py` · 120 linhas · testes: 19

nome	tipo	assinatura	o que faz
CutawayMesh	classe	`(r_faces, theta_faces, *, r_view, phi_cut_deg=270.0, n_phi=72, offset=(0.0, 0.0, 0.0))`	Geometry of one cutaway panel and the map from (nr, ntheta) cell fields to its vertices.
sphere	função	`(radius, center=(0, 0, 0), n=24) -> tuple[np.ndarray, np.ndarray]`	
accretion_scene	função	`(grid, panels, times, *, field='log10_density', unit='', r_view=60.0, gap=0.35, title='Bl…)`	One cutaway panel per entry of `panels` (name -> (T, nr, ntheta) cell values), laid out along `layout` (default: left to right on screen for the view that looks into the cut, `view=1,0.75,-1` in the viewer).

pinneapple_physics.closed_form

Real, closed-form (non-PDE) engineering physics models.

`pinneapple_physics/closed_form/__init__.py` · 73 linhas · testes: 103

pinneapple_physics.closed_form.burgers

Exact solution of the viscous Burgers equation with u(x, 0) = -sin(pi x), u(+-1, t) = 0 on x in [-1, 1].

`pinneapple_physics/closed_form/burgers.py` · 73 linhas · testes: 80

nome	tipo	assinatura	o que faz
burgers_sine_exact	função	`(x, t, nu, n_nodes=120) -> np.ndarray`	`u(x, t)` for arrays `x`, `t` of the same shape (broadcast). `t = 0` returns the initial condition.
burgers_sine_inviscid_exact	função	`(x, t) -> np.ndarray`	Entropy solution `u(x, t)` of `u_t + u u_x = 0`, `u(x, 0) = -sin(pi x)`, `u(+-1, t) = 0` on [-1, 1].

pinneapple_physics.closed_form.cantilever_fatigue

Real, closed-form cantilever-beam bending stress + stress-life fatigue model -- standard mechanical-engineering equations (e.g. Shigley, *Mechanical Engineering Design*; Dowling, *Mechanical Behavior of Materials*), not fabricated:

`pinneapple_physics/closed_form/cantilever_fatigue.py` · 73 linhas · testes: 32

nome	tipo	assinatura	o que faz
stress_from_moment_pa	função	`(width_m, thickness_m, moment_n_m) -> float`	sigma = M*c/I for a solid rectangular cross-section (width x thickness), the shared real computation behind both the scalar fixed-end stress and the spatial stress profile below -- one formula, not two independent implementations.
cantilever_root_bending_stress_pa	função	`(width_m, thickness_m, load_n, length_m) -> float`	The real, closed-form ground-truth bending stress at the fixed (pivot) end of a cantilever beam, where the bending moment is maximum (M = F*L).

nome	tipo	assinatura	o que faz
cantilever_bending_stress_at_position_pa	função	(width_m, thickness_m, load_n, length_m, x_from_root_m) -> float	Real spatial bending-stress profile along the SAME cantilever cantilever_root_bending_stress_pa evaluates at a single point -- not a separate/invented formula. With the beam fixed (clamped) at the pivot end (x=0) and the load applied at the free tip (x=L), the bending moment at position x is
basquin_fatigue_life_cycles	função	(stress_amplitude_pa, fatigue_strength_coeff_pa, fatigue_strength_exponent) -> float	Basquin's stress-life relationship solved for cycles to failure N, given a real stress amplitude and the material's real fatigue strength coefficient (sigma_f') and exponent (b).
basquin_allowable_stress_pa	função	(target_life_cycles, fatigue_strength_coeff_pa, fatigue_strength_exponent) -> float	Basquin's equation solved for the maximum stress amplitude that still achieves at least target_life_cycles -- the real, standard way this design problem is posed in practice (design against an allowable stress, not directly against raw cycles, since cycle counts span many orders of magnitude ...

pinneapple_physics.closed_form.compressible

One-dimensional compressible flow of a calorically perfect gas: isentropic ratios, normal shocks, Pitot pressure and the modified-Newtonian stagnation pressure coefficient.

pinneapple_physics/closed_form/compressible.py · 78 linhas · testes: 15

nome	tipo	assinatura	o que faz
isentropic_ratios	função	(M, gamma=1.4) -> dict[str, np.ndarray]	Stagnation-to-static ratios at Mach M (NACA 1135 eqs. 43-45, 80): T0_T = 1 + (gamma - 1) / 2 M^2, p0_p = T0_T^(gamma / (gamma - 1)), rho0_rho = T0_T^(1 / (gamma - 1)) and the area ratio A_Astar = (1 / M) (2 / (gamma + 1) T0_T)^((gamma + 1) / (2 (gamma - 1))) (inf at M = 0).
normal_shock	função	(M, gamma=1.4) -> dict[str, np.ndarray]	Jump across a normal shock at upstream Mach M >= 1 (NACA 1135 eqs. 93-99):
pitot_pressure_ratio	função	(M, gamma=1.4)	Pitot (stagnation) pressure over free-stream static pressure, p02 / p1.
cp_max_newtonian	função	(M, gamma=1.4)	Stagnation-point pressure coefficient of modified Newtonian theory (Lees 1955), Cp_max = (p02 / p1 - 1) / (gamma M^2 / 2), with p02 the Pitot pressure behind the normal shock (:func:pitot_pressure_ratio); the surface law is then Cp = Cp_max sin^2(theta). Tends to ``((gamma + 1)^2 / (4 ...

pinneapple_physics.closed_form.eddy_coil

Impedance of an air-cored eddy-current probe coil above a conductive plate (Dodd & Deeds, 1968).

pinneapple_physics/closed_form/eddy_coil.py · 69 linhas · testes: 27

nome	tipo	assinatura	o que faz
skin_depth	função	(f, sigma, mu_r=1.0) -> float	
coil_impedance	função	(f, *, r1, r2, l1, l2, turns, sigma, thickness=math.inf) -> dict	Air-coil impedance, its change over the plate, the normalised impedance and the skin depth.

pinneapple_physics.closed_form.electromagnetics

Closed-form solutions of the electromagnetic presets (pinneapple_physics.pde_environment.presets. electromagnetics), registered as pp.solve(problem, "analytic") solutions in :mod:pinneapple_physics.solving.

pinneapple_physics/closed_form/electromagnetics.py · 125 linhas · testes: 41

nome	tipo	assinatura	o que faz
two_dielectric_capacitor	função	(y, V, d, d1, eps1, eps2)	Potential across a parallel-plate capacitor (phi = 0 at y = 0, V at y = d) filled with eps1 for y < d1 and eps2 above: D is uniform, so E_i = D / eps_i with D = V / (d1 / eps1 + (d - d1) / eps2) (Griffiths, *Introduction to Electrodynamics*, 4th ed., ex. 4.7).
wire_az	função	(x, y, a, J, nu=1.0)	A_z of a round conductor (radius a, uniform current density J) in a medium of reluctivity nu, gauge A = 0 at r = a: -J r^2 / (4 nu) + J a^2 / (4 nu) inside, -(J a^2 / (2 nu)) ln(r / a) outside (Ampere's law).
tm_cavity_mode	função	(x, y, t, m, n, Lx, Ly, eps=1.0, mu=1.0)	The TM_mn standing wave of a PEC rectangle: Ez = sin(kx x) sin(ky y) cos(w t), Hx = -(ky / (mu w)) sin(kx x) cos(ky y) sin(w t), Hy = (kx / (mu w)) cos(kx x) sin(ky y) sin(w t) with w = sqrt((kx^2 + ky^2) / (eps mu)) (Pozar sec. 6.3).
skin_effect_slab	função	(x, delta, A0=1.0)	Field diffusing into a conducting half-space x > 0 from a surface value A0 (phasor, exp(+j omega t)): A = A0 exp(-(1 + j) x / delta), delta = sqrt(2 nu / (omega sigma)) (Jackson sec. 5.18, Griffiths sec. 9.4.1).
cylinder_tm_total	função	(x, y, k0, radius, eps_r)	Total Ez of a unit plane wave exp(-j k0 x) on a dielectric cylinder (series of cylinder functions, :func:pinneapple_simulation.electromagnetics.waves.cylinder_scattering_tm).

pinneapple_physics.closed_form.fin_array_conduction

Real, closed-form extended-surface (fin-array) conduction+convection model -- generalized over any array of parallel fin-like extended surfaces on a base (straight rectangular fins on a flat plate, radial vanes on an annular rotor, radiator fins, etc.).

pinneapple_physics/closed_form/fin_array_conduction.py · 125 linhas · testes: 20

nome	tipo	assinatura	o que faz
fin_array_profile_params	função	(k_material_w_mk, h_conv_w_m2k, n_fins, fin_thickness_m, fin_length_m, transverse_extent_…)	Shared real intermediate quantities behind the fin-array model. Computed once so a scalar ground-truth temperature and a spatial (position-dependent) profile evaluated from the SAME real physics can never silently drift into two different implementations.

nome	tipo	assinatura	o que faz
`fin_array_base_temperature_c`	função	`(params, t_ambient_c, power_w) -> float`	The real, closed-form scalar ground truth: temperature at the base of the fin array (before any further, separate resistance -- e.g. a downstream product's own die-to-base spreading term -- is added on top by the caller).
`fin_array_temperature_at_position_c`	função	`(params, t_ambient_c, x_from_root_m, corrected_length_m=None) -> float`	Real spatial temperature profile *inside* the same fin equation `fin_array_base_temperature_c` is built on -- not a separate/invented formula. Standard 1D fin conduction result (Incropera): with the fin root (x=0) at `theta_fin_base_c` above ambient and an adiabatic-tip correction folded into ...

`pinneapple_physics.closed_form.heat`

Transient 1-D heat conduction in a slab, closed form.

`pinneapple_physics/closed_form/heat.py` · 79 linhas · testes: 17

nome	tipo	assinatura	o que faz
`slab_transient`	função	`(x, t, L, alpha, T_left, T_right, T0, n_terms=None) -> np.ndarray`	Temperature `T(x, t)` [same unit as the inputs] of the slab problem above; `x` [m] and `t` [s] broadcast, `alpha` thermal diffusivity [m^2/s].
`pin_fin_theta`	função	`(xi, h, k, D, L)`	Temperature excess theta = (T - T_inf) / (T_b - T_inf) along a pin fin of diameter `D` and length `L` with a convective tip, at `xi = x / L` (Incropera et al., Fundamentals of Heat and Mass Transfer, Table 3.4, case A).
`pin_fin_heat`	função	`(h, k, D, L, T_base, T_inf) -> float`	Heat a pin fin with a convective tip dissipates [W] (the same table, case A).

`pinneapple_physics.closed_form.heat_exchanger`

Heat-exchanger rating: effectiveness-NTU relations, the LMTD correction factor, and tube- and shell-side coefficients of a shell-and-tube exchanger.

`pinneapple_physics/closed_form/heat_exchanger.py` · 155 linhas · testes: 41

nome	tipo	assinatura	o que faz
`effectiveness`	função	`(ntu, cr, arrangement='counterflow', n_shell=1) -> float`	epsilon(NTU, C_r) of the given arrangement ("counterflow", "parallel", "shell_tube", "crossflow").
`ntu_from_effectiveness`	função	`(eps, cr, arrangement='counterflow') -> float`	Inverse of :func:`effectiveness` (bisection; NTU up to 50).
`lmtd_correction_1_2`	função	`(t_hot_in, t_hot_out, t_cold_in, t_cold_out) -> float`	F of a 1-2 exchanger (shell fluid either side), Bowman et al. (1940).
`gnielinski`	função	`(re, pr) -> dict[str, float]`	Tube-side Nusselt number (Gnielinski; laminar fully developed Nu = 3.66 below Re = 2300).

nome	tipo	assinatura	o que faz
`hausen_mean_nusselt`	função	`(re, pr, d_over_l) -> float`	Mean Nusselt number over a heated length L of a tube at constant wall temperature, laminar, velocity already developed (thermal entry): Hausen (1943), Incropera 6th ed. eq. 8.57. Tends to 3.66 for a long tube.
`mean_nusselt_from_temperatures`	função	`(re, pr, d_over_l, t_wall, t_in, t_out) -> float`	Mean Nusselt number of a tube at constant wall temperature from its bulk inlet and outlet temperatures: the energy balance `ln((Tw - Ti) / (Tw - To)) = 4 (L / D) Nu_m / (Re Pr)` (Incropera eq. 8.41b with h_m = Nu_m k / D).
`kern_shell`	função	`(m_dot, mu, k, cp, *, shell_id, baffle_spacing, tube_od, pitch, layout='triangular') -> d…)`	Shell-side coefficient by Kern's method: crossflow area, equivalent diameter, Re, Nu, h.
`one_two_exchanger_ode`	função	`(ntu, cr, n=4001, profiles=False)`	Effectiveness of a 1-2 exchanger by integrating its three energy equations directly. The shell fluid is well mixed across each section, and the two tube passes run in opposite directions (Nagle's model). This is solved as a boundary-value problem by finite differences, so it does not use the closed …

`pinneapple_physics.closed_form.helmholtz_resonator`

Real, closed-form Helmholtz-resonator tuning equation -- the same family of equation that determines the resonant "chuff" frequency of a bottle you blow across, and the standard bass-reflex port tuning equation used throughout loudspeaker cabinet design (Small/Thiele port design method; see e.g. …

`pinneapple_physics/closed_form/helmholtz_resonator.py` · 50 linhas · testes: 4

nome	tipo	assinatura	o que faz
`helmholtz_resonant_frequency_hz`	função	`(port_radius_m, port_length_m, cabinet_volume_l, end_correction_factor=1.7) -> float`	The real, closed-form Helmholtz resonant frequency for a port of the given radius/length feeding a cabinet of the given internal volume. `end_correction_factor` defaults to 1.7 (a port flanged at both ends -- cabinet wall + interior -- a standard loudspeaker-design value); pass a different real …
`port_volume_m3`	função	`(port_radius_m, port_length_m) -> float`	Real port air volume -- used both as a real design constraint (a port shouldn't displace too much of the cabinet's own internal air) and as a real, physically-meaningful minimization objective in multi-objective (Pareto-frontier) port design.

`pinneapple_physics.closed_form.membrane_diffusion`

Real, closed-form membrane-controlled diffusive transport (Fick's first law) -- the same real, standard model used in real reservoir- type transdermal drug patches (e.g. scopolamine, clonidine, fentanyl-class patches; see Robinson & Lee, *Controlled Drug Delivery*), and generically applicable to …

`pinneapple_physics/closed_form/membrane_diffusion.py` · 148 linhas · testes: 6

nome	tipo	assinatura	o que faz
steady_state_release_rate_mg_day	função	(diffusion_coeff_m2_s, partition_coeff, reservoir_conc_kg_m3, area_m2, membrane_thickness…)	The real, closed-form ground-truth steady-state release rate (mg/day) through a rate-limiting membrane, Fick's first law.
depletion_time_days	função	(diffusion_coeff_m2_s, partition_coeff, membrane_thickness_m, reservoir_thickness_m) -> f…)	Real consequence of this model: depletion time depends ONLY on membrane thickness -- area and reservoir concentration cancel out (both the release rate and the total reservoir mass scale with them identically). A genuine derived property of the physics, not a simplification error.
concentration_at_membrane_depth	função	(reservoir_conc_kg_m3, partition_coeff, membrane_thickness_m, z_from_reservoir_m) -> float	Real steady-state concentration profile *inside* the same Fick's-first-law model the release rate is built on -- not a separate/invented formula. Steady-state diffusion through a membrane with a fixed concentration at one face and sink conditions (~zero concentration) at the other is, by …
post_depletion_decay_rate_constant_per_day	função	(diffusion_coeff_m2_s, partition_coeff, membrane_thickness_m, reservoir_thickness_m) -> f…)	Real derived first-order decay constant for the phase after the reservoir's solid/excess drug is exhausted: the remaining dissolved drug is removed through the membrane at a rate proportional to how much is left, using the same real membrane transport coefficient (D*K/L) that governs steady-state …
simulate_release_timeseries	função	(diffusion_coeff_m2_s, partition_coeff, reservoir_conc_kg_m3, area_m2, membrane_thickness…)	Real time-domain simulation of release/depletion over the device's wear life, combining only the real formulas above plus the one derived decay-tail formula -- no invented dynamics.

pinneapple_physics.closed_form.particle_drag

Drag coefficient of a rigid sphere in a steady uniform flow, `C_D = F / (0.5 rho U^2 pi d^2 / 4)` at `Re = U d / nu`.

pinneapple_physics/closed_form/particle_drag.py · 32 linhas · testes: 14

nome	tipo	assinatura	o que faz
clift_sphere_cd	função	(re) -> float	Sphere drag coefficient of Clift, Grace & Weber (1978), Table 5.2, range 20 < Re <= 260 (warns outside).
schiller_naumann_cd	função	(re) -> float	Sphere drag coefficient of Schiller & Naumann (1933), Re < 800 (warns above).

pinneapple_physics.closed_form.pcb_thermal

PCB thermal building blocks (closed form, citable).

pinneapple_physics/closed_form/pcb_thermal.py · 179 linhas · testes: 16

nome	tipo	assinatura	o que faz
Layer	classe	(name, kind, thickness_m, coverage)	

nome	tipo	assinatura	o que faz
`Environment`	classe	`(t_ambient_c, air_velocity_m_s, orientation, emissivity, extra)`	
`stackup`	função	`(n_copper=4, thickness_mm=1.6, outer_oz=1.0, inner_oz=1.0, outer_coverage=0.3, inner_cove…)`	Symmetric stack: signal outer layers (partial copper), plane inner layers, dielectric filling the rest of the board thickness equally.
`effective_conductivity`	função	`(layers) -> Dict[str, float]`	
`via_copper_fraction`	função	`(n_vias, drill_mm, plating_um, area_m2) -> float`	Copper area fraction of plated via barrels over a footprint (unfilled vias).
`convection_coefficients`	função	`(width_m, depth_m, t_surface_c, env) -> Dict[str, float]`	h for the (top, bottom) faces at the given mean surface temperatures.

`pinneapple_physics.closed_form.pipe_flow`

Fully developed pipe and channel flow: Darcy friction factors, pressure drop and near-wall cell sizing.

`pinneapple_physics/closed_form/pipe_flow.py` · 169 linhas · testes: 29

nome	tipo	assinatura	o que faz
`laminar`	função	`(Re) -> float`	Darcy friction factor of laminar fully developed pipe flow, `f = 64 / Re` (Hagen-Poiseuille; White eq. 6.49). Warns for `Re >= 2300`.
`blasius`	função	`(Re) -> float`	Blasius (1913) smooth-pipe correlation `f = 0.316 Re^-0.25` (White eq. 6.55). Warns outside `4e3 <= Re <= 1e5`.
`colebrook`	função	`(Re, rel_rough=0.0, tol=1e-14, max_iter=100) -> float`	Darcy friction factor from the Colebrook (1939) equation (White eq. 6.48),
`haaland`	função	`(Re, rel_rough=0.0) -> float`	Explicit Haaland (1983) approximation of Colebrook (White eq. 6.64), within about 2 % of it:
`dean_number`	função	`(Re, D, Rc) -> float`	Dean number `De = Re sqrt(D / (2 Rc))` of a pipe of diameter D bent to a centre-line radius Rc (same units).
`curved_pipe_ratio`	função	`(De) -> float`	Fully developed laminar friction factor of a curved pipe over that of a straight one at the same Re, White (1929): `f_c / f_s = 1 / (1 - (1 - (11.6 / De)^0.45)^(1 / 0.45))`; 1 for De <= 11.6. Warns above De = 2000 (beyond White's data).
`friction_factor`	função	`(Re, rel_rough=0.0) -> float`	`laminar(Re)` below Re = 2300, `colebrook(Re, rel_rough)` otherwise (the transitional band 2300-4000 is uncertain; Colebrook is used there without a warning).
`darcy_pressure_drop`	função	`(f, L, D, rho, U) -> float`	Darcy-Weisbach pressure drop [Pa] `dp = f (L / D) rho U^2 / 2` (White eq. 6.10) over a length L [m] of a duct of (hydraulic) diameter D [m], density rho [kg/m^3], bulk velocity U [m/s].
`parallel_plates_pressure_drop`	função	`(mu, L, h, U) -> float`	Laminar pressure drop [Pa] between infinite parallel plates a gap `h` [m] apart (plane Poiseuille flow), `dp = 12 mu L U / h^2` with U the mean velocity [m/s] and mu the dynamic viscosity [Pa s] (White §6.6).

nome	tipo	assinatura	o que faz
`first_cell_height`	função	`(y_plus, Re, D, U, nu, f=None, rel_rough=0.0) -> float`	Wall-normal height y [m] of the first cell such that `y+ = y u_tau / nu` equals `y_plus` (the usual y+ estimate; a cell-centred code puts the first centroid at y / 2).

pinneapple_physics.closed_form.pipe_insulation

Heat loss through an insulated pipe and the economic insulation thickness (ISO 12241 / VDI 2055 method).

`pinneapple_physics/closed_form/pipe_insulation.py` · 151 linhas · testes: 27

nome	tipo	assinatura	o que faz
`air`	função	`(t_c) -> dict[str, float]`	
`surface_coefficient`	função	`(ts, ta, d, eps, wind=0.0) -> dict[str, float]`	Outer coefficient (W/m2K) of a horizontal cylinder of diameter d (m) in air: radiation + mixed convection.
`heat_loss`	função	`(d_pipe, thickness, t_in, t_amb, *, k0, k1=0.0, eps=0.9, wind=0.0) -> dict[str, float]`	Heat loss per metre (W/m), surface temperature and coefficients of an insulated (or bare: thickness 0) pipe.
`radial_fd`	função	`(d_pipe, thickness, t_in, t_surface, *, k0, k1=0.0, n=400) -> dict[str, np.ndarray]`	Temperature across the insulation by finite volumes in r with k(T), Picard iterations. The inner and outer temperatures are given. Returns r, T and the heat flow per metre at the inner and outer faces.
`crf`	função	`(rate, years) -> float`	Capital recovery factor `i (1+i)^n / ((1+i)^n - 1)`.
`economic_thickness`	função	`(d_pipe, t_in, t_amb, thicknesses, *, k0, k1, eps, wind, hours, energy_price, efficiency,…)`	Annual cost per metre (energy + annualised installed insulation) for each thickness. `energy_price` is the price per kWh of fuel, which `efficiency` converts to heat. `cost_per_m3` is charged on the insulation volume, `cost_per_m2` on the cladding area. Returns the table, the economic …

pinneapple_physics.closed_form.plate_fin_heatsink

Plate-fin heat sink thermal/hydraulic model (closed form, citable).

`pinneapple_physics/closed_form/plate_fin_heatsink.py` · 309 linhas · testes: 18

nome	tipo	assinatura	o que faz
`HeatSinkGeometry`	classe	`(base_width, base_depth, base_thickness, n_fins, fin_thickness, fin_height, material)`	All lengths in metres. Fins run along `base_depth` (the flow / vertical direction) and are distributed across `base_width`, with a fin at each edge.
`OperatingPoint`	classe	`(power_w, t_ambient_c, air_velocity_m_s, source_width, source_depth, tim_resistance_k_m2_…)`	

nome	tipo	assinatura	o que faz
air_properties	função	(t_c, p_pa=101325.0) -> Dict[str, float]	
geometry_errors	função	(g, op) -> List[str]	
forced_convection	função	(g, v_approach, air) -> Dict[str, float]	
natural_convection	função	(g, dt_k, air) -> Dict[str, float]	
spreading_resistance	função	(k, tb, source_area, plate_area, r_conv) -> Dict[str, float]	Lee et al. (1995). Includes 1D conduction through the base.
evaluate	função	(g, op, max_iter=60, h_scale=1.0, t_root_guess_c=None) -> Dict[str, object]	Thermal resistance network: T_source = T_amb + Q (R_tim + R_spread + R_fins).

pinneapple_physics.closed_form.pump_npsh

Centrifugal pump on a suction / discharge system: duty point, NPSH available against required, cavitation margin.

pinneapple_physics/closed_form/pump_npsh.py · 94 linhas · testes: 27

nome	tipo	assinatura	o que faz
iapws97_psat	função	(t_c) -> float	Saturation pressure of water (Pa) at t_c (°C), IAPWS-IF97 eq. 30.
water_density	função	(t_c) -> dict[str, float]	
atmosphere	função	(z_m) -> float	ISO 2533 pressure (Pa) at altitude z_m (troposphere).
pipe_loss	função	(q, d, length, k_minor, rough, rho, mu) -> dict[str, float]	Head loss (m) of a pipe with minor losses at flow q (m3/s).
fit_quadratic	função	(q, y) -> np.ndarray	Least-squares y = c0 + c1 q + c2 q^2 through catalogue points; returns (c0, c1, c2).
duty_point	função	(head_coef, h_static, system_loss, q_max) -> float	Flow where the pump head equals the system head (bisection; system_loss(q) in m).
npsh_available	função	(p_surface, t_c, z_suction, h_loss) -> dict[str, float]	NPSH available (m) at the pump suction flange for a liquid surface at p_surface (Pa abs) and z_suction (m, liquid level above the pump centre line: negative for a suction lift).

pinneapple_physics.closed_form.rocket_nozzle

Ideal rocket nozzle: 1-D isentropic flow of a calorically perfect gas through a converging-diverging nozzle.

pinneapple_physics/closed_form/rocket_nozzle.py · 114 linhas · testes: 9

nome	tipo	assinatura	o que faz
area_ratio	função	(M, gamma) -> float	A/A* at Mach M (both branches).
mach_from_area_ratio	função	(ratio, gamma, *, supersonic=True, tol=1e-12, max_iter=300) -> float	Invert A/A*(M) = ratio (>= 1) on the supersonic (default) or subsonic branch, by bisection.
pressure_ratio	função	(M, gamma) -> float	Static over stagnation pressure p/p0 at Mach M.
temperature_ratio	função	(M, gamma) -> float	Static over stagnation temperature T/T0 at Mach M.
choked_mass_flow	função	(p0, throat_area, gamma, R_gas, T0) -> float	Mass flow (kg/s) through a choked throat (Sutton & Biblarz eq. 3-24).
characteristic_velocity	função	(gamma, R_gas, T0) -> float	c* = p0 A_t / mdot (m/s): depends on the gas and chamber temperature only (eq. 3-32).
thrust_coefficient	função	(p_exit, p0, p_atm, ratio, gamma) -> float	C_F = momentum term + (p_e - p_a) / p0 * A_e/A_t (eq. 3-30).
specific_impulse	função	(cf, c_star, g0=G0) -> float	Isp = C_F c* / g0 (s), eq. 3-33.
nozzle_performance	função	(*, gamma, R_gas, T_chamber, p_chamber, area_ratio_exit, p_atm=101325.0) -> dict	Exit state and performance of an ideal nozzle expanded to area_ratio_exit = A_e/A_t.

pinneapple_physics.closed_form.thin_airfoil

Closed-form 2-D section aerodynamics: thin-airfoil lift of a NACA 4-digit camber line and a profile-drag correlation for a streamlined section.

pinneapple_physics/closed_form/thin_airfoil.py · 61 linhas · testes: 4

nome	tipo	assinatura	o que faz
naca4_camber_slope	função	(x, m, p) -> np.ndarray	dz/dx of the NACA 4-digit mean line (max camber m at chord fraction p), the two parabolic arcs.
thin_airfoil_cl	função	(m, p, alpha_deg, n_quad=4001) -> float	Thin-airfoil lift coefficient of a NACA 4-digit camber line at alpha_deg (trapezoidal quadrature in theta0; the integrands are bounded, the kink at x = p converges as 1/n_quad).
turbulent_flat_plate_cf	função	(reynolds) -> float	Mean skin-friction coefficient of a fully turbulent flat plate, 0.074 / Re^0.2 (5e5 < Re < 1e7).
hoerner_profile_cd	função	(t_c, reynolds) -> float	Profile drag coefficient of a streamlined 2-D section of thickness ratio t_c (Hoerner form factor, two wetted sides, referred to the chord).

pinneapple_physics.codegen

pinneapple_physics.codegen — automatic physical-code generation.

pinneapple_physics/codegen/__init__.py · 51 linhas · **sem teste direto**

pinneapple_physics.codegen.fdm_script_generator

Standalone finite-difference *script* generator for PINNeAPPle.

pinneapple_physics/codegen/fdm_script_generator.py · 548 linhas · testes: 13 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
PDEParser	classe	()	Classify a pinneapple_physics.pde_environment.ProblemSpec (or any duck-typed object exposing .pde.kind, .coords, .fields, .domain_bounds, .pde.params) into one of the three curated FDM templates FDMScriptGenerator knows how to render.
FDMScriptGenerator	classe	(parser=None) -> None	Renders a curated PDE description into a standalone, runnable FDM Python script (source text, not an in-memory callable).

pinneapple_physics.codegen.fenics_script_generator

Standalone UFL/dolfinx *script* generator for PINNeAPPle.

pinneapple_physics/codegen/fenics_script_generator.py · 614 linhas · testes: 13

nome	tipo	assinatura	o que faz
FEniCSScriptGenerator	classe	()	Renders a curated set of UFL/dolfinx finite-element physics problems into standalone, runnable Python scripts, chosen by inspecting a ProblemSpec-like object's physics shape (see classify_physics_shape / the module docstring).
classify_physics_shape	função	(spec) -> str	Classify a ProblemSpec-like object into "diffusion" | "stokes" | "elasticity" — see module docstring for the exact heuristic.

pinneapple_physics.compile_api

pp.compile: optimise a physics problem once, solve it faster, with the same results.

pinneapple_physics/compile_api.py · 137 linhas · testes: 5

nome	tipo	assinatura	o que faz
Benchmark	classe	(problem, epochs, repeats, baseline_s, compiled_s, speedup, max_param_diff, max_param_abs…)	
CompiledProblem	classe	(problem, optimize='physics') -> None	A problem with its optimisations decided. solve and loss_fn use them.
compile	função	(problem, optimize='physics', **preset_kwargs) -> CompiledProblem	Prepare a problem (a PhysicalProblem, a ProblemSpec or a preset name) for faster solving.

pinneapple_physics.conservation

Conservation as a hard constraint on model outputs: project a predicted field so that its weighted integral (total mass, energy, charge, water) equals a prescribed value exactly.

pinneapple_physics/conservation.py · 83 linhas · testes: 6

nome	tipo	assinatura	o que faz
`ConservationProjection`	classe	`(model, weights, mode='additive', target_fn=None)`	`model` followed by an exact projection. `target_fn(x, y_pred)` gives the conserved total for each batch entry (e.g. `lambda x, y: integral(x_state, w)` for a closed system); by default the integral of the input field `x` itself (same grid as the output).
`integral`	função	`(field, weights, dims=1) -> Tensor`	Weighted sum over the last `dims` dimensions (`weights` has that trailing shape).
`project_integral`	função	`(field, weights, target, mode='additive', dims=None, eps=1e-30) -> Tensor`	Return `field` corrected so that `integral(field, weights) == target` (per leading batch entry).

pinneapple_physics.darcy_checks

Physics checks of a 2-D Darcy field, `-div(a grad u) = f` on the unit square with `u = 0` on the boundary.

`pinneapple_physics/darcy_checks.py` · 857 linhas · testes: 26

nome	tipo	assinatura	o que faz
`CoefficientEnvelope`	classe	`(a_min, a_max, ranges, rel_tol)`	What the training coefficients looked like: the values `a` took and the range of the field statistics.
`DarcyCheck`	classe	`(name, kind, values, value, limit, passed, critical, detail)`	One check over a batch: the per-sample values, the aggregate compared with the limit, pass / fail.
`UseProfile`	classe	`(name, use, limits, reasons)`	What a prediction is used for, and the physics limits that use needs. `limits`: metric (a key of :func:`physics_metrics`) -> limit; `reasons`: why each limit, written before any model is checked.
`darcy_operator_apply`	função	`(u, a) -> np.ndarray`	`A_a u`: the finite-volume operator of `-div(a grad u)` (with `u = 0` on the boundary) applied to `u`.
`darcy_residual`	função	`(u, a, f=1.0) -> np.ndarray`	Finite-volume residual `f - A_a u` per cell (same shape as `u`); zero for the solver's own solution.
`residual_error_bound`	função	`(u, a, f=1.0) -> np.ndarray`	Certified relative error of `u` in the energy norm of the discrete problem, from its residual alone.
`residual_l2_bound`	função	`(u, a, f=1.0) -> np.ndarray`	Certified relative L2 error of `u` (to the discrete solution), from its residual alone.
`boundary_flux_balance`	função	`(u, a, f=1.0) -> dict[str, np.ndarray]`	Net outward flux through the boundary against the integral of the source (per sample).
`boundary_values`	função	`(u) -> np.ndarray`	`u` on the four sides, `1.5 u_0 - 0.5 u_1` from the two cells next to each face: `(batch, 4, n)` (order x_min, x_max, y_min, y_max; `(4, n)` for one field).

nome	tipo	assinatura	o que faz
coefficient_stats	função	(a) -> dict[str, np.ndarray]	Per-sample descriptors of a coefficient field used for the applicability check.
darcy_checks	função	(a, u, f=1.0, *, reference=None, envelope=None, tolerances=None, quantile=0.9, physics_ki…)	Physics checks of predicted Darcy fields u for coefficients a ((n, n) or (batch, n, n)).
physics_metrics	função	(a, u, f=1.0) -> dict[str, np.ndarray]	Per-sample values of the physics checks, keyed like :data:DEFAULT_TOLERANCES (energy_bound, l2_bound, bc, flux, positivity); no reference solution needed.
calibrate_tolerances	função	(a, u, f=1.0, *, quantile=0.99, floor=None) -> dict[str, float]	Limits for :func:darcy_checks from a model's own predictions u on held-out in-distribution inputs a.
profile_verdicts	função	(a, u, f=1.0, *, profiles=None, quantile=0.9) -> dict[str, Any]	Per use profile, over a batch: each metric's quantile value, its limit and ratio = value / limit, the profile's ratio (the worst one, 1 = at the limit) and its decision (APPROVED when every metric is within its limit, else REJECT).
sample_checks	função	(a, u, f=1.0, *, tolerances=None, envelope=None) -> list[dict[str, Any]]	The checks of one prediction ((n, n)), each {"key", "value", "limit", "passed"}, as pinneapple check runs them (physics = "darcy"): the physics metrics against tolerances (default :data:DEFAULT_TOLERANCES) and, with envelope, the coefficient against the training envelope …
sample_verdict	função	(checks) -> str	PASS when every check of :func:sample_checks passes, else FAIL (the exit status of pinneapple check).
estimated_error	função	(u, a, f=1.0) -> np.ndarray	Map of the error of u estimated from its residual alone: e ~ L^-1 r / a_h.
flux_magnitude	função	(u, a) -> np.ndarray	|a grad u| per cell from the solver's own face fluxes (harmonic-mean faces, half-cell Dirichlet faces): each component is the mean of the cell's two face flux densities.
estimated_flux_error	função	(u, a, f=1.0) -> np.ndarray	Map of the error of the flux a grad u estimated from the residual alone (:func:estimated_error), relative to the largest flux of u: |a grad e| / max |a grad u|.
region_masks	função	(n, wall_cells=2, interior_from=8) -> dict[str, np.ndarray]	Boolean masks of an n x n cell grid: wall (cells closer than wall_cells to the boundary) and interior (at least interior_from cells away).
regional_errors	função	(u, a, f=1.0, *, wall_cells=2, interior_from=8) -> dict[str, np.ndarray]	Relative error estimated without a reference (:func:estimated_error) in the wall and interior regions of :func:region_masks, per sample: ||e||_region / ||u||_region.
use_verdicts	função	(a, u, f=1.0, *, envelope=None, wall_cells=2, interior_from=8) -> dict[str, dict[str, Any…)	For one prediction ((n, n)): which uses it can serve, and why (:data:USES), from the residual alone, plus inputs (the coefficient inside the training envelope). Each entry: ok, the measured values, the limits and a one-line reason.
trust_report	função	(checks, *, model='', kinds=None)	The deploy decision (TrustReport: APPROVED / REVIEW / REJECT) of a list of :class:DarcyCheck, e.g. the checks of several :func:darcy_checks calls (fixed limits and calibrated ones); kinds keeps only those.

nome	tipo	assinatura	o que faz
darcy_trust_report	função	(a, u, f=1.0, *, model='', kinds=…, **kwargs)	(TrustReport, checks): the deploy decision (APPROVED / REVIEW / REJECT) of :func:darcy_checks.
save_darcy_npz	função	(path, a, u, **extra)	One Darcy field as the grid result pinneapple check reads (physics = "darcy"): cell-centre axes x, y of the unit square, coefficient a and solution u (float32).
check_config_entry	função	(name, path, *, f=1.0, tolerances=None, envelope=None, coefficient='a', solution='u') -> …)	One [[result]] table of a pinneapple-check.toml for a Darcy prediction saved with :func:save_darcy_npz: physics = "darcy", the source, the tolerances (metric keys of :func:physics_metrics) and the coefficient envelope (bounds of a and the envelope statistics).
nodal_darcy_residual	função	(u, k, f=1.0, dx=1.0, *, cropped=1, cross_denominator=None) -> np.ndarray	-(k lap u + grad k . grad u) - f at the stored nodes whose four neighbours are known: (batch, m, m) with m = n - 1 when the high side is cropped, n otherwise.
nodal_boundary_values	função	(u, *, cropped=1) -> np.ndarray	u extrapolated linearly from the two nearest stored nodes to the zero-ghost nodes on the four sides: (batch, 4, n) (x_min, x_max, y_min, y_max). Low ghost at index -1, high ghost at index n + cropped.
nodal_flux_balance	função	(u, k, f=1.0, dx=1.0, *, cropped=1) -> np.ndarray	Relative imbalance of a control-volume budget around the stored nodes (whole block if not cropped, all but the last row and column if cropped): net outward flux (harmonic-mean face coefficients) against f times the area. Zero for a conservative solution; per sample.
solve_nodal_darcy	função	(k, f=1.0, dx=1.0, *, cross_denominator=None) -> np.ndarray	Direct solve of -(k lap u + grad k . grad u) = f on an n x n node block with zero ghosts at index -1 and n (coefficient clamped at the edges): the reference for :func:nodal_darcy_residual with cropped=0.
check_nodal_darcy_batch	função	(k, u, f=1.0, dx=1.0, *, cropped=1, alt_cross_denominators=None, tolerances=None, floor_f…)	Physics checks of a batch of node-based Darcy training pairs (k, u) before anyone trains on them.

pinneapple_physics.deepxde_bridge

pinneapple_physics.deepxde_bridge -- DeepXDE <-> PINNeAPPle integration glue.

pinneapple_physics/deepxde_bridge/__init__.py · 60 linhas · testes: 9

pinneapple_physics.deepxde_bridge.adapter

ProblemSpec -> real DeepXDE dde.Model translation and training.

pinneapple_physics/deepxde_bridge/adapter.py · 468 linhas · **sem teste direto** · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
DeepXDESolveResult	classe	(deepxde_model, spec, final_loss, train_time_s)	Outcome of :func:solve_with_deepxde.

nome	tipo	assinatura	o que faz
`require_deepxde`	função	`() -> Any`	Import `deepxde`.
`is_deepxde_available`	função	`() -> bool`	Return `True` if `deepxde` can be imported.
`solve_with_deepxde`	função	`(spec, *, n_domain=2000, n_boundary=200, n_initial=100, layers=(50, 50, 50, 50), epochs=1…)`	Translate `spec` into a real DeepXDE problem and train it for real.

`pinneapple_physics.ensemble`

Adaptive ensembles of physics models: PINNs, neural operators (FNO, DeepONet...), GNNs (MeshGraphNet...), numerical solvers and closed forms, selected and combined online from the errors they make on new cases.

`pinneapple_physics/ensemble.py` · 636 linhas · testes: 68

nome	tipo	assinatura	o que faz
`Expert`	classe	`(name, predict, cost, kind)`	`predict(query) -> array` of the common output shape; `cost`: fixed cost per call (any unit), or None to use the measured wall time.
`EnsembleRun`	classe	`(names, errors, ensemble_errors, weights, active, coverage, costs, predictions, suspect)`	
`PhysicsEnsemble`	classe	`(experts, mode='combine', etas=(0.5, 2.0, 8.0), alphas=(0.0, 0.01, 0.05), point_weights=N…)`	Online selection / combination of physics models.
`relative_l2`	função	`(pred, ref, weights=None) -> float`	
`from_callable`	função	`(name, fn, cost=None, kind='numerical') -> Expert`	
`from_solution`	função	`(name, solution, points, cost=None, shape=None) -> Expert`	A `pp.solve` result (PINN, FEM, analytic, external solver...): evaluated at `points(query)`.
`from_torch`	função	`(name, model, to_input, from_output=None, cost=None, kind='neural') -> Expert`	Any `torch.nn.Module` (FNO, DeepONet, MeshGraphNet, a PINN network...): `to_input(query)` builds its input (tensor, GraphBatch...), the output's `.y` is used if present, and `from_output(y, query)` maps it to the common points (e.g. drop the batch and channel axes).
`residual_baseline`	função	`(experts, domain_queries, residual_fn) -> dict[str, float]`	Median physics residual of each expert on queries from its own domain (its training or validation cases), known before operation: the reference for `PhysicsEnsemble(residual_baseline=...)`. Experts missing from `domain_queries` are left out (they fall back to their recent history).

nome	tipo	assinatura	o que faz
`fit_static_weights`	função	`(predictions, references, n_folds=5, point_weights=None, seed=0) -> dict[str, Any]`	Offline ensemble: convex weights (>= 0, sum 1) minimising the summed squared error on a validation set.

pinneapple_physics.ensemble_viz

Animated view of an adaptive physics ensemble: prediction against the reference field, case by case, with the model the ensemble chose and how its weights moved.

`pinneapple_physics/ensemble_viz.py` · 131 linhas · testes: 3

nome	tipo	assinatura	o que faz
`animate_ensemble`	função	`(run, references, path='ensemble.gif', *, x=None, times=None, frames_per_case=4, fps=10, …)`	Write a GIF of the ensemble run. `cases`: subset of case indices to animate (default: all). `regimes`: `[(first_case, label), ...]` drawn as dashed lines on the timeline. Returns the path written.

pinneapple_physics.experiment

`Experiment`: one reproducible run of a method on a problem, with its metrics and provenance.

`pinneapple_physics/experiment.py` · 174 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ExperimentResult`	classe	`(config, problem_fingerprint, solution, metrics, wall_time_s, started_at, environment, n_…)`	
`Experiment`	classe	`(problem, method, options, seed, reference, reference_options, n_eval, metric_names, name…)`	A method applied to a problem, with a seed, options and an optional reference to score against.
`run_all`	função	`(experiments) -> Dict[str, ExperimentResult]`	Run several experiments; keyed by experiment name.

pinneapple_physics.maxwell_checks

Physics checks of electromagnetic fields from any source (a PINN, an FNO, a graph network, a solver, measured data), without a reference solution: every check asks whether the fields obey Maxwell's equations and the balance laws that follow from them.

`pinneapple_physics/maxwell_checks.py` · 549 linhas · testes: 51

nome	tipo	assinatura	o que faz
`MaxwellCheck`	classe	`(name, key, value, limit, passed, critical, detail, kind)`	

nome	tipo	assinatura	o que faz
FieldSet	classe	(grid, fields, *, eps=EPS0, mu=MU0, sigma=0.0, omega=None, interior=1)	Fields on a tensor grid with derivatives; see the module docstring for the layout.
relative_residual	função	(terms, mask=None, floor=0.0) -> float	||sum terms|| / sum ||term|| (L2 over the masked points); NaN when every term is zero or below floor (the law does not constrain the field: e.g. div D of a field that varies only along D's normal direction).
gauss_electric	função	(grid, fields, *, eps=EPS0, tol=None, **kw) -> MaxwellCheck	
gauss_magnetic	função	(grid, fields, *, mu=MU0, tol=None, **kw) -> MaxwellCheck	
faraday	função	(grid, fields, *, mu=MU0, omega=None, tol=None, **kw) -> MaxwellCheck	
ampere_maxwell	função	(grid, fields, *, eps=EPS0, mu=MU0, sigma=0.0, omega=None, tol=None, **kw) -> MaxwellCheck	
continuity	função	(grid, fields, *, omega=None, tol=None, **kw) -> MaxwellCheck	
poynting_balance	função	(grid, fields, *, eps=EPS0, mu=MU0, sigma=0.0, omega=None, tol=None) -> MaxwellCheck	Poynting's theorem over the grid box.
tangential_continuity	função	(F1, F2, normal, *, surface_current=None, tol=None, name='E') -> MaxwellCheck	n x (F1 - F2) = K at interface points: F1, F2 are (n, 3) fields on the two sides at the same points, normal a unit normal (3,) or (n, 3) pointing from side 2 to side 1; K the surface current (for H; zero for E). Value: RMS mismatch over RMS of the tangential fields.
pec_boundary	função	(grid, fields, faces, *, tol=None) -> MaxwellCheck	n x E = 0 on grid faces declared PEC ("x_min", "y_max", ...): RMS tangential E on those faces over RMS |E| in the whole field.
reciprocity	função	(S, *, tol=None) -> MaxwellCheck	S = S^T for a reciprocal network (S of shape (n, n) or (f, n, n)): max over frequencies of ||S - S^T||_F / ||S||_F.
passivity	função	(S, *, lossless=False, tol=None) -> MaxwellCheck	Passivity: I - S^H S positive semi-definite, i.e. the largest singular value of S is at most 1, at every frequency; value = max(sigma_max - 1, 0). lossless=True checks unitarity ||S^H S - I||.
maxwell_checks	função	(grid, fields, *, eps=EPS0, mu=MU0, sigma=0.0, omega=None, laws=None, pec_faces=(), refer…)	Every check the given fields allow: the laws whose terms are present (laws restricts them), the Poynting balance (needs E and H or B, and a t axis or omega), PEC faces, and a relative L2 comparison with reference (a mapping of the same component names; evaluation only).

nome	tipo	assinatura	o que faz
maxwell_trust_report	função	(grid, fields, *, model='', **kwargs)	(TrustReport, checks): the deploy decision (APPROVED / REVIEW / REJECT) of :func:maxwell_checks.
sample_checks	função	(grid, fields, **kwargs) -> list[dict[str, Any]]	The checks as pinneapple check runs them (physics = "maxwell"): {"key", "name", "value", "limit", "passed", "detail"} per check.
sample_verdict	função	(checks) -> str	
save_maxwell_npz	função	(path, grid, fields, **extra)	Write a grid result pinneapple check reads with physics = "maxwell": 1-D axes and component arrays. Complex phasors are stored as <name>_re / <name>_im pairs.
check_config_entry	função	(name, path, *, parameters=None, tolerances=None) -> str	A [[result]] block of pinneapple-check.toml for a field saved by :func:save_maxwell_npz.

pinneapple_physics.metrics

pp.metrics: the error metrics used everywhere in the library, with one convention.

pinneapple_physics/metrics.py · 182 linhas · testes: 36

nome	tipo	assinatura	o que faz
l2	função	(pred, true)	Euclidean norm of the error, per field: ||pred - true||_2.
relative_l2	função	(pred, true)	||pred - true||_2 / ||true||_2 per field (nan where ||true|| = 0).
rmse	função	(pred, true)	Root mean square error per field.
mae	função	(pred, true)	Mean absolute error per field.
max_abs	função	(pred, true)	Largest absolute error per field (the L-infinity norm of the error).
relative_linf	função	(pred, true)	max|pred - true| / max|true| per field (nan where max|true| = 0).
r2	função	(pred, true)	Coefficient of determination per field (nan when true is constant).
per_field	função	(metric, pred, true, fields) -> dict[str, float]	{field: value} for an (N, F) comparison.
summary	função	(pred, true, fields, names=…) -> dict[str, dict[str, float]]	{metric: {field: value}} for several metrics at once (the shape benchmark tables use).

nome	tipo	assinatura	o que faz
pooled	função	(pred, true) -> dict[str, float]	One number per metric over every entry, all fields together: relative_l2, rmse, mse, max_abs, r2. For leaderboards that rank models by a single value; use the per-field functions for anything reported per quantity. When the shapes differ, a 1-D side gets a trailing axis and the ...
rollout_relative_l2	função	(pred_seq, true_seq, *, aggregate='steps') -> float	Relative L2 error of a time rollout: pred_seq and true_seq have shape (T, ...), axis 0 is time.

pinneapple_physics.multiscale

Multiscale physics for adaptive ensembles: filters, closures, multiscale residuals, cross-scale consistency and scale-aware model selection.

pinneapple_physics/multiscale/__init__.py · 90 linhas · testes: 29

pinneapple_physics.multiscale.closures

Subfilter closures for 2-D large-eddy simulation in vorticity form, and the tests that tell whether a closure can be trusted.

pinneapple_physics/multiscale/closures.py · 381 linhas · testes: 29 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Smagorinsky	classe	(cs, delta, name)	nu_t = (cs delta)^2 |S|. delta defaults to the solver's implicit filter width.
DynamicSmagorinsky	classe	(delta, test_ratio, test_kind, name, last_c)	Dynamic Smagorinsky: nu_t = C delta^2 |S| with C = <L_ij M_ij> / <M_ij M_ij> (domain average, clipped at 0) from the test filter of width test_ratio * delta and kind test_kind. The last coefficient is kept in self.last_c.
GradientModel	classe	(delta, coef, name)	Clark gradient (nonlinear) model tau_ij = coef * delta^2 / 12 * d_k u_i d_k u_j (deviatoric part).
NoModel	classe	(name)	No subfilter model (an under-resolved DNS).
AprioriReport	classe	(closure, filter, stress_correlation, tendency_correlation, transfer_true, transfer_model…)	
PosterioriReport	classe	(closure, times, spectrum_error, field_error, dissipation_les, dissipation_ref, stable, t…)	
ClosureHealth	classe	(score, ok, reasons, enstrophy_ratio, tail_ratio, germano)	

nome	tipo	assinatura	o que faz
strain	função	`(w, solver) -> dict[str, np.ndarray]`	Velocity gradients, strain `S11 = u_x`, `S12 = (u_y + v_x) / 2` and `|S| = sqrt(2 S_ij S_ij)`.
stress_tendency	função	`(a, b, solver) -> np.ndarray`	Vorticity tendency `-curl(div tau)` of a deviatoric stress `(tau_xx, tau_xy) = (a, b)`.
true_subfilter	função	`(w_fine, filt, n_coarse=None, length=2 * np.pi) -> dict[str, np.ndarray]`	Exact subfilter quantities of a resolved vorticity field for the filter `filt`: the filtered field `w_bar`, the deviatoric stress `(a, b)`, the vorticity flux `sigma` and the tendency `-div(sigma)`, restricted to `n_coarse` points per axis (exact when the filter removes the modes the …
apriori_test	função	`(w_fine, filt, closure, n_coarse, length=2 * np.pi) -> AprioriReport`	A-priori test of `closure` on a resolved field: filter it with `filt`, restrict to the LES grid of `n_coarse` points, and compare the closure's stress and tendency (computed from the filtered field) with the exact subfilter terms.
spectrum_error	função	`(w, w_ref, kmax, length=2 * np.pi, kmin=1) -> float`	Mean absolute difference of `log10 E(k)` over the shells `kmin <= k <= kmax` (0: same spectrum; 0.3: a factor of 2 on average).
aposteriori_test	função	`(les, w0_bar, ref_bar, times, dt=0.005, t_long=None) -> AposterioriReport`	Run the LES `les` (its closure included) from the filtered initial field `w0_bar` (on its grid) and compare with the filtered reference `ref_bar` (`(len(times), n, n)`, same grid) at `times`; then keep running to `t_long` (default 4 x the last time) for the long-run stability.
closure_health	função	`(w, w0, kc, length=2 * np.pi, *, forced=False, max_enstrophy_growth=1.05, tail_band=(0.05…)`	Label-free health of an LES (or any filtered) prediction `w` at cut-off `kc` (shells), started from `w0` (same filter):
germano_error	função	`(w, closure, solver, test_ratio=2.0, test_kind='sharp', delta=None) -> float`	Label-free cross-scale check of a closure through the Germano identity: the resolved stress between the grid filter and a test filter of width `test_ratio * delta`, `L_ij = hat(u_i u_j) - hat(u_i) hat(u_j)`, is known exactly from the LES field; a consistent closure must reproduce it as …

pinneapple_physics.multiscale.comparison

Compare ensembles on a stream of multiscale queries: scale-blind against scale-aware selection, with and without closure / consistency checks and scale-linking models.

`pinneapple_physics/multiscale/comparison.py` · 439 linhas · testes: 29

nome	tipo	assinatura	o que faz
EnsembleConfig	classe	`(aware, router, checks, experts, error_tolerance, cost_weight)`	`aware`: scale-aware post-filtering (else blind); `router`: :class:`~.scales.ScaleAwareRouter` with the fidelity ladder; `checks`: prediction checks and the closed multiscale residual (else the grid-scale residual); `experts`: the experts it may use; `error_tolerance`: escalate up the …
evaluate_experts	função	`(path, queries, initial_states, experts, checks=None, n_out=None) -> dict[str, np.ndarray]`	Run every expert on every query once. `experts[name](w0, query)` returns the output on the common grid, or a pair `(output, internals)` whose internals are handed to `checks[name](internals, w0, query) -> (score, reason)`. A failure (exception, non-finite values) is stored as NaN. Returns …

nome	tipo	assinatura	o que faz
adapt_output	função	(raw, source, target, aware, length=2 * np.pi) -> np.ndarray	The answer to a query for target from an output that carries source: blind (target applied on top) or scale-aware (:func:~.filters.refilter).
raw_residual	função	(pred, nu, dt, length=2 * np.pi) -> float	Relative residual of the vorticity equation between the two snapshots of pred at the grid scale.
closed_multiscale_residual	função	(pred, nu, dt, filt, h, length=2 * np.pi) -> float	Residual of the filtered vorticity equation at the requested width and at twice it, with the subfilter tendency estimated by the gradient model at each width removed; mean of the two relative residuals.
run_ensembles	função	(queries, references, table, profiles, configs, *, dt, h, length=2 * np.pi, cost_weight=0…)	Every configuration on the same stream, replaying the outputs of :func:evaluate_experts. Queries carry id, nu, re and filter. Costs are the median measured wall times. check_names: experts whose stored checks are used as prediction checks (default: all that have one).
summarise_runs	função	(runs, profiles, configs, widths, *, wrong_factor=1.5, wrong_margin=0.03, solver=None, or…)	Per configuration, pooled over the streams in runs ({seed: (queries, results)}): mean error, error per requested width (with n), spectrum error, wrong selections (error above max(wrong_factor * best, best + wrong_margin), best = best expert available to the configuration on that query), …
closure_flag_table	função	(tables, targets, names, kc, length=2 * np.pi) -> list[dict[str, Any]]	How the label-free checks flagged each expert in names: count, and the expert's field and spectrum error against targets ({seed: [reference at the expert's own filter, per query]}) when flagged and when not, with the reasons given.
markdown_tables	função	(rows, widths, flags=(), sc_rows=(), bench_rows=()) -> str	Markdown of :func:summarise_runs, :func:closure_flag_table, the scale-conditioning study and the closure benchmark.
summary_figure	função	(rows, widths, path, sc_rows=(), colors=…, unit='h') -> Path	Four panels: error per requested width and configuration; error against cost (with the baselines); wrong selections and scale violations; the scale-conditioning study.
frontier_figure	função	(points, path, baselines=(), colors=…, title=…) -> Path	One line per configuration through its operating points (cost, error, label) (e.g. a sweep of the cost weight or of the error tolerance); baselines as grey squares. Lower-left is better.

pinneapple_physics.multiscale.consistency

Cross-scale consistency: a fine and a coarse prediction of the same state must agree once both are brought to the same scale.

pinneapple_physics/multiscale/consistency.py · 114 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ConsistencyReport	classe	(passed, field_error, spectrum_error, integrals, failed, tolerance, reasons)	

nome	tipo	assinatura	o que faz
cross_scale_consistency	função	(fine, coarse, coarse_filter, fine_filter=None, *, length=2 * np.pi, tol=0.05, integrals=…)	Compare fine (filtered by fine_filter, default unfiltered, on its grid) with coarse (filtered by coarse_filter, on a grid of equal or fewer points) after bringing the fine one to the coarse scale and grid. Arrays are (n, n) or with leading axes (e.g. snapshots). integrals …

pinneapple_physics.multiscale.filters

Spatial filters for large-eddy simulation and multiscale checks on periodic grids.

pinneapple_physics/multiscale/filters.py · 271 linhas · testes: 29

nome	tipo	assinatura	o que faz
Filter	classe	(kind, delta)	A filter of width delta (physical length) and kind in :data:FILTER_KINDS. delta = 0 is the identity (a fully resolved field).
transfer	função	(kind, k, delta) -> np.ndarray	1-D transfer function G_hat(k) of a filter of width delta (delta = 0: identity).
wavenumbers	função	(n, length=2 * np.pi) -> np.ndarray	Angular wavenumbers of an n-point periodic grid, in FFT order.
apply_filter	função	(u, filt, length=2 * np.pi, axes=None) -> np.ndarray	Filter a periodic field over axes (default: the last two, or the last one for 1-D data).
restrict	função	(u, n, axes=None) -> np.ndarray	Fourier restriction to a coarser periodic grid of n points per axis: keeps the modes |k| < n / 2 (the Nyquist mode of the coarse grid is dropped, so the result is real). The mean, hence the integral over the domain, is preserved to round-off.
prolong	função	(u, n, axes=None) -> np.ndarray	Fourier interpolation (zero padding) to a finer periodic grid of n points per axis.
post_filter_transfer	função	(source, target, k, max_gain=2.0) -> np.ndarray	1-D transfer of the post-filter that brings a field filtered by source to target: G_target / G_source where that ratio is at most max_gain in magnitude, 0 elsewhere. A mild deconvolution (gain up to 2 by default, e.g. a sharp cut-off at k = 4 from a slightly damped Gaussian output) …
realizability_mismatch	função	(source, target, kmax, *, slope=-2.0, n=4096, max_gain=2.0, kmin=1.0) -> float	Fraction (0 to 1, as an L2 ratio) of the target filter that cannot be obtained by post-filtering a field already filtered by source. The transfer functions are compared on kmin <= k <= kmax (kmin: the fundamental wavenumber, 1 on a 2 pi domain) weighted by a model spectrum of …
refilter	função	(u, source, target, length=2 * np.pi, axes=None, max_gain=2.0) -> np.ndarray	Turn a field filtered by source into the target filter (as far as it can be, see :func:post_filter_transfer).
filter_variable_width	função	(u, kind, delta, length=2 * np.pi, axes=None, decimals=10) -> np.ndarray	Filter whose width varies in space: u_bar(x) = sum_k G_hat(k; delta(x)) u_hat(k) exp(i k x).

nome	tipo	assinatura	o que faz
spectral_derivative	função	(u, axis, length=2 * np.pi, order=1) -> np.ndarray	Exact derivative of the trigonometric interpolant along axis (the Nyquist mode is dropped for odd orders).
commutation_error	função	(u, kind, delta, axis=-1, length=2 * np.pi, axes=None) -> np.ndarray	d(u_bar)/dx - (du/dx)_bar for a filter of width delta (scalar or a field). Zero to round-off for a uniform width; for a width that varies in space it is the term that a residual of the filtered equations picks up from filtering alone, not from the model (to leading order ``(d Delta / dx) d ...
low_modes	função	(kmax) -> list[tuple[int, int]]	Half-plane list of the 2-D integer wavenumbers with |kx|, |ky| <= kmax (sharp square cut-off), mean excluded: one of each pair k, -k.
fourier_basis	função	(modes, n, length=2 * np.pi) -> np.ndarray	(2 len(modes), n * n) real basis cos, sin(kx x + ky y) on an n x n grid (k in units of 2 pi / length).
fourier_coefficients	função	(w, modes, length=2 * np.pi) -> np.ndarray	Coefficients of a periodic field on :func:fourier_basis (exact for modes below the grid's Nyquist).

pinneapple_physics.multiscale.learned

Learned models that connect scales (PyTorch).

pinneapple_physics/multiscale/learned.py · 501 linhas · testes: 29

nome	tipo	assinatura	o que faz
ScaleConditionedFNO2d	classe	(in_channels=1, out_channels=1, width=32, modes=12, layers=4, conditioned=True, delta_ref…)	FNO on a periodic grid whose layers are modulated by the filter width: x -> gelu(gamma(Delta) * (K x + W x) + beta(Delta)). forward(x, delta) with x (B, C, n, n) and delta (B,) or a float. conditioned=False gives the scale-blind network (gamma = 1, beta = 0).
TorchVorticityLES2D	classe	(n, length=2 * math.pi, dtype=torch.float64)	Differentiable dealiased pseudo-spectral 2-D vorticity solver (integrating-factor RK4) with an optional closure closure(w (B, n, n), solver) -> tendency. nu per sample in :meth:rollout.
LearnedClosure	classe	(hidden=16, layers=3, kernel=5)	Periodic CNN closure: inputs omega / rms, u, v (scaled) and the strain components; output the subfilter vorticity flux sigma (2 channels), scaled by delta^2 * rms(omega)^2; tendency -div(sigma).
FourierGalerkinPINN	classe	(kmax=4, hidden=256, scale=5.0, t_ref=0.3, length=2 * math.pi)	Physics-informed network for the large scales of 2-D vorticity, trained on the physics only.
torch_transfer	função	(kind, k, delta) -> torch.Tensor	Transfer function of :func:~.filters.transfer in torch (delta may be a tensor broadcastable to k).
torch_filter	função	(x, kind, delta, length=2 * math.pi) -> torch.Tensor	Filter x (B, C, n, n) with width delta (scalar or (B,)).

nome	tipo	assinatura	o que faz
torch_refilter	função	(x, kind, d_from, d_to, length=2 * math.pi, max_gain=2.0) -> torch.Tensor	Bring x filtered at d_from to d_to (per sample), as :func:~.filters.refilter.
filtering_consistency_loss	função	(model, x, d_small, d_large, kind='gaussian', length=2 * math.pi) -> torch.Tensor	Mean squared mismatch between filter(model(x, d_small)) -> d_large and model(x, d_large), relative to the mean square of the latter.
fit_scale_conditioned	função	(model, inputs, targets, kind='gaussian', steps=2000, batch=16, lr=0.002, consistency_wei…)	Train on one dataset seen at several widths: inputs (N, C, n, n) unfiltered, targets {delta: (N, C', n, n) filtered at delta}. Each step draws a batch and a width; the input is filtered at that width when input_filter; the loss is the relative MSE plus consistency_weight times …
train_closure_aposteriori	função	(closure, solver, w0, nu, reference, dt, save_steps, steps=300, batch=8, lr=0.001, seed=0…)	A-posteriori training: roll the LES out from the filtered initial fields w0 (N, n, n) with closure and match the filtered reference reference (N, len(save_steps), n, n): relative MSE of the fields plus spectrum_weight times the mean squared log ratio of the energy spectra (shells …
as_numpy_closure	função	(closure, n, length=2 * math.pi)	Wrap a learned closure as closure(w, solver) -> tendency for :class:~.ns2d.VorticityNS2D (numpy, no gradient). stress is not defined for it (it models the vorticity flux directly).
periodic_cnn2d	função	(in_channels=2, out_channels=2, width=32, dilations=(1, 2, 4, 8, 4, 2, 1)) -> nn.Sequenti…)	Dilated 2-D CNN with circular padding (a periodic domain), GELU activations.
torus_graph	função	(n, offsets=(1, 2, 4)) -> tuple[torch.Tensor, torch.Tensor]	Edges of an n x n periodic grid graph to the neighbours offsets cells away along each axis (a multiscale graph, so a few message-passing steps reach far), with edge attribute (dx, dy) / 8. Node index i * n + j.
fit_supervised	função	(forward, params, X, Y, steps, seed=0, lr=0.002, batch=16, log='') -> list[float]	Adam with a cosine schedule on the relative mean squared error |f(X) - Y|^2 / |Y|^2 per batch.

pinneapple_physics.multiscale.ns2d

2-D incompressible Navier-Stokes in vorticity form on the periodic square [0, L)^2, for resolved (DNS) and large-eddy (LES) simulation, with the energy spectrum and the residual of the (filtered) vorticity equation.

pinneapple_physics/multiscale/ns2d.py · 218 linhas · testes: 45

nome	tipo	assinatura	o que faz
VorticityNS2D	classe	(n, nu, length, forcing, drag, closure, dealias)	Pseudo-spectral 2-D vorticity solver. closure(omega, solver) -> tendency (physical space, same shape) adds a subfilter model; forcing: (n, n) time-independent vorticity forcing; drag: linear friction mu.

nome	tipo	assinatura	o que faz
`velocity_from_vorticity`	função	`(w, length=2 * np.pi) -> tuple[np.ndarray, np.ndarray]`	`(u, v)` of a periodic vorticity field (zero mean velocity).
`energy_spectrum`	função	`(w, length=2 * np.pi, kmax=None) -> tuple[np.ndarray, np.ndarray]`	Shell-summed kinetic energy spectrum `E(k)` from vorticity, `sum_k E(k) = mean(u^2 + v^2) / 2`. Returns integer shell wavenumbers (in units of `2 pi / L`) and `E` (averaged over leading batch axes).
`random_vorticity`	função	`(rng, n, k0=4.0, energy=0.5, length=2 * np.pi, batch=None) -> np.ndarray`	Random vorticity with energy spectrum `E(k) ~ k^4 exp(-2 (k / k0)^2)` (peak at `k0`), random phases, total kinetic energy `energy`.
`exact_advection`	função	`(w, length=2 * np.pi) -> np.ndarray`	`div(u w)` of the trigonometric interpolant of `w`, without aliasing: the products are formed on a grid 3/2 times finer and truncated back (exact for the modes of the original grid).
`vorticity_residual`	função	`(w_prev, w_next, dt, nu, length=2 * np.pi, forcing=None, drag=0.0, closure_tendency=None)…)`	Residual of the vorticity equation between two snapshots `dt` apart (trapezoidal in time, second order): `R = (w_next - w_prev) / dt + mean(div(u w)) - nu lap(mean w) + mu mean(w) - f - C`, the means taken over the two snapshots, where `C` is the subfilter tendency given in …

`pinneapple_physics.multiscale.residuals`

Physics residuals evaluated at several filter widths.

`pinneapple_physics/multiscale/residuals.py` · 120 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MultiscaleResidual`	classe	`(filters, raw, subfilter, commutation, model)`	
`multiscale_residuals`	função	`(state, residual_fn, filters, *, length=2 * np.pi, subfilter_fn=None, commutation_fn=None…)`	Residual curve of any equation. `residual_fn(filtered_state, filter) -> (R, scale)` evaluates the residual field of the filtered equations and a normalising scale; `subfilter_fn(state, filter)` the subfilter term (from the unfiltered `state`, or modelled); `commutation_fn(state, filter)` …
`ns2d_multiscale_residuals`	função	`(w_prev, w_next, dt, nu, filters, *, length=2 * np.pi, forcing=None, drag=0.0, subfilter=…)`	Multiscale residual of the 2-D vorticity equation between two snapshots `dt` apart (see :func:`~.ns2d.vorticity_residual`). `subfilter`: `"exact"` (computed from the given, unfiltered snapshots: for a resolved field, the model residual is then the true error of the filtered dynamics), …

`pinneapple_physics.multiscale.scales`

Scale-aware model selection: each model declares the scales it resolves, each query the scales it needs.

`pinneapple_physics/multiscale/scales.py` · 245 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ScaleProfile	classe	(name, filter, grid_spacing, closure, re_range, time_step, fidelity, cost, notes)	Scales a model represents. filter: the filter its output carries (Filter("sharp", 0) for a resolved field); grid_spacing: its output grid; re_range: Reynolds numbers it is valid for (training range of a surrogate, resolution limit of a solver); time_step: its time resolution …
ScaleQuery	classe	(filter, re, time_scale, kmax)	Scales a query needs: the filter of the requested output, the Reynolds number of the case and the time scale of the requested quantity (None: not constrained). kmax: largest wavenumber of the output grid (where the filter comparison stops).
ScaleCheck	classe	(name, score, covered, mismatch, reasons)	
ScaleRoute	classe	(chosen, weights, checks, awake, rung, escalated, reasons)	
ScaleAwareRouter	classe	(profiles, query_fn=None, *, strength=1.0, floor=0.0001, abstain_below=0.05, ladder=True,…)	profiles: one :class:ScaleProfile per expert (experts without one are always awake, score 1); query_fn(query) -> ScaleQuery (identity by default). strength: exponent of the compatibility in the prior; abstain_below: compatibility under which an expert sleeps; ladder: keep only …
scale_compatibility	função	(profile, query, *, mismatch_tol=0.25, re_band=0.15, slope=-2.0) -> ScaleCheck	Can profile answer query, and how well do its filtering assumptions match? See the module docs.

pinneapple_physics.multiscale.turbulence

Decaying 2-D turbulence as a multiscale benchmark: resolved reference data, filtered datasets at several widths, query streams that shift the Reynolds number and the requested scale, and the closure benchmark.

pinneapple_physics/multiscale/turbulence.py · 295 linhas · testes: 29

nome	tipo	assinatura	o que faz
DecayingTurbulence2D	classe	(length, n_out, t_out, spinup, k0, energy, re_bands, widths, width_probabilities, kinds, …)	Set-up of the benchmark (defaults: the experiment of docs/core_concepts/multiscale_ensembles.md).
benchmark_closures	função	(bench, data, closures, cases_per_band=8, times=(0.1, 0.2, 0.3), t_long=2.0, dt=0.005) ->…)	Closures on held-out cases of every band, on the LES grid of bench (sharp filter of the dealiased grid).
scale_conditioning_study	função	(bench, data, idx, conditioned, fixed, blind=None, widths=(1, 2, 4, 6, 8), kind='gaussian…)	Errors at each width (widths in units of bench.h) on the cases idx:

pinneapple_physics.multiscale.viz

Animation of a scale-aware ensemble run: the scale each query asks for, the model chosen, the filtered reference against the prediction, their energy spectra up to the requested cut-off, the weights and the timeline of choices.

pinneapple_physics/multiscale/viz.py · 111 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
animate_multiscale_ensemble	função	(references, predictions, active, names, weights, widths, labels, cutoffs, path='multisca…)	One frame per query. references / predictions: 2-D fields at the requested scale; active: model chosen per query; weights: per query {name: weight}; widths: requested filter width per query (in width_unit); labels: text describing each query; cutoffs: requested …

pinneapple_physics.non_hermitian

Non-Hermitian physics: PT-symmetric dimers and chains, exceptional points and how they can (or cannot) be tuned.

pinneapple_physics/non_hermitian/__init__.py · 29 linhas · testes: 8

pinneapple_physics.non_hermitian.ep_tuning

Exceptional-point tuning: can a control parameter move the EP of a PT-symmetric dimer along a desired law?

pinneapple_physics/non_hermitian/ep_tuning.py · 226 linhas · testes: 8

nome	tipo	assinatura	o que faz
EPTuningReport	classe	(ep_law, target, residual, exact, degree, perturbation_class, assumptions, theorem_applie…)	
classify_coupling	função	(b_ab, b_ba=None, lam=None) -> str	"constant", "phase-only", "reciprocal polynomial", "reciprocal non-polynomial" or "non-reciprocal" (b_ba given and not the conjugate of b_ab).
analyze_ep_tuning	função	(b_ab, b_ba=None, target=None, lam=None, dimension=2) -> EPTuningReport	Symbolic check of an EP tuning law. b_ab, b_ba: sympy expressions in lam (default symbol lam, real); b_ba=None means reciprocal (conj(b_ab)). target: desired gamma_EP^2(lam), default J**2 + lam*J*G with the module's symbols (_symbols(): lam real, J > 0, …
coupling_classes	função	() -> list[tuple[str, str, Any, Any]]	The paper's four coupling classes as sympy laws in the module's symbols (_symbols()), one tuple (key, title, b_ab, b_ba) each (b_ba None: reciprocal), ready for :func:analyze_ep_tuning:
route_a_sqrt_coupling	função	(J, G) -> Callable[[float], complex]	b(lam) = sqrt(J^2 + lam J G): gamma_EP^2 = J^2 + lam J G exactly, PT symmetric (defined while the radicand is >= 0; beyond that the coupling becomes imaginary and the EP leaves the real gamma axis).
route_c_asymmetric	função	(J, G)	(b_ab, b_ba) = (J, J + lam G): gamma_EP^2 = J (J + lam G) = J^2 + lam J G exactly, but H12 != conj(H21).
feedback_coupling	função	(J, G, lam, b0=None, iters=60, tol=1e-14) -> tuple[float, int]	Route B illustration: the coupling is updated from a measurement of itself, b <- (b + (J^2 + lam J G) / b) / 2, a polynomial map in (b, 1/b, lam). Its fixed point is b* = sqrt(J^2 + lam J G), the route-A coupling. This shows that a feedback loop *can* produce the non-polynomial dependence the …
theorem_profile	função	()	The theorem as a :class:~pinneapple_physics.physics_audit.ModelProfile: auditing a device design against it says whether the no-go result constrains that design (all four assumptions VALID) or not (one VIOLATED).

nome	tipo	assinatura	o que faz
design_features	função	(H, polynomial) -> dict[str, float]	Features of a device Hamiltonian (at one operating point) for :func:theorem_profile's audit.

pinneapple_physics.non_hermitian.pt_dimer

PT-symmetric dimers and chains: Hamiltonian, spectrum, exceptional points (EPs) and how they coalesce.

pinneapple_physics/non_hermitian/pt_dimer.py · 185 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PTDimer	classe	(coupling, coupling_ba, onsite)	Two-site gain/loss system. coupling: b_ab as a number or a function of the control parameter lam; coupling_ba: b_ba (default conj(b_ab): reciprocal, PT symmetric; give it to model non-reciprocal hopping); onsite: common on-site energy eps.
PTChain	classe	(n, b, b_back)	N-site chain with gain +i gamma on the first site, loss -i gamma on the last and nearest-neighbour coupling b (number or list of N-1 numbers; reciprocal unless reciprocal=False and b_back is given).
pt_violation	função	(H) -> float	||P H* P - H|| / ||H|| with P the exchange (anti-identity) matrix: 0 for a PT-symmetric Hamiltonian.
eigenvector_overlap	função	(H) -> float	Largest |<v_i|v_j>| / (|v_i||v_j|) over pairs of right eigenvectors: 1 when two eigenvectors coalesce (EP), small for well-separated modes of a nearly Hermitian matrix.
petermann_factor	função	(H) -> np.ndarray	K_i = <L_i|L_i><R_i|R_i> / |<L_i|R_i>|^2 for each mode (1 for a Hermitian matrix, diverges at an EP).

pinneapple_physics.non_hermitian.viz

Figures for PT-symmetric dimers and exceptional-point tuning (matplotlib).

pinneapple_physics/non_hermitian/viz.py · 121 linhas · testes: 8

nome	tipo	assinatura	o que faz
plot_spectrum	função	(ax, dimer, lam=0.0, gamma_max=None, n=400)	Real and imaginary parts of the eigenvalues against gamma, the EP, and the eigenvector overlap (twin axis).
plot_ep_laws	função	(ax, J=1.0, G=0.5, lam=None)	gamma_EP^2(lam) of each coupling class against the linear target J^2 + lam J G.
plot_coupling_plane	função	(ax, J=1.0, G=0.5, lam=None)	The coupling b(lam) in the complex plane: gamma_EP is its distance to the origin. A straight (polynomial degree 1) path has a squared distance quadratic in lam; a circle keeps it constant.

nome	tipo	assinatura	o que faz
`plot_chain_threshold`	função	`(ax, sizes=(2, 3, 4, 5), b_values=None)`	PT-breaking threshold gamma_EP^2 of N-site chains with uniform coupling against |b|^2. For a uniform chain H(gamma, b) = |b| H(gamma / |b|, 1) after a gauge transformation, so gamma_EP = kappa_N |b| exactly and the dimer's obstruction carries over to every N. Chains where the control parameter ...
`ep_tuning_figure`	função	`(path, J=1.0, G=0.5) -> str`	

`pinneapple_physics.online_learning`

Online learning with expert advice, shared by every adaptive method in the library (time-series forecasting in `pinneapple_systems.time_series.adaptive` and physics-model ensembles in `pinneapple_physics.ensemble`).

`pinneapple_physics/online_learning.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FixedShare`	classe	`(n, eta, alpha)`	Exponentially weighted average with fixed share (one per horizon).
`AdaHedge`	classe	`(k)`	Parameter-free Hedge: follow the leader while it works, hedge with eta = ln K / Delta otherwise.
`AdaptiveConformal`	classe	`(level=0.9, gamma=0.01, window=500)`	Adaptive conformal inference on absolute residuals: alpha_{t+1} = alpha_t + gamma (alpha - err_t).
`FixedShareGrid`	classe	`(n, etas=(0.5, 2.0, 8.0), alphas=(0.0, 0.01, 0.05))`	Fixed-Share aggregators over `n` experts for every (eta, alpha) in the grid, weighted by AdaHedge.

`pinneapple_physics.paper_problems`

Test problems of published physics-ML papers, as the papers (or, where the paper is silent, their official code) define them: domain, equation, initial / boundary data, training and test grids, and a reference solution.

`pinneapple_physics/paper_problems.py` · 114 linhas · testes: 5

nome	tipo	assinatura	o que faz
`GridCase`	classe	`(name, equation, x_range, t_range, exact, n_train, n_test, bc, lbfgs_steps, coef, source)`	One test problem on a tensor (x, t) grid with an analytic reference.
`tensor_grid`	função	`(x_range, t_range, nx, nt) -> dict[str, np.ndarray]`	The official `get_data` grid: `meshgrid(x, t)` ('xy' indexing, rows are times), flattened row-major.
`time_sequence`	função	`(points, k=5, dt=0.0001) -> np.ndarray`	PINNsFormer's pseudo-sequence: each (x, t) repeated k times with t + i*dt, i = 0..k-1 -> (N, k, 2).
`convection_exact`	função	`(x, t, beta=50.0)`	u_t + beta u_x = 0, u(x, 0) = sin x, periodic on [0, 2 pi]: u = sin(x - beta t).
`reaction_exact`	função	`(x, t, rho=5.0)`	u_t = rho u (1 - u), u(x, 0) = h(x) = exp(-(x - pi)^2 / (2 (pi/4)^2)): logistic solution u = h e^{rho t} / (h e^{rho t} + 1 - h).

nome	tipo	assinatura	o que faz
`wave_exact`	função	`(x, t, c2=4.0)`	u_tt = c2 u_xx on [0, 1], u(0) = u(1) = 0, u(x, 0) = sin(pi x) + 0.5 sin(3 pi x), u_t(x, 0) = 0: u = sin(pi x) cos(c pi t) + 0.5 sin(3 pi x) cos(3 c pi t), c = sqrt(c2).
`pinnsformer_case`	função	`(name) -> GridCase`	

pinneapple_physics.paper_problems_xtfc

The test problems of the X-TFC paper (Schiassi et al., arXiv:2005.10632v1), as the paper states them: equation, domain, constraints, exact solution and the run settings it reports (neurons, activation, weight range, training grid). No result of the paper is in this module; the targets live in ...

`pinneapple_physics/paper_problems_xtfc.py` · 214 linhas · testes: 10

nome	tipo	assinatura	o que faz
`XTFCCase`	classe	`(name, problem, exact, n_train, n_neurons, activation, weight_range, paper_iterations, pa…)`	
`p6_rhs`	função	`(X)`	
`p7_rhs`	função	`(X)`	
`xtfc_case`	função	`(name) -> XTFCCase`	

pinneapple_physics.pcb

Populated-PCB thermal model (the engine of the PCB Hotspot app).

`pinneapple_physics/pcb/__init__.py` · 16 linhas · testes: 10

pinneapple_physics.pcb.engine

PCB Hotspot engine: evaluation, what-if studies and calibration to measurements.

`pinneapple_physics/pcb/engine.py` · 392 linhas · testes: 16

nome	tipo	assinatura	o que faz
`model_scope`	função	`(comps, env) -> Dict[str, Any]`	What the model covers, which way each simplification errs, what to do about it today and what is planned -- shown in every report.
`evaluate`	função	`(board, comps, env, *, quick=False, n_cells=60) -> Dict[str, Any]`	`quick`: one solve on a coarser grid (interactive dragging). Full: band + checks.
`field_3d`	função	`(board, out) -> Dict[str, Any]`	
`what_if`	função	`(board, comps, env, n_cells=48) -> Dict[str, Any]`	Re-solve the board with common fixes and report the effect on the critical component (and on every junction).

nome	tipo	assinatura	o que faz
`calibrate`	função	`(board, comps, env, measurements, *, noise_std_c=1.0, n_ensemble=16, n_iterations=8, seed…)`	Fit surface-cooling and in-plane-spreading scale factors to measured temperatures with PINNeAPPle's Ensemble Kalman Inversion.

`pinneapple_physics.pcb.solver`

3D finite-volume thermal model of a populated PCB.

`pinneapple_physics/pcb/solver.py` · 341 linhas · testes: 16

nome	tipo	assinatura	o que faz
`Component`	classe	`(name, x_mm, y_mm, power_w, package, side, tj_max_c, theta_jb, theta_jc, w_mm, d_mm, h_mm…)`	
`Board`	classe	`(width_mm, depth_mm, n_copper, thickness_mm, outer_oz, inner_oz, outer_coverage, inner_co…)`	
`validate`	função	`(board, comps) -> List[str]`	
`overlaps`	função	`(comps) -> List[str]`	Same-side component pairs whose footprints overlap (a layout error the heat model would otherwise silently superpose).
`solve`	função	`(board, comps, env, *, n_cells=60, h_override=None, k_xy_scale=1.0, h_scale=1.0, max_iter…)`	Steady temperatures. `h_override` fixes {'top','bottom'} coefficients (tests / calibration); `k_xy_scale` / `h_scale` scale the in-plane conductivity and the surface coefficients (calibration / uncertainty).

`pinneapple_physics.pde_environment`

Módulo sem docstring e sem texto.

`pinneapple_physics/pde_environment/__init__.py` · 105 linhas · **sem teste direto**

`pinneapple_physics.pde_environment.builder`

ProblemBuilder — fluent API for constructing ProblemSpec objects.

`pinneapple_physics/pde_environment/builder.py` · 638 linhas · testes: 26

nome	tipo	assinatura	o que faz
`ProblemBuilder`	classe	`(name) -> None`	Fluent builder for `ProblemSpec`.

`pinneapple_physics.pde_environment.capabilities`

PDE family knowledge base — part of pinneapple_environment.

`pinneapple_physics/pde_environment/capabilities.py` · 314 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PDEFamily`	classe	`(kind, aliases, description, default_params, coords_hint, fields_hint, tags)`	Descriptor for a known PDE family.
`list_pde_families`	função	`() -> List[str]`	Return sorted list of canonical PDE kind strings.
`get_pde_family`	função	`(kind_or_alias) -> Optional[PDEFamily]`	Look up a PDEFamily by kind string or alias (case-insensitive).
`identify_pde`	função	`(description) -> List[Tuple[str, float]]`	Score known PDE families against a free-text description.
`suggest_problem_spec`	função	`(description) -> Dict[str, Any]`	Given a free-text description, suggest ProblemSpec constructor arguments.

`pinneapple_physics.pde_environment.condition_sampling`

Sampling points for conditions whose region is given by a selector function.

`pinneapple_physics/pde_environment/condition_sampling.py` · 68 linhas · testes: 15

nome	tipo	assinatura	o que faz
`sample_condition_points`	função	`(select, domain_bounds, coords, n, rng) -> np.ndarray`	Up to `n` points (float32, shape `(m, len(coords))`) accepted by `select(X) -> bool mask`.
`box_face_normals`	função	`(points, domain_bounds, coords, time_coord='t') -> Tuple[np.ndarray, np.ndarray]`	Outward unit normals of the bounding box at `points` (one column per coordinate, 0 for time).

`pinneapple_physics.pde_environment.conditions`

Módulo sem docstring e sem texto.

`pinneapple_physics/pde_environment/conditions.py` · 495 linhas · testes: 26

nome	tipo	assinatura	o que faz
`ConditionSpec`	classe	`(name, kind, fields, selector_type, selector, value_fn, weight, order, deriv_coord, inter…)`	Generic constraint spec.
`DirichletBC`	função	`(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0) -> Con…)`	Construct a Dirichlet boundary condition.
`NeumannBC`	função	`(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0, order=…)`	Construct a Neumann boundary condition.
`RobinBC`	função	`(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0) -> Con…)`	Construct a Robin boundary condition.
`InterfaceBC`	função	`(name, fields, selector_type='all', selector=None, value_fn=None, weight=1.0, k_a=1.0, k_…)`	Construct an interface / coupling boundary condition between two subdomains.
`InitialCondition`	função	`(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0) -> Con…)`	Construct an initial condition.
`DataConstraint`	função	`(name, fields=None, selector_type='all', selector=None, value_fn=None, weight=1.0) -> Con…)`	Construct a data constraint (supervised loss at specific points).

`pinneapple_physics.pde_environment.environment_typing`

Módulo sem docstring e sem texto.

`pinneapple_physics/pde_environment/environment_typing.py` · 6 linhas · testes: 26

`pinneapple_physics.pde_environment.presets`

Módulo sem docstring e sem texto.

`pinneapple_physics/pde_environment/presets/__init__.py` · 14 linhas · testes: 21

`pinneapple_physics.pde_environment.presets._curve_geometry`

Parametric curve/polygon geometry for the two tag-based presets whose domain is NOT a box or a circular cylinder: `axial_compressor_cascade_2d` (a circular-arc cascade blade) and `car_external_aero` (an Ahmed-body- inspired car silhouette). See `tag_geometry.py` for where these are used and …

`pinneapple_physics/pde_environment/presets/_curve_geometry.py` · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`circular_arc_cascade_blade_points`	função	`(n, rng, *, flow_angle_in_deg, flow_angle_out_deg, chord, x_le, y_le) -> np.ndarray`	
`ahmed_body_polygon`	função	`(car_length, car_height, *, slant_angle_deg=_AHMED_DEFAULT_SLANT_ANGLE_DEG, n_arc=24, x0=…)`	Closed polygon (M, 2) tracing the Ahmed-body-inspired silhouette, nose at x=x0, tail at x=x0+car_length, underbody at y=ground_clearance, roof at y=car_height. Vertices are ordered counter-clockwise starting at the bottom of the front fillet.
`naca4_symmetric_polygon`	função	`(chord, thickness_ratio, *, n=80, x0=0.0, y0=0.0) -> np.ndarray`	Closed polygon (M, 2) tracing a symmetric (zero-camber) NACA 4-digit airfoil of the given `chord` and `thickness_ratio` t (e.g. t=0.12 for NACA 0012), leading edge at (x0, y0), trailing edge at (x0+chord, y0). Vertices run leading edge -> upper surface -> trailing edge -> lower surface -> back …
`polygon_perimeter_sample`	função	`(vertices, n, rng) -> np.ndarray`	Sample n points uniformly (by arc length) along the CLOSED polygon whose vertices are given in order (the closing edge back to vertices[0] is included).
`polygon_contains`	função	`(vertices, X) -> np.ndarray`	Vectorized ray-casting point-in-polygon test (closed polygon, vertices in order). Returns a bool array, one per row of X (N, 2).

`pinneapple_physics.pde_environment.presets._utils`

Shared helpers for preset modules (not part of the public API).

`pinneapple_physics/pde_environment/presets/_utils.py` · 10 linhas · **sem teste direto**

`pinneapple_physics.pde_environment.presets.academics`

Módulo sem docstring e sem texto.

`pinneapple_physics/pde_environment/presets/academics.py` · 113 linhas · testes: 37

nome	tipo	assinatura	o que faz
laplace_2d_default	função	() -> ProblemSpec	
poisson_2d_default	função	() -> ProblemSpec	
burgers_1d_default	função	(nu=0.01) -> ProblemSpec	

pinneapple_physics.pde_environment.presets.astroml_bridge

astroML-backed statistics for astronomical data: periodograms, deconvolved densities, fits with errors.

pinneapple_physics/pde_environment/presets/astroml_bridge.py · 576 linhas · testes: 12 · sinais no código: shim ou obsoleto

nome	tipo	assinatura	o que faz
Periodogram	classe	(freq, power, method, n_terms, n_obs, false_alarm_levels, meta)	A periodogram on a frequency grid. freq in cycles per unit of t; power dimensionless.
XDResult	classe	(weights, means, covariances, log_likelihood, model)	Extreme-deconvolution Gaussian mixture of the underlying (error-free) distribution.
LineFit	classe	(slope, intercept, covariance, chi2, dof, method)	y = slope * x + intercept with a 2x2 covariance of (intercept, slope).
astroml_available	função	() -> bool	True if astroML can be imported (does not import the heavy sub-packages).
frequency_grid	função	(t, period_range=None, f_min=None, f_max=None, oversample=10.0, n_max=200000) -> np.ndarr…)	Uniform frequency grid [1 / unit of t] for a periodogram of samples at times t.
lomb_scargle_periodogram	função	(t, y, dy, freq=None, *, period_range=None, oversample=10.0, generalized=True, significan…)	Generalised (floating-mean) Lomb-Scargle periodogram, astroML.time_series.lomb_scargle.
multiterm_periodogram	função	(t, y, dy, freq=None, *, n_terms=3, period_range=None, oversample=10.0, time_unit='d') ->…)	Multi-harmonic periodogram, astroML.time_series.multiterm_periodogram.
bootstrap_peak_distribution	função	(t, y, dy, freq, n_bootstraps=100, random_state=0, generalized=True) -> np.ndarray	Highest periodogram peak of n_bootstraps resamplings of the data (lomb_scargle_bootstrap).
best_period	função	(t, y, dy, *, period_range=None, n_terms=1, oversample=10.0, refine=True) -> float	Period [unit of t] of the highest periodogram peak.
light_curve_sample	função	(t, y, dy, *, band='', time_unit='d', value_unit='mag', **provenance)	An irregularly sampled light curve as a point-domain :class:PhysicalSample (no astroML needed).
xd_fit	função	(X, Xerr, n_components, *, max_iter=200, tol=1e-05, random_state=0) -> XDResult	Extreme deconvolution, astroML.density_estimation.XDGMM.
bayesian_block_edges	função	(t, x=None, sigma=None, *, fitness='events', p0=0.05, gamma=None) -> np.ndarray	Bin edges from Bayesian Blocks (Scargle et al. 2013), astroML.density_estimation.bayesian_blocks.
knuth_histogram	função	(data) -> tuple[np.ndarray, np.ndarray, float]	Histogram with Knuth's (2006) Bayesian optimal bin width: (counts, edges, width).

nome	tipo	assinatura	o que faz
`fit_line_with_errors`	função	`(x, y, dy) -> LineFit`	Weighted least squares with errors on y only, `astroML.linear_model.LinearRegression(y_error=dy)`.
`fit_line_tls`	função	`(x, y, dx, dy, initial=None) -> LineFit`	Line fit with errors on both axes: maximises `astroML.linear_model.TLS_logL` (Hogg, Bovy & Lang 2010).
`load_dataset`	função	`(name, *, data_home=None, download_if_missing=False, **kwargs)`	Call an `astroML.datasets` fetcher by short name (see `DATASETS`) and return what it returns.
`check_period_recovery`	função	`(seed=0, period=0.613, n=150, noise=0.15) -> dict`	Irregular sinusoid (amplitude 0.3, Gaussian noise) over 200 d: recovered period vs the true one.
`check_xd_means`	função	`(seed=0, n=600) -> dict`	Two 2-D Gaussians, per-point errors drawn in [0.2, 1.0]: worst error of the XD means / component spread.
`check_block_step`	função	`(seed=0, t_step=6.3, p0=0.001) -> dict`	Noisy series (300 points over 10 s, step 1 -> 2, noise 0.2): the interior Bayesian-Blocks edge vs the step.
`check_line_fits`	função	`(seed=0) -> dict`	y = 2.5 x - 1 with errors on both axes: worst relative slope error of the WLS and TLS fits.
`validate`	função	`(cases=None, log=print, **kw) -> list[dict]`	Run the synthetic checks; returns one row per case and logs a table. Needs astroML, no network.

`pinneapple_physics.pde_environment.presets.astrophysics`

Astrophysics and space-systems problem presets.

`pinneapple_physics/pde_environment/presets/astrophysics.py` · 1.486 linhas · testes: 72

nome	tipo	assinatura	o que faz
`kepler_exact_state`	função	`(t, mu, a, e) -> Dict[str, np.ndarray]`	Exact Kepler two-body trajectory at times `t` (perigee at t=0).
`kepler_two_body_orbit`	função	`(mu=398600.4418, a=8000.0, e=0.15) -> ProblemSpec`	Restricted two-body (Kepler) orbit -- planar Cartesian formulation.
`cw_exact_velocity`	função	`(t, n, x0, y0, z0, vx0, vy0, vz0) -> Dict[str, np.ndarray]`	Time derivative of :func:`_cw_exact_state` (Clohessy-Wiltshire relative velocity, closed form).
`space_debris_cw_relative_motion`	função	`(n=0.0011, x0=1.0, y0=0.0, z0=0.2, vx0=0.0, vy0=-0.0015, vz0=0.0008) -> ProblemSpec`	Clohessy-Wiltshire (Hill's) equations -- space-debris close-approach / conjunction-assessment and proximity-operations relative motion.
`satellite_j2_perturbation`	função	`(mu=398600.4418, J2=0.00108262668, Re=6378.137, a=7000.0, e=0.001, inclination_deg=98.7) …)`	LEO satellite orbit with the J2 (Earth-oblateness) perturbation.

nome	tipo	assinatura	o que faz
spacecraft_attitude_euler_rotation	função	(I1=100.0, I3=150.0, w1_0=0.05, w3_0=0.5) -> ProblemSpec	Torque-free rigid-body spacecraft attitude dynamics (axisymmetric).
lane_emden_exact_theta	função	(xi, n) -> np.ndarray	Closed-form Lane-Emden solution for n in {0, 1, 5} (Chandrasekhar, 1939). All three verified this session with sympy: substituted into theta'' + (2/xi)theta' + theta^n = 0 and confirmed exact zero residual. Raises for any other n (no closed form exists in general; must be solved numerically, e.g. ...
lane_emden_polytrope	função	(n=1.0, xi_min=0.001, xi_max=3.0) -> ProblemSpec	Lane-Emden equation for a self-gravitating polytropic star.
nfw_potential_exact	função	(r, G, rho_s, rs) -> np.ndarray	Closed-form NFW gravitational potential (Navarro, Frenk & White, 1996/1997). Verified this session with sympy: the spherical Laplacian of this expression equals 4*pi*G*rho_NFW(r) exactly (checked symbolically, zero residual) for the NFW density profile rho(r) = rho_s / [(r/rs)(1+r/rs)^2].
nfw_source_fn	função	(G, rho_s, rs)	Return a ctx["source_fn"] callable for the "poisson" PDE kind: f(X, ctx) = 4*pi*G*rho_NFW(|X|), matching the density profile whose potential is nfw_potential_exact (both verified together, see nfw_potential_exact's docstring).
nfw_dark_matter_potential	função	(G=1.0, rho_s=1.0, rs=1.0, r_max=10.0) -> ProblemSpec	Gravitational potential of a Navarro-Frenk-White (NFW) dark-matter halo -- Poisson's equation with the NFW density profile as source.
sod_exact_solution	função	(x, t, gamma=1.4, rho_l=1.0, u_l=0.0, p_l=1.0, rho_r=0.125, u_r=0.0, p_r=0.1, x0=0.5) -> …)	Exact Riemann solution for the Sod shock tube (Sod, 1978; algorithm per Toro, "Riemann Solvers and Numerical Methods for Fluid Dynamics", 3rd ed., Ch. 4). Returns rho, u, p at positions x and time t.
sod_shock_tube_astro	função	(gamma=1.4, x0=0.5, t_end=0.2) -> ProblemSpec	1D compressible Euler shock tube (Sod, 1978) -- the standard validation case for every astrophysical hydrodynamics code (used to verify e.g. FLASH, Athena++, RAMSES, Enzo on release). Represents an idealized discontinuity (e.g. a contact between two interstellar- medium phases, or the initial ...
cr3bp_omega_gradient	função	(x, y, mu)	Gradient of the CR3BP effective (synodic-frame) potential Omega(x,y) = 0.5*(x^2+y^2) + (1-mu)/r1 + mu/r2, i.e. the RHS of the equations of motion x'' - 2y' = dOmega/dx, y'' + 2x' = dOmega/dy (Szebehely, "Theory of Orbits", 1967, Ch. 1-2). A pure-numpy reference implementation, independent of the ...
cr3bp_collinear_lagrange_point	função	(mu, x0, tol=1e-14, max_iter=100) -> float	Solve for a collinear (L1/L2/L3) Lagrange point via 1D Newton- Raphson on dOmega/dx=0 restricted to y=0 (same algorithm style as this module's _kepler_solve_E: a plain, dependency-free Newton-Raphson, not scipy). x0 must be a starting guess in the correct branch (see cr3bp_lagrange_points for ...
cr3bp_lagrange_points	função	(mu) -> Dict[str, tuple]	All 5 Lagrange-point equilibria of the planar CR3BP for a given mass ratio mu. L4/L5 are the EXACT equilateral-triangle points (x=1/2-mu, y=+-sqrt(3)/2) -- true for any mu, verified symbolically with sympy this session (dOmega/dx and dOmega/dy both reduce to exactly 0 there, since r1=r2=1 by ...

nome	tipo	assinatura	o que faz
`cr3bp_planar_synodic`	função	`(mu=0.012150585609624, dx0=0.0001, dy0=0.0, n_periods=4.0) -> ProblemSpec`	Planar circular restricted three-body problem (CR3BP), Earth-Moon system, synodic (co-rotating) frame -- Szebehely's standard dimensionless normalization: unit distance = Earth-Moon separation (~384,400 km), unit time such that G(m_Earth+m_Moon)=1 and the frame's angular velocity = 1 (so 1 time ...
`schwarzschild_light_bending_u_exact`	função	`(phi, m, b) -> np.ndarray`	First-order (weak-field, m/b << 1) perturbative solution of the EXACT Schwarzschild null-geodesic equation d^2u/dphi^2 + u = 3 m u^2 (u := 1/r, m := GM/c^2), expressed in terms of the asymptotic impact parameter b: u(phi) = cos(phi)/b + (m/b^2)*(1 + sin(phi)^2)
`schwarzschild_deflection_angle_weak_field`	função	`(GM, c, b) -> float`	Einstein's weak-field light-deflection formula, delta_phi = 4GM/(c^2 b) (Einstein, 1916; famously confirmed observationally for starlight grazing the Sun by Dyson, Eddington & Davidson's 1919 eclipse expedition -- twice the Newtonian/light-corpuscle deflection of 2GM/(c^2 b)).
`schwarzschild_light_bending_weak_field`	função	`(GM=1.327124400018e+20, c=299792458.0, b=695700000.0) -> ProblemSpec`	Light deflection by a spherical mass in the Schwarzschild weak-field limit -- the null-geodesic ("photon orbit") equation, u := 1/r as a function of the orbital angle phi:
`shakura_sunyaev_flux_exact`	função	`(r, GM, Mdot, R_in) -> np.ndarray`	Closed-form radial flux profile of a steady, geometrically-thin, optically-thick alpha-disk (Shakura & Sunyaev, 1973, A&A, 24, 337), F(r) := sigma_SB * T_eff(r)^4 (so T_eff(r) = (F(r)/sigma_SB)^(1/4)):
`shakura_sunyaev_teff_exact`	função	`(r, GM, Mdot, R_in, sigma_SB=5.670374419e-08) -> np.ndarray`	Effective temperature profile T_eff(r) = (F(r)/sigma_SB)^(1/4), the classic Shakura-Sunyaev (1973) alpha-disk result. Clamps F to >= 0 before the 1/4 power (r <= R_in gives F=0, i.e. no disk material interior to the zero-torque truncation radius).
`shakura_sunyaev_accretion_disk`	função	`(M_bh_solar=10.0, Mdot_edd_frac=0.1, eta=0.1, r_out_factor=10000.0) -> ProblemSpec`	Steady, geometrically-thin, optically-thick alpha-disk (Shakura & Sunyaev, 1973, A&A, 24, 337) around a compact object -- the canonical accretion-disk model underlying essentially every X-ray-binary and AGN accretion-disk spectral calculation since the 1970s.

`pinneapple_physics.pde_environment.presets.astropy_bridge`

Astropy-backed unit handling and coordinate transforms for astrophysics.

`pinneapple_physics/pde_environment/presets/astropy_bridge.py` · 296 linhas · testes: 22

nome	tipo	assinatura	o que faz
`astro_constant_si`	função	`(name) -> float`	Return the SI-unit float value of `ASTRO_CONSTANTS[name]`.
`constant_provenance`	função	`(name) -> str`	Return the real astropy literature reference string for a constant.
`with_units`	função	`(value, unit_str=None) -> 'u.Quantity'`	Attach real astropy units to a bare number, or parse a unit string.

nome	tipo	assinatura	o que faz
to_si	função	(quantity, target_unit=None) -> float	Convert an astropy Quantity (or a parsable string like "5 Msun") to a plain float, in SI units by default or in target_unit if given.
transform_coordinates	função	(x, y, z, obstime, frame_in='gcrs', frame_out='itrs', vx=None, vy=None, vz=None, unit='km…)	Transform a Cartesian position (and optional velocity) between GCRS and ITRS using real astropy.coordinates machinery.

pinneapple_physics.pde_environment.presets.cfd

Módulo sem docstring e sem texto.

pinneapple_physics/pde_environment/presets/cfd.py · 459 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ns_incompressible_2d_default	função	(Re=100.0, Umax=1.0) -> ProblemSpec	
ns_incompressible_3d_default	função	(Re=100.0, Umax=1.0) -> ProblemSpec	
lid_driven_cavity_3d	função	(Re=100.0, size=1.0, lid_velocity=1.0) -> ProblemSpec	3D Lid-Driven Cavity benchmark.
channel_flow_3d	função	(Re=100.0, length=2.0, height=1.0, width=1.0, Umax=1.0) -> ProblemSpec	3D pressure-driven rectangular channel (Poiseuille) flow.
pipe_flow_3d	função	(Re=100.0, radius=0.5, length=2.0, Umax=1.0) -> ProblemSpec	3D Hagen-Poiseuille pipe flow in a circular cylinder.

pinneapple_physics.pde_environment.presets.electromagnetics

Electromagnetic presets: each has a closed-form solution (pp.solve(problem, "analytic"), registered from :mod:pinneapple_physics.closed_form.electromagnetics), so any method can be scored against the exact field.

pinneapple_physics/pde_environment/presets/electromagnetics.py · 184 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
em_two_dielectric_capacitor	função	(V=1.0, eps1=1.0, eps2=4.0, d1=0.5, h=0.02) -> ProblemSpec	
em_two_dielectric_capacitor_differential	função	(V=1.0, eps1=1.0, eps2=4.0, d1=0.5) -> ProblemSpec	
em_wire_az	função	(a=0.25, J=1.0, nu=1.0) -> ProblemSpec	
em_tm_cavity	função	(m=1, n=1, periods=0.5) -> ProblemSpec	
em_tm_cavity_integral	função	(m=1, n=1, periods=0.5, h=0.02) -> ProblemSpec	
em_skin_effect	função	(delta=0.2, omega=1.0, nu=1.0, length=1.0) -> ProblemSpec	
em_cylinder_scattering	função	(k0=2.0 * math.pi, radius=0.25, eps_cyl=4.0, eps_cyl_i=0.0, half_width=1.0) -> ProblemSpec	

pinneapple_physics.pde_environment.presets.engineering

Engineering problem presets.

pinneapple_physics/pde_environment/presets/engineering.py · 1.169 linhas · testes: 7

nome	tipo	assinatura	o que faz
rocket_nozzle_cfd	função	(gamma=1.4, R_gas=287.0, T_inlet=3500.0, p_inlet=10000000.0, throat_radius=0.1, exit_radi…)	2D axisymmetric compressible flow through a convergent-divergent rocket nozzle.
rocket_structural	função	(E=200000000000.0, nu=0.3, p_internal=10000000.0, T_inner=800.0, T_outer=293.0, alpha_T=1…)	Rocket motor casing under combined internal pressure + thermal gradient.
aircraft_wing_aerodynamics	função	(Re=5000000.0, Ma=0.3, alpha_deg=5.0, chord=1.0, rho_inf=1.225, U_inf=102.0, nu_air=1.5e-…)	2D incompressible/low-Ma airfoil aerodynamics (NACA-like profile).
aircraft_wing_structural	função	(E=70000000000.0, nu=0.33, lift_load=50000.0, span=5.0, thickness=0.1) -> ProblemSpec	2D plane stress model of an aircraft wing spar under lift bending load.
car_external_aero	função	(U_inf=33.3, rho=1.225, nu_air=1.5e-05, car_length=4.5, car_height=1.4) -> ProblemSpec	2D external aerodynamics around a simplified car body.
car_brake_thermal	função	(k_disc=55.0, rho_disc=7100.0, cp_disc=500.0, q_friction=2000000.0, T_ambient=293.0, h_co…)	Transient heat conduction in a brake disc during emergency braking.
car_suspension_fatigue	função	(E=210000000000.0, nu=0.3, F_vertical=8000.0, F_lateral=3000.0, arm_length=0.35, arm_widt…)	Plane stress fatigue analysis of a suspension wishbone arm.
cpu_heatsink_thermal	função	(k_aluminium=205.0, k_copper=400.0, q_cpu=200.0, T_ambient=293.0, h_fin=50.0, die_area=0.…)	3D steady-state heat conduction/convection in a CPU heatsink.
pcb_thermal	função	(k_pcb=0.3, k_z_pcb=0.25, q_components=None, T_ambient=308.0, h_natural=10.0, board_lengt…)	2D PCB heat spreading model with multiple component heat sources.
fan_cooler_cfd	função	(rpm=2000.0, blade_radius=0.05, hub_radius=0.01, rho=1.225, nu_air=1.5e-05, n_blades=7) -…)	2D radial fan flow in a PC cooler.
industrial_furnace_thermal	função	(T_hot_gas=1600.0, T_ambient=300.0, k_refractory=1.5, k_insulation=0.1, h_hot_gas=50.0, h…)	3D steady-state heat transfer in an industrial furnace wall.
refractory_lining	função	(layers=None, T_hot=1700.0, T_cold=350.0, total_thickness=0.5) -> ProblemSpec	Multi-layer refractory wall with different conductivities.
furnace_combustion_zone	função	(T_flame=2200.0, T_wall=1600.0, rho_flue=0.24, nu_flue=0.0002, U_inlet=5.0, furnace_lengt…)	Simplified combustion zone: hot gas flow with volumetric heat release.
datacenter_airflow_2d	função	(U_cold_aisle=2.5, T_cold_air=291.0, T_hot_max=318.0, Q_rack=20000.0, rack_height=2.0, ra…)	2D server rack row cooling (cold-aisle/hot-aisle containment).
datacenter_server_thermal	função	(Q_cpu=250.0, Q_gpu=400.0, Q_ram=20.0, T_inlet_air=295.0, U_fan=3.0, h_forced=120.0, k_bo…)	2D server board conduction + forced convection from fans.

nome	tipo	assinatura	o que faz
datacenter_cfd_3d	função	(room_length=20.0, room_width=10.0, room_height=3.0, n_racks=20, Q_per_rack=15000.0, T_su…)	Simplified 3D datacenter room CFD for hot-spot prediction.

pinneapple_physics.pde_environment.presets.industry

Módulo sem docstring e sem texto.

pinneapple_physics/pde_environment/presets/industry.py · 347 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
steady_heat_conduction_3d_default	função	() -> ProblemSpec	
transient_heat_3d_default	função	() -> ProblemSpec	
linear_elasticity_3d_default	função	() -> ProblemSpec	
darcy_pressure_only_3d_default	função	() -> ProblemSpec	
helmholtz_acoustics_3d_default	função	() -> ProblemSpec	
wave_ultrasound_3d_default	função	() -> ProblemSpec	
reaction_diffusion_2d_default	função	() -> ProblemSpec	

pinneapple_physics.pde_environment.presets.multidisciplinary

Multidisciplinary problem presets.

pinneapple_physics/pde_environment/presets/multidisciplinary.py · 1.100 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
climate_atmosphere_2d	função	(g=9.81, f=0.0001, H0=1000.0, U_ref=10.0, lon_min=0.0, lon_max=360.0, lat_min=-90.0, lat_…)	2D shallow water equations for atmospheric dynamics.
climate_ocean_gyre	função	(beta=2e-11, tau0=0.1, r=1e-07, rho0=1025.0, H=500.0, L=1000000.0, W=500000.0) -> Problem…)	Ocean gyre circulation — Stommel beta-plane model.
crystal_phonon	função	(k=150.0, tau=1e-12, vg=3000.0, T_hot=320.0, T_cold=280.0, L_domain=1e-06) -> ProblemSpec	Phonon transport / lattice dynamics in 1D — simplified Boltzmann transport.
material_fracture_2d	função	(E=210000000000.0, nu=0.3, Gc=2700.0, l0=0.005, Lx=1.0, Ly=1.0) -> ProblemSpec	Phase-field fracture model 2D — Bourdin-Francfort-Marigo formulation.
black_scholes_1d	função	(sigma=0.2, r=0.05, K=100.0, T=1.0) -> ProblemSpec	Black-Scholes PDE for European call option pricing.
heston_pde_2d	função	(kappa=2.0, theta=0.04, sigma_v=0.3, rho=-0.7, r=0.05, K=100.0, T=1.0, S_max=300.0, v_max…)	Heston stochastic volatility PDE for European option pricing (2D).
pk_two_compartment	função	(k12=0.5, k21=0.3, kel=0.2, V1=10.0, dose=500.0, t_end=24.0) -> ProblemSpec	Two-compartment pharmacokinetic (PK) model — ODE system.
drug_diffusion_tissue	função	(D=1e-10, lam=5e-05, C0=1.0, Lx=0.01, Ly=0.005, t_end=3600.0) -> ProblemSpec	Drug diffusion and first-order metabolism in 2D tissue.
sir_epidemic	função	(beta=0.3, gamma=0.1, N=1000000.0, I0=100.0, t_end=200.0) -> ProblemSpec	SIR epidemic model — ODE system.

nome	tipo	assinatura	o que faz
`opinion_dynamics_2d`	função	`(D=0.01, alpha=1.0, Lx=1.0, Ly=1.0, t_end=10.0) -> ProblemSpec`	2D spatial opinion dynamics PDE — continuum Hegselmann-Krause model.

`pinneapple_physics.pde_environment.presets.registry`

Registry mapping problem preset names to factory functions.

`pinneapple_physics/pde_environment/presets/registry.py` · 106 linhas · testes: 119

nome	tipo	assinatura	o que faz
`register_preset`	função	`(name)`	Decorator to register a preset factory function by name.
`get_preset`	função	`(name, **kwargs) -> ProblemSpec`	Instantiate a problem preset by name.
`list_presets`	função	`() -> List[str]`	Return sorted list of all registered preset names.

`pinneapple_physics.pde_environment.presets.solid_mechanics`

Solid mechanics presets — elasticity, axisymmetric, contact problems.

`pinneapple_physics/pde_environment/presets/solid_mechanics.py` · 669 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`axisymmetric_linear_elasticity_2d_default`	função	`(r_min=10.0, r_max=50.0, z_min=0.0, z_max=100.0, E=210000000000.0, nu=0.3, p_inner=0.0, p…)`	2D axisymmetric linear elasticity in (r, z) coordinates.
`thick_walled_cylinder_lame_default`	função	`(a=20.0, b=60.0, L=100.0, p_a=100000000.0, p_b=0.0, E=210000000000.0, nu=0.3) -> ProblemS…)`	Thick-walled hollow cylinder under internal/external pressure (Lamé problem).
`lame_analytical`	função	`(r, a, b, p_a, p_b, E, nu) -> dict`	Lamé analytical solution for thick-walled cylinder.
`threaded_coupling_tc50_box_default`	função	`(clearance=0.1, thread_height=0.8, offset=1.0, E=210000000000.0, nu=0.3, traction_top=100…)`	2D axisymmetric FEM preset for TC50 threaded coupling BOX body.
`threaded_coupling_tc50_rotating_default`	função	`(body='BOX', E=210000000000.0, nu=0.3, p_inner=20000000.0, F_axial=500000.0, T_torque=400…)`	TC50 threaded coupling under combined loading.
`threaded_coupling_tc50_pin_default`	função	`(thread_height=0.8, E=210000000000.0, nu=0.3, traction_top=100000000.0) -> ProblemSpec`	TC50 threaded coupling PIN body — same physics as BOX, different geometry bounds.

`pinneapple_physics.pde_environment.presets.structural`

Structural mechanics problem presets.

`pinneapple_physics/pde_environment/presets/structural.py` · 429 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plane_stress_2d_default`	função	`(E=210000000000.0, nu=0.3, load_x=0.0, load_y=-1000.0) -> ProblemSpec`	2D plane stress linear elasticity.
`plane_strain_2d_default`	função	`(E=210000000000.0, nu=0.3) -> ProblemSpec`	2D plane strain linear elasticity (deep structures, dams, tunnels).
`von_mises_2d_default`	função	`(E=210000000000.0, nu=0.3) -> ProblemSpec`	Plane stress with von Mises yield criterion as extra output field.
`linear_elasticity_3d`	função	`(E=210000000000.0, nu=0.3) -> ProblemSpec`	3D linear elasticity for brackets, frames, and general structures.
`rotary_coupling_torsion_default`	função	`(E=210000000000.0, nu=0.3, r_inner=0.05, r_outer=0.08, torque=10000.0, axial_force=500000…)`	Rotary coupling under combined torsion and axial tension.
`thermoelasticity_2d_default`	função	`(E=70000000000.0, nu=0.33, alpha_T=2.3e-05, dT=100.0) -> ProblemSpec`	2D thermoelastic coupling: thermal expansion due to temperature change.

`pinneapple_physics.pde_environment.presets.tag_geometry`

Real-geometry fixtures for `selector_type="tag"` presets.

`pinneapple_physics/pde_environment/presets/tag_geometry.py` · 816 linhas · testes: 9

nome	tipo	assinatura	o que faz
`build_tag_batch`	função	`(name, spec, *, n_col=20000, n_bc_per_face=3000, seed=7, user_ctx=None)`	Build a real-geometry batch for preset `name` using its :data:`TAG_GEOMETRY_FIXTURES` entry. Raises `KeyError` if `name` has no fixture (i.e. it is one of the `NOT_FIXABLE_WITHOUT_REAL_ GEOMETRY` presets, or an untagged preset that needs no fixture at all).
`tag_solve_kwargs`	função	`(name, spec, *, n_bc_per_face=1000, seed=7) -> Dict[str, Any]`	The extra `solve_pde` keyword arguments preset `name` needs: `{}` when `spec` has no `selector_type="tag"` condition, the :func:`build_tag_batch` geometry when it has a fixture here, and `KeyError` otherwise (the honest failure: no geometry is invented)::
`solve_pde_kwargs_from_batch`	função	`(batch) -> Dict[str, Any]`	Strip the `x_col` key (`solve_pde` samples its own collocation points from `spec.domain_bounds` and doesn't accept one) and convert numpy arrays to torch tensors, ready to splat into `solve_pde(spec, model, **solve_pde_kwargs_from_batch(batch))`.
`per_tag_losses`	função	`(spec, model, batch, *, w_pde=1.0, w_bc=10.0) -> dict[str, float]`	Every loss term of `compile_problem(spec)` for `model` on a :func:`build_tag_batch` batch, as floats: `total`, `pde` and one entry per boundary condition (`bc_<name>`), so the per-tag masking can be seen to do distinct work. Numpy arrays of the batch are converted to torch tensors …
`preset_name`	função	`(spec) -> str`	The registry key this spec came from: `problem_id`, the `meta["preset"]` stamp :func:`~pinneapple_physics.pde_environment.presets.registry.get_preset` writes, or `""`.
`has_default_geometry`	função	`(spec) -> bool`	True when this spec's tag conditions can be satisfied by its own fixture.
`default_geometry`	função	`(spec, *, n_bc_per_face=1000, seed=7) -> Dict[str, Any]`	The `solve_pde` keyword arguments that satisfy this spec's tag conditions with the preset's own box or cylinder geometry, or `{}` when the spec has no fixture.

nome	tipo	assinatura	o que faz
`default_geometry_report`	função	`(spec) -> str`	One line for a card or a log: which geometry a preset uses for its tagged boundaries.

pinneapple_physics.pde_environment.presets.terramechanics

Terramechanics physics residuals and preset for Bekker-Wong PINN surrogate.

`pinneapple_physics/pde_environment/presets/terramechanics.py` · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TerramechanicsResiduals`	classe	`(c_Pa=1400.0, phi_deg=30.0, R_m=0.125, b_m=0.06, n_phys=256, R_factor=0.06, slip_range=(0…)`	Physics residuals for the Bekker-Wong PINN surrogate.
`bekker_wong_surrogate_2d`	função	`(c_Pa=1400.0, phi_deg=30.0, R_m=0.125, b_m=0.06) -> ProblemSpec`	Return a ProblemSpec for the 2-D Bekker-Wong PINN surrogate.

pinneapple_physics.pde_environment.presets.turbomachinery

Turbomachinery problem presets.

`pinneapple_physics/pde_environment/presets/turbomachinery.py` · 480 linhas · testes: 7

nome	tipo	assinatura	o que faz
`axial_compressor_meanline`	função	`(num_stages=5, pressure_ratio=3.0, mass_flow_rate=4.37, rpm=10000.0, inlet_total_pressure…)`	1D mean-line thermodynamic analysis of a multi-stage axial compressor.
`axial_compressor_cascade_2d`	função	`(inlet_mach=0.5, flow_angle_in_deg=50.0, flow_angle_out_deg=20.0, chord=0.1, pitch_to_cho…)`	2D compressible Euler flow through an axial compressor blade cascade.
`axial_compressor_stage_3d`	função	`(rpm=10000.0, pressure_ratio_stage=1.4, mass_flow_rate=4.37, hub_radius=0.1, tip_radius=0…)`	3D single compressor stage in cylindrical coordinates (r, theta, z).

pinneapple_physics.pde_environment.scales

Módulo sem docstring e sem texto.

`pinneapple_physics/pde_environment/scales.py` · 18 linhas · testes: 69

nome	tipo	assinatura	o que faz
`ScaleSpec`	classe	`(L, U, alpha)`	Optional nondimensionalization / scaling parameters. Keep minimal; advanced scaling can be built later.

pinneapple_physics.pde_environment.spec

Módulo sem docstring e sem texto.

`pinneapple_physics/pde_environment/spec.py` · 51 linhas · testes: 154

nome	tipo	assinatura	o que faz
`PDETermSpec`	classe	`(kind, fields, coords, params, meta)`	PDE descriptor used by the compiler.
`ProblemSpec`	classe	`(name, dim, coords, fields, pde, conditions, sample_defaults, scales, field_ranges, refer…)`	Complete problem specification.

pinneapple_physics.pde_environment.turbulence_presets

RANS/LES turbulence closure presets for PINN training.

`pinneapple_physics/pde_environment/turbulence_presets.py` · 992 linhas · testes: 49

nome	tipo	assinatura	o que faz
`KOmegaSSTResiduals`	classe	`(nu=1e-05, rho=1.0, consts=None) -> None`	k-omega SST two-equation turbulence model residuals for PINN.
`SpalartAllmarasResiduals`	classe	`(nu=1e-05) -> None`	Spalart-Allmaras one-equation turbulence model residuals.
`WALEResiduals`	classe	`(nu=1e-05, cw=0.325, delta=1.0, spatial_indices=(0, 1, 2), t_index=3) -> None`	WALE (Wall-Adapting Local Eddy-viscosity) subgrid closure for filtered incompressible Navier-Stokes -- Nicoud & Ducros (1999) -- as an algebraic (no transport equation) 3D LES residual.
`get_rans_preset`	função	`(model_name='k-omega-sst', **kwargs)`	Get a RANS/LES turbulence closure residual object by name.

pinneapple_physics.pde_environment.turbulence_selector

Unified turbulence-model selector.

`pinneapple_physics/pde_environment/turbulence_selector.py` · 171 linhas · testes: 26

nome	tipo	assinatura	o que faz
`TurbulenceModel`	classe	`()`	Turbulence closure selector.
`get_turbulence_closure`	função	`(model, dim=3, solver_family='pinn', **kwargs) -> Optional[Any]`	Return the right turbulence closure object/parameter for a solver family.

pinneapple_physics.physical_problem

`PhysicalProblem`: one description of a physics problem for every part of the library.

`pinneapple_physics/physical_problem.py` · 534 linhas · testes: 12

nome	tipo	assinatura	o que faz
`Parameter`	classe	`(name, value, unit, role, bounds, distribution, description)`	A named physical parameter.

nome	tipo	assinatura	o que faz
`Quantity`	classe	`(name, unit, field, description)`	A quantity of interest: what the user actually wants out of the solution (a field, a coefficient, a peak value). `field` names the model output it derives from, when there is one.
`PhysicalProblem`	classe	`(name, coords, fields, pde, conditions, domain_bounds, geometry, geometry_kind, parameter…)`	A complete physics problem: geometry, physics, conditions, parameters, solver choice and intent.

`pinneapple_physics.physics_audit`

Physics audit: which model to use, when to trust it and when its physical formulation itself must be revisited.

`pinneapple_physics/physics_audit/__init__.py` · 171 linhas · testes: 37

nome	tipo	assinatura	o que faz
`Diagnosis`	classe	`(model, features, audit, validity, attribution, recommendations, unrepresented)`	
`diagnose`	função	`(profile, features, prediction=None, query=None, monitor=None, error=None, error_toleranc…)`	Audit the assumptions of `profile` for the case's `features`, run `monitor` on `prediction` (if both given), attribute the failure (`error`: relative error against a reference, if any; other evidence through `attribution_kwargs`: `resolution`, `measurement_suspect`, `spread`, …

`pinneapple_physics.physics_audit.assumptions`

Physics assumptions of a model, the conditions under which each holds, and an auditor that checks them case by case.

`pinneapple_physics/physics_audit/assumptions.py` · 420 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Condition`	classe	`(feature, lo, hi, band, log)`	`lo <= features[feature] <= hi`. `band`: width of the soft edge, in decades when `log` (default for positive bounds), in feature units otherwise. Inside but within `band` of an edge is marginal; outside by more than `band` is violated. :meth:`score` is 1 inside and decays as exp(-(d / …
`Assumption`	classe	`(name, description, conditions, breaks_into, missing_mechanism)`	A simplification a model rests on. `breaks_into`: the phenomenon that appears when it fails (turbulence, compressibility…); `missing_mechanism`: what the model then lacks (turbulent transport, density variation…).
`AssumptionCheck`	classe	`(name, status, score, evidence, breaks_into, missing_mechanism, features)`	

nome	tipo	assinatura	o que faz
Envelope	classe	(ranges, samples, sample_features, band)	Where a model was trained or validated. ranges: {feature: (lo, hi)}; samples: optional (n, d) training points over sample_features for a nearest-neighbour distance (in units of the median neighbour spacing of the training set, on standardised features, log10 for positive features).
ModelProfile	classe	(name, family, equations, assumptions, mechanisms, envelope, cost, notes)	What the audit knows about a model: family ("PINN", "FNO", "solver"...), governing equations, assumptions (names from :data:ASSUMPTIONS or :class:Assumption objects), mechanisms it represents ("turbulent transport"...), training envelope and relative cost.
AssumptionAudit	classe	(model, status, compatibility, checks, envelope, triggered, missing_mechanisms)	
laminar	função	(re_crit=2300.0, feature='Re', geometry='pipe') -> Assumption	Laminar flow below the transition Reynolds number (2300 in a pipe; about 5e5 on a flat plate, by x).
turbulent	função	(re_min=4000.0, re_max=math.inf, feature='Re') -> Assumption	Fully turbulent flow (wall functions, turbulent correlations such as Blasius or Colebrook). The phenomenon named is the one below re_min; state an upper limit of a correlation through the training envelope or a separate assumption (:func:hydraulically_smooth).
hydraulically_smooth	função	(eps_plus_max=5.0) -> Assumption	Smooth wall: roughness buried in the viscous sublayer, roughness Reynolds number eps+ = Re (eps / D) sqrt(f / 8) below about 5 (see :func:~.regime.roughness_reynolds).
incompressible	função	(ma_max=0.3) -> Assumption	
creeping_flow	função	(re_max=1.0) -> Assumption	
continuum	função	(kn_max=0.01) -> Assumption	
boussinesq	função	(max_beta_dt=0.1) -> Assumption	
constant_properties	função	(max_dt_over_t=0.1) -> Assumption	
small_strain	função	(max_strain=0.01) -> Assumption	
linear_elastic	função	(max_stress_ratio=1.0) -> Assumption	
lumped_capacitance	função	(bi_max=0.1) -> Assumption	
forced_convection	função	(ri_max=0.1) -> Assumption	

nome	tipo	assinatura	o que faz
`diffusion_dominated`	função	`(pe_max=1.0) -> Assumption`	
`advection_dominated`	função	`(pe_min=10.0) -> Assumption`	
`attached_flow`	função	`(alpha_stall_deg=12.0) -> Assumption`	
`shallow_water`	função	`(max_aspect=0.05) -> Assumption`	
`get_assumption`	função	`(a) -> Assumption`	
`audit_assumptions`	função	`(profile, features) -> AssumptionAudit`	Check every assumption of `profile` and its training envelope for one case's `features`.

`pinneapple_physics.physics_audit.attribution`

Physics failure attribution: not only *that* a prediction is wrong (or suspect), but its probable cause, with the evidence and a confidence.

`pinneapple_physics/physics_audit/attribution.py` · 246 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Cause`	classe	`(code, title, probability, score, evidence)`	
`Attribution`	classe	`(failed, causes, confidence, evidence_available, error, notes)`	
`attribute_failure`	função	`(*, validity=None, audit=None, error=None, error_tolerance=0.05, residual_baseline=None, …)`	Weigh the evidence of one case.

`pinneapple_physics.physics_audit.improvement`

Physics-guided model improvement: when no model is good enough, which experiment or change is worth most?

`pinneapple_physics/physics_audit/improvement.py` · 179 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Recommendation`	classe	`(action_id, action, cause, priority, gain, cost, rationale, details, cases)`	
`design_points`	função	`(features, spread=0.15, n=5, seed=0, keys=None) -> list[dict[str, float]]`	High-fidelity design points at and around a case: the case itself, then `n - 1` points spread by `spread` (in decades for positive features) on a Latin hypercube.

nome	tipo	assinatura	o que faz
recommend	função	(attribution, audit=None, graph=None, features=None, model=None, min_probability=0.1) -> …)	Ranked actions for one case. With graph, model and features the "switch formulation" and "add term" actions name the mechanism the model lacks and the models or equations that represent it; with features the high-fidelity actions list design points.
improvement_plan	função	(cases, top=None) -> list[Recommendation]	Aggregate the recommendations of many cases: the same action summed over the cases that call for it (each case's priority), so frequent, confident causes rank first. Items are attributions (recommended with :func:recommend without context) or (attribution, recommendations) pairs.

pinneapple_physics.physics_audit.knowledge

Physics knowledge graph: phenomena, equations, assumptions, parameters, mechanisms and model families, and the relations between them.

pinneapple_physics/physics_audit/knowledge.py · 328 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Node	classe	(id, kind, label, data)	
Edge	classe	(src, rel, dst, data)	
PhysicsKnowledgeGraph	classe	()	
default_graph	função	(models=()) -> PhysicsKnowledgeGraph	The catalogue of assumptions, phenomena and equations above, plus models.

pinneapple_physics.physics_audit.monitor

Physics validity monitor: is a prediction still physically plausible, when no ground truth is available?

pinneapple_physics/physics_audit/monitor.py · 260 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CheckResult	classe	(name, kind, value, tolerance, passed, critical, detail)	
ValidityCheck	classe	(name, kind, fn, tolerance, critical, describe)	fn(prediction, query) -> value; passes when value <= tolerance. critical: a single failure is an alarm (default for bounds: a negative density is never acceptable).
ValidityReport	classe	(status, results, alarms, index)	
PhysicsValidityMonitor	classe	(checks, persistence=3, drift_ratio=3.0, drift_alpha=0.3, baseline_cases=10, baselines=No…)	Run a set of checks on every prediction and keep their history.

nome	tipo	assinatura	o que faz
residual_check	função	(residual_fn, tolerance, name='pde_residual', critical=False) -> ValidityCheck	residual_fn(prediction, query): a residual norm (RMS of the PDE residual, for instance).
conservation_check	função	(total_fn, target_fn, rel_tol=0.01, name='conservation', critical=False, abs_tol=0.0) -> …)	Relative imbalance |total(prediction) - target(query)| / |target(query)|. total_fn is typically a quadrature (np.sum(cell_volumes * rho)); target_fn the initial total plus sources minus outflow. abs_tol: an absolute imbalance that is always accepted (needed when the target is …
bounds_check	função	(lo=None, hi=None, name='bounds', select=None, max_fraction=0.0, critical=True) -> Validi…)	Fraction of values outside [lo, hi] (a field selected by select, all of it by default).
boundary_check	função	(predict_boundary, boundary_value, rel_tol=0.01, name='boundary_condition', critical=Fals…)	Relative RMS mismatch between the prediction on the boundary and the boundary condition: RMS(a - b) divided by scale if given, else by the larger of the RMS of the boundary condition and the RMS of the whole prediction. (Dividing by the RMS of the boundary prediction itself would make a …
constitutive_check	função	(fn, tolerance, name='constitutive', critical=False) -> ValidityCheck	Any relation the prediction must satisfy (equation of state, material law): fn returns its violation.
invariant_check	função	(invariant_fn, expected, rel_tol=0.01, name='invariant', critical=False, abs_tol=0.0) -> …)	Relative change of an invariant (energy of a conservative system, a symmetry measure). abs_tol as in :func:conservation_check, for an invariant whose expected value is zero (an antisymmetry measure).
envelope_check	função	(envelope, features_fn, min_score=0.5, name='training_envelope') -> ValidityCheck	Extrapolation: 1 - envelope score (0 inside the training ranges, -> 1 far outside).

pinneapple_physics.physics_audit.regime

Dimensionless numbers that tell which physics a case is in.

pinneapple_physics/physics_audit/regime.py · 178 linhas · testes: 31

nome	tipo	assinatura	o que faz
reynolds	função	(velocity, length, nu=None, *, rho=None, mu=None) -> float	Re = U L / nu (or rho U L / mu).
mach	função	(velocity, sound_speed=None, *, gamma=1.4, R=287.05, T=None) -> float	Ma = U / a, with a given or a = sqrt(gamma R T) for an ideal gas.
peclet	função	(velocity, length, diffusivity) -> float	Pe = U L / D (mass) or U L / alpha (heat).

nome	tipo	assinatura	o que faz
knudsen	função	`(mean_free_path, length) -> float`	
grashof	função	`(g, beta, dT, length, nu) -> float`	
prandtl	função	`(nu, alpha) -> float`	
rayleigh	função	`(g, beta, dT, length, nu, alpha) -> float`	
richardson	função	`(g, beta, dT, length, velocity) -> float`	Ri = Gr / Re^2 = g beta dT L / U^2.
biot	função	`(h, length, k) -> float`	
froude	função	`(velocity, length, g=9.81) -> float`	
strouhal	função	`(frequency, length, velocity) -> float`	
dominant_frequency	função	`(signal, dt=1.0, discard=0.5, window=True, refine=True, pad=1) -> float`	Frequency (1 / time unit of `dt`) of the strongest spectral peak of a sampled signal, e.g. the vortex-shedding frequency of a wake probe or a lift history, for :func:`strouhal`.
damkohler	função	`(reaction_rate, length, velocity) -> float`	
roughness_reynolds	função	`(re, eps_d, f=None) -> float`	eps+ = Re (eps / D) sqrt(f / 8): roughness height in wall units. `f` (Darcy) defaults to the Haaland estimate, so no solution is needed.
regime_features	função	`(*, velocity=None, length=None, nu=None, sound_speed=None, diffusivity=None, alpha=None, …)`	Every dimensionless number whose inputs are given (SI units). `extra` adds features computed elsewhere (`strain`, `stress_ratio`, `alpha_deg`...).

pinneapple_physics.physics_audit.routing

Physics-aware model routing: weight each model by how well the *current* case fits its physical foundations, not only by the errors it made on earlier cases.

`pinneapple_physics/physics_audit/routing.py` · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
RouteDecision	classe	`(chosen, weights, compatibility, awake, escalate, audits, reasons)`	
PhysicsAwareRouter	classe	`(profiles, features_fn=None, strength=1.0, floor=0.0001, abstain_below=0.05, escalate_bel…)`	`profiles`: one :class:`~.assumptions.ModelProfile` per model (same names as the ensemble's experts; a model without a profile gets compatibility 1). `features_fn(query) -> {feature: value}` (identity if the queries are already feature dictionaries).

pinneapple_physics.pinn_solver

Módulo sem docstring e sem texto.

`pinneapple_physics/pinn_solver/__init__.py` · 28 linhas · testes: 1

pinneapple_physics.pinn_solver.compiler

Módulo sem docstring e sem texto.

`pinneapple_physics/pinn_solver/compiler/__init__.py` · 11 linhas · testes: 5

pinneapple_physics.pinn_solver.compiler.autograd_ops

Módulo sem docstring e sem texto.

`pinneapple_physics/pinn_solver/compiler/autograd_ops.py` · 153 linhas · testes: 8

nome	tipo	assinatura	o que faz
`derivative_cache`	função	`() -> Iterator[Dict[str, int]]`	Memoize first derivatives within the block (one loss evaluation). Yields hit/miss counters.
`ensure_tensor`	função	`(y)`	
`grad`	função	`(y, x) -> torch.Tensor`	
`jacobian`	função	`(Y, x) -> torch.Tensor`	
`divergence`	função	`(V, x, coord_indices=None) -> torch.Tensor`	Divergence sum_i dV_i/dx_{idx_i} of a vector field V.
`laplacian`	função	`(y, x, coord_indices=None) -> torch.Tensor`	Sum of unmixed second derivatives ∂²y/∂x_i².
`time_derivative`	função	`(y, x, t_index) -> torch.Tensor`	
`norm_dot_grad`	função	`(y, x, normals) -> torch.Tensor`	
`mse`	função	`(a, b) -> torch.Tensor`	

pinneapple_physics.pinn_solver.compiler.compile

Módulo sem docstring e sem texto.

`pinneapple_physics/pinn_solver/compiler/compile.py` · 2.604 linhas · testes: 121

nome	tipo	assinatura	o que faz
`compile_problem`	função	`(spec, *, weights=None, cache_derivatives=False)`	Compile `spec` into `loss_fn(model, y_hat, batch) -> {name: loss}`.

pinneapple_physics.pinn_solver.compiler.dataset

Módulo sem docstring e sem texto.

`pinneapple_physics/pinn_solver/compiler/dataset.py` · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SingleBatchDataset`	classe	`(batch)`	Wraps a prebuilt dict-batch (e.g. from STLDomainBatchBuilder) into a Dataset.

nome	tipo	assinatura	o que faz
`dict_collate`	função	`(batch) -> Dict[str, Any]`	Collate list of PINN dict-batches.

`pinneapple_physics.pinn_solver.compiler.em_kinds`

Electromagnetic PDE kinds of the problem compiler: Maxwell's equations in differential and integral form.

`pinneapple_physics/pinn_solver/compiler/em_kinds.py` · 261 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`em_residuals`	função	`(kind, *, fields, xcol, coords, params, ctx, model, field_names, split_fields) -> list[to…)`	

`pinneapple_physics.pinn_solver.compiler.loss`

Módulo sem docstring e sem texto.

`pinneapple_physics/pinn_solver/compiler/loss.py` · 107 linhas · testes: 20

nome	tipo	assinatura	o que faz
`LossWeights`	classe	`(w_pde, w_bc, w_ic, w_data)`	
`AdaptiveWeights`	classe	`(base_weights, momentum, eps, max_ratio, enabled, _ema)`	Self-normalising weights for a multi-term PINN loss, keyed by arbitrary term names (unlike :class:`LossWeights`'s four fixed `w_pde`/`w_bc`/`w_ic`/`w_data` fields, which `compile_problem` reads directly by attribute and is unaffected by this addition -- this class is a standalone utility ...

`pinneapple_physics.pinn_solver.domino`

DoMINO: Domain Decomposition for Physics-Informed Neural Networks.

`pinneapple_physics/pinn_solver/domino.py` · 385 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Subdomain`	classe	`(bounds, overlap=0.1)`	Rectangular subdomain with overlap.
`SubdomainPINN`	classe	`(subdomain, model)`	PINN for a single subdomain.
`DoMINO`	classe	`(subdomains, model_factory, interface_weight=10.0)`	Domain-decomposition PINN trainer.

`pinneapple_physics.pinn_solver.factory`

Módulo sem docstring e sem texto.

`pinneapple_physics/pinn_solver/factory/__init__.py` · 23 linhas · testes: 7

`pinneapple_physics.pinn_solver.factory.autodiff`

Autograd-based derivative computation for PINN residual evaluation.

`pinneapple_physics/pinn_solver/factory/autodiff.py` · 166 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DerivativeKey`	classe	`(base, wrt)`	A stable key for caching derivative computations.
`DerivativeComputer`	classe	`(independent_vars, dependent_vars, device, coord_scales=None) -> None`	Computes and caches torch autograd derivatives required by SymPy Derivative atoms.
`ensure_requires_grad`	função	`(inputs) -> None`	Ensures all tensors in `inputs` have requires_grad=True (in-place).

`pinneapple_physics.pinn_solver.factory.pinn_factory`

PINN model factory: compiles symbolic equations and builds unified loss function.

`pinneapple_physics/pinn_solver/factory/pinn_factory.py` · 412 linhas · testes: 1

nome	tipo	assinatura	o que faz
`NeuralNetwork`	classe	`(num_inputs, num_outputs, num_layers, num_neurons, activation=nn.Tanh())`	Backwards-compatible MLP using the `(num_inputs, num_outputs, num_layers, num_neurons)` API.
`PINN`	classe	`(neural_network, inverse_params_names=None, initial_guesses=None, dtype=torch.float32)`	Backwards-compatible wrapper: `NeuralNetwork` + trainable inverse parameters.
`PINNProblemSpec`	classe	`(pde_residuals, independent_vars, dependent_vars, conditions, inverse_params, loss_weight…)`	A model-agnostic PINN specification.
`PINNFactory`	classe	`(spec)`	Factory that compiles symbolic equations (SymPy) into torch-callable residuals, then creates a unified loss function.

`pinneapple_physics.pinn_solver.factory.signals`

Exogenous signal providers for `PINNFactory`: known inputs such as sensor data or forcing.

`pinneapple_physics/pinn_solver/factory/signals.py` · 62 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TabulatedSignal`	classe	`(x, values, *, input_index=0, extrapolate='clamp')`	Piecewise-linear interpolation of a sampled signal, evaluated in torch.

`pinneapple_physics.pinn_solver.factory.sympy_backend`

SymPy-based compiler for symbolic equations to torch-callable lambdas.

`pinneapple_physics/pinn_solver/factory/sympy_backend.py` · 243 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CompiledEquation`	classe	`(expr, derivatives, call, args_order, exogenous, runtime_constants)`	A compiled symbolic equation.

nome	tipo	assinatura	o que faz
`SympyTorchCompiler`	classe	`(independent_vars, dependent_vars, inverse_params=None, extra_namespace=None)`	Compiles equation strings into torch-callable lambdas using SymPy.
`SympyBackend`	classe	`(*, extra_namespace=None) -> None`	Compatibility wrapper.

`pinneapple_physics.pinn_solver.io`

Módulo sem docstring e sem texto.

`pinneapple_physics/pinn_solver/io/__init__.py` · 28 linhas · **sem teste direto**

`pinneapple_physics.pinn_solver.io.mappings`

Mappings between UPD coordinates/variables and PINN independent/dependent vars.

`pinneapple_physics/pinn_solver/io/mappings.py` · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CoordMapping`	classe	`(ind_vars, coord_sources, coord_transform, normalize_to_unit, _deriv_scale)`	Mapping from UPD coords -> PINN independent variables.
`VarMapping`	classe	`(dep_vars, upd_to_dep, enforce_present)`	Mapping between UPD variable names and PINN dependent variable names.
`PINNMapping`	classe	`(coord, var)`	Full mapping: coordinates -> independent vars, variables -> dependent vars.
`build_default_mapping_atmosphere`	função	`(dep_vars, ind_vars=None, use_lev=False, time_transform='seconds_since_start', angle_tran…)`	Default mapping for UPD atmosphere reanalysis shards: coords: time, lat, lon, optionally lev independent vars: - default: ["t","lat","lon"] or ["t","lev","lat","lon"] transforms: - t: seconds_since_start + minmax to [-1,1] - lat/lon: deg2rad + minmax to [-1,1]

`pinneapple_physics.pinn_solver.io.upd_dataset`

UPD dataset loader and PINN-ready sampling (collocation, conditions, data).

`pinneapple_physics/pinn_solver/io/upd_dataset.py` · 253 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UPDDataset`	classe	`(item, mapping, device='cpu', dtype=torch.float32)`	Reads UPD (Zarr + JSON) and produces PINN-ready samples: - collocation points (inputs only) - condition points (inputs only; one list entry per condition) - optional supervised data pairs from UPD variables

`pinneapple_physics.pinn_solver.schemas`

Módulo sem docstring e sem texto.

`pinneapple_physics/pinn_solver/schemas/__init__.py` · 1 linhas · **sem teste direto**

pinneapple_physics.pinn_solver.schemas.schema_adapter

Adapter from Physical Schema (UPD) to PINN problem and sampling specs.

`pinneapple_physics/pinn_solver/schemas/schema_adapter.py` · 325 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SchemaAdapterOptions`	classe	`(default_loss_weights, default_condition_weight, build_sampling_from_conditions)`	Options controlling how a Physical Schema is converted into a PINN problem.
`SchemaAdapter`	classe	`(mapping, options=None) -> None`	Converts a Physical Schema (UPD/Pinneapple) into: - PINNProblemSpec (for PINNFactory) - Optional: SamplingSpec/ConditionSpec (for UPDDataset)

pinneapple_physics.pinn_solver.stochastic

Latent-conditioned (stochastic) PINN training utilities.

`pinneapple_physics/pinn_solver/stochastic.py` · 378 linhas · testes: 15

nome	tipo	assinatura	o que faz
`LatentConditionedModel`	classe	`(base, latent_dim) -> None`	Wrap any `model(x) -> y` so it additionally accepts a latent code `xi`, concatenated onto `x` before the wrapped model's own forward.
`CorrelatedLatentField`	classe	`(bounds, coord_order, spatial_axes, periodic_axes, latent_dim, grid_res=(64, 24, 32), len…)`	Spatially-correlated replacement for per-point :func:`sample_latent`.
`sample_latent`	função	`(n, latent_dim, device=None, dtype=torch.float32) -> torch.Tensor`	`N(0, I)` latent codes, shape `(n, latent_dim)`.
`ensemble_forward`	função	`(model, x, latent_dim, n_samples, latent_sampler=None) -> torch.Tensor`	Evaluate `n_samples` latent draws at every point in `x` (N, D) in a single batched forward pass. Returns `(N, n_samples, out_dim)`.
`mean_covariance_loss`	função	`(model, x, latent_dim, n_samples, mean_target, cov_target=None, cov_index_pairs=None, fie…)`	Monte-Carlo mean and (optionally) covariance of the model's own latent ensemble at `x`, matched (MSE) against externally supplied real statistics.

pinneapple_physics.solid

Solid mechanics post-processing (strains, stresses, equilibrium residuals, von Mises) by autograd.

`pinneapple_physics/solid/__init__.py` · 13 linhas · **sem teste direto**

pinneapple_physics.solid.axisymmetric

Axisymmetric linear elasticity in cylindrical coordinates `(r, z)`, by autograd.

`pinneapple_physics/solid/axisymmetric.py` · 122 linhas · testes: 8

nome	tipo	assinatura	o que faz
lame_parameters	função	(E, nu) -> tuple[float, float]	(lambda, mu) from Young's modulus E and Poisson's ratio nu (isotropic).
strains_rz	função	(u, x, *, r_min=None, create_graph=True) -> dict[str, torch.Tensor]	Small strains e_rr, e_tt, e_zz, e_rz (and e_rt, e_tz when u has u_theta), each (N, 1).
stresses_rz	função	(u, x, E, nu, *, r_min=None, create_graph=True) -> dict[str, torch.Tensor]	Stresses s_rr, s_tt, s_zz, s_rz (and s_rt, s_tz with torsion), each (N, 1), by Hooke's law.
equilibrium_residual_rz	função	(u, x, E, nu, *, r_min=None) -> torch.Tensor	Pointwise equilibrium residual [R_r, R_z] ((N, 2)), plus R_theta ((N, 3)) with torsion.
von_mises_from_components	função	(s_rr, s_zz, s_tt, s_rz=0.0, s_rt=0.0, s_tz=0.0) -> Any	Von Mises stress sqrt(((s_rr-s_zz)^2 + (s_zz-s_tt)^2 + (s_tt-s_rr)^2 + 6 (s_rz^2 + s_rt^2 + s_tz^2)) / 2).
von_mises_rz	função	(u, x, E, nu, *, r_min=None, create_graph=False) -> torch.Tensor	Von Mises stress (N, 1) of u (all six stress components when u has u_theta).

pinneapple_physics.solving

pp.solve and pp.compare: one entry point to solve a problem by any method and compare methods.

pinneapple_physics/solving.py · 594 linhas · testes: 71

nome	tipo	assinatura	o que faz
MethodNotAvailable	classe	()	The requested method cannot solve this problem (missing solver output, missing option, ...).
Solution	classe	(problem, method, predict_fn, wall_time_s, history, info, model, grid, kind)	
Comparison	classe	(problem, reference, n_points, rows)	
as_problem	função	(problem, **preset_kwargs) -> PhysicalProblem	PhysicalProblem from a PhysicalProblem, a PDE ProblemSpec or a preset name.
register_method	função	(name, kind='custom')	Decorator registering fn(problem, **options) -> Solution under name for solve/compare. kind labels the family of the method (classical, neural, ...) in every Solution it returns.
list_methods	função	(kind=None) -> List[str]	Registered method names, optionally only those of one kind.
list_kinds	função	() -> Dict[str, List[str]]	Methods grouped by kind.
register_analytic	função	(name, matcher)	Decorator: factory(problem) -> fn(X) is the exact solution for every problem where matcher is True.
list_analytic	função	() -> List[str]	

nome	tipo	assinatura	o que faz
analytic_for	função	`(problem) -> Optional[str]`	Name of the closed-form solution this library knows for `problem`, or `None`.
solve	função	`(problem, method='pinn', *, preset_kwargs=None, **options) -> Solution`	Solve `problem` with `method` (a registered name or a callable `(problem, **options) -> Solution`).
compare	função	`(problem, methods, *, reference='reference', options=None, points=None, n_points=4096, se…)`	Solve or reuse each method and score it against `reference` at the same points.

pinneapple_physics.symbolic_pde

pinneapple_symbolic — Symbolic PDE compiler for PINNeAPPle.

`pinneapple_physics/symbolic_pde/__init__.py` · 52 linhas · **sem teste direto**

pinneapple_physics.symbolic_pde.bc

Boundary condition enforcement for PINNeAPPle symbolic PDE module.

`pinneapple_physics/symbolic_pde/bc.py` · 447 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
HardBC	classe	`(distance_fn, bc_value_fn) -> None`	Hard boundary condition enforcement via a distance-function ansatz.
PeriodicBC	classe	`(period, dim=0) -> None`	Periodic boundary condition enforcement via coordinate transformation.
MultiPeriodicBC	classe	`(periods) -> None`	Exact periodic-BC embedding for two or more coordinate axes at once.
DirichletBC	classe	`(value_fn, boundary_fn=None, weight=1.0) -> None`	Soft Dirichlet boundary condition via an MSE penalty term.
NeumannBC	classe	`(flux_fn, normal_fn, boundary_fn=None, weight=1.0) -> None`	Soft Neumann boundary condition via autograd normal derivative.

pinneapple_physics.symbolic_pde.compiler

Symbolic PDE compiler for PINNeAPPle.

`pinneapple_physics/symbolic_pde/compiler.py` · 771 linhas · testes: 5 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
SymbolicPDE	classe	`(expr, coord_syms, field_syms, param_syms=None, grad_method='autograd', grid=None) -> None`	Symbolic PDE residual — compile to a PyTorch autograd callable.
pde_from_sympy	função	`(expr, coords, fields, params=None, grad_method='autograd', grid=None) -> SymbolicPDE`	Create a SymbolicPDE from a SymPy residual expression.
auto_residual	função	`(model, coords_tensor, derivative_fns) -> Dict[str, torch.Tensor]`	Compute field values and named derivative quantities via autograd.

pinneapple_physics.symbolic_pde.gradient_backends

Pluggable derivative backends for :class:`SymbolicPDE`.

`pinneapple_physics/symbolic_pde/gradient_backends.py` · 130 linhas · testes: 5

nome	tipo	assinatura	o que faz
`GridAxisSpec`	classe	`(coord_names, dims, L)`	Maps SymPy coordinate names to axes of a structured-grid field tensor.
`grid_derivative`	função	`(field, wrt, *, grid, method) -> torch.Tensor`	Compute an arbitrary-order partial derivative of a grid-shaped field.

pinneapple_physics.symbolic_pde.pinn

A parsed equation system (:mod:`.text`) as a :class:`~pinneapple_analysis.validation.pinn.PINNProblem`.

`pinneapple_physics/symbolic_pde/pinn.py` · 194 linhas · testes: 10

nome	tipo	assinatura	o que faz
`PINNWithParameters`	classe	`()`	
`pinn_problem`	função	`(system, bounds, *, coefficients=None, unknowns=None, width=32, depth=4, exact=None, name…)`	`(PINNProblem, model)` for `system` on the box `bounds`. `unknowns`: coefficient -> initial guess, trained with the network (read back with `model.estimates()`). `exact`: optional `X -> (N, n_fields)`.
`has_field`	função	`(expr) -> bool`	

pinneapple_physics.symbolic_pde.text

Equations as text, parsed safely into SymPy: `"u_t = alpha*lap(u) + sin(pi*x)"`.

`pinneapple_physics/symbolic_pde/text.py` · 486 linhas · testes: 17

nome	tipo	assinatura	o que faz
`EquationSyntaxError`	classe	`()`	The text is not an equation this parser accepts (the message says what and where).
`Context`	classe	`(coords, fields, time)`	
`Equation`	classe	`(text, lhs, rhs, coefficients, kind, on)`	
`EquationSystem`	classe	`(ctx, pdes, conditions, coefficients, unknowns)`	
`parse_expression`	função	`(text, ctx) -> tuple[sp.Expr, set[str]]`	`text` (no =) as a SymPy expression, and the coefficient names it uses.

nome	tipo	assinatura	o que faz
parse_equation	função	(text, ctx, *, kind='pde', on='') -> Equation	
parse_system	função	(pdes, conditions=None, *, coords, fields, coefficients=None, unknowns=None, time='t') ->…)	Parse PDE texts and conditions ({"kind", "on", "equation"}) into an :class:EquationSystem. EquationSyntaxError names the first text that fails.
classify	função	(system) -> dict[str, Any]	The PDE family of a single-field system (heat, wave, poisson, laplace, helmholtz, burgers, advection_diffusion, advection, reaction_diffusion, or custom), its orders and linearity.
derivative_key	função	(dv) -> str	Environment key of a field derivative: u__xx for u_xx (letters sorted).
derivative_keys	função	(expr) -> dict[str, tuple[str, str]]	{key: (field, wrt letters)} for every field derivative in expr.
compile_expr	função	(expr, backend='numpy')	A function env -> value for expr built as a closure tree (no eval, no lambdify).

pinneapple_physics.tracking

Experiment tracking adapter: send an ExperimentResult to MLflow or Weights & Biases.

pinneapple_physics/tracking.py · 82 linhas · testes: 3

nome	tipo	assinatura	o que faz
flat_metrics	função	(result) -> Dict[str, float]	
flat_params	função	(result) -> Dict[str, str]	
log_result	função	(result, tracker, **kw) -> Any	Send result to tracker; returns the tracker's run id (or what the callable returns).

pinneapple_physics.tribology

Tribology: sliding wear of a bar against a counterface (Archard's law with contact-pressure redistribution).

pinneapple_physics/tribology/__init__.py · 5 linhas · testes: 20

pinneapple_physics.tribology.archard

Sliding wear of a bar end pressed against a rigid, flat counterface.

pinneapple_physics/tribology/archard.py · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
WearMaterial	classe	(name, k, hardness_MPa, E_GPa)	
BarWear	classe	(material, load_N, length_mm, width_mm, crown_radius_mm, layer_mm, nx)	A bar of section length (along sliding, crowned) x width, end radius R, pressed with load F.

nome	tipo	assinatura	o que faz
`winkler_paraboli c_contac t`	fu nç ão	`(F_N, width_mm, R_mm, k_w) -> dict[str, float]`	Closed form for a parabolic gap g = x² / (2R) on a Winkler layer of modulus k_w (MPa/mm), load F over a width b: p = k_w (δ - x²/2R) on |x| < a, a = sqrt(2 R δ), F = (4/3) b k_w a δ.

pinneapple_physics.weather

Módulo sem docstring e sem texto.

`pinneapple_physics/weather/__init__.py` · 1 linhas · **sem teste direto**

pinneapple_physics.weather.data

ERA5 from WeatherBench2 (public Google Cloud bucket, no account) as a local, normalised array for training and evaluating global forecast models, plus the climatology needed for anomaly scores.

`pinneapple_physics/weather/data.py` · 176 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Era5 Stor e`	cl as se	`(path)`	Normalised ERA5 written by :func:`download` (memory mapped).
`chan nel_ name s`	fu nç ão	`(levels=LEVELS) -> list[str]`	
`down load`	fu nç ão	`(dest, years=(1990, 2022), levels=LEVELS, url=ERA5_64x32, log=print, normalization=None) …)`	`normalization`: mean and std per channel to use (a `meta.json` of another store, or the file shipped next to a checkpoint) instead of computing them; a trained model only works with the normalisation it was trained with.
`clim atol ogy`	fu nç ão	`(store, url=CLIMATOLOGY_64x32 ) -> np.ndarray`	Normalised 1990-2019 climatology (dayofyear 1..366, hour 0/6/12/18, channel, lat, lon) in the store's units, cached next to the store.
`clim atol ogy_ at`	fu nç ão	`(clim, time) -> np.ndarray`	

pinneapple_physics.weather.evaluate

Skill of global forecasts by lead time, and how far ahead each forecast can be trusted.

`pinneapple_physics/weather/evaluate.py` · 154 linhas · testes: 8

nome	tipo	assinatura	o que faz
`Publis hedFor ecasts`	cla ss e	`(store, names=None)`	Published forecasts on the 64 x 32 grid (WeatherBench2), returned in the store's channel layout (other channels NaN). Only the 00 and 12 UTC initialisations exist.
`rmse`	fu nç ão	`(f, o, w) -> np.ndarray`	Area-weighted RMSE over the last two axes (lat, lon); `w` (lat,) normalised to mean 1.
`acc`	fu nç ão	`(f, o, clim, w) -> np.ndarray`	

nome	tipo	assinatura	o que faz
`first_crossing`	função	`(leads_h, values, threshold, below=True) -> float`	Lead (hours, linearly interpolated) at which `values` first goes below (or above) `threshold`; inf if it never does within the leads given.
`horizons`	função	`(leads_h, acc_curve, rmse_curve, clim_rmse) -> dict`	
`score_forecasts`	função	`(store, clim, forecasts, init_idx, steps, variables=SCORED) -> dict`	`forecasts`: {name: callable(i0, steps) -> (steps+1, channel, lat, lon) physical units or None}. Returns {name: {var: {"rmse": (n_init, steps+1), "acc": ...}}} on the given initial indices.
`persistence`	função	`(store)`	
`climatology_forecast`	função	`(store, clim)`	

pinneapple_physics.weather.events

Extreme-weather case studies and the ERA5 states around them.

`pinneapple_physics/weather/events.py` · 156 linhas · testes: 8

nome	tipo	assinatura	o que faz
`Event`	classe	`(key, title, init, peak, var, region, center, what, score, vmin, vmax, extra_inits)`	
`by_key`	função	`(key) -> Event`	
`coarsen`	função	`(field, src_lat, src_lon, lat, lon) -> np.ndarray`	Area-weighted mean of a (..., src_lat, src_lon) field in each (lat, lon) model cell.
`arco_states`	função	`(store, times, log=print) -> np.ndarray`	Normalised states (time, channel, lat, lon) on the store's grid at `times` from ARCO ERA5. The 6-hour precipitation is the sum of the six hourly accumulations ending at each time.
`event_states`	função	`(store, event, leads, cache_dir=None, log=print) -> tuple[np.ndarray, np.ndarray]`	(times, normalised states) from init - 6 h to init + leads x 6 h, from the store or from ARCO.
`region_mask`	função	`(lat, lon, box) -> np.ndarray`	

pinneapple_physics.weather.model

Global forecast network on an equiangular latitude-longitude grid.

`pinneapple_physics/weather/model.py` · 122 linhas · testes: 8

nome	tipo	assinatura	o que faz
SConv	classe	`(cin, cout, k=3)`	
Block	classe	`(cin, cout)`	Residual block: two spherical 3x3 convolutions with group norm and GELU.
SphereUNet	classe	`(n_state, n_extra, widths=(128, 192, 256, 384), blocks=2, patch=1, refine=64)`	`patch` > 1 first folds patch x patch cells into channels (pixel unshuffle), so the U-Net runs on a grid that is `patch` times coarser and the output is unfolded back; a light full-resolution layer then smooths the seams.
sphere_pad	função	`(x, p) -> torch.Tensor`	Pad (B, C, lat, lon) by `p` cells: circular in longitude, across the poles in latitude.
toa_insolation	função	`(times, lat, lon) -> np.ndarray`	Cosine of the solar zenith angle clipped at 0 (normalised top-of-atmosphere sunlight), (T, lat, lon).
static_fields	função	`(lat, lon, constants) -> np.ndarray`	

pinneapple_physics.weather.train

Training of the global forecast network: one 6-hour step first, then rollouts of growing length.

`pinneapple_physics/weather/train.py` · 198 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Forecaster	classe	`(store, model)`	A trained network with everything needed to roll it out from an ERA5 time.
train	função	`(store_path, out, *, train_years=(1990, 2017), val_years=(2018, 2019), widths=(192, 256, …)`	Train for `hours` of wall time split by `schedule` ((rollout steps, fraction of the time), ...).

pinneapple_physics.weather.viz

Forecast-versus-truth visuals in the style of global AI weather demos: dark background, the ERA5 field and the forecast on globes side by side, the lead time running, and a skill strip that shows, at each lead, whether the forecast is still good (ACC above 0.6), fading, or no better than ...

`pinneapple_physics/weather/viz.py` · 399 linhas · testes: 8

nome	tipo	assinatura	o que faz
display_units	função	`(var, x) -> np.ndarray`	
coastlines	função	`() -> list[np.ndarray]`	

nome	tipo	assinatura	o que faz
`orthographic`	função	`(field, lat, lon, center=(-15.0, -50.0), size=420)`	Image (size, size) of the field on the visible hemisphere centred at (lat, lon); NaN outside the disk.
`project_lines`	função	`(lines, center, size=420)`	
`regional`	função	`(field, lat, lon, box, size=(360, 260))`	Equirectangular image of a region `box` = (lat_s, lat_n, lon_w, lon_e), smoothly interpolated.
`regional_lines`	função	`(lines, box, size=(360, 260))`	
`forecast_gif`	função	`(path, var, truth, forecast, lat, lon, times, *, skill_leads_h, skill_acc, horizon, cente…)`	Animated GIF: ERA5 (left) and forecast (right) of `var` for each lead, and the skill strip below.
`fill_gaps`	função	`(x, y) -> np.ndarray`	Linear interpolation over NaNs (models published every 12 h on a 6-hour lead axis).
`skill_figure`	função	`(path, leads_h, curves, horizons_by_model, var, metric='acc', dpi=110)`	Skill by lead for every model (dark style), with the useful horizon of each marked.
`upsample_sphere`	função	`(field, lat, lon, factor=4)`	Cubic upsampling of a (lat, lon) field, periodic in longitude, for display only.
`earth2_gif`	função	`(path, var, truth, forecast, lat, lon, times, *, lead_hours, horizon, title, center=(20.0…)`	NVIDIA Earth-2 style animation: ERA5 (left) and the forecast (right) on lit globes over black, smooth fields (cubic upsampling on the sphere, `substeps` interpolated frames between 6-hour states), the globe slowly rotating, the lead in large type and a thin lead bar coloured by the horizon (good, …

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 114 de 166.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.075.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
E55	S-N curve + Goodman + Miner	validated	1	14
E58	Fin array conduction	tested	0	4
E63	Helmholtz resonator	tested	0	4
E64	Membrane diffusion	tested	0	5
E65	Cantilever fatigue	validated	1	4
S38	Maxwell physics checks of predicted fields (Gauss, Faraday, Ampere-Maxwell, continuity, Poynting, interfaces, reciprocity, passivity)	validated	4	2
T10	Domain decomposition: XPINN, DoMINO/D3M	tested	0	2

Limitações conhecidas

- *Nenhuma registrada.*

2. `pinneapple_presets`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
21	5.247	22	31	50 (3 arquivos)	16

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`		1	1	27
`cases`		14	20	4
`core`		1	10	0
`registry`		1	6	0
`report`		1	4	27
`surrogate`		1	6	46
`viz3d`		1	6	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_presets`

Industrial presets: engineering problems already set up, solved and validated with PINNeAPPle.

`pinneapple_presets/__init__.py` · 19 linhas · testes: 27

`pinneapple_presets.__main__`

python -m pinneapple_presets list | card NAME | run NAME [k=v ...] [--surrogate] | validate NAME | surrogate NAME | report NAME|all [--build-surrogate] [--no-3d] | index

`pinneapple_presets/__main__.py` · 83 linhas · testes: 27

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	

`pinneapple_presets.cases`

Módulo sem docstring e sem texto.

`pinneapple_presets/cases/__init__.py` · 1 linhas · **sem teste direto**

`pinneapple_presets.cases.airfoil`

Wing / blade section at Re 4e6: lift, drag and moment from the verified zoo surrogate of OpenFOAM RANS.

`pinneapple_presets/cases/airfoil.py` · 290 linhas · testes: 4

nome	tipo	assinatura	o que faz
`AirfoilSection`	classe	`()`	
`PresetCache`	classe	`()`	
`thin_airfoil_zero_lift`	função	`(code) -> float`	alpha_L0 (deg) of a NACA 4-digit camber line by thin-airfoil theory (numerical quadrature).

`pinneapple_presets.cases.eddy_current`

Eddy-current inspection of aircraft skin: probe impedance, depth of penetration and thinning sensitivity.

`pinneapple_presets/cases/eddy_current.py` · 308 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EddyCurrentInspection`	classe	`()`	

`pinneapple_presets.cases.footbridge`

Steel footbridge: crowd strength, deflection, lateral-torsional buckling and pedestrian vibration (EN 1990 A2).

`pinneapple_presets/cases/footbridge.py` · 375 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SteelFootbridge`	classe	`()`	
`section_class_ok`	função	`(h, b, tf, tw, fy) -> tuple`	EN 1993-1-1 Table 5.2, class 3 limits: web in bending c/t <= 124 eps, outstand flange c/t <= 14 eps (welded: c ignores the weld throat, conservative). Returns (ok, web c/t, web limit, flange c/t, flange limit).

pinneapple_presets.cases.heatsink

Plate-fin heat sink in ducted forced air for a power module (resistance network + 3-D finite-volume base).

`pinneapple_presets/cases/heatsink.py` · 361 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PlateFinHeatsink`	classe	`()`	

pinneapple_presets.cases.insulation

Insulation of a steam line: heat loss, surface temperature and the economic thickness (ISO 12241, VDI 2055).

`pinneapple_presets/cases/insulation.py` · 279 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SteamLineInsulation`	classe	`()`	

pinneapple_presets.cases.ipm_traction

Interior-PM traction motor (2004 Toyota Prius geometry): torque, ripple, MTPA and voltage at speed by FEA.

`pinneapple_presets/cases/ipm_traction.py` · 252 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`IPMTraction`	classe	`()`	
`machine`	função	`(n_gap) -> IPMMachine`	

pinneapple_presets.cases.pcb

Junction temperatures on a power PCB (DC-DC / motor-drive stage) by 3-D finite volumes of the stack-up.

`pinneapple_presets/cases/pcb.py` · 266 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PowerPCB`	classe	`()`	

pinneapple_presets.cases.pipe_network

Water distribution network: pressures, flows and least-cost pipe sizing (global gradient algorithm).

`pinneapple_presets/cases/pipe_network.py` · 289 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WaterNetwork`	classe	`()`	

pinneapple_presets.cases.pressure_vessel

Cylindrical pressure vessel shell and heads to ASME VIII Div. 1 (UG-27, UG-32), checked by Lamé and 3-D FEM.

pinneapple_presets/cases/pressure_vessel.py · 346 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PressureVessel	classe	()	
ring_mesh	função	(ri, ro, h, nr, nt, nz) -> HexMesh	Quarter ring r in [ri, ro], theta in [0, 90 deg], z in [0, h] as an 8-node hexahedral mesh.
lame	função	(p, ri, ro, r, E, nu) -> dict	Lamé stresses at radius r and the plane-strain radial displacement (closed ends ignored).

pinneapple_presets.cases.pump_npsh

Centrifugal pump lifting from a sump: duty point, NPSH available vs required and the cavitation margin.

pinneapple_presets/cases/pump_npsh.py · 276 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PumpNPSH	classe	()	

pinneapple_presets.cases.shell_tube

Shell-and-tube closed-loop water cooler (TEMA E, one shell pass, two tube passes): rating by epsilon-NTU.

pinneapple_presets/cases/shell_tube.py · 304 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ShellTubeCooler	classe	()	

pinneapple_presets.cases.spm_motor

Surface-PM servo motor: torque, ripple, cogging and back-EMF by 2-D nonlinear magnetostatic FEM.

pinneapple_presets/cases/spm_motor.py · 292 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SPMMotor	classe	()	

pinneapple_presets.cases.water_hammer

Water hammer from a valve closure at the end of a steel water main (method of characteristics).

pinneapple_presets/cases/water_hammer.py · 369 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
WaterHammer	classe	()	

nome	tipo	assinatura	o que faz
`rigid_column_max_head`	função	`(*, length, diameter, friction_factor, q0, h_reservoir, h_downstream, t_close, closure_ex…)`	Peak valve head of the same closure by rigid-column theory (incompressible liquid, rigid pipe; Wylie & Streeter 1993 sec. 1.3): `(L / g A) dQ/dt = H_res - h_f(Q) - H_valve`, valve `Q = tau Q0 sqrt(dH/dH0)`. Valid when the closure is much slower than 2L/a; an independent check of the MOC there.

pinneapple_presets.core

Industrial preset: an engineering problem already set up and solved with PINNeAPPle.

`pinneapple_presets/core.py` · 342 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Param`	classe	`(name, unit, default, source, lo, hi, help, choices)`	One input. `lo`..`hi` is the validity domain of the preset (solver and correlations), not a soft hint: a run outside it is flagged and the applicability check contradicts.
`Output`	classe	`(name, unit, help)`	
`Reference`	classe	`(name, params, output, value, unit, source, kind, tol)`	A citable reference value for one output at given parameters. `kind`: "analytic", "standard", "benchmark", "experiment" or "independent_solver"; `tol` is the relative error accepted for a pass.
`SurrogateSpec`	classe	`(inputs, outputs, n_train, n_test, bounds, log, log_outputs)`	Which inputs a surrogate spans (each over its Param domain unless `bounds` narrows it), which outputs it predicts, and the DOE size (Latin hypercube, solver runs) with its hold-out share.
`Result`	classe	`(preset, params, outputs, trust, decision, fidelity, warnings, meta, fields)`	Outputs of one run with its checks, decision and provenance.
`Preset`	classe	`(name, title, sector, problem, question, params, outputs, validity, pipeline, library, su…)`	Base class; a preset sets the class attributes and implements :meth:`solve`, :meth:`checks`, :meth:`decide` and :meth:`references` (and optionally :meth:`scene`, :meth:`convergence`, :meth:`figures`).
`check`	função	`(name, ok, value=None, limit=None, source='', reason='', critical=False) -> Check`	A trust-report check from a pass/fail (None = not run).
`environment`	função	`() -> dict`	Versions and hardware of this run (recorded with every result).
`lhs`	função	`(n, bounds, seed=0, log=()) -> np.ndarray`	Latin hypercube sample of `n` points in the box (`log[i]`: that axis in log space).
`call_quiet`	função	`(fn, *a, **kw)`	

pinneapple_presets.registry

Registry of the industrial presets and the index the Hub reads (`pinneapple_presets/index.json`).

pinneapple_presets/registry.py · 97 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
register	função	(cls) -> type	Class decorator: add a user preset (a :class:Preset sub-class with a unique name).
names	função	() -> list[str]	Names of every industrial preset (built in and registered), sorted.
load	função	(name) -> Preset	The preset name (see :func:names).
list_presets	função	(sector=None) -> list[dict]	Name, title, sector and capabilities of every preset (sector: substring filter).
index	função	() -> dict	Everything the Hub shows: per preset the data sheet, the measured results and the figure paths (relative to the repository root).
write_index	função	(path=INDEX_PATH) -> str	

pinneapple_presets.report

Solve, validate and document one preset: results, figures, 3-D stills, surrogate, inversion, optimisation, design variants, README (data sheet + measured results) and the Lab manifest, in examples/industrial_presets/<name>/.

pinneapple_presets/report.py · 322 linhas · testes: 27

nome	tipo	assinatura	o que faz
write_report	função	(preset, out_dir=None, *, surrogate='load', three_d=True, lab=True) -> dict	surrogate: "build" (run the DOE and retrain), "load" (use the stored model; build if missing) or "skip". Returns the summary written to results/summary.json.
compare_variants	função	(p, variants, res, root, three_d=True) -> dict	Run every variant with the same pipeline; table of the key outputs, a bar chart and a 3-D gallery.
readme	função	(p, s) -> str	
write_catalogue	função	(root=None) -> str	examples/industrial_presets/README.md: one row per preset from its stored summary (measured values only) and the folder's Lab manifest.

pinneapple_presets.surrogate

Surrogate of a preset: Latin-hypercube DOE over its parameters run with the real solver, an anisotropic (ARD) Gaussian process per output, error measured on held-out solver runs, the training box as validity domain.

pinneapple_presets/surrogate.py · 232 linhas · testes: 46

nome	tipo	assinatura	o que faz
ARDGP	classe	()	Gaussian process, squared-exponential kernel with one length scale per input, constant variance and a nugget, fitted by maximum marginal likelihood (L-BFGS on log hyper-parameters, a few restarts). Inputs are scaled to [0, 1] and the target standardised before fitting.

nome	tipo	assinatura	o que faz
`PresetSurrogate`	classe	`(meta, models, test=None)`	Trained surrogate of one preset: one ARD GP per output on unit-scaled inputs.
`surrogate_path`	função	`(name) -> str`	
`load_surrogate`	função	`(name) -> PresetSurrogate | None`	
`build_surrogate`	função	`(preset, *, seed=0, save=True, tol=0.05, n_train=None, n_test=None) -> PresetSurrogate`	Run the DOE with the real solver, fit, measure the hold-out error and (`save`) store the model.
`serve_spec`	função	`(name) -> dict | None`	What a model server needs to expose the surrogate: inputs with bounds and units, outputs, hold-out error and the Python call.

`pinneapple_presets.viz3d`

3-D pictures of preset results on the library's studio scene.

`pinneapple_presets/viz3d.py` · 144 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`box`	função	`(lo, hi, n=(1, 1, 1))`	Closed axis-aligned box from `lo` to `hi` with `n` cells per side on each face; returns (V, F).
`tube`	função	`(path, radius, n_around=16)`	Open tube of `radius` (scalar or one per path point) along the polyline `path` (N x 3); returns (V, F, s) with `s` the index of the path point of each vertex (to put per-point values on the surface).
`extrude`	função	`(section, x)`	Prism of the closed 2-D polygon `section` (M x 2, in the y-z plane, counter-clockwise) along the stations `x`; returns (V, F, i) with `i` the station index of each vertex. Ends are capped with a fan (convex or star-shaped sections).
`merge`	função	`(parts)`	Concatenate (V, F[, values...]) meshes into one; returns the same tuple shape.
`stills`	função	`(scene, out_dir, name, modes=(), size=(1100, 700), views=None, frame=None) -> dict[str, s…)`	Write `out_dir/<name>_viewer.html` (self-contained, regenerated by the scripts, not committed) and one PNG per field in `modes` ("real" for the materials). Returns {mode: png path or None if no browser}.
`field_on`	função	`(idx, values) -> np.ndarray`	Per-vertex values from per-station values (`idx` from :func:`tube` / :func:`extrude`).

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 5 de 21.
- Funções de teste em arquivos que importam algum módulo do pacote: 50.

Limitações conhecidas

- *Nenhuma registrada.*

3. `pinneapple_problemdesign`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
28	5.642	40	21	123 (9 arquivos)	19

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`agent`		1	1	0
`autonomy`		1	5	41
`cli`		1	1	0
`codegen`		1	1	0
`elicitation`		4	3	0
`extractor`		1	1	0
`knowledge`		4	7	22
`merge`		1	1	0
`method_selection`		1	3	11
`policy`		1	1	0
`preset_authoring`		1	5	15
`protocol`		1	4	20
`providers`		2	3	0
`renderers`		3	2	0
`research_loop`		1	3	9
`schema`		1	16	70
`state`		1	2	0
`unified_agent`		1	2	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_problemdesign`

Módulo sem docstring e sem texto.

`pinneapple_problemdesign/__init__.py` · 62 linhas · testes: 58

pinneapple_problemdesign.agent

Design agent for interactive problem elicitation via LLM.

`pinneapple_problemdesign/agent.py` · 194 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DesignAgent`	classe	`(llm, use_orchestrator_bridge=True)`	High-level flow: - ingest user message - structured extraction (json_mode=True) - safe merge into ProblemSpec - non-invention policy => unknown -> Gap, assumptions -> Assumption - stage-based gap generation - once sufficient => build plan (FNO-first) and render report

pinneapple_problemdesign.autonomy

Autonomy safety-boundary gate for the autonomous physics research loop.

`pinneapple_problemdesign/autonomy.py` · 481 linhas · testes: 41

nome	tipo	assinatura	o que faz
`AutonomyLevel`	classe	`()`	How much of the research loop's step-by-step decision-making runs without a human in the loop.
`ConsequentialActionClassifier`	classe	`(*, name_patterns=None, category_patterns=None, tag_patterns=None, exceptions=None) -> No…)`	Explicit, inspectable (no ML/fuzzy matching) classifier for whether a tool call is "consequential": crosses outside pure sandboxed simulation/computation/validation because it costs real money, invokes a licensed external tool per-run, or would affect something outside this sandbox (an external …
`ApprovalRequest`	classe	`(tool_name, tool_args, reason, autonomy_level, is_consequential)`	Everything a human approver needs to decide whether a gated action should proceed. Passed to `approver_fn` by :class:`ApprovalGate`.
`ApprovalDecision`	classe	`(approved, reason, requires_human)`	The outcome of :meth:`ApprovalGate.check`.
`ApprovalGate`	classe	`(autonomy_level, approver_fn=None, iteration_report_every=5, confidence_pause_threshold=N…)`	The gate itself: decides, per tool call, whether it may proceed autonomously or needs a human's explicit sign-off.

pinneapple_problemdesign.cli

CLI for interactive Pinneapple Design agent.

`pinneapple_problemdesign/cli.py` · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`()`	

pinneapple_problemdesign.codegen

Code generation: translate elicited ProblemSpec → concrete Pinneapple API objects.

`pinneapple_problemdesign/codegen.py` · 374 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_pineapple_spec`	função	`(design_spec) -> PinneappleSpec`	Translate an elicited :class:`~pinneapple_problemdesign.schema.ProblemSpec` into a :class:`~pinneapple_problemdesign.schema.PinneappleSpec`.

`pinneapple_problemdesign.elicitation`

Módulo sem docstring e sem texto.

`pinneapple_problemdesign/elicitation/__init__.py` · 6 linhas · **sem teste direto**

`pinneapple_problemdesign.elicitation.questions`

Stage-specific gap questions for problem elicitation.

`pinneapple_problemdesign/elicitation/questions.py` · 101 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_stage_gaps`	função	`(stage, spec_dict) -> List[Gap]`	

`pinneapple_problemdesign.elicitation.stages`

Elicitation stage definitions and STAGES_ORDER.

`pinneapple_problemdesign/elicitation/stages.py` · 76 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Stage`	classe	`(name, description, required_fields)`	

`pinneapple_problemdesign.elicitation.validators`

Validation and suggestion for ProblemSpec (warnings, assumptions).

`pinneapple_problemdesign/elicitation/validators.py` · 42 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`validate_and_suggest`	função	`(spec) -> Tuple[List[str], List[Assumption]]`	

`pinneapple_problemdesign.extractor`

JSON extraction from user text for requirements analysis.

`pinneapple_problemdesign/extractor.py` · 140 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`extract_from_text`	função	`(llm, user_text) -> Tuple[Dict[str, Any], Dict[str, Any]]`	

`pinneapple_problemdesign.knowledge`

Módulo sem docstring e sem texto.

`pinneapple_problemdesign/knowledge/__init__.py` · 10 linhas · **sem teste direto**

pinneapple_problemdesign.knowledge.mapping

Plan builder: dispatches to FNO-first or PINN-first based on task_type.

`pinneapple_problemdesign/knowledge/mapping.py` · 483 linhas · testes: 22

nome	tipo	assinatura	o que faz
`available_orchestrator_tools`	função	`(spec) -> List[Dict[str, object]]`	Read-only lookup of real, currently-registered tools from `pinneapple_worldmodel.orchestrator.PhysicsOrchestrator`'s `PhysicsToolRegistry` that are relevant to *spec*.
`build_plan`	função	`(spec, gaps, use_orchestrator_bridge=True) -> Plan`	Dispatch to the appropriate plan builder based on spec.task_type.
`build_plan_pinn_first`	função	`(spec, gaps) -> Plan`	Plan for PDE-solution and inverse-problem tasks via PINNFactory.
`build_plan_fno_first`	função	`(spec, gaps) -> Plan`	
`build_plan_cfd_first`	função	`(spec, gaps) -> Plan`	Plan for external-aerodynamics/CFD/fluid-dynamics problems, using PINNeAPPle's Lattice-Boltzmann solver, RANS/LES turbulence-closure presets, vortex-identification post-processing, and (optionally) the OpenFOAM bridge, instead of defaulting to generic PINN/FNO advice.

pinneapple_problemdesign.knowledge.physics_knowledge

Phenomenon -> governing-equation -> typical-parameters -> assumptions knowledge base for problem design.

`pinneapple_problemdesign/knowledge/physics_knowledge.py` · 615 linhas · testes: 6

nome	tipo	assinatura	o que faz
`PhenomenonEntry`	classe	`(phenomenon, governing_equation, typical_parameters, assumptions, preset_module, preset_f…)`	One physics-knowledge record, traceable back to a real preset.
`lookup_phenomenon`	função	`(name_or_keyword) -> List[PhenomenonEntry]`	Look up knowledge-base entries by a case-insensitive keyword.

pinneapple_problemdesign.knowledge.pinneapple_capabilities

Mapping of Pinneapple module capabilities for plan generation.

`pinneapple_problemdesign/knowledge/pinneapple_capabilities.py` · 41 linhas · **sem teste direto**

pinneapple_problemdesign.merge

Deep merge of extracted spec into existing ProblemSpec.

`pinneapple_problemdesign/merge.py` · 33 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`merge_into_spec`	função	`(spec, partial) -> None`	

pinneapple_problemdesign.method_selection

Recommend a numerical method + turbulence closure from a computed flow regime.

`pinneapple_problemdesign/method_selection.py` · 346 linhas · testes: 11 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`MethodRecommendation`	classe	`(numerical_method, turbulence_model, flow_regime, rationale, confidence)`	A recommended (numerical_method, turbulence_model) pair for a flow.
`recommend_method`	função	`(velocity, length_scale, kinematic_viscosity, *, geometry='internal_pipe', compressible=F…)`	Recommend a numerical solver + turbulence closure for a flow.
`recommend_method_from_spec`	função	`(spec) -> Optional[MethodRecommendation]`	Convenience wrapper: pull physical parameters out of a `ProblemSpec` and call :func:`recommend_method`, or return `None` if they aren't present.

pinneapple_problemdesign.policy

Non-invention policy: apply assumptions and gaps to design state.

`pinneapple_problemdesign/policy.py` · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`apply_non_invention_policy`	função	`(state, *, unknown_fields, assumptions_suggested, gaps_suggested) -> None`	

pinneapple_problemdesign.preset_authoring

LLM-assisted preset *drafting*, gated by a real, automatically-generated self-consistency check -- not a preset *authoring pipeline* in the sense of producing something trusted for production use.

`pinneapple_problemdesign/preset_authoring.py` · 707 linhas · testes: 15

nome	tipo	assinatura	o que faz
`PresetDraftError`	classe	`(stage, message, *, raw='') -> None`	Raised when the LLM's proposal cannot be turned into a real, checkable PDE.
`ManufacturedSolutionCheck`	classe	`(exact_solution_used, residual_at_exact_solution, tolerance, passed, n_points, notes)`	One method-of-manufactured-solutions self-consistency check.
`DraftedPreset`	classe	`(phenomenon_description, proposed_equation_latex_or_sympy, sympy_pde, proposed_bcs, propo…)`	An LLM-proposed PDE preset, together with the real verification that was run against it. See module docstring for the safety model.

nome	tipo	assinatura	o que faz
`run_manufactured_solution_check`	função	`(sympy_pde, coord_syms, field_syms, *, param_syms=None, params=None, tolerance=0.0001, n_…)`	Run a real method-of-manufactured-solutions self-consistency check against `sympy_pde`, the same technique as `tests/test_manufactured_solutions.py`: compile `sympy_pde` with the real `pinneapple_physics.symbolic_pde` machinery, substitute a simple closed-form exact solution for every field …
`draft_preset`	função	`(phenomenon_description, *, llm_provider='anthropic', model=None, use_knowledge_base=True…)`	Ask an LLM to draft a candidate governing PDE for `phenomenon_description`, then run a real manufactured-solution self-consistency check against it.

`pinneapple_problemdesign.protocol`

LLMProvider protocol, LLMMessage, LLMResponse, GeminiProvider.

`pinneapple_problemdesign/protocol.py` · 100 linhas · testes: 20

nome	tipo	assinatura	o que faz
`LLMMessage`	classe	`(role, content)`	
`LLMResponse`	classe	`(text, raw)`	
`LLMProvider`	classe	`()`	Single interface for the module.
`GeminiProvider`	classe	`(model='gemini-1.5-pro')`	Gemini provider using Google Generative AI SDK.

`pinneapple_problemdesign.providers`

Módulo sem docstring e sem texto.

`pinneapple_problemdesign/providers/__init__.py` · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LLMProvider`	classe	`()`	Provider interface used by pinneapple_problemdesign.

`pinneapple_problemdesign.providers.cosmos_provider`

Cosmos LLM provider for HTTP gateway access.

`pinneapple_problemdesign/providers/cosmos_provider.py` · 187 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CosmosProviderConfig`	classe	`(base_url, api_key, timeout_s, route)`	Cosmos provider configuration.
`CosmosProvider`	classe	`(base_url=None, *, api_key=None, timeout_s=None, route=None, session=None)`	Cosmos-backed LLM provider (via your gateway).

`pinneapple_problemdesign.renderers`

Módulo sem docstring e sem texto.

pinneapple_problemdesign/renderers/__init__.py · 5 linhas · **sem teste direto**

pinneapple_problemdesign.renderers.report_json

JSON renderer for DesignReport.

pinneapple_problemdesign/renderers/report_json.py · 35 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
render_json_report	função	(report, *, indent=2) -> str	

pinneapple_problemdesign.renderers.report_md

Markdown renderer for DesignReport.

pinneapple_problemdesign/renderers/report_md.py · 133 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
render_markdown_report	função	(report, warnings) -> str	

pinneapple_problemdesign.research_loop

AutonomousResearchAgent: the autonomous research cycle on top of UnifiedPhysicsAgent.

pinneapple_problemdesign/research_loop.py · 423 linhas · testes: 9

nome	tipo	assinatura	o que faz
IterationRecord	classe	(iteration, problem_description, approval, agent_result, guardrail_report, confidence, no…)	What happened during one iteration of the loop -- kept in full so a human reviewing a run afterwards can see every decision, not just the final outcome.
ResearchLoopResult	classe	(status, iterations, final_confidence, final_result, memory_record_ids, prior_lessons)	The full, inspectable outcome of an :meth:AutonomousResearchAgent.run call.
AutonomousResearchAgent	classe	(llm, *, autonomy_level=AutonomyLevel.SUPERVISED, approver_fn=None, memory=None, physics_…)	The Phase B research loop: hypothesize (revise description) -> simulate (UnifiedPhysicsAgent) -> validate (PhysicsGuardrail / PhysicsConfidenceScore) -> decide (stop or revise) -> repeat, behind ApprovalGate's non-negotiable safety floor, with ExperimentMemory learning across …

pinneapple_problemdesign.schema

Problem design schema: Assumption, Risk, DataSpec, Gap, ProblemSpec.

pinneapple_problemdesign/schema.py · 290 linhas · testes: 70

nome	tipo	assinatura	o que faz
Assumption	classe	(text, confidence, needs_confirmation)	

nome	tipo	assinatura	o que faz
Risk	classe	(text, level, mitigation)	
DataSpec	classe	(sources, format, sampling, variables_observed, target_variables, known_quality_issues, m…)	
PhysicsSpec	classe	(governing_equations, boundary_conditions, initial_conditions, constraints, parameters_kn…)	
GeometrySpec	classe	(domain, representation, sensors, coordinate_system, cad_format, voxel_resolution, aoa_sw…)	
ValidationSpec	classe	(primary_metrics, acceptance_criteria, robustness_tests, ood_scenarios)	
DeploymentSpec	classe	(environment, latency_budget_ms, update_policy, monitoring)	
ConstraintsSpec	classe	(hardware, max_training_time, interpretability, compliance)	
ProblemSpec	classe	(title, goal, task_type, inputs, outputs, horizon, input_window, frequency, domain_contex…)	
Gap	classe	(id, question, severity, rationale, how_to_obtain, resolved)	
PlanStep	classe	(title, why, actions, pinneapple_modules, exit_criteria)	
Plan	classe	(recommended_approach, alternatives, steps, go_no_go)	
PinneappleSpec	classe	(pde_kind, pde_confidence, coords, fields, pde_params, environment_kwargs, model_name, mo…)	Concrete Pinneapple API objects derived from the elicited design spec.
DesignReport	classe	(spec, gaps, plan, pinneapple_spec, created_at)	
uses_pinn_approach	função	(spec) -> bool	Whether spec should be solved with a PDE-residual PINN pipeline.

nome	tipo	assinatura	o que faz
uses_cfd_approach	função	(spec) -> bool	Whether spec describes an external-aerodynamics/CFD problem that should get the LBM/turbulence-closure/vortex-postprocessing plan (knowledge.mapping.build_plan_cfd_first) instead of generic PINN/FNO advice.

pinneapple_problemdesign.state

DesignState, Turn for problem design conversation state.

pinneapple_problemdesign/state.py · 33 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Turn	classe	(role, content)	
DesignState	classe	(spec, gaps, plan, history, stage, done)	

pinneapple_problemdesign.unified_agent

Compose DesignAgent and PhysicsOrchestrator into one working loop.

pinneapple_problemdesign/unified_agent.py · 572 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
UnifiedAgentResult	classe	(status, design_report, problem_statement, execution_result, design_state, gaps, question…)	Bundled result of :meth:UnifiedPhysicsAgent.run.
UnifiedPhysicsAgent	classe	(llm, *, use_orchestrator_bridge=True, orchestrator=None, orchestrator_verbose=True) -> N…)	Compose a real DesignAgent with a real PhysicsOrchestrator.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 9 de 28.
- Funções de teste em arquivos que importam algum módulo do pacote: 123.

Limitações conhecidas

- *Nenhuma registrada.*

4. pinneapple_science

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
16	4.253	33	49	19 (1 arquivos)	10

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`		1	1	0
`atlas`		1	4	0
`core`		1	7	19
`em`		7	34	19
`instruments`		1	10	19
`numerics`		1	8	19
`plots`		1	7	0
`registry`		1	5	0
`report`		1	6	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_science`

PINNeAPPle science (`pp.science`): experiments by area of science, at academic and research level, theory and practice (a virtual bench), in 0-D (lumped), 1-D, 2-D, 3-D and 3-D + time; and the atlas of every area.

`pinneapple_science/__init__.py` · 25 linhas · testes: 19

`pinneapple_science.__main__`

`python -m pinneapple_science atlas [--html PATH] | list [--area A] [--dim D] | run NAME [--dim D] [--quick] | report NAME|all [--quick] [--no-register] | table`.

`pinneapple_science/__main__.py` · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	

`pinneapple_science.atlas`

The atlas of science areas: what :mod:`pinneapple_science` covers, what the rest of the library already covers, and what is planned.

`pinneapple_science/atlas.py` · 273 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Area`	classe	`(id, title, branch, status, description, subareas)`	
`Atlas`	classe	`(schema, version, statuses, dims, areas)`	
`load`	função	`(path=ATLAS_PATH) -> Atlas`	The atlas (`pp.science.atlas()`): every area and subarea with its status and pointers, from the versioned data file `pinneapple_science/data/atlas.json`.

nome	tipo	assinatura	o que faz
validate	função	(atlas=None, *, check_presets=True, check_lab=True) -> list[str]	Problems with the atlas (empty when it is consistent): unknown statuses, pointers to modules that do not exist, presets or Lab experiments that are not registered, pp.science experiments that are not registered or not listed, and available subareas without an experiment.

pinneapple_science.core

The common shape of a science experiment: what it declares, what every run returns.

pinneapple_science/core.py · 348 linhas · testes: 19 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Reference	classe	(value, source, kind, unit)	A reference value and where it comes from. kind is one of closed_form (a formula, cited), published (a tabulated value from a paper or book), codata (a fundamental constant), limit (an asymptotic limit), conservation (a quantity that must be conserved), convergence (the …
Comparison	classe	(quantity, computed, reference, rel_error, tol, passed, kind, source, unit, how, abs_erro…)	
Result	classe	(experiment, title, area, topic, level, kind, dim, params, values, comparisons, checks, f…)	What a run returns. Build it through the recording methods (value, compare, check, figure, data); trust is assembled from the comparisons and checks when the run ends.
Experiment	classe	(name, title, area, topic, level, kind, dims, description, references, params, quick)	Base class. Subclasses set the class attributes and implement _run(res, dim, p), which fills the :class:Result res for dimension dim with parameters p (defaults merged with the caller's).
jsonable	função	(v) -> Any	Plain JSON types (numpy scalars and arrays to Python numbers and lists; NaN and inf to None).
observed_order	função	(errors, ratio=2.0) -> list[float]	Observed orders log(e_i / e_{i+1}) / log(ratio) of a refinement ladder.
central_derivative	função	(f, x, h) -> float	

pinneapple_science.em

Electromagnetism experiments of :mod:pinneapple_science, built on pp.em (:mod:pinneapple_simulation.electromagnetics) and pp.maxwell (:mod:pinneapple_physics.maxwell_checks).

`pinneapple_science/em/__init__.py` · 3 linhas · **sem teste direto**

`pinneapple_science.em.bench`

Bench experiments (practice): a virtual laboratory with instruments that add noise, calibration error and resolution, a fit with its uncertainty, and the comparison with the value the bench was built with.

`pinneapple_science/em/bench.py` · 220 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Mu0Solenoid`	classe	`()`	
`RCDischarge`	classe	`()`	

`pinneapple_science.em.fdtd_study`

A configurable 3-D + time FDTD study on top of :mod:`pinneapple_simulation.electromagnetics.fdtd`: a domain in wavelengths, sources (Hertzian dipoles, soft E sources) and materials (spheres, boxes, cylinders with any :class:`~pinneapple_simulation.electromagnetics.media.Medium`) from a plain dict, ...

`pinneapple_science/em/fdtd_study.py` · 199 linhas · testes: 19

nome	tipo	assinatura	o que faz
`FDTDConfig`	classe	`(f0, size, cells_per_wavelength, npml, bandwidth, periods, sources, materials)`	
`FDTDRun`	classe	`(config, dx, n, power, current_moment, ntff, frames, frame_times, volume, volume_grid, sl…)`	
`run_fdtd`	função	`(cfg, *, snapshots=24, volume_half=3, volume_steps=24, maxwell_block=True, snap_window=(2…)`	Run the configured study: returns power at f0 (near-to-far-field box two cells inside the CPML), the snapshots of E_z on the three central planes during `snap_window` (periods of f0), a small block of all six components around a point 0.6 wavelength from the first source on `volume_steps` ...
`ladder_values`	função	`(values) -> list[float]`	

`pinneapple_science.em.induction`

Induction and circuits, academic level: Faraday's law with a magnet falling through a loop (3-D + time) and the RC, RL and series RLC circuits (lumped, 0-D in space).

`pinneapple_science/em/induction.py` · 271 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FaradayInduction	classe	()	
Circuits	classe	()	
magnet_rings	função	(a, length, Br, n_rings, n_seg, z_center=0.0)	Segments and per-segment currents of a cylindrical magnet (radius a, length, remanence Br) as n_rings polygonal rings of the bound surface current K = Br / mu0 (A/m).
loop_flux	função	(segs, I_ring, R, n_path=64) -> float	Flux through a circular loop of radius R in the plane z = 0 by Stokes' theorem, oint A . dl (:func:pinneapple_simulation.electromagnetics.statics.flux_through_path). A surface integral of B_z would be singular when the magnet's current sheet crosses the loop's disc; the line integral is ...

pinneapple_science.em.research

Electromagnetism, research level: a configurable FDTD study in 3-D + time with mesh convergence and the observed order, Monte Carlo uncertainty propagation, an inverse problem (permittivity from a measured reflection spectrum) and a surrogate with its hold-out error.

pinneapple_science/em/research.py · 548 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FDTDStudy3D	classe	()	
WaveguideUQ	classe	()	
InversePermittivity	classe	()	
HallSurrogate	classe	()	
slice_viewer	função	(run, path, title='FDTD: E_z in 3-D + time') -> str	A single-file three.js page (Studio viewer): the three central planes as surfaces, E_z r on them animated in time (play, pause, time slider).

pinneapple_science.em.statics

Electro- and magnetostatics, academic level: Coulomb and superposition, Gauss's law, the parallel-plate capacitor with its edge effect, Biot-Savart (wire, loop, solenoid), Ampere's law and the Hall effect.

pinneapple_science/em/statics.py · 901 linhas · testes: 19

nome	tipo	assinatura	o que faz
CoulombSuperposition	classe	()	

nome	tipo	assinatura	o que faz
`GaussLaw`	classe	`()`	
`ParallelPlateCapacitor`	classe	`()`	
`BiotSavart`	classe	`()`	
`AmpereLaw`	classe	`()`	
`HallEffect`	classe	`()`	
`ring_potential`	função	`(points, Q, R) -> np.ndarray`	Potential of a uniformly charged ring (radius `R`, charge `Q`, plane z = 0, centred on the z axis): `phi = Q / (4 pi eps0) (2 / pi) K(m) / sqrt((R + s)^2 + z^2)`, `m = 4 R s / ((R + s)^2 + z^2)` (Jackson, *Classical Electrodynamics*, 3rd ed., sec. 3.3 / problem 3.3; Smythe, *Static and …
`palmer_strip_capacitance`	função	`(w, d, eps_r=1.0) -> float`	Capacitance per unit length of two parallel thin strips (width `w`, gap `d`) with the edge correction of Palmer (1937): `eps (w/d) [1 + d/(pi w) (1 + ln(2 pi w / d))]` (F/m). Asymptotic for `w >> d`. H. B. Palmer, "The capacitance of a parallel-plate capacitor by the Schwartz-Christoffel …
`kirchhoff_disc_capacitance`	função	`(a, d, eps_r=1.0) -> float`	Capacitance of two thin coaxial discs (radius `a`, gap `d`), Kirchhoff (1877): `eps [pi a^2 / d + a (ln(16 pi a / d) - 1)]` (F), asymptotic for `d << a`. G. Kirchhoff, Monatsber. Akad. Wiss. Berlin (1877) 144; see also Jackson, Am. J. Phys. 68, 789 (2000).
`strip_capacitor_2d`	função	`(w, d, h, L=40.0, growth=1.08)`	`(C per unit length, grid, phi)` of two thin strips at +-1/2 V by finite volumes on a graded quarter domain (mirror at x = 0, antisymmetry phi = 0 at y = 0, grounded box at distance `L`).
`disc_capacitor`	função	`(a, d, h, *, cartesian, L=12.0)`	Capacitance of two coaxial thin discs at +-1/2 V: an axisymmetric (r, z) solution, or a 3-D Cartesian one on one eighth of the domain (mirror planes x = 0, y = 0, antisymmetry at z = 0, staircase disc edge).
`hall_bar`	função	`(L, W, beta, n, V=1.0, nx=None)`	Potential in a Hall bar `[0, L] x [0, W]` (contacts at x = 0 and x = L held at 0 and `V`; insulating edges y = 0, W) for Hall parameter `beta = mu B`: Laplace inside, `d phi/dy = beta d phi/dx` on the edges (`J_y = 0` with the Drude conductivity tensor ``sigma0/(1+beta^2) [[1, beta], …

`pinneapple_science.em.waves`

Waves and optics, academic level: radiation of an oscillating dipole, a plane wave in a lossy medium, reflection and refraction at an interface (Fresnel and Snell), waveguide modes, cavity resonances and Mie scattering.

`pinneapple_science/em/waves.py` · 599 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DipoleRadiation`	classe	`()`	

nome	tipo	assinatura	o que faz
`LossyPlaneWave`	classe	`()`	
`FresnelSnell`	classe	`()`	
`WaveguideModes`	classe	`()`	
`CavityResonance`	classe	`()`	
`MieScattering`	classe	`()`	
`dipole_cartesian`	função	`(X, Y, Z, Il, omega)`	Phasor E and H (Cartesian components) of a z-directed Hertzian dipole at the origin (:func:`pinneapple_simulation.electromagnetics.waves.hertzian_dipole_fields`).

`pinneapple_science.instruments`

Virtual laboratory instruments and the fits a bench experiment ends with.

`pinneapple_science/instruments.py` · 194 linhas · testes: 19

nome	tipo	assinatura	o que faz
`Instrument`	classe	`(name, unit, noise, gain_u, offset_u, resolution, seed, _rng, _gain, _offset)`	A reading instrument. `noise`: standard deviation of the random noise per reading (in the unit read); `gain_u`: relative standard uncertainty of the gain (calibration); `offset_u`: standard uncertainty of the zero offset; `resolution`: the quantisation step (0 for none). The gain and offset …
`Oscilloscope`	classe	`(rate, duration, v_range, bits, noise, gain_u, timebase_u, seed)`	A digitising oscilloscope: samples `signal(t)` at `rate` over `duration` after `t0` with vertical noise, a vertical gain error, an ADC of `bits` over the full scale ±`v_range` and a time-base error `timebase_u` (relative).
`LinearFit`	classe	`(slope, intercept, slope_u, intercept_u, cov, chi2, dof, residuals)`	
`CurveFit`	classe	`(params, errors, cov, chi2, dof, residuals)`	
`FieldProbe`	função	`(**kw) -> Instrument`	A Hall magnetic-field probe (tesla).
`Ammeter`	função	`(**kw) -> Instrument`	
`Voltmeter`	função	`(**kw) -> Instrument`	
`linear_fit`	função	`(x, y, sigma_y=None, *, through_origin=False, scale_by_chi2=True) -> LinearFit`	Weighted least squares `y = a x + b` (`b = 0` with `through_origin`). Uncertainties from the covariance `(A^T W A)^{-1}`, scaled by the reduced chi-square when it exceeds one (`scale_by_chi2`), so a scatter larger than the stated sigma is not hidden.

nome	tipo	assinatura	o que faz
`curve_fit`	função	`(model, x, y, p0, sigma_y=None) -> CurveFit`	Non-linear least squares (scipy `curve_fit`) with the same uncertainty rule as :func:`linear_fit`.
`combine`	função	`(*relative_uncertainties) -> float`	Root-sum-square of independent relative standard uncertainties (GUM eq. 10 for a product of powers one).

pinneapple_science.numerics

Small numerical building blocks shared by the experiments: graded tensor grids, a finite-volume solver for `div(eps grad phi) = -rho` in 1-D, 2-D and 3-D with fixed-potential nodes and mirror (symmetry) faces, the discrete field energy, and a scalar Helmholtz eigen-solver for waveguide ...

`pinneapple_science/numerics.py` · 244 linhas · testes: 19

nome	tipo	assinatura	o que faz
`TensorGrid`	classe	`(axes)`	Node coordinates per axis (1, 2 or 3 axes).
`graded_axis`	função	`(core, h, length, growth=1.12, start=0.0) -> np.ndarray`	Nodes from `start`: spacing exactly `h` over at least `core` (rounded up to whole cells), then growing geometrically by `growth` until `start + length`.
`solve_potential`	função	`(grid, fixed, values, *, eps=None, rho=None, tol=1e-10, direct=None, mean='harmonic') -> …)`	Solve `-div(eps grad phi) = rho` with `phi = values` on the `fixed` nodes (boolean array; put the outer box there for a grounded enclosure). Faces without fixed nodes are zero-flux, which is the mirror condition of a symmetry plane. `eps` (relative or absolute, per node) defaults to 1, ...
`field_energy`	função	`(grid, phi, eps=None, mean='harmonic') -> float`	Discrete `1/2 sum eps (grad phi)^2 dV` (per unit length in 2-D), with the face fluxes of the scheme.
`helmholtz_eigs`	função	`(nx, ny, a, b, kind, n=6, inside=None, modes=False)`	The `n` smallest cut-off wavenumbers `k_c` of a waveguide cross-section from the scalar Helmholtz eigenproblem `-lap psi = k_c^2 psi` on an `nx` x `ny` cell-centred grid of the box `a` x `b`: `kind="TM"` (`psi = E_z`, Dirichlet on the wall) or `"TE"` (`psi = H_z`, Neumann). ...
`spectral_peaks`	função	`(signal, dt, f_max, *, n_peaks=8, min_rel=0.001, pad=8) -> np.ndarray`	Frequencies of the strongest local maxima of the spectrum of `signal` below `f_max` (Hann window, `pad`-fold zero padding, parabolic interpolation of the log spectrum), ascending. Peaks below `min_rel` times the largest are dropped.
`refine_ladder`	função	`(base, levels, ratio=2) -> Sequence[int]`	
`gp_loo_scale`	função	`(gp) -> float`	Factor for the predictive standard deviation of a fitted Gaussian process (an object with `L`, the Cholesky factor of its training covariance, and `alpha = K^-1 z`, as :class:`pinneapple_presets.surrogate.ARDGP`) from its leave-one-out residuals on the training data: `sqrt(mean(z_i^2))` with ...

pinneapple_science.plots

Figure conventions of :mod:`pinneapple_science`: one palette (computed = slot 1, reference = slot 2, in that fixed order), a single-hue sequential map for magnitudes, a two-hue diverging map with a neutral middle for signed fields, thin marks, recessive axes. Every function returns a matplotlib …

`pinneapple_science/plots.py` · 83 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`style`	função	`(ax) -> None`	
`figure`	função	`(ncols=1, nrows=1, *, width=4.2, height=3.3, three_d=(), title='')`	`(fig, axes)`: axes is a flat list; indices in `three_d` get 3-D axes.
`badge`	função	`(ax, text, ok=True, loc='lower right') -> None`	A small text box with a measured result (the ink is the text colour; the frame carries pass / fail).
`close`	função	`(fig) -> None`	
`log_ticks_ok`	função	`(values) -> bool`	
`to_rgb`	função	`(fig) -> np.ndarray`	The figure as an RGB array (for animation frames).
`integer_ticks`	função	`(ax, values) -> None`	Plain integer labels at `values` on a log x axis (no minor labels like 3x10^1).

pinneapple_science.registry

The experiments of :mod:`pinneapple_science`, by name (modules are imported on first use).

`pinneapple_science/registry.py` · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register`	função	`(cls) -> type[Experiment]`	Class decorator: add an experiment to the catalogue.
`get`	função	`(name) -> Experiment`	
`names`	função	`() -> list[str]`	
`experiments`	função	`(area=None, *, level=None, kind=None, dim=None) -> list[dict[str, Any]]`	Cards of the experiments matching every filter given (`area` matches the area id or its prefix).
`run`	função	`(name, dim=None, *, quick=False, **params) -> Result`	Run one experiment in one dimension (its first by default) with parameters over its defaults.

pinneapple_science.report

Committed results of :mod:`pinneapple_science`: one folder per experiment with every dimension it runs in, its Lab manifest (`experiment.json`), its Lab seal and a README; and the catalogue with the atlas.

`pinneapple_science/report.py` · 204 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`folder_of`	função	`(name, root=None) -> str`	
`write`	função	`(name, *, root=None, quick=False, register=True, seal=True) -> dict[str, Any]`	Run `name` in every dimension, save the results, write the README and the Lab manifest, register it in the Lab and stamp the figures with its seal. Returns `{"folder", "results", "item", "sealed"}`.
`readme`	função	`(name, results, sealed, item) -> str`	
`table`	função	`(root=None) -> list[dict[str, Any]]`	Experiment x dimension x quantity x reference x measured error, from the committed `result.json` files.
`write_catalogue`	função	`(root=None, register=True) -> str`	`examples/science/README.md` (one row per comparison from the committed results), `ATLAS.md`, `atlas.html` and the folder's manifest.
`write_all`	função	`(*, root=None, quick=False, register=True) -> list[dict[str, Any]]`	

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 6 de 16.
- Funções de teste em arquivos que importam algum módulo do pacote: 19.

Limitações conhecidas

- *Nenhuma registrada.*

5. `pinneapple_condense`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
18	6.006	29	115	27 (3 arquivos)	3

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`algebraic`		1	6	6
`benchmark`		1	14	0
`exceptions`		1	6	14
`falsify`		1	21	6

submódulo	o que faz	módulos	classes + funções	testes*
figures		1	8	0
flow		1	9	6
fv		1	8	21
hierarchy		1	2	14
homogenize		1	7	14
learned		1	10	14
meshless		1	9	14
meshless2d		1	9	6
package		1	7	27
query		1	8	7
query_benchmark		1	1	0
substructure		1	15	21
transient		1	4	14

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_condense`

Adaptive Physics Condensation (work in progress).

`pinneapple_condense/__init__.py` · 2 linhas · testes: 27

`pinneapple_condense.algebraic`

Algebraic condensation: the method of :mod:`.substructure` for *any* assembled sparse operator.

`pinneapple_condense/algebraic.py` · 299 linhas · testes: 6

nome	tipo	assinatura	o que faz
Partition	classe	`(block, sep, interiors, adjacent, shape, size, origin, coords, fieldid)`	Blocks of the unknowns: `block` per unknown (flat id), `sep` mask, and per block its interior and adjacent separator unknowns in canonical local order.
Condensation	classe	`(A, part, timings)`	Exact algebraic condensation of `A x = b` over a :class:`Partition`.
Reduced	classe	`(cond, fc, modes)`	Galerkin-reduced separator problem of a :class:`Condensation` (exact when every face keeps all its modes).
partition	função	`(A, coords, block_size, origin=(0.0, 0.0), fieldid=None, primal=None, h=None) -> Partition`	Blocks of side `block_size` from `origin`; separators from the coupling graph (see the module docstring). `h`: a length below the smallest spacing (for the canonical rounding; default block size / 1e4).
faces	função	`(part, A) -> dict`	Separator unknowns grouped by the set of blocks they couple to. Two blocks: a face, keyed by `(orientation, field, length)` (orientation 0: the two blocks are x-neighbours) with canonical positions along it; three or more blocks, or primal unknowns: kept full.

nome	tipo	assinatura	o que faz
face_modes	função	`(snapshots, fc, m) -> dict`	POD modes per face type from separator snapshots (full-length solution vectors of the snapshot problem with its face description `fc`): the constant plus the `m - 1` leading left singular vectors of the constant-free face traces.

pinneapple_condense.benchmark

The measured studies of the condensation benchmark (each returns plain JSON-able numbers).

pinneapple_condense/benchmark.py · 396 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
peak_rss_gb	função	`() -> float`	
isolated	função	`(func, timeout=3600, module=…, **kwargs) -> dict`	Run `<module>.<func>(**kwargs)` (default: this module) in a fresh Python process; its result plus the peak RSS of that process (`peak_rss_gb`).
mesh_convergence	função	`(shape=(8, 8), levels=((4, 1), (8, 2), (16, 4)), seed=1) -> dict`	Reference solutions at increasing resolution (cells per block side, vertical multiplier), the observed order from the last three levels, the Richardson extrapolation and the error of the default resolution (8, 1).
accuracy_vs_modes	função	`(shape=(12, 12), ms=(1, 2, 4, 6, 8, 12, 16, 24), seed=1, modes='pod') -> dict`	
refinement_study	função	`(shape=(12, 12), m0=2, steps=6, seed=1, hot=True) -> dict`	Doerfler face refinement from `m0` modes (`p`: double the modes of marked faces; `full`: un-reduce them) vs uniform `m`; a power map with a sub-pitch hotspot (`hot`) makes the error local.
correction_study	função	`(nb=32, ms=(6, 12), seed=0, power_scale=4.0, rtol=1e-09, every=10) -> dict`	The exact correction (:meth:`.substructure.Condensed.correct`) on a large hotspot case: iterations to the fine solution from each reduced model, and the Tmax error along the way (every `every` iterations).
homogenization_study	função	`(shape=(12, 12), seed=1) -> dict`	
scaling_point	função	`(nb, m=12, reference=True, flat=True, hier=True, seed=2) -> dict`	One `nb x nb` package: wall time and result of each method that is asked for (call through :func:`isolated` per method for its own peak memory).
configurations	função	`(nb=64, m=12, n_configs=20, seed=0, check=2) -> dict`	Build once, then solve `n_configs` new power maps; the first `check` are also solved by the reference.
learned_study	função	`(nb=32, m=12, n_train=48, seed=1, defects=4, exact_reference=True, tol_T=0.01) -> dict`	

nome	tipo	assinatura	o que faz
`transient_study`	função	`(shape=(6, 6), dt=0.0002, steps=150, mk=((8, 8), (16, 16), (32, 32)), seed=1) -> dict`	
`root_compressibility`	função	`(nb=64, m=12, tol=1e-08) -> dict`	Numerical rank (relative `tol`) of the off-diagonal blocks between the four sides of the root Schur complement: how much an H-matrix representation of the top operator could save.
`meshless_study`	função	`(nb=2, hfs=(6, 8, 10, 12), methods=('gfdm', 'rbf_fd'), interfaces=('pairs', 'naive'), ls_…)`	Track 1b: RBF-FD and GFDM point clouds from the implicit geometry against the fine FV reference, for a range of spacings `pitch / hf`, interface treatments (node pairs with flux continuity and the contact law, or the naive single-node collocation), and least-squares collocation (`ls_hfs`). …
`exceptions_study`	função	`(shape=(12, 12), seed=1, budgets=…) -> dict`	Track 1c: the Keff prior plus residual-selected exception patches against random patches at the same budget, on a smooth power map and on one with a sub-pitch hotspot; and the 1-D column prior for comparison.

`pinneapple_condense.exceptions`

Prior field plus exception patches (track 1c): solve exactly only where a cheap prior does not hold.

`pinneapple_condense/exceptions.py` · 190 linhas · testes: 14

nome	tipo	assinatura	o que faz
`prior`	função	`(pkg, kind='keff') -> tuple[np.ndarray, float]`	The prior on the fine cells `(nx, ny, nz)` and its cost (s).
`block_indicator`	função	`(pkg, sysm, T, f) -> np.ndarray`	`||K T - f||` over the cells of each block (W), `(nbx, nby)`.
`correct`	função	`(pkg, T0, blocks, *, halo=1, blend='core', sysm=None, f=None) -> tuple[np.ndarray, dict]`	Solve the fine problem on the patches around `blocks` (boolean `(nbx, nby)`) with `T0` outside.
`energy_correct`	função	`(pkg, T, sysm) -> tuple[np.ndarray, float]`	Add `alpha * phi` (`phi`: the response to a uniform map, i.e. one block with adiabatic sides, by symmetry) so that the Robin outflow equals the power.
`run`	função	`(pkg, *, budget=0.1, select='residual', halo=1, blend='core', prior_kind='keff', energy=T…)`	The whole method at one budget (fraction of blocks selected); returns the field and its costs.
`pareto`	função	`(pkg, budgets=…, *, halos=(0, 1), selects=('residual', 'random'), prior_kind='keff', seed…)`	Error against the fraction of cells solved exactly, for each selection rule and halo; with the reference.

`pinneapple_condense.falsify`

Falsification studies of the condensation method (`docs/research/condensation_falsification.md`): each function is one pre-registered test, returns plain JSON-able numbers and is meant to run in a fresh process (:func:`.benchmark.isolated` with `module="pinneapple_condense.falsify"`) so that …

nome	tipo	assinatura	o que faz
t1_reference	função	(nb, seed=2, m=12, rtol=1e-10, fine=True, maxiter=2000) -> dict	Reference peak junction temperature of an nb x nb package: the fine AMG solve (fine) and the exact condensation (PCG on the unreduced interfaces, preconditioned by the m-mode model) with its convergence trail (T1b: iterations and time to 1 mK).
t1_point	função	(nb, m, seed=2) -> dict	The memoised hierarchy with m interface modes: peak junction temperature, build and solve time.
f0_validation	função	(poiseuille=(16, 32, 64), cavity=(32, 64, 128), nusselt=(16, 32, 64)) -> dict	The flow solver against plane Poiseuille flow, the Re = 100 cavity and the one-sided-flux parallel-plate Nusselt number (5.385).
f2_stokes	função	(n_pins=16, cells=16, pin=8, ms=(1, 2, 4, 8, 16), n_snap=24, seed=0) -> dict	F2 / F2r: Stokes through an n_pins x n_pins pin array; exact algebraic condensation against the global direct solve, then reduced faces from snapshots of a 4 x 4 array with random smooth inlet profiles.
f1_advection	função	(n_pins=8, cells=16, pin=8, pes=(0.1, 1, 10, 100, 1000), ms=(1, 2, 4, 8, 16), n_snap=24, …)	F1 / F1r: steady advection-diffusion through the pin array (velocity: the Stokes solution), block Peclet sweep; exact condensation of the non-symmetric operator, then reduced faces from a 4 x 4 snapshot problem with random pin sources and inlet profiles at the same Peclet number.
f3_cht	função	(n_pins=(8, 4), cells=16, pin=8, res=(1, 10, 50, 100), ms=(2, 4, 8, 16), n_snap=16, tol=1…)	F3 / F3r: cold plate (fin array in a free-slip strip), Navier-Stokes by Picard, full solve against condensed Picard (every Oseen iterate condensed by content), then the conjugate energy equation; reduced faces for the flow at the largest Re; the fraction of reversed face flux on block faces.
thermal_case	função	(pkg, m=12, rtol=1e-10) -> dict	One package: fine FV peak junction temperature; m-mode condensation (Tmax error, energy-norm effectivity of the residual estimate); exact condensation (PCG to rtol) against the fine solve.
t2_contrast	função	(nb=32, ratios=(1000.0, 10000.0, 100000.0), m=12, seed=2) -> dict	T2: the underfill conductivity lowered so that k_copper / k_underfill = ratio (baseline 401 / 0.5 = 802).
t3_anisotropy	função	(nb=32, factors=(100.0, 0.01), m=12, seed=2) -> dict	T3: TIM and interposer with in-plane / through-thickness conductivity ratio factor.
t8_adversarial	função	(nb=16, m=12, seed=2) -> dict	T8: power maps unlike the snapshot set (same total power as the baseline hotspot map).
t9_thin	função	(nb=16, m=12, seed=2) -> dict	T9: thin layers -- TIM 30 -> 3 -> 1 um (1 um with one cell row), die 100 -> 20 um.
t4_no_repetition	função	(nb=32, m=12, seed=1, n_train=48) -> dict	T4: every block distinct (continuous random variants, four defects). Against one fine FV solve: the exact path (every block condensed, m-mode interfaces, then the PCG correction to the fine solution), and the learned path (:func:.learned.solve_adaptive, training timed separately).
t5_variants	função	(nb=64, Vs=(1, 4, 16, 64, 256), m=12, seed=0) -> dict	T5: V discrete variants at random positions: hierarchy build (memoised subtrees) against the flat condensation build, both including the V block condensations.

nome	tipo	assinatura	o que faz
t6_t7_nonlinear	função	(nb=16, case='k', seed=2, power_scale=2.0) -> dict	T6 (case="k": silicon and copper k(T)) or T7 (case="R": R_c(T) = R_0 (1 + beta (T - T_0)) on the die/TIM contact, beta doubling R_c over the temperature range of the linear solution).
t10_pitch	função	(nb=16, pitches=(4e-05, 2e-05), ms=(2, 4, 8, 12, 16, 24), n_snap=24, seed=2) -> dict	T10: columns of blocks alternating in pitch, through algebraic condensation (each pitch its own block operator, found by content); baseline: the uniform pitch through the same pipeline. Reduced faces from snapshots of a 4 x 4 package with the same column pattern and random power maps.
m1_advection	função	(pins=4, densities=(8, 16, 32), truth_cells=64, pes=(1, 10, 100), methods=('rbf_fd', 'gfd…)	M1: meshless advection-diffusion on point clouds (velocity interpolated from the finest FV Stokes solution) against FV at equal unknowns; truth: FV at truth_cells per unit cell.
m2_stokes	função	(pins=4, densities=(8, 16, 32), truth_cells=64, methods=('rbf_fd', 'gfdm'), seed=0) -> di…)	M2: collocated meshless Stokes on fluid points with pin-wall points; pressure drop against FV.
m3_hybrid	função	(pins=2, densities=(8, 16, 32), truth_cells=64, k_pin=400.0, k_mat=1.0, R_c=0.001, stenci…)	M3: FV-at-interfaces hybrid against FV everywhere at equal unknowns (truth: FV at truth_cells), with the error split between the FV band and the meshless points, and the stencil sensitivity.
m4_meshless_condensed	função	(pins=4, cells=16, seed=0) -> dict	M4: meshless matrices condensed by :mod:.algebraic: the hybrid conduction (replicated point clouds and FV bands: identical blocks expected) and the meshless advection-diffusion (velocity differs per block).
exception_patches	função	(A, b, x0, x_true, block, nbshape, budgets, select='residual', halo=1, seed=0)	Generic prior + exception patches: blocks ranked by the norm of b - A x0 over their unknowns (or random), the patch (selected blocks + halo rings) solved with the prior as Dirichlet data outside.
e1_cfd	função	(n_pins=8, fine=16, coarse=4, budgets=…, Pe_b=100.0, Re=50.0, seed=0) -> dict	E1: residual-selected against random exception patches on the advection-diffusion case (Pe_b = 100) and on the conjugate energy field of the cold plate (Re_fin = 50); prior: the same case on a 4x coarser grid.

pinneapple_condense.figures

Figures of the condensation benchmark (matplotlib, from the JSON of :mod:.benchmark).

pinneapple_condense/figures.py · 214 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
scaling	função	(points, path) -> str	Wall time and peak memory against the number of blocks (two panels, one quantity each).
accuracy	função	(res, path) -> str	

nome	tipo	assinatura	o que faz
`refinement`	função	`(res, path) -> str`	
`keff_maps`	função	`(maps, path) -> str`	Fine junction temperatures, the homogenised block averages, and their difference.
`transient`	função	`(res, path) -> str`	
`meshless_convergence`	função	`(res, path, mesh_error=None) -> str`	|Tmax error| and |energy balance| against the number of nodes, per method and interface treatment.
`meshless_fields`	função	`(res, method, path) -> str`	The point cloud on a vertical slice coloured by temperature, and the junction-temperature error map.
`pareto`	função	`(res, path) -> str`	Error against the fraction of cells solved exactly (exception patches), per selection rule and halo.

`pinneapple_condense.flow`

Steady 2-D incompressible flow and heat on a staggered (MAC) finite-volume grid, assembled as one sparse system: the reference and the operator of the CFD falsification tests (`docs/research/condensation_falsification.md`).

`pinneapple_condense/flow.py` · 330 linhas · testes: 6

nome	tipo	assinatura	o que faz
`Grid`	classe	`(nx, ny, h, solid, walls, closed, lid, index)`	
`ScalarIndex`	classe	`(nx, ny)`	
`assemble_oseen`	função	`(g, nu, inlet=None, adv=None, force=(0.0, 0.0)) -> tuple[sp.csr_matrix, np.ndarray]`	The Oseen (Stokes when `adv` is None) system `A x = b`; `inlet`: u on the inlet faces (ny,); `adv`: advecting face velocities `(au (nx+1, ny), av (nx, ny+1))`; `force`: body force per unit mass.
`split`	função	`(g, x) -> tuple[np.ndarray, np.ndarray, np.ndarray]`	
`solve`	função	`(A, b) -> np.ndarray`	
`picard`	função	`(g, nu, inlet=None, tol=1e-08, maxiter=200, relax=1.0, linear_solver=None, force=(0.0, 0.…)`	Navier-Stokes by Picard (Oseen) iteration from Stokes: `linear_solver(A, b) -> x` (default: direct). Returns the solution, the iteration history of the relative velocity change and the per-iteration systems' solve times.
`assemble_scalar`	função	`(g, u, v, k, rhoc, q, T_in=0.0, scheme='hybrid') -> tuple[sp.csr_matrix, np.ndarray]`	`rho c (u . grad T) - div(k grad T) = q` on every cell; `k`, `rhoc`, `q` are (nx, ny) cell fields (`rhoc` zero in solids: no advection there; face velocities are zero on solid faces anyway). `scheme`: "hybrid" or "upwind".
`scalar_coords`	função	`(g) -> np.ndarray`	

nome	tipo	assinatura	o que faz
pin_array	função	(n_pins, cells, pin, h, inlet_cells=0, outlet_cells=0) -> np.ndarray	Solid mask of an array of square pins: cells per unit cell side, a pin x pin pin centred in each, with inlet_cells / outlet_cells empty columns before and after.

pinneapple_condense.fv

Cell-centred finite volumes for steady and transient conduction on a voxel box, and the reference solver.

pinneapple_condense/fv.py · 174 linhas · testes: 21

nome	tipo	assinatura	o que faz
System	classe	(K, shape, ports, g_top, g_bottom, Gz, C)	An assembled conduction operator: K (W/K), the cell count, the port blocks, the Robin conductances of the top and bottom cell rows ((nx, ny)) and the vertical face conductances Gz ((nx, ny, nz - 1)).
SolveInfo	classe	(method, seconds, iterations, residual)	
assemble	função	(k, dz, R, dx, h_top, h_bottom, *, ports=False, rho_c=None, dy=None) -> System	Assemble the conduction operator of a voxel box (see the module docstring). k: (nx, ny, nz) array or a (kx, ky, kz) tuple; R: contact resistance of each horizontal face, (nz - 1,) or (nx, ny, nz - 1); rho_c: volumetric heat capacity, returned as the diagonal C (J/K) …
assemble_package	função	(pkg, *, transient=False) -> System	The whole package (:class:.package.Package) as one box, adiabatic on the sides: the reference model.
source_vector	função	(pkg, power=None) -> np.ndarray	Heat source of every cell (W): the power map on the active cell row.
factor	função	(K)	Sparse LU of a symmetric matrix (minimum degree on K + K^T, symmetric pivoting).
solve	função	(K, f, *, method='auto', rtol=1e-11, maxiter=4000) -> tuple[np.ndarray, SolveInfo]	Solve K T = f. auto: sparse direct (SuperLU, minimum-degree ordering on K + K^T) up to 6e4 unknowns, else conjugate gradients with an algebraic multigrid pre-conditioner (pyamg, if installed; Jacobi otherwise) to re-lative residual rtol.
solve_package	função	(pkg, power=None, **kw) -> tuple[np.ndarray, SolveInfo]	Reference steady temperature rise of every cell, shape (nx, ny, nz).

pinneapple_condense.hierarchy

Hierarchical (nested) condensation with reuse of identical subtrees.

pinneapple_condense/hierarchy.py · 356 linhas · testes: 14

nome	tipo	assinatura	o que faz
Hierarchy	classe	(pkg, m=6, *, modes='pod', keep_root=True)	Nested condensation of a package (uniform m interface mo-des on every face).

nome	tipo	assinatura	o que faz
estimate	função	`(h, sol, chunk=4096) -> np.ndarray`	Per-face error indicator of a hierarchical solution, as :meth:`.substructure.Condensed.estimate`: the fine residual on the port nodes (interiors are exact) measured in the inverse two-sided face Schur complement.

pinneapple_condense.homogenize

Effective-conductivity (Keff) homogenisation: the baseline every condensation must beat.

`pinneapple_condense/homogenize.py` · 121 linhas · testes: 14

nome	tipo	assinatura	o que faz
effective_layer	função	`(k_m, k_i, phi) -> tuple[float, float]`	(k_xy, k_z) of a layer with a volume fraction `phi` of aligned prisms of conductivity `k_i`.
effective_properties	função	`(stack, variant=BASE) -> dict[str, np.ndarray]`	Per z row of one block: `kxy`, `kz`, `rho_c` (volume average), `dz` and contact `R`.
homogenized_system	função	`(pkg, cells=1, transient=False)`	The homogenised package on `cells x cells` lateral cells per block (`n` must be divisible by it).
coarse_power	função	`(pkg, cells=1, power=None) -> np.ndarray`	
solve_homogenized	função	`(pkg, cells=1, power=None)`	Coarse temperature field `(nbx * cells, nby * cells, nz)` and the solve info.
homogenized_observables	função	`(pkg, T, sysm, cells=1) -> dict[str, np.ndarray]`	Per block: mean junction temperature `Tj_mean`, the die/TIM heat `q_tim`, the heat down the microbump layer `q_ubump_layer` and the share a bump would take by the parallel rule (`q_ubump` estimate).
compare	função	`(pkg, ref, hom) -> dict[str, float]`	What homogenisation erases, against fine-scale reference observables (:func:`.substructure. reference_observables`): the peak junction temperature, block mean and block peak junction temperatures, the intra-block junction spread the coarse model cannot represent, and the interface heats.

pinneapple_condense.learned

Condensation with a learned block operator, error indicators and an error-driven fallback to exact condensation.

`pinneapple_condense/learned.py` · 310 linhas · testes: 14

nome	tipo	assinatura	o que faz
`LearnedBlocks`	classe	`(stack, m, bounds, to_variant, n_train, regressor, seed, timings)`	A learned map `theta -> reduced block operator`. `fit()` condenses `n_train` Latin-hypercube samples of `bounds` exactly and trains the regression.
`AdaptiveResult`	classe	`(obs, tb, exact_blocks, checked, failed, rounds, timings, flagged_std, flagged_ood, estim…)`	
`default_variant`	função	`(theta) -> Variant`	
`reduced_block`	função	`(stack, variant, Phi) -> dict[str, np.ndarray]`	Exact reduced operator of one block on the interface modes `Phi` (no cache: one factorisation).
`assemble_blocks`	função	`(Sr, dofs, n) -> sp.csc_matrix`	Global interface matrix from per-block reduced operators `(N, d, d)` and their dofs `(N, d)`.
`solve_blocks`	função	`(ops, pb, dofs, n)`	Assemble, factorise, solve; return the per-block reduced port values `(N, d)` and the timings.
`block_observables`	função	`(ops, tb, pb, n2) -> dict[str, np.ndarray]`	
`solve_adaptive`	função	`(learned, theta, power, *, tol_std=0.05, tol_T=0.01, n_hot=8, n_audit=16, max_rounds=5, s…)`	Learned condensation with the verification loop of the module docstring. `theta`: `(nbx, nby, p)` block parameters; `power`: `(nbx * n, nby * n)` W per active cell.
`solve_exact`	função	`(learned, theta, power) -> tuple[dict, float]`	Every block condensed exactly (one factorisation per block): the reference of the learned path.
`variant_field`	função	`(shape, seed=0, *, defects=0, defect_k=0.05, bounds=None) -> np.ndarray`	Smooth random block parameters inside `bounds` (a degraded-TIM gradient, scattered bump voids, TSV quality) and `defects` nearly open micro-bumps (`ubump_k = defect_k`, outside the default training box).

`pinneapple_condense.meshless`

Meshless solution of the chip-package benchmark (track 1b): points straight from the implicit geometry, no volume mesh.

`pinneapple_condense/meshless.py` · 637 linhas · testes: 14

nome	tipo	assinatura	o que faz
`Cloud`	classe	`(X, region, kind, normal, partner, area, k, R, seconds, info)`	
`Geometry`	classe	`(pkg)`	Implicit description of a package: what material is at any point, and its surfaces.
`MeshlessSolution`	classe	`(cloud, T, A, b, timings, stencils)`	

nome	tipo	assinatura	o que faz
point_cloud	função	(pkg, *, h_xy=None, per_layer=3, h_max=2.5e-05, per_feature=4, band_h=2.0, gap=0.3, jitte…)	Nodes from the implicit geometry (see the module docstring); h_xy: spacing cap (default: the pitch / 6); per_layer: nodes across each layer; per_feature: nodes across each inclusion. density: an optional callable X -> factor in (0, 1] that thins the interior lattice (adaptive …
region_stencils	função	(X, region, Q, q_region, n, box, reach, normal=None)	Stencils of the n nearest nodes of the same region for every query point. With box = (W0, W1) the package sides are mirror planes (adiabatic): nodes within reach of a side (or two) are reflected and a reflected copy stands for its original node. Returns (Xe, idx_e, col): the …
assemble	função	(cloud, pkg, *, method='rbf_fd', degree=2, n=None, interface='pairs', oversample=False)	The collocation system of the module docstring. Returns (A, b, stencils).
solve	função	(pkg, cloud=None, *, method='rbf_fd', degree=2, n=None, interface='pairs', solver='auto',…)	Point cloud (unless given), assembly, solve. solver: sparse LU, or GMRES with an ILU preconditioner above 2e5 nodes. condition: also a 1-norm condition-number estimate (an extra factorisation).
evaluate	função	(sol, pkg, P, *, degree=None) -> np.ndarray	Temperature at points P (interpolated with stencils of the region each point lies in).
observables	função	(sol, pkg) -> dict	Junction temperatures at the centres of the reference's active cells (Tj as in :func:.substructure.reference_observables), the die/TIM heat, the outflows and the energy balance.

pinneapple_condense.meshless2d

2-D meshless (RBF-FD / GFDM) discretisations of the CFD falsification cases (docs/research/ condensation_falsification.md, section C), on point clouds built from the implicit geometry of a square-pin array.

pinneapple_condense/meshless2d.py · 339 linhas · testes: 6

nome	tipo	assinatura	o que faz
pin_sdf	função	(X, pins, pin_frac, offset=(1.0, 0.0))	Signed distance to the nearest square pin (negative inside); unit cells of side 1 from offset.
cloud	função	(pins, h, pin_frac=0.5, jitter=0.25, inlet_blocks=1, outlet_blocks=1, seed=0, fluid_only=…)	Points over (inlet + pins_x + outlet) x pins_y unit cells: one jittered lattice of spacing h per unit cell, the same in every cell (shifted by integers). fluid_only drops the points inside (and within 0.3 h of) the pins; wall_points adds points on the pin walls (spacing h). …
bilinear_mac	função	(g, u, v, X)	Velocity of a MAC solution (grid g of :mod:.flow, lengths in unit cells) at points X.
snap_to_box	função	(X, box, h)	Points within h / 2 of the box moved onto it (boundary nodes: the conditions are collocated where they hold).

nome	tipo	assinatura	o que faz
knn	função	(X, Q, n, tree=None)	The n nearest nodes of each query point, ties broken by the offset (not by the k-d tree's traversal order): a block replicated by an integer shift then gets the same stencils, so the same matrix.
advection_diffusion	função	(X, box, h, vel, alpha, q, *, method='rbf_fd', n=15, degree=2)	Collocated vel . grad T - alpha lap T = q. Points within 3 h / 8 of the box carry the boundary condition: inlet T = 0; outlet, top and bottom zero normal gradient.
stokes	função	(X, wall, box, h, nu=1.0, *, method='rbf_fd', n=20, degree=2)	Collocated Stokes (u, v, p at every fluid point). Rows: momentum and continuity at interior points; no-slip on pin-wall points (continuity replaced by the normal momentum balance there is not used: pressure at wall points is left to the continuity equation); inlet u = 1, v = 0; outlet p = 0, du/dx ...
hybrid_conduction	função	(pins, cells, pin_frac, k_pin, k_mat, R_c, *, band=2, point_h=None, jitter=0.25, n=12, de…)	Conduction -div(k grad T) = q (source q_pin in the pins) in a pins_x x pins_y array of unit cells, adiabatic sides, cooled on top (Robin h_top, T_inf = 0), with a contact resistance R_c on every pin wall. FV cells (cells per unit cell side) within band cells of a pin wall; ...
fv_conduction	função	(pins, cells, pin_frac, k_pin, k_mat, R_c, q_pin=1.0, h_top=1.0)	The same conduction problem on finite volumes everywhere (the reference and the equal-unknowns baseline).

pinneapple_condense.package

A synthetic multilayer chip package built from one repeated unit cell (a *block*).

pinneapple_condense/package.py · 253 linhas · testes: 27

nome	tipo	assinatura	o que faz
Layer	classe	(name, thickness, nz, matrix, inclusion, side)	One layer of the stack. inclusion: material of the square inclusion centred in each block (bump, TSV), side: its side as a fraction of the block pitch.
Variant	classe	(inclusion_k, removed, contact)	What makes a block differ from the base block. inclusion_k: ((layer, factor), ...) multiplies the inclusion conductivity of a layer (a void fraction lowers it); removed: layers whose inclusion is missing (replaced by the matrix); contact: ((lower, upper, factor), ...) multiplies a ...
Stack	classe	(layers, pitch, n, contact, h_top, h_bottom, active, materials, anisotropy)	The layers (bottom to top), the block pitch and lateral cells per block, contact resistances between layers (m^2 K / W), Robin coefficients on the top of the lid and the bottom of the substrate (W/m^2/K), and the layer whose bottom cell row is the active (heat-generating, junction) surface.
Package	classe	(stack, layout, variants, power)	A lateral (nbx, nby) array of blocks over one stack. layout[I, J] indexes variants; power is the heat (W) of every active cell, shape (nbx * n, nby * n).
reference_stack	função	(n=8, vertical=1) -> Stack	The benchmark stack: 40 um block pitch (one microbump, one TSV and one C4 per block; a simplification: C4 pitch is usually larger than microbump pitch), ~1 mm total height, 8 x 8 cells per block side by default.

nome	tipo	assinatura	o que faz
`hotspot_power`	função	`(stack, shape, total, *, hotspots=3, background=0.4, width=0.08, seed=0, centres=None) ->…)`	A non-uniform power map on the active cells (W per cell): `background` of `total` spread uniformly, the rest in Gaussian hotspots (`width` as a fraction of the die side) at random or given `centres`.
`uniform_package`	função	`(shape, stack=None, *, total_power=None, power_density=1000000.0, seed=0, hotspots=3) -> …)`	A package of identical blocks with a hotspot power map. `power_density`: mean heat flux over the die (W/m^2; 1e6 W/m^2 = 100 W/cm^2) unless `total_power` is given. Change `layout` / `variants` afterwards for heterogeneous blocks.

pinneapple_condense.query

Physics queries on a chip package (track 2): the methods behind :class:`pinneapple_query.Engine` for the package of :mod:`.package`.

`pinneapple_condense/query.py` · 411 linhas · testes: 7

nome	tipo	assinatura	o que faz
`State`	classe	`(power, obs, method, uncertainty, seconds, solver=None, sol=None, field=None, obs_unc=Non…)`	A solved configuration: per-block observables, its source, its own uncertainty (K, the largest over the junction), the uncertainty of each observable (`obs_unc`: same keys as `obs`, absolute; `None` for an exact state), and what is needed to read interior points (the hierarchy and its …
`PackageModel`	classe	`(pkg, m=8, reference_cells=2000000)`	Shared machinery of the methods: the package, the stored states, the hierarchies, extraction of a quantity from a state.
`Cached`	classe	`(model)`	
`Bound`	classe	`(model)`	Rigorous interval from the M-matrix monotonicity (see the module docstring).
`Superposition`	classe	`(model, max_states=64)`	Linear combination of solved states (least squares on the power maps); the residual map bounded by monotonicity, plus the combined uncertainty of the states.
`Condensed`	classe	`(model, expected_s=1.0)`	A new solve with the memoised hierarchy at `m`; uncertainty |T_m - T_2m| (the 2m value is returned).
`Reference`	classe	`(model)`	The fine finite-volume solve: the last resort.
`package_engine`	função	`(pkg, m=12, reference_cells=2000000, calibrate=True) -> tuple[Engine, PackageModel]`	A query engine over a package with the methods of this module. `calibrate`: solve the package's own power map once with the condensed method (part of the build cost), which gives the condensed method its uncertainty prior and the store its first state.

pinneapple_condense.query_benchmark

Measured demonstration of the physics query engine on the chip package (track 2).

pinneapple_condense/query_benchmark.py · 125 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
query_demo	função	(nb=48, m=12, n_points=2000, n_interior=100, seed=0, reference=True) -> dict	

pinneapple_condense.substructure

Static condensation of repeated blocks, reduced interfaces, on-demand recovery and an error-driven refinement.

pinneapple_condense/substructure.py · 661 linhas · testes: 21

nome	tipo	assinatura	o que faz
BlockModel	classe	(stack, variant, S, W, Y, Z, ports, Om, obs_names, seconds, O_p, O_t)	The condensed form of one distinct block (see the module docstring for the symbols).
CondensedState	classe	(t, pb, seconds)	A solved condensed configuration: the interface dofs and the block powers it was solved for.
Condensed	classe	(pkg, m=6, *, refined_x=None, refined_y=None, modes='pod', shift=0.0, face_m=None)	The condensed package: block operators reused across identical blocks, m interface modes per face (refined_x / refined_y: faces kept exact; face_m: modes of every face, x faces first), a global sparse interface system factorised once.
block_functionals	função	(stack, variant, sysb) -> tuple[sp.csr_matrix, list[str]]	Rows of the linear observables of one block over its cells: the n * n junction temperatures, then the vertical heat (W, positive upwards) through the die/TIM interface, the microbump, the TSV and the C4 bump (each at the middle face of its layer, over the inclusion cells), and the Robin outflow ...
block_model	função	(stack, variant=BASE, *, cache=True, shift=0.0) -> BlockModel	Condense one block (cached per (stack, variant, shift)). shift s condenses K + s C instead of K (the Laplace-domain operator at rate s [1/s]; one implicit-Euler step for s = 1/dt).
clear_cache	função	() -> None	
face_weights	função	(stack) -> np.ndarray	Area (m^2) of each port node of a face, order (lateral, k).
trace_snapshots	função	(stack, patch=4, shifts=(0.0,), *, global_size=16, global_maps=24) -> dict[str, np.ndarra…)	Interface traces the face modes are built from (cached per stack), two groups of equal weight:
layered_modes	função	(stack, orient, m, shifts=(0.0,), *, groups='layer') -> np.ndarray	m face modes that never tie two layers together: each layer gets its own constant, and the remaining m - n_layers modes are the leading per-layer POD modes of the traces (:func:trace_snapshots), allotted greedily by singular value across layers. A mode spanning several layers would fix ...

nome	tipo	assinatura	o que faz
interface_modes	função	(bm, orient, m, method='pod', shifts=(0.0,)) -> np.ndarray	m modes of a shared face (orient "x" or "y"), columns orthonormal in the area-weighted inner product, the first one constant. method="pod" (default): the rest are the leading POD modes of the traces of :func:trace_snapshots (load-informed Ritz-type modes); "cc": the ...
reference_observables	função	(pkg, T) -> dict[str, np.ndarray]	The observables of :meth:Condensed.observables from a full cell field (nx, ny, nz) (same rows).
energy_norm	função	(K, e) -> float	
errors	função	(obs, ref, total_power) -> dict[str, float]	Error of observables against a reference: Tmax and junction temperatures (K), interface heats relative to the reference total of that interface, and the energy balance (outflow - power, relative).
tmax_cells	função	(T, active_k) -> tuple[float, float]	(max over every cell, max over the active row): equal when all heat is generated on the active row with non-negative power (discrete maximum principle: every other cell is a weighted mean of its neighbours).
iterate_refinement	função	(pkg, m, tol, *, fraction=0.5, max_steps=6, power=None)	Solve, estimate, refine until sqrt(sum eta^2) <= tol * sqrt(t^T A t) (relative energy estimate).

pinneapple_condense.transient

Transient conduction: Craig-Bampton condensation of the blocks, implicit Euler, and the fine reference.

pinneapple_condense/transient.py · 199 linhas · testes: 14

nome	tipo	assinatura	o que faz
CBBlock	classe	(K, M, W, G, Ot, Op, lam)	
CBTransient	classe	(pkg, m=8, k=8, shifts=TRANSIENT_SHIFTS)	Craig-Bampton model of a package (m interface modes per face, k interior modes per block).
cb_block	função	(pkg_stack, variant, Phi_f, k) -> CBBlock	Craig-Bampton reduction of one block onto Phi_f interface modes and k fixed-interface modes.
reference_transient	função	(pkg, powers, dt, every=1) -> dict	Fine-grid implicit Euler with the same step (C / dt + K factorised once, or AMG-CG for big grids).

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 15 de 18.
- Funções de teste em arquivos que importam algum módulo do pacote: 27.

Limitações conhecidas

- *Nenhuma registrada.*

6. `pinneapple_query`

Objetivo

`pinneapple_query` é o motor genérico de consultas físicas (Physics Query Engine). A ideia: pedir uma grandeza física (por exemplo `T`) num lugar físico estável (por exemplo `die@0.3,0.7`), sob certas condições (por exemplo um mapa de potência), com uma precisão-alvo. A resposta traz o valor, a unidade, a incerteza, o domínio de validade, a proveniência e o método que a produziu, e diz se o valor foi *lido* de um estado já resolvido (`read`), *previsto* para condições não resolvidas (`predicted`) ou obtido por uma nova solução (`solved`).

O problema que resolve é escolher, a cada pergunta, o caminho mais barato que ainda cumpre a precisão pedida. Um `Planner` pergunta a cada `Method` registrado por uma `Offer` (se pode responder, a que custo esperado, com que limite de erro, dentro de que validade) e escolhe a oferta válida mais barata cujo limite atende o alvo; se nenhuma atende, fica com a de menor limite de erro; um método de último recurso (`fallback`, tipicamente uma solução completa) só entra quando ninguém oferece. O `Engine` guarda um log de cada resposta (método, tipo, latência) para auditoria e aceita ganchos (`hooks`).

O pacote é pequeno e só tem a parte genérica: `core` (consulta, ofertas, planejador, respostas) e `refs` (referências físicas independentes de malha, como `ubump[12,40]`, `interface:die/tim` ou `lid:top`). Os métodos de uma física concreta ficam junto dela: para o encapsulamento de chips, `pinneapple_condense.query.package_engine` registra os métodos `cached`, `superposition`, `bound`, `condensed` e `reference`. No resto da plataforma, o nó `query.package` de `pinneapple_flow` expõe isso como um passo de fluxo, e `examples/physics_query/run_demo.py` é a demonstração medida (resultado gravado em `examples/physics_query/results/query_demo.json`). É uma linha de pesquisa ("track 2" no CHANGELOG), não uma API estável de `pp.*`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
3	252	7	2	7 (1 arquivos)	2

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`core`	Consulta (`Query`), oferta (`Offer`), resposta (`Answer`), interface de método (`Method`), planejador (`Planner`) e motor (`Engine`).	1	7	0
`refs`	Referências físicas estáveis (`PhysicalRef`) e o parser das suas formas em texto (`parse_ref`).	1	2	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_query`

Fachada do pacote: reexporta `Engine`, `Planner`, `Method`, `Offer`, `Answer`, `Query`, `condition_key`, `PhysicalRef` e `parse_ref`; o docstring mostra o uso com `pinneapple_condense.query.package_engine`.

`pinneapple_query/__init__.py` · 17 linhas · testes: 7

`pinneapple_query.core`

Motor de consultas: o `Planner` coleta ofertas dos métodos e escolhe a mais barata que cumpre a precisão; o `Engine` executa, mede o tempo, registra as ofertas e grava um log de auditoria. `Method` é só interface (levanta NotImplementedError); [sem teste direto] — coberto por `tests/test_query.py` via `pinneapple_condense.query`.

`pinneapple_query/core.py` · 155 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Query`	classe	`(quantity, where, time, conditions, accuracy, rigorous)`	Uma consulta: grandeza, referência física (`where`), tempo, condições, precisão absoluta desejada e se só valem métodos com limite rigoroso; `key` é o hash das condições.
`Offer`	classe	`(method, cost_s, bound, valid, rigorous, why)`	Oferta de um método para uma consulta: custo esperado em segundos, limite de erro esperado, se é válida, se o limite é rigoroso e o porquê.
`Answer`	classe	`(value, unit, uncertainty, kind, method, validity, provenance, seconds, offers, rigorous,…)`	Resposta: valor, unidade, incerteza, tipo (`read`, `predicted` ou `solved`), método, validade, proveniência, tempo gasto, ofertas consideradas e, se rigorosa, o intervalo.
`Method`	classe	`()`	Interface de um modo de responder: `offer` decide barato se pode responder e `answer` calcula; ambos levantam NotImplementedError na classe base.
`Planner`	classe	`(methods, fallback=None)`	Escolhe o método: a oferta válida mais barata que atende a precisão; senão a de menor limite de erro; o `fallback` só quando nenhum método oferece (sem fallback, levanta LookupError).
`Engine`	classe	`(planner, units=None, default_accuracy=None)`	Ponto de entrada `query(quantity, where, ...)`: aplica a precisão padrão por grandeza, consulta o `Planner`, mede a latência, anexa as ofertas, chama os ganchos e grava o log de auditoria.
`condition_key`	função	`(conditions) -> str`	Hash SHA-256 estável (16 caracteres) das condições de uma consulta; arrays entram pelos seus bytes.

`pinneapple_query.refs`

Referências físicas independentes de malha (componente + coordenada fracionária local, índice de bloco, faixa de blocos, interface ou face) e o parser das formas de texto. [sem teste direto] — `tests/test_query.py` testa `parse_ref` pela fachada do pacote.

`pinneapple_query/refs.py` · 80 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicalRef	classe	(component, local, index, span, kind)	Referência física imutável: componente, coordenadas locais fracionárias, índice de feature repetida, faixa de blocos e tipo (point, feature, region, interface ou face:...); str() volta à forma de texto.
parse_ref	função	(where) -> PhysicalRef	Converte textos como die@0.31,0.72, ubump[12,40], interface:die/tim[3:9,0:4], region:die[...] ou lid:top em PhysicalRef; rejeita coordenadas locais fora de [0, 1].

Exemplos de uso reais do repositório

Referências físicas e seus tipos — tests/test_query.py

```
        def test_physical_references():
            r = parse_ref("die@0.25,0.5")
            assert (r.component, r.local, r.kind) == ("die", (0.25, 0.5),
"point")
            assert parse_ref("ubump[3,4]").index == (3, 4) and
parse_ref("ubump[3,4]").kind == "feature"
            i = parse_ref("interface:die/tim[0:2,1:3]")
            assert i.kind == "interface" and i.span == ((0, 2), (1, 3)) and
i.component == "die/tim"
            assert parse_ref("lid:top").kind == "face:top"
            assert str(parse_ref("die@0.25,0.5,1")) == "die@0.25,0.5,1"
            with pytest.raises(ValueError):
                parse_ref("die@1.5,0.2")                                  #
local coordinates are fractions
```

Resolver uma vez, depois ler do estado resolvido — tests/test_query.py

```
        def test_solved_then_read_with_physical_points(engine):
            pkg, eng, model = engine
            a = eng.query("Tj_max", "region:die", accuracy=1e-9)          #
nothing solved: the reference solves
            assert a.kind == "solved" and a.method == "reference"
            b = eng.query("T", "die@0.4,0.6")
            c = eng.query("T", "die@0.4,0.6,0.5")
            assert b.kind == c.kind == "read" and b.method == "cached"
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 1 de 3.
- Funções de teste em arquivos que importam algum módulo do pacote: 7.

Limitações conhecidas

- O pacote só traz a parte genérica: sem métodos concretos ele não responde nada. Hoje os únicos métodos estão em pinneapple_condense.query (encapsulamento de chips).
- Method.offer e Method.answer levantam NotImplementedError: cada física precisa implementar sua própria subclasse.
- Os módulos core e refs não têm teste que os importe diretamente; a cobertura vem de tests/test_query.py, que passa pelos métodos de pinneapple_condense.
- As formas de referência (die@..., ubump[...], tsv[...], c4[...], lid:top) foram pensadas para encapsulamento de chips; outra geometria precisa definir seus próprios nomes de componente.

- A escolha do `Planner` confia nos custos e limites de erro que cada método declara na oferta; o motor não verifica se esses limites são verdadeiros.
- Sem precisão-alvo (nem na consulta nem em `default_accuracy`), o planejador escolhe a oferta de menor limite de erro, que pode ser a mais cara.
- É linha de pesquisa ("track 2" no CHANGELOG), sem entrada em `pp.*`.

Volume 3 — Simulação numérica

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Solvers clássicos (FDM, FEM, FVM, espectral, SPH, LBM), pontes para solvers externos, eletromagnetismo, voo, multicorpos, veículos e rovers.

Sumário

1. pinneapple_simulation

1. `pinneapple_simulation`

Objetivo

Esta parte cobre quatro submódulos de `pinneapple_simulation`. `external_solvers` liga o PINNeAPPle a programas de simulação de fora (OpenFOAM, CalculiX, OpenRadioss, ANSYS Fluent, FEniCS, MATLAB, Modelica/FMI, MuJoCo, Genesis, NVIDIA Newton, FluidX3D, GeoTaichi, TurboDesigner) e lê formatos abertos de CFD/FEM; quase tudo termina num `PhysicalSample` (UPD) para treino e checagem. `flight` é um simulador de voo 6-DOF com aerodinâmica de vortex lattice, trim, piloto automático, decolagem e turbulência. `meshless` compara métodos sem malha (Kansa, RBF-FD, PINN) com FD e FEM no mesmo problema 2-D. `pipelines` encadeia FreeCAD, malha e OpenFOAM num fluxo reproduzível, com validação contra correlações publicadas, varredura e surrogate.

Esta parte cobre a física de corpos e partículas do `pinneapple_simulation`: dinâmica multicorpo própria (`multibody`), simuladores de partículas em PyTorch (`particle_dynamics`: corpos rígidos, MPM, GeoMPM, SPH, dinâmica molecular), robôs no NVIDIA Newton com checagens independentes (`robot_sim`), terramecânica de rodas planetárias (`terramechanics`) e o rover completo em terreno deformável (`vehicle`, com o VIPER). O fio comum é sempre ter uma referência independente para checar o resultado: modelo analítico, balanço de energia, dados publicados ou substituto PINN comparado com o modelo de origem.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
298	66.085	364	931	1.119 (109 arquivos)	113

Pasta de testes dedicada: `tests/pinneapple_simulation` (16 funções de teste).

Submódulos

sub-mó-dulo	o que faz	mó-du-los	clas-ses + fun-ções	tes tes *
accelerated_foam		9	46	20
electromagnetics		56	345	88
external_solvers	Pontes para solvers e ferramentas externas (execução, montagem de casos, leitura de resultados) e leitores de formatos CFD/FEM, com saída em PhysicalSample.	66	192	151
flight	Dinâmica de voo 6-DOF: aeronave rígida com vortex lattice, motores, trem de pouso, trim, controle, decolagem, turbulência e cenas 3-D.	17	108	19
geophysics		4	19	11
meshless	Métodos sem malha e com malha resolvendo os mesmos problemas 2-D, e o estudo de refinamento que os compara.	8	19	22
multibody	Árvore multicorpo de base flutuante com juntas de revolução e utilitários de braço de base fixa.	3	18	45
numerical_solvers		73	295	206
particle_dynamics	Simuladores de partículas diferenciáveis em PyTorch: corpos rígidos, MPM, GeoMPM, SPH e dinâmica molecular.	8	21	151
pipelines	Pipelines de ponta a ponta que encadeiam CAD (FreeCAD), malhador e solver (OpenFOAM) num fluxo checado e reproduzível.	14	59	14
robot_sim	Robôs simulados no NVIDIA Newton (com backend Isaac Sim) e checados contra o modelo multicorpo próprio.	6	17	39
rover		17	88	21
terramechanics	Contato roda-solo para rovers: solos, Wong-Reece, terreno deformável, bancada e substitutos PINN.	8	38	34
vehicle	Veículo planetário com rodas (VIPER) do spec ao CAD, simulação, dashboard e animação.	8	28	38

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_simulation`

pinneapple_simulation — Physics simulation back-ends and data generation.

`pinneapple_simulation/__init__.py` · 163 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
simulate	função	`(scenario, **kwargs) -> Any`	Run a named physics scenario and return its output.
generate_data	função	`(scenario, n_samples=10, **kwargs) -> list`	Generate a list of simulation outputs for training data.

`pinneapple_simulation.accelerated_foam`

Faster implicit pressure solves for OpenFOAM, verified against plain OpenFOAM.

pinneapple_simulation/accelerated_foam/__init__.py · 41 linhas · testes: 15

pinneapple_simulation.accelerated_foam.benchmark

The comparisons behind docs/core_concepts/openfoam_acceleration.md.

pinneapple_simulation/accelerated_foam/benchmark.py · 387 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
openfoam_comparison	função	(work_dir, lib, n=128, end_time=0.5, inners=('PCG', 'GAMG'), tolerance=1e-06, rel_tol=0.0…)	Lid-driven cavity (icoFoam), plain vs warm-started pressure solves; one row per run.
solver_entries	função	(fv_solution, field_p) -> dict	The key value; entries of solvers { <field_p> { ... } } in an fvSolution text.
set_solver_entries	função	(fv_solution, field_p, entries) -> str	Replace the body of solvers { <field_p> {...} }; other entries ($p in pFinal) are untouched.
prepare_tutorial	função	(tutorial_dir, case_dir, p_solver, field_p, libs=(), setup=('blockMesh',), end_time=None,…)	Copy an installed tutorial, set the solver of field_p (the Final entry inherits it when the tutorial writes it as $p), write fields in ASCII with 17 digits, and run the setup commands. drop_functions removes the tutorial's function objects (post-processing only; some packaged …
read_solver_dict	função	(tutorial_dir, field_p) -> dict	The tutorial's own solver controls of field_p (strings).
tutorial_comparison	função	(tutorial_dir, work_dir, lib, application, field_p, compare_fields=('U',), setup=('blockM…)	An installed tutorial run three ways: its own field_p solver (plain), the same solver wrapped by pinneWarmStart (warm), and a reference with tolerance ref_tolerance; relTol 0.
offline_benchmark	função	(systems, guesses, solver_specs, tolerance=1e-06, rel_tol=0.0) -> list[dict]	Re-solve a run's systems in order with every (guess, solver) pair.
default_solver_specs	função	(include_amg=True, include_torch=True) -> dict	
default_guesses	função	(n_snapshots=8) -> dict	
markdown_table	função	(rows, columns=None) -> str	

pinneapple_simulation.accelerated_foam.build

Compile the pinneWarmStart linear-solver library against an installed OpenFOAM.

pinneapple_simulation/accelerated_foam/build.py · 155 linhas · testes: 15

nome	tipo	assinatura	o que faz
include_dirs	função	(openfoam_src=None, work=None) -> list[Path]	Header directories for OpenFOAM and OSspecific/POSIX: from $FOAM_SRC lnInclude when present, else built from a source tree. Raises FileNotFoundError when neither is available.
compile_flags	função	(label_size=None, precision=None, api=None) -> list[str]	
build_plugin	função	(openfoam_src=None, out_dir=None, force=False, cxx=None) -> Path	Compile libpinneWarmStart.so; return its path (cached unless force).
find_plugin	função	(openfoam_src=None) -> Path | None	$PINNEAPPLE_WARMSTART_LIB if set, else a build (cached); None when it cannot be built.

pinneapple_simulation.accelerated_foam.case

Small OpenFOAM cases for the pressure-solve benchmarks, a runner and a log parser.

pinneapple_simulation/accelerated_foam/case.py · 395 linhas · testes: 20

nome	tipo	assinatura	o que faz
FieldSolves	classe	(n_solves, iterations, per_solve, initial_residuals, final_residuals)	Linear solves of one field, from a solver log.
RunResult	classe	(case_dir, wall_time, execution_time, solves, log)	
find_bashrc	função	(bashrc=None) -> str | None	The OpenFOAM environment script: bashrc if given, else $FOAM_BASHRC, else a usual location.
openfoam_available	função	(solver='icoFoam') -> bool	True when solver can be run (on PATH, or through an environment script).
plain_solver	função	(kind='PCG', tolerance=1e-06, rel_tol=0.0, **extra) -> dict	OpenFOAM's own pressure solver: PCG (DIC preconditioner) or GAMG (GaussSeidel smoother).
warm_solver	função	(inner='PCG', tolerance=1e-06, rel_tol=0.0, guess='projection', n_snapshots=8, **extra) -…)	The pinneWarmStart wrapper around the same inner solver and tolerances as :func:plain_solver.
write_cavity	função	(case_dir, n=64, end_time=0.25, nu=0.01, lid=1.0, courant=0.5, p_solver=None, p_final_sol…)	Write a 2-D lid-driven cavity (unit square, n x n cells) run by icoFoam.
run_command	função	(case_dir, command, bashrc=None, log_name=None, env=None) -> tuple[str, float]	Run an OpenFOAM command in case_dir; return (log text, wall seconds). Raises on failure.
parse_log	função	(log) -> tuple[dict[str, FieldSolves], float]	Per-field linear solves and the final ExecutionTime of an OpenFOAM solver log.
run_case	função	(case_dir, solver='icoFoam', bashrc=None, mesh=True, env=None) -> RunResult	blockMesh (if mesh) then solver; the solver log is parsed into a :class:RunResult.
time_dirs	função	(case_dir) -> list[str]	
read_field	função	(case_dir, name, time_name=None) -> np.ndarray	Internal field name at time_name (default: latest), as a NumPy array.

nome	tipo	assinatura	o que faz
field_difference	função	(case_a, case_b, name, time_name=None) -> float	Relative L2 difference ||a - b|| / ||a|| of a field between two runs at the same time.

pinneapple_simulation.accelerated_foam.guess_server

Serve initial guesses from Python to a running OpenFOAM solver (pinneWarmStart with guess external).

pinneapple_simulation/accelerated_foam/guess_server.py · 138 linhas · testes: 15

nome	tipo	assinatura	o que faz
GuessServer	classe	(pipe_dir, guessers)	Answers guess requests on pipe_dir/request with guessers[field] in a background thread.

pinneapple_simulation.accelerated_foam.ldu

OpenFOAM lduMatrix systems in Python: read the systems written by pinneWarmStart and convert them.

pinneapple_simulation/accelerated_foam/ldu.py · 197 linhas · testes: 15

nome	tipo	assinatura	o que faz
LduSystem	classe	(lower_addr, upper_addr, diag, upper, source, psi0, psi, lower, meta, _csr)	One linear system A x = b as OpenFOAM handed it to the linear solver.
from_csr	função	(A, b) -> LduSystem	An :class:LduSystem from a symmetric sparse matrix (upper triangle = faces), for tests and synthetic cases.
norm_factor	função	(system, x, Ax=None) -> float	OpenFOAM's residual normalisation: sum(|A x - A1 xbar| + |b - A1 xbar|) + small.
normalised_residual	função	(system, x, nf=None) -> float	sum|b - A x| / normFactor -- the residual OpenFOAM prints and tests against tolerance.
read_system	função	(directory) -> LduSystem	Read one system written by pinneWarmStart (dumpDir): meta.txt and system.bin.
read_dump	função	(dump_dir, field_name='p') -> list[LduSystem]	All systems of field_name in a dump directory, in solve order.

pinneapple_simulation.accelerated_foam.learned

A learned correction of the initial guess: a small graph network on the matrix graph (PyTorch).

pinneapple_simulation/accelerated_foam/learned.py · 162 linhas · testes: 15

nome	tipo	assinatura	o que faz
GraphGuessNet	classe	(hidden=32, layers=4, seed=0)	Message-passing network h <- tanh(W_self h + W_nb (Â h)) on the normalised matrix graph.
LearnedWarmStart	classe	(n_snapshots=8, hidden=32, layers=4, seed=0)	Projection guess + learned correction (safeguarded). Same interface as the other warm starts.
projection_inputs	função	(systems, n_snapshots=8) -> list[np.ndarray]	The projection guess of every system of a run, from the run's own converged solutions (psi).

pinneapple_simulation.accelerated_foam.solvers

Krylov solvers for extracted pressure systems, with OpenFOAM's stopping test.

pinneapple_simulation/accelerated_foam/solvers.py · 230 linhas · testes: 15

nome	tipo	assinatura	o que faz
SolveResult	classe	(x, iterations, initial_residual, final_residual, wall_time, converged, history, setup_ti…)	
jacobi	função	(system)	
dic	função	(system)	OpenFOAM's DIC: M = (D* + L) D*^-1 (D* + U) with D* from the face sweep, applied by two sparse triangular solves.
amg	função	(system, **kwargs)	Smoothed-aggregation AMG V-cycle from pyamg (MIT licence; optional) applied to -A (SPD).
pcg	função	(system, x0=None, preconditioner='dic', tolerance=1e-06, rel_tol=0.0, max_iter=1000, min_…)	Preconditioned CG, OpenFOAM's PCG algorithm and stopping test, in NumPy.
torch_pcg	função	(system, x0=None, tolerance=1e-06, rel_tol=0.0, max_iter=1000, device=None, dtype='float6…)	Jacobi-preconditioned CG in PyTorch (GPU when available).

pinneapple_simulation.accelerated_foam.warm_start

Initial guesses for a sequence of pressure systems.

pinneapple_simulation/accelerated_foam/warm_start.py · 121 linhas · testes: 15

nome	tipo	assinatura	o que faz
ProjectionWarmStart	classe	(n_snapshots=8)	Galerkin projection of each new system on the span of the last n_snapshots solutions and of the incoming field. Call :meth:guess before a solve and :meth:update with the converged solution after it.
PODWarmStart	classe	(energy=0.99999, max_rank=32, n_recent=0)	Galerkin projection on a POD basis of stored solutions (other runs or earlier steps) and the current field.
galerkin	função	(system, basis, drop=1e-12) -> np.ndarray | None	x = V c with (V^T A V) c = V^T b; dependent columns of V (n, k) are dropped. None if empty.

pinneapple_simulation.electromagnetics

Electromagnetics and electrodynamics in SI: references, materials, solvers and machines.

pinneapple_simulation/electromagnetics/__init__.py · 67 linhas · testes: 88

pinneapple_simulation.electromagnetics.analytic

Closed-form references for verifying the 2D A_z solvers.

pinneapple_simulation/electromagnetics/analytic.py · 226 linhas · testes: 22

nome	tipo	assinatura	o que faz
`Layer`	classe	`(r_in, r_out, mu_r, Br_cos, Br_sin, J_cos, J_sin)`	Annulus `r_in < r < r_out` with relative permeability `mu_r`; radial remanence `Br_cos cos(p phi) + Br_sin sin(p phi)` and current density `J_cos cos(p phi) + J_sin sin(p phi)`.
`RadialLayers`	classe	`(layers, p)`	Exact solution for one harmonic `p >= 1` in concentric layers; `A = 0` at the outer radius, regular at `r = 0` if the first layer starts there (otherwise `dA/dr = 0`, an ideal iron wall).
`wire_field`	função	`(r, current, radius, R_outer)`	(A_z, |B|) of a round wire (uniform J) centred in a circle of radius `R_outer` with A=0.
`coax_field`	função	`(r, current, a, b, c)`	|B| of a coax: inner conductor radius `a` (+I), outer conductor `b < r < c` (-I).
`magnetized_cylinder`	função	`(x, y, Br, a, R, alpha=0.0)`	A_z of a cylinder (radius `a`, `mu_r = 1`, remanence `Br` along angle `alpha`) in air with `A = 0` at radius `R`. Equivalent surface current `K = M sin(phi - alpha)`; returns (A, B_inside).
`magnetized_ring`	função	`(x, y, Br, a, b, r0, R, alpha=0.0)`	A_z of a uniformly magnetised ring `a < r < b` (`mu_r = 1`, remanence `Br` along `alpha`) in air between two circles with `A = 0`: `r0 < a` and `R > b`. Superposition of two cylinders (:func:`magnetized_cylinder`, radii `b` and `a`) plus the harmonic ``C (1/r - r/R^2) sin(phi - …
`skin_effect_wire`	função	`(r, current, radius, sigma, omega, mu=MU0)`	Phasor current density J(r) and impedance per metre Z = R + j omega L_int of a round wire.
`round_conductor_eddy_loss_low_freq`	função	`(B_peak, omega, sigma, a) -> float`	Time-averaged eddy loss per metre in a round conductor (radius `a`, no net current) in a uniform transverse field `B_peak sin(omega t)`, resistance limited (field of the eddy currents neglected): `P = pi sigma omega^2 B^2 a^4 / 8` W/m.

pinneapple_simulation.electromagnetics.antenna

Printed end-fire antennas (antipodal Vivaldi) with an external FEM solver: limits, checks, runner, surrogates.

`pinneapple_simulation/electromagnetics/antenna/__init__.py` · 30 linhas · testes: 31

pinneapple_simulation.electromagnetics.antenna.ava

An open parametric antipodal Vivaldi antenna (AVA) for the EMerge FEM solver, written from scratch.

`pinneapple_simulation/electromagnetics/antenna/ava.py` · 386 linhas · testes: 31

nome	tipo	assinatura	o que faz
`InvalidGeometry`	classe	`()`	The parameters do not describe a buildable antenna (overlapping metal, slots through the arm, …).

nome	tipo	assinatura	o que faz
AVAParams	classe	(Lb, W, Wa, R, xs, Wg, Lo, nslot, Ls, ws, ps, dir_a, dir_b, dir_x, res, er, tand, h, wf, …)	Design parameters, millimetres (R in 1/mm, res and air in wavelengths).
Layout	classe	(params, top, bottom, director, wf, min_metal)	Conductor polygons (mm, closed, counter-clockwise not guaranteed) and the derived dimensions.
microstrip_width	função	(z0, er, h) -> float	Width of a microstrip of impedance z0 on a substrate er, h (Pozar eq. 3.197, same unit as h).
layout	função	(p) -> Layout	The conductor polygons of design p; :class:InvalidGeometry if it cannot be built.
use_superlu	função	() -> None	Force EMerge's SuperLU direct solver.
nearfield_planes	função	(p) -> dict[str, dict[str, np.ndarray]]	The two near-field cut planes of design p (mm): board (z = h/2, the substrate mid-plane) and axis (x = 0, through the end-fire axis), each {"origin", "u", "v", "nu", "nv"}: points origin + i/(nu-1) u + j/(nv-1) v. The extent is the board plus nf_margin on every side, the …
plane_points	função	(pl) -> np.ndarray	(nv, nu, 3) points of a plane of :func:nearfield_planes (mm).
simulate	função	(params, freqs_ghz, out_dir=None, *, superlu=True, log='WARNING') -> list[dict[str, Any]]	Build the design in EMerge, solve each frequency (one mesh per frequency, size res * lambda) and return one row per frequency (the columns of :mod:.runner's table).
main	função	(argv=None) -> int	

pinneapple_simulation.electromagnetics.antenna.bounds

Physical limits on the directivity of a travelling-wave (end-fire) or aperture (broadside) antenna.

pinneapple_simulation/electromagnetics/antenna/bounds.py · 87 linhas · testes: 23

nome	tipo	assinatura	o que faz
to_dbi	função	(x)	
from_dbi	função	(g)	
wavelength	função	(f_hz)	Free-space wavelength [m] at f_hz.
endfire_directivity	função	(length_m, f_hz, kind='hw')	Maximum directivity (linear) of an end-fire antenna of length length_m at f_hz.
endfire_directivity_dbi	função	(length_m, f_hz, kind='hw')	
aperture_gain	função	(area_m2, f_hz, efficiency=1.0)	Broadside aperture gain (linear) eta 4 pi A / lambda^2.
aperture_gain_dbi	função	(area_m2, f_hz, efficiency=1.0)	

nome	tipo	assinatura	o que faz
`min_length`	função	`(gain_dbi, f_hz, kind='hw')`	Shortest end-fire length [m] whose directivity limit reaches `gain_dbi` at `f_hz`.
`feasibility_frequency`	função	`(gain_dbi, length_m, kind='hw')`	Lowest frequency [Hz] at which an end-fire antenna of length `length_m` can reach `gain_dbi`.

pinneapple_simulation.electromagnetics.antenna.checks

Consistency checks of one antenna result, as :class:`pinneapple_analysis.trust.Check` / `TrustReport`.

`pinneapple_simulation/electromagnetics/antenna/checks.py` · 172 linhas · testes: 23

nome	tipo	assinatura	o que faz
`check_realized_gain`	função	`(g_dbi, g_real_dbi, s11_db, *, tol_db=0.1) -> Check`	`G_realized = G + 10 log10(1 - |S11|^2)` within `tol_db`.
`check_efficiency_ratio`	função	`(g_dbi, d_dbi, eff, *, tol_db=0.1) -> Check`	`e_rad = G / D` within `tol_db`.
`check_efficiency_le_one`	função	`(eff, *, tol=0.01) -> Check`	Radiation efficiency at most 1 (+ `tol` for the far-field quadrature). Critical.
`check_peak_ge_boresight`	função	`(peak_dbi, bore_dbi, *, tol_db=0.05) -> Check`	
`check_directivity_limit`	função	`(d_dbi, length_m, f_hz, *, margin_db=0.5, kind='hw') -> Check`	Directivity below the end-fire limit of :mod:`.bounds` for a length `length_m` (+ `margin_db`).
`check_s11_resolved`	função	`(n_solved, *, interpolated, min_points=5) -> Check`	An S11 from a rational (vector-fit) interpolation needs at least `min_points` solved frequencies.
`check_power_closure`	função	`(p_pattern, p_surface, *, rtol=0.1) -> Check`	Far-field power (integral of the pattern) against the near-field Poynting flux through the same surface.
`antenna_checks`	função	`(row, *, length_m=None, n_solved=None, interpolated=False, min_points=5, tol_db=0.1, marg…)`	Every check above on one table row (the columns of :mod:`.runner`).

pinneapple_simulation.electromagnetics.antenna.design

Design loop of a printed end-fire antenna: DOE, multi-objective optimisation, active learning with solver verification, multi-fidelity bias correction and a trust card per prediction.

`pinneapple_simulation/electromagnetics/antenna/design.py` · 326 linhas · testes: 23

nome	tipo	assinatura	o que faz
`OptimizationOutcome`	classe	`(designs, Gmin, S11max, area_cm2, n_evals, wall_s, method, notes)`	
`ALRound`	classe	`(round, n_train_designs, new_tags, rmse_prospective, mean_std, coverage95)`	

nome	tipo	assinatura	o que faz
`ava_transform`	função	`(sample, *, fixed=None) -> dict[str, Any]`	A raw sample of :data:`AVA_RANGES` to AVA parameters: `dir_x` from a fraction of `Lb` to mm, director off when `dir_a < 4` mm, slot fields ignored when `nslot = 0`; `fixed` overrides (e.g. `res`).
`is_buildable`	função	`(params) -> bool`	
`training_box`	função	`(designs, keys=('Lb', 'W', 'Wa'), ranges=None) -> dict[str, tuple[float, float]]`	`ranges` with `keys` narrowed to the span of `designs`: optimise where the surrogate was trained.
`doe`	função	`(n, ranges=None, *, method='lhs', seed=0, transform=ava_transform, valid=is_buildable, ma…)`	`n` valid designs from a space-filling sample; returns `(designs, {"drawn": .., "invalid": ..})`.
`band_metrics`	função	`(models, params, freqs, *, gain='Greal_bore_dBi', s11='S11_dB') -> dict[str, np.ndarray]`	Predicted minimum gain, its std at the argmin, and maximum S11 over `freqs` for each design.
`optimize`	função	`(models, freqs, *, ranges=None, fixed=None, budget=600, seed=0, method='nsga2', s11_max=-…)`	Multi-objective search on the surrogates `models[gain]` and `models[s11]` (fit beforehand).
`active_learning`	função	`(simulate, table, freqs, *, target='Greal_bore_dBi', rounds=2, per_round=2, ranges=None, …)`	Variance-driven active learning with verification by the simulator.
`multi_fidelity`	função	`(coarse, fine, target) -> dict[str, Any]`	Coarse-to-fine bias of `target` per frequency, validated leaving one fine design out.
`prediction_card`	função	`(model, params, f_ghz, train, *, target='Greal_bore_dBi', std_max_db=1.0, lodo_rmse=None,…)`	Trust card of one surrogate prediction (mean, std in `details`).

`pinneapple_simulation.electromagnetics.antenna.package`

Read an external study package from a local folder and re-run one of its designs.

`pinneapple_simulation/electromagnetics/antenna/package.py` · 83 linhas · testes: 23

nome	tipo	assinatura	o que faz
`Package`	classe	`(root, table_path, table, script, argv, flags, length_col)`	
`load_package`	função	`(root) -> Package`	The package in folder `root` (see the module docstring).

`pinneapple_simulation.electromagnetics.antenna.plots`

Figures of an end-fire antenna study (matplotlib; each function draws on an `ax` and returns it).

`pinneapple_simulation/electromagnetics/antenna/plots.py` · 87 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
directivity_vs_length	função	(table, ax=None, *, column='D_peak_dBi', target_dbi=10.0, length_col='Lb', label=None)	column against L / lambda with the Hansen-Woodyard and ordinary end-fire limits and the target.
lodo_rmse_bars	função	(results, ax=None, *, title='leave-one-design-out RMSE')	Bars of RMSE per model for one or more targets: {target: ValidationResult}.
parity_plot	função	(result, model, ax=None)	Held-out prediction against the simulated value for one model of a ValidationResult.
layout_plot	função	(lay, ax=None, *, colors=…)	Top (red), bottom (blue) and director (green) conductors of an :class:.ava.Layout.

pinneapple_simulation.electromagnetics.antenna.runner

Drive an external parametric simulator script, one process per sample (and per frequency), and read its tables.

pinneapple_simulation/electromagnetics/antenna/runner.py · 309 linhas · testes: 34

nome	tipo	assinatura	o que faz
RunRecord	classe	(tag, freqs, status, returncode, wall_s, max_rss_mb, rows, message, retried)	One process: which design and frequencies, how it ended, what it cost.
CLISimulator	classe	(argv, workdir, *, per_frequency=True, timeout=3600.0, params_flag='--params', freq_flag=…)	Run argv + [params_flag, json, freq_flag, f..., out_flag, dir, tag_flag, tag] per design.
ResultTable	classe	(rows)	Rows of (design, frequency); columns are the union of the rows' keys.
ava_simulator	função	(workdir, **kw) -> CLISimulator	A :class:CLISimulator on the open AVA model of :mod:.ava (SuperLU, one frequency per process).
read_table	função	(path) -> ResultTable	Read a results table (CSV, TSV or whitespace separated, one header line, # comments skipped).
write_table	função	(table, path) -> str	

pinneapple_simulation.electromagnetics.antenna.scene3d

3-D views of an antipodal Vivaldi from its solved fields, on the generic studio scene of PINNeAPPle.

pinneapple_simulation/electromagnetics/antenna/scene3d.py · 333 linhas · testes: 8

nome	tipo	assinatura	o que faz
FarField	classe	(f_ghz, theta, phi, U, P_rad, P_acc)	
antenna_surfaces	função	(params, *, copper_mm=COPPER_MM)	Substrate and copper of the design as studio Surface objects (mm, group "antenna").
read_farfield	função	(path) -> FarField	A far-field file of :mod:.ava (theta, phi 1-D; E_theta, E_phi on the (phi, theta) grid).

nome	tipo	assinatura	o que faz
farfield_metrics	função	(ff) -> dict[str, float]	Peak and boresight (+y) directivity and gain of the far-field grid, in dBi.
balloon	função	(ff, centre, radius, *, g_max=None, dyn_db=30.0, quantity='gain')	Radiation balloon (vertices, faces, per-vertex fields gain_dBi and directivity_dBi).
read_nearfield	função	(path) -> dict[str, dict[str, np.ndarray]]	{plane: {"points": (nv, nu, 3) mm, "E": (3, nv, nu) complex}} of a near-field file of :mod:.ava.
plane_surface	função	(points, name, group='cuts')	A studio Surface of a structured (nv, nu, 3) grid of points.
solved_files	função	(run_dir) -> dict[float, dict[str, str]]	{f_GHz: {"ff": path, "nf": path?}} of every far-field file under run_dir (any depth).
frequency_scene	função	(params, files, *, rows=(), dyn_db=30.0, title='') -> tuple[Any, dict]	Antenna, gain balloon and |E| cuts over the solved frequencies of files (:func:solved_files).
phase_scene	função	(params, nf_path, *, n_phase=24, component=0, title='')	The cut planes at one frequency over one period: Re(E_c e^{j wt}) over the 99.5th percentile of |E_c| (clipped to +-1) on jet, wt in degrees (component 0 = x, the polarisation of the AVA aperture field).
build_3d	função	(params, run_dir, out_dir, *, rows=(), name='design', render=True, size=(960, 540), sampl…)	Every 3-D output of one design from the files of its runs (run_dir) into out_dir:

pinneapple_simulation.electromagnetics.antenna.study

End-to-end studies of a printed end-fire antenna, as library calls (the example and the Flow Studio call these).

pinneapple_simulation/electromagnetics/antenna/study.py · 228 linhas · testes: 8

nome	tipo	assinatura	o que faz
limits_report	função	(target_dbi=10.0, f_min_ghz=2.0, f_max_ghz=4.0, n=5, lengths_mm=(150.0, 206.0, 260.0)) ->…)	Shortest end-fire length for target_dbi at n frequencies of the band (Hansen-Woodyard and ordinary) and the lowest frequency each of lengths_mm can reach it.
check_summary	função	(table, *, n_solved=None, interpolated=False, margin_db=0.5) -> dict[str, Any]	Counts of check outcomes and decisions over every row, and how often each check failed.
validate_targets	função	(table, targets=TARGETS) -> dict[str, sg.ValidationResult]	Leave-one-design-out of the default surrogates for each target present in table.
study_figure	função	(table, vals, path, *, title='', target_dbi=10.0, column='D_peak_dBi') -> str	Directivity against L/lambda with the end-fire limits and the target, and the LODO RMSE per model.

nome	tipo	assinatura	o que faz
open_study	função	(out, *, freqs=(2.0, 2.5, 3.0, 3.5, 4.0), n_designs=16, n_fine=4, coarse_res=0.25, fine_r…)	The open-model study (needs EMerge). Writes tables, figures and summary.json to out.
package_study	função	(path, out, *, rerun=None, rerun_freqs=1, interpolated=False, timeout=3600.0, target_dbi=…)	Reproduce an external package: checks of every row, limits, LODO, figures, and a re-run of design rerun (default: the first; "none" to skip) at its first rerun_freqs frequencies.
render_3d	função	(designs, out, *, freqs=(2.0, 2.5, 3.0, 3.5, 4.0), res=0.25, nf_step=2.0, timeout=3600.0,…)	Solve each named design at freqs with its near field (nf_step mm on the two cut planes; one EMerge process per frequency) and write its 3-D outputs into out (:func:.scene3d.build_3d). Returns, per design, the files, the run cost, the checks of its rows and the largest difference …

pinneapple_simulation.electromagnetics.antenna.surrogates

Surrogates of an antenna quantity over (design, frequency), and honest validation of them.

pinneapple_simulation/electromagnetics/antenna/surrogates.py · 301 linhas · testes: 23 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
FrequencyMean	classe	()	Mean of the training targets at each frequency (linear interpolation between frequencies).
HWOffset	classe	()	Hansen-Woodyard limit plus one fitted offset (dB).
LinearPhysics	classe	(ridge=0.001)	Ridge regression on :func:physical_features (standardised; constant features dropped).
GPModel	classe	(prior='hw', restarts=4, seed=0, features=None)	GP on :func:physical_features with the HW limit as prior mean (prior="hw") or a constant.
ValidationResult	classe	(target, scheme, rows, failed)	Out-of-sample predictions of each model; rmse[model] in the target's unit.
physical_features	função	(params, f_ghz) -> np.ndarray	(n, 5) features of each (design, frequency): log(L/lambda), W/lambda, Wa/lambda, nslot > 0 and dir_a > 0. Lengths in mm (Lb, W, Wa).
columns	função	(*names)	A features function for :class:GPModel: the named design parameters as they are.
default_models	função	() -> dict[str, Callable[[], _Model]]	
lodo	função	(table, target, models=None) -> ValidationResult	Leave-one-design-out: each design (all its frequencies) predicted by models trained on the others.
holdout	função	(table, target, test_tags, models=None) -> ValidationResult	Train on every design not in test_tags; predict those.

pinneapple_simulation.electromagnetics.constants

Physical constants of electromagnetics, SI, CODATA 2022 (NIST, https://physics.nist.gov/cuu/Constants/).

`pinneapple_simulation/electromagnetics/constants.py` · 29 linhas · testes: 36

pinneapple_simulation.electromagnetics.eddy

Time-harmonic 2D eddy currents in A_z with solid conductors (linear media).

`pinneapple_simulation/electromagnetics/eddy.py` · 93 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TimeHarmonicSolution`	classe	`(A, u, J_nodal, loss, geometry)`	
`solve_time_harmonic`	função	`(mesh, nu, sigma, omega, conductors, currents, J_source=None, dirichlet=None, dirichlet_v…)`	Solve for A (complex). `conductors`: element masks of solid conductors; `currents`: their net current phasors (A). `sigma` (M,) is used inside conductors only.

pinneapple_simulation.electromagnetics.electrostatics

2-D electrostatics on P1 triangles: `-div(eps grad phi) = rho` with electrodes at fixed potentials.

`pinneapple_simulation/electromagnetics/electrostatics.py` · 61 linhas · testes: 26

nome	tipo	assinatura	o que faz
`ElectrostaticSolution`	classe	`(phi, E, eps, energy, geometry)`	
`solve_electrostatic`	função	`(mesh, eps_r, electrodes=None, rho=None, dirichlet=None, dirichlet_values=None) -> Electr…)`	Potential for element relative permittivities `eps_r` (scalar or (M,)), charge density `rho` (C/m^3 per element) and electrodes `{name: (node indices, potential)}` (or raw `dirichlet` nodes and values). Nodes in no electrode carry the natural condition `D.n = 0` (a symmetry plane or an …
`capacitance`	função	`(sol, V) -> float`	`C = 2 W / V^2` (F/m) for one electrode at `V` and the others grounded, no free charge.

pinneapple_simulation.electromagnetics.fdtd

Finite-difference time-domain (FDTD) solution of Maxwell's curl equations on the Yee grid, in 1-D, 2-D and 3-D.

`pinneapple_simulation/electromagnetics/fdtd.py` · 912 linhas · testes: 10

nome	tipo	assinatura	o que faz
`YeeGrid`	classe	`(shape, spacing, courant, dt, allow_unstable)`	Node counts `shape = (nx, ny, nz)` (use 1 for invariant axes), spacings and the time step (`courant` is the fraction of the stability limit).

nome	tipo	assinatura	o que faz
`Probe`	classe	`(component, index, values, times)`	
`DFTMonitor`	classe	`(components, freqs, region, data)`	Running DFT `F(omega) = sum_n f(t_n) exp(-j omega t_n) dt` of components over a region (slices).
`PlaneWaveTFSF`	classe	`(axis, polarization, waveform, box)`	Total-field / scattered-field box `box = ((i0, i1), (j0, j1), (k0, k1))` (node index ranges, inclusive of i0, exclusive of i1) for a plane wave travelling along +`axis` with electric field `polarization` and waveform `waveform(t)` of the incident E at the box's first plane.
`FDTD`	classe	`(grid, boundaries=None, *, npml=10, pml_order=3.0, pml_kappa_max=1.0, pml_alpha_max=None,…)`	Yee leapfrog solver. `boundaries` maps `"x"`, `"y"`, `"z"` to a kind or a `(low, high)` pair of `"pec"`, `"pmc"`, `"periodic"`, `"bloch"` or `"cpml"`; `bloch` takes `bloch_k` (rad/m) per axis.
`NearToFarField`	classe	`(sim, freqs, box)`	Near-to-far-field transform on a closed box of node planes (3-D grids).
`fill_fraction`	função	`(grid, inside, oversample=8) -> np.ndarray`	Fraction of each node's cell (the box of one spacing centred on the node, along the active axes) for which `inside(x, y, z)` is true, by `oversample` points per active axis. Node `(i, j, k)` is at `(i dx, j dy, k dz)`.
`courant_limit`	função	`(spacing, dims=None) -> float`	Largest stable time step `1 / (c sqrt(sum_i 1/dx_i^2))` over the active axes (TH eq. 4.65).
`numerical_dispersion`	função	`(omega, dt, spacing, direction, n=1.0) -> float`	Wavenumber `k` of the Yee scheme along unit `direction` at angular frequency `omega` in a medium of index `n`: the root of `(n / (c dt) sin(omega dt / 2))^2 = sum_i (sin(k u_i dx_i / 2) / dx_i)^2` (TH eq. 4.12), solved by bisection/Newton from the continuum value.
`gaussian_pulse`	função	`(t0, tau) -> Callable[[float], float]`	`exp(-((t - t0)/tau)^2)`.
`modulated_gaussian`	função	`(f0, bandwidth, t0=None) -> Callable[[float], float]`	`sin(2 pi f0 (t - t0)) exp(-((t - t0)/tau)^2)` with `tau = 1 / (pi bandwidth)` (zero mean, no DC).
`ricker`	função	`(fp, t0=None) -> Callable[[float], float]`	Ricker wavelet with peak frequency `fp`.
`sinusoid`	função	`(f0, ramp_periods=3.0) -> Callable[[float], float]`	`sin(2 pi f0 t)` switched on with a smooth ramp over `ramp_periods` periods.
`poynting_flux_dft`	função	`(mon, axis, spacing) -> np.ndarray`	Time-averaged power `(1/2) Re int (E x H*) . n dA` through a plane monitor normal to `axis` from its DFT phasors (all four transverse components must be monitored); per unit length along invariant axes.

`pinneapple_simulation.electromagnetics.fem`

2D magnetostatics in the z-component of the magnetic vector potential, P1 triangles.

`pinneapple_simulation/electromagnetics/fem.py` · 244 linhas · testes: 36

nome	tipo	assinatura	o que faz
`Geometry`	classe	`(area, gx, gy, rows, cols, n)`	Per-element areas and barycentric gradients of a P1 mesh, plus sparse-assembly indices.
`MagnetostaticSolution`	classe	`(A, B, nu, energy_functional, iterations, residual, geometry, tangent, P, info)`	
`flux_density`	função	`(mesh, A) -> np.ndarray`	B per element (T) from nodal A_z (Wb/m).
`constraint_map`	função	`(n, dirichlet=None, dirichlet_values=None, periodic=None)`	Sparse map `A = P a + A_D`: Dirichlet nodes take their values, a periodic slave node takes `sign * A[master]` (chains are resolved), the remaining nodes are the unknowns `a`.
`solve_magnetostatic`	função	`(mesh, nu, J=None, Br=None, nonlinear=(), dirichlet=None, dirichlet_values=None, periodic…)`	Solve the A_z magnetostatic problem.

`pinneapple_simulation.electromagnetics.hysteresis`

Nonlinear magnetisation: the Froehlich-Kennelly saturation curve and the Jiles-Atherton hysteresis model.

`pinneapple_simulation/electromagnetics/hysteresis.py` · 198 linhas · testes: 26

nome	tipo	assinatura	o que faz
`FrohlichKennelly`	classe	`(alpha, beta)`	`B = mu0 H + H / (alpha + beta H)` for `H >= 0` (odd extension). `alpha` in m/H, `beta` in 1/T.
`HysteresisLoop`	classe	`(H, M)`	
`JilesAtherton`	classe	`(Ms, a, k, c, alpha)`	Jiles-Atherton parameters: saturation magnetisation `Ms` (A/m), anhysteretic shape `a` (A/m), pinning `k` (A/m), reversibility `c` (0..1), inter-domain coupling `alpha`.
`langevin`	função	`(x)`	`L(x) = coth x - 1/x` with its series `x/3 - x^3/45` near 0.

`pinneapple_simulation.electromagnetics.machine`

Radial-flux machine with surface permanent magnets (or none: a reluctance rotor), 2D A_z FEM.

`pinneapple_simulation/electromagnetics/machine.py` · 473 linhas · testes: 47

nome	tipo	assinatura	o que faz
`ScaledCurve`	classe	`(curve, w)`	Density-interpolated reluctivity `nu = nu0 + w (nu_iron(s) - nu0)` with per-element weights `w` (topology optimization of iron). `curve` is a B-H curve object or a linear `mu_r`.
`MachineSolution`	classe	`(theta, currents, mesh, sol, J)`	

nome	tipo	assinatura	o que faz
`SPMMachine`	classe	`(pole_pairs, slots, r_rotor, magnet_thickness, magnet_arc, magnetization, Br, mu_r_magnet…)`	Surface-PM (`magnet_thickness > 0`) or reluctance rotor inside a slotted stator.
`belt_winding`	função	`(slots, pole_pairs, phases=3) -> list[tuple[int, int]]`	Single-layer 60-degree phase-belt layout: (phase, sign) per slot, slot `k` centred at `k * 2 pi / slots`. Phase axes follow the positive rotation (A, then B 120 electrical degrees ahead), so a counter-clockwise rotor sees the sequence A, B, C.

pinneapple_simulation.electromagnetics.machines

Electric machines: parametric cross-sections, conforming meshes, non-linear 2-D FEA and post-processing.

`pinneapple_simulation/electromagnetics/machines/__init__.py` · 27 linhas · testes: 48

pinneapple_simulation.electromagnetics.machines.design

Rotor design of the V-magnet IPM machine with the design-study core of :mod:`pinneapple_design.design_optimizer`: the FEA as a reference evaluator, a Gaussian-process surrogate with active learning, NSGA-II, re-verification of every Pareto design by FEA, and the FEA calls this saves against a GA …

`pinneapple_simulation/electromagnetics/machines/design.py` · 175 linhas · testes: 2

nome	tipo	assinatura	o que faz
`rotor_evaluator`	função	`(**kwargs)`	An :class:`IPMRotorEvaluator` (keyword arguments of its constructor). The class is built on first use: the design core it subclasses imports torch.
`rotor_design_problem`	função	`(evaluator=None, ripple_max=0.3)`	Maximise `torque`, minimise `magnet_mass`, subject to `ripple <= ripple_max`.
`hybrid_vs_fea_only`	função	`(problem, *, n_initial=24, rounds=3, n_verify=8, ga_budget=400, surrogate_budget=1500, se…)`	Run both studies on `problem` (its evaluator is the FEA) and compare them at equal quality.

pinneapple_simulation.electromagnetics.machines.drive

Drive model of a synchronous machine: dq steady state, MTPA and flux weakening under inverter limits, efficiency map and drive-cycle energy.

`pinneapple_simulation/electromagnetics/machines/drive.py` · 484 linhas · testes: 11

nome	tipo	assinatura	o que faz
`LinearDQ`	classe	`(pole_pairs, psi_m, L_d, L_q, core_loss_coeffs)`	Machine with constant inductances: `psi_d = psi_m + L_d i_d`, `psi_q = L_q i_q`.

nome	tipo	assinatura	o que faz
FluxMap	classe	(pole_pairs, i_d, i_q, psi_d, psi_q, T, loss_i_d, loss_i_q, S_iron, magnet_per_f2, loss_c…)	Tabulated machine from the FEA. Grids i_d (ascending, <= 0) and i_q (ascending, >= 0); the machine's symmetry extends the maps to negative i_q (psi_d even, psi_q and torque odd).
Drive	classe	(machine, R, I_max, V_max, n_d, L_end)	A machine (flux map) fed by a voltage-source inverter: current and voltage amplitude limits, phase resistance, end-winding leakage inductance L_end (:mod:.endwinding). n_d sets the resolution of the search over i_d (refined by golden section).
Vehicle	classe	(mass, c_rr, cd_a, wheel_radius, gear_ratio, gear_efficiency, rotating_mass_factor, air_d…)	Longitudinal vehicle: test mass (kg), rolling resistance, drag area C_d A (m^2), wheel radius (m), single gear ratio and its efficiency, rotating-mass factor (equivalent inertia as a fraction of the mass).
v_max_svpwm	função	(v_dc) -> float	Largest phase-voltage amplitude of space-vector PWM in its linear range: V_dc / sqrt(3).
copper_resistance	função	(series_turns, conductor_area, stack_length, end_length, temperature=20.0, rho20=1.724e-0…)	DC phase resistance: rho(T) N 2 (L + l_end) / (A a^2) with N series turns per phase, A the copper area of one conductor, l_end the end-turn length per side, a parallel paths (IEC 60028 standard copper 1.724e-8 Ohm m at 20 degC by default).
prius_phase_resistance	função	(temperature=100.0) -> tuple[float, dict]	Phase resistance of the 2004 Prius motor from the winding data of pyleecan's Toyota_Prius.json: 72 series turns, 13 strands of 0.912 mm per conductor, 83.82 mm stack. The end-turn length per side is an estimate: a half circle over the coil pitch (6 slots at the mid-slot radius) plus twice the …
wltc_class3b	função	() -> tuple[np.ndarray, np.ndarray]	WLTC class 3b: time (s) and vehicle speed (m/s), 1 Hz, 0..1800 s (UN GTR No. 15, Annex 1).
cycle_energy	função	(drive, vehicle, t, v, envelope_speeds=25) -> dict	Battery-side energy of drive over the speed trace (t s, v m/s): motoring P_mech + losses, regeneration P_mech - losses (negative) up to the braking-torque envelope, the rest by friction brakes. Operating points come from a table of :meth:Drive.operating_point on a speed x …

pinneapple_simulation.electromagnetics.machines.endwinding

End windings: resistance and leakage inductance of the coil ends, outside the 2-D cross-section.

pinneapple_simulation/electromagnetics/machines/endwinding.py · 77 linhas · testes: 5

nome	tipo	assinatura	o que faz
coil_end_path	função	(r_c, phi_a, phi_b, overhang, n_arc=48) -> np.ndarray	Polyline (n, 3) of one coil end from the stack face at angle phi_a to phi_b (z >= 0 outside).

nome	tipo	assinatura	o que faz
`end_winding_paths`	função	`(layout, slots, coil_pitch, r_c, overhang) -> list[tuple[int, np.ndarray]]`	(phase, path) of every coil end of a single-layer winding: each slot with a positive coil side is joined to the slot `coil_pitch` further on, which must carry the same phase with the opposite sign.
`end_winding_inductance`	função	`(geometry, layout, turns_per_coil, coil_pitch, overhang, pieces_per_metre=300.0) -> dict`	Phase self and mutual inductances of both coil ends (H), and the dq end leakage `L_aa - M_ab`, in air and with the end face as an iron mirror.

`pinneapple_simulation.electromagnetics.machines.export`

Export of machine meshes and fields: VTK unstructured grids (ParaView), CSV, HDF5, MATLAB, DXF.

`pinneapple_simulation/electromagnetics/machines/export.py` · 58 linhas · testes: 6

nome	tipo	assinatura	o que faz
`write_vtu`	função	`(path, points, cells, cell_data=None, point_data=None) -> Path`	Write an unstructured grid; `points` (N, 2 or 3), `cells` (M, 3) triangles or (M, 6) wedges; data arrays of shape (M,) / (M, c) and (N,) / (N, c).
`solution_to_vtu`	função	`(path, ms) -> Path`	A 2-D machine solution (`MachineSolution`): region index and B, |B| per cell, A_z per node.

`pinneapple_simulation.electromagnetics.machines.geometry`

Parametric cross-sections of radial-flux machines: V-shaped interior magnets and round-bottom slots.

`pinneapple_simulation/electromagnetics/machines/geometry.py` · 320 linhas · testes: 18

nome	tipo	assinatura	o que faz
`Polygon`	classe	`(name, xy, magnetization)`	A named closed polygon (vertices counter-clockwise or clockwise, last vertex not repeated).
`SectorGeometry`	classe	`(r_in, r_out, span, fill, polygons, arc_nodes_in, arc_nodes_out)`	Annular sector `r_in < r < r_out`, `-span/2 < phi < span/2`, holding polygons (later ones win where they overlap) in a background region `fill`. `arc_nodes_in`/`arc_nodes_out`: extra angles that must be nodes on the inner/outer arc (corners of polygons that touch the arcs).
`VRotor`	classe	`(poles, r_out, r_in, H0, H1, H2, H3, W0, W1, W2, W3, W4, magnet_segments)`	Interior-PM rotor with one V of two magnets per pole (pyleecan `HoleM50` parametrisation).
`RoundBottomSlots`	classe	`(slots, r_in, r_out, W0, H0, H1, W1, W2, H2, R1)`	Stator lamination with open slots: opening, straight-sided body, semicircular or filleted bottom.
`MachineGeometry`	classe	`(rotor, stator, length, stacking_factor)`	Rotor + stator + air gap. `gap_layers` rings split the gap (moving band in the middle).

nome	tipo	assinatura	o que faz
`polygon_area`	função	`(xy) -> float`	Signed area of a closed polygon (counter-clockwise positive).
`arc`	função	`(center, radius, a0, a1, h, even=False) -> np.ndarray`	Points of a circular arc from angle `a0` to `a1` (both included), spacing at most `h`; `even`: an even number of intervals, so the mid-angle is a vertex (arcs symmetric about a mirror axis).
`prius_2004`	função	`() -> MachineGeometry`	2004 Toyota Prius traction motor: dimensions of pyleecan's `Toyota_Prius.json` (pyleecan 1.4.2, Apache-2.0), which models the machine of the ORNL evaluation reports (ORNL/TM-2004/185, ORNL/TM-2006/423).
`geometry_errors`	função	`(sector, tol=1e-09) -> list[str]`	Reasons why a sector is not a valid cross-section (empty when valid): polygons that self-intersect, cross each other, overlap (a vertex strictly inside another polygon), or leave the annular sector.

`pinneapple_simulation.electromagnetics.machines.getdp`

Independent cross-check of the machine FEA with gmsh (mesh) and GetDP (solver).

`pinneapple_simulation/electromagnetics/machines/getdp.py` · 469 linhas · testes: 3

nome	tipo	assinatura	o que faz
`GmshGetDPError`	classe	`()`	
`available`	função	`() -> bool`	
`write_geo`	função	`(path, geometry, theta=0.0, n_gap=960, h_max=None, grading=0.2, h=0.001) -> dict`	gmsh geometry of the full machine at rotor angle `theta`. Returns the physical-region numbers, the polygon areas of the slots and the magnetisation angles of the magnets (global frame).
`write_pro`	função	`(path, geometry, info, steel, Br, mu_r, layout, turns_per_slot, currents, parallel_paths=…)`	GetDP problem for the regions of :func:`write_geo`.
`solve`	função	`(geometry, theta, currents, steel, Br, mu_r, layout, turns_per_slot, n_gap=960, workdir=N…)`	Mesh with gmsh and solve with GetDP; returns torque (N m), phase flux linkages (Wb), element count, times.
`write_geo_3d`	função	`(path, geometry, n_gap, layers_stack, end_air, layers_air) -> dict`	The cross-section of :func:`write_geo` (rotor at angle 0) extruded over half the stack (`z` from the mid-stack plane to the end face) into prisms, plus an air extension of length `end_air` beyond the end face (`end_air = 0`: none).

nome	tipo	assinatura	o que faz
write_pro_3d	função	(path, info, steel, Br, mu_r, stacking_factor, pole_half_angle, r_gap, with_end_air) -> N…)	GetDP problem: magnetic scalar potential H = -grad(phi), div(mu H + Br) = 0, nonlinear iron by Picard iteration on the secant permeability (Newton on mu(h) diverges for this formulation), natural boundaries (no flux through the shaft, outer and mid-stack surfaces), phi fixed at one …
end_leakage_3d	função	(geometry, steel, Br, mu_r, n_gap=240, layers_stack=6, end_air=0.04, layers_air=6, workdi…)	No-load air-gap flux under a pole of the 3-D half stack with an air region beyond the end face, divided by the same model without it (identical mesh in the stack: the 2-D field extruded). Returns both fluxes, the ratio, and the element counts and times.

pinneapple_simulation.electromagnetics.machines.interop

Interoperability of the machine models: pyleecan, FEMM, MATLAB, DXF and the PINNeAPPle data contracts.

pinneapple_simulation/electromagnetics/machines/interop.py · 357 linhas · testes: 8

nome	tipo	assinatura	o que faz
from_pyleecan	função	(machine) -> tuple[MachineGeometry, dict]	Geometry and winding data of a pyleecan MachineIPMSM file (or the parsed dict).
to_pyleecan	função	(geometry) -> dict	pyleecan field names and values of the rotor hole, the stator slot and the laminations.
cross_section_polygons	função	(geometry, h=0.001) -> list[tuple[str, np.ndarray, float | None]]	Every polygon of the full cross-section as (region, xy, magnetisation angle): pole k turned by k * pole pitch (magnetisation reversed on odd poles), slot k by k * slot pitch.
femm_lua	função	(geometry, steel, magnet, layout, turns_per_slot, currents=(0.0, 0.0, 0.0), magnet_temper…)	Lua script for FEMM 4.2 (magnetics, planar, millimetres) of the full machine at the rotor angle of the mesher's sector geometry (pole 0 on +x). layout: (phase, sign) per slot; currents: phase currents (A).
read_femm_lua_polygons	função	(script) -> list[np.ndarray]	Closed polygons (metres) traced by consecutive mi_addsegment calls of a script from :func:femm_lua.
to_mat	função	(path, **arrays) -> Path	Write named arrays (or nested dicts of arrays) to a MATLAB .mat file (scipy.io.savemat).
from_mat	função	(path) -> dict	
to_dxf	função	(path, geometry, h=0.001, unit_scale=1000.0) -> Path	Cross-section as closed POLYLINE entities (DXF R12, ASCII, millimetres by default), one layer per region kind (MAGNET, POCKET, SLOT, OPENING, WEDGE) plus the four lamination circles on layer LAMINATION.

nome	tipo	assinatura	o que faz
`read_dxf_polylines`	função	`(path, unit_scale=1000.0) -> dict[str, list[np.ndarray]]`	Closed POLYLINEs of a DXF R12 file per layer (coordinates divided by `unit_scale`).
`rotor_parameter_space`	função	`(name='v-ipm-rotor')`	`ParameterSpace` of the rotor design variables (:data:`~.geometry.ROTOR_VARIABLES`, metres).
`fluxmap_dataset`	função	`(fm, name='ipm-flux-map')`	A flux map (attributes `i_d`, `i_q` grids and `psi_d`, `psi_q`, `T` tables, e.g. a drive `FluxMap`) as a table (pandas DataFrame: i_d, i_q, psi_d, psi_q, torque) and its `DatasetSpec`.
`elmer_case`	função	`(directory, geometry, steel, magnet, layout, turns_per_slot, currents=(0.0, 0.0, 0.0), th…)`	An ElmerFEM case (MagnetoDynamics2D, steady) of the machine at rotor angle `theta`: `machine.geo` (gmsh, the same geometry as the GetDP cross-check) and `case.sif`. Mesh it with `gmsh -2 machine.geo -format msh22` and convert with `ElmerGrid 14 2 machine.msh -autoclean`; body numbers in …

`pinneapple_simulation.electromagnetics.machines.mesher`

Conforming Delaunay meshes of machine sectors and their assembly into a rotating machine mesh.

`pinneapple_simulation/electromagnetics/machines/mesher.py` · 573 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SectorMesh`	classe	`(points, tris, region, polygon, arc_in, arc_out, side0, side1)`	
`MachineMesh`	classe	`(geometry, n_gap=960, h_max=None, grading=0.2, smooth=3, mirror=True)`	Mesh of a complete radial-flux machine (:class:`~.geometry.MachineGeometry`).
`mesh_sector`	função	`(geom, size, n_in, n_out, smooth=3, mirror=False, max_rounds=60) -> SectorMesh`	Mesh `geom` with element size `size(r)` (metres, a function of radius, or a constant); the arcs get `n_in` and `n_out` equal intervals (plus the extra nodes of `geom.arc_nodes_in/out`). `smooth` rounds of Laplacian smoothing of the interior points improve the angles (boundaries stay …
`replicate_sector`	função	`(sm, copies, span) -> Mesh2D`	The full circle from `copies` rotated copies of a sector mesh. Region index per element: the polygon index (fill = -1 shifted to 0, polygons from 1); `boundary`: the nodes of both arcs; extra attributes `sector` and `local` give each element's copy and its index in the sector.

`pinneapple_simulation.electromagnetics.machines.pmsm`

Interior permanent-magnet synchronous machine: 2-D nonlinear A_z FEA and its post-processing.

`pinneapple_simulation/electromagnetics/machines/pmsm.py` · 361 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Winding`	classe	`(turns_per_slot, parallel_paths, layout)`	Phase, orientation and turns of the coil side in each slot (single layer), series turns per phase, parallel paths. `layout`: (phase, sign) per slot, default a 60-degree phase belt.
`IPMMachine`	classe	`(geometry, steel, magnet, magnet_temperature, winding, n_gap, h_max, grading, tol)`	

`pinneapple_simulation.electromagnetics.machines.robustness`

Step skew, manufacturing tolerances and a 3-D view of the field of an IPM machine.

`pinneapple_simulation/electromagnetics/machines/robustness.py` · 132 linhas · testes: 6

nome	tipo	assinatura	o que faz
`step_offsets`	função	`(steps, step_angle) -> np.ndarray`	Rotor angles of the `steps` segments, centred on zero.
`step_skew_factor`	função	`(h, steps, step_angle, period)`	Factor on harmonic `h` (per `period`) of the segment-mean of a quantity: `|sin(K x) / (K sin x)|`, `x = h pi step_angle / period`.
`skewed_no_load`	função	`(machine, thetas, steps, step_angle) -> dict`	Cogging torque and phase flux linkages of the step-skewed rotor at the rotor angles `thetas`.
`skewed_torque`	função	`(machine, i_d, i_q, thetas, steps, step_angle) -> np.ndarray`	Torque of the step-skewed rotor at rotor angles `thetas` (mechanical), with phase currents fixed in the dq frame of the rotor at `theta` (the segments see the same stator currents, turned by their offset).
`tolerance_model`	função	`(base, I_peak=250.0, gamma_deg=50.0, n_gap=288, n_positions=6, variables=TOLERANCE_VARIAB…)`	`fn(X) -> (n, 2)` [mean torque (N m), ripple] for rows of `variables`: `Br_scale` (remanence factor), `H3` magnet thickness, `W4` magnet width (m), `airgap` (m, the rotor radius changes, the stator stays). Rows whose geometry is invalid give NaN. The function counts its FEA calls in …
`skewed_field_3d`	função	`(machine, theta, currents, steps, step_angle, layers_per_step=1) -> dict`	Prism (wedge) mesh of the stack with the 2-D field of every segment: points (N, 3), wedges (M, 6), cell data `B` (M, 3), `|B|` and region index. Segment `k` occupies `z` in `[k, k+1] L / K`.

`pinneapple_simulation.electromagnetics.machines.steel`

Catalogue electrical steel and magnet data for machine models, with their sources.

`pinneapple_simulation/electromagnetics/machines/steel.py` · 191 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SteelGrade	classe	(name, H, B, loss_f, loss_B, loss_W_per_kg, thickness, resistivity, density, source)	
MagnetGrade	classe	(name, Br20, mu_r, alpha_Br, resistivity, density, source)	
LossFit	classe	(model, rms_rel_error, max_rel_error, fitted)	
TableIronLoss	classes	(grade)	Specific iron loss interpolated from a catalogue table (same interface as IronLossModel).
stacked	função	(curve, kf, n=200) -> BHCurve	B-H curve of a lamination stack: B = kf B_steel(H) + (1 - kf) mu0 H.
fit_iron_loss	função	(grade, classical_from_resistivity=True, B_max=1.8, f_max=2500.0) -> LossFit	Least-squares fit (relative errors) of k_h f^alpha B^beta + k_c (f B)^2 + k_e (f B)^1.5 to the grade's loss table. With classical_from_resistivity the classical eddy coefficient is the thin-sheet value pi^2 d^2 / (6 rho_e density) and only k_h, alpha, beta, k_e are fitted.

pinneapple_simulation.electromagnetics.machines.thermal

Lumped thermal network of a radial-flux machine, and the demagnetisation check of its magnets.

pinneapple_simulation/electromagnetics/machines/thermal.py · 283 linhas · testes: 8

nome	tipo	assinatura	o que faz
ThermalNetwork	classe	(names, C, links, boundaries)	
MachineThermalData	classe	(k_lam, k_winding, k_cu, copper_fill, h_gap, h_end, jacket_GA, coolant_T, k_magnet_contac…)	Material and boundary data of :func:machine_network (SI). Defaults are typical textbook values, stated as assumptions: laminations radial k = 28 W/m K, winding (copper, resin, insulation) equivalent k = 1.0 W/m K across the slot, copper k = 390 W/m K along the conductors with copper fill 0.45, …
intrinsic_coercivity	função	(Hcj20, temperature, beta=-0.0055) -> float	Hcj(T) = Hcj20 (1 + beta (T - 20)) (A/m); beta is the reversible temperature coefficient of Hcj (typically -0.5 to -0.6 %/K for sintered NdFeB; take the grade's datasheet value).
cylinder_conductance	função	(k, r1, r2, length, fraction=1.0) -> float	Radial conduction through a cylindrical shell (or fraction of it): 2 pi k L f / ln(r2 / r1).
machine_network	função	(geometry, data=None, end_length=0.16) -> ThermalNetwork	Seven-node network of a machine with a water jacket on the stator back (see the module docstring).

nome	tipo	assinatura	o que faz
reverse_field	função	`(B, angle, Br, mu_r) -> np.ndarray`	Field opposing the magnetisation (A/m) in linear magnet elements: `H_rev = -(B . m - Br) / (mu0 mu_r)` with `m = (cos angle, sin angle)`, from `B = mu0 mu_r H + Br m`.
demagnetisation_margin	função	`(machine, ms, Hcj, margin_ratio=0.9) -> dict`	Field opposing the magnetisation in every magnet element of the solution `ms` of `machine` (an :class:`~.pmsm.IPMMachine`), against `Hcj` (A/m at the magnet temperature; the knee of a square-loop Nd-FeB magnet). Returns the worst reverse field, the margin `Hcj - H_rev` (A/m), the …
demagnetisation_refinement	função	`(geometry, magnet_temperature, currents_d, Hcj, gradings=(0.2, 0.1, 0.05, 0.025), n_gap=9…)`	Mesh study of :func:`demagnetisation_margin`: the machine meshed with decreasing `grading` (element size growth away from the gap, :class:`~.mesher.MachineMesh`), so the magnet elements shrink at a fixed gap mesh. `currents_d`: label -> d-axis current (A, rotor at 0); `Hcj`: grade -> …
magnet_temperature_sweep	função	`(geometry, temperatures, I_peak=250.0, n_gap=960, n_positions=12, n_flux=24, magnet=None)…)`	Machine performance against the magnet temperature (`Br(T)` of the grade): MTPA torque and angle at `I_peak`, torque ripple, magnet flux linkage and back-EMF constant (fundamental, from `n_flux` angles over an electrical period), frozen-permeability L_d, L_q at the MTPA point.
iron_split	função	`(m, I_peak=250.0, gamma_deg=50.3, rpm=1500.0, steps=24) -> dict`	Fractions of the iron loss in the stator yoke, the teeth and the rotor of an :class:`~pinneapple_simulation.electromagnetics.machines.IPMMachine` at one operating point (default 250 A MTPA, 50.3 deg, 1500 rpm): how a lumped thermal network distributes the iron loss of a drive cycle.

pinneapple_simulation.electromagnetics.machines.verification

Verification of the machine FEA: closed forms for the mesher and solver, internal consistency of the torque, flux-linkage and inductance post-processing, and a cross-check against a published benchmark machine.

`pinneapple_simulation/electromagnetics/machines/verification.py` · 365 linhas · testes: 30

nome	tipo	assinatura	o que faz
case	função	`(name)`	
run	função	`(names=None, quick=False) -> list[CaseResult]`	
prius_torque_curve	função	`(n_gap=960, I_peak=250.0, phi0_deg=YANG_2013_PHI0_DEG, n_positions=12) -> dict`	Mean Maxwell-stress and dq torque of the Prius model against the current angle (pyleecan `Phi0`).
yang_comparison	função	`(curve) -> dict`	Per-point errors of a torque-angle curve against Yang et al. 2013, and the angle offset `delta` (deg) that minimises the rms error of `T(Phi0 + delta)` (cubic interpolation of the curve).

pinneapple_simulation.electromagnetics.materials

Magnetic materials for 2D A_z magnetostatics: linear media, permanent magnets and nonlinear B-H curves.

pinneapple_simulation/electromagnetics/materials.py · 147 linhas · testes: 26

nome	tipo	assinatura	o que faz
LinearMaterial	classe	(mu_r, sigma)	Linear, isotropic medium: relative permeability mu_r and electrical conductivity sigma (S/m).
Magnet	classe	(Br, mu_r, direction, sigma)	Linear permanent magnet: B = mu0 mu_r H + Br.
BHCurve	classe	(B, H)	Single-valued B-H curve from points, monotone cubic in H(B), vacuum slope above the last point.
BrauerCurve	classe	(k1, k2, k3)	nu(s) = k1 exp(k2 s) + k3 with s = |B|^2 (closed-form energy and derivative).
illustrative_steel	função	(mu_r_init=4000.0, Js=2.0, n=60) -> BHCurve	Smooth saturating curve B = mu0 H + Js (2/pi) atan(pi (mu_r_init - 1) mu0 H / (2 Js)).

pinneapple_simulation.electromagnetics.media

Electric and magnetic media for wave problems: linear, anisotropic and dispersive (Drude, Lorentz, Debye).

pinneapple_simulation/electromagnetics/media.py · 212 linhas · testes: 36

nome	tipo	assinatura	o que faz
Drude	classe	(omega_p, gamma)	chi(omega) = -omega_p^2 / (omega^2 - j omega gamma); plasma frequency omega_p and collision rate gamma in rad/s. With gamma = 0 it is a collisionless plasma.
Lorentz	classe	(delta_eps, omega0, delta)	chi(omega) = delta_eps omega_0^2 / (omega_0^2 + 2 j omega delta - omega^2); resonance omega_0 (rad/s), damping delta (1/s), strength delta_eps = eps_s - eps_inf.
Debye	classe	(delta_eps, tau)	chi(omega) = delta_eps / (1 + j omega tau); relaxation time tau (s).
Medium	classe	(eps_inf, mu_r, sigma, sigma_m, poles)	Isotropic linear medium: eps_inf, mu_r, conductivity sigma (S/m), magnetic conductivity sigma_m (ohm/m, used by PMLs) and dispersive poles.
Anisotropic	classe	(eps_r, mu_r, sigma)	Linear anisotropic medium with relative tensors eps_r and mu_r (3x3) and conductivity sigma.
drude_metal	função	(omega_p, gamma=0.0, eps_inf=1.0) -> Medium	
is_passive	função	(medium, omegas, tol=1e-12) -> bool	Passivity of a medium under exp(+j omega t): Im(eps_r) <= 0 at every given frequency.
rotation	função	(axis, angle) -> np.ndarray	Rotation matrix about axis by angle (rad), Rodrigues' formula.

pinneapple_simulation.electromagnetics.mesh

Triangle meshes for 2D magnetostatics: a polar ring mesher with a moving band, and a gmsh importer.

pinneapple_simulation/electromagnetics/mesh.py · 301 linhas · testes: 26

nome	tipo	assinatura	o que faz
`Mesh2D`	classe	`(points, tris, region, region_names, layer, boundary, rings)`	P1 triangle mesh with one region name per element.
`Ring`	classe	`(r, n, offset, rotating)`	A circle of `n` equally spaced nodes at radius `r` (`n=1` with `r=0` is the centre).
`zipper`	função	`(ia, ang_a, ib, ang_b, closed=True) -> np.ndarray`	Triangulate the strip between two rings of nodes (indices `ia`/`ib`, angles `ang_a`/`ang_b`).
`polar_mesh`	função	`(rings, classify, theta=0.0, span=TWO_PI, region_names=None) -> Mesh2D`	Mesh the disc (or sector `[0, span]`) between concentric rings.
`rect_mesh`	função	`(x, y, classify=None) -> Mesh2D`	Structured triangle mesh of the rectangle spanned by the node coordinates `x` and `y` (1-D, increasing; any spacing), two triangles per cell with diagonals alternating from cell to cell, so the mesh has no preferred direction. `classify(xc, yc)` names the region of each element from its …
`mesh_from_gmsh`	função	`(dim=2, model=None) -> Mesh2D`	Convert the current gmsh model (already meshed) into a :class:`Mesh2D`.
`boundary_nodes`	função	`(tris) -> np.ndarray`	Nodes on edges that belong to exactly one triangle.

`pinneapple_simulation.electromagnetics.mie`

Mie scattering of a plane wave by a homogeneous sphere (and a perfectly conducting one).

`pinneapple_simulation/electromagnetics/mie.py` · 127 linhas · testes: 26

nome	tipo	assina-tura	o que faz
`MieResult`	classe	`(x, m, an, bn, qext, qsca, qback, g)`	
`mie_coefficients`	função	`(m_bh, x, n_max=None)`	`a_n, b_n` (n = 1..n_max) in Bohren & Huffman's convention (`Im(m_bh) >= 0` absorbs).
`mie`	função	`(m, x, n_max=None) -> MieResult`	Efficiencies of a homogeneous sphere, `m` in this package's `exp(+j omega t)` convention.
`mie_pec`	função	`(x, n_max=None) -> MieResult`	Perfectly conducting sphere: `a_n = psi_n'(x) / xi_n'(x)`, `b_n = psi_n(x) / xi_n(x)`, the `|m| -> infinity` limit of the coefficients above; its small-sphere limit is `Q_sca = (10/3) x^4` (Jackson eq. 10.18 divided by `pi a^2`).
`rayleigh_qsca`	função	`(m, x) -> float`	Small-sphere limit `Q_sca = (8/3) x^4 |(m^2 - 1) / (m^2 + 2)|^2` (B&H eq. 5.8).

`pinneapple_simulation.electromagnetics.periodic`

Periodic structures (infinite arrays, FSS) in unit-cell Floquet runs, and the campaigns built on them.

`pinneapple_simulation/electromagnetics/periodic/__init__.py` · 15 linhas · testes: 5

`pinneapple_simulation.electromagnetics.periodic.benchmark`

Unit-cell campaigns and the equal-budget benchmark of an operator surrogate against Bayesian optimisation.

`pinneapple_simulation/electromagnetics/periodic/benchmark.py` · 212 linhas · testes: 5

nome	tipo	assinatura	o que faz
`cell_simulator`	função	`(workdir, scans, *, phi=0.0, timeout=3600.0, log=None, resume=False) -> CLISimulator`	Runner of :mod:`.tcda` with every frequency and the scan angles `scans` (deg) in one process per cell.
`campaign`	função	`(n, freqs, scans, workdir, *, ranges=None, fixed=None, method='lhs', seed=0, tag='c', log…)`	`n` cells from a `method` sample of `ranges` (plus `fixed` parameters), all solved.
`to_arrays`	função	`(table, names, freqs, scans, column='S11_dB') -> tuple[np.ndarray, np.ndarray, list[str]]`	`X (n, d)`, `Y (n, n_f, n_scan)` and the tags of the cells with every (frequency, scan) solved.
`worst_case`	função	`(Y) -> np.ndarray`	Worst active reflection (dB) over the band and the scan range, per cell.
`fit_fno`	função	`(X, Y, *, wall_s=60.0, seed=0, val_frac=0.15, width=24, threads=1)`	FNO from the parameters to the (n_f, n_scan) map: `pinneapple_analysis.surrogate` FNO candidate, trained for `wall_s` seconds on `threads` CPU threads (one thread: a small FNO runs ~100x more steps per second than with four threads that compete with each other or with a solver). Returns …
`held_out`	função	`(predict, X, Y) -> dict[str, float]`	Errors on cells the model never saw: RMSE over every (f, scan) point and of the worst case, in dB.
`equal_budget`	função	`(solve, ranges, budget, *, n_verify=4, seed=0, fno_wall_s=60.0, n_candidates=4000, log=pr…)`	Both strategies with `budget` solver calls each (one call = one cell, all frequencies and scans).
`solver_from`	função	`(sim, names, freqs, scans, fixed=None)`	A `solve` callable for :func:`equal_budget` from a :func:`cell_simulator`.

pinneapple_simulation.electromagnetics.periodic.tcda

A generic tightly coupled dipole array (TCDA) unit cell for EMerge Floquet runs, written from scratch.

`pinneapple_simulation/electromagnetics/periodic/tcda.py` · 165 linhas · testes: 5

nome	tipo	assinatura	o que faz
`InvalidCell`	classe	`()`	Parameters that do not describe a buildable cell.
`TCDACell`	classe	`(a, h, w, g, s, t_s, d_s, er_s, z_port, res, air)`	
`simulate`	função	`(cell, freqs_ghz, scans_deg=((0.0, 0.0),), *, log='WARNING') -> list[dict]`	Active reflection of `cell` at every frequency (GHz) and scan `(theta, phi)` (degrees); one row each.
`main`	função	`(argv=None) -> int`	

pinneapple_simulation.electromagnetics.postprocess

Post-processing of A_z solutions: torque, flux linkage, back-EMF, iron losses, magnet eddy losses.

`pinneapple_simulation/electromagnetics/postprocess.py` · 285 linhas · testes: 22

nome	tipo	assinatura	o que faz
`CoilSide`	classe	`(region, phase, turns, sign)`	A coil side: `region` name, phase index, turns `N` and orientation `sign` (+1 / -1).
`IronLossModel`	classe	`(k_h, alpha, beta, k_c, k_e, density)`	Loss per kg for an alternating component of amplitude B (T) at frequency f (Hz): `k_h f^alpha B^beta + k_c f^2 B^2 + k_e f^1.5 B^1.5`; `density` in kg/m^3.
`torque_maxwell_stress`	função	`(mesh, B, r1, r2, length=1.0, elements=None) -> float`	Arkkio torque (N m) on everything inside the air annulus `r1 < r < r2`.
`torque_maxwell_stress_gradient`	função	`(mesh, geometry, r1, r2, elements, length=1.0) -> np.ndarray`	The Arkkio torque of :func:`torque_maxwell_stress` as a quadratic form `T = sum_e B_e^T S_e B_e` of the nodal potential: returns an object with `value(A)` and `gradient(A)` (dT/dA, for adjoints).
`torque_coenergy`	função	`(coenergy_at, theta, delta, length=1.0, stencil='central') -> float`	Virtual-work torque `dW'/dtheta` (N m) from co-energies per metre `coenergy_at(angle)`.
`region_integral_A`	função	`(geometry, tris, A, mask) -> float`	int A dS over the elements in `mask` (P1: area times mean of the vertex values).
`flux_linkage`	função	`(mesh, geometry, A, coil_sides, n_phases, length=1.0, parallel_paths=1) -> np.ndarray`	Flux linkage per phase (Wb-turn).
`back_emf`	função	`(psi, theta, omega_mech) -> np.ndarray`	`e = omega dpsi/dtheta` for flux linkages sampled over one full period of `theta` (uniform, endpoint excluded); spectral derivative along axis 0.

nome	tipo	assinatura	o que faz
classical_eddy_coefficient	função	(sigma, thickness, density) -> float	k_c = pi^2 sigma d^2 / (6 density) (W/kg per (Hz T)^2) for laminations of thickness d under sinusoidal flux, low-frequency limit.
element_flux_density_history	função	(B_steps, thetas, rotor_mask) -> np.ndarray	Stack B per step to (steps, M, 2), turning rotor elements into the rotor frame.
flux_density_ellipses	função	(B_hist, mask, n_harmonics=None)	Semi-axes of the ellipse traced by each harmonic of B per element: (b_major, b_minor), each of shape (harmonics, elements in mask), from B_hist (steps, M, 2) sampled uniformly over one period.
iron_loss	função	(B_hist, f_fundamental, model, area, mask, length=1.0, n_harmonics=None)	Iron loss (W) of the elements in mask from B_hist (steps, M, 2) sampled uniformly over one period of the fundamental. Returns (total W, loss density per element W/m^3).
iron_loss_components	função	(B_hist, area, mask, alpha, beta, density, length=1.0, n_harmonics=None) -> np.ndarray	Frequency-free sums (S_h, S_c, S_e) such that the loss of :class:IronLossModel with exponents alpha, beta at fundamental frequency f is k_h f^alpha S_h + k_c f^2 S_c + k_e f^1.5 S_e (W): one field solution period gives the iron loss at every speed (the field waveform is fixed ...
magnet_eddy_loss_resistance_limited	função	(mesh, geometry, A_hist, period, sigma, pieces, length=1.0)	Time-averaged eddy loss (W) in magnet pieces from nodal A_hist (steps, N) in the magnet's own frame, sampled uniformly over period seconds. pieces: element masks, one per insulated piece.

pinneapple_simulation.electromagnetics.reflector

Reflector antennas and compact ranges by physical optics: geometry, PO currents and near fields, quiet zones.

pinneapple_simulation/electromagnetics/reflector/__init__.py · 18 linhas · testes: 6

pinneapple_simulation.electromagnetics.reflector.analytic

Closed forms that check the physical-optics code of :mod:.po.

pinneapple_simulation/electromagnetics/reflector/analytic.py · 55 linhas · testes: 6

nome	tipo	assinatura	o que faz
disk_on_axis	função	(z, radius, k, E0=1.0) -> np.ndarray	E_x (complex) at heights z > 0 on the axis of a PEC disk of radius under a normally incident x-polarised plane wave of amplitude E0 (PO current, exact integral).
go_aperture_taper_db	função	(x, y, focal, q, feed_axis) -> np.ndarray	Relative GO amplitude [dB, 0 at the maximum over the given points] at aperture points (x, y) of the paraboloid z = (x^2 + y^2) / (4F) fed at its focus by a cos^q feed pointing along feed_axis.

pinneapple_simulation.electromagnetics.reflector.compact_range

A single offset-reflector compact range: feed, reflector, quiet zone, and its parameter studies.

pinneapple_simulation/electromagnetics/reflector/compact_range.py · 153 linhas · testes: 20

nome	tipo	assinatura	o que faz
CompactRange	classe	(freq_hz, reflector, feed_gain_dbi, pol, qz_z, qz_diameter, qz_centre_y, feed_offset, mes…)	
scaled	função	(cr, side=None, roll=None, feed_gain_dbi=None) -> CompactRange	cr with a section of side (square, same exponent), roll radius roll and feed gain; the clearance between the axis and the section's lower junction stays, the quiet zone follows the section centre line.
qz_field	função	(cr, points_xy=None) -> dict[str, Any]	PO field on the quiet-zone plane: {"xy", "E" (M, 3), "mesh", "seconds"}; xy relative to the QZ centre (default: a disk of eval_factor x the QZ diameter, step lambda / qz_step_per_wavelength).
evaluate	função	(cr, *, amp_tol=0.5, phase_tol=5.0) -> dict[str, Any]	Quiet-zone metrics of cr (co-polar: E_y for "V", E_x for "H") with the mesh size and the cost.
sweep	função	(base, gains, sides, rolls=(None,), *, log=None)	Every combination; returns a :class:~pinneapple_simulation.electromagnetics.antenna.runner.ResultTable (tag, inputs, metrics; f_GHz constant).
go_check	função	(cr) -> dict[str, float]	PO against geometrical optics on the quiet zone: the quadratic fit of the PO amplitude (its smooth part) against the GO aperture taper of :func:~.analytic.go_aperture_taper_db at the same (x, y); returns the largest and RMS difference in dB over the QZ disk (the remainder is the diffraction …

pinneapple_simulation.electromagnetics.reflector.geometry

Offset paraboloid reflectors with optional rolled edges, as triangle meshes for :mod:.po.

pinneapple_simulation/electromagnetics/reflector/geometry.py · 172 linhas · testes: 6

nome	tipo	assinatura	o que faz
OffsetReflector	classe	(focal, half_x, half_y, centre_y, n, roll, end_angle, blend, roll_aspect)	
ReflectorMesh	classe	(vertices, faces, centres, normals, areas, roll)	
superellipse_radius	função	(phi, a, b, n)	
mesh_reflector	função	(r, h) -> ReflectorMesh	Triangle mesh with edges of about h (use lambda/8 or less for PO).
flat_disk	função	(radius, h) -> ReflectorMesh	A flat disk of radius in z = 0 (normal +z), for the PO validation against the closed form.

pinneapple_simulation.electromagnetics.reflector.po

Physical optics on a triangulated PEC surface: feed fields, PO currents and their exact radiated field.

pinneapple_simulation/electromagnetics/reflector/po.py · 118 linhas · testes: 6

nome	tipo	assinatura	o que faz
wavenumber	função	(f_hz) -> float	
q_from_gain	função	(gain_dbi) -> float	q of a cos^q feed of directivity gain_dbi (D = 2 (2q + 1)).
gain_from_q	função	(q) -> float	
cosq_feed	função	(points, k, *, position, axis, q, pol=(0.0, 1.0, 0.0))	E and H (N, 3) complex of a cos^q feed at position pointing along axis.
plane_wave	função	(points, k, direction, pol, amplitude=1.0)	E, H of a plane wave travelling along direction with E along pol (made transverse).
po_currents	função	(normals, centres, H_inc, source=None)	J = 2 n x H (N, 3) on lit triangles. source: a point (feed) or None for a field arriving against the normals (plane wave): a triangle is lit when its normal faces the source.
radiate	função	(centres, areas, J, obs, k, chunk=2000000)	E (M, 3) complex at the points obs (M, 3) radiated by currents J (N, 3) on triangles of areas (N,) at centres (N, 3), with the complete dyadic Green's function.

pinneapple_simulation.electromagnetics.reflector.qz

Quiet-zone metrics of a collimated field (a compact range, a plane-wave synthesiser, a near-field probe scan).

pinneapple_simulation/electromagnetics/reflector/qz.py · 69 linhas · testes: 6

nome	tipo	assinatura	o que faz
disk_grid	função	(diameter, step, centre=(0.0, 0.0)) -> np.ndarray	Points (M, 2) of a square grid of step inside a disk of diameter.
qz_metrics	função	(xy, E_co, diameter, *, centre=(0.0, 0.0), E_x=None, amp_tol=0.5, phase_tol=5.0) -> dict[…]	Metrics of the co-polar field E_co (M,) complex at points xy (M, 2); see the module docstring.

pinneapple_simulation.electromagnetics.reflector.study

Compact-range studies: validation of the PO code, a (feed gain x section size) sweep, a surrogate with held-out error and the trade-off map of the quiet-zone quality.

pinneapple_simulation/electromagnetics/reflector/study.py · 212 linhas · testes: 6

nome	tipo	assinatura	o que faz
validate	função	(base, *, disk_radius_wl=3.0, mesh_densities=(8, 12, 16), rolls_wl=(3.0, 6.0), log=print)…)	The three checks of the module docstring, at the frequency of base.
fit_surrogates	função	(table, metrics=METRICS, *, test_frac=0.25, seed=0)	Per roll radius: a GP per metric on (feed gain, side), fitted on all rows, and two held-out RMSEs: models refitted without a random test_frac of the designs (interpolation inside the grid), and without every design of one interior side at a time (interpolation across sizes, the harder case). ...
tradeoff_map	função	(table, models, path, *, roll, qz_diameter, amp_tol=0.5, phase_tol=5.0, mark=None, metric…)	Two maps over (feed gain, side) from the GPs: the predicted metric and the predicted usable quiet-zone diameter (within +-amp_tol dB and +-phase_tol deg), with the simulated designs, the required diameter as a white line, the feasible region (usable >= qz_diameter) hatched (or a note when ...
qz_figure	função	(cr, path, *, length_unit=1.0, unit_name='m', title='') -> dict[str, Any]	Co-polar amplitude (dB re centre) and phase (deg re centre) over the evaluated region of the quiet-zone plane, the QZ circle, and the horizontal and vertical cuts through the centre. Returns the metrics.
qz_study	função	(base, gains, sides, rolls, out, *, mark=None, side_unit=1.0, side_label='section side [m…)	Sweep, surrogates with held-out error and one trade-off map per roll radius into out; table: an existing sweep to reuse instead of solving again.

pinneapple_simulation.electromagnetics.reflector.study_report

A compact-range study that plans itself: problem -> which checks and sweeps to run -> results -> datasheet.

pinneapple_simulation/electromagnetics/reflector/study_report.py · 1.018 linhas · testes: 14

nome	tipo	assinatura	o que faz
Problem	classe	(freq_hz, focal, section, offset, roll, feed_gain_dbi, qz_diameter, qz_z, n, pols, band_h…)	A compact-range design and what it must deliver.
PlannedStep	classe	(key, title, reason, required)	
from_text	função	(text, **defaults) -> Problem	:meth:Problem.from_text.
plan	função	(problem) -> list[PlannedStep]	The steps this problem needs, and **why** -- from thresholds on dimensionless numbers only.
field_on_points	função	(cr, pts, *, mesh_J=None) -> np.ndarray	PO field at absolute points pts (M, 3); the model is rebuilt unless mesh_J is given.

nome	tipo	assinatura	o que faz
qz_metrics_at	função	(cr, z=None, *, amp_tol=None, phase_tol=None) -> dict[str, float]	Disk metrics on the quiet-zone plane at z (default: the design plane).
cut_metrics_at	função	(cr, z=None, *, half=None, points=None, cuts=('horizontal', 'vertical')) -> dict[str, dic…)	Peak-to-peak amplitude and phase and peak cross-polar over cuts through the quiet-zone centre, referred to the centre point -- the definitions a range datasheet uses.
main_roll	função	(p) -> float	
run	função	(p, out='cr_study', *, steps=None, parallel=0, log=print, reuse=True) -> dict[str, Any]	Execute the plan (or the steps named) and return the study; every number is cached in <out>/study.json so the document can be re-rendered without solving again.
report	função	(p, study, out='report', *, pdf_name=None) -> dict[str, str | None]	Render the study as a technical datasheet (HTML, and PDF when a browser is available).

pinneapple_simulation.electromagnetics.reflector.study_report_figures

Figures for a compact-range study: geometry, illumination, validation, quiet zone, sweeps.

pinneapple_simulation/electromagnetics/reflector/study_report_figures.py · 431 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_figures	função	(p, study, out) -> dict[str, str]	Every figure of the report, as {key: png path}. A study that was not run loses its figures and nothing else.

pinneapple_simulation.electromagnetics.reflector.study_report_text

The datasheet of a compact-range study: every sentence is computed from the study's numbers.

pinneapple_simulation/electromagnetics/reflector/study_report_text.py · 578 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_datasheet	função	(p, study, figs) -> Datasheet	

pinneapple_simulation.electromagnetics.statics

Electrostatics and magnetostatics in closed form, and Biot-Savart integration of filamentary currents.

pinneapple_simulation/electromagnetics/statics.py · 293 linhas · testes: 50

nome	tipo	assinatura	o que faz
parallel_plate_capacitance	função	(area, gap, eps_r=1.0) -> float	C = eps A / d (fringing neglected), F.

nome	tipo	assinatura	o que faz
coax_capacitance	função	(a, b, eps_r=1.0) -> float	Per-unit-length capacitance of a coaxial line, 2 pi eps / ln(b/a) (F/m).
coax_inductance	função	(a, b, mu_r=1.0) -> float	External per-unit-length inductance of a coaxial line, mu ln(b/a) / (2 pi) (H/m).
spherical_capacitance	função	(a, b=math.inf, eps_r=1.0) -> float	Concentric spheres 4 pi eps a b / (b - a); isolated sphere 4 pi eps a for b = inf.
point_charge_field	função	(points, q, at=(0.0, 0.0, 0.0), eps_r=1.0)	(phi, E) of a point charge: phi = q / (4 pi eps r), E = q r_hat / (4 pi eps r^2).
line_charge_field	função	(points, lam, eps_r=1.0)	E of an infinite line charge on the z axis, lam / (2 pi eps s) radially (V/m).
infinite_wire_B	função	(points, current)	B of an infinite straight filament on the z axis carrying current towards +z: mu0 I / (2 pi s) in the azimuthal direction (T).
polyline_segments	função	(vertices) -> tuple[np.ndarray, np.ndarray]	Segments (start, end) of an open polyline through vertices (n, 3).
circle_segments	função	(radius, n=360, center=(0.0, 0.0, 0.0), z=0.0)	A circle of n straight segments in a plane z = const (counter-clockwise seen from +z).
helix_segments	função	(radius, length, turns, n_per_turn=72, z0=None)	A helical coil along z from z0 (default -length/2) to z0 + length.
biot_savart	função	(points, starts, ends, current=1.0, chunk=4096)	B (T) at points (..., 3) from straight filament segments starts -> ends (m, 3) each carrying current (A, scalar or per segment). Exact field of each finite segment (Jackson problem 5.1 / Griffiths eq. 5.37): B = mu0 I / (4 pi) (a x b)(|a| + |b|) / (|a| |b| (|a| |b| + a.b)) with ``a ...
vector_potential	função	(points, starts, ends, current=1.0, chunk=4096)	A (T m) at points (..., 3) from straight filament segments starts -> ends (Coulomb gauge), exact per segment: A = mu0 I / (4 pi) u ln((d1 + d2 + L) / (d1 + d2 - L)) with u the unit vector of the segment, L its length and d1, d2 the distances to its ends (Hanson and ...
flux_through_path	função	(path_starts, path_ends, starts, ends, current=1.0, gauss=4) -> float	Magnetic flux (Wb) of filaments starts -> ends through a closed polygonal path, by Stokes' theorem Phi = oint A . dl (Gauss-Legendre with gauss points per path segment). Unlike a surface integral of B it stays accurate when a filament pierces the spanning surface.
loop_on_axis_B	função	(z, radius, current)	Axial B of a circular loop in the plane z = 0: mu0 I R^2 / (2 (R^2 + z^2)^{3/2}).
loop_field	função	(points, radius, current)	B (T) of a circular loop of radius R in the plane z = 0, centred on the axis, everywhere off the wire, with complete elliptic integrals (m = k^2 = 4 R s / ((R + s)^2 + z^2))::

nome	tipo	assinatura	o que faz
`finite_solenoid_on_axis_B`	função	`(z, radius, length, n_turns, current)`	Axial B of a tightly wound finite solenoid (current sheet `K = N I / L`) centred at z = 0: `(mu0 N I / (2 L)) [(z + L/2) / sqrt(R^2 + (z + L/2)^2) - (z - L/2) / sqrt(R^2 + (z - L/2)^2)]`.
`helmholtz_coil_center_B`	função	`(radius, current, turns=1) -> float`	Field at the centre of a Helmholtz pair (spacing = radius): `(4/5)^{3/2} mu0 n I / R`.
`mutual_inductance_coaxial_loops`	função	`(r1, r2, d) -> float`	Maxwell's formula for two coaxial circular filaments at axial distance `d`: `M = mu0 sqrt(r1 r2) [(2/k - k) K(k^2) - (2/k) E(k^2)]`, `k^2 = 4 r1 r2 / ((r1 + r2)^2 + d^2)`.
`loop_self_inductance`	função	`(R, a, internal=True) -> float`	Thin round wire loop (loop radius `R` >> wire radius `a`): `mu0 R (ln(8R/a) - 2 + Y)` with `Y = 1/4` for uniform current (low frequency, `internal=True`) and 0 at high frequency (Rosa & Grover, *Formulas and tables for the calculation of mutual and self-inductance*, Bull. Bureau of ...
`filament_inductance`	função	`(paths, wire_radius, pieces_per_metre=2000.0, gauss=6, mirror_z=None) -> np.ndarray`	Inductance matrix (H) of thin conductors along polylines `paths` (each an (n, 3) array, closed or open; open paths give the partial inductances of their current segments).

pinneapple_simulation.electromagnetics.units

SI dimensions of electromagnetic quantities and a dimensional check of Maxwell's equations.

`pinneapple_simulation/electromagnetics/units.py` · 145 linhas · testes: 26

nome	tipo	assinatura	o que faz
`UnitProblem`	classe	`(where, message)`	
`dimension`	função	`(unit) -> Dim`	Dimension vector of a unit expression.
`term_dimension`	função	`(term, quantities=QUANTITIES) -> Dim`	Dimension of a product of factors (see :data:`EQUATIONS`).
`check_equations`	função	`(equations=EQUATIONS, quantities=QUANTITIES) -> list[UnitProblem]`	Every term of every law must have the dimension of the first term; returns the violations.
`check_units`	função	`(declared) -> list[UnitProblem]`	Compare declared units of named quantities (`{"E": "V/m", "B": "mT"}`) with :data:`QUANTITIES`.

pinneapple_simulation.electromagnetics.verification

Verification suite of the electromagnetics package: each case runs a solver or a reference implementation against an independent result (closed form, a published value, another library, a limit) and reports the measured error and, where a mesh or step is refined, the measured order of convergence.

`pinneapple_simulation/electromagnetics/verification.py` · 893 linhas · testes: 12

nome	tipo	assinatura	o que faz
`CaseResult`	classe	`(name, what, reference, error, tolerance, orders, expected_order, detail, seconds)`	
`case`	função	`(name)`	
`observed_orders`	função	`(errors, ratio=2.0) -> list[float]`	
`run`	função	`(names=None, quick=False) -> list[CaseResult]`	
`to_markdown`	função	`(results) -> str`	
`write_markdown_block`	função	`(path, results, marker='verification-table') -> None`	Replace the text between `<!-- marker:start -->` and `<!-- marker:end -->` in a Markdown file with the table of `results` and their notes, so a page shows exactly what the last run measured.

`pinneapple_simulation.electromagnetics.waves`

Closed-form wave electromagnetics, `exp(+j omega t)` convention (see :mod:`.media`).

`pinneapple_simulation/electromagnetics/waves.py` · 372 linhas · testes: 51

nome	tipo	assinatura	o que faz
`MultilayerResult`	classe	`(r, t, R, T)`	
`free_space_wavelength`	função	`(f) -> float`	
`fresnel`	função	`(n1, n2, theta_i)`	Amplitude reflection and transmission coefficients `(rs, rp, ts, tp)` at a planar interface from medium `n1` to `n2` (non-magnetic, complex indices allowed, `Im(n) <= 0` for loss).
`brewster_angle`	função	`(n1, n2) -> float`	`atan(n2 / n1)`: p-polarised reflection vanishes.
`critical_angle`	função	`(n1, n2) -> float`	`asin(n2 / n1)` for `n1 > n2` (total internal reflection above it).
`multilayer`	função	`(n_layers, d_layers, wavelength, theta_i=0.0, pol='s', n_in=1.0, n_out=1.0) -> Multilayer…)`	Reflection and transmission of a stack of planar layers (indices `n_layers`, thicknesses `d_layers` in m) between half-spaces `n_in` and `n_out`, by Born & Wolf's characteristic matrices. `wavelength` may be an array (vacuum wavelength, m). Field coefficients refer to E for s and to H for …
`skin_depth`	função	`(omega, sigma, mu_r=1.0, eps_r=1.0, exact=False)`	Skin depth (m). Good-conductor form `sqrt(2 / (omega mu sigma))`; `exact=True` gives `1 / |Im k|` with `k^2 = omega^2 mu eps - j omega mu sigma`, valid for any loss tangent.

nome	tipo	assinatura	o que faz
surface_impedance	função	(omega, sigma, mu_r=1.0)	Surface impedance of a good conductor, Zs = (1 + j) / (sigma delta) = (1 + j) sqrt(omega mu / (2 sigma)).
rect_waveguide_cutoff	função	(m, n, a, b, eps_r=1.0, mu_r=1.0) -> float	Cut-off frequency (Hz) of the TE_mn / TM_mn mode of an a x b rectangular guide: f_c = c / (2 pi sqrt(eps_r mu_r)) sqrt((m pi / a)^2 + (n pi / b)^2).
rect_waveguide_modes	função	(a, b, n_modes=10, eps_r=1.0, max_index=8)	The lowest n_modes modes as (f_c, "TE"/"TM", m, n) sorted by cut-off. TE needs m + n > 0, TM needs m, n >= 1.
rect_te_mn_fields	função	(x, y, m, n, a, b)	Transverse profile of TE_mn in an a x b guide (un-normalised): Hz = cos(m pi x/a) cos(n pi y/b), E_t proportional to (n/b cos(m pi x/a) sin(n pi y/b), -m/a sin(m pi x/a) cos(n pi y/b)).
circular_waveguide_cutoff	função	(kind, m, n, radius, eps_r=1.0) -> float	Cut-off (Hz) of TE_mn (n-th zero of J_m') or TM_mn (n-th zero of J_m) of a circular guide.
circular_waveguide_modes	função	(radius, n_modes=10, eps_r=1.0, max_m=6, max_n=4)	
propagation_constant	função	(f, f_cutoff, eps_r=1.0, mu_r=1.0)	beta = k sqrt(1 - (fc/f)^2) above cut-off; below it -j alpha (evanescent, exp(-j beta z)).
rect_cavity_resonance	função	(m, n, p, a, b, d, eps_r=1.0, mu_r=1.0) -> float	Resonance (Hz) of mode (m, n, p) of an a x b x d PEC box: f = c / (2 sqrt(eps_r mu_r)) sqrt((m/a)^2 + (n/b)^2 + (p/d)^2).
rect_cavity_modes	função	(a, b, d, n_modes=10, eps_r=1.0, max_index=6)	Lowest resonances of a PEC box with multiplicity: (f, [labels]). A mode exists as TE_mnp (relative to z) if (m or n) and p >= 1 and as TM_mnp if m, n >= 1; at least two indices must be non-zero.
cylindrical_cavity_resonance	função	(kind, m, n, p, radius, length, eps_r=1.0) -> float	TE_mnp (p >= 1) or TM_mnp (p >= 0) resonance of a closed circular cylinder (Pozar sec. 6.4).
rect_cavity_te101_q	função	(a, b, d, sigma, eps_r=1.0) -> float	Conductor-loss Q of the TE101 mode (Pozar eq. 6.46): Qc = (k a d)^3 b eta / (2 pi^2 Rs) / (2 a^3 b + 2 b d^3 + a^3 d + a d^3).
hertzian_dipole_fields	função	(r, theta, current_moment, omega)	Complete fields (E_r, E_theta, H_phi) of a z-directed Hertzian dipole with moment I l (A m) at the origin (Balanis eqs. 4-8a..c, exp(+j omega t))::
hertzian_dipole_power	função	(current_moment, omega) -> float	Time-averaged radiated power eta k^2 |I l|^2 / (12 pi) (W, peak phasor).
hertzian_dipole_radiation_resistance	função	(length, wavelength) -> float	R = (2 pi / 3) eta (l / lambda)^2 = 80 pi^2 (l/lambda)^2 ohm with eta = 120 pi.
dipole_pattern	função	(theta, length_wavelengths=0.5)	Far-field E_theta pattern of a thin centre-fed dipole of length L with sinusoidal current (Balanis eq. 4-58a): [cos(k L/2 cos t) - cos(k L/2)] / sin t (unnormalised). L = lambda/2 gives cos(pi/2 cos t) / sin t.

nome	tipo	assinatura	o que faz
cylinder_tm_coefficients	função	(ka, eps_r, n_max=None)	Coefficients (n, a_n, b_n) of the TM_z scattering by a cylinder of radius a (ka = k0 a): scattered Ez = E0 sum j^-n a_n H_n^(2)(k rho) e^{j n phi}, inside E0 sum j^-n b_n J_n(k1 rho) e^{j n phi} for the incident E0 exp(-j k x). eps_r=None is a perfect conductor (``a_n = ...
cylinder_scattering_tm	função	(x, y, k0, radius, eps_r, E0=1.0, total=True)	Phasor Ez at points (x, y) for a plane wave E0 exp(-j k0 x) (exp(+j omega t)) on a circular cylinder centred at the origin (eps_r=None: PEC). total=False returns the scattered field outside (and the total field inside).
half_wave_dipole_radiation_resistance	função	(eta=ETA0) -> float	R = eta / (4 pi) Cin(2 pi) with Cin(x) = gamma + ln x - Ci(x) (Balanis eq. 4-93): 73.08 ohm in free space (input resistance of an infinitely thin lambda/2 dipole).
half_wave_dipole_directivity	função	() -> float	
spp_wavenumber	função	(omega, eps_metal, eps_dielectric=1.0)	Surface plasmon polariton on a flat interface: k_sp = (omega/c) sqrt(eps_m eps_d / (eps_m + eps_d)) (Maier eq. 2.14), with Im(k_sp) <= 0 for a lossy metal under exp(+j omega t).
plasma_dispersion_omega	função	(k, omega_p)	Transverse waves in a collisionless plasma: omega^2 = omega_p^2 + c^2 k^2.

pinneapple_simulation.external_solvers

Ponto de entrada das integrações externas: reexporta OpenFOAM e, com imports opcionais (`try/except`), os leitores CFD, MATLAB, Modelica, FEniCS, MuJoCo, Genesis e Newton, para o pacote importar mesmo sem as ferramentas instaladas.

`pinneapple_simulation/external_solvers/__init__.py` · 226 linhas · testes: 16

pinneapple_simulation.external_solvers.ansys

Pacote do acoplamento com ANSYS Fluent em modo batch (journal `.jou`); reexporta `journal_builder` e `runner`. [sem teste]

`pinneapple_simulation/external_solvers/ansys/__init__.py` · 23 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.ansys.journal_builder

Gera o texto de um journal TUI do Fluent (ler caso/malha, condições de contorno simples, inicializar, iterar, gravar) a partir de `FluentCaseConfig`. O docstring separa comandos de alta e média confiança e lista o que não foi implementado; nunca foi rodado contra um Fluent real.

`pinneapple_simulation/external_solvers/ansys/journal_builder.py` · 188 linhas · testes: 15 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`FluentCaseConfig`	classe	`(output_case_file, case_file, mesh_file, steady, iterations, time_steps, max_iterations_p…)`	Configuração de uma rodada batch do Fluent: caso ou malha de entrada, regime permanente/transiente, iterações e condições de velocity-inlet/pressure-outlet.
`build_journal`	função	`(config) -> str`	Gera o texto do journal TUI do Fluent para uma `FluentCaseConfig`, na ordem ler, contorno, inicializar, resolver, gravar e sair.

pinneapple_simulation.external_solvers.ansys.runner

Lança o executável `fluent` real com o journal gerado, guarda logs e entrega o resultado ao leitor `fluent_mesh_to_upd`. Só os caminhos de geração de journal e de 'executável não encontrado' são testados (não há Fluent licenciado no ambiente).

`pinneapple_simulation/external_solvers/ansys/runner.py` · 182 linhas · testes: 15

nome	tipo	assinatura	o que faz
`FluentRunConfig`	classe	`(executable, precision, n_procs, work_dir, timeout_s, extra_args)`	Como invocar o executável do Fluent (nome, precisão, número de processos, pasta, timeout), sem dizer o que simular.
`FluentRunResult`	classe	`(success, upd_result, exit_code, wall_time_s, stdout_log_path, stderr_log_path, journal_p…)`	Resultado de uma rodada do Fluent: sucesso, `PhysicalSample` lido, código de saída, tempo, caminhos de log e journal, e erro de leitura.
`run_fluent_case`	função	`(case_config, run_config=None) -> FluentRunResult`	Monta o journal, lança o Fluent em batch e empacota o resultado com `fluent_mesh_to_upd`; falha explicitamente se o executável não existir.

pinneapple_simulation.external_solvers.calculix

Ponte para o CalculiX (FEM estrutural implícito): decks `.inp`, execução do `ccx`, leitura de `.frd` e um `ccx` virtual.

`pinneapple_simulation/external_solvers/calculix/__init__.py` · 23 linhas · testes: 14

pinneapple_simulation.external_solvers.calculix.frd

Lê e escreve arquivos de resultado `.frd` do CalculiX (ASCII) e lê deslocamentos nodais de `.dat`.

`pinneapple_simulation/external_solvers/calculix/frd.py` · 135 linhas · testes: 7

nome	tipo	assinatura	o que faz
`FrdResults`	classe	`(node_ids, coords, steps)`	Resultados lidos de um `.frd`: ids e coordenadas dos nós e os passos com seus campos (acesso por `field`).
`read_frd`	função	`(path) -> FrdResults`	Lê um `.frd` ASCII do CalculiX em `FrdResults`.
`write_frd`	função	`(path, node_ids, coords, elements=None, element_type='C3D8', fields=None, step_time=1.0, …)`	Escreve um `.frd` ASCII com nós, elementos e blocos `DISP`/`STRESS`.

nome	tipo	assinatura	o que faz
`read_dat_displacements`	função	`(path) -> Dict[int, np.ndarray]`	Lê deslocamentos nodais de um arquivo `.dat` (saída `*NODE PRINT`).

`pinneapple_simulation.external_solvers.calculix.inp`

Contêiner simples de modelo CalculiX, escritor e leitor de decks `.inp`, e uma malha hexaédrica de viga em balanço. [sem teste direto]

`pinneapple_simulation/external_solvers/calculix/inp.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CalculixModel`	classe	`(nodes, elements, element_type, nsets, material, E, nu, density, boundary, cloads, title)`	Modelo CalculiX mínimo: nós, elementos, conjuntos, material elástico, apoios e cargas pontuais.
`write_inp`	função	`(model, path, *, job_outputs=('U',), stress_outputs=('S',)) -> str`	Escreve um `CalculixModel` como deck `.inp` com pedidos de saída de deslocamento e tensão.
`read_inp`	função	`(path) -> CalculixModel`	Lê malha, material, apoios e cargas pontuais de um deck `.inp`, seguindo `*INCLUDE`.
`expand_target`	função	`(model, target) -> List[int]`	Devolve os ids de nós por trás de um alvo de `*BOUNDARY`/`*CLOAD` (id de nó ou nome de NSET).
`cantilever_hex_mesh`	função	`(L=1.0, b=0.1, h=0.1, nx=20, ny=2, nz=2, element_type='C3D20R') -> CalculixModel`	Cria uma viga em balanço de hexaedros ao longo de x, engastada em x = 0, com elementos de 8 ou 20 nós.

`pinneapple_simulation.external_solvers.calculix.runner`

Executa `ccx` com o binário nativo se estiver no PATH, senão com uma imagem Docker construída localmente a partir de um `Dockerfile` (Debian `calculix-ccx`).

`pinneapple_simulation/external_solvers/calculix/runner.py` · 100 linhas · testes: 14

nome	tipo	assinatura	o que faz
`CcxRun`	classe	`(returncode, stdout, frd, dat)`	Resultado de uma execução do `ccx`: código de retorno, saída e caminhos do `.frd` e `.dat`.
`docker_available`	função	`() -> bool`	Diz se o Docker responde (`docker info`).
`ensure_image`	função	`(image=IMAGE, attempts=3) -> str`	Constrói a imagem Docker do CalculiX se faltar, tentando registries alternativos a cada falha.
`ccx_backend`	função	`() -> str | None`	Diz qual backend está disponível: 'native', 'docker' ou None.
`run_ccx`	função	`(workdir, job, *, threads=1, timeout=3600.0, backend=None) -> CcxRun`	Roda `ccx -i <job>` numa pasta que contém `<job>.inp`, pelo backend escolhido.

pinneapple_simulation.external_solvers.calculix.study

Estudos CalculiX em poucas linhas: malha (hexaedros, tetraedros gmsh ou deck existente), conjuntos por geometria, um passo de análise (estático, frequência, flambagem, calor), execução e resultados em arrays.

pinneapple_simulation/external_solvers/calculix/study.py · 503 linhas · testes: 7

nome	tipo	assinatura	o que faz
Material	classe	(name, E, nu, density, conductivity, expansion, plastic)	Material do estudo (E, nu, densidade, condutividade, expansão, plasticidade) e seu trecho de deck.
FEModel	classe	(nodes, elements, element, material, title)	Modelo de elementos finitos do estudo: malha (caixa, hexmesh, .inp ou gmsh), seleção de nós e faces por geometria, e o deck.
Static	classe	(fix, loads, pressure, gravity, nlgeom, increments, prescribed)	Passo estático linear ou não linear: apoios, forças, pressões, gravidade e deslocamentos prescritos.
Frequency	classe	(n_modes, fix)	Passo de frequências naturais e modos de vibração (requer densidade).
Buckle	classe	(n_modes, fix, loads, pressure)	Passo de flambagem linear: fatores de carga das cargas de referência e formas de flambagem.
Heat	classe	(temperature, film, flux, initial)	Passo de condução de calor permanente com temperaturas, convecção (film) e fluxos.
Result	classe	(model, u, stress, strain, reactions, peeq, temperature, frequencies, buckling_factors, m…)	Resultados de um estudo: deslocamentos, tensões, deformações, reações, temperaturas, frequências e modos, com von Mises e deslocamento máximo.
solve	função	(model, step, workdir, *, threads=2, keep=True) -> Result	Escreve workdir/job.inp, roda o ccx e lê os resultados num Result.
geo_plate_with_hole	função	(L, W, t, d) -> str	Gera a geometria (texto para gmsh) de uma placa com furo central.
geo_l_bracket	função	(a=0.1, b=0.08, w=0.04, t=0.01, hole=0.012, fillet=0.01) -> str	Gera a geometria (texto para gmsh) de uma cantoneira em L com filete e furo de parafuso.

pinneapple_simulation.external_solvers.calculix.virtual_ccx

Um ccx substituto (pinneapple-ccx): mesma linha de comando, lê o .inp e escreve o .frd com campos previstos por um modelo (PINN/surrogate), recusando pontos fora do envelope de validade ou caindo no ccx real com --fallback.

pinneapple_simulation/external_solvers/calculix/virtual_ccx.py · 117 linhas · testes: 7

nome	tipo	assinatura	o que faz
OutOfEnvelope	classe	()	Erro levantado quando o caso pedido sai do envelope de validade do modelo preditor.
check_envelope	função	(model, coords, envelope, tol=1e-06) -> None	Confere caixa da geometria, módulo E e cargas contra o envelope declarado pelo preditor e levanta OutOfEnvelope se sair.

nome	tipo	assinatura	o que faz
`predict_to_frd`	função	`(inp_path, predictor, frd_path=None) -> str`	Lê um `.inp`, prevê os campos nodais com o preditor e escreve o `.frd` correspondente.
`load_predictor`	função	`(spec) -> Callable`	Carrega um preditor a partir de uma string `pacote.modulo:atributo`.
`main`	função	`(argv=None) -> int`	Linha de comando `pinneapple-ccx -i job [--predictor ...] [--fallback]`, espelhando `ccx -i job`.

`pinneapple_simulation.external_solvers.campaign_cases`

Escreve os arquivos de uma campanha de simulação planejada: tabelas de casos, um runner Python, casos OpenFOAM (aerofólio CST ou cópias de um caso base) e decks CalculiX, mais contratos e README; usado pelo app Campaign Planner.

`pinneapple_simulation/external_solvers/campaign_cases.py` · 349 linhas · testes: 7

nome	tipo	assinatura	o que faz
`write_case_tables`	função	`(cases, out_dir, space=None, stem='cases', formats=('csv', 'parquet', 'json')) -> list[Pa…)`	Grava a tabela de casos em CSV, Parquet e/ou JSON, com o espaço de parâmetros ao lado.
`write_python_runner`	função	`(out_dir, parameters, outputs=()) -> Path`	Gera `run_cases.py`, que percorre os casos e chama a função `simulate` do usuário (o modelo gerado levanta `NotImplementedError` até ser preenchido).
`airfoil_space`	função	`(budget=40) -> ParameterSpace`	Espaço de parâmetros da campanha de aerofólio: pesos CST de espessura e arqueamento e ângulo de ataque.
`airfoil_feasible`	função	`(cases) -> np.ndarray`	Marca as formas de aerofólio válidas (superfícies que não se cruzam e permanecem suaves).
`airfoil_existing`	função	`()`	Devolve como DataFrame as rodadas OpenFOAM já existentes no dataset do zoo `airfoil_rans_cst_re4e6`.
`write_openfoam_airfoil`	função	`(cases, out_dir, reynolds=AIRFOIL_REYNOLDS, iterations=1200, limit=None) -> list[Path]`	Escreve um caso simpleFoam por linha (aerofólio CST) com `params.json`, `coefficients.py` e `Allrun`.
`write_openfoam_from_base`	função	`(base, cases, out_dir, edits, derived=None, solver='simpleFoam', procs=1, pre=('checkMesh…)`	Copia um caso OpenFOAM base aplicando edições de dicionário por caso e grava o `plan.json` do runner de campanhas.
`cantilever_space`	função	`(budget=30) -> ParameterSpace`	Espaço de parâmetros de uma viga em balanço com carga na ponta (dimensões, material, carga), esbelta para valer a teoria de vigas.
`write_calculix_cantilever`	função	`(cases, out_dir, elements_per_height=2) -> list[Path]`	Escreve um `job.inp` CalculiX por caso da viga em balanço, com a deflexão de Euler-Bernoulli em `params.json`.
`write_calculix_from_template`	função	`(template, cases, out_dir, name='job.inp') -> list[Path]`	Preenche os marcadores `{parametro}` de um deck CalculiX para cada caso.

nome	tipo	assinatura	o que faz
`write_campaign`	função	`(plan, out_dir, *, outputs=(), units=None, openfoam=None, calculix=None, openfoam_base=No…)`	Grava um `CampaignPlan` inteiro: tabelas, runner, casos OpenFOAM/CalculiX que se aplicam, contratos e README.

`pinneapple_simulation.external_solvers.cfd_formats`

Pacote de leitores de formatos abertos CFD/FEM além do OpenFOAM: CGNS, Exodus II, malha Fluent (só nós), ponte Abaqus `.inp`/`.odb` e exportações do STAR-CCM+. [sem teste direto do `__init__`]

`pinneapple_simulation/external_solvers/cfd_formats/__init__.py` · 48 linhas · **sem teste direto**

`pinneapple_simulation.external_solvers.cfd_formats._abaqus_odb_export_script`

Script que roda dentro do Python do próprio Abaqus (`abaqus python ...`) para exportar campos de um `.odb` para `.npz`; nunca é importado pelo código normal. [sem teste]

`pinneapple_simulation/external_solvers/cfd_formats/_abaqus_odb_export_script.py` · 87 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`()`	Lê os argumentos JSON, abre o `.odb` com a API do Abaqus e grava os campos pedidos num `.npz`.

`pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader`

Lê nós e elementos de um `.inp` do Abaqus e faz a ponte para resultados `.odb` chamando o Abaqus instalado como subprocesso; empacota tudo como `PhysicalSample`.

`pinneapple_simulation/external_solvers/cfd_formats/abaqus_reader.py` · 320 linhas · testes: 5

nome	tipo	assinatura	o que faz
`read_abaqus_inp_mesh`	função	`(path)`	Extrai os blocos `*NODE` e `*ELEMENT` de um `.inp` do Abaqus.
`abaqus_inp_to_upd`	função	`(path)`	Empacota a malha de um `.inp` (sem campos de resultado) como `PhysicalSample`.
`export_odb_fields`	função	`(odb_path, out_npz_path, *, step_name=None, frame_index=-1, field_outputs=('U', 'S'), aba…)`	Exporta campos de um `.odb` para `.npz` rodando o Python do Abaqus instalado.
`load_exported_odb_npz`	função	`(npz_path)`	Carrega o `.npz` exportado de um `.odb` como `PhysicalSample`.

`pinneapple_simulation.external_solvers.cfd_formats.cgns_reader`

Lê coordenadas e campos de solução de arquivos CGNS/HDF5 e os empacota como `PhysicalSample` com domínio de malha.

pinneapple_simulation/external_solvers/cfd_formats/cgns_reader.py · 186 linhas · testes: 14

nome	tipo	assinatura	o que faz
read_cgns_mesh_and_fields	função	(path, *, fields=None, zone_index=0)	Lê coordenadas de nós e campos de solução de um arquivo CGNS/HDF5.
cgns_to_upd	função	(path, *, fields=None, zone_index=0)	Lê um CGNS/HDF5 e empacota como PhysicalSample com geometry.nodes.

pinneapple_simulation.external_solvers.cfd_formats.convert

Despacho por formato de arquivos CFD/FEM/CAD: nuvem de pontos portátil de qualquer formato suportado, superfície triangulada dos formatos com conectividade, e reparo/simplificação na conversão.

pinneapple_simulation/external_solvers/cfd_formats/convert.py · 142 linhas · testes: 6

nome	tipo	assinatura	o que faz
detect_format	função	(filename) -> str	Devolve o nome do formato pelo sufixo do arquivo, ou ValueError listando os suportados.
read_point_bundle	função	(path, fmt=None) -> dict[str, Any]	Lê um formato volumétrico como dicionário de coordenadas, campos, proveniência e formato.
tet_boundary_faces	função	(tets) -> np.ndarray	Devolve os triângulos usados por um só tetraedro, ou seja, a fronteira de uma malha tetraédrica.
read_surface_mesh	função	(path, fmt=None)	Lê uma superfície triangulada (MeshData) de STL, OBJ, PLY, STEP ou .inp do Abaqus.
convert_surface	função	(path, out_path, *, fmt=None, repair=False, simplify_target_faces=None) -> dict[str, Any]	Lê uma superfície, opcionalmente repara e simplifica, e grava em STL, OBJ ou PLY.

pinneapple_simulation.external_solvers.cfd_formats.exodus_reader

Lê coordenadas e campos nodais de Exodus II clássico (NetCDF-3) via scipy.io.netcdf_file; arquivos NetCDF-4/HDF5 levantam NotImplementedError com a explicação, e a conectividade de elementos não é lida.

pinneapple_simulation/external_solvers/cfd_formats/exodus_reader.py · 177 linhas · testes: 5 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
read_exodus	função	(path, *, fields=None, time_index=-1)	Lê coordenadas e campos nodais de um Exodus II em NetCDF clássico.
exodus_to_upd	função	(path, *, fields=None, time_index=-1)	Lê um Exodus II e empacota como PhysicalSample com domínio de malha.

pinneapple_simulation.external_solvers.cfd_formats.fluent_mesh_reader

Lê só as coordenadas de nós (seção 10, ASCII) de malhas .msh do Fluent/Gambit; células, faces, zonas e seções binárias levantam NotImplementedError de propósito.

`pinneapple_simulation/external_solvers/cfd_formats/fluent_mesh_reader.py` · 162 linhas · testes: 5 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`read_fluent_mesh_nodes`	função	`(path)`	Lê as coordenadas de nós das seções 10 de um `.msh` ASCII do Fluent/Gambit.
`fluent_mesh_to_upd`	função	`(path)`	Empacota os nós de um `.msh` do Fluent como `PhysicalSample` sem campos.

`pinneapple_simulation.external_solvers.cfd_formats.starccm_reader`

Lê exportações do STAR-CCM+: CGNS (reaproveita `cgns_reader`) e tabelas texto/CSV (parser próprio). O formato nativo `.sim`/`.ccm` não é suportado, por ser binário proprietário.

`pinneapple_simulation/external_solvers/cfd_formats/starccm_reader.py` · 243 linhas · testes: 9 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`read_starccm_cgns`	função	`(path, *, fields=None, zone_index=0)`	Lê uma exportação CGNS do STAR-CCM+, delegando a `read_cgns_mesh_and_fields`.
`starccm_cgns_to_upd`	função	`(path, *, fields=None, zone_index=0)`	Empacota uma exportação CGNS do STAR-CCM+ como `PhysicalSample`, delegando a `cgns_to_upd`.
`read_starccm_table`	função	`(path, *, coord_names=('X', 'Y', 'Z'))`	Lê uma tabela texto/CSV exportada do STAR-CCM+, com cabeçalhos que podem ter unidades.
`starccm_table_to_upd`	função	`(path, *, coord_names=('X', 'Y', 'Z'))`	Empacota uma tabela exportada do STAR-CCM+ como `PhysicalSample` com as coordenadas como nós.

`pinneapple_simulation.external_solvers.fenics`

Pacote da integração FEniCS; reexporta a camada de workflow de `fenics.solver`.

`pinneapple_simulation/external_solvers/fenics/__init__.py` · 30 linhas · **sem teste direto**

`pinneapple_simulation.external_solvers.fenics.solver`

Camada de workflow sobre `FEniCSBridge` (em `pinneapple_simulation.numerical_solvers.fenics_bridge`): configuração, solve com empacotamento em `PhysicalSample` e varreduras de parâmetros. Requer FEniCS instalado.

`pinneapple_simulation/external_solvers/fenics/solver.py` · 180 linhas · testes: 151

nome	tipo	assinatura	o que faz
`FEniCSConfig`	classe	`(pde, domain, bcs, params, solver_opts)`	Configuração de um solve FEniCS: identificador da EDP, domínio, condições de contorno, parâmetros e opções do solver.
`FEniCSWorkflow`	classe	`(base_config) -> None`	Atalho para solves FEniCS repetidos sobre uma configuração base, com `solve` e `sweep` em grade de parâmetros.
`solve_and_package`	função	`(config)`	Roda um solve via `FEniCSBridge` e devolve um `PhysicalSample`.

nome	tipo	assinatura	o que faz
`dof_to_upd`	função	`(dof_vector, mesh_coords, field_name='u')`	Empacota um vetor de DOFs do FEniCS e as coordenadas da malha como `PhysicalSample`.

pinneapple_simulation.external_solvers.fluidx3d

Ponte para o FluidX3D (LBM em GPU, OpenCL): escreve o `main_setup()` de um túnel de vento no checkout do usuário, compila e roda com `make.sh` e lê forças e campos `.vtk`. O PINNeAPPle não distribui código do FluidX3D, e o uso segue a licença dele (proíbe uso comercial e militar).

`pinneapple_simulation/external_solvers/fluidx3d.py` · 225 linhas · testes: 9

nome	tipo	assinatura	o que faz
`WindTunnelSetup`	classe	`(length_m, velocity_m_s, nu_m2_s, rho_kg_m3, stl, box, memory_mb, body_cells, lbm_velocit…)`	Descrição de um túnel de vento no FluidX3D: corpo STL (ou esfera), velocidade, viscosidade, caixa, memória, tempo e exportações.
`fluidx3d_home`	função	`() -> str | None`	Devolve a pasta do checkout do FluidX3D (`FLUIDX3D_HOME`) ou None.
`fluidx3d_available`	função	`() -> bool`	Diz se há um checkout configurado e um driver OpenCL instalado.
`write_setup`	função	`(setup, home=None) -> str`	Substitui o `main_setup()` de `src/setup.cpp` pelo gerado, guardando backup do original.
`enable_features`	função	`(features, home=None) -> str`	Ativa `#defines` em `src/defines.hpp` (com backup) e desliga `BENCHMARK`.
`run_fluidx3d`	função	`(setup=None, home=None, device=None, timeout=24 * 3600, log='log.pinneapple') -> subproce…)`	Escreve setup e defines, compila e roda com `make.sh`, gravando a saída num log.
`parse_forces`	função	`(text) -> np.ndarray`	Extrai as linhas (t, Fx, Fy, Fz) impressas pelo setup gerado.
`read_results`	função	`(home=None, log='log.pinneapple', export_dir=None) -> dict`	Lê o histórico de forças do log e o último campo exportado de cada grandeza.
`results_to_samples`	função	`(results, source='fluidx3d') -> list`	Converte os últimos campos exportados numa lista de `PhysicalSample` em grade regular.

pinneapple_simulation.external_solvers.genesis

Pacote da integração com Genesis AI (simulação multifísica para robótica); reexporta configuração, construtor de cena, runner e adaptadores. [sem teste]

`pinneapple_simulation/external_solvers/genesis/__init__.py` · 70 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.genesis.adapter

Converte dicionários de trajetória do Genesis em `PhysicalSample` (UPD). [sem teste]

`pinneapple_simulation/external_solvers/genesis/adapter.py` · 96 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`genesis_traj_to_upd`	função	`(traj, time_array=None, field_map=None, meta_extra=None, dt=None)`	Converte um dicionário de trajetória do Genesis num `PhysicalSample`.
`genesis_trajs_to_upd`	função	`(trajs, time_array=None, field_map=None, meta_extra=None, dt=None) -> list`	Converte uma lista de trajetórias do Genesis em `PhysicalSamples`.

`pinneapple_simulation.external_solvers.genesis.runner`

Executa o laço `scene.step()` de uma cena Genesis e coleta estados (corpo rígido, partículas MPM/SPH ou função própria) por passo. [sem teste]

`pinneapple_simulation/external_solvers/genesis/runner.py` · 125 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GenesisRunner`	classe	`(config, state_fn=None, controller_fn=None) -> None`	Executa uma cena Genesis e coleta observações de trajetória por passo de tempo.

`pinneapple_simulation.external_solvers.genesis.scene_builder`

Configuração de cena Genesis (backend, passo, ambientes paralelos, entidades, opções de corpo rígido/MPM/SPH) e a função que a materializa num `gs.Scene`. [sem teste]

`pinneapple_simulation/external_solvers/genesis/scene_builder.py` · 180 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EntitySpec`	classe	`(morph, material, surface, vis_mode, name)`	Especificação de uma entidade da cena Genesis (morph, material, superfície, nome).
`GenesisConfig`	classe	`(backend, precision, dt, substeps, n_steps, n_envs, env_spacing, gravity, requires_grad, …)`	Configuração completa de uma cena Genesis: backend, precisão, passo, ambientes, gravidade, entidades e opções por física.
`build_scene`	função	`(config)`	Materializa uma `GenesisConfig` num `gs.Scene`.

`pinneapple_simulation.external_solvers.geotaichi`

Ponte para o GeoTaichi (MPM, DEM, FEM, IGA; licença GPL-3.0), rodado como programa separado: executa um script e lê os quadros de partículas `.npz`, que também servem para o solver `geo_mpm` do próprio PINNeAPPle.

`pinneapple_simulation/external_solvers/geotaichi.py` · 89 linhas · testes: 4

nome	tipo	assinatura	o que faz
`geotaichi_home`	função	`() -> str | None`	Devolve a pasta do checkout GeoTaichi (`GEOTAICHI_HOME`) ou None.

nome	tipo	assinatura	o que faz
`geotaichi_available`	função	`() -> bool`	Diz se há um checkout do GeoTaichi configurado e se o Taichi é importável pelo interpretador que vai rodá-lo.
`run_geotaichi`	função	`(script, workdir, args=(), python=None, timeout=24 * 3600, log='log.geotaichi') -> subpro…)`	Roda um script do GeoTaichi num processo próprio dentro de `workdir`, com log.
`read_geotaichi`	função	`(path, kind='MPM', allow_pickle=False) -> list[dict]`	Lê os quadros de partículas de uma rodada; entradas pickled são puladas, salvo `allow_pickle=True`.
`frames_to_samples`	função	`(frames, source='geotaichi', fields=None, times=None) -> list`	Converte cada quadro em um `PhysicalSample`: posições como nuvem de pontos e os demais arrays como campos.

pinneapple_simulation.external_solvers.matlab

Pacote da integração MATLAB: Engine API, runner por subprocesso e E/S de `.mat`.

`pinneapple_simulation/external_solvers/matlab/__init__.py` · 27 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.matlab.engine

Invólucro fino sobre `matlab.engine.MatlabEngine` (chamar funções, rodar scripts, avaliar, trocar variáveis). Requer a Engine API do MATLAB.

`pinneapple_simulation/external_solvers/matlab/engine.py` · 88 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MATLABEngine`	classe	`(start_options=None) -> None`	Invólucro da Engine do MATLAB com `run_function`, `run_script`, `eval`, `get`, `put` e `quit`.

pinneapple_simulation.external_solvers.matlab.field_reader

Converte a saída `.mat` de um solver MATLAB em `PhysicalSample`. [sem teste]

`pinneapple_simulation/external_solvers/matlab/field_reader.py` · 61 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mat_to_upd`	função	`(mat_path, *, field_keys=None, coord_keys=None, units=None)`	Carrega um `.mat` e empacota seus arrays como `PhysicalSample`.

pinneapple_simulation.external_solvers.matlab.mat_io

Lê e grava arquivos `.mat` com `scipy.io`. [sem teste]

`pinneapple_simulation/external_solvers/matlab/mat_io.py` · 34 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_mat`	função	`(path) -> Dict[str, np.ndarray]`	Carrega um `.mat` como dicionário de arrays numpy.
`save_mat`	função	`(path, arrays) -> None`	Grava um dicionário de arrays (ou escalares) num `.mat`.

pinneapple_simulation.external_solvers.matlab.runner

Roda scripts e funções MATLAB por subprocesso em modo `-batch`, trocando dados por `.mat`. [sem teste]

`pinneapple_simulation/external_solvers/matlab/runner.py` · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_matlab_script`	função	`(script_path, *, matlab_executable='matlab', timeout=300, extra_flags=None) -> subprocess…)`	Roda um script MATLAB por subprocesso em modo `-batch`.
`run_matlab_function`	função	`(func_name, args, out_mat_path, *, matlab_executable='matlab', timeout=300) -> None`	Chama uma função MATLAB por um script-invólucro gerado e grava a saída num `.mat`.

pinneapple_simulation.external_solvers.modelica

Pacote da integração Modelica: simulação de FMUs (fmpy), OpenModelica (OMPython ou `omc` direto), inspeção de FMUs, compatibilidade FMI e varreduras.

`pinneapple_simulation/external_solvers/modelica/__init__.py` · 35 linhas · testes: 5

pinneapple_simulation.external_solvers.modelica.fmi_compat

Roda FMUs por `simulate_fmu` em cada interface que oferecem (Co-Simulation, Model Exchange) e monta a tabela de compatibilidade pedida pela lista de ferramentas FMI.

`pinneapple_simulation/external_solvers/modelica/fmi_compat.py` · 122 linhas · testes: 2

nome	tipo	assinatura	o que faz
`CompatRow`	classe	`(fmu, fmi_version, interface, exporter, platform, passed, detail)`	Uma linha da tabela de compatibilidade FMI: FMU, versão, interface, exportador, plataforma, se passou e detalhe.
`platform_tag`	função	`() -> str`	Devolve a plataforma binária FMI desta máquina (por exemplo `x86_64-linux`).
`check_fmu`	função	`(fmu, fmi_type, exporter='', stop_time=None, tol=None) -> CompatRow`	Simula uma FMU numa interface e informa se funcionou.
`check_fmus`	função	`(fmus, exporter='', interfaces=…) -> list[CompatRow]`	Aplica `check_fmu` a cada FMU e a cada interface que ela implementa.
`markdown_table`	função	`(rows) -> str`	Monta tabelas Markdown por ferramenta exportadora e versão FMI, com células 'CS, ME'.

pinneapple_simulation.external_solvers.modelica.fmi_runner

Simula uma FMU com fmpy e devolve as saídas como arrays ou como `PhysicalSample`.

`pinneapple_simulation/external_solvers/modelica/fmi_runner.py` · 107 linhas · testes: 2

nome	tipo	assinatura	o que faz
`FMUSimConfig`	classe	`(start_time, stop_time, step_size, output_interval, parameters, output_variables, fmi_typ…)`	Configuração de uma simulação de FMU: tempos, passo, intervalo de saída, parâmetros, variáveis e tipo FMI.
`simulate_fmu`	função	`(fmu_path, config) -> Dict[str, np.ndarray]`	Simula uma FMU com fmpy e devolve as variáveis de saída como arrays.
`fmu_to_upd`	função	`(fmu_path, config)`	Simula uma FMU e empacota o resultado como `PhysicalSample`.

`pinneapple_simulation.external_solvers.modelica.fmu_inspect`

Inspeção estática de um `.fmu` antes de usá-lo: versão FMI, interfaces, binários por plataforma, símbolos exportados de ELF64 contra os exigidos pelo padrão e consistência do modelDescription.

`pinneapple_simulation/external_solvers/modelica/fmu_inspect.py` · 233 linhas · testes: 6

nome	tipo	assinatura	o que faz
`BinaryCheck`	classe	`(platform, path, format, exported, missing)`	Resultado da checagem de um binário da FMU: plataforma, caminho, formato, funções exportadas e faltantes.
`FMUReport`	classe	`(path, fmi_version, model_name, interfaces, binaries, problems, n_variables)`	Relatório de inspeção de uma FMU: versão, modelo, interfaces, binários, problemas e número de variáveis.
`elf_exported_functions`	função	`(data) -> List[str]`	Lista as funções globais exportadas na tabela `.dynsym` de um objeto ELF64 little-endian.
`inspect_fmu`	função	`(path) -> FMUReport`	Inspeciona estaticamente um `.fmu` sem carregar nem executar código.

`pinneapple_simulation.external_solvers.modelica.om_runner`

Carrega e simula um modelo Modelica com OpenModelica via OMPython e empacota o resultado como `PhysicalSample`. [sem teste]

`pinneapple_simulation/external_solvers/modelica/om_runner.py` · 211 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OMSimConfig`	classe	`(model_name, start_time, stop_time, number_of_intervals, tolerance, parameters, output_va…)`	Configuração de uma simulação OpenModelica: modelo, tempos, intervalos, tolerância, parâmetros e saídas.
`simulate_openmodelica`	função	`(model_file, config, *, working_dir=None) -> Dict[str, np.ndarray]`	Carrega e simula um modelo Modelica com OpenModelica e devolve arrays por variável.
`om_to_upd`	função	`(model_file, config, **sim_kwargs)`	Simula com OpenModelica e devolve um `PhysicalSample`.

`pinneapple_simulation.external_solvers.modelica.omc`

Usa o compilador `omc` sem OMPython, local ou pela imagem Docker oficial, para gerar FMUs e simular arquivos `.mo`.

pinneapple_simulation/external_solvers/modelica/omc.py · 140 linhas · testes: 5

nome	tipo	assinatura	o que faz
omc_availablle	função	() -> str | None	Diz se o omc está disponível: 'local', 'docker' ou None.
run_omc_script	função	(script, workdir, timeout=1800.0) -> str	Roda um script .mos do omc numa pasta e devolve a saída, com RuntimeError em caso de erro.
build_fmu	função	(models, model_name, out_dir, fmi_version='2.0', fmu_type='cs', libraries=()) -> Path	Compila um modelo de arquivos .mo numa FMU.
simulate_mo	função	(models, model_name, stop_time=1.0, start_time=0.0, number_of_intervals=500, tolerance=1e…)	Simula um modelo com omc e devolve {variavel: array} incluindo time.

pinneapple_simulation.external_solvers.modelica.result_reader

Converte dicionários de resultado Modelica/FMI em PhysicalSample. [sem teste]

pinneapple_simulation/external_solvers/modelica/result_reader.py · 53 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
modelica_result_to_upd	função	(result, *, time_key='time', field_keys=None, units=None, source='modelica')	Empacota um dicionário de resultado Modelica como PhysicalSample.

pinneapple_simulation.external_solvers.modelica.sweep

Varreduras de parâmetros de uma FMU por hipercubo latino, gerando dados de treino e teste para surrogates de um modelo Modelica.

pinneapple_simulation/external_solvers/modelica/sweep.py · 86 linhas · testes: 5

nome	tipo	assinatura	o que faz
FMUDataset	classe	(names, params, time, outputs, bounds)	Dataset de uma varredura de FMU: nomes, parâmetros, tempo, saídas e limites, com split, save e load.
latin_hypercube	função	(n, d, seed=0) -> np.ndarray	Gera n pontos em [0, 1]^d por hipercubo latino.
fmu_sweep	função	(fmu, ranges, n, stop_time, output_interval, outputs, seed=0, fixed=None, start_time=0.0,…)	Simula n amostras de hipercubo latino dos intervalos de parâmetros numa grade de saída comum.

pinneapple_simulation.external_solvers.mujoco

Pacote da integração MuJoCo: carregamento de modelo, runner de trajetórias, rollouts paralelos MJX e adaptadores UPD. [sem teste]

pinneapple_simulation/external_solvers/mujoco/__init__.py · 53 linhas · **sem teste direto**

`pinneapple_simulation.external_solvers.mujoco.adapter`

Converte dicionários de trajetória MuJoCo em `PhysicalSample`. [sem teste]

`pinneapple_simulation/external_solvers/mujoco/adapter.py` · 78 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`trajectory_to_upd`	função	`(traj, field_map=None, meta_extra=None)`	Converte um dicionário de trajetória MuJoCo num `PhysicalSample`.
`trajectories_to_upd`	função	`(trajs, field_map=None, meta_extra=None) -> list`	Converte uma lista de trajetórias MuJoCo em `PhysicalSamples`.

`pinneapple_simulation.external_solvers.mujoco.loader`

Configuração MuJoCo e funções para carregar o `MjModel` (arquivo ou XML) e alocar o `MjData`. [sem teste]

`pinneapple_simulation/external_solvers/mujoco/loader.py` · 92 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MuJoCoConfig`	classe	`(model_path, model_xml, assets, dt, n_steps, n_substeps, gravity, extra_opts)`	Configuração de uma simulação MuJoCo: modelo (arquivo ou XML), passo, número de passos, gravidade e opções.
`load_model`	função	`(config)`	Carrega o `MjModel` a partir da configuração.
`make_data`	função	`(model)`	Aloca um `MjData` novo para o modelo.

`pinneapple_simulation.external_solvers.mujoco.mjx_backend`

Backend MJX (JAX/GPU) que roda muitos ambientes MuJoCo em paralelo com `jax.vmap` para gerar datasets. Requer `mujoco` e `jax`. [sem teste]

`pinneapple_simulation/external_solvers/mujoco/mjx_backend.py` · 118 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MJXParallelRunner`	classe	`(config, n_envs=256, device=None) -> None`	Roda N ambientes MuJoCo independentes em paralelo com MJX e `jax.vmap`, com saída em arrays ou UPD.

`pinneapple_simulation.external_solvers.mujoco.runner`

Roda um modelo MuJoCo com controlador opcional e coleta a trajetória (qpos, qvel etc.). [sem teste]

`pinneapple_simulation/external_solvers/mujoco/runner.py` · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MuJoCoRunner`	classe	`(config, controller=None) -> None`	Roda um modelo MuJoCo com controlador opcional e coleta trajetórias.

`pinneapple_simulation.external_solvers.newton_physics`

Pacote do backend de corpos rígidos NVIDIA Newton (XPBD, semi-implícito, Featherstone, MuJoCo-Warp) com checagens analíticas.

`pinneapple_simulation/external_solvers/newton_physics/__init__.py` · 28 linhas · testes: 8

`pinneapple_simulation.external_solvers.newton_physics.runner`

Integra uma `RigidScene` com um solver do Newton e grava a trajetória; velocidades iniciais com solvers de coordenadas de junta levantam `NotImplementedError`. [sem teste direto]

`pinneapple_simulation/external_solvers/newton_physics/runner.py` · 182 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`RigidTrajectory`	classe	`(names, t, pos, quat, vel, omega, mass, inertia, com, gravity, solver, dt, wall_time_s, m…)`	Estados gravados de uma simulação de corpos rígidos (posições, quatérnios, velocidades, massas, inércias) com busca de corpo por nome.
`simulate`	função	`(scene, duration, dt=0.001, solver='xpbd', iterations=8, record_every=10, device='cpu', l…)`	Integra a cena por um tempo com o solver escolhido e grava a trajetória a cada `record_every` passos.

`pinneapple_simulation.external_solvers.newton_physics.scene`

Descrição de cena de corpos rígidos (corpos, formas, juntas, chão) em Python puro, que só importa Newton/Warp ao chamar `build`.

`pinneapple_simulation/external_solvers/newton_physics/scene.py` · 280 linhas · testes: 8

nome	tipo	assinatura	o que faz
`Shape`	classe	`(kind, size, vertices, faces, boxes)`	Uma forma de colisão de um corpo no referencial do corpo (esfera, caixa, cápsula, cilindro, malha ou composta).
`Body`	classe	`(name, shape, pos, quat, vel, omega, density, mass, mu, restitution)`	Um corpo rígido da cena: forma, pose, velocidades iniciais, densidade ou massa, atrito e restituição.
`Joint`	classe	`(kind, parent, child, anchor, axis, damping)`	Uma junta entre dois corpos (revoluta ou fixa) com âncora, eixo e amortecimento.
`RigidScene`	classe	`(gravity, bodies, joints, ground_mu, ground_restitution)`	Cena de corpos, juntas e chão opcional, montada num modelo Newton só quando pedida.

pinneapple_simulation.external_solvers.newton_physics.validation

Checa um solver do Newton contra resultados fechados: queda livre, atrito, pêndulo, quique, giro livre e contato em repouso. [sem teste direto]

pinneapple_simulation/external_solvers/newton_physics/validation.py · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
free_fall	função	(solver='xpbd', dt=0.001, **kw) -> dict	Checa queda livre de uma esfera contra z(t) = z0 - g t²/2.
sliding_friction	função	(solver='xpbd', dt=0.001, mu=0.4, v0=2.0, **kw) -> dict	Checa a distância de parada de uma caixa com atrito de Coulomb contra v0²/(2 mu g).
pendulum_period	função	(solver='xpbd', dt=0.001, length=1.0, amp_deg=5.0, **kw) -> dict	Checa o período de um pêndulo em pequena amplitude contra a fórmula fechada.
bounce	função	(solver='xpbd', dt=0.0001, e=0.5, **kw) -> dict	Checa a altura do primeiro quique contra e² vezes a altura de queda.
torque_free_spin	função	(solver='xpbd', dt=0.001, **kw) -> dict	Checa a conservação do momento angular de uma caixa girando sem gravidade.
resting_contact	função	(solver='xpbd', dt=0.001, **kw) -> dict	Checa que uma caixa em repouso no chão não afunda nem escorrega.
validate	função	(solver='xpbd', cases=None, log=print, **kw) -> list[dict]	Roda as checagens analíticas para um solver e devolve e registra uma tabela.

pinneapple_simulation.external_solvers.openfoam

Pacote OpenFOAM: montagem de casos, execução, amostragem, leitura binária, empacotamento UPD, arquivos `.splash` e campanhas. [sem teste direto do `__init__`]

pinneapple_simulation/external_solvers/openfoam/__init__.py · 8 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.openfoam.binary_reader

Primitivas para ler arquivos FoamFile em formato binário (cabeçalho, listas, pontos, faces, internalField); formato ASCII ou tipos de lista não suportados levantam `NotImplementedError`. [sem teste direto]

pinneapple_simulation/external_solvers/openfoam/binary_reader.py · 254 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
FoamFileHeader	classe	(klass, fmt, body_start)	Cabeçalho de um FoamFile: classe, formato e posição onde começa o corpo.
read_header	função	(data) -> FoamFileHeader	Localiza o fim do bloco `FoamFile {...}` e lê suas chaves.
read_binary_block	função	(data, pos, n_components, dtype=_SCALAR_DTYPE) -> Tuple[np.ndarray, int]	Lê um bloco binário `<count> (<bytes>)` a partir de uma posição.

nome	tipo	assinatura	o que faz
`read_label_list`	função	`(data, path_hint='') -> np.ndarray`	Lê uma lista binária de rótulos (inteiros) de um FoamFile.
`read_vector_field_points`	função	`(data, path_hint='') -> np.ndarray`	Lê uma lista binária de pontos/vetores de um FoamFile.
`read_face_compact_list`	função	`(data, path_hint='') -> Tuple[np.ndarray, np.ndarray]`	Lê uma `faceCompactList` (offsets e índices de pontos).
`read_internal_field`	função	`(data, path_hint='') -> Tuple[np.ndarray, bool, Optional[int]]`	Lê o `internalField` de um campo volumétrico, uniforme ou não.

`pinneapple_simulation.external_solvers.openfoam.campaign`

Campanhas de parâmetros de rodadas OpenFOAM divididas entre várias máquinas: copia o caso base, aplica edições, roda com MPI, amostra linhas e grava um JSON por rodada, com trava por arquivo e publicação opcional por git.

`pinneapple_simulation/external_solvers/openfoam/campaign.py` · 686 linhas · testes: 22

nome	tipo	assinatura	o que faz
`CampaignRun`	classe	`(id, params, mesh, worker, group)`	Uma rodada da campanha: id, parâmetros, malha, worker e grupo.
`Campaign`	classe	`(base, runs, name, solver, procs, end_time, pre, derived, edits, area_patch, setup, sampl…)`	Plano de campanha (caso base, rodadas, solver, edições, amostragem, resultados) com leitura/escrita JSON, preparação e execução por worker.
`GitPublisher`	classe	`(name=None, email=None, remote='origin', tries=6)`	Faz commit e push de um arquivo de resultado no branch atual, com rebase sobre o que outros workers enviaram.
`evaluate`	função	`(expr, names) -> float | list`	Avalia uma expressão aritmética restrita sobre nomes e funções de `math`.
`format_template`	função	`(template, values) -> str`	Formata um template restrito a `{nome}` e `{nome:spec}` numéricos, sem acesso a atributos.
`resolve`	função	`(params, derived=None) -> dict`	Devolve os parâmetros mais os valores derivados, na ordem dada.
`get_entry`	função	`(text, key) -> str`	Lê o valor de uma chave `a/b/c` num texto de dicionário OpenFOAM.
`set_entry`	função	`(text, key, value) -> str`	Define uma chave `a/b/c` num texto de dicionário OpenFOAM, criando níveis faltantes.
`edit_file`	função	`(path, key, value, root=None) -> None`	Define uma chave num arquivo de dicionário OpenFOAM, exigindo que o arquivo esteja dentro de `root` se dado.
`reset_nonuniform`	função	`(case, n_cells) -> list[str]`	Após troca de malha, reinicia campos de `0/` com número errado de valores para o valor mais comum.
`n_cells`	função	`(case) -> int`	Devolve o número de células da malha de um caso.
`area_patch`	função	`(case, patch, area, centre, normal, from_patches=None, rest='wall', plane_tol=0.03, bashr…)`	Reconstrói um patch de contorno com as faces de um plano mais próximas do centro até atingir a área pedida.

nome	tipo	assinatura	o que faz
`write_decompose`	função	`(case, n, method='auto', bashrc=None) -> str`	Escreve o `decomposeParDict` para n processos (scotch se houver, senão `simple`).
`parse_solver_log`	função	`(text, every=200) -> dict`	Lê do log a saúde de uma rodada transiente: último tempo, Courant, limites de fração de fase, término e primeiro erro fatal.
`worker_commands`	função	`(plan, n, procs=4, publish_git=True) -> list[str]`	Gera uma linha de shell por máquina para rodar cada fatia da campanha.
`main`	função	`(argv=None) -> None`	Linha de comando do runner de campanhas (`--worker`, `--shard`, `--procs`, `--publish-git`, `--commands`).

`pinneapple_simulation.external_solvers.openfoam.case_builder`

Copia um caso-modelo OpenFOAM e escreve `0/T` e `transportProperties` de um cenário (problema térmico com obstáculo). [sem teste]

`pinneapple_simulation/external_solvers/openfoam/case_builder.py` · 105 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OpenFOAMCaseTemplate`	classe	`(template_dir, obstacle_stl)`	Caso-modelo OpenFOAM (pasta e STL do obstáculo) com validação dos arquivos exigidos.
`stage_case_for_scenario`	função	`(*, tpl, out_case_dir, scenario) -> None`	Copia o caso-modelo para a pasta de saída e escreve `0/T` e `transportProperties` do cenário.

`pinneapple_simulation.external_solvers.openfoam.export_bundle`

Monta uma pasta-pacote copiando geometria, specs, pontos derivados e sensores amostrados; os nomes de arquivo estão fixos para um projeto de canal com cilindro. [sem teste]

`pinneapple_simulation/external_solvers/openfoam/export_bundle.py` · 38 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`export_bundle`	função	`(*, project_dir, bundle_dir) -> None`	Copia geometria, specs, pontos derivados e sensores de um projeto para uma pasta-pacote.

`pinneapple_simulation.external_solvers.openfoam.field_reader`

Lê o internalField de campos OpenFOAM (por padrão `p` e `U`) e empacota como `PhysicalSample`. [sem teste direto]

`pinneapple_simulation/external_solvers/openfoam/field_reader.py` · 173 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`openfoam_case_to_upd`	função	`(case_dir, *, time=None, fields=('p', 'U'))`	Lê o internalField dos campos pedidos num tempo e empacota como `PhysicalSample`.

pinneapple_simulation.external_solvers.openfoam.mesh_reader

Reconstrói centros e tamanhos de célula a partir de um polyMesh binário, no contrato de malha dos dataloaders de `pinneapple_data`. [sem teste direto]

`pinneapple_simulation/external_solvers/openfoam/mesh_reader.py` · 125 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MeshGeometry`	classe	`(nodes, cell_size, cell_delta, bounds_min, bounds_max, n_points, n_faces, n_internal_face…)`	Geometria mínima de malha (centros, tamanhos de célula, limites, contagens) no contrato dos dataloaders.
`load_mesh`	função	`(source, prefix='constant/polyMesh/') -> MeshGeometry`	Constrói centros e tamanhos de célula a partir de um polyMesh binário (pasta ou arquivo).

pinneapple_simulation.external_solvers.openfoam.runner

Executa um caso OpenFOAM via bash (blockMesh, snappyHexMesh opcional, solver serial ou MPI, comandos extras). [sem teste]

`pinneapple_simulation/external_solvers/openfoam/runner.py` · 43 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OpenFOAMRunConfig`	classe	`(foam_bashrc, solver, n_procs, use_snappy, extra_cmds)`	Como rodar um caso OpenFOAM: bashrc, solver, número de processos, snappyHexMesh e comandos extras.
`run_openfoam_case`	função	`(case_dir, cfg) -> None`	Roda malha, solver (serial ou MPI) e comandos extras num caso OpenFOAM.

pinneapple_simulation.external_solvers.openfoam.sampling

Escreve um `sampleDict` de nuvem de pontos, roda `postProcess -func sample` e lê os `.raw` resultantes em DataFrame. [sem teste]

`pinneapple_simulation/external_solvers/openfoam/sampling.py` · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`write_sample_dict_cloud`	função	`(case_dir, *, set_name, points, fields) -> Path`	Escreve `system/sampleDict` para amostragem em nuvem nos pontos dados.
`run_sampling`	função	`(case_dir, foam_bashrc, *, time=None) -> None`	Roda `postProcess -func sample` (último tempo por padrão).
`read_sampled_scalar_field`	função	`(case_dir, *, set_name, field) -> pd.DataFrame`	Lê um campo escalar amostrado (`x y z valor`) em DataFrame.

pinneapple_simulation.external_solvers.openfoam.splash_packer

Empacota um caso OpenFOAM terminado num arquivo `.splash` (zip + `splash-manifest.json` com tamanho e sha256 por arquivo), lê o manifesto e verifica a integridade.

`pinneapple_simulation/external_solvers/openfoam/splash_packer.py` · 138 linhas · testes: 9

nome	tipo	assinatura	o que faz
`pack_splash_archive`	função	`(case_dir, out_path, *, case_name=None, summary=None, generated_by=…, splash_version='1.0…)`	Compacta um caso OpenFOAM terminado num `.splash` com manifesto de tamanhos e hashes.
`read_splash_manifest`	função	`(splash_path) -> Dict[str, Any]`	Lê só o `splash-manifest.json` de um `.splash`.
`verify_splash_archive`	função	`(splash_path) -> Iterable[str]`	Lista os arquivos cujo tamanho ou sha256 não bate com o manifesto (vazio se íntegro).

pinneapple_simulation.external_solvers.openradioss

Ponte para o OpenRadioss (FEA explícito): edição de deck para DoE, execução via Docker e leitores de resultados ANIM->VTK.

`pinneapple_simulation/external_solvers/openradioss/__init__.py` · 18 linhas · testes: 2

pinneapple_simulation.external_solvers.openradioss.deck

Edita decks OpenRadioss: lê e troca espessuras de `/PROP/SHELL`, escreve o engine com saída ANIM e prepara a pasta de um caso. [sem teste]

`pinneapple_simulation/external_solvers/openradioss/deck.py` · 124 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`read_shell_thickness`	função	`(deck_text, prop_id) -> float`	Lê a espessura `Thick` de um `/PROP/SHELL/<id>`.
`set_shell_thickness`	função	`(deck_text, thickness_by_prop) -> str`	Devolve uma cópia do deck com as espessuras trocadas por id de propriedade.
`write_engine_anim`	função	`(path, run_name, t_end, anim_dt, *, extra_anim=('/ANIM/VECT/DISP',), header='', tail='') …)`	Escreve um arquivo engine que pede saída ANIM em vez de H3D.
`stage_case`	função	`(template_starter, engine_file, out_dir, thickness_by_prop, *, extra_files=None) -> Path`	Cria a pasta de um caso com o starter de espessura alterada e o engine original.

pinneapple_simulation.external_solvers.openradioss.log_reader

Lê do log do engine os erros de balanço de energia (`ERR=...%`).

`pinneapple_simulation/external_solvers/openradioss/log_reader.py` · 38 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`read_energy_errors`	função	`(case_dir_or_log) -> np.ndarray`	Devolve cada `ERR=...%` impresso pelo engine, em ordem.
`read_energy_error`	função	`(case_dir_or_log) -> float`	Devolve o maior |ERR| de balanço de energia do log, ou `nan`.

pinneapple_simulation.external_solvers.openradioss.runner

Roda starter, engine e conversão ANIM->VTK do OpenRadioss via Docker, um caso ou vários em paralelo limitado. [sem teste]

pinneapple_simulation/external_solvers/openradioss/runner.py · 111 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
OpenRadiossDockerConfig	classe	(openradioss_dir, image, arch, threads, docker_bin, extra_path)	Configuração da execução via Docker: pasta do OpenRadioss, imagem, arquitetura, threads e binário do docker.
run_openradioss_case	função	(case_dir, run_name, cfg, *, starter=None, engine=None) -> Path	Roda um caso preparado (starter, engine e conversão) e levanta erro se o solver falhar.
run_openradioss_cases	função	(case_dirs, run_name, cfg, *, max_parallel=4, on_done=None) -> List[Path]	Roda vários casos com paralelismo limitado, um solver de uma thread por caso.
docker_available	função	(cfg) -> bool	Diz se o Docker configurado responde.

pinneapple_simulation.external_solvers.openradioss.vtk_reader

Lê resultados OpenRadioss convertidos por anim_to_vtk (VTK legado ASCII) e séries de deslocamento. [sem teste]

pinneapple_simulation/external_solvers/openradioss/vtk_reader.py · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
RadiossMesh	classe	(points, node_ids, cells, cell_types, part_ids, time)	Malha lida de um VTK do OpenRadioss: pontos, ids de nós, células, tipos, partes e tempo.
read_vtk_mesh	função	(path) -> RadiossMesh	Lê um arquivo VTK legado ASCII convertido por anim_to_vtk.
read_case_displacements	função	(frames_dir) -> Tuple[np.ndarray, np.ndarray]	Devolve tempos e deslocamentos (T, N, 3) a partir de arquivos disp_XXX.txt/time_XXX.txt.

pinneapple_simulation.external_solvers.turbodesigner

Pacote da ponte TurboDesigner (análise de linha média de compressor axial). [sem teste]

pinneapple_simulation/external_solvers/turbodesigner/__init__.py · 43 linhas · **sem teste direto**

pinneapple_simulation.external_solvers.turbodesigner.bridge

Envolve o pacote turbodesigner para obter campos por estação de um compressor axial como fonte de dados para PINN; sem o pacote, usa um cálculo analítico simplificado (fallback_if_missing=True). [sem teste]

pinneapple_simulation/external_solvers/turbodesigner/bridge.py · 400 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TurboDesignerConfig`	classe	`(pressure_ratio, num_stages, mass_flow_rate, rpm, inlet_total_pressure, inlet_total_tempe…)`	Parâmetros de um compressor axial no TurboDesigner (razão de pressão, estágios, vazão, rotação, condições de entrada).
`TurboDesignerWorkflow`	classe	`(base_config) -> None`	Atalho para solves repetidos do TurboDesigner sobre uma configuração base, com `solve`, `solve_upd` e `sweep`.
`run_turbodesigner`	função	`(cfg, *, fallback_if_missing=True) -> Dict[str, np.ndarray]`	Roda a análise de linha média e devolve arrays por estação; sem o pacote, usa o cálculo analítico simplificado.
`turbodesigner_to_upd`	função	`(cfg, **run_kwargs)`	Roda o TurboDesigner e empacota o resultado como `PhysicalSample`.

pinneapple_simulation.flight

Flight dynamics: a 6-DOF rigid aircraft driven by vortex-lattice aerodynamics, thrust, gravity and landing-gear contact, with trim, stability derivatives, a pilot / autopilot, take-off and turbulence-encounter simulations, and 3-D scenes of the flights.

`pinneapple_simulation/flight/__init__.py` · 59 linhas · testes: 9

pinneapple_simulation.flight.aero

Quasi-steady aerodynamics of a whole aircraft for flight simulation, from the vortex lattice of :mod:`pinneapple_design.aero.vlm`.

`pinneapple_simulation/flight/aero.py` · 239 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ControlSurface`	classe	`(name, surface, eta_in, eta_out, cf_c, antisymmetric, tau)`	Panels of the lattice moved by one control: `surface` 0 wing / 1 tail, spanwise range |eta| in [eta_in, eta_out] of that surface's half-span, `sign` per side (+1 symmetric, -1 antisymmetric: right side gets +delta), chord ratio for the effectiveness. A positive deflection is trailing edge down …
`Fin`	classe	`(area, height, ac, rudder_cf_c, sweep_deg, stall_deg)`	Vertical tail as one lifting line: area [m^2], height [m], aerodynamic centre in airframe axes, rudder chord ratio, quarter-chord sweep [deg].
`AeroModel`	classe	`(af, cg, cd0, fin, controls, cma_fus, cnb_fus, cd_gear, cd_flap_per_rad, nc, ns_wing, ns_…)`	Built by :meth:`from_airframe`; :meth:`loads` gives forces and moments about the CG in body axes.
`flap_effectiveness`	função	`(cf_c, correction=0.85) -> float`	Thin-airfoil flap effectiveness d(alpha_0)/d(delta) for a plain flap of chord ratio cf/c: tau = 1 - (theta_f - sin theta_f) / pi, cos theta_f = 2 cf/c - 1, times an empirical `correction` for real viscous flow at moderate deflections (an assumption; 0.7-0.9 is the usual range).
`helmbold_lift_slope`	função	`(aspect_ratio, sweep_half_chord_deg=0.0, eta=0.95) -> float`	Lift-curve slope [/rad] of a finite wing (Helmbold / DATCOM form, Raymer eq. 12.6, subsonic, M ~ 0).

nome	tipo	assinatura	o que faz
`fuselage_munk_factor`	função	`(wing_qc_fraction) -> float`	Raymer's K_fus [per degree] against the wing root quarter-chord position as a fraction of the fuselage length (Aircraft Design fig. 16.14), interpolated between the two values the aero-optimizer models use (0.012 at 30 %, 0.025 at 45 %): an approximation of the chart, clipped to that range.
`fuselage_moments`	função	`(length, max_width, volume, wing_area, mac, span, depth, wing_qc_fraction)`	(Cm_alpha, Cn_beta) of the fuselage [/rad]: Raymer eq. 16.25 Cm_a = K_f W_f^2 L_f / (c S) (per degree x 57.3) and eq. 16.47 Cn_b = -1.3 (volume / (S b)) (D_f / W_f) (sign: body axes, a positive Cn_beta is stable, the fuselage is unstable so the value is negative).

`pinneapple_simulation.flight.aircraft`

The aircraft as a 6-DOF rigid body: mass properties, vortex-lattice aerodynamics, engines and landing gear.

`pinneapple_simulation/flight/aircraft.py` · 155 linhas · testes: 9

nome	tipo	assinatura	o que faz
`Controls`	classe	`(elevator, aileron, rudder, flap, throttle, steer, brake, gear_down)`	Positive elevator / rudder / aileron / flap = trailing edge down on the right side (elevator down = nose down, rudder: trailing edge left = nose left, aileron: right aileron down = roll left), radians; throttle and brake 0..1; steer = nose-wheel angle [rad], positive turns right.
`Aircraft`	classe	`(name, af, aero, mass, cg, inertia, engines, gear, cabin, tail_point, wingtip, clmax_take…)`	

`pinneapple_simulation.flight.catalog`

Four aircraft for flight simulation, built from the library's parametric geometry and weight / drag models.

`pinneapple_simulation/flight/catalog.py` · 383 linhas · testes: 9

nome	tipo	assinatura	o que faz
`ExplicitAirframe`	classe	`(wing_z, htail_spec, vtail_spec)`	An :class:`Airframe` whose wing height and tails are given explicitly (for a high wing on a T-tail aircraft, whose tail the parametric sizing rules would put on the fuselage).
`profile_cd0`	função	`(re, tc) -> float`	Turbulent flat-plate friction (0.074 Re^-0.2) x thickness form factor (1 + 2 t/c + 60 (t/c)^4), both sides, on the planform area (the glider-wing template's model, :mod:`pinneapple_design.intent.templates.glider_wing`).
`light_ga`	função	`() -> Aircraft`	
`regional_turboprop`	função	`() -> Aircraft`	
`a320`	função	`() -> Aircraft`	

nome	tipo	assinatura	o que faz
`motor_glider`	função	`() -> Aircraft`	
`build`	função	`(name) -> Aircraft`	
`names`	função	`() -> list[str]`	

`pinneapple_simulation.flight.control`

Pilot and autopilot models: pitch-attitude, altitude, speed, wings-level / heading hold, yaw damper, and the take-off schedule (ground roll on the centreline, rotation at V_R, climb-out at a target speed).

`pinneapple_simulation/flight/control.py` · 221 linhas · testes: 9

nome	tipo	assinatura	o que faz
`LoopDesign`	classe	`(omega_pitch, zeta_pitch, omega_roll, zeta_roll, omega_yaw, zeta_yaw, pitch_integral, q_f…)`	
`InnerLoops`	classe	`(ac, derivs, design=None)`	Attitude loops of one aircraft: :meth:`elevator`, :meth:`aileron`, :meth:`rudder` give surface increments.
`StickFixed`	classe	`(controls)`	Controls frozen at the trim values.
`Autopilot`	classe	`(ac, trim, derivs, design=None, altitude=None, heading=None, speed=None, omega_h=0.25, ta…)`	Altitude hold (through pitch attitude), wings level / heading hold (through bank), yaw damper (washed-out yaw rate), autothrottle on airspeed, about a trim state. `loops` scales the attitude bandwidths.
`TakeoffSchedule`	classe	`(V_R, theta_rotation, rotation_rate, V_climb, theta_max, theta_min, runway_heading, flap,…)`	
`TakeoffPilot`	classe	`(ac, sched, derivs, design=None)`	Full throttle and the take-off flap from brake release; on the ground the nose wheel and the rudder hold the runway centreline (y = 0 in NED for a north runway, `runway_heading` otherwise) and the ailerons keep the wings level; at airspeed V_R the pitch attitude is commanded up at …
`wrap`	função	`(a) -> float`	

`pinneapple_simulation.flight.encounter`

Turbulence and gust encounters in flight, and their load-factor statistics.

`pinneapple_simulation/flight/encounter.py` · 140 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`gust_sensitivity`	função	`(ac, V, rho, cl_alpha) -> float`	dn / dw_gust [1/(m/s)] of a fixed wing in quasi-steady flow.
`stall_angle`	função	`(ac, trim_alpha, trim_CL, cl_alpha) -> float`	Angle of attack [rad] at which the linear lift reaches the aircraft's clean CLmax (`ac.clmax_clean`, an assumption of the catalog): alpha_trim + (CLmax - CL_trim) / CL_alpha. Beyond it the linear lattice is outside its validity (no stall in the model).
`encounter_metrics`	função	`(ac, hist, t_skip=5.0, alpha_stall=None) -> dict[str, Any]`	Statistics after `t_skip`. With `alpha_stall` [rad] the run is also checked against the model's validity: the fraction of time above the stall angle and `departed` (bank beyond 90 deg or alpha 10 deg past stall), in which case the numbers describe a loss of control the quasi-steady linear …
`turbulence_encounter`	função	`(ac, V, altitude, spec=None, wind=None, duration=60.0, dt=0.01, controller='autopilot', f…)`	Fly `duration` seconds through turbulence. `spec` defaults to severe at `altitude` (von Karman); `field` "3d" (:class:`FrozenField3D`, resolved to wavelengths of ~4 MAC) or "line" (:class:`FrozenLine`, the same gust across the span); a ready `wind` field overrides both. Returns (history, …
`discrete_gust`	função	`(ac, V, altitude, U, H=None, duration=None, dt=0.005)`	Stick-fixed response to an up-gust of amplitude U [m/s] (full 1-cosine, half-length H, default 12.5 MAC) starting 2 s ahead of the aircraft. Returns (history, dnz_max).
`pratt_check`	função	`(ac, V, altitude, U_de) -> dict[str, float]`	Simulated peak dn for a 12.5-MAC 1-cosine gust against the Pratt-Walker formula at the same equivalent gust and speed. The model is quasi-steady, so its peak sits between Pratt (with K_g) and the unalleviated value.

`pinneapple_simulation.flight.figures`

Static figures of the flight-dynamics results (matplotlib PNG): turbulence spectra against their targets, take-off trajectories, crosswind tracking, turbulence-encounter load factors and the gust-sensitivity check.

`pinneapple_simulation/flight/figures.py` · 212 linhas · testes: 9

nome	tipo	assinatura	o que faz
`psd_figure`	função	`(curves, path) -> str`	One panel per entry of `curves`: dict(title, W (rad/m or rad/s), measured, target, xlabel, markers=[(W, label)]).
`takeoff_figure`	função	`(runs, metrics, path) -> str`	Height of the lowest wheel and airspeed against runway distance, pitch attitude against time.
`lateral_figure`	função	`(runs, path, title='Crosswind take-off') -> str`	Lateral deviation from the centreline, bank angle and rudder against time for one or more runs.
`encounter_traces_figure`	função	`(runs, path, window=20.0, t0=10.0, notes=None) -> str`	Small multiples: vertical load-factor increment at the CG and roll angle, one row per aircraft; `notes` adds a remark to an aircraft's row title (e.g. the turbulence level shown).
`encounter_bars_figure`	função	`(table, path) -> str`	RMS vertical load-factor increment at the CG and at the cabin stations, per aircraft (grouped bars). Rows with `status == "departed"` (the run left the model's validity) get a note instead of bars.

nome	tipo	assinatura	o que faz
`gust_sensitivity_figure`	função	`(table, pratt, path) -> str`	Left: measured RMS Δn_z against the quasi-steady prediction A sigma_w (A = rho V CL_a / 2 W/S), y = x for reference. Right: discrete 1-cosine gust peak from the simulation against the Pratt formula.
`table_markdown`	função	`(rows, cols, digits=None) -> str`	Markdown table from dict rows; `cols` = (key, header) pairs; floats rounded per `digits` (default 2).

`pinneapple_simulation.flight.gear`

Landing-gear contact with a flat runway: spring-damper legs with rolling, braking and tyre side forces.

`pinneapple_simulation/flight/gear.py` · 96 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GearLeg`	classe	`(name, contact, stiffness, damping, mu_roll, mu_brake, mu_side, slip_ref, steerable, brak…)`	
`size_gear`	função	`(legs, mass, cg, static_compression=0.25, zeta=0.6, g=9.80665) -> list[GearLeg]`	Spring and damper of every leg so that the static compression under its share of the weight is `static_compression`; the shares come from moment balance about the CG in x (nose / mains) and split equally between legs at the same x station (left / right).
`gear_forces`	função	`(legs, x, C, cg, steer=0.0, brake=0.0, ground_z=0.0)`	(F_b, M_b, normal loads list) of all legs for state x (13-vector) and body->NED matrix C.

`pinneapple_simulation.flight.mass`

Mass, centre of gravity and inertia tensor from a component breakdown.

`pinneapple_simulation/flight/mass.py` · 83 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Component`	classe	`(name, mass, center, inertia)`	
`point`	função	`(name, mass, center) -> Component`	
`rod`	função	`(name, mass, center, length, axis=0) -> Component`	Slender rod of `length` along airframe axis `axis`.
`plate`	função	`(name, mass, center, span, chord, taper=1.0) -> Component`	Thin plate in the x-y plane: `span` along y, `chord` along x (a wing, a tailplane, wing fuel). With `taper` < 1 the mass per unit span follows the local chord (linear taper from the root to the tips): Ixx = m (b/2)^2 (1 + 3 taper) / (6 (1 + taper)), m b^2 / 12 for a rectangular plate.
`fin_plate`	função	`(name, mass, center, height, chord) -> Component`	Thin plate in the x-z plane (a vertical tail).

nome	tipo	assinatura	o que faz
`cylinder`	função	`(name, mass, center, length, radius, shell=False) -> Component`	Cylinder along x (a fuselage, a nacelle); `shell=True` for a thin-walled one.
`mass_properties`	função	`(components) -> tuple[float, np.ndarray, np.ndarray]`	(mass, cg in airframe axes, inertia about the CG in body axes).
`balance`	função	`(components, x_target, movable) -> list[Component]`	Shift the component named `movable` along x so that the CG is at `x_target` (airframe axes).

`pinneapple_simulation.flight.propulsion`

Engines for flight simulation: thrust against speed and altitude, and where it acts.

`pinneapple_simulation/flight/propulsion.py` · 74 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Turbofan`	classe	`(static_thrust, position, cant_deg, name)`	
`Propeller`	classe	`(power, diameter, efficiency, figure_of_merit, kind, position, cant_deg, name)`	
`thrust_direction_body`	função	`(cant_deg) -> np.ndarray`	

`pinneapple_simulation.flight.rigid_body`

6-DOF rigid-body equations in body axes with a unit quaternion for the attitude.

`pinneapple_simulation/flight/rigid_body.py` · 116 linhas · testes: 9

nome	tipo	assinatura	o que faz
`quat_multiply`	função	`(a, b) -> np.ndarray`	
`quat_to_dcm`	função	`(q) -> np.ndarray`	Rotation matrix body -> NED (columns: the body axes in NED).
`euler_to_quat`	função	`(phi, theta, psi) -> np.ndarray`	Roll, pitch, yaw [rad] (3-2-1: yaw, then pitch, then roll) -> quaternion body -> NED.
`quat_to_euler`	função	`(q) -> tuple[float, float, float]`	(phi, theta, psi) [rad]. At the gimbal lock (|theta| = 90 deg) only phi -+ psi is defined: the split puts it all in psi (phi = 0), so the result stays finite and round-trips the attitude.
`state_vector`	função	`(pos=(0, 0, 0), vel_body=(0, 0, 0), euler=(0, 0, 0), omega=(0, 0, 0)) -> np.ndarray`	

nome	tipo	assinatura	o que faz
`rigid_body_derivative`	função	`(x, F_b, M_b, mass, inertia, inertia_inv=None, k_norm=1.0) -> np.ndarray`	Time derivative of the 13-state for total body force F_b (gravity included) and moment M_b about the CG.
`gravity_body`	função	`(q, mass, g=G0) -> np.ndarray`	
`rk4_step`	função	`(f, t, x, dt) -> np.ndarray`	One classical Runge-Kutta step; the quaternion is renormalised afterwards.
`kinetic_energy`	função	`(x, mass, inertia) -> float`	
`angular_momentum_ned`	função	`(x, inertia) -> np.ndarray`	Angular momentum about the CG in NED (constant for a torque-free body).

pinneapple_simulation.flight.simulate

Time integration of an :class:`~.aircraft.Aircraft` with a controller and a wind field, and the recorded history.

`pinneapple_simulation/flight/simulate.py` · 165 linhas · testes: 9

nome	tipo	assinatura	o que faz
`FlightHistory`	classe	`(aircraft, t, x, euler, airspeed, alpha, beta, CL, nz, ny, f_spec, thrust, gear_loads, co…)`	
`simulate`	função	`(ac, x0, controller, t_end, dt=0.01, wind=None, record_every=1, record_gamma=False, stop=…)`	`hold_wind`: sample the wind at the aircraft's sample points at the start of each step and at their predicted end-of-step positions, and interpolate linearly in time over the RK stages (two field evaluations per step instead of four; second order in the step). False samples the field at every …
`energy_check`	função	`(ac, x0, t_end=5.0, dt=0.01) -> dict[str, float]`	Energy and momentum consistency of the integrator with the aerodynamics, engines and gear off (a body in vacuum under gravity, spinning): kinetic + potential energy and the angular momentum in NED are conserved.
`rms`	função	`(a) -> float`	

pinneapple_simulation.flight.takeoff

Take-off: ground roll, rotation at V_R, lift-off, climb-out over the 35 ft screen, and a closed-form check.

`pinneapple_simulation/flight/takeoff.py` · 184 linhas · testes: 9

nome	tipo	assinatura	o que faz
default_schedule	função	(ac, kind='transport') -> TakeoffSchedule	V_R = 1.05 x the take-off-flap stall speed (CLmax_TO of the aircraft), climb at 1.25 V_s (between V2 = 1.13 V_SR and V2 + 10 kt for a transport), rotation at 3 deg/s to 12 deg (transport) or 6 deg/s to 11 deg (light); an assumption of a typical technique, not a certified schedule.
takeoff_derivatives	função	(ac, V) -> dict[str, float]	Control derivatives for the pilot's gain design: about a trimmed climb-out at V with the take-off flap.
simulate_takeoff	função	(ac, schedule=None, wind=None, t_max=90.0, dt=0.01, h_stop=400 * FT, design=None, record_…)	
takeoff_metrics	função	(ac, hist) -> dict[str, Any]	
raymer_takeoff	função	(ac, V_LOF=None, mu=0.02, t_rotation=3.0, h_obstacle=SCREEN, theta_ground=0.0) -> dict[st…)	Raymer's closed-form take-off distance with this aircraft's thrust, lift and drag (see the module text). V_LOF defaults to 1.1 V_stall (take-off flap). Ground-roll lift and drag from the lattice at the ground attitude with the take-off flap and the gear down, ground effect on the induced drag …
raymer_variants	função	(ac, metrics, kind='light') -> dict[str, dict[str, float]]	The two closed-form references the take-off results quote (Raymer, Aircraft Design, sec. 17.8):

pinneapple_simulation.flight.trim

Trim, ground equilibrium and stability derivatives of an :class:~.aircraft.Aircraft.

pinneapple_simulation/flight/trim.py · 190 linhas · testes: 9

nome	tipo	assinatura	o que faz
TrimResult	classe	(V, altitude, gamma, alpha, theta, elevator, throttle, x, controls, residual, CL, feasibl…)	
trim	função	(ac, V, altitude, gamma=0.0, flap=0.0, gear_down=False, psi=0.0) -> TrimResult	Steady straight flight, wings level; the residual is the norm of (u_dot, w_dot, q_dot * c) [m/s^2].
ground_equilibrium	função	(ac, flap=0.0, throttle=0.0, psi=0.0, x_pos=0.0, y_pos=0.0) -> np.ndarray	State at rest on the gear: height and pitch attitude with zero vertical force and pitching moment.
stability_derivatives	função	(ac, tr, d_angle=math.radians(0.5), d_rate=0.01, d_ctl=math.radians(1.0)) -> dict[str, fl…)	Aerodynamic derivatives [/rad] about the trim state (engines included at the trim throttle: the thrust line contributes to Cm through its moment arm only through the trim, not the derivatives, which use the aero loads).

nome	tipo	assinatura	o que faz
`modes`	função	`(ac, tr, eps=1e-05) -> dict[str, object]`	Eigenvalues of the 12-state linearisation (position, velocity, attitude, rates; the quaternion's vector part as the attitude) about a trimmed state, with the controls fixed, by central differences of the full model.
`classify_modes`	função	`(eigs, tol=1e-06) -> dict[str, dict[str, float]]`	Name the complex pairs by frequency (short period: the fastest longitudinal-looking pair is left to the caller with the eigenvectors; here only frequency, damping and the real roots are returned sorted).

`pinneapple_simulation.flight.turbulence`

Atmospheric turbulence and gusts for flight simulation (MIL-F-8785C / MIL-HDBK-1797).

`pinneapple_simulation/flight/turbulence.py` · 476 linhas · testes: 19 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`TurbulenceSpec`	classe	`(sigma_u, sigma_v, sigma_w, L_u, L_v, L_w, model, altitude, severity)`	RMS intensities [m/s] and scale lengths [m] of the three turbulence components, and the spectral form.
`WindField`	classe	`()`	A wind field: `velocity(t, points)` -> (M, 3) NED wind [m/s] at (M, 3) NED points [m].
`SumWind`	classe	`(fields)`	
`ConstantWind`	classe	`(ned)`	Uniform steady wind `ned` [m/s] (the velocity of the air, e.g. (0, -10, 0) blows towards the west).
`ShearWind`	classe	`(w20, from_deg, z0, ground_z)`	Mean wind with the MIL-F-8785C logarithmic profile W(h) = W20 ln(h / z0) / ln(20 ft / z0); `from_deg` the direction the wind comes from (meteorological, degrees from north), `z0` the roughness length (0.15 ft = 0.046 m for category C flight phases, MIL-F-8785C 3.7.2.1). Below z0 the wind is ...
`FrozenLine`	classe	`(spec, length, dx, heading_deg, origin, seed, series)`	One frozen realisation of the three components along a straight line through `origin` with heading `heading_deg`, length `length` [m] at spacing `dx` [m], by spectral synthesis: each Fourier coefficient has the exact target amplitude sqrt(Phi dOmega) and a random phase, so the periodogram ...
`DrydenFilter`	classe	`(spec, V, dt, seed=0)`	MIL-HDBK-1797 Dryden forming filters, driven by white noise, for one point flying at true airspeed V:
`FrozenField3D`	classe	`(spec, n_shells, n_dirs, k_min, k_max, dx_min, heading_deg, seed)`	Isotropic, divergence-free von Karman turbulence in 3-D from random Fourier modes: u(x) = sum_n a_n sigma_n cos(k_n . x + phi_n), with |k_n| on log-spaced shells between `k_min` and `k_max`, directions uniform on the sphere (`n_dirs` per shell), sigma_n a random unit vector normal to k_n (so ...
`DiscreteGust`	classe	`(amplitude, length, start, shape, direction, heading_deg, origin)`	1-cosine gust met by the aircraft when its path coordinate s (along `heading_deg` from `origin`) passes `start`. `shape="ramp"`: MIL-F-8785C 3.7.4, v = V_m/2 (1 - cos(pi x / d_m)) for 0 <= x <= d_m and V_m beyond; `shape="full"`: v = V_m/2 (1 - cos(pi x / H)) for 0 <= x <= 2H and 0 ...

nome	tipo	assinatura	o que faz
`turbulence_spec`	função	`(altitude, severity='moderate', model='von_karman', *, w20=None, probability=None) -> Tur…)`	Intensities and scale lengths at `altitude` [m] (above ground level for the low-altitude model) for a severity `light` / `moderate` / `severe`. `w20` [m/s] overrides the 20 ft wind of the severity (low altitude), `probability` the exceedance probability (medium / high altitude, a key of …
`psd`	função	`(spec, Omega, component='w') -> np.ndarray`	One-sided spatial PSD [(m/s)^2 / (rad/m)] of component u, v or w at spatial frequency Omega [rad/m].
`psd_temporal`	função	`(spec, omega, V, component='w') -> np.ndarray`	One-sided temporal PSD [(m/s)^2 / (rad/s)] seen at true airspeed V (frozen field: Omega = omega / V).
`welch_psd`	função	`(x, d, nseg=16)`	One-sided PSD of a sampled signal against angular frequency (rad per unit of the sample spacing `d`): Welch's method with a Hann window, 50 % overlap. Returns (Omega, Phi) with integral of Phi dOmega = variance.
`path_axes`	função	`(heading_deg) -> np.ndarray`	Rows: the u (along the heading, horizontal), v (to the right) and w (down) unit vectors in NED.
`von_karman_energy`	função	`(k, L, sigma=1.0)`	Isotropic von Karman energy spectrum E(k) = C sigma^2 L (a L k)^4 / (1 + (a L k)^2)^(17/6) [m^3/s^2], a = 1.339, with C fixed numerically so that the kinetic energy (integral of E) is 3 sigma^2 / 2. Its one-dimensional longitudinal and transverse spectra are then the MIL von Karman Phi_u and Phi_w …
`isotropic_line_spectra`	função	`(k1, L, sigma=1.0)`	One-sided longitudinal (F11) and transverse (F22) spectra [(m/s)^2 / (rad/m)] of the isotropic field with the energy spectrum :func:`von_karman_energy`, by quadrature of E(k) (Batchelor 1953): F11(k1) = int_k1^inf E(k)/k (1 - k1^2/k^2) dk, F22(k1) = 1/2 int_k1^inf E(k)/k (1 + k1^2/k^2) dk.
`pratt_gust_load`	função	`(wing_loading, cl_alpha, mac, U_de, V_e, rho0=1.225, rho=1.225, g=9.80665)`	Pratt-Walker gust load factor increment (FAR 25.341 before amendment 25-86, CS-23.341):

`pinneapple_simulation.flight.verification`

Verification of the flight-dynamics building blocks, as numbers a guide or a CI job can quote.

`pinneapple_simulation/flight/verification.py` · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`band_ratios`	função	`(W, P, target, lo, hi, nbins=8)`	
`turbulence_checks`	função	`(seed=3) -> dict[str, Any]`	
`trim_checks`	função	`(names=None, hold=10.0) -> list[dict[str, Any]]`	
`conservation_check`	função	`() -> dict[str, Any]`	
`lattice_check`	função	`() -> list[dict[str, Any]]`	

nome	tipo	assinatura	o que faz
`run`	função	`() -> dict[str, Any]`	

pinneapple_simulation.flight.visuals

3-D scenes of a flight: the aircraft flying its recorded trajectory over a runway, the gust field around it as arrows, the wing and tail loading (vortex-lattice dCp) and a head-up display, as a :class:`pinneapple_tools.visualization.studio.Scene` with an …

`pinneapple_simulation/flight/visuals.py` · 236 linhas · testes: 9

nome	tipo	assinatura	o que faz
`body_vertices`	função	`(ac, V_airframe) -> np.ndarray`	Airframe-axes points (x aft, y right, z up) -> body frame x forward, y left, z up about the CG.
`aircraft_surfaces`	função	`(ac, group='aircraft', parts=None)`	The visual geometry of the aircraft (its `parts_fn`: airframe parts or geometry-library bodies) as studio surfaces in the body frame.
`lattice_surface`	função	`(ac, group='aircraft', lift=0.6)`	The vortex-lattice panels as a sheet just above the upper skin (offset `lift` x local thickness), one quad (two triangles, four own vertices) per panel, in the body frame.
`runway_surfaces`	função	`(length=3000.0, width=45.0, x0=-300.0, grass=1500.0)`	Runway along +x (north) from x0, centreline dashes, threshold bars, grass around (scene axes z up).
`frame_indices`	função	`(hist, n_frames) -> np.ndarray`	
`flight_scene`	função	`(ac, hist, *, n_frames=240, wind=None, runway=True, runway_length=None, gust_grid=7, gust…)`	Animated scene of `hist`. `wind`: the field the aircraft flew through, sampled on a heading-aligned horizontal grid around the aircraft (`gust_grid` x `gust_grid` points over 2 spans, at the CG height and one semi-span above and below) and drawn as arrows coloured by the vertical gust; …
`gust_wisps`	função	`(ac, hist, wind, n_lat=5, n_vert=3, spacing=None, exaggeration=0.6, ahead=0.15)`	The frozen gust field as static wisps along the flown track: `n_lat` x `n_vert` lines offset from the CG track by +-1 span on the local horizontal normal and +-0.5 span vertically (extended by `ahead` x the flown distance along the last heading), sampled every `spacing` (default span / 4) …
`scene_summary`	função	`(sc) -> dict[str, Any]`	

pinneapple_simulation.geophysics

Earth-system building blocks, each verified against an exact solution or a standard test:

`pinneapple_simulation/geophysics/__init__.py` · 40 linhas · testes: 11

pinneapple_simulation.geophysics.energy_balance

Two-layer energy-balance model of global-mean surface warming (Held et al. 2010, J. Climate 23:2418; Geoffroy et al. 2013a, J. Climate 26:1841; efficacy of deep-ocean heat uptake: Geoffroy et al. 2013b, J. Climate 26:1859):

`pinneapple_simulation/geophysics/energy_balance.py` · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TwoLayerEBM`	classe	`(lam, gamma, C, C0, eps)`	
`fit_two_layer`	função	`(T_obs, forcing, dt=1.0, fit_eps=False, guess=None, sigma=None) -> dict[str, object]`	Least-squares fit of (lambda, gamma, C, C0[, eps]) to a warming series driven by `forcing`.

pinneapple_simulation.geophysics.richards

Unsaturated flow in soil: the Richards equation in 1-D with the classical water-retention / conductivity models.

`pinneapple_simulation/geophysics/richards.py` · 316 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VanGenuchten`	classe	`(theta_r, theta_s, alpha, n, Ks, l)`	
`BrooksCorey`	classe	`(theta_r, theta_s, h_b, lam, Ks)`	
`ClappHornberger`	classe	`(theta_s, psi_s, b, Ks, theta_r)`	
`Gardner`	classe	`(theta_r, theta_s, alpha, Ks)`	
`Boundary`	classe	`(kind, value)`	`kind`: "head" (value = h), "flux" (value = Darcy flux, positive upward, i.e. infiltration < 0) or "free_drainage" (bottom only). `value` may be a function of time.
`RichardsResult`	classe	`(z, t, h, theta, flux_top, flux_bottom, mass_balance_ratio, n_steps, n_picard)`	
`solve_richards`	função	`(soil, depth, n_cells, h0, t_end, top, bottom, dt0=1.0, dt_min=1e-06, dt_max=None, t_out=…)`	Column from z = -depth (bottom) to z = 0 (surface). `h0`: initial head (scalar or per cell, bottom first). `sink(z, t)`: volumetric sink [1/T] (root water uptake). Returns profiles at `t_out` (default: 11 times).
`gardner_steady_infiltration`	função	`(soil, z_above_table, q0) -> np.ndarray`	Exact steady head profile above a water table (h = 0 at z = 0) under a constant infiltration rate q0 > 0 (downward) for the Gardner model: K(z) = q0 + (Ks - q0) exp(-alpha z), h = ln(K/Ks)/alpha.

pinneapple_simulation.geophysics.sphere_shallow_water

Shallow-water equations on the rotating sphere: spectral-transform solver and the Williamson et al. test suite.

pinneapple_simulation/geophysics/sphere_shallow_water.py · 298 linhas · testes: 11

nome	tipo	assinatura	o que faz
SphericalHarmonics	classe	(trunc, nlat=None, nlon=None, radius=EARTH['a'])	Scalar and vector spectral transforms for triangular truncation trunc on a Gaussian grid.
SphereShallowWater	classe	(sh, zeta, delta, phi, f, phi_s, omega, g, hyperdiffusion, t, history)	State: spectral vorticity zeta, divergence delta, fluid geopotential phi (= g h).
legendre_tables	função	(mu, M, N) -> tuple[np.ndarray, np.ndarray]	Normalised associated Legendre functions P[j, m, n] (orthonormal on [-1, 1], n <= N + 1) and H[j, m, n] = (1 - mu^2) dP_n^m/dmu (n <= N) at the points mu; zero where n < m.
stable_dt	função	(trunc, gh_max, u_max, radius=EARTH['a'], safety=0.7) -> float	Explicit RK4 time step: the fastest resolved wave is (sqrt(gh) + |u|) at wavenumber ~ N/a; RK4 is stable on the imaginary axis up to 2.8.
williamson_case2	função	(sh, alpha=0.0, gh0=29400.0, **kw) -> SphereShallowWater	Steady zonal geostrophic flow, u0 = 2 pi a / 12 days, rotated by alpha (rad). The Coriolis parameter is rotated with it, so the initial state is an exact steady solution.
williamson_case5	função	(sh, **kw) -> SphereShallowWater	Zonal flow (u0 = 20 m/s, h0 = 5960 m) over a conical mountain of 2000 m centred at (lon, lat) = (3 pi/2, pi/6), radius pi/9. h is the fluid depth (free surface minus mountain).
williamson_case6	função	(sh, R=4, omega_w=7.848e-06, K=7.848e-06, h0=8000.0, **kw) -> SphereShallowWater	Rossby-Haurwitz wave of wavenumber R (Williamson et al. 1992 eqs. 143-148).
rossby_haurwitz_speed	função	(R=4, omega_w=7.848e-06, omega=EARTH['omega']) -> float	Eastward angular phase speed of the Rossby-Haurwitz wave in the non-divergent barotropic model [rad/s] (Haurwitz 1940); the shallow-water wave moves slightly slower.
height_errors	função	(sh, h, h_ref) -> dict[str, float]	Normalised l1, l2 and l_inf errors of Williamson et al. (1992), eqs. 82-84.

pinneapple_simulation.meshless

Meshless and mesh-based solvers of the same 2-D problems, and the study that compares them.

pinneapple_simulation/meshless/__init__.py · 23 linhas · testes: 14

pinneapple_simulation.meshless.contracts

A meshless study as the apps' StudyResult contract (pinneapple_data.contracts): one table row per method and spacing, the convergence and the winners as metrics, and checks that carry the value they were decided on.

pinneapple_simulation/meshless/contracts.py · 50 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
meshless_checks	função	(res) -> list	Observed order against the expected one where the solution is smooth enough to expect it, and stability.
study_result	função	(res, *, study_id, title, limits=(), app='meshless')	

pinneapple_simulation.meshless.domains

2-D domains and node sets for the meshless study.

`pinneapple_simulation/meshless/domains.py` · 218 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NodeSet`	classe	`(X, boundary, h)`	
`Domain2D`	classe	`(name, sdf, pieces, lo, hi, description, cloud, eval_points)`	
`polygon_domain`	função	`(vertices, name='polygon', holes=()) -> Domain2D`	A simple polygon (vertices in order), optionally with circular holes `((cx, cy), r)`.
`domain`	função	`(name) -> Domain2D`	A preset: `square`, `lshape`, `disk`, `annulus`, `plate_hole`.
`point_cloud_domain`	função	`(X, boundary, *, eval_fraction=0.1, seed=0, name='point cloud') -> Domain2D`	An uploaded point cloud: nodes `X` (N, 2) with a boundary flag. `eval_fraction` of the interior points are held out as evaluation points (not used by any method).

pinneapple_simulation.meshless.methods

Five ways to solve the same 2-D problem on the same domain.

`pinneapple_simulation/meshless/methods.py` · 427 linhas · testes: 2

nome	tipo	assinatura	o que faz
`Discretization`	classe	`(method, nodes, evaluate, setup_s, L, K, M, kansa, info, options)`	
`default_eps`	função	`(domain) -> float`	Shape parameter of the multiquadric: 3 / diameter (a fixed, not node-scaled, value: spectral convergence until the condition number stops it).
`rbf_fd_weights`	função	`(X, Q, idx, op='lap', degree=4) -> np.ndarray`	RBF-FD weights (q, n) of the stencils `idx` (q, n) (rows of `X`) for the operator `op` at the points `Q`: `lap` (the Laplacian) or `eval` (interpolation). PHS r^3 + all monomials up to `degree`, in local coordinates centred at the point and scaled by the stencil radius.
`fd_nodes`	função	`(domain, h)`	Grid nodes inside the domain and the points where grid lines cross the boundary.
`discretize`	função	`(method, domain, nodes, **options) -> Discretization`	Build the operator of `method` on `nodes` (FD builds its own grid nodes at the same spacing).
`solve`	função	`(problem, disc, *, seed=0) -> dict[str, Any]`	Solve on the discretization; returns `u` (nodal values, or the network for the PINN) and the costs.

pinneapple_simulation.meshless.pinn

PINN on the meshless node set: the interior nodes are the collocation points, the boundary nodes carry the Dirichlet condition, and training stops at a wall-clock budget (`pinneapple_analysis.cost.train_within_budget`), so the PINN is compared with the other methods at a stated cost. …

pinneapple_simulation/meshless/pinn.py · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
train_pinn	função	(problem, disc, *, budget_s=5.0, width=32, depth=3, lr=0.002, bc_weight=10.0, n_times=8, …)	

pinneapple_simulation.meshless.problems

Linear 2-D problems with Dirichlet conditions, defined by expressions in x, y (and t).

pinneapple_simulation/meshless/problems.py · 147 linhas · testes: 22

nome	tipo	assinatura	o que faz
Problem	classe	(kind, exact, source, boundary, initial, k, alpha, t_end, n_steps)	

pinneapple_simulation.meshless.stencils

Stencil weights in any dimension for meshless discretisations: RBF-FD and GFDM (weighted least squares).

pinneapple_simulation/meshless/stencils.py · 105 linhas · testes: 2

nome	tipo	assinatura	o que faz
monomials	função	(dim, degree) -> list[tuple[int, ...]]	Exponent tuples of every monomial of total degree <= degree in dim variables (constant first).
stencil_weights	função	(X, Q, idx, ops=('lap',), *, method='rbf_fd', degree=2, batch=2000) -> dict[str, np.ndarr…)	Weights {op: (q, n)} (see the module docstring). X: (N, dim) nodes; Q: (q, dim) query points; idx: (q, n) stencil node indices.

pinneapple_simulation.meshless.study

Refinement study of meshless and mesh-based methods on one problem and one domain.

pinneapple_simulation/meshless/study.py · 258 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
MeshlessStudy	classe	(problem, domain, rows, convergence, eps_sweep, wins, notes, reference, fields, skipped, …)	
meshless_study	função	(problem, dom, h=None, *, methods=…, options=None, pinn_levels='finest', n_eval=2000, tol…)	Run every method at every spacing in h (default: three halvings from diameter / 10). options: per method, e.g. {"kansa": {"eps": 2.0}, "rbf_fd": {"degree": 3}, "pinn": {"budget_s": 5}}. pinn_levels: finest (default, the PINN trains once) or all.

pinneapple_simulation.multibody

Dinâmica de corpos rígidos em coordenadas reduzidas: uma base flutuante com árvores de juntas de revolução; reexporta a árvore e os utilitários de braço.

pinneapple_simulation/multibody/__init__.py · 26 linhas · testes: 45

pinneapple_simulation.multibody.arm

Braços seriais de base fixa sobre `MultibodyTree`: dinâmica inversa e direta, referência analítica do braço plano de dois elos, trajetórias quínticas, balanço de energia e checagem de tensão na raiz dos elos.

pinneapple_simulation/multibody/arm.py · 161 linhas · testes: 39

nome	tipo	assinatura	o que faz
`fixed_base_inverse_dynamics`	função	`(tree, q, qd, qdd) -> np.ndarray`	Calcula os torques de junta ao longo de uma trajetória (q, qd, qdd) com a base fixa na origem.
`fixed_base_forward`	função	`(tree, q0, qd0, tau_fn, t) -> tuple[np.ndarray, np.ndarray]`	Integra (RK4) os ângulos e velocidades de junta sob torques dados por `tau_fn(t, q, qd)`.
`planar_two_link_torques`	função	`(m1, m2, l1, lc1, lc2, I1, I2, g, q, qd, qdd) -> np.ndarray`	Torques em forma fechada de um braço plano de dois elos num plano vertical, usados como referência independente.
`quintic`	função	`(q0, q1, T, t) -> tuple[np.ndarray, np.ndarray, np.ndarray]`	Gera uma trajetória quíntica repouso-a-repouso de `q0` a `q1` em `T` segundos (posição, velocidade, aceleração).
`energy_balance`	função	`(tree, t, q, qd, tau) -> dict`	Compara o trabalho dos torques (trapézio) com a variação de energia cinética + potencial e devolve o desvio relativo.
`link_world_points`	função	`(tree, q, points) -> dict[str, np.ndarray]`	Posições no mundo de pontos fixos em cada corpo, para cada configuração.
`lumped_link`	função	`(name, length, mass_per_m, point_masses=(), width=0.05)`	Cria um `Body` de elo reto: viga uniforme mais massas pontuais (motor, carga), com massa, centro de massa e inércia (eixos paralelos).
`link_root_checks`	função	`(tau, section, *, yield_strength, fatigue_strength=None, safety=3.0, names=None) -> tuple…)`	Tensão de flexão na raiz de cada elo a partir dos torques de junta, checada contra escoamento/segurança e, opcionalmente, fadiga.

pinneapple_simulation.multibody.tree

Árvore multicorpo de base flutuante com juntas de revolução: cinemática, jacobianos, matriz de massa, forças de viés, limites de junta por mola de penalidade e passo de tempo linearmente implícito. [sem teste]

pinneapple_simulation/multibody/tree.py · 332 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Body`	classe	`(name, mass, com, inertia, parent_joint)`	Corpo rígido com massa, centro de massa e tensor de inércia no referencial do corpo.
`RevoluteJoint`	classe	`(name, parent, child, origin, axis, R0, limits, damping)`	Junta de revolução entre pai e filho, com origem, eixo, rotação inicial, limites (mola de penalidade) e amortecimento.
`Kinematics`	classe	`(R, p, c, axis, o, Jv, Jw, omega, vel)`	Poses e velocidades no mundo de todos os corpos para um estado, com jacobianos.
`MultibodyTree`	classe	`(gravity=(0.0, 0.0, -1.62))`	Árvore multicorpo (o primeiro corpo é a base flutuante) com cinemática, matriz de massa, forças de viés, gravidade, passo de tempo e energia.

nome	tipo	assinatura	o que faz
`cross`	função	`(a, b)`	Produto vetorial de arrays (..., 3), mais barato que `np.cross` para arrays pequenos.
`skew`	função	`(a)`	Matriz antissimétrica de um vetor 3D.
`quat_to_matrix`	função	`(q)`	Converte quatérnio em matriz de rotação.
`axis_angle`	função	`(axis, angle)`	Matriz de rotação a partir de eixo e ângulo.
`quat_mul`	função	`(a, b)`	Produto de dois quatérnios.
`quat_from_matrix`	função	`(R)`	Converte matriz de rotação em quatérnio.

`pinneapple_simulation.numerical_solvers`

Módulo sem docstring e sem texto.

`pinneapple_simulation/numerical_solvers/__init__.py` · 73 linhas · testes: 111

nome	tipo	assinatura	o que faz
`openfoam_available`	função	`() -> bool`	Return True if the OpenFOAM bridge is importable.
`fenics_available`	função	`() -> bool`	Return True if the FEniCS bridge is importable.

`pinneapple_simulation.numerical_solvers._bc`

Read Dirichlet conditions of a ProblemSpec for grid / node-based solvers.

`pinneapple_simulation/numerical_solvers/_bc.py` · 82 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`dirichlet_conditions`	função	`(spec) -> List`	
`select`	função	`(cond, pts, edges=None, boundary=None) -> torch.Tensor`	Indices of `pts` the condition applies to.
`values`	função	`(cond, pts, field=0) -> torch.Tensor`	Dirichlet values at `pts` for one field (column `field` of a multi-field value_fn).
`boundary_rect`	função	`(nodes, edges) -> torch.Tensor`	

`pinneapple_simulation.numerical_solvers.adapters`

Módulo sem docstring e sem texto.

`pinneapple_simulation/numerical_solvers/adapters/__init__.py` · 20 linhas · **sem teste direto**

pinneapple_simulation.numerical_solvers.adapters.upd_mesh

UPD mesh adapter for solver inputs (vertices, faces, topology).

`pinneapple_simulation/numerical_solvers/adapters/upd_mesh.py` · 163 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mesh_to_fvm_topology`	função	`(mesh, *, device=None, dtype=None) -> Dict[str, torch.Tensor]`	Build a minimal FVM topology from a surface mesh (triangles/quads).
`mesh_to_fem_assembly_inputs`	função	`(mesh, *, device=None, dtype=None) -> Dict[str, torch.Tensor]`	Minimal FEM assembly inputs (MVP): - nodes (N,3) - elements (E,k) connectivity The actual K,f assembly is handled by user-provided assemble_fn (in FEMSolver).

pinneapple_simulation.numerical_solvers.adapters.upd_signal

UPD signal/sample adapter for solver field extraction and stacking.

`pinneapple_simulation/numerical_solvers/adapters/upd_signal.py` · 184 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`list_field_vars`	função	`(sample) -> List[str]`	Tries to list available physical field variables from a PhysicalSample-like object.
`list_signal_axes`	função	`(sample, var) -> List[str]`	Best-effort axis naming for a field. If sample has sample.axes[var] or sample.dims[var], use it. Fallback: ["dim0","dim1",...]
`extract_1d_signal`	função	`(sample, *, var, axis=-1, reduce='mean', index=None, device=None, dtype=None) -> Tuple[to…)`	Extract a 1D signal from a (possibly multi-dim) physical field.
`to_signal_batch`	função	`(samples, *, var, axis=-1, reduce='mean', index=None, device=None, dtype=None) -> Tuple[t…)`	Convert a list of samples into a padded batch tensor (B,Tmax).

pinneapple_simulation.numerical_solvers.base

Base SolverOutput and SolverBase for numerical solvers.

`pinneapple_simulation/numerical_solvers/base.py` · 34 linhas · testes: 151

nome	tipo	assinatura	o que faz
`SolverOutput`	classe	`(result, losses, extras)`	Standard output for solvers / numerical methods. result: - main output tensor (spectra, modes, field solution, etc.) extras: - metadata (freqs, imfs, diagnostics, residuals)
`SolverBase`	classe	`()`	Base class for numerical solvers.

pinneapple_simulation.numerical_solvers.beam_bvp_fdm

Euler-Bernoulli beam deflection: 4th-order finite-difference solve of the beam BVP

pinneapple_simulation/numerical_solvers/beam_bvp_fdm.py · 296 linhas · testes: 5

nome	tipo	assinatura	o que faz
SectionProps	classe	(I_m4, c_m, Q_m3, t_m, A_m2)	Cross-section properties required for stress calculations.
BeamBVPFDMSolver	classe	(nx=200, bc_type=BC_SIMPLY_SUPPORTED)	Thin SolverBase/registry wrapper. solve_euler_bernoulli_beam_1d is the primary entry point and can be used directly without this wrapper.
compute_stresses	função	(moment, shear, section)	Bending stress (extreme fibre), shear stress (neutral axis), von Mises.
second_derivative	função	(f, dz) -> np.ndarray	Second derivative of f, second-order accurate everywhere including the two boundary points (3-point central stencil in the interior, a 3-point one-sided stencil at each edge).
solve_euler_bernoulli_beam_1d	função	(L_m=10.0, E_Pa=200000000000.0, I_m4=8.33e-06, q_N_per_m=1000.0, nx=200, bc_type=BC_SIMPL…)	Solve the 1D Euler-Bernoulli beam BVP EI*w''''=q under a uniform distributed load q_N_per_m, for one of the three BC_* boundary condition sets. section (default: an 0.10x0.10 m rectangular section) supplies the cross-section geometry used for the returned stresses -- pass a SectionProps built …

pinneapple_simulation.numerical_solvers.beam_fe

Euler-Bernoulli beam by Hermite finite elements: static deflection and natural modes.

pinneapple_simulation/numerical_solvers/beam_fe.py · 74 linhas · testes: 27

nome	tipo	assinatura	o que faz
beam_matrices	função	(L, EI, m, n_el)	Global stiffness, consistent mass and node positions of a uniform beam.
beam_static	função	(L, EI, q, n_el=40, ends=('pinned', 'pinned')) -> dict	Deflection under a uniform load q (N/m, positive down) with consistent nodal loads.
beam_modes	função	(L, EI, m, n_el=40, n_modes=4, ends=('pinned', 'pinned')) -> dict	Natural frequencies (Hz) and mass-normalised mode shapes (deflection at the nodes).

pinneapple_simulation.numerical_solvers.bekker_wong

Bekker-Wong numerical solver for rigid-wheel / deformable-soil interaction.

pinneapple_simulation/numerical_solvers/bekker_wong.py · 347 linhas · testes: 50

nome	tipo	assinatura	o que faz
BekkerWongSolver	classe	(soil=None, wheel=None, rear_angle_ratio=1.0 / 3.0)	Numerical Bekker-Wong solver.
bekker_wong_forces_torch	função	(slip, z, *, R, b, c, tan_phi, K, k_c, k_phi, n, a0=0.4, a1=0.15, rear_angle_ratio=1.0 / …)	Bekker-Wong (F_x, F_z, M_y) for batches, piecewise Gauss-Legendre on the three smooth sub-intervals [theta_r, min(0,theta_m)], [.., theta_m], [theta_m, theta_f].

pinneapple_simulation.numerical_solvers.ceemdan

CEEMDAN: Complete Ensemble Empirical Mode Decomposition with Adaptive Noise.

pinneapple_simulation/numerical_solvers/ceemdan.py · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CEEMDANSolver	classe	(n_ensembles=50, noise_std=0.2, max_imfs=10, seed=None)	
ceemdan_1d	função	(x, *, n_ensembles=50, noise_std=0.2, max_imfs=10, seed=None) -> np.ndarray	Approximate CEEMDAN. Returns imfs (K,T).

pinneapple_simulation.numerical_solvers.cfd_pipeline

Full CAD-to-CFD pipeline: STEP/STL → mesh → Navier-Stokes solve.

pinneapple_simulation/numerical_solvers/cfd_pipeline.py · 954 linhas · testes: 151

nome	tipo	assinatura	o que faz
CFDMesh	classe	(nodes, elements, boundary_tags) -> None	Unstructured mesh for CFD computations.
NSFlowSolver	classe	(mesh, nu=0.001, rho=1.0, solver='direct') -> None	Mesh-native incompressible Navier-Stokes solver.
CADToCFDPipeline	classe	(nu=0.001, rho=1.0) -> None	Full pipeline: CAD file → mesh → NS solve → PINN training data.

pinneapple_simulation.numerical_solvers.compressible_fv

Compressible Navier-Stokes / Euler finite-volume solver (2-D and 3-D Cartesian), PyTorch.

pinneapple_simulation/numerical_solvers/compressible_fv.py · 299 linhas · testes: 15

nome	tipo	assinatura	o que faz
CompressibleConfig	classe	(shape, lengths, gamma, Pr, mu_ref, omega, cfl, limiter, bc, inflow, p_out, device, dtype…)	
CompressibleFV	classe	(cfg)	
sod_exact	função	(x, t, left=(1.0, 0.0, 1.0), right=(0.125, 0.0, 0.1), gamma=1.4, x0=0.5)	Exact solution of the 1-D Riemann problem (Toro, *Riemann Solvers and Numerical Methods for Fluid Dynamics*, 3rd ed., 2009, ch. 4); returns rho, u, p on x for `left`/`right` = (rho, u, p).

pinneapple_simulation.numerical_solvers.cylinder_wake

Confined cylinder wake with the D2Q9 lattice-Boltzmann solver: one run, its verification and the zoo dataset `cylinder_wake_lbm`.

pinneapple_simulation/numerical_solvers/cylinder_wake.py · 178 linhas · testes: 3

nome	tipo	assinatura	o que faz
probe_strouhal	função	(signal, dt) -> float	Strouhal number of a probe record sampled every dt convective units: strongest peak of the Hann-windowed, 4x zero-padded spectrum with Gaussian peak interpolation (a fraction of a bin).
simulate	função	(Re, D=SETUP['D'], height=SETUP['height'], length=SETUP['length'], u_in=SETUP['u_in'], co…)	One run. Returns the setup, the convective times t and vorticity frames vorticity (n, nx, ny) of the last keep_time convective units (None: the second half of the run), the probe probe_uy (/U) over the same times, wake_amplitude (its rms), amplitude_growth (probe rms of …
verify	função	(Ds=(20, 30, 40), Re=100.0, channel_Re=2.0) -> dict	Developed channel profile (plane Poiseuille; at channel_Re on D the entrance length is ~2.5 D, so the profile is developed 9 D downstream) on the dataset grid and a 2x coarser one, and the resolution study of the wake at Re. At the dataset's Reynolds numbers the 12 D channel is far shorter …
generate	função	(out, workers=1, name='cylinder_wake_lbm') -> dict	Run :data:PLAN and write out/<name>/<split>[-NN].npz (one row per snapshot) and meta.json.

pinneapple_simulation.numerical_solvers.dfsph

DFSPH: Divergence-Free SPH (Bender & Koschier 2015 / 2017).

pinneapple_simulation/numerical_solvers/dfsph.py · 317 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
DFSPHSolver	classe	(h=0.04, mass=1.0, rho0=1000.0, mu=0.02, gravity=None, boundary_lo=None, boundary_hi=None…)	Divergence-Free SPH (DFSPH) solver.

pinneapple_simulation.numerical_solvers.discrete_time_pinn

Discrete-time physics-informed neural network (PINN), per Raissi, Perdikaris & Karniadakis, "Physics Informed Deep Learning (Part II): Data-driven Discovery of Nonlinear Partial Differential Equations" (arXiv:1711.10566), Section 2 ("Discrete Time Models"), specialized here to the pure …

pinneapple_simulation/numerical_solvers/discrete_time_pinn.py · 408 linhas · testes: 7

nome	tipo	assinatura	o que faz
DiscreteTimeRKNet	classe	(q, hidden_layers=(50, 50, 50, 50), activation='tanh')	x -> (q+1) discrete-time IRK network (Raissi et al. Part II, Section 2). Column i<q is the i-th RK stage prediction U_i(x); column q is the final-time prediction U_{q+1}(x). Plain tanh MLP, matching the original paper's architecture choice.
DiscreteTimeResult	classe	(model, A, b, c, q, dt, x_span, loss_history)	
DiscreteTimePINNSolver	classe	(pde='burgers_1d', q=32, hidden_layers=(30, 30, 30), activation='tanh', n_boundary=20, bc…)	Thin SolverBase/registry wrapper around train_discrete_time. The functional API (train_discrete_time, DiscreteTimeRKNet, stage_derivatives, discrete_time_rk_residual) is the primary entry point.

nome	tipo	assinatura	o que faz
`stage_derivatives`	função	`(model, x)`	U(x), U_x(x), U_xx(x), each (N, q+1), via forward-mode AD.
`discrete_time_rk_residual`	função	`(model, x, u_n_data, dt, A, b, nonlinear_op) -> 'tuple[torch.Tensor, torch.Tensor]'`	Stage and final-time IRK consistency residuals (module docstring point 4). Returns (res_stage, res_final), shapes (N, q) and (N, 1).
`train_discrete_time`	função	`(pde, x_n, u_n, dt, q=32, x_span=None, hidden_layers=(30, 30, 30), activation='tanh', par…)`	Train a `DiscreteTimeRKNet` for one implicit-RK step of `pde` in {"burgers_1d", "allen_cahn_1d"}, from known data (x_n, u_n) at t_n to t_n + dt, via L-BFGS (module docstring point 5).

pinneapple_simulation.numerical_solvers.eddy_current_fdm

2D axisymmetric magnetostatic / eddy-current FDM solver.

`pinneapple_simulation/numerical_solvers/eddy_current_fdm.py` · 240 linhas · testes: 74

nome	tipo	assinatura	o que faz
`EddyCurrentFDMSolver`	classe	`(nr=60, nz=120)`	Thin `SolverBase`/registry wrapper. The functional API (`solve_axisymmetric_eddy_current`, `build_axisymmetric_helmholtz_system`, `annular_current_source`) is the primary entry point and can be used directly without this wrapper.
`build_axisymmetric_helmholtz_system`	função	`(r, z, k2, rhs_2d)`	Assemble the sparse complex system M @ a = b for the axisymmetric operator
`annular_current_source`	função	`(r, z, r_center, z_center, width_z, width_r, n_turns, current_amp) -> np.ndarray`	Uniform azimuthal current density J_theta (A/m^2) in a rectangular annular band centered at (r_center, z_center) -- the standard lumped-coil approximation for an axisymmetric current loop / solenoid turn, used to drive the eddy-current or magnetostatic equation above. The band is ``|z - z_center| …
`solve_axisymmetric_eddy_current`	função	`(r, z, omega, mu, sigma, source_current_density) -> np.ndarray`	Solve for the azimuthal vector potential A_theta(r,z) at angular frequency `omega` (rad/s), given material maps mu(r,z) [H/m] and sigma(r,z) [S/m] and a source current density J_source(r,z) [A/m^2], all as (len(r), len(z)) arrays matching the r,z grids. `omega=0` solves the magnetostatic limit (no …
`eddy_current_density`	função	`(A, omega, sigma) -> np.ndarray`	Induced eddy-current density J_theta = -i*omega*sigma*A, given the solved vector potential A and the same sigma(r,z) map used to solve it.
`axial_flux_density`	função	`(A, r, z) -> np.ndarray`	B_z(r,z) = (1/r) d(r*A)/dr = A/r + dA/dr, computed via central differences (the axisymmetric curl relating A_theta to the axial field component). Complex A gives the complex phasor B_z (in-phase and quadrature parts); A = B0 r / 2 gives B_z = B0, on the axis too.

pinneapple_simulation.numerical_solvers.eemd

EEMD: Ensemble Empirical Mode Decomposition.

`pinneapple_simulation/numerical_solvers/eemd.py` · 167 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
EEMDSolver	classe	(n_ensembles=50, noise_std=0.2, max_imfs=10, seed=None)	
emd_1d	função	(x, *, max_imfs=10, max_sift=50, stop_thresh=0.05) -> List[np.ndarray]	Very small EMD implementation for 1D signals.
eemd_1d	função	(x, *, n_ensembles=50, noise_std=0.2, max_imfs=10, seed=None) -> np.ndarray	EEMD for 1D: returns imfs array (K, T).

pinneapple_simulation.numerical_solvers.elasticity3d_fdm

Static 3D linear (isotropic, small-strain) elasticity via finite differences: solves the Navier-Cauchy equilibrium equations

pinneapple_simulation/numerical_solvers/elasticity3d_fdm.py · 182 linhas · **sem teste direto** · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
Elasticity3DFDMSolver	classe	(n_iter=20000, omega=1.0, tol=1e-10)	Thin SolverBase/registry wrapper. The functional API (solve_static_elasticity_3d, lame_parameters) is the primary entry point and can be used directly without this wrapper.
lame_parameters	função	(E, nu) -> Tuple[float, float]	Lame's first parameter (lambda) and shear modulus (mu) from Young's modulus E and Poisson's ratio nu.
solve_static_elasticity_3d	função	(lam, mu, body_force, u_bc, dx, dy, dz, n_iter=20000, omega=1.0, tol=1e-10) -> dict	Solve the static Navier-Cauchy equations on a structured grid via damped-Jacobi relaxation.

pinneapple_simulation.numerical_solvers.etdrk4

ETDRK4: fourth-order exponential time differencing Runge-Kutta for stiff semilinear problems `du/dt = L u + N(u, t)` with a *diagonal* linear part.

pinneapple_simulation/numerical_solvers/etdrk4.py · 151 linhas · testes: 17

nome	tipo	assinatura	o que faz
SimulationBlowUp	classe	(message, *, step, t)	The state became non-finite or exceeded the magnitude limit.
ETDRK4Result	classe	(t, u, steps, blew_up, meta)	
ETDRK4	classe	(linear, nonlinear, dt, *, contour_points=32, radius=1.0)	Fixed-step ETDRK4 integrator.
check_finite	função	(u, *, limit=1e+37, step=-1, t=float('nan')) -> None	Raise :class:SimulationBlowUp if u has a NaN/inf or |u| > limit.
fourier_wavenumbers	função	(n, length=2 * np.pi, *, real=False) -> np.ndarray	Angular wavenumbers k of an n-point periodic grid of the given length, in FFT order (np.fft.rfft order when real).

pinneapple_simulation.numerical_solvers.external_flow

External flow around any body in OpenFOAM, in one call: STL -> snappyHexMesh -> simpleFoam (k-omega SST, wall functions) -> forces, skin pressure and friction, streamlines and slice planes -> a 3D

scene to render or browse.

`pinneapple_simulation/numerical_solvers/external_flow.py` · 728 linhas · testes: 40

nome	tipo	assinatura	o que faz
`CaseFields`	classe	`(case, time=None)`	The reconstructed mesh and the latest U, p of a case, with the helpers the results need: wall forces and skin coefficients on given patches, a velocity interpolator, streamline tracing, plane sampling.
`FlowResult`	classe	`(coefficients, surface, bodies, lines, line_speed, slices, info, geometry)`	
`ExternalFlow`	classe	`(bodies, speed, alpha, beta, nu, rho, ref_area, ref_length, half_model, ground, ground_mo…)`	Steady incompressible flow around `bodies` (dict name -> (vertices, faces), an STL path, or a studio Scene).
`flow_directions`	função	`(alpha_deg, beta_deg=0.0) -> Tuple[np.ndarray, np.ndarray, np.ndarray]`	Unit drag (along the flow), lift (up, normal to it in the x-z plane) and side directions.
`write_flow_fields`	função	`(case, speed, alpha_deg, beta_deg, nu, iterations, procs, walls, symmetry, ground=None, g…)`	0/ fields, physical properties, controlDict, schemes and solvers of a steady external-flow case. `walls` is the patch group of the bodies; `ground`: name of a road patch (moving with `ground_velocity`); `turbulence`: kOmegaSST (wall functions) or laminar (no k, omega, nut fields).
`snappy_dict`	função	`(geometry, features, surfaces, regions, location) -> str`	snappyHexMeshDict body (castellate + snap, no layers: wall functions).
`run_case`	função	`(case, procs=4, log=print, all_times=False) -> Dict[str, float]`	blockMesh, surfaceFeatureExtract, snappyHexMesh, checkMesh and simpleFoam in parallel, then reconstruct (the latest time, or every written time with `all_times`). Returns the seconds per step; each step's log is in `case/log.<step>`.
`latest_time`	função	`(case) -> str`	
`openfoam_available`	função	`() -> bool`	True when the OpenFOAM tools this module calls can be found.

`pinneapple_simulation.numerical_solvers.fdm`

Finite Difference Method (FDM) — problem-agnostic numerical solver.

`pinneapple_simulation/numerical_solvers/fdm.py` · 919 linhas · testes: 3

nome	tipo	assinatura	o que faz
`FDMSolver`	classe	`(nx=64, ny=64, nt=500, iters=8000, omega=1.5, theta=0.5, tol=1e-09)`	Problem-agnostic FDM solver.

`pinneapple_simulation.numerical_solvers.fdm3d`

3D Finite Difference Method (FDM) solvers.

`pinneapple_simulation/numerical_solvers/fdm3d.py` · 911 linhas · testes: 151

nome	tipo	assinatura	o que faz
`SolverOutput3D`	classe	`(u, coords, meta)`	Output from a 3D FDM solver.
`HeatConfig3D`	classe	`(nx, ny, nz, nt, lx, ly, lz, dt, alpha, ic_fn, source_fn)`	Configuration for 3D heat conduction solver.
`HeatConduction3D`	classe	`(cfg=None)`	3D heat equation solver using explicit Euler + central differences.
`NavierStokesConfig3D`	classe	`(nx, ny, nz, nt, lx, ly, lz, dt, nu, rho, pressure_iters, ic_fn, force_fn)`	Configuration for 3D incompressible Navier-Stokes solver.
`NavierStokes3D`	classe	`(cfg=None)`	3D incompressible Navier-Stokes via fractional-step (projection) method.
`ElasticWaveConfig3D`	classe	`(nx, ny, nz, nt, lx, ly, lz, dt, c, ic_u_fn, ic_du_fn, source_fn)`	Configuration for 3D elastic wave solver.
`ElasticWave3D`	classe	`(cfg=None)`	3D elastic (acoustic) wave equation solver via Störmer-Verlet integration.
`LidDrivenCavityConfig3D`	classe	`(nx, ny, nz, nt, size, dt, nu, rho, lid_velocity, lid_axis, pressure_iters)`	Configuration for the 3D lid-driven cavity solver.
`LidDrivenCavitySolver3D`	classe	`(cfg=None)`	3D Lid-Driven Cavity flow solver.
`ChannelFlowConfig3D`	classe	`(nx, ny, nz, nt, lx, ly, lz, dt, nu, rho, dp_dx, pressure_iters)`	Configuration for the 3D pressure-driven channel flow solver.
`ChannelFlowSolver3D`	classe	`(cfg=None)`	3D pressure-driven channel (Poiseuille-like) flow solver.

`pinneapple_simulation.numerical_solvers.fem`

Finite Element Method — problem-agnostic P1/Q1 solver.

`pinneapple_simulation/numerical_solvers/fem.py` · 368 linhas · testes: 8

nome	tipo	assinatura	o que faz
`FEMSolver`	classe	`(nx=32, ny=32, solver='cg', tol=1e-08, max_iter=4000)`	Problem-agnostic FEM solver.

`pinneapple_simulation.numerical_solvers.fenics_bridge`

FEniCS/dolfinx bridge for pinneapple.

`pinneapple_simulation/numerical_solvers/fenics_bridge.py` · 647 linhas · testes: 151

nome	tipo	assinatura	o que faz
`FEnicsBridge`	classe	`(mesh_nx=32, mesh_ny=32, element_degree=1, solver_backend='dolfinx') -> None`	Bridge that formulates and solves FEM problems using FEniCS/dolfinx.

`pinneapple_simulation.numerical_solvers.fft`

FFT solver for spectrum computation with rFFT/FFT support.

`pinneapple_simulation/numerical_solvers/fft.py` · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FFTSolver`	classe	`(*, dims=None, real_input=True, output='magnitude_phase', norm=None)`	FFT Solver (PyTorch backend).

pinneapple_simulation.numerical_solvers.free_surface

Free-surface flow of a liquid and a gas in OpenFOAM (interFoam, volume of fluid), in one object: an initial liquid level, inlets with a velocity vector, open boundaries, k-omega SST, and turbulence sampled on lines and averaged in time. Built for melt runners, launders, spillways and channels.

`pinneapple_simulation/numerical_solvers/free_surface.py` · 626 linhas · testes: 12

nome	tipo	assinatura	o que faz
`Phase`	classe	`(name, rho, nu)`	
`SampleLine`	classe	`(name, start, end, n)`	
`FreeSurfaceFlow`	classe	`(regions, roles, mesh, inlet_velocity, level, liquid, gas, sigma, gravity, p_ref, turbule…)`	
`LiveLineSampler`	classe	`(case, lines, fields, keep_every=None, end_time=float('inf'), out_name='lines', poll_s=3.…)`	Samples lines from a running case as each time is written, then optionally deletes that time.
`split_surface`	função	`(V, F, boxes, rest='walls') -> dict[str, tuple[np.ndarray, np.ndarray]]`	Split one closed surface into named regions: a triangle goes to the first box that holds its centroid, otherwise to `rest`. Feed the result to `FreeSurfaceFlow(regions=...)`.
`cell_centres`	função	`(poly_dir) -> np.ndarray`	Cell centres (mean of face centres) from an ASCII constant/polyMesh.
`write_profile`	função	`(case, tname, idx, res, fields, out_name='lines') -> None`	
`sample_lines`	função	`(case, lines, fields) -> int`	Sample every written time of a finished case (serial or decomposed) onto `lines`. Returns the count.
`read_profiles`	função	`(case, names=None, t_start=0.0, t_end=float('inf'), out_name='lines') -> dict`	{line: {"s": distance along the line, "xyz", "times", : (n_times, n_points), "mean_"}}.
`time_average`	função	`(profile, field, window=None, origin='min') -> tuple[np.ndarray, np.ndarray]`	Time-mean of `field` along a line against the distance from the line's first sampled point (`origin="min"`), optionally cut to `window` = (s0, s1).

pinneapple_simulation.numerical_solvers.fvm

Finite Volume Method — problem-agnostic cell-centered structured solver.

`pinneapple_simulation/numerical_solvers/fvm.py` · 345 linhas · testes: 5

nome	tipo	assinatura	o que faz
`FVMSolver`	classe	`(nx=64, ny=64, nt=200, dt=None, safety=0.4)`	Problem-agnostic FVM solver on a structured cell-centred 2-D grid.

pinneapple_simulation.numerical_solvers.gated_xtfc

Gated X-TFC: X-TFC with Gaussian RBF features whose centers and widths follow a soft domain split, for 1D second-order linear boundary value problems with sharp layers (forward and inverse).

pinneapple_simulation/numerical_solvers/gated_xtfc.py · 309 linhas · testes: 8

nome	tipo	assinatura	o que faz
LinearBVP1D	classe	(a, b, c, f, u_left, u_right, nu, exact)	a u'' + b u' + c u = f on [0, 1], u(0) = u_left, u(1) = u_right.
GatedLayout	classe	(centers, widths, collocation, splits, eps_b)	
GatedSolution	classe	(problem, layout, coef, solve_seconds)	
InverseFit	classe	(nu, splits, eps_scales, log_evidence, eta, solution, history, seconds)	
convection_diffusion	função	(nu) -> LinearBVP1D	u' - nu u'' = 0, u(0) = 0, u(1) = 1; u = (e^{x/nu} - 1)/(e^{1/nu} - 1) (paper eq. 1-2).
twin_boundary_layer	função	(nu) -> LinearBVP1D	2(2x - 1) u' - nu u'' + 4u = 0, u(0) = u(1) = 1; u = exp(-2x(1 - x)/nu) (paper Sec. 4.4).
gated_layout	função	(splits, eps_scales, nu, n_centers, n_colloc=None, k=1.5) -> GatedLayout	Centers, widths and collocation points of the soft split (per block n_centers centers and n_colloc collocation points). With one split this is the paper's two-block layout; with several, each split blends the widths of its two neighbouring blocks by its own logistic gate.
solve_forward	função	(problem, layout, lam=1e-12) -> GatedSolution	argmin (1/N) ||R c + r0||^2 + lam ||c||^2 at the layout's collocation points (stacked least squares).
validation_energy	função	(sol, x_val) -> float	Mean squared strong-form residual on the validation points.
search_gates_nested	função	(problem, x_val, *, n_centers, n_colloc=None, k=1.5, lam=1e-12, xs_bounds=(0.8, 0.999), e…)	One split: outer bounded Brent search over x_s, inner over eps_scale, minimising the validation residual energy (the paper's nested fminbnd). Returns the best solution and the search record.
log_evidence	função	(Phi, y, *, eta0=1e-07, eta_bounds=(1e-12, 0.01), max_iter=50, damping=0.5) -> dict	Bayesian linear model y = Phi c + e, e ~ N(0, I) (whitened), c ~ N(0, eta^-1 I): the prior precision eta by the evidence fixed point gamma = sum k/(k + eta), eta <- gamma / ||m||^2 (damped in log space, clipped), then the log-evidence, posterior mean and the …
fit_inverse	função	(make_problem, x_data, y_data, *, sigma, beta_pde, n_centers, n_colloc, k=1.5, log10_nu_b…)	Infer nu (and the gate) by maximising the evidence of the stacked whitened data + physics system over (log10 nu, x_s, eps_scale) with the library's Bayesian optimiser (GP + expected improvement).

nome	tipo	assinatura	o que faz
`search_gates_bo`	função	`(problem, x_val, *, split_bounds, eps_bounds=(10.0, 100.0), n_centers, n_colloc=None, k=1…)`	Several splits: minimise the validation residual energy (log10) over all splits and their eps_scales with the library's Bayesian optimiser (the paper's choice for the multi-split twin boundary layer).

pinneapple_simulation.numerical_solvers.heat_fv_reference

Independent reference solutions for the 2D and 3D cases: cell-centred finite volumes, sparse direct solve.

`pinneapple_simulation/numerical_solvers/heat_fv_reference.py` · 123 linhas · testes: 3

nome	tipo	assinatura	o que faz
`footprint`	função	`(x, y, xc, yc, half, width)`	Smooth indicator of a square device footprint (1 inside, 0 outside, tanh edges of `width`).
`footprint_area`	função	`(xc, yc, half, width, lx, ly, n=2000) -> float`	Area (m2) of the smooth `footprint` over the plate [0, lx] x [0, ly] (all lengths in m).
`plate_2d`	função	`(h, *, lx, ly, t, k, power, t_air, device, nx=200, ny=120)`	Temperature (°C) at the cell centres of an (ny, nx) grid; returns (x, y, T).
`block_3d`	função	`(h, *, lx, ly, lz, k, power, t_air, device, nx=40, ny=40, nz=12)`	Temperature (°C) at the cell centres of an (nz, ny, nx) grid; returns (x, y, z, T).
`interp_grid`	função	`(xs, ys, field, px, py)`	Bilinear interpolation of a cell-centred 2D field at points (px, py), clamped to the grid.
`convergence`	função	`(fn, coarse, fine, **kw) -> float`	Largest temperature difference (K) between two grids, compared at the coarse cell centres.

pinneapple_simulation.numerical_solvers.hilbert_huang

Hilbert-Huang transform (EMD + Hilbert) solver for time series.

`pinneapple_simulation/numerical_solvers/hilbert_huang.py` · 225 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HilbertHuangSolver`	classe	`(*, max_imfs=8, max_sift=50, stop_mean_tol=0.001, dt=1.0)`	Hilbert–Huang Transform (HHT) MVP for 1D signals.

pinneapple_simulation.numerical_solvers.immersed_boundary_fdm

Immersed-boundary-method (IBM) finite-difference solver for incompressible Navier-Stokes on an arbitrary 3D geometry.

`pinneapple_simulation/numerical_solvers/immersed_boundary_fdm.py` · 964 linhas · testes: 2

nome	tipo	assinatura	o que faz
`ImmersedBoundaryFDMSolver`	classe	`(flow_type='internal', **solver_kwargs)`	Thin `SolverBase`/registry wrapper. The functional API (`solve_ibm_internal_flow`, `solve_ibm_external_flow`, `solve_ibm_navier_stokes`) is the primary entry point and can be used directly without this wrapper.
`field_divergence`	função	`(u, v, w, dx, dy, dz) -> np.ndarray`	Central-difference divergence of a velocity field -- exposed standalone for verifying the near-zero-divergence property of a projected field.
`interpolate_to_points`	função	`(grid_axes, field, query_points, fill_value=0.0) -> np.ndarray`	Trilinear interpolation of a (nx,ny,nz) grid field onto arbitrary query points, via `scipy.interpolate.RegularGridInterpolator`.
`solve_ibm_internal_flow`	função	`(wall_points, bbox, *, Re=100.0, U_in=1.0, rho=1.0, nx=32, ny=32, nz=32, n_iter=500, n_fr…)`	IBM/FDM incompressible Navier-Stokes on an enclosed domain whose bounding box IS the fluid domain.
`solve_ibm_external_flow`	função	`(wall_points, bbox, *, Re=100.0, U_in=1.0, rho=1200.0, nx=32, ny=32, nz=32, n_iter=1000, …)`	IBM/FDM incompressible Navier-Stokes for external (free-stream) flow around an arbitrary solid immersed body.
`solve_ibm_navier_stokes`	função	`(wall_points, bbox, flow_type='internal', **kwargs) -> Dict[str, Any]`	Dispatch to `solve_ibm_internal_flow` (`flow_type="internal"`) or `solve_ibm_external_flow` (`flow_type="external"`). All other keyword arguments are forwarded to the selected solver.

`pinneapple_simulation.numerical_solvers.internal_flow`

Internal flow in pipes, bends and static mixers in OpenFOAM, in one call.

`pinneapple_simulation/numerical_solvers/internal_flow.py` · 817 linhas · testes: 35

nome	tipo	assinatura	o que faz
`Route`	classe	`(D, segments)`	A pipe centreline: straight runs and circular bends, lengths and radii in diameters. Starts at the origin along +x; bends turn in the plane given by `plane_deg` (0: towards +y, 90: towards +z).
`InternalResult`	classe	`(route, info, sections, profiles, slices, lines, line_speed, wall, timings)`	
`InternalFlow`	classe	`(route, Re, nu, turbulent, mixer, mixer_start_D, scalar, inlet, n_core, n_radial, wall_gr…)`	A pipe `route` at Reynolds number `Re` (bulk velocity and diameter). `turbulent`: None picks laminar below Re 2300. `mixer`: number of Kenics elements starting at `mixer_start_D` (straight routes). `scalar`: carry a passive scalar entering on the upper half of the inlet (c = 1 where the ...
`kenics_elements`	função	`(route_D, n_elements, start_D, length_D=1.5, thickness_D=0.06, width_D=0.98, nu=10, ns=48…)`	Kenics static-mixer elements in a straight pipe along +x starting at `start_D`: plates twisted by 180 deg, alternately left- and right-handed, each turned 90 deg from the previous one. A closed triangulated surface.

nome	tipo	assinatura	o que faz
`oblock_mesh_dict`	função	`(route, n_core=8, n_radial=10, wall_grading=0.25, axial_cell_D=0.2, core=0.36, convert=1.…)`	blockMeshDict of the O-grid pipe swept along `route`: five blocks per station interval (a square core of half-diagonal `core` x radius, four curved blocks), arcs on the wall, patches inlet / outlet / wall. `wall_grading`: wall cell / core-side cell in the radial direction.
`pipe_surface`	função	`(st, r, nth=64, sub=4) -> tuple[np.ndarray, np.ndarray]`	Triangulated wall of a swept circular section (stations from `Route.stations`), subdividing straight runs.
`ito_bend_loss`	função	`(Re, radius_ratio, angle_deg=90.0) -> float`	Total loss coefficient of a smooth pipe bend in turbulent flow (Ito 1960, J. Basic Eng. 82: 131-143), with `radius_ratio` = bend radius / pipe radius, valid for Re (r/R)^2 > 91; includes friction over the bend and the excess loss recovered downstream.
`colebrook`	função	`(Re, rel_roughness=0.0) -> float`	Darcy friction factor of fully developed turbulent pipe flow (Colebrook-White): :func:`pinneapple_physics.closed_form.pipe_flow.colebrook`.

`pinneapple_simulation.numerical_solvers.irk_gauss_legendre`

Butcher tableaus for the q-stage Gauss-Legendre implicit Runge-Kutta (IRK) family -- a general-purpose, arbitrarily-high-order (order 2q), A-stable and B-stable time integrator for any ODE-IVP system dy/dt = f(t,y), stiff or non-stiff. Pure NumPy/SciPy, no model/framework dependency, so it can be ...

`pinneapple_simulation/numerical_solvers/irk_gauss_legendre.py` · 236 linhas · testes: 7

nome	tipo	assinatura	o que faz
`build_gauss_legendre_tableau`	função	`(q) -> Tuple[np.ndarray, np.ndarray, np.ndarray]`	Build the q-stage Gauss-Legendre IRK tableau (A, b, c) on [0,1].
`validate_tableau`	função	`(A, b, c, q, tol=1e-09) -> dict`	Hard-assert the order conditions that certify a genuine 2q-order collocation method (C(q) and B(2q) jointly). Raises AssertionError with diagnostic residuals on failure; returns a dict of residuals on success.
`get_irk_tableau`	função	`(q, cache_dir=None) -> Tuple[np.ndarray, np.ndarray, np.ndarray]`	Build-or-load-cached, then validate, the q-stage Gauss-Legendre IRK tableau. Raises loudly (AssertionError/RuntimeError) rather than returning a silently-wrong tableau.
`step_irk`	função	`(rhs_fn, t, y, h, A, b, c, jac_fn=None, newton_tol=1e-10, newton_max_iter=25) -> np.ndarr…)`	Advance y(t) -> y(t+h) by one fully-implicit Runge-Kutta step using tableau (A, b, c) (e.g. from `get_irk_tableau`). Solves the q coupled stage equations

`pinneapple_simulation.numerical_solvers.isph`

ISPH: Incompressible SPH via pressure-projection (Cummins & Rudman 1999).

`pinneapple_simulation/numerical_solvers/isph.py` · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ISPHSolver	classe	(h=0.04, mass=1.0, rho0=1000.0, mu=0.02, gravity=None, boundary_lo=None, boundary_hi=None…)	Incompressible SPH (ISPH) using prediction-correction pressure projection.

pinneapple_simulation.numerical_solvers.jax_cfd_solver

JAX-CFD adapter — real differentiable Navier-Stokes via Google's jax-cfd.

pinneapple_simulation/numerical_solvers/jax_cfd_solver.py · 358 linhas · testes: 9

nome	tipo	assinatura	o que faz
JAXCFDSolver	classe	(nx=64, ny=64, viscosity=0.01, density=1.0, domain=None, max_velocity=1.0, cfl_safety=0.5…)	2-D incompressible Navier-Stokes solver backed by jax_cfd.base.
jax_cfd_available	função	() -> bool	Return True if both jax and jax_cfd can be imported.
taylor_green_velocity_fns	função	() -> Tuple[Callable, Callable]	2-D Taylor-Green vortex velocity functions u(x,y), v(x,y).
shear_layer_velocity_fns	função	(sigma=15.0, width=0.05) -> Tuple[Callable, Callable]	2-D double shear-layer velocity functions u(x,y), v(x,y).

pinneapple_simulation.numerical_solvers.kuramoto_sivashinsky

Kuramoto-Sivashinsky in one dimension, u_t + (u^2 / 2)_x + u_xx + u_xxxx = 0 on a periodic interval, with the ETDRK4 scheme of :mod:.etdrk4 (Kassam & Trefethen 2005 set-up, 2/3 dealiasing).

pinneapple_simulation/numerical_solvers/kuramoto_sivashinsky.py · 60 linhas · testes: 7

nome	tipo	assinatura	o que faz
KuramotoSivashinsky1D	classe	(n, length, dt)	

pinneapple_simulation.numerical_solvers.lbm

Lattice Boltzmann Method — D2Q9 BGK + D3Q19 BGK.

pinneapple_simulation/numerical_solvers/lbm.py · 739 linhas · testes: 14

nome	tipo	assinatura	o que faz
LBMSolver	classe	(nx=128, ny=64, Re=100.0, u_in=0.05, rho_out=1.0, obstacle_mask=None, Cs=0.0)	D2Q9 Lattice Boltzmann solver.
LBMSolver3D	classe	(nx=32, ny=32, nz=32, Re=100.0, u_in=0.05, obstacle_mask=None, Cs=0.0)	D3Q19 Lattice Boltzmann solver.
cylinder_mask	função	(nx, ny, cx, cy, r) -> torch.Tensor	Create a circular obstacle mask (True = solid). cx, cy, r in lattice units.
rectangle_mask	função	(nx, ny, x0, x1, y0, y1) -> torch.Tensor	Rectangular obstacle mask.

nome	tipo	assinatura	o que faz
airfoil_naca_mask	função	(nx, ny, chord, aoa_deg=0.0, naca='0012') -> torch.Tensor	Approximate NACA 4-digit airfoil mask centred in the domain. chord: chord length in lattice units.
naca4_mask	função	(nx, ny, naca='4412', chord=80.0, aoa_deg=0.0, x_le=None, y_le=None, n_pts=400) -> torch.…)	NACA 4-digit airfoil mask with camber (True = solid), aerodynamic convention.
airfoil_naca_mask_3d	função	(nx, ny, nz, chord, aoa_deg=0.0, naca='0012', z0=0, z1=None) -> torch.Tensor	3-D obstacle mask: the 2-D NACA airfoil cross-section (airfoil_naca_mask) extruded uniformly along the z (span) axis from z0 to z1 (exclusive).

pinneapple_simulation.numerical_solvers.lbm_tunnel

3-D lattice Boltzmann wind tunnel in PyTorch, with the memory and boundary techniques FluidX3D made popular.

pinneapple_simulation/numerical_solvers/lbm_tunnel.py · 479 linhas · testes: 9

nome	tipo	assinatura	o que faz
Units	classe	(m, s, kg)	Conversion between SI and lattice units, fixed by a length, a velocity and a density given in both systems (lattice length in cells, lattice velocity in cells/step, lattice density usually 1).
WindTunnelLBM	classe	(shape, nu, u_in=(0.0, 0.0, 0.0), velocity_set_name='D3Q19', collision='srt', smagorinsky…)	3-D lattice Boltzmann flow in a box (lattice units: cell = 1, step = 1, rho0 = 1).
velocity_set	função	(name='D3Q19')	(c [q,3] int, w [q], opposite index [q]) for D3Q19 or D3Q27.
sphere	função	(shape, center, radius) -> torch.Tensor	
box	função	(shape, lo, hi) -> torch.Tensor	
voxelize_stl	função	(mesh, shape, center, size, axis=0) -> torch.Tensor	Solid mask of a closed surface (path or trimesh.Trimesh), scaled so its extent along axis is size cells and centred at center (cells). Inside test by ray parity (trimesh).
write_vtk	função	(path, data, origin=(0.0, 0.0, 0.0), spacing=1.0) -> None	Legacy VTK, BINARY big-endian STRUCTURED_POINTS. data is (nx, ny, nz) or (3, nx, ny, nz).
read_vtk	função	(path) -> dict	Legacy VTK structured points (binary big-endian or ASCII), as written by FluidX3D's VTK export and by :func:write_vtk. Returns {"data": (nx,ny,nz) or (k,nx,ny,nz), "origin", "spacing", "dimensions", "name"}.
drag_sphere_schiller_naumann	função	(re) -> float	Drag coefficient of a sphere, Schiller & Naumann (1933), valid up to Re ~ 800.
stokes_sphere_field	função	(shape, center, radius, u_inf)	Stokes flow past a fixed sphere (uniform stream u_inf), velocity (3, nx, ny, nz); zero inside.

pinneapple_simulation.numerical_solvers.meshfree

Meshfree solvers: RBF collocation and Moving Least Squares (MLS).

pinneapple_simulation/numerical_solvers/meshfree.py · 377 linhas · testes: 8 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
RBFCollocationSolver	classe	(rbf='gaussian', eps=3.0, coeff=1.0, reg=1e-08)	Meshfree RBF collocation solver (Kansa method).
MLSApproximation	classe	(degree=1, support_radius=0.2, weight='quartic')	Meshfree Moving Least Squares approximation.

pinneapple_simulation.numerical_solvers.nbody_rebound

REBOUND N-body gravitational integrator.

pinneapple_simulation/numerical_solvers/nbody_rebound.py · 270 linhas · testes: 4

nome	tipo	assinatura	o que faz
NBodySolver	classe	(bodies, integrator='ias15', G=1.0, dt=None, move_to_com=True)	Thin, real wrapper around rebound.Simulation.

pinneapple_simulation.numerical_solvers.neighbors

Neighbor search utilities (grid hashing) for particle methods.

pinneapple_simulation/numerical_solvers/neighbors.py · 113 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
NeighborList	classe	(j, indptr)	Compressed neighbor list representation.
build_neighbor_list	função	(pos, *, h, max_neighbors=64) -> NeighborList	Build neighbor list for 2D/3D particles using uniform grid.

pinneapple_simulation.numerical_solvers.nonlinear_beam_fem

Spectral-element FEM for a geometrically nonlinear (Von Karman) transient Euler-Bernoulli beam, time-integrated with the Newmark-beta method and a full Newton-Raphson iteration at every timestep to resolve the nonlinear residual.

pinneapple_simulation/numerical_solvers/nonlinear_beam_fem.py · 843 linhas · testes: 2

nome	tipo	assinatura	o que faz
NonlinearBeamNewmarkFEM	classe	(num_elements, nodes_per_element, length, area, moment_of_inertia, E, density, damping_co…)	Spectral-element / Newmark-beta engine for the transient geometrically nonlinear (Von Karman) Euler-Bernoulli beam described in the module docstring. Construct once per problem (mesh + BCs are fixed at construction), then call run() to march through time, or natural_frequencies_hz() for the …
NonlinearBeamFEMSolver	classe	(num_elements=50, nodes_per_element=NODES_PER_ELEMENT)	Thin SolverBase/registry wrapper. solve_nonlinear_beam_transient and NonlinearBeamNewmarkFEM are the primary entry points and can be used directly without this wrapper.

nome	tipo	assinatura	o que faz
gauss_lobatto_legendre_points	função	(n_points) -> np.ndarray	Gauss-Lobatto-Legendre points on [-1, 1] for n_points nodes (endpoints included, interior points at the roots of P'_{n-1}).
generate_1d_mesh	função	(num_elements, nodes_per_element, dofpn, length)	Uniform 1D mesh of num_elements elements, each with nodes_per_element nodes placed at Gauss-Lobatto-Legendre points (so nodes_per_element=2 is a plain linear mesh; higher values place spectral interior nodes).
gauss_legendre_quadrature	função	(n_points)	Gauss-Legendre quadrature points and weights on [-1, 1].
lagrange_shape_functions_1d	função	(nodes_per_element, n_quad_points)	Lagrange shape functions and their first derivative (built on GLL nodes), evaluated at n_quad_points Gauss-Legendre quadrature points.
hermite_shape_functions_1d	função	(n_quad_points, h)	Cubic Hermite shape functions (value + slope DOF at each of the two element endpoints) and their first/second derivatives, evaluated at n_quad_points Gauss-Legendre quadrature points on the element of length h. Fixed at two endpoints regardless of mesh order -- see NODES_PER_ELEMENT.
neumann_boundary_time_series	função	(num_elements, nodes_per_element, dofpn, conn, boundary_n, timesteps, delta_t, f_drive)	Pre-evaluate the Neumann ("N") boundary loads at every timestep, scattered onto the (element, local-dof) layout used during assembly. A load value is const + sin_amp*sin(2*pi*f_drive*t) + cos_amp*cos(2*pi*f_drive*t).
dirichlet_boundary_values	função	(num_elements, nodes_per_element, dofpn, conn, boundary_d)	Pre-scatter the Dirichlet ("D") boundary values onto the (element, local-dof) layout; np.nan marks an unconstrained (element, local-dof).
apply_neumann_bc	função	(i, t, load_vec_elem, boundary_n_series, ratio)	Add the pre-evaluated Neumann load (scaled by the current load-step ratio) into an element's residual/load vector.
apply_dirichlet_bc	função	(i, newton_iter, stiff_mat_elem, load_vec_elem, boundary_d_values)	Enforce Dirichlet constraints on one element's tangent matrix and residual by row/column elimination (constrained DOFs decouple from the rest of the system; their prescribed value is only injected on the first Newton iteration of a load step, since later iterations solve for the *correction* to an …
cantilever_frequency_roots	função	(n) -> np.ndarray	First n positive roots of cos(x)cosh(x) = -1, the transcendental frequency equation for a clamped-free (cantilever) Euler-Bernoulli beam -- solved by bracketed root-finding since no closed form exists. Useful as an independent closed-form check for natural_frequencies_hz when the caller's …
solve_nonlinear_beam_transient	função	(boundary, L_m, A_m2, I_m4, E_Pa, rho_kg_m3, damping_coeff=0.0, axial_dist_load_N_per_m=0…)	Solve the transient geometrically nonlinear (Von Karman) Euler- Bernoulli beam FEM problem described in the module docstring, for a caller-supplied boundary dict (Dirichlet + Neumann DOF specification -- see module docstring for its shape). newmark_beta/newmark_gamma use the standard …

nome	tipo	assinatura	o que faz
`acceleration_from_displacement`	função	`(w, dt) -> np.ndarray`	Convenience/diagnostic: second time-derivative of a displacement time-history via a proper 3-point second-derivative stencil (reuses `beam_bvp_fdm.second_derivative`, which is exact for a quadratic displacement history and doesn't compound `np.gradient`'s first-order edge truncation the way ...

`pinneapple_simulation.numerical_solvers.ns3d_spectral`

Pseudo-spectral solver for the 3-D incompressible Navier-Stokes equations on the periodic box [0, 2*pi)^3 (rotational form, 2/3-rule dealiasing, classical RK4, exact integrating factor for viscosity).

`pinneapple_simulation/numerical_solvers/ns3d_spectral.py` · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpectralNS3D`	classe	`(N, nu, dt)`	
`taylor_green_ic`	função	`(N)`	

`pinneapple_simulation.numerical_solvers.openfoam_bridge`

OpenFOAM bridge for pinneapple.

`pinneapple_simulation/numerical_solvers/openfoam_bridge.py` · 751 linhas · testes: 151

nome	tipo	assinatura	o que faz
`OpenFOAMBridge`	classe	`(solver='simpleFoam', n_cores=1, n_iterations=500, openfoam_bin=None) -> None`	Bridge that generates OpenFOAM cases, runs the solver, and extracts fields.
`generate_case`	função	`(problem_spec, case_dir, mesh_cfg=None) -> Path`	Generate an OpenFOAM case directory from *problem_spec*.
`run_openfoam`	função	`(case_dir, solver='simpleFoam', n_cores=1, n_iter=500) -> 'subprocess.CompletedProcess[st…)`	Run an OpenFOAM solver inside *case_dir*.
`extract_fields`	função	`(case_dir, time='latestTime', field_names=None) -> Dict[str, 'Any']`	Extract field arrays from a finished OpenFOAM run.
`openfoam_to_dataset`	função	`(case_dir, field_names=None, time='latestTime') -> Dict[str, 'Any']`	High-level helper: extract fields and return as a flat dataset dict.

`pinneapple_simulation.numerical_solvers.operator_benchmarks`

Reference solvers and input distributions of the standard operator-learning benchmarks.

`pinneapple_simulation/numerical_solvers/operator_benchmarks.py` · 277 linhas · testes: 132

nome	tipo	assinatura	o que faz
`FourierInterpolant`	classe	`(values, *, max_modes=None, dtype=None)`	Evaluate a periodic grid field (period 1 in every axis, nodes `j / n`) at arbitrary points, with autograd.

nome	tipo	assinatura	o que faz
periodic_grf	função	(n_samples, shape, *, alpha, tau, sigma=None, seed=None) -> np.ndarray	Zero-mean Gaussian random fields on the periodic unit box with covariance sigma^2 (-lap + tau^2)^-alpha.
threshold_coefficient	função	(field, high=12.0, low=3.0) -> np.ndarray	Piecewise-constant coefficient high where field >= 0 and low elsewhere (FNO Darcy inputs).
spectral_downsample	função	(u, factor, axes) -> np.ndarray	Point values on every factor-th node of a periodic grid (exact for band-limited fields).
burgers_1d	função	(u0, nu, t_end, *, dt=0.0001, n_save=1, dealias=True) -> tuple[np.ndarray, np.ndarray]	Viscous Burgers on the periodic interval [0, 1).
li_forcing	função	(n) -> np.ndarray	f(x, y) = 0.1 (sin(2 pi (x + y)) + cos(2 pi (x + y))) on the n x n periodic grid (Li et al. 2021).
navier_stokes_2d_vorticity	função	(w0, nu, t_end, *, forcing=None, dt=0.001, n_save=1, dealias=True) -> tuple[np.ndarray, n…)	2-D incompressible Navier-Stokes, vorticity form, periodic unit square, grid x_i = i / n.
advection_diffusion_2d	função	(u0, velocity, kappa, dt, n_steps) -> np.ndarray	u_t + c . grad u = kappa lap u on the periodic unit square, exact in time for the resolved modes: each Fourier mode is multiplied by exp((-i k . c - kappa |k|^2) dt) per step. u0: (n, n) or (batch, n, n) (axis 0 is x). Returns (n_steps + 1, ...) states, the first one u0.
darcy_2d	função	(a, f=1.0, *, axis_weights=(1.0, 1.0)) -> np.ndarray	-div(a grad u) = f on the unit square, u = 0 on the boundary; a (and f) at the n x n cell centres ((i + 1/2) h, (j + 1/2) h), h = 1/n. Returns u at the cell centres.
darcy_matrix	função	(a, axis_weights=(1.0, 1.0)) -> sp.csc_matrix	The n^2 x n^2 finite-volume matrix of :func:darcy_2d (A u = f with u raveled in C order). It is symmetric positive definite, and A - min(a) * darcy_matrix(ones) is positive semi-definite (every face coefficient is a harmonic mean, never below min a).

pinneapple_simulation.numerical_solvers.operator_datasets

Operator-learning benchmark datasets: 1-D Burgers, 2-D Darcy, 2-D Navier-Stokes (vorticity).

pinneapple_simulation/numerical_solvers/operator_datasets.py · 309 linhas · testes: 17

nome	tipo	assinatura	o que faz
burgers_inputs	função	(n_samples, n, seed) -> np.ndarray	

nome	tipo	assinatura	o que faz
`fourier_pad`	função	`(u, n_new, axes) -> np.ndarray`	Band-limited (trigonometric) interpolation of periodic grid data onto `n_new` points along each of `axes`; exact for fields whose Fourier content is below the coarse Nyquist mode (the Nyquist mode is dropped).
`solve_burgers`	função	`(u0, dt=None) -> np.ndarray`	
`darcy_inputs`	função	`(n_samples, n, seed, *, tau=None, high=None, low=None) -> np.ndarray`	Coefficients of the Darcy benchmark; `tau` (inverse correlation length), `high` and `low` override the dataset's values to draw shifted inputs (other length scale, other contrast) from the same family.
`block_average`	função	`(x, factor) -> np.ndarray`	Cell averages on a grid `factor` times coarser over the last two axes (`(..., n, n)`, `n % factor == 0`): the coarse-grid version of a cell-averaged field (`u` of the dataset) or of a coefficient.
`solve_darcy`	função	`(a, refine=None) -> np.ndarray`	Solve on the grid refined `refine` times (the piecewise-constant coefficient is represented exactly) and return the cell averages of u on the coefficient's own grid.
`ns_inputs`	função	`(n_samples, n, seed) -> np.ndarray`	
`solve_ns`	função	`(w0, n_solve=None, dt=None) -> np.ndarray`	Solve on `n_solve^2` (inputs given on a coarser grid are Fourier-padded) and return the point values on the stored `n_store^2` grid at t = 0, 1, ..., t_end.
`rel_l2`	função	`(a, b) -> np.ndarray`	Per-sample relative L2 error of `a` against reference `b` (all axes but the first).
`write_npz`	função	`(path, arrays) -> None`	Deflated npz with fixed zip timestamps (the SHA-256 depends only on the data).
`darcy_set`	função	`(n_samples, seed, *, n=None, tau=None, high=None, low=None, workers=1) -> tuple[np.ndarra…)`	`(a, u)` pairs of the Darcy benchmark drawn and solved exactly as the zoo dataset `darcy_2d_li2021` (same solver, refinement and source), optionally from a shifted input distribution: `tau` (correlation length ~ 1 / tau; the dataset uses 3), `high` / `low` (the two coefficient values; 12 ...
`generate`	função	`(out, workers=1, only=None) -> None`	Generate the three datasets (or `only` one) into `out/<name>/` (resumable for Navier-Stokes).
`convergence`	função	`(out, workers=1, n_samples=4, only=None) -> dict`	Resolution and time-step study on held-aside inputs (seed + 7): each setting is compared with a finer reference at the stored nodes, all saved times; the dataset setting must be converged to well below the error level operator-learning methods reach on these problems (FNO: 3e-3 to 1e-2 rel. L2). ...

`pinneapple_simulation.numerical_solvers.particles`

Lagrangian particles in a liquid: soft-sphere DEM contacts, drag, buoyancy and turbulent dispersion, in a prescribed carrier flow (one-way coupling). Fast enough for tens of thousands of particles on a laptop, and meant for process pictures and trends (suspension, mixing, settling), not for ...

pinneapple_simulation/numerical_solvers/particles.py · 284 linhas · testes: 20

nome	tipo	assinatura	o que faz
Liquid	classe	(rho, mu)	
StirredTank	classe	(R, H, D_imp, C, swirl, loop, turbulence, T_L, blades, pitch_deg, down_pumping)	Flat-bottom cylindrical tank (axis z, bottom at z = 0, liquid up to H) with a centred impeller of diameter D_imp at clearance C. swirl and loop: swirl and circulation speeds as fractions of the tip speed; turbulence: velocity fluctuation as a fraction of the tip speed; T_L: ...
DEM	classe	(x, d, rho_p, liq=WATER, e=0.4, t_c=None, dt=0.0005, g=9.81, seed=0)	Equal spheres (diameter d) with linear spring-dashpot normal contacts (restitution e, collision time t_c), drag towards the liquid velocity, buoyancy-reduced gravity, and contacts with the tank (wall, bottom, surface). Verlet neighbour lists with a skin.
drag_factor	função	(Re) -> np.ndarray	Schiller-Naumann correction to Stokes drag (Re < 1000).
terminal_velocity	função	(d, rho_p, liq=WATER, g=9.81) -> float	Settling velocity of an isolated sphere with Schiller-Naumann drag (fixed point).
packed_bed	função	(tank, n, d, seed=0) -> np.ndarray	Initial positions: a loose cubic lattice filling the tank from the bottom (it settles in the first second).
suspend	função	(tank, n=20000, d=0.003, rho_p=1200.0, rpm=lambda t: min(150.0, 6.0 * t), t_end=30.0, dt=…)	Suspension of an initially settled bed as the impeller speeds up. Returns frames (positions, speed), times, rpm, the fraction of particles in the top third, the fraction off the bottom and the largest contact overlap.
zwietering_njs	função	(tank, d, rho_p, mass_ratio, liq=WATER, S=6.0, g=9.81) -> float	Just-suspended impeller speed (rpm) of Zwietering (1958): N_js = S nu^0.1 (g drho/rho_L)^0.45 X^0.13 d^0.2 / D^0.85, X in mass percent of solids to liquid. S depends on the impeller and the geometry (about 4-8 for pitched-blade turbines).

pinneapple_simulation.numerical_solvers.pipe_network

Steady flow in a looped pipe network (water distribution) by the global gradient algorithm.

pinneapple_simulation/numerical_solvers/pipe_network.py · 184 linhas · testes: 27

nome	tipo	assinatura	o que faz
Network	classe	(junctions, reservoirs, pipes, headloss, coords)	Nodes by name: elevation (m), demand (m3/s, positive = drawn off) for junctions, head (m) for fixed-head nodes. Pipes: (name, from, to, length m, diameter m, roughness) with roughness = Hazen-Williams C or Darcy-Weisbach absolute roughness in m, per headloss ("HW" or "DW").
solve_network	função	(net, tol=1e-10, max_iter=100) -> dict	Heads, pressures (head - elevation), flows, velocities and head losses; iterations, the continuity residual per junction and the energy residual per pipe (both checks of the converged solution).

nome	tipo	assinatura	o que faz
`two_loop_network`	função	`(diameters_in=(18, 10, 16, 4, 16, 10, 10, 1), headloss='HW', roughness=None) -> Network`	The two-loop network of Alperovits & Shamir (1977), "Design of optimal water distribution systems", Water Resources Research 13(6): one reservoir at 210 m, six junctions, eight 1000 m pipes, Hazen-Williams C = 130. Default diameters (inches): the least-cost design reported by Savic & Walters (1997) …
`optimize_diameters`	função	`(net, sizes_m, unit_cost, p_min, *, pop=100, generations=400, seed=0, penalty=1000000.0, …)`	Least-cost discrete diameters with every junction pressure >= `p_min` (m): a genetic algorithm (integer genes, tournament selection, uniform crossover, creep mutation, elitism) with a penalty per metre of pressure deficit; every candidate is evaluated with :func:`solve_network`. Returns the best …

pinneapple_simulation.numerical_solvers.plane_stress_fem

Linear plane-stress finite elements on any triangle mesh (constant-strain triangles), for parts whose geometry comes from elsewhere (a B-rep face, a CAD tessellation).

`pinneapple_simulation/numerical_solvers/plane_stress_fem.py` · 137 linhas · testes: 39

nome	tipo	assinatura	o que faz
`PlaneStressSolution`	classe	`(V, F, u, stress, area, reaction)`	
`heywood_kt_net`	função	`(d_over_w) -> float`	Net-section stress concentration of a central hole in a strip under tension, `2 + (1 - d/W)^3`.
`boundary_edges`	função	`(F) -> np.ndarray`	Edges `(m, 2)` that belong to one triangle only.
`solve_plane_stress`	função	`(V, F, *, E, nu, thickness, fixed, traction=(), body_force=(0.0, 0.0)) -> PlaneStressSolu…)`	`V (n, 2)` nodes [m], `F (m, 3)` triangles; `fixed = {"x": nodes, "y": nodes}` (zero displacement); `traction`: `[(edges (k, 2), (tx, ty) [Pa]), ...]` on boundary edges; `body_force` [N/m^3].
`patch_test`	função	`(V, F, *, E=1.0, nu=0.3) -> float`	Impose the displacement of a uniform stress state on the boundary nodes and return the largest relative deviation of the element stresses from it (exact elements give round-off).

pinneapple_simulation.numerical_solvers.problem_runner

Generate PINN training datasets from ProblemSpec definitions.

`pinneapple_simulation/numerical_solvers/problem_runner.py` · 570 linhas · testes: 22

nome	tipo	assinatura	o que faz
`generate_pinn_dataset`	função	`(problem_spec, *, n_col=None, n_bc=None, n_ic=None, n_data=0, seed=0, solver_cfg_override…)`	Generate a PINN training dataset from a ProblemSpec.

pinneapple_simulation.numerical_solvers.reaction_diffusion

One-dimensional reaction-diffusion on a periodic interval, `u_t = D u_xx + R(u)`, with the ETDRK4 scheme of :mod:`.etdrk4` (stiff diffusion exact, reaction explicit, 2/3 dealiasing).

pinneapple_simulation/numerical_solvers/reaction_diffusion.py · 65 linhas · testes: 4

nome	tipo	assinatura	o que faz
`ReactionDiffusion1D`	classe	`(reaction, diffusion, n, length, dt)`	

pinneapple_simulation.numerical_solvers.registry

Solver registry.

pinneapple_simulation/numerical_solvers/registry.py · 137 linhas · testes: 206

nome	tipo	assinatura	o que faz
`SolverSpec`	classe	`(name, family, cls, description, tags)`	
`SolverRegistry`	classe	`(_REG)`	
`SolverCatalog`	classe	`(registry)`	Convenience wrapper (dict-like catalog) around SolverRegistry.
`register_all`	função	`() -> Dict[str, str]`	Import solver modules so their @register decorators execute.

pinneapple_simulation.numerical_solvers.shallow_water_fv

2D shallow-water finite-volume solver for free-surface water: walls, gates, virtual sensors, live health checks, CSV export and comparison with experiments.

pinneapple_simulation/numerical_solvers/shallow_water_fv.py · 352 linhas · testes: 4

nome	tipo	assinatura	o que faz
`Sensor`	classe	`(name, kind, x, y, z, faces, values)`	
`ShallowWater2D`	classe	`(nx, ny, dx, dy=None, *, g=G, rho=RHO, cfl=0.45, second_order=True)`	Water on an `nx x ny` grid of cell size `dx x dy` (domain boundary: solid walls).
`ritter`	função	`(x, t, h0, x0=0.0, g=G)`	Dam break on a dry bed (Ritter 1892): depth and velocity.
`stoker`	função	`(x, t, h0, h1, x0=0.0, g=G)`	Dam break on a wet bed h1 < h0 (Stoker 1957): rarefaction + constant state + bore.

pinneapple_simulation.numerical_solvers.solid_fem

3-D linear elasticity with 8-node hexahedra and incompatible modes (Wilson-Taylor; the element Abaqus calls C3D8I), so slender parts in bending do not lock. Structured box meshes, clamps, supports, nodal and pressure loads, nodal stresses (extrapolated from the Gauss points and averaged) and von …

pinneapple_simulation/numerical_solvers/solid_fem.py · 415 linhas · testes: 47

nome	tipo	assinatura	o que faz
HexMesh	classe	(nodes, elements, shape, size)	
FEMResult	classe	(mesh, u, stress, reactions, info)	
SolidFEM	classe	(mesh, E, nu, incompatible, _fixed, _f)	
box_mesh	função	(L, W, H, nx, ny, nz, origin=(0.0, 0.0, 0.0)) -> HexMesh	Structured hexahedral mesh of a box [0, L] x [-W/2, W/2] x [-H/2, H/2] (shifted by origin).
elasticity_matrix	função	(E, nu) -> np.ndarray	
von_mises	função	(s) -> np.ndarray	von Mises stress of Voigt stresses (..., 6).
fea_figure	função	(res, field='von_mises', scale=None, bands=12, title='', unit='MPa', factor=1e-06, view=(…)	The deformed boundary mesh coloured by field in discrete bands with black element edges and a post-processor legend (S, Mises), the undeformed outline in grey. scale: deformation scale factor (default: the largest displacement drawn as 8 % of the part's length). Any nodal values (a ...

pinneapple_simulation.numerical_solvers.spectral

Spectral solvers (MVP).

pinneapple_simulation/numerical_solvers/spectral.py · 49 linhas · testes: 5

nome	tipo	assinatura	o que faz
SpectralSolver	classe	()	
poisson_periodic_fft	função	(f, *, Lx=1.0, Ly=1.0, eps=1e-12) -> torch.Tensor	Solve ∇²u = f with periodic BC on 2D domain via FFT.

pinneapple_simulation.numerical_solvers.sph

SPH: Weakly-compressible Smoothed Particle Hydrodynamics (WCSPH) MVP.

pinneapple_simulation/numerical_solvers/sph.py · 167 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SPHState	classe	(pos, vel, rho, p)	
SPHSolver	classe	(h=0.04, mass=1.0, rho0=1000.0, c0=20.0, mu=0.02, gravity=None, boundary_lo=None, boundar…)	

pinneapple_simulation.numerical_solvers.sph_boundaries

Boundary handling helpers for SPH (MVP).

`pinneapple_simulation/numerical_solvers/sph_boundaries.py` · 88 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BoxBoundary`	classe	`(lo, hi)`	Axis-aligned box boundary [min,max] in each dimension.
`wall_repulsion_force`	função	`(pos, vel, boundary, *, k=50.0, d0=0.02) -> torch.Tensor`	
`reflect_box`	função	`(pos, vel, lo, hi, *, restitution=0.0) -> Tuple[torch.Tensor, torch.Tensor]`	Hard boundary reflection for an axis-aligned box.

`pinneapple_simulation.numerical_solvers.ssa`

SSA: Singular Spectrum Analysis for 1D time series.

`pinneapple_simulation/numerical_solvers/ssa.py` · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SSASolver`	classe	`(L=64, r=10)`	
`ssa_decompose`	função	`(x, L=64, r=10) -> Tuple[np.ndarray, np.ndarray, np.ndarray]`	Return reconstructed components (r,T), singular values, and U.

`pinneapple_simulation.numerical_solvers.sst`

SST: Synchrosqueezed transform (MVP).

`pinneapple_simulation/numerical_solvers/sst.py` · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SSTSolver`	classe	`(fs=1.0, nperseg=128, noverlap=None, nfft=None)`	
`sst_stft_1d`	função	`(x, *, fs=1.0, nperseg=128, noverlap=None, nfft=None, eps=1e-08) -> Tuple[np.ndarray, np.…)`	Return (SST_like, freqs, times).

`pinneapple_simulation.numerical_solvers.stable_fluids`

Stable Fluids: unconditionally stable 2-D incompressible flow on a periodic grid.

`pinneapple_simulation/numerical_solvers/stable_fluids.py` · 198 linhas · testes: 17

nome	tipo	assinatura	o que faz
`StableFluidsState`	classe	`(velocity, scalars, t)`	
`StableFluids2D`	classe	`(nx, ny, *, length=(2 * np.pi, 2 * np.pi), viscosity=0.0, confinement=0.0, scalar_dissipa…)`	Periodic box `[0, Lx) x [0, Ly)` with `nx x ny` cell-centred samples.
`stable_time_step`	função	`(max_velocity, dx, *, courant=0.5, viscosity=0.0, ndim=2, implicit_diffusion=True) -> flo…)`	Advective CFL step `courant * dx / max_velocity`, capped by the explicit-diffusion limit `dx^2 / (2 ndim nu)` unless diffusion is implicit (as in :class:`StableFluids2D`).

pinneapple_simulation.numerical_solvers.stiff_kinetics_ode

Stiff-ODE convenience wrapper for coupled reaction-network / chemical- or biological-kinetics systems: dy_i/dt = rate_fn(t, y)[i], y_i(t0) = y0[i].

`pinneapple_simulation/numerical_solvers/stiff_kinetics_ode.py` · 100 linhas · testes: 7

nome	tipo	assinatura	o que faz
`solve_stiff_reaction_network`	função	`(rate_fn, y0, t_span, t_eval=None, jac_fn=None, method='Radau', rtol=1e-09, atol=1e-12, c…)`	Integrate dy/dt = rate_fn(t, y), y(t0) = y0, over t_span = (t0, tf) with a stiff-appropriate implicit method.

pinneapple_simulation.numerical_solvers.stl

STL: Seasonal-Trend decomposition using Loess.

`pinneapple_simulation/numerical_solvers/stl.py` · 40 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`STLSolver`	classe	`(period=24, robust=True)`	

pinneapple_simulation.numerical_solvers.symbolic_fd

Finite differences for any parsed equation system on a box: steady (Newton) or method of lines (BDF).

`pinneapple_simulation/numerical_solvers/symbolic_fd.py` · 498 linhas · testes: 17

nome	tipo	assinatura	o que faz
`FDError`	classe	`()`	The system cannot be solved by this finite-difference scheme (and why).
`FDResult`	classe	`(axes, fields, t, info)`	
`FDOperator`	classe	`()`	A parsed system compiled on a grid: the right-hand sides, the face conditions and the initial state. Arrays may carry leading axes (an ensemble); coefficients named in `keep` stay symbolic and are read from the environment (`extra`), so an ensemble of coefficient values can be stepped at once.
`compile_system`	função	`(system, bounds, *, n=64, coefficients=None, keep=()) -> FDOperator`	Compile `system` for finite differences (see :func:`solve_fd`, which uses it).
`initial_state`	função	`(op, t0) -> dict[tuple[str, bool], np.ndarray]`	Initial values (and `f_t` for second order) of every field on the grid, faces set by their conditions.
`solve_fd`	função	`(system, bounds, *, n=64, n_save=41, coefficients=None, relax=20000.0, rtol=1e-06, atol=1…)`	Solve `system` on the box `bounds` (one interval per coordinate, `t` included when time-dependent) with `n` nodes per spatial axis. Returns :class:`FDResult`; FDError when the system is outside the scheme.

pinneapple_simulation.numerical_solvers.thermal_channel_2d

Transient 2-D heated-channel CFD: incompressible Navier-Stokes + energy, with time-varying actuators (inlet velocity U_in(t), heater power Q_h(t)).

pinneapple_simulation/numerical_solvers/thermal_channel_2d.py · 231 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ChannelConfig	classe	(nx, ny, L, Re, Pr, heater, dt, gamma_safety, record_every)	
ThermalChannel2D	classe	(cfg=None)	

pinneapple_simulation.numerical_solvers.thermal_channel_datasets

Heated-channel CFD dataset (zoo thermal_channel_cfd): verification and actuated trajectories.

pinneapple_simulation/numerical_solvers/thermal_channel_datasets.py · 181 linhas · testes: 2

nome	tipo	assinatura	o que faz
verify	função	() -> dict	Poiseuille (3 grids), fully developed Nusselt number (2 grids) and the three-grid study of T_out, T_max, dp.
make_actuation	função	(family, seed)	(U spec, Q spec) of one trajectory: random_steps_ramps (training family), sinusoid_chirp and edge_steps (unseen families).
eval_act	função	(spec, t, which) -> float	
simulate	função	(args) -> dict	One trajectory: warm-up, then the recorded horizon (fields every REC).
generate	função	(out, workers=1, name='thermal_channel_cfd') -> dict	Run :data:PLAN and write out/<name>/<split>[-NN].npz and meta.json.

pinneapple_simulation.numerical_solvers.tvd_advection

Second-order TVD (Total Variation Diminishing) upwind scheme for 1D scalar advection, with a van Leer flux limiter.

pinneapple_simulation/numerical_solvers/tvd_advection.py · 69 linhas · testes: 5

nome	tipo	assinatura	o que faz
van_leer_limiter	função	(r) -> np.ndarray	van Leer TVD flux limiter: Phi(r) = (r + |r|) / (1 + |r|). r is the ratio of consecutive solution gradients (upstream/downstream); Phi(r)=0 for r<=0 (a local extremum -- falls back to pure upwind) and Phi(r) -> 1 as r -> infinity (smooth region -- full 2nd-order correction).
tvd_advection_rhs	função	(f, v, dz) -> np.ndarray	Second-order TVD-upwind estimate of df/dt = -v*df/dz on a uniform grid of spacing dz, given the field f (shape (N,)) and a constant velocity v (v>0: flow toward increasing index; v<0: toward decreasing index). Domain-boundary interface fluxes use plain first-order upwind (no downstream neighbor …

pinneapple_simulation.numerical_solvers.vmd

VMD: Variational Mode Decomposition.

pinneapple_simulation/numerical_solvers/vmd.py · 143 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
VMDSolver	classe	(K=4, alpha=2000.0, tau=0.0, DC=False, init='uniform', tol=1e-07, max_iter=500)	
vmd_1d	função	(x, *, K=4, alpha=2000.0, tau=0.0, DC=False, init='uniform', tol=1e-07, max_iter=500) -> …)	Returns (modes[K,T], center_freqs[K]).

pinneapple_simulation.numerical_solvers.water_hammer_moc

Water hammer in a reservoir - pipe - valve system by the method of characteristics (MOC).

pinneapple_simulation/numerical_solvers/water_hammer_moc.py · 130 linhas · testes: 17

nome	tipo	assinatura	o que faz
wave_speed	função	(K, rho, D, e, E, c1=1.0) -> float	Pressure-wave speed in a liquid-filled elastic pipe (Korteweg; Wylie & Streeter eq. 2.29): a = sqrt((K/rho) / (1 + c1 K D / (E e))). c1 = 1 - nu^2 for a pipe anchored against axial movement.
joukowsky_head	função	(a, dv) -> float	Head rise of an instantaneous velocity change: dH = a dV / g (Joukowsky 1898).
michaud_head	função	(a, dv, length, t_close) -> float	Michaud's (1878) estimate for a closure slower than 2L/a: dH = 2 L dV / (g t_c); equals Joukowsky for t_c <= 2L/a.
valve_closure_moc	função	(*, length, diameter, wave_speed, friction_factor, q0, h_reservoir, h_downstream=0.0, t_c…)	Transient heads and flows after the valve at the end of the pipe starts closing at t = 0.

pinneapple_simulation.numerical_solvers.wavelet

Wavelet-based decomposition/denoising features.

pinneapple_simulation/numerical_solvers/wavelet.py · 63 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
WaveletSolver	classe	(wavelet='db4', level=4, mode='periodization')	
wavedec_features	função	(x, wavelet='db4', level=4, mode='periodization') -> Tuple[np.ndarray, Dict[str, Any]]	Return concatenated detail coefficients as features.

pinneapple_simulation.numerical_solvers.xtfc_general

General X-TFC on a box: any number of dimensions and fields, value and derivative constraints on faces, linear or non-linear residuals.

pinneapple_simulation/numerical_solvers/xtfc_general.py · 335 linhas · testes: 10

nome	tipo	assinatura	o que faz
`Constraint`	classe	`(dim, at, order)`	`d^order u / dx_dim^order` at `x_dim = at` is prescribed (by the problem's constraint function).
`XTFCProblem`	classe	`(bounds, fields, constraint_fn, residual, derivatives, linear)`	A differential problem on `bounds` for X-TFC.
`XTFCResult`	classe	`(beta, iterations, converged, step_norms, lstsq_seconds, total_seconds, n_neurons, _probl…)`	
`activation_derivative`	função	`(name, z, n) -> np.ndarray`	`d^n act / dz^n` at `z` (`name` in `tanh`, `logistic`).
`switching_functions`	função	`(constraints) -> np.ndarray`	Coefficients (monomial basis, row k = s_k) of the switching functions of constraints on one dimension.
`solve_xtfc`	função	`(problem, points, *, n_neurons, activation='tanh', weight_range=(-1.0, 1.0), seed=0, max_…)`	Solve `problem` on the training `points` (N, ndim) by X-TFC.
`grid`	função	`(bounds, n, kind='uniform') -> np.ndarray`	Tensor grid (N, ndim) of `n` points per dimension, `uniform` or `cgl` (Chebyshev-Gauss-Lobatto), boundary included.

`pinneapple_simulation.numerical_solvers.xtfc_ivp`

X-TFC (eXtreme Theory of Functional Connections) solver for systems of ODEs posed as initial-value problems:

`pinneapple_simulation/numerical_solvers/xtfc_ivp.py` · 438 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`XTFCIVPSolver`	classe	`(n_basis=50, n_collocation=100, activation='tanh', max_iter=50, tol=1e-10, seed=0, colloc…)`	Thin `SolverBase`/registry wrapper around `solve_xtfc_ode` for callers that want the uniform `SolverOutput` interface. The functional API (`solve_xtfc_ode`, `xtfc_predict`) is the primary entry point and can be used directly without this wrapper.
`chebyshev_gauss_lobatto`	função	`(n, lo=-1.0, hi=1.0) -> Array`	N Chebyshev-Gauss-Lobatto points on [lo, hi], ascending order. Standard collocation choice for spectral/TFC methods -- clusters points quadratically near both endpoints, which both improves the conditioning of the least-squares system (vs. uniform spacing, which is offered as a fallback via …
`xtfc_predict`	função	`(t_query, w, b, beta, y0, activation, t0, tf) -> Array`	Evaluate the trained TFC constrained expression at arbitrary query times -- a pure function of the persisted (w, b, beta, y0, activation, t0, tf), so a solved model can be re-evaluated later from just those small arrays without re-running solve_xtfc_ode.
`solve_xtfc_ode`	função	`(rhs_fn, y0, t_span, n_basis=50, n_collocation=100, activation='tanh', max_iter=50, tol=1…)`	Solve dy_i/dt = rhs_fn(t, y)[:, i], y_i(t0) = y0[i], for t in t_span, via X-TFC (see module docstring for the full method).

pinneapple_simulation.numerical_solvers.xtfc_pde

X-TFC (eXtreme Theory of Functional Connections) solver for 1D-space(+time) PDEs posed on a rectangle x in [x0, x1], t in [t0, tf], with one initial condition (at t=t0) and two Dirichlet boundary conditions (at x=x0, x=x1):

`pinneapple_simulation/numerical_solvers/xtfc_pde.py` · 584 linhas · testes: 6

nome	tipo	assinatura	o que faz
`XTFCPDESolver`	classe	`(pde='burgers_1d', n_basis=60, n_collocation_x=25, n_collocation_t=25, activation='tanh',…)`	Thin `SolverBase`/registry wrapper around `solve_xtfc_pde` for callers that want the uniform `SolverOutput` interface. The functional API (`solve_xtfc_pde`, `xtfc_pde_predict`) is the primary entry point.
`xtfc_pde_predict`	função	`(x_query, t_query, wx, wt, b, beta, pde, activation, x_span, t_span) -> Array`	Evaluate the trained 2D TFC constrained expression at arbitrary (paired) query points -- a pure function of the persisted (wx, wt, b, beta, pde, activation, x_span, t_span), mirroring `xtfc_predict` in `xtfc_ivp.py`.
`solve_xtfc_pde`	função	`(pde, x_span=None, t_span=None, n_basis=60, n_collocation_x=25, n_collocation_t=25, activ…)`	Solve `pde` in {"heat_1d", "burgers_1d"} via X-TFC (see module docstring for the full method).

pinneapple_simulation.numerical_solvers.xtfc_subdomain

Subdomain-chained X-TFC for extremely stiff ODE-IVP systems (target stiffness ratio -- max|eigenvalue(Jacobian)| / min|eigenvalue(Jacobian)| -- roughly 1e10 to 1e14), plus a generic "extra scalar parameter" input.

`pinneapple_simulation/numerical_solvers/xtfc_subdomain.py` · 395 linhas · testes: 7

nome	tipo	assinatura	o que faz
`XTFCSubdomainSolver`	classe	`(n_subdomains=12, first_width=None, spacing='log', n_basis=30, n_collocation=40, activati…)`	Thin `SolverBase`/registry wrapper around `solve_xtfc_subdomains`. The functional API (`solve_xtfc_subdomains`, `log_subdomain_boundaries`) is the primary entry point.
`log_subdomain_boundaries`	função	`(t0, tf, n_subdomains, first_width=None) -> Array`	n_subdomains+1 boundaries [t0, ..., tf] with logarithmically GROWING widths -- tiny near t0 (to resolve a fast initial transient spanning many decades), geometrically larger towards tf.
`solve_xtfc_subdomains`	função	`(rhs_fn, y0, t_span, n_subdomains=12, first_width=None, spacing='log', n_basis=30, n_coll…)`	Solve a (possibly extremely stiff) dy_i/dt = rhs_fn(t, y)[:, i], y_i(t0) = y0[i] IVP over log-spaced subdomains of t_span, chaining continuity across subdomain boundaries (see module docstring).

pinneapple_simulation.particle_dynamics

Simulação dinâmica diferenciável em PyTorch puro: corpos rígidos, MPM (MLS-MPM) e SPH; reexporta também os parâmetros de solo/roda de Bekker-Wong.

`pinneapple_simulation/particle_dynamics/__init__.py` · 61 linhas · **sem teste direto**

pinneapple_simulation.particle_dynamics._utils

Utilitário privado que monta o vetor de gravidade (padrão -9,81 no eixo y) usado pelos simuladores de partículas. [sem teste]

`pinneapple_simulation/particle_dynamics/_utils.py` · 25 linhas · **sem teste direto**

pinneapple_simulation.particle_dynamics.geo_mpm

MPM para geomecânica (colapso granular, deslizamentos, rompimento de barragem, sólidos moles): MLS-MPM com APIC em unidades físicas, materiais areia (Drucker-Prager), fluido e elástico, com quadros no formato do GeoTaichi.

`pinneapple_simulation/particle_dynamics/geo_mpm.py` · 355 linhas · testes: 4

nome	tipo	assinatura	o que faz
`Sand`	classe	`(friction_deg, E, nu, rho)`	Material areia: plasticidade de Drucker-Prager sem coesão, com ângulo de atrito, E, nu e densidade.
`Fluid`	classe	`(K, rho, gamma)`	Material fluido fracamente compressível, pressão de J (tipo Tait).
`Elastic`	classe	`(E, nu, rho)`	Material sólido hiperelástico corrotacional fixo.
`GeoMPM`	classe	`(domain, dx, gravity=None, floor='friction', walls='slip', wall_friction=0.5, cfl=0.3, fl…)`	Simulador MLS-MPM 2D/3D em unidades físicas com corpos, paredes (slip, sticky ou atrito), passo estável por CFL e gravação de quadros.
`box`	função	`(lo, hi)`	Corpo retangular (ou caixa): devolve um teste de pertencimento para `add_body`.
`write_particle_frame`	função	`(out, i, fr) -> str`	Grava um quadro de partículas em `.npz` no layout do GeoTaichi.
`read_particle_frames`	função	`(out, kind='MPM', allow_pickle=False) -> list[dict]`	Lê os quadros de uma corrida deste módulo ou do GeoTaichi (MPM ou DEM); `allow_pickle` só para arquivos confiáveis.
`runout`	função	`(position, axis=0, floor_band=None, quantile=0.995) -> float`	Posição da frente de um depósito ao longo de um eixo, por um quantil alto robusto a partículas soltas.

pinneapple_simulation.particle_dynamics.molecular_dynamics

Dinâmica molecular NVE com potencial de Lennard-Jones par a par e integração velocity-Verlet, em PyTorch.

`pinneapple_simulation/particle_dynamics/molecular_dynamics.py` · 274 linhas · testes: 8

nome	tipo	assinatura	o que faz
`MDState`	classe	`(pos, vel)`	Contêiner de posições e velocidades de um sistema de N partículas.
`MolecularDynamicsSimulator`	classe	`(epsilon=1.0, sigma=1.0, mass=1.0, cutoff=None, box_size=None, dim=3) -> None`	Simulador mínimo NVE de Lennard-Jones com velocity-Verlet, raio de corte e caixa periódica opcionais.

nome	tipo	assinatura	o que faz
`lj_potential`	função	`(dist, epsilon, sigma) -> torch.Tensor`	Potencial de par de Lennard-Jones U(r) = 4ε((σ/r)^12 - (σ/r)^6).
`lj_force_magnitude`	função	`(dist, epsilon, sigma) -> torch.Tensor`	Módulo analítico da força radial de Lennard-Jones, F(r) = -dU/dr.

`pinneapple_simulation.particle_dynamics.mpm`

Método do Ponto Material (MLS-MPM) diferenciável para sólidos elásticos, fluidos viscosos e neve com plasticidade de estiramento crítico, com etapas P2G, atualização da grade e G2P.

`pinneapple_simulation/particle_dynamics/mpm.py` · 481 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MPMState`	classe	`(pos, vel=None, F=None, C=None, Jp=None) -> None`	Contêiner do estado das partículas MPM: posição, velocidade, gradiente de deformação F, matriz afim C e Jp.
`MPMSimulator`	classe	`(grid_resolution=64, dt=0.0001, material='elastic', dim=2, E=10000.0, nu=0.2, rho=1.0, gr…)`	Simulador MPM diferenciável (`nn.Module`) com materiais elástico, fluido ou neve, em 2D/3D.

`pinneapple_simulation.particle_dynamics.particles`

Simulação por partículas: SPH fracamente compressível (equação de Tait, viscosidade laminar) sobre uma classe base com busca de vizinhos O(N²); a base levanta NotImplementedError em `forward`, e o DEM citado no docstring não está implementado aqui.

`pinneapple_simulation/particle_dynamics/particles.py` · 334 linhas · testes: 151 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`ParticleSystem`	classe	`(n_particles, dim=2, device='cpu') -> None`	Base de sistemas de partículas com busca de vizinhos por força bruta O(N²); `forward` levanta NotImplementedError.
`SPHParticles`	classe	`(n_particles, smoothing_length, rho0=1000.0, c0=10.0, nu=0.0001, dim=2, gamma=7.0, gravit…)`	SPH fracamente compressível: densidade por soma de kernel, pressão de Tait, viscosidade e gravidade.

`pinneapple_simulation.particle_dynamics.rigid_body`

Dinâmica de corpos rígidos diferenciável em 2D/3D com integração Euler simplética, para aprendizado de robôs e acoplamento com PINNs.

`pinneapple_simulation/particle_dynamics/rigid_body.py` · 306 linhas · testes: 151

nome	tipo	assinatura	o que faz
`RigidBodyState`	classe	`(n_bodies=1, dim=2, device='cpu', dtype=torch.float32) -> None`	Contêiner mutável do estado (posição, orientação, velocidades) de vários corpos rígidos.
`RigidBody`	classe	`(mass, inertia, dim=2) -> None`	Corpo rígido único com massa, inércia e integrador simplético.

nome	tipo	assinatura	o que faz
`RigidBodySystem`	classe	`(bodies, gravity=(0.0, -9.81, 0.0)) -> None`	Sistema de vários corpos rígidos com gravidade opcional.

pinneapple_simulation.particle_dynamics.terramechanics

Dataclasses de parâmetros de solo e roda para a interação roda-solo de Bekker-Wong.

`pinneapple_simulation/particle_dynamics/terramechanics.py` · 48 linhas · testes: 28

nome	tipo	assinatura	o que faz
`SoilParams`	classe	`(c, phi_deg, K, k_c, k_phi, n, rho, g, a0, a1)`	Parâmetros de solo de Bekker-Wong (coesão, atrito, módulos de afundamento, densidade).
`WheelParams`	classe	`(R, b, n_wheels, mass_rover)`	Geometria de roda rígida e massa do rover, com peso por roda.

pinneapple_simulation.pipelines

End-to-end pipelines that chain external tools (CAD, mesher, solver) into one checked, reproducible run.

`pinneapple_simulation/pipelines/__init__.py` · 5 linhas · **sem teste direto**

pinneapple_simulation.pipelines.freecad_openfoam

Headless, reproducible pipelines from a parametric FreeCAD model to a checked OpenFOAM result.

`pinneapple_simulation/pipelines/freecad_openfoam/__init__.py` · 29 linhas · testes: 14

pinneapple_simulation.pipelines.freecad_openfoam.__main__

`python -m pinneapple_simulation.pipelines.freecad_openfoam`:

`pinneapple_simulation/pipelines/freecad_openfoam/__main__.py` · 46 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	

pinneapple_simulation.pipelines.freecad_openfoam.cad

The FreeCAD step: a parametric document in, named boundary patches out.

`pinneapple_simulation/pipelines/freecad_openfoam/cad.py` · 365 linhas · testes: 14

nome	tipo	assinatura	o que faz
`PatchSpec`	classe	`(name, where, kind)`	One boundary patch: the rule that selects its faces and its kind (`inlet`, `outlet`, `wall`, `symmetry`); a plain string is a rule, the kind then follows the name (inlet*, outlet*, else wall).
`Face`	classe	`(index, type, area, centre, points, triangles)`	

nome	tipo	assinatura	o que faz
`CadGeometry`	classe	`(faces, info)`	Faces of one solid in document units (mm), with what the backend reported (volume, validity, files).
`FreeCADBackend`	classe	`(freecadcmd=None, timeout_s=600.0)`	FreeCAD through `freecadcmd` (headless). `timeout_s` per job.
`FakeFreeCAD`	classe	`()`	The templates of :mod:`.models` built in NumPy: the same faces (types, centres, outward triangles) and spreadsheet parameters as the FreeCAD documents, without FreeCAD. Documents are JSON: `{"fake_freecad": template, "parameters": {...}, "solid": name}`.
`rule_eval`	função	`(expr, names, functions=None) -> Any`	Value of a patch rule: numbers, names, arithmetic, comparisons (chained), `and or not` and a few `math` functions (plus `functions`); anything else raises `ValueError`.
`assign_patches`	função	`(geom, patches, parameters=None) -> dict[str, list[int]]`	Face indices of each patch: the first rule a face satisfies takes it; `rest` takes what is left. Raises `ValueError` when a face is left without a patch or a patch gets no face.
`write_stl`	função	`(path, geom, assignment, scale=0.001) -> Path`	ASCII STL with one `solid <patch>` per patch (snappyHexMesh turns each into a region), points times `scale` (mm -> m by default), merged across faces so shared edges stay shared.
`find_freecadcmd`	função	`() -> str | None`	`PINNEAPPLE_FREECADCMD`, the automation toolchain's FreeCAD (`/opt/freecad` AppImage), or PATH.
`get_backend`	função	`(name='auto')`	`"freecad"`, `"fake"`, or `"auto"` (FreeCAD when `freecadcmd` is found, else the fake).

`pinneapple_simulation.pipelines.freecad_openfoam.case`

OpenFOAM case templates for the meshed geometry: steady `simpleFoam` or transient `pimpleFoam`, incompressible, laminar or k-omega SST, with an optional passive scalar (a temperature at low heating, or a concentration).

`pinneapple_simulation/pipelines/freecad_openfoam/case.py` · 221 linhas · testes: 14

nome	tipo	assinatura	o que faz
`ScalarSpec`	classe	`(name, D, inlet, walls, initial)`	A passive scalar: `D` its diffusivity (m²/s; nu / Pr for a temperature), `inlet` its value on the inlets, `walls` patch -> fixed value or `"zeroGradient"` (walls not named: zero gradient).
`Physics`	classe	`(velocity, nu, rho, turbulence, solver, iterations, residual_tol, end_time, dt, max_coura…)`	Incompressible flow: `velocity` (m/s, uniform on the inlets), `nu` (m²/s), `rho` (kg/m³, for forces and pressures in Pa), `turbulence` laminar | kOmegaSST, `solver` simpleFoam | pimpleFoam. Steady: `iterations` and `residual_tol` (residualControl). Transient: `end_time`, `dt`, ...
`write_internal_case`	função	`(case, kinds, inlet_normals, physics, procs=1, size=None) -> dict[str, Any]`	0.orig fields, constant/ and system/ dictionaries and `Allrun` of an internal-flow case.

`pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job`

Módulo sem docstring e sem texto.

`pinneapple_simulation/pipelines/freecad_openfoam/freecad_scripts/fc_job.py` · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sheet`	função	`(doc, parameters)`	
`cylinder`	função	`(doc, name, radius, height, placement)`	
`build_pipe_bend`	função	`(doc, p)`	Straight inlet leg along +x ending at the origin, a bend of radius Rc (centre line) through `angle` degrees about the z axis, a straight outlet leg; diameter D.
`build_heated_pipe`	função	`(doc, p)`	Straight pipe along +x of diameter D: an unheated entry length L_entry from x = 0, then a heated length L_heated (two cylinders, so the wall has one face per section).
`build_spheroid`	função	`(doc, p)`	Spheroid centred at the origin, length L along x (the flow direction), diameter D across (L = D: sphere).
`set_parameters`	função	`(doc, parameters)`	
`faces_of`	função	`(shape, tol)`	
`main`	função	`()`	

`pinneapple_simulation.pipelines.freecad_openfoam.mesh`

Meshing the exported geometry: blockMesh background + snappyHexMesh (always available), or cfMesh's `cartesianMesh` when it is installed (`mesher="auto"` picks it then).

`pinneapple_simulation/pipelines/freecad_openfoam/mesh.py` · 193 linhas · testes: 14

nome	tipo	assinatura	o que faz
`MeshSpec`	classe	`(cells_across, length, level, mesher, feature_angle, max_non_ortho)`	`cells_across`: cells across `length` (a parameter name, mm, or a number in mm) near the walls; `level`: snappyHexMesh surface level (the background is `level` times coarser by 2); `mesher`: snappy, cfmesh or auto. `max_non_ortho` is the snappy quality limit.
`has_cfmesh`	função	`(bashrc=None) -> bool`	
`split_procs`	função	`(procs, size) -> tuple[int, int, int]`	Split `procs` into (nx, ny, nz) along the longest sides of the box first.
`decompose_dict`	função	`(case, procs, size) -> None`	hierarchical decomposition (scotch / metis can be stubs in some builds).
`allmesh_script`	função	`(procs, mesher='snappy') -> str`	

nome	tipo	assinatura	o que faz
`allrun_script`	função	`(solver, procs, scalar=False) -> str`	Copy `0.orig` to `0` and run `solver` (decomposed with MPI when `procs > 1`, then the latest time reconstructed); the solver log is `log.<solver>` in the case. `scalar`: then 20 steady iterations of `scalarTransportFoam` on the converged flow (log `log.scalarTransport`).
`write_internal_mesh`	função	`(case, stl, patches, inside, cell_size, spec, procs=1, bashrc=None) -> dict[str, Any]`	Dictionaries and `Allmesh` of an internal-flow mesh of the closed multi-solid `stl` (metres).

pinneapple_simulation.pipelines.freecad_openfoam.models

Parametric FreeCAD templates the pipeline ships, with their default patch rules and a NumPy twin for tests.

`pinneapple_simulation/pipelines/freecad_openfoam/models.py` · 196 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Model`	classe	`(name, title, flow, solid, parameters, patches, fake)`	
`build_document`	função	`(template, output, parameters=None, backend='auto') -> dict[str, Any]`	Write the parametric document of `template` (defaults overridden by `parameters`) with FreeCAD, or a fake document with :class:`~.cad.FakeFreeCAD` (`backend="fake"`, or `"auto"` without FreeCAD).

pinneapple_simulation.pipelines.freecad_openfoam.pipeline

A parametric FreeCAD model to a checked OpenFOAM result, as one reproducible workflow.

`pinneapple_simulation/pipelines/freecad_openfoam/pipeline.py` · 231 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PipelineSpec`	classe	`(id, title, template, document, solid, flow, parameters, inputs, patches, physics, mesh, …)`	One FreeCAD -> OpenFOAM study.
`evaluate`	função	`(expr, names) -> Any`	A number, or the value of an expression (see :func:`.cad.rule_eval`) with the reference functions.
`resolve_physics`	função	`(physics, inputs) -> dict[str, Any]`	Physics values with expressions evaluated in order (a later key may use an earlier one, e.g. `velocity = Re * nu / (D * 1e-3)`); nested dicts (the scalar) too.
`run_pipeline`	função	`(spec, folder, inputs=None, approve=True, approver='pipeline')`	Run (or resume) the study in `folder` with `inputs` (document parameters and other inputs). `approve`: every step is approved (the solver is `expensive`; a resumed step overwrites its own earlier files); without it the run stops at the first step that asks. Returns the automation …

pinneapple_simulation.pipelines.freecad_openfoam.post

Post-processing of a finished internal-flow case: what the pipeline reports and checks.

`pinneapple_simulation/pipelines/freecad_openfoam/post.py` · 322 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Section`	classe	`(name, origin, normal, radius)`	A disc of `radius` (m) centred at `origin` (m) with unit `normal` (the positive flow direction).
`CaseResult`	classe	`(case, scalar=None)`	Fields of the latest time of a case on its mesh: cell centres, volumes, U, p and the scalar, the boundary patches and their face values, with nearest-cell sampling.
`section_average`	função	`(res, sec) -> dict[str, float]`	Area-averaged p (kinematic), flux and bulk scalar over the part of the disc inside the fluid.
`wall_loads`	função	`(case, walls, nu, rho) -> dict[str, Any]`	Pressure and viscous force (N) on the wall patches and y+ of their first cells, from the written fields: :meth:`~pinneapple_simulation.numerical_solvers.external_flow.CaseFields.wall` (laminar sublayer, log law above y+ = 11.25), y+ = d u_tau / nu with d the wall distance of the first cell centre.
`read_case`	função	`(case, kinds, rho, sections=(), scalar=None, nu=1e-06) -> dict[str, Any]`	Every measured quantity of a finished case (see the module docstring); pressures in Pa.
`plane_figure`	função	`(res, path, origin, normal, title, rho, scalar_label=None, n=360) -> Path`	Speed, pressure (and the scalar) sampled on a plane through the domain; outside the fluid is blank.
`mesh_figure`	função	`(res, path, origin, normal, title) -> Path`	The cells cut by a plane, each drawn as a square of its size at its centre (snappyHexMesh cells are cubes away from the walls), coloured by size.
`geometry_figure`	função	`(geom, assignment, path, title) -> Path`	The exported solid with its faces coloured by patch (3-D, matplotlib).

pinneapple_simulation.pipelines.freecad_openfoam.report

Validation of a study: the same case on successively finer meshes, Richardson extrapolation (Roache's GCI, :func:`pinneapple_analysis.verification.convergence.richardson_extrapolate`), the error of every mesh and of the extrapolated value against the reference, a table and figures.

`pinneapple_simulation/pipelines/freecad_openfoam/report.py` · 154 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
mesh_study	função	(factory, resolutions, folder, reference, inputs=None, h_of=None, **kw) -> dict[str, Any]	Run factory(resolution, **kw) for each resolution (coarse to fine) in folder/r<resolution> (resumed when already run) and extrapolate the measured value of reference reference. h_of: the cell size of a resolution (default 1 / resolution: cells across a length; for snappy surface ...
mesh_study_figure	função	(studies, path, labels=None) -> Path	One panel per mesh study: measured value against cell size, the reference and its tolerance band, the Richardson-extrapolated value at h = 0.
sweep_figure	função	(rows, inputs, output, study, path, unit='') -> Path	The sweep (output over the first two inputs) and the surrogate's predictions against the solver on the training and the held-out cases.
validation_markdown	função	(studies) -> str	The measured validation table (one row per mesh, then the extrapolated value).

pinneapple_simulation.pipelines.freecad_openfoam.steps

The three pipeline actions that are not plain OpenFOAM commands, as functions of their parameters and a step folder. The automation adapters (freecadcmd.export, openfoam.prepare, openfoam.post in :mod:pinneapple_tools.automation.adapters.pipeline) wrap them; each returns ``{"outputs", ...

pinneapple_simulation/pipelines/freecad_openfoam/steps.py · 340 linhas · testes: 14

nome	tipo	assinatura	o que faz
point_inside	função	(V, F, p) -> bool	Whether p is inside the closed triangle mesh (parity of crossings along three skew rays, majority).
auto_procs	função	(cells, cores=None, per_proc=20000.0) -> int	Processes for a mesh of about cells cells: one per per_proc cells, at most the cores here.
cad_step	função	(params, step_dir) -> dict[str, Any]	Set the parameters, recompute, export, assign patches, write geometry.stl (m) and geometry.png.
prepare_step	função	(params, step_dir) -> dict[str, Any]	Mesh dictionaries + case + Allmesh / Allrun in step_dir/case.
post_step	função	(params, step_dir) -> dict[str, Any]	Measured quantities, figures, derived quantities and reference errors of a finished case.

pinneapple_simulation.pipelines.freecad_openfoam.studies

The worked, validated studies of the pipeline, as :class:~.pipeline.PipelineSpec factories (generic geometry, no client data). Each compares a measured quantity with a closed form or a published correlation (:mod:pinneapple_physics.closed_form); the measured numbers are in ...

pinneapple_simulation/pipelines/freecad_openfoam/studies.py · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
pipe_bend	função	(cells_across=16, backend='auto', procs='auto', bend_window=True) -> PipelineSpec	Laminar flow through a pipe bend; inputs: the FreeCAD parameters (D, Rc, angle, L_in, L_out) and Re. bend_window=False drops the sections inside the bend and the White (1929) reference, which need a bend of at least 165° (for sweeps over the angle).
heated_pipe	função	(cells_across=16, backend='auto', procs='auto') -> PipelineSpec	Laminar thermal entry at constant wall temperature (passive scalar T, inlet 0, heated wall 1).
sphere	função	(surface_level=4, backend='auto', procs='auto') -> PipelineSpec	Sphere (the spheroid template with L = D) in a uniform stream at low Reynolds number.

pinneapple_simulation.pipelines.freecad_openfoam.sweep

Parameter sweeps / designs of experiments over the inputs of a :class:.pipeline.PipelineSpec, and a surrogate of the results measured on held-out cases.

pinneapple_simulation/pipelines/freecad_openfoam/sweep.py · 152 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Sweep	classe	(spec, design, folder, ids)	spec run at every point of design (list of input dicts) under folder.
doe	função	(bounds, n, method='lhs', seed=0, fixed=None) -> list[dict[str, float]]	n points in the box bounds (name -> (low, high)): lhs (Latin hypercube), sobol or grid (n levels per input, n**d points); fixed values are added to every point.
case_id	função	(i, inputs) -> str	
fit_surrogate	função	(rows, inputs, outputs, *, test_frac=0.25, val_frac=0.15, seed=0, budget=5.0, methods='au…)	A surrogate of outputs from inputs on the finished cases: :func:pinneapple_analysis.surrogate. run_study with a random held-out test set (no distribution-shift split: sweeps are small). Returns the study; study.ranking("test") holds the held-out errors, study.model.predict the …

pinneapple_simulation.robot_sim

Robot Sim do PINNeAPPle: robôs no NVIDIA Newton com as checagens do PINNeAPPle (torques contra um modelo multicorpo independente, energia, convergência em dt, cargas estruturais) e um backend Isaac Sim com a mesma interface.

pinneapple_simulation/robot_sim/__init__.py · 18 linhas · testes: 39

pinneapple_simulation.robot_sim.arms

Braços prontos para o robot sim (o TwoLinkArm de tubos de alumínio), montados com os dados de material e modelos de elo do PINNeAPPle, para que as mesmas massas vão ao Newton (URDF), ao Isaac Sim (UsdPhysics) e à árvore multicorpo própria. [sem teste direto]

pinneapple_simulation/robot_sim/arms.py · 74 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TwoLinkArm	classe	(l1, l2, tube, wall, material, motor, payload, shoulder_z, base_mass, effort_limits, grav…)	Braço plano de dois elos no plano x-z com tubos quadrados de alumínio, motor e carga; gera o spec para Newton/Isaac e a árvore multicorpo.

pinneapple_simulation.robot_sim.checks

Checagem em uma chamada de um movimento de pegar-e-colocar de um braço no robot sim, usada pelo nó `lib.robot.arm_check` do Flow Studio.

pinneapple_simulation/robot_sim/checks.py · 50 linhas · testes: 39

nome	tipo	assinatura	o que faz
arm_pick_and_place	função	(payload=5.0, duration=0.8, q_start=(-0.6, 1.2), q_end=(0.9, -0.4), dt=0.0005, kp=(350.0,…)	Simula e checa um movimento de pegar-e-colocar do braço (torques contra modelo independente, energia, cargas na raiz dos elos).

pinneapple_simulation.robot_sim.learn

Pequeno laço de aprendizado em CPU (método da entropia cruzada, CEM) contra rollouts do `RobotSim`.

pinneapple_simulation/robot_sim/learn.py · 48 linhas · testes: 39

nome	tipo	assinatura	o que faz
CEMResult	classe	(best, best_cost, history, evaluations)	Resultado do CEM: melhores parâmetros, custo, histórico e número de avaliações.
cem	função	(objective, mean, std, *, iterations=6, population=8, elite=0.25, lower=None, upper=None,…)	Minimiza `objective(params)` pelo método da entropia cruzada, com limites opcionais.

pinneapple_simulation.robot_sim.sim

Simulação de robôs com NVIDIA Newton (`SolverFeatherstone`, CPU ou GPU via Warp), gravando torques, energia, poses e IMU; o `IsaacSimBackend` só escreve um script e levanta NotImplementedError/RuntimeError fora do Isaac Sim. [sem teste direto]

pinneapple_simulation/robot_sim/sim.py · 275 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
RobotRun	classe	(t, q, qd, tau, body_pos, body_quat, body_vel, body_omega, body_names, joint_names, masse…)	Registro de uma corrida (tempo, q, qd, torques, poses e velocidades dos corpos, saturação) com energia, trabalho do atuador e IMU.
RobotSim	classe	(builder_fn, *, gravity=9.81, ground=True, device='cpu', name='robot')	Modelo Newton de um robô (base fixa ou flutuante) construído de URDF, MJCF, USD ou spec, com `run` e controladores.
IsaacSimBackend	classe	(usd_path, robot_prim)	Mesma interface `run` para o Isaac Sim; aqui só escreve o script autônomo e levanta erro fora do Isaac Sim.

nome	tipo	assinatura	o que faz
`newton_available`	função	`() -> bool`	Diz se o NVIDIA Newton (e Warp) pode ser importado.
`torque_trajectory`	função	`(fn)`	Controlador em malha aberta com torques `fn(t)`.
`pd_tracking`	função	`(reference, kp, kd, feedforward=None)`	Controlador PD de rastreamento com feedforward opcional.
`convergence_order`	função	`(errors, dts) -> float`	Ordem observada de precisão: inclinação de log(erro) contra log(dt) por mínimos quadrados.

`pinneapple_simulation.robot_sim.spec`

Robô como dados simples (elos, juntas, acionamentos), exportável como URDF e como UsdPhysics para Omniverse/Isaac Sim. [sem teste direto]

`pinneapple_simulation/robot_sim/spec.py` · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Link`	classe	`(name, mass, size, position, com, inertia, box_offset, color, collide, shape)`	Elo do robô: massa, tamanho, posição, centro de massa, inércia, forma e colisão.
`Joint`	classe	`(name, parent, child, anchor, axis, limits_deg, drive, stiffness, damping, max_force, max…)`	Junta do robô: pai, filho, âncora, eixo, limites, acionamento e limites de força/velocidade.
`ArticulationSpec`	classe	`(name, links, joints, fixed_base, gravity, ground, info)`	Articulação completa (elos, juntas, base fixa, gravidade, chão) como dados simples.
`box_inertia`	função	`(mass, size) -> np.ndarray`	Inércia principal de uma caixa sólida em torno do centro.
`cylinder_inertia`	função	`(mass, radius, length) -> np.ndarray`	Inércia principal de um cilindro sólido com eixo em y.
`spec_to_urdf`	função	`(spec) -> str`	Escreve o URDF do spec: inerciais, colisores e visuais caixa/cilindro, juntas de revolução com limites.

`pinneapple_simulation.rover`

Six-wheel rocker-bogie rover on lunar terrain with Bekker-Wong wheel-soil forces.

`pinneapple_simulation/rover/__init__.py` · 31 linhas · testes: 21

`pinneapple_simulation.rover.autolab`

PINNeAPPle Autonomous Physics Lab, rover demo: *the rover that learns the planet*.

`pinneapple_simulation/rover/autolab/__init__.py` · 38 linhas · testes: 8

`pinneapple_simulation.rover.autolab.estimation`

What the rover measures, and what it infers about the soil from it.

`pinneapple_simulation/rover/autolab/estimation.py` · 281 linhas · testes: 8

nome	tipo	assinatura	o que faz
Sensors	classe	(sinkage_sd, slip_sd, torque_rel, torque_abs, load_rel, seed)	Noise levels (one standard deviation) of the rover's measurements; stated assumptions.
UnitBelief	classe	(base, mean, cov, n_obs, history, chi2_dof, scale)	
SoilBelief	classe	(prior_soils, wheel, g=1.62, prior_sd=PRIOR_SD, obs_sd_sinkage=0.008, obs_sd_torque=(2.0,…)	Gaussian beliefs over the soil parameters of each mapped unit (see the module docstring).
soil_theta	função	(soil) -> np.ndarray	
theta_soil	função	(base, theta) -> Soil	
measure	função	(run, sensors=None, wheels=None, every=4, min_speed=0.02, max_slip=0.7, slip_window=0.3, …)	Noisy measurements from a vehicle run (front wheels by default), one row per wheel and kept sample.

pinneapple_simulation.rover.autolab.mission

A mission in plain language -> the rover's lab loop, through a fixed set of tools.

pinneapple_simulation/rover/autolab/mission.py · 393 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
MissionSpec	classe	(text, goal, start, slip_limit, risk_max, objective, speed, max_replans, parser, quotes, …)	
RuleParser	classe	()	Deterministic parser with the extract(text, schema) interface of an LLM adapter (English and Portuguese keywords).
StepLog	classe	(stage, tool, summary, seconds)	
MissionResult	classe	(spec, routes, runs, belief, planners, measurements, comparison, report, log, retreat, ou…)	
RoverMission	classe	(site, text, *, llm=None, spec=None, dt=0.004, terrain_cell=0.03, planner_cell=0.5, n_sam…)	The rover lab loop on site for one request (see the module docstring).
parse_mission	função	(text, site, llm=None) -> MissionSpec	Request -> :class:MissionSpec. Values an LLM returns are kept only when their quote is in the text; missing values take the site's start / goal or the defaults (listed in defaults_used).

nome	tipo	assinatura	o que faz
traction_monitor	função	(slip_limit, window_s=2.0, output_dt=0.05, front=None, t_min=2.0)	Stop rule for :meth:VehicleSimulator.run: the mean slip of the (front) wheels over the last window_s seconds above slip_limit ends the run as "stalled". Samples before t_min seconds of driving (the start-up ramp) are not monitored. output_dt is not used (kept for compatibility): …
retreat_point	função	(run, site, back=2.0) -> tuple	Where the rover backs out to after a stall: back metres before the first point where a front wheel entered the unit it stalled on, on its own track (the reversal itself is not simulated).
to_json	função	(obj)	JSON-safe copy of mission outputs (arrays to lists, numbers rounded).

pinneapple_simulation.rover.autolab.planner

Risk-aware route planning on a site from what the rover believes about its soils.

pinneapple_simulation/rover/autolab/planner.py · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
UnitTables	classe	(grades, slip, energy, mean_slip, mean_energy)	
Route	classe	(waypoints, cells, length_m, energy_J, max_risk, mean_risk, max_slip, profile, belief_lab…)	
RiskPlanner	classe	(site, belief, wheel, mass, slip_limit=0.4, g=1.62, n_wheels=4, cell=0.5, n_samples=24, r…)	Plans on site with belief; mass [kg], wheel, n_wheels and g describe the rover.
densify	função	(P, step)	
simplify	função	(P, tol, accept=None)	Douglas-Peucker simplification of a polyline (tolerance tol metres); accept(a, b) can veto a shortcut (e.g. one that crosses risky ground), which is then split further.

pinneapple_simulation.rover.autolab.render

Blender Cycles renders of a rover mission (pip install bpy): the VIPER model of the repository (data/geometries/viper_rover/viper_rover.light.glb) on the site, the terrain with the ruts the multibody runs left, coloured by the learned traverse risk or by soil, and both routes.

pinneapple_simulation/rover/autolab/render.py · 322 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
available	função	() -> bool	

nome	tipo	assinatura	o que faz
`import_viper`	função	`(sc, path=VIPER_GLB)`	Import the VIPER glb under an empty (origin on the ground under the chassis, x forward, z up).
`pose_at`	função	`(run, k)`	(location, rotation matrix) of the rover at output sample `k`: contact-point centroid and chassis axes.
`place`	função	`(root, run, k)`	
`terrain_grid`	função	`(ter, x_range, y_range, step)`	
`terrain_colors`	função	`(site, X, Y, planner=None, mode='risk', shade_with=None)`	Soil albedo (`mode="soil"`) or the learned worst-heading risk blended over it (`"risk"`).
`build_terrain`	função	`(site, runs, cell=0.03, upto=None)`	A fresh deformable terrain with the ruts of `runs` replayed (all of each run, or up to `upto[i]` s).
`route_lines`	função	`(runs, routes, learned_planner=None)`	Caption lines of a render: what each route was planned with and what the multibody run measured.
`render_overview`	função	`(site, runs, routes, planner, out_png, *, size=(1600, 900), samples=48, step=0.06, lines=…)`	
`render_hero`	função	`(site, runs, out_png, *, size=(1600, 900), samples=96, window=6.0, cell=0.02, grains=2000…)`	Close-up of the stall of `runs[0]` (studio background, hard shadow, sand grains on the berms).
`render_traverse`	função	`(site, runs, out_dir, *, size=(800, 450), samples=16, every_s=2.5, step=0.06, fps=12, gif…)`	Frames of `runs` one after the other (chase camera, ruts growing), then `traverse.mp4` and `traverse.gif`. Returns (frame paths, mp4, gif).

`pinneapple_simulation.rover.autolab.report`

Report of a rover mission: `summary.json` (every number), `trust_report.md`, `report.md` and the figures.

`pinneapple_simulation/rover/autolab/report.py` · 206 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_info`	função	`() -> dict`	Versions and hardware of this run.
`mission_numbers`	função	`(mission, res) -> dict`	The measured and predicted numbers of a mission, labelled by where they come from.
`write_report`	função	`(mission, res, out_dir, figures=True, info=None) -> dict`	
`report_markdown`	função	`(n) -> str`	

`pinneapple_simulation.rover.autolab.site`

A synthetic planetary site whose truth is known: relief, soils, and what an orbiter would have said about them.

`pinneapple_simulation/rover/autolab/site.py` · 190 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Mound`	classe	`(x, y, sx, sy, height)`	Gaussian hill `height * exp(-((x-x0)^2/sx^2 + (y-y0)^2/sy^2) / 2)` [m].
`Patch`	classe	`(unit, x, y, a, b, angle_deg)`	An ellipse of one soil unit (rotated by `angle_deg`); later patches cover earlier ones.
`SoilUnit`	classe	`(name, truth, orbital, orbital_note, color)`	A mapped unit: its name, its true soil (hidden) and the soil the orbital map assigns to it.
`SyntheticSite`	classe	`(x_range, y_range, units, patches, mounds, tilt_deg, tilt_dir_deg, start, goal, descripti…)`	
`soft_sand`	função	`() -> Soil`	The hidden soft sand of the demo site: GRC-1 with a looser state (see :data:`SOFT_SAND_NOTE`).
`demo_site`	função	`() -> SyntheticSite`	A 40 m x 24 m site of compact regolith (GRC-1) cut by a north-south ridge (3.6 m high, flanks up to about 20 deg). The ridge's pass (about 14 deg) is filled with soft sand that the orbital map labels as compact regolith; its northern part is firm gravelly ground (JSC-1A). A second soft-sand patch ...

`pinneapple_simulation.rover.autolab.verify`

Trust report of a rover mission: did it work, does the physics hold, and can the learned soil be believed.

`pinneapple_simulation/rover/autolab/verify.py` · 182 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`monitor_slip`	função	`(run, front, window_s=2.0, t_min=2.0)`	The traction monitor's quantity: front-wheel mean slip averaged over the last `window_s` seconds (per output sample, once `window_s` seconds have been monitored from `t_min` on, exactly as :func:`~.mission.traction_monitor` decides; nan before).
`timestep_check`	função	`(mission, route, seconds=6.0) -> dict`	
`verify_mission`	função	`(mission, routes, runs, planners, belief, meas_all, comparison, do_timestep=True)`	

`pinneapple_simulation.rover.autolab.visuals`

Figures of a rover mission (matplotlib): site and risk maps with both routes, the soil belief converging, the wheel models against the measurements, and the traverses. Every curve is computed from the mission's own runs.

`pinneapple_simulation/rover/autolab/visuals.py` · 194 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_site_map`	função	`(site, planners, routes, runs, path, slip_limit=0.4)`	Truth soils with relief and routes; the risk the rover saw before (orbital prior) and after learning.
`plot_learning`	função	`(belief, unit, truth, path, title='')`	Mean +- 2 sd of each soil parameter of `unit` after every filter update, with the prior and the truth.
`plot_model_comparison`	função	`(comparison, path)`	Sinkage and torque against slip: measured (multibody + sensor noise) vs analytic (orbital prior, learned) vs the zoo surrogate, per unit the rover drove on.
`plot_traverse`	função	`(runs, site, path, slip_limit=0.4, front=(0, 1))`	Per run vs distance: the traction monitor's slip, front-wheel sinkage, cumulative drive energy, elevation with the soil unit under the rover.
`plot_route_risk`	função	`(planners, routes, path, slip_limit=0.4, risk_max=0.05)`	Predicted slip and risk along every route, under the orbital prior and under the learned belief.

`pinneapple_simulation.rover.autolab.wheel_models`

Wheel-soil laws at the operating point a rover needs, and the three ways to get them.

`pinneapple_simulation/rover/autolab/wheel_models.py` · 128 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GradeTable`	classe	`(grades, slip, sinkage, torque, energy_per_m, max_dp_over_w)`	Operating points of a quasi-static rover on one soil, per grade [deg] (arrays over `grades`).
`grade_table`	função	`(soil, wheel, mass, g=1.62, n_wheels=4, grades=GRADES, slips=SLIPS, n_gl=12) -> GradeTable`	Slip, sinkage, torque and energy per metre a `mass` kg rover on `n_wheels` needs on each grade.
`surrogate_steady_state`	função	`(predict, load, slip, z_max=0.2, iters=50)`	Sinkage with `Fz(slip, z) = load` for a `predict(slip, z) -> (N, 3)` contact model (bisection), and the forces there. Returns dict like :meth:`WongReeceModel.steady_state`.
`compare_contact_models`	função	`(measured, soil_believed, wheel, surrogate=None, g=1.62, in_range=None) -> dict`	Score the analytic law (believed soil) and a surrogate against measured operating points.
`max_climbable_grade`	função	`(table, slip_limit) -> float`	Largest grade of the table the rover climbs at a slip at or below `slip_limit` [deg] (nan if none).

`pinneapple_simulation.rover.checks`

Verification of the rover model against closed forms, the standalone solver and its own balance laws.

`pinneapple_simulation/rover/checks.py` · 149 linhas · testes: 12

nome	tipo	assinatura	o que faz
`single_wheel_check`	função	`(state, rover, soils, rear_angle_ratio=1.0 / 3.0) -> dict`	
`pressure_only`	função	`(soil) -> SoilParams`	The soil's pressure-sinkage law with shear switched off (c = 0, K very large) and the stress peak at the bottom of the wheel (a0 = a1 = 0); use with `rear_angle_ratio=0` for front-only contact.
`bekker_rigid_wheel_sinkage`	função	`(W, b, D, k, n) -> float`	Bekker's closed form for the static sinkage of a rigid wheel: z = [3 W / ((3 - n) b k sqrt(D))]^(2/(2n+1)).
`static_sinkage_check`	função	`(rover=None, soil=None) -> dict`	
`balance_check`	função	`(state, rover) -> dict`	Net force (including gravity, the lateral reaction, the tow load and the d'Alembert force) and net moment about the centre of mass, recomputed from the stored wheel forces; scaled by m g and m g L.
`surrogate_vs_numerical`	função	`(states, surrogate, numerical, soils) -> dict`	Errors of `surrogate` against `numerical` at every loaded (slip, sinkage) of the given states.
`dp_grade_estimate`	função	`(rover, soil, s_limit, g=1.62, rear_angle_ratio=1.0 / 3.0, tol=0.01) -> dict`	Grade where six equally loaded wheels at slip `s_limit` give drawbar pull = m g sin(a) (bisection).

`pinneapple_simulation.rover.contact`

Wheel-soil force laws for the rover: (slip, sinkage, soil) -> (F_x, F_z, M_y) per wheel.

`pinneapple_simulation/rover/contact.py` · 167 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BekkerWongContact`	classe	`(wheel_radius, wheel_width, rear_angle_ratio=1.0 / 3.0, method='gauss', n_gl=48)`	Bekker-Wong forces for N wheels at once (`soils[k]` selected per wheel with `soil_idx`).
`RangeLog`	classe	`(n_calls, n_out, slip_out, sinkage_out, soil_mismatch, worst)`	Counts of surrogate inputs inside / outside the training box.
`SurrogateContact`	classe	`(predict, slip_range, sinkage_range, soil, wheel_radius, wheel_width)`	Wrap `predict(slip, z) -> (N, 3)` trained on `soil` for a wheel (R, b) on slip x sinkage box ranges.

`pinneapple_simulation.rover.dynamics`

Rocker-bogie rover on lunar terrain with Bekker-Wong wheel forces.

`pinneapple_simulation/rover/dynamics.py` · 380 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`StepState`	classe	`(t, pose, v, omega, s_cmd, wheel_centre, contact_point, normal, tangent, sinkage, slip, F…)`	Converged state of one step (arrays per wheel in WHEEL_NAMES order).
`RoverSimulator`	classe	`(rover, terrain, contact=None, g=LUNAR_G, s_limit=0.95, conv_tol=1e-06)`	Quasi-static rocker-bogie rover with Bekker-Wong wheels on a :class:`LunarTerrain`.

nome	tipo	assinatura	o que faz
`Trajectory`	classe	`(states, rover, mode, failed_at, terrain)`	

pinneapple_simulation.rover.geometry

Parametric six-wheel rocker-bogie rover: parameters, kinematics and meshes (glTF / STL export).

`pinneapple_simulation/rover/geometry.py` · 237 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RoverParams`	classe	`(wheel_radius, wheel_width, n_grousers, grouser_height, track, wheelbase, pivot_height, b…)`	Generic small lunar rover (every value is a generic choice; see the module docstring).
`Pose`	classe	`(x, y, z, yaw, pitch, roll, rocker, bogie_l, bogie_r, extra)`	Rover configuration: body origin (rocker pivot axis centre), attitude and suspension angles [m, rad].
`rot_x`	função	`(a)`	
`rot_y`	função	`(a)`	
`rot_z`	função	`(a)`	
`body_rotation`	função	`(yaw, pitch, roll) -> np.ndarray`	Body-to-world rotation. `pitch` > 0 is nose up, `roll` > 0 is left side up [rad].
`quat_from_matrix`	função	`(R) -> np.ndarray`	(w, x, y, z) of a rotation matrix.
`link_frames`	função	`(p, pose) -> dict`	World positions of the pivots and wheel centres plus link rotations.
`rover_meshes`	função	`(p) -> dict`	Meshes in their own link frames: `body` (body frame), `rocker_L/R` (rocker frame, origin at the pivot), `bogie_L/R` (bogie frame, origin at the bogie pivot) and `wheel` (wheel frame, axle along y).
`assembled_mesh`	função	`(p, pose, wheel_spin=None)`	All parts in world coordinates: list of (name, vertices, faces).
`export_rover`	função	`(p, path, pose=None) -> str`	Write the rover (level, on its wheels) as .glb or .stl with the studio writer.

pinneapple_simulation.rover.terrain

Lunar terrain for rover simulation: a height function with a planar slope, craters and rocks, plus soil regions.

`pinneapple_simulation/rover/terrain.py` · 207 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Crater`	classe	`(x, y, diameter, rim_ratio, depth_ratio, crest_width)`	
`Rock`	classe	`(x, y, radius, height)`	
`SoilRegion`	classe	`(soil, kind, params, name)`	Soil inside a circle (`kind="circle"`: centre, radius) or an axis-aligned box (`"box"`: x0, x1, y0, y1).
`LunarTerrain`	classe	`(slope_deg, slope_dir_deg, ramp, craters, rocks, soil, regions)`	Height function and soil map.
`random_rocks`	função	`(rng, n, x_range, y_range, radius_range, height_ratio_range, keep_out=None) -> list[Rock]`	`n` rocks with uniform positions and radii (no published size-frequency law is used); `keep_out(x, y)` returns True where no rock may be placed (e.g. the rover path).
`random_craters`	função	`(rng, n, x_range, y_range, diameter_range, depth_ratio_range=(0.05, 0.15), rim_ratio=0.01…)`	`n` small degraded craters (scenery): positions uniform, diameters from a power law `N(>D) ~ D^-slope` truncated to `diameter_range`, depth/diameter uniform in `depth_ratio_range` (below Pike's 0.2 for fresh craters, since small lunar craters are mostly degraded). `keep_out(x, y, D)` …

`pinneapple_simulation.rover.viz`

Plots, animations and the 3D twin view of a rover :class:`~.dynamics.Trajectory`.

`pinneapple_simulation/rover/viz.py` · 225 linhas · testes: 12

nome	tipo	assinatura	o que faz
`plot_wheel_series`	função	`(traj, path, title='', x='distance') -> str`	
`plot_rover_series`	função	`(trajs, path, title='') -> str`	
`animate`	função	`(traj, path_gif, *, color_by='sinkage', path_mp4=None, frames=80, window=(1.7, 1.1), fps=…)`	3D animation following the rover over the terrain (`window`: half sizes of the terrain patch shown around it [m]); wheels coloured by `color_by` ("sinkage" [mm] or "slip").
`twin_scene`	função	`(traj, title='Lunar rover', margin=1.5, grid=(160, 100))`	A `pinneapple_twin3d.Scene` of the run (y up for the viewer): terrain coloured by height, links and wheels moving with the trajectory, wheel fields `sinkage` [mm] and `slip` [-] per step.

`pinneapple_simulation.terramechanics`

Terramecânica de rodas de rovers planetários: solos com proveniência, modelo de roda de Wong-Reece e extensões, terreno deformável com sulcos, bancada de roda única com dados publicados e substitutos PINN do contato.

pinneapple_simulation/terramechanics/__init__.py · 37 linhas · testes: 34

pinneapple_simulation.terramechanics.mobility

Triagem de mobilidade de roda com a PINN `bekker_wong_surrogate_2d`: treina, acha o afundamento que sustenta a carga, lê tração na barra contra escorregamento e rechecagem de cada restrição física.

pinneapple_simulation/terramechanics/mobility.py · 159 linhas · testes: 5

nome	tipo	assinatura	o que faz
`contact_entry_angle`	função	`(R_m, sinkage_m) -> float`	Ângulo de entrada θ1 = arccos(1 - z/R) de uma roda rígida afundada z.
`mohr_coulomb_traction_limit`	função	`(c_Pa, phi_deg, R_m, b_m, sinkage_m, Fz_N) -> float`	Maior tração na barra permitida pela resistência ao cisalhamento do solo, c A + Fz tan(φ).
`train_wheel_surrogate`	função	`(*, c_Pa, phi_deg, R_m, b_m, epochs=2000, n_collocation=1024, architecture='modified_mlp'…)`	Treina a PINN `bekker_wong_surrogate_2d` para um solo e uma roda.
`analyze_wheel_surrogate`	função	`(model, spec, *, c_Pa, phi_deg, R_m, b_m, Fz_N, slip_min=0.0, slip_max=0.7, n_slip=15, n_…)`	Tração contra escorregamento no afundamento que sustenta `Fz_N`, com checagem de cada restrição física.
`mobility_analysis`	função	`(*, c_Pa=1400.0, phi_deg=30.0, R_m=0.125, b_m=0.06, Fz_N=50.0, slip_min=0.0, slip_max=0.7…)`	Treina, checa as salvaguardas e analisa a mobilidade da roda em uma chamada.

pinneapple_simulation.terramechanics.soils

Banco de solos para modelos roda-solo (Bekker, Janosi-Hanamoto, Wong-Reece, escorregamento-afundamento, descarregamento) em que cada parâmetro registra a fonte ('secondary' ou 'estimate'). [sem teste direto]

pinneapple_simulation/terramechanics/soils.py · 222 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Soil`	classe	`(name, c, phi_deg, K, k_c, k_phi, n0, rho, n1, a0, a1, c3, Ky, k_u0, A_u, description, so…)`	Parâmetros de um solo com proveniência por parâmetro (`sources`).
`register_soil`	função	`(soil) -> Soil`	Registra um solo no banco.
`get_soil`	função	`(name) -> Soil`	Busca um solo pelo nome.
`list_soils`	função	`() -> list[str]`	Lista os nomes dos solos registrados.

pinneapple_simulation.terramechanics.surrogate

Substituto PINN de um modelo de contato roda-solo, (escorregamento, afundamento) -> (tração, carga normal, torque), treinado com termos físicos unilaterais e conjunto de teste separado, salvo como TorchScript + JSON.

pinneapple_simulation/terramechanics/surrogate.py · 292 linhas · testes: 20

nome	tipo	assinatura	o que faz
`ContactSurrogate`	classe	`(net, x_lo, x_hi, y_lo, y_hi, slip_range, sinkage_range, meta)`	Rede treinada com sua normalização e caixa de treino; prevê forças, diz se está no domínio e salva.
`PhysicsMismatchWarning`	classe	`()`	Aviso de que o substituto foi treinado com uma física diferente do modelo com que é usado.
`ZooContactSurrogate`	classe	`(zoo_model)`	Adapta um modelo de contato do model zoo (`pp.zoo.load`) à interface de `ContactSurrogate`.
`load_surrogate`	função	`(stem) -> ContactSurrogate`	Carrega um `ContactSurrogate` salvo (TorchScript + JSON).
`contact_net`	função	`(hidden=96, depth=4, n_fourier=16, scale=2.0, seed=0)`	Rede MLP residual com features de Fourier aleatórias e tanh, 2 entradas -> 3 saídas.
`train_contact_surrogate`	função	`(model, slip_range=(0.0, 0.8), sinkage_range=(0.002, 0.06), *, n_grid=40, n_lhs=2000, n_t…)`	Treina o substituto em grade + hipercubo latino com termos físicos e devolve o relatório de erro no teste separado.
`compare_on_states`	função	`(sur, model, slip, z) -> dict`	Compara substituto e referência em estados dados e conta os que estão fora da caixa de treino.

`pinneapple_simulation.terramechanics.terrain`

Terreno deformável para simulação roda-solo: memória plástica de afundamento (sulcos, multipassagem), marcas de garras, mapas de tensão de contato e rochas rígidas.

`pinneapple_simulation/terramechanics/terrain.py` · 341 linhas · testes: 20

nome	tipo	assinatura	o que faz
`RigidRock`	classe	`(x, y, a, b, c, burial, yaw, z0, seed)`	Rocha rígida elipsoidal parcialmente enterrada, com altura, penetração e malha.
`DeformableTerrain`	classe	`(base, x_range, y_range, cell=0.01, rebound=None, relief=0.0, relief_seed=0, berm=0.3, wa…)`	Grade de memória de afundamento sobre uma superfície base, com sulcos, bermas, marcas de garras e rochas.
`fractal_relief`	função	`(shape, cell, amplitude, seed=0, beta=2.2, min_wavelength=0.05) -> np.ndarray`	Superfície aleatória gaussiana com espectro em lei de potência e RMS dado.

`pinneapple_simulation.terramechanics.testbed`

Bancada de roda única: tração, torque e afundamento contra escorregamento a carga fixa, comparação com medições publicadas embarcadas e calibração por mínimos quadrados com pontos retidos.

`pinneapple_simulation/terramechanics/testbed.py` · 253 linhas · testes: 20

nome	tipo	assinatura	o que faz
`TestbedData`	classe	`(dataset, wheel_kind, wheel, soil, load, slip, drawbar_pull, sinkage, torque, source)`	Pontos medidos de uma roda num solo a uma carga (tração, afundamento, torque, fonte).
`ValidationReport`	classe	`(data, model, curve, errors)`	Relatório de validação: dados, modelo, curva e erros.

nome	tipo	assinatura	o que faz
`list_datasets`	função	`() -> list[tuple[str, str]]`	Lista os conjuntos embarcados (dataset, tipo de roda).
`load_dataset`	função	`(dataset, wheel, lambda_grouser=1.0) -> TestbedData`	Carrega os pontos embarcados ('ding2011', 'hurrell2025', 'wanghan2016') para roda lisa ou com garras.
`run_testbed`	função	`(model, load, slips) -> dict`	Tração, torque, afundamento e DP/W do modelo contra escorregamento a carga fixa.
`validate`	função	`(data, soil=None, g=9.81, **model_kw) -> ValidationReport`	Roda a bancada nos escorregamentos medidos e relata os erros contra a medição.
`holdout_split`	função	`(data, every=2, offset=0) -> np.ndarray`	Máscara de treino que separa pontos para ajuste e pontos retidos.
`calibrate`	função	`(data, params=('n0', 'n1', 'K'), bounds=None, g=9.81, train=None) -> dict`	Ajuste por mínimos quadrados de parâmetros do solo à tração e ao afundamento medidos.
`calibrate_holdout`	função	`(data, params=('n0', 'n1', 'K'), every=2, offset=0, g=9.81) -> dict`	Calibração de escorregamento-afundamento com pontos retidos para medir o erro fora do ajuste.
`plot_validation`	função	`(reports, path, title='', calibrated=None) -> str`	Gráfico de tração/carga e afundamento contra escorregamento: pontos medidos e curvas do modelo.

`pinneapple_simulation.terramechanics.wheel`

Geometria de roda rígida para terramecânica: raio, largura, garras (retas ou chevron), massa e inércia de giro. [sem teste direto]

`pinneapple_simulation/terramechanics/wheel.py` · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Grousers`	classe	`(count, height, thickness, shape, chevron_angle_deg, shear_radius_factor)`	Garras (lugs) de uma roda rígida: número, altura, espessura, forma e ângulo chevron.
`Wheel`	classe	`(radius, width, grousers, mass, spin_inertia)`	Roda rígida: raio, largura, garras, massa e inércia de giro.

`pinneapple_simulation.terramechanics.wong_reece`

Modelo de Wong-Reece roda rígida / solo deformável com escorregamento-afundamento, garras, multipassagem, bulldozing e força lateral. [sem teste direto]

`pinneapple_simulation/terramechanics/wong_reece.py` · 335 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ContactResult`	classe	`(Fx, Fy, Fz, My, theta_f, theta_r, z_eff, bulldozing, j_exit, theta, sigma, tau_x, tau_y)`	Forças e momento por ponto de operação, com as tensões no arco quando pedidas.
`WongReeceModel`	classe	`(soil, wheel, g=1.62, bulldozing='lateral', n_gl=24, slip_sinkage='drive')`	Roda de Wong-Reece num solo: tensões normal e cisalhante, forças, regime permanente e configuração física registrável.

nome	tipo	assinatura	o que faz
`bearing_capacity_factors`	função	`(phi_deg) -> tuple[np.ndarray, np.ndarray]`	Fatores de capacidade de carga de Terzaghi (N_c, N_γ) nas formas fechadas de Wong (2008).
`bulldozing_resistance`	função	`(soil, depth, width, g)`	Resistência de bulldozing de uma cunha de solo empurrada por uma face.
`slip_ratio`	função	`(v_long, omega, radius, v_eps=0.001)`	Escorregamento longitudinal (tração ou frenagem) a partir da velocidade do centro e da rotação da roda.
`slip_angle`	função	`(v_long, v_lat, v_eps=0.001)`	Ângulo de deriva de uma roda a partir das componentes da velocidade do centro.
`physics_mismatch`	função	`(recorded, current, rtol=1e-09) -> list[str]`	Lista as chaves em que uma configuração física gravada difere da atual.

`pinneapple_simulation.vehicle`

Veículos planetários com rodas: descrição (chassi + módulos de canto) com CAD, modelo multicorpo construído a partir dela e simulador em terreno deformável com rodas de Wong-Reece.

`pinneapple_simulation/vehicle/__init__.py` · 28 linhas · testes: 38

`pinneapple_simulation.vehicle.build`

Constrói o modelo multicorpo de um `VehicleSpec`: chassi como base flutuante e, por canto, braço de suspensão, módulo de direção e roda, com massas e inércias tiradas do CAD. [sem teste direto]

`pinneapple_simulation/vehicle/build.py` · 60 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VehicleModel`	classe	`(spec, tree, susp, steer, wheel, wheel_body, origins)`	Modelo multicorpo de um veículo: spec, árvore, índices de juntas e corpos, com transformações para o CAD.
`build_vehicle`	função	`(spec, gravity=1.62) -> VehicleModel`	Constrói o `VehicleModel` com corpos nomeados como no CAD (`chassis`, `arm_<c>`, `steer_<c>`, `wheel_<c>`).

`pinneapple_simulation.vehicle.dashboard`

Dashboard centrado em animação de uma corrida do veículo (sobre `pinneapple_tools.visualization.dashboard`): curvas da bancada, escorregamento por roda, PINN contra Wong-Reece e cena 3D. [sem teste direto]

`pinneapple_simulation/vehicle/dashboard.py` · 240 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`render_frames`	função	`(run, terrain, n=150, *, field=None, clim=None, size=(1280, 720), camera=None, preset='lu…)`	Renderiza `n` quadros distribuídos pela corrida com o renderizador planetário.
`vehicle_dashboard`	função	`(run, frames, ks, *, model=None, surrogate=None, title=None, subtitle='', field_label=Non…)`	Monta o dashboard da corrida com gráficos, tiles, painel de parâmetros e animação.
`vehicle_scene3d`	função	`(run, ks, terrain=None, cell=0.03, preset='lunar_low_sun') -> dict`	Payload 3D ao vivo: corpos do CAD, suas poses e o terreno final com sulcos.

pinneapple_simulation.vehicle.scenarios

Sítios lunares e estudos prontos: terreno plano ou inclinado, cratera com rochas e varredura de rampas em processos paralelos.

`pinneapple_simulation/vehicle/scenarios.py` · 87 linhas · testes: 17

nome	tipo	assinatura	o que faz
`flat_site`	função	`(x_range=(-3.0, 12.0), y_range=(-3.0, 3.0), cell=0.01, relief=0.005, seed=2, slope_deg=0.…)`	Regolito plano (ou com inclinação uniforme) com relevo aleatório fino.
`crater_site`	função	`(crater=(5.0, 0.5, 4.0), depth_ratio=0.08, rim_ratio=0.015, rock=(2.0, 0.65, 0.16, 0.13, …)`	Cratera degradada com borda e uma rocha sobre regolito.
`grade_sweep`	função	`(factory, soil='GRC-1', grades=(0, 5, 10, 15, 20), duration=12.0, speed=0.1, workers=4, c…)`	Sobe rampas uniformes em linha reta em processos paralelos e resume cada caso.

pinneapple_simulation.vehicle.sim

Simulação no tempo de um veículo com rodas em terreno deformável: dinâmica multicorpo mais rodas de Wong-Reece, rochas por penalidade, atuadores e seguimento de rota. [sem teste direto]

`pinneapple_simulation/vehicle/sim.py` · 571 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VehicleRun`	classe	`(model, t, q, v, wheel, joints, chassis, stresses, terrain_events, meta)`	Amostras gravadas de uma corrida (estado, rodas, juntas, chassi, tensões) com distância, energia, resumo, CSV e reprodução do terreno.
`VehicleSimulator`	classe	`(model, terrain, soil, g=1.62, soil_damping=0.7, rock_stiffness=300000.0, rock_damping=40…)`	Simulador de veículo em terreno deformável com rodas de Wong-Reece, rochas, atuadores e seguimento de rota.

pinneapple_simulation.vehicle.spec

Descrição de veículo compartilhada por CAD, modelo multicorpo e simulador: chassi e módulos de canto (suspensão -> direção -> roda motriz) com limites de junta e de atuador. [sem teste direto]

`pinneapple_simulation/vehicle/spec.py` · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Actuator`	classe	`(torque_limit, speed_limit, kp, ki, kd)`	Limites de torque e velocidade e ganhos de um servo de junta.
`CornerModule`	classe	`(name, susp_pivot, susp_axis, steer_origin, steer_axis, wheel_centre, wheel_axis, susp_li…)`	Um canto do veículo: pivô e eixo da suspensão, direção, centro e eixo da roda, limites e atuadores.
`VehicleSpec`	classe	`(name, wheel, corners, total_mass, max_speed, sources, cad)`	Descrição completa do veículo: roda, cantos, massa total, velocidade máxima, fontes e CAD.

pinneapple_simulation.vehicle.viper

NASA VIPER em escala real a partir de números públicos (fontes em `VIPER_SOURCES`, estimativas marcadas), com CAD paramétrico próprio.

pinneapple_simulation/vehicle/viper.py · 323 linhas · testes: 32

nome	tipo	assinatura	o que faz
viper	função	(wheel=None, total_mass=450.0) -> VehicleSpec	VehicleSpec do VIPER na altura nominal em terreno plano.
viper_cad	função	(spec) -> VehicleCAD	CAD paramétrico do VIPER (chassi, braços, direção, rodas) com massas.

pinneapple_simulation.vehicle.viz

Imagens e animações de um VehicleRun: CAD posado pelo estado multicorpo, terreno com sulcos reproduzido a cada instante, tensões de contato em cores, GIF/MP4 e gráficos por roda.

pinneapple_simulation/vehicle/viz.py · 352 linhas · testes: 11

nome	tipo	assinatura	o que faz
posed_parts	função	(model, q, *, skip=())	Malhas no mundo de cada peça do CAD para uma posição generalizada q.
terrain_at	função	(run, terrain, t_end, cache=None) -> DeformableTerrain	Cópia do terreno com os sulcos e marcas da corrida até t_end.
frame_view	função	(run, terrain, k, *, field=None, clim=None, half_window=(3.2, 2.4), far_margin=14.0, far_…)	Monta a vista de superfície de um quadro: veículo posado, terreno próximo e campo distante.
far_surface	função	(ter, X, Y, relief=0.03, feather=1.0)	Altura do solo no campo distante, misturando o terreno deformável com a superfície base.
follow_camera	função	(run, k, offset=(-4.4, -5.0, 2.1), look=(0.5, 0.0, 0.75), smooth=15)	Câmera que segue o veículo com rumo suavizado.
render_frame	função	(run, terrain, k, path, *, field=None, clim=None, camera=None, size=(1600, 900), title=No…)	Renderiza um quadro da corrida em imagem.
animate	função	(run, terrain, gif, *, mp4=None, field=None, clim=None, frames=120, fps=15, size=(1280, 7…)	GIF (e MP4 com ffmpeg) da corrida.
write_gif	função	(path, imgs, fps, size=None) -> str	Grava um GIF animado de quadros RGB, com redução opcional.
beauty_view	função	(spec, terrain=None, preset='lunar_south_pole', earth=False, horizon=4000.0, sun_azimuth_…)	Vista de superfície usada por beauty.
beauty	função	(spec, path, *, terrain=None, camera=BEAUTY_CAMERA, size=(1600, 900), title='', preset='l…)	Imagem do veículo na pose de referência sobre um trecho de terreno.
plot_run	função	(run, path, title='')	Gráficos por roda contra distância (afundamento, escorregamento, carga, tração, torque, força lateral) e arfagem/rolagem do chassi.
plot_testbed	função	(curves, path, title='')	Curvas de roda única contra escorregamento.
chassis_rotation	função	(run, k)	Matriz de rotação do chassi na amostra k.

Exemplos de uso reais do repositório

Colapso de coluna granular com GeoMPM — `examples/geomechanics/01_granular_column_collapse.py`

```
        from pinneapple_simulation.particle_dynamics.geo_mpm import GeoMPM,
Sand, box, runout
        ...
            sim = GeoMPM(domain=(L0 * (2 + 2.5 * a), a * L0 + 0.1), dx=0.01,
floor="sticky", walls="slip")
            sim.add_body(box((0, 0), (L0, a * L0)), Sand(friction_deg=PHI))
        ...
            sim.run(t_end=0.6 + 0.4 * a, save_every=0.05, out=run_dir)
```

Braço de dois elos no Robot Sim (Newton) — `examples/nvidia/03_robotsim_pick_and_place/run.py`

```
            a = TwoLinkArm()
        ...
            sim = RobotSim.from_spec(a.spec(), ground=False)
        ...
                    run = sim.run(DURATION, dts[-1], q0=q0,
controller=pd_tracking(ref, kp, kd, feedforward=feed),
```

VIPER: CAD e imagem — `examples/use_cases/terramechanics/lunar_rover_3d.py`

```
            spec = viper()
        ...
            spec.cad.export(str(media / "viper.glb"))
            spec.cad.export(str(OUT / "viper.stl"))
            viz.beauty(spec, str(media / "viper_beauty.png"), camera=((4.3,
-3.9, 1.75), (0.1, 0.0, 0.95)),
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 185 de 298.
- Funções de teste em arquivos que importam algum módulo do pacote: 1.119.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
S1	Finite differences 1D/2D/3D	validated	2	11
S2	Finite elements (linear) + nonlinear beam FEM	validated	6	15
S3	Finite volumes	validated	2	2
S4	Spectral (Fourier pseudo-spectral)	validated	3	2

id	método	status	testes contra referência	outros testes
S5	Lattice Boltzmann D2Q9 / D3Q19 (+ Smagorinsky LES)	validated	2	21
S6	SPH / ISPH / DFSPH	untested	0	0
S7	Meshfree RBF collocation (Kansa)	validated	1	1
S8	X-TFC (IVP, PDE, subdomains)	validated	4	8
S9	Discrete-time PINN / implicit Runge-Kutta Gauss-Legendre	tested	0	2
S10	Stiff kinetics ODE	validated	1	0
S11	TVD advection	validated	1	0
S12	Immersed boundary (FDM)	validated	1	1
S13	Beam BVP FDM / 3D elasticity FDM	validated	2	0
S14	Eddy current FDM	validated	3	3
S15	Bekker-Wong terramechanics	validated	1	19
S16	N-body (REBOUND)	validated	1	2
S17	JAX-CFD	tested	0	3
S18	Particle dynamics: rigid body, MPM, molecular dynamics	tested	0	13
S24	External solvers: OpenFOAM, FEniCS, ANSYS/CFD formats, MATLAB, Modelica/FMI, MuJoCo, Genesis, TurboDesigner	validated	1	80
S26	2D shallow-water finite volumes (MUSCL + HLL, wet/dry, walls, gates, virtual sensors, health checks)	validated	1	3
S27	ETDRK4 exponential integrator for stiff semilinear PDEs (contour phi-functions, blow-up guard)	validated	5	1
S28	Stable fluids 2D (semi-Lagrangian advection, spectral projection, vorticity confinement)	validated	2	4
S29	2-D A_z magnetostatics, P1 FEM with Newton (nonlinear B-H, magnets, (anti-)periodic constraints)	validated	1	6
S30	Time-harmonic 2-D eddy currents (A_z, solid conductors with imposed net currents)	validated	1	1

id	método	status	testes contra referência	outros testes
S31	Surface-PM machine model: Maxwell-stress and co-energy torque, cogging, back-EMF, losses	validated	1	8
S32	2-D electrostatics, P1 FEM (capacitance from field energy)	validated	2	0
S33	Biot-Savart integration of straight filament segments, loop and solenoid fields	validated	3	0
S34	Mie series for dielectric and perfectly conducting spheres	validated	2	0
S35	Fresnel coefficients and characteristic matrices of planar multilayers	validated	2	0
S36	Jiles-Atherton hysteresis model (RK4 along an applied field path)	validated	1	0
S37	Dispersive media (Drude, Lorentz, Debye) and their auxiliary-differential-equation coefficients	validated	1	0
S40	Yee FDTD in 1-D/2-D/3-D (CPML, Courant check, Drude/Lorentz/Debye ADE, PEC/PMC/periodic/Bloch, TF/SF plane waves, DFT monitors, sub-cell averaging)	validated	4	6
S41	Near-to-far-field transform on a closed box (FDTD radiation patterns)	validated	1	0

Limitações conhecidas

- `robot_sim` depende do NVIDIA Newton/Warp (`newton_available`); o `IsaacSimBackend` só escreve um script e levanta erro fora de uma máquina com Isaac Sim e GPU RTX.
- `ParticleSystem.forward` levanta NotImplementedError (classe base); o docstring de `particles` cita DEM, mas só o SPH está implementado.
- Vários módulos centrais não têm teste direto: `multibody.tree`, `terramechanics.wong_reece`, `terramechanics.soils`, `vehicle.sim`, `vehicle.build`, `vehicle.spec`, `vehicle.dashboard`, `robot_sim.sim`, `robot_sim.spec`.
- Os solos em `terramechanics.soils` trazem valores 'secondary' (não conferidos com a tabela original) ou 'estimate' (estimativa de engenharia), segundo o próprio docstring.
- O VIPER é um modelo paramétrico a partir de números públicos; o docstring avisa que os PDFs da NASA não puderam ser abertos e que alguns valores são estimativas.
- `read_particle_frames` com `allow_pickle=True` só deve ser usado com arquivos confiáveis (GeoTaichi grava objetos em pickle).
- Os substitutos PINN de contato só valem dentro da caixa de treino; fora dela `compare_on_states` e `in_range` sinalizam, e `PhysicsMismatchWarning` avisa física diferente.

Volume 4 — Aprendizado de máquina e dados

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Arquiteturas e treino de redes (PINN, operadores neurais, grafos), dados e amostragem, adaptação, modelos de mundo, percepção, computação quântica e a arena de benchmarks.

Sumário

1. pinneapple_neural
2. pinneapple_data
3. pinneapple_pdb
4. pinneapple_adaptation
5. pinneapple_arena
6. pinneapple_worldmodel
7. pinneapple_perception
8. pinneapple_quantum

1. `pinneapple_neural`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
198	36.217	334	150	438 (49 arquivos)	152

Pasta de testes dedicada: `tests/pinneapple_neural` (36 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`_licencas`		1	4	20
`architectures`		147	270	64
`callbacks`		1	3	4
`lpm`		1	6	5
`physicsnemo_bridge`		4	15	19
`predictor`		4	27	0
`trainer`		34	132	151
`utils`		1	2	5
`workflows`		4	21	10

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_neural`

pinneapple_neural — Neural network architectures, training, and inference.

`pinneapple_neural/__init__.py` · 256 linhas · testes: 20

nome	tipo	assinatura	o que faz
`build_model`	função	`(name, **kwargs)`	Build a model from the registry by name.
`train_model`	função	`(model, loss_fn, train_loader, val_loader=None, *, epochs=5000, device='cpu', **cfg_kwarg…)`	Train a model with the given loss function.
`predict`	função	`(model, x, *, device='cpu', batch_size=4096)`	Run batched inference on input `x`.
`train_and_predict`	função	`(model, loss_fn, train_loader, x_test, *, val_loader=None, epochs=5000, device='cpu')`	Convenience: train then evaluate on test points.

`pinneapple_neural._licencas`

License guards for optional third-party integrations.

`pinneapple_neural/_licencas.py` · 44 linhas · testes: 20

nome	tipo	assinatura	o que faz
`ResearchOnlyWarning`	classe	`()`	A research-only (non-production / non-commercial) component was used.
`ResearchOnlyError`	classe	`()`	A research-only component was requested in commercial mode.
`commercial_mode`	função	`() -> bool`	
`require_research_use`	função	`(component, notice=NOETHER_NOTICE) -> None`	Block `component` in commercial mode; otherwise warn that it is research only.

`pinneapple_neural.architectures`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/__init__.py` · 43 linhas · testes: 28

`pinneapple_neural.architectures._registry_bridge`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/_registry_bridge.py` · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register_family_registry`	função	`(family_registry, *, family, description_getter=None, tags_getter=None, capabilities_gett…)`	Registers a dictionary-like family registry into the global ModelRegistry.

`pinneapple_neural.architectures.adapters`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/adapters/__init__.py` · 27 linhas · **sem teste direto**

`pinneapple_neural.architectures.adapters.ae`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/adapters/ae.py` · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AutoEncoderAdapter`	classe	`()`	Adapter for autoencoders.

`pinneapple_neural.architectures.adapters.base`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/adapters/base.py` · 63 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelAdapter`	classe	`()`	Protocol for model adapters.
`select_adapter`	função	`(spec) -> ModelAdapter`	Pick an adapter based on model spec.

`pinneapple_neural.architectures.adapters.conv`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/adapters/conv.py` · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ConvAdapter`	classe	`()`	Adapter for grid/raster models (Conv1D/2D/3D, AFNO, UNO-grid-mode).

`pinneapple_neural.architectures.adapters.dynamics`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/adapters/dynamics.py` · 32 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DynamicsAdapter`	classe	`()`	Adapter for time-evolution / neural-ODE-style models.

`pinneapple_neural.architectures.adapters.gnn`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/adapters/gnn.py` · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GNNAdapter`	classe	`()`	Adapter for graph/mesh-based models.

`pinneapple_neural.architectures.adapters.operators`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/adapters/operators.py` · 49 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OperatorAdapter`	classe	`()`	Forward adapter for Neural Operators.

`pinneapple_neural.architectures.adapters.pinn`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/adapters/pinn.py` · 37 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PINNAdapter`	classe	`()`	Adapter for pointwise coordinate PINNs.

`pinneapple_neural.architectures.adapters.ts`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/adapters/ts.py` · 44 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TimeSeriesAdapter`	classe	`()`	Adapter for time-series models.

`pinneapple_neural.architectures.afno`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/afno.py` · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AFNOLayer`	classe	`(hidden_dim, n_modes_h=12, n_modes_w=12, mlp_ratio=4.0, dropout=0.0, num_blocks=8, hidden…)`	Single AFNO layer: block-diagonal token mixing in Fourier space + channel MLP.
`AFNO`	classe	`(in_channels, out_channels, hidden_dim=64, n_layers=4, n_modes_h=12, n_modes_w=12, mlp_ra…)`	Adaptive Fourier Neural Operator for structured-grid PDEs.

`pinneapple_neural.architectures.autoencoders`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/autoencoders/__init__.py` · 5 linhas · **sem teste direto**

`pinneapple_neural.architectures.autoencoders.ae_2d`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/autoencoders/ae_2d.py` · 133 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Autoencoder2D`	classe	`(in_channels, latent_dim, img_size, base_channels=32, norm='group', gn_groups=8, output_a…)`	Conv2D autoencoder for images / 2D fields.

`pinneapple_neural.architectures.autoencoders.ae_rom_hybrid`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/autoencoders/ae_rom_hybrid.py` · 320 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AEROMHybrid`	classe	`(input_dim, latent_dim, hidden=(512, 256), activation='gelu', rom_weight=1.0, pred_weight…)`	AE-ROM Hybrid (Koopman/latent linear dynamics) com: - AE denso (encoder/decoder) - Dinâmica contínua linear no latente: dz/dt = K z + c - Passo discreto via Euler: z_next_pred = z + dt*(K z + c) - Loss de previsão em x (decode do z_next_pred) - Rollout multi-step (latente e x) - Regularização de …

`pinneapple_neural.architectures.autoencoders.base`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/autoencoders/base.py` · 51 linhas · testes: 5 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`AEOutput`	classe	`(x_hat, z, losses, extras)`	Standardized output for all AEs.
`AEBase`	classe	`()`	Base class for all autoencoders in Pinneapple.

`pinneapple_neural.architectures.autoencoders.dense_ae`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/autoencoders/dense_ae.py` · 54 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DenseAutoencoder`	classe	`(input_dim, latent_dim, hidden=(512, 256), activation='gelu')`	Plain MLP autoencoder for vector inputs.

`pinneapple_neural.architectures.autoencoders.kae`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/autoencoders/kae.py` · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`KAEAutoencoder`	classe	`(input_dim, latent_dim, hidden=(512, 256), activation='gelu', mmd_weight=1.0, mmd_sigma=1…)`	Practical MMD-WAE style Autoencoder: - Dense AE - MMD penalty on latent space to match a simple prior.

`pinneapple_neural.architectures.autoencoders.koopman_pi_ae`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/autoencoders/koopman_pi_ae.py` · 331 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsInformedKoopmanAutoencoder`	classe	`(input_dim, latent_dim, hidden=(512, 256), activation='gelu', koopman_weight=1.0, *, pred…)`	Koopman AE (prático / literature-grade): - Dense AE - Operador linear treinável no latente (K, opcionalmente com bias b) - Suporte explícito a dt via gerador contínuo A (K_eff = expm(A*dt)) se desejado - Loss no latente: ||z_next - K_eff z||^2 - Loss de previsão no espaço original: ||x_next - …

`pinneapple_neural.architectures.autoencoders.registry`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/autoencoders/registry.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AutoencoderCatalog`	classe	`(registry)`	Group entrypoint: cat = AutoencoderCatalog() model = cat.build("vae", input_dim=..., latent_dim=...)
`register_into_global`	função	`() -> None`	

`pinneapple_neural.architectures.autoencoders.vae`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/autoencoders/vae.py` · 122 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VariationalAutoencoder`	classe	`(input_dim, latent_dim, hidden=(512, 256), beta=1.0, activation='gelu', clamp_logvar=(-10…)`	MLP VAE for vector inputs.

`pinneapple_neural.architectures.base`

BaseModel and ModelOutput for Pinneapple model family.

`pinneapple_neural/architectures/base.py` · 93 linhas · testes: 5 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`ModelOutput`	class e	`(y, losses, extras)`	
`BaseModel`	class e	`(family, name)`	Unified base class for all Pinneapple models.

`pinneapple_neural.architectures.benchmarks`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/benchmarks/__init__.py` · 5 linhas · **sem teste direto**

`pinneapple_neural.architectures.benchmarks.generic_pinn_models`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/benchmarks/generic_pinn_models.py` · 122 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FourierFeatures`	classe	`(in_dim, num_f=64, scale=10.0)`	
`GenericMLP`	classe	`(in_dim=3, out_dim=1, width=128, depth=4, act='tanh')`	
`GenericFourierMLP`	classe	`(in_dim=3, out_dim=1, num_f=64, scale=10.0, width=128, depth=4)`	
`Sine`	classe	`(w0=30.0)`	
`GenericSIREN`	classe	`(in_dim=3, out_dim=1, width=128, depth=4, w0=30.0)`	
`GenericResMLP`	classe	`(in_dim=3, out_dim=1, width=128, depth=6)`	
`GenericLinear`	classe	`(in_dim=3, out_dim=1)`	

`pinneapple_neural.architectures.benchmarks.registry`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/benchmarks/registry.py` · 32 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register_into_global`	função	`() -> None`	Register benchmark models into the global ModelRegistry.

`pinneapple_neural.architectures.catalog`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/catalog.py` · 33 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelCatalog`	classe	`(autoencoders, pinns, transformers, recurrent, convolutions, neural_operators, continuous…)`	

`pinneapple_neural.architectures.classical_ts`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/__init__.py` · 5 linhas · **sem teste direto**

`pinneapple_neural.architectures.classical_ts.arima`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/arima.py` · 250 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ARIMA`	classe	`(dim=1, p=3, d=0, q=0, l2=1e-06, use_bias=True)`	ARIMA(p,d,q) MVP without external deps:

`pinneapple_neural.architectures.classical_ts.base`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/base.py` · 34 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ClassicalTSOutput`	classe	`(y, losses, extras)`	Output for classical time-series / filtering models.
`ClassicalTSBase`	classe	`()`	

`pinneapple_neural.architectures.classical_ts.ekf`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/ekf.py` · 125 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ExtendedKalmanFilter`	classe	`(f, h, Q, R, F_jac, H_jac)`	EKF (robust MVP): x_t = f(x_{t-1}, u_t) + w y_t = h(x_t) + v

`pinneapple_neural.architectures.classical_ts.enkf`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/enkf.py` · 122 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EnsembleKalmanFilter`	classe	`(f, h, Q, R, *, ensemble_size=64)`	Ensemble Kalman Filter (EnKF) MVP:

`pinneapple_neural.architectures.classical_ts.kalman`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/kalman.py` · 130 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`KalmanFilter`	classe	`(A, H, Q, R, *, B=None)`	Linear Kalman Filter (batch, robust):

`pinneapple_neural.architectures.classical_ts.registry`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/registry.py` · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ClassicalTSCatalog`	classe	`(registry)`	
`register_into_global`	função	`() -> None`	

`pinneapple_neural.architectures.classical_ts.tcn`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/tcn.py` · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TCN`	classe	`(in_dim, out_dim, *, channels=(64, 64, 64), kernel=3, dropout=0.1)`	TCN for sequence-to-sequence regression.

`pinneapple_neural.architectures.classical_ts.ukf`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/ukf.py` · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UnscentedKalmanFilter`	classe	`(f, h, Q, R, *, alpha=0.001, beta=2.0, kappa=0.0, jitter=1e-06, jitter_growth=10.0, max_c…)`	UKF (sigma-point filter), batch-friendly.

`pinneapple_neural.architectures.classical_ts.var`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/classical_ts/var.py` · 165 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VAR`	class e	`(dim, p=1, l2=1e-06, use_bias=True)`	VAR(p) with ridge regression (closed form).

`pinneapple_neural.architectures.continuous`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/__init__.py` · 5 linhas · **sem teste direto**

`pinneapple_neural.architectures.continuous.base`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/base.py` · 38 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ContOutput`	classe	`(y, losses, extras)`	Standard output for continuous-time / probabilistic dynamics models.
`ContinuousModelBase`	classe	`()`	

`pinneapple_neural.architectures.continuous.bayesian_rnn`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/bayesian_rnn.py` · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LockedDropout	classe	`(p=0.0)`	Locked (aka variational) dropout for sequences: uses the same dropout mask across the time dimension T.
BayesianRNN	classe	`(input_dim, out_dim, hidden_dim=128, num_layers=2, dropout=0.1, cell='gru', min_logvar=-1…)`	Bayesian RNN (MVP via MC Dropout / variational dropout approximation).

pinneapple_neural.architectures.continuous.deep_state_space

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/continuous/deep_state_space.py · 214 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ContOutput	classe	`(y, losses, extras)`	
DeepStateSpaceModel	classe	`(input_dim, out_dim, latent_dim=32, hidden_dim=128, mlp_depth=2, min_logvar=-10.0, max_lo…)`	Variational Deep State Space Model (DSSM / deep SSM) - MVP (filtering):
gaussian_nll_diag	função	`(mu, logvar, y) -> torch.Tensor`	
kl_diag_gaussians	função	`(mu_q, logvar_q, mu_p, logvar_p) -> torch.Tensor`	

pinneapple_neural.architectures.continuous.hamiltonian

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/continuous/hamiltonian.py · 69 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
HamiltonianNeuralNetwork	classe	`(dim_q, hidden=128, num_layers=3)`	HNN MVP: - Learn Hamiltonian H(q,p) - Dynamics: dq/dt = dH/dp, dp/dt = -dH/dq

pinneapple_neural.architectures.continuous.latent_ode

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/continuous/latent_ode.py · 249 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LatentODE	classe	`(obs_dim, latent_dim=32, hidden=128, *, ode_method='dopri5', enc_method='dopri5', min_log…)`	Latent ODE (improved, literature-aligned): - Encoder: GRU-ODE recognition model (handles irregular t) - Latent dynamics: NeuralODE with adaptive solver (default dopri5) - Decoder: Gaussian likelihood (mu_x, log_sigma_x) -> NLL

pinneapple_neural.architectures.continuous.neural_cde

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/neural_cde.py` · 156 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeuralCDE`	classe	`(input_dim, hidden_dim, out_dim, *, f_hidden=128, g_hidden=128, num_layers=2, activation=…)`	Neural CDE (literature-style MVP, no torchcde):

`pinneapple_neural.architectures.continuous.neural_gp`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/neural_gp.py` · 248 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeuralGaussianProcess`	classe	`(in_dim, out_dim, *, embed_dim=64, mlp_hidden=128, mlp_layers=2, rff_features=512, rff_le…)`	Neural Gaussian Process (RFF + Bayesian Linear head):

`pinneapple_neural.architectures.continuous.neural_ode`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/neural_ode.py` · 91 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeuralODE`	classe	`(state_dim, hidden=128, num_layers=3, method='rk4')`	Neural ODE MVP: dy/dt = f(t, y) where f is a neural net. Integrate over time grid with Euler or RK4.

`pinneapple_neural.architectures.continuous.neural_sde`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/neural_sde.py` · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeuralSDE`	classe	`(state_dim, *, hidden=128, num_layers=3, diffusion='diag', activation='tanh', sigma_eps=1…)`	Neural SDE (Euler–Maruyama) for *solving* problems via simulation:

`pinneapple_neural.architectures.continuous.ode_rnn`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/ode_rnn.py` · 54 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ODERNN`	classe	`(input_dim, hidden_dim=128)`	ODE-RNN MVP: - Integrate hidden state with Neural ODE between observation times - Update with GRUCell when observation arrives

pinneapple_neural.architectures.continuous.registry

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/registry.py` · 80 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ContinuousCatalog`	classe	`(registry)`	
`register_into_global`	função	`() -> None`	

pinneapple_neural.architectures.continuous.symplectic_ode

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/symplectic_ode.py` · 147 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SymplecticODENet`	classe	`(dim_q, hidden=128)`	Symplectic ODE-Net:

pinneapple_neural.architectures.continuous.symplectic_rnn

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/continuous/symplectic_rnn.py` · 128 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SymplecticRNN`	classe	`(dim_q, *, hidden=128, num_layers=2, activation='tanh')`	Symplectic RNN (MVP, leap-frog-style update):

pinneapple_neural.architectures.convolutions

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/convolutions/__init__.py` · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.convolutions.base

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/convolutions/base.py` · 36 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ConvOutput`	classe	`(y, losses, extras)`	Standard output for convolutional models in Pinneapple.
`ConvModelBase`	classe	`()`	Base class for convolution models.

pinneapple_neural.architectures.convolutions.conv1d

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/convolutions/conv1d.py` · 265 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Conv1DModel`	classe	`(in_channels, out_channels, *, mode='denoise', cond_channels=0, hidden_channels=64, num_b…)`	Flexible 1D Conv model supporting: - causal forecasting - denoising - operator learning - PDE 1D surrogate

pinneapple_neural.architectures.convolutions.conv2d

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/convolutions/conv2d.py` · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Conv2DModel`	classe	`(in_channels, out_channels, hidden_channels=32, num_blocks=4, kernel_size=3, dropout=0.0,…)`	Conv2D regression model (ResNet-like CNN).

pinneapple_neural.architectures.convolutions.conv3d

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/convolutions/conv3d.py` · 174 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ResidualBlock3D`	classe	`(channels, kernel_size=3, dropout=0.0, norm_groups=8) -> None`	A stable 3D residual block: Conv -> GroupNorm -> GELU -> Dropout -> Conv -> GroupNorm -> GELU Residual is applied outside (so the caller can scale it).
`Conv3DModel`	classe	`(in_channels, out_channels, hidden_channels=32, num_blocks=4, kernel_size=3, dropout=0.0,…)`	Conv3D regression model.

pinneapple_neural.architectures.convolutions.registry

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/convolutions/registry.py` · 60 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ConvolutionCatalog`	classe	`(registry)`	
`register_into_global`	função	`() -> None`	

pinneapple_neural.architectures.graphnn

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/graphnn/__init__.py` · 8 linhas · **sem teste direto**

pinneapple_neural.architectures.graphnn.base

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/graphnn/base.py` · 54 linhas · testes: 15

nome	tipo	assinatura	o que faz
`GraphBatch`	classe	`(x, edge_index, pos, edge_attr, mask)`	Minimal graph batch format (no external deps).
`GraphOutput`	classe	`(y, losses, extras)`	Standard output for GNN family. y: - node-level: (B, N, out_dim) - graph-level: (B, out_dim) (not used in this MVP)
`GraphModelBase`	classe	`()`	Base for graph models.

`pinneapple_neural.architectures.graphnn.equivariant_gnn`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/graphnn/equivariant_gnn.py` · 229 linhas · testes: 5

nome	tipo	assinatura	o que faz
`EquivariantGNN`	classe	`(node_dim, pos_dim, out_dim, *, hidden=128, layers=4, edge_dim=0, dropout=0.0, update_pos…)`	EGNN-style MVP (E(n)-equivariant):

`pinneapple_neural.architectures.graphnn.gnn`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/graphnn/gnn.py` · 123 linhas · testes: 5

nome	tipo	assinatura	o que faz
`GraphNeuralNetwork`	classe	`(node_dim, out_dim, *, edge_dim=0, hidden=128, layers=4, dropout=0.0)`	Message Passing Neural Network (MVP).

`pinneapple_neural.architectures.graphnn.gnn_ode`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/graphnn/gnn_ode.py` · 212 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FourierTimeFeatures`	classe	`(n_frequencies, *, max_frequency=10.0)`	Fourier features for scalar time t. Produces [sin(2π f_i t), cos(2π f_i t)] features.
`GraphNeuralODE`	classe	`(node_dim, *, edge_dim=0, hidden=128, layers=3, method='rk4', time_features=0, time_max_f…)`	Graph Neural ODE (MVP):

`pinneapple_neural.architectures.graphnn.graphcast`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/graphnn/graphcast.py` · 289 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GraphCast`	classe	`(node_in, out_dim, *, hidden=256, processor_blocks=8, pos_dim=0, edge_dim=0, dropout=0.0)`	GraphCast-inspired MVP (fixed mesh + variable graph support).

pinneapple_neural.architectures.graphnn.mesh_graph_net

MeshGraphNet — Encoder-Process-Decoder GNN for mesh-based simulation.

pinneapple_neural/architectures/graphnn/mesh_graph_net.py · 375 linhas · testes: 15 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
MeshGraphNet	classe	(node_in_dim, out_dim, *, edge_in_dim=0, pos_dim=0, hidden_dim=128, n_layers=2, n_message…)	Encoder-Process-Decoder GNN for unstructured mesh simulation.

pinneapple_neural.architectures.graphnn.mgn_dynamics

Transient mesh dynamics on top of :class:MeshGraphNet.

pinneapple_neural/architectures/graphnn/mgn_dynamics.py · 270 linhas · testes: 6

nome	tipo	assinatura	o que faz
MeshGraph	classe	(edge_index, edge_attr, node_type)	One static mesh: topology, raw edge features and node types.
Normalizer	classe	(dim, eps=1e-08) -> None	Per-feature standardiser with fixed statistics (stored as buffers).
MeshDynamicsMGN	classe	(vel_dim=2, num_node_types=4, edge_dim=3, out_p=True, **mgn_kwargs) -> None	Autoregressive MeshGraphNet for fields (vel, p) on a transient mesh.
triangles_to_edges	função	(cells) -> torch.Tensor	Bidirectional, de-duplicated edge_index (2, E) from triangle cells (T, 3).
edge_features	função	(pos, edge_index) -> torch.Tensor	Relative displacement (dst - src) and its norm → (E, pos_dim + 1).
train_dynamics	função	(model, data, *, steps, lr=0.0001, lr_final=1e-06, noise_std=0.02, device='cpu', log_ever…)	One-step training on random (trajectory, time) pairs with Adam and exponential LR decay.
eval_rollout	função	(model, data, *, n_steps, device='cpu', normal_type=0) -> dict[str, float]	Velocity RMSE on NORMAL nodes: mean over trajectories of the n_steps free rollout and of one step.
frozen_ic_rmse	função	(data, *, n_steps, normal_type=0) -> float	Baseline for :func:eval_rollout: velocity RMSE on NORMAL nodes when the velocity stays at its t = 0 value over the same horizon, mean over trajectories.

pinneapple_neural.architectures.graphnn.registry

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/graphnn/registry.py · 64 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GraphCatalog	classe	(registry)	
register_into_global	função	() -> None	

pinneapple_neural.architectures.graphnn.spatiotemporal_gnn

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/graphnn/spatiotemporal_gnn.py` · 117 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpatiotemporalGNN`	classe	`(node_in, out_dim, *, edge_dim=0, spatial_hidden=128, spatial_layers=3, temporal_hidden=1…)`	Spatiotemporal GNN (MVP):

pinneapple_neural.architectures.graphnn.utils

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/graphnn/utils.py` · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`scatter_add`	função	`(src, index, dim_size) -> torch.Tensor`	Scatter-add with shared-graph batching.
`scatter_add_flat`	função	`(src, index, dim_size) -> torch.Tensor`	Flattened variable-graph batching. src: (E_total, F) index: (E_total,) destination indices [0..N_total-1] returns: (N_total, F) where out[index[e], :] += src[e, :]
`flatten_graph_batch`	função	`(x_list, edge_index_list, pos_list=None, edge_attr_list=None)`	Turns per-sample graphs into a single flattened graph with node index offsets.

pinneapple_neural.architectures.group_b_registry

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/group_b_registry.py` · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GroupBCatalog`	classe	`(registry)`	
`register_into_global`	função	`() -> None`	

pinneapple_neural.architectures.hash_grid

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/hash_grid.py` · 219 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HashGridEncoding`	classe	`(in_dim=3, n_levels=16, n_features_per_level=2, log2_hashmap_size=19, base_resolution=16,…)`	Multi-resolution hash grid encoding (Instant-NGP style).
`HashGridMLP`	classe	`(in_dim=3, out_dim=1, hidden_dim=64, n_hidden=2, n_levels=8, n_features_per_level=2, log2…)`	MLP with multi-resolution hash grid encoding for fast 3-D field representation.

pinneapple_neural.architectures.instantiate

Model instantiation helpers.

pinneapple_neural/architectures/instantiate.py · 173 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
InstantiateReport	classe	(original, normalized, dropped, used_aliases)	Diagnostics about how kwargs were normalized.
normalize_kwargs	função	(kwargs) -> Tuple[Dict[str, Any], Dict[str, str]]	Normalize user kwargs to a canonical set.
filter_supported_kwargs	função	(model_cls, kwargs) -> Tuple[Dict[str, Any], Dict[str, Any]]	Keep only kwargs accepted by model_cls.__init__.
instantiate	função	(model_cls, kwargs=None, *, strict=False) -> Tuple[BaseModel, InstantiateReport]	Instantiate a model class with normalized kwargs.

pinneapple_neural.architectures.modified_mlp

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/modified_mlp.py · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FourierFeatureEmbedding	classe	(in_dim, n_fourier=64, sigma=1.0, trainable=False)	Random Fourier feature embedding for coordinate inputs.
ModifiedMLP	classe	(in_dim, out_dim, hidden_dim=128, n_layers=6, n_fourier=32, sigma=1.0, activation=nn.Tanh…)	Modified MLP with Fourier feature embedding and highway U/V connections.

pinneapple_neural.architectures.neural_operators

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/neural_operators/__init__.py · 51 linhas · testes: 2

pinneapple_neural.architectures.neural_operators.base

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/neural_operators/base.py · 40 linhas · testes: 5

nome	tipo	assinatura	o que faz
OperatorOutput	class e	(y, losses, extras)	Standard output for neural operators.
NeuralOperatorBase	class e	()	Base class for Neural Operators.

pinneapple_neural.architectures.neural_operators.deeponet

DeepONet operator learning with branch-trunk architecture (Lu et al., Nat. Mach. Intell. 3, 2021).

pinneapple_neural/architectures/neural_operators/deeponet.py · 79 linhas · testes: 5

nome	tipo	assinatura	o que faz
DeepONet	classe	(branch_dim, trunk_dim, out_dim, hidden=128, modes=64, depth=1, trunk_activation=False)	Unstacked DeepONet, one set of branch coefficients per output channel:

nome	tipo	assinatura	o que faz
`mlp`	função	`(in_dim, hidden, out_dim, depth=1, act=nn.GELU, last_act=False) -> nn.Sequential`	MLP with hidden widths `hidden` (a list) or `depth` layers of width `hidden` (an int).

pinneapple_neural.architectures.neural_operators.fno

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/neural_operators/fno.py` · 530 linhas · testes: 39

nome	tipo	assinatura	o que faz
`FourierNeuralOperator`	classe	`(in_channels, out_channels, width=64, modes=16, layers=4, *, use_grid=False)`	FNO-1D (MVP) - extendable to 2D/3D by changing spectral conv + projections.
`FNO2d`	classe	`(in_channels, out_channels, width=32, modes1=12, modes2=12, layers=4, *, use_grid=True)`	FNO-2D: Fourier Neural Operator for 2D spatial fields.
`FNO3d`	classe	`(in_channels, out_channels, width=32, modes1=8, modes2=8, modes3=8, layers=4, *, use_grid…)`	FNO-3D: Fourier Neural Operator for 3D spatial (or 2D-space + time) fields -- the natural fit for a structured, periodic-in-2-of-3-axes domain like a periodic channel (matches the setup FNO was originally demonstrated on: Li et al. 2020, Navier-Stokes on a periodic torus).
`MLPFNOSurrogate`	classe	`(in_channels, out_channels, fno_width=32, fno_modes=12, fno_layers=4, mlp_hidden=128, mlp…)`	Hybrid FNO backbone + MLP decoder for scattered-point evaluation.

pinneapple_neural.architectures.neural_operators.fno_export

ONNX export of Fourier neural operators.

`pinneapple_neural/architectures/neural_operators/fno_export.py` · 121 linhas · testes: 9

nome	tipo	assinatura	o que faz
`RealSpectralConv2d`	classe	`(conv, H, W)`	Real-arithmetic equivalent of :class:`~.fno._SpectralConv2d` on an `H x W` grid.
`to_exportable`	função	`(model, grid) -> nn.Module`	A copy of `model` with every 2-D spectral convolution in real arithmetic for an `H x W` grid.
`export_onnx`	função	`(model, example, path, *, grid=None, opset=17, atol=0.0001) -> float`	Export `model` (an FNO, or any module containing 2-D spectral convolutions) to ONNX at `path` with a dynamic batch axis, check onnxruntime against PyTorch on `example` and return the largest absolute difference (raises when it exceeds `atol` times the output scale). `grid` defaults to the …

pinneapple_neural.architectures.neural_operators.gno

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/neural_operators/gno.py · 146 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GalerkinNeuralOperator	classe	(in_dim, out_dim, basis_dim=64, basis_hidden=128, op_hidden=128, op_depth=2, orthonormali…)	Galerkin Neural Operator (mesh-based, 1D/2D/3D): - Learn a basis Phi(x) from coords (x in R^d, d can be 1,2,3) - Project u -> coefficients using quadrature/lumped-mass weights - Apply operator in coefficient space - Reconstruct y = Phi * c_out

pinneapple_neural.architectures.neural_operators.ms_deeponet

Multi-scale DeepONet: one trunk per spatial scale, each seeing the coordinates stretched by its own factor (the MscaleDNN idea, Liu, Cai and Xu 2020), so high-frequency parts of the output are learned by trunks that see them as low frequencies. The branch gives the coefficients of all scales.

pinneapple_neural/architectures/neural_operators/ms_deeponet.py · 69 linhas · testes: 5

nome	tipo	assinatura	o que faz
MultiScaleDeepONet	classe	(branch_dim, trunk_dim, out_dim, *, hidden=128, scales=(32, 64, 128), scale_factors=None,…)	y(u, x) = sum_s < B_s(u), T_s(c_s x) > + bias

pinneapple_neural.architectures.neural_operators.noether_bridge

Noether (Emmi AI) architecture bridge for PINNeAPPle.

pinneapple_neural/architectures/neural_operators/noether_bridge.py · 296 linhas · testes: 5

nome	tipo	assinatura	o que faz
NoetherModelConfig	classe	(model_name, config_kwargs, device)	Minimal config wrapper forwarded verbatim to noether's own config classes.
NoetherUPT	classe	(**config_kwargs)	Universal Physics Transformer (Noether/Emmi AI).
NoetherABUPT	classe	(**config_kwargs)	Anchored-Branched Universal Physics Transformer (Noether/Emmi AI).
NoetherTransformer	classe	(**config_kwargs)	Vanilla Transformer backbone from Noether.
NoetherTransolver	classe	(**config_kwargs)	Transolver / Transolver++ Physics-Attention backbone (Noether/Emmi AI).
NoetherAeroUPT	classe	(**config_kwargs)	AeroUPT — UPT with surface/volume domain routing for aerodynamics.
NoetherAeroABUPT	classe	(**config_kwargs)	AeroABUPT — AB-UPT with surface/volume domain routing.
NoetherAeroTransformer	classe	(**config_kwargs)	AeroTransformer — Transformer for CFD aerodynamics.
NoetherAeroTransolver	classe	(**config_kwargs)	AeroTransolver — Transolver for CFD aerodynamics.

pinneapple_neural.architectures.neural_operators.pino

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/neural_operators/pino.py · 232 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicsInformedNeuralOperator	classe	(operator, *, lambda_pde=1.0, lambda_bc=1.0, lambda_ic=1.0, lambda_physics=1.0, prefix='p…)	Generic PINO wrapper.
GridSpec	classe	(L, dims)	Descreve grid uniforme (PINO clássico). L: tamanho físico por eixo (mesmo length de dims). dims: quais dims de u correspondem aos eixos (ex.: (-2, -1) para 2D).
ResidualSpec	classe	(residual, orders, method)	Spec genérico para PDE residual. residual(u=..., deriv=..., params=..., **extra) -> r onde deriv é o dict retornado por compute_derivatives.
pino_physics_fn	função	(u, *, grid, spec, params=None, bc_mask=None, bc_value=None, ic_mask=None, ic_value=None,…)	Generic PINO physics loss: - computes derivatives on grid (FFT/FD) - evaluates residual r - returns pde/bc/ic + physics_total
spectral_derivative	função	(u, *, dim, L, order=1) -> torch.Tensor	
fd_derivative	função	(u, *, dim, dx, order=1) -> torch.Tensor	
compute_derivatives	função	(u, *, grid, orders=(1, 2), method='spectral') -> Dict[Tuple[int, int], torch.Tensor]	Retorna derivadas por eixo e ordem: key = (axis_index, order) onde axis_index é o índice dentro de grid.dims/grid.L.

pinneapple_neural.architectures.neural_operators.registry

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/neural_operators/registry.py · 123 linhas · testes: 8

nome	tipo	assinatura	o que faz
NeuralOperatorCatalog	classe	(registry)	
register_into_global	função	() -> None	

pinneapple_neural.architectures.neural_operators.transolver

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/neural_operators/transolver.py · 145 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicsAttention	classe	(dim, heads=8, slices=32, dropout=0.0)	Multi-head Physics-Attention (slice -> attend -> deslice).

nome	tipo	assinatura	o que faz
`TransolverBlock`	classe	`(dim, heads, slices, mlp_ratio=2, dropout=0.0)`	
`Transolver`	classe	`(in_dim, out_dim, dim=128, depth=4, heads=8, slices=32, mlp_ratio=2, dropout=0.0)`	Point-cloud operator: (B, N, in_dim) -> (B, N, out_dim).
`TransolverLite`	classe	`(in_dim=5, out_dim=101, d_model=64, n_heads=4, n_slices=8, n_blocks=3, dropout=0.1)`	Linear input projection -> `n_blocks` slice-attention blocks -> per-node linear head.

`pinneapple_neural.architectures.neural_operators.uno`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/neural_operators/uno.py` · 494 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpectralConvND`	classe	`(dim, in_channels, out_channels, modes)`	Spectral convolution generalized to 1D/2D/3D (rfftn). Applies a learned complex linear map on low-frequency modes.
`OperatorBlockND`	classe	`(dim, width, modes)`	
`DownBlockND`	classe	`(dim, in_ch, out_ch, modes, depth)`	
`UpBlockND`	classe	`(dim, in_ch, skip_ch, out_ch, modes, depth)`	
`UNONdGrid`	classe	`(dim, in_channels, out_channels, width=64, levels=3, depth_per_level=2, modes=None, add_c…)`	U-shaped Neural Operator for regular grids (1D/2D/3D). Input: (B, Cin, *spatial) Output: (B, Cout, *spatial)
`PointKernelOperator`	classe	`(dim, in_ch, out_ch, hidden=64, k=16)`	Continuous operator on irregular points using kNN message passing with a learned kernel. x: (B, N, Cin) coords: (B, N, dim) out: (B, N, Cout)
`PointOperatorBlock`	classe	`(dim, width, k=16)`	
`UNONdPoints`	classe	`(dim, in_channels, out_channels, width=128, levels=3, depth_per_level=2, k=16, pool_ratio…)`	U-shaped operator for irregular meshes (points). This builds a multi-scale pyramid by subsampling points and using kNN operators. Input: x (B,N,Cin) coords (B,N,dim) Output: y (B,N,Cout)
`UniversalUNO`	classe	`(dim, in_channels, out_channels, *, mesh_mode='auto', width_grid=64, levels_grid=3, depth…)`	Unified U-NO module: - grid mode: (B,C,*spatial) -> (B,Cout,*spatial) - points mode: (B,N,Cin) + coords(B,N,dim) -> (B,N,Cout)

`pinneapple_neural.architectures.physics_aware`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/physics_aware/__init__.py` · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.physics_aware.base

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/physics_aware/base.py · 25 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicsAwareOutput	classe	(y, losses, extras)	
PhysicsAwareBase	classe	()	

pinneapple_neural.architectures.physics_aware.pann

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/physics_aware/pann.py · 135 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicsAwareNeuralNetwork	classe	(backbone, *, physics_loss_fn=None, physics_weight=1.0, physics_components_prefix='physic…)	Physics-Aware Neural Network:

pinneapple_neural.architectures.physics_aware.registry

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/physics_aware/registry.py · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicsAwareCatalog	classe	(registry)	
register_into_global	função	() -> None	

pinneapple_neural.architectures.physics_aware.spn

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/physics_aware/spn.py · 53 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
StructurePreservingNetwork	classe	(backbone, *, projector)	Structure-Preserving Network (MVP):

pinneapple_neural.architectures.pinns

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/pinns/__init__.py · 57 linhas · **sem teste direto**

pinneapple_neural.architectures.pinns.base

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/pinns/base.py · 261 linhas · testes: 5

nome	tipo	assinatura	o que faz
PINNOutput	classe	(y, losses, extras)	Standard output for all PINN-family models.
PINNBase	classe	()	Base class for all PINN-family models.

pinneapple_neural.architectures.pinns.bpinn

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/pinns/bpinn.py` · 389 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
BayesianLinear	classe	`(in_features, out_features, *, prior_std=1.0, init_rho=-3.0, bias=True)`	Linear layer whose weights are Gaussian random variables.
BPINNConfig	classe	`(in_dim, out_dim, hidden, activation, prior_std, init_rho, noise_pde, noise_obs, kl_weigh…)`	Configuration for BayesianPINN.
BayesianPINN	classe	`(in_dim=2, out_dim=1, hidden=(64, 64, 64), activation='tanh', *, config=None, prior_std=1…)`	Bayesian Physics-Informed Neural Network via mean-field VI.

pinneapple_neural.architectures.pinns.inverse

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/pinns/inverse.py` · 118 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
InversePINN	classe	`(in_dim, out_dim, hidden=(128, 128, 128, 128), activation='tanh', inverse_params=None, in…)`	Catalog InversePINN (returns PINNOutput), with a compatibility hook for PINNFactory.

pinneapple_neural.architectures.pinns.physics_diffusion

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/pinns/physics_diffusion.py` · 787 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
NoiseSchedulerBase	classe	`()`	Protocol for noise schedulers.
VPNoiseScheduler	classe	`(T=1000, beta_min=0.0001, beta_max=0.02)`	Variance-preserving (VP-DDPM) linear beta schedule.
EDMNoiseScheduler	classe	`(sigma_min=0.002, sigma_max=80.0, rho=7.0)`	Elucidated Diffusion Model (EDM) schedule.
SinusoidalEmbedding	classe	`(dim)`	Sinusoidal positional embedding for noise level / timestep.
FiLMLayer	classe	`(feat_dim, cond_dim)`	Feature-wise Linear Modulation — conditions activations on noise level.
MLPDenoiser	classe	`(x_dim, t_emb_dim=64, hidden=(256, 256, 256), activation='gelu', predict_x0=False)`	Time-conditioned MLP denoiser for 1-D physics fields.
DSMLoss	classe	`(denoiser, scheduler, loss_weight_fn=None)`	Denoising score-matching loss.
GuidanceBase	classe	`()`	Protocol for DPS-style guidance.

nome	tipo	assinatura	o que faz
PDEResidualGuidance	classe	(residual_fn, weight=0.1)	Physics-constrained guidance via PDE residual.
DataConsistencyGuidance	classe	(mask, y_obs, weight=0.5)	DPS guidance for data assimilation / inverse problems.
ComposedGuidance	classe	(*guidances)	Sum of multiple guidance terms.
PIDiffConfig	classe	(x_dim, t_emb_dim, hidden, activation, scheduler_type, T, beta_min, beta_max, sigma_min, …)	
PhysicsInformedDiffusion	classe	(x_dim=64, hidden=(256, 256, 256), activation='gelu', *, denoiser=None, scheduler=None, c…)	Physics-Informed Diffusion Model.

pinneapple_neural.architectures.pinns.pielm

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/pinns/pielm.py · 141 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PIELM	classe	(in_dim, out_dim, hidden_dim=1024, activation='tanh', freeze_random=True)	
PIELMFactoryAdapter	classe	(pielm, *, inverse_params_names=None, initial_guesses=None, dtype=torch.float32)	

pinneapple_neural.architectures.pinns.pinn_lstm

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/pinns/pinn_lstm.py · 75 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PINNLSTM	classe	(in_dim, out_dim, *, seq_len, hidden_dim=128, num_layers=2, dropout=0.0, inverse_params_n…)	

pinneapple_neural.architectures.pinns.pinnsformer

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/pinns/pinnsformer.py · 212 linhas · testes: 1

nome	tipo	assinatura	o que faz
PINNsFormer	classe	(in_dim, out_dim, *, seq_len=32, d_model=128, nhead=4, num_layers=4, dim_feedforward=256,…)	Transformer-based PINN.

pinneapple_neural.architectures.pinns.registry

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/pinns/registry.py · 78 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PINNCatalog	classe	(registry)	
register_into_global	função	() -> None	

pinneapple_neural.architectures.pinns.vanilla

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/pinns/vanilla.py` · 94 linhas · testes: 13

nome	tipo	assinatura	o que faz
`VanillaPINN`	classe	`(in_dim, out_dim, hidden=(128, 128, 128, 128), activation='tanh', *, inverse_params_names…)`	

pinneapple_neural.architectures.pinns.vpinn

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/pinns/vpinn.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VPINN`	classe	`(in_dim, out_dim, hidden=(128, 128, 128), activation='tanh', *, inverse_params_names=None…)`	
`pinn_factory_adapter`	função	`(loss_fn)`	

pinneapple_neural.architectures.pinns.xpinn

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/pinns/xpinn.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SubnetWrapper`	classe	`(in_dim, out_dim, hidden=(128, 128, 128, 128), activation='tanh')`	
`XPINN`	classe	`(n_subdomains, in_dim, out_dim, hidden=(128, 128, 128, 128), activation='tanh', interface…)`	

pinneapple_neural.architectures.pinns.xtfc

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/pinns/xtfc.py` · 681 linhas · testes: 4

nome	tipo	assinatura	o que faz
`TFCConstrainedExpression`	classe	`(tfc_obj, dim=1)`	Wraps the `tfc` library to create constrained expressions for 1D or nD problems that exactly satisfy boundary/initial conditions.
`XTFCConfig`	classe	`(in_dim, out_dim, rf_dim, activation, freeze_random, rf_kind, rff_sigma, use_bias, init_s…)`	
`XTFC`	classe	`(in_dim, out_dim, rf_dim=2048, activation=None, freeze_random=True, g_fn=None, B_fn=None,…)`	XtFC — eXtreme Theory of Functional Connections PINN.
`XTFCFactoryModel`	classe	`(xtfc, inverse_params_names=None, initial_guesses=None, dtype=torch.float32)`	Thin wrapper around XTFC that exposes a standard `forward(x) -> Tensor` interface compatible with PINNFactory and the Trainer.

nome	tipo	assinatura	o que faz
`tfc_available`	função	`() -> bool`	Return True if the `tfc` library is installed.
`build_xtfc`	função	`(in_dim, out_dim, *, rf_dim=2048, activation=None, freeze_random=True, g_fn=None, B_fn=No…)`	Convenience constructor for XTFC.

`pinneapple_neural.architectures.recurrent`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/recurrent/__init__.py` · 5 linhas · **sem teste direto**

`pinneapple_neural.architectures.recurrent.base`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/recurrent/base.py` · 36 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RNNOutput`	classe	`(y, losses, extras)`	Standard output for recurrent models in Pinneapple.
`RecurrentModelBase`	classe	`()`	Base class for time-series recurrent models.

`pinneapple_neural.architectures.recurrent.gru`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/recurrent/gru.py` · 199 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GRUModel`	classe	`(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, *, pool='last', tim…)`	GRU forecaster (baseline+).
`BiGRUModel`	classe	`(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, *, pool='last', tim…)`	Bidirectional GRU forecaster (baseline+).

`pinneapple_neural.architectures.recurrent.lstm`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/recurrent/lstm.py` · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LSTMModel`	classe	`(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, *, pool='last', tim…)`	LSTM forecaster (baseline+).
`BiLSTMModel`	classe	`(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, *, pool='last', tim…)`	Bidirectional LSTM forecaster (baseline+).

`pinneapple_neural.architectures.recurrent.registry`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/recurrent/registry.py` · 57 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RecurrentCatalog`	classe	`(registry)`	
`register_into_global`	função	`() -> None`	

`pinneapple_neural.architectures.recurrent.seq2seq`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/recurrent/seq2seq.py` · 215 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Seq2SeqRNN`	classe	`(in_dim, out_dim, horizon, hidden_dim=128, num_layers=2, dropout=0.0, cell='gru', future_…)`	Seq2Seq RNN forecaster.

`pinneapple_neural.architectures.register_all`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/register_all.py` · 44 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register_all`	função	`() -> None`	Populate ModelRegistry with every family. Idempotent — safe to call multiple times.

`pinneapple_neural.architectures.registry`

ModelRegistry and ModelSpec for model registration and lookup.

`pinneapple_neural/architectures/registry.py` · 101 linhas · testes: 64

nome	tipo	assinatura	o que faz
`ModelSpec`	classe	`(name, family, cls, description, tags, input_kind, supports_physics_loss, expects, predic…)`	
`ModelRegistry`	classe	`(_REGISTRY)`	Central registry for all Pinneapple models.

`pinneapple_neural.architectures.reservoir_computing`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/reservoir_computing/__init__.py` · 5 linhas · **sem teste direto**

`pinneapple_neural.architectures.reservoir_computing.base`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/reservoir_computing/base.py` · 48 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RCOutput`	class e	`(y, losses, extras)`	Standard output for reservoir computing models.

nome	tipo	assinatura	o que faz
RCBase	class e	()	

pinneapple_neural.architectures.reservoir_computing.elm

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/reservoir_computing/elm.py · 88 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ExtremeLearningMachine	classe	(in_dim, out_dim, hidden_dim=1024, activation='tanh', l2=1e-06, freeze_random=True)	Extreme Learning Machine (ELM): - Random hidden layer features: H = phi(X W + b) - Solve output weights with ridge regression (closed-form)

pinneapple_neural.architectures.reservoir_computing.esn

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/reservoir_computing/esn.py · 340 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
EchoStateNetwork	classe	(in_dim, out_dim, reservoir_dim=1024, spectral_radius=0.9, leak=1.0, input_scale=1.0, l2=…)	Echo State Network (ESN): h_t = (1-leak)*h_{t-1} + leak*tanh(x_t W_in + h_{t-1} W) y_t = phi(h_t, x_t) W_out (W_out trained by ridge)

pinneapple_neural.architectures.reservoir_computing.esn_rc

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/reservoir_computing/esn_rc.py · 113 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ESNRC	classe	(in_dim, out_dim, reservoir_dim=1024, *, spectral_radius=0.9, leak=1.0, input_scale=1.0, …)	ESN-RC (Reservoir Computing) variant: - Uses ESN reservoir but augments readout with nonlinear transforms (e.g., h, h^2, h^3) to approximate richer dynamics while keeping closed-form ridge training.

pinneapple_neural.architectures.reservoir_computing.hybrid_rbf

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/reservoir_computing/hybrid_rbf.py · 158 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
HybridRBFNetwork	classe	(in_dim, out_dim, num_centers=512, sigma=None, l2=1e-06, learn_centers=False, use_linear=…)	Hybrid RBF Network: y = [Phi(x), x, 1] W where W is solved by ridge regression in a single linear system.

pinneapple_neural.architectures.reservoir_computing.koopman

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/reservoir_computing/koopman.py · 155 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
KoopmanOperator	classe	(in_dim, *, lift_dim=1024, lift='rff', poly_degree=2, rff_lengthscale=1.0, l2=1e-06, use_…)	Koopman Operator (EDMD-style MVP):

pinneapple_neural.architectures.reservoir_computing.rbf

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/reservoir_computing/rbf.py · 263 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
RBFNetwork	classe	(in_dim, out_dim, num_centers=512, sigma=None, l2=1e-06, learn_centers=False, *, normaliz…)	RBF Network (Gaussian features) + ridge-regression readout.
sigma_is_default	função	(log_sigma, eps=1e-06) -> bool	

pinneapple_neural.architectures.reservoir_computing.registry

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/reservoir_computing/registry.py · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ReservoirCatalog	classe	(registry)	
register_into_global	função	() -> None	

pinneapple_neural.architectures.rom

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/rom/__init__.py · 39 linhas · testes: 25

pinneapple_neural.architectures.rom.base

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/rom/base.py · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ROMOutput	classe	(y, losses, extras)	Output for Reduced Order Models.
ROMBase	classe	()	

pinneapple_neural.architectures.rom.contracts

ROM study results as the apps' open contracts (`pinneapple_data.contracts`): a `ModelRef` per exported reduced model and a `StudyResult` for the study, so a ROM built in the ROM Lab is an input of the uncertainty, optimisation, twin or gateway apps.

`pinneapple_neural/architectures/rom/contracts.py` · 120 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`model_ref`	função	`(rm, *, name, uri, payload=None, row=None, split='test', dataset_hash='', dataset_name=''…)`	`ModelRef` of a `ReducedModel`: inputs with their training range, outputs (the fields), the held-out metrics of its study row and the SHA-256 of the file at `uri` (`payload`: its bytes).
`rom_checks`	função	`(res, tolerance) -> list`	Checks of a study, each with its value: the best ROM against its baseline (nearest snapshot or persistence), against the tolerance, and divergence of the time steppers.
`study_result`	função	`(res, *, study_id, title, tolerance, models=(), artifacts=(), limits=(), app='rom', datas…)`	`StudyResult` of a ROM study: one table row per (method, rank), the best model, checks with values.

`pinneapple_neural.architectures.rom.deep_uq_rom`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/rom/deep_uq_rom.py` · 61 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DeepUQROM`	classe	`(r, hidden=256, layers=2, dropout=0.0)`	Deep UQ-ROM (MVP scaffold): - A small latent predictor that outputs mean and log-variance.

`pinneapple_neural.architectures.rom.dmd`

Dynamic Mode Decomposition (DMD) — batch-capable, stabilised, with spectral analysis.

`pinneapple_neural/architectures/rom/dmd.py` · 346 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DynamicModeDecomposition`	classe	`(r=64, center='global', l2=0.0)`	Truncated (rank-r) DMD in reduced space.
`DMDc`	classe	`(affine=False)`	Dynamic Mode Decomposition with control (Proctor, Brunton & Kutz 2016), NumPy.

`pinneapple_neural.architectures.rom.havok`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/rom/havok.py` · 225 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HAVOK`	classe	`(delays=50, r=64, center=True, svd_eps=1e-12, decode_mode='last')`	HAVOK (MVP-ish, closer to literature): 1) build Hankel (delay embedding) from scalar or low-dim observations 2) center + SVD of Hankel and project to low-rank subspace (HAVOK core) 3) fit linear dynamics in that subspace via DMD (no forced stitching across trajectories) 4) rollout in subspace; ...

`pinneapple_neural.architectures.rom.koopman`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/rom/koopman.py` · 269 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`KoopmanOperator`	classe	`(k_dim, structure='full', num_blocks=4)`	Learnable Koopman operator K.
`KoopmanAutoencoder`	classe	`(in_dim, k_dim=128, hidden=128, n_layers=3, activation='tanh', k_structure='full', num_bl…)`	Deep Koopman autoencoder: learned lifting + linear dynamics.

`pinneapple_neural.architectures.rom.neural_rom`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/rom/neural_rom.py` · 237 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeuralROM`	classe	`(in_dim, r=32, hidden_enc=128, n_layers_enc=3, dynamics='mlp', hidden_dyn=128, n_layers_d…)`	Fully neural ROM: nonlinear encoder + latent dynamics + nonlinear decoder.

`pinneapple_neural.architectures.rom.opinf`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/rom/opinf.py` · 387 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OperatorInference`	classe	`(r, *, use_quadratic=True, use_bias=True, l2_linear=1e-06, l2_quad=1e-06, l2_bias=0.0, ce…)`	Operator Inference (OpInf) for latent dynamics.

`pinneapple_neural.architectures.rom.parametric`

Parametric reduced-order models: design parameters -> full field, without running the solver.

`pinneapple_neural/architectures/rom/parametric.py` · 198 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ParametricPOD`	classe	`(r, energy, regressor, kernel, smoothing, normalize, seed, mean_, basis_, sv_)`	POD basis + coefficient regression. `r`: largest number of modes; `energy`: keep the modes holding this fraction of the snapshot variance (whichever is smaller). Parameters are scaled to [0, 1] on the training box. `normalize`: split each snapshot into its amplitude (its L2 norm, regressed as …
`latin_hypercube`	função	`(n, bounds, seed=0) -> tuple[np.ndarray, list[str]]`	`n` points of a Latin hypercube in the box `bounds` (name -> (lo, hi)). Returns (points, names).

`pinneapple_neural.architectures.rom.pod`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/rom/pod.py` · 252 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
POD	classe	(r=64, center=True, energy=None, randomised=False, random_seed=0)	POD via full or randomised truncated SVD for ROM compression.

pinneapple_neural.architectures.rom.registry

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/rom/registry.py · 82 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ROMCatalog	classe	(registry)	
register_into_global	função	() -> None	

pinneapple_neural.architectures.rom.rom_hybrid

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/rom/rom_hybrid.py · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ROMHybrid	classe	(field_dim, r=64, l2=1e-06, use_corrector=True, corrector_hidden=64)	Hybrid ROM combining three components:

pinneapple_neural.architectures.rom.sindy

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/rom/sindy.py · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SINDy	classe	(r, *, library='poly', poly_degree=2, threshold=0.05, n_iter=10, l2=0.0001, solver='stlsq…)	SINDy for latent dynamics.

pinneapple_neural.architectures.rom.steppers

One-step rollout functions of linear reduced-order models fitted on snapshot trajectories: `step(u) -> u_next` in the full space, the shape `pp.validate(step, {"rollout": ...})` audits (horizon, drift, spectrum, invariants).

pinneapple_neural/architectures/rom/steppers.py · 48 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
pod_dmd_stepper	função	(snapshots, r=8)	POD (rank `r`, centred) of `snapshots (n_t, *shape)` and exact DMD of its coefficients. Returns `(step, max_abs_eig)`: `step(u)` advances one snapshot interval in the full space; `max_abs_eig` is the largest DMD eigenvalue modulus (above 1: the ROM amplifies).
pod_opinf_stepper	função	(train, energy) -> tuple[Callable[[np.ndarray], np.ndarray], int]	POD of `train (n_traj, n_t, n_x)` keeping `energy` of the snapshot energy, and linear operator inference (no quadratic term, no bias) on the coefficient trajectories. Returns `(step, rank)`.

pinneapple_neural.architectures.rom.study

Reduced-order model study: which ROM, at which rank, for which error and speed, measured on held-out snapshots.

pinneapple_neural/architectures/rom/study.py · 787 linhas · testes: 11 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Split	classe	(train, test, how)	
ReducedModel	classe	(method, mode, rank, arrays, config, fit_s, _cache)	A fitted ROM: predict and a portable .npz form (to_bytes / from_bytes).
ROMStudyResult	classe	(mode, split, singular_values, rows, projection, skipped, budget_s, hf_seconds, metric, m…)	
default_ranks	função	(n_train, n_values, sv=None, max_rank=64) -> list[int]	1, 2, 4, 8, ... up to the data's rank, plus the ranks holding 99 %, 99.9 % and 99.99 % of the energy.
split_snapshots	função	(snaps, *, how='auto', test_fraction=0.2, seed=0, test_idx=None) -> Split	Held-out snapshots. Time series: forecast (the last test_fraction of the times, default) or interleaved (every k-th time inside the window). Parametric: random (seeded). test_idx overrides.
field_scales	função	(snaps, train) -> np.ndarray	One positive scale per value: the RMS of each field over the training snapshots, so fields in different units (a displacement in mm and a stress in MPa) weigh the same in the POD.
fit_reduced_model	função	(snaps, train, method, rank=None, *, budget_s=5.0, seed=0, mode=None, scale='auto', log_p…)	Fit one ROM on the snapshots train (indices). rank: number of POD modes (ignored by nearest and the parametric neural). budget_s: wall-clock budget of the neural surrogate. scale: "auto" (each field divided by its training RMS when there are several fields), ...
rom_study	função	(snaps, *, methods=None, ranks=None, split='auto', test_fraction=0.2, test_idx=None, budg…)	Fit every method at every rank on the training snapshots and score it on the held-out ones (see the module docstring). hf_seconds: wall time of the full-order model per snapshot (default: snaps.meta). scale: with several fields ("auto") each is divided by its training RMS before the ...

pinneapple_neural.architectures.rom.toy_systems

Synthetic linear dynamical systems for ROM demonstrations and tests (POD compression, DMD rollout).

pinneapple_neural/architectures/rom/toy_systems.py · 27 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
make_linear_system	função	(D, seed=0) -> torch.Tensor	A contractive random linear operator A of shape (D, D): a Gaussian matrix scaled by 1 / sqrt(D) whose singular values are rescaled to a largest value of 0.95 (so x_{k+1} = A x_k decays).

nome	tipo	assinatura	o que faz
generate_sequences	função	`(B, T, D, A, noise=0.01, seed=123) -> torch.Tensor`	`B` trajectories of `T` steps of `x_{k+1} = A x_k + noise * N(0, I)` from Gaussian initial states in `D` dimensions, shape `(B, T, D)`; deterministic for a given `seed`.

pinneapple_neural.architectures.rom.trusted

Latent surrogate that says when not to trust it: error bounds, envelope check and physics residual per query.

`pinneapple_neural/architectures/rom/trusted.py` · 192 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TrustedLatentSurrogate	classe	`(rom, spread, members, subsample, neighbours, alpha, alpha_envelope, alpha_residual, resi…)`	POD latent surrogate + per-query error bound, envelope p-value and physics residual.

pinneapple_neural.architectures.rom.views

A held-out snapshot on its body: the full-order field, a prediction (a ROM, a surrogate) and their difference on the mesh, with the deformed shape when the snapshots carry a displacement, the boundary-condition and load regions of the dataset, the error hotspots and a short text on where the error …

`pinneapple_neural/architectures/rom/views.py` · 122 linhas · testes: 11

nome	tipo	assinatura	o que faz
zones	função	`(snaps, points, grow=0.1) -> dict[str, np.ndarray]`	The dataset's named node sets grown by `grow` times the body size ("clamp zone", "loaded tip zone") and the remaining nodes ("rest of the body"): the regions errors are compared over.
reconstruction_view	função	`(snaps, model, index, *, reported=None, grow=0.1, max_faces=None, n_hotspots=3, full_labe…)`	View of snapshot `index` of `snaps` against `model` (a `ReducedModel`): :func:`prediction_view` of the model's prediction at the snapshot's time or parameters.
prediction_view	função	`(snaps, index, pred, *, label='Prediction', note='', reported=None, grow=0.1, max_faces=N…)`	View of snapshot `index` of `snaps` against `pred`, a predicted snapshot row in the layout of `snaps` (a ROM, a surrogate). Named node sets of the dataset become markers (kind `bc` / `load` / `region`) and, grown by `grow` times the body size, the regions the error is compared over …

pinneapple_neural.architectures.siren

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/siren.py` · 135 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SineLayer	classe	`(in_features, out_features, bias=True, is_first=False, omega_0=30.0)`	Single sinusoidal layer with principled weight initialization.

nome	tipo	assinatura	o que faz
SIREN	classe	`(in_dim, out_dim, hidden_dim=256, n_layers=5, omega_0=30.0, outermost_linear=True, final_…)`	Sinusoidal Representation Network for physics-informed learning.

pinneapple_neural.architectures.transformers

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/transformers/__init__.py` · 5 linhas · **sem teste direto**

pinneapple_neural.architectures.transformers.autoformer

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/transformers/autoformer.py` · 367 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SeriesDecomp	classe	`(kernel_size=25)`	Moving-average decomposition: x = seasonal + trend.
AutoCorrelation	classe	`(topk_factor=1.0, dropout=0.0)`	Autoformer-style Auto-Correlation: - compute cross-correlation between Q and K via FFT - pick top-k delays - aggregate V by shifting (time-delay aggregation)
AutoCorrelationLayer	classe	`(d_model, nhead, topk_factor=1.0, dropout=0.0)`	Multi-head wrapper with projections like attention.
EncoderLayer	classe	`(d_model, nhead, d_ff, kernel_size, dropout, topk_factor)`	
DecoderLayer	classe	`(d_model, nhead, d_ff, kernel_size, dropout, topk_factor)`	
Autoformer	classe	`(in_dim, out_dim, horizon, d_model=128, nhead=4, num_layers=2, kernel_size=25, dropout=0.…)`	Autoformer (minimal faithful): - Decompose input into seasonal/trend - Encoder: AutoCorrelation + decomposition - Decoder: AutoCorrelation (self + cross) + decomposition, accumulates trend - Output: y = seasonal_out + trend_out (projected to out_dim)

pinneapple_neural.architectures.transformers.base

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/transformers/base.py` · 48 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
TSOutput	class e	`(y, losses, extras)`	Standard output for time-series transformers in Pinneapple.
TimeSeriesModelBase	class e	`()`	Base class for sequence forecasting models.

pinneapple_neural.architectures.transformers.fedformer

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/transformers/fedformer.py · 89 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FEDformer	classe	(in_dim, out_dim, horizon, d_model=128, topk_freq=32)	Fast baseline inspired by FEDformer ideas: - learn a 1D projection for spectral features (cheap multivariate proxy) - select top-k frequency bins (batch-consistent) - mix last-token embedding + spectral features with a small MLP - horizon embedding to avoid constant repeated outputs

pinneapple_neural.architectures.transformers.informer

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/transformers/informer.py · 132 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SinusoidalPositionalEncoding	classe	(d_model, max_len=10000)	
Informer	classe	(in_dim, out_dim, horizon, d_model=128, nhead=4, num_layers=4, dim_feedforward=256, dropo…)	Efficient encoder-only baseline: - Transformer encoder (standard attention) - Optional distilling (Conv1d stride=2) on early layers only - Attention pooling to get sequence context - Forecast head uses context + (optional) known future covariates per horizon

pinneapple_neural.architectures.transformers.registry

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/transformers/registry.py · 61 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TransformerCatalog	classe	(registry)	
register_into_global	função	() -> None	

pinneapple_neural.architectures.transformers.tft

Módulo sem docstring e sem texto.

pinneapple_neural/architectures/transformers/tft.py · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GRN	classe	(d, hidden, dropout=0.0)	Gated Residual Network (lite): y = LayerNorm(x + sigmoid(Wg x) * Drop(W2 gelu(W1 x)))

nome	tipo	assinatura	o que faz
`GLU`	classe	`(d)`	Gated Linear Unit used for skip gating.
`GroupVSN`	classe	`(d_model, num_groups, hidden, dropout=0.0)`	Variable selection network (lite) over groups, not per-variable.
`TemporalFusionTransformer`	classe	`(in_dim, out_dim, horizon, d_model=128, nhead=4, num_enc_layers=2, num_dec_layers=2, drop…)`	TFT-lite (strong, honest): - Input projection - Grouped variable selection (past groups + optional future group) - Causal TransformerEncoder over past - TransformerDecoder with learnable future queries (multi-step) attending to past memory - GRN/GLU gating around blocks for stability and TFT-like …

`pinneapple_neural.architectures.transformers.timesnet`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/transformers/timesnet.py` · 199 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TimesNet`	classe	`(in_dim, out_dim, horizon, d_model=128, num_blocks=2, top_k_periods=3, head_window=64, dr…)`	TimesNet-like forecasting: 1) FFT period discovery (Top-K) 2) Reshape into 2D maps per period 3) 2D inception blocks (multi-scale) 4) Weighted aggregation across periods 5) Flatten head for horizon prediction

`pinneapple_neural.architectures.transformers.transformer`

Módulo sem docstring e sem texto.

`pinneapple_neural/architectures/transformers/transformer.py` · 113 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VanillaTransformer`	classe	`(in_dim, out_dim, d_model=128, nhead=4, num_layers=4, dim_feedforward=256, dropout=0.1, p…)`	Causal Transformer encoder for autoregressive forecasting.

`pinneapple_neural.callbacks`

Physics checks during training in PyTorch Lightning, PhysicsNeMo and Hugging Face (#704).

`pinneapple_neural/callbacks.py` · 274 linhas · testes: 4

nome	tipo	assinatura	o que faz
`PhysicsWarning`	classe	`()`	A physics metric worsened for several consecutive checks while the training loss improved.
`PhysicsMonitor`	classe	`(problem, data=None, *, reference=None, query=None, level='quick', every=100, patience=3,…)`	The checks of `pp.validate(model, problem, data, reference, query, level)` as training metrics.

nome	tipo	assinatura	o que faz
PhysicsNeMoHook	classe	(monitor)	For a PhysicsNeMo training loop: hook(model, step, loss, log=...) after each optimiser step.

pinneapple_neural.lpm

Large Physics Model (LPM) building blocks: one model for a whole design space.

pinneapple_neural/lpm.py · 212 linhas · testes: 5

nome	tipo	assinatura	o que faz
FourierEncoding	classe	(n_freq=6, include_input=True)	x -> [x, sin(2^k pi x), cos(2^k pi x)] for k = 0..n_freq-1 (per coordinate).
GeometryEncoder	classe	(d, width=64, code=32)	DeepSets: per-point MLP on (position, normal) then mean+max pooling -> geometry code.
Design	classe	(points, normals, query, params, target)	One simulation of the design space: its geometry, operating point and a target field.
LargePhysicsModel	classe	(dim, n_fields, n_params=0, radii=(0.1, 0.4, 1.2), n_freq=6, width=128, code=32, n_heads=…)	
multiscale_neighbourhood	função	(query, points, normals, radii)	Per query point and radius: mean offset to neighbours, mean neighbour normal, occupancy.
fit	função	(model, designs, *, epochs=2000, lr=0.002, freeze_geometry=False, seed=0) -> list[float]	Train (or fine-tune with freeze_geometry=True) on a list of designs.

pinneapple_neural.physicsnemo_bridge

pinneapple_neural.physicsnemo_bridge -- PhysicsNeMo <-> PINNeAPPle integration glue.

pinneapple_neural/physicsnemo_bridge/__init__.py · 86 linhas · testes: 19

pinneapple_neural.physicsnemo_bridge.active_learning_bridge

PINNeAPPle active learning -> PhysicsNeMo retrain bridge.

pinneapple_neural/physicsnemo_bridge/active_learning_bridge.py · 226 linhas · testes: 19

nome	tipo	assinatura	o que faz
RetrainConfig	classe	(n_rounds, n_epochs_per_round, batch_size, lr, selection_mode, device, verbose)	Configuration for the "retrain" half of the AL <-> retrain loop.
ALRoundResult	classe	(round_index, n_points_added, n_points_total, mean_train_loss, train_losses)	Bookkeeping for a single AL-select + retrain round.
ALRetrainResult	classe	(rounds, x_collocation, final_loss)	Full result of :func:active_learning_retrain_loop.
active_learning_retrain_loop	função	(model, loss_fn, residual_fn, bounds, *, x_initial=None, al_config=None, retrain_config=N…)	Run the "AL selects -> retrains -> AL selects -> ..." loop (Padrao 2).

pinneapple_neural.physicsnemo_bridge.adapters

PhysicsNeMo -> PINNeAPPle model adapters (UQ + Digital Twin).

`pinneapple_neural/physicsnemo_bridge/adapters.py` · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsNemoModelAdapter`	classe	`(model, field_names) -> None`	Wrap a trained model so its `forward()` output is one plain tensor.
`is_physicsnemo_available`	função	`() -> bool`	Return `True` if `physicsnemo` (or the legacy `modulus` name) can be imported.
`require_physicsnemo_model`	função	`(model) -> Any`	Validate that `model` is a physicsnemo-compatible trained model.
`wrap_for_uq`	função	`(model, field_names=None, *, mc_dropout_config=None) -> MCDropoutWrapper`	Wrap a trained model for MC-Dropout uncertainty quantification.
`build_digital_twin_for_model`	função	`(model, field_names, coord_names=None, *, config=None) -> DigitalTwin`	Wrap a trained model into a :class:`DigitalTwin`.

pinneapple_neural.physicsnemo_bridge.field_operators

PhysicsNeMo field operators on a structured grid -> PINNeAPPle predictors (pattern 1: PhysicsNeMo trains, PINNeAPPle checks).

`pinneapple_neural/physicsnemo_bridge/field_operators.py` · 241 linhas · testes: 3

nome	tipo	assinatura	o que faz
`GridFieldPredictor`	classe	`(module, config, name)`	A trained PhysicsNeMo model on an `n x n` grid as a NumPy predictor `predict(a) -> u`.
`TrainingResult`	classe	`(run, epochs, final_loss, history)`	What :func:`train_grid_operator` spent and reached.
`grid_graph`	função	`(n) -> tuple[np.ndarray, np.ndarray]`	Graph of an `n x n` cell-centred grid: `edge_index (2, E)` (both directions, node `i * n + j`) and `edge_attr (E, 3) = (dx, dy, |d|)` of each edge in cell units.
`physicsnemo_fno`	função	`(n, *, latent_channels=32, num_fno_layers=4, num_fno_modes=12, decoder_layers=1, decoder_…)`	A 2-D `physicsnemo.models.fno.FNO` (coefficient -> solution) wrapped as a :class:`GridFieldPredictor`.
`physicsnemo_meshgraphnet`	função	`(n, *, processor_size=15, hidden_dim=64, aggregation='sum', seed=0) -> GridFieldPredictor`	A `physicsnemo.models.meshgraphnet.MeshGraphNet` on the graph of the `n x n` grid (:func:`grid_graph`).
`train_grid_operator`	função	`(pred, a, u, budget, *, batch_size=8, lr=0.001, weight_decay=1e-05, seed=0, evaluate=None…)`	Train `pred` on `(a, u)` pairs until `budget` (a `ComputeBudget`) is spent.

pinneapple_neural.predictor

Módulo sem docstring e sem texto.

`pinneapple_neural/predictor/__init__.py` · 69 linhas · **sem teste direto**

pinneapple_neural.predictor.infer

Inference utilities for trained PINN and neural network models.

`pinneapple_neural/predictor/infer.py` · 190 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
InferenceResult	classe	(coords, fields, grid_shape, model_id, metadata)	Container for model inference output.
batched_inference	função	(model, x, *, batch_size=4096, device=None, use_amp=False, amp_dtype=torch.float16) -> to…)	Run model inference over a large tensor in batches to avoid OOM.
infer	função	(model, x, *, device='cpu', field_names=None) -> np.ndarray	Run a forward pass on numpy input.
infer_on_grid_1d	função	(model, x_range, t_range, *, nx=200, nt=200, device='cpu', field_names=None, coord_names=…)	Inference on a 1D+time rectangular grid.
infer_on_grid_2d	função	(model, x_range, y_range, *, nx=100, ny=100, device='cpu', field_names=None, coord_names=…)	Inference on a 2D rectangular grid.

pinneapple_neural.predictor.postprocess

Production-quality 3D field visualization: streamlines, isosurfaces, volume rendering.

`pinneapple_neural/predictor/postprocess.py` · 762 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FlowVisualizer	classe	(model, coord_ranges, device='cpu') -> None	High-level flow visualization class.
compute_streamlines	função	(u_field, v_field, x_grid, y_grid, seed_points=None, n_seeds=20, max_length=1.0, dt=0.01)…)	Compute 2D streamlines via RK4 integration.
compute_isosurface	função	(field_3d, x, y, z, level=0.0) -> Tuple[np.ndarray, np.ndarray]	Extract isosurface from 3D scalar field using marching cubes.
plot_streamlines_2d_from_model	função	(model, x_range=(0.0, 1.0), y_range=(0.0, 1.0), n_grid=50, n_seeds=30, figsize=(10, 6), t…)	Plot 2D streamlines of velocity field predicted by a PINN model.
plot_isosurface_3d	função	(model, x_range=(0.0, 1.0), y_range=(0.0, 1.0), z_range=(0.0, 1.0), n_grid=30, level=0.0,…)	Plot 3D isosurface of a scalar field predicted by a PINN model.
plot_volume_slice	função	(model, axis='z', slice_val=0.5, x_range=(0.0, 1.0), y_range=(0.0, 1.0), z_range=(0.0, 1.…)	Plot a 2D slice of a 3D field predicted by a PINN model.

pinneapple_neural.predictor.visualize

Visualization utilities for PINN inference results.

`pinneapple_neural/predictor/visualize.py` · 1.103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_field_1d	função	(result, field_name, *, ax=None, title=None, cmap='RdBu_r', save_path=None, show=False) -…)	2D colormesh plot of a 1D+time field (x vs t).
plot_field_2d	função	(result, field_name, *, ax=None, title=None, cmap='viridis', save_path=None, show=False) …)	Colormesh plot of a 2D field (x vs y).

nome	tipo	assinatura	o que faz
`plot_error_map_1d`	função	`(result, reference, field_name, *, ax=None, title=None, save_path=None, show=False) -> Any`	Plot absolute error |prediction - reference| on a 1D+time grid.
`plot_error_map_2d`	função	`(result, reference, field_name, *, ax=None, title=None, save_path=None, show=False) -> Any`	Plot absolute error for a 2D field.
`plot_loss_curve`	função	`(history, *, keys=None, ax=None, title='Training loss', log_scale=True, save_path=None, s…)`	Plot training/validation loss curves from a history list.
`plot_model_comparison_1d`	função	`(results, field_name, *, reference=None, title=None, cmap='RdBu_r', save_path=None, show=…)`	Side-by-side comparison of multiple models on a 1D+time field.
`plot_model_comparison_2d`	função	`(results, field_name, *, reference=None, title=None, cmap='viridis', save_path=None, show…)`	Side-by-side comparison for 2D fields.
`render_visualizations`	função	`(viz_cfgs, *, model_results, problem_spec=None, reference_result=None, out_dir='.') -> Di…)`	Dispatch visualization from YAML-driven config list.
`plot_velocity_slice`	função	`(u, x, y, z, *, slice_axis='z', slice_idx=None, component=0, title=None, cmap='RdBu_r', a…)`	2D colormesh of a velocity slice through a 3D field.
`plot_velocity_magnitude_slice`	função	`(u, x, y, z, *, slice_axis='z', slice_idx=None, title=None, cmap='viridis', ax=None, save…)`	Plot |u| = sqrt(u^2 + v^2 + w^2) on a 2D slice.
`plot_streamlines_2d`	função	`(u, v, x, y, *, density=1.5, title=None, background_field=None, cmap='viridis', ax=None, …)`	Streamline plot for 2D velocity field.
`plot_centerline_velocity`	função	`(u_centerline, coord, *, reference_coord=None, reference_u=None, reference_label='Ghia et…)`	1D centerline velocity profile with optional literature reference overlay.
`plot_vorticity_slice`	função	`(u, v, x, y, z, *, slice_axis='z', slice_idx=None, title=None, cmap='RdBu_r', ax=None, sa…)`	Plot z-component of vorticity (omega_z = dv/dx - du/dy) on a 2D slice.
`plot_internal_flow_summary`	função	`(solver_output, *, time_step=-1, slice_axis='z', title='Internal Flow Summary', save_path…)`	Multi-panel summary figure for 3D internal flow (LDC or channel).
`plot_design_opt_convergence`	função	`(history_objectives, *, title=…, xlabel='Iteration', ylabel='Objective', log_scale=False,…)`	Plot convergence curve of objective function over optimization iterations.
`plot_pareto_front_2d`	função	`(objectives, pareto_mask, *, xlabel='Objective 1', ylabel='Objective 2', title='Pareto Fr…)`	Scatter plot with Pareto-optimal points highlighted.

`pinneapple_neural.trainer`

Módulo sem docstring e sem texto.

`pinneapple_neural/trainer/__init__.py` · 191 linhas · testes: 1

`pinneapple_neural.trainer.adaptive_sweep`

Automatic (adaptive) hyperparameter search.

`pinneapple_neural/trainer/adaptive_sweep.py` · 178 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AdaptiveSweepConfig`	classe	`(param_space, n_trials, direction, seed, pruning)`	
`run_adaptive_sweep`	função	`(trial_fn, cfg) -> List[Dict[str, Any]]`	Run an adaptive (Bayesian/TPE via Optuna, or random-search fallback) hyperparameter search.

`pinneapple_neural.trainer.advisor`

Módulo sem docstring e sem texto.

`pinneapple_neural/trainer/advisor.py` · 378 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Suggestion`	classe	`(code, priority, description, config_patch, rationale)`	A single actionable improvement recommendation.
`DiagnosticReport`	classe	`(model_name, signals, suggestions, metrics_summary, diagnosis_text)`	Complete diagnostic output for one model run.
`TrainingAdvisor`	classe	`(plateau_rel_std_thresh=0.005, divergence_rel_thresh=-0.05, slow_start_thresh=0.05, high_…)`	Inspects training histories and final metrics to produce ranked Suggestion objects for improving the next training run.

`pinneapple_neural.trainer.audit`

Auditability: seed, deterministic mode, RunLogger, env fingerprint.

`pinneapple_neural/trainer/audit.py` · 76 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RunLogger`	classe	`(out_dir, run_name, flush_every)`	JSONL logger for industry-grade auditability.
`set_seed`	função	`(seed) -> None`	
`set_deterministic`	função	`(deterministic=True) -> None`	
`env_fingerprint`	função	`() -> Dict[str, Any]`	

`pinneapple_neural.trainer.callbacks`

Training callbacks: EarlyStopping and ModelCheckpoint.

`pinneapple_neural/trainer/callbacks.py` · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EarlyStopping`	classe	`(monitor, patience, min_delta, mode, best, bad_epochs, stop)`	
`ModelCheckpoint`	classe	`(path, monitor, mode, best)`	

`pinneapple_neural.trainer.causal`

Causal training for temporal PDEs.

`pinneapple_neural/trainer/causal.py` · 408 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CausalWeightScheduler`	classe	`(n_time_chunks=100, epsilon=1.0, update_every=100)`	Compute per-point causal weights for temporal PDE training.
`CausalPINNTrainer`	classe	`(model, epsilon=1.0, n_time_chunks=50, update_every=100)`	PINN trainer with causal weighting for temporal PDEs.

`pinneapple_neural.trainer.checkpoint`

Checkpoint save/load for model, optimizer, config, and normalizers.

`pinneapple_neural/trainer/checkpoint.py` · 40 linhas · testes: 3

nome	tipo	assinatura	o que faz
`Checkpoint`	classe	`(model_state, optim_state, cfg, meta, normalizers)`	
`save_checkpoint`	função	`(path, ckpt) -> None`	
`load_checkpoint`	função	`(path) -> Checkpoint`	

`pinneapple_neural.trainer.collocation`

Adaptive collocation point sampling for PINN training.

`pinneapple_neural/trainer/collocation.py` · 413 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AdaptiveCollocationConfig`	classe	`(n_active, n_pool, resample_every, residual_frac, residual_batch, replace)`	Configuration for `AdaptiveCollocationSampler`.
`AdaptiveCollocationSampler`	classe	`(pool, config=None, residual_fn=None) -> None`	Residual-based Adaptive Refinement (RAR) for PINN collocation points.
`estimate_channel_scales`	função	`(Y, quantile=0.95, min_scale=1e-06, channel_names=None) -> List[float]`	Estimate per-channel output scales as the q-th quantile of |Y_i|.
`filter_nan_rows`	função	`(*tensors, warn=True) -> Tuple[torch.Tensor, ...]`	Remove rows where any tensor has NaN or Inf.

`pinneapple_neural.trainer.datamodule`

DataModule, loaders, and item adapters for training.

`pinneapple_neural/trainer/datamodule.py` · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ItemAdapter`	classe	`()`	Adapter that standardizes how we convert dataset items into dict batches.
`FnAdapter`	classe	`(x_fn, y_fn, meta_fn=None)`	Convenience adapter for Pinneapple PhysicalSample-like objects (or any object).
`AdaptedSequenceDataset`	classe	`(samples, adapter)`	Wrap a Python sequence and apply an adapter in **getitem**. Useful for: list[PhysicalSample] or list[dict] etc.

nome	tipo	assinatura	o que faz
`DataModule`	classe	`(dataset, split, group_ids, batch_size, num_workers, pin_memory, persistent_workers, drop…)`	Builds train/val/test DataLoaders with consistent split + preprocess.

`pinneapple_neural.trainer.distributed`

Distributed Data Parallel (DDP) training for PINNeAPPle.

`pinneapple_neural/trainer/distributed.py` · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DDPTrainerConfig`	classe	`(backend, world_size, master_addr, master_port, gradient_sync_every, find_unused_paramete…)`	Configuration for DDP training.
`DDPPINNTrainer`	classe	`(model, config=None)`	Multi-GPU PINN trainer using PyTorch DDP.
`is_distributed`	função	`() -> bool`	Return True when running inside an active distributed process group.
`get_rank`	função	`() -> int`	Return the rank of the current process (0 when not distributed).
`get_world_size`	função	`() -> int`	Return the total number of processes (1 when not distributed).

`pinneapple_neural.trainer.graybox`

Gray-box PINN training (AI-Aristotle style): a main network learns the state variables while one or more small auxiliary networks learn an unknown functional term in an otherwise-known governing equation.

`pinneapple_neural/trainer/graybox.py` · 280 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GrayBoxNet`	classe	`(n_in=1, n_out=1, width=32, depth=3)`	Small network approximating an unknown functional term in a PDE/ODE. Kept shallow/narrow by design: if the term is later distilled into a closed-form expression (e.g. via symbolic regression), a simple network is much easier to distill accurately than a large one.
`GrayBoxConfig`	classe	`(epochs_supervised, epochs_joint, lr_main, lr_graybox, lr_params, w_data, w_pde, pde_warm…)`	
`train_graybox`	função	`(main_model, graybox_nets, known_residual_fn, obs_coords, obs_values, collocation_sampler…)`	Two-phase gray-box training: Phase 1 fits `main_model` to (obs_coords, obs_values) alone; Phase 2 jointly trains `main_model`, the gray-box networks, and any unknown scalar parameters against a weighted sum of the data loss and `known_residual_fn`'s PDE residual, with the PDE weight auto-balanced …
`distill_to_symbolic`	função	`(graybox_net, coord_bounds, n_samples=2000, var_names=None, n_iterations=40) -> Dict[str,…)`	Sample a trained gray-box network on its input domain and run PySR symbolic regression to recover a closed-form analytical expression for the unknown term it learned. Requires the optional `pysr` package.

pinneapple_neural.trainer.heteroscedastic

Heteroscedastic (input-dependent variance) uncertainty quantification.

pinneapple_neural/trainer/heteroscedastic.py · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
HeteroscedasticWrapper	classe	(base_network, n_out_physical)	Wraps a base network (output dim = 2 * n_out_physical) into a model exposing .forward_uq(x) -> (mean, log_variance), each of shape (N, n_out_physical). .forward(x) returns the mean only, for drop-in compatibility with code expecting a single-prediction model.
init_logvar_bias	função	(output_layer, n_out_physical, bias_value=-4.6) -> None	Bias-initialize a network's final linear layer's log-variance output columns (indices [n_out_physical:]) toward a small initial predicted uncertainty (log(sigma^2) ~= bias_value, i.e. sigma ~= exp(bias_value/2) ~= 0.1 at the default). Call once, right after constructing the base network and before ...
heteroscedastic_nll_loss	função	(pred_mean, pred_logvar, target) -> torch.Tensor	Gaussian negative log-likelihood: L = 0.5*(log(sigma^2) + (y-mu)^2/sigma^2), with log-variance clamped to [-10, 10] for numerical stability.
compute_aleatoric_epistemic_uq	função	(model, coords, n_mc=50, forward_kwargs=None) -> Dict[str, torch.Tensor]	Decompose predictive uncertainty via the law of total variance:

pinneapple_neural.trainer.hpc

Advanced HPC utilities for large-scale PINN training.

pinneapple_neural/trainer/hpc.py · 819 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FSDPConfig	classe	(sharding_strategy, mixed_precision, min_num_params, activation_checkpointing, cpu_offloa…)	Configuration for FSDP wrapping.
CUDAGraphModule	classe	(module, warmup_steps=3, device='cuda') -> None	Wrap a module to use CUDA graph replay for static-shape inference.
TorchRunConfig	classe	(n_nodes, n_procs_per_node, master_addr, master_port, rdzv_backend, max_restarts)	Configuration for a torchrun launch.
SLURMConfig	classe	(job_name, n_nodes, n_gpus_per_node, n_cpus_per_task, time_limit, partition, account, mem…)	SLURM batch script configuration.
ProfilerConfig	classe	(trace_dir, record_shapes, profile_memory, with_flops, with_stack, schedule_wait, schedul…)	Configuration for PINNeAPPleProfiler.
PINNeAPPleProfiler	classe	(config=None) -> None	Context-manager profiler that wraps torch.profiler.profile.
AutoBatchSizeFinder	classe	(trial_fn, lo=1, hi=8192, verbose=True) -> None	Find the maximum batch size that fits in GPU memory via binary search.

nome	tipo	assinatura	o que faz
`wrap_fsdp`	função	`(model, cfg=None, *, process_group=None) -> nn.Module`	Wrap a model with Fully Sharded Data Parallel.
`wrap_zero_optimizer`	função	`(optimizer_class, params, **optimizer_kwargs) -> torch.optim.Optimizer`	Wrap an optimizer with ZeroRedundancyOptimizer (ZeRO-1 sharding).
`register_powersgd_hook`	função	`(model, *, matrix_approximation_rank=1, start_powersgd_iter=10, use_error_feedback=True) …)`	Register a PowerSGD gradient compression hook on a DDP model.
`register_topk_hook`	função	`(model, *, compression_ratio=0.1) -> None`	Register a Top-K gradient sparsification hook on a DDP model.
`build_torchrun_cmd`	função	`(script, cfg, *, script_args='') -> str`	Generate a `torchrun` launch command string.
`build_slurm_script`	função	`(torchrun_cmd, slurm, *, output_dir='slurm_logs') -> str`	Generate a complete SLURM batch script as a string.

pinneapple_neural.trainer.inference

Inference utilities: load_checkpoint and predict.

`pinneapple_neural/trainer/inference.py` · 22 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_checkpoint`	função	`(model, path, map_location='cpu') -> Dict[str, Any]`	
`predict`	função	`(model, x, device=None) -> torch.Tensor`	

pinneapple_neural.trainer.inverse_pinn

Inverse PINN: jointly optimize network weights and unknown scalar physical parameters from sparse sensor observations plus a PDE/ODE residual.

`pinneapple_neural/trainer/inverse_pinn.py` · 218 linhas · testes: 6

nome	tipo	assinatura	o que faz
`InverseParam`	classe	`(initial_value, log_space=False, name='param')`	Wraps a scalar as a learnable parameter. Supports a log-space transform for positivity-constrained parameters (the raw trainable value is log(param); `.value` exponentiates it, so it is always > 0 and unconstrained-Adam-friendly even near 0).
`InversePINN`	classe	`(model, inverse_params, residual_fn, sensor_points, sensor_values, domain_bounds, n_col=2…)`	Jointly optimizes network weights (theta) and unknown physical parameters (lambda) from sparse sensor observations plus a caller- supplied PDE/ODE residual (see module docstring).

pinneapple_neural.trainer.lbfgs_finetune

Multi-round L-BFGS fine-tuning: resample-per-round quasi-Newton polish.

`pinneapple_neural/trainer/lbfgs_finetune.py` · 112 linhas · testes: 6

nome	tipo	assinatura	o que faz
LBFGSFineuneResult	classe	(round_losses, round_iters)	Per-round bookkeeping from :func:multi_round_lbfgs.
multi_round_lbfgs	função	(params, closure_builder, n_rounds, n_steps_per_round, lr=1.0, history_size=50, line_sear…)	Run n_rounds independent :class:torch.optim.LBFGS phases, each max_iter=n_steps_per_round, each against a freshly built closure (so a new mini-batch, if the caller's closure_builder resamples one) -- rather than one L-BFGS phase for the whole budget.

pinneapple_neural.trainer.loss_balancer

Advanced automatic loss weight optimisation for PINNs.

pinneapple_neural/trainer/loss_balancer.py · 1.229 linhas · testes: 8

nome	tipo	assinatura	o que faz
ReLoBRaLo	classe	(loss_names, *, tau=0.1, alpha=0.999, update_every=1, clip_min=0.001, clip_max=1000.0, in…)	Relative Loss Balancing via Residuals and Losses.
SoftAdapt	classe	(loss_names, *, tau=0.1, beta=0.0, normalized=True, update_every=1, clip_min=0.001, clip_…)	SoftAdapt — Adaptive loss weighting via rates of change.
AugmentedLagrangian	classe	(loss_names, *, pde_key='pde', rho=0.1, lambda_init=1.0, lambda_min=0.0, lambda_max=10000…)	Augmented Lagrangian method for PINN loss balancing.
InverseDirichlet	classe	(model, loss_names, *, shared_layer=None, update_every=50, alpha=0.9, clip_min=0.001, cli…)	Inverse Dirichlet (gradient statistics) weighting.
LRAnnealing	classe	(model, loss_names, *, reference='pde', update_every=100, alpha=0.1, clip_min=0.001, clip…)	Learning-rate annealing for PINN loss weights (Algorithm 1 of Wang, Teng & Perdikaris, 2021 — "Understanding and mitigating gradient flow pathologies in physics-informed neural networks", SIAM J. Sci. Comput. 43(5):A3055– A3081, doi:10.1137/20M1318043). Not the NTK paper cited for …
PCGrad	classe	(model, loss_names, *, update_every=1, weights=None) -> None	PCGrad — Projecting Conflicting Gradients.

nome	tipo	assinatura	o que faz
AutoBalancerConfig	classe	(method, check_every, health_window, plateau_rel_thresh, divergence_thresh, max_switches,…)	Configuration for the AutoBalancer meta-scheduler.
AutoBalancer	classe	(model, loss_names, *, config=None, pde_key='pde') -> None	Auto-balancing meta-scheduler for PINN loss weights.
JointAdaptiveWeights	classe	(loss_names, *, initial_weights=None, sp_beta=1.0, max_raw_init=20.0, scale=None) -> None	Jointly-trained adaptive loss weights via softplus parameterisation.
AutoPDEWeightCalibrator	classe	(target_ratio=0.01, *, once=True, clip_min=1e-20, clip_max=1000000.0) -> None	Automatically calibrates the PDE loss weight relative to BC/data losses.

pinneapple_neural.trainer.losses

Loss functions for supervised and physics-aware training.

pinneapple_neural/trainer/losses.py · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SupervisedLoss	classe	(kind)	
PhysicsLossHook	classe	(physics_loss_fn)	
CombinedLoss	classe	(supervised, physics, w_supervised, w_physics)	
build_loss	função	(*, problem_spec, model_capabilities=None, weights=None, supervised_kind='mse', physics_l…)	Helper to build a CombinedLoss for Arena.

pinneapple_neural.trainer.metrics

Metrics for evaluation (MSE, MAE, RMSE, R2, MetricBundle).

pinneapple_neural/trainer/metrics.py · 109 linhas · **sem teste direto** · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Metric	classe	(name)	
MSE	classe	(name)	
MAE	classe	(name)	
RMSE	classe	(name)	
R2	classe	(name, eps)	
RelL2	classe	(name, eps)	
MaxError	classe	(name)	
Metrics	classe	()	
MetricBundle	classe	(metrics)	
RegressionMetrics	classe	(eps)	

nome	tipo	assinatura	o que faz
`default_metrics`	função	`() -> List[Metric]`	
`regression_metrics_bundle`	função	`() -> Metrics`	

`pinneapple_neural.trainer.metrics_cfg`

Build Metrics objects from YAML-driven configuration lists.

`pinneapple_neural/trainer/metrics_cfg.py` · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_metrics_from_cfg`	função	`(cfg) -> Metrics`	Build a MetricBundle from a list of metric names or config dicts.

`pinneapple_neural.trainer.monotonic`

Monotonicity constraints for regression surrogates: a hinge penalty on input-output derivatives.

`pinneapple_neural/trainer/monotonic.py` · 55 linhas · testes: 2

nome	tipo	assinatura	o que faz
`monotonicity_penalty`	função	`(model, z, cols, signs, *, scale=None, margin=0.0, output=0, create_graph=True) -> torch.…)`	Mean squared hinge on `signs[j] * d out / d z[:, cols[j]] * scale[j] + margin` (zero when every sampled derivative has the required sign with at least `margin` to spare).
`monotone_violation_share`	função	`(model, z, cols, signs, output=0) -> list[float]`	Share of the rows of `z` where `d out / d z[:, col]` has the wrong sign, per constrained column.

`pinneapple_neural.trainer.multiadam`

MultiAdam: Adam with separate moment statistics per loss term.

`pinneapple_neural/trainer/multiadam.py` · 103 linhas · testes: 4

nome	tipo	assinatura	o que faz
`MultiAdam`	classe	`(params, lr=0.001, betas=(0.99, 0.99), eps=1e-08)`	Adam with one `(m, v)` pair per loss term (Yao et al., ICML 2023).

`pinneapple_neural.trainer.multistart`

Multi-restart PINN training with NaN watchdog and best-model selection.

`pinneapple_neural/trainer/multistart.py` · 363 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MultiRestartConfig`	classe	`(n_restarts, seed_base, nan_patience, max_failed, verbose, keep_all)`	Configuration for `MultiRestartTrainer`.
`RestartResult`	classe	`(model, history, score, restart_idx, seed, nan_detected)`	Outcome of a single training restart.
`MultiRestartResult`	classe	`(best, all_results, n_valid, n_nan)`	Aggregated output of `MultiRestartTrainer.fit()`.

nome	tipo	assinatura	o que faz
`MultiRestartTrainer`	classe	`(model_factory, train_fn, *, config=None, score_fn=None, on_restart_end=None) -> None`	Runs a training function N times with different seeds, keeps the best.

pinneapple_neural.trainer.noether_trainer

Noether surrogate trainer for PINNeAPPle.

`pinneapple_neural/trainer/noether_trainer.py` · 344 linhas · testes: 5

nome	tipo	assinatura	o que faz
`NoetherTrainConfig`	classe	`(epochs, batch_size, lr, weight_decay, field_weights, loss_fn, warmup_steps, grad_clip, d…)`	Training configuration for NoetherSurrogateTrainer.
`NoetherTrainResult`	classe	`(train_losses, val_losses, best_val_loss, best_epoch, checkpoint_path, extras)`	Summary returned by `NoetherSurrogateTrainer.train()`.
`NoetherSurrogateTrainer`	classe	`(model, train_loader, val_loader=None, config=None) -> None`	Train a Noether (UPT / AB-UPT / Transolver …) model with PINNeAPPle data and report training results in a PINNeAPPle-compatible way.

pinneapple_neural.trainer.normalizers

Normalizers — re-exported from pinneapple_data.transforms (single source of truth).

`pinneapple_neural/trainer/normalizers.py` · 5 linhas · **sem teste direto**

pinneapple_neural.trainer.parallel

Parallelization and GPU optimization utilities for pinneapple training.

`pinneapple_neural/trainer/parallel.py` · 508 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AMPContext`	classe	`(device='cuda', dtype=torch.bfloat16, enabled=True) -> None`	Context manager for automatic mixed precision training.
`CUDAPrefetcher`	classe	`(loader, device='cuda') -> None`	Asynchronous GPU prefetcher that overlaps CPU→GPU data transfer with GPU computation using CUDA streams.
`GradAccumConfig`	classe	`(accumulate_steps, clip_grad_norm)`	Configuration for gradient accumulation.
`GradAccumTrainer`	classe	`(model, optimizer, cfg=None, amp_ctx=None) -> None`	Mixin / wrapper that adds gradient accumulation to any training loop.
`SweepConfig`	classe	`(param_grid, n_jobs, backend, timeout_per_trial)`	Hyperparameter sweep configuration.
`ThroughputMonitor`	classe	`(device_index=0) -> None`	Training throughput and GPU health monitor.
`best_device`	função	`(prefer_cuda=True, allow_mps=True) -> torch.device`	Return the best available device: CUDA, then Apple MPS (unless `allow_mps=False`), then CPU.

nome	tipo	assinatura	o que faz
`count_gpus`	função	`() -> int`	Return the number of available CUDA GPUs.
`gpu_info`	função	`() -> List[Dict[str, Any]]`	Return info dict for each available GPU.
`maybe_compile`	função	`(model, *, mode='default', fullgraph=False, dynamic=False) -> nn.Module`	Wrap model with torch.compile() if supported (PyTorch >= 2.0).
`wrap_data_parallel`	função	`(model, device_ids=None) -> nn.Module`	Wrap model in DataParallel if multiple GPUs are available.
`unwrap_model`	função	`(model) -> nn.Module`	Unwrap DataParallel / DistributedData-Parallel to get the raw model.
`run_parallel_sweep`	função	`(trial_fn, cfg) -> List[Dict[str, Any]]`	Run a hyperparameter sweep in parallel.
`enable_gradient_checkpointing`	função	`(model) -> nn.Module`	Enable gradient checkpointing on transformer / large MLP models.

`pinneapple_neural.trainer.preprocess`

Preprocessing pipeline and preprocessing steps for training batches.

`pinneapple_neural/trainer/preprocess.py` · 445 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PreprocessStep`	classe	`()`	
`PreprocessPipeline`	classe	`(steps)`	
`NormalizeStep`	classe	`(key, dim, store_key, enabled, scaler)`	Fits a StandardScaler on `key` from train batches, then normalizes in apply(). Works for typical (B,T,D) or (B,D) tensors.
`SolverFeatureStep`	classe	`(solver, mode, select_var_dim, reduce_fft_to, fit_noop)`	
`MissingValueStep`	classe	`(key, strategy, enabled)`	Fill missing values (NaNs) in x.
`WinsorizeStep`	classe	`(key, q_low, q_high, enabled, low_, high_)`	Quantile clipping (winsorization) fit on the train fold.
`RobustScaleStep`	classe	`(key, eps, enabled, median_, iqr_)`	Robust scaling: (x - median) / (IQR + eps), fit on the train fold.
`LogStep`	classe	`(key, enabled, clamp_min)`	Optional log transform for non-negative signals (fold-safe).
`DifferencingStep`	classe	`(key, time_dim, order, enabled, anchor_key)`	Differencing along the time axis to improve stationarity.

pinneapple_neural.trainer.self_scaled_qn

Self-scaled quasi-Newton optimizers: BFGS, SSBFGS and SSBroyden (dense inverse Hessian).

pinneapple_neural/trainer/self_scaled_qn.py · 186 linhas · testes: 4

nome	tipo	assinatura	o que faz
SelfScaledQuasiNewton	classe	(params, variant='ssbroyden', max_iter=20, tolerance_grad=1e-12, tolerance_change=1e-14, …)	Dense BFGS / SSBFGS / SSBroyden with strong-Wolfe line search (Urban et al., JCP 2025).
self_scaled_parameters	função	(s, y, g, alpha, Hy, n, variant)	Return (tau, phi) for one update (eqs. 11-23). Tensors are 1-D, g is the old gradient.

pinneapple_neural.trainer.splits

Train/val/test split strategies — re-exported from pinneapple_data.splits.

pinneapple_neural/trainer/splits.py · 5 linhas · **sem teste direto**

pinneapple_neural.trainer.time_marching

Sequential time-marching for stiff/long-horizon PDE problems.

pinneapple_neural/trainer/time_marching.py · 388 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TimeMarchingTrainer	classe	(model_factory, t_start, t_end, n_windows=10, epochs_per_window=2000, n_col=1000, ic_weig…)	Trains a PINN by marching sequentially through time windows.

pinneapple_neural.trainer.trainer

Unified Trainer with physics-aware losses, callbacks, and audit logging.

pinneapple_neural/trainer/trainer.py · 397 linhas · testes: 151

nome	tipo	assinatura	o que faz
TrainConfig	classe	(epochs, lr, weight_decay, grad_clip, amp, device, seed, deterministic, log_dir, run_name…)	
Trainer	classe	(model, loss_fn, metrics=None, preprocess=None, early_stopping=None, checkpoint=None)	Trainer for PINNs and general supervised models.

pinneapple_neural.trainer.two_phase

Two-phase PINN trainer: supervised (Phase 1) → physics (Phase 2).

pinneapple_neural/trainer/two_phase.py · 414 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TwoPhaseConfig	classe	(phase1_epochs, phase1_lr, phase2_epochs, phase2_lr, weight_decay, grad_clip, device, amp…)	

nome	tipo	assinatura	o que faz
`TwoPhaseHistory`	classe	`(phase1_epochs, phase1_loss, phase2_epochs, phase2_loss, phase2_pde_loss, phase2_bc_loss)`	
`TwoPhaseTrainer`	classe	`(model, supervised_loss_fn, physics_loss_fn, config=None)`	Auto-scheduled two-phase trainer for PINNs.
`UnnormModel`	classe	`(model, Y_mean, Y_std)`	Wraps a normalised model to output physical-scale predictions.

`pinneapple_neural.trainer.viz`

Visualization utilities for training history.

`pinneapple_neural/trainer/viz.py` · 57 linhas · **sem teste direto** · sinais no código: shim ou obsoleto

nome	tipo	assinatura	o que faz
`plot_history`	função	`(history, keys=('train_total', 'val_total'), *, log_scale=True, show=True)`	Plot training history.

`pinneapple_neural.trainer.weight_scheduler`

Automatic loss weight scheduling for PINN training.

`pinneapple_neural/trainer/weight_scheduler.py` · 799 linhas · testes: 11

nome	tipo	assinatura	o que faz
`WeightSchedulerConfig`	classe	`(method, initial_weights, update_every, lr, alpha, clip_min, clip_max, ema_decay)`	Configuration for automatic PINN loss weight scheduling.
`SelfAdaptiveWeights`	classe	`(loss_names, config) -> None`	Learnable loss weights trained by gradient *ascent*.
`GradNormBalancer`	classe	`(model, loss_names, shared_layer=None, config=None) -> None`	Gradient-norm-based weight balancing.
`LossRatioBalancer`	classe	`(loss_names, target_ratios=None, config=None) -> None`	Keep loss component ratios near target values using EMA smoothing.
`NTKWeightBalancer`	classe	`(model, loss_names, n_proj=50, config=None) -> None`	NTK-trace based weight balancing.
`WeightScheduler`	classe	`(model, loss_names, config=None) -> None`	Unified interface for automatic PINN loss weight scheduling.

`pinneapple_neural.utils`

Small helpers shared by trainers, solvers and examples.

`pinneapple_neural/utils.py` · 92 linhas · testes: 5

nome	tipo	assinatura	o que faz
`as_tensor`	função	`(out, *, attrs=OUTPUT_TENSOR_ATTRS) -> torch.Tensor`	Return the prediction Tensor of a model output.

nome	tipo	assinatura	o que faz
fit_supervised	função	(model, x, y, *, steps=1000, lr=0.001, batch=32, loss='mse', weight_decay=0.0, seed=0, fo…)	Train model to map x to y (tensors or arrays, first axis = samples) with Adam on random mini-batches for steps steps; returns the loss history. loss: "mse" or "rel_l2" (per-sample relative L2, for fields). forward(model, xb) overrides the call (default …

pinneapple_neural.workflows

End-to-end train/predict workflows over PINNeAPPle's neural architectures -- reusable, product-facing entry points (as opposed to pinneapple_neural.architectures/trainer, which expose the lower-level building blocks these workflows compose).

pinneapple_neural/workflows/__init__.py · 56 linhas · **sem teste direto**

pinneapple_neural.workflows.dense_volume_operator

A reusable train/predict workflow for a grid-based neural operator (:class:~pinneapple_neural.architectures.neural_operators.fno.FNO3d) trained on a DENSE-in-time real volume time series -- e.g. the output of :func:pinneapple_data.adapters.load_dense_volumes from a .splash OpenFOAM archive, ...

pinneapple_neural/workflows/dense_volume_operator.py · 248 linhas · testes: 3

nome	tipo	assinatura	o que faz
DenseVolumeOperatorConfig	classe	(in_channels, out_channels, width, modes, layers, epochs, lr, val_frac, batch_size, rollo…)	
train_dense_volume_operator	função	(frames, cfg, *, seed=0) -> Tuple[Checkpoint, List[Dict[str, Any]]]	Train an :class:FNO3d on a real dense volume time series.
load_dense_volume_operator	função	(checkpoint, *, device='cpu') -> nn.Module	Reconstruct the real FNO3d from a checkpoint produced by :func:train_dense_volume_operator (or the same-shaped legacy checkpoints train_fno3d.py already produced before this module existed -- both save the same meta keys).
predict_single_step	função	(checkpoint, input_frame, *, device='cpu') -> np.ndarray	One forward pass: input_frame (C, D, H, W), real (not normalized) units in, real units out. This is the regime the network is most accurate in -- see the module docstring.
rollout_dense_volume_operator	função	(checkpoint, initial_frame, n_steps, *, device='cpu') -> np.ndarray	Autoregressive rollout from a real starting frame. Returns (n_steps + 1, C, D, H, W) real-unit frames, index 0 = the given initial_frame unchanged, indices 1..n_steps = the model's own successive predictions fed back in as input.

pinneapple_neural.workflows.hybrid_surrogate_physics

Hybrid model: a data-driven surrogate for intermediate quantities + an explicit physics post-model.

pinneapple_neural/workflows/hybrid_surrogate_physics.py · 517 linhas · testes: 10

nome	tipo	assinatura	o que faz
PhysicsPostModel	classe	(name, source)	An explicit physics model applied to the surrogate's intermediate quantities.
CallablePostModel	classe	(fn, name, source)	Wrap a plain function as a :class:PhysicsPostModel.
SurrogatePrediction	classe	(mean, std, sampler)	Predictive distribution of a surrogate over m flat outputs at n points.
Surrogate	classe	()	Minimal surrogate interface: fit(X, Y) with X (n, d), Y (n, m); predict(X) returns a :class:SurrogatePrediction, a (mean, std) tuple (std may be None) or a mean array.
GaussianProcessSurrogate	classe	(*, n_restarts=2, random_state=0, use_sklearn=None)	One independent Gaussian process per output column.
PCASurrogate	classe	(base, *, n_components=None, variance=None)	Reduce the outputs with PCA, fit base on the scores, map predictions back.
TransformedSurrogate	classe	(base, forward, inverse, name='transform')	Fit base on forward(Y); predictions are mapped back with inverse.
PerQuantitySurrogate	classe	(blocks)	One surrogate per block of output columns (e.g. one per intermediate quantity).
HybridPrediction	classe	(intermediates, derived_at_mean, derived_mean, derived_std, derived_lower, derived_upper,…)	Output of :meth:HybridSurrogatePhysics.predict.
HybridSurrogatePhysics	classe	(surrogate, post_model, *, param_names=None)	Surrogate of intermediate quantities composed with an explicit :class:PhysicsPostModel.

pinneapple_neural.workflows.parametric_dense_volume_operator

A joint, multi-case extension of :mod:dense_volume_operator: one :class:~pinneapple_neural.architectures.neural_operators.fno.FNO3d trained across SEVERAL real dense-in-time volume time series that share the same physical problem but differ in a scalar (or low-dim) physical parameter theta ...

pinneapple_neural/workflows/parametric_dense_volume_operator.py · 382 linhas · testes: 3

nome	tipo	assinatura	o que faz
ParametricCase	classe	(name, frames, theta)	One real training case: a dense-in-time volume sequence plus the scalar (or low-dim) physical parameter(s) that produced it.
ParametricDenseVolumeOperatorConfig	classe	(state_channels, theta_dim, width, modes, layers, epochs, lr, val_frac, batch_size, rollo…)	

nome	tipo	assinatura	o que faz
`train_parametric_dense_volume_operator`	função	`(cases, cfg, *, seed=0) -> Tuple[Checkpoint, List[Dict[str, Any]]]`	Jointly train one theta-conditioned :class:`FNO3d` across cases.
`load_parametric_dense_volume_operator`	função	`(checkpoint, *, device='cpu') -> nn.Module`	
`predict_single_step_parametric`	função	`(checkpoint, input_frame, theta, *, device='cpu') -> np.ndarray`	One forward pass at a given (possibly never-trained-on) `theta`. `input_frame`: (state_c, D, H, W), real units in, real units out.
`rollout_parametric_dense_volume_operator`	função	`(checkpoint, initial_frame, theta, n_steps, *, device='cpu') -> np.ndarray`	Autoregressive rollout at a given `theta`, re-attaching the (constant) theta channel after every step -- see module docstring ("theta is a condition, not a predicted state"). Returns `(n_steps + 1, state_c, D, H, W)`, index 0 = `initial_frame`.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 46 de 198.
- Funções de teste em arquivos que importam algum módulo do pacote: 438.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
S21	Digital twin state estimation: KF/EKF/UKF/EnKF, prognostics, signal reconstruction	tested	2	39
T1	Base trainer: AMP, gradient accumulation, auto batch size, CUDA graphs, profiler	tested	0	9
T2	Distributed: DDP, FSDP, SLURM, torchrun, PowerSGD	untested	0	0
T3	Causal training	untested	0	0
T4	Time marching	untested	0	0
T5	Two phases Adam -> L-BFGS + L-BFGS finetune	tested	0	6
T6	Multi-restart	untested	0	0
T7	Adaptive residual-based collocation	untested	0	0

id	método	status	testes contra referência	outros testes
T8	Loss balancing: ReLoBRaLo, SoftAdapt, PCGrad, Augmented Lagrangian, Inverse Dirichlet, LR Annealing, AutoBalancer	validated	1	6
T9	Weight schedulers: SA-PINN, GradNorm, LossRatio, NTK	tested	0	2
T10	Domain decomposition: XPINN, DoMINO/D3M	tested	0	2
T11	Inverse PINN (learnable physical parameters)	tested	0	3
T12	Gray-box + symbolic regression (PySR)	untested	0	0
T13	Uncertainty: heteroscedastic, B-PINN, MC Dropout, deep ensembles	tested	0	20
T14	SINDy	tested	0	2
T15	Adaptive hyperparameter sweep / architecture search (arena NAS)	untested	0	0
T16	Noether trainer (research only)	tested	0	1
T17	Post-training advisor / audit	untested	0	0
T20	Self-scaled quasi-Newton: BFGS / SSBFGS / SSBroyden (new, 2026-09-24)	validated	1	3
T22	Large Physics Model blocks (Fourier encoding, multi-scale neighbourhoods, geometry code, ensemble UQ, OOD score, fine-tuning)	tested	0	1
T23	MultiAdam: per-loss-term Adam moments for multi-term PINN losses	validated	1	3

Limitações conhecidas

- *Nenhuma registrada.*

2. `pinneapple_data`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
115	29.019	129	256	507 (54 arquivos)	70

Pasta de testes dedicada: `tests/pinneapple_data` (33 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
active_learning		1	5	22
adapters		5	19	0
audit		1	10	49
augmentation		1	3	0
cache		1	2	0
cache_bytes		1	3	1
cae		13	74	62
chunking		1	2	0
collate		1	3	0
collocation		1	2	7
contracts		4	27	175
data_health		1	6	37
dataloaders		1	3	0
datasets		9	28	10
device		1	2	0
doe		4	10	21
expressions		1	5	22
formfill		14	31	44
geometry_features		1	3	0
grf_sampler		1	5	0
lineage		3	4	13
noether_dataset		1	3	5
parameter_sampling		1	2	5
physical_sample		1	1	28
physical_units		1	12	52
physics_case		1	2	47
prefetch_adaptive		1	2	0
preflight		4	11	13
process_optimizer		1	7	16
serialization		1	9	11
simulation_metadata		5	25	25
snapshots		1	3	31
splits		1	2	0
stl_import		1	3	0
synth		15	20	0
transforms		1	3	2
udr1		1	3	36
unified		1	9	24
upd_dataset		1	1	5
upd_types		1	5	0
validators		1	3	0
zarr_cached_store		1	2	0
zarr_cached_store_bytes		1	2	0
zarr_iterable		1	1	0
zarr_prefetch		1	2	0
zarr_shard_iterable		1	2	0
zarr_shards		1	2	0
zarr_store		1	1	1

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_data`

Módulo sem docstring e sem texto.

`pinneapple_data/__init__.py` · 117 linhas · testes: 78

`pinneapple_data.active_learning`

Active Learning for adaptive collocation in PINN training.

`pinneapple_data/active_learning.py` · 674 linhas · testes: 22

nome	tipo	assinatura	o que faz
`ActiveLearningConfig`	classe	`(strategy, n_candidates, n_select, n_initial, n_iterations, exploitation_weight, seed)`	
`ResidualBasedAL`	classe	`(config, bounds) -> None`	Sample collocation points where PDE residual is highest (RAR/RAD).
`VarianceBasedAL`	classe	`(config, bounds) -> None`	Sample where model output variance is highest (requires a UQ model).
`CombinedAL`	classe	`(config, bounds, residual_weight=0.6, variance_weight=0.3, diversity_weight=0.1) -> None`	Weighted combination: exploit high residual + explore uncertain regions.
`AdaptiveCollocationTrainer`	classe	`(model, physics_fn, sampler, al_config=None, optimizer=None, device='cpu') -> None`	Wraps a PINN model with active-learning collocation updates.

`pinneapple_data.adapters`

Módulo sem docstring e sem texto.

`pinneapple_data/adapters/__init__.py` · 63 linhas · **sem teste direto**

`pinneapple_data.adapters.geom_adapter`

Geometry loading and attachment utilities for PhysicalSample via pinneapple_geom.

`pinneapple_data/adapters/geom_adapter.py` · 286 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometryLoadOptions`	classe	`(scale, translate, rotate_euler_deg, repair, compute_normals, boundary_labels)`	Configuration container for geometry or mesh loading behavior.
`load_geometry_asset`	função	`(geom, *, options=None) -> Any`	Load or construct a GeometryAsset via pinneapple_geom.
`stl_to_upd`	função	`(path, *, repair=True, compute_normals=True, units='m') -> Any`	Load an STL and package it as a UPD-aligned PhysicalSample.
`attach_geometry`	função	`(sample, geom_asset) -> Any`	Attach a GeometryAsset to a PhysicalSample-like structure.

pinneapple_data.adapters.pinn_batch_builders

PINN batch builders: convert various data sources into the unified dict-batch format.

`pinneapple_data/adapters/pinn_batch_builders.py` · 149 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PINNBatch`	classe	`(batch)`	In-memory PINN-ready batch (thin wrapper around the dict).
`build_from_bundle`	função	`(bundle, *, n_collocation=4096, n_boundary=2048, n_data=2048, device=None, dtype=torch.fl…)`	Create a PINN-style batch from an Arena BundleData.
`build_from_solver`	função	`(problem_spec, geometry, solver_cfg, *, device=None, dtype=torch.float32) -> PINNBatch`	Build a PINN batch by running a solver or using pre-computed tensors.
`build_from_real_data`	função	`(adapter_cfg, *, device=None, dtype=torch.float32) -> PINNBatch`	Build a PINN batch from real measurement data.

pinneapple_data.adapters.splash_archive_adapter

Loader for a Splash-CFD `.splash` case (a zipped OpenFOAM case directory) into plain numpy arrays -- both a flat point-cloud form (for PINN-style training) and a dense structured-grid volume time series (for grid-based neural operators like :class:`~pinneapple_neural. …

`pinneapple_data/adapters/splash_archive_adapter.py` · 315 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SplashMesh`	classe	`(n_cells, n_points, n_faces, n_internal_faces, cell_centers, cell_size, cell_delta, bound…)`	
`FieldSnapshot`	classe	`(time, fields)`	
`SplashCase`	classe	`(path, mesh, time_dirs, transport, meta)`	
`load_mesh`	função	`(zf, prefix='constant/polyMesh/') -> SplashMesh`	Build cell centers/sizes from the binary polyMesh inside a .splash zip. `zf` is an open `zipfile.ZipFile`.
`save_mesh_cache`	função	`(mesh, path) -> None`	
`load_mesh_cache`	função	`(path) -> SplashMesh`	
`open_case`	função	`(path, mesh_cache_dir=None) -> SplashCase`	Open a `.splash` file, reconstructing (or loading a cached) mesh.
`load_dense_volumes`	função	`(splash_path, field_components, nx, ny, nz, *, mesh_cache_dir=None) -> Tuple[np.ndarray, …)`	Load every time directory of a `.splash` archive as a dense volume time series, for a structured single hex block mesh.

pinneapple_data.adapters.upd_adapter

UPD shard loading and PhysicalSample conversion adapters.

pinneapple_data/adapters/upd_adapter.py · 288 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
load_upd_item	função	(upd) -> Tuple[xr.Dataset, Dict[str, Any]]	Load a UPD shard dataset and its associated metadata.
upd_to_physical_sample	função	(upd, *, schema_override=None, provenance_extra=None) -> Any	Convert a UPD shard into a PhysicalSample instance.
attach_upd_state	função	(sample, upd) -> Any	Attach UPD dataset state and metadata to an existing sample object.

pinneapple_data.audit

Physics audit of a training dataset: does the data solve its PDE? Split leakage, near duplicates, diversity and units (#700). NumPy/SciPy only.

pinneapple_data/audit.py · 519 linhas · testes: 49

nome	tipo	assinatura	o que faz
AuditCheck	classe	(id, status, summary, value, threshold, evidence)	
AuditCard	classe	(dataset, splits, physics, checks, tolerances)	
DatasetPhysics	classe	(name, columns, check, tolerances, units)	Per-sample physics of a dataset. check(sample) -> {name: relative error} for one sample (a mapping of column -> array without the sample axis); tolerances maps each name to its limit (keys of :data:DEFAULT_TOLERANCES or numbers).
darcy_consistency	função	(a, u, f=1.0) -> dict[str, float]	Darcy sample checks in the discretisation of operator_benchmarks.darcy_2d (cell-centred, h = 1/n): solve = relative L2 distance of u to the solution of its own discrete problem; bc = RMS difference of the boundary values (linear extrapolation of the first two cells to each face) ...
effective_sample_count	função	(x) -> float	Participation ratio (sum l)^2 / sum l^2 of the principal-component spectrum of the samples (rows): about the number of independent directions the samples span; 1 when every sample is a multiple of one pattern.
near_duplicates	função	(x, near=0.001) -> dict[str, Any]	Exact and near duplicates inside one set (rows), relative to the median nearest-neighbour distance.
leakage	função	(train, test, near=0.001) -> dict[str, Any]	Nearest training sample of each test sample; a test sample closer than near x the median training nearest-neighbour distance is a (near) copy.
constraint_mask	função	(expr, cols) -> np.ndarray	Where the declared relation expr ("Cd > 0", "abs(u) <= 1", "a == b * c" to 1e-6 relative) holds, element-wise over cols: the evaluator the constraints check uses, for checking predictions too.
audit_dataset	função	(splits, *, name='dataset', physics=None, inputs=None, units=None, expected_units=None, c…)	Audit splits ({split: {column: array}}).

nome	tipo	assinatura	o que faz
`audit_zoo_dataset`	função	`(name, *, offline=None, **kw) -> AuditCard`	Audit a `pp.datasets` entry with the physics and constraints its catalog entry states.

`pinneapple_data.augmentation`

Point-cloud dataset augmentation strategies for PDE/PINN training data.

`pinneapple_data/augmentation.py` · 139 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`augment_noise`	função	`(coords, fields, n_copies, std_fraction=0.05, seed=0) -> Dict[str, Any]`	Add Gaussian noise (scaled to each field's own std) to `n_copies` tiled copies of the dataset. Coordinates are unchanged; only field values are perturbed.
`augment_transform`	função	`(coords, fields, n_copies, max_rotation_deg=15.0, scale_range=(0.9, 1.1), seed=0) -> Dict…)`	Random in-plane rotation (about the first two coordinate axes) plus isotropic scaling of the coordinate frame, `n_copies` times. Field values are treated as physical quantities sampled at each point, not coordinate-derived geometry — a rigid rotation/scale of the frame leaves field *values* …
`augment_interpolation`	função	`(coords, fields, n_new_points, method='linear', seed=0) -> Dict[str, Any]`	Generate `n_new_points` new samples inside the convex hull of the original point cloud, with field values estimated via scattered-data interpolation (falling back to nearest-neighbor wherever the requested method produces NaN, e.g. outside the interpolation method's support).

`pinneapple_data.cache`

Thread-safe LRU cache for in-memory data with configurable capacity.

`pinneapple_data/cache.py` · 120 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CacheStats`	classe	`(hits, misses, evictions)`	Statistics container for the LRUCache.
`LRUCache`	classe	`(max_items=256)`	Thread-safe LRU cache.

`pinneapple_data.cache_bytes`

Byte-constrained LRU cache for memory-bounded caching of tensors and samples.

`pinneapple_data/cache_bytes.py` · 179 linhas · testes: 1

nome	tipo	assinatura	o que faz
`ByteCacheStats`	classe	`(hits, misses, evictions, bytes_in_use)`	Statistics container for the ByteLRUCache.
`ByteLRUCache`	classe	`(max_bytes=512 * 1024 * 1024, weigher=default_weigher)`	Thread-safe LRU cache constrained by max_bytes (not max items).
`default_weigher`	função	`(value) -> int`	Estimate memory usage in bytes.

pinneapple_data.cae

CAE data layer: read meshes and results from OpenFOAM, CalculiX/Abaqus, Gmsh, VTK, STL and point tables into one `Mesh` type; mesh quality (checkMesh-equivalent finite-volume metrics, element shape metrics, surface checks); per-cell Courant number and wall distance.

`pinneapple_data/cae/__init__.py` · 14 linhas · testes: 44

pinneapple_data.cae.compare

Compare two results field by field: simulation vs simulation (different meshes or models), vs experiment (sensor tables), vs AI predictions (point tables or meshes), vs analytical or benchmark data.

`pinneapple_data/cae/compare.py` · 420 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Interpolator`	classe	`(src_pts, dst_pts, method='auto')`	Values of a candidate field at query points: identical locations, linear (Delaunay), nearest outside.
`locations`	função	`(m, field) -> Tuple[np.ndarray, str, Optional[np.ndarray]]`	(points where the field lives, 'cell'|'point', weights).
`cell_centres_volumes`	função	`(m) -> Tuple[np.ndarray, Optional[np.ndarray]]`	
`field_value`	função	`(m, f) -> np.ndarray`	
`fields_of`	função	`(m) -> Dict[str, Dict[str, Any]]`	
`match_fields`	função	`(ref, cand, mapping=None) -> List[Tuple[str, str]]`	Pairs (reference field, candidate field): explicit mapping first, then same name (case-insensitive), then aliases (U ~ velocity, p ~ pressure, T ~ temperature, DISP ~ displacement, ...).
`tolerance_verdicts`	função	`(result, tolerance) -> Optional[str]`	Mark every field of a :func:`compare` result `"pass"` / `"fail"` by its relative L2 error against `tolerance` (a fraction, e.g. 0.05 for 5 %), in `field["verdict"]`. Returns `"PASS"` when every judged field passes, `"FAIL"` otherwise, `None` when nothing was judged (no tolerance or no ...
`compare`	função	`(ref, cand, mapping=None, method='auto', fields=None) -> Dict[str, Any]`	
`surface`	função	`(m, max_tris=300000) -> Optional[Dict[str, Any]]`	Boundary surface for display: triangles with the cell each one belongs to. For 2D OpenFOAM cases (empty patches) only one of the two empty sides is kept.
`views`	função	`(ref, res, max_points=60000) -> Dict[str, Any]`	Per-field display payload: a surface coloured per cell (or per node), a line plot, or a 2D/3D point map.

`pinneapple_data.cae.dataset`

Neutral physical dataset: any supported result (OpenFOAM, CalculiX, VTK/VTU, Gmsh, point tables) normalised into one representation (geometry, mesh, coordinates, fields with quantity and unit, metadata) and written to VTK (.vtu), HDF5, Parquet, CSV, NPZ, JSON or a PINNeAPPle dataset (UPD Zarr ...

`pinneapple_data/cae/dataset.py` · 507 linhas · testes: 14

nome	tipo	assinatura	o que faz
`quantity_of`	função	`(name) -> Tuple[Optional[str], Optional[str]]`	
`poly_cells`	função	`(m)`	OpenFOAM cells as VTK cells: hexahedra, wedges, tetrahedra and pyramids are rebuilt from their faces; any other cell becomes a VTK polyhedron (list of faces). Returns (blocks [(type, conn)], polyhedra faces, cell order).
`field_scalar`	função	`(values) -> Tuple[Optional[np.ndarray], str]`	One scalar per point or cell: the value itself, the magnitude of a vector (up to 3 components) or the von Mises stress of a symmetric tensor in Voigt order `xx yy zz xy yz zx`. Returns `(scalar, kind)`; `scalar` is `None` for other shapes.
`build_dataset`	função	`(m, *, unit_system='auto', to_si=False, rho=None, location='auto', files=None) -> Dict[st…)`	Normalise a Mesh into the neutral dataset (a dict holding numpy arrays and metadata).
`describe`	função	`(ds) -> Dict[str, Any]`	JSON-safe description (the schema the user sees): no arrays.
`export`	função	`(ds, fmt) -> Tuple[bytes, str, str]`	(content, media type, file extension).
`write_vtu`	função	`(ds) -> bytes`	
`write_hdf5`	função	`(ds) -> bytes`	
`write_parquet`	função	`(ds) -> bytes`	
`write_csv`	função	`(ds) -> bytes`	
`write_npz`	função	`(ds) -> bytes`	
`write_json`	função	`(ds, max_values=2000000) -> bytes`	

nome	tipo	assinatura	o que faz
`write_pinneapple`	função	`(ds) -> bytes`	A PINNeAPPle dataset: one PhysicalSample (state = coordinates and fields at the chosen location) written with the library's UPD Zarr store (pinneapple_data.serialization.save_zarr), zipped. Load it with `load_zarr` after unzipping; units, quantities and the source are in the sample's provenance.
`preview`	função	`(ds, max_fields=6, max_tris=200000) -> Dict[str, Any]`	Display payload of a dataset from :func:`build_dataset`: the boundary surface (always, so a bare mesh shows too) coloured by the first `max_fields` fields reduced to one scalar each (:func:`field_scalar`), rounded to 5 significant digits. Empty dict when the mesh has no surface.

`pinneapple_data.cae.foam`

OpenFOAM readers: polyMesh (points, faces, owner, neighbour, boundary) and volume fields of a time directory.

`pinneapple_data/cae/foam.py` · 372 linhas · testes: 6

nome	tipo	assinatura	o que faz
`header`	função	`(text) -> Dict[str, str]`	
`read_points`	função	`(b) -> np.ndarray`	
`read_labels`	função	`(b) -> np.ndarray`	
`read_faces`	função	`(b) -> Tuple[np.ndarray, np.ndarray]`	Faces as (flat node list, offsets).
`read_boundary`	função	`(b) -> List[Dict[str, Any]]`	
`read_polymesh`	função	`(fs, root='') -> Mesh`	
`dims_unit`	função	`(dims) -> Optional[str]`	
`read_boundary_values`	função	`(b, mesh, width) -> Optional[np.ndarray]`	Value on every boundary face (NaN for empty patches), from the boundaryField of a field file: the written 'value' (uniform or a list), zero for noSlip, the owner-cell value where nothing is written (zeroGradient).
`read_field`	função	`(b, n_cells) -> Tuple[np.ndarray, Optional[str], str]`	internalField of a vol*Field: (values per cell, unit, class).
`time_dirs`	função	`(fs, root='') -> List[str]`	Numeric time directories under the case root, ascending.
`read_time`	função	`(fs, mesh, root='', time=None, only=None) -> Dict[str, Any]`	Load the volume fields of one time directory (latest by default) into `mesh.cell_data`.

nome	tipo	assinatura	o que faz
`boundary_conditions`	função	`(fs, mesh, root='', time=None) -> Dict[str, Dict[str, str]]`	Patch -> field -> condition as written in the boundaryField of each field of one time directory (the first, which holds the initial and boundary conditions, by default): `"fixedValue uniform (1 0 0)"`, `"noSlip"`, `"zeroGradient"`. Empty patches are left out.

pinneapple_data.cae.frd

CalculiX result file (.frd, ASCII): nodes, elements and the result blocks (DISP, STRESS, NDTEMP, FLUX, ...).

`pinneapple_data/cae/frd.py` · 117 linhas · testes: 6

nome	tipo	assinatura	o que faz
`read_frd`	função	`(b) -> Mesh`	

pinneapple_data.cae.geometry

Finite-volume geometry of any mesh, computed the way OpenFOAM does (primitiveMesh): face centres and area vectors by triangle decomposition about the face average point, cell centres and volumes by pyramid decomposition about the average of the face centres. FE volume meshes are turned into the ...

`pinneapple_data/cae/geometry.py` · 249 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FaceMesh`	classe	`(points, face_nodes, face_offsets, owner, neighbour, n_cells, patch_of_face, patch_names,…)`	
`signed_volume_sign`	função	`(btype, pts, conn) -> np.ndarray`	Sign of a simple volume estimate of each element (tetra decomposition about its first corner).
`oriented_blocks`	função	`(mesh) -> List[str]`	Element node-order conventions differ (VTK, Gmsh, Abaqus wedges...). If most elements of a block come out with negative volume, the whole block uses the opposite convention: flip it, so only truly inverted elements stay negative. Returns the names of the flipped blocks.
`face_mesh`	função	`(mesh) -> FaceMesh`	OpenFOAM polyMesh as is; FE volume meshes converted (internal faces first, then boundary faces).
`face_geometry`	função	`(fm) -> Dict[str, np.ndarray]`	Face centres (nf,3), area vectors (nf,3), and the per-face corner groups used later.
`cell_geometry`	função	`(fm, fg) -> Dict[str, np.ndarray]`	
`cell_adjacency`	função	`(fm) -> sp.csr_matrix`	

nome	tipo	assinatura	o que faz
regions	função	(adj) -> np.ndarray	
element_adjacency_by_nodes	função	(mesh) -> sp.csr_matrix	Cells sharing at least one node (for surface/planar meshes and the node-connectivity check).
solution_dirs	função	(fm, fg) -> Optional[np.ndarray]	Directions with no 'empty' patch (OpenFOAM 2D/1D cases): boolean (3,).

pinneapple_data.cae.inp

Abaqus / CalculiX input deck (.inp): keyword blocks, nodes, elements (to VTK types), node and element sets.

pinneapple_data/cae/inp.py · 296 linhas · testes: 6

nome	tipo	assinatura	o que faz
Block	classe	(keyword, params, data, line, file)	
Deck	classe	(blocks, includes, missing_includes)	
vtk_type	função	(abq) -> Optional[str]	
read_deck	função	(text, name='model.inp', files=None, _depth=0) -> Deck	
resolve_sets	função	(deck, kind) -> Dict[str, List[str]]	NSET or ELSET name (upper case) -> member labels; sets may contain other sets' names.
deck_mesh	função	(deck) -> Mesh	
deck_conditions	função	(deck) -> List[Dict[str, object]]	The supports and loads of a deck as node ids (labels of *NODE), one entry per keyword block and kind: *BOUNDARY (kind bc: the constrained degrees of freedom and their values), *CLOAD / *CFLUX (kind load: the sum of the nodal forces or fluxes per direction), *DLOAD / …

pinneapple_data.cae.io

Read whatever was uploaded into a Mesh: OpenFOAM case or polyMesh, Abaqus/CalculiX .inp, CalculiX .frd, STL, Gmsh .msh, VTK (.vtk/.vtu) and the other meshio formats, and point clouds with fields (CSV, HDF5, NPZ).

pinneapple_data/cae/io.py · 296 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FormatError`	classe	`()`	
`kind_of`	função	`(fs) -> Tuple[str, Optional[str]]`	(kind, main file or case root). Kinds: openfoam, inp, frd, stl, meshio, points.
`read_any`	função	`(fs, fields=True, time=None) -> Mesh`	
`read_stl`	função	`(b) -> Mesh`	
`read_meshio`	função	`(name, b) -> Mesh`	
`read_points`	função	`(name, b) -> Mesh`	A table of points with field columns (CSV/TXT, Parquet, NPZ, HDF5): a point cloud mesh with point_data.

`pinneapple_data.cae.metrics`

Per-cell flow metrics on a finite-volume mesh: Courant number and distance to the nearest wall.

`pinneapple_data/cae/metrics.py` · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`courant_number`	função	`(face_area_vectors, face_owner, face_neighbour, U, cell_volumes, dt, where='auto') -> np.…)`	Cell Courant number `Co_P = 0.5 dt sum_f |U_f . S_f| / V_P` over the faces f of cell P.
`wall_distance`	função	`(cell_centres, wall_face_centres, wall_face_normals=None) -> np.ndarray`	Distance from each cell centre to the nearest wall face centre (`scipy.spatial.cKDTree` query).
`mesh_courant_number`	função	`(mesh, U, dt, where='auto') -> np.ndarray`	:func:`courant_number` on a :class:`Mesh` (cell or face velocity `U`, see there).
`mesh_wall_distance`	função	`(mesh, patches=None, project=True) -> np.ndarray`	:func:`wall_distance` of every cell of a :class:`Mesh` to the faces of `patches` (default: every patch of type `wall`). `project` uses the face normals (see :func:`wall_distance`).

`pinneapple_data.cae.model`

The mesh/result container shared by the CAE readers, the mesh-quality checks, the comparator and the interop export.

`pinneapple_data/cae/model.py` · 173 linhas · testes: 39

nome	tipo	assinatura	o que faz
`CellBlock`	classe	`(type, conn, ids)`	
`PolyMesh`	classe	`(face_nodes, face_offsets, owner, neighbour, n_cells, patches)`	OpenFOAM polyMesh: faces (flat node list + offsets), owner/neighbour, boundary patches.
`Mesh`	classe	`(points, blocks, poly, boundary_blocks, cell_sets, node_sets, point_data, cell_data, unit…)`	

pinneapple_data.cae.quality

Mesh quality: finite-volume metrics as OpenFOAM's checkMesh computes them, element shape metrics for finite-element meshes, connectivity, and surface (STL) checks; problem regions are clusters of bad cells.

pinneapple_data/cae/quality.py · 306 linhas · testes: 16

nome	tipo	assinatura	o que faz
fv_metrics	função	(mesh) -> Dict[str, Any]	
element_metrics	função	(mesh) -> Dict[str, np.ndarray]	Scaled Jacobian, skewness and edge ratio for every cell of the mesh's top dimension (global cell order).
surface_checks	função	(mesh) -> Dict[str, Any]	Triangle/quad surface (STL, shell mesh): open and non-manifold edges, orientation, degenerate faces, shells.

pinneapple_data.cae.report

Mesh health report: checks with a verdict, statistics and histograms, problem regions (clusters of bad cells, with where they are) and what to do about each issue. Also the payload for a 3D view (boundary surface coloured by a metric, plus the bad cells).

pinneapple_data/cae/report.py · 371 linhas · testes: 8

nome	tipo	assinatura	o que faz
build	função	(mesh, target='auto', max_regions=12) -> Dict[str, Any]	target: 'cfd' (finite-volume thresholds lead), 'fea' (element-shape thresholds lead) or 'auto'.
view_payload	função	(mesh, rep, max_tris=250000) -> Dict[str, Any]	Boundary surface (triangles) with each face's owner-cell metric values, and the problem regions' cells.
worst_cells	função	(mesh, rep, n=40) -> List[Dict[str, Any]]	The n worst cells of every metric in a report from :func:build, worst first.

pinneapple_data.cae.upload

Expand an upload (loose files and/or zip archives) into {relative path: bytes}, with limits.

pinneapple_data/cae/upload.py · 45 linhas · testes: 62

nome	tipo	assinatura	o que faz
expand	função	(files, max_unzipped=MAX_UNZIPPED) -> Dict[str, bytes]	

pinneapple_data.chunking

Chunking specifications and tensor iteration for streaming large arrays.

pinneapple_data/chunking.py · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ChunkSpec	classe	(mode, chunk_size, dim, dims, chunk_sizes)	Generic chunking specification for tensors.
iter_chunks	função	(x, spec) -> Iterator[torch.Tensor]	Iterate over chunks of a tensor according to a ChunkSpec.

pinneapple_data.collate

Collate functions for PINN batches and UPD supervision batches.

pinneapple_data/collate.py · 132 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
collate_pinn_batches	função	(batches) -> Dict[str, Any]	Collate a list of PINN batch dicts into a single batch dict.
move_batch_to_device	função	(batch, device) -> Dict[str, Any]	Move all tensors in a collated PINN batch dict to *device*. Leaves meta unchanged.
collate_upd_supervised	função	(samples) -> Dict[str, Any]	Collate a list of PhysicalSample objects for supervised training.

pinneapple_data.collocation

Automated collocation point generation for PINN training.

pinneapple_data/collocation.py · 558 linhas · testes: 7

nome	tipo	assinatura	o que faz
CollocationConfig	classe	(n_col, n_bc, n_ic, n_data, strategy, seed, oversample_bc, include_time, t_range)	Configuration for collocation point generation.
CollocationSampler	classe	(*, bounds, coord_names, sdf_fn=None, boundary_samplers=None, condition_value_fns=None, c…)	Unified collocation point generator for PINN training.

pinneapple_data.contracts

Open data contracts shared by the PINNeAPPle apps: ParameterSpace, DatasetSpec, ModelRef, StudyResult.

pinneapple_data/contracts/__init__.py · 51 linhas · testes: 175

pinneapple_data.contracts.expressions

Safe evaluation of constraint expressions: re-exported from :mod:pinneapple_data.expressions, which needs only NumPy (pinneapple check imports it without the contract models and their pydantic dependency).

pinneapple_data/contracts/expressions.py · 12 linhas · **sem teste direto**

pinneapple_data.contracts.models

The four data contracts shared by the PINNeAPPle apps and the library.

pinneapple_data/contracts/models.py · 675 linhas · testes: 40

nome	tipo	assinatura	o que faz
Contract	classe	(schema_version, metadata)	Base of the four contracts: JSON round trip, files, schema.
Parameter	classe	(name, kind, low, high, values, value, unit, log, group, default, description)	
ParameterSpace	classe	(kind, name, description, parameters, constraints, budget)	The parameters of a study and the constraints between them.

nome	tipo	assinatura	o que faz
Column	classe	(name, role, unit, dtype, description)	
FileRef	classe	(path, format, sha256, size, rows, description)	
Provenance	classe	(app, study_id, source, run_ids, library_version, created, notes)	
Splits	classe	(train, val, test, seed, id_column)	Either explicit row ids per split or fractions with a seed (and optionally the column holding ids).
DatasetSpec	classe	(kind, name, version, description, columns, files, n_rows, parameter_space, splits, prove…)	A table of cases or samples: what the columns are, where the files are, how it is split.
Variable	classe	(name, unit, low, high, description)	
DatasetRef	classe	(name, version, content_hash, uri)	
ModelRef	classe	(kind, name, version, model_type, uri, sha256, framework, inputs, outputs, dataset, metri…)	Where a trained model is and what it may be used for.
CheckRecord	classe	(name, status, value, threshold, evidence, detail)	One check. PASS / FAIL / WARN need a record (value or evidence): a check is only reported as run if it ran.
Table	classe	(columns, file)	Small tables inline (columns of equal length); large ones as a file.
StudyResult	classe	(kind, id, study_type, app, title, status, parameter_space, dataset, models, table, metri…)	The outcome of a study, with what went in, what came out and which checks ran.
from_dict	função	(data) -> Contract	Build the contract named by `data["kind"]`.
load	função	(path) -> Contract	Read a contract from JSON or YAML, whichever of the four it is.
validate	função	(data) -> list[str]	Problems with a contract given as a dict (empty list when valid), one readable line each.
json_schemas	função	() -> dict[str, dict[str, Any]]	
export_schemas	função	(directory) -> list[Path]	Write `<Kind>.schema.json` for each contract into `directory`.
sha256_file	função	(path, chunk=1 << 20) -> str	

pinneapple_data.contracts.tables

Tables on disk with their contract: CSV, Parquet and HDF5, each with a JSON sidecar.

`pinneapple_data/contracts/tables.py` · 150 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sidecar_path`	função	`(path) -> Path`	
`table_format`	função	`(path) -> str`	
`write_sidecar`	função	`(path, contract) -> Path`	Write `<path>.contract.json`.
`read_sidecar`	função	`(path) -> Contract | None`	
`write_table`	função	`(df, path, contract=None) -> Path`	Write a DataFrame (or a dict of columns) as CSV / Parquet / HDF5 / JSON records, plus the sidecar.
`read_table`	função	`(path)`	Read a table and its contract: `(DataFrame, contract or None)`.
`convert`	função	`(src, dst) -> Path`	Copy a table to another format (by suffix), carrying its contract along.

`pinneapple_data.data_health`

Engineering data health: a physics-aware quality report for sensor / test / plant data.

`pinneapple_data/data_health.py` · 998 linhas · testes: 37

nome	tipo	assinatura	o que faz
`Issue`	classe	`(category, severity, title, detail, column, affected_fraction, count, spans, suggestion, …)`	
`LoadInfo`	classe	`(format, rows, columns, delimiter, decimal, encoding, sheet, sheets, units_from_file, tru…)`	
`load_table`	função	`(data, filename, sheet=None, max_rows=1000000) -> Tuple[pd.DataFrame, LoadInfo]`	Read an uploaded table. Supported: .csv .tsv .txt .xlsx .xlsm .xls .parquet .json .jsonl .h5 .hdf5
`detect_time_column`	função	`(df) -> Tuple[Optional[str], Optional[pd.Series], str]`	
`analyze`	função	`(df, units=None, time_column=None, n_bins=160, series_points=600) -> Dict[str, Any]`	
`example_chiller_plant`	função	`(seed=7, days=7, inject=True) -> Tuple[pd.DataFrame, Dict[str, Any]]`	One week of 1-minute data from a chilled-water plant, with eight realistic faults injected.

`pinneapple_data.dataloaders`

DataLoader builders for UPD shards and PhysicalSample-based PINN training.

`pinneapple_data/dataloaders.py` · 353 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DataLoaderSpec`	classe	`(batch_size, num_workers, shuffle, pin_memory, drop_last)`	Configuration container for torch.utils.data.DataLoader parameters.
`build_upd_dataloader`	função	`(*, zarr_path, meta_path, mapping, sampling, loader=None, device='cpu', dtype=torch.float…)`	Construct a PyTorch DataLoader from a single UPD shard (Zarr + JSON).

nome	tipo	assinatura	o que faz
`build_physical_sample_dataloader`	função	`(sample, *, mapping, sampling, loader=None, device='cpu', dtype=torch.float32, length=100…)`	Construct a PyTorch DataLoader from a PhysicalSample instance.

pinneapple_data.datasets

Pre-built datasets for PINNeAPPle — physics, geometry, and time series.

`pinneapple_data/datasets/__init__.py` · 152 linhas · testes: 10

nome	tipo	assinatura	o que faz
`load_dataset`	função	`(dataset_id, **kwargs) -> Dict[str, np.ndarray]`	Load a pre-built dataset by ID.
`list_datasets`	função	`(category=None, verbose=True) -> List[DatasetInfo]`	Print and return all registered datasets.
`dataset_info`	função	`(dataset_id) -> DatasetInfo`	Return metadata for a specific dataset.
`dataset_ids`	função	`(category=None) -> List[str]`	Return list of all registered dataset IDs.

pinneapple_data.datasets.deepmind_mgn

DeepMind `meshgraphnets` datasets: partial download, TFRecord reader and conversion to tensors.

`pinneapple_data/datasets/deepmind_mgn.py` · 116 linhas · testes: 6

nome	tipo	assinatura	o que faz
`download_prefix`	função	`(dataset, split, dest, nbytes) -> Path`	Download `meta.json` and the first `nbytes` of `<split>.tfrecord` of a DeepMind dataset into `dest`.
`read_trajectories`	função	`(data_dir, split, max_traj) -> list[dict[str, np.ndarray]]`	Read up to `max_traj` trajectories of `<data_dir>/<split>.tfrecord` as dicts of numpy arrays.
`cylinder_flow_to_tensors`	função	`(traj, node_type_map=CYLINDER_TYPE_MAP) -> tuple[MeshGraph, torch.Tensor, torch.Tensor]`	`cylinder_flow` trajectory -> `(MeshGraph, velocity (T, N, 2), pressure (T, N, 1))`.

pinneapple_data.datasets.download

Download a URL once into a local cache, with retries (standard library only).

`pinneapple_data/datasets/download.py` · 85 linhas · testes: 4

nome	tipo	assinatura	o que faz
`cache_path`	função	`(url, cache_dir) -> Path`	The file :func:`fetch_cached` uses for `url`: `<cache_dir>/<md5(url)><ext>`.
`fetch_cached`	função	`(url, cache_dir, *, tries=4, timeout=60, backoff=2.0) -> Path`	Return a local copy of `url`, downloading it only if it is not cached yet.

pinneapple_data.datasets.external_sources

External data source integrations for PINNeAPPle.

`pinneapple_data/datasets/external_sources.py` · 1.168 linhas · testes: 5

nome	tipo	assinatura	o que faz
`MaterialsProjectClient`	classe	`(api_key=None) -> None`	Client for the Materials Project API.
`NOMADClient`	classe	`()`	Client for the NOMAD Laboratory REST API (public, no key needed).
`NASAClient`	classe	`()`	Client for NASA Open Data Portal (data.nasa.gov).
`NOAAClient`	classe	`()`	Client for NOAA NCEI / ERDDAP APIs.
`CopernicusClient`	classe	`()`	Client for the Copernicus Data Space (CDS + CDSE).
`BrazilDataCubeClient`	classe	`()`	Client for the Brazil Data Cube STAC API.
`CPTECClient`	classe	`()`	Client for CPTEC/INPE weather and climate data.
`ONSClient`	classe	`()`	Client for ONS (Brazilian National Grid Operator) open data.
`NRELClient`	classe	`(api_key=None) -> None`	Client for NREL open data.
`DesignSafeClient`	classe	`()`	Client for DesignSafe-CI (earthquake / structural data).
`OpenFOAMClient`	classe	`(openfoam_data_dir=None) -> None`	Provides representative CFD data from OpenFOAM tutorial cases.
`SU2Client`	classe	`(su2_results_dir=None) -> None`	Provides aerodynamic data representative of SU2 tutorial cases.
`NASAPowerClient`	classe	`()`	Client for the NASA POWER daily point API (meteorology and solar resource, no API key).
`OpenMeteoClient`	classe	`()`	Client for the Open-Meteo Historical Weather API (ERA5-based reanalysis, hourly, no API key; CC BY 4.0).

`pinneapple_data.datasets.geometry`

Pre-built geometry datasets for PINNeAPPle benchmarks.

`pinneapple_data/datasets/geometry.py` · 269 linhas · **sem teste direto**

`pinneapple_data.datasets.physics`

Pre-built physics datasets for PINNeAPPle benchmarks.

`pinneapple_data/datasets/physics.py` · 450 linhas · testes: 4

nome	tipo	assinatura	o que faz
`burgers_periodic_solve`	função	`(u0, T, nu, *, length=1.0, refine=None, cfl=0.5) -> np.ndarray`	Solve periodic viscous Burgers `u_t + (u^2/2)_x = nu u_xx` from `u0` (`(..., nx)`) to time `T`.
`burgers_periodic_pairs`	função	`(n, nx=128, T=1.0, nu=0.01, seed=0, *, length=1.0, refine=None)`	`n` random pairs `(u(x, 0), u(x, T))` of periodic viscous Burgers on `[0, length)` (operator learning).

`pinneapple_data.datasets.real_world`

Real-world datasets loaded from the web or optional Python libraries.

`pinneapple_data/datasets/real_world.py` · 1.665 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_real_dataset`	função	`(dataset_id) -> Dict[str, np.ndarray]`	Load a real-world dataset by ID.

pinneapple_data.datasets.registry

Central registry for pre-built PINNeAPPle datasets.

`pinneapple_data/datasets/registry.py` · 65 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DatasetInfo`	classe	`(id, name, category, description, fields, tags, license, reference)`	
`DatasetRegistry`	classe	`(_store)`	Central registry for pre-built PINNeAPPle datasets.

pinneapple_data.datasets.timeseries

Pre-built time-series datasets for PINNeAPPle benchmarks.

`pinneapple_data/datasets/timeseries.py` · 355 linhas · **sem teste direto**

pinneapple_data.device

Device utilities for pinning and moving PhysicalSample tensors between CPU and GPU.

`pinneapple_data/device.py` · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pin_sample`	função	`(sample) -> Any`	Pin all tensor entries inside a PhysicalSample-like structure.
`to_device_sample`	função	`(sample, device, *, dtype=None, non_blocking=True) -> Any`	Move all tensor entries inside a PhysicalSample-like structure to a target device.

pinneapple_data.doe

Design of experiments for simulation campaigns: designs, coverage, redundancy, priorities, campaign plans.

`pinneapple_data/doe/__init__.py` · 20 linhas · testes: 21

pinneapple_data.doe.campaign

Plan a simulation campaign: which cases to run, in which order, and what they add to what already ran.

`pinneapple_data/doe/campaign.py` · 174 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CampaignPlan`	classe	`(space, design, cases, unit, coverage_planned, coverage_existing, coverage_combined, redu…)`	
`plan_campaign`	função	`(space, n=None, method='lhs', *, existing=None, existing_id_column=None, outputs=(), seed…)`	Design `n` cases (default `space.budget`), drop those repeating an existing run (`drop_repeats`), order the rest by priority and measure the coverage of the plan, of what exists, and of both together. Existing cases outside the space (bounds or constraints) are left out of the coverage and …

pinneapple_data.doe.coverage

How well a set of cases covers a parameter space, which planned cases repeat what already ran, and in which order to run them.

`pinneapple_data/doe/coverage.py` · 119 linhas · testes: 14

nome	tipo	assinatura	o que faz
`coverage`	função	`(unit, names=None, *, n_probe=2 ** 14, seed=0, bins_2d=None, levels=None) -> dict[str, An…)`	Space-coverage metrics of points in [0, 1]^d.
`redundancy`	função	`(planned_unit, existing_unit=None, *, tol=None, existing_ids=None) -> dict[str, Any]`	Which planned cases (nearly) repeat an existing case or another planned case.
`prioritize`	função	`(planned_unit, existing_unit=None) -> dict[str, Any]`	Run order: greedy maximin over the planned cases, starting from what already exists, so every prefix of the list covers the space as evenly as possible (stop the campaign early and the cases run are still spread out). `score` is each case's distance to everything before it (existing included): …

pinneapple_data.doe.designs

Designs of experiments over a :class:`~pinneapple_data.contracts.ParameterSpace`.

`pinneapple_data/doe/designs.py` · 283 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Design`	classe	`(cases, unit, method, info)`	The cases of a design: values (DataFrame with every parameter), unit coordinates and how they were made.
`design`	função	`(space, n, method='lhs', *, seed=0, existing=None, outputs=(), levels=3, n_restarts=20, f…)`	`n` cases of `space` by `method` (see the module docstring). `existing` is a DataFrame of cases already run (a column per parameter, plus `outputs` for adaptive infill).
`fractional_factorial_unit`	função	`(k, n) -> tuple[np.ndarray, dict]`	Two-level 2^(k-p) design with at least `n` runs (or 2^k if smaller): the first k-p factors are crossed in full, each further factor is the product of a distinct subset of them (largest subsets first, which keeps the resolution high). Returns unit coordinates (0 / 1) and the generators and …
`greedy_maximin`	função	`(points, n, fixed=None, weights_fixed=None) -> np.ndarray`	Indices of `n` of `points` picked one by one, each the farthest from those already picked and from `fixed`. `weights_fixed` (one per fixed point, >= 1) favours the neighbourhoods to refine: a candidate's distance to the fixed points is multiplied by the weight of its nearest fixed point.
`min_distance`	função	`(unit) -> float`	

pinneapple_data.expressions

Safe evaluation of constraint expressions such as `"chord - 2 * thickness >= 0.01"`.

`pinneapple_data/expressions.py` · 131 linhas · testes: 22

nome	tipo	assinatura	o que faz
`ExpressionError`	classe	`()`	The expression uses something that is not allowed, or a name that is not a parameter.
`names_in`	função	`(expression) -> set[str]`	Parameter names an expression refers to (function names excluded).
`evaluate`	função	`(expression, values, functions=None) -> np.ndarray`	Evaluate `expression` with `values` (name -> scalar or column) and return a boolean (or numeric) array. `functions`: the names it may call (default :data:`FUNCTIONS`).
`check`	função	`(expression, names, functions=(), strings=False) -> None`	Raise :class:`ExpressionError` unless `expression` uses only the grammar above, the variables `names` and calls of `functions` (and no string literals unless `strings`).
`names_in_tree`	função	`(tree) -> set[str]`	

pinneapple_data.formfill

Fill engineering forms from the documents that hold the data: read datasheets and specifications, find every item a form asks for with its source, report conflicts between documents and missing data, and fill the fillable PDF.

`pinneapple_data/formfill/__init__.py` · 28 linhas · testes: 44

pinneapple_data.formfill.extract

Read datasheets and specifications, find each answer a form asks for, and keep where every value came from.

`pinneapple_data/formfill/extract.py` · 698 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Document`	classe	`(name, pages, tables, role, has_text, ocr_pages, ocr_conf, unread_pages)`	
`Candidate`	classe	`(key, value, raw, doc, page, snippet, method, confidence, unit, si, gauge, ocr)`	
`Decision`	classe	`(key, label, section, status, value, display, chosen, candidates, required, note)`	
`Compilation`	classe	`(decisions, tables, documents, spec)`	

nome	tipo	assinatura	o que faz
`read_document`	função	`(source, name='', role='', ocr='auto', max_ocr_pages=30) -> Document`	A PDF (path or bytes) or plain text (`.txt`/`.md` path, or bytes that are not a PDF).
`parse_value`	função	`(f, text) -> Optional[Dict[str, Any]]`	Interpret `text` as an answer to `f`; `None` when it does not look like one.
`extract_rules`	função	`(doc, spec) -> List[Candidate]`	Candidates for every item of `spec` whose label appears in `doc` (tables first, then text lines), plus the rows of the spec's tables.
`check_answer`	função	`(f, value) -> Any`	An engineer's answer for `f`, validated (`ValueError` names the accepted options).
`compile`	função	`(docs, candidates, spec, overrides=None) -> Compilation`	One decision per item of `spec`. Documents earlier in `docs` win conflicts (order = priority); the conflict is still reported with every source. `overrides` (an engineer's answers) win over everything and are marked as such.

pinneapple_data.formfill.fill

Write a compilation into the user's copy of a fillable PDF form.

`pinneapple_data/formfill/fill.py` · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pdf_widgets`	função	`(pdf) -> List[Dict[str, Any]]`	Every form widget of a PDF: qualified name, type (`/Tx`, `/Btn`…), page, tooltip, button states and whether it belongs to a radio group.
`form_values`	função	`(comp, spec=None) -> Tuple[Dict[str, str], Dict[str, str]]`	`(text fields, button states)` for the PDF, from the decisions and the table rows.
`fill_pdf`	função	`(blank_pdf, comp, spec=None, extra_text=None) -> bytes`	The filled form as PDF bytes. `blank_pdf` is the user's copy of the fillable form `spec` describes.
`read_filled`	função	`(pdf) -> Dict[str, Any]`	Field name -> value of a filled form (for checks and round trips).

pinneapple_data.formfill.llm

Optional extraction with a local LLM served by Ollama, for documents the label rules cannot read (free-form specifications, odd layouts, forms whose items have no usual label).

`pinneapple_data/formfill/llm.py` · 230 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OllamaError`	classe	`()`	
`OllamaClient`	classe	`(url=None, model=None, timeout=600.0, num_ctx=16384)`	Minimal client for the Ollama HTTP API (`/api/chat`, `/api/tags`).
`document_text`	função	`(doc, pages=None) -> str`	Page-marked text of a document, its tables written as `cell | cell` rows.

nome	tipo	assinatura	o que faz
verify_quote	função	(doc, quote, page) -> Optional[int]	Page (1-based) where quote occurs in the document text, preferring the stated page; None if absent.
extract_with_llm	função	(doc, spec, client=None, max_chars=24000) -> Tuple[List[Candidate], List[Dict[str, Any]]]	(accepted candidates, rejected answers with the reason) for one document.

pinneapple_data.formfill.ocr

OCR for scanned pages (pages without a text layer), with Tesseract.

pinneapple_data/formfill/ocr.py · 241 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
OcrPage	classe	(lines, tables, confidence, word_conf, angle)	
ocr_available	função	() -> bool	
ocr_image	função	(img) -> OcrPage	OCR one page image (PIL).
ocr_pdf_page	função	(pdf_bytes, index, dpi=DPI) -> OcrPage	OCR page index (0-based) of a PDF.

pinneapple_data.formfill.pipeline

One call from source documents to a compiled form, and the helpers a front end needs around it.

pinneapple_data/formfill/pipeline.py · 66 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
compile_documents	função	(docs, spec, client=None, overrides=None, max_ocr_pages=None) -> Tuple[Compilation, List[…]	Read every (name, data, role) document, extract candidates with the rules of spec (and with the local LLM client too, when given) and compile them into one decision per form item.
check_overrides	função	(spec, raw) -> Dict[str, Any]	User answers {item key: value} checked against spec (:func:check_answer): empty values are dropped, an unknown key or an invalid value raises ValueError with a message for the user.
output_filename	função	(comp, prefix='', day=None) -> str	File name for the filled form: <spec id>_<value of spec.filename_key or "filled">_<YYYYMMDD>.pdf, made of safe characters only.

pinneapple_data.formfill.report

The compiled datasheet as a PDF (needs reportlab): every item of the spec with its value and source, the open items (missing required data, conflicts between documents) first, side-by-side columns where the spec has them (shell side | tube side), the table rows and the documents read.

pinneapple_data/formfill/report.py · 166 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
render_datasheet	função	(comp, spec=None, title=None) -> bytes	

pinneapple_data.formfill.spec

What a form asks for, how documents phrase each item, and where each answer goes in the fillable PDF.

`pinneapple_data/formfill/spec.py` · 259 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Field`	classe	`(key, label, section, kind, pdf, states, patterns, quantity, required, source_hint, optio…)`	
`TableSpec`	classe	`(key, label, columns, header, combine, defaults, pdf_rows, description)`	A repeated-row item (nozzle schedule, line list...): found as a document table whose header matches `header` (every regex must match some header cell), filled into `pdf_rows` (one dict column -> PDF field per row).
`FormSpec`	classe	`(id, title, fields, sections, tables, description, notes_field, left_blank, filename_key,…)`	

pinneapple_data.formfill.specs

Built-in form templates. Add one by writing a module that builds a `FormSpec` and registering it here, or ship it as JSON and load it with `FormSpec.load`.

`pinneapple_data/formfill/specs/__init__.py` · 41 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`get_spec`	função	`(spec_id) -> FormSpec`	
`list_specs`	função	`() -> List[Dict[str, object]]`	

pinneapple_data.formfill.specs.asme_udr1

Built-in template: ASME BPVC Section VIII Division 1 Form U-DR-1, and where each answer goes in its fillable PDF.

`pinneapple_data/formfill/specs/asme_udr1.py` · 326 linhas · **sem teste direto**

pinneapple_data.formfill.specs.psv

Built-in template: pressure relief valve datasheet, with the data API 520 Part I (sizing) and API 526 (flanged steel valves) ask for. The output is a compiled datasheet generated by the tool, not a reproduction of a standard's form.

`pinneapple_data/formfill/specs/psv.py` · 115 linhas · **sem teste direto**

pinneapple_data.formfill.specs.pump

Built-in template: centrifugal pump datasheet, with the data API 610 asks the purchaser for. The output is a compiled datasheet generated by the tool, not a reproduction of the standard's data sheet.

`pinneapple_data/formfill/specs/pump.py` · 110 linhas · **sem teste direto**

`pinneapple_data.formfill.specs.shell_tube`

Built-in template: shell-and-tube heat exchanger datasheet, with the data a TEMA specification sheet and API 660 ask for. The output is a compiled datasheet generated by the tool, not a reproduction of a standard's form.

`pinneapple_data/formfill/specs/shell_tube.py` · 110 linhas · **sem teste direto**

`pinneapple_data.formfill.specs.tank`

Built-in template: atmospheric storage tank datasheet, with the data API 650 asks the purchaser for (Annex L lists them). The output is a compiled datasheet generated by the tool, not a reproduction of the standard's data sheet.

`pinneapple_data/formfill/specs/tank.py` · 117 linhas · **sem teste direto**

`pinneapple_data.geometry_features`

Geometry featurization: surface point-cloud sampling and a coarse signed-distance-field (SDF) grid from a triangle mesh — generic inputs (vertices/faces arrays, matching this package's own `stl_import.STLMesh` convention), no file-format or storage coupling. Useful as a preprocessing step for any …

`pinneapple_data/geometry_features.py` · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sample_point_cloud`	função	`(vertices, faces, n_samples=2048) -> np.ndarray`	Sample `n_samples` points uniformly (area-weighted) from a mesh's surface. `n_samples` is silently capped at 3x the face count for a very coarse mesh, since sampling far more points than the mesh has resolution to support doesn't add real information.
`compute_sdf_grid`	função	`(vertices, faces, grid_resolution=16, bounds=None) -> Dict[str, Any]`	Evaluate a signed distance field on a regular grid spanning the mesh's bounding box (or a caller-supplied `bounds` = [[xmin,ymin,zmin], [xmax,ymax,zmax]], e.g. to featurize several meshes on one shared grid).
`featurize_mesh`	função	`(vertices, faces, n_point_cloud_samples=2048, sdf_grid_resolution=16) -> Dict[str, Any]`	Convenience wrapper: point cloud + SDF grid in one call, with per-stage errors reported rather than raised (a degenerate/non-watertight mesh may still yield a usable point cloud even if the SDF query fails).

`pinneapple_data.grf_sampler`

Gaussian Random Field (GRF) input-function sampler, the standard way to draw random forcing/coefficient functions u(x) for operator-learning benchmarks (e.g. Lu, Jin & Karniadakis 2019, "DeepONet", arXiv:1910.03193, which draws every one of its input functions from a zero-mean GRF with an …

`pinneapple_data/grf_sampler.py` · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GRFDraw`	classe	`(x_grid, u_grid, spline)`	One sampled function u(x), x in [x_min, x_max].
`GRFDraw2D`	classe	`(x_grid, y_grid, u_grid, interp)`	One sampled 2D field u(x,y), (x,y) in [0,1]x[0,1] (normalized).
`sample_grf_batch`	função	`(n_draws, x_min=0.0, x_max=1.0, n_grid=1000, length_scale=0.2, jitter=1e-10, seed=None) -…)`	Draw n_draws independent zero-mean GRF paths sharing ONE Cholesky factorization of the RBF covariance on a common fine grid.
`sample_grf_paths`	função	`(n_draws, x_min=0.0, x_max=1.0, n_grid=1000, length_scale=0.2, jitter=1e-10, seed=None) -…)`	Alias for sample_grf_batch (kept for readability at call sites that draw a single small batch rather than a full dataset-generation pass).
`sample_grf_2d_batch`	função	`(n_draws, nx=64, ny=64, length_scale=0.2, seed=None) -> List[GRFDraw2D]`	Draw n_draws independent zero-mean, unit-variance 2D GRF paths on the normalized [0,1]x[0,1] domain via FFT spectral synthesis (see module docstring above). One (nx,ny) white-noise draw + one 2D FFT + one inverse 2D FFT per sample — no shared factorization to amortize across draws the way …

`pinneapple_data.lineage`

Engineering model lineage (digital thread): artifacts and derivations, checks, questions, W3C PROV export, and auto-detection of the lineage from a project's files.

`pinneapple_data/lineage/__init__.py` · 7 linhas · testes: 13

`pinneapple_data.lineage.detect`

Auto-detect a project's lineage from its files: OpenFOAM cases (blockMeshDict → mesh → run), Gmsh .geo → .msh, CalculiX decks and results, datasets exported by the Interoperability Hub (linked to their source files by sha256), Comparator / Mesh Quality / Preflight reports, PINNeAPPle model cards, …

`pinneapple_data/lineage/detect.py` · 304 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`detect_project`	função	`(fs, project='') -> Lineage`	

`pinneapple_data.lineage.graph`

Engineering model lineage: artifacts (CAD, mesh, simulation, dataset, model, prediction, ...) linked by "derived from" edges into a digital thread, with the checks that make it trustworthy and the questions it answers.

`pinneapple_data/lineage/graph.py` · 365 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Artifact`	classe	`(id, kind, name, file, version, software, parameters, timestamp, hash, owner, origin, inp…)`	

nome	tipo	assinatura	o que faz
Lineage	classe	(artifacts=(), project='')	
sha256	função	(b) -> str	

pinneapple_data.noether_dataset

Noether ↔ PINNeAPPle data bridge.

pinneapple_data/noether_dataset.py · 216 linhas · testes: 5

nome	tipo	assinatura	o que faz
NoetherDatasetBridge	classe	(noether_dataset, input_fields=None, target_fields=None, transform=None, device=None) -> …)	Bridge a noether.data.base.Dataset to a plain PyTorch Dataset.
PhysicalSampleDataset	classe	(samples, input_fields=None, output_fields=None, include_coords=False, transform=None) ->…)	Convert a list of PINNeAPPle PhysicalSample (UPD) to a Noether-compatible {field: tensor} PyTorch Dataset.
split_physical_samples	função	(samples, val_fraction=0.1, seed=42, **dataset_kwargs)	Split a list of PhysicalSamples into train and validation datasets.

pinneapple_data.parameter_sampling

Design-of-experiments parameter-space sampling.

pinneapple_data/parameter_sampling.py · 118 linhas · testes: 5

nome	tipo	assinatura	o que faz
sample_parameters	função	(param_ranges, n_samples, method='lhs', seed=0) -> list[dict[str, float]]	Sample n_samples points from a bounded parameter space.
saltelli_perturbation_sweep	função	(nominal, param_ranges, n_base=8, seed=0) -> list[dict[str, float]]	Build a Saltelli-style sample set for first-order/total-order Sobol global sensitivity analysis, without requiring the SALib dependency.

pinneapple_data.physical_sample

PhysicalSample dataclass and training bridge for UPD-aligned physical data.

pinneapple_data/physical_sample.py · 241 linhas · testes: 28

nome	tipo	assinatura	o que faz
PhysicalSample	classe	(state, geometry, schema, domain, provenance, extras)	Unified Physical Sample (UPD-aligned) used across Pinneapple.

pinneapple_data.physical_units

Physical units and quantities for engineering data (PINNeAPPle Physical Data Layer).

pinneapple_data/physical_units.py · 436 linhas · testes: 52

nome	tipo	assinatura	o que faz
`Unit`	classe	`(symbol, factor, dim, offset, gauge)`	A unit: `SI value = value * factor + offset`.
`UnitError`	classe	`()`	
`Quantity`	classe	`(name, si_unit, dim, keywords, hard, typical, display)`	
`QuantityGuess`	classe	`(quantity, confidence, reason, alternatives, is_difference)`	
`parse_unit`	função	`(text) -> Unit`	Parse a unit expression: `°C`, `kg/m^3`, `W/(m·K)`, `m³/h`, `N*m`, `psig`.
`try_parse_unit`	função	`(text) -> Optional[Unit]`	
`convert`	função	`(values, src, dst, difference=False)`	Convert values (scalar or numpy array) from `src` to `dst`.
`dim_str`	função	`(dim) -> str`	
`split_header`	função	`(header) -> Tuple[str, Optional[str]]`	Split a column header into (name, unit text). `"T_supply [°C]"` -> `("T_supply", "°C")`; also understands `Pressure (bar)`, `flow, m3/h`, `power in kW` and suffixes like `T_inlet_degC`, `pressure_bar` or `vibration_mm_s`. Returns `(header, None)` when no unit is found. A short …
`infer_quantity`	função	`(name, unit=None) -> QuantityGuess`	Infer the physical quantity of a column from its unit dimension and name keywords.
`describe_unit`	função	`(u) -> Dict[str, object]`	
`si_unit_for`	função	`(quantity) -> str`	

`pinneapple_data.physics_case`

PhysicsCase -- a minimal bridge between the three disjoint "partial canonical representations" of a physics problem already in this repo:

`pinneapple_data/physics_case.py` · 405 linhas · testes: 47

nome	tipo	assinatura	o que faz
`BenchmarkComparison`	classe	`(benchmark, x_vars, y_vars, n_reference_points, n_compared_points, rmse, relative_l2_erro…)`	Plain numeric comparison produced by `PhysicsCase.validate_against_benchmark()`.

nome	tipo	assinatura	o que faz
`PhysicsCase`	classe	`(name, geometry, geometry_kind, physics, solver, solver_config, results, reference_benchm…)`	One physics case: geometry + physics spec + solver choice + results, referenced together.

`pinneapple_data.prefetch_adaptive`

Adaptive prefetch IterableDataset for Zarr-backed UPD stores with configurable queue control.

`pinneapple_data/prefetch_adaptive.py` · 281 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AdaptivePrefetchConfig`	classe	`(queue_max, min_target_fill, max_target_fill, target_fill_init, control_period_s, increas…)`	Configuration for adaptive prefetching behavior.
`AdaptivePrefetchZarrUPDIterable`	classe	`(root, *, fields=None, coords=None, dtype=None, start=0, end=None, stride=1, sample_ctor=…)`	IterableDataset that streams samples from a cached Zarr-backed UPD store with adaptive prefetching.

`pinneapple_data.preflight`

Simulation pre-flight: check an OpenFOAM case or a CalculiX/Abaqus deck before spending solver hours on it.

`pinneapple_data/preflight/__init__.py` · 74 linhas · testes: 13

nome	tipo	assinatura	o que faz
`preflight`	função	`(fs) -> Dict[str, Any]`	
`is_result_file`	função	`(path) -> bool`	A solver result rather than an input: inside an OpenFOAM time directory other than `0` (`0.5/U`, `1e-3/p`) or a CalculiX / VTK result file. Not needed for a pre-flight; drop such files before checking.
`drop_results`	função	`(fs) -> Dict[str, bytes]`	`fs` without the solver result files (:func:`is_result_file`).
`attach_snippets`	função	`(report, fs, context=4, width=160) -> None`	Add to every finding that names a file and line the source lines around it, in place: `finding["snippet"] = {"start": first line number, "lines": [...], "at": line}` (`context` lines on each side, each cut to `width` characters). File names are resolved against the case root of the report.

`pinneapple_data.preflight.calculix`

Pre-flight rules for CalculiX (and flat Abaqus) input decks: mesh, sections and materials, the properties each procedure needs, unit consistency, supports and loads, initial conditions and step settings.

`pinneapple_data/preflight/calculix.py` · 450 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check`	função	`(fs, main) -> Report`	

pinneapple_data.preflight.common

Findings and the pre-flight report shared by the OpenFOAM and CalculiX rule sets.

`pinneapple_data/preflight/common.py` · 83 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Finding`	classe	`(section, status, title, detail, fix, file, line, entity, rule, evidence)`	
`Report`	classe	`(solver, findings, info)`	
`line_of`	função	`(text, pattern, flags=re.M) -> Optional[int]`	
`num`	função	`(v) -> Optional[float]`	Last number in an OpenFOAM entry ('nu [0 2 -1 0 0 0 0] 1e-05' -> 1e-05, 'uniform 0' -> 0).
`vec`	função	`(v) -> Optional[List[float]]`	

pinneapple_data.preflight.openfoam

Pre-flight rules for OpenFOAM cases: mesh, transport/thermo properties, boundary and initial conditions, schemes, linear solvers and time stepping, checked before the solver runs.

`pinneapple_data/preflight/openfoam.py` · 551 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check`	função	`(fs, root='') -> Report`	

pinneapple_data.process_optimizer

Data-driven process optimization for industrial plants (plain machine learning, no physics model).

`pinneapple_data/process_optimizer.py` · 436 linhas · testes: 16

nome	tipo	assinatura	o que faz
`suggest_roles`	função	`(columns) -> Dict[str, Any]`	Guess target / levers / context from column names (to be reviewed by a person).
`clean_for_modeling`	função	`(df, report) -> Tuple[pd.DataFrame, List[str]]`	Time-sorted numeric table with the problems found by the health report set to missing.
`time_series_cv`	função	`(X, y, n_folds=4, l2_grid=L2_GRID, seed=0) -> Dict[str, Any]`	Forward-chaining cross-validation on the training period: fold k trains on the first k blocks and is scored on the next one (never on the past), for every L2 regularisation strength in `l2_grid`. Picks the strength with the lowest mean validation MAE; the per-fold errors show how stable the model …
`fit_and_optimize`	função	`(data, target, levers, context, goal='minimize', constraints=(), bounds=None, max_move=0.…)`	Train on the older part of the record, evaluate and optimize on the newest part. The L2 regularisation of the KPI model is chosen by forward-chaining cross-validation inside the training part (`cv_folds`; 0 keeps the default) and the fold errors go in the report under `model.cv`.

nome	tipo	assinatura	o que faz
`plant_physics`	função	`(oat, rh, load, chws_sp, cws_sp, pump_pct)`	True plant behaviour used by the example (not visible to the ML model).
`example_plant_operations`	função	`(seed=11, days=60) -> Tuple[pd.DataFrame, Dict[str, Any]]`	60 days of 15-minute data from a chilled-water plant run by operators with manual setpoint habits.
`true_saving`	função	`(result) -> Dict[str, Any]`	For the example: score the recommendations with the plant's true physics.

`pinneapple_data.serialization`

Serialization utilities for PhysicalSample to/from PT, Zarr, and HDF5 formats.

`pinneapple_data/serialization.py` · 338 linhas · testes: 11

nome	tipo	assinatura	o que faz
`save_manifest`	função	`(path, manifest) -> None`	
`load_manifest`	função	`(path) -> Dict[str, Any]`	
`save_pt`	função	`(samples, path, *, compress=False) -> None`	Save a sequence of PhysicalSamples to a PyTorch (.pt) file.
`load_pt`	função	`(path) -> List[PhysicalSample]`	Load PhysicalSamples from a PyTorch (.pt) file.
`save_hdf5`	função	`(samples, path) -> None`	Save PhysicalSamples to an HDF5 file.
`save_hdf5`	função	`(samples, path) -> None`	
`load_hdf5`	função	`(path) -> List[PhysicalSample]`	
`save_zarr`	função	`(samples, root, *, compressor='default') -> None`	Save PhysicalSamples to a Zarr directory store using UPDZarrStore.
`load_zarr`	função	`(root)`	Load PhysicalSamples from a Zarr directory store using UPDZarrStore.

`pinneapple_data.simulation_metadata`

Simulation metadata extraction (PINNeAPPle): solver, mesh, time step, boundary conditions, parameters and convergence from the files a simulation leaves behind.

`pinneapple_data/simulation_metadata/__init__.py` · 346 linhas · testes: 25

nome	tipo	assinatura	o que faz
`read_upload`	função	`(files) -> dict[str, bytes]`	Turn uploaded files (zips are expanded, with size limits) into {relative path: bytes}.
`parse_residual_table`	função	`(text) -> dict[str, Any] | None`	
`extract`	função	`(fs) -> dict[str, Any]`	

`pinneapple_data.simulation_metadata.calculix`

CalculiX / Abaqus input decks (.inp) and CalculiX run files (.sta, .cvg, .dat, solver output).

`pinneapple_data/simulation_metadata/calculix.py` · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`parse_inp`	função	`(text, include=None) -> Dict[str, Any]`	Parse an input deck into model / steps / materials / BCs / loads. `include` maps file names to the text of *INCLUDE'd files that were uploaded too.
`parse_sta`	função	`(text) -> List[Dict[str, Any]]`	
`parse_cvg`	função	`(text) -> List[Dict[str, Any]]`	
`parse_dat`	função	`(text) -> Dict[str, Any]`	Largest printed displacement (or min/max of other printed results) at the last time of each set.
`parse_ccx_log`	função	`(text) -> Dict[str, Any]`	
`assess`	função	`(sta, cvg, deck, log) -> Dict[str, Any]`	

`pinneapple_data.simulation_metadata.convergence`

Convergence assessment from residual histories: a verdict, the reasons and what to do next.

`pinneapple_data/simulation_metadata/convergence.py` · 228 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`downsample`	função	`(x, series, k=1500) -> Dict[str, Any]`	
`assess_steady`	função	`(log, targets) -> Dict[str, Any]`	
`assess_transient`	função	`(log, tolerances, max_co, algorithm, reached_end) -> Dict[str, Any]`	
`assess_table`	função	`(iters, residuals, is_log10) -> Dict[str, Any]`	Generic residual history (Fluent / STAR-CCM+ / SU2 exports).

`pinneapple_data.simulation_metadata.foam_dict`

A tolerant parser for OpenFOAM dictionary files (controlDict, fvSchemes, fvSolution, 0/U, ...).

`pinneapple_data/simulation_metadata/foam_dict.py` · 176 linhas · testes: 16

nome	tipo	assinatura	o que faz
`parse`	função	`(text, max_list=2000) -> Dict[str, Any]`	Parse a FoamFile. Very long lists (mesh points, nonuniform fields) are summarised as `"<list of N>"` beyond `max_list` entries to keep the result small.

nome	tipo	assinatura	o que faz
`header`	função	`(text) -> Dict[str, str]`	The FoamFile header (class, object, format, note) without parsing the body.
`scalar`	função	`(v) -> Optional[float]`	Last number in an entry: 'nu [0 2 -1 0 0 0 0] 1e-05' -> 1e-05; 'uniform 0' -> 0.0.
`vector`	função	`(v) -> Optional[List[float]]`	'uniform (10 0 0)' -> [10, 0, 0].
`words`	função	`(v) -> List[str]`	
`boundary`	função	`(text) -> List[Dict[str, Any]]`	Patches of a polyMesh/boundary file: name, type, nFaces, startFace, inGroups.

`pinneapple_data.simulation_metadata.openfoam`

OpenFOAM case + log extraction for the simulation metadata record.

`pinneapple_data/simulation_metadata/openfoam.py` · 523 linhas · testes: 9

nome	tipo	assinatura	o que faz
`parse_log`	função	`(text) -> dict[str, Any]`	Parse an OpenFOAM solver log (or blockMesh / checkMesh log).
`find_root`	função	`(fs) -> str | None`	Case root: the directory that holds system/controlDict (shortest such path).
`parse_dat`	função	`(text) -> tuple[list[str], np.ndarray]`	Parse an OpenFOAM function-object table (forceCoeffs.dat, forces.dat, probes, residuals.dat, ...).
`read_dat`	função	`(path) -> tuple[list[str], np.ndarray]`	:func:`parse_dat` of the file at `path` (an OpenFOAM `postProcessing/<name>/<time>/<table>.dat`).
`extract_case`	função	`(fs, root) -> dict[str, Any]`	
`summarise`	função	`(case) -> dict[str, Any]`	Turn the raw case extraction into the record sections.

`pinneapple_data.snapshots`

Snapshot matrices for reduced-order models: many results of one model (time steps or parameter points) on one mesh, stacked into `X` with one row per snapshot.

`pinneapple_data/snapshots.py` · 413 linhas · testes: 31

nome	tipo	assinatura	o que faz
`SnapshotSet`	classe	`(X, fields, times, params, param_names, locations, source, meta, cells, snapshot_points)`	`X` (n_snapshots, n_values) plus what each row and column is.
`snapshots_from_arrays`	função	`(arrays, *, times=None, params=None, param_names=None, locations=None, where=None, source…)`	Build a set from field arrays with the snapshots on the first axis: (n, m) scalars or (n, m, c) vectors (more axes are flattened into m). `cells`: (VTK type, connectivity) of the point mesh; `snapshot_points`: (n, m, 3) node coordinates per snapshot.

nome	tipo	assinatura	o que faz
`load_snapshots`	função	`(files, *, fields=None, param_columns=None) -> SnapshotSet`	Read an upload into a :class:`SnapshotSet` (see the module docstring for the formats).

`pinneapple_data.splits`

Train/val/test split strategies (random, time, group).

`pinneapple_data/splits.py` · 104 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SplitSpec`	classe	`(method, train, val, test, seed)`	Split strategies: - random: shuffle then split by ratios - time: keep order; split by ratios (train earliest, test latest) - group: ensure groups don't leak (requires group_ids)
`split_indices`	função	`(n, spec, *, group_ids=None) -> Dict[str, np.ndarray]`	

`pinneapple_data.stl_import`

STL import utilities (ASCII + binary).

`pinneapple_data/stl_import.py` · 93 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`STLMesh`	classe	`(verts, faces)`	
`load_stl_bytes`	função	`(data, *, dedup_eps=1e-06) -> STLMesh`	
`load_stl`	função	`(path, *, dedup_eps=1e-06) -> STLMesh`	

`pinneapple_data.synth`

Módulo sem docstring e sem texto.

`pinneapple_data/synth/__init__.py` · 38 linhas · **sem teste direto**

`pinneapple_data.synth.base`

Base SynthConfig, SynthOutput, and SynthGenerator protocol for synthetic data generation.

`pinneapple_data/synth/base.py` · 66 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SynthConfig`	classe	`(seed, device, dtype)`	Configuration container for synthetic data generation.
`SynthOutput`	classe	`(samples, extras)`	Standard output structure for synthetic generators.
`SynthGenerator`	classe	`()`	Protocol defining the interface for synthetic data generators.

`pinneapple_data.synth.curvefit`

Curve-fit synthetic generator for polynomial ridge regression and sample synthesis.

`pinneapple_data/synth/curvefit.py` · 153 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CurveFitSynthGenerator`	classe	`(cfg=None)`	Synthetic generator that fits a polynomial trend to 1D data and synthesizes samples.

`pinneapple_data.synth.distributions`

Distribution-based synthetic generator for Gaussian, uniform, and mixture-of-Gaussians sampling.

`pinneapple_data/synth/distributions.py` · 154 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DistributionSynthGenerator`	classe	`(cfg=None)`	Synthetic generator for sampling inputs from simple probability distributions.

`pinneapple_data.synth.geometry`

Geometry synthesis and mesh augmentation from trimesh-like inputs.

`pinneapple_data/synth/geometry.py` · 221 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometrySynthGenerator`	classe	`(cfg=None)`	Geometry synthesis and augmentation from an existing triangle mesh.

`pinneapple_data.synth.geometry_cadquery`

CadQuery-based geometry synthesis: parametric templates and STL-driven variants.

`pinneapple_data/synth/geometry_cadquery.py` · 532 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ParametricCadQuerySynthGenerator`	classe	`(cfg=None)`	Generate geometry variants from a parametric CadQuery template.
`STLTemplateSynthGenerator`	classe	`(cfg=None)`	Synthetic geometry generator driven by an existing STL/mesh template.

`pinneapple_data.synth.images`

Image reconstruction and inpainting synthetic generator.

`pinneapple_data/synth/images.py` · 262 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ImageReconstructionSynthGenerator`	class e	`(cfg=None)`	Synthetic generator for image reconstruction tasks.

`pinneapple_data.synth.operators`

Módulo sem docstring e sem texto.

`pinneapple_data/synth/operators/__init__.py` · 6 linhas · **sem teste direto**

`pinneapple_data.synth.operators.masks`

Random block mask generation for 2D grids.

pinneapple_data/synth/operators/masks.py · 38 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
random_block_mask	função	(H, W, frac=0.2, seed=42) -> torch.Tensor	Generate a random rectangular block mask within a 2D grid.

pinneapple_data.synth.operators.noise

Gaussian noise addition for synthetic data augmentation.

pinneapple_data/synth/operators/noise.py · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
add_gaussian_noise	função	(x, std=0.01, seed=42) -> torch.Tensor	Add Gaussian noise to a tensor.

pinneapple_data.synth.operators.transforms

Tensor normalization transforms for synth operators.

pinneapple_data/synth/operators/transforms.py · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
normalize_01	função	(x, eps=1e-08) -> torch.Tensor	Normalize a tensor to the [0, 1] range.

pinneapple_data.synth.pde

PDE/ODE synthetic generator for heat, advection, and logistic equations.

pinneapple_data/synth/pde.py · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SimplePhysicalSample	classe	(fields, coords, meta)	MVP duck-typed PhysicalSample to keep synth module usable standalone.
PDESynthGenerator	classe	(cfg=None)	PDE/ODE synthetic generator.

pinneapple_data.synth.pde_symbolic

Symbolic PDE parser and finite-difference-based synthetic trajectory generator.

pinneapple_data/synth/pde_symbolic.py · 498 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SymbolicFDSynthGenerator	classe	(cfg=None)	Symbolic equation -> automatic finite difference operator -> synthetic dataset.
make_fd_residual_fn	função	(equation, *, parameters=None, bc='periodic')	Create a residual evaluator using finite-difference approximations.

pinneapple_data.synth.registry

SynthCatalog factory for building synthetic generators by name from a registry.

pinneapple_data/synth/registry.py · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SynthCatalog`	classe	`(registry)`	Factory-style catalog for synthetic data generators.

`pinneapple_data.synth.sample_adapter`

PhysicalSample-like conversion and torchify utilities for synth module outputs.

`pinneapple_data/synth/sample_adapter.py` · 192 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicalSampleLike`	classe	`(fields, coords, meta)`	Protocol defining the minimal interface of a PhysicalSample-like object.
`has_pinnego_physical_sample`	função	`() -> bool`	Check whether `pinneapple_data.physical_sample.PhysicalSample` is importable.
`to_physical_sample`	função	`(sample_like, *, device=None, dtype=None) -> Any`	Best-effort conversion: - if already a PhysicalSample -> return - if duck-typed {fields, coords, meta} -> convert - else if dict with keys -> interpret

`pinneapple_data.transforms`

Data transforms and normalizers for preprocessing pipelines.

`pinneapple_data/transforms.py` · 140 linhas · testes: 2 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Normalizer`	classe	`()`	Abstract base for all normalizers (fit / transform / inverse).
`StandardScaler`	classe	`(eps, mean_, std_)`	Zero-mean unit-variance scaler.
`MinMaxScaler`	classe	`(eps, min_, max_)`	Scales each feature to [0, 1].

`pinneapple_data.udr1`

ASME Form U-DR-1 shortcuts over the generic `pinneapple_data.formfill` engine (template `asme_u-dr-1`).

`pinneapple_data/udr1/__init__.py` · 30 linhas · testes: 36

nome	tipo	assinatura	o que faz
`extract_rules`	função	`(doc) -> List[Candidate]`	
`compile_requirements`	função	`(docs, candidates, overrides=None) -> Compilation`	
`fill_form`	função	`(blank_pdf, comp) -> bytes`	

`pinneapple_data.unified`

Engineering data standardizer: many files, many conventions -> one physical schema (PINNeAPPle UPD).

`pinneapple_data/unified.py` · 584 linhas · testes: 24

nome	tipo	assinatura	o que faz
canonical_name	função	(base, quantity) -> str	'CHW Supply Temp' + temperature -> 'chw_supply_temperature'; 'chws_temp' -> same.
conversion_text	função	(src, dst, difference=False) -> str	Human-readable conversion, e.g. '°F → °C: (x − 32) × 0.5556'.
propose_mapping	função	(df, source, system='metric', time_column=None, units=None, tz=None) -> Dict[str, Any]	Draft the mapping of one source from its headers (to be reviewed by a person).
make_recipe	função	(mappings, system='metric', resample='auto', layout='wide', agg='mean') -> Dict[str, Any]	
standardize	função	(sources, recipe) -> Dict[str, Any]	Apply a recipe to sources [(file name, dataframe, raw bytes)]. Returns a dict with `wide` and `long` DataFrames, the `manifest`, per-source notes and cross-source `agreement`.
write	função	(result, fmt='csv', layout='wide') -> Tuple[bytes, str, str]	Serialize a result. Returns (bytes, media type, file name).
to_physical_sample	função	(result)	The wide table as a PINNeAPPle :class:`~pinneapple_data.physical_sample.PhysicalSample` (xarray Dataset on a `time` coordinate, units in attributes).
example_sources	função	(seed=3) -> List[Tuple[str, bytes]]	The same chilled-water plant recorded by a building management system (CSV, local time, °C, semicolon + decimal comma), a SCADA REST API (JSON, Unix ms, °F, gpm, tons) and a test rig (Excel, ISO UTC, K, L/s, W).
before_after_series	função	(result, k=500) -> Dict[str, Any]	Per variable of a :func:`standardize` result: the raw values of every source (in their original units) and the standardized values, on the common UTC time axis, downsampled to at most `k` evenly spaced points.

pinneapple_data.upd_dataset

Hardened UPDDataset wrapper with validation, persistence, and save/load support.

pinneapple_data/upd_dataset.py · 192 linhas · testes: 5

nome	tipo	assinatura	o que faz
UPDDataset	classe	(samples, manifest)	Hardened dataset wrapper.

pinneapple_data.upd_types

Pure data-structure types for the UPD (Unified Physical Data) pipeline.

pinneapple_data/upd_types.py · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
UPDItem	classe	(zarr_path, meta_path)	One UPD shard item: points to a Zarr store and a JSON metadata file.
ConditionSpec	classe	(name, type, equation, n, options)	Sampling definition for a single PINN condition.

nome	tipo	assinatura	o que faz
`SamplingSpec`	classe	`(n_collocation, conditions, n_data, replace, seed)`	Overall sampling plan for a UPD shard.
`Batch`	classe	`(collocation, conditions, data)`	PINN-ready training batch.
`seconds_since`	função	`(t, t0=None) -> np.ndarray`	Convert a `datetime64` array to float seconds elapsed since *t0*.

`pinneapple_data.validators`

PhysicalSample validation: units, ranges, non-negativity, and monotonic coordinate checks.

`pinneapple_data/validators.py` · 161 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ValidationIssue`	classe	`(level, message, field)`	Single validation issue from validate_physical_sample.
`validate_physical_sample`	função	`(sample, *, units_policy='warn', required_units=None, ranges=None, non_negative=None, mon…)`	Validate a PhysicalSample and return a list of issues.
`assert_valid_physical_sample`	função	`(*args, **kwargs) -> None`	Validate a PhysicalSample and raise ValueError if any errors are found.

`pinneapple_data.zarr_cached_store`

Cached wrapper around UPDZarrStore with LRU sample and field-level caching.

`pinneapple_data/zarr_cached_store.py` · 231 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZarrCacheConfig`	classe	`(max_samples, max_fields, enable_field_cache)`	Configuration for Zarr-backed cache limits.
`CachedUPDZarrStore`	classe	`(root, *, cache=None, mode='r')`	Cached wrapper around UPDZarrStore.

`pinneapple_data.zarr_cached_store_bytes`

Byte-cached UPDZarrStore wrapper with configurable sample and field cache budgets.

`pinneapple_data/zarr_cached_store_bytes.py` · 226 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZarrByteCacheConfig`	classe	`(max_sample_bytes, max_field_bytes, enable_field_cache)`	Configuration for byte-bounded Zarr cache limits.
`CachedUPDZarrStoreBytes`	classe	`(root, *, cache=None, mode='r')`	Cached wrapper around UPDZarrStore using ByteLRUCache.

`pinneapple_data.zarr_iterable`

IterableDataset for streaming PhysicalSamples from a UPDZarrStore with worker-aware sharding.

`pinneapple_data/zarr_iterable.py` · 69 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZarrUPDIterable`	classe	`(root, fields, coords, start, stop, step, shuffle, seed)`	IterableDataset over a UPDZarrStore.

`pinneapple_data.zarr_prefetch`

Prefetch IterableDataset for Zarr-backed UPD stores with background producer and optional GPU transfer.

`pinneapple_data/zarr_prefetch.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PrefetchConfig`	classe	`(prefetch, queue_max, use_sample_cache, pin_memory, target_device, transfer_non_blocking)`	Industrial prefetch controls.
`PrefetchZarrUPDIterable`	classe	`(root, *, fields=None, coords=None, dtype=None, start=0, end=None, stride=1, sample_ctor=…)`	Streaming dataset backed by Zarr with caching + background prefetch, plus pinned memory and async CPU->GPU transfer.

`pinneapple_data.zarr_shard_iterable`

Shard-aware IterableDataset for balanced iteration over multi-shard Zarr UPD stores.

`pinneapple_data/zarr_shard_iterable.py` · 162 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ShardAwareConfig`	classe	`(pin_memory, target_device, transfer_non_blocking, use_sample_cache)`	Configuration for shard-aware iteration behavior.
`ShardAwareZarrUPDIterable`	classe	`(root, *, fields=None, coords=None, dtype=None, sample_ctor=None, cache=None, cfg=None)`	Shard-aware iterator: - reads root/index.json - balances shard parts across DataLoader workers - iterates each assigned zarr store sequentially

`pinneapple_data.zarr_shards`

Sharded Zarr writer for partitioning UPD samples by key with index-based discovery.

`pinneapple_data/zarr_shards.py` · 154 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ShardSpec`	classe	`(key_fn, max_per_shard)`	Simple shard naming.
`UPDZarrShardedWriter`	classe	`(root, shard_spec, *, shards_dirname='shards', index_filename='index.json')`	Writes multiple shards: root/ shards/ /part=0000/data.zarr /part=0001/data.zarr index.json

`pinneapple_data.zarr_store`

Zarr-backed UPD store for PhysicalSample persistence with columnar layout.

`pinneapple_data/zarr_store.py` · 396 linhas · testes: 1

nome	tipo	assinatura	o que faz
`UPDZarrStore`	classe	`(root, mode='r')`	Minimal UPD Zarr store:

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 45 de 115.
- Funções de teste em arquivos que importam algum módulo do pacote: 507.

Limitações conhecidas

- *Nenhuma registrada.*

3. `pinneapple_pdb`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
7	1.929	9	14	100 (5 arquivos)	5

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`benchmarks`		1	5	35
`builder`		1	5	0
`derived`		1	3	0
`shard`		1	6	0
`templates`		1	1	0
`validate`		1	3	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_pdb`

Módulo sem docstring e sem texto.

`pinneapple_pdb/__init__.py` · 36 linhas · testes: 65

`pinneapple_pdb.benchmarks`

A small, curated, NAME-based catalog of real benchmark datasets.

`pinneapple_pdb/benchmarks.py` · 984 linhas · testes: 35

nome	tipo	assinatura	o que faz
`BenchmarkEntry`	classe	`(name, description, reference_source, x_vars, y_vars, reference_x, reference_y, verificat…)`	A single named, real reference dataset: enough to plug directly into `pinneapple_llm.guardrail.PhysicsGuardrail._check_reference` as `(reference_x, reference_y)`.
`register_benchmark`	função	`(name)`	Decorator to register a zero-argument `BenchmarkEntry` factory under `name` -- mirrors `pinneapple_physics.pde_environment.presets.registry.register_preset`'s decorator pattern. The factory is called fresh on every `get_benchmark`/`benchmark_catalog` lookup (not memoised), so the …
`get_benchmark`	função	`(name) -> BenchmarkEntry`	Resolve a benchmark dataset by name.
`list_benchmarks`	função	`() -> List[str]`	Return the sorted list of all registered benchmark dataset names.
`benchmark_catalog`	função	`() -> Dict[str, BenchmarkEntry]`	Return name -> `BenchmarkEntry` for every registered benchmark, built (and independently re-verified) fresh.

`pinneapple_pdb.builder`

Builder for physical datasets from Earth data hubs (NASA CMR, earthaccess).

`pinneapple_pdb/builder.py` · 539 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HubQuery`	classe	`(provider, short_name, keyword)`	Query parameters for Earth data hub search.
`SpaceTime`	classe	`(time_start, time_end, bbox, stride, chunking)`	Temporal and spatial extent plus stride/chunking options.
`VariableSelection`	classe	`(include, exclude, packs)`	Variable include/exclude/pack selection.
`PhysicalSchema`	classe	`(physical_system, governing_equations, ics, bcs, forcings, units_policy, regime_tags, val…)`	Physical system metadata: equations, BCs, ICs, units policy, regime tags.
`PhysicalDatasetBuilder`	classe	`(packs=None)`	MVP-1 builder: - time-window sharding (and optional spatial tiles) - derived: vorticity/divergence - validations: units/ranges/monotonic - CLI friendly

`pinneapple_pdb.derived`

Derived variable computation (e.g. vorticity, divergence) for physical datasets.

`pinneapple_pdb/derived.py` · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DerivedSpec`	classe	`(derived, u_name, v_name)`	Specification for derived variables to add (e.g. vorticity, divergence).
`add_vorticity_divergence`	função	`(ds, u_name=None, v_name=None) -> xr.Dataset`	Add vorticity and divergence from U/V on lat/lon grid; returns new dataset.
`apply_derived`	função	`(ds, spec) -> xr.Dataset`	Apply derived variables from spec to dataset; returns new dataset.

pinneapple_pdb.shard

Sharding and subsetting of physical datasets by time and spatial tiles.

`pinneapple_pdb/shard.py` · 72 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ShardSpec`	classe	`(time_window, tile_deg, add_regime_tags)`	Specification for time-window and optional spatial tiling of datasets.
`iter_time_windows`	função	`(time_values, window) -> Iterator[Tuple[pd.Timestamp, pd.Timestamp]]`	Yield (start, end) time pairs for non-overlapping windows over time_values.
`subset_time`	função	`(ds, a, b) -> xr.Dataset`	Select time slice [a, b) from dataset.
`iter_tiles`	função	`(lat_vals, lon_vals, dlat, dlon)`	Yield (lat0, lat1, lon0, lon1) tile bounds aligned to dlat/dlon grid.
`subset_tile`	função	`(ds, lat0, lat1, lon0, lon1) -> xr.Dataset`	Select spatial tile [lat0,lat1] x [lon0,lon1]; assumes coords sorted.
`regime_tags_for`	função	`(ds) -> list[str]`	Derive regime tags (e.g. tropics, polar) from dataset lat extent.

pinneapple_pdb.templates

Physical schema templates for common datasets (e.g. atmosphere reanalysis).

`pinneapple_pdb/templates.py` · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`schema_templates`	função	`() -> Dict[str, Dict[str, Any]]`	Return mapping of template_id -> PhysicalSchema-compatible dict.

pinneapple_pdb.validate

Validation and standardization for physical datasets (UPD).

`pinneapple_pdb/validate.py` · 154 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ValidationSpec`	classe	`(require_units, ranges, enforce_lev_monotonic, enforce_time_monotonic)`	Specification for dataset validation (units, ranges, monotonicity).
`standardize_dims`	função	`(ds) -> xr.Dataset`	Standardize dimension names and transpose data vars to canonical order (time, lev, lat, lon).
`validate_dataset`	função	`(ds, spec) -> List[str]`	Validate dataset against spec; returns list of issue messages (empty if valid).

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 2 de 7.
- Funções de teste em arquivos que importam algum módulo do pacote: 100.

Limitações conhecidas

- *Nenhuma registrada.*

4. `pinneapple_adaptation`

Objetivo

`pinneapple_adaptation` reúne duas formas de reaproveitar um PINN já treinado em um problema físico parecido, sem treinar do zero. A primeira é transfer learning (`transfer_learning`, antigo `pinneapple_transfer`): pegar um modelo treinado, congelar parte das camadas e ajustar o resto no novo problema, por exemplo Burgers com outra viscosidade. A segunda é meta-learning (`meta_learning`, antigo `pinneapple_meta`): treinar uma inicialização compartilhada, com MAML ou Reptile, que se adapta a qualquer membro de uma família de EDPs em poucos passos de gradiente.

O pacote trabalha com qualquer `torch.nn.Module` e com funções de perda física no formato `(model, batch) -> perda`. Na prática, essas funções vêm do compilador de problemas de `pinneapple_physics` (`compile_problem` sobre um `ProblemSpec` de um preset, como `laplace_2d_default` ou `burgers_1d_default`), como mostra o teste `tests/test_adaptation_and_active_learning.py`. O catálogo de métodos (`pinneapple_catalog.methods`) lista o meta-learning como método de treino (item T18).

Na raiz há dois atalhos: `fine_tune`, que monta um `TransferConfig` e um `TransferTrainer`, chama `prepare()` e depois `finetune()`; e `adapt_model`, que escolhe entre meta-learning (quando o alvo é um dict com chave `support`) e transfer learning. Os nomes antigos `transfer` e `meta` continuam como apelidos dos submódulos, para compatibilidade.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
13	2.946	10	9	154 (2 arquivos)	2

Submódulos

sub-mó-dulo	o que faz	mó du-los	clas-ses + fun-ções	te st es *
`meta_learning`	Meta-learning (MAML e Reptile) sobre famílias paramétricas de EDPs: amostrador de tarefas, treinadores, configurações e um invólucro `MetaModel` para adaptar e prever.	6	8	15 1
`transfer_learning`	Transfer learning e fine-tuning de PINNs: congelamento de camadas, taxas de aprendizado por camada, descongelamento progressivo, interpolação de pesos entre variantes e adaptação de domínio por MMD.	6	9	15 1

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_adaptation`

Raiz do pacote: reexporta as classes de `transfer_learning` e `meta_learning`, mantém os apelidos antigos `transfer`/`meta` e define os atalhos `fine_tune` e `adapt_model`.

`pinneapple_adaptation/__init__.py` · 106 linhas · testes: 3

nom e	ti po	assinatura	o que faz
`fine_tune`	fu nç ão	`(source_model, target_physics_fn, target_data=None, *, strategy='finetune', epochs=500, l…)`	Atalho que cria `TransferConfig` + `TransferTrainer`, chama `prepare()` e `finetune()` e devolve o dict de resultado (modelo, histórico, métricas).
`adapt_model`	fu nç ão	`(model, target, *, mode='auto', **kwargs)`	Adapta um modelo a uma tarefa nova: usa `meta_adapt` se o alvo for um dict com `support` (ou `mode='meta'`), senão `fine_tune`.

`pinneapple_adaptation.meta_learning`

Reexporta configurações, amostrador e treinadores de meta-learning e oferece `meta_train` (constrói e treina) e `meta_adapt` (adapta a uma tarefa nova).

`pinneapple_adaptation/meta_learning/__init__.py` · 113 linhas · testes: 3

nom e	tip o	assinatura	o que faz
`meta_train`	fun ção	`(model, sampler, *, algorithm='reptile', **cfg_kwargs)`	Constrói um `ReptileTrainer` (padrão) ou `MAMLTrainer`, executa `train()` e devolve o treinador já treinado.
`meta_adapt`	fun ção	`(meta_trainer, task, n_steps=10)`	Chama `adapt` do treinador meta-treinado para uma tarefa nova e devolve uma cópia adaptada do modelo.

`pinneapple_adaptation.meta_learning.config`

Dataclasses `MAMLConfig` e `ReptileConfig` com os hiperparâmetros dos laços interno e externo, validados em `__post_init__`.

`pinneapple_adaptation/meta_learning/config.py` · 134 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MAMLConfig`	classe	`(n_inner_steps, inner_lr, outer_lr, n_tasks_per_batch, n_meta_epochs, first_order, device…)`	Hiperparâmetros do MAML: passos e taxa do laço interno, taxa externa, tarefas por lote, épocas, flag `first_order`, dispositivo, semente e checkpoint.
`ReptileConfig`	classe	`(n_inner_steps, inner_lr, outer_lr, n_tasks_per_batch, n_meta_epochs, device, seed, epsil…)`	Hiperparâmetros do Reptile, incluindo o passo de interpolação `epsilon` (validado no intervalo (0, 1]).

`pinneapple_adaptation.meta_learning.maml`

Treinador MAML para famílias de EDPs, em versão de primeira ordem (padrão) ou de segunda ordem; usa `torch.func.functional_call` quando existe e cai para cópia profunda + SGD em PyTorch antigo.

`pinneapple_adaptation/meta_learning/maml.py` · 480 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MAMLTrainer`	classe	`(model, config, task_sampler) -> None`	Treina um modelo com MAML sobre uma distribuição de tarefas de EDP, com `meta_update`, `train`, `adapt`, `save` e `load`.

`pinneapple_adaptation.meta_learning.meta_model`

Invólucro `MetaModel` de um modelo meta-treinado: adapta a uma função física nova, faz predição e salva/carrega o estado.

`pinneapple_adaptation/meta_learning/meta_model.py` · 191 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MetaModel`	classe	`(model, meta_type='reptile', device='cpu') -> None`	Invólucro de um modelo meta-treinado (rótulo `reptile` ou `maml`) para adaptar em poucos passos, prever e serializar.

`pinneapple_adaptation.meta_learning.reptile`

Treinador Reptile: treina em cada tarefa separadamente e interpola os pesos compartilhados na direção dos pesos de cada tarefa, sem gradientes de segunda ordem.

`pinneapple_adaptation/meta_learning/reptile.py` · 294 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ReptileTrainer`	classe	`(model, config, task_sampler) -> None`	Treinador Reptile com `meta_update`, `train`, `adapt` (devolve cópia adaptada), `save` e `load`.

`pinneapple_adaptation.meta_learning.task_sampler`

`PDETaskSampler`: sorteia parâmetros de EDP em intervalos dados e monta tarefas com conjuntos de suporte e de consulta para o meta-treino.

`pinneapple_adaptation/meta_learning/task_sampler.py` · 196 linhas · testes: 151

nome	tipo	assinatura	o que faz
PDETaskSampler	classe	(param_ranges, physics_fn_factory, data_factory=None, n_support=64, n_query=64, input_dim…)	Gera tarefas de meta-treino: parâmetros sorteados uniformemente, função física da fábrica e pontos de suporte/consulta (de data_factory ou uniformes em [0,1]^d).

pinneapple_adaptation.transfer_learning

Reexporta TransferConfig, TransferTrainer, utilitários de congelamento, ParametricFamilyTransfer e PhysicsTransferAdapter. [sem teste direto do __init__]

pinneapple_adaptation/transfer_learning/__init__.py · 49 linhas · **sem teste direto**

pinneapple_adaptation.transfer_learning.adapter

PhysicsTransferAdapter: combina a perda física do domínio alvo com uma regularização pela física de origem e uma perda MMD (kernel RBF) entre features dos dois domínios.

pinneapple_adaptation/transfer_learning/adapter.py · 346 linhas · testes: 151

nome	tipo	assinatura	o que faz
PhysicsTransferAdapter	classe	(model, source_spec, target_spec, source_physics_fn=None, source_weight=0.1, mmd_bandwidt…)	Adapta um PINN de um domínio físico de origem a um alvo relacionado, somando perda alvo, regularização da física de origem e perda MMD entre features.

pinneapple_adaptation.transfer_learning.config

Dataclass TransferConfig com estratégia de transferência (finetune, feature_extract, partial_freeze, progressive), prefixos de congelamento, taxas de aprendizado e pesos de perda.

pinneapple_adaptation/transfer_learning/config.py · 98 linhas · testes: 151

nome	tipo	assinatura	o que faz
TransferConfig	classe	(strategy, freeze_prefix, unfreeze_prefix, layer_lr_scale, base_lr, finetune_lr, epochs, …)	Configuração de um fine-tuning: estratégia, prefixos a congelar/descongelar, escala de taxa por camada, taxas base e de ajuste, épocas, pesos de perda física/dados e warmup.

pinneapple_adaptation.transfer_learning.freeze

Funções para congelar/descongelar parâmetros por prefixo de nome, montar grupos de otimizador com taxa de aprendizado por camada e contar parâmetros treináveis. [sem teste direto]

pinneapple_adaptation/transfer_learning/freeze.py · 202 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`freeze_layers`	função	`(model, prefixes) -> int`	Congela (`requires_grad=False`) os parâmetros cujo nome começa por algum prefixo e devolve quantos foram congelados.
`unfreeze_layers`	função	`(model, prefixes) -> int`	Descongela os parâmetros cujo nome começa por algum prefixo e devolve quantos foram alterados.
`freeze_all_except`	função	`(model, trainable_prefixes) -> None`	Congela todos os parâmetros que não casam com os prefixos treináveis.
`layer_lr_groups`	função	`(model, base_lr, scale_dict) -> List[dict]`	Monta grupos de parâmetros para o otimizador com taxa de aprendizado por camada (taxa base vezes a escala do prefixo).
`count_trainable`	função	`(model) -> Dict[str, int]`	Conta parâmetros treináveis, congelados e totais de um modelo.

`pinneapple_adaptation.transfer_learning.parametric`

`ParametricFamilyTransfer`: guarda variantes de um PINN ajustadas para valores diferentes de um parâmetro físico e interpola linearmente os pesos entre as duas mais próximas.

`pinneapple_adaptation/transfer_learning/parametric.py` · 272 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ParametricFamilyTransfer`	classe	`(base_model, param_name) -> None`	Coleção de PINNs ajustados por valor de um parâmetro escalar: devolve a variante mais próxima, interpola pesos entre vizinhas e salva/carrega a família inteira.

`pinneapple_adaptation.transfer_learning.trainer`

`TransferTrainer`: copia o modelo de origem, aplica a estratégia de congelamento (`prepare`) e roda o laço de fine-tuning, inclusive com cronograma de descongelamento progressivo.

`pinneapple_adaptation/transfer_learning/trainer.py` · 465 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TransferTrainer`	classe	`(source_model, config) -> None`	Treinador de transferência: copia o modelo de origem, aplica a estratégia (`prepare`), ajusta no problema alvo (`finetune`) ou com descongelamento por época (`progressive_unfreeze`).

Exemplos de uso reais do repositório

Fine-tuning de um PINN num preset de Laplace — `tests/test_adaptation_and_active_learning.py`

```
            from pinneapple_adaptation import fine_tune

            spec = laplace_2d_default()
            loss_fn = compile_problem(spec)
            bounds = spec.domain_bounds
            coords = list(spec.coords)
            torch.manual_seed(0)
            model = torch.nn.Sequential(torch.nn.Linear(2, 32),
torch.nn.Tanh(), torch.nn.Linear(32, 1))

            def target_physics_fn(m, batch):
```

```
                x_col = torch.as_tensor(
                    np.stack([np.random.uniform(*bounds[c], size=256) for c
in coords], axis=1).astype(np.float32)
                )
                x_col.requires_grad_(True)
                y = m(x_col)
                return loss_fn(m, y, _empty_batch(x_col, len(coords), 1))
```

Meta-treino Reptile sobre Burgers com viscosidade variável — `tests/test_adaptation_and_active_learning.py`

```
            sampler = PDETaskSampler(param_ranges={"nu": (0.001, 0.05)},
                                     physics_fn_factory=physics_fn_factory,
input_dim=2, seed=0)
            torch.manual_seed(0)
            model = torch.nn.Sequential(torch.nn.Linear(2, 32),
torch.nn.Tanh(), torch.nn.Linear(32, 1))

            trainer = meta_train(model, sampler, algorithm="reptile",
                                 n_meta_epochs=5, n_tasks_per_batch=2,
n_inner_steps=3, seed=0)
            assert hasattr(trainer, "adapt"), "meta_train() should return a
trainer with .adapt()"
            assert len(trainer._history) == 5, "meta_train() must have
actually run .train(), not just constructed the trainer"
            assert all(np.isfinite(h["meta_loss"]) for h in
trainer._history)

            task = sampler.sample_batch(1)[0]
            adapted = trainer.adapt(task, n_steps=5)
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 11 de 13.
- Funções de teste em arquivos que importam algum módulo do pacote: 154.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
T18	Meta-learning: MAML, Reptile	tested	0	6
T19	Transfer learning (freeze, adapters, parametric)	tested	0	3

Limitações conhecidas

- O docstring da raiz cita `fast_adapt` no exemplo de uso, mas essa função não existe; o nome real é `meta_adapt`.
- `MAMLTrainer` e `ReptileTrainer` não usam o mesmo contrato de função física: MAML chama `physics_fn(forward_ou_modelo, x)` e Reptile chama `physics_fn(model, batch) -> (perda, dict)`. Uma mesma fábrica só serve aos dois se tratar as duas formas.
- No PyTorch sem `torch.func`, o MAML usa um caminho alternativo (cópia profunda + SGD) que só faz a aproximação de primeira ordem.
- Em `PhysicsTransferAdapter`, `source_spec` e `target_spec` só ficam guardados como referência; não entram no cálculo, a menos que se faça subclasse.

- `ParametricFamilyTransfer.interpolate_weights` não extrapola: fora do intervalo registrado, ou com menos de duas variantes, devolve a variante mais próxima.
- `fine_tune` e `meta_train` tinham bugs (não chamavam `prepare()` / `train()`); foram corrigidos e agora têm teste de regressão, mas o `__init__` de `transfer_learning` e o módulo `freeze` não têm teste direto.

5. `pinneapple_arena`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
54	21.853	69	243	99 (12 arquivos)	39

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`arena`		1	3	8
`autoresearch`		1	8	5
`config`		1	10	13
`custom_builder`		1	2	0
`dataset_bench`		1	3	0
`dataset_problems`		1	1	0
`decision_mode`		1	8	16
`external`		28	186	5
`model_factory`		1	5	10
`nas`		1	4	5
`pointwise`		11	35	16
`problems`		1	27	15
`regimes`		1	3	19
`showcase`		1	2	5
`trainer`		1	10	25
`visualizer`		1	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_arena`

pinneapple_arena — YAML/JSON-driven multi-model physics benchmark runner.

`pinneapple_arena/__init__.py` · 157 linhas · testes: 52

`pinneapple_arena.arena`

Arena — main orchestrator for multi-model physics benchmarks.

pinneapple_arena/arena.py · 628 linhas · testes: 8

nome	tipo	assinatura	o que faz
Arena	classe	(config, device=None)	Multi-model physics benchmark runner.
rank_by_accuracy	função	(train_results, eval_results, field_names) -> List[Tuple[str, float]]	Pure-accuracy ranking: (name, mean_rel_error) sorted ascending (best first). This is Arena's default, unchanged ranking behaviour.
physics_aware_rank	função	(train_results, eval_results, field_names, n_std=2.0, residual_threshold=None) -> Dict[st…)	Additive, opt-in comparison view layered on top of the default pure-accuracy ranking.

pinneapple_arena.autoresearch

Autoresearch mode: an automated experiment loop over a single training script.

pinneapple_arena/autoresearch.py · 284 linhas · testes: 5

nome	tipo	assinatura	o que faz
Trial	classe	(index, metric, status, kept, description, seconds, changes)	
RandomSearchProposer	classe	(space, seed=0, max_changes=2)	Samples one or two tunables at a time from a discrete space (no LLM, reproducible).
LLMProposer	classe	(provider='anthropic', model=None, llm_call=None, history_len=15, **llm_kwargs)	Asks an LLM for the next edit of the TUNABLE block, given program.md and the trial log.
AutoResearch	classe	(workdir, proposer, budget_s=300.0, python=sys.executable)	
read_tunables	função	(code) -> Dict[str, Any]	
apply_tunables	função	(code, changes) -> str	Rewrite values inside the TUNABLE block only; unknown names are rejected.
run_trial	função	(workdir, budget_s, *, python=sys.executable, grace_s=30.0)	Run train.py once. Returns (metric, status, stdout_tail, seconds).
init_workdir	função	(path, *, overwrite=False) -> str	Create program.md + train.py (1D Poisson PINN template) in path.

pinneapple_arena.config

ArenaConfig — structured configuration for PINNeAPPle Arena benchmarks.

pinneapple_arena/config.py · 329 linhas · testes: 13

nome	tipo	assinatura	o que faz
NetworkConfig	classe	(hidden, activation, width, modes, modes2, layers, use_grid, hidden_dim, n_layers, n_mess…)	Neural network architecture hyperparameters.
TrainingConfig	classe	(epochs, lr, weight_decay, grad_clip, optimizer, scheduler, batch_size, seed)	Training hyperparameters for a single model run.

nome	tipo	assinatura	o que faz
`ModelConfig`	classe	`(name, type, network, training)`	Full specification for one model in the benchmark.
`ProblemConfig`	classe	`(name, params, physics_preset, grid_n, n_col, n_bc, n_train_supervised, n_mesh_nodes)`	Physics problem selection and parameters.
`OutputConfig`	classe	`(dir, prefix, save_figures, dpi, dark_theme, show)`	Output / visualisation settings.
`InverseConfig`	classe	`(enabled, params, n_obs, noise_std, method, n_iters, lambda_reg, sensor_locations)`	Inverse problem configuration (identifying unknown physics parameters).
`UQConfig`	classe	`(enabled, method, n_samples, dropout_rate, coverage)`	Uncertainty quantification configuration.
`DatasetConfig`	classe	`(dataset_id, input_fields, output_fields, n_train, n_val, split_seed)`	Use a pinneapple_data dataset instead of a built-in Arena problem.
`DecisionConfig`	classe	`(enabled, budget, stop_when_passed, thresholds, evidence_path, problem)`	Decision mode: the decision engine chooses which configured models to train.
`ArenaConfig`	classe	`(problem, models, output, inverse, uq, dataset, decision)`	Top-level Arena configuration.

`pinneapple_arena.custom_builder`

High-level API for defining custom physics problems.

`pinneapple_arena/custom_builder.py` · 661 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EasyArenaProblem`	classe	`(name, description, coords, fields, pde_evaluator, bcs, params, analytical_fn, bc_weight)`	Auto-generated ArenaProblem from :func:`define_problem`.
`define_problem`	função	`(name, coords, fields, pde, bcs, params=None, description='', analytical=None, bc_weight=…)`	Define a custom physics problem with a declarative, high-level API.

`pinneapple_arena.dataset_bench`

dataset_bench.py -- High-level benchmark API using pinneapple_data datasets.

`pinneapple_arena/dataset_bench.py` · 723 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`list_benchmarks`	função	`(category=None, verbose=True) -> List[str]`	Print and return available benchmark preset IDs.
`get_benchmark_preset`	função	`(dataset_id) -> Dict[str, Any]`	Return the preset configuration dict for a dataset_id.
`benchmark_dataset`	função	`(dataset_id, *, models=None, epochs=None, mode=None, n_train=None, n_bc=None, grid_n=None…)`	Run a full Arena benchmark on a pinneapple_data dataset.

`pinneapple_arena.dataset_problems`

DatasetProblem — ArenaProblem backed by a real pinneapple_data dataset.

pinneapple_arena/dataset_problems.py · 520 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DatasetProblem`	classe	`(dataset_id, name, description, input_fields, output_fields, mode, dataset_params, pinn_r…)`	ArenaProblem backed by a pinneapple_data dataset.

pinneapple_arena.decision_mode

Arena as a decision laboratory: the decision engine picks which models to train.

pinneapple_arena/decision_mode.py · 320 linhas · testes: 16

nome	tipo	assinatura	o que faz
`ArenaExecutor`	classe	`(arena)`	Executor: train + evaluate the chosen configured model with the Arena's own code.
`ArenaDecisionReport`	classe	`(budget, stop_reason, ran, not_trained, accepted, ranking, evidence, facts)`	
`catalog_family`	função	`(model_type) -> Optional[str]`	Map an Arena model type to an `ARCHITECTURE_CATALOG` key, or `None`.
`arena_choice`	função	`(arena) -> Tuple[PhysicsChoice, Dict[str, Optional[str]]]`	
`arena_model_rules`	função	`(families)`	Rules for the `arena_model` choice: the `ModelSelector` score of each option's catalog family, minus a penalty for families that already failed.
`default_decider`	função	`(families) -> PhysicsDecider`	
`arena_problem_facts`	função	`(arena, overrides=None) -> Dict[str, Any]`	Facts the Arena actually knows about its problem. Nothing else is filled in: the adapter lists the rest as unknown. `overrides` (e.g. a representation or Reynolds number) are given facts and win.
`run_decision_mode`	função	`(arena, *, budget=3, decider=None, verifier=None, store=None, problem=None, stop_when_pas…)`	Run the decision loop over `arena`'s configured models (see module docstring).

pinneapple_arena.external

External model catalog: references to third-party physics-ML libraries and the glue that runs them.

pinneapple_arena/external/__init__.py · 68 linhas · testes: 5

nome	tipo	assinatura	o que faz
`licence_counts`	função	`() -> dict[str, int]`	Number of catalog entries per licence (`"none found"` when the repository has no licence file).

pinneapple_arena.external.__main__

Command line for the external model catalog (`python -m pinneapple_arena.external`).

`pinneapple_arena/external/__main__.py` · 134 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`cmd_list`	função	`(args) -> None`	
`print_result`	função	`(r) -> None`	
`cmd_run`	função	`(args) -> None`	
`main`	função	`() -> None`	

pinneapple_arena.external.arch_meta

Architecture-family labels (the `arch` field on ModelSpec).

`pinneapple_arena/external/arch_meta.py` · 16 linhas · **sem teste direto**

pinneapple_arena.external.benchmark

Benchmark / comparison harness — the successor to the standalone pipeline.py smoke-test script, now callable from the API so the frontend's Benchmark and Compare views can run it on demand against any subset of the catalog.

`pinneapple_arena/external/benchmark.py` · 53 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`select_slugs`	função	`(slugs=None, category=None, domain=None) -> list[str]`	
`run_benchmark`	função	`(slugs, include_heavy=False) -> dict`	
`run_compare`	função	`(slugs, include_heavy=False) -> dict`	

pinneapple_arena.external.dataset_probe

"Probe" action for the dataset catalog: load one real sample of one subset and report its fields/shapes — the dataset equivalent of a model's /infer.

`pinneapple_arena/external/dataset_probe.py` · 123 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`local_commands`	função	`(spec, subset, split) -> str`	
`probe_dataset`	função	`(spec, subset, split, stream=False, dataset_root=None) -> ModelResult`	Load sample 0 of `subset/split`. `dataset_root`: a local download (default: the `EXTERNAL_CATALOG_DATASET_ROOT` environment variable); otherwise `stream=True` streams one sample.

pinneapple_arena.external.datasets_data

Single source of truth for the external dataset catalog — the dataset counterpart of models_data.py. See DatasetSpec in schema.py for why datasets live in their own catalog instead of being forced into ModelSpec.

`pinneapple_arena/external/datasets_data.py` · 87 linhas · **sem teste direto**

`pinneapple_arena.external.demo_datasets`

Standardized synthetic dataset generators — one per physical-problem shape, reused across many models' demo runs (`pinneapple_arena/external/models_data.py` points each ModelSpec at one of these via `dataset_fn`).

`pinneapple_arena/external/demo_datasets.py` · 230 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ds_pde_1d`	função	`(n=64) -> dict`	
`ds_burgers`	função	`(nx=64, nt=32) -> dict`	
`ds_grid_2d`	função	`(nx=16, ny=16) -> dict`	
`ds_molecular`	função	`(n_atoms=6) -> dict`	
`ds_weather_grid`	função	`(nlat=16, nlon=32, nlevels=4) -> dict`	
`ds_em_grid`	função	`(nx=20, ny=20) -> dict`	
`ds_seismic`	função	`(nx=32, nz=32) -> dict`	
`ds_spin_lattice`	função	`(n_sites=6) -> dict`	
`ds_power_system`	função	`() -> dict`	
`ds_traffic`	função	`(nx=50, nt=50) -> dict`	
`ds_epidemic`	função	`(n_days=60) -> dict`	
`ds_combustion`	função	`(n_species=5, n_t=50) -> dict`	
`ds_cosmology`	função	`(n_particles=200) -> dict`	
`ds_room`	função	`() -> dict`	
`ds_graph_mesh`	função	`(n_nodes=30) -> dict`	
`ds_rigidbody`	função	`(n=8) -> dict`	
`ds_image_field`	função	`(size=32) -> dict`	

nome	tipo	assinatura	o que faz
ds_timeseries	função	(n=100) -> dict	
ds_pedestrians	função	(n=20) -> dict	
ds_airfoil	função	() -> dict	
ds_option_pricing	função	(n=64) -> dict	
ds_lorenz	função	(n_steps=2000, dt=0.01) -> dict	Classic chaotic Lorenz-63 trajectory — used both to *discover* (PySINDy) and to *integrate forward* (Diffrax) the same governing ODE.
ds_tle	função	() -> dict	A real, valid two-line element set (ISS, illustrative epoch) for orbit-propagation demos — TLE format needs a real-looking checksum/epoch to parse, so this is copied from a genuine public ISS TLE rather than invented.
ds_quantum_circuit	função	(n_qubits=2) -> dict	
ds_crystal	função	() -> dict	A tiny synthetic cubic unit cell (plain arrays only — no pymatgen/ASE dependency here; the consuming infer_* function builds a real Structure object from these if the library needs one).

pinneapple_arena.external.domain_meta

Physical-domain metadata: label, glyph and description per domain key.

`pinneapple_arena/external/domain_meta.py` · 56 linhas · **sem teste direto**

pinneapple_arena.external.finetune

Fine-tuning / retraining runners — one per pip-installable model in the catalog (light or heavy; manual-install-only repos have nothing to run in-app regardless, same as for inference).

`pinneapple_arena/external/finetune.py` · 1.013 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
finetune_deepxde	função	(params) -> ModelResult	
finetune_neuraloperator	função	(params) -> ModelResult	
finetune_pina	função	(params) -> ModelResult	
finetune_sciann	função	(params) -> ModelResult	
finetune_jax_md	função	(params) -> ModelResult	
finetune_ceviche	função	(params) -> ModelResult	
finetune_netket	função	(params) -> ModelResult	
finetune_pinnicle	função	(params) -> ModelResult	
finetune_pybamm	função	(params) -> ModelResult	
finetune_socialforce	função	(params) -> ModelResult	
finetune_phiflow	função	(params) -> ModelResult	
finetune_jax_cfd	função	(params) -> ModelResult	
finetune_taichi	função	(params) -> ModelResult	
finetune_warp	função	(params) -> ModelResult	
finetune_devito	função	(params) -> ModelResult	

nome	tipo	assinatura	o que faz
`finetune_deepwave`	função	`(params) -> ModelResult`	
`finetune_pyroomacoustics`	função	`(params) -> ModelResult`	
`finetune_ml4gw`	função	`(params) -> ModelResult`	
`finetune_reservoirflow`	função	`(params) -> ModelResult`	
`finetune_photontorch`	função	`(params) -> ModelResult`	
`finetune_neuralfoil`	função	`(params) -> ModelResult`	
`finetune_mujoco_mjx`	função	`(params) -> ModelResult`	
`finetune_brax`	função	`(params) -> ModelResult`	
`finetune_torax`	função	`(params) -> ModelResult`	
`finetune_physicsnemo`	função	`(params) -> ModelResult`	
`finetune_mace`	função	`(params) -> ModelResult`	
`finetune_nequip`	função	`(params) -> ModelResult`	
`finetune_deepmd_kit`	função	`(params) -> ModelResult`	
`finetune_schnetpack`	função	`(params) -> ModelResult`	
`finetune_neuralgcm`	função	`(params) -> ModelResult`	
`finetune_pysindy`	função	`(params) -> ModelResult`	
`finetune_diffrax`	função	`(params) -> ModelResult`	
`finetune_tensorcircuit_ng`	função	`(params) -> ModelResult`	
`finetune_dsgp4`	função	`(params) -> ModelResult`	
`finetune_chgnet`	função	`(params) -> ModelResult`	
`run_finetune`	função	`(slug, params=None) -> ModelResult`	

`pinneapple_arena.external.installer`

Install-state helpers shared by every runner and by the API layer.

`pinneapple_arena/external/installer.py` · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`is_installed`	função	`(module_name) -> bool`	True when `module_name` can be found, without importing it (an import of a heavy library has side effects, e.g. torch's default device). A library that is found but fails to import is reported by its runner as `failed` with the real exception.
`pip_install`	função	`(package, timeout=900) -> tuple[bool, str]`	Best-effort `pip install`. Used only when a user explicitly opts in (the API's `install=true` flag) — never triggered implicitly.

`pinneapple_arena.external.local_command`

Builds the exact command a user can run on their own machine to get a real result for a model the hosted app can't run itself — because it's a heavy GPU dependency, or a git-only / non-pip research repo, or fine-tuning simply isn't wired up for it yet in-app.

`pinneapple_arena/external/local_command.py` · 37 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`local_command_for`	função	`(spec, action='infer', **params) -> str`	`action` is "infer" or "finetune": the command line of `python -m pinneapple_arena.external` (PINNeAPPle installed), or the clone + build steps of a manual-install repository.
`pip_install_command`	função	`(spec) -> str | None`	

pinneapple_arena.external.models_data

Single source of truth for the model catalog.

`pinneapple_arena/external/models_data.py` · 1.302 linhas · **sem teste direto**

pinneapple_arena.external.result

The result contract every model runner (inference *and* fine-tune) returns. Ported from the original suite_core.py with one addition: `ModelResult` now also carries an optional `metrics["loss_curve"]` for fine-tune runs and a `job_kind` field ("infer" | "finetune" | "benchmark") set by the caller.

`pinneapple_arena/external/result.py` · 141 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelResult`	classe	`(name, slug, category, domains, repo, status, dataset_name, dataset_description, input_sh…)`	
`Timer`	classe	`()`	`with Timer() as t: ...` then read `t.elapsed` at any point, including from an `except` clause before the block exits.
`summarize_array`	função	`(arr) -> dict`	
`not_installed`	função	`(name, slug, category, domains, repo, pip_hint, dataset=None, local_command=None) -> Mode…)`	
`manual_install`	função	`(name, slug, category, domains, repo, instructions, dataset=None, local_command=None) -> …)`	
`skipped_heavy`	função	`(name, slug, category, domains, repo, pip_hint, dataset=None, local_command=None) -> Mode…)`	
`failed`	função	`(name, slug, category, domains, repo, exc, duration, dataset=None) -> ModelResult`	
`ok`	função	`(name, slug, category, domains, repo, dataset, duration, output_shape=None, output_summar…)`	

pinneapple_arena.external.runners

Módulo sem docstring e sem texto.

`pinneapple_arena/external/runners/__init__.py` · 1 linhas · **sem teste direto**

pinneapple_arena.external.runners.applied_domains

Auto-migrated from the original inference_suite.py (single-file registry). Each infer_* function builds a minimal real instance of the actual library and runs one real forward pass / inference call — see pinneapple_arena/external/result.py for the ModelResult contract every function returns.

`pinneapple_arena/external/runners/applied_domains.py` · 236 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_simvascular`	função	`(ds) -> ModelResult`	
`infer_pyroomacoustics`	função	`(ds) -> ModelResult`	
`infer_rirpinn`	função	`(ds) -> ModelResult`	
`infer_epipinn`	função	`(ds) -> ModelResult`	
`infer_pise`	função	`(ds) -> ModelResult`	
`infer_pinn_ac_opf`	função	`(ds) -> ModelResult`	
`infer_piml_converter`	função	`(ds) -> ModelResult`	
`infer_ml4gw`	função	`(ds) -> ModelResult`	
`infer_physics_informed_am`	função	`(ds) -> ModelResult`	
`infer_pinn_swe_open`	função	`(ds) -> ModelResult`	
`infer_pinn_corrosion_fatigue`	função	`(ds) -> ModelResult`	
`infer_pinn_wind_bearing`	função	`(ds) -> ModelResult`	
`infer_pinnicle`	função	`(ds) -> ModelResult`	
`infer_physnet_thermal`	função	`(ds) -> ModelResult`	
`infer_neuralsolvers`	função	`(ds) -> ModelResult`	
`infer_pinn_multivp`	função	`(ds) -> ModelResult`	
`infer_socialforce`	função	`(ds) -> ModelResult`	
`infer_spdiff`	função	`(ds) -> ModelResult`	
`infer_stiff_pinn`	função	`(ds) -> ModelResult`	
`infer_crnn`	função	`(ds) -> ModelResult`	
`infer_neuralfoil`	função	`(ds) -> ModelResult`	
`infer_pinn_option_pricing`	função	`(ds) -> ModelResult`	

`pinneapple_arena.external.runners.dynamics_robotics_astro`

Nonlinear dynamics / system identification, robotics-adjacent differentiable engines, and astro (astrophysical fluids / orbital mechanics / ocean) models added from the second catalog-expansion research pass.

`pinneapple_arena/external/runners/dynamics_robotics_astro.py` · 210 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_pysindy`	função	`(ds) -> ModelResult`	
`infer_diffrax`	função	`(ds) -> ModelResult`	
`infer_genesis`	função	`(ds, include_heavy=False) -> ModelResult`	
`infer_nimble_physics`	função	`(ds) -> ModelResult`	
`infer_dojo_jl`	função	`(ds) -> ModelResult`	
`infer_astronomix`	função	`(ds) -> ModelResult`	
`infer_dsgp4`	função	`(ds) -> ModelResult`	
`infer_veros`	função	`(ds) -> ModelResult`	
`infer_diffpd`	função	`(ds) -> ModelResult`	

nome	tipo	assinatura	o que faz
`infer_difftaichi`	função	`(ds) -> ModelResult`	
`infer_diffsph`	função	`(ds) -> ModelResult`	

`pinneapple_arena.external.runners.em_photonics`

Auto-migrated from the original inference_suite.py (single-file registry). Each infer_* function builds a minimal real instance of the actual library and runs one real forward pass / inference call — see pinneapple_arena/external/result.py for the ModelResult contract every function returns.

`pinneapple_arena/external/runners/em_photonics.py` · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_ceviche`	função	`(ds) -> ModelResult`	
`infer_photontorch`	função	`(ds) -> ModelResult`	
`infer_hpinn`	função	`(ds) -> ModelResult`	

`pinneapple_arena.external.runners.fluids_structural`

Auto-migrated from the original inference_suite.py (single-file registry). Each infer_* function builds a minimal real instance of the actual library and runs one real forward pass / inference call — see pinneapple_arena/external/result.py for the ModelResult contract every function returns.

`pinneapple_arena/external/runners/fluids_structural.py` · 163 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_phiflow`	função	`(ds) -> ModelResult`	
`infer_jax_cfd`	função	`(ds) -> ModelResult`	
`infer_jax_fluids`	função	`(ds) -> ModelResult`	
`infer_deepflame`	função	`(ds) -> ModelResult`	
`infer_fluidx3d`	função	`(ds) -> ModelResult`	
`infer_lesnets`	função	`(ds) -> ModelResult`	
`infer_twophasepinn`	função	`(ds) -> ModelResult`	
`infer_meshgraphnets_pytorch`	função	`(ds) -> ModelResult`	
`infer_meshgraphnets_original`	função	`(ds) -> ModelResult`	
`infer_gns`	função	`(ds) -> ModelResult`	
`infer_graingnn`	função	`(ds) -> ModelResult`	
`infer_taichi`	função	`(ds) -> ModelResult`	
`infer_warp`	função	`(ds) -> ModelResult`	
`infer_dl4to`	função	`(ds) -> ModelResult`	

`pinneapple_arena.external.runners.geophysics_energy`

Auto-migrated from the original inference_suite.py (single-file registry). Each infer_* function builds a minimal real instance of the actual library and runs one real forward pass / inference call — see pinneapple_arena/external/result.py for the ModelResult contract every function returns.

`pinneapple_arena/external/runners/geophysics_energy.py` · 111 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
infer_devito	função	(ds) -> ModelResult	
infer_deepwave	função	(ds) -> ModelResult	
infer_torax	função	(ds, include_heavy=False) -> ModelResult	
infer_pybamm	função	(ds) -> ModelResult	
infer_reservoirflow	função	(ds) -> ModelResult	

pinneapple_arena.external.runners.materials2_and_misc

Second wave of materials-ML potentials plus a grab-bag of other domains (biomolecular structure, quantum circuits, newer neural-operator paper code, wind-wake surrogates, wildfire PINNs) added from the second catalog-expansion research pass. Same runner contract as every other module — see ...

pinneapple_arena/external/runners/materials2_and_misc.py · 202 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
infer_openfold	função	(ds) -> ModelResult	
infer_tensorcircuit_ng	função	(ds) -> ModelResult	
infer_transolver	função	(ds) -> ModelResult	
infer_u_no	função	(ds) -> ModelResult	
infer_torchmd	função	(ds) -> ModelResult	
infer_fairchem	função	(ds, include_heavy=False) -> ModelResult	
infer_mattersim	função	(ds, include_heavy=False) -> ModelResult	
infer_chgnet	função	(ds) -> ModelResult	
infer_orb_models	função	(ds) -> ModelResult	
infer_grainlearning	função	(ds) -> ModelResult	
infer_wakenet	função	(ds) -> ModelResult	
infer_bpinn_wildfire	função	(ds) -> ModelResult	

pinneapple_arena.external.runners.materials_md

Auto-migrated from the original inference_suite.py (single-file registry). Each infer_* function builds a minimal real instance of the actual library and runs one real forward pass / inference call — see pinneapple_arena/external/result.py for the ModelResult contract every function returns.

pinneapple_arena/external/runners/materials_md.py · 121 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
infer_jax_md	função	(ds) -> ModelResult	
infer_mace	função	(ds, include_heavy=False) -> ModelResult	
infer_nequip	função	(ds, include_heavy=False) -> ModelResult	

nome	tipo	assinatura	o que faz
infer_allegro	função	(ds) -> ModelResult	
infer_deepmd_kit	função	(ds, include_heavy=False) -> ModelResult	
infer_schnetpack	função	(ds, include_heavy=False) -> ModelResult	

pinneapple_arena.external.runners.pinn_general

Auto-migrated from the original inference_suite.py (single-file registry). Each infer_* function builds a minimal real instance of the actual library and runs one real forward pass / inference call — see pinneapple_arena/external/result.py for the ModelResult contract every function returns.

pinneapple_arena/external/runners/pinn_general.py · 167 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
infer_deepxde	função	(ds) -> ModelResult	
infer_pina	função	(ds) -> ModelResult	
infer_physicsnemo	função	(ds, include_heavy=False) -> ModelResult	
infer_sciann	função	(ds) -> ModelResult	
infer_neuralpde_jl	função	(ds) -> ModelResult	
infer_pinns_torch	função	(ds) -> ModelResult	
infer_pinns_original	função	(ds) -> ModelResult	
infer_pinn_tfi_hss	função	(ds) -> ModelResult	
infer_neuraloperator	função	(ds) -> ModelResult	
infer_deeponet_original	função	(ds) -> ModelResult	
infer_deeponet_fno_benchmark	função	(ds) -> ModelResult	
infer_pino	função	(ds) -> ModelResult	

pinneapple_arena.external.runners.quantum_cosmology

Auto-migrated from the original inference_suite.py (single-file registry). Each infer_* function builds a minimal real instance of the actual library and runs one real forward pass / inference call — see pinneapple_arena/external/result.py for the ModelResult contract every function returns.

pinneapple_arena/external/runners/quantum_cosmology.py · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
infer_netket	função	(ds) -> ModelResult	
infer_ferminet	função	(ds) -> ModelResult	
infer_flowpm_jaxpm	função	(ds) -> ModelResult	
infer_camels	função	(ds) -> ModelResult	

`pinneapple_arena.external.runners.registry`

Runner registry: maps every model slug to its (dataset_fn, infer_fn) pair.

`pinneapple_arena/external/runners/registry.py` · 268 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`call_infer`	função	`(slug, include_heavy=False) -> ModelResult`	Build the demo dataset and run the real infer_* function for `slug`. Any non-"ok" result gets a `local_command` telling the user exactly how to get a real result on their own machine instead.
`get_dataset_preview`	função	`(slug) -> dict`	Return the demo dataset's metadata + a JSON-safe preview of its arrays, without running inference — used by the "sample input" panel in the UI.

`pinneapple_arena.external.runners.rigidbody_robotics`

Auto-migrated from the original inference_suite.py (single-file registry). Each infer_* function builds a minimal real instance of the actual library and runs one real forward pass / inference call — see pinneapple_arena/external/result.py for the ModelResult contract every function returns.

`pinneapple_arena/external/runners/rigidbody_robotics.py` · 65 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_brax`	função	`(ds, include_heavy=False) -> ModelResult`	
`infer_mujoco_mjx`	função	`(ds, include_heavy=False) -> ModelResult`	

`pinneapple_arena.external.runners.weather_climate`

Auto-migrated from the original inference_suite.py (single-file registry). Each infer_* function builds a minimal real instance of the actual library and runs one real forward pass / inference call — see pinneapple_arena/external/result.py for the ModelResult contract every function returns.

`pinneapple_arena/external/runners/weather_climate.py` · 62 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`infer_graphcast`	função	`(ds) -> ModelResult`	
`infer_pangu_weather`	função	`(ds) -> ModelResult`	
`infer_fourcastnet`	função	`(ds) -> ModelResult`	
`infer_neuralgcm`	função	`(ds, include_heavy=False) -> ModelResult`	

`pinneapple_arena.external.schema`

Data model for one catalog entry (`ModelSpec`) plus the physical-domain and architecture-family metadata used to group/label entries in both the API and the frontend.

`pinneapple_arena/external/schema.py` · 111 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelSpec`	classe	`(slug, name, org, arch, domains, desc, tags, repo, ref, pip_package, import_name, heavy, …)`	

nome	tipo	assinatura	o que faz
DomainMeta	classe	(key, label, glyph, desc)	
ArchMeta	classe	(key, label)	
SubDataset	classe	(name, label, domains, dims, desc)	
DatasetSpec	classe	(slug, name, org, desc, tags, repo, ref, size, subsets, splits, pip_package, import_name,…)	

pinneapple_arena.external.status

Cheap, no-inference status check for a model — used by the catalog list/ filter view so the UI can show an install-state badge on every card without actually running all 71 models on every page load.

pinneapple_arena/external/status.py · 29 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
install_state	função	(spec) -> str	

pinneapple_arena.model_factory

Build any Arena-supported model from a ModelConfig.

pinneapple_arena/model_factory.py · 324 linhas · testes: 10

nome	tipo	assinatura	o que faz
is_graph_model	função	(cfg) -> bool	
is_pinn_model	função	(cfg) -> bool	
is_inverse_model	função	(cfg) -> bool	
is_operator_model	função	(cfg) -> bool	
build_model	função	(cfg, in_dim=2, out_dim=1, edge_in_dim=0) -> nn.Module	Instantiate any pinneapple model from a ModelConfig.

pinneapple_arena.nas

Architecture + hyperparameter search over the existing model catalog.

pinneapple_arena/nas.py · 447 linhas · testes: 5

nome	tipo	assinatura	o que faz
ArchitectureCandidate	classe	(name, param_space, build_kwargs_fn)	One searchable architecture: its ModelRegistry key, its own hyperparameter search space (in the same ParamSpec mini-language AdaptiveSweepConfig.param_space uses — ("float"|"int", (lo, hi)), optionally (..., "log"), or ("categorical", (choices,))), and a function that turns …
ArchitectureSearchSpace	classe	(candidates, lr_param_space)	Which architectures are candidates, plus each one's hyperparameter search space and a shared learning-rate range applied to every candidate (learning rate isn't architecture-specific, so it lives outside candidates rather than being duplicated in each one).

nome	tipo	assinatura	o que faz
`NASResult`	classe	`(best_architecture, best_hyperparams, best_score, all_trials, study)`	
`search_architecture`	função	`(problem, search_space=None, n_trials=20, physics_aware=False, lam=1.0, epochs_per_trial=…)`	Search over `search_space` (architecture name + each architecture's hyperparameters, jointly) for the model that best solves `problem`.

pinneapple_arena.pointwise

Pointwise field benchmarks: the problems, reference data, losses, model adapters and training protocol of the PINNeAPPle Arena app (`apps/pinn_arena`).

`pinneapple_arena/pointwise/__init__.py` · 71 linhas · testes: 16

pinneapple_arena.pointwise.adapters

Input adapters between the arena's pointwise data and PINNeAPPle architectures.

`pinneapple_arena/pointwise/adapters.py` · 181 linhas · testes: 16

nome	tipo	assinatura	o que faz
`ArchitectureNotApplicable`	classe	`()`	The architecture cannot be trained on pointwise coordinate→field data.
`adapt_model`	função	`(arch_id, model, in_dim, out_dim) -> Any`	Return `model` wrapped so it accepts (N, in_dim) and returns (N, out_dim).
`check_applicable`	função	`(arch_id) -> None`	

pinneapple_arena.pointwise.architectures

Architecture catalog of the PINNeAPPle Arena: `ARCHITECTURES_CATALOG: dict[str, ArchitectureInfo]`, descriptions of the PINNeAPPle registry models the arena can select (`pp.list_models()` / `pp.build_model()`).

`pinneapple_arena/pointwise/architectures.py` · 1.722 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`registry_key`	função	`(arch_id) -> str`	The `pp.build_model` key of a catalog entry.
`catalog_status`	função	`() -> dict[str, bool]`	`{catalog id: registered}`: whether the PINNeAPPle registry has the entry's model at all.

pinneapple_arena.pointwise.datasets

Reference datasets of the arena problems: analytic solutions where one exists, otherwise a numerical solution (finite differences, spectral, panel method, ...) written for that problem.

`DatasetGenerator(problem_id, params).generate(n)` returns ``{"coords": (N, in_dim), "solution": (N, out_dim), ...

`pinneapple_arena/pointwise/datasets.py` · 3.180 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DatasetGenerator`	classe	`(problem_id, parameters)`	Generates reference (ground truth) solutions for a given physics problem. Uses PINNeAPPle numerical solvers when available; falls back to analytical.

`pinneapple_arena.pointwise.geometries`

Pre-built 3D geometry catalog for the PINNeAPPle Arena.

`pinneapple_arena/pointwise/geometries.py` · 503 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`get_geometry_mesh_config`	função	`(geometry_id, resolution) -> dict`	Return a mesh configuration dictionary for a given geometry preset and resolution, suitable for use with PINNeAPPle geometry builders.

`pinneapple_arena.pointwise.leaderboard`

Global ranking of architectures over many arena experiments (the PINNeAPPle Arena leaderboard).

`pinneapple_arena/pointwise/leaderboard.py` · 212 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`aggregate`	função	`(experiments, families=None) -> dict`	`{"total_experiments", "total_architectures", "problems_covered", "entries"}`; `families` maps an architecture id to its family label (e.g. from `pinneapple_arena.pointwise.ARCHITECTURES_CATALOG`).

`pinneapple_arena.pointwise.losses`

PDE-residual losses of the arena problems, `(model, {"X": collocation points}) -> (loss, info)`.

`pinneapple_arena/pointwise/losses.py` · 732 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsLossGenerator`	classe	`(problem_id, parameters)`	Compiles physics (PDE residual) losses for Arena experiments.

`pinneapple_arena.pointwise.problems`

Physics problems catalog for the PINNeAPPle Arena.

`pinneapple_arena/pointwise/problems.py` · 1.875 linhas · **sem teste direto**

`pinneapple_arena.pointwise.schema`

Descriptors of the arena catalogs: physics problems, geometry presets and architectures (pydantic v2).

`pinneapple_arena/pointwise/schema.py` · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsArea`	classe	`()`	
`Dimension`	classe	`()`	
`DifficultyLevel`	classe	`()`	
`ArchitectureFamily`	classe	`()`	
`GeometryType`	classe	`()`	
`ProblemParameter`	classe	`(name, default, min_val, max_val, description, unit)`	A configurable scalar parameter for a physics problem.
`ColumnHints`	classe	`(coord_cols, output_cols, time_col, csv_header, notes)`	Describes the expected column layout for a dataset associated with this problem. Used to guide users when uploading CSV / NPZ / JSON files and to auto-populate the column mapping UI.
`PhysicsProblem`	classe	`(id, name, description, domain, dimension, equations, applications, initial_conditions, b…)`	Full descriptor for a physics problem available in the arena.
`GeometryPreset`	classe	`(id, name, description, geometry_type, dimensions, supported_problems, mesh_resolution_de…)`	Pre-built 3-D geometry available for 3-D experiments.
`MeshInfo`	classe	`(n_points, n_elements, bounds, quality_score)`	Summary statistics returned after meshing a geometry.
`ArchitectureInfo`	classe	`(id, name, family, description, detailed_description, supports_physics_loss, supports_3d,…)`	Metadata for a neural architecture available in the arena.

`pinneapple_arena.pointwise.tables`

Numeric tables uploaded to the arena (CSV / TSV / TXT with an auto-detected separator, JSON records, NPZ): `parse_numeric_table(filename, content) -> (columns, array, n_skipped)`.

`pinneapple_arena/pointwise/tables.py` · 111 linhas · testes: 16

nome	tipo	assinatura	o que faz
`parse_numeric_table`	função	`(filename, content) -> tuple[list[str], np.ndarray, int]`	`(columns, float32 array (rows, columns), n_skipped)` of an uploaded table (format from the suffix).

`pinneapple_arena.pointwise.training`

Training and scoring of one architecture on a pointwise field problem (the PINNeAPPle Arena protocol).

`pinneapple_arena/pointwise/training.py` · 706 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ArchitectureUnavailable`	classe	`()`	The registry could not build the architecture for this problem's data (unknown id, or a constructor that needs inputs a pointwise problem does not have). The architecture is reported as failed, never replaced.
`best_device`	função	`() -> str`	cuda > mps > cpu, as a string (`pinneapple_neural.trainer.best_device`).
`device_info`	função	`() -> dict`	The compute device and, for CUDA, its name, memory and CUDA version.
`field_metrics`	função	`(y_pred, y_true) -> dict`	Error metrics of a prediction against its reference, pooled over all points and fields (`pinneapple_physics.metrics` on the flattened arrays): `l2_relative`, `l1` (mean absolute error), `rmse`, `max_error`, `r2` and `mape` (percent, over the points where the reference is not ~0).
`prediction_quality`	função	`(y_pred, y_true, metrics) -> dict`	Detect degenerate model outputs and produce a prediction quality score (a heuristic of this arena, not a physics check: it flags constant, zero, NaN and worse-than-the-mean predictions).
`problem_in_dim`	função	`(problem_id, parameters=None) -> int`	Returns input dimensionality (number of coordinates) for the problem.
`problem_out_dim`	função	`(problem_id) -> int`	Returns output dimensionality (number of fields) for the problem.
`instantiate_model`	função	`(arch_id, in_dim, out_dim) -> Any`	Build `arch_id` from the PINNeAPPle registry (`pp.build_model`) for `(N, in_dim) -> (N, out_dim)` data, wrapped by the pointwise / closed-form adapters where it takes grids or sequences.
`model_forward`	função	`(model, X_t) -> Any`	Unified forward pass that handles diverse model return types.
`train_torch_model`	função	`(model, X_train, y_train, X_val, y_val, X_col, physics_loss_fn, n_epochs, lr, device, pro…)`	Train a torch model with: - Supervised data loss on X_train / y_train - Optional PDE residual loss on SEPARATE X_col (physics-informed) - Cosine annealing LR schedule - Early stopping on held-out X_val / y_val - Best-weights restore before returning
`train_closed_form_model`	função	`(model, X_train, y_train, X_val, y_val, progress_callback=None) -> dict[str, Any]`	Train a ClosedFormAdapter model (ELM, hybrid RBF, ESN readouts) by fit().
`run_inverse_solver`	função	`(model, problem_id, inverse_cfg, coords, solution, device) -> dict`	Wrap a trained/fresh model with PINNeAPPle's InverseProblemSolver and call invert() to recover unknown PDE parameters.
`predict`	função	`(model, X, device='cpu') -> tuple[np.ndarray, float]`	`(prediction, milliseconds)` of a trained model on `X` (no gradients).

nome	tipo	assinatura	o que faz
`rank_models`	função	`(results) -> list[dict]`	Completed results ranked by relative L2 error (ascending, `rank` 1..n); failed ones after, `rank` None.

`pinneapple_arena.problems`

Arena physics problem registry.

`pinneapple_arena/problems.py` · 1.206 linhas · testes: 15 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`ArenaProblem`	classe	`(name, description, domain, input_dim, output_dim, physics_preset)`	
`KovasznayNS`	classe	`()`	
`Stokes2D`	classe	`()`	2D Stokes (Poiseuille) flow: u = 1-y², v = 0, p = -2x.
`Heat1D`	classe	`()`	1D heat: u_t = alpha*u_xx, u = exp(-alpha*pi^2*t)*sin(pi*x).
`Heat2D`	classe	`()`	2D heat: u_t = alpha*(u_xx+u_yy).
`ConvectionDiffusion1D`	classe	`()`	Steady 1D convection-diffusion: c*u_x = eps*u_xx. Analytical: u = (exp(c*x/eps) - 1)/(exp(c/eps) - 1) on [0,1], u(0)=0, u(1)=1.
`DarcyFlow2D`	classe	`()`	2D Darcy: -Δp = f, p = sin(πx)sin(πy), f = 2π²sin(πx)sin(πy).
`Laplace2D`	classe	`()`	2D Laplace: Δu = 0, u = sin(πx)sinh(πy)/sinh(π).
`Poisson2D`	classe	`()`	2D Poisson: Δu = -2π²sin(πx)sin(πy), u = sin(πx)sin(πy).
`Helmholtz2D`	classe	`()`	2D Helmholtz: u_xx + u_yy + k²u = f, u = sin(πx)sin(πy).
`Biharmonic2D`	classe	`()`	2D Biharmonic (thin plate): Δ²u = f, u = sin(πx)sin(πy), f = 4π⁴*u.
`AdvectionDiffusion1D`	classe	`()`	1D advection-diffusion: u_t + c*u_x = nu*u_xx. Analytical: u = exp(-nu*pi²*t)*sin(pi*(x - c*t)).
`Wave1D`	classe	`()`	1D wave: u_tt = c²u_xx, u = sin(πx)cos(πct).
`KleinGordon1D`	classe	`()`	1D Klein-Gordon: u_tt - u_xx + m²u = 0. Analytical: u = cos(kx)cos(ωt), ω² = k² + m².
`NLS1D`	classe	`()`	1D NLS bright soliton: |ψ(x,t)| = A*sech(A*(x-vt)). We solve for (u, v) = (Re ψ, Im ψ) using the real/imaginary split. Solves i*psi_t + psi_xx + 2*|psi|^2*psi = 0 (nonlinearity coefficient 2).

nome	tipo	assinatura	o que faz
`Burgers1D`	classe	`()`	1D Burgers: u_t + u*u_x = nu*u_xx, u(x,0) = -sin(pi*x), u(-1,t)=u(1,t)=0.
`KdV1D`	classe	`()`	1D KdV: u_t + 6u*u_x + u_xxx = 0. Soliton: u = -2c*sech²(sqrt(c)*(x - 4c*t)).
`AllenCahn`	classe	`()`	1D Allen-Cahn: u_t = eps²u_xx + u - u³. Tanh interface.
`FisherKPP1D`	classe	`()`	1D Fisher-KPP: u_t = D*u_xx + r*u*(1-u). Exact Ablowitz-Zeidler traveling wave: u = [1/(1+exp(w*(x-v*t)))]^2 with v = 5*sqrt(D*r/6) and w = sqrt(r/(6D)).
`FitzHughNagumo1D`	classe	`()`	1D FitzHugh-Nagumo excitable media (two fields: v, w). v_t = v_xx + v(1-v)(v-a) - w + I, w_t = eps*(v - gamma*w).
`LinearElasticity2D`	classe	`()`	2D plane-strain linear elasticity (manufactured solution). Displacement: u = sin(πx)sin(πy), v = 0. Body force computed from Lamé equations.
`BlackScholes1D`	classe	`()`	1D Black-Scholes PDE for a European call option. V_t + r*S*V_S + sigma^2*S^2/2*V_SS - r*V = 0. Analytical via BS formula (evaluated on (log-price, time) grid).
`AllenCahn`	classe	`()`	
`get_problem`	função	`(name) -> ArenaProblem`	
`register_problem`	função	`(problem) -> 'ArenaProblem'`	Register a custom ArenaProblem instance so Arena can find it by name.
`list_problems`	função	`() -> List[str]`	
`list_problems_by_domain`	função	`() -> Dict[str, List[str]]`	

`pinneapple_arena.regimes`

Regime arena: which method works best for a physical problem, per data regime, under one protocol.

`pinneapple_arena/regimes.py` · 387 linhas · testes: 19

nome	tipo	assinatura	o que faz
`ArenaResult`	classe	`(problem, protocol, regimes, methods, rankings, overall, baselines, audit, created, hardw…)`	
`arena_markdown`	função	`(r, title='') -> str`	A comparative report from `ArenaResult.to_dict()` (also from a stored result file).

nome	tipo	assinatura	o que faz
`run_regime_arena`	função	`(data, *, methods='auto', budget=10.0, seeds=3, regimes='auto', few_data_fraction=0.25, s…)`	See the module docstring. `extrapolation`: the shift split for the extrapolation regime (`data.meta['shift']` or `"edge"` by default; see `splits.make_splits`); `constraints` default to `data.meta['constraints']`; `pde` to `data.meta['pde']`.

`pinneapple_arena.showcase`

Four contrasting PINN architectures trained on the same viscous Burgers problem, ranked by accuracy and by Arena's physics-aware view.

`pinneapple_arena/showcase.py` · 158 linhas · testes: 5

nome	tipo	assinatura	o que faz
`field_fit_slice`	função	`(xy_eval, y_ref, y_pred, grid_n) -> dict[str, Any]`	Reference and prediction on the last time row of the evaluation grid (t = 1 for `burgers_1d`).
`run_benchmark`	função	`(problem_name=PROBLEM_NAME, architectures=None, seed=None, progress_cb=None, n_col=N_COL,…)`	Train every architecture on the same collocation points, evaluate on the same grid, rank.

`pinneapple_arena.trainer`

Unified training engine for Arena benchmarks.

`pinneapple_arena/trainer.py` · 616 linhas · testes: 25

nome	tipo	assinatura	o que faz
`TrainResult`	classe	`(name, model, train_losses, train_time, metrics, physics_residual, uq_result, inverse_res…)`	
`train_pinn`	função	`(model, cfg, pinn_residuals_fn, xy_int, xy_bc, uv_bc, problem_params, device='cpu', log_i…)`	Train a PINN model.
`train_supervised`	função	`(model, cfg, X_train, Y_train, device='cpu', log_interval=100) -> TrainResult`	
`grid_layout`	função	`(x) -> Tuple[np.ndarray, int, int]`	Order of the rows of x (n, 2) that lays them out as a full tensor grid, y-major: (order, ny, nx). Raises ValueError when the points are not a complete grid (an FNO cannot take scattered points).
`train_grid_operator`	função	`(model, cfg, X_grid, Y_grid, device='cpu', log_interval=100) -> TrainResult`	Train an FNO-type model on whole fields: every step sees the full training grid (ordered), never a random subset of points reshaped as if it were an image.
`train_graph`	função	`(model, cfg, node_feats, edge_index, edge_attr, node_targets, device='cpu', log_interval=…)`	

nome	tipo	assinatura	o que faz
evaluate_model	função	(result, cfg, xy_eval, Y_ref, field_names, device='cpu', node_positions=None, edge_index=…)	
run_uq	função	(result, xy_eval, uq_cfg, device='cpu') -> Optional[Any]	Run UQ analysis using pinneapple_analysis.uq_predict.
run_inverse	função	(result, xy_eval, Y_ref, inv_cfg, device='cpu') -> Optional[Any]	Run inverse problem using pinneapple_analysis.invert.
load_pinneapple_dataset	função	(dataset_id, input_fields, output_fields, n_train=1000, n_val=200, split_seed=42)	Load a pinneapple_data dataset and split into train/val arrays.

`pinneapple_arena.visualizer`

Dark-theme visualization for Arena benchmark results.

`pinneapple_arena/visualizer.py` · 452 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_field_comparison	função	(eval_results, field_names, xy_eval, grid_n, problem_name, dark_theme=True, dpi=150, save…)	Multi-panel field comparison: reference | model1 | model2 | … | error.
plot_loss_curves	função	(train_results, dark_theme=True, dpi=150, save_path=None, show=False)	
plot_metrics_table	função	(eval_results, field_names, train_results, dark_theme=True, dpi=150, save_path=None, show…)	
plot_streamlines	função	(eval_results, xy_eval, grid_n, u_field='u', v_field='v', field_names=None, problem_name=…)	
plot_uq	função	(uq_result, xy_eval, field_names, grid_n=None, title='UQ', dark_theme=True, dpi=150, save…)	Plot UQ mean +/- std from a pinneapple_analysis UQResult.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 15 de 54.
- Funções de teste em arquivos que importam algum módulo do pacote: 99.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
T15	Adaptive hyperparameter sweep / architecture search (arena NAS)	untested	0	0
T21	Autoresearch loop (fixed-budget trials, keep/revert, random or LLM proposer)	tested	0	2

Limitações conhecidas

- *Nenhuma registrada.*

6. `pinneapple_worldmodel`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
31	15.016	90	49	94 (6 arquivos)	23

Pasta de testes dedicada: `tests/pinneapple_worldmodel` (8 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`benchmark`		1	3	0
`camera_system`		1	3	0
`curriculum`		1	3	0
`dataset`		1	3	20
`dataset_factory`		1	4	0
`dataset_packager`		1	4	0
`generate_datasets`		1	4	0
`geometry`		1	17	0
`inspection_data`		1	11	39
`mega_model`		1	5	0
`meta_learning`		1	3	0
`model`		1	2	20
`model_zoo`		1	4	0
`orchestrator`		1	3	20
`photorealistic_enhancer`		1	3	0
`physical_world`		1	12	39
`physics_renderer`		1	3	0
`physics_solver_bridge`		1	4	0
`physics_tools`		1	2	35
`pipeline`		1	3	0
`reality_randomizer`		1	2	0
`robot_perception`		1	13	39
`scenario`		1	1	0
`scenario_generator`		1	2	0

submódulo	o que faz	módulos	classes + funções	testes*
simulator		1	2	0
specialist_trainer		1	2	0
synthetic_factory		1	7	0
train		1	5	0
trainer		1	3	0
training_hooks		1	6	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

pinneapple_worldmodel

pinneapple_worldmodel — Generalist Physics AI World Model.

pinneapple_worldmodel/__init__.py · 409 linhas · testes: 11

pinneapple_worldmodel.benchmark

Standard physics AI benchmark suite.

pinneapple_worldmodel/benchmark.py · 564 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
BenchmarkTask	classe	(name, scenario_name, n_trajectories, rollout_steps, seed, physics_checks, metrics, descr…)	A single benchmark evaluation task.
BenchmarkResult	classe	(task_name, next_step_rmse, rollout_rmse, rel_l2, div_step, physics_scores, n_trajectorie…)	Results for one BenchmarkTask.
PhysicsBenchmark	classe	(device='cpu', verbose=True) -> None	Evaluate physics AI models on standard tasks.

pinneapple_worldmodel.camera_system

Stage 5 — Camera System.

pinneapple_worldmodel/camera_system.py · 392 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CameraConfig	classe	(name, sensor_type, position, look_at, fov_deg, resolution, fps, near_clip, far_clip, noi…)	Single virtual camera / sensor configuration.
MultiCameraArray	classe	(cameras, sync_fps)	A set of cameras sharing the same simulation field.
CameraSystem	classe	(array=None, domain_bounds=None) -> None	Project physics fields to camera-plane observations.

pinneapple_worldmodel.curriculum

Curriculum learning for physics world models.

pinneapple_worldmodel/curriculum.py · 283 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CurriculumStage	classe	(scenarios, n_samples, epochs, target_val_loss, lr, rollout_steps, description)	One stage in the curriculum.
CurriculumConfig	classe	(stages, model_config, device, batch_size, patience, save_dir, verbose)	Full curriculum configuration.
PhysicsCurriculum	classe	(config) -> None	Train a world model through a staged physics curriculum.

pinneapple_worldmodel.dataset

World model dataset: (state_t, params) → state_{t+1} pairs.

pinneapple_worldmodel/dataset.py · 382 linhas · testes: 20

nome	tipo	assinatura	o que faz
WorldModelDataset	classe	(trajectories, *, horizon=1, normalize=True, transform=None) -> None	One-step or multi-step prediction dataset from a list of trajectories.
DatasetConfig	classe	(scenarios, n_samples_per_scenario, horizon, normalize, validate_physics, save_dir, devic…)	Configuration for :class:DatasetBuilder.
DatasetBuilder	classe	(config) -> None	Orchestrates physics simulation → :class:WorldModelDataset assembly.

pinneapple_worldmodel.dataset_factory

Multi-source physics dataset factory.

pinneapple_worldmodel/dataset_factory.py · 725 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FactoryConfig	classe	(sources, scenarios, n_samples_per_scenario, horizon, validate_physics, device, save_dir,…)	Configuration for :class:PhysicsDatasetFactory.
DatasetEntry	classe	(name, dataset, source, scenario, physics_tags, n_samples, generation_time_s, metadata)	One dataset slice from the catalog.
DatasetCatalog	classe	() -> None	Registry of all generated physics datasets.
PhysicsDatasetFactory	classe	(config) -> None	Generate multi-source physics training datasets.

pinneapple_worldmodel.dataset_packager

Stage 8 — Dataset Packager.

pinneapple_worldmodel/dataset_packager.py · 439 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PackagerConfig	classe	(output_dir, format, zarr_chunks, zarr_compressor, save_vtk, save_videos, sample_id_digit…)	Controls how the packager writes the dataset.
PackagedSample	classe	(sample_id, sample_dir, field_paths, video_paths, metadata_path)	Metadata for a single packaged sample.

nome	tipo	assinatura	o que faz
`DatasetManifest`	classe	`(output_dir) -> None`	Index of all samples in a Physical AI dataset.
`DatasetPackager`	classe	`(config=None) -> None`	Write one physics sample to the canonical dataset structure.

`pinneapple_worldmodel.generate_datasets`

Dataset generation for Physics World Model training.

`pinneapple_worldmodel/generate_datasets.py` · 441 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate`	função	`(scenarios=None, sources=None, n_samples=100, output_dir='./data/worldmodel', device='cpu…)`	Generate multi-source physics datasets and return a DatasetCatalog.
`generate_via_factory`	função	`(scenarios=None, sources=None, n_samples=100, output_dir='./data/worldmodel', device='cpu…)`	Generate datasets using the high-level PhysicsDatasetFactory.
`load_catalog`	função	`(path) -> DatasetCatalog`	Load a previously saved DatasetCatalog from disk.
`main`	função	`() -> None`	

`pinneapple_worldmodel.geometry`

Physical domain geometry for physics simulation and world model training.

`pinneapple_worldmodel/geometry.py` · 569 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometryBase`	classe	`()`	Abstract base class for all geometry objects.
`Rectangle`	classe	`(lo, hi) -> None`	Axis-aligned rectangle `[lo_x, hi_x] × [lo_y, hi_y]`.
`Circle`	classe	`(center, radius) -> None`	2-D filled circle.
`Polygon`	classe	`(vertices) -> None`	2-D filled polygon defined by ordered vertices.
`Box3D`	classe	`(lo, hi) -> None`	Axis-aligned 3-D box.
`Sphere`	classe	`(center, radius) -> None`	3-D filled sphere.
`Union`	classe	`(a, b) -> None`	CSG union: points inside *a* OR *b*.
`Intersection`	classe	`(a, b) -> None`	CSG intersection: points inside *a* AND *b*.
`Difference`	classe	`(a, b) -> None`	CSG difference: points inside *a* but NOT inside *b*.
`BoundaryRegion`	classe	`(name, bc_type, value, sampler)`	A named subset of the domain boundary with its own BC type.

nome	tipo	assinatura	o que faz
`PhysicsDomain`	classe	`(geometry, boundaries, name, tags)`	Complete domain specification: geometry + boundary regions + metadata.
`make_unit_square`	função	`(*, with_boundary_labels=True) -> PhysicsDomain`	Unit square $[0,1]^2$ — standard benchmark domain.
`make_cavity`	função	`(L=1.0) -> PhysicsDomain`	Lid-driven cavity: unit square with moving lid (y=L, u=1).
`make_channel`	função	`(length=4.0, height=1.0, *, inlet_velocity=1.0) -> PhysicsDomain`	Poiseuille channel flow domain.
`make_channel_with_cylinder`	função	`(length=4.0, height=1.0, cyl_center=(1.0, 0.5), cyl_radius=0.1) -> PhysicsDomain`	2-D channel with circular obstacle — classic benchmark.
`make_l_shaped`	função	`() -> PhysicsDomain`	L-shaped domain (stress concentration benchmark).
`make_annulus`	função	`(inner_r=0.3, outer_r=1.0, center=(0.0, 0.0)) -> PhysicsDomain`	Annular domain — useful for heat/EM problems in cylindrical geometry.

`pinneapple_worldmodel.inspection_data`

Synthetic thermal-inspection data with labels from physics, scene variations written as OpenUSD stages, and the check that an augmentation kept the label true.

`pinneapple_worldmodel/inspection_data.py` · 224 linhas · testes: 39

nome	tipo	assinatura	o que faz
`Variation`	classe	`(gain, offset, noise, blur, zoom, angle_deg, shift, glint)`	
`thermal_plate`	função	`(defect, n=48, *, k=1.0, k_void=0.15, q=1.0) -> dict`	Steady conduction on the unit square with a low-conductivity disk `defect = (x, y, r)`. Returns the field, the defect-free field, the coefficient and the label (the cell where the void raises the temperature most).
`sample_variation`	função	`(rng, *, glint_p=0.2) -> Variation`	
`camera_transform`	função	`(var, H, W)`	Plate unit square (u, v) -> image pixel (col, row) and its inverse, for the variation's camera.
`write_variation_stage`	função	`(path, plate, var, *, cmap='gray') -> str`	
`thermal_image`	função	`(stage_path, var, H=64, W=64, seed=0) -> np.ndarray`	The camera image `(H, W)` of a variation stage.
`nominal_image`	função	`(stage_path, plate, var, H=64, W=64) -> np.ndarray`	What the same camera would see on the defect-free plate (the inspector's reference): the nominal field through the variation's gain, offset, pose and blur, no glint, no noise.

nome	tipo	assinatura	o que faz
label_check	função	(image, nominal, var, label_xy, *, tol=0.08) -> dict	Does the image still show the physical anomaly where the label says? The residual against the nominal image, smoothed (2 px), its maximum mapped back to the plate; tol in plate units (0.08 = about 4 cells). A glint that outshines the defect moves the maximum and fails; so does a label on …
inspection_dataset	função	(n, seed, workdir, *, glint_p=0.25, H=64, W=64, keep_stages=0) -> dict	n samples: a random subsurface void, its steady temperature field, a random camera variation written as a USD stage and rendered from it, the physical label in pixels, and the label check. Stages beyond the first keep_stages are deleted after rendering.
inspection_sample	função	(x=0.4, y=0.6, r=0.08, seed=0, glint=False, usd_path='inspection_sample.usda') -> dict	One synthetic thermal-inspection sample: the plate with a void at (x, y), radius r (plate units), a random camera variation (with a specular glint if asked) written as a USD stage at usd_path, and the label check of its image.
cleanup	função	(paths) -> None	

pinneapple_worldmodel.mega_model

Physics Foundation Model — the generalist mega-model.

pinneapple_worldmodel/mega_model.py · 668 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FoundationConfig	classe	(n_modes, width, depth, context_dim, descriptor_vocab, n_heads, n_context_layers, lora_ra…)	Hyper-parameter configuration for :class:PhysicsFoundationModel.
LoRALinear	classe	(in_features, out_features, rank=8) -> None	Low-Rank Adaptation layer for fast domain adaptation.
PhysicsDescriptorEncoder	classe	(context_dim=64, max_scalar_features=32, n_heads=4, n_layers=2, dropout=0.1) -> None	Encode a physics descriptor dict into a fixed-size context vector.
FoundationFNOBlock	classe	(width, n_modes, context_dim, lora_rank=8, dropout=0.1, spatial_dim=2) -> None	One FNO block with context injection and optional LoRA.
PhysicsFoundationModel	classe	(config, *, n_fields=1, grid_shape=(64, 64)) -> None	Generalist physics AI foundation model.

pinneapple_worldmodel.meta_learning

Meta-learning for the physics world model.

pinneapple_worldmodel/meta_learning.py · 512 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
MetaConfig	classe	(algorithm, n_meta_epochs, n_inner_steps, inner_lr, outer_lr, n_tasks_per_batch, n_suppor…)	Configuration for :class:MetaLearner.
TaskDistribution	classe	(catalog, n_support=32, n_query=32, device='cpu') -> None	Sample physics tasks from a :class:~.dataset_factory.DatasetCatalog.

nome	tipo	assinatura	o que faz
`MetaLearner`	classe	`(config, model_config=None) -> None`	MAML / Reptile meta-learning for physics world models.

`pinneapple_worldmodel.model`

Physics World Model architecture.

`pinneapple_worldmodel/model.py` · 320 linhas · testes: 20

nome	tipo	assinatura	o que faz
`WorldModelConfig`	classe	`(n_modes, width, depth, context_dim, embed_dim, activation, dropout, rollout_steps)`	Hyperparameters for :class:`PhysicsWorldModel`.
`PhysicsWorldModel`	classe	`(config, *, n_fields, grid_shape) -> None`	FNO-based physics world model: (state_t, context) → state_{t+1}.

`pinneapple_worldmodel.model_zoo`

Specialist model registry and checkpoint management.

`pinneapple_worldmodel/model_zoo.py` · 494 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZooEntry`	classe	`(name, model, scenario, physics_tags, metrics, n_params, trained_at, checkpoint_path, met…)`	One entry in the :class:`ModelZoo`.
`EnsembleModel`	classe	`(models, weights=None, aggregation='mean') -> None`	Averaging ensemble of multiple PhysicsWorldModel specialists.
`WaMaModel`	classe	`()`	Model created by averaging the weights of multiple specialist models.
`ModelZoo`	classe	`(root_dir=None) -> None`	Registry of trained specialist physics models.

`pinneapple_worldmodel.orchestrator`

General-purpose physics problem solver — tool-based orchestrator.

`pinneapple_worldmodel/orchestrator.py` · 766 linhas · testes: 20

nome	tipo	assinatura	o que faz
`ProblemStatement`	classe	`(kind, description, pde_hint, domain_hint, scenarios, params, observations, model, n_samp…)`	A high-level description of a physics problem to solve.
`OrchestratorResult`	classe	`(kind, artifacts, plan, elapsed_s, logs)`	Output of :class:`PhysicsOrchestrator.solve`.
`PhysicsOrchestrator`	classe	`(registry=None, verbose=True, tool_gate=None) -> None`	Solve arbitrary physics problems by chaining Pineapple tools.

`pinneapple_worldmodel.photorealistic_enhancer`

Stage 7 — Photorealistic Enhancer.

`pinneapple_worldmodel/photorealistic_enhancer.py` · 283 linhas · **sem teste direto** · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`EnhancerConfig`	classe	`(backend, strength, style_prompt, negative_prompt, model_id, device, batch_size, apply_to…)`	Configuration for the photo-realistic enhancer.
`PhotorealisticEnhancer`	classe	`(config=None) -> None`	Enhance visual realism of rendered frames WITHOUT touching physics data.
`register_enhancer`	função	`(name, fn) -> None`	Register a custom enhancement backend.

`pinneapple_worldmodel.physical_world`

PINNeAPPle World Model: physics-labelled synthetic video, a world model that trains on a CPU, and the physics checks that decide whether any video (simulated, predicted or generated by a foundation model such as NVIDIA Cosmos) is physically consistent.

`pinneapple_worldmodel/physical_world.py` · 489 linhas · testes: 39

nome	tipo	assinatura	o que faz
`Video`	classe	`(frames, dt, px_per_m, labels, params)`	
`WorldModel`	classe	`(H=64, W=96, latent=32, seed=0)`	Encoder (conv) -> latent `z` -> GRU dynamics -> decoder (deconv); trained to predict frame t+1 from frames <= t. Small enough to train on a CPU in minutes on 64 x 96 frames.
`StateEncoder`	classe	`(H=64, W=96, seed=0)`	A small CNN that reads the object's centre (pixels) from a frame, trained on the exact labels of the synthetic videos (the labels are free because the frames come from a simulation).
`CosmosAdapter`	classe	`(base_url, api_key=None, timeout=600.0)`	`generate(prompt, frames=None, seed=0)` on an NVIDIA Cosmos world foundation model NIM: `POST {base}/infer` with `prompt`, `video` (base64 MP4 of `frames` when given), `seed`, `video_params`; the answer's `b64_video` is decoded to frames (needs `imageio` with an FFmpeg plugin). ...
`bouncing_ball_scene`	função	`(*, seed=0, T=48, fps=24.0, H=64, W=96, randomize=True, defect=False, e=None) -> Video`	A thrown ball bouncing on the floor, rendered with domain randomization; exact labels.
`dataset`	função	`(n, *, seed=0, **kw) -> list[Video]`	
`track_object`	função	`(frames) -> dict[str, np.ndarray]`	Centre `(cx, cy)` [px] and area [px] of the brightest-contrast object per frame (NaN when none is found).
`physics_checks`	função	`(video, *, dt=None, px_per_m=None, g=G, g_rtol=0.1, area_rtol=0.25, max_speed=10.0) -> di…)`	Physics checks of a ball video from its pixels; `video` is a :class:`Video` or frames (then give `dt` and `px_per_m`). Returns `{"checks": [...], "passed": bool, "track": ..., "g_fit": ...}`.
`video_check`	função	`(seed=0, edit='none') -> dict[str, Any]`	A synthetic bouncing-ball video (exact physics, randomized camera), optionally edited (`"time_warp"`, `"teleport"`, `"fade"`, `"gravity_flip"`), judged by :func:`physics_checks` from its pixels alone.

nome	tipo	assinatura	o que faz
`corrupt`	função	`(video, kind, seed=0) -> Video`	Non-physical edits: `"time_warp"` (motion played at a varying rate), `"teleport"` (frames swapped in the middle), `"fade"` (the object fades out), `"gravity_flip"` (the video played backwards in time after the first bounce, so the ball gains energy).
`state_pairs`	função	`(videos) -> tuple[np.ndarray, np.ndarray]`	`(s_t, s_t+1)` with `s = (x, y, vx, vy)` in metres from the exact labels (the training data of a learned state dynamics).
`render_like`	função	`(video, x_m, y_m) -> np.ndarray`	Frames of a trajectory `(x_m, y_m)` rendered with `video`'s camera and appearance (its randomization parameters), noise from a fixed seed.

`pinneapple_worldmodel.physics_renderer`

Stage 4 — Physics Renderer.

`pinneapple_worldmodel/physics_renderer.py` · 444 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RendererConfig`	classe	`(resolution, fps, sensors, colormap_rgb, colormap_thermal, colormap_depth, colormap_ir, d…)`	Configuration for the PhysicsRenderer.
`RenderResult`	classe	`(frames, video_paths, png_dirs, field_stats)`	Output of PhysicsRenderer.render().
`PhysicsRenderer`	classe	`(config=None) -> None`	Render physical field trajectories into video frames.

`pinneapple_worldmodel.physics_solver_bridge`

Stage 2 — Physics Solver Layer + Stage 3 — Physical Ground Truth Dataset.

`pinneapple_worldmodel/physics_solver_bridge.py` · 837 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SolverOutput`	classe	`(fields, t_coords, x_coords, y_coords, z_coords, params, solver_name, metadata)`	Multivariate physical field history from one simulation run.
`SolverBridgeConfig`	classe	`(solver, generate_temperature, generate_concentration, openfoam_case_dir, fenics_problem,…)`	Configuration for the PhysicsSolverBridge (Stage 2).
`PhysicsSolverBridge`	classe	`(config=None) -> None`	Dispatch a physics scenario to the appropriate solver backend.
`GroundTruthPackager`	classe	`(output_dir, format='zarr', zarr_chunks=None, overwrite=True) -> None`	Organise SolverOutput into the canonical Physical AI dataset structure.

`pinneapple_worldmodel.physics_tools`

Pinneapple module capability inventory for tool-based orchestration.

`pinneapple_worldmodel/physics_tools.py` · 983 linhas · testes: 35

nome	tipo	assinatura	o que faz
`PhysicsTool`	classe	`(name, category, description, input_schema, output_schema, module_path, fn, tags)`	One registered physics capability.
`PhysicsToolRegistry`	classe	`() -> None`	Catalogue of all Pinneapple capabilities as callable tools.

pinneapple_worldmodel.pipeline

Physics AI Pipeline — unified entry point.

`pinneapple_worldmodel/pipeline.py` · 498 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsAIConfig`	classe	`(scenarios, sources, n_samples, device, save_dir, specialist_epochs, specialist_lr, speci…)`	Complete configuration for :class:`PhysicsAIPipeline`.
`PhysicsAIPipelineResult`	classe	`(catalog, zoo, meta_model, mega_model, benchmark, elapsed_s)`	All outputs of :class:`PhysicsAIPipeline.run`.
`PhysicsAIPipeline`	classe	`(config) -> None`	Build a generalist physics AI world model using all Pinneapple tools.

pinneapple_worldmodel.reality_randomizer

Stage 6 — Reality Randomizer.

`pinneapple_worldmodel/reality_randomizer.py` · 328 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RandomizerConfig`	classe	`(seed, p_sensor_noise, gaussian_std, poisson_scale, p_jpeg, jpeg_quality_lo, jpeg_quality…)`	Controls which augmentations are applied and their strength.
`RealityRandomizer`	classe	`(config=None) -> None`	Apply reality-gap augmentations to rendered image sequences.

pinneapple_worldmodel.robot_perception

Robot perception data from the robot sim: arm motions simulated by PINNeAPPle Robot Sim (NVIDIA Newton), written as OpenUSD timelines with a camera, seen by a randomized synthetic camera, labelled with the simulator's exact joint angles, and checked for what a perception model of a robot must ...

`pinneapple_worldmodel/robot_perception.py` · 283 linhas · testes: 39

nome	tipo	assinatura	o que faz
`CameraVariation`	classe	`(zoom, offset, gain, noise, blur, albedo, background, occluder)`	
`ImageRegressor`	classe	`(H, W, n_out, seed=0)`	A small CNN from a grey image `(H, W)` to `n_out` numbers.

nome	tipo	assinatura	o que faz
`forward_points`	função	`(arm, q) -> dict[str, np.ndarray]`	Elbow and tool point `(T, 2)` in the x-z plane for joint angles `q (T, 2)` (the simulator's convention: angles from the horizontal, positive turning x towards -z).
`arm_motions`	função	`(arm, n, seed, *, duration=0.8, dt=0.001, fps=30.0, kp=(350.0, 110.0), kd=(5.0, 2.5), q_b…)`	`n` random point-to-point moves simulated in Newton, sampled at `fps` (or the given `moves`: `[(q_start, q_end)]`).
`sample_camera`	função	`(rng, *, occluder_p=0.3) -> CameraVariation`	
`write_motion_stage`	função	`(path, arm, motion, var, *, payload=None) -> str`	`payload`: `(V, F, grey)`, a mesh carried at the tool point (in the tool frame: x along the forearm).
`camera_frames`	função	`(path, arm, motion, var, H=72, W=72, seed=0) -> np.ndarray`	`(T, H, W)` grey frames of the stage at each recorded time.
`visibility`	função	`(arm, motion, var, *, margin=0.03) -> np.ndarray`	Per frame: are the elbow and the tool point inside the image and not behind the occluder?
`payload_track_check`	função	`(frames, arm, motion, var, *, tol_px=2.0, offset=(0.0, 0.0)) -> dict`	Does each frame show the carried part where the kinematics put it? The centroid of the pixels brighter than the links (the part is drawn white, the links grey: threshold half-way between the two after the camera's gamma and gain) against the tool point, plus the part's `offset` (x, z in the tool …
`arm_camera_sample`	função	`(payload=5.0, seed=0, occluder=False, usd_path='arm_camera_sample.usda') -> dict`	One labelled perception sample: a random arm move in the robot sim (Newton), its USD timeline with a side camera (random variation, an occluder if asked) at `usd_path`, the camera frames, and the label checks (the forward kinematics against the simulator's link poses, the frames that show the …
`angle_features`	função	`(q) -> np.ndarray`	
`feature_angles`	função	`(f) -> np.ndarray`	
`perception_checks`	função	`(q_pred, motion, arm, *, qd_max=12.0) -> dict`	Errors of a predicted angle sequence for one motion, all frames visible.

`pinneapple_worldmodel.scenario`

Physics scenario definitions for world model training.

`pinneapple_worldmodel/scenario.py` · 253 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicsScenario	classe	(name, pde_kind, grid_shape, t_span, n_steps, domain_bounds, param_ranges, ic_type, bc_ty…)	Complete specification of a physics simulation problem.

pinneapple_worldmodel.scenario_generator

Stage 1 — Scenario Generator.

pinneapple_worldmodel/scenario_generator.py · 360 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ScenarioSpec	classe	(scenario, domain, geometry, fluid, boundary_conditions, initial_conditions, sensor_confi…)	Extended scenario specification produced by the generator.
ScenarioGenerator	classe	(default_grid_2d=(64, 64), default_n_steps=32, device='cpu') -> None	Convert a user prompt or JSON dict into a :class:ScenarioSpec.

pinneapple_worldmodel.simulator

Physics trajectory simulator.

pinneapple_worldmodel/simulator.py · 630 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TrajectoryData	classe	(states, params, scenario_name, metadata)	One physics trajectory: sequence of field snapshots + metadata.
PhysicsSimulator	classe	(scenario, *, device='cpu', verbose=False) -> None	Generate physics trajectories from a :class:PhysicsScenario.

pinneapple_worldmodel.specialist_trainer

Specialist model training using Pinneapple's full training stack.

pinneapple_worldmodel/specialist_trainer.py · 423 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SpecialistConfig	classe	(model, epochs, lr, batch_size, device, val_fraction, patience, grad_clip, rollout_steps,…)	Training configuration for :class:SpecialistTrainer.
SpecialistTrainer	classe	(config, *, physics_loss_fn=None, pretrained_zoo=None) -> None	Train specialist models and populate a :class:~.model_zoo.ModelZoo.

pinneapple_worldmodel.synthetic_factory

Physics Synthetic Data Factory for Physical AI.

pinneapple_worldmodel/synthetic_factory.py · 525 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SyntheticFactoryConfig	classe	(scenario_input, n_samples, output_dir, device, render_fps, sensors, render_resolution, a…)	Single configuration data-class covering all 9 stages.
FactoryResult	classe	(config, spec, samples, output_dir, elapsed_s)	Output of SyntheticDataFactory.generate_dataset().

nome	tipo	assinatura	o que faz
SyntheticDataFactory	classe	(config=None) -> None	Physics Synthetic Data Factory for Physical AI.
generate_pipe_flow	função	(n_samples=20, output_dir='./pipe_dataset', **kw) -> FactoryResult	Generate a pipe flow dataset (NS2D, Re=100-5000).
generate_cylinder_flow	função	(n_samples=20, output_dir='./cylinder_dataset', **kw) -> FactoryResult	Generate flow-around-cylinder dataset (classic von Karman vortex shedding).
generate_heat_transfer	função	(n_samples=20, output_dir='./heat_dataset', **kw) -> FactoryResult	Generate 2D heat conduction dataset.
generate_chemical_mixing	função	(n_samples=20, output_dir='./mixing_dataset', **kw) -> FactoryResult	Generate chemical mixing / advection-diffusion dataset.

pinneapple_worldmodel.train

Physics World Model training entry point.

pinneapple_worldmodel/train.py · 593 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
train_specialist	função	(catalog, output_dir, *, epochs=50, batch_size=32, lr=0.001, device='cpu', patience=10, n…)	Train one specialist model per scenario in the catalog.
train_meta	função	(catalog, output_dir, *, algorithm='reptile', n_meta_epochs=100, n_inner_steps=5, inner_l…)	Meta-train a shared initialisation over the full scenario catalog.
train_foundation	função	(catalog, output_dir, *, base_ckpt=None, epochs=30, batch_size=16, lr=0.0005, device='cpu…)	Build and fine-tune the PhysicsFoundationModel (mega-model with LoRA).
run_pipeline	função	(scenarios=None, sources=None, n_samples=100, output_dir='./checkpoints/worldmodel', devi…)	Run the complete 5-stage Physics AI pipeline.
main	função	() -> None	

pinneapple_worldmodel.trainer

World model training loop.

pinneapple_worldmodel/trainer.py · 421 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
WorldModelLoss	classe	(rollout_steps=1, rollout_weight=0.5, physics_loss_fn=None, physics_weight=0.01) -> None	Multi-step rollout loss for world model training.
WorldModelTrainConfig	classe	(epochs, lr, weight_decay, batch_size, val_fraction, device, amp, grad_clip, rollout_step…)	Training hyperparameters for :class:WorldModelTrainer.
WorldModelTrainer	classe	(model, config, *, physics_loss_fn=None) -> None	Train a :class:PhysicsWorldModel on a :class:WorldModelDataset.

`pinneapple_worldmodel.training_hooks`

Stage 9 — Model Training Layer.

`pinneapple_worldmodel/training_hooks.py` · 478 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsAIDataset`	classe	`(dataset_dir, fields=('u', 'v', 'p', 'T'), sensors=(), max_samples=None, device='cpu') ->…)`	Base dataset over a packaged Physical AI dataset directory.
`CosmosEncoderDataset`	classe	`(dataset_dir, video_sensors=('rgb', 'thermal'), field_names=('u', 'v', 'p', 'T'), **kwarg…)`	Dataset for training a video → latent embedding encoder.
`PhysicsDecoderDataset`	classe	`(dataset_dir, embedding_dim=512, field_names=('u', 'v', 'p', 'T'), device='cpu', max_samp…)`	Dataset for training a latent embedding → physical field decoder.
`InversePINNDataset`	classe	`(dataset_dir, param_keys=None, video_sensor='rgb', **kwargs) -> None`	Dataset for inverse PINN training: video frames → physical parameters.
`NeuralOperatorDataset`	classe	`(dataset_dir, field_names=('u', 'v', 'p'), horizon=1, subsample_t=1, device='cpu', max_sa…)`	Dataset for training neural operators on (state_t, state_{t+1}) pairs.
`TrainingHooks`	classe	`(dataset_dir, device='cpu') -> None`	Convenience factory for all training dataset types.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 8 de 31.
- Funções de teste em arquivos que importam algum módulo do pacote: 94.

Limitações conhecidas

- *Nenhuma registrada.*

7. `pinneapple_perception`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
7	1.037	6	19	29 (3 arquivos)	2

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`audio_modal`		1	1	0
`change_detection`		1	10	13

submódulo	o que faz	módulos	classes + funções	testes*
image_geometry		1	2	0
piv_tools		1	6	6
sentinel2		1	4	13
video_piv		1	2	6

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

pinneapple_perception

pinneapple_perception — extract physics observations from images, video, and audio, for use as `DataConstraint` targets or boundary geometry anywhere else in PINNeAPPle.

pinneapple_perception/__init__.py · 55 linhas · testes: 23 · sinais no código: experimental

pinneapple_perception.audio_modal

Extract dominant vibration/acoustic frequencies from an audio recording (e.g. a struck beam, plate, or shell's impulse response) via FFT peak-picking with sub-bin parabolic frequency refinement. Depends only on numpy/scipy (already core PINNeAPPle dependencies).

pinneapple_perception/audio_modal.py · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
extract_dominant_frequencies	função	(waveform, sample_rate, n_peaks=5, min_freq=0.0, max_freq=None, window='hann') -> List[Tu…)	Return the n_peaks most prominent frequency components of waveform, as a list of (frequency_Hz, amplitude) tuples sorted by descending amplitude.

pinneapple_perception.change_detection

Change detection between two multispectral images of the same place: per-band mean absolute reflectance change and NDVI change against thresholds, a comparison figure and a self-contained HTML report.

pinneapple_perception/change_detection.py · 288 linhas · testes: 13 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
BandChangeResult	classe	(band, mean_abs_diff_raw, mean_abs_diff_reflectance, threshold, exceeds_threshold)	
NdviChangeResult	classe	(mean_abs_diff, threshold, exceeds_threshold)	

nome	tipo	assinatura	o que faz
`PairAnalysis`	classe	`(date_a, date_b, band_results, ndvi_result, any_change, narrative_html)`	
`band_difference`	função	`(img_a, img_b, band_index, band_name, threshold=DEFAULT_BAND_DIFF_THRESHOLD, reflectance_…)`	Real per-band mean-absolute-difference change check, same computation as the original notebook's `detecta_mudanca` (mean of |img_b - img_a| for one band, normalized, compared to a threshold), but with an explicit, physically-meaningful `reflectance_scale` (default the real Sentinel-2 SR scale, …
`compute_ndvi`	função	`(red, nir, eps=1e-06) -> np.ndarray`	Real NDVI: (NIR - RED) / (NIR + RED + eps). `red` and `nir` must both be REAL band arrays (Sentinel-2 B4 and B8 respectively) -- this function has no fallback that fabricates either band. Same formula as the original notebook's `calcula_ndvi`, applied to a real NIR input instead of the original's …
`ndvi_change`	função	`(ndvi_a, ndvi_b, threshold=DEFAULT_NDVI_THRESHOLD) -> NdviChangeResult`	
`analyze_pair`	função	`(rgb_a, rgb_b, date_a, date_b, nir_a=None, nir_b=None, band_diff_threshold=DEFAULT_BAND_D …)`	Real per-pair change analysis: three band-difference checks (red/green/blue) always run; NDVI is computed and checked only if real `nir_a`/`nir_b` arrays are supplied (never fabricated). `date_a` should chronologically precede `date_b` -- the caller is responsible for ordering (this fixes a real, …
`rgb_preview_stretch`	função	`(img, normalizer=…) -> np.ndarray`	Purely a display contrast stretch (clip to [0,1] after dividing by a fixed constant) -- matches the original notebook's `img/3000` preview trick. This has NO physical-reflectance meaning (see this module's docstring); it is only ever used for the RGB preview panels, never for the quantitative …
`render_comparison_figure`	função	`(rgb_a, rgb_b, date_a, date_b, ndvi_a=None, ndvi_b=None) -> bytes`	Render the same-style comparison grid the original notebook produced (RGB pair + diff, per-band diff maps, and -- when real NDVI arrays are supplied -- an NDVI before/after/delta row), as real PNG bytes (matplotlib `Agg` backend, no display needed).
`render_html_report`	função	`(pairs, images_png, lon, lat, buffer_m, has_ndvi) -> str`	

`pinneapple_perception.image_geometry`

Extract boundary/geometry point clouds from an image -- e.g. a photo or scan of a physical part's cross-section -- for use as a domain boundary in `pinneapple_design.geometry` or as boundary-condition selector data. Depends only on numpy/scipy (already core PINNeAPPle dependencies).

`pinneapple_perception/image_geometry.py` · 114 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`extract_boundary_points`	função	`(image, threshold=None, units_per_pixel=1.0, ordered=True) -> np.ndarray`	Extract the boundary of the foreground region(s) in `image` as a point cloud.

nome	tipo	assinatura	o que faz
`estimate_bounding_circle`	função	`(points) -> Tuple[float, float, float]`	Least-squares circle fit (algebraic Kasa method) to a boundary point cloud -- a quick, standard way to validate/characterize a roughly-circular extracted boundary (e.g. a pipe or shaft cross-section). Returns (center_x, center_y, radius).

pinneapple_perception.piv_tools

Around :mod:`.video_piv`: reading frames from a video or image files, the vector-grid size a setting gives, a summary of a PIV result, and window / search-margin advice from the expected displacement.

`pinneapple_perception/piv_tools.py` · 128 linhas · testes: 6

nome	tipo	assinatura	o que faz
`FFmpegMissing`	classe	`()`	`ffmpeg` or `ffprobe` is not on PATH.
`decode_video`	função	`(path) -> np.ndarray`	(T, H, W) float64 grayscale frames of the video at `path` (ffprobe for the size, ffmpeg for the pixels).
`decode_image_frames`	função	`(files) -> np.ndarray`	(T, H, W) grayscale frames from `(file name, bytes)` pairs, sorted by file name (use zero-padded names).
`piv_grid_count`	função	`(height, width, window_size, search_margin, step=None, *, margin=None) -> tuple[int, int]`	(rows, cols) of interrogation windows `piv_velocity_field` uses on an H x W frame.
`summarize_piv`	função	`(seq, frame_shape, window_size, search_margin, step=None) -> dict`	Counts and speeds of a `piv_velocity_sequence` result on frames of shape (T, H, W).
`piv_parameter_guide`	função	`(displacement_px, width_px, height_px) -> dict`	Window size, search margin and step for an expected displacement (px per frame), with warnings.

pinneapple_perception.sentinel2

Sentinel-2 surface-reflectance images of a point from Google Earth Engine, as arrays for :mod:`pinneapple_perception.change_detection` (bands B4, B3, B2 and the real NIR band B8).

`pinneapple_perception/sentinel2.py` · 136 linhas · testes: 13

nome	tipo	assinatura	o que faz
`EarthEngineUnavailable`	classe	`()`	Raised when ee.Initialize() genuinely fails -- no credential, no project, or the Earth Engine API not enabled for the given project. Carries the real underlying exception's message plus the concrete remediation steps from this module's docstring, so a caller sees exactly what a real deployment ...
`Sentinel2Frame`	classe	`(date, rgb, nir)`	

nome	tipo	assinatura	o que faz
`ee_initialize`	função	`(project=None) -> None`	Real ee.Initialize() call. Tries a service account credential if EE_SERVICE_ACCOUNT_EMAIL + EE_SERVICE_ACCOUNT_KEY_FILE are set (the real, non-interactive path for a server deployment); otherwise falls through to ee's default persistent-credential lookup (populated by a prior real `earthengine …
`fetch_sentinel2_sequence`	função	`(lon, lat, buffer_m=5000.0, start_date='2024-01-01', end_date='2025-07-27', max_cloud_pct…)`	Real fetch of the n_images most recent, least-cloudy Sentinel-2 SR images intersecting a buffered point, returned oldest-to-newest (chronological order -- the original notebook fetched newest-first and then had two of its own cells label "before/after" in opposite directions for the same pairs; …

`pinneapple_perception.video_piv`

Extract a velocity field from a video (image pair/sequence) using cross-correlation Particle Image Velocimetry (PIV) -- the actual standard technique experimental fluid dynamicists use to turn a video of a seeded/textured flow into a velocity field, not a generic computer- vision optical-flow …

`pinneapple_perception/video_piv.py` · 221 linhas · testes: 6 · sinais no código: experimental

nome	tipo	assinatura	o que faz
`piv_velocity_field`	função	`(frame_a, frame_b, window_size=32, search_margin=16, step=None, dt=1.0, units_per_pixel=1…)`	Cross-correlation PIV velocity field between two grayscale frames.
`piv_velocity_sequence`	função	`(frames, dt=1.0, **piv_kwargs) -> Dict[str, np.ndarray]`	Run `piv_velocity_field` on every consecutive pair in a video (`frames`: (T, H, W) array) and stack the results with a `t` column, ready for a 3D (x, y, t) `DataConstraint`.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 5 de 7.
- Funções de teste em arquivos que importam algum módulo do pacote: 29.

Limitações conhecidas

- *Nenhuma registrada.*

8. `pinneapple_quantum`

Objetivo

Sem texto de objetivo.

Números do inventário

mó-dulos	li-nhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
16	2.636	17	24	0 (0 arquivos)	16

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`backends`		2	7	0
`circuits`		2	4	0
`data`		2	7	0
`loss`		2	10	0
`models`		3	2	0
`pipeline`		2	5	0
`training`		2	4	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_quantum`

pinneapple_quantum — PINNeAPPle Quantum Module (PQM).

`pinneapple_quantum/__init__.py` · 210 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_circuit`	função	`(n_qubits=4, depth=2, ansatz='hardware_efficient', encoding='angle', n_observables=1, bac…)`	Shortcut to build a :class:`QuantumModel` from flat arguments.
`backend_info`	função	`(provider='pennylane') -> dict`	Return info dict for the specified backend.

`pinneapple_quantum.backends`

Módulo sem docstring e sem texto.

`pinneapple_quantum/backends/__init__.py` · 4 linhas · **sem teste direto**

`pinneapple_quantum.backends.backend`

Quantum backend adapter — PennyLane, Qiskit, or classical simulator.

`pinneapple_quantum/backends/backend.py` · 324 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`BackendConfig`	classe	`(provider, device_name, diff_method, shots)`	Configuration for the quantum execution backend.
`QuantumBackend`	classe	`(config)`	Abstract base for quantum backends.
`PennyLaneBackend`	classe	`()`	PennyLane backend (supports CPU/GPU simulators and real QPUs).
`QiskitBackend`	classe	`()`	Qiskit / Aer backend.

nome	tipo	assinatura	o que faz
`ClassicalBackend`	classe	`()`	Classical state-vector simulator.
`ClassicalQNode`	classe	`(circuit_fn, n_qubits)`	Callable that executes a quantum circuit description on the classical engine.
`get_backend`	função	`(config=None) -> 'QuantumBackend'`	Instantiate the correct backend from a config (or defaults).

`pinneapple_quantum.circuits`

Módulo sem docstring e sem texto.

`pinneapple_quantum/circuits/__init__.py` · 8 linhas · **sem teste direto**

`pinneapple_quantum.circuits.base`

Quantum circuit builder — ansatz and encoding definitions.

`pinneapple_quantum/circuits/base.py` · 300 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`QuantumCircuitConfig`	classe	`(n_qubits, depth, ansatz, encoding, n_observables)`	Full specification for a variational quantum circuit (VQC).
`ClassicalVQC`	classe	`(config)`	Variational quantum circuit **approximated** entirely in PyTorch.
`build_pennylane_circuit`	função	`(config, backend)`	Construct a PennyLane QNode implementing the given circuit config.
`build_classical_circuit`	função	`(config, backend)`	Construct the same circuit as :func:`build_pennylane_circuit`, but recorded against a gate-recorder protocol (`recorder.rx/ry/rz/cnot/hadamard/cz/measure_z`) that :class:`~pinneapple_quantum.backends.backend.ClassicalQNode` replays on an exact state-vector engine — used when no quantum package ...

`pinneapple_quantum.data`

Módulo sem docstring e sem texto.

`pinneapple_quantum/data/__init__.py` · 20 linhas · **sem teste direto**

`pinneapple_quantum.data.quantum_dataset`

Quantum dataset layer — states, operators, measurements, and domain samplers.

`pinneapple_quantum/data/quantum_dataset.py` · 290 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`QuantumState`	classe	`(coords, psi_real, psi_imag, meta)`	A pure quantum state ψ(x) over a spatial domain.
`QuantumObservable`	classe	`(expectation, variance, operator_names)`	A measured observable produced by a quantum circuit.

nome	tipo	assinatura	o que faz
`QuantumCollocationDataset`	classe	`(x_col, x_bc, psi_exact=None, batch_size=256)`	Dataset of domain collocation points for quantum PINN training.
`SpinChainDataset`	classe	`(n_spins=4, n_samples=1024, J=1.0, h=0.5, seed=42)`	Dataset of spin-chain configurations and their energy labels.
`sample_domain_1d`	função	`(x_min=0.0, x_max=1.0, n_col=1000, n_bc=50, device='cpu', dtype=torch.float32) -> Tuple[t…)`	Sample collocation and boundary points for a 1D quantum domain.
`sample_domain_2d`	função	`(x_min=-2.0, x_max=2.0, y_min=-2.0, y_max=2.0, n_col=4096, n_bc=200, device='cpu', dtype=…)`	Sample collocation and boundary points for a 2D quantum domain.
`exact_eigenstates_1d_harmonic`	função	`(x, n_state=0, omega=1.0, hbar=1.0, mass=1.0) -> Tuple[torch.Tensor, float]`	Analytically compute the n-th eigenstate of the 1D harmonic oscillator.

pinneapple_quantum.loss

Módulo sem docstring e sem texto.

`pinneapple_quantum/loss/__init__.py` · 26 linhas · **sem teste direto**

pinneapple_quantum.loss.schrodinger

Physics losses for the Schrödinger equation and quantum Hamiltonians.

`pinneapple_quantum/loss/schrodinger.py` · 344 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`harmonic_potential`	função	`(x, omega=1.0, mass=1.0) -> torch.Tensor`	V(x) = ½mω²‖x‖² — quantum harmonic oscillator.
`infinite_well_potential`	função	`(x, width=1.0) -> torch.Tensor`	V(x) = 0 inside (0, L), handled via Dirichlet BCs. Returns 0 in domain.
`coulomb_potential`	função	`(x, charge=1.0, eps=0.001) -> torch.Tensor`	V(r) = −e²/r — hydrogen-like atom potential (regularized at origin).
`double_well_potential`	função	`(x, a=1.0, b=0.0) -> torch.Tensor`	V(x) = (x² − a)² + bx — symmetric / asymmetric double well.
`get_potential`	função	`(name_or_fn) -> Callable`	Resolve a potential by name string or return it directly if callable.
`energy_loss`	função	`(model, x, potential='harmonic_oscillator', hbar=1.0, mass=1.0, **potential_kwargs) -> Tu…)`	Variational energy (Rayleigh quotient) for the time-independent Schrödinger equation.
`pde_residual_loss`	função	`(model, x, eigenvalue, potential='harmonic_oscillator', hbar=1.0, mass=1.0, **potential_k…)`	Strong-form PDE residual for the time-independent Schrödinger equation.
`time_dependent_loss`	função	`(model, x, t, potential='harmonic_oscillator', hbar=1.0, mass=1.0, **potential_kwargs) ->…)`	PDE residual for the time-dependent Schrödinger equation.
`normalization_loss`	função	`(model, x, domain_volume=1.0) -> torch.Tensor`	Soft constraint enforcing wavefunction normalization: ∫|ψ|² dx ≈ 1.

nome	tipo	assinatura	o que faz
`hamiltonian_expectation`	função	`(model, x, H_matrix) -> torch.Tensor`	Discrete Hamiltonian expectation ⟨ψ|H|ψ⟩ for matrix Hamiltonians.

`pinneapple_quantum.models`

Módulo sem docstring e sem texto.

`pinneapple_quantum/models/__init__.py` · 5 linhas · **sem teste direto**

`pinneapple_quantum.models.hybrid_model`

HybridModel — classical neural network + variational quantum circuit.

`pinneapple_quantum/models/hybrid_model.py` · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HybridModel`	classe	`(classical_net, quantum_model, mode='classical_first', out_dim=None)`	Classical–quantum hybrid model that combines an `nn.Module` with a :class:`QuantumModel` into a single differentiable computational graph.

`pinneapple_quantum.models.quantum_model`

QuantumModel — variational quantum circuit as a PyTorch nn.Module.

`pinneapple_quantum/models/quantum_model.py` · 133 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`QuantumModel`	classe	`(circuit_config, backend_config=None, output_scale=1.0)`	A variational quantum circuit (VQC) exposed as a standard PyTorch `nn.Module`.

`pinneapple_quantum.pipeline`

Módulo sem docstring e sem texto.

`pinneapple_quantum/pipeline/__init__.py` · 16 linhas · **sem teste direto**

`pinneapple_quantum.pipeline.presets`

High-level pipeline presets for common quantum ML workflows.

`pinneapple_quantum/pipeline/presets.py` · 490 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`vq_pinn`	função	`(equation='schrodinger_1d', potential='harmonic_oscillator', n_qubits=4, depth=3, ansatz=…)`	Build and train a Variational Quantum PINN (VQ-PINN).
`hybrid_pinn`	função	`(classical_in=1, classical_hidden=32, n_qubits=4, depth=2, potential='harmonic_oscillator…)`	Hybrid PINN: classical MLP pre-encodes features → quantum circuit → output.
`learn_hamiltonian`	função	`(data, labels, n_spins=4, epochs=2000, lr=0.001, device='cpu', seed=42) -> Dict[str, Any]`	Learn a matrix Hamiltonian from energy measurement data.

nome	tipo	assinatura	o que faz
`quantum_kernel`	função	`(x_train, x_test=None, n_qubits=4, encoding='angle', backend='pennylane') -> torch.Tensor`	Compute a quantum-feature-map kernel matrix K(xᵢ, xⱼ) for use with classical kernel methods (SVM, GP regression) in an elevated feature space.
`solve`	função	`(problem='quantum', equation='schrodinger', potential='harmonic', method='vq_pinn', backe…)`	High-level entry-point that mirrors the PINNeAPPle `solve()` API.

`pinneapple_quantum.training`

Módulo sem docstring e sem texto.

`pinneapple_quantum/training/__init__.py` · 9 linhas · **sem teste direto**

`pinneapple_quantum.training.trainer`

Quantum-aware hybrid trainer with parameter-shift support.

`pinneapple_quantum/training/trainer.py` · 345 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`QTrainerConfig`	classe	`(epochs, lr, optimizer, quantum_gradient, lambda_norm, lambda_bc, print_every, device)`	Configuration for the quantum-aware trainer.
`QTrainHistory`	classe	`(epochs, total_loss, pde_loss, energy, norm_sq)`	Training history container.
`QTrainer`	classe	`(model, physics_loss_fn, bc_loss_fn=None, config=None)`	Hybrid classical-quantum trainer.
`parameter_shift_gradient`	função	`(loss_fn, weights, shift=math.pi / 2) -> torch.Tensor`	Compute exact gradient of `loss_fn` w.r.t. `weights` using the parameter-shift rule.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 0 de 16.
- Funções de teste em arquivos que importam algum módulo do pacote: 0.

Limitações conhecidas

- *Nenhuma registrada.*

Volume 5 — Confiança, validação e decisão

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

A camada que confere cada resultado contra a física: validação, incerteza, inversão, o linter `pinneapple check`, registros de evidência, segurança, decisão e a cadeia de porquês.

Sumário

1. pinneapple_analysis
2. pinneapple_check
3. pinneapple_veriphysics
4. pinneapple_security
5. pinneapple_decision
6. pinneapple_reason

1. `pinneapple_analysis`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
159	42.004	249	450	939 (83 arquivos)	56

Pasta de testes dedicada: `tests/pinneapple_analysis` (169 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`benchmark`		18	113	44
`cost`		4	25	55
`data_assimilation`		4	13	7
`inspection`	Suíte de ensaio não destrutivo (NDE): gera sinais de inspeção sintéticos para seis modalidades e treina uma PINN com a mesma EDP que gerou os dados.	8	21	13

sub-módulo	o que faz	módulos	classes + funções	testes *
`inverse_problems`	Problemas inversos: modelos diretos, identificabilidade, calibração, estimadores (mínimos quadrados, adjunto, autograd, MCMC, EKI, PINN), regularização, ruído, sensibilidade e descoberta de termos faltantes.	19	95	151
`proof`		3	14	11
`reproduction`		1	10	5
`retrieval`		3	11	32
`sensitivity`		6	21	28
`state_estimation`		2	4	22
`study_contracts`		1	7	4
`study_models`		1	3	25
`surrogate`		11	41	32
`trust`		4	11	13
`uncertainty`		21	79	185
`validation`	Validação de modelos de IA física: o relatório único `pp.validate`, checagens clássicas de PINN (conservação, contorno, simetria), métricas de campo e espectrais, testes metamórficos, rollout, injeção de falhas e a arena red team / blue team.	33	151	151
`verification`		19	78	43

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_analysis`

pinneapple_analysis — Model analysis, validation, and inversion.

`pinneapple_analysis/__init__.py` · 173 linhas · testes: 56

nome	tipo	assinatura	o que faz
`analyze_model`	função	`(model, spec, x_test, *, uq_method='mc_dropout', run_validate=True, run_uq=True) -> dict`	Run validation and uncertainty quantification in one call.
`invert`	função	`(model, y_obs, sensor_locs, *, noise_std=0.01, lambda_reg=0.001, method='adam', n_iters=2…)`	High-level inverse-problem shortcut (Gaussian misfit + Tikhonov).

pinneapple_analysis.benchmark

Equal-budget benchmarks of physics-AI methods: prepare -> validate -> run -> report.

`pinneapple_analysis/benchmark/__init__.py` · 34 linhas · testes: 44

pinneapple_analysis.benchmark.__main__

Módulo sem docstring e sem texto.

`pinneapple_analysis/benchmark/__main__.py` · 7 linhas · **sem teste direto**

pinneapple_analysis.benchmark.advisor

Diagnostics and suggestions: what in a benchmark's setup is likely to hurt its result, with the evidence and a fix.

`pinneapple_analysis/benchmark/advisor.py` · 222 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Diagnosis`	classe	`(severity, topic, subject, finding, evidence, suggestion, edit)`	
`diagnose`	função	`(cfg, plan, extracted, checks, arch_cfgs=None, loss_terms=None) -> list[Diagnosis]`	`extracted`: {(problem, method): {level: {seed: {set: {metric: {"values": [...]}}}}}} (report.collect).
`apply_suggestions`	função	`(cfg, diagnoses, path) -> BenchmarkConfig`	Write a new configuration with every suggested edit applied (for the user to review and run).

pinneapple_analysis.benchmark.api

`pp.benchmark`: prepare -> validate -> run -> report, with correctness gates and an advisor.

`pinneapple_analysis/benchmark/api.py` · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BenchmarkResult`	classe	`(ws, cfg, plan, arch, banners, blocked)`	
`prepare`	função	`(config=None, out_dir='benchmark_out', *, force=False, adopt=False, log=print, **kwargs) …)`	Create and validate the artifacts (solver config, geometry, data, cost, hashes) in `out_dir/prepared/`, for the user to inspect before any training. `adopt`: use artifacts you placed there yourself (they are hashed and checked the same way). Returns {problem: manifest}.
`run`	função	`(config=None, out_dir='benchmark_out', *, force=False, prepare_missing=True, stages=…, lo…)`	Run the benchmark (see the module docstring). Re-running with the same `out_dir` resumes from the cache; a different configuration is refused unless `force`. Missing artifacts are prepared first (`prepare_missing`); stale or failed ones stop the run unless `allow_failed_validation`.
`load`	função	`(out_dir) -> BenchmarkResult`	Open a finished (or partial) benchmark folder.

nome	tipo	assinatura	o que faz
report	função	(out_dir) -> BenchmarkResult	

pinneapple_analysis.benchmark.builtin

Built-in benchmark problems and methods (importing this module registers them).

pinneapple_analysis/benchmark/builtin/__init__.py · 9 linhas · **sem teste direto**

pinneapple_analysis.benchmark.builtin.airfoil

airfoil: the aero app's 2-D airfoil (issue #299). OpenFOAM v1912 simpleFoam, k-omega SST, Re 4e6, the app's O-grid (pinneapple_design.aero). Data: Latin-hypercube CST shapes x angles plus the NACA polars (the app's campaign, apps/aero_optimizer/tools/generate_dataset.py), and an OOD set ...

pinneapple_analysis/benchmark/builtin/airfoil.py · 654 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
AirfoilProblem	classe	()	
MGNMethod	classe	()	
MLPMethod	classe	()	
MLPFieldsMethod	classe	()	
PINNRansMethod	classe	()	
openfoam_available	função	() -> bool	

pinneapple_analysis.benchmark.builtin.airfoil_impl

Airfoil benchmark implementation (moved from PINNeAPPle-Benchmark experiment 09): MLP ensemble, MeshGraphNet, POD + MLP fields, potential-flow and RANS PINNs on the aero app's 2-D airfoil (OpenFOAM v1912 simpleFoam, k-omega SST, Re 4e6). The data directory is set by set_data_dir (the problem's ...

pinneapple_analysis/benchmark/builtin/airfoil_impl.py · 599 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PotentialNet	classe	(cfg, alpha_deg)	psi = U (y cos a - x sin a) + Gamma/(4 pi) ln r^2 + (1 - r^2/R^2) N(features), Gamma trainable.
RansNet	classe	(cfg, alpha_deg, wall)	(u, v, p) with no-slip exact on the wall: u = tanh(d / DELTA_W) (cos a + N_u), v likewise, p = N_p.
EddyViscosity	classe	(model, p0, e)	Prandtl mixing length with van Driest damping, evaluated from the current network and frozen (detached) for the step (a Picard linearisation): nu_t = l^2 |S|, l = min(kappa d (1 - exp(-d+/26)), 0.085 delta), with the friction velocity and the boundary-layer thickness of a turbulent flat plate at ...
set_data_dir	função	(prepared) -> None	

nome	tipo	assinatura	o que faz
`load_runs`	função	`(dirs)`	The aero app's run filter (apps/aero_optimizer/tools/train_surrogate.py): usable status, deep stall dropped, oscillating runs averaged over their last snapshots.
`split`	função	`(recs, frac=0.15, seed=3)`	The aero app's split: 15 % of the LHS shapes held out with all their angles; the NACA polars apart.
`splits`	função	`()`	
`eval_splits`	função	`(final, S)`	
`coefs`	função	`(recs)`	
`coef_scores`	função	`(true, pred) -> dict`	
`field_scores`	função	`(true, pred) -> dict`	Relative L2 per run of the velocity (Ux, Uy together) and of the pressure, on the graph nodes.
`train_mlp`	função	`(cfg, seed, budget, *, final) -> dict`	
`graph_data`	função	`(lay, recs)`	
`train_mgn`	função	`(cfg, seed, budget, *, final, batch=4) -> dict`	
`train_mlp_fields`	função	`(cfg, seed, budget, *, final) -> dict`	Field baseline for a like-for-like field comparison with the MeshGraphNet: POD of the training fields (p, Ux, Uy on the same graph nodes), and an MLP from the shape parameters and the angle to the POD coefficients.
`find_case`	função	`(case) -> dict`	
`collocation_pool`	função	`(shape, d_min) -> torch.Tensor`	Cell centres of the case's own O-grid (clustered at the wall, leading and trailing edges) inside r < R.
`wall_distance`	função	`(x, y, p0, e)`	Distance to the wall polygon (segments p0 -> p0 + e), differentiable almost everywhere.
`rans_residuals`	função	`() -> list[str]`	Steady incompressible RANS with an eddy viscosity nut(x, y) and its gradient (nutx, nuty): continuity and the x and y momentum equations, div((nu + nut)(grad u + grad u^T)) written out (the PINN's residuals).
`train_pinn`	função	`(cfg, seed, budget, *, case, kind='potential', ckpt=None) -> dict`	A PINN for one airfoil at one angle, no simulation data. kind = "potential" or "rans" (see module docstring).

pinneapple_analysis.benchmark.builtin.fields

Field-operator problems (input field -> output field on a grid) and the generic methods that learn them.

pinneapple_analysis/benchmark/builtin/fields.py · 614 linhas · testes: 29 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
FieldData	classe	(prepared)	The prepared arrays with pointwise Gaussian normalisers computed on the training split only.
FieldOperatorProblem	classe	()	Base of the grid-to-grid problems. The table reports, per method, the relative L2 of the output field and the relative error of an integral quantity (qoi) on the PINN instances, the whole test set and the OOD set.
ArrayProblem	classe	(id, *, train, val, test, ood=None, data_cost_s=None, qoi=None, pinn_setup=None, title=No…)	Wrap your own arrays as a benchmark problem.
FNOMethod	classe	()	
DeepONetMethod	classe	()	
PINNMethod	classe	()	Per-instance PINN through the problem's pinn_setup (or the problem's own train_pinn).
train_operator	função	(problem, prepared, method, cfg, seed, budget, *, final=True, **_) -> dict	
train_field_pinn	função	(problem, prepared, cfg, seed, budget, *, split, instance, **_)	
pinn_checks	função	(problem, prepared, cfg) -> list[Check]	Autograd derivatives against finite differences; the PDE residual of a known exact solution through the same residual code (catches a wrong PDE or a sign error); a tiny collocation set can be fitted.
finite	função	(x) -> bool	

pinneapple_analysis.benchmark.builtin.inverse_heat

inverse_heat: the convection coefficient h of a heat-spreader plate from 8 thermocouples (issue #299, case 2).

pinneapple_analysis/benchmark/builtin/inverse_heat.py · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
InverseHeatProblem	classe	()	

`pinneapple_analysis.benchmark.builtin.inverse_heat_impl`

Inverse heat benchmark implementation (moved from PINNeAPPle-Benchmark experiment 09): the convection coefficient h of the heat-spreader plate from 8 thermocouples; the plate PINN (run_plate under a budget) against surrogate + calibration (MLP, DeepONet on finite-volume runs) and the finite-volume ...

`pinneapple_analysis/benchmark/builtin/inverse_heat_impl.py` · 228 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Surrogate`	classe	`(method, cfg, xy)`	(log h, power) normalised -> (T - T_air) field (60 x 100), normalised.
`Data`	classe	`()`	
`set_data_dir`	função	`(prepared, smoke=False) -> None`	
`single_thread`	função	`()`	
`fv_field`	função	`(h, power)`	
`readings`	função	`(power, h_true, seed)`	
`generate`	função	`(out, n_train, n_val, workers=4) -> dict`	
`train_surrogate`	função	`(method, cfg, seed, budget)`	
`calibrate`	função	`(field_fn, power, t_meas, x, y)`	Golden-section search of h in [1, 300] W/m2K for the sensor misfit of field_fn(h, power) (degC field).
`surrogate_job`	função	`(method, cfg, seed, budget, *, final) -> dict`	
`pinn_job`	função	`(cfg, seed, budget, *, kind, case) -> dict`	
`classical_job`	função	`(seed) -> dict`	Golden-section search through the finite-volume model itself (the solver as the 'surrogate').

`pinneapple_analysis.benchmark.builtin.standard`

The standard operator-learning problems (Li et al. 2021, "Fourier Neural Operator for Parametric PDEs"), data from `pinneapple_simulation.numerical_solvers.operator_benchmarks`:

`pinneapple_analysis/benchmark/builtin/standard.py` · 478 linhas · testes: 16 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Burgers`	classe	`()`	
`Darcy`	classe	`()`	
`NavierStokes`	classe	`()`	

nome	tipo	assinatura	o que faz
`Bilinear`	classe	`(a)`	Bilinear interpolant of a cell-centred n x n field on the unit square and its gradient; values outside the outermost centres are held constant. No autograd: the coefficient does not depend on the network.
`HardNet`	função	`(problem, cfg, init=None, t_end=10.0)`	A library NeuralNetwork with input features and hard initial/boundary conditions:

`pinneapple_analysis.benchmark.checks`

Correctness gates of a benchmark run.

`pinneapple_analysis/benchmark/checks.py` · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`architecture_gates`	função	`(plan) -> list[Check]`	
`job_checks`	função	`(rec) -> list[Check]`	Checks of one finished job, from its run file.
`training_gates`	função	`(ws, plan) -> tuple[list[Check], dict[str, list[Check]]]`	All job checks, aggregated per (problem, method), plus the cross-run checks.
`determinism_check`	função	`(plan, pid, mid, steps=20) -> Check`	Run one job twice in this process with the same seed and a step budget: the results must agree.
`seed_spread`	função	`(values, subj, what) -> Check`	
`row_status`	função	`(arch, train, skipped, did_not_converge) -> str`	
`summarise`	função	`(checks) -> dict[str, Any]`	

`pinneapple_analysis.benchmark.cli`

`pinneapple benchmark prepare|run|report` (also `python -m pinneapple_analysis.benchmark`).

`pinneapple_analysis/benchmark/cli.py` · 64 linhas · testes: 16

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	

`pinneapple_analysis.benchmark.config`

Benchmark configuration: the same fields from Python keyword arguments or a YAML file, resolved and hashed.

`pinneapple_analysis/benchmark/config.py` · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BenchmarkConfig`	classe	`(problems, methods, budget, problem_budgets, budget_levels, seeds, n_trials, search_fract…)`	Everything a benchmark run depends on.

pinneapple_analysis.benchmark.registry

Problems and methods: base classes, the registry and the decorators that extend it.

`pinneapple_analysis/benchmark/registry.py` · 251 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Check`	classe	`(name, stage, subject, passed, value, threshold, detail, severity)`	One gate or validation item. `passed` None means not applicable (always with a reason in `detail`).
`Column`	classe	`(key, label, set, metric, kind, pct)`	A table column: `metric` of the instances in `set` (e.g. rel_l2 on test), `kind` field | integral.
`Problem`	classe	`(**options)`	Base class of a benchmark problem. Subclasses set `id`, `title` and implement the hooks they need.
`Method`	classe	`(id, title, per_instance, outputs)`	Base class of a benchmark method.
`register_problem`	função	`(problem_id)`	
`register_method`	função	`(method_id)`	
`get_problem`	função	`(p) -> Problem`	
`get_method`	função	`(m) -> Method`	
`problems`	função	`() -> list[str]`	
`methods`	função	`() -> list[str]`	

pinneapple_analysis.benchmark.report

The benchmark report: tables (n in every header, integral quantities and fields apart, data cost, break-even, a checks badge per row), the Checks, Validation, Trust card and Diagnostics sections, figures and summary.json.

`pinneapple_analysis/benchmark/report.py` · 648 linhas · testes: 16

nome	tipo	assinatura	o que faz
`fmt_s`	função	`(x) -> str`	
`collect`	função	`(plan) -> dict`	{(problem, method): {level: {seed: extracted}}} plus costs, from the run files.
`equal_flops_table`	função	`(plan, col, pid) -> dict | None`	Rows of the equal-FLOPs arm: the main metric on the instances common to all methods, and on the full set.
`cell`	função	`(seeds, set_, metric) -> dict | None`	
`build_tables`	função	`(plan, col, row_status) -> dict`	Rows per problem: {method: {column key: cell, ...}} plus costs and statuses.

nome	tipo	assinatura	o que faz
markdown	função	(plan, tables, checks, validation, trust, diagnoses, banners, studies=None) -> str	
write_csv	função	(path, plan, tables) -> None	
figures	função	(plan, col, tables, fig_dir, banners) -> list[str]	
summary_metrics	função	(prob)	(label, kind, (set, metric), (ood set, metric) or None, pct) lines of the summary figure for a problem.
to_dataframe	função	(plan, tables)	
collect_studies	função	(plan) -> dict	Budget study and ablations (setup audit), from the run files.
studies_markdown	função	(plan, studies) -> list[str]	
study_metric_key	função	(prob, method_id, pts) -> str | None	The metric column a budget-study curve shows: the method's own convergence metric (main_metric_for), so a field model is judged on its field and not on an integral it cannot predict; else the first column.
studies_figure	função	(plan, studies, fig_dir) -> list[str]	

pinneapple_analysis.benchmark.runner

The benchmark machinery: the out_dir workspace, the prepare stage, the job cache and the search, final and equal-FLOPs stages, executed as one process per job (threads_per_run threads each, n_workers at a time).

pinneapple_analysis/benchmark/runner.py · 397 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ConfigMismatch	classe	()	
PreparationError	classe	()	
Workspace	classe	(out_dir, config)	
Plan	classe	(ws, cfg, problems, methods)	The job specs of one benchmark: ids compatible with experiment 09 of PINNeAPPle-Benchmark.
prepare_problem	função	(ws, problem, *, n_workers=1, force=False, adopt=False, log=print) -> dict[str, Any]	Create (or reuse) the problem's artifacts, then verify the solver, check the data and run the validation stack on them. Returns the manifest (status ok / failed).

nome	tipo	assinatura	o que faz
require_prepared	função	(ws, problem, allow_failed) -> dict	
run_job	função	(spec, out_dir, objects=None) -> str	Run one job in this process and write runs/.json (the result, or the traceback).
execute_jobs	função	(ws, jobs, *, n_workers, objects=None, log=print) -> None	
lvl	função	(x) -> str	

pinneapple_analysis.benchmark.validation

PINNeAPPle's validation stack applied to a benchmark's prepared artifacts, as recorded checks.

pinneapple_analysis/benchmark/validation.py · 159 linhas · testes: 16

nome	tipo	assinatura	o que faz
physics_audit_check	função	(subject, name, family, equations, assumptions, features) -> Check	
envelope_checks	função	(subject, train, test, ood, features=None) -> list[Check]	
gci_check	função	(subject, quantity, values_fine_to_coarse, sizes_fine_to_coarse, max_gci=0.05) -> Check	
mesh_quality_check	função	(subject, mesh, target='cfd') -> Check	
preflight_check	função	(subject, case_dir) -> Check	
yplus_check	função	(subject, first_cell, length, velocity, nu, target=1.0) -> Check	First-cell y+ from the flat-plate estimate (length: the reference length, e.g. the chord).
field_features	função	(x) -> dict[str, float]	Scalar descriptors of an input field: amplitude and spectral centroid (for the applicability envelope).
field_validation	função	(problem, out, grid_study=None) -> list[Check]	
trust_card	função	(out_dir, checks, tier, limitations) -> dict[str, Any]	The six questions of pinneapple_lab's trust card, answered from the benchmark's own checks.

pinneapple_analysis.cost

Computational-cost validation per operation (time, memory, FLOPs, scaling, budgets, regressions).

pinneapple_analysis/cost/__init__.py · 50 linhas · testes: 42

pinneapple_analysis.cost.equal_budget

Equal-budget comparison of methods: a compute budget, a training loop that stops at it, FLOPs, search, amortised cost.

pinneapple_analysis/cost/equal_budget.py · 446 linhas · testes: 55

nome	tipo	assinatura	o que faz
ComputeBudget	classe	(wall_s, flops, steps)	A training budget: any combination of wall-clock seconds, FLOPs and optimiser steps (None = no limit).
BudgetRun	classe	(steps, wall_s, flops, flops_per_step, stopped_by, budget, history, eval_wall_s)	What a budgeted training run used and how it went.
SearchTrial	classe	(config, score, info)	
SearchResult	classe	(best_config, best_score, trials, n_trials, seed)	
AmortizedCost	classe	(data_s, train_s, infer_s, per_query_train_s)	Cost of answering new queries, in seconds (or any one unit used throughout).
count_flops	função	(fn) -> tuple[int, dict[str, int], Any]	Run fn() once and count its FLOPs, forward and backward (including torch.autograd.grad).
cosine_lr	função	(progress, lr0, final_fraction=0.03) -> float	Cosine decay from lr0 to final_fraction * lr0 over the budget (progress in [0, 1]).
set_lr	função	(optimizer, lr) -> None	
train_within_budget	função	(step, budget, *, flops_per_step=None, count_first_step=True, evaluate=None, eval_at=(), …)	Call step(progress) (one optimiser step; may return the loss) until budget is spent.
equal_budget_search	função	(space, objective, *, n_trials, seed=0, minimize=True) -> SearchResult	Random search over the grid space with exactly n_trials distinct configurations.
relative_l2	função	(pred, true, *, sample_axis=0, eps=1e-12) -> np.ndarray	Per-sample relative L2 error ||pred - true|| / ||true|| over every axis except sample_axis.
time_per_query	função	(fn, *, repeats=20, warmup=3, n_per_call=1, no_grad=True) -> dict[str, float]	Median wall time of fn() divided by n_per_call (instances answered per call), under no_grad unless no_grad=False (a query that needs derivatives, e.g. a PINN's velocity from its stream function).
hardware_record	função	() -> dict[str, Any]	CPU model, cores, memory, GPU, library versions and thread count: the line every budget table needs.

pinneapple_analysis.cost.profiler

Computational-cost validation per operation: measure, fit scaling, check budgets, catch regressions.

pinneapple_analysis/cost/profiler.py · 437 linhas · testes: 11

nome	tipo	assinatura	o que faz
`CostRecord`	classe	`(name, n_runs, time_s, time_min_s, time_iqr_s, times_s, peak_mem_bytes, mem_source, flops…)`	
`ScalingFit`	classe	`(quantity, exponent, ci, r2, coefficient, declared, expected_exponent, verdict)`	
`ScalingStudy`	classe	`(name, sizes, records, fits)`	
`Budget`	classe	`(max_time_s, max_mem_bytes, max_flops, max_time_exponent)`	Limits for one operation. Any limit left as `None` is not checked.
`BudgetExceeded`	classe	`()`	
`Regression`	classe	`(name, quantity, baseline, current, ratio)`	
`CostLedger`	classe	`()`	Named operations with their costs; save a baseline, compare later runs against it.
`measure`	função	`(fn, *args, repeats=5, warmup=1, name=None, flops=True, memory=True, size=None, **kwargs)…)`	Cost of one operation `fn(*args, **kwargs)`.
`expected_exponent`	função	`(declared, sizes) -> float`	Log-log slope of the declared complexity over the sizes actually used (`n log n` is not 1.0).
`scaling_study`	função	`(make_call, sizes, *, name='operation', repeats=5, warmup=1, declared=None, declared_flop…)`	Measure an operation over problem sizes and fit `cost ~ n^k` for time (and FLOPs when countable).

`pinneapple_analysis.cost.torch_ops`

Cost per operation for PyTorch models: the PINN training step broken into its parts, and a per-operator table.

`pinneapple_analysis/cost/torch_ops.py` · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pinn_operation_costs`	função	`(model, x, *, repeats=5, warmup=1, optimizer=None) -> Dict[str, CostRecord]`	Cost of each stage of a PINN step on collocation points `x` of shape `(N, d)`.
`profile_ops`	função	`(fn, *, top=15, sort_by='self_cpu_time_total') -> List[Dict[str, Any]]`	Per-PyTorch-operator table for one call: calls, self CPU time (ms) and FLOPs (matmul/conv/attention).

`pinneapple_analysis.data_assimilation`

Data assimilation: variational (4D-Var through autograd) on top of the ensemble methods in `pinneapple_analysis.inverse_problems` (EKI) and `pinneapple_analysis.state_estimation` (Kalman filters), plus Gaussian-process field reconstruction from sparse sensors and greedy sensor placement …

`pinneapple_analysis/data_assimilation/__init__.py` · 16 linhas · testes: 6

pinneapple_analysis.data_assimilation.models

Small chaotic models used to test data-assimilation methods, written in PyTorch so they are differentiable.

`pinneapple_analysis/data_assimilation/models.py` · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`rk4`	função	`(f, x, dt) -> torch.Tensor`	
`lorenz96_tendency`	função	`(x, F=8.0) -> torch.Tensor`	
`lorenz96_step`	função	`(x, dt=0.05, F=8.0) -> torch.Tensor`	
`lorenz63_step`	função	`(x, dt=0.01, sigma=10.0, rho=28.0, beta=8.0 / 3.0) -> torch.Tensor`	

pinneapple_analysis.data_assimilation.sensor_placement

Field reconstruction from sparse sensors with a Gaussian process, and greedy placement of the next sensors.

`pinneapple_analysis/data_assimilation/sensor_placement.py` · 236 linhas · testes: 7

nome	tipo	assinatura	o que faz
`GaussianProcessField`	classe	`(lengthscales=1.0, variance=1.0, nu=2.5, noise=1e-10, mean=0.0) -> None`	Gaussian-process regression of a scalar field from point sensors (Rasmussen & Williams 2006, ch. 2).
`SensorPlacement`	classe	`(indices, points, integrated_variance)`	Result of :func:`greedy_sensor_placement`.
`matern_kernel`	função	`(A, B, lengthscales=1.0, variance=1.0, nu=2.5) -> np.ndarray`	Anisotropic Matern covariance `k(a, b)` between the rows of `A` (n, d) and `B` (m, d).
`greedy_sensor_placement`	função	`(gp, candidates, n_sensors, domain=None, weights=None, existing=None) -> SensorPlacement`	Choose `n_sensors` locations among `candidates` that greedily minimise the integrated posterior variance.

pinneapple_analysis.data_assimilation.var4d

Strong-constraint 4D-Var through any differentiable PyTorch model: the adjoint comes from autograd.

`pinneapple_analysis/data_assimilation/var4d.py` · 174 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Observation`	classe	`(step, y, H, R)`	Observations `y` at model step `step`; `H` maps the state to observation space (default identity) and `R` is the error covariance (scalar variance, vector of variances, or full matrix).
`Var4DResult`	classe	`(x0, trajectory, cost_initial, cost_final, cost_history, n_iterations, background_term, o…)`	

nome	tipo	assinatura	o que faz
`Var4D`	classe	`(model, B_sqrt, dtype=torch.float64)`	`model(x) -> x_next`; `B_sqrt`: square root of the background covariance (scalar std, vector of stds, or a matrix L with B = L L^T).
`gradient_check`	função	`(var, x0, xb, obs, n_steps=None, eps=1e-06, seed=0) -> dict[str, float]`	Directional derivative of J from autograd versus a central finite difference along a random direction.
`twin_experiment`	função	`(n=40, window_steps=20, obs_every=2, obs_fraction=0.5, obs_std=1.0, background_std=1.0, d…)`	Identical-twin 4D-Var on Lorenz-96: a truth run, noisy observations of every `1/obs_fraction`-th variable every `obs_every` steps, a background perturbed by `background_std`. Returns RMSE of background and analysis at the start and end of the window, and of a forecast of one more window from …

pinneapple_analysis.inspection

Pacote de inspeção/NDE: reexporta `InspectionPINN`, `train_generic_inspection_pinn` e as funções `generate_*`, `make_*_physics_loss` e `train_*` de seis modalidades (correntes parasitas, MFL, acústica, EMAT, ondas guiadas, phased array). Todos os sinais são sintéticos e ilustrativos, nunca dados medidos.

`pinneapple_analysis/inspection/__init__.py` · 151 linhas · testes: 13

pinneapple_analysis.inspection.acoustic

Gera, por diferenças finitas no tempo (FDTD 1D), o espectro de condição de uma barra golpeada (impact-echo/ressonância), em que um defeito reduz a velocidade de onda local; inclui perda física da equação da onda e treino da PINN. [sem teste]

`pinneapple_analysis/inspection/acoustic.py` · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_acoustic_synthetic`	função	`(*, L=1.0, c0=5000.0, n_points=60, duration=0.0015, defect_position=0.6, defect_width=0.0…)`	Simula por FDTD 1D a barra sem e com defeito e devolve séries temporais, espectros e os dados de treino da PINN para o deslocamento `u(x, t)`.
`make_acoustic_physics_loss`	função	`(x_grid, c_field) -> PhysicsLossFn`	Cria a perda física `(model, batch) -> (loss, comps)` da equação da onda `u_tt - c(x)^2 u_xx = 0` via autograd, com `c(x)` tirado do mapa FDTD fixo.
`train_acoustic`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados (se não vierem), monta uma `InspectionPINN` de entrada (x, t) e treina com a perda acústica; devolve modelo, saída do treino e dados.

pinneapple_analysis.inspection.base

Base comum da inspeção: a MLP `InspectionPINN` (subclasse de `PINNBase`) e o wrapper de treino que usa o `Trainer` de `pinneapple_neural` com `CombinedLoss` (supervisionada + hook de física). [sem teste]

`pinneapple_analysis/inspection/base.py` · 251 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`InspectionPINN`	classe	`(in_dim, out_dim, hidden=(64, 64, 64), activation='tanh', *, modality='generic', dtype=to…)`	MLP coordenadas -> sinal de inspeção, no padrão de `VanillaPINN`; a física entra por um resíduo externo (`make_*_physics_loss`).
`train_generic_inspection_pinn`	função	`(model, synthetic_data, physics_loss_fn=None, *, epochs=200, lr=0.001, batch_size=32, val…)`	Converte um dict de arrays em lotes `{x, y, x_col}`, liga a perda física opcional por `PhysicsLossHook` e chama `Trainer.fit`, devolvendo o dict do treino.

`pinneapple_analysis.inspection.eddy_current`

Sinal sintético de varredura de sonda de correntes parasitas, construído sobre o solver axissimétrico de Helmholtz complexo de `pinneapple_simulation.numerical_solvers.eddy_current_fdm`, mais o treino da PINN com esse mesmo resíduo. A varredura lateral é uma aproximação axissimétrica de um problema 3D. [sem teste]

`pinneapple_analysis/inspection/eddy_current.py` · 281 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_eddy_current_synthetic`	função	`(scan_positions=None, *, nr=24, nz=40, r_bounds=(0.001, 0.05), z_bounds=(-0.03, 0.03), om…)`	Gera um conjunto sintético de varredura de sonda de correntes parasitas resolvendo o Helmholtz axissimétrico para cada posição de defeito.
`make_eddy_current_physics_loss`	função	`(omega, mu, sigma) -> PhysicsLossFn`	Cria a perda física com as partes real e imaginária do operador de Helmholtz complexo axissimétrico, com J=0 (válido no volume do condutor).
`train_eddy_current`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados se preciso, monta uma `InspectionPINN` e treina com a perda do resíduo de Helmholtz.

`pinneapple_analysis.inspection.emat`

Gerador simplificado de sinal EMAT: a força de Lorentz vira um termo de força escalar (Gaussiana no espaço, tone-burst no tempo) numa equação da onda elástica 1D; não resolve o acoplamento eletromagnético-elástico completo. [sem teste]

`pinneapple_analysis/inspection/emat.py` · 267 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_emat_synthetic`	função	`(*, L=0.3, rho=7800.0, G0=80000000000.0, n_points=60, duration=6e-05, coil_position=0.05,…)`	Gera um sinal EMAT pitch-catch sintético: um tone-burst tipo Lorentz lança uma onda elástica que um defeito (queda local de rigidez) reflete/atenua.
`make_emat_physics_loss`	função	`(x_grid, c_field, coil_x0, footprint_sigma, burst_freq, n_cycles, t0_burst, J0_over_rho) …)`	Cria a perda física da equação da onda forçada `u_tt - c(x)^2 u_xx - F(x,t) = 0`, com `F` em forma fechada.
`train_emat`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados EMAT se preciso e treina uma `InspectionPINN` com a perda da onda forçada.

pinneapple_analysis.inspection.guided_wave_ultrasonic

Ondas guiadas em 1D com a equação dispersiva tipo Klein-Gordon `u_tt = c0^2 u_xx - beta(x) u`, como substituto qualitativo das curvas de Rayleigh-Lamb; o defeito é um aumento local de `beta`. [sem teste]

`pinneapple_analysis/inspection/guided_wave_ultrasonic.py` · 210 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_guided_wave_synthetic`	função	`(*, L=1.0, c0=3000.0, beta0=0.0, n_points=80, duration=0.0006, burst_freq=50000.0, n_cycl…)`	Gera um conjunto pulso-eco de onda guiada: um tone-burst injetado em x=0 é parcialmente refletido por um aumento local de `beta(x)`.
`make_guided_wave_physics_loss`	função	`(x_grid, beta_field, c0) -> PhysicsLossFn`	Cria a perda física da equação dispersiva `u_tt - c0^2 u_xx + beta(x) u = 0`.
`train_guided_wave_ultrasonic`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados de onda guiada se preciso e treina uma `InspectionPINN` com a perda dispersiva.

pinneapple_analysis.inspection.magnetic_flux_leakage

Verdade de referência por diferenças finitas 2D para MFL (vazamento de fluxo magnético) com permeabilidade não linear (saturação tipo Frohlich-Kennelly, iteração de Picard) e treino PIML com um resíduo simplificado. [sem teste]

`pinneapple_analysis/inspection/magnetic_flux_leakage.py` · 308 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`solve_mfl_nonlinear`	função	`(nx, ny, Lx, Ly, phi0, mu_r_max, H_sat, defect_mask, n_picard=6) -> Tuple[np.ndarray, np.…)`	Resolve o potencial escalar magnetostático não linear do MFL por iteração de Picard, com a região de defeito fixa em `mu0`.
`generate_magnetic_flux_leakage_synthetic`	função	`(*, nx=36, ny=14, Lx=0.2, Ly=0.02, phi0=500.0, mu_r_max=500.0, H_sat=2000.0, defect_cente…)`	Resolve a verdade de referência MFL e lê o campo de vazamento `Bz` na superfície como sinal de varredura ao longo de x.
`make_mfl_physics_loss`	função	`(x_grid, y_grid, mu_field) -> PhysicsLossFn`	Cria uma perda física simplificada `mu * laplaciano(phi)`, que descarta o termo cruzado `grad(mu)·grad(phi)` (exata só onde `mu` varia pouco).
`train_magnetic_flux_leakage`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera os dados MFL se preciso e treina uma `InspectionPINN` com a perda magnetostática simplificada.

pinneapple_analysis.inspection.phased_array_ultrasonic

B-scan sintético de um array linear em pulso-eco (wavelets de Ricker) reconstruído por delay-and-sum / Total Focusing Method; a perda física de Helmholtz é declarada como regularizador, não lei exata. [sem teste]

`pinneapple_analysis/inspection/phased_array_ultrasonic.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`generate_phased_array_synthetic`	função	`(*, n_elements=24, array_length=0.02, c=5900.0, burst_freq=5000000.0, defect_x=0.0, defec…)`	Gera um B-scan pulso-eco sintético de um array de elementos e sua reconstrução por delay-and-sum (TFM).
`make_phased_array_physics_loss`	função	`(k0) -> PhysicsLossFn`	Cria a perda de consistência de Helmholtz 2D `laplaciano(p) + k0^2 p = 0` aplicada à intensidade da imagem (um regularizador, não lei física exata).
`train_phased_array_ultrasonic`	função	`(synthetic_data=None, *, hidden=(64, 64, 64), epochs=200, lr=0.001, batch_size=32, device…)`	Gera o B-scan se preciso e treina uma `InspectionPINN` com o regularizador de Helmholtz.

`pinneapple_analysis.inverse_problems`

Reexporta o kit de problemas inversos (modelos de ruído, regularizadores, operadores de observação, EKI, sensibilidade, termos faltantes, `InverseProblemSolver`, e o núcleo de identificabilidade/calibração/studio). O docstring ainda usa o nome antigo `pinneapple_inverse`.

`pinneapple_analysis/inverse_problems/__init__.py` · 261 linhas · testes: 17

`pinneapple_analysis.inverse_problems.calibration`

Calibração de modelo: separa medições de ajuste e de validação, faz mínimos quadrados ponderados, roda a identificabilidade no ponto calibrado (parâmetro não estimável fica sem intervalo) e dá intervalos por Jacobiano e por bootstrap. [sem teste direto]

`pinneapple_analysis/inverse_problems/calibration.py` · 241 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CalibrationResult`	classe	`(parameters, fit, metrics, rows, identifiability, bootstrap, warnings, seconds)`	Resultado da calibração: parâmetros, ajuste, métricas, linhas, identificabilidade, bootstrap, avisos e tempo; `theta` dá os valores e `to_dict` serializa.
`residual_metrics`	função	`(pred, obs) -> dict[str, Any]`	Calcula RMSE, MAE, erro máximo, viés, chi2 e chi2 por ponto da predição contra as medições, no total e por observável.
`calibrate`	função	`(model, obs, parameters=None, *, holdout=None, holdout_fraction=0.0, holdout_by=None, los…)`	Calibra os parâmetros contra as observações com validação separada, perda robusta opcional, bootstrap e identificabilidade.

`pinneapple_analysis.inverse_problems.contracts`

Converte relatórios de identificabilidade, calibrações e comparações de métodos em `StudyResult` de `pinneapple_data.contracts`, e parâmetros em/de `ParameterSpace`, para encadear os apps.

`pinneapple_analysis/inverse_problems/contracts.py` · 246 linhas · testes: 13

nome	tipo	assinatura	o que faz
parameters_from_space	função	(space, nominal=None) -> list[Parameter]	Cria a lista de Parameter a partir de um ParameterSpace; parâmetros categóricos não são estimáveis e levantam erro.
to_parameter_space	função	(params, name, description='', groups=None)	Cria um ParameterSpace: parâmetros livres viram contínuos nos seus limites e os fixos viram constantes.
identifiability_study	função	(report, params, *, app='identifiability', title='', model='', study_id=None)	Converte um relatório de identificabilidade em StudyResult (uma linha por parâmetro, posto e condicionamento como métricas).
calibration_study	função	(result, params, *, app='calibration', title='', model='', model_spec=None, study_id=None)	Converte um resultado de calibração em StudyResult, com um ModelRef do modelo calibrado quando há métricas de validação.
inverse_study	função	(comparison, params, *, app='inverse', title='', study_id=None)	Converte a saída de compare_methods em StudyResult com uma linha por método e parâmetro.

pinneapple_analysis.inverse_problems.counterfactual_attribution

Atribuição exata por valores de Shapley de uma variação medida de um escalar entre poucas causas candidatas, via rodadas contrafactuais de qualquer modelo; inclui um classificador de ambiguidade quando duas causas não se distinguem.

pinneapple_analysis/inverse_problems/counterfactual_attribution.py · 152 linhas · testes: 7

nome	tipo	assinatura	o que faz
CandidateCause	classe	(name, apply_baseline, apply_current, signature_measurements)	Causa candidata: nome, funções que a colocam no estado de base ou atual, e medições que a assinam.
ShapleyAttributionResult	classe	(measured_delta, contributions, residual, baseline_value, current_value)	Resultado da atribuição: variação medida, contribuição por causa, resíduo de fechamento (deve ser ~0) e valores de base e atual.
ExactShapleyAttributor	classe	(value_fn, baseline_state)	Avalia uma função de valor em todas as combinações de causas e devolve a decomposição exata de Shapley da variação.
AmbiguityResult	classe	(indeterminate, reason, recommended_measurement)	Diz se a atribuição é indeterminada, por quê, e qual medição é recomendada para desempatar.
classify_ambiguity	função	(contributions, contribution_uncertainty, signature_measurements, available_measurements)…)	Decide se as contribuições são indistinguíveis dadas as incertezas e as medições disponíveis.

pinneapple_analysis.inverse_problems.ensemble_kalman

Ensemble Kalman Inversion (EKI) sem derivadas: atualiza um ensemble de parâmetros só com avaliações do modelo direto; inclui a versão regularizada por Tikhonov (TEKI).

pinneapple_analysis/inverse_problems/ensemble_kalman.py · 454 linhas · testes: 151

nome	tipo	assinatura	o que faz
`EKIConfig`	classe	`(n_ensemble, n_iterations, noise_cov, noise_std, init_spread, lambda_reg, prior_mean, pri…)`	Configuração da EKI: tamanho do ensemble, iterações, ruído, espalhamento inicial, regularização, prior, tolerância de colapso e semente.
`EKIHistory`	classe	`(iterations, data_misfit, ensemble_spread, theta_mean_history)`	Histórico de convergência da EKI (desajuste, espalhamento, média de theta por iteração).
`EnsembleKalmanInversion`	classe	`(forward_fn, config=None) -> None`	EKI padrão (Iglesias et al. 2013): inicializa o ensemble, aplica passos tipo Kalman e para por colapso do ensemble.
`IteratedEKI`	classe	`()`	EKI regularizada por Tikhonov (TEKI, Chada et al. 2020).

`pinneapple_analysis.inverse_problems.estimation`

Estimadores comuns a identificabilidade, calibração e studio: mínimos quadrados não lineares em parâmetros escalados, L-BFGS-B com gradiente exato (adjunto ou autograd) e Metropolis adaptativo. [sem teste direto]

`pinneapple_analysis/inverse_problems/estimation.py` · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Prior`	classe	`(mean, std, L, weight)`	Prior Gaussiano nos parâmetros escalados mais penalidade opcional de suavidade `weight * |L z|^2`.
`FitResult`	classe	`(z, theta, pred, chi2, prior_penalty, jac, nfev, success, message, seconds, derivatives, …)`	Resultado de um ajuste: z, theta, predições, chi2, penalidade do prior, Jacobiano, avaliações e diagnóstico; `dof` dá os graus de liberdade.
`PosteriorSamples`	classe	`(names, samples_z, samples, acceptance, n_evaluations, seconds)`	Amostras da posterior (em z e unidades físicas) com taxa de aceitação; `summary` dá média, mediana, desvio, intervalos e correlações.
`least_squares_fit`	função	`(model, params, obs, *, z0=None, free=None, prior=None, loss='linear', max_nfev=200, deri…)`	Minimiza chi2 (mais prior) nos parâmetros escalados por região de confiança, com subconjunto livre e perdas robustas opcionais.
`gradient_fit`	função	`(model, params, obs, *, z0=None, prior=None, gradient='adjoint', max_iter=300) -> FitResu…)`	L-BFGS-B no desajuste com gradiente exato pelo adjunto do modelo ou por autograd.
`metropolis`	função	`(model, params, obs, *, z0, cov0=None, n_steps=4000, burn=None, prior=None, seed=0, thin=…)`	Metropolis de passeio aleatório adaptativo (Haario et al. 2001) em z, com verossimilhança Gaussiana e limite de tempo.

`pinneapple_analysis.inverse_problems.forward`

Linguagem comum dos problemas inversos: `Parameter`, `ParameterSet`, `Observations`, a base `ForwardModel` (cujo `predict` levanta NotImplementedError até ser sobrescrito) e modelos por função, expressão de texto ou tabela; mais o Jacobiano em coordenadas escaladas. [sem teste direto]

`pinneapple_analysis/inverse_problems/forward.py` · 484 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Parameter`	classe	`(name, value, lower, upper, unit, log, fixed, description, truth)`	Um parâmetro ajustável com valor nominal, limites, unidade, escala log opcional, flag de fixo e valor verdadeiro opcional.
`ParameterSet`	classe	`(parameters)`	Conjunto de parâmetros livres e o mapa entre theta (dict em unidades físicas) e z (vetor escalado).
`Observations`	classe	`(inputs, sigma, values, observable, meta)`	Medições: entradas por coluna, observável de cada linha, valores (opcionais antes do experimento) e sigma do ruído; permite subconjunto, divisão e concatenação.
`ForwardModel`	classe	`(name, inputs, outputs, cost, description)`	Classe base de modelo direto: define entradas, saídas e parâmetros padrão; `predict` levanta NotImplementedError se não for sobrescrito.
`CallableModel`	classe	`(fn, *, inputs, outputs=('y',), parameters=(), name='callable', cost='cheap', description…)`	Embrulha qualquer função Python ou solver externo `fn(theta, inputs)` como modelo direto.
`ExpressionModel`	classe	`(expressions, *, inputs, parameters, name='expression', description='')`	Modelo escrito como texto, uma expressão por observável.
`TabulatedModel`	classe	`(table, *, parameters, inputs, output, name='simulated results', smoothing=0.0, observabl…)`	Usa uma tabela de resultados simulados como modelo, interpolando sobre parâmetros e entradas.
`count_evaluations`	função	`(model) -> int`	Devolve quantas vezes o modelo foi avaliado (contador interno `_n_eval`).
`jacobian`	função	`(model, params, z, obs, *, method='auto', rel_step=0.0001) -> tuple[np.ndarray, np.ndarra…)`	Calcula predições e Jacobiano em relação a z, pelo método do modelo, autograd ou diferenças finitas.

`pinneapple_analysis.inverse_problems.heat_diffusivity`

Recupera a difusividade `alpha` da equação do calor 1D a partir de leituras ruidosas num instante, com uma PINN que aprende campo e `alpha` juntos, e relata a informação de Fisher/limite de Cramér-Rao.

`pinneapple_analysis/inverse_problems/heat_diffusivity.py` · 162 linhas · testes: 4

nome	tipo	assinatura	o que faz
`u_exact`	função	`(t, x, alpha) -> np.ndarray`	Solução exata `exp(-pi^2 alpha t) sin(pi x)` da equação do calor do caso de teste.
`identifiability_report`	função	`(t_obs, x_obs, alpha, noise_std) -> dict[str, Any]`	Informação de Fisher e limite de Cramér-Rao para `alpha` a partir das leituras com ruído Gaussiano.
`run_inverse_discovery`	função	`(true_alpha=TRUE_ALPHA, initial_guess=INITIAL_GUESS, noise_std=NOISE_STD, n_observations=…)`	Gera as leituras, treina rede e `alpha` juntos e devolve um resultado serializável com histórico, `alpha` recuperado e identificabilidade.

pinneapple_analysis.inverse_problems.heat_transfer

Transferência de calor inversa: estima o coeficiente de convecção h de uma aleta (1D, também com leituras do usuário) e de uma placa 2D por PINN; o caso 3D não é treinado aqui, é servido a partir de resultados gravados em examples/use_cases/fin_convection_inverse/results/3d.

pinneapple_analysis/inverse_problems/heat_transfer.py · 325 linhas · testes: 15

nome	tipo	assinatura	o que faz
InputError	classe	()	Erro de entrada inválida (subclasse de ValueError) dos casos de transferência de calor.
fin_exact	função	(xi, h, k, d, length) -> np.ndarray	Temperatura adimensional de uma aleta pino com ponta convectiva (Incropera, Tabela 3.4, caso A).
fin_q_exact	função	(h, k, d, length, dt) -> float	Calor que sai da base da aleta pela solução analítica.
run_fin	função	(*, k, d_mm, length_mm, t_base, t_air, sensors_mm, readings, h_guess=100.0, steps=2000, s…)	Aprende o perfil de temperatura e h de uma aleta a partir de leituras de termopares.
fin_synthetic_readings	função	(*, k, d_mm, length_mm, t_base, t_air, sensors_mm, h_true, noise, seed=0) -> list[float]	Gera leituras sintéticas de termopares da aleta analítica mais ruído Gaussiano.
run_plate	função	(*, power, h_true, noise, h_guess=60.0, steps=1500, seed=0, progress=None, budget=None, w…)	Gera leituras sintéticas da placa pelo modelo de volumes finitos e treina a PINN para T(x, y) e h, comparando com o modelo.
plate_precomputed	função	() -> dict	Carrega o resultado gravado da placa 2D (campo da PINN e de referência) da pasta de resultados do exemplo.
block_precomputed	função	() -> dict	Carrega o resultado gravado do bloco 3D (campo da PINN, referência e sorteios de h) da pasta de resultados do exemplo.

pinneapple_analysis.inverse_problems.identifiability

Responde se os parâmetros podem ser recuperados das medições antes de estimar: informação de Fisher, sensibilidade local, verossimilhança de perfil e posterior, mais projeto D-ótimo de novas medições. (A flag 'experimental' vem de 'experimental design', não de status.) [sem teste direto]

pinneapple_analysis/inverse_problems/identifiability.py · 457 linhas · **sem teste direto** · sinais no código: experimental

nome	tipo	assinatura	o que faz
IdentifiabilityReport	classe	(parameters, analysis_point, data, n_observations, rank, n_parameters, condition_number, …)	Relatório de identificabilidade: posto, condicionamento, autovalores, correlações, ambiguidades e status por parâmetro.
d_optimal_design	função	(F, Jc_w, n_pick=3, distinct=True) -> list[dict[str, Any]]	Seleção gulosa D-ótima de medições candidatas a partir da informação de Fisher, com o ganho de log det de cada escolha.

nome	tipo	assinatura	o que faz
`profile_likelihood`	função	`(model, params, obs, z_best, chi2_min, std_z, *, n_points=15, max_nfev=60, time_budget=60…)`	Perfil de verossimilhança de cada parâmetro livre, com intervalo de 95% e veredito.
`analyze_identifiability`	função	`(model, obs, parameters=None, *, candidates=None, n_suggest=3, profile='auto', n_profile=…)`	Roda os métodos de identificabilidade sobre modelo e medições e sugere novas medições.

`pinneapple_analysis.inverse_problems.missing_term`

Identificação de termos faltantes numa EDP parcialmente conhecida: biblioteca de candidatos, regressão esparsa SINDy (STRidge/LASSO), análise do resíduo de uma PINN treinada e um termo neural treinado em conjunto.

`pinneapple_analysis/inverse_problems/missing_term.py` · 500 linhas · testes: 151

nome	tipo	assinatura	o que faz
`CandidateLibrary`	classe	`(poly_order=2, include_trig=False, custom_terms=None) -> None`	Monta a matriz Θ de termos candidatos (monômios, trigonométricos, personalizados) a partir de dados de colocação.
`SINDyResult`	classe	`(coefficients, active_terms, active_coefficients, residual_norm, n_terms_selected)`	Resultado da regressão SINDy: coeficientes, termos ativos e norma do resíduo; `equation` escreve a forma simbólica.
`SINDyIdentifier`	classe	`(threshold=0.001, max_iter=20, alpha=1e-05, method='stridge') -> None`	Identifica termos faltantes por regressão esparsa (Brunton et al. 2016).
`ResidualAnalysisResult`	classe	`(residual_field, sindy_result, coords, mean_abs_residual, max_abs_residual)`	Saída de `ResidualAnalyzer.analyze`: campo de resíduo, resultado SINDy opcional e estatísticas.
`ResidualAnalyzer`	classe	`(pde_residual_fn, library=None, identifier=None) -> None`	Avalia o resíduo da EDP conhecida numa PINN treinada e opcionalmente roda SINDy sobre ele.
`NeuralTermConfig`	classe	`(hidden_dims, activation, term_weight, data_weight, n_iters, lr, device)`	Configuração do treino conjunto PINN + termo neural.
`NeuralTermDiscovery`	classe	`(known_residual_fn, config=None) -> None`	Descobre um termo faltante parametrizado por uma rede neural e permite destilá-lo.

`pinneapple_analysis.inverse_problems.models`

Modelos diretos físicos para identificabilidade, calibração e inversão: aleta analítica, aleta em volumes finitos com zonas de `h` (com adjunto), placa dissipadora, resfriamento concentrado e cinética de Arrhenius. [sem teste direto]

`pinneapple_analysis/inverse_problems/models.py` · 476 linhas · **sem teste direto** · sinais no código: experimental

nome	tipo	assinatura	o que faz
`PinFinModel`	classe	`(d_mm=5.0, length_mm=50.0)`	Aleta pino analítica com ponta convectiva (caso do Inverse Heat Lab), com autograd e caminho PINN para `h`.

nome	tipo	assinatura	o que faz
`RodModel`	classe	`(d_mm=5.0, length_mm=100.0, n_zones=1, n_cells=200)`	Aleta em volumes finitos com `h` por zonas (um campo), com sensibilidades exatas, gradiente adjunto e gêmeo em torch.
`PlateModel`	classe	`(nx=60, ny=36)`	Placa dissipadora do Inverse Heat Lab resolvida por volumes finitos, com cache por `h`.
`LumpedCoolingModel`	classe	`(area_m2=0.01)`	Curva de resfriamento concentrado; `h` e C só aparecem como `h A / C`, logo os dois livres são estruturalmente ambíguos.
`ArrheniusModel`	classe	`()`	Conversão de primeira ordem de Arrhenius; A e Ea são ambíguos com uma só temperatura.

`pinneapple_analysis.inverse_problems.noise_models`

Modelos de ruído / termos de desajuste de dados em PyTorch: Gaussiano, Huber, Cauchy, Student-t e heteroscedástico.

`pinneapple_analysis/inverse_problems/noise_models.py` · 260 linhas · testes: 151

nome	tipo	assinatura	o que faz
`DataMisfitBase`	classe	`()`	Base abstrata de termo de desajuste / log-verossimilhança negativa.
`GaussianMisfit`	classe	`(noise_cov=None, noise_std=1.0, reduction='mean') -> None`	Desajuste Gaussiano (L2/Mahalanobis).
`HuberMisfit`	classe	`(delta=1.0, noise_std=1.0, smooth=True) -> None`	Desajuste de Huber (ou pseudo-Huber), robusto a outliers moderados.
`CauchyMisfit`	classe	`(gamma=1.0, noise_std=1.0) -> None`	Desajuste de Cauchy/Lorentz, de cauda pesada e muito robusto.
`StudentTMisfit`	classe	`(nu=5.0, noise_std=1.0) -> None`	Log-verossimilhança negativa de Student-t; `nu` controla a robustez.
`HeteroscedasticMisfit`	classe	`(log_noise_var) -> None`	Desajuste Gaussiano com variância de ruído por observação.

`pinneapple_analysis.inverse_problems.obs_operator`

Operadores de observação H (estado -> observáveis): pontual, linear, integral e composto. O docstring cita um `BoundaryObsOperator` que não existe no arquivo.

`pinneapple_analysis/inverse_problems/obs_operator.py` · 284 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ObsOperatorBase`	classe	`()`	Base abstrata de operador de observação; oferece `jacobian_theta`.
`PointObsOperator`	classe	`(sensor_locations, output_indices=None) -> None`	Avalia o modelo num conjunto fixo de posições de sensores.
`LinearObsOperator`	classe	`(A, x_eval) -> None`	Operador linear `H(u) = A u` (ex.: tomografia).
`IntegralObsOperator`	classe	`(x_quad, weights, kernel_fn=None) -> None`	Funcional integral aproximado por quadratura ponderada com núcleo opcional.
`ComposedObsOperator`	classe	`(operators, x_obs_list=None) -> None`	Empilha vários operadores de observação num vetor único.

pinneapple_analysis.inverse_problems.presets

Problemas inversos prontos com resposta conhecida (medições sintéticas com semente fixa) e construção de modelos a partir de JSON, para apps e agentes. [sem teste direto]

`pinneapple_analysis/inverse_problems/presets.py` · 326 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Preset`	classe	`(id, title, summary, model_spec, parameters, observations, validation, candidates, method…)`	Problema pronto: modelo em JSON, parâmetros, observações, validação, métodos e lição; `build` monta os objetos.
`model_from_spec`	função	`(spec) -> ForwardModel`	Constrói um `ForwardModel` a partir de um dict JSON (aleta, expressão, tabela etc.).
`synthetic`	função	`(model, params, obs, seed, rel_sigma=False) -> Observations`	Gera observações sintéticas: modelo na verdade mais ruído N(0, sigma) com semente, arredondado à resolução do instrumento.
`get_preset`	função	`(name) -> Preset`	Devolve um preset pelo nome.
`list_presets`	função	`(tag=None) -> list[dict[str, Any]]`	Lista os presets, opcionalmente filtrados por tag.

pinneapple_analysis.inverse_problems.regularization

Regularizadores diferenciáveis em PyTorch (Tikhonov, L1, variação total, composto) e escolha automática de lambda pela curva L.

`pinneapple_analysis/inverse_problems/regularization.py` · 329 linhas · testes: 151

nome	tipo	assinatura	o que faz
`RegularizerBase`	classe	`()`	Base abstrata de regularizador R(θ) com peso.
`TikhonovRegularizer`	classe	`(lambda_reg, prior=None, covariance=None) -> None`	Regularizador de Tikhonov/ridge, com prior e covariância opcionais.
`SparsityRegularizer`	classe	`(lambda_reg, eps=1e-06) -> None`	Regularizador L1/LASSO com aproximação suave.
`TotalVariationRegularizer`	classe	`(lambda_reg, eps=1e-06) -> None`	Variação total anisotrópica para parâmetros de campo 1D/2D.
`CompositeRegularizer`	classe	`(terms=None) -> None`	Soma ponderada de vários regularizadores.
`LCurveResult`	classe	`(lambdas, data_misfits, regularization_norms, optimal_lambda, curvatures)`	Resultado da curva L: lambdas, desajustes, normas, curvaturas e lambda ótimo.
`LCurveSelector`	classe	`(train_fn, lambdas=None) -> None`	Escolhe lambda pela máxima curvatura da curva L, chamando uma função de treino para cada valor.

pinneapple_analysis.inverse_problems.sensitivity

Análise de sensibilidade: Jacobiano local e matriz de Fisher, análise de identificabilidade por autovalores com critérios D/A/E e índices de Sobol globais pelo método de Saltelli.

pinneapple_analysis/inverse_problems/sensitivity.py · 439 linhas · testes: 151 · sinais no código: experimental

nome	tipo	assinatura	o que faz
LocalSensitivityResult	classe	(jacobian, fisher_information, singular_values, sensitivity_indices, param_names)	Resultado da sensibilidade local: Jacobiano, Fisher, valores singulares e índices.
LocalSensitivity	classe	(forward_fn, noise_std=1.0, param_names=None) -> None	Sensibilidade linearizada em torno de um ponto nominal, com Jacobiano por diferenças finitas opcional.
IdentifiabilityResult	classe	(eigenvalues, eigenvectors, condition_number, d_optimality, pseudo_d_optimality, a_optima…)	Resultado da análise de identificabilidade por Fisher: autovalores, condicionamento, critérios D/A/E e índices identificáveis.
IdentifiabilityAnalyzer	classe	(tol=1e-06) -> None	Analisa a identificabilidade a partir dos autovalores da matriz de Fisher.
SobolResult	classe	(S1, ST, param_names, n_samples)	Índices de Sobol de primeira ordem e totais.
GlobalSensitivity	classe	(forward_fn, param_bounds, n_samples=1024, seed=None) -> None	Sensibilidade global por variância (índices de Sobol pelo método de Saltelli).

pinneapple_analysis.inverse_problems.solver

`InverseProblemSolver`: liga modelo (PINN ou `nn.Module`), operador de observação, desajuste e regularização, otimizando por Adam, L-BFGS ou EKI.

pinneapple_analysis/inverse_problems/solver.py · 426 linhas · testes: 151

nome	tipo	assinatura	o que faz
InverseSolverConfig	classe	(method, n_iters, lr, lbfgs_max_iter, device, grad_clip, print_every, eki_n_ensemble, eki…)	Configuração do `InverseProblemSolver`: método, iterações, taxa de aprendizado e opções de EKI.
InverseSolverResult	classe	(theta_opt, final_misfit, final_regularization, final_total_loss, loss_history, n_iters, …)	Resultado: theta ótimo, desajuste, regularização, perda total, histórico, método e convergência.
InverseProblemSolver	classe	(model, obs_operator, data_misfit=None, regularizer=None, config=None, extra_loss_fn=None…)	Resolvedor unificado de problemas inversos com informação física.

pinneapple_analysis.inverse_problems.studio

Interface única para formular um problema inverso, resolvê-lo com vários métodos (`least_squares`, `adjoint`, `autodiff`, `bayesian`, `pinn`) e ranqueá-los pelo erro nas medições de validação. [sem teste direto]

pinneapple_analysis/inverse_problems/studio.py · 317 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
InverseProblem	classe	(model, observations, parameters, validation, name, description, prior_std, smoothness, f…)	Problema inverso: modelo, observações, parâmetros, validação, prior e zonas de campo.

nome	tipo	assinatura	o que faz
available_methods	função	(problem) -> dict[str, dict[str, Any]]	Diz quais métodos se aplicam ao problema e por que não, quando não se aplicam.
solve_inverse	função	(problem, method, *, identifiability=None, pinn_steps=1500, mcmc_steps=3000, seed=0, prog…)	Resolve o problema com um método e devolve um dict serializável com estimativas, incerteza, resíduos e métricas.
compare_methods	função	(problem, methods=…, *, identifiability=True, pinn_steps=1500, seed=0, progress=None) -> …)	Roda vários métodos sobre os mesmos dados e os ranqueia pelo RMSE de validação.

pinneapple_analysis.proof

Proof graphs: mathematical arguments as graphs of assumptions, lemmas and theorems with machine checks (symbolic, exhaustive, numerical, counterexample search) and statuses that propagate along the dependencies.

pinneapple_analysis/proof/__init__.py · 29 linhas · testes: 11

pinneapple_analysis.proof.graph

Proof graphs: a mathematical argument as a graph of assumptions, definitions, lemmas, theorems and conjectures, each with machine checks, and a status that only rises as far as its evidence and its dependencies allow.

pinneapple_analysis/proof/graph.py · 334 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CheckOutcome	classe	(name, passed, exact, detail, counterexample, seconds)	
Check	classe	(name, fn)	
Node	classe	(id, kind, statement, depends_on, checks, argument, covered, supported_by, status, outcom…)	
ProofGraph	classe	(title='')	
symbolic_identity	função	(name, lhs, rhs=0, simplify=None) -> Check	
symbolic_true	função	(name, statement) -> Check	
exhaustive	função	(name, predicate, domain, describe='') -> Check	
numeric	função	(name, predicate, sampler, n=2000, seed=0) -> Check	
counterexample_search	função	(name, violation, bounds, tol=1e-08, seed=0, maxiter=300, restarts=3) -> Check	violation(x) < -tol means x is a counterexample. Differential evolution minimises it from restarts seeds; the smallest value found is reported.

nome	tipo	assinatura	o que faz
`custom`	função	`(name, fn) -> Check`	

pinneapple_analysis.proof.pt_chains

Does the no-go theorem for linear exceptional-point tuning extend from PT dimers to longer chains?

`pinneapple_analysis/proof/pt_chains.py` · 262 linhas · testes: 4

nome	tipo	assinatura	o que faz
`chain_hamiltonian`	função	`(gamma, bonds) -> np.ndarray`	N-site chain, +i gamma on the first site and -i gamma on the last, H[k, k+1] = b_k, H[k+1, k] = conj(b_k).
`threshold_numeric`	função	`(bonds, gmax=None, tol=1e-11) -> float`	Smallest gamma at which the spectrum stops being real, by bisection on the matrix alone. The real-spectrum set in gamma need not be an interval; a scan on a fine grid finds the first breaking before bisecting.
`tetramer_threshold_branches`	função	`(A, B) -> list[float]`	Candidate gamma^2 values: branch I (A^2 / B) and the real non-negative roots of branch II gamma^2 = 2A - B +/- 2 sqrt(A (A - 2B)).
`build_pt_chain_graph`	função	`(max_deg=4, n_numeric=300, search_iters=120, searches=True) -> ProofGraph`	The graph above. `searches=False` leaves out the (slow) counterexample searches of L6 and C1.

pinneapple_analysis.reproduction

Neutral tools for reproducing published numbers: one evaluator for the error metrics papers report, seed statistics, and the status rule fixed before any run (examples/paper_reproductions).

`pinneapple_analysis/reproduction.py` · 139 linhas · testes: 5

nome	tipo	assinatura	o que faz
`SeedStats`	classe	`(values, median, min, max, n_failed)`	Every seed's value with the median and the min-max; no best-seed selection anywhere.
`rel_l2`	função	`(pred, ref) -> float`	`||pred - ref||_2 / ||ref||_2` (also PINNsFormer's rRMSE). `inf` when `pred` is not finite.
`rmae`	função	`(pred, ref) -> float`	`sum |pred - ref| / sum |ref|` (PINNsFormer's rMAE).
`max_abs`	função	`(pred, ref) -> float`	
`mean_abs`	função	`(pred, ref) -> float`	
`mse`	função	`(pred, ref) -> float`	
`rmse`	função	`(pred, ref) -> float`	

nome	tipo	assinatura	o que faz
evaluate	função	(pred, ref, metrics=('rel_l2', 'rmae', 'max_abs')) -> dict[str, float]	The named metrics of pred against ref (keys of :data:METRICS).
seed_stats	função	(values) -> SeedStats	Median and min-max over seeds. A diverged seed (inf / nan) counts as inf in the median, so a failure raises the median instead of being dropped.
status	função	(ours_median, paper, *, spread=None, ordering_holds=None, reduced_scale=False, factor=2.0…)	Status of one target by the protocol's rule (see the module docstring).

pinneapple_analysis.retrieval

pinneapple_analysis.retrieval -- real, local semantic search over a real corpus (verification-module docstrings + real preset knowledge- graph summaries + optionally real external literature fetched live from arXiv; see corpus.py's own docstring for the full source list and honesty scope). No ...

pinneapple_analysis/retrieval/__init__.py · 58 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
build_default_index	função	(*, model_name=DEFAULT_MODEL_NAME, include_literature=False, literature_topics=None, lite…)	Build a real index over the real default corpus in one call -- convenience wrapper over build_corpus() + RetrievalIndex.build(). See build_corpus's own docstring for what include_literature adds and why it's opt-in.

pinneapple_analysis.retrieval.corpus

Real, non-fabricated corpus sources for semantic retrieval.

pinneapple_analysis/retrieval/corpus.py · 360 linhas · testes: 32

nome	tipo	assinatura	o que faz
Document	classe	(doc_id, source, title, text)	
fetch_literature_documents	função	(query, *, k=5, timeout_s=30) -> List[Document]	Fetch real papers from the live arXiv API for one query -- a thin wrapper over ArxivSource.search (the same real search pinneapple_llm.research.search_literature uses), turning each real ArxivPaper into a :class:Document. Never fabricates a result: a network failure raises (the caller ...
build_literature_corpus	função	(topics=None, *, k_per_topic=5, timeout_s=30) -> List[Document]	Fetch real papers for several topics from the live arXiv API, de-duplicated by arXiv id. Skips (logs a warning, does not raise) a topic whose fetch fails -- e.g. no network for that one call -- so one bad query doesn't abort the whole corpus build; if EVERY topic fails this returns an empty list, ...

nome	tipo	assinatura	o que faz
`build_corpus`	função	`(*, include_literature=False, literature_topics=None, literature_k_per_topic=5) -> List[D…)`	Assemble the real, current corpus -- recomputed fresh every call (never a stale cached snapshot), so it always reflects whatever modules/presets are actually registered right now.
`iter_repo_files`	função	`(repo_root, *, extensions=DEFAULT_CODE_EXTENSIONS, exclude_dirs=DEFAULT_EXCLUDE_DIRS, max…)`	Real `os.walk` over *repo_root*, returning every real file path whose extension is in *extensions*, skipping *exclude_dirs* (by directory *name*, matched at any depth) and any file over *max_file_bytes* (a large data/generated file is skipped, not truncated silently into a misleading partial …
`chunk_text`	função	`(text, *, max_chars=4000, overlap=200) -> List[str]`	Simple, real character-window chunker -- splits on a blank-line boundary near each window edge when one exists (so a chunk doesn't cut a function/class in half more often than necessary), falling back to a hard cut otherwise. No chunk is ever padded/rewritten -- each is a verbatim slice of *text*. …
`file_fingerprint`	função	`(path) -> str`	Real content hash (sha256, hex) of a file's current bytes on disk -- the change-detection primitive the org-management code-index refresh job uses to decide whether a file needs re-embedding. Distinct from mtime: a hash catches a change even if mtime didn't update (e.g. a git checkout that …
`code_repo_documents`	função	`(repo_root, *, repo_name=None, extensions=DEFAULT_CODE_EXTENSIONS, exclude_dirs=DEFAULT_E…)`	Real source-code Documents for one local repository clone.

pinneapple_analysis.retrieval.index

A real, local, embedding-based semantic index over a :class:`~pinneapple_analysis.retrieval.corpus.Document` corpus.

`pinneapple_analysis/retrieval/index.py` · 133 linhas · testes: 17

nome	tipo	assinatura	o que faz
`SearchResult`	classe	`(document, score)`	
`RetrievalIndex`	classe	`(documents, embeddings, model_name) -> None`	A built, queryable index over a real document list. Never constructed with a partial/fabricated embedding -- `build()` embeds every document for real before the index is usable.

pinneapple_analysis.sensitivity

Global and local sensitivity analysis: which parameters really control the result.

`pinneapple_analysis/sensitivity/__init__.py` · 42 linhas · testes: 24

pinneapple_analysis.sensitivity.effects

Effect curves, local derivatives and the "which parameters can be frozen" decision.

`pinneapple_analysis/sensitivity/effects.py` · 157 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`effect_curves`	função	`(fn, space, n_grid=21, n_background=128, n_ice=20, seed=0, outputs=None) -> dict`	Partial dependence and ICE curves of every input. Cost `d * n_grid * n_background` evaluations.
`local_derivatives`	função	`(fn, x0, space=None, torch_fn=None, rel_step=0.0001, outputs=None) -> dict`	Jacobian `dy/dx` (m, d) at `x0`.
`freeze_recommendation`	função	`(indices, threshold=0.01, morris_result=None) -> list[dict]`	Per input: `freeze` when the upper bound of the total-index interval is below `threshold` for every output; `inconclusive` when the estimate is below but the interval is not (more samples needed); `keep` otherwise. Evidence lists the total index and interval per output (and Morris …
`verify_freeze`	função	`(fn, space, frozen, n=1024, seed=0, at=None) -> dict`	Run the model on `n` samples with all inputs varying, and on the same samples with `frozen` inputs held at their nominal value (the median, or `at[name]`). Returns per output the variance ratio (frozen / full) and the largest and 95th-percentile pointwise change relative to the output's …

`pinneapple_analysis.sensitivity.morris`

Morris elementary effects: a cheap screening of which inputs matter, `r (d + 1)` evaluations.

`pinneapple_analysis/sensitivity/morris.py` · 111 linhas · testes: 13

nome	tipo	assinatura	o que faz
`MorrisResult`	classe	`(names, outputs, r, levels, delta, n_evals, mu, mu_star, sigma, mu_star_ci, level, effect…)`	
`morris_design`	função	`(d, r=20, levels=4, seed=0) -> tuple[np.ndarray, np.ndarray, np.ndarray]`	Unit-hypercube trajectories `(r (d + 1), d)`, the input moved at each step `(r, d)` and its sign.
`morris`	função	`(fn, space, r=20, levels=4, seed=0, n_boot=500, level=0.95, outputs=None) -> MorrisResult`	Morris screening of `fn` over `space` with `r` trajectories (cost `r (d + 1)` evaluations).

`pinneapple_analysis.sensitivity.sobol`

Variance-based (Sobol) sensitivity indices with bootstrap confidence intervals.

`pinneapple_analysis/sensitivity/sobol.py` · 255 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SobolIndices`	classe	`(names, outputs, n_base, n_evals, variance, S1, ST, S1_ci, ST_ci, S2, S2_ci, level, n_boo…)`	Sobol indices per output (rows) and input (columns), with `level` bootstrap intervals `*_ci` of shape `(2, m, d)` (lower, upper). `S2` is `(m, d, d)` when second order was requested.

nome	tipo	assinatura	o que faz
`saltelli_design`	função	`(space, n, second_order=False, seed=0) -> tuple[np.ndarray, dict]`	Inputs `X` to evaluate, stacked as `[A; B; AB_1..AB_d; (BA_1..BA_d)]`, and the layout to read them back. `n` is rounded up to a power of two (the balance property of the Sobol sequence).
`sobol_from_evaluations`	função	`(Y, layout, outputs=None, n_boot=200, level=0.95, seed=0, X=None) -> SobolIndices`	Indices from model outputs `Y` on the inputs of :func:`saltelli_design` (e.g. runs done elsewhere: a solver campaign, a cluster job).
`sobol_indices`	função	`(fn, space, n=1024, second_order=False, n_boot=200, level=0.95, seed=0, outputs=None) -> …)`	First-order and total Sobol indices (and optionally second order) of `fn` over `space`.
`first_order_given_data`	função	`(X, y, names=None, n_bins=None, n_boot=200, level=0.95, seed=0) -> dict`	First-order indices from an existing sample `(X, y)` with independent inputs: `Var(E[y | x_i]) / Var(y)` estimated on `n_bins` equal-count bins of each input (default `sqrt(n)`, at most 50), with the bias of the bin means (within-bin variance / bin size) subtracted. Interactions and totals …
`sobol_from_runs`	função	`(space, n, run_ids, Y, second_order=False, seed=0, n_boot=200, level=0.95, outputs=None) …)`	Indices from runs of the design `saltelli_design(space, n, second_order, seed)` done elsewhere (OpenFOAM, CalculiX, a campaign runner): `run_ids` are the row numbers of the design (any order), `Y` their outputs. Every run of the design is needed; a missing or failed run raises `ValueError` …

pinneapple_analysis.sensitivity.study

One sensitivity study: which parameters really control each response, and which can be frozen.

`pinneapple_analysis/sensitivity/study.py` · 136 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`study_cost`	função	`(d, n_sobol=1024, second_order=False, r_morris=20, n_grid=15, n_background=64, n_verify=5…)`	Model evaluations of each part of :func:`sensitivity_study` (`n_sobol` rounded up to a power of two).
`sensitivity_study`	função	`(model, space, outputs=None, *, n_sobol=1024, second_order=False, r_morris=20, n_grid=15,…)`	Run the study.

pinneapple_analysis.sensitivity.testfunctions

Benchmark functions with closed-form Sobol indices, to check a sensitivity estimator against the exact answer.

`pinneapple_analysis/sensitivity/testfunctions.py` · 53 linhas · testes: 28

nome	tipo	assinatura	o que faz
`ishigami`	função	`(X, a=7.0, b=0.1) -> np.ndarray`	

nome	tipo	assinatura	o que faz
`ishigami_space`	função	`() -> list[dict]`	
`ishigami_indices`	função	`(a=7.0, b=0.1) -> dict`	Exact variance, first-order, total and second-order (x1, x3) indices of the Ishigami function.
`sobol_g`	função	`(X, a) -> np.ndarray`	
`sobol_g_space`	função	`(d) -> list[dict]`	
`sobol_g_indices`	função	`(a) -> dict`	Exact variance, first-order and total indices of the G function with coefficients `a`.

pinneapple_analysis.state_estimation

pinneapple_analysis.state_estimation -- Sequential state estimation (data assimilation) building blocks.

`pinneapple_analysis/state_estimation/__init__.py` · 49 linhas · testes: 22 · sinais no código: shim ou obsoleto

pinneapple_analysis.state_estimation.kalman

Extended Kalman Filter (EKF) and Ensemble Kalman Filter (EnKF) for sequential state estimation / data assimilation.

`pinneapple_analysis/state_estimation/kalman.py` · 346 linhas · testes: 6 · sinais no código: shim ou obsoleto

nome	tipo	assinatura	o que faz
`ExtendedKalmanFilter`	classe	`(n_state, n_obs, f, h, Q=None, R=None, F_jac=None, H_jac=None, *, eps_fd=1e-05) -> None`	Extended Kalman Filter for state estimation.
`EnsembleKalmanFilter`	classe	`(n_state, n_obs, f, h, Q=None, R=None, n_ens=100, inflation=1.02, seed=0) -> None`	Stochastic Ensemble Kalman Filter.
`gaspari_cohn`	função	`(d) -> np.ndarray`	Gaspari & Cohn (1999, QJRMS 125:723) fifth-order compactly supported correlation of the normalised distance `d` = distance / half-width (zero for d >= 2).
`enkf_analysis`	função	`(ens, y, hx, r_var, *, loc_xy=None, loc_yy=None, inflation=1.0, rng=None) -> np.ndarray`	Stochastic EnKF analysis (perturbed observations; Burgers et al. 1998, Mon. Wea. Rev. 126:1719) of an ensemble `ens` (N, n_state) given observations `y` (m,), their forecast equivalents `hx` (N, m) and independent observation-error variances `r_var` (m,).

pinneapple_analysis.study_contracts

The shared data contracts (:mod:`pinneapple_data.contracts`) for uncertainty and sensitivity studies.

`pinneapple_analysis/study_contracts.py` · 237 linhas · testes: 4

nome	tipo	assinatura	o que faz
`is_parameter_space`	função	`(obj) -> bool`	
`space_from_contract`	função	`(space) -> tuple[list[dict], dict[str, Any]]`	`(parameters, fixed)` for a study from a `ParameterSpace` (object or dict).
`space_to_contract`	função	`(params, name='study', fixed=None, units=None)`	A `ParameterSpace` with each parameter's support (0.1-99.9 % range for an unbounded distribution) as bounds and the distributions in `metadata['distributions']`, so :func:`space_from_contract` reads it back exactly.
`model_from_ref`	função	`(ref, onnx=None, outputs=None) -> StudyModel`	`zoo:<name>` -> the zoo model; an ONNX reference (framework `onnx` or a `.onnx` URI) -> the graph in `onnx` bytes, named by the reference's inputs and outputs.
`model_to_ref`	função	`(model)`	A `ModelRef` for a study model: inputs with their validity range, outputs, held-out card metrics.
`uncertainty_result`	função	`(out, model, study_id, app='uncertainty', title='')`	`StudyResult` (`study_type='uncertainty'`) from an `uncertainty_study` dict.
`sensitivity_result`	função	`(out, model, study_id, app='sensitivity', title='')`	`StudyResult` (`study_type='sensitivity'`) from a `sensitivity_study` dict (or the runs-elsewhere dict with `sobol`, `freeze` and `parameters`).

`pinneapple_analysis.study_models`

One model interface for uncertainty and sensitivity studies: `predict(X) -> (n, m)` with input/output names, units and validity range, whatever the model is.

`pinneapple_analysis/study_models.py` · 211 linhas · testes: 25

nome	tipo	assinatura	o que faz
`StudyModel`	classe	`(name, kind, input_names, output_names, predict_fn, units, bounds, description, torch_fn,…)`	
`builtin`	função	`(name) -> StudyModel`	`"ishigami"` or `"sobol_g"` (6 inputs, a = 0, 1, 4.5, 9, 99, 99), with their exact Sobol indices in `metadata["exact"]`.
`columns`	função	`(table, names) -> np.ndarray`	Stack the named columns of a table (dict of arrays or a DataFrame) into `(n, k)` floats.

`pinneapple_analysis.surrogate`

Surrogate models from simulation results: select, compare at equal budget, measure on unseen data, package.

`pinneapple_analysis/surrogate/__init__.py` · 40 linhas · testes: 32

`pinneapple_analysis.surrogate.contracts`

The surrogate study, the regime arena and the gateway in the shared data contracts (`pinneapple_data.contracts`).

`pinneapple_analysis/surrogate/contracts.py` · 217 linhas · testes: 4

nome	tipo	assinatura	o que faz
`read_table_with_roles`	função	`(path, *, max_rows=None)`	`(DataFrame, {column: role} or None)` from CSV / Parquet / JSON / HDF5 / NPZ; the roles come from a `DatasetSpec` embedded in the file or in its `.contract.json` sidecar. HDF5 files not written by the contracts and NPZ files are read by `surrogate.read_table` (no roles).
`data_from_spec`	função	`(df, spec, *, group=None, grid_shape=None, meta=None) -> SurrogateData`	Inputs and outputs from the spec's column roles. A single output declared as a field (`spec.metadata["fields"] = {"u": [nx, nt]}`) or given as indexed columns `u_0, u_1, ...` becomes a field. `group` defaults to `spec.metadata["group"]`.
`dataset_spec`	função	`(data, name, *, app='', description='')`	
`model_ref`	função	`(model, *, name, version, uri, app='', dataset_name='')`	
`study_result`	função	`(study, *, id, app='surrogate', ref=None, title='')`	`study`: a `surrogate.StudyResult`; `ref`: the `ModelRef` of its model, when published.
`arena_study_result`	função	`(result, *, id, app='arena', title='')`	`result`: `ArenaResult.to_dict()`. The table has one row per method and regime.

`pinneapple_analysis.surrogate.data`

Surrogate training data: input parameters -> output quantities (scalars) or fields on a grid.

`pinneapple_analysis/surrogate/data.py` · 365 linhas · testes: 13

nome	tipo	assinatura	o que faz
`SurrogateData`	classe	`(X, Y, input_names, output_names, coords, coord_names, groups, meta)`	`X` (n, d) inputs; `Y` (n, m) scalar outputs or (n, *grid_shape) fields.
`grid_coords`	função	`(shape, ranges=None) -> np.ndarray`	Coordinates `(prod(shape), len(shape))` of a tensor-product grid in `ij` order (the order fields are flattened in), each axis spanning `ranges[i]` (default [0, 1]).
`inspect_table`	função	`(df, roles=None) -> dict[str, Any]`	What a table offers a surrogate: numeric columns with ranges, fields (indexed column groups `u_0 .. u_k`, at least 4), text columns (candidate group labels) and suggested inputs / outputs (from `roles`, e.g. a DatasetSpec's column roles, else: fields and the last numeric column are outputs).
`read_table`	função	`(source, name=None, *, fmt=None)`	Read a table from a path or bytes into a `pandas.DataFrame` (CSV, Parquet, JSON records/columns, HDF5 datasets, NPZ arrays). For HDF5 and NPZ, 1-D arrays become columns and 2-D arrays a become `a_0..a_k`.
`field_columns`	função	`(columns, name) -> list[str]`	The columns of field `name` (`name_0, name_1, ...` or `name[0], ...`) ordered by index.

nome	tipo	assinatura	o que faz
`from_table`	função	`(df, inputs, outputs, *, group=None, grid_shape=None, coords=None, coord_names=(), meta=N…)`	Pick `inputs` and `outputs` from a DataFrame. A single output name that is not a column but has indexed columns (`u_0, u_1, ...`) is a field; `grid_shape` reshapes it (default: 1-D). Rows with missing values in the chosen columns are dropped (their count goes to `meta['dropped_rows']`).
`from_arrays`	função	`(arrays, *, input_names=None, output_names=None, meta=None) -> SurrogateData`	From `{"X": (n, d), "Y": (n, m) or (n, *grid), "coords"?, "groups"?, "input_names"?, "output_names"?}` (the layout an NPZ or HDF5 export of a simulation campaign uses).
`lattice_order`	função	`(points) -> tuple[tuple[int, ...], np.ndarray] | None`	`(shape, order)` when the points are a tensor-product lattice (`points[order]` reshapes to `shape` with the first coordinate slowest), else None.
`from_snapshots`	função	`(snaps, *, meta=None) -> SurrogateData`	A parametric `pinneapple_data.snapshots.SnapshotSet` whose point fields live on a tensor-product lattice of nodes (a structured mesh, possibly scaled per snapshot) as one field dataset: `Y` (n, *lattice, channels), the channels being every component of every point field (``ux, uy, uz, ...
`snapshot_values`	função	`(snaps, data, Y) -> np.ndarray`	Fields predicted on the lattice of :func:`from_snapshots` (n, *lattice, channels) back to snapshot rows (n, n_values) in the layout of `snaps` (`snaps.field_values(name, rows)` then reads them).

`pinneapple_analysis.surrogate.examples`

Example surrogate datasets that need no download.

`pinneapple_analysis/surrogate/examples.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`airfoil_rans`	função	`() -> SurrogateData`	
`burgers_fields`	função	`(n=96, nu_range=(0.004, 0.1), nx=48, nt=12, t_max=0.9, seed=0) -> SurrogateData`	
`cantilever_3d`	função	`(path, n_timed=2) -> SurrogateData`	See the module docstring. `path`: the snapshot file (`load_snapshots` of the same file gives the mesh of each design and the clamp and load sets for a 3-D view; `snapshot_values` maps predictions back).
`burgers_fdm_baseline`	função	`(nx=201, cfl=0.4)`	A numerical-solver baseline for `burgers_fields`: the library's explicit upwind/FTCS finite-difference scheme (`pinneapple_simulation.numerical_solvers.fdm`) on `nx` points with a stable time step, interpolated to the dataset grid. Returns `solve(X, coords) -> (n, n_points)`; its error …

`pinneapple_analysis.surrogate.heldout`

Held-out errors of a field surrogate trained on snapshots (`data.from_snapshots`), in the terms of the snapshots: per field over all held-out samples, per region of the body, and on the scalar quantities an engineer reads off the field (the largest displacement at a loaded face, the largest …

pinneapple_analysis/surrogate/heldout.py · 100 linhas · testes: 13

nome	tipo	assinatura	o que faz
region_rel_l2	função	(pred, true, regions) -> dict[str, float]	Relative L2 error restricted to each region's points, over all samples: pred/true (n, m) or (n, m, components) with the points on axis 1.
held_out_report	função	(snaps, data, model, idx, *, grow=0.1) -> dict[str, Any]	Errors of model (a SurrogateModel or anything with predict_array) on the samples idx: fields (relative L2 over all of them, per region: the dataset's node sets grown by grow times the body, see rom.views.zones; per_sample: the mean and median of the per-sample errors ...

pinneapple_analysis.surrogate.limits

Where a surrogate may be used: validity bounds, an out-of-distribution gate, and where its errors concentrate.

pinneapple_analysis/surrogate/limits.py · 147 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
OODGate	classe	(X_train, names, knn_limit=3.0)	
per_sample_error	função	(pred, true, scale) -> np.ndarray	Per sample: RMS over outputs of (pred - true) / scale (scale: per-output std of the training targets; for fields the per-sample relative L2 is used instead, see study).
limitations	função	(gate, X_eval, err, sets, X_train, *, field_err=None, coords=None, coord_names=(), metric…)	X_eval / err: held-out samples (test and shift) and their per-sample error; sets: name -> boolean mask over them. field_err: (n_eval, n_points) pointwise absolute error, for the error map.

pinneapple_analysis.surrogate.methods

Surrogate candidates: which methods apply to a dataset, and how each is trained under a ComputeBudget.

pinneapple_analysis/surrogate/methods.py · 502 linhas · testes: 18 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
TrainContext	classe	(Xtr, Ytr, Xva, Yva, budget, seed, grid_shape, coords, pde, options)	What a method gets: raw arrays (Y flattened to (n, p)), training-split scalers, the field layout.
Fitted	classe	(method, n_params, info)	A trained candidate: predict(X_raw) -> Y_raw with Y flattened (n, p).
NonNegative	classe	(inner, mask)	A candidate trained on the square root of the outputs in mask (a boolean over the flattened outputs, e.g. a von Mises stress channel): it predicts their square, so they are never negative.
Candidate	classe	(id, title, family, iterative, fit, check)	

nome	tipo	assinatura	o que faz
`candidate_ids`	função	`() -> list[str]`	
`applicable`	função	`(kind, n_train, grid_shape=(), has_pde=False) -> dict[str, str | None]`	`{method: None if applicable else the reason it is not}` for every registered candidate.
`fit_candidate`	função	`(method, ctx) -> tuple[Fitted, float]`	Train `method`; returns the fitted model and the wall time of the fit (one-shot fits: the whole fit).

`pinneapple_analysis.surrogate.model`

A trained surrogate packaged for use: predict with the OOD gate, save / load, export, model card.

`pinneapple_analysis/surrogate/model.py` · 179 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`SurrogateModel`	classe	`(fitted, gate, *, input_names, output_names, grid_shape=(), coords=None, coord_names=(), …)`	

`pinneapple_analysis.surrogate.pde`

PDE declarations for physics-informed surrogates: which residual a PINN candidate adds to its data loss.

`pinneapple_analysis/surrogate/pde.py` · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PDEDeclaration`	classe	`(type, coord_index, coefficients)`	
`parse_pde`	função	`(decl, coord_names, input_names) -> PDEDeclaration | None`	Validate a declaration against the data; `None` passes through.

`pinneapple_analysis.surrogate.splits`

Train / validation / held-out test / distribution-shift splits for surrogate studies.

`pinneapple_analysis/surrogate/splits.py` · 105 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Splits`	classe	`(train, val, test, shift, shift_kind, shift_description, seed, info)`	
`make_splits`	função	`(data, *, test_frac=0.2, val_frac=0.15, shift='auto', shift_frac=0.15, seed=0) -> Splits`	Disjoint index sets; the shift set is removed before the random train/val/test split.

`pinneapple_analysis.surrogate.study`

Surrogate study: train every applicable candidate on the same data and budget, measure on unseen data, pick one.

pinneapple_analysis/surrogate/study.py · 380 linhas · testes: 13

nome	tipo	assinatura	o que faz
StudyResult	classe	(data, splits, budget, seeds, methods, best, model, limitations, trust, metric, created, …)	
evaluate	função	(kind, pred, true, scale, output_names) -> dict[str, Any]	Metrics of flattened predictions (n, p) against the truth, scale the training targets' std.
run_study	função	(data, *, methods='auto', budget=10.0, seeds=1, splits=None, split_options=None, pde=None…)	See the module docstring. budget: a ComputeBudget, a dict or seconds of wall clock per training run; seeds: a count or explicit seeds; pde: a declaration (pde.py) or data.meta['pde']; target: the held-out error a deployable model must reach (default 0.10 nRMSE, 0.05 …
nonnegative_mask	função	(data) -> np.ndarray | None	Boolean mask over the flattened outputs named in data.meta["nonnegative"]: output names for scalar data, channel names (meta["channels"], the last grid axis) for fields. None when nothing is declared.

pinneapple_analysis.trust

pinneapple_analysis.trust — pre-consumption trust scoring for a trained model's own predictions. See trust_gate.py for how this differs from pinneapple_llm.guardrail.PhysicsGuardrail (verifies LLM-proposed specs, not deployed-model predictions) and …

pinneapple_analysis/trust/__init__.py · 20 linhas · **sem teste direto**

pinneapple_analysis.trust.field_card

Trust card of a field on a grid: the physics checks a result must pass before anyone uses it.

pinneapple_analysis/trust/field_card.py · 358 linhas · testes: 12

nome	tipo	assinatura	o que faz
FieldCheck	classe	(name, kind, passed, value, threshold, critical, detail)	One check: kind is sanity, physics, reference or generalization (the lab vocabulary); passed is None when the check could not run, with the reason in detail.
FieldTrustCard	classe	(checks, stages, decision, reasons, name)	
field_checks	função	(grid, values, *, pde=None, source=None, alpha=1.0, time_coord='t', dirichlet=None, refer…)	The checks of one scalar field values (shape (len(axis_0), len(axis_1), ...)) on grid ({coord: axis} in the order of the array dimensions). Every argument after values is optional; a check without its input is reported as not run, with the reason, never silently dropped.

nome	tipo	assinatura	o que faz
trust_card	função	(grid, values, *, name='field', is_ml=True, limitations=(), min_coverage=0.5, **kwargs) -…)	Run field_checks and turn them into a trust card: the deploy decision (TrustReport: a failing critical check rejects, any other failure or low coverage asks for review) and the six questions of the lab trust card. kwargs go to field_checks.

pinneapple_analysis.trust.trust_gate

pinneapple_analysis.trust.trust_gate — TrustGate: pre-consumption trust scoring of a trained model's own prediction, before it is allowed to feed a live digital twin.

pinneapple_analysis/trust/trust_gate.py · 199 linhas · testes: 10

nome	tipo	assinatura	o que faz
TrustScore	classe	(combined, ood_score, residual_score, ensemble_score, details)	
TrustGate	classe	(*, ood_weight=0.4, residual_weight=0.4, ensemble_weight=0.2, ood_level=0.01)	

pinneapple_analysis.trust.trust_report

pinneapple_analysis.trust.trust_report — relatório de confiança para decidir o deploy de um modelo.

pinneapple_analysis/trust/trust_report.py · 179 linhas · testes: 13 · sinais no código: experimental

nome	tipo	assinatura	o que faz
Check	classe	(name, status, critical, weight, value, limit, source, reason)	
TrustReport	classe	(checks, min_coverage, min_score, model, details)	
checagem_erro_relativo	função	(nome, erro, limite, *, fonte='', critica=False, peso=1.0, motivo_nao_executada='') -> Ch…)	Acurácia contra referência: erro relativo menor ou igual ao limite.
checagem_booleana	função	(nome, passou, *, fonte='', critica=False, peso=1.0, motivo_nao_executada='') -> Check	Consistência física (conservação, condições de contorno, simetria) ou comparação qualitativa.
from_trust_score	função	(ts, *, limite_sub_score=0.7, fonte='TrustGate') -> List[Check]	Converte o TrustScore do TrustGate em checagens (OOD, resíduo e ensemble).

pinneapple_analysis.uncertainty

pinneapple_uq — Uncertainty Quantification for physics-informed neural networks.

pinneapple_analysis/uncertainty/__init__.py · 186 linhas · testes: 39

pinneapple_analysis.uncertainty.aleatoric

Aleatoric (data) uncertainty via heteroscedastic NLL regression.

pinneapple_analysis/uncertainty/aleatoric.py · 164 linhas · testes: 157

nome	tipo	assinatura	o que faz
AleatoricHead	classe	(base, out_dim, in_dim, *, hidden=64) -> None	Wrap a deterministic model with a learned aleatoric variance head.
aleatoric_nll_loss	função	(mean, log_var, target, *, min_log_var=-10.0, max_log_var=10.0) -> Tensor	Heteroscedastic Gaussian NLL loss.

pinneapple_analysis.uncertainty.applicability

Where is a model's prediction reliable? Applicability gates on the inputs and a routing rule that sends the cases the model cannot answer to the high-fidelity solver.

pinneapple_analysis/uncertainty/applicability.py · 163 linhas · testes: 8

nome	tipo	assinatura	o que faz
ApplicabilityDomain	classe	(names, lo, hi, center, scale, k, level, threshold, n_ref, bounds_source, _tree, _gate)	k-NN density gate plus range gate on standardised inputs. Build with :meth:fit.
route_cases	função	(status, pred, lower, upper, tolerance=None, limits=None) -> list[dict]	Routing rule, one entry per case: {"route": "surrogate" | "high_fidelity", "reasons": [...]}.
routing_summary	função	(routes) -> dict	Counts of each route and of each reason code (a case can have several).

pinneapple_analysis.uncertainty.calibration

Calibration metrics for probabilistic regression models.

pinneapple_analysis/uncertainty/calibration.py · 302 linhas · testes: 185

nome	tipo	assinatura	o que faz
CalibrationMetrics	classe	()	Collection of calibration and sharpness metrics for regression UQ.

pinneapple_analysis.uncertainty.conformal

Split Conformal Prediction for regression.

pinneapple_analysis/uncertainty/conformal.py · 281 linhas · testes: 151

nome	tipo	assinatura	o que faz
ConformalPredictor	classe	(model, alpha=0.1) -> None	Split conformal predictor for regression tasks.
conformal_quantile	função	(scores, alpha) -> float	The split-conformal quantile of any array of non-conformity scores (NumPy, list or tensor): the ceil((n + 1)(1 - alpha))-th smallest score, clipped to the largest (see _conformal_quantile).
conformal_quantile_valid	função	(n, alpha) -> bool	True when n calibration scores are enough for the finite-sample guarantee at level alpha (n >= ceil(1 / alpha) - 1); otherwise the quantile is clipped to the largest score and the guarantee does not hold.

pinneapple_analysis.uncertainty.conformal_study

Split-conformal prediction intervals for any model's predictions (NumPy), with the coverage *measured* on a held-out test split that the calibration never saw.

`pinneapple_analysis/uncertainty/conformal_study.py` · 195 linhas · testes: 8

nome	tipo	assinatura	o que faz
`ConformalStudy`	classe	`(alpha, q, normalized, n_cal, n_test, coverage, coverage_ci, mean_width, median_width, le…)`	Split-conformal calibration and its held-out evaluation.
`conformal_scores`	função	`(y, pred, scale=None) -> np.ndarray`	Non-conformity scores: `|y - pred|`, divided by `scale` when given.
`conformal_quantile`	função	`(scores, alpha=0.1) -> float`	The finite-sample conformal quantile: the `ceil((n + 1)(1 - alpha))`-th smallest score (the largest when that exceeds `n`; with `n < 1 / alpha - 1` the guarantee needs more calibration points).
`conformal_interval`	função	`(pred, q, scale=None) -> tuple[np.ndarray, np.ndarray]`	`(lower, upper) = pred -+ q` (times `scale` for normalised scores).
`min_calibration_size`	função	`(alpha) -> int`	Smallest calibration set for which the conformal quantile is finite at level `1 - alpha`.
`coverage_by_group`	função	`(covered, groups, labels=None, level=0.95, width=None) -> list[dict]`	Measured coverage per group (e.g. bins of an input or of the prediction): where the marginal guarantee hides under-coverage. `groups` holds a group index per point.
`quantile_bins`	função	`(x, n_bins=5) -> tuple[np.ndarray, list[str]]`	Equal-count bins of `x`: a bin index per point and a label `[lo, hi]` per bin.
`conformal_study`	função	`(y_cal, pred_cal, y_test, pred_test, alpha=0.1, scale_cal=None, scale_test=None, level=0.…)`	Calibrate on `(y_cal, pred_cal)` and measure the coverage on `(y_test, pred_test)`.
`split_indices`	função	`(n, fractions=(0.5, 0.5), seed=0) -> list[np.ndarray]`	Random disjoint index sets with the given fractions of `range(n)` (the last takes the remainder).

pinneapple_analysis.uncertainty.core

Shared types and unified interface for pinneapple_uq.

`pinneapple_analysis/uncertainty/core.py` · 265 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UQResult`	classe	`(mean, std, aleatoric_std, epistemic_std, samples, metadata)`	Container for predictions with associated uncertainty estimates.
`uq_predict`	função	`(model, x, method='mc_dropout', device=None, **kwargs) -> UQResult`	Unified interface for uncertainty-aware prediction.

pinneapple_analysis.uncertainty.decomposition

Variance decomposition: total predictive uncertainty = aleatoric + epistemic.

`pinneapple_analysis/uncertainty/decomposition.py` · 143 linhas · testes: 6

nome	tipo	assinatura	o que faz
`decompose_uncertainty`	função	`(model, x, *, n_samples=100, has_aleatoric=True, device=None) -> UQResult`	Decompose predictive uncertainty into aleatoric and epistemic parts.

pinneapple_analysis.uncertainty.distributions

Uncertain inputs of a study: one distribution per parameter, sampled by Monte Carlo, Latin hypercube or Sobol.

`pinneapple_analysis/uncertainty/distributions.py` · 187 linhas · testes: 8

nome	tipo	assinatura	o que faz
`UncertainParameter`	classe	`(name, dist, params, unit, nominal)`	One uncertain input. `ppf` maps probabilities in [0, 1] to values; `bounds` is the support (or the 0.1 %-99.9 % range for an unbounded distribution).
`as_parameter`	função	`(spec, name=None) -> UncertainParameter`	One parameter from a dict, a `(low, high)` pair (uniform) or an :class:`UncertainParameter`.
`as_space`	função	`(space) -> list[UncertainParameter]`	A list of parameters from a list of specs, a `{name: spec or (low, high)}` mapping, or an object with a `parameters` attribute (e.g. a ParameterSpace contract).
`to_values`	função	`(space, U) -> np.ndarray`	Map points of the unit hypercube (n, d) to parameter values through each inverse CDF.
`unit_sample`	função	`(d, n, method='lhs', seed=0) -> np.ndarray`	`n` points of the unit hypercube: `"mc"` (plain random), `"lhs"` (Latin hypercube) or `"sobol"` (scrambled Sobol sequence; `n` is used as is, a power of two keeps its balance properties).
`sample_space`	função	`(space, n, method='lhs', seed=0) -> np.ndarray`	`n` samples (n, d) of the parameters of `space` (see :func:`as_space`).

pinneapple_analysis.uncertainty.drift

Drift monitoring of a deployed surrogate: accuracy, calibration, input shift and residual trend in one report.

`pinneapple_analysis/uncertainty/drift.py` · 208 linhas · testes: 6

nome	tipo	assinatura	o que faz
`DriftThresholds`	classe	`(nrmse_review, nrmse_retrain, bias_z, coverage_z, ks_alpha, oob_review, oob_retrain, tren…)`	Thresholds of the `ok` / `review` / `retrain` decision of :func:`assess_drift`.
`DriftReport`	classe	`(n, nrmse, bias, bias_z, level, coverage, ks_stat, ks_pvalue, out_of_bounds, trend_tau, t…)`	Result of :func:`assess_drift`. Per-output arrays have shape `(m,)`; per-feature arrays `(d,)`.
`assess_drift`	função	`(y_true, y_pred, sigma=None, X_ref=None, X_new=None, bounds=None, level=0.95, thresholds=…)`	Check a batch of surrogate predictions for accuracy loss, miscalibration, input shift and residual trend.

pinneapple_analysis.uncertainty.ensemble

Deep Ensemble uncertainty quantification.

pinneapple_analysis/uncertainty/ensemble.py · 251 linhas · testes: 151

nome	tipo	assinatura	o que faz
`EnsembleConfig`	classe	`(n_members, diversity_reg, seed)`	Configuration for Deep Ensemble UQ.
`EnsembleUQ`	classe	`(models, config=None) -> None`	Deep Ensemble wrapper for predictive uncertainty quantification.

pinneapple_analysis.uncertainty.margin

Does a design that passes at its nominal dimensions still pass with its manufacturing tolerances? Monte Carlo margin study against an upper limit, with checks on the study itself.

pinneapple_analysis/uncertainty/margin.py · 130 linhas · testes: 6

nome	tipo	assinatura	o que faz
`corner_stackup`	função	`(fn, nominal, tolerances) -> dict[str, Any]`	Evaluate `fn` at the 2^d corners of the tolerance box `nominal +- tolerances`.
`summarize_samples`	função	`(values, limit, marginal_fraction=0.05) -> dict[str, Any]`	Distribution statistics and the p95 verdict against an upper `limit`.
`margin_study`	função	`(fn, nominal, tolerances, limit, *, n=200, n_small=50, dist='uniform', seed=None, check_s…)`	Monte Carlo margin study of `fn` (`(m, d)` inputs -> `(m,)`) against the upper `limit`.

pinneapple_analysis.uncertainty.mc_dropout

Monte Carlo Dropout for epistemic uncertainty quantification.

pinneapple_analysis/uncertainty/mc_dropout.py · 414 linhas · testes: 163

nome	tipo	assinatura	o que faz
`MCDropoutConfig`	classe	`(n_samples, dropout_p, seed)`	Configuration for Monte Carlo Dropout.
`MCDropoutWrapper`	classe	`(model, config) -> None`	Wraps any `nn.Module` and adds Monte Carlo Dropout at inference time.
`MCDropout`	classe	`(model, config=None) -> None`	Convenience class that applies MC Dropout to any callable model.

pinneapple_analysis.uncertainty.pce

Polynomial chaos expansion by least squares: output mean, variance and Sobol indices from one regression.

pinneapple_analysis/uncertainty/pce.py · 123 linhas · testes: 8

nome	tipo	assinatura	o que faz
`PCEResult`	classe	`(names, degree, n_terms, n_evals, mean, std, S1, ST, loo_q2, alphas, coef)`	Fitted expansion. Output arrays are per output (`m`).
`multi_indices`	função	`(d, degree) -> np.ndarray`	All multi-indices of `d` variables with total degree `<= degree` (constant first).

nome	tipo	assinatura	o que faz
`legendre_design`	função	`(xi, alphas) -> np.ndarray`	Design matrix of orthonormal Legendre products (w.r.t. the uniform density on [-1, 1]^d).
`n_terms`	função	`(d, degree) -> int`	
`polynomial_chaos`	função	`(fn, space, degree=3, n=None, seed=0, max_terms=500) -> PCEResult`	Fit a total-degree Legendre chaos to `fn` on `n` Latin-hypercube samples of `space`.

`pinneapple_analysis.uncertainty.posterior_metrics`

Posterior-matching metrics for generative / probabilistic inverse solvers (PosteriorBench).

`pinneapple_analysis/uncertainty/posterior_metrics.py` · 172 linhas · testes: 7

nome	tipo	assinatura	o que faz
`normalize_weights`	função	`(weights, n) -> np.ndarray`	
`weighted_mmd`	função	`(pred, ref, pred_weights=None, ref_weights=None) -> float`	
`weighted_wasserstein_1d`	função	`(x, y, wx=None, wy=None) -> float`	
`grf_direction`	função	`(h, w, rng, alpha=2.0, tau=3.0) -> np.ndarray`	Unit-norm smooth random direction: GRF with spectrum ~ (pi^2 |k|^2 + tau^2)^(-alpha/2).
`sliced_wasserstein`	função	`(pred, ref, pred_weights=None, ref_weights=None, n_projections=128, seed=0, alpha=2.0, ta…)`	
`radial_power_spectrum`	função	`(samples, weights=None) -> np.ndarray`	
`spectral_relative_geomean`	função	`(pred_spectrum, ref_spectrum, eps=1e-10) -> float`	
`evaluate_posterior`	função	`(pred, ref, pred_weights=None, ref_weights=None, *, n_projections=128, seed=0, swd_mean=N…)`	Compare a predicted posterior ensemble (N,H,W) with a reference one (M,H,W). Lower is better.

nome	tipo	assinatura	o que faz
`crps_gaussian`	função	`(mu, sigma, y) -> np.ndarray`	Closed form for a Gaussian predictive N(mu, sigma^2) (Gneiting et al., Mon. Weather Rev. 133 (2005) 1098-1118, Eq. 5). Elementwise.
`crps_ensemble`	função	`(samples, y, *, fair=False) -> np.ndarray`	Ensemble CRPS, `E|X - y| - 0.5 E|X - X'|` with members on axis 0 (Gneiting & Raftery, JASA 102 (2007) 359-378, Eq. 21). `fair=True` uses the M(M-1) normalisation of the spread term (Ferro, QJRMS 2014), unbiased for the CRPS of the distribution the members are drawn from. Computed in O(M log M) ...

`pinneapple_analysis.uncertainty.quantile`

Quantile (distribution-free) uncertainty estimation.

`pinneapple_analysis/uncertainty/quantile.py` · 212 linhas · testes: 161

nome	tipo	assinatura	o que faz
`QuantileConfig`	classe	`(quantiles)`	Configuration for quantile prediction.
`QuantileHead`	classe	`(base, cfg, hidden_dim=64, rearrange=True) -> None`	Wrap a point-prediction model and project outputs to multiple quantiles.
`QuantileLoss`	classe	`(quantiles) -> None`	Multi-quantile pinball loss compatible with the Pinneapple Trainer.
`pinball_loss_torch`	função	`(y_pred, y_true, q) -> torch.Tensor`	Pinball (quantile) loss for a single quantile level *q*.
`crossing_rate`	função	`(y_pred, quantiles) -> float`	Fraction of predictions whose quantiles are out of order.

`pinneapple_analysis.uncertainty.reachability`

RESEARCH ONLY -- set-based encounter feasibility for relative motion (collision risk, rendezvous).

`pinneapple_analysis/uncertainty/reachability.py` · 115 linhas · testes: 7

nome	tipo	assinatura	o que faz
`BodyState`	classe	`(position, velocity, pos_uncertainty, vel_uncertainty, max_accel, latency)`	
`encounter_feasibility`	função	`(a, b, horizon, *, contact_distance=0.0, n=2001, refine=True) -> Dict[str, object]`	Margin(t) = |c_a - c_b| - R_a - R_b - contact_distance on [0, horizon].
`time_to_reach_region`	função	`(chaser, target_center, target_radius=0.0, horizon=1000000.0) -> Optional[float]`	Earliest time at which a (static) target region is inside the chaser's reachable ball.

`pinneapple_analysis.uncertainty.resampling`

Resampling confidence intervals for dependent data: moving-block and group bootstrap, skill scores, jackknife.

`pinneapple_analysis/uncertainty/resampling.py` · 206 linhas · testes: 9

nome	tipo	assinatura	o que faz
block_bootstrap	função	(x, stat=np.mean, block=20, n=2000, seed=0, level=0.95) -> tuple[float, float, float]	Moving-block bootstrap of stat(x) for a serially dependent sample (Kunsch 1989).
group_bootstrap	função	(x, groups, stat=np.mean, n=2000, seed=0, level=0.95) -> tuple[float, float, float]	Cluster bootstrap of stat(x): resample whole groups with replacement.
skill_score	função	(err_model, err_ref, *, block=None, groups=None, n=2000, seed=0, level=0.95) -> tuple[flo…)	Skill score 1 - E|e_model| / E|e_ref| with a paired bootstrap interval.
jackknife	função	(samples, fn, n_blocks=20)	Delete-one-block jackknife estimate and standard error of fn(samples).

pinneapple_analysis.uncertainty.residual_bound

Residual-to-error calibration (#699): turn the PDE residual of a prediction, which needs no true answer, into an error bound an engineer can use ("the error is below X with 90 % confidence").

pinneapple_analysis/uncertainty/residual_bound.py · 242 linhas · testes: 19

nome	tipo	assinatura	o que faz
LinearSPDProblem	classe	(A, f, L, c, weight, shape, _L_solve)	A u = f with A SPD and A - c L positive semi-definite. weight is the quadrature weight of the inner products (the cell area h^2 on a uniform grid), so norms approximate their continuous counterparts.
CoverageReport	classe	(label, coverage, target, n, holds, tolerance, median_bound, note)	
ResidualLoss	classe	(problems)	A differentiable physics loss for training: mean_i ||f_i - A_i u_i||^2 / ||f_i||^2 over a fixed set of problems (e.g. :func:darcy_problem of each training input), in PyTorch with sparse matrices. Call loss(u, idx) with predictions u ((B, ...), any shape with n values per ...
ResidualErrorCalibrator	classe	(alpha=0.1, mode='ratio')	Split-conformal map from a residual measure to an error bound.
darcy_problem	função	(a, f=1.0) -> LinearSPDProblem	-div(a grad u) = f on the unit square, u = 0 on the boundary, in the discretisation of operator_benchmarks.darcy_2d (a, f at the n x n cell centres).
heat_conduction_problem	função	(k, q) -> LinearSPDProblem	Steady conduction -div(k grad T) = q with T = 0 on the boundary of the unit square: the same operator as Darcy, with conductivity k and heat source q.
energy_error_bound	função	(problem, u) -> dict[str, float]	Guaranteed bound on ||u_h - u||_A from the residual alone, absolute and relative to ||u||_A.
coverage_study	função	(res, err, *, alpha=0.1, n_cal=None, n_splits=200, seed=0) -> dict[str, float]	Coverage of the mode="ratio" bound over n_splits random calibration / test splits of one pool of exchangeable queries (n_cal calibration queries, default half). One split's coverage fluctuates by about sqrt(alpha (1 - alpha) / n); the mean over splits is what the conformal guarantee ...

pinneapple_analysis.uncertainty.study

One uncertainty study of one model output: is the prediction reliable here, and which cases need the solver?

`pinneapple_analysis/uncertainty/study.py` · 220 linhas · testes: 12

nome	tipo	assinatura	o que faz
`uncertainty_study`	função	`(model, output, X_cal, y_cal, X_test, y_test, *, alpha=0.1, scale='absolute', X_ref=None,…)`	Run the study (see the module docstring). `model` is a :class:`pinneapple_analysis.study_models.StudyModel`; arrays are `(n, d)` inputs in `model.input_names` order and `(n,)` true outputs. `tolerance` (output units) or `tolerance_pct` (percent of the prediction) is the interval …

pinneapple_analysis.uncertainty.tolerance

Monte-Carlo propagation of input (manufacturing, operating) tolerances through any model.

`pinneapple_analysis/uncertainty/tolerance.py` · 166 linhas · testes: 20

nome	tipo	assinatura	o que faz
`ToleranceResult`	classe	`(mean, std, percentiles, n, inputs, outputs, p_within, p_within_ci, p_within_each)`	Result of :func:`propagate`. Output arrays have shape `(m,)` (`m = 1` for a scalar model).
`wilson_interval`	função	`(k, n, level=0.95) -> tuple[float, float]`	Wilson score interval for a binomial proportion `k / n` (Wilson 1927; Brown, Cai & DasGupta 2001, eq. 4):
`propagate`	função	`(fn, nominal, tolerances, n=10000, dist='normal', limits=None, seed=0, level=0.95, percen…)`	Propagate independent input tolerances through `fn` by Monte-Carlo sampling (JCGM 101:2008).
`propagate_space`	função	`(fn, space, n=4096, method='lhs', limits=None, seed=0, level=0.95, percentiles=(1, 5, 50,…)`	Propagate independent inputs with any distribution (see :mod:`pinneapple_analysis.uncertainty.distributions`) through `fn`.
`src`	função	`(X, y) -> tuple[np.ndarray, float]`	Standardised regression coefficients of `y` on the columns of `X` (the samples of :func:`propagate`) and the `R^2` of the linear fit: how much of the output spread each tolerance explains, when the response is near linear over the tolerance band.

pinneapple_analysis.validation

Reexporta as checagens clássicas (`PhysicsValidator`, `ConservationCheck`, `BoundaryCheck`, `SymmetryCheck`, `validate_model`), comparações com solução analítica/solver, funções espectrais, metamórficas e do relatório-artefato. O docstring vem depois de `from __future__` e por isso não vira docstring do módulo.

`pinneapple_analysis/validation/__init__.py` · 82 linhas · testes: 21

pinneapple_analysis.validation.analytical

Compara as predições de um modelo com uma solução analítica em pontos aleatórios do domínio, ou com dados de um solver de referência (RMSE, L2 relativo, erro máximo). [sem teste direto]

`pinneapple_analysis/validation/analytical.py` · 187 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`compare_to_analytical`	função	`(model, analytical_fn, coord_names, domain_bounds, n_points=5000, field_names=None, devic…)`	Compara as predições do modelo com uma função analítica em pontos amostrados no domínio.
`validate_against_solver`	função	`(model, solver_data, coord_names, field_names, device='cpu') -> dict`	Compara as predições do modelo com dados de referência de um solver.

pinneapple_analysis.validation.arena

Red team / blue team para validação: geradores adversariais de casos errados porém plausíveis contra as checagens, e um laço que as endurece; o módulo é chamável como `run_arena`.

`pinneapple_analysis/validation/arena/__init__.py` · 43 linhas · testes: 12

pinneapple_analysis.validation.arena.__main__

Ponto de entrada `python -m pinneapple_analysis.validation.arena`: roda a arena em Darcy 2D e Burgers 1D. [sem teste direto]

`pinneapple_analysis/validation/arena/__main__.py` · 49 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Executa a arena pela linha de comando e devolve o código de saída.

pinneapple_analysis.validation.arena.blue

Blue team: limiares calibrados com taxa de falso alarme declarada (teste conformal dividido combinado) e endurecimento com checagens extras até rejeitar as fugas guardadas. [sem teste direto]

`pinneapple_analysis/validation/arena/blue.py` · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Defence`	classe	`(checks, thresholds, kind, alpha, alpha_cal, scales, statistic_threshold, n_calibration, …)`	Conjunto de checagens com um limiar cada (rejeita quando algum valor passa do limiar), com estatística combinada e descrição.
`score_matrix`	função	`(checks, cases) -> dict[str, np.ndarray]`	Calcula o valor de cada checagem em cada caso.
`calibrate_scores`	função	`(checks, values, alpha=0.01, *, safety=0.5, seed=0, false_alarms=None, kind=…, floor=0.01…)`	Calibra os limiares a partir de valores já calculados nos casos limpos, mantendo os falsos alarmes guardados abaixo do limiar.
`calibrate`	função	`(checks, clean_cases, alpha=0.01, *, safety=0.5, seed=0, false_alarms=()) -> Defence`	Teste conformal dividido combinado das checagens sobre casos limpos.

nome	tipo	assinatura	o que faz
harden	função	(current, library, clean_cases, *, escapes=(), false_alarms=(), alpha=0.01, safety=0.5, s…)	Calibra a defesa e adiciona checagens da biblioteca até rejeitar todas as fugas guardadas (ou nenhuma ajudar).

pinneapple_analysis.validation.arena.cases

Guarda casos de regressão da arena e os reexecuta (usado por `pinneapple check --arena-cases`). [sem teste direto]

pinneapple_analysis/validation/arena/cases.py · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ReplayResult	classe	(path, problem, rows)	Resultado da reexecução de um arquivo de casos; aponta regressões.
save_cases	função	(path, problem, defence, findings, meta=None) -> Path	Grava problema, defesa e achados num arquivo de casos.
load_cases	função	(path) -> dict[str, Any]	Lê um arquivo de casos.
builtin_cases	função	() -> list[Path]	Lista os arquivos de casos que vêm com a biblioteca (um por problema).
load_defence	função	(path) -> tuple[Any, Defence]	Devolve o problema e a defesa endurecida guardados num arquivo de casos.
replay	função	(path) -> ReplayResult	Regenera cada caso guardado e o confere contra a defesa guardada.

pinneapple_analysis.validation.arena.generators

Geradores do red team: famílias parametrizadas de saídas erradas porém plausíveis que a busca de evasão explora. [sem teste direto]

pinneapple_analysis/validation/arena/generators.py · 164 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Generator	classe	(name, bounds, fn, problems, physics_consistent, description)	Família parametrizada de saídas erradas porém plausíveis, com limites e flag de consistência física.
get_generator	função	(name) -> Generator	Devolve um gerador pelo nome.
generators_for	função	(problem) -> list[Generator]	Lista os geradores aplicáveis a um problema.

pinneapple_analysis.validation.arena.loop

Laço da arena: rodadas de red team e depois blue team, o relatório e a auditoria de um modelo externo contra uma defesa. [sem teste direto]

pinneapple_analysis/validation/arena/loop.py · 396 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ArenaBudget	classe	(n_bases, evals, fa_seeds, fa_evals, n_calibration, n_holdout, n_mutation_cases, n_datase…)	Quanto o red team busca e quantos casos limpos o blue team usa para calibrar.

nome	tipo	assinatura	o que faz
`RoundReport`	classe	`(round, defence_before, defence_after, mutation_before, mutation_after, escapes, attempts…)`	Relatório de uma rodada: defesa antes/depois, fugas, falsos alarmes e escore de mutação.
`ArenaReport`	classe	`(problem, alpha, min_error, rounds, defence, stored, settings, _problem)`	Relatório completo da arena, com rodadas, defesa final, casos guardados e saída em Markdown/JSON.
`ModelAudit`	classe	`(problem, defence, true_error, rejected, seeds, min_error)`	Saídas de um modelo julgadas por uma defesa contra o solver de referência (fugas e falsos alarmes).
`run_arena`	função	`(problem='darcy_2d', checks=None, *, budget=None, rounds=1, alpha=0.01, safety=0.5, min_e…)`	Roda rodadas de red team e blue team num problema e devolve o `ArenaReport`.
`audit_model`	função	`(problem, predict, defence=None, *, n_cases=50, seed=0, min_error=0.1, varied=True, **pro…)`	Passa um modelo externo (FNO, MeshGraphNet, PINN…) por uma defesa.

`pinneapple_analysis.validation.arena.red`

Red team: busca fugas (erradas mas aceitas) e falsos alarmes (corretas mas rejeitadas) contra uma defesa, mais o escore de mutação. [sem teste direto]

`pinneapple_analysis/validation/arena/red.py` · 178 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Finding`	classe	`(kind, problem, seed, variation, generator, params, true_error, values, margin, found, me…)`	Uma fuga, um falso alarme ou a melhor tentativa de um deles, com parâmetros e erro verdadeiro.
`evasion_search`	função	`(problem, defence, generator, base, *, budget=200, method='de', seed=0, min_error=0.1) ->…)`	Busca a pior saída de um gerador que a defesa ainda aceita.
`false_alarm_search`	função	`(problem, defence, *, seeds, budget=100, method='de', search_seed=0) -> list[Finding]`	Para cada semente, busca o caso correto com maior margem de rejeição da defesa.
`mutation_score`	função	`(matrix, thresholds, min_error=0.1) -> dict[str, Any]`	Fração dos mutantes com erro relevante que os limiares rejeitam, no total e por falha.
`mutation_escapes`	função	`(matrix, thresholds, *, seed, min_error=0.1, per_fault=1) -> list[Finding]`	Lista os mutantes relevantes que passam, como achados reproduzíveis.

`pinneapple_analysis.validation.arena.search`

Minimizadores caixa-preta no cubo unitário para o red team: evolução diferencial (SciPy), CMA-ES e L-BFGS-B por diferenças finitas. [sem teste direto]

`pinneapple_analysis/validation/arena/search.py` · 130 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SearchResult`	classe	`(x, fun, evaluations, method)`	Resultado de uma minimização: ponto, valor, avaliações e método.
`cmaes`	função	`(f, dim, budget, seed=0, sigma0=0.3, x0=None) -> SearchResult`	CMA-ES (mu/mu_w, lambda) no cubo [0, 1]^dim.

nome	tipo	assinatura	o que faz
`differential_evolution`	função	`(f, dim, budget, seed=0) -> SearchResult`	Evolução diferencial do SciPy dentro de um orçamento de avaliações.
`lbfgs`	função	`(f, dim, budget, seed=0) -> SearchResult`	L-BFGS-B com gradiente por diferenças finitas a partir de alguns pontos iniciais.
`minimize`	função	`(f, dim, budget, method='de', seed=0) -> SearchResult`	Minimiza uma função no cubo unitário com o método escolhido (`de`, `cmaes` ou `lbfgs`).

`pinneapple_analysis.validation.boundary`

Checagens de condições de contorno de uma PINN treinada (Dirichlet, Neumann, periodicidade) consultando o modelo em pontos de borda.

`pinneapple_analysis/validation/boundary.py` · 255 linhas · testes: 151

nome	tipo	assinatura	o que faz
`BoundaryCheck`	classe	`(device='cpu') -> None`	Checa condições de contorno consultando o modelo em pontos de borda.

`pinneapple_analysis.validation.conservation`

Checagens de leis de conservação (massa, energia, integral genérica) por integração numérica sobre o domínio.

`pinneapple_analysis/validation/conservation.py` · 375 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ConservationCheck`	classe	`(device='cpu') -> None`	Checa leis de conservação por integração numérica no domínio.

`pinneapple_analysis.validation.core`

Estruturas de dados do validador clássico: `CheckResult` e o `ValidationReport` agregado (não confundir com o de `validate.py`). [sem teste direto]

`pinneapple_analysis/validation/core.py` · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CheckResult`	classe	`(name, passed, value, threshold, description, unit)`	Resultado de uma checagem física: nome, aprovado, valor, limiar e descrição.
`ValidationReport`	classe	`(model_name, timestamp, checks)`	Relatório agregado do validador clássico, com resumo em tabela, `to_dict` e `save`.

`pinneapple_analysis.validation.error_map`

Mostra onde um campo está errado: erro ponto a ponto contra a referência, aglomerados de erro alto (hotspots), participação por região e um texto que compara com o número global.

`pinneapple_analysis/validation/error_map.py` · 182 linhas · testes: 9

nome	tipo	assinatura	o que faz
`ErrorMap`	classe	`(error, rel_l2, max_error, rms_error, hotspots, regions, unit, label)`	Resultado de `error_map`: erro por ponto, L2 relativo, erro máximo e RMS, hotspots e regiões.
`pointwise_error`	função	`(pred, ref) -> np.ndarray`	Calcula `|pred - ref|` por ponto (norma Euclidiana para vetores).
`error_hotspots`	função	`(points, err, *, n=5, quantile=0.9, link=None, weights=None) -> list[dict[str, Any]]`	Agrupa os pontos de erro acima de um quantil em aglomerados e devolve os de maior erro.
`grow_region`	função	`(points, idx, radius) -> np.ndarray`	Devolve os pontos a menos de um raio de um conjunto de índices.
`error_by_region`	função	`(err, regions, *, weights=None) -> list[dict[str, Any]]`	Por região nomeada: participação nos pontos e no erro quadrático e razão do RMS local para o global.
`error_map`	função	`(points, pred, ref, *, regions=None, weights=None, n_hotspots=5, quantile=0.9, unit='', l…)`	Calcula erro ponto a ponto, hotspots e participação por região de uma predição contra a referência.

pinneapple_analysis.validation.fault_injection

Injeção de falhas: mede com que frequência cada checagem de validação pega defeitos conhecidos de modelo e de dados.

`pinneapple_analysis/validation/fault_injection/__init__.py` · 36 linhas · testes: 20

pinneapple_analysis.validation.fault_injection.__main__

Ponto de entrada `python -m pinneapple_analysis.validation.fault_injection`: regenera a matriz de detecção (poder das checagens). [sem teste direto]

`pinneapple_analysis/validation/fault_injection/__main__.py` · 59 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	Regenera a matriz de detecção pela linha de comando e devolve o código de saída.

pinneapple_analysis.validation.fault_injection.checks

As checagens em teste, como objetos `ValidityCheck` de `physics_audit.monitor` que pontuam um `Case`, incluindo checagens de conjunto de dados. [sem teste direto]

`pinneapple_analysis/validation/fault_injection/checks.py` · 289 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`score`	função	`(check, case) -> float`	Valor de uma checagem num caso (`inf` quando não dá para calcular).
`flags`	função	`(check, case, tolerance=None) -> bool`	Diz se a checagem rejeita o caso, com tolerância opcional.
`check_library`	função	`(problem) -> dict[str, Any]`	Todas as checagens disponíveis para um problema, com tolerância ajustada à mão.

nome	tipo	assinatura	o que faz
`default_checks`	função	`(problem) -> list`	As checagens que um usuário roda hoje (rodada 0 da arena).
`dataset_checks`	função	`(problem, case_checks=None) -> list`	Checagens de um conjunto de dados inteiro: quase-duplicatas, vazamento treino/teste e pior valor por amostra.

`pinneapple_analysis.validation.fault_injection.faults`

Biblioteca de falhas: defeitos controlados da saída de um modelo ou de um conjunto de dados, cada um com uma magnitude. [sem teste direto]

`pinneapple_analysis/validation/fault_injection/faults.py` · 248 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Fault`	classe	`(name, target, apply, magnitudes, documented, unit, description)`	Uma falha controlada de modelo ou de dados, com magnitudes e função `apply`.
`model_faults`	função	`(problem=None) -> list[Fault]`	Lista as falhas de saída de modelo, opcionalmente filtradas por problema.
`data_faults`	função	`(problem=None) -> list[Fault]`	Lista as falhas de conjunto de dados, opcionalmente filtradas por problema.
`get_fault`	função	`(name) -> Fault`	Devolve uma falha pelo nome.

`pinneapple_analysis.validation.fault_injection.matrix`

Matriz de detecção: cada checagem contra cada falha e magnitude e contra casos limpos, com taxas de detecção, falso alarme, AUC e magnitude mínima detectável. [sem teste direto]

`pinneapple_analysis/validation/fault_injection/matrix.py` · 282 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DetectionMatrix`	classe	`(problem, level, tolerances, clean, scores, errors, faults, min_rate, trials)`	Valores das checagens em casos limpos e em cada falha e magnitude, com taxas de detecção, falso alarme e AUC.
`FaultInjectionReport`	classe	`(problem, model, data, settings)`	Relatório da injeção de falhas (nível modelo e nível dados), com poder das checagens e saída em Markdown.
`auc`	função	`(clean, faulty) -> float`	AUC ROC de um escore separando casos com falha de casos limpos (Mann-Whitney).
`fault_rng`	função	`(seed, fault, magnitude, case_seed) -> np.random.Generator`	Fluxo aleatório de uma injeção de falha, para reproduzir um teste isolado.
`run_fault_injection`	função	`(problem='darcy_2d', *, checks=None, data_checks=None, model_faults=None, data_faults=Non…)`	Monta a matriz de detecção das checagens contra a biblioteca de falhas num problema.

pinneapple_analysis.validation.fault_injection.problems

Problemas de benchmark com solver de referência verificado (Darcy 2D, Burgers 1D) como fonte de casos corretos; a base `BenchmarkProblem` tem ganchos que levantam NotImplementedError até a subclasse implementar. [sem teste direto]

`pinneapple_analysis/validation/fault_injection/problems.py` · 274 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Case`	classe	`(problem, inputs, field, params, seed, variation, meta)`	Um par entrada/saída de um problema, reproduzível por semente e variação.
`Dataset`	classe	`(train, test)`	Divisão de treino e teste de casos.
`BenchmarkProblem`	classe	`(**options)`	Base de problema com solver de referência verificado; subclasses implementam a solução e o espaço de variação.
`DarcyProblem`	classe	`()`	Problema de Darcy `-div(a grad u) = f` no quadrado unitário com `u = 0` na borda.
`BurgersProblem`	classe	`()`	Problema de Burgers viscoso 1D periódico.
`get_problem`	função	`(problem, **options) -> BenchmarkProblem`	Devolve um problema pelo nome (`darcy_2d`, `burgers_1d`) ou o próprio objeto.

pinneapple_analysis.validation.field_metrics

Métricas de erro no estilo PDEBench para campos em grade, só NumPy (RMSE, nRMSE, erro de contorno, conservado, por faixa de frequência, Pearson).

`pinneapple_analysis/validation/field_metrics.py` · 154 linhas · testes: 17

nome	tipo	assinatura	o que faz
`FieldErrorReport`	classe	`(rmse, nrmse, max_error, crmse, brmse, frmse_low, frmse_mid, frmse_high, bands, vrmse, pe…)`	Conjunto de métricas de erro de um campo em grade, serializável com `to_dict`.
`spectral_error_shells`	função	`(pred, target, ndim) -> np.ndarray`	Erro quadrático médio por casca de número de onda radial inteiro.
`field_error_metrics`	função	`(pred, target, *, ndim, bands=(4, 12), eps=1e-12) -> FieldErrorReport`	Calcula o `FieldErrorReport` de uma predição contra o alvo.

pinneapple_analysis.validation.gallery

Final comum dos exemplos de `examples/validation/`: relatório do modelo bom, trust card, saídas lidas por `pinneapple check` e uma falha real pega numa variante quebrada.

`pinneapple_analysis/validation/gallery.py` · 67 linhas · testes: 4

nome	tipo	assinatura	o que faz
`GalleryResult`	classe	`(name, family, physics, good, bad, caught, expected, files)`	Resultado de um exemplo da galeria: relatórios bom e ruim, se a falha foi pega e arquivos gravados.

nome	tipo	assinatura	o que faz
`conclude`	função	`(name, good, bad, *, expect, family, physics, out_dir=None) -> GalleryResult`	Fecha um exemplo da galeria e levanta `AssertionError` se a checagem esperada não pegar a falha.

`pinneapple_analysis.validation.metamorphic`

Testes metamórficos para substitutos: relações que a física garante (simetria, invariância, monotonicidade, superposição, consistência de resolução), testáveis sem solução de referência; a base `Relation.violations` levanta NotImplementedError.

`pinneapple_analysis/validation/metamorphic.py` · 497 linhas · testes: 9 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Relation`	classe	`(name, tol, output, atol, scale, per_sample, kind)`	Base de relação metamórfica; `violations` devolve a violação relativa por amostra e a evidência.
`Symmetry`	classe	`(transform_in, transform_out)`	Relação de simetria `f(T x) = S(x, f(x))`.
`Similarity`	classe	`(transform_in, dimensionless)`	Relação de semelhança: uma grandeza adimensional não muda sob a transformação da entrada.
`Invariance`	classe	`(transform_in)`	Relação de invariância `f(T x) = f(x)`.
`Monotonicity`	classe	`(input, direction, step, increase, bounds)`	A saída não diminui (ou não aumenta) quando uma entrada cresce.
`Superposition`	classe	`(affine, homogeneity, linear_inputs, combine)`	Relação de superposição (aditividade e, opcionalmente, homogeneidade) de modelos lineares ou afins.
`ResolutionConsistency`	classe	`(restrict, quantity)`	Grandezas integrais devem ser iguais quando a entrada muda de resolução.
`RelationResult`	classe	`(name, kind, passed, worst, tol, n_tested, counterexample, search, note)`	Resultado de uma relação: aprovação, pior violação, contraexemplo e nota.
`MetamorphicReport`	classe	`(results)`	Relatório de todas as relações metamórficas testadas.
`box_sampler`	função	`(bounds, fixed=None, log=()) -> Sampler`	Amostrador uniforme (ou log-uniforme) numa caixa de entradas nomeadas.
`field_sampler`	função	`(make) -> Sampler`	Amostrador de lotes de campos a partir de um gerador de uma amostra.
`reflect_field`	função	`(axis) -> Callable[[np.ndarray], np.ndarray]`	Cria a reflexão de um lote de campos ao longo de um eixo espacial.
`rotate_field`	função	`(k=1, axes=(0, 1)) -> Callable[[np.ndarray], np.ndarray]`	Cria a rotação de um lote de campos por quartos de volta.
`check_relations`	função	`(model, relations, sampler, *, n=64, seed=0, search='random', max_examples=50) -> Metamor…)`	Roda cada relação em amostras aleatórias e, com `search='hypothesis'`, faz busca por propriedades (cai para busca aleatória se `hypothesis` faltar).

nome	tipo	assinatura	o que faz
relations_from_spec	função	(spec) -> list[Relation]	Cria relações de um modelo tabular a partir de uma lista declarativa (TOML/JSON).

pinneapple_analysis.validation.physics_learning

Verifica se um modelo aprendeu a física ou só uma perda baixa: escolhe pontos informativos, checa resíduo discreto, respostas a intervenções e suficiência das entradas, e compara com as métricas usuais.

pinneapple_analysis/validation/physics_learning.py · 499 linhas · testes: 7

nome	tipo	assinatura	o que faz
Finding	classe	(check, passed, value, limit, detail, provenance, points)	Um achado da verificação: checagem, aprovação, valor, limite e detalhe.
VerificationReport	classe	(model, findings, points, reasons, insufficient, trust)	Relatório de verificação de aprendizado da física, com achados, pontos e confiança.
select_points	função	(case, theta, k=48, seed=0, interventions=INTERVENTIONS, pool=3000) -> tuple[np.ndarray, …)	Escolhe pontos informativos pela resposta da referência a cada intervenção e por cotas perto de interfaces, fonte e bordas.
random_points	função	(case, k, seed=0) -> np.ndarray	Sorteia pontos aleatórios do caso (base de comparação).
fv_residual	função	(model, case, th, rows) -> np.ndarray	Resíduo de volumes finitos dos valores do próprio modelo nas células dadas, relativo à escala de calor.
structural_checks	função	(model, case, theta, X, probes=True) -> list[Finding]	Checagens no campo do modelo que não precisam de referência: resíduo, interfaces, bordas e balanço global de calor.
intervention_checks	função	(model, case, theta, X, interventions=INTERVENTIONS) -> list[Finding]	Compara a resposta do modelo a cada intervenção com a resposta da referência.
sufficiency	função	(model, case, theta, X) -> tuple[Finding, list[dict]]	Aponta parâmetros físicos fora das entradas do modelo que mudam a temperatura de referência acima da tolerância.
verify	função	(model, case=None, theta=None, *, k=48, seed=0, points=None, probes=True) -> Verification…)	Roda todas as checagens do módulo num modelo e devolve o VerificationReport.
baselines	função	(model, case=None, *, n_theta=12, n_points=400, seed=1) -> dict	Calcula o que as métricas usuais dizem (erro L2 em pontos aleatórios e perda física média).
detection_study	função	(severities=(0.25, 0.5, 1.0, 2.0), budgets=(4, 8, 16, 48), *, seed=0, extra_models=None) …)	Estudo de detecção do verificador contra as métricas usuais, com modelos defeituosos e de controle.

pinneapple_analysis.validation.physics_learning_cases

Caso de verificação para physics_learning: seção transversal die/TIM/tampa aquecida na junção, com solver de referência, modelos de controle, modelos defeituosos, substituto POD+GP e PINN.

pinneapple_analysis/validation/physics_learning_cases.py · 359 linhas · testes: 7

nome	tipo	assinatura	o que faz
`Section`	classe	`(nx, nz, _cache)`	Grade e solver de referência da seção transversal die/TIM/tampa.
`Model`	classe	`(name, fn, inputs, defect, reinit)`	Modelo sob teste `fn(X, theta) -> T`, com entradas, defeito e cópia não treinada opcional.
`good_models`	função	`(sec, amplitudes=(0.003,), modes=(1,)) -> list[Model]`	Modelos de controle: a referência e a referência com erros suaves que respeitam contorno e simetria.
`defective_models`	função	`(sec, severity=1.0) -> list[Model]`	Modelos precisos pelas métricas usuais mas errados num aspecto físico, com severidade ajustável.
`training_distribution`	função	`(n, rng, *, correlated=True) -> list[dict]`	Amostras de parâmetros como as de treino, com resfriamento correlacionado à potência.
`surrogate_model`	função	`(sec, n_train=48, *, correlated=True, seed=0) -> Model`	Substituto POD + processo Gaussiano do campo, treinado em soluções de referência.
`pinn_model`	função	`(sec, *, hard_sides=False, adam_steps=3000, lbfgs_steps=500, n_collocation=3000, seed=0, …)`	PINN do caso nominal treinada com `train_pinn`, com uma saída por camada e condições de interface e contorno.

`pinneapple_analysis.validation.pinn`

Validação de PINN: resíduo em pontos nunca usados no treino, detectores dos modos de falha conhecidos, comparação com solução exata e um monitor durante o treino; traz problemas prontos (Burgers, Poisson, Kovasznay).

`pinneapple_analysis/validation/pinn.py` · 576 linhas · testes: 8

nome	tipo	assinatura	o que faz
`PINNProblem`	classe	`(name, bounds, residual, boundary, initial, exact, lhs, forcing, time_axis, scale, residu…)`	Descrição de um problema para `validate_pinn`: domínio, resíduo, contorno, condição inicial, solução exata e escalas.
`PINNReport`	classe	`(problem, checks, evidence)`	Relatório de validação de PINN com checagens, evidência e modos de falha detectados.
`PINNMonitor`	classe	`(problem, *, every=100, warmup=0, patience=2, stop_on=…, n_points=1024, grid=(32, 32), **…)`	Roda as checagens de `validate_pinn` a cada tantas épocas durante o treino, com parada opcional.
`d`	função	`(u, X, i, order=1)`	Derivada de ordem dada de `u` em relação a uma coordenada, por autograd.
`validate_pinn`	função	`(model, problem, *, n_points=4096, grid=None, train_points=None, tolerances=None, seed=0)…)`	Valida uma PINN treinada num problema e devolve o `PINNReport`.
`mlp`	função	`(n_in, n_out, width=32, depth=4)`	Cria uma MLP tanh, a rede usual de PINN.
`train_pinn`	função	`(problem, model=None, *, n_collocation=2000, n_condition=400, adam_steps=1500, lbfgs_step…)`	Treina uma PINN com restrições suaves (Adam e depois L-BFGS) e monitor opcional.
`cole_hopf`	função	`(x, t, nu, n=2001) -> np.ndarray`	Solução exata de Burgers viscoso com `u(x, 0) = -sin(pi x)` pela transformada de Cole-Hopf.

nome	tipo	assinatura	o que faz
`burgers_1d`	função	`(nu=0.05 / math.pi) -> PINNProblem`	Problema de Burgers 1D (Raissi et al. 2019, viscosidade maior por padrão) com solução exata.
`poisson_2d`	função	`(kx=1, ky=2, exact=True) -> PINNProblem`	Problema de Poisson 2D manufaturado no quadrado unitário.
`kovasznay`	função	`(re=40.0) -> PINNProblem`	Escoamento de Kovasznay (Navier-Stokes estacionário) com saídas (u, v, p).

`pinneapple_analysis.validation.report_artifact`

O relatório de validação como artefato: esquema JSON versionado, hashes de proveniência, diferença entre dois relatórios, seção para model card do Hugging Face, badge e assinaturas.

`pinneapple_analysis/validation/report_artifact.py` · 341 linhas · testes: 26

nome	tipo	assinatura	o que faz
`ReportDiff`	classe	`(old, new, lost, gained, changed, added, removed, moved, envelope, decision)`	Diferença entre dois relatórios: checagens perdidas, ganhas, alteradas e decisão.
`as_dict`	função	`(report) -> dict[str, Any]`	Converte relatório (objeto, dict ou caminho JSON) em dict.
`check_report`	função	`(report) -> list[str]`	Lista os problemas de um relatório contra o esquema e o vocabulário de IDs; vazio se válido.
`hash_model`	função	`(model) -> str | None`	SHA-256 dos parâmetros de um modelo ou do arquivo ONNX; None para um callable simples.
`hash_data`	função	`(*objs) -> str | None`	SHA-256 dos dados usados num relatório.
`diff`	função	`(old, new, *, rtol=0.1) -> ReportDiff`	Calcula o que mudou entre dois relatórios de validação.
`model_card_section`	função	`(report, *, title='Physics validation') -> str`	Gera um bloco Markdown para model card do Hugging Face.
`shields_endpoint`	função	`(report, label='physics-validated') -> dict[str, Any]`	Gera o JSON de um badge de endpoint do shields.io.
`badge`	função	`(report, label='physics-validated') -> str`	Gera um badge SVG autocontido com o resultado das checagens.
`sign_report`	função	`(report, key) -> dict[str, Any]`	Assina o relatório com `pinneapple_security.signing.sign_json`.
`verify_report`	função	`(envelope, key) -> dict[str, Any]`	Verifica a assinatura de um envelope e devolve o relatório, levantando erro se não confere.

`pinneapple_analysis.validation.rollout`

Validação de rollout de substitutos autorregressivos: deriva de invariantes, espectro de energia ao longo do horizonte, tempo de divergência ('trust horizon') e estatísticas de longo prazo.

`pinneapple_analysis/validation/rollout.py` · 326 linhas · testes: 7

nome	tipo	assinatura	o que faz
`RolloutReport`	classe	`(checks, evidence)`	Relatório das checagens de rollout com evidência.
`rollout`	função	`(step, u0, n_steps) -> np.ndarray`	Gera a trajetória de um modelo autorregressivo, parando cedo se o estado deixar de ser finito.
`invariant_series`	função	`(traj, ndim, invariants=('mass', 'energy')) -> dict[str, np.ndarray]`	Séries temporais dos invariantes (massa, energia) de uma trajetória.
`invariant_drift`	função	`(pred, ndim, ref=None, *, dt=1.0, invariants=('mass', 'energy'), window=1) -> dict[str, d…)`	Taxa e deriva máxima de cada invariante, e desvio contra a referência se houver.
`spectral_drift`	função	`(pred, ndim, ref=None, *, ref_spectrum=None, periodic=True, window=1, floor=1e-06) -> dic…)`	Viés de energia em alto número de onda a cada horizonte, contra a referência.
`divergence_time`	função	`(pred, ref, ndim, *, dt=1.0, err_tol=0.2, spec_tol=0.25, inv_tol=0.05, periodic=True, inv…)`	Primeiro horizonte em que cada critério cruza a tolerância, e o 'trust horizon' (o mais cedo).
`long_run_statistics`	função	`(pred, ref, ndim, *, dt=1.0, burn_in=0, periodic=True) -> dict[str, Any]`	Compara medidas invariantes de uma corrida longa do modelo com uma corrida longa de referência.
`validate_rollout`	função	`(pred, ndim, ref=None, *, dt=1.0, conserved=(), invariants=('mass', 'energy'), long_ref=N…)`	Roda as checagens `rollout.*` de uma trajetória (aprova / falha / não aplicável).

`pinneapple_analysis.validation.schema`

Vocabulário do relatório de validação num só lugar: status, fatores e níveis de credibilidade e os IDs estáveis de checagens usados por `pp.validate`, trust card e `pinneapple check`.

`pinneapple_analysis/validation/schema.py` · 120 linhas · testes: 9

nome	tipo	assinatura	o que faz
`CheckSpec`	classe	`(id, factor, title, rule, strength, critical, better)`	Especificação de uma checagem: ID, fator de credibilidade, título, regra, força e criticidade.
`spec_of`	função	`(check_id) -> CheckSpec`	Devolve a especificação de um ID de checagem; sub-checagens herdam da família.

`pinneapple_analysis.validation.spectral`

Fidelidade espectral de um campo previsto (só NumPy): espectro de energia, viés em alto número de onda, erro de fase, coerência, funções de estrutura e erros de gradiente.

`pinneapple_analysis/validation/spectral.py` · 369 linhas · testes: 9

nome	tipo	assinatura	o que faz
`SpectralComparison`	classe	`(k, E_pred, E_ref, ratio, phase_error, coherence, bands, band_ratio, band_phase_error, ba…)`	Comparação espectral: espectros, razão, erro de fase, coerência e viés por faixa.
`StructureFunctions`	classe	`(r, S, S3_signed, flatness)`	Funções de estrutura por defasagem, com flatness.

nome	tipo	assinatura	o que faz
SpectralFidelityReport	classe	(rel_l2, spectrum, structure_pred, structure_ref, gradients, tol, checks)	Relatório de fidelidade espectral com L2 relativo, espectro, funções de estrutura e gradientes.
energy_spectrum	função	(u, ndim=None, *, periodic=True, window='hann', kmax=None) -> tuple[np.ndarray, np.ndarra…)	Espectro de energia por cascas de um campo.
fit_power_law	função	(k, E, kmin, kmax) -> tuple[float, float]	Ajusta uma lei de potência E = C k^slope num intervalo de k.
spectral_comparison	função	(pred, ref, ndim, *, periodic=True, window='hann', bands=(1 / 3, 2 / 3), floor=1e-10) -> …)	Compara os espectros de predição e referência.
high_wavenumber_bias	função	(pred, ref, ndim, **kw) -> float	Viés de energia na faixa alta de número de onda.
structure_functions	função	(u, ndim, *, orders=(2, 3), lags=None, periodic=True) -> StructureFunctions	Calcula funções de estrutura ao longo de cada eixo espacial.
gradient_metrics	função	(pred, ref, ndim, spacing=1.0, *, periodic=False, vector_axis=None, walls=(), conductivit…)	Erros RMS relativos de grandezas baseadas em gradiente.
spectral_fidelity	função	(pred, ref, ndim, *, periodic=True, window='hann', bands=(1 / 3, 2 / 3), tol=0.25, spacin…)	Fidelidade espectral: aprova quando o viés de alta frequência fica dentro da tolerância.

pinneapple_analysis.validation.symmetry

Checagens de simetria (reflexão e rotação 2D) de uma PINN treinada.

pinneapple_analysis/validation/symmetry.py · 280 linhas · testes: 151

nome	tipo	assinatura	o que faz
SymmetryCheck	classe	(device='cpu') -> None	Checa propriedades de simetria espacial de um modelo treinado.

pinneapple_analysis.validation.validate

pp.validate: uma chamada escolhe as checagens que se aplicam ao modelo e ao problema, roda-as e devolve um ValidationReport com evidência, credibilidade por fator e decisão de implantação, em níveis quick, standard e thorough.

pinneapple_analysis/validation/validate.py · 759 linhas · testes: 23

nome	tipo	assinatura	o que faz
ValidationCheck	classe	(id, status, summary, value, threshold, evidence)	Uma checagem do relatório: ID, status, resumo, valor, limiar e evidência.
ValidationReport	classe	(model, problem, level, checks, provenance)	Relatório de pp.validate com decisão, veredito, credibilidade e saídas JSON, HTML e trust card.
as_field_model	função	(model) -> Callable[[np.ndarray], np.ndarray]	Converte modelo torch, ONNX ou callable num operador de campo em NumPy.
validate	função	(model, problem=None, data=None, reference=None, level='standard', *, query=None, method=…)	Valida um modelo num problema no nível pedido e devolve o ValidationReport.

pinneapple_analysis.validation.validator

`PhysicsValidator`: orquestra uma suíte de checagens (conservação, contorno, simetria, personalizada) sobre um modelo treinado, inclusive a partir de um `ProblemSpec`.

`pinneapple_analysis/validation/validator.py` · 357 linhas · testes: 151

nome	tipo	assinatura	o que faz
`PhysicsValidator`	classe	`(model, coord_names, domain_bounds, device='cpu') -> None`	Orquestra uma suíte de checagens físicas sobre um modelo treinado.
`validate_model`	função	`(model, spec, coord_names=None, domain_bounds=None, device='cpu') -> ValidationReport`	Validação rápida de uma vez a partir de um `ProblemSpec` ou argumentos explícitos.

pinneapple_analysis.verification

pinneapple_analysis.verification -- Physics verification building blocks.

`pinneapple_analysis/verification/__init__.py` · 366 linhas · **sem teste direto** · sinais no código: experimental

pinneapple_analysis.verification.architecture_critique

Adversarial review of a proposed Physics-AI architecture/pipeline design -- "assume the role of a Principal Engineer trying to prove this architecture is wrong" as a structured, checked-menu LLM tool, not free-form commentary.

`pinneapple_analysis/verification/architecture_critique.py` · 267 linhas · testes: 12

nome	tipo	assinatura	o que faz
`CritiqueFinding`	classe	`(category, verdict, severity, reasoning)`	
`AdversarialReviewReport`	classe	`(architecture_description, findings, overall_reasoning, provider, raw_llm_response)`	
`run_adversarial_review`	função	`(architecture_description, spec=None, *, provider='ollama', model=None, api_key=None, con…)`	Run a structured adversarial review of *architecture_description* (free-text: the proposed model/pipeline design, e.g. "FNO surrogate trained on 200 OpenFOAM runs, fine-tuned with a physics residual loss, deployed for real-time inference") against the fixed :data:`FAILURE_MODE_CHECKLIST`. See the …

pinneapple_analysis.verification.architecture_recommendation

Which PINNeAPPle neural architecture to reach for FIRST, given a problem's real characteristics -- data availability, whether an existing solver can generate more, whether the model needs to generalize across parameters/geometry, and whether the target is a forward or inverse problem. The sibling …

pinneapple_analysis/verification/architecture_recommendation.py · 453 linhas · testes: 19

nome	tipo	assinatura	o que faz
ArchitectureCandidate	classe	(name, registry_key, category, when_to_use, strengths, weaknesses, source, implementation…)	One named architecture family and when its own well-known design intent (cited in source) actually applies -- never an accuracy claim, only a documented-scope-match input (same discipline as solver_orchestration.ExternalTool/_CLASSICAL_SOLVER_SCOPE).
ArchitectureRecommendation	classe	(recommended, reasoning, decision_path, alternatives, caveats)	Result of :func:recommend_architecture. decision_path makes the reasoning auditable step by step (matching this whole engine's "explain, don't just assert" norm) rather than a single opaque pick.
recommend_architecture	função	(*, n_high_fidelity_simulations=0, has_lots_of_data=None, has_analytical_solver=False, ne…)	Recommend which PINNeAPPle neural architecture family to reach for first, from the problem's own real characteristics -- see the module docstring for the full decision structure and honesty scope (this never claims accuracy; only documented-design-intent match).

pinneapple_analysis.verification.causal_discrepancy

Two roadmap points from the product README's honest 21-point scope table:

pinneapple_analysis/verification/causal_discrepancy.py · 483 linhas · testes: 5

nome	tipo	assinatura	o que faz
InterventionRunResult	classe	(param_value, final_loss, loss_pct_change, residual_value, residual_pct_change, guardrail…)	
InterventionReport	classe	(base_description, param_name, baseline_rejected, rejection_reason, query_point, baseline…)	
DiscrepancyModel	classe	(physics_model, correction_model, in_dim, out_dim, n_train, n_test, variance_explained_te…)	y(x) ~= physics_model(x) + delta(x) -- Kennedy & O'Hagan (2001) model-form discrepancy. delta (correction_model) is a small MLP fit by MSE regression on the real residual reference_y - physics_model(reference_x).
run_intervention	função	(base_description, param_name, param_values, *, physical_parameters=None, provider='ollam…)	"What happens if I increase pressure 20%?" -- run the real pipeline once as a baseline, then once per value in param_values with param_name overridden in the drafted preset's kwargs (e.g. param_name="nu" for a Burgers preset's viscosity), and report REAL percentage changes vs. the …

nome	tipo	assinatura	o que faz
`fit_discrepancy_model`	função	`(spec, physics_model, reference_x, reference_y, *, hidden_dim=32, n_layers=2, epochs=400,…)`	Fit a small MLP discrepancy-correction model on the real residual between `reference_y` and `physics_model(reference_x)` (Kennedy & O'Hagan 2001's model-form discrepancy technique -- see this module's docstring and `DiscrepancyModel.reference`).

`pinneapple_analysis.verification.convergence`

Grid-convergence / mesh-independence study automation.

`pinneapple_analysis/verification/convergence.py` · 477 linhas · testes: 43

nome	tipo	assinatura	o que faz
`ConvergenceResult`	classe	`(observed_order, extrapolated_value, gci_fine, gci_coarse, asymptotic_ratio, is_asymptoti…)`	The result of a three-grid Richardson extrapolation / GCI analysis. See the module docstring's "Formula references" section for the exact formulas and citation (Roache, 1994).
`MeshIndependenceStudyResult`	classe	`(resolutions, values, refinement_ratios, refinement_ratio_used, convergence)`	A full mesh-independence study: the raw per-resolution values (for plotting a convergence curve) plus the three-finest-grid :class:`ConvergenceResult`.
`richardson_extrapolate`	função	`(f_coarse, f_medium, f_fine, r, *, p_assumed=None, safety_factor=_DEFAULT_SAFETY_FACTOR, …)`	Richardson-extrapolate a quantity of interest computed at three systematically-refined resolutions and compute Roache's (1994) Grid Convergence Index. See the module docstring for the exact formulas.
`mesh_independence_study`	função	`(solve_fn, resolutions, quantity_extractor, *, refinement_ratio=None, p_assumed=None, saf…)`	Run a mesh-independence / grid-convergence study by calling an arbitrary solver at increasing resolutions and Richardson-extrapolating the three finest results.

`pinneapple_analysis.verification.dimensional_analysis`

Real, first-principles dimensionless-number calculations + a simple, honestly-scoped flow-regime classifier.

`pinneapple_analysis/verification/dimensional_analysis.py` · 175 linhas · testes: 27

nome	tipo	assinatura	o que faz
`DimensionlessNumbers`	classe	`(reynolds, prandtl, peclet_heat, peclet_mass, mach, froude, weber, damkohler, nusselt_cor…)`	Every dimensionless group actually computable from the physical parameters supplied to :func:`compute_dimensionless_numbers` -- fields are `None` when the inputs needed for that particular number weren't given, rather than silently defaulted, so a caller can tell "not computed" apart from …
`compute_dimensionless_numbers`	função	`(*, velocity=None, length=None, density=None, dynamic_viscosity=None, kinematic_viscosity…)`	Compute every dimensionless number the supplied subset of parameters actually supports. All formulas are standard textbook definitions (e.g. Incropera & DeWitt, *Fundamentals of Heat and Mass Transfer*; White, *Viscous Fluid Flow*):

nome	tipo	assinatura	o que faz
`classify_flow_regime`	função	`(re, *, geometry='internal_pipe') -> str`	A simple, HONESTLY GEOMETRY-SPECIFIC flow-regime classification -- real transition Reynolds numbers depend on the flow geometry, so this function requires the caller to say which one its thresholds should assume, rather than presenting one universal answer.

`pinneapple_analysis.verification.evidence_graph`

A run-level Evidence Graph -- ties one analysis run's :class:`~pinneapple_analysis.verification.provenance.ProvenanceRecord` together with its (optional) real :class:`~pinneapple_analysis.verification.physics_confidence_score.PhysicsConfidenceScore` and any solver-vs-solver ...

`pinneapple_analysis/verification/evidence_graph.py` · 235 linhas · testes: 8

nome	tipo	assinatura	o que faz
`build_evidence_graph`	função	`(provenance, confidence=None, comparisons=None) -> 'nx.DiGraph'`	Build a small evidence graph for one run.
`evidence_summary`	função	`(g) -> Dict[str, Any]`	A structured summary of an evidence graph -- counts of evidence nodes by kind and by supports/contradicts, plus the claim's own stated trustworthiness. Never re-derives trustworthiness itself.
`explain_trust`	função	`(g) -> str`	A plain-text "why should I trust this" rendering of an evidence graph -- mirrors `knowledge_graph.explain_preset`'s pattern of producing an LLM-context-ready string from real graph contents, but grounded in one run's actual evidence instead of general physics knowledge. Never invents a reason not ...

`pinneapple_analysis.verification.forecast_skill`

Forecast skill against a reference forecast, with confidence intervals that respect serial correlation.

`pinneapple_analysis/verification/forecast_skill.py` · 96 linhas · testes: 19

nome	tipo	assinatura	o que faz
`SkillResult`	classe	`(leads, diff, skill_score, horizon, n_starts, block)`	
`block_bootstrap_indices`	função	`(n, block, n_boot, rng) -> np.ndarray`	(n_boot, n) resampled indices: moving blocks of `block` consecutive samples, concatenated and cut to `n`.
`block_bootstrap_mean`	função	`(d, block, n_boot=2000, seed=0, level=0.95) -> tuple[float, float, float]`	Mean of `d` along axis 0 and its `level` interval (moving-block bootstrap).

nome	tipo	assinatura	o que faz
`block_bootstrap_ratio`	função	`(a, b, block, n_boot=2000, seed=0, level=0.95) -> tuple[float, float, float]`	Skill score 1 - mean(a) / mean(b) and its interval (paired moving-block bootstrap).
`batch_means_ci`	função	`(x, n_batches=8, level=0.95) -> tuple[float, float]`	Mean of a correlated time series and the half-width of its interval from `n_batches` batch means (Student t with n_batches - 1 degrees of freedom). The batches must be longer than the correlation time.
`skill_vs_lead`	função	`(err_model, err_ref, leads, block, *, n_boot=2000, level=0.95, seed=0) -> SkillResult`	Paired comparison of two forecasts verified from the same starts. `err_*`: (starts, leads), e.g. the MAE of each forecast at each lead. The **horizon** is the last lead up to which the model is better than the reference at every lead with `level` confidence (0 if it is not better at the first …

`pinneapple_analysis.verification.geometry_intelligence`

Geometry Intelligence -- the concrete implementation of the founder's architectural point 4 ("CAD/mesh upload -> automatic semantic region labeling -> boundary conditions"), built entirely from real, already-tested PINNeAPPle geometry infrastructure …

`pinneapple_analysis/verification/geometry_intelligence.py` · 756 linhas · testes: 9

nome	tipo	assinatura	o que faz
`BoundaryLoop`	classe	`(n_vertices, closed, perimeter, is_planar, planarity_rms_deviation, enclosed_area, circul…)`	One closed boundary loop bounding a surface patch -- either a crease edge chain shared with a DIFFERENT patch, or a true open mesh-boundary edge chain (the patch has an actual hole/opening, no covering face at all -- the convention many real CFD surface meshes use to represent an inlet/outlet …
`SurfaceRegion`	classe	`(region_id, face_ids, n_faces, area, area_fraction_of_total_surface, area_fraction_of_lar…)`	One connected surface patch found by normal-vector-similarity segmentation, with every geometric property the classifier's heuristics (and, optionally, the LLM disambiguation step) actually use -- computed, real numbers, never placeholders.
`GeometryClassification`	classe	`(regions, semantic_labels, boundary_condition_types, ambiguous_region_ids, llm_used, llm_…)`	
`classify_geometry`	função	`(mesh, problem_description='', *, provider=None, model=None, api_key=None, angle_toleranc…)`	Classify every boundary surface of `mesh` into a physically- meaningful region. See the module docstring for the full two-stage design (real geometric segmentation, then optional LLM semantic disambiguation) and exactly which heuristics are used for each of the three geometric candidate labels …

pinneapple_analysis.verification.geometry_ood_guardrail

`GeometryOODGuardrail` -- flags when an input geometry is out-of-distribution (OOD) relative to the geometries a model/preset was actually designed and validated against.

`pinneapple_analysis/verification/geometry_ood_guardrail.py` · 456 linhas · testes: 39 · sinais no código: experimental

nome	tipo	assinatura	o que faz
`GeometryOODResult`	classe	`(score, mahalanobis_sq, dof, status, n_reference, feature_values, feature_z_scores, top_d…)`	Real, auditable output of one :meth:`GeometryOODGuardrail.query` call.
`GeometryOODGuardrail`	classe	`(*, warn_p=0.05, reject_p=0.01)`	Fits a per-feature Gaussian reference distribution over a set of real "in-distribution" geometries, and scores new geometries against it.
`extract_geometry_features`	função	`(mesh) -> Dict[str, float]`	Extract the 13 real, computed shape descriptors in :data:`FEATURE_NAMES` from a mesh.
`default_reference_meshes`	função	`() -> List[MeshData]`	A real, generic reference catalog built from `pinneapple_design.geometry.gen.primitives.build_mesh` -- box, cylinder, and channel domains (the base shapes several real presets in this catalog already use: box for cavity-type domains, cylinder for cylinder-in-crossflow/pipe domains, channel for …

pinneapple_analysis.verification.knowledge_graph

A real Physics Knowledge Graph, built directly from PINNeAPPle's own already-existing preset registry -- founder's roadmap point 16 ("Relacionando: Phenomenon -> Equation -> Assumptions -> Parameters -> Boundary conditions -> Numerical methods -> Solvers -> Benchmarks -> Experimental datasets -> …

`pinneapple_analysis/verification/knowledge_graph.py` · 644 linhas · testes: 14 · sinais no código: experimental; placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`build_knowledge_graph`	função	`() -> 'nx.MultiDiGraph'`	Build the real Physics Knowledge Graph fresh from the live preset registry + the live validation test files. Not a cached/serialized snapshot -- call this again any time the preset library changes and the graph reflects it immediately.
`find_presets_for_phenomenon`	função	`(g, phenomenon) -> List[str]`	Every real preset name whose equation belongs to `phenomenon`.
`find_verified_equations`	função	`(g) -> List[str]`	Every `pde_kind` with at least one real, AST-confirmed `verified_by` edge to a VerificationMethod.

nome	tipo	assinatura	o que faz
`explain_preset`	função	`(g, preset_name) -> Dict[str, Any]`	A structured, honest summary of one real preset -- its equation, phenomenon, real parameter defaults, real citation(s) if any, and whether/how its equation is verified. `llm_context` is a plain-text rendering meant to be dropped directly into an LLM prompt as grounding context.
`graph_stats`	função	`(g) -> Dict[str, Any]`	Real, honest audit counts -- not just graph-size trivia. In particular the citation and verification fractions are a genuine finding about PINNeAPPle's own documentation/testing completeness, not a knowledge-graph gimmick.

`pinneapple_analysis.verification.mesh_intelligence`

Mesh Intelligence -- the concrete implementation of the founder's architectural point 5 ("mesh quality, y+, is this mesh good enough").

`pinneapple_analysis/verification/mesh_intelligence.py` · 475 linhas · testes: 14

nome	tipo	assinatura	o que faz
`TriangleQualityStats`	classe	`(aspect_ratio_convention, aspect_ratio, aspect_ratio_mean, aspect_ratio_max, worst_aspect…)`	Per-triangle quality metrics and their summary statistics. Every array here has one entry per mesh face -- nothing is subsampled or approximated.
`OrthogonalityResult`	classe	`(computable, reason)`	Real orthogonal-quality (the angle between adjacent CELL centroids and their shared FACE normal) requires a volumetric cell mesh -- it is not a meaningful, well-defined quantity for a pure triangle-soup SURFACE mesh (no volume cells, no cell-to-cell adjacency of the kind the metric is actually …
`YPlusEstimate`	classe	`(computable, reason, target_yplus, reynolds_length, skin_friction_coefficient, wall_shear…)`	See the module docstring's 'y+ required first-cell height' section for the full formula chain and citations. Every field is `None` when its inputs weren't computable, exactly like `dimensional_analysis.DimensionlessNumbers` -- never silently defaulted.
`MeshQualityReport`	classe	`(n_faces, n_vertices, triangle_quality, orthogonality, yplus, is_good_enough, failures, t…)`	
`estimate_required_first_cell_height`	função	`(*, reynolds_length, velocity, kinematic_viscosity, target_yplus=1.0) -> YPlusEstimate`	Estimate the first-cell (near-wall prism-layer) height needed to reach `target_yplus` for a flat-plate turbulent boundary layer, from ONLY the flow's own Reynolds number/velocity/viscosity -- i.e. BEFORE a mesh even exists, exactly the way a CFD analyst sizes a boundary layer during mesh …
`assess_mesh_quality`	função	`(mesh, *, aspect_ratio_threshold=100.0, skewness_threshold=0.95, max_bad_element_fraction…)`	Assess whether `mesh` is "good enough" for the physics being asked about -- a structured report (which metric failed, by how much), never a single opaque boolean, per this product's own stated ethic (see `pipeline.py`'s `PhysicsGuardrail` usage for the same philosophy applied to physics …

pinneapple_analysis.verification.nozzle_crosscheck

Rocket nozzle screening by two methods that do not share code: a PINN and the ideal 1-D isentropic nozzle.

pinneapple_analysis/verification/nozzle_crosscheck.py · 146 linhas · testes: 3

nome	tipo	assinatura	o que faz
scaled_field_head	função	(net, scale, offset)	Wrap net so that y = net(x) * scale + offset per output field (fixed, non-trainable buffers).
train_nozzle_pinn	função	(*, throat_radius, exit_radius, nozzle_length, p_chamber, T_chamber, gamma, R_gas, epochs…)	Train the rocket_nozzle_cfd PINN, run the guardrail and return the exit-plane section averages.
screen_nozzle	função	(*, throat_radius, exit_radius, p_chamber, T_chamber, gamma=1.2, R_gas=287.0, nozzle_leng…)	Ideal-nozzle estimate and, with run_pinn, the PINN estimate and their disagreement.

pinneapple_analysis.verification.physics_confidence_score

PhysicsConfidenceScore: a transparent, componentized aggregate score built ONLY from real, already-executed checks -- never a black-box number.

pinneapple_analysis/verification/physics_confidence_score.py · 777 linhas · testes: 36

nome	tipo	assinatura	o que faz
CalibrationSummary	classe	(ece, coverage, target_coverage, sharpness)	A small, honest container for the real numbers a caller already computed via pinneapple_analysis.uncertainty.calibration.CalibrationMetrics's static methods.
ConfidenceComponent	classe	(name, score, source_summary)	One real, already-executed check turned into a 0-1 score by a plain, documented arithmetic formula (never fitted/learned/opaque).
PhysicsConfidenceScore	classe	(overall_score, components, coverage)	A transparent aggregate over whichever of the five real checks a caller actually ran. See this module's docstring for the full design rationale -- in short: overall_score is a plain arithmetic mean of components' scores (never a hidden weighting), components contains only the checks …
compute_physics_confidence	função	(*, guardrail_report=None, convergence_result=None, calibration_metrics=None, benchmark_c…)	Aggregate whichever real, already-executed checks a caller supplies into one transparent :class:PhysicsConfidenceScore.
score_against_reference	função	(name, x, y_ref, preds, *, x_vars, y_vars, sigma=None, convergence=None, guardrail=None, …)	Score several models' predictions against one held-out reference dataset.

pinneapple_analysis.verification.pipeline

Problem description -> dimensional analysis -> LLM-drafted preset -> PINN training -> guardrail -> selection.

pinneapple_analysis/verification/pipeline.py · 166 linhas · testes: 6

nome	tipo	assinatura	o que faz
CandidateResult	classe	(architecture, final_loss, guardrail_trustworthy, guardrail_checks, residual_value, model)	
AnalysisResult	classe	(provenance, best_candidate, all_candidates, rejected, rejection_reason)	
rank_key	função	(candidate) -> tuple	Sort key for physics-aware model selection: (untrustworthy?, re-computed residual, training loss).
guardrail_checks_as_dicts	função	(report) -> list[dict[str, Any]]	A GuardrailReport's checks as plain dicts (name, passed, detail, value, threshold).
analyze	função	(description, *, physical_parameters=None, flow_geometry='internal_pipe', provider='ollam…)	Run the pipeline in the module docstring for one description (blocking: training happens inside).

pinneapple_analysis.verification.provenance

A real, structured provenance record for one analysis run -- the concrete implementation of the user's architectural point 17 ("Provenance / reproducibility... cada resultado precisa ter um lineage"). Every field here is something the pipeline actually knows at run time (not a placeholder) -- …

pinneapple_analysis/verification/provenance.py · 67 linhas · testes: 8 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
ProvenanceRecord	classe	(run_id, created_unix, problem_description, dimensionless_numbers, flow_regime, drafted_p…)	

pinneapple_analysis.verification.solver_orchestration

Solver orchestration across PINNeAPPle's real solver families -- the concrete, honestly-scoped first pass at the user's architectural point 6 ("Solver orchestration across FEM/FVM/FDM/DEM/LBM/... families, with solver-vs-solver comparison"). Nothing here invents a new solver: every family this …

pinneapple_analysis/verification/solver_orchestration.py · 729 linhas · testes: 19

nome	tipo	assinatura	o que faz
SolverRecommendation	classe	(recommended_family, reasoning, matched_rule, fallback_families, caveats)	The result of :func:select_solver_family.

nome	tipo	assinatura	o que faz
`ComparisonReport`	classe	`(label_a, label_b, n_points, max_abs_diff, mean_abs_diff, rmse, relative_rmse, worst_inde…)`	A real, honest numeric comparison between two already-produced result arrays evaluated at the same query points. Every number here is a plain, correct array computation -- nothing is estimated or guessed.
`list_available_solver_families`	função	`() -> Dict[str, dict]`	Introspect, RIGHT NOW, which solver families this process can actually exercise on this machine. Every entry is computed live -- nothing here is a hardcoded table that could drift out of sync with what's actually installed.
`select_solver_family`	função	`(spec, dimensionless_numbers=None, *, catalog=None) -> SolverRecommendation`	Recommend which currently-available solver family to use for *spec* (a `pinneapple_physics.ProblemSpec`), and say exactly why.
`compare_solvers`	função	`(spec, model_or_result_a, model_or_result_b, *, query_points=None, labels=('solver_a', 's…)`	Compare two already-produced results for the SAME `ProblemSpec` *spec*, evaluated at the SAME set of query points, and report real, honest difference metrics.

`pinneapple_analysis.verification.tool_recommendation`

External tool/player recommendation -- the sibling of `solver_orchestration` for tools OUTSIDE PINNeAPPle: given a physics problem, which real, named CFD/FEM/multiphysics/meshing/visualization/UQ software (open-source or commercial) is actually a documented fit, and what are the real tradeoffs …

`pinneapple_analysis/verification/tool_recommendation.py` · 601 linhas · testes: 18

nome	tipo	assinatura	o que faz
`ExternalTool`	classe	`(name, category, license, typical_cost_tier, pde_kinds, dims, strengths, caveats, source)`	One real, named external tool and its documented scope.
`ToolRecommendation`	classe	`(recommended, reasoning, alternatives, caveats)`	Result of :func:`recommend_tools`. Mirrors `solver_orchestration.SolverRecommendation`'s shape/honesty contract, for the external-tool axis instead of PINNeAPPle's internal solver families.
`Player`	classe	`(name, focus_areas, tool_keys, notes)`	One real company/ecosystem and what it's known for. `tool_keys` links to `TOOL_CATALOG` entries this player makes/sells, where one exists in the (starter) catalog -- a player can be real and relevant with an empty `tool_keys` (e.g. a cloud/compute provider that hosts other vendors' software …
`recommend_tools`	função	`(spec, dimensionless_numbers=None, *, catalog=None, category=None) -> ToolRecommendation`	Recommend external tools whose documented scope matches *spec* (a `pinneapple_physics.ProblemSpec`), ranked the same documented-scope -matching way `solver_orchestration.select_solver_family` ranks PINNeAPPle's own internal solvers -- see the module docstring for exactly what this does and does …

nome	tipo	assinatura	o que faz
`get_tools_by_category`	função	`(category, *, catalog=None) -> List[str]`	List catalog tool keys in *category* -- the entry point for non-PDE-scoped categories (`"meshing"`, `"visualization"`, `"UQ"`) that :func:`recommend_tools` never scope-matches by design (a mesh generator or a UQ library doesn't "solve" a `pde.kind`).
`get_player_for_tool`	função	`(tool_key, *, player_catalog=None) -> Optional[str]`	Reverse lookup: which `PLAYER_CATALOG` key makes the given `TOOL_CATALOG` tool, if any (`None` for a community/consortium project with no single corporate player behind it, e.g. FEniCSx/Gmsh/ OpenFOAM's Foundation side/SU2/MOOSE/CalculiX/OpenTURNS).
`buy_vs_build_recommendation`	função	`(need, *, guidance=None) -> Optional[str]`	Look up qualitative buy-vs-build guidance for a named *need* (a key into :data:`BUY_VS_BUILD_GUIDANCE`, e.g. `"CFD_high_fidelity"`). Returns `None` (not a guess) for a need this starter table doesn't cover yet -- callers should extend `BUY_VS_BUILD_GUIDANCE` or pass a custom `guidance=` …
`list_buy_vs_build_needs`	função	`(*, guidance=None) -> List[str]`	

`pinneapple_analysis.verification.uncertainty_inverse`

Wiring PINNeAPPle's REAL uncertainty-quantification and inverse-problem machinery into the Physics Verification Engine pipeline -- roadmap points 13 ("Inverse modelling") and 14 ("Uncertainty Quantification: aleatoric / epistemic / parameter / model-form") from `README.md`'s scope table.

`pinneapple_analysis/verification/uncertainty_inverse.py` · 752 linhas · testes: 13 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`InverseParamAdapter`	classe	`(base, unknown_param_names, initial_guess=None) -> None`	Gives an arbitrary PINNeAPPle model an `inverse_params` `nn.ParameterDict` -- the real, documented calling convention `InverseProblemSolver` requires (see this module's docstring).
`UncertaintyReport`	classe	`(epistemic_std, aleatoric_std, parameter_std, total_std, method, n_samples, metadata)`	Structured UQ report distinguishing uncertainty *types*, not a single opaque confidence score -- the explicit value-add of roadmap point 14.
`InverseProblemResult`	classe	`(estimated_params, parameter_uncertainty, converged, final_misfit, method, n_iters, loss_…)`	Result of estimating unknown physical parameter(s) from observed data via the real `InverseProblemSolver`.
`quantify_uncertainty`	função	`(spec, model, *, method='ensemble', n_samples=20, models=None, x_eval=None, n_eval=200, s…)`	Quantify predictive uncertainty for a trained PINNeAPPle model, using the real `pinneapple_analysis.uncertainty` classes -- never a fabricated confidence number. See :class:`UncertaintyReport` for exactly which class computes which field, and which fields are honestly `None` for a given …

nome	tipo	assinatura	o que faz
`solve_inverse_problem`	função	`(spec, observed_data, unknown_param_names, *, model=None, initial_guess=None, n_outer=5, …)`	Estimate unknown physical parameter(s) from sparse observed data, using the real `pinneapple_analysis.inverse_problems.InverseProblemSolver` (Adam optimiser over `PointObsOperator` + `GaussianMisfit`), wired with an alternating field/parameter scheme -- see the module docstring for exactly ...

Exemplos de uso reais do repositório

Correntes parasitas: varredura sintética e treino da PINN — `examples/inspection/01_eddy_current_pinn.py`

```
data = generate_eddy_current_synthetic(
    nr=24, nz=40, scan_positions=np.linspace(-0.01, 0.01, 9),
)
...
out = train_eddy_current(
    synthetic_data=data,
    hidden=[64, 64, 64],
    epochs=60,
    lr=1e-3,
    batch_size=32,
    log_dir="examples/_out/inspection_eddy_current",
    run_name="eddy_current_demo",
)
```

Problema inverso a partir de um preset — `examples/quickstart/05_inverse_problem.py`

```
import pinneapple_analysis.inverse_problems as pinv

preset = pinv.get_preset("fin_h")
built = preset.build()
problem = pinv.InverseProblem(
    model=built["model"], observations=built["observations"],
    parameters=built["parameters"], validation=built["validation"],
name="fin_h",
)
result = pinv.solve_inverse(problem, "least_squares")
comparison = pinv.compare_methods(problem, methods=["least_squares",
"bayesian"])
print("best:", comparison["best"])
```

pp.validate num FNO exportado em ONNX (Darcy 2D) — `examples/validation/02_fno_darcy_onnx.py`

```
data = {"train": {"a": a[:n_train], "u": u[:n_train]}, "test": {"a":
a[n_train:], "u": u[n_train:]}}
good = pp.validate(str(path), "darcy_2d", data=data,
level="standard")
shifted, _ = darcy_set(8, n_grid, seed=7, high=100.0, low=1.0)
bad = pp.validate(str(path), "darcy_2d", data={"train":
data["train"]}, query=shifted, level="standard")
return conclude("02_fno_darcy_onnx", good, bad, expect=CATCHES,
family=FAMILY, physics=PHYSICS, out_dir=out_dir)
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 103 de 159.

- Funções de teste em arquivos que importam algum módulo do pacote: 939.

Limitações conhecidas

- `inspection`: todos os sinais são sintéticos e ilustrativos; as simplificações (EMAT escalar, Klein-Gordon no lugar de Rayleigh-Lamb, varredura axissimétrica em correntes parasitas, MFL sem termo cruzado, Helmholtz como regularizador no phased array) estão declaradas nos docstrings. Os submódulos não têm teste direto; só o pacote é importado por `tests/test_inspection.py`.
- `inverse_problems`: convivem duas camadas, o kit PyTorch antigo (`InverseProblemSolver`, misfits, regularizadores, EKI) e o núcleo novo em NumPy/SciPy (`forward`, `estimation`, `identifiability`, `calibration`, `studio`, `presets`); vários módulos do núcleo novo não têm teste direto.
- `inverse_problems.heat_transfer`: o caso 3D não é treinado sob demanda, é lido de resultados gravados; o docstring de `obs_operator` cita um `BoundaryObsOperator` inexistente.
- `validation`: há duas classes `ValidationReport` diferentes (`core.ValidationReport` do validador clássico e `validate.ValidationReport` de `pp.validate`).
- `validation.arena` e `validation.fault_injection`: só os pacotes são importados por testes; os submódulos (blue, red, loop, matrix, faults etc.) não têm teste direto, e só Darcy 2D e Burgers 1D vêm prontos.
- `validation.metamorphic`: a busca por propriedades usa `hypothesis` se instalado; senão cai para busca aleatória.

2. `pinneapple_check`

Objetivo

`pinneapple_check` é um linter de física para CI. Ele falha um pull request quando um resultado de simulação ou um modelo de Physics-AI viola a física: conservação, unidades, condições de contorno e iniciais, resíduo da EDP, limites, uso fora da distribuição de treino, regressão contra uma referência gravada e integridade dos artefatos. Cada violação tem um id de regra estável (`PC101`...), severidade, explicação e dica de correção.

O uso é pela linha de comando `pinneapple check [paths]` (ou `python -m pinneapple_check`), pela GitHub Action (`action.yml` na raiz do repositório) ou por Python com `pinneapple_check.run`. Funciona sem configuração (descobre casos OpenFOAM e arquivos de campo e infere a física pelos nomes) ou com `pinneapple-check.toml`. A saída sai em texto, JSON, SARIF, JUnit, anotações do GitHub ou resumo Markdown, e uma baseline permite adotar o checador num repositório que já tem violações.

O pacote importa só NumPy na partida e reaproveita módulos de outros pacotes sem executar seus `__init__` (`_reuse`): leitores OpenFOAM e de unidades de `pinneapple_data`, o envelope de treino de `pinneapple_physics.physics_audit`, métricas de `pinneapple_analysis`, o catálogo de métodos (`pinneapple_catalog`, regra PC901), `pinneapple_security` para hashes, assinaturas e varredura de checkpoints, e os estágios do trust card de `pinneapple_lab`. O relatório vira

evidência para o trust card e para o mapa de aplicabilidade de `pinneapple_veriphysics`. O mesmo `cli.main` serve de despachante do comando `pinneapple` (doctor, recommend, card, new, design, benchmark).

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
23	4.547	22	71	144 (12 arquivos)	11

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`	Entrada `python -m pinneapple_check`.	1	0	0
`_reuse`	Importa módulos isolados de outros pacotes PINNeAPPle sem seus `__init__`.	1	14	0
`balances`	Balanços globais de massa, calor e força em grades tensoriais.	1	3	0
`baseline`	Baseline para falhar só em violações novas.	1	3	0
`checks`	As checagens de resultados, modelos, datasets e integridade.	6	15	22
`cli`	Linha de comando `pinneapple check` e despachante `pinneapple`.	1	2	54
`config`	Leitura de `pinneapple-check.toml` e descoberta de alvos.	1	10	5
`engine`	Orquestra configuração, alvos e resultados num `Report`.	1	1	5
`evidence`	Trust card e checagens de salvaguarda para o veriphysics.	1	2	22
`fields`	Leitura de resultados em `FieldData`.	1	12	16
`fv`	Operadores e resíduos de volumes finitos em malhas OpenFOAM.	1	4	16
`models`	Carga segura de modelos como funções.	1	5	22
`numerics`	Derivadas em grade, nuvem de pontos e modelo.	1	5	5
`physics`	Equações governantes e tabela de parâmetros.	1	4	5
`report`	Formatos de saída (texto, JSON, SARIF, JUnit, GitHub, resumo).	1	7	22
`result`	`Outcome` e `Report`.	1	3	0
`rules`	Tabela de regras PC com seleção por prefixo.	1	3	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_check`

Ponto de entrada do `pinneapple check`, um linter de física para CI: reexporta `run` e as classes de relatório; importa só NumPy na partida.

`pinneapple_check/__init__.py` · 39 linhas · testes: 74

pinneapple_check.__main__

Permite `python -m pinneapple_check [paths]`, equivalente a `pinneapple check [paths]`. [sem teste direto]

`pinneapple_check/__main__.py` · 8 linhas · **sem teste direto**

pinneapple_check._reuse

Carrega módulos isolados de outros pacotes (leitores OpenFOAM e de unidades do `pinneapple_data`, envelope do `physics_audit`, métricas do `pinneapple_analysis`, catálogo, estágios do `pinneapple_lab`, checagens Darcy/Maxwell) sem executar o `__init__` desses pacotes. [sem teste direto]

`pinneapple_check/_reuse.py` · 98 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`leaf`	função	`(dotted) -> types.ModuleType`	Importa um módulo pelo nome pontilhado sem executar o `__init__` dos pacotes pais.
`foam`	função	`()`	Leitor OpenFOAM de `pinneapple_data.cae.foam`.
`cae_io`	função	`()`	Módulo de E/S CAE de `pinneapple_data.cae.io`.
`cae_geometry`	função	`()`	Geometria de malha de `pinneapple_data.cae.geometry`.
`units`	função	`()`	Unidades físicas de `pinneapple_data.physical_units`.
`envelope_cls`	função	`()`	Classe `Envelope` de `pinneapple_physics.physics_audit.assumptions`.
`field_metrics`	função	`()`	Métricas de campo de `pinneapple_analysis.validation.field_metrics`.
`spectral`	função	`()`	Métricas espectrais de `pinneapple_analysis.validation.spectral`.
`metamorphic`	função	`()`	Testes metamórficos de `pinneapple_analysis.validation.metamorphic`.
`dataset_audit`	função	`()`	Auditoria de dataset de `pinneapple_data.audit`.
`method_catalog`	função	`()`	Catálogo de métodos de `pinneapple_catalog.methods`.
`trust_stages`	função	`()`	Estágios do trust card (`STAGES`) de `pinneapple_lab.curation`.
`darcy_checks`	função	`()`	Checagens de Darcy de `pinneapple_physics.darcy_checks`.
`maxwell_checks`	função	`()`	Checagens de Maxwell de `pinneapple_physics.maxwell_checks`.

pinneapple_check.balances

Balanços globais na fronteira de uma caixa em grade tensorial: fluxo líquido de volume, fluxo líquido de calor e força resultante, cada um com o fluxo absoluto total para normalizar. [sem teste direto]

`pinneapple_check/balances.py` · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`grid_mass`	função	`(coords, names, vel) -> dict`	Fluxo líquido de volume pela fronteira da grade e o fluxo absoluto total.
`grid_heat`	função	`(coords, names, T, k, q) -> dict`	Fluxo de calor que entra no domínio (k dT/dn) mais a fonte.

nome	tipo	assinatura	o que faz
`grid_force`	função	`(coords, names, disp, E, nu, f) -> dict`	Resultante das trações de fronteira σ·n mais a força de corpo (deformação plana em 2D).

`pinneapple_check.baseline`

Linha de base: adota o checador num repositório que já tem violações e falha só nas novas. [sem teste direto]

`pinneapple_check/baseline.py` · 34 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_baseline`	função	`(path) -> set[str]`	Lê o arquivo de baseline como conjunto de impressões de violações.
`write_baseline`	função	`(report, path) -> int`	Grava as violações atuais do relatório como baseline e devolve quantas.
`apply_baseline`	função	`(report, path) -> None`	Marca no relatório as violações que já estão na baseline.

`pinneapple_check.checks`

Pacote das checagens executadas por `run`: resultados, modelos, conjuntos de dados e integridade de artefatos. [sem teste direto]

`pinneapple_check/checks/__init__.py` · 9 linhas · **sem teste direto**

`pinneapple_check.checks.common`

Checagens comuns a resultados e modelos: valores finitos (PC403), limites declarados (PC401), admissibilidade física (PC402), unidades e referências. [sem teste direto]

`pinneapple_check/checks/common.py` · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Ctx`	classe	`(cfg, target, enabled, outcomes)`	Contexto de um alvo onde os resultados são coletados, com regras ativas, tolerâncias e localização.
`fmt`	função	`(x) -> str`	Formata um número para mensagens.
`admissible`	função	`(name, unit) -> tuple[float | None, float | None, str] | None`	Limites físicos rígidos (baixo, alto, descrição) inferidos do nome e da unidade de um campo.
`check_values`	função	`(ctx, name, v, unit, loc, where=None) -> None`	Aplica PC403 (não finito), PC401 (limites declarados) e PC402 (admissibilidade) a um campo.
`check_units`	função	`(ctx, name, file_unit, declared, values, loc, used) -> None`	Confere a unidade do arquivo contra a declarada e a plausibilidade das magnitudes (regras PC5xx).
`rel_l2`	função	`(a, b) -> float`	Erro L2 relativo entre dois arrays.

`pinneapple_check.checks.datasets`

Checagens de um conjunto de treino (`[[dataset]]`) usando a auditoria de `pinneapple_data.audit`: regras PC451 (física/restrições), PC452 (diversidade/duplicatas) e PC453 (vazamento treino/teste). [sem teste direto]

`pinneapple_check/checks/datasets.py` · 53 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_dataset`	função	`(ctx, t) -> None`	Roda a auditoria física de um conjunto de treino e emite PC451-PC453.

`pinneapple_check.checks.integrity`

Integridade de artefatos com `pinneapple_security`: manifestos SHA-256 (PC801, PC804), assinaturas Ed25519 destacadas (PC802) e pins de hash por alvo. [sem teste direto]

`pinneapple_check/checks/integrity.py` · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_pin`	função	`(ctx, path, expected) -> None`	Confere o hash SHA-256 de um arquivo contra o pin declarado.
`check_integrity`	função	`(ctx, cfg) -> None`	Confere manifestos e assinaturas declarados na configuração.

`pinneapple_check.checks.models`

Checagens de um modelo: carga segura, status do método no catálogo, entradas dentro da faixa de treino, física das predições em pontos amostrados (resíduo, divergência, balanços), condições de contorno/iniciais e regressão. [sem teste direto]

`pinneapple_check/checks/models.py` · 397 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`read_table`	função	`(path) -> tuple[dict[str, np.ndarray], dict[str, int]]`	Lê colunas de um CSV ou arrays de um NPZ, com o número de linha de cada registro do CSV.
`check_model`	função	`(ctx, t) -> LoadedModel | None`	Carrega e checa um alvo de modelo, devolvendo o `LoadedModel` ou None.

`pinneapple_check.checks.results`

Checagens de um resultado gravado (caso OpenFOAM, grade, nuvem de pontos, tabela de QoI): valores, unidades, contorno, resíduos da EDP, balanços e regressão contra referência.

`pinneapple_check/checks/results.py` · 707 linhas · testes: 22

nome	tipo	assinatura	o que faz
`evaluate_expr`	função	`(value, coords, n) -> np.ndarray`	Avalia um valor declarado: número, vetor ou expressão das coordenadas (ex.: 'sin(pi*x)').
`check_result`	função	`(ctx, t) -> FieldData | None`	Carrega e checa um alvo de resultado, devolvendo o `FieldData` ou None.
`report_equations`	função	`(ctx, eqs, why, where, rule_div, rule_res, tol_div, tol_res, loc, subject_prefix='') -> N…)`	Emite os resultados de divergência e resíduo de um conjunto de equações.
`check_balance`	função	`(ctx, rule, what, b, tol, loc, unit='', closed_scale=None, subject=None) -> None`	Emite o resultado de um balanço global contra a tolerância.

pinneapple_check.cli

Linha de comando `pinneapple check [paths]` (formatos, seleção de regras, baseline, `--explain`) e o despachante `pinneapple <comando>` (doctor/recommend/card/new, check, design, benchmark).

`pinneapple_check/cli.py` · 207 linhas · testes: 54

nome	tipo	assinatura	o que faz
`check_main`	função	`(argv=None) -> int`	Executa `pinneapple check` com os argumentos e devolve o código de saída.
`main`	função	`(argv=None) -> int`	Despacha `pinneapple <comando>`: doctor/recommend/card/new, check, e design/benchmark quando instalados.

pinneapple_check.config

Configuração por `pinneapple-check.toml` (ou `[tool.pinneapple.check]` no `pyproject.toml`): alvos de resultado, modelo, dataset e integridade, seleção de regras, tolerâncias e descoberta automática sem configuração.

`pinneapple_check/config.py` · 534 linhas · testes: 5

nome	tipo	assinatura	o que faz
`ConfigError`	classe	`()`	Erro de configuração inutilizável (código de saída 2).
`Reference`	classe	`(path, fields, rel_l2, qoi)`	Referência gravada para regressão: caminho, campos, tolerância L2 relativa e QoIs.
`ResultTarget`	classe	`(kind, time, fields, flux, conserved, envelope)`	Alvo de resultado: tipo, tempo, campos, fluxos, quantidades conservadas e envelope.
`ModelTarget`	classe	`(architecture, model_config, hub, zoo, inputs, outputs, train_range, train_samples, queri…)`	Alvo de modelo: arquitetura, hub/zoo, entradas/saídas, faixa de treino, consultas, domínio, método e relações.
`DatasetTarget`	classe	`(zoo, inputs, constraints, train, test, audit)`	Alvo de dataset: zoo, entradas, restrições, partições treino/teste e opções de auditoria.
`Integrity`	classe	`(manifests, public_key, signed, line)`	Configuração de integridade: manifestos, chave pública e arquivos assinados.
`Config`	classe	`(root, file, select, ignore, extend_select, fail_on, baseline, severity, exclude, results…)`	Configuração completa: raiz, seleção/ignorar de regras, fail_on, baseline, severidades, exclusões e alvos.
`find_config`	função	`(start) -> tuple[Path | None, dict[str, Any]]`	Sobe a partir de `start` até o primeiro `pinneapple-check.toml` ou `pyproject.toml` com a seção do checador.
`load_config`	função	`(file=None, start=None, root=None) -> Config`	Interpreta um arquivo de configuração (encontrado a partir de `start` se não for dado).
`discover`	função	`(paths, exclude, root) -> list[ResultTarget]`	Descobre resultados sob os caminhos quando a configuração não declara nenhum: casos OpenFOAM e arquivos de campo.

pinneapple_check.engine

Executa as checagens: configuração -> alvos -> resultados (`Outcome`) -> `Report`, aplicando seleção de regras e baseline.

`pinneapple_check/engine.py` · 93 linhas · testes: 5

nome	tipo	assinatura	o que faz
`run`	função	`(paths=None, *, config=None, select=None, ignore=None, extend_select=None, baseline=None,…)`	Checa os alvos da configuração e devolve o `Report`.

pinneapple_check.evidence

Traduz o relatório em evidência: trust card por alvo (os seis estágios de `pinneapple_lab.curation`) e as checagens de salvaguarda lidas por `pinneapple_veriphysics.applicability.build_applicability_map`.

`pinneapple_check/evidence.py` · 66 linhas · testes: 22

nome	tipo	assinatura	o que faz
`trust_card`	função	`(report) -> dict[str, dict[str, Any]]`	Trust card por alvo: para cada estágio, título, pergunta, status e evidências.
`guardrail_checks`	função	`(report, target) -> list[dict[str, Any]]`	Lista `{name, passed, detail}` para o veriphysics, uma entrada por nome de evidência.

pinneapple_check.fields

Lê resultados de qualquer formato (OpenFOAM, NPZ/HDF5, meshio, tabela de QoI) para uma estrutura única, `FieldData`.

`pinneapple_check/fields.py` · 434 linhas · testes: 16

nome	tipo	assinatura	o que faz
`LoadError`	classe	`()`	Erro ao ler um resultado.
`FieldData`	classe	`(kind, fields, units, files, source, axes, points, mesh, geom, boundary, patches, phi, ti…)`	Estrutura única de um resultado: campos, unidades, eixos ou pontos, malha, contorno, fluxo `phi`, tempo, snapshots e QoIs.
`scalar`	função	`(v) -> np.ndarray`	Converte um valor em array escalar.
`velocity_components`	função	`(fd) -> list[np.ndarray] | None`	Velocidade como lista de componentes (de `U`/`u` ou de escalares `u, v, w`).
`velocity_name`	função	`(fd) -> str | None`	Nome do campo de velocidade encontrado, ou None.
`load_result`	função	`(path, kind=None, time=None, fields=None) -> FieldData`	Lê um resultado escolhendo o leitor pelo tipo ou pela extensão.
`load_openfoam`	função	`(case, time=None, only=None) -> FieldData`	Lê um caso OpenFOAM (malha, campos, patches, `phi`) num instante.
`fv_geometry`	função	`(mesh) -> dict[str, np.ndarray]`	Centros e vetores de área das faces, centros e volumes das células, e direções resolvidas.

nome	tipo	assinatura	o que faz
`read_surface_scalar`	função	`(b, poly) -> tuple[np.ndarray, np.ndarray]`	Lê um `surfaceScalarField` (`phi`) nas faces internas e de fronteira.
`load_arrays`	função	`(path, kind=None) -> FieldData`	Lê NPZ/HDF5 como grade (se houver coordenadas 1D) ou nuvem de pontos, com unidades opcionais.
`load_meshio`	função	`(path) -> FieldData`	Lê uma malha via meshio para `FieldData`.
`load_qoi`	função	`(path) -> FieldData`	Lê uma tabela de quantidades de interesse.

`pinneapple_check.fv`

Operadores de volumes finitos sobre uma malha OpenFOAM (teorema de Gauss nas faces de cada célula) e os resíduos conservativos e balanços de fronteira construídos sobre eles.

`pinneapple_check/fv.py` · 189 linhas · testes: 16

nome	tipo	assinatura	o que faz
`FV`	classe	`(fd)`	Operadores de volumes finitos (fluxo, gradiente, laplaciano, somas por patch) sobre uma malha OpenFOAM.
`fv_residuals`	função	`(fv, physics, p)`	Resíduos por célula (por unidade de volume) das equações da física, com os motivos das que não rodaram.
`fv_mass_balance`	função	`(fv) -> dict[str, Any] | None`	Balanço de massa na fronteira usando `phi` ou a velocidade interpolada.
`fv_heat_balance`	função	`(fv, p) -> dict[str, Any] | None`	Balanço de calor na fronteira da malha.

`pinneapple_check.models`

Carrega um modelo como função simples `(N, n_inputs) -> {saída: (N,)}`, com varredura de segurança do checkpoint antes de carregar.

`pinneapple_check/models.py` · 192 linhas · testes: 22

nome	tipo	assinatura	o que faz
`ModelLoadError`	classe	`()`	Erro ao carregar um modelo.
`UnsafeCheckpoint`	classe	`(report)`	Checkpoint recusado pela varredura de segurança, com o relatório.
`LoadedModel`	classe	`(fn, inputs, outputs, train_range=None, file=None)`	Modelo carregado como função, com entradas, saídas, faixa de treino e arquivo.
`load_model`	função	`(target) -> LoadedModel`	Carrega o modelo de um alvo (checkpoint, hub ou zoo).
`scan`	função	`(p)`	Varre um checkpoint com `pinneapple_security` antes de carregar.

pinneapple_check.numerics

Derivadas de campos escalares nomeados em três discretizações (grade, nuvem de pontos por mínimos quadrados, modelo por diferenças centrais) atrás de uma interface única.

`pinneapple_check/numerics.py` · 243 linhas · testes: 5

nome	tipo	assinatura	o que faz
`GridD`	classe	`(axes, comps)`	Derivadas por diferenças finitas numa grade tensorial.
`PointD`	classe	`(points, comps, axes, k=None)`	Derivadas por ajuste quadrático de mínimos quadrados ponderados em nuvem de pontos; pontos na casca ficam de fora.
`ModelD`	classe	`(f, X, inputs, h, spans=None)`	Derivadas por diferenças centrais de um modelo nos pontos `X`.
`components`	função	`(fields, vector_ndim, ncomp=3) -> dict[str, np.ndarray]`	Separa campos vetoriais em escalares nomeados (`U` -> `u, v, w`).
`rel_rms`	função	`(residual, terms, mask=None, floor=0.0) -> float`	RMS do resíduo sobre RMS da soma dos termos absolutos (0 = exato, 1 = termos não se equilibram).

pinneapple_check.physics

Equações governantes (escoamento incompressível, calor, elasticidade, conservação, Darcy, Maxwell) escritas sobre os provedores de derivadas, mais a tabela de parâmetros com dimensões e padrões; o docstring ainda diz 'quatro físicas'.

`pinneapple_check/physics.py` · 195 linhas · testes: 5

nome	tipo	assinatura	o que faz
`Equation`	classe	`(name, residual, terms, mask, description, scale)`	Uma equação avaliada: nome, resíduo, termos, máscara, descrição e escala.
`parse_parameters`	função	`(physics, given) -> tuple[dict[str, Any], list[tuple[str, str]]]`	Converte parâmetros para SI e lista os de dimensão errada (regra PC504).
`infer_physics`	função	`(names) -> str | None`	Infere a física só pelos nomes dos campos (ex.: U e p -> escoamento incompressível).
`equations`	função	`(physics, D, p, conserved=None) -> tuple[list[Equation], list[str]]`	Resíduos da física num provedor de derivadas, com os motivos das equações que não puderam rodar.

pinneapple_check.report

Formatos de saída: `text`, `json`, `sarif` (GitHub code scanning), `junit`, `github` (anotações) e `summary` em Markdown para o job summary.

`pinneapple_check/report/__init__.py` · 216 linhas · testes: 22

nome	tipo	assinatura	o que faz
`render`	função	`(report, fmt, **kw) -> str`	Renderiza o relatório no formato pedido.
`text`	função	`(report, color=None, show_passed=False, show_fixes=True, **_) -> str`	Saída de texto legível, com cores e dicas de correção.

nome	tipo	assinatura	o que faz
`to_json`	função	`(report, **_) -> str`	Saída JSON do relatório.
`sarif`	função	`(report, **_) -> str`	Saída SARIF para o GitHub code scanning.
`junit`	função	`(report, **_) -> str`	Saída JUnit XML.
`github`	função	`(report, **_) -> str`	Anotações por workflow commands do GitHub.
`summary`	função	`(report, **_) -> str`	Resumo em Markdown para o job summary do GitHub Action.

`pinneapple_check.result`

O que uma checagem produz: um `Outcome` por (regra, alvo, sujeito), aprovado ou não, e o `Report` com código de saída. [sem teste direto]

`pinneapple_check/result.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Location`	classe	`(file, line, column)`	Arquivo, linha e coluna de uma violação.
`Outcome`	classe	`(rule, target, subject, passed, message, location, value, threshold, severity, baselined,…)`	Uma avaliação de regra: aprovada ou violação, mensagem, valor, limite, severidade e dica.
`Report`	classe	`(outcomes, targets, config_file, root, notes)`	Relatório com todos os resultados, violações, erros, avisos, código de saída e trust card.

`pinneapple_check.rules`

Tabela de regras do `pinneapple check`: cada regra tem id (`PC101`), nome, severidade padrão, estágio do trust card, explicação e dica de correção; seleção por prefixos no estilo ruff. [sem teste direto]

`pinneapple_check/rules.py` · 290 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Rule`	classe	`(id, name, title, severity, stage, evidence, explanation, fix)`	Regra do checador: id, nome, severidade, estágio do trust card, explicação e correção.
`get_rule`	função	`(rule_id) -> Rule`	Busca uma regra pelo id.
`select_rules`	função	`(select=None, ignore=None, extend_select=None) -> set[str]`	Ids de regras ativas por prefixos no estilo ruff (`select`, `ignore`, `extend_select`).

Exemplos de uso reais do repositório

Uso por Python (docstring do pacote) — `pinneapple_check/__init__.py`

```python
from pinneapple_check import run
report = run(["cases/"])
assert not report.errors, [o.message for o in report.errors]
```

GitHub Action e linha de comando — `README.md`

```
        # .github/workflows/physics-check.yml, in a job with `permissions:
security-events: write`
        - uses: actions/checkout@v4
        - uses: PINNeAPPle-Labs/PINNeAPPle@main    # pin a tag or a commit
SHA

        pinneapple check cases/ models/            # locally: same rules,
readable output, exit code 1 on a violation
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 12 de 23.
- Funções de teste em arquivos que importam algum módulo do pacote: 144.

Limitações conhecidas

- Os resíduos e balanços têm erro de discretização próprio; em malhas grossas o valor é dominado por ele, por isso as tolerâncias padrão são frouxas (docs/guides/validation_methods_reference.md).
- Condições de contorno checadas são só Dirichlet; Neumann só aparece pelos balanços. Um resíduo pequeno não detecta condição de contorno ou parâmetro errado que a EDP ainda satisfaz.
- A detecção de fora-da-distribuição (PC601/PC602) usa a caixa de treino e distâncias padronizadas; não cobre bem entradas correlacionadas.
- Unidades precisam estar no arquivo ou declaradas; a admissibilidade (PC402) infere a grandeza pelo nome e unidade do campo.
- torch só é necessário para carregar checkpoints `.pt`; `design` e `benchmark` no despachante só funcionam se esses pacotes estiverem instalados.
- Muitos submódulos (`checks.*`, `rules`, `result`, `baseline`, `balances`, `_reuse`) não têm teste que os importe diretamente; são exercitados pelos testes de `run`/CLI.
- O docstring de `physics` fala em 'quatro físicas', mas o código trata seis (escoamento incompressível, calor, elasticidade, conservação, Darcy, Maxwell).

3. `pinneapple_veriphysics`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
13	3.411	17	38	98 (11 arquivos)	1

Pasta de testes dedicada: `tests/pinneapple_veriphysics` (49 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
applicability		1	5	37
decision		1	1	18
evidence_package		1	9	22
evidence_report		1	1	18
example_cases		1	2	0
execution_log		1	3	5
ledger		1	8	22
mesh_views		1	1	3
recommend		1	3	1
robustness		1	6	5
scope		1	4	5
validation_lab		1	12	42

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_veriphysics`

pinneapple_veriphysics — the verification and evidence layer of Veriphysics, as part of PINNeAPPle.

`pinneapple_veriphysics/__init__.py` · 35 linhas · testes: 8

`pinneapple_veriphysics.applicability`

Applicability map: a structured evidence chain for one Veriphysics run.

`pinneapple_veriphysics/applicability.py` · 488 linhas · testes: 37

nome	tipo	assinatura	o que faz
Tolerances	classe	(plateau_rel, spread_rel, ece, coverage)	Acceptance tolerances of the applicability map (#6). The defaults are the heuristics above; a request can replace any of them with criteria agreed for the target application. The map records which were used and whether they came from the request.
ChecklistItem	classe	(key, question, status, tier, summary, evidence)	
VariableEnvelope	classe	(name, kind, verified_range, verified_point, extrapolation_supported, note)	
ApplicabilityMap	classe	(checklist, envelope, verified, inferred, unsupported, tolerances)	
build_applicability_map	função	(*, architecture, coords, fields, domain_bounds, physical_parameters, guardrail_checks, c…)	Classify the facts of one run. `heldout_error` (optional) is `{"metric": str, "value": float, "threshold": float, "description": str}` from an out-of-sample test the caller ran. `extrapolation`, `ensemble` and `weight_sensitivity` are the dicts returned by …

pinneapple_veriphysics.decision

`DecisionRecord` -- the human-facing output of one Veriphysics run: "here's what I recommend, here's the evidence, here's what I'd try next if you disagree." Built ONLY from real objects the orchestrator already produced (a `ProvenanceRecord`, a ...

`pinneapple_veriphysics/decision.py` · 157 linhas · testes: 18

nome	tipo	assinatura	o que faz
`DecisionRecord`	classe	`(run_id, problem_description, recommended_strategy, reasoning, trust_score, trust_coverag…)`	

pinneapple_veriphysics.evidence_package

Engineering evidence: turn experiments and simulations into a reproducible report and a zip package.

`pinneapple_veriphysics/evidence_package.py` · 638 linhas · testes: 22

nome	tipo	assinatura	o que faz
`Source`	classe	`(name, kind, sha256, bytes, raw, url, data, contract, problems)`	
`load_source`	função	`(name, data, url='') -> Source`	Read one file and decide what it is (never fails: unknown content is kept as a referenced file).
`build_report`	função	`(meta, sources) -> dict[str, Any]`	The report as data. `meta`: title, project, author, purpose, methodology, conditions (dict), limitations (list), baseline and candidate (labels matched against run names), references (list).
`make_figures`	função	`(report, outdir) -> dict[str, str]`	PNG figures from the report's own numbers: baseline vs candidate metrics, test status per group.
`render_html`	função	`(report, figures=None) -> str`	Self-contained HTML (figures inlined as PNG data URIs).
`render_pdf`	função	`(report, figures=None, path=None) -> bytes`	PDF through :class:`pinneapple_tools.reporting.paperkit.Paper` (needs reportlab).
`to_study_result`	função	`(report, app='evidence') -> dict[str, Any]`	The report as a `StudyResult` (`pinneapple_data.contracts`, study type `evidence`) for the next app: one check per test (PASS / FAIL with its record as evidence, NOT_RUN otherwise), the test counts as metrics, the artifacts by SHA-256 and the limitations as limits.
`build_package`	função	`(report, sources, *, html=None, pdf=None, figures=None) -> bytes`	Zip: report.json/html/pdf, evidence-ledger.json, figures/, sources/ (the original bytes) and MANIFEST.json with the SHA-256 of every other file.
`verify_package`	função	`(data) -> list[str]`	Problems found in an evidence package (empty list: every file matches the manifest).

pinneapple_veriphysics.evidence_report

Evidence Report PDF generator.

`pinneapple_veriphysics/evidence_report.py` · 206 linhas · testes: 18

nome	tipo	assinatura	o que faz
`render_evidence_report_pdf`	função	`(decision) -> bytes`	Render *decision* (a real, already-computed `DecisionRecord`) as an Evidence Report PDF and return the raw PDF bytes.

pinneapple_veriphysics.example_cases

Fixed catalog of pre-defined "example case" problems -- the "small, fixed set of 2-3 example problem classes" the MVP 1 scope freeze requires (`helm/docs/PINNeAPPle90/sprint1/04-escopo-minimo-mvps.md`): Burgers 1D (already existed) plus two turbomachinery cases (added here) that run for real ...

`pinneapple_veriphysics/example_cases.py` · 228 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ExampleCase`	classe	`(key, label, preset, preset_kwargs, description, physical_parameters, flow_geometry, epoc…)`	
`get_example_case`	função	`(key) -> ExampleCase`	Look up a registered example case by key, or raise `ValueError` naming the actual registered keys -- mechanically checked, never a silent fallback to a different case than the caller asked for.

pinneapple_veriphysics.execution_log

A real, thread-safe execution log collector -- so a caller (the API, the UI) can see WHAT the orchestrated pipeline is actually doing while it runs (which step it's on, which tool/solver/architecture it just picked), not just its final result. Every entry is written by the pipeline itself, at the ...

`pinneapple_veriphysics/execution_log.py` · 51 linhas · testes: 5

nome	tipo	assinatura	o que faz
`LogEntry`	classe	`(timestamp_unix, message)`	
`LogCollector`	classe	`() -> None`	Pass an instance of this into `analyze()`/`formulate_and_recommend()` (via their optional `log=` parameter) to observe the real steps they take as they take them. Safe to read from another thread while the pipeline is still running (e.g. an API handler polling it).
`emit`	função	`(log, message) -> None`	Convenience helper: log to *log* if one was actually given, otherwise a real no-op -- every call site in the pipeline can call this unconditionally without an `if log is not None` guard.

pinneapple_veriphysics.ledger

Evidence ledger: a test may be reported as passed only when a record shows that it ran.

`pinneapple_veriphysics/ledger.py` · 242 linhas · testes: 22

nome	tipo	assinatura	o que faz
EvidenceEntry	classe	`(check_id, name, category, tool, tool_version, inputs_sha256, outputs, started_at, finish…)`	One recorded execution of a check. `passed` is None while it runs or when it crashed.
EvidenceLedger	classe	`(entries)`	An append-only list of :class:`EvidenceEntry`. Several entries may share a `check_id` (re-runs): the status of a check is that of its latest valid entry.
canonical_json	função	`(obj) -> str`	Sorted keys, no whitespace: the same object always gives the same bytes.
sha256_bytes	função	`(data) -> str`	
sha256_json	função	`(obj) -> str`	
sha256_file	função	`(path, chunk=1 << 20) -> str`	
utc_now	função	`(t=None) -> str`	
record_checks	função	`(ledger, checks, *, prefix, inputs_sha256, started_at, finished_at, tool='', tool_version…)`	Record the checks of a trust card (`pp.trust_card(...).checks` or its `to_dict()["checks"]`): one entry per check that ran, under `f"{prefix}/{name}"`. A check with `passed is None` (N/A) did not run and gets no entry, so it reads "not run".

pinneapple_veriphysics.mesh_views

3-D views of the reference comparisons of a Validation Lab run: for every result with a reference whose data is 3-D and lives on a mesh (OpenFOAM case, VTU/VTK with cells), the prediction, the reference and the pointwise error on the mesh, the regions with the largest error and the declared …

`pinneapple_veriphysics/mesh_views.py` · 164 linhas · testes: 3

nome	tipo	assinatura	o que faz
mesh_views	função	`(workdir, config=None, *, report=None, max_faces=None, n_regions=8, mesh_unit='m') -> lis…)`	One view per 3-D result target with a reference: `{"target", "view" (FieldView), "errors" {field: error map dict}, "fields", "interpolated", "notes"}`. `report`: the lab report, whose PC701 values are quoted next to the pointwise numbers. `mesh_unit`: the length unit of the coordinates …

pinneapple_veriphysics.recommend

The "formulation + method intelligence" half of `pipeline.analyze()`, factored out so it can run on its own -- fast (no training, one `draft_problem` LLM call), for the "Solution Recommendation" screen: internal solver/architecture choice and external tools/players, shown side by side with …

`pinneapple_veriphysics/recommend.py` · 295 linhas · testes: 1

nome	tipo	assinatura	o que faz
`CandidateDetail`	classe	`(key, name, category, is_recommended, when_to_use, license, typical_cost_tier, player, st…)`	One real, documented option -- an architecture or an external tool -- with its real strengths/weaknesses, never a fabricated score. `is_recommended` distinguishes the primary pick from an alternative; every field beyond that comes straight from the real catalog entry (`ARCHITECTURE_CATALOG` / …
`RecommendationResult`	classe	`(rejected, rejection_reason, dimensionless_numbers, flow_regime, drafted_preset, drafted_…)`	
`formulate_and_recommend`	função	`(description, *, physical_parameters=None, flow_geometry='internal_pipe', provider='ollam…)`	Formulate the problem (dimensional analysis + LLM preset draft) and compute internal + external method-intelligence recommendations -- no training, no PhysicsGuardrail check. Fast enough for an interactive "recommend before you commit" UI screen.

pinneapple_veriphysics.robustness

Robustness studies that feed the applicability map with *measured* evidence for the three items that used to be `NOT_RUN`:

`pinneapple_veriphysics/robustness.py` · 300 linhas · testes: 5

nome	tipo	assinatura	o que faz
`ensemble_study`	função	`(spec, build_model, best_model, *, n_members, epochs, n_collocation, n_eval=2048, margins…)`	Train *n_members* extra seeds and measure member disagreement in the interior and on each outside shell. Returns the models too (key `"members"`) so callers can reuse them; strip before serializing.
`extrapolation_study`	função	`(spec, model, *, residual_threshold, ensemble=None, margins=DEFAULT_MARGINS, n_eval=2048,…)`	Residual (and ensemble spread, if given) on shells outside the training box. `supported_margin` is the largest margin such that it and every smaller margin keep residual <= threshold and spread <= `spread_tol` (default `SPREAD_REL_TOL`); `0.0` means even the smallest shell fails.
`weight_sensitivity_study`	função	`(spec, build_model, best_model, *, epochs, n_collocation, factors=DEFAULT_WEIGHT_FACTORS,…)`	Retrain with `w_pde` scaled by each factor (other weights at their defaults) and measure the RMS change of the prediction vs *best_model* relative to the field's RMS magnitude. Note: the seed differs from the original run's, so a part of the difference is seed noise -- compare with the ensemble's …

nome	tipo	assinatura	o que faz
`calibration_against_reference`	função	`(members, reference_x, reference_y) -> dict[str, Any]`	Real calibration of the ensemble against supplied reference data via `CalibrationMetrics` -- the only way to turn the spread into a *calibrated* uncertainty. Returns ece/coverage/sharpness.
`condition_coverage`	função	`(spec, n=50000) -> dict[str, Any]`	How many points each auto-sampled (`callable`/`all`) boundary, initial or data condition selects under the SAME sampler the installed `pinneapple_physics.solve_pde` uses, so this reports what training will actually enforce.
`parameter_shift_study`	função	`(make_spec, base_kwargs, build_model, base_model, *, residual_threshold, epochs, n_colloc…)`	Parameter generalization: how far can a physical parameter move before the surrogate trained at the base value stops being usable?

pinneapple_veriphysics.scope

Scope of use: which uses a result can serve, where it fails, and why -- beyond one approve / reject verdict.

`pinneapple_veriphysics/scope.py` · 148 linhas · testes: 5

nome	tipo	assinatura	o que faz
`UseScope`	classe	`(use, ok, reason, why, evidence, values, limits, fraction_ok, precondition)`	One use of a result. `ok`: True (usable), False (not), None (not assessed).
`ScopeOfUse`	classe	`(subject, uses, figure, notes)`	The uses a result can and cannot serve, with an optional figure (a path to a PNG) and notes (for instance how an estimate was validated).
`scope_from_verdicts`	função	`(subject, verdicts, *, evidence='estimated', figure=None, notes=()) -> ScopeOfUse`	A :class:`ScopeOfUse` from per-use verdicts `{use: {"ok", "reason", "why", "values", "limits"}}` (the form of `darcy_checks.use_verdicts`). `evidence`: one kind for every use, or `{use: kind}`.
`scope_from_batch`	função	`(subject, verdicts, *, min_fraction=0.9, evidence='estimated', figure=None, notes=()) -> …)`	A :class:`ScopeOfUse` over a batch of per-sample verdicts: a use is OK when at least `min_fraction` of the samples serve it (0.9: nine in ten, the batch convention of `pinneapple_physics.darcy_checks`); the share is kept in `fraction_ok` and the reason quotes the median measured values.

pinneapple_veriphysics.validation_lab

Validation Lab: run the physics checks of `pinneapple check` on predictions, results or a zoo model and report them in three separate groups, with an evidence record for every check that ran.

`pinneapple_veriphysics/validation_lab.py` · 459 linhas · testes: 42

nome	tipo	assinatura	o que faz
`CodeNotAllowed`	classe	`()`	The configuration would import or unpickle user-supplied code.
`LabReport`	classe	`(check_report, ledger, config_text, inputs, started_at, finished_at, versions, title)`	
`category_of`	função	`(rule_id) -> str | None`	Group of a `pinneapple check` rule (None for PC002, which is about coverage, not a check).
`to_toml`	função	`(doc) -> str`	Write a `pinneapple-check.toml`: `check` as a table, `result`/`model`/`dataset` as arrays of tables, nested values as inline tables. Only what the configuration needs (no dates, no nulls).
`model_ref_target`	função	`(ref) -> dict[str, Any]`	A `[[model]]` table from a model reference: `{"zoo": name}`, a ModelRef with `uri = "zoo:<name>"`, or one with `source == "zoo"` and `name`. `inputs`/`outputs` may be names or `{"name": ...}` variables; `train_range`, `queries`, `bounds`, `units`, `tolerances` and ...
`config_from_spec`	função	`(spec) -> str`	`pinneapple-check.toml` from a plain description of what to check::
`assert_no_code`	função	`(cfg) -> None`	Refuse targets that execute code from the inputs: a `file.py:function` or checkpoint model, a Hub download. Zoo models (shipped with the library, hashes checked) and data files are allowed.
`write_inputs`	função	`(root, files) -> list[str]`	Write uploaded files (`{relative path: bytes}`, e.g. from `pinneapple_data.cae.upload.expand`) under `root`; a name that would land outside `root` is refused.
`tables_to_csv`	função	`(root, config_text=None) -> tuple[str | None, dict[str, str]]`	Parquet files and JSON tables (a list of row objects, or an object of equal-length columns) under `root` become CSV next to them, which `pinneapple check` reads as point clouds or quantities of interest. Returns the configuration with the references renamed and `{old: new}`. Other JSON ...
`run_validation`	função	`(workdir, config=None, *, allow_code=False, title='Validation') -> LabReport`	Run `pinneapple check` on the inputs under `workdir` with `config` (default `workdir/pinneapple-check.toml`) and record every outcome in an evidence ledger.
`summarize`	função	`(lab) -> dict[str, Any]`	Compact view of a lab report dict: decision, per-group counts and groups that did not run.
`render_html`	função	`(lab) -> str`	Self-contained HTML of a lab report dict (no scripts, no external files).

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 12 de 13.
- Funções de teste em arquivos que importam algum módulo do pacote: 98.

Limitações conhecidas

- *Nenhuma registrada.*

4. `pinneapple_security`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
20	3.398	33	93	105 (10 arquivos)	0

Pasta de testes dedicada: `tests/pinneapple_security` (60 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`		1	1	10
`anomaly`		1	3	6
`archives`		1	7	22
`attestation`		1	7	10
`audit`		1	1	54
`classification`		1	4	10
`compliance`		1	3	10
`cps`		1	9	5
`cps_bench`		1	7	5
`dp`		1	7	9
`encryption`		1	9	6
`integrate`		1	8	10
`integrity`		1	8	14
`model_security`		1	6	6
`passwords`		1	4	56
`privacy`		1	11	9
`secrets_scan`		1	4	10
`signing`		1	12	25
`web`		1	15	40

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_security`

pinneapple_security — data and process security for scientific and industrial Physics AI.

`pinneapple_security/__init__.py` · 67 linhas · testes: 64

pinneapple_security.__main__

python -m pinneapple_security <command>: manifests, signatures, scans and log sanitising from the shell.

pinneapple_security/__main__.py · 143 linhas · testes: 10

nome	tipo	assinatura	o que faz
main	função	(argv=None) -> int	

pinneapple_security.anomaly

Physics-based detection of manipulated sensor data (false data injection, spoofing, replay) in industrial streams.

pinneapple_security/anomaly.py · 102 linhas · testes: 6

nome	tipo	assinatura	o que faz
ResidualCUSUM	classe	(k, h, mu, sigma, fitted, s_hi, s_lo, t, alarms)	
balance_residual	função	(inflows, outflows, accumulation=None) -> np.ndarray	Conservation residual Σin − Σout − d(storage)/dt per sample (mass, energy or charge balance).
replay_score	função	(x, window=50) -> dict[str, Any]	Smallest normalised distance between the last window samples and any earlier window. Near 0 → the latest data repeats an old recording sample for sample (replay attack) — live noise never matches that closely.

pinneapple_security.archives

Open uploaded zip archives safely: no member outside the destination (zip-slip), no symbolic links, and limits on the number of members, the total uncompressed size, the size of one member and the compression ratio (zip bombs).

pinneapple_security/archives.py · 153 linhas · testes: 22

nome	tipo	assinatura	o que faz
UnsafeArchive	classe	()	The archive breaks a safety rule or a limit.
ArchiveLimits	classe	(max_members, max_total_bytes, max_member_bytes, max_ratio)	
check_member_name	função	(name) -> str	The member name as a safe relative POSIX path, or :class:UnsafeArchive.
check_zip	função	(z, limits=DEFAULT_LIMITS) -> list[zipfile.ZipInfo]	Check names, link entries and declared sizes; return the file members (directories left out).
safe_extract_zip	função	(source, dest, limits=DEFAULT_LIMITS) -> list[Path]	Extract source (path, bytes, file object or ZipFile) under dest; return the written files.

nome	tipo	assinatura	o que faz
`read_members`	função	`(source, limits=DEFAULT_LIMITS, skip=None) -> dict[str, bytes]`	All file members in memory, `{safe name: bytes}`, under the same checks and counted limits. `skip`: a predicate on the member name for members to leave out (not read, not counted).
`gunzip`	função	`(data, max_bytes=1 << 30) -> bytes`	Decompress gzip `data`, refusing output above `max_bytes` (a gzip bomb) without producing it first.

pinneapple_security.attestation

Provenance attestations (in-toto Statement v1 with a SLSA Provenance v1 predicate) and a CycloneDX SBOM.

`pinneapple_security/attestation.py` · 153 linhas · testes: 10

nome	tipo	assinatura	o que faz
`subject`	função	`(name, path=None, sha256=None) -> dict[str, Any]`	
`git_commit`	função	`(path='.') -> dict[str, Any] | None`	Commit of the repository containing `path` and whether the work tree is dirty (None outside git).
`provenance`	função	`(external_parameters, dependencies=(), started_on=None, finished_on=None, builder_id=…, i…)`	
`statement`	função	`(subjects, predicate, predicate_type=SLSA_PROVENANCE) -> dict[str, Any]`	
`attest_files`	função	`(artifacts, parameters, inputs=(), repo='.') -> dict[str, Any]`	Statement for output files (`name → path` or paths) built from input files with `parameters`.
`attest_experiment`	função	`(result, artifacts=(), inputs=(), repo='.') -> dict[str, Any]`	Statement for a `pp.Experiment` result: the record itself (by digest) and any saved files are the subjects, the config and problem fingerprint are the parameters, the metrics go in `internalParameters`.
`sbom`	função	`(name='pinneapple-environment', version='', packages=None) -> dict[str, Any]`	CycloneDX 1.5 SBOM of the installed distributions (all, or only `packages`).

pinneapple_security.audit

Tamper-evident audit trail: an append-only JSON-lines log where every record carries the hash of the previous one.

`pinneapple_security/audit.py` · 133 linhas · testes: 54

nome	tipo	assinatura	o que faz
`AuditLog`	classe	`(path, key=None)`	

pinneapple_security.classification

Data classification and handling policy for datasets, models and results.

pinneapple_security/classification.py · 106 linhas · testes: 10

nome	tipo	assinatura	o que faz
DataLabel	classe	(level, owner, personal_data, export_control, licence, retention_days, allowed_countries,…)	
Decision	classe	(allowed, reasons, requires)	
HandlingPolicy	classe	(publish_levels, share_levels, cloud_levels)	Default rules (organisation-specific: subclass or edit). approvals: approvals the caller already holds, e.g. {"owner", "dpo", "export_officer"}.
combine	função	(labels) -> DataLabel	Label of something derived from all labels: strictest level, any personal data, every export control.

pinneapple_security.compliance

Which pinneapple_security capability produces evidence for which control of the frameworks industry and research are audited against — and a self-assessment report from the evidence a project actually has.

pinneapple_security/compliance.py · 106 linhas · testes: 10

nome	tipo	assinatura	o que faz
controls_for	função	(capability) -> list[dict[str, Any]]	
assess	função	(evidence, frameworks=()) -> dict[str, Any]	evidence: capability → anything truthy (a path, a verification result with ok, True). A control is covered when at least one of its capabilities has evidence, partial when some do but a verification failed.
report_markdown	função	(result, title=…) -> str	

pinneapple_security.cps

Physics-residual detection of false data injection in cyber-physical systems (water, process, HVAC, grids).

pinneapple_security/cps.py · 305 linhas · testes: 5 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ResidualComponent	classe	(name)	
MassBalance	classe	(level, inflows, outflows, dt, window, area, name)	Storage balance of one volume over a window of window samples: area * (level[t] - level[t-w]) - dt * sum(inflows - outflows)[t-w:t], in volume units (m^3 for water).

nome	tipo	assinatura	o que faz
`FlowLaw`	classe	`(flow, up, down, exponent, midpoint, coef, name)`	Flow through a valve, orifice, pipe or weir driven by a head difference: `q^2 = a + b * (up - down)` (turbulent square-root law, exponent 1/2; `exponent` 1 gives a linear laminar or pump-curve law). `a` absorbs the elevation offset between the two levels. Residual `q - q_law` in flow units. …
`CusumAlarm`	classe	`(k, h, upper_only)`	Alarm logic shared by every detector here: standardised scores `self.z(D)` (T, m) → CUSUM → threshold calibrated to a false-alarm budget. Subclasses define `z` and `channels`; `upper_only` for scores where only large values are suspicious. Physics and data-driven detectors use the same …
`PhysicsResidualDetector`	classe	`(components, k, h, mu_, sigma_)`	Grey-box physics residuals + per-channel two-sided CUSUM, threshold calibrated to a false-alarm budget.
`cusum`	função	`(z, k=0.5, h=None, upper_only=False) -> tuple[np.ndarray, list[int]]`	Two-sided CUSUM of standardised scores `z` (T,) or (T, m). Returns the statistic (T,), the maximum over channels and both sides, and the alarm times when `h` is given (the sums restart after each alarm, so the alarm count is a false-alarm count on clean data). NaNs (warm-up) leave the sums …
`roc_auc`	função	`(scores, labels) -> float`	Area under the ROC curve (Mann-Whitney U with average ranks for ties). NaN scores are dropped.
`window_auc`	função	`(stat_attacked, episodes, stat_clean) -> float`	Episode-level ROC AUC: the largest statistic inside each attack episode against the largest statistic in every non-overlapping window of the same length of a clean trace. Separability of attacked from clean windows, independent of any threshold.
`episode_metrics`	função	`(alarms, episodes, n, samples_per_day=1440, grace=60) -> dict[str, Any]`	Detection rate and delay per attack episode `(start, end)` (end exclusive) and false alarms per day outside the episodes. An alarm in `[start, end)` detects the episode; the delay is in samples. Alarms in the `grace` samples after an episode (the attack being withdrawn is itself a …

`pinneapple_security.cps_bench`

Open, reproducible benchmark for false-data-injection detection: a three-tank gravity water network.

`pinneapple_security/cps_bench.py` · 331 linhas · testes: 5

nome	tipo	assinatura	o que faz
`PCAResidualDetector`	classe	`(variance)`	Squared prediction error (Q statistic) of a PCA model of the sensors and their one-step increments.
`ARResidualDetector`	classe	`(lags, ridge, upper_only)`	Data-driven linear twin: each sensor predicted from `lags` past samples of every sensor (ridge regression); per-sensor two-sided CUSUM of the standardised one-step prediction residuals.
`IsolationForestDetector`	classe	`(n_estimators, seed, max_fit)`	Isolation forest (Liu, Ting & Zhou 2008) on sensors and their increments; needs scikit-learn.

nome	tipo	assinatura	o que faz
`simulate_three_tank`	função	`(days=7.0, seed=0, dt=60.0, substeps=12, demand_scale=1.0, pump_band=(1.0, 2.5)) -> dict[…)`	Simulate the plant; returns sensor columns (`SENSORS`) plus the true states `true_h1..3`. `demand_scale` and `pump_band` (T1 levels that switch the pump on and off) change the operating regime without changing the physics: a detector that learned the physics should not care, one that …
`inject`	função	`(D, kind, episodes) -> dict[str, np.ndarray]`	Copy of `D` with the reported readings rewritten in each `(start, end)` episode.
`physics_detector`	função	`(dt=60.0, window=1, k=0.5) -> PhysicsResidualDetector`	Grey-box twin of the plant: three storage balances and two valve laws, constants identified from data.
`run_benchmark`	função	`(seed=0, train_days=7.0, calib_days=7.0, test_days=10.0, episodes=20, length=240, false_a…)`	Fit every detector on a trusted trace, calibrate it to `false_alarms_per_day` on a second clean trace, then, for each operating regime (nominal, and a shifted regime the detectors never saw), score one clean test trace and one attacked copy per attack with `episodes` episodes of `length` …

`pinneapple_security.dp`

Differential privacy: noise mechanisms, private statistics, a privacy accountant and DP-SGD for PyTorch models.

`pinneapple_security/dp.py` · 155 linhas · testes: 9

nome	tipo	assinatura	o que faz
`RDPAccountant`	classe	`()`	Privacy spent by `steps` applications of the sampled Gaussian mechanism (sampling rate `q`, noise multiplier `sigma` = noise std / clipping norm).
`DPSGD`	classe	`(model, optimizer, loss_fn, noise_multiplier, max_grad_norm, sample_rate, seed=None)`	DP-SGD wrapper: `step(xb, yb)` computes per-sample gradients of `loss_fn(model_output, y)` (one scalar per sample), clips each to `max_grad_norm`, adds N(0, (σ·C)²) noise to the sum and lets `optimizer` apply the noisy mean. `sample_rate` = batch size / dataset size (Poisson sampling is …
`laplace_mechanism`	função	`(value, sensitivity, epsilon, rng=None) -> Any`	
`gaussian_sigma`	função	`(sensitivity, epsilon, delta) -> float`	σ = Δ_2 √(2 ln(1.25/δ)) / ε (Dwork & Roth Thm A.1; stated for ε ≤ 1).
`gaussian_mechanism`	função	`(value, sensitivity, epsilon, delta, rng=None) -> Any`	
`dp_mean`	função	`(x, lower, upper, epsilon, rng=None) -> float`	ε-DP mean with values clipped to [lower, upper] and the record count treated as public.

nome	tipo	assinatura	o que faz
dp_histogram	função	(x, bins, epsilon, rng=None) -> np.ndarray	ε-DP counts per bin (one record changes one count by 1); negative noisy counts are set to 0.

pinneapple_security.encryption

Encryption at rest: AES-256-GCM for bytes, files and arrays, with keys from a passphrase (scrypt) or random.

pinneapple_security/encryption.py · 106 linhas · testes: 6

nome	tipo	assinatura	o que faz
DecryptionError	classe	()	
derive_key	função	(passphrase, salt) -> bytes	
new_key	função	() -> bytes	
encrypt_bytes	função	(data, secret) -> bytes	
decrypt_bytes	função	(blob, secret) -> bytes	
encrypt_file	função	(path, secret, out=None, remove_plain=False) -> str	
decrypt_file	função	(path, secret, out=None) -> str	
encrypt_array	função	(a, secret) -> bytes	
decrypt_array	função	(blob, secret)	

pinneapple_security.integrate

pinneapple_security in the rest of the library: signed model versions in the ModelStore, signed model cards and TrustReports, attested experiments, audited actions.

pinneapple_security/integrate.py · 134 linhas · testes: 10

nome	tipo	assinatura	o que faz
secure_save	função	(store, problem_id, model, signing_key, *, metadata=None, stage='development', trust_repo…)	ModelStore.save + a manifest of the version folder, signed, + a safety scan of the checkpoint + an audit record. Returns the version.
resign_version	função	(store, problem_id, version, signing_key, *, actor='', audit=None, reason='') -> str	Re-sign a version after a legitimate change (e.g. ModelStore.promote rewrites metadata.json); the audit record keeps the old and new roots, so the change stays traceable.
verify_version	função	(store, problem_id, version, key) -> dict[str, Any]	Signature valid, manifest matches the signed root, files match the manifest, checkpoint scan safe.
sign_record	função	(obj, key) -> dict[str, Any]	

nome	tipo	assinatura	o que faz
`verify_record`	função	`(envelope, key, expected=None) -> Any`	
`sign_model_card`	função	`(card, key, path=None) -> dict[str, Any]`	Envelope over `card.to_dict()` (TrustReport and override included); written next to the card if `path`.
`verify_model_card`	função	`(card, envelope, key) -> dict[str, Any]`	
`attest_and_sign_experiment`	função	`(result, key, directory=None, inputs=(), repo='.') -> dict[str, Any]`	In-toto/SLSA statement for a `pp.Experiment` result, signed. With `directory`, the result is saved there first, its files become subjects, and `attestation.json` is written next to them.

`pinneapple_security.integrity`

Integrity: content digests of files, arrays and records, and Merkle manifests of datasets and model folders.

`pinneapple_security/integrity.py` · 139 linhas · testes: 14

nome	tipo	assinatura	o que faz
`Manifest`	classe	`(files, algorithm, root)`	Files of a folder with their digests. `files`: relative path (POSIX) → {"sha256", "size"}.
`sha256_bytes`	função	`(data) -> str`	
`sha256_file`	função	`(path) -> str`	
`canonical_json`	função	`(obj) -> bytes`	Canonical JSON bytes (sorted keys, compact separators, UTF-8); non-JSON values go through `str`.
`digest_record`	função	`(obj) -> str`	
`digest_array`	função	`(a) -> str`	SHA-256 of an array's dtype, shape and C-ordered bytes (NumPy arrays and PyTorch tensors).
`merkle_root`	função	`(leaves) -> str`	Merkle root (SHA-256) of hex leaf digests; an odd node is paired with itself. Empty input → hash of b''.
`build_manifest`	função	`(folder, include=(), exclude=…) -> Manifest`	Manifest of every file under `folder` (glob `include` filters; `exclude` drops names, folders, paths).

`pinneapple_security.model_security`

Model security: scan checkpoints for code execution before loading, load safely, and measure adversarial sensitivity of surrogates.

`pinneapple_security/model_security.py` · 165 linhas · testes: 6

nome	tipo	assinatura	o que faz
`UnsafeModelError`	classe	`()`	
`ScanReport`	classe	`(path, globals, dangerous, unknown, errors)`	

nome	tipo	assinatura	o que faz
scan_pickle_bytes	função	(data, name='<bytes>') -> ScanReport	
scan_checkpoint	função	(path) -> ScanReport	Scan a torch.save zip archive (every *.pkl member) or a legacy/plain pickle file.
safe_load	função	(path, map_location='cpu', expected_sha256=None)	Verify the digest (if given), scan, then torch.load(weights_only=True). Raises :class:UnsafeModelError.
adversarial_sensitivity	função	(model, X, eps=0.01, steps=20, step_frac=0.25, input_range=None) -> dict[str, Any]	Worst-case relative output change under an L∞ input perturbation of eps × the input range (per feature).

pinneapple_security.passwords

Password hashing for logins: scrypt from the standard library by default, argon2id when argon2-cffi is installed, and verification of bcrypt hashes ($2b$..., as written by caddy hash-password or htpasswd -B) when bcrypt is installed.

pinneapple_security/passwords.py · 119 linhas · testes: 56

nome	tipo	assinatura	o que faz
is_hash	função	(value) -> bool	Whether value looks like a hash this module can verify (as opposed to a plain-text password).
hash_password	função	(password, scheme='auto') -> str	A salted hash of password. scheme: "scrypt", "argon2" (needs argon2-cffi) or "auto" (argon2 when installed, else scrypt).
verify_password	função	(password, stored) -> bool	Whether password matches the hash stored. Unknown formats and missing libraries give False.
needs_rehash	função	(stored) -> bool	True for a scrypt hash weaker than the current parameters (argon2: its own check).

pinneapple_security.privacy

Personal and sensitive data in text and tables: detection, redaction, pseudonymisation, k-anonymity, log sanitising.

pinneapple_security/privacy.py · 180 linhas · testes: 9

nome	tipo	assinatura	o que faz
Finding	classe	(kind, start, end, text)	
valid_cpf	função	(s) -> bool	
valid_cnpj	função	(s) -> bool	
luhn_ok	função	(s) -> bool	
iban_ok	função	(s) -> bool	

nome	tipo	assinatura	o que faz
`find_pii`	função	`(text, kinds=PII_KINDS) -> list[Finding]`	Non-overlapping findings, earlier kinds in `PII_KINDS` winning ties (a CNPJ is not also a phone).
`redact`	função	`(text, kinds=PII_KINDS, label=…) -> str`	
`pseudonymize`	função	`(value, key, prefix='id_', length=16) -> str`	Keyed, deterministic token for `value` (HMAC-SHA256, truncated to `length` hex characters).
`pseudonymize_column`	função	`(values, key, prefix='id_') -> list[str]`	
`k_anonymity`	função	`(rows, quasi_identifiers, k=5) -> dict[str, Any]`	`rows`: a pandas DataFrame or a list of dicts. Returns the table's k, the groups smaller than `k` and the share of rows in them.
`sanitize_log`	função	`(text, extra=(), keep_paths=True) -> tuple[str, dict[str, int]]`	Strip personal and infrastructure identifiers from a log. `extra`: literal strings to hide too (project or client names). Returns the clean text and how many of each kind were replaced.

`pinneapple_security.secrets_scan`

Find credentials committed to code, notebooks, configs and solver cases before they are shared or published.

`pinneapple_security/secrets_scan.py` · 116 linhas · testes: 10

nome	tipo	assinatura	o que faz
`SecretFinding`	classe	`(rule, path, line, excerpt)`	
`shannon_entropy`	função	`(s) -> float`	
`scan_text`	função	`(text, path='<text>', entropy_threshold=4.5) -> list[SecretFinding]`	
`scan_path`	função	`(root, exts=None, max_bytes=5000000, entropy_threshold=4.5) -> list[SecretFinding]`	

`pinneapple_security.signing`

Signatures for artifacts and records: Ed25519 (public-key, needs `cryptography`) or HMAC-SHA256 (shared key, stdlib).

`pinneapple_security/signing.py` · 189 linhas · testes: 25

nome	tipo	assinatura	o que faz
`VerificationError`	classe	`()`	
`VerifyKey`	classe	`(algorithm, raw)`	Public half (Ed25519) or the shared secret (HMAC). `raw`: 32 bytes.
`SigningKey`	classe	`(algorithm, raw)`	
`pae`	função	`(payload_type, payload) -> bytes`	DSSE pre-authentication encoding.
`generate_key`	função	`() -> SigningKey`	New Ed25519 key (needs `cryptography`).

nome	tipo	assinatura	o que faz
hmac_key	função	(secret=None) -> SigningKey	Shared-secret key (stdlib only). Anyone who can verify can also sign: use it inside one organisation.
sign_bytes	função	(data, key) -> dict[str, str]	
verify_bytes	função	(data, signature, key) -> None	
sign_json	função	(payload, key, payload_type=PAYLOAD_TYPE) -> dict[str, Any]	DSSE envelope over the canonical JSON of payload.
verify_json	função	(envelope, key) -> Any	Check the envelope with key; returns the payload. Raises :class:VerificationError.
sign_file	função	(path, key, sig_path=None) -> str	Detached signature <path>.sig (JSON) over the file's SHA-256 and name.
verify_file	função	(path, key, sig_path=None) -> dict[str, Any]	

pinneapple_security.web

HTTP hardening for the PINNeAPPle web apps (ASGI: FastAPI / Starlette), with no dependency beyond the standard library at import time.

pinneapple_security/web.py · 496 linhas · testes: 40

nome	tipo	assinatura	o que faz
SecurityHeaders	classe	(app, csp=None, hsts=…, frame_options='DENY', extra=None)	ASGI middleware adding the security headers to every response (an existing header set by the route wins).
BodyLimit	classe	(app, max_bytes, limits=None)	ASGI middleware: 413 for a request body above max_bytes (Content-Length checked first, then the bytes actually received, so chunked uploads can not pass the limit). limits: per path prefix overrides.
RateRule	classe	(name, count, seconds, methods, prefixes)	count requests per seconds for requests whose method is in methods (empty: any) and whose path starts with one of prefixes (empty: any).
RateLimiter	classe	(rules=(), max_keys=50000, clock=time.monotonic)	Token buckets per (rule, key): a burst up to count and a steady count/seconds refill. Memory is bounded (least recently used keys are dropped past max_keys). Thread-safe.
LoginThrottle	classe	(max_failures, window, lock_seconds, clock, max_keys, _fails, _locked, _lock)	Lock a client IP or a user name out for lock_seconds after max_failures failed logins within window seconds. A success clears the IP's and the user's failure count.
UnsafeURL	classe	()	A URL the server must not fetch.
AuthAudit	classe	(app_name, path=None, key=None)	Auth and admin events (login ok/failed/locked, API key created/revoked, admin denied, rate limited) as one JSON log line each on the pinneapple.security logger, and, with path, in a hash-chained :class:pinneapple_security.audit.AuditLog (key: HMAC key for the chain). Secrets are never …
client_ip	função	(peer, forwarded_for, trusted=DEFAULT_TRUSTED_PROXIES) -> str	The caller's address. X-Forwarded-For is used only when the direct peer is a trusted proxy (by default a loopback or private address: the Docker network Caddy sits on), and then the right-most entry that is not itself a trusted proxy is taken. A client that reaches the app directly from a …

nome	tipo	assinatura	o que faz
`inline_hashes`	função	`(page) -> tuple[set[str], set[str]]`	CSP hashes of the inline `<script>` blocks (including import maps) and of the inline event handlers (`onsubmit="..."`) of an HTML page.
`build_csp`	função	`(script_hashes=(), handler_hashes=(), *, connect_src=(), img_src=(), frame_ancestors="'no…)`	The Content-Security-Policy of an app page: everything from the app's own origin, the app's own inline scripts by hash (no `'unsafe-inline'` for scripts), WebGL/three.js friendly (`blob:` workers and images). `frame_src`: other origins the page may embed; `frame_ancestors`: who may embed it …
`parse_rate`	função	`(spec, default=None) -> tuple[float, float] | None`	`"30/60"` (30 requests per 60 s), `"30/min"`, `"5/s"`, `"1000/h"`; `off`/`0` → None.
`same_origin_ok`	função	`(method, headers, allowed_origins=()) -> bool`	False for a state-changing browser request coming from another site (cross-site request forgery: a page elsewhere posting to the app with the user's cached Basic-auth credentials). Uses the browser-set `Sec-Fetch-Site` header, then `Origin` compared with `Host` (`X-Forwarded-Host` behind a …
`check_cors_origins`	função	`(origins, allow_credentials) -> list[str]`	Validate a CORS allow-list: full `scheme://host[:port]` origins only, and no `*` with credentials.
`check_frame_origins`	função	`(origins) -> list[str]`	Validate origins allowed to embed an app (`frame-ancestors`) or to be embedded by it (`frame-src`): full `scheme://host[:port]` origins only; no `*`, no scheme wildcards, no paths.
`check_outbound_url`	função	`(url, allow_hosts=(), resolve=None) -> str`	Refuse a URL a user asked the server to call unless it is http(s) to a public address, or its host is in `allow_hosts` (e.g. `surrogate` for an app on the same Docker network). Loopback, link-local (cloud metadata at 169.254.169.254), private, multicast and reserved addresses are refused, for …

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 20 de 20.
- Funções de teste em arquivos que importam algum módulo do pacote: 105.

Limitações conhecidas

- *Nenhuma registrada.*

5. `pinneapple_decision`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
12	2.499	37	20	44 (3 arquivos)	9

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`adapter`		1	2	0
`api`		1	5	16
`backends`		1	3	0
`constraints`		1	5	0
`debias`		1	8	21
`decider`		1	1	0
`loop`		1	7	0
`schema`		1	10	0
`selectors`		1	4	0
`tree`		1	9	0
`verifiers`		1	3	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_decision`

pinneapple_decision -- probabilistic decision layer over the Physics AI pipeline.

`pinneapple_decision/__init__.py` · 116 linhas · testes: 44

`pinneapple_decision.adapter`

Problem adapter: real `ProblemSpec` objects and free-form dicts -> decision state.

`pinneapple_decision/adapter.py` · 508 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AdaptedProblem`	classe	`(problem, given, inferred, unknown, unrecognized, source)`	Result of :func:`adapt_problem`.
`adapt_problem`	função	`(obj) -> AdaptedProblem`	Adapt a `ProblemSpec`, a free-form dict, or `None` to the decision layer.

`pinneapple_decision.api`

Module-level API used by `import pinneapple as pp; pp.decide(...)`.

`pinneapple_decision/api.py` · 59 linhas · testes: 16

nome	tipo	assinatura	o que faz
decision_engine	função	(executor=None, verifier=None, decider=None, store=None, *, reset=False) -> PhysicsDecisi…)	Return the default engine. Passing any argument (or reset=True) replaces it with a new engine built from those arguments (evidence is not carried over).
decide	função	(problem=None, objective=None, *, state=None, choice=None, engine=None) -> Decision	Decide which experiment to run next. problem may be a ProblemSpec or a free-form dict (see adapter.py). Facts that could not be determined are listed in decision.diagnostics["problem_facts"]["unknown"].
execute	função	(decision, problem=None, *, engine=None) -> ExecutionResult	Execute a decision issued by the same engine (constraints are re-checked first).
verify	função	(result, *, engine=None) -> Verification	Verify a result; only verified results become evidence for the next decision.
run_tree	função	(problem, tree=None, *, max_experiments=3, engine=None)	Walk the physics-AI decision tree (tree.physics_ai_tree() by default).

pinneapple_decision.backends

Scoring backends: turn (state, choice, options in a given order) into scores.

pinneapple_decision/backends.py · 153 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ScoringBackend	classe	(name, has_label_prior)	
RuleBasedBackend	classe	(rules=None)	Deterministic backend over per-choice rule functions (see selectors.py).
LogProbLLMBackend	classe	(logprob_fn, *, template=DEFAULT_TEMPLATE, name='llm-logprob')	Scores options by next-token log-probability of their letter label.

pinneapple_decision.constraints

Hard constraints on options: physics/geometry, resources, implementation.

pinneapple_decision/constraints.py · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Constraint	classe	(name, check)	
NoFeasibleOptionError	classe	(choice, excluded)	
ConstraintViolationError	classe	()	Raised when a decision is about to be executed but its option no longer passes.
resolve	função	(constraint) -> Constraint	
apply_constraints	função	(choice, state) -> Tuple[List[str], Dict[str, str]]	Return (feasible_options, {excluded_option: reason}) in the choice's option order.

pinneapple_decision.debias

L0 debiasing without labels (AnyJev-like; own implementation, see README).

`pinneapple_decision/debias.py` · 161 linhas · testes: 21

nome	tipo	assinatura	o que faz
`BatchPrior`	classe	`(min_prior_n=8)`	Running mean of L0 distributions per (choice name, feasible option set).
`log_softmax`	função	`(xs) -> List[float]`	
`softmax_dict`	função	`(logits) -> Dict[str, float]`	
`normalize`	função	`(p) -> Dict[str, float]`	
`rotations`	função	`(options, max_permutations=None) -> List[List[str]]`	
`raw_distribution`	função	`(score_fn, options) -> Dict[str, float]`	
`permutation_marginalized`	função	`(score_fn, options, *, combine='geometric', max_permutations=None) -> Tuple[Dict[str, flo…)`	Distribution averaged over cyclic rotations of the option order.
`apply_prior`	função	`(dist, prior, strength=1.0) -> Dict[str, float]`	

pinneapple_decision.decider

PhysicsDecider: constraints -> scoring -> L0 debiasing -> Decision.

`pinneapple_decision/decider.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsDecider`	classe	`(backend=None, *, debias=True, prior='content_free', prior_strength=1.0, combine='geometr…)`	Decide which option of a :class:`PhysicsChoice` to run next.

pinneapple_decision.loop

OBSERVE -> UNDERSTAND -> DECIDE -> EXECUTE -> VALIDATE -> LEARN.

`pinneapple_decision/loop.py` · 215 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ValidationRequiredError`	classe	`()`	A result was offered as evidence before it was verified.
`Executor`	classe	`()`	
`CallableExecutor`	classe	`(fn)`	Wrap `fn(decision, problem) -> ExecutionResult | dict`.

nome	tipo	assinatura	o que faz
`EvidenceStore`	classe	`(path=None)`	In-memory evidence log, optionally persisted as JSON.
`DecisionLoop`	classe	`(executor=None, verifier=None, decider=None, store=None)`	
`PhysicsDecisionEngine`	classe	`()`	High-level facade: `decide(problem, objective=...)` / `decide(state={...})`, `execute(decision)`, `verify(result)`.
`choice_for_objective`	função	`(objective, has_previous_result=False) -> PhysicsChoice`	

`pinneapple_decision.schema`

Core types of the decision layer: choices, decisions, evidence.

`pinneapple_decision/schema.py` · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DecisionLevel`	classe	`()`	What was done to the backend's scores before reading them.
`OptionInfo`	classe	`(name, description, implementation, representations, supports_varying_geometry, relative_…)`	Metadata of one option, used by rules and constraints (never by the LLM to decide).
`PhysicsChoice`	classe	`(name, question, options, constraints, option_info)`	A typed question with a closed set of options.
`ExecutionResult`	classe	`(status, metrics, diagnostics, artifacts, decision_id)`	What an executor returns. `diagnostics` are short tags a rule or LLM can read on the next decision (e.g. `"residual_localized"`, `"late_time_error"`).
`Verification`	classe	`(passed, checks, tags, source, notes)`	Outcome of verifying one :class:`ExecutionResult`.
`Decision`	classe	`(choice, question, selected, probabilities, level, requires_validation, calibrated_confid…)`	
`Evidence`	classe	`(decision, result, verification)`	One OBSERVE -> DECIDE -> EXECUTE -> VALIDATE record.
`DecisionState`	classe	`(problem, previous_result, verification, history, facts)`	Everything a backend may look at: the problem plus the evidence so far.
`problem_from_spec`	função	`(spec) -> Dict[str, Any]`	Map a `pinneapple_problemdesign.ProblemSpec` to this module's problem dict.
`dumps`	função	`(obj) -> str`	

`pinneapple_decision.selectors`

The three MVP decisions: model, training strategy, validation strategy.

`pinneapple_decision/selectors.py` · 358 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelSelector`	classe	`()`	
`TrainingStrategySelector`	classe	`()`	
`ValidationStrategySelector`	classe	`()`	
`FactQuestion`	classe	`(name, question, fact, hints=None)`	A yes/no question whose answer is a *fact* about the problem (decision tree).

`pinneapple_decision.tree`

Declarative decision tree walked by :class:`~pinneapple_decision.loop.DecisionLoop`.

`pinneapple_decision/tree.py` · 297 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ChoiceNode`	classe	`(id, choice, edges, fact, fact_options, plan_key, plan_updates)`	
`ExecuteNode`	classe	`(id, next, plan_updates)`	
`ValidateNode`	classe	`(id, on_pass, on_fail, plan_updates)`	
`TerminalNode`	classe	`(id, outcome, plan_updates)`	
`DecisionTree`	classe	`(name, root, nodes)`	
`TreeStep`	classe	`(node, kind, selected, resolved_by, decision_id, passed, note)`	
`TreeRun`	classe	`(tree, outcome, path, plan, experiments, evidence, facts)`	
`physics_ai_tree`	função	`() -> DecisionTree`	The tree proposed for the physics-AI workflow (see module docstring).
`run_tree`	função	`(loop, problem, tree=None, *, max_experiments=3, max_steps=200) -> TreeRun`	Walk `tree` with `loop` (a :class:`DecisionLoop`). See the module docstring.

`pinneapple_decision.verifiers`

Verifiers: turn an ExecutionResult into a Verification.

`pinneapple_decision/verifiers.py` · 92 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Verifier`	classe	`()`	
`ThresholdVerifier`	classe	`(thresholds=None)`	
`VeriPhysicsVerifier`	classe	`(min_trust=70.0, min_coverage=0.5, fallback=None)`	

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 3 de 12.
- Funções de teste em arquivos que importam algum módulo do pacote: 44.

Limitações conhecidas

- *Nenhuma registrada.*

6. `pinneapple_reason`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
18	3.757	17	60	17 (1 arquivos)	8

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`		1	1	0
`cases`		1	5	17
`chain`		1	13	17
`engine`		1	3	0
`library`		1	11	17
`llm`		1	2	17
`modes`		7	20	17
`narrative`		1	2	0
`props`		1	6	17
`report`		1	4	0
`trust`		1	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_reason`

pinneapple_reason: the chain of whys of an engineering design, computed.

`pinneapple_reason/__init__.py` · 77 linhas · testes: 17

nome	tipo	assinatura	o que faz
`write_report`	função	`(chain, out_dir, lang='pt', **kw)`	Write chain.json, report.md, report.html, the figures and the trust card to `out_dir`.
`trust_report`	função	`(chain, **kw)`	
`run_case`	função	`(name, **kw) -> Chain`	Reason through one of :data:`cases.CASES`.

nome	tipo	assinatura	o que faz
`verify`	função	`(concept, requirements) -> list[Verification]`	Run every failure mode of `requirements` on `concept` (no countermeasures).
`describe_library`	função	`(lang='pt') -> dict`	The library: failure modes by domain and their countermeasures.

pinneapple_reason.__main__

`python -m pinneapple_reason`: run a case, list the library, explain a saved chain.

`pinneapple_reason/__main__.py` · 45 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	

pinneapple_reason.cases

Worked cases at different scales and in different fields, each starting from the naive concept.

`pinneapple_reason/cases.py` · 150 linhas · testes: 17

nome	tipo	assinatura	o que faz
`roof`	função	`()`	A house roof in a warm, humid climate.
`footbridge`	função	`()`	A 30 m steel footbridge (two welded girders, 2.5 m deck).
`power_board`	função	`()`	A 20 A MOSFET switch on a 4-layer board inside an enclosure at 40 °C.
`pumped_main`	função	`()`	A pumping station lifting water from a sump into a tank, and the gravity main from the tank to a valve.
`get`	função	`(name)`	

pinneapple_reason.chain

The chain of whys: requirement -> naive concept -> physical check -> failure found -> countermeasure -> check ...

`pinneapple_reason/chain.py` · 409 linhas · testes: 17

nome	tipo	assinatura	o que faz
`Requirement`	classe	`(id, text, modes, limits, source)`	What the design must do (`function`) and the failure modes that would stop it (`modes`).
`Concept`	classe	`(name, params, elements, description, sources, version)`	One version of the design: its parameters (SI units), the elements it has and where each value came from.
`Validity`	classe	`(name, value, lo, hi, note)`	One applicability condition of the model a check used: `lo <= value <= hi` (either bound may be None).
`Verification`	classe	`(mode, requirement, passed, demand, capacity, unit, margin, demand_label, capacity_label,…)`	The result of one failure mode checked on one concept.
`Finding`	classe	`(requirement, mode, title, verification)`	The first failure mode that occurs: which requirement it breaks and the number that proves it.

nome	tipo	assinatura	o que faz
`Option`	classe	`(countermeasure, element, title, feasible, reason, changes, cost, effect, target_margin_b…)`	A countermeasure the engine tried for a finding: what it changed, what it costs and what it did to every requirement (margins before / after), or why it could not be applied.
`Step`	classe	`(version, concept, verifications, finding, options, chosen, why)`	Version `version` of the design, every check run on it, the first failure (if any), the countermeasures tried for it and the one chosen (`chosen`: index into `options`).
`Chain`	classe	`(title, requirements, steps, status, notes, meta)`	The whole reasoning: requirements, every version of the design and why each one was made.
`tr`	função	`(pt, en) -> dict[str, str]`	A text in Portuguese and English.
`text_of`	função	`(t, lang='pt') -> str`	The text in `lang` (falls back to the other language, or to `str(t)`).
`param_text`	função	`(name, value, lang='pt') -> str`	`name = value unit` with the parameter's name in `lang` (raw name and value when unknown).
`element_label`	função	`(name, lang='pt') -> str`	The name of an element in `lang` (the raw name when it is not a built-in one).
`cost_text`	função	`(cost, lang='pt') -> str`	What a countermeasure costs, in words: its `what` template filled with the quantity ({q}; {pct} as a percentage, {mm} in millimetres), or `quantity unit`.

`pinneapple_reason.engine`

The reasoning engine: check every requirement, find the first failure, try the countermeasures, keep the best, check again, until every requirement passes.

`pinneapple_reason/engine.py` · 211 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Advisor`	classe	`()`	
`Reasoner`	classe	`(library=LIBRARY, advisor=None, max_steps=12, progress=None)`	
`reason`	função	`(requirements, concept, title='', *, advisor=None, max_steps=12, library=LIBRARY, progres…)`	Build the chain of whys for `requirements` starting from the naive `concept`.

`pinneapple_reason.library`

The library of failure modes and countermeasures, by physical domain.

`pinneapple_reason/library.py` · 246 linhas · testes: 17

nome	tipo	assinatura	o que faz
`Infeasible`	classe	`(pt, en)`	A countermeasure cannot be applied to this concept (the message says why, in both languages).
`Need`	classe	`(name, unit, label)`	A parameter a failure mode reads: SI unit and meaning.

nome	tipo	assinatura	o que faz
`FailureMode`	classe	`(name, domain, title, summary, evaluate_fn, needs, sources, countermeasures, uncertain, k…)`	A failure mode and its quantitative check.
`Countermeasure`	classe	`(name, element, title, effect, addresses, apply_fn, cost_fn, unit_cost, unit_cost_note, r…)`	A change that acts on the mechanism of a failure mode.
`Library`	classe	`()`	
`param`	função	`(p, name, default=None) -> Any`	
`self_check`	função	`(name, passed, value=None, limit=None, source='', reason='') -> Check`	An internal check of a computation (residual, closure, agreement with an independent solution).
`register_mode`	função	`(m, replace=False) -> FailureMode`	
`register_countermeasure`	função	`(c, replace=False) -> Countermeasure`	
`validity`	função	`(name, value, lo=None, hi=None, note='') -> Validity`	
`iter_known_answers`	função	`(names=None)`	`(mode, case)` for every known-answer case of the library (or of `names`).

pinneapple_reason.llm

An LLM as an advisor of the reasoning: it may propose candidate countermeasures and a first concept, never a number that decides anything.

`pinneapple_reason/llm.py` · 63 linhas · testes: 17

nome	tipo	assinatura	o que faz
`ExtractLLM`	classe	`()`	
`LLMAdvisor`	classe	`(llm, max_proposals=3)`	

pinneapple_reason.modes

Built-in failure modes and countermeasures, one module per physical domain (imported on first use of the library). Add a domain by writing a module like these and calling `register_mode` / `register_countermeasure`.

`pinneapple_reason/modes/__init__.py` · 6 linhas · **sem teste direto**

pinneapple_reason.modes._util

Shared helpers of the built-in failure modes.

`pinneapple_reason/modes/_util.py` · 69 linhas · testes: 17

nome	tipo	assinatura	o que faz
`verdict`	função	`(*, demand, capacity, unit, scale=None, demand_label, capacity_label, mechanism, method, …)`	A verification that passes when `demand <= capacity`; margin = (capacity - demand) / scale.

nome	tipo	assinatura	o que faz
`q`	função	`(value, unit, label) -> list`	
`target_margin`	função	`(p) -> float`	
`size_to_pass`	função	`(mode, p, values, make, limits=None, add=()) -> tuple[dict[str, Any], Any]`	The first candidate (in order) whose changes make `mode` pass with the case's design margin.
`fmt`	função	`(x, nd=3) -> str`	

`pinneapple_reason.modes.corrosion`

Corrosion: galvanic coupling of two metals in an aerated electrolyte.

`pinneapple_reason/modes/corrosion.py` · 91 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`faraday_rate`	função	`(i, metal) -> float`	Uniform wall-loss rate (m/s) of `metal` dissolving at current density `i` (A/m^2).

`pinneapple_reason.modes.fluids`

Liquids in pipes: pump cavitation (NPSH), water hammer on valve closure, column separation.

`pinneapple_reason/modes/fluids.py` · 328 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`npsh`	função	`(p) -> dict`	
`main_wave_speed`	função	`(p) -> float`	
`transient`	função	`(p, instantaneous_frictionless=False) -> dict`	

`pinneapple_reason.modes.structural`

Structures: bending strength, deflection, lateral-torsional buckling and pedestrian-induced resonance of steel girders; flexural buckling of a column; fatigue (Basquin); wind uplift of a roof covering.

`pinneapple_reason/modes/structural.py` · 571 linhas · testes: 17

nome	tipo	assinatura	o que faz
`i_section`	função	`(h, b, tf, tw) -> dict`	
`crowd_load`	função	`(span) -> float`	EN 1991-2 5.3.2.1 footbridge uniformly distributed load, Pa.
`girder_loads`	função	`(p) -> dict`	
`chi`	função	`(lam, alpha) -> float`	EN 1993-1-1 6.3.1.2 / 6.3.2.2 reduction factor for slenderness `lam` and imperfection factor `alpha`.
`m_cr`	função	`(sec, L_cr, C1=1.0) -> float`	Elastic critical moment of a doubly symmetric I-beam, load at the shear centre (Timoshenko & Gere).

nome	tipo	assinatura	o que faz
`frequency`	função	`(p) -> dict`	
`pedestrian_response`	função	`(p) -> dict`	

`pinneapple_reason.modes.thermal`

Heat: condensation under a roof on a clear night, radiant heat from a sun-baked roof, Joule heating of a power device.

`pinneapple_reason/modes/thermal.py` · 547 linhas · testes: 17

nome	tipo	assinatura	o que faz
`roof_network`	função	`(p, case) -> dict`	Steady temperatures (°C) and heat fluxes (W/m^2) of the roof for `case` = "night" or "day".
`electrothermal`	função	`(p) -> dict`	Junction temperature with P = I^2 R_on(T_j) on the concept's thermal path (fixed point).

`pinneapple_reason.modes.water`

Water: rain getting through the laps of a roof covering (capillarity, wind pressure, ponding / falling film).

`pinneapple_reason/modes/water.py` · 213 linhas · testes: 17

nome	tipo	assinatura	o que faz
`lap_heads`	função	`(p) -> dict`	Heads acting on the lap (m of water) and the lap's rise, for the parameters `p`.
`minimum_slope`	função	`(p, margin=0.0) -> float`	Smallest pitch (deg) at which the lap holds `(1 + margin)` times the head, by bisection (the film depth falls as the slope rises). `inf` if not even 75° is enough.

`pinneapple_reason.narrative`

The chain told in words: one paragraph per version, each claim followed by the number that proves it.

`pinneapple_reason/narrative.py` · 60 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`option_line`	função	`(o, lang) -> str`	
`explain`	função	`(chain, lang='pt') -> str`	

`pinneapple_reason.props`

Physical constants, properties and small closed-form relations the failure modes share, each with its source.

`pinneapple_reason/props.py` · 76 linhas · testes: 17

nome	tipo	assinatura	o que faz
`air`	função	`(t_c, p_pa=101325.0) -> dict[str, float]`	Air properties at `t_c` (k, nu, Pr, rho, cp, alpha, beta).
`saturation_pressure`	função	`(t_c) -> float`	Saturation vapour pressure of water over a flat water surface, Pa (Magnus form, Alduchov & Eskridge 1996; within 0.4 % of the reference values for -40..50 °C).

nome	tipo	assinatura	o que faz
`dew_point`	função	`(t_c, rh) -> float`	Dew-point temperature, °C, of air at `t_c` with relative humidity `rh` (0-1) (inverse Magnus form).
`sky_temperature`	função	`(t_air_k) -> float`	Effective clear-sky temperature, K: T_sky = 0.0552 T_air^1.5 (Swinbank 1963).
`wind_convection`	função	`(v) -> float`	External convection coefficient of a surface in wind, W/(m^2 K): h = 2.8 + 3.0 V (Watmuff et al. 1977).
`atmospheric_pressure`	função	`(z_m) -> float`	Standard-atmosphere pressure at altitude `z_m`, Pa (ISO 2533, troposphere).

`pinneapple_reason.report`

Reports of a chain: `chain.json`, `report.md`, `report.html` (self-contained), `trust.json` and figures.

`pinneapple_reason/report.py` · 487 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`timeline_figure`	função	`(chain, path, lang='pt') -> Path`	Version cards left to right; the arrow between two cards says what failed (with its numbers) and what was added.
`margins_figure`	função	`(chain, path, lang='pt') -> Path`	
`demand_capacity_figure`	função	`(chain, path, lang='pt') -> Path`	
`write_report`	função	`(chain, out_dir, lang='pt', card=True) -> dict[str, Path]`	Write the chain, its figures, the trust report and the reports; returns the paths.

`pinneapple_reason.trust`

Trust of a chain: the checks behind the final design, as a :class:`~pinneapple_analysis.trust.TrustReport` and as the six-stage trust card of :func:`pinneapple_lab.curation.trust_stages`.

`pinneapple_reason/trust.py` · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`known_answer_checks`	função	`(modes) -> list[Check]`	
`uncertainty_checks`	função	`(chain) -> list[Check]`	Worst margin of each final check when one uncertain premise at a time moves by +- its relative spread.
`chain_checks`	função	`(chain) -> list[tuple[Check, str]]`	`(check, kind)` for the final version of `chain`.
`trust_report`	função	`(chain, **kw) -> TrustReport`	

nome	tipo	assinatura	o que faz
`trust_card`	função	`(chain, out_dir='.', report=None) -> dict[str, dict[str, Any]]`	The six questions (data and geometry, model, physics, benchmark, uncertainty, decision) answered from the chain's own checks with :func:`pinneapple_lab.curation.trust_stages`.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 10 de 18.
- Funções de teste em arquivos que importam algum módulo do pacote: 17.

Limitações conhecidas

- *Nenhuma registrada.*

Volume 6 — Projeto e sistemas

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Geometria, otimização de projeto, projeto a partir de intenção, sistemas dinâmicos, séries temporais, co-simulação e gêmeos digitais.

Sumário

1. pinneapple_design
2. pinneapple_systems
3. pinneapple_twin3d

1. `pinneapple_design`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
196	48.153	269	740	744 (54 arquivos)	86

Pasta de testes dedicada: `tests/pinneapple_design` (88 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`aero`	Projeto aerodinâmico de ponta a ponta: perfis CST, malha O-grid e casos OpenFOAM, surrogates (MLP e MeshGraphNet), desempenho da aeronave leve e do avião comercial, busca NSGA-II e verificação por CFD.	23	111	72
`bricks`		14	108	24
`design_optimizer`	Otimização de projeto guiada por física: espaço de parâmetros, objetivos e restrições, otimizadores (gradiente, bayesiano, evolutivo), estudos de projeto com orçamento de avaliações, verificação em alta fidelidade e presets de dissipador e perfil.	18	98	179
`fitmesh`		12	38	21
`geometry`		56	255	195

sub-módulo	o que faz	módulos	classes + funções	testes *
geometry_library	Biblioteca de geometrias 3D paramétricas e realistas (aeronaves, veículos, navios, turbinas, prédios, anatomia) com registro, construtores, checagens de malha, exportadores (STL, OBJ, glTF, USD, STEP), renderizador por software e arquivos pré-gerados no Hugging Face Hub.	15	91	42
intent		19	84	24
qualitative		5	25	13
surrogate_inverse		8	104	94
thermal		19	75	46
topology		6	17	9

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_design`

pinneapple_design — Geometry construction and physics-guided design optimization.

`pinneapple_design/__init__.py` · 179 linhas · testes: 34

nome	tipo	assinatura	o que faz
build_domain	função	`(name, dim=2, **kwargs)`	Return a physics domain object by name.
sample_domain	função	`(domain, n_interior=4096, n_boundary=512)`	Sample interior and boundary collocation points from a domain.
optimize	função	`(param_space, surrogate, objective, *, constraints=None, method='bayesian', n_trials=100,…)`	Run physics-guided design optimization.

`pinneapple_design.aero`

Pacote de projeto aerodinâmico: formas de perfil, malhas O-grid, casos OpenFOAM, desempenho de aeronave e busca guiada por surrogate.

`pinneapple_design/aero/__init__.py` · 2 linhas · testes: 17

`pinneapple_design.aero.aircraft`

Converte a polar de uma seção em números de aeronave leve monomotor a hélice (classe Cessna 172 por padrão): velocidade máxima, CO2 por 100 km, velocidade de estol e checagem de requisitos.

`pinneapple_design/aero/aircraft.py` · 154 linhas · testes: 14

nome	tipo	assinatura	o que faz
`Aircraft`	classe	`(mass, wing_area, aspect_ratio, oswald, cd0_rest, power_kw, prop_efficiency, bsfc, cruise…)`	Dataclass com os dados da aeronave leve (massa, área e alongamento da asa, Oswald, CD0 do resto, potência, hélice, consumo, cruzeiro), com peso e corda derivados.
`Requirements`	classe	`(max_stall_speed, min_thickness, max_abs_cm, max_cruise_cl_ratio)`	Requisitos do design: velocidade de estol máxima, espessura mínima, |Cm| máximo e razão máxima entre CL de cruzeiro e CLmax.
`Polar`	classe	`(alpha, cl, cd, cm)`	Polar de seção a partir de coeficientes em poucos ângulos: cl(alpha) e cd, cm como funções de cl na parte colada.
`isa_density`	função	`(h) -> float`	Densidade do ar na atmosfera padrão (ISA, troposfera) na altitude `h`.
`performance`	função	`(polar, ac=Aircraft(), req=Requirements()) -> Dict[str, Any]`	Calcula o desempenho da aeronave a partir da polar: velocidade máxima, consumo e CO2 por 100 km, velocidade de estol e grandezas de cruzeiro.
`evaluate`	função	`(shape_props, polar, ac=Aircraft(), req=Requirements()) -> Dict[str, Any]`	Desempenho mais o requisito geométrico; o design é viável quando todas as checagens passam.

`pinneapple_design.aero.aircraft3d`

Avalia a aeronave inteira em 3D: vortex lattice (asa + cauda, trimada), arrasto de seção faixa a faixa pelo surrogate de CFD, arrasto e pesos por build-up de Raymer, CG, ponto neutro e estol pela seção crítica.

`pinneapple_design/aero/aircraft3d.py` · 295 linhas · testes: 6

nome	tipo	assinatura	o que faz
`Requirements3D`	classe	`(max_stall_speed, min_thickness, min_static_margin, max_static_margin, max_tail_incidence…)`	Requisitos do design 3D: estol, espessura, margem estática mínima e máxima, incidência da cauda, estação de início do estol, envergadura e razão de CL de cruzeiro.
`Aircraft3D`	classe	`(x, wing_polar, tail_polar, ac=Aircraft(), req=Requirements3D(), base_weights=None, nc=4,…)`	Avalia um design 3D a partir das polares de seção (do surrogate de CFD): condição de voo, arrasto, estol e avaliação completa.
`airframe_from`	função	`(x, base=None) -> Airframe`	Monta um `Airframe` a partir do vetor de design (6 pesos CST + parâmetros de planta).
`baseline_x`	função	`() -> np.ndarray`	Vetor de design de referência: perfil NACA 2412 mais a planta de base.
`wing_weight`	função	`(af, Wdg, q, tc, nz=5.7, Wfw=70 * 3.785 * 0.72) -> float`	Massa da asa (kg) pela fórmula de Raymer para aviação geral, a partir da massa bruta de projeto, pressão dinâmica e combustível na asa.
`htail_weight`	função	`(af, Wdg, q, nz=5.7) -> float`	Massa da cauda horizontal pela fórmula de Raymer (aviação geral).
`vtail_weight`	função	`(af, Wdg, q, nz=5.7) -> float`	Massa da cauda vertical pela fórmula de Raymer (aviação geral).
`fuselage_wetted`	função	`(af)`	Área molhada da fuselagem e seu fator de forma, integrando o perímetro das seções.

nome	tipo	assinatura	o que faz
`fuselage_cd0`	função	`(af, V, rho, mu=1.75e-05, wet=None) -> float`	CD0 da fuselagem (atrito turbulento x fator de forma x área molhada sobre a área da asa).
`vtail_cd0`	função	`(af, V, rho, mu=1.75e-05, vt=None) -> float`	CD0 da cauda vertical por build-up de componente.

`pinneapple_design.aero.airframe`

Geometria paramétrica 3D de aeronave leve (fuselagem, asa com o perfil otimizado, caudas por coeficiente de volume, hélice, trem de pouso) com exportação para glTF binário, OpenUSD e STL.

`pinneapple_design/aero/airframe.py` · 511 linhas · testes: 20

nome	tipo	assinatura	o que faz
`Part`	classe	`(name, vertices, faces, material, group, aero)`	Uma peça da malha da aeronave: nome, vértices, faces, material, grupo (a hélice gira) e se entra na forma limpa de CFD.
`Airframe`	classe	`(length, width, height, wing_area, aspect_ratio, taper, sweep_le, dihedral, twist, incide…)`	Aeronave leve paramétrica (fuselagem, asa, caudas por coeficiente de volume) que gera suas peças com `build` e expõe envergadura, corda média e dados das caudas.
`vertex_normals`	função	`(V, F) -> np.ndarray`	Normais por vértice somando as normais das faces vizinhas.
`to_glb`	função	`(parts, scalars=None, fields=None) -> bytes`	Escreve glTF 2.0 binário com materiais PBR e, opcionalmente, campos por vértice (ex.: pressão de superfície).
`to_usda`	função	`(parts, title='aircraft') -> str`	Escreve OpenUSD em texto, uma Mesh por peça com materiais UsdPreviewSurface (Omniverse, usdview).
`to_stl`	função	`(parts, solid='aircraft') -> bytes`	STL binário das peças dadas, uma superfície fechada por peça.

`pinneapple_design.aero.airliner`

Geometria paramétrica de avião comercial de corredor único (classe A320/737) com a mesma interface de `Airframe`, para usar o mesmo pipeline de vortex lattice e exportação. [sem teste]

`pinneapple_design/aero/airliner.py` · 310 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Airliner`	classe	`(length, width, height, wing_area, aspect_ratio, taper, sweep_le, dihedral, twist, incide…)`	Avião comercial paramétrico (fuselagem circular com janelas e pintura, asa enflechada, turbofans, caudas) com a mesma interface de `Airframe`.
`sweep_le_from_qc`	função	`(sweep_qc, ar, taper) -> float`	Converte o enflechamento a 1/4 da corda em enflechamento do bordo de ataque, dados alongamento e afilamento.

`pinneapple_design.aero.airliner3d`

Física do avião comercial (classe A320): cruzeiro transônico com Prandtl-Glauert, arrasto de onda (Korn + regra de Lock), alcance de Breguet com MTOW iterado, e lado de baixa velocidade. [sem teste]

`pinneapple_design/aero/airliner3d.py` · 340 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Mission`	classe	`(passengers, mass_per_pax, range_km, cruise_altitude, tsfc, reserve, oew_rest, high_lift_…)`	Missão de projeto do avião comercial: passageiros, massa por passageiro, alcance, altitude de cruzeiro, consumo específico do turbofan e reserva.
`RequirementsAL`	classe	`(max_span, max_vref_kt, buffet_margin, min_static_margin, max_static_margin, max_stall_st…)`	Requisitos do avião comercial: envergadura máxima (código de aeródromo), velocidade de aproximação, margem de buffet, margem estática e estação de estol.
`AirlinerModel`	classe	`(x, wing_polar, tail_polar, mission=Mission(), req=RequirementsAL(), nc=4, ns_wing=12, ns…)`	Avalia um design de avião comercial: faixas da asa, arrasto de onda, cruzeiro, arrasto parasita, estol, volume de combustível e avaliação completa.
`isa`	função	`(h)`	Atmosfera padrão até a estratosfera baixa: densidade, velocidade do som e viscosidade (Sutherland) na altitude `h`.
`airliner_from`	função	`(x) -> Airliner`	Monta um `Airliner` a partir do vetor de design (seção CST + planta + Mach + posição da asa).
`baseline_x_al`	função	`() -> np.ndarray`	Vetor de design de referência do avião comercial (NACA 2412 + planta de base).
`wing_weight_transport`	função	`(af, mtow, tc, nz=3.75) -> float`	Massa da asa pela fórmula de Raymer para aviões de transporte, com um fator de correção do módulo.

`pinneapple_design.aero.buildup`

Arrasto de sustentação nula (CD0) de uma aeronave da biblioteca de geometrias a partir da malha: atrito turbulento de placa plana por componente, fatores de forma de Raymer, interferência e excrescências. [sem teste]

`pinneapple_design/aero/buildup.py` · 57 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sutherland_mu`	função	`(T) -> float`	Viscosidade dinâmica do ar pela lei de Sutherland, em Pa.s.
`friction_drag`	função	`(geometry, ac, *, rho, V, T, mach, components=None, interference=1.08) -> dict[str, Any]`	CD0 (sobre a área da asa) dos corpos de uma `Geometry` da biblioteca cujos nomes casam com os componentes dados.

`pinneapple_design.aero.case`

Monta, roda e lê um caso OpenFOAM 2D (simpleFoam, k-omega SST resolvido até a parede) de um perfil em um ângulo de ataque; também empacota o caso em zip pronto para rodar.

`pinneapple_design/aero/case.py` · 302 linhas · testes: 22

nome	tipo	assinatura	o que faz
write_case	função	(case, shape, alpha_deg, reynolds=4000000.0, iterations=1200, tu=0.001, grid=GridSpec(), …)	Escreve o diretório do caso OpenFOAM (polyMesh, condições de contorno, controle) para uma forma, ângulo e Reynolds.
read_internal	função	(path) -> np.ndarray	Lê o internalField de um arquivo de campo ASCII do OpenFOAM como array (n,) ou (n, 3).
wall_geometry	função	(P2) -> Dict[str, np.ndarray]	Por face de parede: centro, comprimento, normal para dentro do corpo e distância do primeiro centro de célula.
forces	função	(P2, p_wall, U_first, nu, alpha_deg) -> Dict[str, Any]	Cl, Cd, Cm (a 1/4 da corda) e as distribuições de Cp e Cf a partir das células junto à parede.
time_dirs	função	(case)	Lista os diretórios de tempo numéricos de um caso OpenFOAM, em ordem.
read_coefficients	função	(case, P2, nu, alpha_deg, ni) -> Dict[str, Any]	Coeficientes em cada escrita guardada (checagem de convergência) e o resultado completo na última.
run_case	função	(case, shape, alpha_deg, reynolds=4000000.0, iterations=1200, grid=GridSpec(), timeout=18…)	Escreve e roda o simpleFoam e devolve coeficientes, veredito de convergência e tempo.
case_zip	função	(shape, alpha_deg, reynolds=4000000.0, iterations=1200, grid=None) -> bytes	Caso OpenFOAM pronto para rodar (com script de coeficientes e Allrun) como bytes de zip.

pinneapple_design.aero.case3d

Caso OpenFOAM 3D da aeronave inteira (meio modelo com plano de simetria): STL do Airframe, snappyHexMesh, simpleFoam com funções de parede e leitura de forças, Cp, Cf e linhas de corrente. [sem teste]

pinneapple_design/aero/case3d.py · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
write_case3d	função	(case, af, alpha_deg=2.0, speed=55.0, nu=1.5e-05, iterations=800, level_wing=(5, 6), proc…)	Escreve o caso 3D: STLs dos corpos do Airframe, snappyHexMesh e configuração do simpleFoam.
run_case3d	função	(case, procs=4, log=print) -> Dict[str, float]	Roda o caso 3D em paralelo (delegando a run_case).
read_result3d	função	(case, info, af, n_lines=34) -> Dict[str, Any]	Lê forças (pressão + atrito), Cp e Cf por face, linhas de corrente, esteira e fatias no plano de simetria.
surface_scalars	função	(af, parts, surface, key='cp') -> Dict[str, np.ndarray]	Passa um campo de superfície do CFD (cp ou cf) para os vértices da malha visual, por interpolação de faces vizinhas e espelhamento.

pinneapple_design.aero.dataset

Conjunto de treino OpenFOAM dos surrogates de perfil: plano de rodadas (hipercubo latino de formas x ângulos + polares NACA), execução de uma rodada e regra do valor estacionário.

pinneapple_design/aero/dataset.py · 92 linhas · testes: 13

nome	tipo	assinatura	o que faz
`plan`	função	`(n_shapes, n_alpha, seed=7) -> list[dict[str, Any]]`	Lista de rodadas: formas válidas por hipercubo latino x ângulos em faixas, mais as polares NACA de referência; determinística pela semente.
`run_job`	função	`(job, out) -> tuple[str, Any, Any]`	Roda um job no OpenFOAM e grava `out/runs/<id>.json` (e `.npz` com campos de célula quando rodou).
`steady_values`	função	`(runs, max_osc=MAX_CL_OSC) -> list[dict[str, Any]]`	Cl, Cd, Cm estacionários de cada rodada; rodadas instáveis são médias das últimas escritas ou descartadas se oscilam demais.

`pinneapple_design.aero.field_dataset`

Grava o dataset de campos do zoo (`airfoil_rans_cst_re4e6_fields`): campos de célula p, Ux, Uy, nut de cada rodada da campanha OpenFOAM, em arquivos `.npz` por split.

`pinneapple_design/aero/field_dataset.py` · 98 linhas · testes: 4

nome	tipo	assinatura	o que faz
`write_field_dataset`	função	`(run_dir, out, name=…, shard_rows=SHARD_ROWS) -> dict`	Escreve o dataset de campos do zoo (train, test, naca_reference em `.npz` e `meta.json`) a partir dos arquivos de rodada da campanha.

`pinneapple_design.aero.geometry`

Formas de perfil CST (Kulfan) com 6 pesos de espessura e arqueamento, ajuste de NACA 4 dígitos, propriedades derivadas, validade, hipercubo latino e alças de edição.

`pinneapple_design/aero/geometry.py` · 153 linhas · testes: 72

nome	tipo	assinatura	o que faz
`Loop`	classe	`(xy, n_te)`	Laço fechado de parede para a O-grid: base do bordo de fuga, superfície inferior e superior.
`cst_surface`	função	`(w, x, te) -> np.ndarray`	Avalia uma superfície CST (função de classe x Bernstein + espessura do bordo de fuga) nas estações x.
`surfaces`	função	`(shape, x) -> tuple[np.ndarray, np.ndarray]`	y das superfícies superior e inferior nas estações x (corda 1).
`fit_cst`	função	`(xu, yu, xl, yl) -> np.ndarray`	Pesos CST por mínimos quadrados para coordenadas dadas de extradorso e intradorso.
`naca4`	função	`(code, n=200) -> np.ndarray`	Ajuste CST de um perfil NACA 4 dígitos, ex.: `naca4("2412")`.
`properties`	função	`(shape) -> dict[str, float]`	Espessura e arqueamento máximos e suas posições, raio do bordo de ataque e área da seção.
`valid`	função	`(shape) -> tuple[bool, str]`	Testa se a forma é válida (superfícies não se cruzam, curvatura limitada) e devolve o motivo quando não é.
`outline`	função	`(shape, n=80) -> dict[str, list]`	Contorno do perfil em espaçamento cosseno, como dicionário de listas (para o navegador).
`latin_hypercube`	função	`(n, bounds, rng) -> np.ndarray`	Amostra `n` pontos por hipercubo latino dentro dos limites dados.

nome	tipo	assinatura	o que faz
`wall_loop`	função	`(shape, n_surf=80, n_te=4) -> Loop`	Constrói o `Loop` de parede da forma, com pontos agrupados no bordo de ataque e de fuga.
`handles`	função	`(shape, xh=HANDLE_X) -> tuple[np.ndarray, np.ndarray]`	y das superfícies superior e inferior nas estações de alça (três por superfície).
`shape_from_handles`	função	`(y_upper, y_lower, xh=HANDLE_X) -> np.ndarray`	Forma CST cujas superfícies passam pelas alturas de alça dadas.
`out_of_bounds`	função	`(shape) -> list`	Nomes dos pesos CST fora da caixa de treino dos surrogates (`BOUNDS`).

`pinneapple_design.aero.geometry_onnx`

Exporta a geometria CST como grafos ONNX float64 (geometria e conversão de alças para forma) que reproduzem as funções NumPy, para uso no navegador.

`pinneapple_design/aero/geometry_onnx.py` · 188 linhas · testes: 13

nome	tipo	assinatura	o que faz
`torch_geometry`	função	`(n_outline=N_OUTLINE)`	Módulo torch que leva a forma (n, 6) ao dicionário de saídas de geometria em float64.
`torch_from_handles`	função	`()`	Módulo torch que leva alturas de alça (n, 6) à forma CST (n, 6), como `shape_from_handles`.
`export_onnx`	função	`(directory, *, presets=None) -> dict[str, Any]`	Grava `cst_geometry.onnx`, `cst_from_handles.onnx` e `cst_geometry.json` no diretório e devolve o JSON.
`reference`	função	`(shapes) -> dict[str, np.ndarray]`	Valores NumPy que os grafos ONNX devem reproduzir (usado nos testes).

`pinneapple_design.aero.mesh`

Gera a O-grid estruturada em volta do perfil, com a mesma topologia para todo design, e escreve direto o polyMesh do OpenFOAM.

`pinneapple_design/aero/mesh.py` · 182 linhas · testes: 21

nome	tipo	assinatura	o que faz
`GridSpec`	classe	`(n_surf, n_te, nj, first, radius, span)`	Parâmetros da O-grid: células ao longo da superfície e do bordo de fuga, camadas até o campo distante, altura da primeira célula e raio externo.
`grid_points`	função	`(shape, g=GridSpec()) -> np.ndarray`	Pontos da grade (nj+1, ni, 2), com j = 0 na parede.
`topology`	função	`(g=GridSpec()) -> Dict[str, np.ndarray]`	Faces da grade 2D na ordem do OpenFOAM, como índices (i, j) dos pontos.
`write_polymesh`	função	`(shape, case, g=GridSpec()) -> np.ndarray`	Escreve `constant/polyMesh` para a forma e devolve os pontos.
`cell_centres`	função	`(P2) -> np.ndarray`	Centros das células (nj, ni, 2) a partir dos pontos da grade.

pinneapple_design.aero.optimize

Busca de projeto NSGA-II sobre forma do perfil + área de asa no surrogate, com três objetivos (velocidade máxima, CO2, velocidade de estol), viabilidade e nível de confiança por design.

`pinneapple_design/aero/optimize.py` · 183 linhas · testes: 23

nome	tipo	assinatura	o que faz
`Engine`	classe	`(bundle)`	Junta os surrogates treinados e a avaliação da aeronave: carrega o bundle, prevê coeficientes (MLP e GNN), avalia e faz a busca NSGA-II.
`baseline`	função	`(engine, aircraft, req, name='NACA 2412') -> Dict[str, Any]`	Avalia o design de referência (um perfil NACA com a área de asa da aeronave).
`summarize`	função	`(rec) -> Dict[str, Any]`	Forma compacta de um design enviada ao navegador para a nuvem de designs.
`infeasibility_reasons`	função	`(designs) -> dict[str, int]`	Conta, por requisito violado, por que os designs de uma busca ficaram fora do conjunto de Pareto.

pinneapple_design.aero.optimize3d

Busca NSGA-II da aeronave inteira (perfil + planta + posição da asa), avaliada com vortex lattice, surrogate de seção e build-up de componentes.

`pinneapple_design/aero/optimize3d.py` · 107 linhas · testes: 6

nome	tipo	assinatura	o que faz
`Engine3D`	classe	`(engine)`	Motor da busca da aeronave inteira: pesos de base, avaliação 3D de um lote e busca NSGA-II.
`summarize3d`	função	`(r) -> Dict[str, Any]`	Forma compacta de um registro da busca 3D (velocidades, CO2, viabilidade, margem estática, etc.).

pinneapple_design.aero.optimize_airliner

Busca NSGA-II do avião comercial: seção da asa, planta, Mach de cruzeiro e posição da asa; objetivos CO2 por passageiro-km, Mach e velocidade de aproximação. [sem teste]

`pinneapple_design/aero/optimize_airliner.py` · 102 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EngineAL`	classe	`(engine)`	Motor da busca do avião comercial: avaliação de um lote de designs com a missão e busca NSGA-II.
`summarize_al`	função	`(r) -> Dict[str, Any]`	Forma compacta de um registro da busca do avião comercial (CO2 por passageiro-km, Mach, Vref, MTOW, etc.).

pinneapple_design.aero.panel

Método de painéis de Hess-Smith (escoamento potencial incompressível) em volta de um perfil CST: a referência invíscida do benchmark de perfil.

`pinneapple_design/aero/panel.py` · 128 linhas · testes: 31

nome	tipo	assinatura	o que faz
`PanelResult`	classe	`(nodes, sigma, gamma, alpha_deg, x_mid, y_mid, cp, cl_kutta_joukowski, cl_pressure, cd_pr…)`	Resultado do método de painéis: nós, intensidades de fonte, vórtice, pontos médios, com cálculo de velocidade em pontos.
`airfoil_nodes`	função	`(shape, n_per_side=100, close_te=False) -> np.ndarray`	Nós dos painéis em sentido horário, agrupados nas pontas; opcionalmente fecha o bordo de fuga rombudo.
`panel_solve`	função	`(shape=None, alpha_deg=0.0, *, nodes=None, n_per_side=100, close_te=False) -> PanelResult`	Resolve o escoamento potencial em volta de um perfil CST (ou de nós dados) num ângulo de ataque.

`pinneapple_design.aero.render_blender`

Renderização fotorrealista de um `.glb` de aeronave com Blender Cycles, com opção de colorir por Cp/Cf e desenhar linhas de corrente; exige `bpy`. [sem teste]

`pinneapple_design/aero/render_blender.py` · 250 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`render`	função	`(glb, out, view='hero', samples=96, size=(1600, 900), cp=False, lines_npz=None, sun_eleva…)`	Renderiza um `.glb` com Blender Cycles numa vista, com opção de campo Cp/Cf, modo estúdio e linhas de corrente.
`main`	função	`(argv=None)`	Ponto de entrada de linha de comando do render (argparse).

`pinneapple_design.aero.surrogate`

Surrogates do escoamento no perfil: uma MeshGraphNet nas células da O-grid (campos + Cl, Cd, Cm) e um ensemble de MLPs cuja dispersão é a incerteza.

`pinneapple_design/aero/surrogate.py` · 134 linhas · testes: 9

nome	tipo	assinatura	o que faz
`Layout`	classe	`(grid, i_step, max_dist, j_sel, i_sel, edge_index)`	Subconjunto fixo de células da O-grid perto do perfil usado como grafo, com as arestas multiescala.
`AeroGNN`	classe	`(node_in, edge_in=3, hidden=48, mp=8, n_fields=4)`	MeshGraphNet que prevê os campos p, Ux, Uy, log(1+nut/nu) por nó e, por uma leitura agregada na parede, Cl, Cd, Cm.
`MLPEnsemble`	classe	`(k=5, width=96)`	K MLPs pequenas em (forma, alpha) -> (Cl, log Cd, Cm); a dispersão entre elas é a incerteza.
`node_features`	função	`(lay, shape, alpha, pos=None) -> np.ndarray`	Atributos de nó do grafo: posição, log da distância à parede, nível, flag de parede, direção do escoamento e pesos da forma.
`edge_features`	função	`(lay, pos) -> np.ndarray`	Atributos de aresta: direção unitária e log do comprimento.
`coef_targets`	função	`(cl, cd, cm)`	Empilha (Cl, log Cd, Cm) como alvos de treino.
`coef_from_targets`	função	`(t)`	Inverte `coef_targets`: devolve Cl, Cd, Cm.

pinneapple_design.aero.train

Treina os dois surrogates de perfil com as rodadas OpenFOAM e mede o erro em formas nunca vistas (split por forma) e nas polares NACA.

`pinneapple_design/aero/train.py` · 334 linhas · testes: 13

nome	tipo	assinatura	o que faz
`load_runs`	função	`(dirs, max_osc=MAX_CL_OSC) -> list[dict[str, Any]]`	Carrega os registros de rodada utilizáveis de cada diretório, com coeficientes estacionários e o caminho dos campos.
`split`	função	`(recs, frac=0.15, seed=3)`	Divide em (treino, teste, referência) separando formas inteiras para teste.
`coefs`	função	`(recs) -> np.ndarray`	Array (n, 3) de (Cl, Cd, Cm) dos registros.
`metrics`	função	`(true, pred) -> dict[str, dict[str, float]]`	Erros médio e máximo e R^2 por coeficiente; para Cd também erro relativo médio e percentil 90.
`train_mlp`	função	`(tr, k=5, width=96, epochs=6000, seed=0)`	Treina k MLPs em reamostragens bootstrap (Adam + agenda cosseno) e devolve modelo e normalização.
`predict_mlp`	função	`(model, norm, recs)`	Média e dispersão do ensemble para (Cl, Cd, Cm).
`graph_data`	função	`(lay, recs)`	Monta atributos de nó e aresta e alvos de campo e coeficiente de cada rodada a partir dos `.npz`.
`train_gnn`	função	`(tr, te, minutes=60.0, seed=0, batch=4, log=print, *, epochs=None, hidden=48, mp=8, layou…)`	Treina a rede de grafos por um orçamento de tempo e/ou épocas, com taxa de aprendizado em cosseno.
`predict_gnn`	função	`(model, norm, lay, recs)`	Prevê (Cl, Cd, Cm) com a GNN e calcula o RMSE de cada campo contra as rodadas.
`train_surrogates`	função	`(recs, gnn_minutes=60.0, gnn_from=None, log=print, *, mlp_k=5, mlp_width=96, mlp_epochs=6…)`	Divide os dados, treina os dois surrogates e mede; devolve `(bundle, metrics)`.

pinneapple_design.aero.verify

Confere a busca com OpenFOAM: escolhe designs ao longo da frente de Pareto, roda suas polares e compara surrogate x CFD; as rodadas servem para aprendizado ativo. [sem teste]

`pinneapple_design/aero/verify.py` · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pick`	função	`(front, n) -> list[int]`	Escolhe `n` designs espaçados ao longo da frente, sempre com as duas pontas.
`cfd_eval`	função	`(runs, shape, area, ac, req) -> dict[str, Any]`	Desempenho e checagens da aeronave a partir da polar OpenFOAM de uma forma com dada área de asa.
`verification_jobs`	função	`(designs, chosen, round_)`	Gera os jobs OpenFOAM (um por design e ângulo) no formato do dataset.
`compare_design`	função	`(engine, did, d, runs, ac, req) -> dict[str, Any]`	Compara surrogate e OpenFOAM para um design da busca.

pinneapple_design.aero.vlm

Vortex lattice (ferraduras) para asa + cauda horizontal de um `Airframe`; a solução linear dá sustentação, momento, carregamento, arrasto induzido, trimagem e ponto neutro.

`pinneapple_design/aero/vlm.py` · 257 linhas · testes: 6

nome	tipo	assinatura	o que faz
`Lattice`	classe	`(P1, P2, C, nrm0, surf, strip, chord, yc, dy, strip_surf, tail_rot)`	Malha de ferraduras: pontos dos vórtices ligados, pontos de controle, normais, superfície (asa ou cauda) e faixa.
`VLMSolution`	classe	`(L, G, s_ref, c_ref, b_ref, x_ref, CL, Cm, CLw, cl_strip, Q)`	Solução do VLM linear em alpha e incidência da cauda: coeficientes, trimagem, cl por faixa e ponto neutro.
`build_lattice`	função	`(af, nc=5, ns_wing=14, ns_tail=6, tail=True) -> Lattice`	Monta o `Lattice` da asa e da cauda de um `Airframe` (enflechamento, diedro, torção, arqueamento).
`horseshoe_induced`	função	`(P, L, legs=True)`	Velocidade induzida pelas ferraduras do lattice em pontos dados (perna ligada e pernas livres).
`solve`	função	`(af, x_ref, nc=5, ns_wing=14, ns_tail=6, tail=True) -> VLMSolution`	Resolve o VLM para o `Airframe` e devolve um `VLMSolution`.
`streamlines`	função	`(sol, alpha, it, seeds, steps=220, h=0.08, core=0.08) -> np.ndarray`	Linhas de corrente do escoamento potencial (corrente livre + ferraduras com núcleo dessingularizado).

pinneapple_design.bricks

Interlocking toy bricks: geometry, physics and a self-modifying design loop.

`pinneapple_design/bricks/__init__.py` · 67 linhas · testes: 24

pinneapple_design.bricks.assembly

Grid assemblies of interlocking bricks: placement, stud-tube connectivity and validity rules.

`pinneapple_design/bricks/assembly.py` · 315 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Brick`	classe	`(part, x, y, z, rot, colour)`	
`Ground`	classe	`(kind, regions)`	`kind`: `"table"` (smooth, friction `mu`) or `"baseplate"` (studs). `regions`: rectangles in stud units where the ground exists (`None` = everywhere).
`Connection`	classe	`(lower, upper, z, rect, studs)`	Contact between `lower` (brick index, `-1` = ground) and `upper` at plate height `z`.
`Assembly`	classe	`(bricks=None, ground=None, name='')`	A list of bricks on a stud grid plus the ground they stand on.
`colour_rgb`	função	`(name) -> tuple[float, float, float]`	

nome	tipo	assinatura	o que faz
stack	função	`(part_name, n, *, offsets=None, colours=('red',), ground=None) -> Assembly`	`n` copies of a part stacked straight up, brick `k` shifted by `offsets[k]` studs along x.

pinneapple_design.bricks.closed_form

Closed-form references for brick statics (used to validate :mod:`.statics`, :mod:`.dynamics` and :mod:`.fea`).

`pinneapple_design/bricks/closed_form.py` · 77 linhas · testes: 24

nome	tipo	assinatura	o que faz
harmonic	função	`(n) -> float`	
harmonic_overhang	função	`(n, length) -> float`	Maximum overhang of `n` unconnected identical blocks, one per level: (L / 2) H_n.
harmonic_stack	função	`(n, length, edge=0.0) -> np.ndarray`	Left-edge x positions of the blocks (bottom first) of the harmonic stack on a table ending at `edge`.
tip_over_offset	função	`(length) -> float`	Largest overhang of a single uniform block on a table edge: half its length.
stud_cantilever_tension	função	`(weight, d, studs_x, x1) -> float`	Equal tension per stud [N] that holds a part of weight `weight` [N] with its centre of mass at `d`, pivoting about the patch edge `x1` (0 if the centre of mass is inside the patch).
beam_reactions	função	`(span, w, P) -> float`	Each support reaction of a symmetric simply supported beam [N].
beam_moment	função	`(x, span, w, P) -> float`	Bending moment at `x` (from a support) of the simply supported beam with a central load [N mm].
beam_midspan_deflection	função	`(span, E, inertia, P=0.0, w=0.0) -> float`	Euler-Bernoulli mid-span deflection of a simply supported beam (consistent units).
tower_tip_force	função	`(weight, half_base, height) -> float`	Horizontal force at `height` that tips a rigid block of weight `weight` about the base edge at `half_base` from its centre of mass line: F = W b / h.

pinneapple_design.bricks.dynamics

Rigid-body replay of a brick build with NVIDIA Newton (:mod:`pinneapple_simulation.external_solvers.newton_physics`).

`pinneapple_design/bricks/dynamics.py` · 239 linhas · testes: 24

nome	tipo	assinatura	o que faz
Replay	classe	`(t, pos, quat, broken, max_displacement_mm, moved, solver, wall_time_s, max_tilt_deg)`	

nome	tipo	assinatura	o que faz
table_boxes	função	(asm, margin_mm=80.0) -> list[tuple[float, float, float, float]]	The ground regions clipped to the build's neighbourhood, (x0, x1, y0, y1) in millimetres.
clusters	função	(asm, broken=()) -> tuple[list[list[int]], set[int]]	Groups of bricks held together by intact stud joints, and the groups welded to a baseplate.
rigid_scene	função	(asm, broken=(), joint=None, dead_loads=())	Newton :class:RigidScene of the build (metres): one rigid body per cluster of bricks held by intact stud joints (a compound of their boxes), welded to the world when it is held by a baseplate. dead_loads: :class:~.statics.Load whose downward part is a dead weight, added as a small box at …
replay	função	(asm, broken=(), *, duration=0.8, dt=0.001, solver='xpbd', iterations=16, frames=40, load…)	Simulate the build released at rest with broken joints let go. loads (:class:~.statics.Load): a downward load is a dead weight (a small box of that weight at the load point, falling with the part instead of pushing it with a constant force); horizontal parts are constant forces with their …
harmonic_threshold	função	(n=4, length_mm=63.8, *, factors=…, duration=0.8, mu=0.4, tol_mm=1.0, solver='xpbd') -> l…)	Newton on the harmonic stack of n smooth blocks with every offset scaled by f: does the top block stay (closed form: stands for f <= 1, falls for f > 1)? One row per factor.
cross_check	função	(goal, candidates, *, max_n=30, duration=0.4, tol_mm=2.0, dt=0.00025, iterations=32, seed…)	Does Newton agree with the statics verdict? For up to max_n evaluated candidates (.asm, .ev) whose verdict Newton can judge with every stud joint intact (stable: nothing should move by more than tol_mm; mechanism: something should), replay them under the goal's loads. Joint …
friction_creep	função	(asm, loads, *, dts=(0.001, 0.00025), duration=0.6, iterations=32) -> list[dict]	Largest displacement of a build that should hold still under loads, for several time steps: a sustained sideways load makes XPBD friction creep by an amount that shrinks with dt (a solver artefact).

pinneapple_design.bricks.fea

Linear-elastic finite elements on the voxelised build (Hex8 with incompatible modes, :mod:solid_fem).

pinneapple_design/bricks/fea.py · 127 linhas · testes: 24

nome	tipo	assinatura	o que faz
VoxelModel	classe	(mesh, element_brick, element_volume)	
FEAResult	classe	(fem, model, brick_displacement, brick_von_mises, max_displacement)	
voxel_mesh	função	(asm, per_stud=2) -> VoxelModel	Hexahedral mesh of the occupied cells, per_stud elements across a stud pitch, one per plate height.

nome	tipo	assinatura	o que faz
solve	função	`(asm, loads=(), *, E=E_ABS_MPA, nu=NU_ABS, per_stud=2, gravity=True, supports='ground', f…)`	Static linear elasticity of the voxelised build under gravity and point loads (:class:~.statics.Load).

pinneapple_design.bricks.goals

Design goals for the self-modifying loop: each one scores a build with the statics check and says why it fails.

pinneapple_design/bricks/goals.py · 285 linhas · testes: 24 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Evaluation	classe	`(score, feasible, violation, n_bricks, status, explanation, metrics, critical)`	
Goal	classe	`(joint=None)`	Base class. evaluate returns an :class:Evaluation whose violation follows one scale for every goal: invalid build >= 5, unsupported parts >= 2, goal-specific geometric miss (e.g. no deck) in [2, 3), fails under its own weight in [1, 2), overloaded (carries itself but not the load) in [0, …
CantileverGoal	classe	`(max_bricks=10, joint=None)`	Reach as far past a table edge as possible with at most max_bricks 1-wide parts, under gravity.
BridgeGoal	classe	`(gap=12, load_n=1.0, max_bricks=24, joint=None)`	Span a gap (in studs) between two tables and carry load_n newtons at mid-span on the deck, with the fewest parts. Feasible when the statics check carries gravity plus the load (load factor >= 1).
TowerGoal	classe	`(max_bricks=16, push_n=0.01, base=6, joint=None)`	The tallest tower on a base x base stud footprint that survives a horizontal push push_n [N] at its top (and gravity) with at most max_bricks parts; it stands on a smooth table, so it can tip as well as break.
reference_cantilever	função	`(n, length_mm=63.8) -> dict[str, float]`	Closed-form reaches for n parts of length length_mm: a single block (L/2), harmonic stacking of unconnected blocks ((L/2) H_n) and the centre-of-mass bound for joined parts (n L / 2: all the length in one straight beam with its centre of mass on the edge; stud joints need overlaps, so real …

pinneapple_design.bricks.ldraw

LDraw interchange: write and read .ldr / .mpd files, and load real part shapes from the LDraw parts library.

pinneapple_design/bricks/ldraw.py · 281 linhas · testes: 24

nome	tipo	assinatura	o que faz
ReadReport	classe	`(assembly, skipped)`	
PartsLibrary	classe	`(root)`	Resolve and triangulate LDraw .dat files (line types 1, 3, 4; BFC winding is not enforced).

nome	tipo	assinatura	o que faz
`to_ldr`	função	`(asm, name='model.ldr', *, steps=True, author='PINNeAPPle') -> str`	LDraw text of the build; with `steps` one `0 STEP` per layer (bottom first), as building instructions.
`write_ldr`	função	`(asm, path, **kw) -> str`	
`write_mpd`	função	`(models, path) -> str`	Several builds in one multi-part document; the first is the main model and references the others.
`from_ldr`	função	`(text, *, ground=None, tol=0.5) -> ReadReport`	Read .ldr or .mpd text. Catalogue parts on the grid (positions within `tol` LDU, quarter-turn yaws) become bricks; everything else is listed in `skipped`. Sub-models of an .mpd are expanded.
`read_ldr`	função	`(path, **kw) -> ReadReport`	
`default_cache`	função	`() -> str`	
`fetch_parts_library`	função	`(cache_dir=None, urls=LIBRARY_URLS, timeout=120.0) -> str`	Download and unpack the LDraw parts library (CC BY 2.0) into `cache_dir` once; returns the folder that holds `parts/` and `p/`. Raises `RuntimeError` with every URL tried when it cannot be downloaded.

`pinneapple_design.bricks.mesh`

Triangle meshes of bricks and assemblies, and export to glTF (per-brick colours), STL and USD.

`pinneapple_design/bricks/mesh.py` · 119 linhas · testes: 24

nome	tipo	assinatura	o que faz
`box_mesh`	função	`(lo, hi) -> tuple[np.ndarray, np.ndarray]`	Closed, outward-oriented box with four vertices per face (sharp edges under smooth-normal shading).
`cylinder_mesh`	função	`(cx, cy, z0, r, h, n=16) -> tuple[np.ndarray, np.ndarray]`	Closed cylinder on `z0`: a smooth side and flat caps with their own vertices.
`merge`	função	`(meshes) -> tuple[np.ndarray, np.ndarray]`	
`brick_mesh`	função	`(b, *, studs=True, segments=16, local=False) -> tuple[np.ndarray, np.ndarray]`	Mesh of one placed brick in millimetres (world coordinates, or relative to its centre with `local`).
`assembly_meshes`	função	`(asm, **kw)`	`(vertices_mm, faces, srgb)` per brick.
`studio_scene`	função	`(asm, title='brick build', fields=None)`	The build as a `pp.viz` studio Scene (metres, z up) with one material per colour, and that material table.
`write_gltf`	função	`(asm, path, title='brick build') -> str`	Binary glTF (.glb) with one node per brick and its colour, in metres.
`write_usd`	função	`(asm, path, title='brick build') -> str`	
`write_stl`	função	`(asm, path, title='brick build') -> str`	Binary STL of the whole build in millimetres.

pinneapple_design.bricks.parts

Interlocking toy bricks: nominal dimensions, the part catalogue, masses and colours.

pinneapple_design/bricks/parts.py · 215 linhas · testes: 24

nome	tipo	assinatura	o que faz
PartType	classe	(name, nx, ny, h, ldraw, mass, mass_source, smooth)	A rectangular brick or plate: nx x ny studs, h plates high (3 = brick, 1 = plate).
Colour	classe	(name, ldraw, rgb)	
JointModel	classe	(clutch_n, clutch_band, mu, shear_n, edge_inset_mm, source, notes)	Strength of a stud joint and contact friction used by the statics check.
fit_mass_model	função	() -> np.ndarray	Least-squares coefficients (a, b) of m = a * studs + b * perimeter_mm * height_mm [kg] fitted to the published brick masses.
estimate_mass	função	(nx, ny, h) -> float	Mass of an nx x ny part h plates high [kg] from the shell model of :func:fit_mass_model.
mass_model_residuals	função	() -> dict[str, float]	Relative error of the shell model on each published mass (leave-nothing-out fit).
part	função	(name) -> PartType	A catalogue part by name ("brick_2x4") or LDraw file ("3001.dat").
smooth_block	função	(nx, ny, h, mass=None) -> PartType	A stud-less block (no clutch with anything) of the given footprint, e.g. for unconnected stacking tests.
colour	função	(name_or_code) -> Colour	

pinneapple_design.bricks.report

Summaries of a brick experiment: sensitivity to the joint model, the strategy comparison chart, a Markdown report.

pinneapple_design/bricks/report.py · 191 linhas · testes: 24

nome	tipo	assinatura	o que faz
sensitivity	função	(goal, asm, factors=None, shear_caps=(None, 1.0, 0.3)) -> list[dict]	Re-evaluate asm with the clutch scaled over its assumed band and with capped stud shear.
plot_comparison	função	(summaries, path, *, ylabels=None, display=None) -> str	Best feasible score versus physics evaluations: one panel per goal, per strategy the median over the seeds (a seed without a feasible build counts as the worst, so the line starts once most seeds are feasible) and the range of the feasible seeds, with the feasible count at the end. …
write_report	função	(path, *, validation, comparisons, best, sensitivities, references=None, files=None, join…)	Markdown report: what was measured, the closed-form checks (and where they disagree), the strategy comparison with its spread over seeds, the best builds, the sensitivity to the joint model and its limits.

pinneapple_design.bricks.search

The self-modifying loop: mutate a build, simulate every candidate, keep what the physics accepts.

`pinneapple_design/bricks/search.py` · 371 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Candidate`	classe	`(asm, ev, edit, generation, parent, index)`	
`SearchResult`	classe	`(strategy, goal, seed, budget, history, generations, best_curve, wall_time_s, meta)`	
`mutate`	função	`(goal, asm, rng, last=None, edit=None) -> tuple[Assembly, str]`	One random edit (or `edit`). Returns the new build and the edit name; the build may be invalid.
`repair`	função	`(goal, asm) -> Assembly`	Drop parts that overlap an earlier one or float, so most edits give a buildable candidate.
`evolve`	função	`(goal, budget=200, *, seed=0, mu=10, lam=10, progress=None) -> SearchResult`	(mu + lambda) evolution of builds with NSGA-II selection; `budget` physics evaluations.
`surrogate_search`	função	`(goal, budget=200, *, seed=0, mu=10, lam=10, pool=8, kappa=1.0, warmup=20, progress=None)…)`	Same loop, but each generation proposes `pool * lam` children, a GP surrogate trained on all simulated builds ranks them by `mean + kappa * std` of a penalised score, and only the top `lam` are simulated.
`compare`	função	`(goal_factory, *, budget=200, seeds=(0, 1, 2, 3, 4), strategies=('evolution', 'surrogate'…)`	Run every strategy with every seed at the same evaluation budget. `kw` (mu, lam) go to both strategies, `surrogate_kw` (pool, kappa, warmup) to the surrogate one. Returns the runs and a summary per strategy: `best_median` over all seeds with a seed that found no feasible build counted as the …

pinneapple_design.bricks.statics

Static equilibrium of a brick build: contact forces, stud-joint utilisation, failure explanations.

`pinneapple_design/bricks/statics.py` · 589 linhas · testes: 24

nome	tipo	assinatura	o que faz
`Body`	classe	`(lo, hi, mass, com, label)`	
`Interface`	classe	`(lower, upper, z, rect, studs, label)`	Contact patch between `lower` (`-1` = ground) and `upper` at height `z` [mm].

nome	tipo	assinatura	o que faz
Load	classe	(body, point, force, label)	
Structure	classe	(bodies, interfaces, loads, gravity)	
StaticsResult	classe	(status, utilisation, interface_utilisation, interface_compression, interface_tension, bo…)	
FailureRound	classe	(result, broken, new)	
from_assembly	função	(asm, loads=(), broken=()) -> Structure	The :class:Structure of an :class:~pinneapple_design.bricks.assembly.Assembly (cell-bounds boxes).
analyse	função	(structure, joint=None) -> StaticsResult	Equilibrium under gravity and structure.loads with the smallest peak stud utilisation.
capacity	função	(structure, joint=None, lam_max=1000000.0) -> float	Largest factor lam on structure.loads (gravity fixed) with every stud within its clutch and an equilibrium that exists (0 when even gravity alone is not carried; lam_max when the loads are carried without limit).
convex_hull	função	(points) -> np.ndarray	Counter-clockwise convex hull of 2-D points (Andrew's monotone chain).
signed_distance_inside	função	(poly, p) -> float	Distance from p to the boundary of the convex polygon poly (ccw): > 0 inside, < 0 outside.
centre_of_pressure	função	(s, members, z0) -> np.ndarray | None	Where the resultant of the weights and loads on members crosses the plane z = z0 (None if the resultant does not point down).
tipping_report	função	(s, joint=None) -> dict | None	The worst support polygon left by a centre of pressure, treating stud joints as unbreakable.
analyse_assembly	função	(asm, loads=(), joint=None, broken=()) -> StaticsResult	:func:analyse of an assembly (gravity + loads), with the stud joints in broken released.
progressive_failure	função	(asm, loads=(), joint=None, max_rounds=50) -> list[FailureRound]	Release the most utilised stud joints one round at a time until the build stands or becomes a mechanism.
top_load	função	(asm, force, *, at=None, label='load') -> Load	A point load on the highest part covering at = (x, y) [mm] (default: the centre of the top part), applied at the top face.
single_stud_torsion	função	(asm, loads=(), joint=None, spread_mm=1.5) -> dict	Does a verdict rest on single-stud joints having no torsion capacity? A lone stud is a pivot about the vertical in this model; re-solve with each single-stud joint given three contact points spread_mm apart (a little torsion and shear moment) and report both statuses.

pinneapple_design.bricks.validation

Checks of the brick physics against closed forms, gathered into a trust report.

pinneapple_design/bricks/validation.py · 146 linhas · testes: 24

nome	tipo	assinatura	o que faz
harmonic_limit_lp	função	(n=6, L=63.8, tol=0.0001, joint=None) -> float	Largest factor on the harmonic offsets the LP still finds stable (bisection; closed form: 1 for sharp edges, joint.edge_inset_mm = 0).
validate	função	(*, newton=None, fea=True, joint=None) -> dict	Run every closed-form check. newton: include the Newton replay checks (default: when installed). The closed forms have sharp edges, so the statics checks use joint with edge_inset_mm = 0.

pinneapple_design.bricks.visuals

Pictures of the loop: the evolution as a timeline, the twin3d web viewer, GIF/MP4 frames, Blender renders and building instructions.

pinneapple_design/bricks/visuals.py · 508 linhas · testes: 24

nome	tipo	assinatura	o que faz
Item	classe	(brick, pos, quat, value)	
Marker	classe	(size, pos, quat)	
Frame	classe	(items, caption, short, kind, generation, markers, scenery)	
build_frame	função	(asm, values=None, **kw) -> Frame	A frame of asm at rest; values: stud utilisation per brick (colours the bricks by load path).
scenery	função	(asms, thickness=80.0, margin=48.0) -> list[tuple[np.ndarray, np.ndarray]]	Table blocks under the builds (their ground regions near the parts, or one block for unlimited ground), as tall as the drop to the floor in :mod:.dynamics by default.
utilisation	função	(asm, loads=(), joint=None) -> np.ndarray	Worst stud utilisation of each brick's joints (0 = no stud in tension, 1 = at the clutch limit).
replay_frames	função	(asm, *, loads=(), joint=None, frames=14, duration=0.6, caption='', short='', generation=…)	Progressive failure (statics) then a Newton replay with the failed joints released: one frame per sample, broken joints marked in red and their bricks at :data:BROKEN_VALUE.
evolution_timeline	função	(result, goal, *, generations=None, hold=3, replay=True, frames_per_replay=14, duration=0…)	Frames of a :class:~.search.SearchResult: per generation the best build (coloured by stud utilisation) and, when replay, one rejected candidate of that generation collapsing in Newton.
frame_bounds	função	(frames, margin=6.0) -> tuple[np.ndarray, np.ndarray]	

nome	tipo	assinatura	o que faz
`timeline_scene`	função	`(frames, title='self-modifying brick build', source='')`	A twin3d :class:`~pinneapple_twin3d.Scene` (millimetres, z up) with one time step per frame: brick motion, visibility, the stud utilisation field and the captions. Export it with `scene.export(folder)`.
`frame_geometry`	função	`(frame, *, colour_by='auto', ghost=None)`	The frame as a duck-typed geometry for :func:`pinneapple_design.geometry_library.render.render_geometry`.
`render_frames`	função	`(frames, *, size=(960, 540), azimuth=250.0, elevation=20.0, colour_by='auto', ssaa=1, lab…)`	Software-rendered RGB frames with one fixed camera framing every frame.
`write_gif`	função	`(path, images, fps=8.0) -> str`	
`write_mp4`	função	`(path, images, fps=8.0) -> str`	H.264 MP4 through imageio-ffmpeg when installed, else the `ffmpeg` program.
`blender_available`	função	`() -> bool`	
`render_blender`	função	`(asm, out, *, values=None, view='iso', samples=64, size=(1280, 720), title=None, tables=N…)`	Cycles render of the build in its brick colours, or coloured by `values` per brick (stud utilisation). `tables`: table boxes [mm] drawn with the build (default: :func:`scenery` of the build; `[]` for none).
`turntable_blender`	função	`(asm, folder, *, frames=24, elevation=0.45, samples=32, size=(960, 540), gif=None, mp4=No…)`	`frames` Cycles renders around the build (one per view direction); returns the PNG paths and, with `gif` / `mp4`, also writes the turntable as an animation.
`building_steps`	função	`(asm) -> list[list[int]]`	Brick indices per building step: one step per layer (plate height), bottom first.
`write_instructions`	função	`(asm, folder, *, title='Building instructions', size=(800, 520)) -> str`	Step images (earlier steps in grey, the new parts in colour), a part list, an LDraw file with one STEP per layer and `instructions.md` tying them together. Returns the Markdown path.

pinneapple_design.design_optimizer

Ponto de entrada do otimizador de projeto: reexporta o laço fechado (espaço de parâmetros, surrogate, otimizador, refinamento por PINN, Pareto) e a camada de estudos de projeto.

`pinneapple_design/design_optimizer/__init__.py` · 116 linhas · testes: 36

pinneapple_design.design_optimizer.adjoint

Otimização de forma por adjunto contínuo com autograd do PyTorch: parametrização de forma (FFD, Bézier, NACA), solver adjunto que calcula dJ/d(forma) e um objetivo de arrasto.

`pinneapple_design/design_optimizer/adjoint.py` · 549 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ShapeParametrization`	classe	`(control_points, device='cpu') -> None`	Parametrização de forma para otimização por gradiente (pontos de controle FFD, curvas de Bézier ou família NACA), com deformação de malha.
`ContinuousAdjointSolver`	classe	`(primal_model, pde_residual_fn, objective_fn) -> None`	Solver adjunto por diferenciação automática: resolve a equação adjunta, calcula a sensibilidade de forma e roda a otimização.
`DragAdjointObjective`	classe	`(surface_pts, nu, alpha=0.0) -> None`	Objetivo de força de arrasto (pressão + viscoso) aproximado como média ponderada sobre pontos de superfície.

`pinneapple_design.design_optimizer.airfoil_drag`

Estudo resolvido: reduz o arrasto de perfil de uma seção NACA 4 dígitos mantendo a sustentação, com MLP surrogate treinada na verdade de forma fechada (perfil fino + Hoerner) e o `DesignOptLoop`.

`pinneapple_design/design_optimizer/airfoil_drag.py` · 175 linhas · testes: 5

nome	tipo	assinatura	o que faz
`section_aero`	função	`(m, t_c, p=CAMBER_POSITION, alpha_deg=ANGLE_OF_ATTACK_DEG, reynolds=REYNOLDS) -> dict[str…)`	(Cl, Cd) de um design pela verdade de forma fechada.
`airfoil_outline`	função	`(m, t_c, p=CAMBER_POSITION, n_pts=160) -> dict[str, list[float]]`	Pontos de superfície da seção em escala real.
`train_surrogate`	função	`(seed, n_train=N_TRAIN_SAMPLES, n_val=N_VAL_SAMPLES, epochs=SURROGATE_EPOCHS) -> dict[str…)`	Treina a MLP (m, t/c) -> (Cl, Cd) e valida num lote com outra semente.
`run_design_optimization`	função	`(n_iterations=150, seed=None, surrogate_epochs=SURROGATE_EPOCHS) -> dict[str, Any]`	Treina o surrogate, roda o laço fechado e devolve um resultado JSON com base, final, trajetória e erros.

`pinneapple_design.design_optimizer.compare`

Compara métodos de otimização no mesmo problema, mesmo orçamento e mesmas sementes, usando hipervolume normalizado como métrica de convergência. [sem teste]

`pinneapple_design/design_optimizer/compare.py` · 158 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MethodComparison`	classe	`(problem, results, ideal, nadir, budget, rows, curves)`	Resultado de uma comparação de métodos: tabela, união das avaliações, frentes, vencedor e exportação para dicionário.
`compare_methods`	função	`(problem, methods, *, budget=300, seeds=(0,), tol=0.01, options=None, progress=None) -> M…)`	Roda cada método uma vez por semente com o mesmo orçamento e tabula o resultado.

`pinneapple_design.design_optimizer.constraints`

Restrições de projeto como penalidades diferenciáveis: caixa, conservação de massa (divergência), razão entre parâmetros, suavidade (variação total) e um conjunto que as agrega.

pinneapple_design/design_optimizer/constraints.py · 254 linhas · testes: 151

nome	tipo	assinatura	o que faz
ConstraintBase	classe	()	Base abstrata de restrição, com penalty e satisfied.
BoxConstraint	classe	(theta_min, theta_max, weight=100.0) -> None	Restrição de limites theta_min <= theta <= theta_max com barreira ReLU unilateral.
MassConservationConstraint	classe	(weight=10.0, coords=None, n_neighbors=None) -> None	Penaliza o quadrado médio da divergência estimada do campo de velocidade.
GeometricConstraint	classe	(min_val=0.1, max_val=10.0, param_pairs=None, weight=50.0) -> None	Restringe razões entre pares de parâmetros a um intervalo.
ManufacturabilityConstraint	classe	(smoothness_weight=1.0) -> None	Penaliza variação de alta frequência do vetor de projeto pela variação total.
ConstraintSet	classe	(constraints)	Agrega várias restrições: soma de penalidades e checagem de todas.

pinneapple_design.design_optimizer.evaluators

Avaliadores que ligam modelos e solvers a um DesignProblem: modelo do pp.zoo, módulo torch diferenciável e comando externo via CSV. [sem teste]

pinneapple_design/design_optimizer/evaluators.py · 168 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ZooEvaluator	classe	(name, space, fixed=None, derived=None)	Usa um modelo do pp.zoo como avaliador (fidelidade surrogate, não diferenciável por este caminho); designs fora da faixa de validade ficam inválidos.
TorchModuleEvaluator	classe	(module, outputs, *, input_transform=None, output_transform=None, name='torch-module', fi…)	Usa um módulo torch como avaliador diferenciável, com transformações de entrada e saída opcionais.
CommandEvaluator	classe	(command, space, outputs, *, timeout=600.0, name='external solver', fidelity='reference',…)	Roda um solver externo por lote: o comando lê um CSV de designs e escreve um CSV de saídas, sem shell, em diretório temporário.
from_model_ref	função	(ref, space, *, fixed=None, outputs=None) -> Evaluator	Cria o avaliador certo a partir de um ModelRef ou dicionário com uri (zoo, TorchScript, etc.).

pinneapple_design.design_optimizer.methods

Métodos de otimização sobre um DesignProblem sob o mesmo orçamento: random (hipercubo latino), NSGA-II próprio e do pymoo, SciPy DE e SLSQP, bayesiano (ParEGO) e gradiente por autograd. [sem teste]

pinneapple_design/design_optimizer/methods.py · 501 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
OptimizationResult	classe	(problem, method, seed, budget, evaluations, wall_s, status, message, notes)	Todas as avaliações de uma rodada de um método, com custo, frente de Pareto, melhor design e curva de hipervolume.
normalise_front	função	(F, ideal, nadir) -> np.ndarray	Normaliza objetivos entre os pontos ideal e nadir.

nome	tipo	assinatura	o que faz
`weight_vectors`	função	`(n_obj, n, seed=0) -> np.ndarray`	Gera `n` vetores de peso no simplex (espaçados para dois objetivos, Dirichlet com semente acima disso).
`available_methods`	função	`(problem=None) -> dict[str, dict[str, Any]]`	Lista cada método com sua família e se pode rodar aqui (pymoo instalado, avaliador diferenciável).
`run`	função	`(problem, method='nsga2', *, budget=400, seed=0, initial=None, progress=None, **options) …)`	Roda um método no problema com no máximo `budget` avaliações; métodos impossíveis voltam com status skipped.

`pinneapple_design.design_optimizer.objective`

Objetivos de projeto sobre um campo previsto: proxies de arrasto, eficiência térmica, tensão e peso, e soma ponderada.

`pinneapple_design/design_optimizer/objective.py` · 138 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ObjectiveBase`	classe	`()`	Classe base abstrata para objetivos de projeto.
`DragObjective`	classe	`(pressure_weight=1.0, viscous_weight=0.1) -> None`	Proxy de arrasto: média de |u|^2 + peso x p^2 sobre o campo (sem normais de superfície).
`ThermalEfficiencyObjective`	classe	`(heat_field_idx=0, pressure_drop_weight=0.5) -> None`	Eficiência térmica (fluxo de calor sobre queda de pressão proxy), devolvida negativa para minimizar.
`StructuralObjective`	classe	`()`	Proxy de tensão de von Mises pela média dos quadrados do campo.
`WeightMinimizationObjective`	classe	`()`	Proxy de volume pela norma L2 do vetor de parâmetros.
`CompositeObjective`	classe	`(terms=None) -> None`	Soma ponderada de vários objetivos.

`pinneapple_design.design_optimizer.optimizer`

Espaço de parâmetros em caixa e os três otimizadores do laço: gradiente (Adam/L-BFGS), bayesiano com processo gaussiano e evolutivo (CMA-ES ou AG de reserva).

`pinneapple_design/design_optimizer/optimizer.py` · 666 linhas · testes: 179

nome	tipo	assinatura	o que faz
`ParamSpace`	classe	`(bounds, x0)`	Espaço de parâmetros limitado por caixa, com recorte aos limites.
`OptState`	classe	`(step, best_x, best_y, last_x, last_y)`	Retrato do estado do otimizador em uma iteração.
`DesignOptimizerConfig`	classe	`(method, n_iters, lr, grad_optimizer, grad_clip, n_initial_random, acquisition, xi, kappa…)`	Configuração única dos back-ends (método gradient/bayesian/evolutionary, iterações, taxa de aprendizado, etc.).
`GradientDesignOptimizer`	classe	`(theta_init, cfg) -> None`	Otimizador por gradiente que aplica um passo Adam ou L-BFGS por chamada de `step`.
`BayesianDesignOptimizer`	classe	`(cfg, seed=0) -> None`	Otimizador bayesiano sequencial com processo gaussiano (scikit-learn ou GP NumPy de reserva).

nome	tipo	assinatura	o que faz
`EvolutionaryDesignOptimizer`	classe	`(cfg, seed=0, multi_objective=False) -> None`	Otimizador populacional: CMA-ES quando `cma` está instalado, senão um AG (mu+lambda) com torneio, cruzamento e mutação.

pinneapple_design.design_optimizer.pareto

Utilidades multiobjetivo: dominância, máscara de Pareto, ordenação não dominada e distância de aglomeração do NSGA-II, representantes da frente e contêiner `ParetoFront`.

`pinneapple_design/design_optimizer/pareto.py` · 341 linhas · testes: 9

nome	tipo	assinatura	o que faz
`ParetoFront`	classe	`(objectives, params, mask)`	Contêiner de uma frente de Pareto: objetivos, parâmetros e máscara, com filtro, hipervolume e gráfico.
`pareto_dominates`	função	`(a, b) -> bool`	True se a solução `a` domina `b` (minimização).
`compute_pareto_front`	função	`(objectives) -> np.ndarray`	Máscara booleana das soluções Pareto-ótimas.
`fast_non_dominated_sort`	função	`(objectives) -> list[np.ndarray]`	Divide as soluções em frentes sucessivas (NSGA-II).
`crowding_distance`	função	`(objectives) -> np.ndarray`	Distância de aglomeração do NSGA-II dentro de uma frente.
`representatives`	função	`(objectives, senses, names, ids=None, balanced='balanced', weights=None) -> dict[str, Any]`	Designs representativos de uma frente: o melhor em cada objetivo e um compromisso equilibrado.

pinneapple_design.design_optimizer.pipeline

Laço fechado de otimização guiada por física (`DesignOptLoop`): avalia o surrogate, otimiza com o back-end escolhido, refina por PINN e monta a frente de Pareto.

`pinneapple_design/design_optimizer/pipeline.py` · 502 linhas · testes: 151

nome	tipo	assinatura	o que faz
`DesignOptConfig`	classe	`(n_iterations, optimizer_cfg, refine_top_k, save_history, verbose, convergence_tol, seed,…)`	Configuração do `DesignOptLoop`: iterações, configuração do otimizador interno, refinamento top-k e outros.
`DesignOptResult`	classe	`(best_theta, best_objective, history_theta, history_objectives, history_all_objectives, p…)`	Resultado completo de um `DesignOptLoop`: melhor design, histórico, frente de Pareto, com gráfico de convergência e resumo.
`DesignOptLoop`	classe	`(param_space, surrogate, objective, *, constraints=None, refinement=None, cfg=None) -> No…)`	Orquestra o laço fechado: parametriza, avalia com o surrogate, otimiza, refina por PINN e monta a frente de Pareto.

pinneapple_design.design_optimizer.presets

Problemas prontos: dissipador de aletas planas (fidelidades surrogate, forma fechada e FVM) e perfil CST (ensemble do zoo), com bases de comparação e geração de corpos/arquivos STL e STEP.

`pinneapple_design/design_optimizer/presets.py` · 466 linhas · testes: 31 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`HeatsinkOperating`	classe	`(power_w, t_ambient_c, air_velocity_m_s, source_width_mm, source_depth_mm, tim_k_mm2_w, m…)`	Ponto de operação do dissipador: potência, temperatura ambiente, velocidade do ar, tamanho da fonte, resistência de interface.
`HeatsinkLimits`	classe	`(t_source_max_c, pressure_drop_max_pa, fin_stress_max_mpa, fin_gap_min_mm, fin_aspect_max)`	Limites do dissipador: temperatura da fonte, queda de pressão, tensão na aleta e limites de extrusão.
`CostModel`	classe	`(material_per_kg, material_yield, per_fin, fixed, currency)`	Modelo de custo unitário com valores assumidos (preço do material x massa / rendimento + custo por aleta + fixo).
`HeatsinkEvaluator`	classe	`(fidelity='surrogate', operating=None, cost=None)`	Avaliador do dissipador: geometria em forma fechada mais o modelo térmico/hidráulico da fidelidade escolhida.
`AirfoilEvaluator`	classe	`()`	Avaliador do perfil: ensemble `airfoil_cst_rans_mlp` (média e dispersão de Cl, Cd, Cm) mais espessuras em forma fechada.
`heatsink_problem`	função	`(fidelity='surrogate', operating=None, limits=None, cost=None, variables=None, objectives…)`	Monta o problema de dimensionamento do dissipador de aletas planas na fidelidade pedida.
`heatsink_baseline`	função	`() -> dict[str, float]`	Design convencional de partida do dissipador para comparação.
`airfoil_problem`	função	`(cl_min=0.6, cm_min=-0.12, aft_min=0.012, alpha_range=(0.0, 8.0)) -> DesignProblem`	Problema do perfil CST: minimizar Cd e maximizar altura da longarina com restrições de Cl, Cm e espessura traseira.
`airfoil_baseline`	função	`() -> dict[str, float]`	Design de referência do perfil: NACA 2412 (ajuste CST) a 4 graus.
`preset_info`	função	`(name) -> dict[str, Any]`	O que um formulário precisa para montar um preset: variáveis, padrões, objetivos possíveis, base e referência.
`plate_fin_bodies`	função	`(design) -> list[tuple[str, np.ndarray, np.ndarray]]`	Malhas fechadas (metros) da placa base e de cada aleta do dissipador.
`airfoil_bodies`	função	`(design, span=0.2, chord=1.0, n=81) -> list[tuple[str, np.ndarray, np.ndarray]]`	Segmento de asa extrudado e fechado da seção CST.
`preset_bodies`	função	`(preset, design) -> list[tuple[str, np.ndarray, np.ndarray]]`	Corpos sólidos de um design de preset (heatsink ou airfoil).
`design_file`	função	`(bodies, fmt, name='design') -> bytes`	Bytes de STL ou STEP do design, escritos pelos exportadores da biblioteca de geometrias.

pinneapple_design.design_optimizer.refinement

Refina previsões do surrogate continuando o treino de uma PINN com perda de dados + resíduo de EDP.

`pinneapple_design/design_optimizer/refinement.py` · 225 linhas · testes: 151

nome	tipo	assinatura	o que faz
`RefinementConfig`	classe	`(n_epochs, lr, pde_weight, bc_weight, data_weight, n_collocation, verbose)`	Configuração do refinamento por PINN: épocas, taxa de aprendizado e pesos das perdas.
`RefinementResult`	classe	`(theta, u_refined, loss_history, improvement_ratio)`	Resultado de um refinamento: parâmetros, saída refinada, histórico de perda e razão de melhora.
`PINNRefinement`	classe	`(pinn_model, pde_residual_fn, domain=None, cfg=None) -> None`	Refina a previsão do surrogate treinando uma PINN com perda de dados + resíduo de EDP, para um ou para os k melhores designs.

pinneapple_design.design_optimizer.report

Leva estudos de projeto para dentro e fora dos contratos de dados compartilhados (`ParameterSpace`, `StudyResult`, `ModelRef`) e treina surrogates GP a partir de tabelas do usuário. [sem teste]

`pinneapple_design/design_optimizer/report.py` · 344 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SurrogateFit`	classe	`(space, gp, outputs, metrics, n_train, n_test)`	Surrogate GP treinado numa tabela, com seu erro em dados separados.
`to_parameter_space`	função	`(space, name, description='', budget=None)`	Converte um espaço de projeto em `pinneapple_data.contracts.ParameterSpace`.
`designs_frame`	função	`(problem, ev, idx=None, status=None)`	Tabela pandas com um design por linha: variáveis, saídas, viabilidade, fidelidade e status de verificação.
`study_result`	função	`(problem, result, *, study_id, app, title='', candidates=(), verification=None, compariso…)`	Empacota uma otimização ou comparação de métodos como `StudyResult`, registrando só as checagens que rodaram.
`surrogate_from_table`	função	`(df, inputs, outputs, *, test_fraction=0.2, seed=0, integer=(), bounds=None) -> Surrogate…)`	Ajusta um `GPSurrogate` por saída a partir das linhas de uma tabela e mede o erro em linhas separadas.
`model_ref_for_surrogate`	função	`(fit, name, uri, dataset_name='')`	`ModelRef` de um surrogate treinado em tabela, com faixa de validade e métricas.
`table_evaluator`	função	`(space, df, outputs, *, tol=1e-06, name='your solver results') -> FunctionEvaluator`	Avaliador de referência que responde a partir de uma tabela de resultados de solver, casando linhas pelos valores das variáveis.
`verify_front`	função	`(problem, ev, front, n, reference=None)`	Escolhe candidatos ao longo da frente e os reavalia com a referência, quando existe.
`write_design_files`	função	`(folder, problem, ev, candidates, status=None, *, labels=None) -> list[str]`	Grava os arquivos do estudo numa pasta: avaliações, candidatos com status, variáveis para rodar no solver e o `ParameterSpace`.

nome	tipo	assinatura	o que faz
`chart_points`	função	`(problem, ev, front=(), *, limit=2000, labels=None) -> list[dict[str, Any]]`	Seleciona e afina pontos para um gráfico de trade-off, mantendo sempre a frente.
`verify_uploaded_results`	função	`(space, candidates, results, objectives=(), constraints=(), *, tol=1e-06)`	Verifica candidatos exportados com resultados de solver rodados fora, casando linhas por valores das variáveis.

`pinneapple_design.design_optimizer.spec`

Monta problemas de projeto a partir de specs JSON simples (fonte preset, zoo ou tabela), como os enviados por um app ou API. [sem teste]

`pinneapple_design/design_optimizer/spec.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpecError`	classe	`()`	Erro de spec que não vira problema, com mensagem legível.
`objectives_from`	função	`(items) -> list[Objective]`	Converte a lista de objetivos da spec em `Objective`; exige ao menos um.
`constraints_from`	função	`(items) -> list[Constraint]`	Converte a lista de restrições da spec em `Constraint` com limites inferior/superior.
`zoo_problem`	função	`(model, objectives, constraints=(), *, variables=None, fixed=None) -> DesignProblem`	Problema sobre um modelo do `pp.zoo`: cada entrada não fixa vira variável limitada pela faixa de validade.
`problem_from_spec`	função	`(spec, table=None) -> tuple[DesignProblem, Any]`	Monta `(problem, fit)` a partir de uma spec; `fit` só existe para fonte tabela.
`reference_for_spec`	função	`(spec) -> Evaluator | None`	Referência de alta fidelidade da spec: o motor FVM para o dissipador, None nos outros casos.

`pinneapple_design.design_optimizer.study`

Descrição do problema de projeto usada por todos os métodos: variáveis, espaço, objetivos, restrições, avaliador, registro das avaliações, relatório de viabilidade e hipervolume.

`pinneapple_design/design_optimizer/study.py` · 494 linhas · testes: 49 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Variable`	classe	`(name, lower, upper, kind, unit, description)`	Uma variável de projeto em `[lower, upper]`; `kind="integer"` arredonda.
`DesignSpace`	classe	`(variables)`	Lista ordenada de variáveis: limites, reparo, mapeamento ao cubo unitário, amostragem e conversão de/para `ParameterSpace`.
`Objective`	classe	`(name, sense, unit, label)`	Minimizar ou maximizar uma saída nomeada do avaliador.
`Constraint`	classe	`(name, upper, lower, unit, kind, label)`	Restrição `lower <= saída <= upper` com violação normalizada, margem e tipo (física, fabricação, validade).

nome	tipo	assinatura	o que faz
`Evaluator`	classe	`(name, fidelity, outputs, differentiable, relaxable, description)`	Base de avaliador: mapeia designs X (n, d) em saídas nomeadas, com fidelidade e flag de diferenciável; `evaluate` é abstrato.
`FunctionEvaluator`	classe	`(fn, outputs, *, name='function', fidelity='model', torch_fn=None, description='', relaxa…)`	Avaliador a partir de uma função vetorizada (e, opcionalmente, de sua versão torch).
`Evaluations`	classe	`(X, Y, F, G, feasible, relaxed, fidelity, wall_s, tag)`	Todos os designs avaliados num estudo, em ordem: matriz de objetivos, violações, viabilidade e frente de Pareto.
`DesignProblem`	classe	`(name, space, objectives, constraints, evaluator, description, notes)`	Problema de otimização: variáveis, objetivos, restrições e avaliador, com registro das avaliações.
`constants_of`	função	`(ps) -> dict[str, Any]`	Parâmetros constantes de um `ParameterSpace` ou dicionário.
`feasibility_report`	função	`(problem, ev, idx, active_tol=0.05) -> list[dict[str, Any]]`	Por design: valor, margem e status (violada, ativa, inativa) de cada restrição.
`hypervolume`	função	`(F, ref, n_mc=20000, seed=0) -> float`	Hipervolume dominado pelas linhas de F até `ref`: exato para 1 e 2 objetivos, Monte Carlo acima.

`pinneapple_design.design_optimizer.surrogate`

Envoltório `PhysicsSurrogate` que dá uma interface uniforme (previsão, jacobiano, treino, salvar/carregar) a qualquer `nn.Module` que mapeia parâmetros em campo.

`pinneapple_design/design_optimizer/surrogate.py` · 268 linhas · testes: 151

nome	tipo	assinatura	o que faz
`SurrogateConfig`	classe	`(model_name, in_dim, out_channels, hidden_dim, n_layers, device, checkpoint_path)`	Configuração do `PhysicsSurrogate`: modelo (nome registrado ou MLP), dimensões e camadas.
`PhysicsSurrogate`	classe	`(model, cfg=None) -> None`	Envoltório fino de um `nn.Module` theta -> u com previsão, jacobiano, treino em dataset e checkpoint.

`pinneapple_design.design_optimizer.verify`

Confirmação em alta fidelidade dos designs otimizados e otimização assistida por surrogate GP com aprendizado ativo. [sem teste]

`pinneapple_design/design_optimizer/verify.py` · 254 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VerificationReport`	classe	`(rows, errors, reference, wall_s, evaluations)`	Relatório de verificação: linhas por design, erros por saída, dados da referência e tempo gasto.
`GPSurrogate`	classe	`(problem, outputs, validity=None, name='gp-surrogate', safe=None, kappa=2.0)`	Processo gaussiano por saída (Matern 5/2, ARD, scikit-learn) em entradas no cubo unitário; saídas positivas em escala log.
`ActiveLearningResult`	classe	`(problem, reference_evals, rounds, final, verification, wall_s, notes)`	Resultado da otimização assistida: avaliações de referência, rodadas, resultado final e verificação.

nome	tipo	assinatura	o que faz
`select_candidates`	função	`(ev, front, n) -> list[int]`	Escolhe `n` designs espaçados ao longo de uma frente, com as duas pontas.
`verify_candidates`	função	`(problem, ev, idx, reference, reference_problem=None) -> VerificationReport`	Reavalia designs escolhidos com a referência e compara cada saída (erro relativo, viabilidade, status).
`surrogate_assisted`	função	`(problem, reference, *, outputs=None, method='nsga2', n_initial=24, rounds=3, n_verify=6,…)`	Otimiza num GP da referência e o melhora onde o otimizador procura (aprendizado ativo).

`pinneapple_design.fitmesh`

Fit-for-purpose geometry and mesh: from a problem description to checked CAD, planned meshes and one report.

`pinneapple_design/fitmesh/__init__.py` · 26 linhas · testes: 21

`pinneapple_design.fitmesh.api`

HTTP API over :class:`FitMeshSession`: one session per study, one call per step, the files and the report.

`pinneapple_design/fitmesh/api.py` · 146 linhas · testes: 12

nome	tipo	assinatura	o que faz
`create_app`	função	`(root='fitmesh_runs')`	

`pinneapple_design.fitmesh.cad`

Build the CAD variants, export STEP and STL, and check each one before it is meshed.

`pinneapple_design/fitmesh/cad.py` · 115 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CadVariant`	classe	`(index, params, step, stl, checks, comparison, volume, area, bbox)`	
`build_variant`	função	`(tpl, params, out_dir, index, accuracy='design') -> CadVariant`	
`compare_with_references`	função	`(tpl, params) -> list[dict[str, Any]]`	Relative difference of each shared dimension with each published reference object.

`pinneapple_design.fitmesh.catalog`

Parametric geometry templates with the references that make them checkable.

`pinneapple_design/fitmesh/catalog.py` · 460 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Param`	classe	`(name, value, unit, typical, limits, role, sensitivity)`	
`Reference`	classe	`(citation, what, params)`	

nome	tipo	assinatura	o que faz
`Template`	classe	`(key, title, physics, keywords, description, params, references, quantities, build, volum…)`	
`naca4_points`	função	`(m, p, t, chord, n=80) -> np.ndarray`	Closed NACA 4-digit section (x, z), trailing edge closed (a4 = -0.1036), cosine spacing.
`polygon_area`	função	`(pts) -> float`	

`pinneapple_design.fitmesh.context`

Read a problem description into the facts the geometry and mesh choices depend on, and ask for what is missing.

`pinneapple_design/fitmesh/context.py` · 208 linhas · testes: 12

nome	tipo	assinatura	o que faz
`Fact`	classe	`(value, source)`	
`ProblemContext`	classe	`(text, facts)`	
`parse`	função	`(text, llm=None) -> ProblemContext`	Facts from a free-text problem description (English or Portuguese).
`wall_unit_height`	função	`(y_plus, speed, length, nu, re, internal=False) -> dict[str, float]`	First-cell height for a target y+ from flat-plate skin friction at the reference length.

`pinneapple_design.fitmesh.layers`

Prism (inflation) layer sizing: a geometric stack of wall layers.

`pinneapple_design/fitmesh/layers.py` · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`layer_total`	função	`(first_height, expansion, n_layers) -> float`	Total thickness `h1 (r^n - 1) / (r - 1)` of `n` layers (`n h1` when `r = 1`).
`size_layers`	função	`(first_height, expansion=None, n_layers=None, total_thickness=None) -> dict[str, float]`	Solve the geometric layer stack for the one missing quantity among `expansion`, `n_layers` and `total_thickness` (exactly one must be `None`).

`pinneapple_design.fitmesh.llm`

Optional language-model help for every step, with a local model if you like (Ollama via `pinneapple_llm`).

`pinneapple_design/fitmesh/llm.py` · 251 linhas · testes: 12

nome	tipo	assinatura	o que faz
`LLMAssist`	classe	`(provider, model, api_key, conversation_store, call, provider_kwargs)`	Which model to ask. `provider`: "ollama" (local), "anthropic" or "openai"; `call` overrides everything (a function `(prompt, system) -> str` returning JSON), which is how tests and custom backends plug in.

nome	tipo	assinatura	o que faz
`read_problem`	função	`(llm, text) -> dict[str, Any]`	
`rank_geometries`	função	`(llm, ctx) -> dict[str, Any]`	
`propose_parameters`	função	`(llm, tpl, ctx, base) -> list[dict[str, Any]]`	
`free_geometry`	função	`(llm, description) -> dict[str, Any]`	A CadQuery script from the model, checked statically; run it with :func:`run_free_geometry`.
`check_code`	função	`(code) -> list[str]`	Static checks before anything runs: syntax, imports on the allow-list, no dunder or I/O names, build().
`run_free_geometry`	função	`(code, params, out_dir, tol, timeout=120) -> dict[str, Any]`	Run model-written CadQuery in a separate isolated interpreter (`python -I`), with a timeout.
`review_mesh_plan`	função	`(llm, plan, ctx) -> MeshPlan`	

pinneapple_design.fitmesh.mesh_gen

Generate the planned meshes and measure their quality.

`pinneapple_design/fitmesh/mesh_gen.py` · 407 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MeshResult`	classe	`(variant, level, tool, path, files, cells, seconds, quality, layers, error)`	
`generate`	função	`(tpl, var, plan, out_dir, level=1.0, procs=1, max_cells=3000000.0, log=print) -> MeshResu…)`	Mesh one CAD variant at `level` x the planned sizes (1.0 = the plan; 1.5, 2.25 = the coarser family).
`quality`	função	`(path, target) -> dict[str, Any]`	

pinneapple_design.fitmesh.mesh_plan

Recommend a mesh that is fit for the purpose: fine enough for the quantities that matter, no finer.

`pinneapple_design/fitmesh/mesh_plan.py` · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MeshPlan`	classe	`(kind, tool, elements, purpose, sizes, layers, domain, estimated_cells, estimated_cost, f…)`	
`mesh_kind`	função	`(tpl, ctx) -> str`	
`plan_mesh`	função	`(tpl, params, ctx, budget_cells=5000000.0) -> MeshPlan`	

pinneapple_design.fitmesh.report

One final trust report for the whole study, plus the grid-convergence calculation it relies on.

`pinneapple_design/fitmesh/report.py` · 226 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`gci`	função	`(values, sizes, safety=None) -> dict[str, Any]`	Grid-convergence index from three meshes (Roache 1994; procedure of Celik et al. 2008, J. Fluids Eng. 130).
`build_report`	função	`(session) -> dict[str, Any]`	Collect every stage of a :class:`FitMeshSession` into one report with a verdict per stage.
`render_html`	função	`(rep) -> str`	
`write_report`	função	`(rep, stem) -> dict[str, str]`	

pinneapple_design.fitmesh.session

The interactive study: describe the problem, answer the questions, choose, and get one final report.

`pinneapple_design/fitmesh/session.py` · 295 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FitMeshSession`	classe	`(text, out_dir, llm=None, log=print)`	

pinneapple_design.fitmesh.suggest

Rank geometry templates for a problem and propose the parameters to vary, their ranges and the samples.

`pinneapple_design/fitmesh/suggest.py` · 94 linhas · testes: 12

nome	tipo	assinatura	o que faz
`suggest_geometries`	função	`(ctx, top=3) -> list[dict[str, Any]]`	Templates ranked by physics match and keyword hits, each with why it was picked and its references.
`scale_to_context`	função	`(tpl, ctx) -> dict[str, float]`	Reference values, scaled so the reference length matches the size given in the problem (if any).
`suggest_parameters`	função	`(tpl, ctx, base=None) -> list[dict[str, Any]]`	Every parameter with a vary / keep recommendation, a proposed range and the reason.
`sample`	função	`(tpl, base, ranges, n, seed=0, include_base=True) -> list[dict[str, float]]`	Latin hypercube over the selected ranges (scipy.stats.qmc); integer parameters are rounded.

pinneapple_design.geometry

pinneapple_geom

`pinneapple_design/geometry/__init__.py` · 228 linhas · testes: 1

pinneapple_design.geometry.bodies

Closed triangulated bodies for external-flow studies: the Ahmed body, a sphere, a cylinder, a box, an annulus.

`pinneapple_design/geometry/bodies.py` · 141 linhas · testes: 43

nome	tipo	assinatura	o que faz
ahmed_body	função	(slant_deg=25.0, scale=1.0, clearance=0.05, n_nose=10) -> Mesh	The Ahmed body (Ahmed, Ramm & Faltin 1984): 1.044 x 0.389 x 0.288 m, front corners rounded with 0.1 m radius, a 0.222 m long rear slant at slant_deg. Its lower face sits clearance above z = 0 (the road; the stilts are left out). Nose at x = 0. The classic drag coefficient at 25 degrees is …
sphere	função	(radius=0.5, centre=(0.0, 0.0, 0.0), n=48) -> Mesh	UV sphere.
cylinder	função	(radius=0.5, length=2.0, axis='y', n=48) -> Mesh	Closed cylinder centred at the origin; axis "y" is the classic cross-flow cylinder, "x" an axial one.
box	função	(size=(1.0, 1.0, 1.0), centre=(0.0, 0.0, 0.0)) -> Mesh	Axis-aligned box (e.g. a building block).
annulus	função	(r_inner, r_outer, height, n=72, z0=0.0) -> Mesh	Closed annular ring (a tube with flat ends) around the z axis, from z0 to z0 + height: outer and inner walls plus the two ring faces, outward-facing triangles. For r_inner = 0 use :func:cylinder.

pinneapple_design.geometry.builders

Módulo sem docstring e sem texto.

pinneapple_design/geometry/builders/__init__.py · 15 linhas · **sem teste direto**

pinneapple_design.geometry.builders.analytic_domain_batch_builder

Analytic (mesh-free) domain-batch builders for axis-aligned canonical shapes: boxes/rectangles and cylinders.

pinneapple_design/geometry/builders/analytic_domain_batch_builder.py · 709 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
sample_box_tag_batch	função	(spec, tag_faces, *, n_col=20000, n_bc_per_face=3000, seed=7, user_ctx=None) -> Dict[str,…)	Sample a real box-domain batch for a ProblemSpec whose domain_bounds literally describe an axis-aligned box (or rectangle, or hyper-rectangle -- works for any number of coordinates).
sample_cylinder_tag_batch	função	(spec, *, axis_coord, cross_coords, radius, tag_faces, n_col=20000, n_bc_per_face=3000, s…)	Sample a real cylinder-domain batch (circular cross-section, extruded along axis_coord) -- e.g. pipe_flow_3d.
sample_axisymmetric_wall_tag_batch	função	(spec, *, axis_coord, radial_coord, r_wall_fn, dr_wall_fn, tag_faces, n_col=20000, n_bc_p…)	Sample a real axisymmetric (r, z) domain whose wall radius varies along the axis as r_wall_fn(z) (e.g. a conical nozzle) -- a genuine, non-cylindrical generalization of :func:sample_cylinder_tag_batch needed because the wall is not at a constant radius.

nome	tipo	assinatura	o que faz
`sample_annulus_tag_batch`	função	`(spec, tag_faces, *, cross_coords, inner_radius, outer_radius, n_col=20000, n_bc_per_face…)`	Sample a real annular (ring) cross-section domain: `inner_radius <= sqrt(x^2+y^2) <= outer_radius`. `tag_faces`: tag_name -> "inner" (the r=inner_radius circle) or "outer" (the r=outer_radius circle).
`sample_box_with_curve_tag_batch`	função	`(spec, *, shape_coords, box_tag_faces=None, curve_tags=None, inside_body_fn=None, n_col=2…)`	Sample a 2D box domain (`shape_coords`, e.g. ("x","y")) with zero or more solid bodies/curves embedded in it -- e.g. an airfoil/blade/car-body silhouette that is NOT a face of the bounding box.

pinneapple_design.geometry.builders.stl_domain_batch_builder

Módulo sem docstring e sem texto.

`pinneapple_design/geometry/builders/stl_domain_batch_builder.py` · 463 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VoxelInsideConfig`	classe	`(pitch, target_resolution, fill)`	
`SDFConfig`	classe	`(mode, sdf_fn, inside_if_leq)`	
`TagHeuristics`	classe	`(enabled, inlet_outlet_axis, inlet_is_min, outlet_is_min, plane_eps_frac, require_normal_…)`	
`STLDomainBatchConfig`	classe	`(n_col, n_bc, n_ic, n_data, normalize_unit_box, inside_mode, max_rejection_rounds, oversa…)`	
`STLDomainBatchBuilder`	classe	`(cfg)`	

pinneapple_design.geometry.core

Módulo sem docstring e sem texto.

`pinneapple_design/geometry/core/__init__.py` · 12 linhas · **sem teste direto**

pinneapple_design.geometry.core.geometry

GeometrySpec, GeometryAsset, and mesh transform utilities for Pinneapple geometry.

`pinneapple_design/geometry/core/geometry.py` · 422 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometrySpec`	classe	`(kind, name, path, params)`	Declarative geometry specification.
`GeometryAsset`	classe	`(mesh, bounds, units, boundary_groups, meta)`	Unified geometry container used across Pinneapple.

pinneapple_design.geometry.core.mesh

MeshData container for triangle meshes with PINN sampling and collocation support.

`pinneapple_design/geometry/core/mesh.py` · 566 linhas · testes: 174

nome	tipo	assinatura	o que faz
`MeshData`	classe	`(vertices, faces, normals)`	Lightweight mesh container (numpy-only).

`pinneapple_design.geometry.core.registry`

Geometry asset builder registry and build_geometry_asset factory.

`pinneapple_design/geometry/core/registry.py` · 288 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_geometry_asset`	função	`(spec, *, options=None) -> GeometryAsset`	Build a GeometryAsset from a GeometrySpec or spec dict.
`load_geometry_asset`	função	`(geom, *, options=None) -> GeometryAsset`	Convenience wrapper: - Path / str -> infer spec - GeometryAsset -> returned as-is - dict -> GeometrySpec

`pinneapple_design.geometry.csg`

Constructive Solid Geometry (CSG) for 2D/3D PINN collocation domains.

`pinneapple_design/geometry/csg.py` · 570 linhas · testes: 195 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`SDFShape`	classe	`(bounds_min, bounds_max)`	Base class for SDF-based 2D shapes.
`CSGRectangle`	classe	`(x_min, y_min, x_max, y_max) -> None`	Axis-aligned rectangle.
`CSGCircle`	classe	`(center_x, center_y, radius) -> None`	2D circle.
`CSGEllipse`	classe	`(center_x, center_y, a, b) -> None`	2D axis-aligned ellipse.
`CSGPolygon`	classe	`(vertices) -> None`	Arbitrary polygon defined by an ordered list of 2D vertices.
`CSGUnion`	classe	`()`	Boolean union of two SDFShapes: min(sdf_a, sdf_b).
`CSGIntersection`	classe	`()`	Boolean intersection of two SDFShapes: max(sdf_a, sdf_b).
`CSGDifference`	classe	`()`	Boolean difference A minus B: max(sdf_a, -sdf_b).
`lshape`	função	`(width1, height1, width2, height2) -> CSGDifference`	L-shaped domain.
`annulus`	função	`(cx, cy, r_inner, r_outer) -> CSGDifference`	Annular (ring) domain: outer circle minus inner circle.
`channel_with_hole`	função	`(length, height, hole_cx, hole_cy, hole_r) -> CSGDifference`	Rectangular channel with a circular obstacle (hole).
`t_junction`	função	`(h_length, h_height, v_x_start, v_width, v_height) -> CSGUnion`	T-junction domain: horizontal channel plus a vertical branch.

`pinneapple_design.geometry.gen`

Módulo sem docstring e sem texto.

`pinneapple_design/geometry/gen/__init__.py` · 17 linhas · testes: 9

pinneapple_design.geometry.gen.airfoil

Parametric airfoil shape generators.

`pinneapple_design/geometry/gen/airfoil.py` · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`naca_parametric`	função	`(m=0.0, p=0.0, t_c=0.12, n_pts=100) -> torch.Tensor`	Generate a NACA 4-digit airfoil surface as control-point coordinates.

pinneapple_design.geometry.gen.cadquery_advanced

Advanced parametric CadQuery geometry for physics-AI design spaces, plus reproductions of the classic CadQuery documentation examples.

`pinneapple_design/geometry/gen/cadquery_advanced.py` · 292 linhas · testes: 9

nome	tipo	assinatura	o que faz
`pipe_bend_flanged`	função	`(p)`	Bend of mean radius R and angle theta, tube OD/wall t, bolted flange at each end.
`pipe_tee`	função	`(p)`	Equal tee: run along X (length L), branch along +Z (height H), same OD/wall.
`concentric_reducer`	função	`(p)`	Truncated cone from D1 to D2 over length L, wall t (loft of circles, hollowed by a loft).
`helical_coil`	função	`(p)`	Solid tube of radius r swept along a helix (coil radius Rc, pitch, turns).
`axial_fan`	função	`(p)`	Hub cylinder + n blades lofted from a root to a tip NACA section with twist, polar pattern.
`involute_tooth_profile`	função	`(module, teeth, pressure_deg=20.0, n=12) -> List[Tuple[float, float]]`	Closed outline of a spur gear (involute flanks from base to tip circle, root arcs).
`spur_gear`	função	`(p)`	
`pin_fin_heatsink`	função	`(p)`	
`occ_bottle`	função	`(p)`	CadQuery docs, "The Classic OCC Bottle".
`pillow_block`	função	`(p)`	CadQuery docs, "Parametric Bearing Pillow Block".
`lego_brick`	função	`(p)`	CadQuery docs example "Lego Brick" (studs on top, tubes underneath).
`register_advanced_templates`	função	`(reg)`	Add every builder above to a :class:`CadQueryRegistry` (with its parameter schema).

pinneapple_design.geometry.gen.cadquery_gen

CadQuery mesh import, parametric CAD spec, and STEP/STL conversion.

`pinneapple_design/geometry/gen/cadquery_gen.py` · 404 linhas · testes: 18

nome	tipo	assinatura	o que faz
`CadQueryTessellationOptions`	classe	`(linear_deflection, angular_deflection)`	CadQuery tessellation controls (affect mesh fidelity and speed).
`ParametricCadSpec`	classe	`(builder_id, params, schema, meta)`	Future-ready declarative CAD spec.
`CadQueryBuildOptions`	classe	`(tessellation, heal, center_to_origin, normalize_to_unit_box)`	Build controls beyond tessellation.
`CadQueryRegistry`	classe	`(builders, schemas)`	Registry of parametric CAD builders.
`cadquery_available`	função	`() -> bool`	
`cadquery_to_trimesh`	função	`(cq_obj, *, options=None)`	Convert an in-memory CadQuery object (Workplane/Solid/Compound) to trimesh.Trimesh.
`build_mesh_from_step`	função	`(step_path, *, options=None) -> MeshData`	Load a STEP file with CadQuery and return a triangle mesh as MeshData.
`build_mesh_from_cadquery_object`	função	`(cq_obj, *, options=None) -> MeshData`	Convert an in-memory CadQuery object (Workplane/Solid/Compound) to MeshData.
`build_mesh_from_parametric_spec`	função	`(spec, registry, *, options=None) -> MeshData`	Build a MeshData from a ParametricCadSpec using a registry builder.
`register_default_templates`	função	`(reg) -> CadQueryRegistry`	Register a couple of templates to prove the concept. (You can add your cold-plate / channel / fin arrays here.)
`export_cadquery`	função	`(cq_obj, out_path, *, fmt=None) -> Path`	Export a CadQuery object to STEP/STL (or other supported formats).

`pinneapple_design.geometry.gen.car2d`

Parametric 2-D car side profile (an Ahmed-type road-vehicle body, no wheels) and its rasterisation.

`pinneapple_design/geometry/gen/car2d.py` · 92 linhas · testes: 20

nome	tipo	assinatura	o que faz
`CarProfile`	classe	`(slant_deg, windshield_deg, hood, clearance, diffuser_deg, nose, height, slant_length)`	
`sample_designs`	função	`(n, seed=0) -> list[dict[str, float]]`	Latin-hypercube designs over `DESIGN_SPACE`.

`pinneapple_design.geometry.gen.domains`

Pre-built physics domains for PINN training.

`pinneapple_design/geometry/gen/domains.py` · 754 linhas · testes: 151 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`BoundaryRegion`	classe	`(name, kind, conditions)`	A named boundary region with an associated SDF-like sampler.
`PhysicsDomain2D`	classe	`(bounds_min, bounds_max, boundary_regions)`	Base class for 2D physics domains.

nome	tipo	assinatura	o que faz
`ChannelDomain2D`	classe	`(length=2.0, height=1.0, origin=(0.0, 0.0), inlet_velocity=1.0)`	Rectangular channel: flow from left (inlet) to right (outlet).
`ChannelWithObstacleDomain2D`	classe	`(length=2.0, height=1.0, origin=(0.0, 0.0), obstacle_center=(0.5, 0.5), obstacle_radius=0…)`	Rectangular channel with a circular obstacle.
`LidDrivenCavityDomain2D`	classe	`(size=1.0, origin=(0.0, 0.0), lid_velocity=1.0)`	Unit square cavity with moving lid.
`LShapeDomain2D`	classe	`(size=2.0, origin=(-1.0, -1.0))`	L-shaped 2D domain for heat/Laplace problems.
`AnnularDomain2D`	classe	`(center=(0.0, 0.0), r_inner=0.2, r_outer=1.0, inner_condition=None, outer_condition=None)`	Annular (ring) domain between inner and outer circles.
`MultiObstacleDomain2D`	classe	`(length=3.0, height=1.0, origin=(0.0, 0.0), obstacles=None, inlet_velocity=1.0)`	Rectangular channel with multiple arbitrary circular obstacles.
`TJunctionDomain2D`	classe	`(main_length=3.0, main_height=0.5, branch_width=0.5, branch_length=1.0, origin=(0.0, 0.0)…)`	T-junction: horizontal main channel + vertical branch from the bottom.
`SDFDomain2D`	classe	`(sdf_fn, *, bounds_min, bounds_max, boundary_samplers=None, boundary_conditions=None)`	Flexible domain from any SDF callable and named sampler dict.
`get_domain`	função	`(name, **kwargs) -> PhysicsDomain2D`	Instantiate a built-in domain by name.
`list_domains`	função	`() -> List[str]`	List all registered built-in domain names.

`pinneapple_design.geometry.gen.domains3d`

Pre-built 3D physics domains for PINN training.

`pinneapple_design/geometry/gen/domains3d.py` · 597 linhas · testes: 151 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`PhysicsDomain3D`	classe	`(bounds_min, bounds_max, boundary_regions)`	Base class for 3D physics domains.
`LidDrivenCavityDomain3D`	classe	`(size=1.0, origin=(0.0, 0.0, 0.0), lid_velocity=1.0, lid_axis='x')`	Unit cube [ox, ox+L]^3 with a moving lid on the top face (z = oz+Lz).
`ChannelDomain3D`	classe	`(length=2.0, height=1.0, width=1.0, origin=(0.0, 0.0, 0.0), inlet_velocity=1.0, flow_prof…)`	Rectangular duct: x in [ox, ox+Lx], y in [oy, oy+Ly], z in [oz, oz+Lz].
`PipeFlowDomain3D`	classe	`(radius=0.5, length=2.0, origin=(0.0, 0.0, 0.0), inlet_velocity=1.0)`	Circular cross-section pipe, flow in the x-direction.
`get_domain_3d`	função	`(name, **kwargs) -> PhysicsDomain3D`	Instantiate a built-in 3D domain by name.
`list_domains_3d`	função	`() -> List[str]`	List all registered built-in 3D domain names.

`pinneapple_design.geometry.gen.genetic_sdf`

Módulo sem docstring e sem texto.

`pinneapple_design/geometry/gen/genetic_sdf.py` · 110 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeneticSDF`	classe	`(sdf, genome, bounds_min, bounds_max, dim)`	A callable SDF domain with an explicit genome.
`sdf_union`	função	`(a, b) -> torch.Tensor`	
`sdf_intersection`	função	`(a, b) -> torch.Tensor`	
`make_union_circles_sdf`	função	`(*, n_circles=3, seed=0, bounds_min=(0.0, 0.0), bounds_max=(1.0, 1.0), r_min=0.12, r_max=…)`	Create a random "genetic" domain as union of circles.

`pinneapple_design.geometry.gen.primitives`

Trimesh-based primitive mesh builders and architected geometries (woven, braid).

`pinneapple_design/geometry/gen/primitives.py` · 753 linhas · testes: 66 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`MeshBuilder`	classe	`()`	
`register_builder`	função	`(*names)`	
`list_builders`	função	`() -> Tuple[str, ...]`	Every registered builder name (all aliases included, e.g. both `"box"` and `"cube"`) -- the real, checkable catalog for anything that needs to know what :func:`build_mesh` can build without hardcoding a guessed list (e.g. an LLM-drafting layer presenting a catalog of real options, mirroring …
`build_mesh`	função	`(name, **params) -> MeshData`	General factory: - calls a registered builder -> trimesh.Trimesh - optional transforms: scale / rotate / translate - optional boolean: boolean= {op:'cut'|'union'|'intersect', other:{name:'sphere', ...}} -> If no boolean engine exists, it falls back to concatenate (non-watertight).
`build_primitive`	função	`(name, **params) -> MeshData`	
`project_points_to_surface`	função	`(points, surface, *, method='nearest') -> np.ndarray`	Placeholder / hook for future "fidelity maximum" mapping: - Given points (N,3), project them onto a patient-specific surface mesh.

`pinneapple_design.geometry.gen.sdf_shapes`

Rich SDF (Signed Distance Function) library for 2D and 3D shapes.

`pinneapple_design/geometry/gen/sdf_shapes.py` · 629 linhas · testes: 151

nome	tipo	assinatura	o que faz
`SDF`	classe	`(fn, *args, **kwargs)`	Object-oriented wrapper around an SDF callable.
`sdf2d_circle`	função	`(p, center, radius) -> np.ndarray`	SDF for a 2D circle. Negative inside.

nome	tipo	assinatura	o que faz
sdf2d_rectangle	função	(p, center, half_extents) -> np.ndarray	SDF for an axis-aligned rectangle. Negative inside.
sdf2d_rounded_rectangle	função	(p, center, half_extents, radius) -> np.ndarray	SDF for a rectangle with rounded corners.
sdf2d_ellipse	função	(p, center, semi_axes, iterations=4) -> np.ndarray	SDF for a 2D ellipse via iterative Newton refinement (Inigo Quilez).
sdf2d_capsule	função	(p, a, b, radius) -> np.ndarray	SDF for a 2D capsule (stadium) between points a and b.
sdf2d_line_segment	função	(p, a, b, thickness=0.0) -> np.ndarray	SDF to a line segment (with optional thickness).
sdf2d_triangle	função	(p, v0, v1, v2) -> np.ndarray	SDF for a 2D triangle (signed: negative inside).
sdf2d_annulus	função	(p, center, r_inner, r_outer) -> np.ndarray	SDF for a 2D annular region (ring). Negative between the two radii.
sdf2d_sector	função	(p, center, radius, angle_start, angle_end) -> np.ndarray	SDF for a circular sector (pie slice).
sdf2d_convex_polygon	função	(p, vertices) -> np.ndarray	SDF for a convex polygon given as ordered vertices (Nx2 array).
sdf3d_sphere	função	(p, center, radius) -> np.ndarray	SDF for a 3D sphere.
sdf3d_box	função	(p, center, half_extents) -> np.ndarray	SDF for an axis-aligned 3D box.
sdf3d_cylinder	função	(p, center, radius, height, axis=2) -> np.ndarray	SDF for a finite cylinder aligned with axis (0=x, 1=y, 2=z).
sdf3d_torus	função	(p, center, R, r, axis=2) -> np.ndarray	SDF for a torus with major radius R and tube radius r.
sdf3d_capsule	função	(p, a, b, radius) -> np.ndarray	SDF for a 3D capsule between points a and b.
sdf3d_rounded_box	função	(p, center, half_extents, radius) -> np.ndarray	SDF for a 3D box with rounded edges.
sdf3d_cone	função	(p, apex, height, radius) -> np.ndarray	SDF for an upward-pointing, base-capped cone with apex at apex.
sdf_union	função	(a, b) -> np.ndarray	Boolean union: min(a, b).
sdf_intersection	função	(a, b) -> np.ndarray	Boolean intersection: max(a, b).
sdf_difference	função	(a, b) -> np.ndarray	Boolean difference: a minus b = max(a, -b).
sdf_smooth_union	função	(a, b, k=0.1) -> np.ndarray	Smooth minimum (R-function). k controls blend radius.
sdf_smooth_intersection	função	(a, b, k=0.1) -> np.ndarray	Smooth maximum.
sdf_smooth_difference	função	(a, b, k=0.1) -> np.ndarray	Smooth difference.
sdf_translate	função	(sdf_fn, offset) -> Callable[[np.ndarray], np.ndarray]	Translate an SDF by applying an inverse offset to the query point.
sdf_scale	função	(sdf_fn, factor) -> Callable[[np.ndarray], np.ndarray]	Uniformly scale an SDF.
sdf_rotate_2d	função	(sdf_fn, angle_rad) -> Callable[[np.ndarray], np.ndarray]	Rotate a 2D SDF by angle_rad (counter-clockwise).

nome	tipo	assinatura	o que faz
sdf_mirror_x	função	(sdf_fn) -> Callable[[np.ndarray], np.ndarray]	Mirror an SDF across the y-axis (flip x).
sdf_mirror_y	função	(sdf_fn) -> Callable[[np.ndarray], np.ndarray]	Mirror an SDF across the x-axis (flip y).
sdf_onion	função	(sdf_fn, thickness) -> Callable[[np.ndarray], np.ndarray]	Create a shell (onion skin) of given thickness.
sdf_elongate_2d	função	(sdf_fn, dx=0.0, dy=0.0) -> Callable[[np.ndarray], np.ndarray]	Elongate a 2D SDF shape along x and/or y axes.
sdf_repeat_2d	função	(sdf_fn, period, repetitions=None) -> Callable[[np.ndarray], np.ndarray]	Tile a 2D SDF with given (px, py) period.
circle	função	(center=(0.0, 0.0), radius=0.5) -> SDF	
rectangle	função	(center=(0.0, 0.0), half_extents=(0.5, 0.5)) -> SDF	
ellipse	função	(center=(0.0, 0.0), semi_axes=(0.5, 0.3)) -> SDF	
annulus	função	(center=(0.0, 0.0), r_inner=0.2, r_outer=0.5) -> SDF	
capsule2d	função	(a=(0.0, -0.5), b=(0.0, 0.5), radius=0.2) -> SDF	
sphere3d	função	(center=(0.0, 0.0, 0.0), radius=0.5) -> SDF	
box3d	função	(center=(0.0, 0.0, 0.0), half_extents=(0.5, 0.5, 0.5)) -> SDF	
cylinder3d	função	(center=(0.0, 0.0, 0.0), radius=0.5, height=1.0, axis=2) -> SDF	
torus3d	função	(center=(0.0, 0.0, 0.0), R=0.5, r=0.15, axis=2) -> SDF	

pinneapple_design.geometry.gen.threaded_connection

Threaded connection geometry for rotary coupling PINN simulations.

pinneapple_design/geometry/gen/threaded_connection.py · 442 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ThreadProfile	classe	(name, tpi, taper, h_factor, clearance_factor)	Geometric parameters for a single API rotary shouldered thread form.
ConnectionSpec	classe	(name, r_bore, r_outer, z_thread_start, z_thread_end, z_shoulder_top, thread, body, n_thr…)	Full geometry specification for one body (BOX or PIN) of an NC connection.
ThreadedConnectionMesher	classe	(spec, lc=1.0, lc_thread=None)	Generates a 2D axisymmetric mesh of an API threaded connection using gmsh.
TC50Geometry	classe	(body='BOX', lc=0.8, lc_thread=None)	TC50 threaded coupling geometry (large-diameter).
TC46Geometry	classe	(body='BOX', lc=0.8, lc_thread=None)	TC46 threaded coupling geometry (medium-diameter).

nome	tipo	assinatura	o que faz
TC38Geometry	classe	(body='BOX', lc=0.8, lc_thread=None)	TC38 threaded coupling geometry (standard).
build_thread_polygon	função	(spec, n_threads=None) -> List[Tuple[float, float]]	Builds the (r, z) polygon that describes the threaded surface profile.
get_connection_geometry	função	(connection, body='BOX', lc=0.8) -> ThreadedConnectionMesher	Factory: retorna o mesher para uma conexão pelo nome.

pinneapple_design.geometry.io

Geometry I/O helpers.

pinneapple_design/geometry/io/__init__.py · 34 linhas · **sem teste direto**

pinneapple_design.geometry.io.cadquery_bridge

CadQuery solid-to-PhysicalSample conversion via tessellation.

pinneapple_design/geometry/io/cadquery_bridge.py · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
cadquery_solid_to_upd	função	(solid, *, tess_linear_deflection=0.2, tess_angular_deflection=0.3, units='mm', meta=None…)	Convert a CadQuery solid/workplane to a triangle mesh and wrap as PhysicalSample.

pinneapple_design.geometry.io.iges

IGES file import utilities.

pinneapple_design/geometry/io/iges.py · 157 linhas · testes: 6

nome	tipo	assinatura	o que faz
IgesImportConfig	classe	(kind, mesh_size, algorithm_2d, algorithm_3d, curvature_refine, optimize, heal_shapes)	
iges_to_mesh	função	(iges_path, *, cfg=None, cache_dir=None) -> MeshData	Convert an IGES file into MeshData via gmsh.

pinneapple_design.geometry.io.meshio_bridge

Meshio mesh loading and UPD PhysicalSample packaging for VTK/VTU/MSH/XDMF.

pinneapple_design/geometry/io/meshio_bridge.py · 149 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
load_meshio	função	(path, *, require_triangles=True) -> MeshData	Load a mesh file using meshio and return a triangle MeshData.
save_meshio	função	(mesh, path, *, file_format=None, point_data=None, cell_data=None) -> None	Save MeshData via meshio.
meshio_to_upd	função	(path)	Load mesh + point data from any meshio-supported format (VTK, Exodus, etc.)

pinneapple_design.geometry.io.sdf_meshing

Meshing utilities for implicit SDF geometries.

`pinneapple_design/geometry/io/sdf_meshing.py` · 172 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SDFGrid2D`	classe	`(bounds_min, bounds_max, resolution)`	
`sample_boundary_points_sdf2d`	função	`(sdf, *, grid=None, n=2048, rng=None, band=0.01) -> np.ndarray`	Sample boundary points from a 2D SDF by rejection within a thin band.
`marching_squares_contours`	função	`(sdf, *, grid=None, level=0.0) -> list[np.ndarray]`	Extract contour polylines for the level set sdf(x)=level.
`sdf2d_to_tri_mesh`	função	`(sdf, *, grid=None, mesh_size=0.03, cache_path=None) -> MeshData`	Build a 2D triangulated mesh of the zero level-set using gmsh.

pinneapple_design.geometry.io.step

STEP file import utilities.

`pinneapple_design/geometry/io/step.py` · 132 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`StepImportConfig`	classe	`(kind, mesh_size, algorithm_2d, algorithm_3d, curvature_refine, optimize)`	
`step_to_mesh`	função	`(step_path, *, cfg=None, cache_dir=None) -> MeshData`	Convert a STEP file into MeshData via gmsh.

pinneapple_design.geometry.io.stl

STL loading utilities returning MeshData.

`pinneapple_design/geometry/io/stl.py` · 27 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_stl`	função	`(path, *, repair=True, compute_normals=True) -> MeshData`	Convenience STL loader (via trimesh) returning MeshData.

pinneapple_design.geometry.io.trimesh_bridge

Trimesh to MeshData adapter for loading, repair, and conversion.

`pinneapple_design/geometry/io/trimesh_bridge.py` · 111 linhas · testes: 8

nome	tipo	assinatura	o que faz
`TrimeshBridge`	classe	`()`	Adapter between trimesh.Trimesh and Pinneapple MeshData.

pinneapple_design.geometry.mesh

Módulo sem docstring e sem texto.

`pinneapple_design/geometry/mesh/__init__.py` · 26 linhas · **sem teste direto**

pinneapple_design.geometry.mesh.mesher_2d

2D mesh generation utilities.

pinneapple_design/geometry/mesh/mesher_2d.py · 436 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Mesh2D	classe	(vertices, triangles, boundary_edges, boundary_points, meta)	Lightweight 2D triangle mesh.
mesh_rectangle_structured	função	(x_range=(0.0, 1.0), y_range=(0.0, 1.0), nx=20, ny=20) -> Mesh2D	Generate a structured triangular mesh of a rectangle.
mesh_sdf_2d	função	(sdf_fn, bounds_min, bounds_max, *, mesh_size=0.05, mesh_size_boundary=None, boundary_n_p…)	Generate a quality 2D mesh inside an SDF domain using gmsh.
mesh_polygon_2d	função	(vertices, *, mesh_size=None, holes=None) -> Optional['Mesh2D']	Triangulate a 2D polygon using scipy.spatial.Delaunay (basic) or the `triangle` library (quality mesh, if available).
mesh_quality_report	função	(mesh) -> Dict[str, Any]	Generate a comprehensive mesh quality report.

pinneapple_design.geometry.mesh.mesher_3d

3D mesh generation from surfaces and SDFs.

pinneapple_design/geometry/mesh/mesher_3d.py · 839 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Mesh3D	classe	(vertices, tets, surface_faces, boundary_groups, meta)	Tetrahedral volume mesh.
PhysicsPointCloud3D	classe	(interior, boundary, boundary_normals, near_wall, near_wall_dist, boundary_groups, meta)	Collocation + boundary + near-wall points for physics training.
mesh_from_surface	função	(surface, *, mesh_size=0.0, mesh_size_boundary=0.0, boundary_layer=False, bl_thickness=0.…)	Generate a tetrahedral volume mesh from a closed 3D surface mesh.
mesh_from_sdf_3d	função	(sdf_fn, bounds_min, bounds_max, *, resolution=64, mesh_size=0.0, clean=True, volume=True…)	Extract a surface from a 3D SDF via marching cubes, then volume-mesh it.
sample_physics_points	função	(mesh, *, n_interior=10000, n_boundary=2000, n_near_wall=0, near_wall_thickness=0.0, curv…)	Generate physics-ready collocation points from a 3D mesh.
surface_from_pointcloud	função	(points, *, normals=None, method='poisson', depth=8, alpha=0.0, density_quantile=0.05) ->…)	Reconstruct a surface mesh from a point cloud.

pinneapple_design.geometry.mesh_collocator

MeshCollocator — 3D collocation point sampler for PINN training.

pinneapple_design/geometry/mesh_collocator.py · 388 linhas · testes: 151

nome	tipo	assinatura	o que faz
`MeshCollocatorConfig`	classe	`(n_interior, n_boundary, domain, boundary_mode, hollow, seed, max_rejection_tries)`	Configuration for MeshCollocator.
`CollocationBatch3D`	classe	`(X_interior, X_boundary, normals_boundary, boundary_groups)`	Collocation batch ready for PINN training.
`MeshCollocator`	classe	`(mesh, config=None, *, inner_mesh=None)`	Collocation point sampler for 3D mesh geometry.

`pinneapple_design.geometry.ops`

Módulo sem docstring e sem texto.

`pinneapple_design/geometry/ops/__init__.py` · 53 linhas · **sem teste direto**

`pinneapple_design.geometry.ops.clean`

Comprehensive 3D mesh cleaning pipeline.

`pinneapple_design/geometry/ops/clean.py` · 602 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CleanMeshConfig`	classe	`(merge_vertices, merge_tol, remove_degenerate, remove_duplicates, remove_unreferenced, fi…)`	Full configuration for the clean_mesh pipeline.
`mesh_quality_report_3d`	função	`(mesh) -> Dict[str, Any]`	Comprehensive quality report for a 3D surface mesh.
`smooth_mesh`	função	`(mesh, *, method='taubin', iterations=5, lam=0.5, mu=-0.53) -> MeshData`	Laplacian or Taubin mesh smoothing.
`remove_outlier_vertices`	função	`(mesh, *, nb_neighbors=20, std_ratio=2.0) -> MeshData`	Statistical outlier vertex removal.
`keep_largest_component`	função	`(mesh) -> MeshData`	Keep only the largest connected component of the mesh.
`decimate_mesh`	função	`(mesh, *, target_faces=None, ratio=0.5) -> MeshData`	Mesh decimation (polygon reduction).
`remove_self_intersections`	função	`(mesh) -> MeshData`	Remove self-intersecting faces using pymeshlab.
`clean_mesh`	função	`(mesh, *, config=None, verbose=False) -> MeshData`	Comprehensive 3D surface mesh cleaning pipeline.

`pinneapple_design.geometry.ops.curvature`

Discrete differential geometry on triangle surface meshes (NumPy/SciPy).

`pinneapple_design/geometry/ops/curvature.py` · 188 linhas · testes: 5

nome	tipo	assinatura	o que faz
`boundary_vertices`	função	`(mesh_or_vertices, faces=None) -> np.ndarray`	Boolean mask of vertices on a boundary edge (edge used by one face).

nome	tipo	assinatura	o que faz
euler_characteristic	função	(mesh_or_vertices, faces=None) -> int	
mixed_areas	função	(mesh_or_vertices, faces=None) -> np.ndarray	Per-vertex mixed Voronoi area; the areas sum to the surface area.
cotan_laplacian	função	(mesh_or_vertices, faces=None) -> sparse.csr_matrix	Symmetric cotangent matrix C with C_ij = (cot a_ij + cot b_ij)/2 off the diagonal and C_ii = -sum_j C_ij; (C f)_i is the Laplacian of f integrated over vertex i's mixed area (Eq. 8), so the pointwise Laplace-Beltrami is (C f)_i / A_i.
laplace_beltrami	função	(f, mesh_or_vertices, faces=None) -> np.ndarray	Pointwise Laplace-Beltrami of a vertex field f (n,) or (n, d).
gaussian_curvature	função	(mesh_or_vertices, faces=None, *, boundary='geodesic') -> np.ndarray	Angle-defect Gaussian curvature per vertex.
mean_curvature	função	(mesh_or_vertices, faces=None, *, signed=True, boundary='nan') -> np.ndarray	Mean curvature H per vertex from Delta x = -2 H n.
principal_curvatures	função	(mesh_or_vertices, faces=None) -> Tuple[np.ndarray, np.ndarray]	

pinneapple_design.geometry.ops.features

Face/vertex normals, areas, and mesh feature computation.

pinneapple_design/geometry/ops/features.py · 88 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
compute_face_normals	função	(mesh) -> np.ndarray	Compute face normals (M,3).
compute_vertex_normals	função	(mesh) -> np.ndarray	Compute vertex normals by area-weighted face normals.
compute_face_areas	função	(mesh) -> np.ndarray	Compute triangle face areas (M,).
compute_curvature_proxy	função	(mesh) -> np.ndarray	Very lightweight curvature proxy per face.

pinneapple_design.geometry.ops.lbm_bridge

Bridge from a surface mesh (STL / trimesh) to a boolean LBM obstacle mask.

pinneapple_design/geometry/ops/lbm_bridge.py · 249 linhas · testes: 7

nome	tipo	assinatura	o que faz
mesh_to_obstacle_mask	função	(mesh, bounds, resolution, *, aoa_deg=0.0, rotation_axis='y', oracle_supersample=2, devic…)	Voxelize a surface mesh into a boolean 3D LBM obstacle/solid mask.

pinneapple_design.geometry.ops.meshfree

Meshfree geometry utilities.

pinneapple_design/geometry/ops/meshfree.py · 397 linhas · testes: 151

nome	tipo	assinatura	o que faz
RBFInterpolator	classe	(kernel='gaussian', eps=1.0, reg=1e-10) -> None	Scattered data interpolation via Radial Basis Functions.
ImplicitSurfaceRBF	classe	(kernel='gaussian', eps=2.0, offset=0.01, reg=1e-08) -> None	Reconstruct a continuous SDF approximation from a point cloud with normals.

pinneapple_design.geometry.ops.occ_features

Direct, parametric OpenCASCADE (OCC) solid-modeling feature operations.

pinneapple_design/geometry/ops/occ_features.py · 206 linhas · testes: 9

nome	tipo	assinatura	o que faz
fillet_edges	função	(solid, radius, edge_selector=None)	Apply a real, parametric constant-radius fillet to a solid's edges.
chamfer_edges	função	(solid, distance, edge_selector=None)	Apply a real, parametric chamfer to a solid's edges.
shell_solid	função	(solid, thickness, faces_to_remove=None)	Hollow out a solid into a shell of the given wall thickness.

pinneapple_design.geometry.ops.pointcloud

Point cloud utilities.

pinneapple_design/geometry/ops/pointcloud.py · 80 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PointCloud	classe	(points, normals, features)	
mesh_to_pointcloud	função	(mesh, *, n_surface=4096, seed=0, device=None, dtype=torch.float32, include_normals=True)…)	
sample_pointcloud_from_mesh	função	(points, cells, *, n=8192, seed=0) -> np.ndarray	Convenience wrapper used by the webapp.
sdf2d_to_pointcloud	função	(sdf, *, grid=None, n_boundary=4096, band=0.01, seed=0, device=None, dtype=torch.float32)…)	

pinneapple_design.geometry.ops.remesh

Surface remeshing via pymeshlab isotropic explicit remeshing.

pinneapple_design/geometry/ops/remesh.py · 58 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
remesh_surface	função	(mesh, *, max_edge_length=None, target_edge_length=None) -> MeshData	Surface remeshing (best-effort).

pinneapple_design.geometry.ops.repair

Best-effort mesh repair using trimesh (duplicates, degenerate, normals, holes).

pinneapple_design/geometry/ops/repair.py · 61 linhas · testes: 1

nome	tipo	assinatura	o que faz
repair_mesh	função	(mesh, *, remove_duplicates=True, remove_degenerate=True, fix_normals=True, fill_holes=Fa…)	Best-effort mesh repair using trimesh.

pinneapple_design.geometry.ops.simplify

Mesh simplification via trimesh or Open3D quadric decimation.

pinneapple_design/geometry/ops/simplify.py · 187 linhas · testes: 1

nome	tipo	assinatura	o que faz
simplify_mesh	função	(mesh, *, target_faces=10000, backend='auto', on_missing='error') -> MeshData	Simplify a mesh.

pinneapple_design.geometry.ops.voxelize

Voxelization and meshfree sampling utilities for PINNeAPPle geometries.

pinneapple_design/geometry/ops/voxelize.py · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
VoxelGrid	classe	(data, origin, voxel_size, dim, meta)	Regular axis-aligned 3-D (or 2-D) voxel grid.
voxelize_domain	função	(bounds, resolution, device=torch.device('cpu')) -> VoxelGrid	Create a fully-occupied VoxelGrid for a rectangular domain.
voxelize_sdf	função	(sdf_fn, bounds, resolution, *, threshold=0.0, device=torch.device('cpu')) -> VoxelGrid	Voxelize a signed-distance field: voxel is occupied when sdf ≤ threshold.
voxelize_pointcloud	função	(points, bounds, resolution, *, values=None, aggregate='mean', device=torch.device('cpu')…)	Rasterize a point cloud into a voxel grid.
voxelgrid_to_collocation	função	(grid, threshold=0.5, include_values=False)	Extract occupied voxel centres as collocation points for PINNeAPPle.
sample_voxelgrid	função	(grid, n_samples, strategy='random', threshold=0.5) -> torch.Tensor	Sample collocation points from a VoxelGrid.

pinneapple_design.geometry.optimize.loop

Geometry optimization loop helpers.

pinneapple_design/geometry/optimize/loop.py · 124 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GeometryOptimizer	classe	(space, *, seed=0, sigma0=0.2)	UI-friendly geometry optimizer backed by EvolutionaryDesignOptimizer.

pinneapple_design.geometry.retrieval

Geometry retrieval: find the parts most similar to a query part.

pinneapple_design/geometry/retrieval.py · 122 linhas · testes: 3

nome	tipo	assinatura	o que faz
`GeometryIndex`	classe	`(ids, vectors, captions)`	Cosine-similarity index over part vectors (brute force; fine up to ~10^5-10^6 parts).
`shape_descriptor`	função	`(mesh, n_pairs=20000, bins=48, seed=0) -> np.ndarray`	D2 histogram (`bins`) + 3 sorted normalised principal extents, L2-normalised.
`load_caption_index`	função	`(parquet_paths, id_column='abc_id', limit=None) -> GeometryIndex`	Index finalrev-style parquet shards (columns `abc_id`, `caption`, `embedding`).

`pinneapple_design.geometry.sample`

Módulo sem docstring e sem texto.

`pinneapple_design/geometry/sample/__init__.py` · 31 linhas · testes: 3

`pinneapple_design.geometry.sample.barycentric`

Barycentric sampling and interpolation on triangles.

`pinneapple_design/geometry/sample/barycentric.py` · 66 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sample_barycentric_uv`	função	`(n, rng=None) -> Tuple[np.ndarray, np.ndarray, np.ndarray]`	Sample barycentric weights (w0, w1, w2) uniformly over a triangle.
`sample_points_on_triangles`	função	`(tri_v0, tri_v1, tri_v2, n, rng=None) -> np.ndarray`	Sample n points uniformly on a single triangle defined by (v0,v1,v2).
`interpolate_on_triangles`	função	`(values_v0, values_v1, values_v2, w0, w1, w2) -> np.ndarray`	Barycentric interpolation of any per-vertex values.

`pinneapple_design.geometry.sample.grids`

Uniform and Latin hypercube sampling in axis-aligned boxes.

`pinneapple_design/geometry/sample/grids.py` · 63 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sample_uniform_box`	função	`(bounds_min, bounds_max, n, rng=None) -> np.ndarray`	Uniform random samples inside an axis-aligned bounding box.
`sample_latin_hypercube_box`	função	`(bounds_min, bounds_max, n, rng=None) -> np.ndarray`	Latin Hypercube Sampling (LHS) in an axis-aligned box. Good for collocation coverage without clustering.

`pinneapple_design.geometry.sample.marching_squares`

Módulo sem docstring e sem texto.

`pinneapple_design/geometry/sample/marching_squares.py` · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`marching_squares_boundary`	função	`(sdf, *, bounds_min=(0.0, 0.0), bounds_max=(1.0, 1.0), resolution=256, device=None, dtype…)`	Return boundary polyline vertices as (M,2) tensor.
`sample_boundary_points`	função	`(sdf, n, *, bounds_min=(0.0, 0.0), bounds_max=(1.0, 1.0), resolution=256, device=None, dt…)`	Sample n boundary points from the extracted contour vertices.

`pinneapple_design.geometry.sample.particles`

Particle samplers for SPH and particle-based datasets.

`pinneapple_design/geometry/sample/particles.py` · 51 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sample_box_particles`	função	`(*, lo, hi, spacing, jitter=0.0, device=None, dtype=torch.float32) -> torch.Tensor`	Regular grid of particles in an axis-aligned box.
`sample_circle_particles`	função	`(*, center, radius, spacing, jitter=0.0, device=None, dtype=torch.float32) -> torch.Tensor`	2D particles inside a circle.

`pinneapple_design.geometry.sample.points`

Surface and boundary point sampling for triangle meshes.

`pinneapple_design/geometry/sample/points.py` · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sample_surface_points`	função	`(mesh, n, *, rng=None, return_normals=True, return_face_id=True)`	Sample n points uniformly over the surface (triangle areas as weights).
`sample_surface_points_weighted`	função	`(mesh, n, *, face_weights, rng=None, return_normals=True, return_face_id=True)`	Sample n points on surface using user-provided per-face weights.

`pinneapple_design.geometry.trough`

Longitudinal section of a melt runner (trough) as a 2-D OpenFOAM mesh: a sloped floor, a weir (skimmer) at the downstream end, a jet entering through the top, and open air above.

`pinneapple_design/geometry/trough.py` · 115 linhas · testes: 12

nome	tipo	assinatura	o que faz
`TroughSection`	classe	`(length, slope_deg, weir_height, top, inlet_x, inlet_width, cell, thickness)`	

`pinneapple_design.geometry.vehicle_cad`

Parametric vehicle CAD: closed-mesh primitives for rover-class vehicles, parts with materials and masses, mass properties (centre of mass and inertia from the meshes) and assembly into bodies for multibody simulation.

`pinneapple_design/geometry/vehicle_cad.py` · 715 linhas · testes: 19

nome	tipo	assinatura	o que faz
CADPart	classe	(name, body, vertices, faces, material, mass, group)	
VehicleCAD	classe	(title, parts, frames, notes)	Parts grouped into bodies; every coordinate is in one assembly frame (the vehicle at its reference pose).
orient_outward	função	(mesh) -> Mesh	Flip the winding of a closed mesh whose signed volume is negative.
signed_volume	função	(V, F) -> float	
box	função	(size, center=(0, 0, 0), R=None) -> Mesh	
beveled_box	função	(size, center=(0, 0, 0), bevel=0.02, n=3, R=None) -> Mesh	Box with rounded edges of radius bevel (convex hull of eight corner spheres).
cylinder	função	(p0, p1, radius, n=32) -> Mesh	Closed cylinder (or frustum, radius=(r0, r1)) between two points.
tube_shell	função	(axis_p0, axis_p1, r_in, r_out, n=48) -> Mesh	Closed annular tube (a wheel rim): inner and outer cylinders with annular end faces.
revolve_closed	função	(profile, axis_origin=(0, 0, 0), axis=(0, 0, 1), n=48) -> Mesh	Closed solid of revolution of a closed polygon profile of (r, h) points about axis.
paraboloid_dish	função	(diameter, depth, thickness, center, axis, n=40, m=8) -> Mesh	Dish antenna reflector: a paraboloid shell of depth over diameter (its vertex at center).
loft	função	(rings, cap='fan') -> Mesh	Closed solid skinned through rings (m, n, 3): m closed polylines of n points each, in corresponding order.
offset_polygon	função	(poly, d) -> np.ndarray	Convex polygon (k, 2) moved inward by d along every edge normal (outward for d < 0).
chamfered_rect	função	(sx, sy, c=0.0) -> np.ndarray	Rectangle sx x sy centred at the origin with its four corners cut at 45 deg by c (CCW).
extrude_polygon	função	(poly, z0, z1, chamfer_top=0.0, chamfer_bottom=0.0, R=None, t=(0, 0, 0)) -> Mesh	Prism of the convex polygon poly (k, 2) from z0 to z1, its top and bottom edges chamfered at 45 deg by chamfer_top / chamfer_bottom (the cap polygons are inset by that amount), then rotated by R and moved by t. Use it for body panels with bevelled vertical edges (cut the ...
chamfered_box	função	(size, center=(0, 0, 0), edge=0.0, top=0.0, bottom=0.0, R=None) -> Mesh	Box with flat 45 deg chamfers: edge on the four vertical edges, top / bottom on the edges of the top and bottom faces (a machined or sheet-metal look, as opposed to the rounded :func:beveled_box).

nome	tipo	assinatura	o que faz
box_beam	função	(p0, p1, section0, section1=None, up=(0, 0, 1), chamfer=0.0) -> Mesh	Straight (optionally tapered) box-section beam from p0 to p1. section = (width, height): height along up (projected normal to the beam), width across; section1 tapers it to the far end; chamfer cuts the four long edges (as a fraction of the smaller side of each section).
ladder_rail	função	(p0, p1, width, rail=(0.03, 0.04), rungs=4, side=(0, 1, 0), rung=(0.02, 0.02)) -> Mesh	Ladder / guide-rail frame: two parallel box rails width apart (centre to centre) along side, from p0 to p1, tied by rungs cross members (deployment rails, linear stages, truss frames).
helix_sweep	função	(origin, axis, r_in, r_out, thickness, pitch, length, n_per_turn=24, phase=0.0) -> Mesh	Helical flight (auger flute, screw thread): a rectangular section from r_in to r_out and thickness along the axis swept along a helix of pitch for length from origin along axis.
mli_blanket	função	(width, height, thickness=0.012, wrinkle=0.004, n=(28, 14), seed=0) -> Mesh	Multi-layer-insulation blanket in the x-z plane centred at the origin: a slab whose outer face (-y, like :func:solar_panel) is crumpled by wrinkle [m] of smooth random relief, the back face flat.
rotate	função	(mesh, R, about=(0, 0, 0)) -> Mesh	
translate	função	(mesh, t) -> Mesh	
rot	função	(axis, angle_rad)	
merge	função	(meshes) -> Mesh	
mesh_mass_properties	função	(V, F, mass) -> tuple[np.ndarray, np.ndarray, float]	Centre of mass and inertia tensor about it [kg m^2] of a closed mesh of uniform density holding mass, and its volume [m^3] (Eberly, "Polyhedral mass properties", 2002).
combine_mass	função	(items) -> tuple[float, np.ndarray, np.ndarray]	Total mass, centre of mass and inertia about it of (mass, com, inertia about own com) items.
rover_wheel	função	(radius, width, *, rim_thickness=0.004, grousers=0, grouser_height=0.0, grouser_thickness …)	A rigid rover wheel in its own frame (axle along y, centre at the origin): rim shell, grousers (straight lugs or chevrons), a spoked web on the inner side, hub (drive actuator housing). Returns parts (suffix, mesh, material, mass_share); mass shares are by volume x material density ratio.
solar_panel	função	(width, height, thickness=0.025, cells=(6, 4), gap=0.012, frame=0.03)	Vertical solar panel in the x-z plane (facing -y... rotate as needed), centred at the origin: aluminium frame and substrate plus a grid of cells standing proud by 1 mm.
radiator	função	(width, depth, thickness=0.02, fins=12, fin_height=0.03)	Horizontal radiator plate (x-y plane, top at z = 0) with fins parallel fins along x.

nome	tipo	assinatura	o que faz
`camera_head`	função	`(width, depth, height, baseline, lens_radius=0.022, lights=2)`	Mast camera head facing +x: housing, a stereo pair of lenses `baseline` apart, LED light bars.
`actuator`	função	`(p0, p1, radius, cap_radius=None, cap_length=None, flange=0.012, housing='white_paint', t…)`	Rotary actuator in assembly coordinates: a cylindrical housing from `p0` to `p1` (output end at `p0`), flange rings at both ends and a motor / gearbox cap beyond `p1` (BLDC motor plus strain-wave gear class joints: suspension, steering, mast pan).
`pan_tilt_head`	função	`(width=0.36, depth=0.15, height=0.12, baseline=0.24, lens_radius=0.02, lights=6, aft_came…)`	Mast head on a pan-tilt unit, in its own frame (pan axis = z through the origin, looking +x): pan drum, a U-yoke carrying the tilt axle, a camera box with a stereo pair of lenses `baseline` apart, a row of `lights` LEDs under them and an optional aft-looking camera. The top of the head is at …
`drill_assembly`	função	`(rail_height=1.3, rail_width=0.16, rail_offset=0.075, auger_radius=0.0125, flight_radius=…)`	Rotary-percussive planetary drill with a deployment rail, in its own frame: drill axis = z through the origin, footpad bottom at z = 0, the rail frame (and the mount face) on the -x side. Parts: a ladder guide rail of `rail_height` with a backing deployment stage, the drill head …

`pinneapple_design.geometry.vehicles3d`

Parametric 3-D vehicles for CFD and photoreal rendering: a road car and a launch vehicle.

`pinneapple_design/geometry/vehicles3d.py` · 687 linhas · testes: 4

nome	tipo	assinatura	o que faz
`RoadCar`	classe	`(length, width, roof_height, clearance, hood_height, deck_height, windshield_deg, slant_d…)`	A four-door coupé / sedan: `style` "fastback" (roof runs into a slanted rear window down to the tail) or "notchback" (rear window, then a trunk deck). Dimensions in metres.
`LaunchVehicle`	classe	`(length, diameter, fairing_length, fairing_diameter, interstage_x, fin_root, fin_tip, fin…)`	A two-stage launcher with a tangent-ogive fairing and four tail fins in a + configuration.
`FormulaCar`	classe	`(wheelbase, front_axle_x, ride_height, front_track, rear_track, wheel_radius, front_tyre_…)`	A parametric open-wheel racing car in the spirit of current Formula 1 (not any team's car): monocoque with sidepods and engine cover, floor with a diffuser, multi-element front and rear wings with endplates (DRS flap open or closed), driver helmet, halo and four exposed wheels. Metres, x from the …
`road`	função	`(length=9.0, width=6.5, x0=-2.2) -> Part`	A flat asphalt plane at z = 0 (drawn only).
`naca4`	função	`(m=0.06, p=0.4, t=0.1, n=40) -> np.ndarray`	Closed NACA 4-digit profile (x, z) on a unit chord, trailing edge -> upper surface -> leading edge -> lower surface, with a finite trailing edge (0.3 % thick) so the solid stays closed.

pinneapple_design.geometry_library

Fachada da biblioteca de geometrias (exposta como `pp.geometries`): listar, construir, exportar, renderizar miniaturas, carregar arquivos pré-gerados e exportar a galeria web.

`pinneapple_design/geometry_library/__init__.py` · 96 linhas · testes: 34

nome	tipo	assinatura	o que faz
`list`	função	`(category=None) -> builtins.list[str]`	Nomes das geometrias da biblioteca, opcionalmente filtrados por categoria.
`categories`	função	`() -> dict[str, builtins.list[str]]`	Dicionário categoria -> nomes das geometrias.
`export`	função	`(name, path, fmt=None, lod='light', **params) -> str`	Constrói a geometria com os parâmetros dados e grava no formato pedido (ou pela extensão).
`render`	função	`(name, path=None, lod='light', size=(960, 640), **kw)`	Renderiza a miniatura de uma geometria com parâmetros padrão (câmera e parâmetros físicos opcionais).
`thumbnail`	função	`(name)`	Caminho da miniatura PNG versionada, renderizando-a se não existir.
`export_web`	função	`(directory, **kw) -> dict[str, Any]`	Gera o índice da galeria e as miniaturas para o navegador.

pinneapple_design.geometry_library.core

Registro das entradas, especificação de parâmetros com faixas e fontes, o corpo fechado `Body` e o objeto `Geometry` devolvido por `build`.

`pinneapple_design/geometry_library/core.py` · 321 linhas · testes: 21

nome	tipo	assinatura	o que faz
`Param`	classe	`(name, default, unit, range, doc, choices)`	Um parâmetro físico: padrão, unidade, faixa admissível e o que controla.
`Source`	classe	`(what, ref, estimate)`	De onde vem um número do mundo real; `estimate=True` marca valor escolhido por julgamento, sem fonte publicada.
`Body`	classe	`(name, vertices, faces, materials, group, aero, closed)`	Uma superfície fechada (estanque, normais para fora) ou uma folha aberta só para desenho, com material por corpo ou por face.
`GeometrySpec`	classe	`(name, title, category, description, builder, params, dimensions, frame, sources, physics…)`	Especificação de uma entrada: nome, título, categoria, construtor, parâmetros, dimensões, fontes e usos físicos.
`Geometry`	classe	`(name, bodies, params, lod, spec)`	Geometria construída: corpos fechados com materiais, em metros, com malha, dimensões, volume, checagens, exportação e renderização.
`register`	função	`(spec) -> GeometrySpec`	Adiciona uma `GeometrySpec` ao registro, recusando nomes repetidos e chaves de dimensão inválidas.
`get_spec`	função	`(name) -> GeometrySpec`	Devolve a especificação registrada de um nome (carrega as entradas se preciso).
`params_hash`	função	`(name, params, lod) -> str`	Hash curto (SHA-256) de nome, nível de detalhe e parâmetros.
`build`	função	`(name, lod='light', **params) -> Geometry`	Constrói uma geometria da biblioteca no nível de detalhe `light` ou `fine` com os parâmetros físicos dados.

nome	tipo	assinatura	o que faz
`names`	função	`() -> list[str]`	Lista ordenada dos nomes registrados.
`info`	função	`(name) -> dict[str, Any]`	Informações de uma entrada: dimensões, parâmetros com faixas, fontes e usos.
`summary_table`	função	`() -> str`	Tabela Markdown da biblioteca (nome, título, tamanho, usos).

`pinneapple_design.geometry_library.entries`

Pacote de entradas: cada módulo dentro dele registra suas geometrias ao ser importado (descoberta automática). [sem teste]

`pinneapple_design/geometry_library/entries/__init__.py` · 7 linhas · **sem teste direto**

`pinneapple_design.geometry_library.entries._adapt`

Adaptador que transforma as listas de peças dos modelos anteriores (`Part` de aero, `CADPart` de veículo) em corpos fechados da biblioteca, soldando peças por material. [sem teste]

`pinneapple_design/geometry_library/entries/_adapt.py` · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`bodies_from_items`	função	`(items, prefix='') -> list[Body]`	Agrupa peças por grupo, solda vértices e separa em cascas conexas, cada uma virando um `Body` com material por face.
`base_name`	função	`(name, material) -> str`	Nome da pele de onde veio uma peça dividida por material (ex.: `fuselage_window` → `fuselage`).
`drop_degenerate`	função	`(bodies) -> list[Body]`	Remove faces degeneradas (vértices repetidos) dos corpos.
`close_sheets`	função	`(bodies, thickness=0.004) -> list[Body]`	Dá espessura às folhas abertas dos modelos antigos para que todo corpo seja uma casca fechada.

`pinneapple_design.geometry_library.entries.aircraft_new`

Registra três aeronaves movidas a rotor a partir de dados publicados: turboélice regional classe ATR 72-600, helicóptero leve classe H145 e drone quadricóptero classe F450. [sem teste]

`pinneapple_design/geometry_library/entries/aircraft_new.py` · 880 linhas · **sem teste direto**

`pinneapple_design.geometry_library.entries.industrial_bio`

Registra geometrias industriais e biomédicas de referência: dissipador de pinos, rotor centrífugo fechado, prédio alto padrão CAARC e arco da aorta torácica. [sem teste]

`pinneapple_design/geometry_library/entries/industrial_bio.py` · 656 linhas · **sem teste direto**

`pinneapple_design.geometry_library.entries.legacy`

Registra as primeiras entradas, embrulhando modelos já existentes no repositório: avião comercial A320, carro de passeio, carro de Fórmula, lançador de dois estágios e o rover NASA VIPER. [sem teste]

`pinneapple_design/geometry_library/entries/legacy.py` · 288 linhas · **sem teste direto**

`pinneapple_design.geometry_library.entries.marine_rail_space`

Registra entradas marinhas, ferroviárias e espaciais: navio KCS em escala real, casco Wigley analítico, trem de alta velocidade classe ICE 3 e lançador classe Falcon 9. [sem teste]

`pinneapple_design/geometry_library/entries/marine_rail_space.py` · 890 linhas · **sem teste direto**

`pinneapple_design.geometry_library.entries.wind_turbine`

Registra a turbina eólica de referência NREL 5 MW (Jonkman et al. 2009): pás pela tabela publicada de corda/torção/perfil, cubo, nacele e torre. [sem teste]

`pinneapple_design/geometry_library/entries/wind_turbine.py` · 160 linhas · **sem teste direto**

`pinneapple_design.geometry_library.exporters`

Escritores de arquivo determinísticos: STL binário, OBJ + MTL, glTF binário, OpenUSD ASCII e STEP AP214 (B-rep facetado), mais leitura de STL e checagem do STEP via OpenCASCADE.

`pinneapple_design/geometry_library/exporters.py` · 180 linhas · testes: 21

nome	tipo	assinatura	o que faz
`normalise_format`	função	`(fmt) -> str`	Normaliza o nome do formato (aceita aliases como usd e stp) e valida.
`stl_bytes`	função	`(V, F, header) -> bytes`	STL binário de uma malha (V, F) com cabeçalho de 80 bytes.
`to_stl`	função	`(geom, aero_only=False) -> bytes`	Bytes STL de uma `Geometry` (opcionalmente só os corpos aerodinâmicos).
`to_obj`	função	`(geom) -> bytes`	Texto OBJ de uma `Geometry`, um grupo por superfície com referência ao MTL.
`to_mtl`	função	`(geom) -> bytes`	Arquivo MTL com as cores dos materiais usados.
`to_glb`	função	`(geom) -> bytes`	Bytes glTF binário da cena da geometria com a tabela de materiais.
`to_usda`	função	`(geom) -> bytes`	Texto OpenUSD da cena da geometria.
`to_step`	função	`(geom, aero_only=False) -> bytes`	Arquivo STEP AP214 com um `FACETED_BREP` por corpo fechado (folhas abertas são ignoradas).
`export`	função	`(geom, path, fmt=None) -> str`	Grava a geometria num caminho no formato dado ou deduzido da extensão.
`read_stl`	função	`(path)`	Lê STL binário como triângulos (m, 3, 3).
`step_check`	função	`(path) -> dict`	Relê um STEP com OpenCASCADE (`cadquery`): número de sólidos e volume total.

pinneapple_design.geometry_library.kit

Blocos de construção de superfícies fechadas para as entradas: lofts, sólidos de revolução, seções NACA e asas/pás, caixas arredondadas, cilindros, tubos, extrusões e rotações.

pinneapple_design/geometry_library/kit.py · 481 linhas · testes: 21

nome	tipo	assinatura	o que faz
rot	função	(axis, angle_deg) -> np.ndarray	Matriz de rotação em torno de um eixo por um ângulo em graus (Rodrigues).
tf	função	(mesh, R=None, t=(0, 0, 0), about=(0, 0, 0), scale=1.0) -> Mesh	Aplica escala, rotação e translação a uma malha.
mirror	função	(mesh, axis=1) -> Mesh	Espelha uma malha num eixo, invertendo a ordem das faces.
with_mirror	função	(mesh, axis=1) -> list[Mesh]	Lista com a malha e sua cópia espelhada.
merge	função	(meshes) -> Mesh	Junta várias malhas numa só.
ear_clip_fast	função	(P2) -> np.ndarray	Triangulação por ear clipping de polígono simples anti-horário, com testes vetorizados.
loft	função	(rings, start='cap', end='cap') -> Mesh	Superfície fechada por anéis, com pontas planas (cap) ou em polo.
revolve	função	(x, r, n=48, axis='x', center=(0.0, 0.0, 0.0), phase=0.0) -> Mesh	Sólido de revolução de um meridiano r(x) em torno de um eixo.
superellipse	função	(hw, hh, n, e=2.0, zc=0.0, yc=0.0, top_e=None) -> np.ndarray	Anel de pontos de uma superelipse (com expoente separado para a metade de cima, opcional).
body_of_sections	função	(xs, hw, zb, zt, n=48, e=2.4, e_top=None) -> Mesh	Corpo tipo fuselagem com seções superelípticas ao longo de x.
naca4	função	(m=0.0, p=0.4, t=0.12, n=40, closed_te=True) -> np.ndarray	Contorno NACA 4 dígitos em sentido anti-horário, com opção de bordo de fuga fechado.
thick_te	função	(sec, te) -> np.ndarray	Abre o bordo de fuga até uma espessura finita.
section_ring	função	(sec2d, chord, twist_deg, le, frame=None, pivot=0.25) -> np.ndarray	Posiciona uma seção de corda unitária em 3D com torção e bordo de ataque dados.
wing	função	(stations, n=36, frame=None, tip='cap') -> Mesh	Loft de asa ou pá por estações (bordo de ataque, corda, torção, espessura, arqueamento), fechado nas pontas.
blend_sections	função	(a, b, w) -> np.ndarray	Interpola linearmente duas seções com peso w.
round_section	função	(n) -> np.ndarray	Círculo unitário no formato de seção, com o mesmo número de pontos de naca4.
box	função	(size, center=(0, 0, 0), R=None) -> Mesh	Caixa fechada de tamanho dado, com rotação e centro opcionais.
rounded_box	função	(size, center=(0, 0, 0), radius=0.05, n_side=4, R=None) -> Mesh	Caixa com arestas e cantos arredondados (soma de Minkowski de caixa e esfera).

nome	tipo	assinatura	o que faz
`cylinder`	função	`(p0, p1, radius, n=32, r1=None) -> Mesh`	Cilindro (ou tronco de cone) fechado entre dois pontos.
`ellipsoid`	função	`(center, radii, nu=24, nv=14, R=None) -> Mesh`	Elipsoide fechado com semieixos dados.
`tube`	função	`(path, radius, n=16, radii=None) -> Mesh`	Tubo fechado varrido ao longo de uma polilinha com referenciais de transporte paralelo.
`tube_loop`	função	`(path, radius, n=16) -> Mesh`	Tubo fechado ao longo de uma polilinha fechada (anéis, aros).
`torus`	função	`(center, R_major, r_minor, nu=48, nv=16, axis=(0, 0, 1)) -> Mesh`	Toro fechado com raios maior e menor.
`ear_clip`	função	`(poly) -> np.ndarray`	Triangula um polígono simples por ear clipping, em qualquer sentido.
`extrude`	função	`(poly2d, z0, z1, plane='xy') -> Mesh`	Prisma: polígono extrudado num eixo com tampas trianguladas.
`sweep_profile`	função	`(profile2d, path, up=(0, 0, 1)) -> Mesh`	Perfil 2D fechado varrido ao longo de um caminho, com tampas (trilhos, alças, nervuras).
`is_closed_volume`	função	`(mesh) -> bool`	True se a malha encerra volume positivo.

`pinneapple_design.geometry_library.meshops`

Operações de malha em NumPy/SciPy: soldar vértices, orientar normais, estanqueidade, volume, autointerseção, solidificar folhas e união booleana (via `manifold3d`).

`pinneapple_design/geometry_library/meshops.py` · 413 linhas · testes: 42

nome	tipo	assinatura	o que faz
`concat`	função	`(meshes) -> Mesh`	Concatena malhas reindexando as faces.
`compact`	função	`(V, F) -> Mesh`	Remove vértices não referenciados.
`weld`	função	`(V, F, tol=1e-09) -> Mesh`	Funde vértices coincidentes por tolerância relativa e remove faces colapsadas.
`area`	função	`(V, F) -> float`	Área total da malha.
`volume`	função	`(V, F) -> float`	Volume encerrado com sinal (positivo para normais para fora).
`face_normals`	função	`(V, F, unit=True) -> np.ndarray`	Normais das faces, unitárias ou não.
`edge_stats`	função	`(V, F) -> dict[str, int]`	Conta arestas de borda, não manifold e com orientação inconsistente.
`components`	função	`(F, n_vertices) -> np.ndarray`	Rótulo de componente conexo de cada face.
`orient_outward`	função	`(V, F) -> Mesh`	Inverte cada componente fechado com volume negativo.
`quality`	função	`(V, F, closed=True) -> dict[str, float]`	Resumo de qualidade da malha: contagens de arestas, estanqueidade, volume e área.
`self_intersections`	função	`(V, F, max_pairs=20000000, return_pairs=False)`	Conta pares de triângulos não adjacentes que se cruzam, com busca por k-d trees.

nome	tipo	assinatura	o que faz
`union`	função	`(meshes) -> Mesh`	União booleana de malhas fechadas com `manifold3d`.
`crease_normals`	função	`(V, F, angle_deg=38.0) -> np.ndarray`	Normais por canto que mantêm arestas vivas acima de um ângulo.
`solidify`	função	`(V, F, thickness)`	Transforma uma folha aberta numa casca fina fechada.
`fix_winding`	função	`(V, F) -> Mesh`	Torna a orientação das faces consistente em cada componente e orienta para fora.
`refan_concave_caps`	função	`(V, F, materials=None, min_valence=5)`	Retriangula tampas planas em leque que se dobram sobre si (ex.: tampa de perfil arqueado).

`pinneapple_design.geometry_library.render`

Renderizador por software (NumPy + Pillow, sem GPU) para miniaturas, folha de contatos e GIF giratório, com sombras, oclusão ambiente e supersampling. [sem teste]

`pinneapple_design/geometry_library/render.py` · 362 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`rasterize`	função	`(sx, sy, depth, F, W, H, persp=True, max_frag=3000000)`	Rasteriza triângulos num z-buffer com id de triângulo e coordenadas baricêntricas por pixel.
`render_geometry`	função	`(geom, path=None, size=(960, 640), azimuth=None, elevation=None, ssaa=2, background='stud…)`	Renderiza uma `Geometry` e devolve uma imagem PIL (e grava se `path` for dado).
`contact_sheet`	função	`(geoms, path, cols=4, tile=(480, 330), title='') -> str`	Grade de miniaturas com nomes, em PNG.
`turntable`	função	`(geoms, path, frames=24, cols=4, tile=(240, 165), duration_ms=90) -> str`	GIF animado com todas as geometrias girando numa grade.

`pinneapple_design.geometry_library.store`

Arquivos pré-gerados: manifesto com SHA-256, cópias locais, cache e Hugging Face Hub; `load` busca nessa ordem e reconstrói pelo código se o arquivo faltar.

`pinneapple_design/geometry_library/store.py` · 138 linhas · testes: 21

nome	tipo	assinatura	o que faz
`file_name`	função	`(name, lod, fmt) -> str`	Nome do arquivo pré-gerado: `<nome>.<lod>.<formato>`.
`thumbnail_path`	função	`(name) -> Path`	Caminho da miniatura PNG de uma entrada.
`sha256`	função	`(path) -> str`	SHA-256 de um arquivo, lido em blocos.
`read_manifest`	função	`() -> dict[str, Any]`	Lê o manifesto de arquivos (ou devolve um vazio).
`write_manifest`	função	`(m) -> None`	Grava o manifesto de arquivos em JSON.

nome	tipo	assinatura	o que faz
`cache_dir`	função	`() -> Path`	Diretório de cache das geometrias (`PINNEAPPLE_GEOMETRY_CACHE` ou `~/.cache/pinneapple/geometries`).
`offline`	função	`() -> bool`	True se o modo offline estiver ligado por variável de ambiente.
`hub_url`	função	`(name, fname) -> str`	URL do arquivo no dataset do Hugging Face Hub.
`load`	função	`(name, format='stl', lod='light', *, allow_build=True, allow_download=True) -> Path`	Caminho do arquivo pré-gerado de uma entrada, buscando repositório, cache e Hub, com conferência de hash.

`pinneapple_design.geometry_library.web`

Gera o índice JSON da galeria e miniaturas JPEG de cada entrada para o navegador.

`pinneapple_design/geometry_library/web.py` · 75 linhas · testes: 13

nome	tipo	assinatura	o que faz
`index`	função	`() -> dict[str, Any]`	Índice da galeria com o que cada cartão precisa (título, dimensões, arquivo, fontes, parâmetros, código).
`export_web`	função	`(directory, *, thumb_size=(360, 240)) -> dict[str, Any]`	Grava `index.json` e `thumbs/<nome>.jpg` no diretório e devolve o índice.

`pinneapple_design.intent`

Design from intent: a plain-language goal in, a buildable and verified design package out.

`pinneapple_design/intent/__init__.py` · 73 linhas · testes: 24

`pinneapple_design.intent.api`

`design(text)`: from a sentence to a verified design package, a list of questions, or a refusal.

`pinneapple_design/intent/api.py` · 193 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NeedsInput`	classe	`(spec)`	The request is in scope but a critical value is missing: answer `questions` and call again with `answers={slot: value or (value, unit)}`.
`design`	função	`(text, *, answers=None, llm=None, template=None, seed=0, out_dir=None, verify=True, n_scr…)`	Design from a plain-language goal.

`pinneapple_design.intent.cli`

`pinneapple design "<goal>"`: the design-from-intent pipeline from a shell.

`pinneapple_design/intent/cli.py` · 64 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	

`pinneapple_design.intent.figures`

Figures of a design package (matplotlib, Agg). Every plotted number comes from a model run of the package; each figure says which model in its title or legend.

`pinneapple_design/intent/figures.py` · 300 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pareto_figure`	função	`(ctx, x, y, xlabel, ylabel, out_dir) -> str | None`	Every reduced-model evaluation of the optimisation: feasible / infeasible, and the chosen design.
`loop_figures`	função	`(t, spec, d, ctx, out_dir) -> dict[str, str]`	
`loop_gif`	função	`(t, d, traj, out_dir, *, n_frames=48, size=(720, 480)) -> str`	The car going round, from the Newton trajectory (software renderer, one fixed camera).
`bracket_figures`	função	`(t, spec, d, ctx, out_dir) -> dict[str, str]`	
`heatsink_figures`	função	`(t, spec, d, ctx, out_dir) -> dict[str, str]`	
`wing_figures`	função	`(t, spec, d, ctx, out_dir) -> dict[str, str]`	

`pinneapple_design.intent.materials`

Materials with cited properties for the design templates.

`pinneapple_design/intent/materials.py` · 128 linhas · testes: 21

nome	tipo	assinatura	o que faz
`Prop`	classe	`(value, unit, source, note)`	
`Material`	classe	`(key, label, category, props, processes, notes)`	
`heat_sink_material`	função	`(key) -> Material`	The heat-sink model's own material (conductivity and density exactly as its physics uses them).
`get`	função	`(key) -> Material`	
`candidates`	função	`(*needs, category=None) -> list[Material]`	Materials with a cited value for every property in `needs`.
`find`	função	`(text) -> list[str]`	Material keys named in free text ("aluminium", "steel", "PLA", "balsa", ...).

`pinneapple_design.intent.package`

DesignPackage: everything needed to build and check the design, and the files it is written to.

`pinneapple_design/intent/package.py` · 400 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DesignPackage`	classe	`(spec, plan, optimisation, candidate, design, robustness, alternatives, verification, ass…)`	
`make_geometry`	função	`(name, title, bodies)`	A :class:`pinneapple_design.geometry_library.core.Geometry` from bodies (exports STL / STEP / glTF).
`write_html`	função	`(pkg, path) -> str`	
`write_pdf`	função	`(pkg, path) -> str`	A short PDF: the verdict, the design, the verification and the figures (matplotlib).

`pinneapple_design.intent.parse`

Intent -> RequirementSpec: pick a template, read the numbers with their units, fill the template's slots.

`pinneapple_design/intent/parse.py` · 416 linhas · testes: 21

nome	tipo	assinatura	o que faz
`OutOfScope`	classe	`(text, kind, reason, needs, closest)`	The request cannot be designed here: `reason` says why and `needs` what would be required.
`Mention`	classe	`(value, unit, start, end, quote)`	
`IntentLLM`	classe	`()`	
`CallLLM`	classe	`(provider='anthropic', model=None, **kw)`	Adapter from :func:`pinneapple_llm.call_llm` (Anthropic / OpenAI / Ollama) to :class:`IntentLLM`.
`register`	função	`(template) -> ProblemTemplate`	Add a template (an instance) to the registry used by :func:`formulate`.
`templates`	função	`() -> dict[str, ProblemTemplate]`	
`get_template`	função	`(name) -> ProblemTemplate`	
`find_quantities`	função	`(text) -> list[Mention]`	Every number in `text` with the unit that follows it (the longest run of up to three tokens that parses as a unit, e.g. `m / s`, `deg C`, `W/m2K`). Numbers glued to letters (`6061`, `NACA2412`) are skipped.
`llm_schema`	função	`(tpls) -> dict[str, Any]`	
`select_template`	função	`(text, tpls=None) -> list[tuple[str, float]]`	Templates ranked by keyword score (sum of the weights of the keywords found in the text).
`out_of_scope`	função	`(text) -> tuple[str, str] | None`	
`formulate`	função	`(text, *, llm=None, answers=None, template=None) -> RequirementSpec | OutOfScope`	Turn a request into a :class:`RequirementSpec` (or an :class:`OutOfScope` with what is needed).

pinneapple_design.intent.physics

Módulo sem docstring e sem texto.

pinneapple_design/intent/physics/__init__.py · 1 linhas · testes: 21

pinneapple_design.intent.physics.bracket

Angle bracket under a tip load: beam theory, an in-house hexahedral FE model, and the CalculiX model with the filleted corner and the bolt holes.

pinneapple_design/intent/physics/bracket.py · 181 linhas · testes: 21

nome	tipo	assinatura	o que faz
beam	função	(P, b, w, t, E)	Cantilever of length b - t (from the top of the mounting leg) under a tip load: root bending stress M c / I and tip deflection P L^3 / (3 E I) (vectorised).
l_mesh	função	(a, b, w, t, n_t=2, h=None, n_w=4, fillet=0.0)	Structured hexahedra of the L (no holes; the fillet, if any, as a staircase of whole elements): a :class:HexMesh, the clamped nodes (bottom of the mounting leg) and the loaded nodes (top face of the arm).
hex_model	função	(P, a, b, w, t, E, nu, n_t=2, fillet=0.0)	Solve the L with :class:SolidFEM (C3D8I, incompatible modes). Returns the tip deflection (mean x displacement of the loaded face), the nominal root stress (bending stress on the arm's outer face at the section z = 2.5 t, scaled to the root moment arm b - t), and the result.
geo_shelf_bracket	função	(a, b, w, t, fillet, hole) -> str	gmsh (OpenCASCADE) script: the L with a filleted inner corner and two bolt holes in the mounting leg.
gmsh_exe	função	() -> str | None	
export_cad	função	(geo, workdir) -> dict[str, str]	STEP (the OpenCASCADE B-rep, exact holes and fillet) and STL (surface triangulation) from the gmsh script.
calculix_model	função	(P, a, b, w, t, fillet, hole, E, nu, rho, size, workdir)	Mesh with gmsh (C3D10 tetrahedra), clamp the mounting leg's bottom face (bolted), load the arm's top face, solve with CalculiX. Returns (model, result, peak_vm, peak_where, delta, reaction).
section_stress	função	(m, r, b, w, t, z_frac=0.45)	Mean |sigma_zz| on the arm's outer face at z = z_frac b (away from the corner and the load), and the beam-theory value M c / I there.
mesh_sizes	função	(t) -> tuple[float, float]	Two quadratic-tetrahedron sizes: about two and three elements through the thickness.

pinneapple_design.intent.physics.loop_track

Loop-the-loop track for a small car: the track curve, a 1-D model of the car along it, the closed form, and the Newton rigid-body scene of the same track.

pinneapple_design/intent/physics/loop_track.py · 390 linhas · testes: 21

nome	tipo	assinatura	o que faz
`LoopTrack`	classe	`(radius, clothoid, run_in, run_out, width, pitch)`	
`Run`	classe	`(completes, v_top, v_exit, n_min, n_max, top_margin, energy_lost, s_fail, profile)`	Result of the 1-D model for a batch of designs (arrays of shape `(n_designs,)`) plus the profiles of the first design.
`NewtonLoopResult`	classe	`(completes, v_top, max_gap, s_reached, t, pos, vel, s, wall_s, solver, dt, meta)`	
`along_track`	função	`(radius, clothoid, v0, mass, *, crr=0.02, cd=0.6, area=0.0006, com_height=0.008, wheel_in…)`	Integrate `d(v^2)/ds` along the centre-of-mass path for a batch of designs (all arguments broadcast).
`closed_form`	função	`(radius, clothoid, v0, mass, *, crr=0.02, cd=0.6, area=0.0006, com_height=0.008, wheel_in…)`	Screening estimate (vectorised): energy to the top with the losses at the mean speed, the top condition and the peak load.
`track_meshes`	função	`(track, thickness=0.003, rail=0.008, n=600)`	Closed meshes of the running strip and the two side rails, in metres: `{"strip": (V, F), "rail_l": ...}`. The strip lies below the running surface (normal offset `-thickness`..0).
`car_mesh`	função	`(length, width, height, wheel_r)`	A simple car for drawings and files: rounded body plus four wheels, resting on z = 0, nose along +x.
`newton_scene`	função	`(track, mass, v0, *, radius=0.006, mu=0.0, thickness=0.003, rail=0.008, n=900)`	The track (one static triangle mesh: strip + rails) and the car as a sphere of the wheel radius resting on the running surface, launched along +x, as a :class:`~pinneapple_simulation.external_solvers.newton_physics. RigidScene`.
`newton_check`	função	`(track, mass, v0, *, dt=2.5e-05, duration=None, iterations=12, record_every=None, radius=…)`	Run the car through the loop in NVIDIA Newton (XPBD contact on the track mesh) and measure whether it stays on the track and reaches the end, the speed at the top and the largest gap to the running surface in the loop.

`pinneapple_design.intent.pipeline`

RequirementSpec -> PhysicsPlan -> multi-fidelity optimisation -> verification and robustness.

`pinneapple_design/intent/pipeline.py` · 450 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PhysicsPlan`	classe	`(template, title, physics, assumptions, variables, objectives, failure_modes, fidelities,…)`	
`CallLog`	classe	`(calls, wall)`	Model calls per fidelity: designs evaluated and wall time.

nome	tipo	assinatura	o que faz
`Candidate`	classe	`(x, design, outputs, closed_form, rank)`	
`OptimisationResult`	classe	`(candidates, method, n_screen, n_feasible_screen, n_reduced, log, study, notes)`	
`Robustness`	classe	`(p_success, ci, n, per_mode, margin_p05, inputs, sensitivity, level, calibration)`	
`Verification`	classe	`(ran, label, reason, result, comparison)`	
`Assessment`	classe	`(trust, verdict, probability, ci, assumptions, reasons)`	
`plan`	função	`(spec, template) -> PhysicsPlan`	
`decode`	função	`(vs, x) -> dict[str, Any]`	
`optimise`	função	`(spec, template, *, seed=0, n_screen=2000, budget=600, n_candidates=3, slow_budget=48, pr…)`	Screen, optimise, and return up to `n_candidates` feasible designs (best first objective, or nearest the ideal point of the Pareto front). `options` are passed to every `evaluate` call (calibration factors).
`robustness`	função	`(spec, template, cand, *, n=2000, seed=0, level=REDUCED, log=None, calibration=None, opti…)`	P(every failure-mode margin >= 0) under the template's uncertain inputs, by Monte Carlo on `level`.
`verify`	função	`(spec, template, cand, *, log=None, workdir=None, options=None) -> Verification`	Run the full solver on `cand` (if installed) and compare with the reduced and closed-form models evaluated with the same options as the full run.
`assess`	função	`(spec, template, cand, rob, ver, *, agree_tol=0.1) -> Assessment`	

`pinneapple_design.intent.spec`

Typed requirement objects: what the user asked for, with units, bounds and the provenance of every value.

`pinneapple_design/intent/spec.py` · 241 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpecError`	classe	`()`	A value that cannot be accepted: unknown unit, wrong dimension, or outside the template's range.
`Quantity`	classe	`(name, value, unit, provenance, source, quote)`	

nome	tipo	assinatura	o que faz
`Slot`	classe	`(name, label, unit, display_unit, keywords, cues, critical, default, default_source, boun…)`	A value a template needs or can use.
`Question`	classe	`(slot, text, unit, choices, why)`	
`ObjectiveChoice`	classe	`(name, sense, label, keywords)`	An objective a template can optimise: `name` is an output of its models, `sense` min / max.
`RequirementSpec`	classe	`(text, template, values, objectives, questions, issues, unsupported, parser, match_score)`	
`to_si`	função	`(value, unit) -> float`	`value` in `unit` converted to SI (absolute temperatures keep their offset).
`from_si`	função	`(value, unit) -> float`	
`same_dimension`	função	`(a, b) -> bool`	

`pinneapple_design.intent.template`

Problem templates: how a kind of request maps to physics, design variables, models and failure modes.

`pinneapple_design/intent/template.py` · 218 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Fidelity`	classe	`(level, label, method, modules, executables, check)`	
`FailureMode`	classe	`(name, label, description, critical)`	
`DesignVariable`	classe	`(name, lower, upper, unit, label, integer, choices)`	
`Uncertain`	classe	`(name, nominal, spread, dist, unit, source)`	An uncertain input of the robustness study: `nominal` and a symmetric `half_width` (uniform) or a standard deviation (normal), with the reason for the range.
`FullResult`	classe	`(outputs, checks, level, label, wall_s, artifacts, notes, compare, compare_options, compa…)`	What the high-fidelity solver measured for one design.
`ProblemTemplate`	classe	`(name, title, domain, keywords, physics, assumptions, slots, objectives, failure_modes, f…)`	Base class. Subclasses set the class attributes and implement :meth:`evaluate` and the package hooks.

`pinneapple_design.intent.templates`

Built-in problem templates. A new domain plugs in by subclassing :class:`~pinneapple_design.intent.template.ProblemTemplate` and calling :func:`pinneapple_design.intent.register`.

pinneapple_design/intent/templates/__init__.py · 12 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
builtin	função	() -> list	

pinneapple_design.intent.templates.bracket

Angle (shelf) bracket that holds a load at the end of its arm: thickness, width, mounting-leg length, material.

pinneapple_design/intent/templates/bracket.py · 346 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
BracketTemplate	classe	()	

pinneapple_design.intent.templates.example_spring

Template-authoring example: a helical compression spring (the worked example of the guide docs/guides/design_from_intent.md). It is not registered by default; register(CoilSpringTemplate()) adds it.

pinneapple_design/intent/templates/example_spring.py · 91 linhas · testes: 21

nome	tipo	assinatura	o que faz
CoilSpringTemplate	classe	()	

pinneapple_design.intent.templates.glider_wing

Wing of a model glider: span, aspect ratio, taper and section for the lowest sink rate.

pinneapple_design/intent/templates/glider_wing.py · 297 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
GliderWingTemplate	classe	()	
profile_cd0	função	(re, tc)	Wing profile drag coefficient on the planform area: 2 (1 + 0.2 t/c) x C_f(Re) x FF (wetted area of a thin wing about twice the planform).

pinneapple_design.intent.templates.heatsink

Passive (or fan-cooled) plate-fin heat sink for a heat source: base, fins, material.

pinneapple_design/intent/templates/heatsink.py · 327 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
HeatSinkTemplate	classe	()	

pinneapple_design.intent.templates.loop_car

Toy car through a loop-the-loop: track radius, clothoid entry, launch speed, car mass, track strip.

pinneapple_design/intent/templates/loop_car.py · 469 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LoopCarTemplate	classe	()	

pinneapple_design.qualitative

Qualitative physics preview of geometries and geometry changes, before the quantitative analysis.

`pinneapple_design/qualitative/__init__.py` · 36 linhas · testes: 13

pinneapple_design.qualitative.descriptors

Geometric descriptors of closed triangulated bodies and assemblies of named parts.

`pinneapple_design/qualitative/descriptors.py` · 259 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Descriptors`	classe	`(volume, wetted_area, centroid, extents, length, frontal_area, equivalent_diameter, slend…)`	Geometric descriptors of one body (SI units). `direction` is the flow / load reference direction.
`Assembly`	classe	`(parts)`	Named parts of one geometry. Parts are treated as touching, not overlapping (areas add).
`as_mesh`	função	`(geom) -> tuple[np.ndarray, np.ndarray]`	`(vertices, faces)` from a tuple, a trimesh-like object (`.vertices`/`.faces`) or an `Assembly`.
`extrude`	função	`(polygon, depth=1.0, axis='y') -> tuple[np.ndarray, np.ndarray]`	Closed prism from a counter-clockwise 2-D polygon (n, 2) in the (x, z) plane, extruded along `axis` by `depth` (centred), so 2-D sections can be compared with the same models.
`face_geometry`	função	`(V, F)`	Per-face area, unit normal and centroid.
`describe`	função	`(geom, direction=(1.0, 0.0, 0.0)) -> Descriptors`	Descriptors of a closed, outward-oriented triangulated body. The projected area is `0.5 * sum |n . d| A` (exact for convex bodies, an upper bound otherwise).
`section_profile`	função	`(geom, axis=0, n_slices=40, n_grid=48)`	Section properties along `axis` from a voxelization by ray parity: slice positions `x` (centres), area `A(x)`, centroid of each section in the two other coordinates, second moments `I(x)` about the section's centroidal axes (`I_a`: about the first remaining axis, i.e. bending in the …

pinneapple_design.qualitative.models

Cheap physical models that turn a geometry into first estimates, the mechanisms behind them and a surface map.

`pinneapple_design/qualitative/models.py` · 501 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`Quantity`	classe	`(name, unit, better, pt, en)`	

nome	tipo	assinatura	o que faz
`Evaluation`	classe	`(quantities, terms, by_part, face_field, field_label, notes, caveats, mesh, extra)`	Result of one model on one geometry.
`QualitativeModel`	classe	`(**conditions)`	
`ExternalFlow`	classe	`()`	
`ConvectiveCooling`	classe	`()`	
`Cantilever`	classe	`()`	
`ScalingModel`	classe	`(formulas, better=None, direction=(1.0, 0.0, 0.0), **conditions)`	User formulas of the geometric descriptors, e.g. `ScalingModel({"conduction_resistance": lambda d, c: d.length / (c["k"] * d.frontal_area)}, better={"conduction_resistance": "min"}, k=200)`. The mechanisms are the descriptors: each one's share of a change comes from swapping it alone (in log ...
`sheltered_faces`	função	`(V, F, normals, centroids, direction, grid=96, tol=0.05) -> np.ndarray`	Windward faces hidden behind other surfaces seen from upstream (in the wake of something in front of them): the windward triangles are rasterized on the plane normal to the flow, keeping the most upstream depth per cell; a windward face whose cell has a surface more than `tol` x body length ...

`pinneapple_design.qualitative.preview`

Qualitative preview: how each geometry, or each change to a part, should affect the result, before any simulation; then the quantitative check.

`pinneapple_design/qualitative/preview.py` · 523 linhas · testes: 13

nome	tipo	assinatura	o que faz
`Objective`	classe	`(quantity, sense, model)`	
`VariantPreview`	classe	`(name, evaluation, change, mechanisms, parts, confidence, reasons, good)`	
`QualitativePreview`	classe	`(model, objective, baseline, variants, geometries, lang)`	

nome	tipo	assinatura	o que faz
`QuantitativeCheck`	classe	`(preview, values, seconds)`	
`PartSensitivity`	classe	`(objective, model, base_value, rows, lang)`	
`parse_objective`	função	`(objective, model=None) -> Objective`	`"minimizar arrasto"`, `"maximize heat rejection"`, `("tip_deflection", "min")` or an `Objective`. Downforce means lift with sense "min".
`preview`	função	`(geometries, objective, conditions=None, baseline=None, model=None, lang='pt') -> Qualita…)`	Qualitative comparison of `geometries` (name -> mesh tuple, trimesh-like object or `Assembly`) for an `objective` such as "minimizar arrasto", "maximize heat rejection", "aumentar rigidez por massa" or `("tip_deflection", "min")`. `conditions` are the model's operating conditions (speed, …
`part_sensitivity`	função	`(assembly, objective, conditions=None, step=0.2, model=None, lang='pt', axes='xyz', about…)`	For each part: grow it by `step` along each axis (about its minimum corner by default, i.e. away from where it is usually attached) and remove it; the change of the objective ranks where to act first.

`pinneapple_design.qualitative.quantitative`

Quantitative counterparts of the qualitative models, for `QualitativePreview.quantify`.

`pinneapple_design/qualitative/quantitative.py` · 132 linhas · testes: 13

nome	tipo	assinatura	o que faz
`hex8_incompatible_stiffness`	função	`(size, E, nu) -> np.ndarray`	24 x 24 stiffness of a rectangular brick `size = (hx, hy, hz)` with Wilson's incompatible modes condensed.
`voxel_fem_cantilever`	função	`(geom, E=70000000000.0, nu=0.3, load=1000.0, axis=None, load_axis=2, clamp='min', n_slice…)`	Tip deflection, stiffness, maximum von Mises stress and mass of a body clamped at one end of `axis` (default the longest) with `load` (N) along `load_axis` spread over the other end face. `cells_across`: voxels along the larger section dimension (the other is scaled to keep the voxels near …

`pinneapple_design.surrogate_inverse`

Surrogate-driven inverse design on closed-form physics, with the answer re-checked against the physics.

`pinneapple_design/surrogate_inverse/__init__.py` · 38 linhas · testes: 91

`pinneapple_design.surrogate_inverse.brake_rotor`

Vented brake rotor: the radial vanes between the two friction rings, as a fin array, under a sustained load.

`pinneapple_design/surrogate_inverse/brake_rotor.py` · 269 linhas · testes: 94

nome	tipo	assinatura	o que faz
`RotorDesign`	classe	`(vane_count, vane_gap_m, vane_thickness_m)`	
`OptimizationResult`	classe	`(design, surrogate_predicted_temp_c)`	
`profile_params`	função	`(design, power_w, h_conv) -> dict[str, float]`	Formula quantities, plus `theta_vane_base_c` (alias of the fin-root rise) and `lc_radial_m` (corrected length along the vane's radial extent, for the radial colour profile).
`rotor_temperature_c`	função	`(design, power_w, h_conv) -> float`	Ground truth: friction-ring temperature under a sustained heat input.
`thermal_resistance_k_w`	função	`(design, h_conv) -> float`	
`vane_temperature_at_radial_position_c`	função	`(design, power_w, h_conv, r_fraction) -> float`	The fin profile (same `m` and root rise) along the vane's radial extent: hottest at the inner edge (`r_fraction = 0`), coolest at the outer edge. Used to colour the vanes; the scalar model integrates over the gap direction, this profile is a display of the same fin parameters along the radius.
`thermal_field`	função	`(design, power_w, h_conv) -> dict[str, float]`	
`constraint_violations`	função	`(design) -> dict[str, float]`	
`design_to_dict`	função	`(design) -> dict`	
`generate_dataset`	função	`(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]`	
`train_surrogate`	função	`(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit`	
`predict_temperature_c`	função	`(surrogate, design, power_w, h_conv) -> float`	

nome	tipo	assinatura	o que faz
optimize_design	função	(surrogate, power_w, h_conv, steps=400, lr=0.05, penalty_weight=100.0) -> OptimizationRes…)	Minimise the surrogate's rotor temperature over (N, gap, t): same loss form (log R plus the relative spacing shortfall), sigmoid-bounded descent and integer rounding as :func:.heatsink_fins.optimize_design.
verify_design	função	(design, power_w, h_conv, surrogate_predicted_temp_c) -> VerificationCard	
monte_carlo_uncertainty	função	(design, power_w, h_conv, safety_threshold_c, n_samples=200, seed=None, thickness_toleran…)	Vane thickness and gap drawn uniformly within their tolerance bands (vane count is a part count and stays fixed), each sample through the formula (never the surrogate). Verdict on the 95th percentile p95: FRAGILE if above the threshold, MARGINAL within 5 % below it, ROBUST otherwise. …
mesh	função	(design) -> meshes.Mesh	Two friction rings and the radial vanes, in millimetres.
export_stl_bytes	função	(design) -> bytes	

pinneapple_design.surrogate_inverse.core

The machinery shared by every case: a small MLP surrogate, gradient descent on the design through the frozen surrogate, and a verification card that re-checks the answer against the ground truth.

pinneapple_design/surrogate_inverse/core.py · 267 linhas · testes: 16

nome	tipo	assinatura	o que faz
MLPSurrogate	classe	(bounds, hidden=48, n_layers=3, log_output=False)	tanh MLP with its inputs mapped from their bounds to [-1, 1] and its output standardised.
SurrogateFit	classe	(model, val_mean_rel_err, val_p95_rel_err, val_max_rel_err, n_train, n_val, epochs, secon…)	A trained surrogate with its held-out errors (relative, on rows it never trained on).
CheckResult	classe	(name, verdict, detail, value, limit)	
VerificationCard	classe	(checks, values, covered, not_covered)	The final design re-checked against the ground truth (never the surrogate grading itself).
fit_surrogate	função	(X, y, bounds, *, hidden=48, n_layers=3, epochs=1500, lr=0.003, seed=0, log_output=False,…)	Full-batch Adam with a cosine learning-rate decay on (X, y); the last val_fraction of the rows (the data are expected in random order) is held out and scored as |pred - y| / max(|y|, rel_floor).

nome	tipo	assinatura	o que faz
`freeze`	função	`(model) -> nn.Module`	Stop gradients into the weights: from here on only the design moves.
`descend`	função	`(loss_fn, x0, *, bounds, steps=400, lr=0.01, reparam='clamp', project=None) -> dict[str, …)`	Adam on the design variables `x0` to minimise `loss_fn(values)`.
`banded`	função	`(name, error, pass_limit, warn_limit, *, pass_text, warn_text, fail_text) -> CheckResult`	PASS when `error <= pass_limit`, WARNING up to `warn_limit`, FAIL above.
`relative_error`	função	`(predicted, reference, floor) -> float`	
`constraint_checks`	função	`(violations, ok_text=…, skip=()) -> list[CheckResult]`	One check per signed violation (`<= 0` is satisfied).

`pinneapple_design.surrogate_inverse.heatsink_fins`

Straight-fin CPU heat sink: fin count, height and thickness on a fixed 80 x 80 mm aluminium base.

`pinneapple_design/surrogate_inverse/heatsink_fins.py` · 264 linhas · testes: 91

nome	tipo	assinatura	o que faz
`HeatsinkDesign`	classe	`(fin_count, fin_height_m, fin_thickness_m)`	
`OptimizationResult`	classe	`(design, surrogate_predicted_temp_c, steps_run)`	
`profile_params`	função	`(design, power_w, h_conv) -> dict[str, float]`	The fin-array formula's intermediate quantities (`m`, `lc_m`, resistances, fin-root rise).
`junction_temperature_c`	função	`(design, power_w, h_conv) -> float`	Ground truth: base (junction) temperature. Used for the training labels and for the verification.
`thermal_resistance_k_w`	função	`(design, h_conv) -> float`	`(T_j - T_amb) / P`: independent of the power in this model.
`fin_temperature_at_height_c`	função	`(design, power_w, h_conv, z_above_root_m) -> float`	The 1-D fin profile `theta(z) = theta_root cosh(m (Lc - z)) / cosh(m Lc)` of the same formula.
`thermal_field`	função	`(design, power_w, h_conv) -> dict[str, float]`	What a viewer needs to colour the fins with the fin profile (evaluated per vertex).
`constraint_violations`	função	`(design) -> dict[str, float]`	Signed violations (`<= 0` satisfied).
`design_to_dict`	função	`(design) -> dict`	

nome	tipo	assinatura	o que faz
generate_dataset	função	(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]	Uniform designs, h spread around the fan profiles; label = thermal resistance from the formula.
train_surrogate	função	(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit	MLP on log R; fit.val_mean_rel_err is the held-out error on R, i.e. on the temperature rise.
predict_temperature_c	função	(surrogate, design, power_w, h_conv) -> float	
optimize_design	função	(surrogate, power_w, h_conv, steps=400, lr=0.05, penalty_weight=100.0) -> OptimizationRes…)	Minimise the surrogate's junction temperature over (N, H, t) from the baseline.
verify_design	função	(design, power_w, h_conv, surrogate_predicted_temp_c) -> VerificationCard	Re-run the formula on the final design: surrogate error on the temperature rise (PASS <= 5 %, WARNING <= 15 %), every design rule, and a plausibility bound on the temperature.
fin_cell_check	função	(design, power_w, h_conv) -> dict	The formula against 2-D finite-volume conduction in one fin pitch (same h, same power per fin).
mesh	função	(design) -> meshes.Mesh	Base plate plus fins, in millimetres.
export_stl_bytes	função	(design) -> bytes	

pinneapple_design.surrogate_inverse.meshes

Triangle meshes of the five designs, in millimetres, for a 3-D viewer or an STL download.

pinneapple_design/surrogate_inverse/meshes.py · 93 linhas · testes: 88

nome	tipo	assinatura	o que faz
merge	função	(parts) -> Mesh	Concatenate meshes (faces re-indexed).
bbox_size	função	(mesh) -> tuple[float, float, float]	
to_stl	função	(mesh, name) -> bytes	Binary STL (the header starts with the part name, never with solid).
fin_array	função	(base_w_mm, base_d_mm, base_t_mm, n_fins, fin_h_mm, fin_t_mm) -> Mesh	Base plate (centred on the origin, top face at z = base_t / 2) with n_fins straight fins spread evenly across its width, first and last flush with the edges.
vented_rotor	função	(r_mean_mm, radial_len_mm, ring_t_mm, gap_mm, n_vanes, vane_t_mm, n=96) -> Mesh	Two annular friction rings (z = 0 to ring_t and above the vent gap) joined by n_vanes radial vane blocks in the gap, around the z axis.
layered_disc	função	(radius_mm, layer_thicknesses_mm, n=96) -> Mesh	Stacked coaxial discs (z up from 0), e.g. a drug reservoir under a rate-controlling membrane.
rectangular_beam	função	(width_mm, thickness_mm, length_mm) -> Mesh	A plain box: x = width, y = thickness (the bending axis), z = length, centred on the origin (12 triangles).

nome	tipo	assinatura	o que faz
`flanged_tube`	função	`(radius_mm, length_mm, flange_radius_mm, flange_t_mm, n=64) -> Mesh`	A port: a cylinder of `radius` x `length` along z from 0, with a flange disc at its base.

`pinneapple_design.surrogate_inverse.reflex_port`

Bass-reflex port: radius and length that tune a cabinet to a target frequency, and the trade-off between a small port and an accurate tuning.

`pinneapple_design/surrogate_inverse/reflex_port.py` · 232 linhas · testes: 88

nome	tipo	assinatura	o que faz
`PortDesign`	classe	`(radius_m, length_m)`	
`OptimizationResult`	classe	`(design, surrogate_predicted_freq_hz)`	
`FrontierPoint`	classe	`(volume_weight, design, surrogate_predicted_freq_hz, real_achieved_freq_hz, real_freq_err…)`	
`resonant_frequency_hz`	função	`(design, cabinet_volume_l) -> float`	Ground truth (training labels and verification).
`acoustic_field`	função	`(design, cabinet_volume_l) -> dict[str, float]`	One value per design (the lumped model has no field along the port): the tuning within the target range.
`constraint_violations`	função	`(design, cabinet_volume_l) -> dict[str, float]`	
`design_to_dict`	função	`(design) -> dict`	
`generate_dataset`	função	`(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]`	
`train_surrogate`	função	`(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit`	
`predict_frequency_hz`	função	`(surrogate, design, cabinet_volume_l) -> float`	
`optimize_design`	função	`(surrogate, cabinet_volume_l, target_freq_hz, steps=400, lr=0.05, penalty_weight=100.0) -…)`	Minimise `((f_surrogate - f_target) / f_target)^2` plus a penalty on the port-volume rule, from the baseline, in the sigmoid-bounded space of :func:`.core.descend` (radius and length bounded, comparable step sizes).
`sweep_pareto_frontier`	função	`(surrogate, cabinet_volume_l, target_freq_hz, n_points=8, steps=400, lr=0.05, penalty_wei…)`	Volume-vs-accuracy trade-off for `lambda` in `linspace(0.05, 0.95, n_points)`.

nome	tipo	assinatura	o que faz
`verify_design`	função	`(design, cabinet_volume_l, target_freq_hz, surrogate_predicted_freq_hz) -> VerificationCa…)`	
`mesh`	função	`(design) -> meshes.Mesh`	Port tube with its flange, in millimetres.
`export_stl_bytes`	função	`(design) -> bytes`	

`pinneapple_design.surrogate_inverse.suspension_arm`

Suspension arm as a rectangular cantilever: the lightest width x thickness that still reaches a fatigue life.

`pinneapple_design/surrogate_inverse/suspension_arm.py` · 255 linhas · testes: 88

nome	tipo	assinatura	o que faz
`ArmDesign`	classe	`(width_m, thickness_m)`	
`OptimizationResult`	classe	`(design, surrogate_predicted_stress_pa)`	
`SensitivityResult`	classe	`(width_m, thickness_m, load_n, life_cycles, mass_kg, d_life_d_width, d_life_d_thickness, …)`	
`bending_stress_pa`	função	`(design, load_n) -> float`	Ground truth: bending stress at the clamped root, where the moment is largest.
`bending_stress_at_position_pa`	função	`(design, load_n, x_from_root_m) -> float`	`sigma(x) = 6 F (L - x) / (w t^2)`: largest at the root, zero at the loaded tip.
`fatigue_life_cycles`	função	`(design, load_n) -> float`	
`allowable_stress_for_life_pa`	função	`(target_life_cycles) -> float`	
`stress_field`	função	`(design, load_n) -> dict[str, float]`	
`constraint_violations`	função	`(design, load_n, target_life_cycles) -> dict[str, float]`	
`design_to_dict`	função	`(design) -> dict`	
`generate_dataset`	função	`(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]`	
`train_surrogate`	função	`(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit`	MLP on `log sigma` (the stress spans about 30 MPa to 6 GPa over the space).

nome	tipo	assinatura	o que faz
`predict_stress_pa`	função	`(surrogate, design, load_n) -> float`	
`optimize_design`	função	`(surrogate, load_n, target_life_cycles, steps=500, lr=0.0006, penalty_weight=5e-11) -> Op…)`	Minimise `w t` (mass at fixed length and density) plus `penalty_weight * max(0, sigma - sigma_allow)^2` through the surrogate's stress, from the baseline, clamped to the bounds.
`verify_design`	função	`(design, load_n, target_life_cycles, surrogate_predicted_stress_pa) -> VerificationCard`	
`compute_sensitivity`	função	`(width_m, thickness_m, load_n, step_m=H_M) -> SensitivityResult`	Derivatives of life and mass at the design point by central differences on the closed form (never the surrogate), with a step of `step_m` metres.
`sensitivity_to_dict`	função	`(r) -> dict`	
`mesh`	função	`(design) -> meshes.Mesh`	The beam as a plain box in millimetres (x width, y thickness, z length): the model has no fillets or holes.
`export_stl_bytes`	função	`(design) -> bytes`	

`pinneapple_design.surrogate_inverse.transdermal_patch`

Membrane-controlled release patch: area, membrane thickness and reservoir concentration for a target release rate and wear time, with the smallest area.

`pinneapple_design/surrogate_inverse/transdermal_patch.py` · 249 linhas · testes: 88

nome	tipo	assinatura	o que faz
`PatchDesign`	classe	`(area_m2, membrane_thickness_m, reservoir_conc_kg_m3)`	
`OptimizationResult`	classe	`(design, surrogate_predicted_rate_mg_day)`	
`release_rate_mg_day`	função	`(design) -> float`	Ground truth: steady release rate (training labels and verification).
`depletion_time_days`	função	`(design) -> float`	
`reservoir_mass_mg`	função	`(design) -> float`	

nome	tipo	assinatura	o que faz
concentration_at_membrane_depth	função	(design, z_from_reservoir_m) -> float	Linear steady profile C(z) = K C_res (1 - z / L) (sink at the outer face).
decay_rate_constant_per_day	função	(design) -> float	First-order constant of the tail after depletion, k = D K / (L L_res) = 1 / t_dep (area cancels).
simulate_release_timeseries	função	(design, duration_days=None, n_points=100) -> list[dict[str, float]]	Rate, cumulative release and nominal reservoir left over time (default: 1.5 x the depletion time).
concentration_field	função	(design) -> dict[str, float]	
constraint_violations	função	(design, target_wear_days) -> dict[str, float]	
design_to_dict	função	(design) -> dict	
generate_dataset	função	(n_samples, seed=0) -> tuple[torch.Tensor, torch.Tensor]	
train_surrogate	função	(n_samples=6000, epochs=1500, seed=0) -> SurrogateFit	MLP on log rate (the rate spans about 0.5 to 2600 mg/day over the design space).
predict_rate_mg_day	função	(surrogate, design) -> float	
optimize_design	função	(surrogate, target_rate_mg_day, target_wear_days, steps=1200, lr=0.15, weight_rate=80.0, …)	Minimise A / A_max + w_r (rel. rate error)^2 + w_w (rel. wear shortfall)^2 in a sigmoid-bounded space (area, membrane and concentration differ by six orders of magnitude; in that space one learning rate moves all three). The wear term uses the closed-form depletion time and aims 3 % above the …
verify_design	função	(design, target_rate_mg_day, target_wear_days, surrogate_predicted_rate_mg_day) -> Verifi…)	
mesh	função	(design) -> meshes.Mesh	Reservoir disc with the membrane on top, in millimetres (radius from the area).

nome	tipo	assinatura	o que faz
`export_stl_bytes`	função	`(design) -> bytes`	

`pinneapple_design.thermal`

Thermal design: heat sinks (sizing with physics + surrogate screening).

`pinneapple_design/thermal/__init__.py` · 2 linhas · testes: 3

`pinneapple_design.thermal.board_cooling`

The self-optimizing engineer: cool an overheating part on a board with a heat sink and the enclosure fans.

`pinneapple_design/thermal/board_cooling/__init__.py` · 29 linhas · testes: 10

`pinneapple_design.thermal.board_cooling.agent`

The self-optimizing engineer: a cooling request in plain language -> plan -> controlled tools -> evidence.

`pinneapple_design/thermal/board_cooling/agent.py` · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`t_read_mission`	função	`(state, text) -> dict[str, Any]`	
`t_baseline`	função	`(state) -> dict[str, Any]`	
`t_screen_natural`	função	`(state, gaps_mm=(4.0, 6.0, 8.0, 11.0)) -> dict[str, Any]`	Still air: the largest heat sink the envelope allows, for a few fin gaps (the optimum gap of a natural- convection sink is several mm). If even the best one misses the limit, forced air is needed.
`t_sample`	função	`(state, n=64, seed=0, workers=4) -> dict[str, Any]`	
`t_learn`	função	`(state, test_fraction=0.2, seed=0, max_error_k=3.0, min_coverage=0.9, add=32, max_n=320, …)`	Fit and measure on held-out designs; while the hold-out misses `max_error_k` or `min_coverage`, run `add` more reference solves (new Latin-hypercube points) and measure again, up to `max_n` solves. The learning curve (hold-out metrics per sample size) is part of the evidence.
`t_optimize`	função	`(state, rounds=5, k_sigma=2.0, seed=0) -> dict[str, Any]`	
`t_verify`	função	`(state) -> dict[str, Any]`	
`t_deliver`	função	`(state, out_dir='out', render=True, samples=24) -> dict[str, Any]`	

nome	tipo	assinatura	o que faz
`toolbox`	função	`() -> Toolbox`	
`template_plan`	função	`(text, *, n=64, out_dir='out', render=True, workers=4) -> TemplatePlanner`	
`run_mission`	função	`(text, *, out_dir='out', planner='template', n=64, render=True, workers=4, llm=None, prov…)`	Plan and run the cooling study. `planner`: "template" (deterministic) or "llm" (falls back to the template, with the reason in `run.plan.notes`, when no model answers or its plan is not valid).

`pinneapple_design.thermal.board_cooling.learn`

Fast approximation of the coupled reference: junction temperatures and the board-top temperature field.

`pinneapple_design/thermal/board_cooling/learn.py` · 107 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CoolingSurrogate`	classe	`(t_air_c, modes, tj_, map_, fit_seconds, n_train)`	
`errors`	função	`(pred_tj, pred_map, Y, maps) -> dict[str, Any]`	
`holdout`	função	`(data, t_air_c, *, test_fraction=0.2, seed=0, modes=16) -> dict[str, Any]`	Train on part of the reference solves, measure on the rest: errors of the surrogate and of the nearest snapshot, coverage of the +-2 std band, prediction time against the reference time.

`pinneapple_design.thermal.board_cooling.mission`

The cooling mission: a board with an overheating part, the limits, the space, the fan budget, the design space.

`pinneapple_design/thermal/board_cooling/mission.py` · 212 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CoolingDesign`	classe	`(base_width_mm, base_depth_mm, base_thickness_mm, n_fins, fin_thickness_mm, height_mm, ai…)`	One heat sink on the hot part plus the fan air speed. Lengths in mm, speed in m/s (approach velocity of the enclosure fans over the board and through the fins; 0 = natural convection).
`DesignSpace`	classe	`(lo, hi, min_gap_mm, min_fin_height_mm)`	Box bounds of the 7 design variables (`VARIABLES`) plus the geometric rules (fin gap, fin height).

nome	tipo	assinatura	o que faz
`CoolingMission`	classe	`(board, components, hot, t_ambient_c, tj_limit_c, max_width_mm, max_depth_mm, max_height_…)`	Board, parts, the overheating part (`hot`), air temperature, limits and the room for the cure.
`default_mission`	função	`(power_w=40.0) -> CoolingMission`	A 160 x 100 mm, 4-layer controller board: a `power_w` SoC (BGA 35 x 35, junction limit 90 °C), two 2 W regulators (QFN-32) and a 1.5 W memory (BGA 17 x 17), air at 35 °C inside the enclosure. Package data: the typical values of `pinneapple_physics.closed_form.pcb_thermal.PACKAGES`.
`parse_mission`	função	`(text) -> CoolingMission`	Deterministic reading of a cooling request (Portuguese or English): the numbers it states replace the defaults; `provenance` maps each field to the quoted words or to `"default"`.

`pinneapple_design.thermal.board_cooling.optimize`

Constrained search on the surrogate, verification on the reference, and the loop between them.

`pinneapple_design/thermal/board_cooling/optimize.py` · 141 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`constraint_violation`	função	`(mission, X, tj_upper, fan_w) -> np.ndarray`	Sum of the positive parts of every constraint, each scaled to a comparable size (1 = 1 K, 10 % of the fan budget, 0.1 mm of fin gap). `tj_upper`: (n, n_parts) pessimistic junction temperatures to hold.
`optimize`	função	`(mission, sur, *, k_sigma=2.0, seed=0, maxiter=120, popsize=24) -> dict[str, Any]`	Minimum-mass design on the surrogate. Returns the design, its prediction and the per-generation history of the best member (mass, predicted pessimistic hot-part temperature, violation).
`design_loop`	função	`(mission, data, *, rounds=5, k_sigma=2.0, seed=0, log=None) -> dict[str, Any]`	Learn -> optimise -> verify on the reference -> add the verified point, until confirmed. `data`: the output of :func:`.simulate.simulate_many` (extended in place with the verified points).

`pinneapple_design.thermal.board_cooling.report`

Deliverables of a cooling study: figures, 3-D scene and viewer, Blender renders, the report and its trust report.

`pinneapple_design/thermal/board_cooling/report.py` · 377 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`figure_maps`	função	`(state, path) -> str`	
`figure_evolution`	função	`(state, path) -> str`	Maximum (hot-part junction) temperature through the study: baseline, still air, the optimiser's best member per generation (surrogate, pessimistic) and each reference-verified optimum.

nome	tipo	assinatura	o que faz
`figure_holdout`	função	`(state, path) -> str`	Surrogate vs reference on the held-out designs, and the surrogate's map error on the verified design.
`impact_gif`	função	`(state, path, hold=6) -> str`	2-D animation of the board map from the bare board to the verified design (one frame per stage, held).
`render_3d`	função	`(state, out_dir, *, render=True, samples=24, size=(1280, 720)) -> dict[str, Any]`	glTF / USD of the verified design, the self-contained timeline viewer and, with Blender, stills of the bare and the verified board and the animation of the stages (GIF + MP4).
`write_report`	função	`(state, out_dir, files, extra) -> str`	
`deliver`	função	`(state, out_dir, *, render=True, samples=24) -> dict[str, Any]`	

pinneapple_design.thermal.board_cooling.scene

3-D scene of the cooled board: enclosure, fans, board temperature map, parts, heat sink and the warm air.

`pinneapple_design/thermal/board_cooling/scene.py` · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`fan_positions`	função	`(mission) -> tuple[list[tuple[float, float, float]], float]`	Centres (x, y, z) of the three fans on the inlet wall and their radius [mm].
`heat_sink_mesh`	função	`(mission, design, n_max, field3d=None)`	Vertices, faces and per-vertex temperature of the heat sink with `n_max` fin slots (unused fins collapsed to a point on the base). `design` None: everything collapsed (no heat sink).
`air_paths`	função	`(mission, result, n_x=18, heights=(4.0, 12.0, 22.0))`	Air paths along +y and the bulk temperature of each strip (energy balance), for a forced-air result.
`board_scene`	função	`(mission, result, *, title='', paths=True)`	One state as a static scene: field `T` (°C) on the board top, the parts and the heat sink.
`timeline_scene`	função	`(mission, states, *, title='', t_range=T_RANGE)`	The states (label, result) as frames of one animation: board map, part temperatures and heat sink shape and temperature change per frame; the HUD carries the hot part's junction temperature (nominal and pessimistic), the heat-sink mass and the fan air power. The air paths of the last state are …

pinneapple_design.thermal.board_cooling.simulate

Reference simulation of a cooled board: the 3D board model and the heat sink model, coupled.

`pinneapple_design/thermal/board_cooling/simulate.py` · 183 linhas · testes: 10

nome	tipo	assinatura	o que faz
map_axes	função	(board) -> tuple[np.ndarray, np.ndarray]	(y, x) cell centres of MAP_SHAPE in mm.
sink_mass_and_air	função	(mission, d) -> dict[str, float]	Closed-form heat-sink quantities that do not need the coupled solve: mass, pressure drop and air power (pinneapple_physics.closed_form.plate_fin_heatsink).
simulate	função	(mission, design, *, fidelity='train', pessimistic=True, tol=0.0001, max_iter=12) -> dict…	Junction temperatures, board-top map, heat paths and checks of design (None: the bare board in still air, the starting point). fidelity: a key of FIDELITY or {"n_cells", "hs_grid"}.
response	função	(mission, result) -> np.ndarray	The scalar responses a surrogate learns, in a fixed order: junction temperatures (nominal, then pessimistic) of mission.components.
simulate_many	função	(mission, X, *, fidelity='train', workers=4) -> dict[str, np.ndarray]	Reference solves of many designs in parallel: responses (n, 2 n_parts), board-top maps (n, ny, nx), wall seconds per solve, board energy-balance errors and coupling convergence flags.

pinneapple_design.thermal.board_cooling.verify

Independent verification of the chosen design and its trust report.

pinneapple_design/thermal/board_cooling/verify.py · 109 linhas · testes: 10

nome	tipo	assinatura	o que faz
verify	função	(mission, design, *, coarse=None, prediction=None, holdout=None, train_X=None, test_X=Non…)	Fine solve of design and the trust report. coarse: its solve on the training grid; prediction: the surrogate's {"tj", "tj_std", "map"} for it; holdout: :func:.learn.holdout output; train_X / test_X: the designs the surrogate was fitted on and the held-out ones its …

pinneapple_design.thermal.fin_array_speed

How much faster is a trained surrogate than a numerical solve, and how far apart are they? A measured sweep over seven plate-fin heat sinks.

pinneapple_design/thermal/fin_array_speed.py · 160 linhas · testes: 3

nome	tipo	assinatura	o que faz
design_by_name	função	(name) -> dict[str, Any]	One of CURATED_DESIGNS (KeyError otherwise).
design_of	função	(spec) -> hf.HeatsinkDesign	
train_surrogate	função	(n_samples=6000, epochs=1500, seed=0)	heatsink_fins.train_surrogate; returns (model, val_mean_rel_err) (held-out error on the thermal resistance, i.e. on the temperature rise).
time_surrogate	função	(model, d, power_w, h_conv, reps=200) -> dict[str, float]	Mean wall time of one forward pass (reps calls after a warm-up call) and the predicted temperature.
reference_solve	função	(d, power_w, h_conv, refine=True) -> dict[str, Any]	Finite-volume fin pitch (pinneapple_design.thermal.fin_cell), timed end to end.

nome	tipo	assinatura	o que faz
measure_design	função	(model, spec, timing_reps=200, refine=True) -> dict[str, Any]	Closed form, timed surrogate and timed reference for one design.
summarize	função	(designs) -> dict[str, Any]	
run_sweep	função	(model, progress_cb=None, timing_reps=200, refine=True) -> dict[str, Any]	Measure every design of CURATED_DESIGNS; progress_cb(i, n) after each.
thermal_field	função	(name='baseline') -> dict[str, Any]	heatsink_fins.thermal_field of one design, plus its base temperature.

pinneapple_design.thermal.fin_cell

Finite-volume conduction in one fin pitch of a straight-fin heat sink: an independent check of the 1-D fin-array formula (:mod:pinneapple_physics.closed_form.fin_array_conduction).

pinneapple_design/thermal/fin_cell.py · 147 linhas · testes: 16

nome	tipo	assinatura	o que faz
FinCellResult	classe	(t_junction_c, t_fin_root_c, t_fin_tip_c, energy_balance_rel_error, n_cells, grid_change_…)	
solve_fin_cell	função	(*, n_fins, fin_thickness_m, fin_height_m, fin_spacing_m, depth_m, base_thickness_m, k_w_…)	Steady conduction in one fin pitch (module docstring). power_w is the whole array's power.

pinneapple_design.thermal.heatsink

Plate-fin heat-sink sizing (the engine of the HeatSink Sizer app).

pinneapple_design/thermal/heatsink/__init__.py · 18 linhas · **sem teste direto**

pinneapple_design.thermal.heatsink.engine

Physics engine of the HeatSink Sizer: evaluates one design, fully verified.

pinneapple_design/thermal/heatsink/engine.py · 318 linhas · testes: 46

nome	tipo	assinatura	o que faz
DesignInput	classe	(base_width_mm, base_depth_mm, base_thickness_mm, n_fins, fin_thickness_mm, fin_height_mm…)	
OperatingInput	classe	(power_w, t_ambient_c, air_velocity_m_s, source_width_mm, source_depth_mm, source_x_mm, s…)	
base_theta	função	(g, op, r_conv, n=40) -> Dict[str, Any]	

nome	tipo	assinatura	o que faz
physics_resistance	função	(design, op, n=36) -> Dict[str, float]	Base-to-ambient resistance (no TIM) from the FVM-resolved hotspot, plus pressure drop -- the quantity the surrogate is trained to reproduce.
field_3d	função	(g, op, fvm, nominal, t_amb, max_cells=64, n_z=14) -> Dict[str, Any]	Surface temperatures of the whole heat sink for the 3D viewer.
model_scope	função	(mode) -> Dict[str, Any]	What the model covers, which way each simplification errs, what to do about it today and what is planned -- shown in every report.
evaluate_design	função	(design, op, *, grid=48, map_size=40) -> Dict[str, Any]	

pinneapple_design.thermal.heatsink.field

3D finite-volume conduction solve of the heat-sink base plate.

pinneapple_design/thermal/heatsink/field.py · 111 linhas · testes: 18

nome	tipo	assinatura	o que faz
solve_base_field	função	(*, base_width, base_depth, base_thickness, k, power_w, r_fins_k_w, source_width, source_…)	

pinneapple_design.thermal.heatsink.monotone

Heat-sink surrogate trained with monotonicity constraints (zoo model heatsink_natural_mlp).

pinneapple_design/thermal/heatsink/monotone.py · 163 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
random_designs	função	(mode, n, rng) -> np.ndarray	Feature rows of random in-range designs, sampled log-uniformly like the dataset.
split_indices	função	(n) -> dict[str, np.ndarray]	The app's split: numpy default_rng(0).permutation(n): test 15 %, cal 10 %, val 10 %, train the rest.
train_monotone	função	(mode='natural', *, seed=0, epochs=600, lbfgs_rounds=4, lbfgs_steps=200, n_random=20000, …)	Train the constrained surrogate on the app's dataset; returns the bundle that torch.save stores (state, normalisation, split-conformal quantiles of log R, history, summary metrics).

pinneapple_design.thermal.heatsink.sizer

Sizing: requirements in, verified heat-sink designs out.

pinneapple_design/thermal/heatsink/sizer.py · 251 linhas · testes: 18

nome	tipo	assinatura	o que faz
SizingRequest	classe	(power_w, t_limit_c, t_ambient_c, air_velocity_m_s, source_width_mm, source_depth_mm, tim…)	
size	função	(req, surrogates) -> Dict[str, Any]	

pinneapple_design.thermal.heatsink.surrogate

Neural surrogate of the physics engine, used only to *screen* designs.

`pinneapple_design/thermal/heatsink/surrogate.py` · 334 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Surrogate	classe	`(mode, model, x_mean, x_std, y_mean, y_std, conformal_log_q, report)`	
n_fins_for	função	`(width_mm, gap_mm, t_mm)`	
features	função	`(mode, W, D, tb, N, t, H, k, sw, sd, Ta, Q, V=None) -> np.ndarray`	Vectorised feature map (all arguments broadcastable arrays).
reference_solve	função	`(params, mode='forced') -> dict | None`	The physics engine on one design given in the surrogate's dataset columns (`base_width_mm` ... `fin_gap_mm` or `n_fins`, `k_w_mk`, `source_frac_w/_d` or `source_width_mm/_depth_mm`, `t_ambient_c`, `air_velocity_m_s`, `power_w`): the high-fidelity answer the zoo surrogates ...
reference_solve_batch	função	`(X, names, output='r_base_k_w', mode='forced') -> np.ndarray`	:func:`reference_solve` on every row of `X` (columns named by `names`); NaN where the engine rejects the design.
generate_dataset	função	`(mode, n, seed=0, workers=4) -> List[Dict[str, float]]`	
rows_to_xy	função	`(mode, rows) -> Tuple[np.ndarray, np.ndarray]`	
train	função	`(mode, rows, seed=0, epochs=400) -> Surrogate`	
load_surrogates	função	`() -> Dict[str, Surrogate]`	
main	função	`(n_forced=6000, n_natural=5000, workers=4, reuse=False) -> None`	

pinneapple_design.thermal.heatsink.trust

Trust report for the heat-sink surrogates.

`pinneapple_design/thermal/heatsink/trust.py` · 202 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
surrogate_report	função	`(sur, *, X, Y, split, history, train_seconds, n_rows) -> Dict`	

pinneapple_design.topology

Density-based topology optimization: SIMP, density filter, Heaviside projection, OC and MMA.

`pinneapple_design/topology/__init__.py` · 19 linhas · testes: 9

pinneapple_design.topology.driver

Optimization loop: design variables -> (symmetry) -> density filter -> Heaviside projection -> problem.

`pinneapple_design/topology/driver.py` · 127 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DesignChain`	classe	`(n_el, filt=None, symmetry=None, beta=0.0, eta=0.5)`	`x_phys = H_beta(W (S x))` and its adjoint (chain rule).
`TopOptResult`	classe	`(x, x_phys, objective, history)`	
`optimize`	função	`(problem, volfrac, filt=None, optimizer='oc', max_iter=200, tol=0.01, x0=None, symmetry=N…)`	Run density-based topology optimization of `problem` (see module docstring).
`finite_difference_check`	função	`(fun, x, idx, h=1e-06) -> np.ndarray`	Relative errors `|g_adj - g_fd| / max(|g_fd|, tiny)` of the gradient of `fun` (returning `(value, gradient)`) at the entries `idx`, central differences with step `h`.

pinneapple_design.topology.filters

Density filter and Heaviside projection for density-based topology optimization.

`pinneapple_design/topology/filters.py` · 115 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DensityFilter`	classe	`(centroids, r_min, volumes=None)`	Linear density filter `x_tilde = W x` with `W = diag(1/(H v)) H diag(v)`.
`SymmetryMap`	classe	`(S)`	Element densities from fewer design variables: `x_elem = S x_var` (a 0/1 sparse matrix).
`heaviside`	função	`(x, beta, eta=0.5) -> np.ndarray`	
`heaviside_derivative`	função	`(x, beta, eta=0.5) -> np.ndarray`	

pinneapple_design.topology.magnetic

Topology optimization of rotor iron for torque, on the 2D A_z machine model.

`pinneapple_design/topology/magnetic.py` · 79 linhas · testes: 9

nome	tipo	assinatura	o que faz
`RotorTorqueProblem`	classe	`(machine, thetas, I_peak, gamma, objective='mean', T_ref=1.0)`	

`pinneapple_design.topology.optimizers`

Design-update rules: optimality criteria (OC) and the method of moving asymptotes (MMA).

`pinneapple_design/topology/optimizers.py` · 117 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MMA`	classe	`(n, m, xmin=0.0, xmax=1.0, move=0.5, asyinit=0.5, asyincr=1.2, asydecr=0.7, c=1000.0, d=1…)`	Method of moving asymptotes for `min f0(x) s.t. f_i(x) <= 0, xmin <= x <= xmax`.
`oc_update`	função	`(x, dfdx, dvdx, volume_target, volume_of, move=0.2, eta=0.5, xmin=0.0, xmax=1.0)`	One OC step: `x_new = clip(x (max(-df/dx, 0) / (lambda dv/dx))^eta)` with `volume_of(x_new) = volume_target` found by bisection on `lambda`. `volume_of` maps design variables to the volume.

`pinneapple_design.topology.problems`

Density-based topology-optimization problems on structured grids of square bilinear (Q4) elements.

`pinneapple_design/topology/problems.py` · 264 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Q4Grid`	classe	`(nelx, nely, h=1.0)`	
`Compliance2D`	classe	`(nelx, nely, fixed_dofs, loads, penal=3.0, E0=1.0, Emin=1e-09, nu=0.3)`	
`ThermalCompliance2D`	classe	`(nelx=60, nely=60, sink_fraction=0.1, q=1.0, penal=3.0, kmin=0.001)`	Square plate, uniform heat source `q` per unit area, `T = 0` on the middle `sink_fraction` of the left edge, insulated elsewhere.
`StokesDissipation2D`	classe	`(nelx, nely, inlets, outlets, mu=1.0, alpha_max=None, alpha_min=None, q=0.1, penalty=1000…)`	Velocity-driven Stokes flow with Brinkman penalization on a `Lx x 1` box.
`elasticity_ke`	função	`(nu=0.3, E=1.0, h=1.0, thickness=1.0) -> np.ndarray`	8x8 plane-stress stiffness of a square Q4 element, dofs (ux, uy) per node.
`conduction_ke`	função	`(h=1.0) -> np.ndarray`	4x4 conduction matrix (unit conductivity) of a square Q4 element.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 110 de 196.
- Funções de teste em arquivos que importam algum módulo do pacote: 744.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
S39	Density-based topology optimisation (SIMP, filter, projection, OC, MMA; compliance, thermal, Stokes-Brinkman, machine rotor)	validated	1	2

Limitações conhecidas

- `aero.case`, `aero.case3d`, `aero.dataset` e `aero.verify` precisam do OpenFOAM instalado para rodar; `render_blender` precisa do `bpy` (Blender).
- Os modelos de aeronave são de ordem reduzida (polar elíptica, build-up de Raymer, vortex lattice com esteira plana sem roll-up); o docstring de `vlm.streamlines` avisa que a esteira não mostra o enrolamento.
- Os surrogates de perfil valem só dentro da caixa de treino dos pesos CST (`BOUNDS`); `out_of_bounds` existe para sinalizar isso.
- Vários módulos de `aero` não têm teste direto: `airliner`, `airliner3d`, `buildup`, `case3d`, `optimize_airliner`, `render_blender`, `verify`.
- Dependências opcionais mudam o que roda: `pymoo-nsga2` exige pymoo, o otimizador evolutivo usa `cma` se houver, o bayesiano e o `GPSurrogate` usam scikit-learn; métodos impossíveis voltam com status `skipped`.
- `Evaluator.torch` e os avaliadores não diferenciáveis levantam `NotImplementedError` (métodos por gradiente só funcionam com avaliadores diferenciáveis, como a fidelidade surrogate do dissipador).
- Os objetivos de `objective.py` (arrasto, eficiência térmica, tensão, peso) são proxies sobre o campo previsto, não grandezas físicas exatas.
- O preset de perfil não traz solver de referência: os candidatos ficam como 'só surrogate' até serem verificados com OpenFOAM fora do pacote; o modelo de custo do dissipador é uma suposição do usuário, não dado de mercado.
- `meshops.union` exige `manifold3d`; `exporters.step_check` exige `cadquery` (OpenCASCADE); o carregamento do Hub depende de rede (há modo offline e reconstrução pelo código).
- As dimensões das entradas vêm de dados publicados, mas algumas são estimativas: `Source(estimate=True)` marca valores escolhidos por julgamento.
- O STEP gerado é um B-rep facetado (triângulos planos), não superfícies NURBS exatas.
- Os módulos de `entries` e o renderizador por software não têm teste direto.

2. `pinneapple_systems`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
182	37.262	301	515	579 (40 arquivos)	78

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`component_library`		7	15	36
`component_modeling`		8	12	165
`cosimulation`		10	23	151
`digital_twin`		33	129	172
`excitation`		1	12	6
`operations`		10	73	25
`process_components`		17	112	171
`sports`		12	87	16
`time_series`		83	351	151

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_systems`

pinneapple_systems — Multi-component physical systems (time series, co-simulation, digital twins).

`pinneapple_systems/__init__.py` · 280 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`cosim_from_models`	função	`(models_dict, connections) -> 'CoSimGraph'`	Build a CoSimGraph from a dict of named nn.Module instances.
`forecast`	função	`(data, horizon, *, model='auto', **kwargs)`	Quick time-series forecast with auto model selection.

`pinneapple_systems.component_library`

pinneapple_systems.component_library — the piece `pinneapple_systems.component_modeling`'s own docstring says pinneapple deliberately did NOT build: a registry of NAMED PLANT COMPONENTS (keyed by `component_type`, e.g. "pipe"/"pump"/"valve"), each pairing an existing ...

`pinneapple_systems/component_library/__init__.py` · 44 linhas · testes: 36

`pinneapple_systems.component_library.base`

pinneapple_systems.component_library.base — `ComponentModel`: a `BaseModel` that additionally knows its physical constraint (a `Physics`) and offers single-call `.fit()`/`.evaluate()` convenience, closing the two real gaps found against a plain `BaseModel`/`PINNBase`:

`pinneapple_systems/component_library/base.py` · 173 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ComponentModel`	classe	`(*, architecture, architecture_kwargs=None, physics=None)`	A named plant component: an existing architecture + an optional `Physics` constraint.

`pinneapple_systems.component_library.benchmark`

pinneapple_systems.component_library.benchmark — train every architecture registered under one `component_type` and rank by final validation loss (real numbers from `ComponentModel.fit()`, not a display placeholder).

`pinneapple_systems/component_library/benchmark.py` · 77 linhas · **sem teste direto** · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`ComponentBenchmarkResult`	classe	`(component_type, ranked)`	
`benchmark_component_type`	função	`(component_type, train_loader, val_loader=None, *, epochs=50, lr=0.001, eval_coords=None,…)`	Trains every architecture registered for `component_type` on the same data and ranks them by final validation loss (ascending — lower is better). If `eval_coords`/`eval_targets` are given, also attaches RMSE/MAPE/R^2 per architecture via `ComponentModel.evaluate()`.

`pinneapple_systems.component_library.components`

pinneapple_systems.component_library.components — six reference plant components. Deliberately six, not padded to ten: each pairs a genuinely different (physics shortcut, backbone architecture, component_type) combination rather than near-duplicating the others.

`pinneapple_systems/component_library/components.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PipeVanillaPINN`	classe	`(in_dim=3, out_dim=3, hidden=(128, 128, 128, 128), **kwargs)`	
`HeatExchangerVanillaPINN`	classe	`(in_dim=2, out_dim=1, hidden=(64, 64, 64), **kwargs)`	
`TankShellVanillaPINN`	classe	`(in_dim=2, out_dim=2, hidden=(64, 64, 64), **kwargs)`	
`BatteryCellVanillaPINN`	classe	`(in_dim=2, out_dim=1, hidden=(64, 64, 64), **kwargs)`	
`PumpModifiedMLP`	classe	`(in_dim=3, out_dim=3, hidden_dim=128, n_layers=6, **kwargs)`	
`ValveModifiedMLP`	classe	`(in_dim=2, out_dim=1, hidden_dim=64, n_layers=3, **kwargs)`	

`pinneapple_systems.component_library.physics`

pinneapple_systems.component_library.physics — `Physics`: the thin attachment layer between a named plant component and the constraint it must satisfy.

`pinneapple_systems/component_library/physics.py` · 88 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Physics	classe	(name, loss_fn, params)	A named physical constraint, reduced to a single Trainer-shaped loss_fn. params is kept alongside for introspection (e.g. shown on a Toolbox/Component detail view) even though it is already baked into loss_fn via closure.

pinneapple_systems.component_library.registry

pinneapple_systems.component_library.registry — ComponentRegistry: a registry of NAMED PLANT COMPONENTS keyed by component_type (e.g. "pipe", "pump", "valve") — several competing architectures may register under the same component_type, mirroring how ...

pinneapple_systems/component_library/registry.py · 122 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ComponentSpec	classe	(component_type, name, cls, physics, description, tags)	
ComponentRegistry	classe	(_REGISTRY)	Central registry for named plant components.
register_component	função	(*, component_type, name, physics=None, description='', tags=None)	Module-level alias for ComponentRegistry.register — reads better as a bare decorator: @register_component(component_type=..., name=...).

pinneapple_systems.component_library.toolbox

pinneapple_systems.component_library.toolbox — Toolbox: a named, versioned, installable bundle of component_types (e.g. "Fluid Mechanics Toolbox v1.0" bundling pipe+pump+valve).

pinneapple_systems/component_library/toolbox.py · 110 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Toolbox	classe	(name, version, component_types, description, depends_on)	
ToolboxRegistry	classe	(_REGISTRY, _INSTALLED)	Registry of :class:Toolbox bundles, plus a tiny installed-set.

pinneapple_systems.component_modeling

pinneapple_systems.component_modeling — control, optimization, and uncertainty-quantification tooling that operates on ANY differentiable component model, independent of what that component represents physically.

pinneapple_systems/component_modeling/__init__.py · 60 linhas · **sem teste direto**

pinneapple_systems.component_modeling.bayesian

pinneapple_systems.component_modeling.bayesian — SWAGApproximation: a real, architecture-agnostic approximate Bayesian posterior over a model's weights, via SWAG (Stochastic Weight Averaging Gaussian — Maddox et al. 2019, "A Simple Baseline for Bayesian Uncertainty in Deep Learning").

pinneapple_systems/component_modeling/bayesian.py · 127 linhas · testes: 151

nome	tipo	assinatura	o que faz
SWAGApproximation	classe	(model)	

pinneapple_systems.component_modeling.control

pinneapple_systems.component_modeling.control — PID feedback control for an arbitrary differentiable or black-box component.

pinneapple_systems/component_modeling/control.py · 113 linhas · testes: 165

nome	tipo	assinatura	o que faz
PIDController	classe	(kp, ki, kd, setpoint, output_limits=None, bias=0.0)	Standard discrete-time PID: u = bias + Kp*e + Ki*integral(e) + Kd*de/dt, e = setpoint - measurement.
run_closed_loop	função	(plant, *, setpoint, kp=1.0, ki=0.1, kd=0.0, steps=50, dt=0.1, process_noise=0.0, measure…)	Closes a discrete-time control loop around plant.

pinneapple_systems.component_modeling.edge

pinneapple_systems.component_modeling.edge — package a trained model as a self-contained edge-deployment bundle (ONNX graph + manifest.json zipped together), and a minimal runtime that can execute that bundle with onnxruntime + numpy alone (no torch/pinneapple import at all).

pinneapple_systems/component_modeling/edge.py · 96 linhas · testes: 151

nome	tipo	assinatura	o que faz
EdgeRuntime	classe	(zip_path)	Minimal edge-device inference runtime — onnxruntime + numpy only, no torch/pinneapple import at all. Loads an export_edge_package() zip and runs real ONNX inference.
export_edge_package	função	(model, output_dir, *, name, sample_input, opset_version=17, extra_manifest=None) -> Dict…)	Exports model to ONNX and zips it with a manifest.json into a self-contained edge deployment package. Returns {"manifest": {…}, "onnx_path": …, "zip_path": …}.

pinneapple_systems.component_modeling.ensemble

pinneapple_systems.component_modeling.ensemble — DeepEnsemble: N independently-initialized copies of the same architecture, trained separately, combined at inference time.

pinneapple_systems/component_modeling/ensemble.py · 84 linhas · testes: 151

nome	tipo	assinatura	o que faz
DeepEnsemble	classe	(model_factory, n_members=5, base_seed=0)	

pinneapple_systems.component_modeling.mc_dropout

pinneapple_systems.component_modeling.mc_dropout — MC-Dropout epistemic uncertainty: n_samples stochastic forward passes with dropout kept active, reduced to a (mean, std) estimate.

`pinneapple_systems/component_modeling/mc_dropout.py` · 41 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mc_dropout_uncertainty`	função	`(model, x, n_samples=50) -> Tuple[torch.Tensor, torch.Tensor]`	Returns (mean, std) over `n_samples` stochastic forward passes with dropout active. Raises ValueError if `model` has no `nn.Dropout` submodule (nothing would actually be stochastic).

`pinneapple_systems.component_modeling.mpc`

pinneapple_systems.component_modeling.mpc — Model Predictive Control (receding horizon) around an arbitrary differentiable component.

`pinneapple_systems/component_modeling/mpc.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_mpc`	função	`(plant, *, in_dim, setpoint, control_input_index=0, observed_output_index=0, horizon=10, …)`	Receding-horizon MPC: at each of `steps` outer steps, optimize a length-`horizon` control sequence (Adam, `opt_iters` iterations, gradients flowing through `plant` itself) minimizing::

`pinneapple_systems.component_modeling.physics_residuals`

pinneapple_systems.component_modeling.physics_residuals — generic, closed-form PDE residuals for a handful of standard physical constraints, computed via autograd against any differentiable model's coordinate -> field(s) map.

`pinneapple_systems/component_modeling/physics_residuals.py` · 126 linhas · testes: 8 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`incompressible_continuity_residual`	função	`(model, coords) -> torch.Tensor`	Mass-conservation (div(u) = 0) residual for a velocity-field output. Assumes the model's first `dim` output columns are velocity components aligned with the first `dim` coordinate axes (u, v, [w]).
`heat_conduction_residual`	função	`(model, coords, *, k=50.0, Q=0.0) -> torch.Tensor`	Steady-state heat diffusion residual: k*laplacian(T) + Q, T = model's first output column, laplacian summed over every axis of `coords` (works for any spatial dimension).
`linear_elasticity_residual`	função	`(model, coords, *, E=210000000000.0, nu=0.3) -> torch.Tensor`	Navier-Lame equilibrium (isotropic linear elasticity, no body force): (lambda+mu)*grad(div(u)) + mu*laplacian(u) = 0. Assumes the model's first `dim` output columns are displacement components [u, v, (w)]. Works in 2D or 3D.
`species_diffusion_residual`	função	`(model, coords, *, D=1e-05, k=0.0) -> torch.Tensor`	Simplified steady-state diffusion-reaction (Fick's second law at steady state, first-order decay): D*laplacian(C) - k*C = 0. Assumes the model's first output column is concentration. A generic Fickian diffusion-reaction constraint — not tied to any specific transport system.

`pinneapple_systems.cosimulation`

PINNeAPPle native co-simulation engine.

pinneapple_systems/cosimulation/__init__.py · 86 linhas · **sem teste direto**

pinneapple_systems.cosimulation.adapters

Adapters connecting pinneapple_cosim to PINNeAPPle's model and problem ecosystems.

pinneapple_systems/cosimulation/adapters.py · 432 linhas · testes: 151

nome	tipo	assinatura	o que faz
PINNeProblemNode	classe	(name, model, spec, compiled_fn, coord_ports, field_ports, extra_input_ports=None, physic…)	Co-sim node backed by a PINNeAPPle ProblemSpec + compiled physics loss.
PINNeAPPleModelNode	classe	(name, model, input_ports, output_ports, batch_port_map=None) -> None	Wraps any BaseModel from ModelRegistry as a co-sim node.
SymbolicPDENode	classe	(name, model, symbolic_pde, coord_ports, field_ports, physics_weight=1.0) -> None	Co-sim node whose physics residual is defined by a SymPy expression.
TimeSeriesCoSimNode	classe	(name, model, input_len, horizon, extra_input_ports=None, extra_output_ports=None) -> None	Wraps a PINNeAPPle TSModelBase as a co-simulation node.

pinneapple_systems.cosimulation.connection

Connection dataclass for co-simulation graphs.

pinneapple_systems/cosimulation/connection.py · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Connection	classe	(src_node, src_port, dst_node, dst_port)	Immutable directed edge between two node ports.

pinneapple_systems.cosimulation.engine

Co-simulation engine: time-stepped execution with algebraic-loop iteration.

pinneapple_systems/cosimulation/engine.py · 257 linhas · testes: 151

nome	tipo	assinatura	o que faz
CoSimEngine	classe	(graph, recorder=None, loop_solver='gauss_seidel', max_iter=50, tol=1e-06, retain_graph=F…)	Time-stepped co-simulation engine over a CoSimGraph.

pinneapple_systems.cosimulation.graph

Co-simulation graph: node registry, connection management, topology analysis.

pinneapple_systems/cosimulation/graph.py · 187 linhas · testes: 151

nome	tipo	assinatura	o que faz
CoSimGraph	classe	() -> None	Registry of nodes and connections with lazy topology caching.

pinneapple_systems.cosimulation.losses

Composite loss functions for training differentiable co-simulation graphs.

`pinneapple_systems/cosimulation/losses.py` · 225 linhas · testes: 151

nome	tipo	assinatura	o que faz
`DataLoss`	classe	`(weight=1.0) -> None`	MSE between predicted port values and observations.
`PhysicsLoss`	classe	`(weight=1.0) -> None`	Sum of physics residuals from all PINNNodes in the graph.
`CouplingLoss`	classe	`(weight=0.1) -> None`	Penalise interface inconsistencies between connected ports.
`CoSimLoss`	classe	`(data_weight=1.0, physics_weight=1.0, coupling_weight=0.1) -> None`	Composite loss: data fidelity + physics residuals + coupling consistency.

`pinneapple_systems.cosimulation.node`

Co-simulation node abstractions.

`pinneapple_systems/cosimulation/node.py` · 291 linhas · testes: 151

nome	tipo	assinatura	o que faz
`CoSimNode`	classe	`(name, input_ports, output_ports) -> None`	Abstract base class for all co-simulation nodes.
`TorchNode`	classe	`(name, model, input_ports, output_ports) -> None`	Wraps a `nn.Module`; gradients flow through `step()`.
`AnalyticalNode`	classe	`(name, fn, input_ports, output_ports) -> None`	Wraps a pure Python/NumPy callable. No autograd — outputs are detached.
`PINNNode`	classe	`(name, model, input_ports, output_ports, physics_fn, physics_weight=1.0) -> None`	`TorchNode` extended with a physics-residual loss.
`BlackBoxNode`	classe	`(name, fn, input_ports, output_ports) -> None`	Opaque callable node — fully detached from autograd.
`split_output_ports`	função	`(out, ports) -> Dict[str, torch.Tensor]`	Split *out*'s last dimension evenly across *ports*.

`pinneapple_systems.cosimulation.recorder`

Trajectory recording for co-simulation runs.

`pinneapple_systems/cosimulation/recorder.py` · 177 linhas · testes: 151

nome	tipo	assinatura	o que faz
`Trajectory`	classe	`(node, port) -> None`	Time series for a single (node, port) signal.
`TrajectoryRecorder`	classe	`() -> None`	Records time-stamped port outputs from multiple nodes.

`pinneapple_systems.cosimulation.trainer`

CoSimTrainer — trains co-simulation graphs using PINNeAPPle's training stack.

`pinneapple_systems/cosimulation/trainer.py` · 331 linhas · testes: 151

nome	tipo	assinatura	o que faz
`CoSimTrainer`	classe	`(graph, engine, criterion=None, weight_scheduler_cfg=None, early_stopping=None, checkpoin…)`	Trains a differentiable co-simulation graph with PINNeAPPle's training stack.

pinneapple_systems.cosimulation.utils

Graph algorithms for co-simulation topology analysis.

pinneapple_systems/cosimulation/utils.py · 180 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
tarjan_scc	função	(adj) -> List[List[str]]	Find all Strongly Connected Components using Tarjan's algorithm.
condensation_order	função	(adj) -> Tuple[List[List[str]], List[List[str]]]	Compute execution order via SCC condensation.
topological_sort	função	(adj) -> List[str]	Topological sort for pure DAGs (no cycles).

pinneapple_systems.digital_twin

pinneapple_digital_twin — Digital twins built on top of pinneapple surrogate models.

pinneapple_systems/digital_twin/__init__.py · 121 linhas · testes: 15

pinneapple_systems.digital_twin.assimilation

Data assimilation methods for digital twins.

pinneapple_systems/digital_twin/assimilation/__init__.py · 5 linhas · **sem teste direto**

pinneapple_systems.digital_twin.assimilation.kalman

pinneapple_systems.digital_twin.assimilation.kalman compatibility shim -- the real ExtendedKalmanFilter / EnsembleKalmanFilter implementation moved to pinneapple_analysis.state_estimation.kalman (a capability audit found no general-purpose, standalone data-assimilation module usable ...

pinneapple_systems/digital_twin/assimilation/kalman.py · 20 linhas · testes: 157 · sinais no código: shim ou obsoleto

pinneapple_systems.digital_twin.conditioning

Causal, sample-by-sample telemetry conditioning for production digital twins.

pinneapple_systems/digital_twin/conditioning.py · 195 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
SignalProfile	classe	(kind, tau_s, hampel_window, hampel_k, max_consecutive_outliers, max_rate, valid_range, m…)	How to condition one kind of signal. Use a preset (PROFILES) or build your own.
ConditionedSample	classe	(t, raw, value, flags)	
StreamingConditioner	classe	(profile='generic', **overrides)	Condition one signal, one reading at a time (causal; O(window) per reading).
TelemetryConditioner	classe	(profiles=None, default='generic')	Per-sensor, per-field conditioners for a whole twin.

pinneapple_systems.digital_twin.io

I/O adapters for digital twin data streams.

pinneapple_systems/digital_twin/io/__init__.py · 26 linhas · testes: 8

pinneapple_systems.digital_twin.io.mqtt_broker

A small embedded MQTT 3.1.1 broker for demos, tests and single-asset edge deployments.

pinneapple_systems/digital_twin/io/mqtt_broker.py · 263 linhas · testes: 14

nome	tipo	assinatura	o que faz
MQTTBroker	classe	(host='127.0.0.1', port=1883, username=None, password=None, max_packet=256 * 1024)	Run with :meth:start (background thread with its own event loop) and :meth:stop.
topic_matches	função	(filt, topic) -> bool	MQTT topic-filter matching (+ one level, # the rest; $ topics need an explicit first level).

pinneapple_systems.digital_twin.io.sensors

Sensor management for digital twins.

pinneapple_systems/digital_twin/io/sensors.py · 151 linhas · testes: 151

nome	tipo	assinatura	o que faz
Sensor	classe	(sensor_id, coords, field_names, calibration_offset, calibration_scale, noise_std, transf…)	Represents a single physical sensor.
SensorRegistry	classe	() -> None	Registry of all sensors in the digital twin.

pinneapple_systems.digital_twin.io.stream

Real-time data stream adapters for digital twins.

pinneapple_systems/digital_twin/io/stream.py · 576 linhas · testes: 172

nome	tipo	assinatura	o que faz
BaseStream	classe	(sensor_id, field_names) -> None	Abstract base class for all data stream adapters.
FileWatchStream	classe	(path, sensor_id, field_names, *, poll_interval=1.0, format='jsonl') -> None	Polls a JSON-lines file (or CSV/Parquet) for new rows.
MQTTStream	classe	(broker, topic, sensor_id, field_names, *, port=1883, keepalive=60, username=None, passwo…)	Subscribes to an MQTT topic and emits Observations.
HTTPPollStream	classe	(url, sensor_id, field_names, *, poll_interval=5.0, headers=None, transform=None) -> None	Periodically polls a REST endpoint (GET) and emits Observations.
KafkaStream	classe	(bootstrap_servers, topic, sensor_id, field_names, *, group_id='pinneapple_dt', auto_offs…)	Reads messages from an Apache Kafka topic.
OPCUAStream	classe	(server_url, node_ids, sensor_id, field_names, *, poll_interval=1.0) -> None	Polls an OPC-UA server for real-time tag values.

nome	tipo	assinatura	o que faz
`ModbusStream`	classe	`(host, port, register_map, sensor_id, field_names, *, poll_interval=1.0, device_id=1) -> …)`	Polls a Modbus TCP server's holding registers for real-time values.
`MockStream`	classe	`(sensor_id, field_names, generator_fn, *, tick_interval=0.1, coords=None) -> None`	Synthetic data stream for testing.

`pinneapple_systems.digital_twin.io.usgs`

Client of the USGS Water Services "Instantaneous Values" API (public, no key): river discharge, gage height and other parameters of US stream gauges every 5-15 minutes.

`pinneapple_systems/digital_twin/io/usgs.py` · 100 linhas · testes: 8

nome	tipo	assinatura	o que faz
`USGSDataError`	classe	`()`	The USGS service could not be reached or returned no usable data.
`Series`	classe	`(site_id, site_name, parameter_cd, unit, timestamps, values, fetched_at, from_cache)`	
`monitoring_url`	função	`(site_id) -> str`	Public USGS page of a gauge, for a reader to check the data.
`parse_iv`	função	`(payload, site_id, parameter_cd) -> Series`	The first time series of an IV JSON payload, malformed readings skipped, sorted by time.
`fetch_iv`	função	`(site_id, parameter_cd=DISCHARGE, period='P5D', timeout_s=10.0, cache_ttl_s=90.0) -> Seri…)`	Download one parameter of one gauge over the trailing `period` (ISO 8601 duration, e.g. `"P5D"`).

`pinneapple_systems.digital_twin.monitoring`

Monitoring and anomaly detection for digital twins.

`pinneapple_systems/digital_twin/monitoring/__init__.py` · 17 linhas · **sem teste direto**

`pinneapple_systems.digital_twin.monitoring.anomaly`

Anomaly detection for digital twins.

`pinneapple_systems/digital_twin/monitoring/anomaly.py` · 415 linhas · testes: 166

nome	tipo	assinatura	o que faz
`AnomalyEvent`	classe	`(timestamp, sensor_id, field_name, observed, predicted, score, detector, metadata)`	Records a detected anomaly.
`ThresholdDetector`	classe	`(thresholds, rel_thresholds=None, eps=1e-08) -> None`	Simple threshold-based anomaly detector.
`ZScoreDetector`	classe	`(z_threshold=3.0, window_size=100, min_samples=10) -> None`	Rolling z-score anomaly detector.
`MahalanobisDetector`	classe	`(threshold=3.0, field_order=None) -> None`	Multivariate Mahalanobis distance anomaly detector.
`ResidualDetector`	classe	`(z_threshold=3.0, window_size=100, min_samples=10) -> None`	Checks the innovation residuals produced by the assimilation filter.

nome	tipo	assinatura	o que faz
`AnomalyMonitor`	classe	`() -> None`	Composite anomaly monitor that runs multiple detectors in sequence.

pinneapple_systems.digital_twin.pipe_loop

The digital twin that knows when it is wrong (Autonomous Physics Lab, demo 3): a pumped, heated water loop with an elbow and a T junction, a fast model of its sensors, and a diagnosis that tells a changed process from a wrong sensor from a model outside its domain -- with physics residuals, a ...

`pinneapple_systems/digital_twin/pipe_loop/__init__.py` · 31 linhas · testes: 7

pinneapple_systems.digital_twin.pipe_loop.agent

The digital twin that knows when it is wrong: a monitoring mission -> plan -> controlled tools -> evidence.

`pinneapple_systems/digital_twin/pipe_loop/agent.py` · 217 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`t_read_mission`	função	`(state, text) -> dict[str, Any]`	
`t_build_twin`	função	`(state, n_train=120, n_test=40, seed=0) -> dict[str, Any]`	
`t_calibrate`	função	`(state, n_runs=12) -> dict[str, Any]`	
`t_monitor`	função	`(state) -> dict[str, Any]`	
`t_request_simulation`	função	`(state, n_new=40, widen=0.25) -> dict[str, Any]`	For every showcase case diagnosed as outside the domain: run the reference at the operating point, compare it with the readings (does the plant still obey the physics?), extend the fast model's envelope by `widen` of the box beyond the input that left it, add `n_new` reference solves there, ...
`t_benchmark`	função	`(state, n_per_kind=20, n_calibration=12, workers=4) -> dict[str, Any]`	
`t_trust`	função	`(state) -> dict[str, Any]`	
`t_deliver`	função	`(state, out_dir='out', render=True, samples=16) -> dict[str, Any]`	

nome	tipo	assinatura	o que faz
`toolbox`	função	`() -> Toolbox`	
`template_plan`	função	`(text, *, out_dir='out', render=True, n_per_kind=20, workers=4) -> TemplatePlanner`	
`run_mission`	função	`(text=DEFAULT_TEXT, *, out_dir='out', planner='template', render=True, n_per_kind=20, wor…)`	

`pinneapple_systems.digital_twin.pipe_loop.benchmark`

Controlled test cases with a known answer, and the confusion matrix of the diagnosis.

`pinneapple_systems/digital_twin/pipe_loop/benchmark.py` · 123 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`make_cases`	função	`(n_per_kind=20, seed=0) -> list[Scenario]`	
`run_benchmark`	função	`(loop=None, *, n_per_kind=20, n_calibration=12, seed=0, n_train=120, workers=4, log=None)…)`	Fit the twin's fast model, calibrate the diagnoser on healthy runs, diagnose every controlled case and return the confusion matrix, isolation scores, false alarms, delays and timings.

`pinneapple_systems.digital_twin.pipe_loop.diagnose`

How the twin knows it is wrong: physics residuals, a fast model, a Kalman filter and its domain of validity.

`pinneapple_systems/digital_twin/pipe_loop/diagnose.py` · 313 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Residuals`	classe	`(loop)`	Physics residuals of one reading vector, in sensor units, and their noise standard deviation (propagated from SIGMA by finite differences).
`Surrogate`	classe	`(n_train, seed, model, lo, hi)`	POD + Gaussian-process model of the ten readings over NORMAL_BOX (the twin's fast model).
`LossTracker`	classe	`(loop, q_walk=0.02, k0_sigma=0.5)`	EKF on the extra loss coefficient of each pipe (random walk), observed through the measured pressure drop of each pipe given its measured flow.
`Diagnoser`	classe	`(loop, surrogate, window, thresholds, ood_threshold)`	Windowed evidence -> hypothesis. `window` steps; thresholds from :meth:`calibrate` (healthy runs).

pinneapple_systems.digital_twin.pipe_loop.plant

The plant: a pumped, heated water loop with an elbow and a T junction, and its reference simulator.

`pinneapple_systems/digital_twin/pipe_loop/plant.py` · 258 linhas · testes: 7

nome	tipo	assinatura	o que faz
`Pipe`	classe	`(length, diameter, roughness, k_minor, ua_w_k)`	
`Loop`	classe	`(pipe_A, pipe_B, pipe_C, h0_m, k_pump, kv0_B, kv0_C, z_B, z_C, t_room_c, p_atm_head_m, wa…)`	Geometry and machines of the loop (SI).
`Faults`	classe	`(k_extra, bias, noise_gain)`	What is wrong at one instant (all zero: healthy).
`Scenario`	classe	`(name, kind, seed, duration_s, dt_s, onset_s, ramp_s, target, magnitude, mode)`	A controlled test case: an operating history and (optionally) one fault with its onset and ramp.
`water`	função	`(t_c) -> tuple[float, float]`	(density kg/m3, dynamic viscosity Pa s) at `t_c` °C.
`steady_state`	função	`(loop, u, k_extra=None) -> dict[str, float]`	True plant state for inputs `u` (keys of INPUTS): flows, heads, pressures and temperatures. Water properties at the mean of the inlet and heater-outlet temperatures (two passes).
`readings`	função	`(state) -> np.ndarray`	True sensor values in sensor units (bar, m3/h, °C), order SENSORS.
`quasi_steady_valid`	função	`(loop, stroke_s, factor=10.0) -> bool`	The quasi-steady model holds when a valve moves over many acoustic round trips: stroke >= factor 2L/a.
`operating_history`	função	`(sc) -> tuple[np.ndarray, dict[str, np.ndarray]]`	Slow, smooth operator moves inside NORMAL_BOX (sum of a few low-frequency sines per input, seeded); a domain scenario then drives its `target` input to `magnitude` (outside the box) from the onset.
`run_plant`	função	`(loop, sc) -> dict[str, Any]`	Simulate one scenario: inputs, true sensor values, noisy readings and the fault applied at each step.
`with_loop`	função	`(loop, **kw) -> Loop`	

pinneapple_systems.digital_twin.pipe_loop.report

Deliverables of the pipe-loop twin: control-room figures, confusion matrix, trust report, 3-D scene and report.

`pinneapple_systems/digital_twin/pipe_loop/report.py` · 341 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`trust_report`	função	`(state) -> TrustReport`	

nome	tipo	assinatura	o que faz
`control_room`	função	`(case, path, *, title='', cursor=None) -> str`	
`figure_confusion`	função	`(b, path) -> str`	
`figure_request`	função	`(rs, path) -> str | None`	
`impact_gif`	função	`(case, path, n=14) -> str`	
`render_3d`	função	`(case, out_dir, *, render, samples, size=(1280, 720)) -> dict[str, Any]`	
`deliver`	função	`(state, out_dir, *, render=True, samples=16) -> dict[str, Any]`	
`write_report`	função	`(state, out_dir, files, env) -> str`	

pinneapple_systems.digital_twin.pipe_loop.scene

3-D control-room scene of the loop: pipes coloured by pressure, a transparent elbow with its velocity profile, the T junction, valves, tanks, the pump and the sensors coloured by the twin's verdict, frame by frame.

`pinneapple_systems/digital_twin/pipe_loop/scene.py` · 181 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`tube`	função	`(path, r=R_PIPE, n_theta=20, n_seg=40)`	Triangulated tube along a polyline (resampled to `n_seg` rings); returns vertices, faces and the arc-length fraction 0..1 of every vertex.
`elbow_streamlines`	função	`(v_bulk, n_r=3, n_t=8)`	Lines through the elbow and the straight runs next to it at a few radii, valued by the 1/7 power-law speed.
`loop_scene`	função	`(run, diagnosis, *, n_frames=24, title='')`	Animated scene of one monitored run: pressures on the walls, verdict on the sensors, HUD with the evidence.

pinneapple_systems.digital_twin.product

The closed digital-twin product: spec, commissioning, live run, evidence, drift policy.

`pinneapple_systems/digital_twin/product/__init__.py` · 83 linhas · testes: 24

pinneapple_systems.digital_twin.product.assimilation

The twin's estimator: an ensemble Kalman filter over the state the model cannot know.

`pinneapple_systems/digital_twin/product/assimilation.py` · 418 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TwinEstimate`	classe	`(theta, theta_std, innovation, n_obs)`	One analysis instant: the latent state, its spread, and the innovation.
`LatentStateTwin`	classe	`(model, spec, *, n_ens=64, inflation=1.02, seed=0) -> None`	Estimates the latent state of one asset from a spec and the model behind it.

pinneapple_systems.digital_twin.product.commission

Commissioning: the two gates a twin passes before it is allowed to run.

pinneapple_systems/digital_twin/product/commission.py · 588 linhas · testes: 24

nome	tipo	assinatura	o que faz
read_timestamp	função	(frame) -> float	The frame's timestamp, under any of the conventional keys.
read_readings	função	(frame, spec) -> dict[str, float]	The sensor values present in a frame, ignoring keys that are not sensors.
preflight	função	(spec, history) -> CheckReport	Check that this twin can be commissioned at all, before spending a backtest on it.
backtest	função	(spec, model, history, *, n_ens=64, inflation=1.02, seed=0, warmup_steps=0, calibrate_fra…)	Replay the history through the twin and measure it against the held-out sensors.

pinneapple_systems.digital_twin.product.evidence

The evidence a twin has to produce to be trusted, and the shapes it is filed in.

pinneapple_systems/digital_twin/product/evidence.py · 284 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CheckItem	classe	(check_id, status, title, detail, why, fix, source)	One line of a preflight checklist.
CheckReport	classe	(title, items)	An ordered checklist with a verdict: any FAIL blocks commissioning.
FieldMetrics	classe	(field_name, unit, n, rmse, mae, bias, p95_abs, coverage, span, window_s, coverage_raw, b…)	Agreement between the twin and reality, on observations it never assimilated.
TrackingReport	classe	(metrics, gates, breaches, alarm_lead_s, forecast_error_s, final_latent, n_steps)	Per-field tracking metrics plus the gate verdict and the alarm lead times.
TwinEvidence	classe	(asset_id, spec_digest, model_id, created_unix, preflight, tracking, decisions, unverifie…)	Everything a commissioned twin produces, in one file, keyed by the spec digest.

pinneapple_systems.digital_twin.product.model

What a model must accept to be run as a twin, and an adapter so existing surrogates fit.

pinneapple_systems/digital_twin/product/model.py · 155 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
TwinModelError	classe	()	The model does not honour the twin contract.
BiasModel	classe	(model, field_names) -> None	Wrap a plain model(X) so it satisfies the twin contract with an output bias.
as_array_dict	função	(out, field_names) -> dict[str, np.ndarray]	Normalise a model output to {field: (n,) float32}.
ensure_twin_model	função	(model, field_names, n_latent)	Return a callable that honours the contract, wrapping model when it does not.

pinneapple_systems.digital_twin.product.policy

What the twin does when it stops agreeing with the plant.

`pinneapple_systems/digital_twin/product/policy.py` · 166 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TwinMode`	classe	`()`	
`DriftPolicy`	classe	`(window, target_coverage, hard_break, max_sustained, recalibrate_after, hysteresis)`	Thresholds for the consistency ladder.
`DriftMonitor`	classe	`(policy, _window, _mode, _off_run, _alarm_run, _ok_run, decisions)`	Rolling consistency of the twin's band, converted into a mode with hysteresis.

pinneapple_systems.digital_twin.product.runner

The product surface: one call to commission a twin, one to run it, one to file the evidence.

`pinneapple_systems/digital_twin/product/runner.py` · 346 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RunResult`	classe	`(n_steps, n_observed, n_holdout, final_mode, coverage, latent, modes_seen, decisions)`	What happened while the twin ran.
`commission`	função	`(spec, model, history, *, n_ens=64, inflation=1.02, seed=0, warmup_steps=0, calibrate_fra…)`	Run both gates and return the evidence file.
`run_live`	função	`(spec, model, frames, *, n_ens=64, inflation=1.02, seed=0, policy=None, evidence=None, th…)`	Assimilate a run of frames, monitor consistency, and record what the twin decided.
`describe`	função	`(spec, latent, fields=None)`	Describe the twin for the industry standards the platform already speaks.
`export_scene`	função	`(spec, frames, folder, *, mesh=None, field_on_mesh=None, times=None) -> str`	Write a `pinneapple_twin3d` scene for this twin: sensors with measured series and envelopes.

pinneapple_systems.digital_twin.product.spec

The twin contract: what a digital twin is asked to deliver, signed before it runs.

`pinneapple_systems/digital_twin/product/spec.py` · 438 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SpecError`	classe	`()`	The spec is not usable: a missing claim, a contradiction, an unobservable field.
`FieldSpec`	classe	`(name, unit, kind, lo, hi, alarm_above, alarm_below)`	One quantity the twin estimates.
`SensorSpec`	classe	`(sensor_id, unit, field, lo, hi, at, max_gap_s, noise_sigma, holdout, driver)`	A measurement available to the twin, with the envelope that makes it checkable.

nome	tipo	assinatura	o que faz
`ModelSpec`	classe	`(model_id, version, accuracy_floor, validity)`	The model behind the twin, and the accuracy it claims.
`AcceptanceGates`	classe	`(max_rmse_frac, max_bias_frac, min_coverage, min_alarm_lead_s)`	The thresholds that decide whether a twin goes live.
`TwinSpec`	classe	`(asset_id, asset_name, asset_type, fields, sensors, model, asset_unit, coord_names, sampl…)`	A complete, checkable definition of one digital twin.

`pinneapple_systems.digital_twin.prognostics`

Predictive maintenance: trend-based Remaining Useful Life (RUL) estimation.

`pinneapple_systems/digital_twin/prognostics.py` · 628 linhas · testes: 15

nome	tipo	assinatura	o que faz
`HealthIndicatorHistory`	classe	`(timestamps, values, source)`	A scalar health/degradation-indicator time series.
`RULEstimate`	classe	`(estimated_rul, rul_confidence_interval, failure_time_estimate, degradation_model_used, f…)`	Result of a trend-based Remaining Useful Life estimation.
`estimate_rul`	função	`(health_indicator_series, timestamps=None, failure_threshold=None, model='linear', confid…)`	Trend-based Remaining Useful Life (RUL) estimation.
`estimate_rul_from_twin`	função	`(twin, failure_threshold, *, detector=None, sensor_id=None, field_name=None, n=None, mode…)`	Convenience wrapper: estimate RUL directly from a live `DigitalTwin`.

`pinneapple_systems.digital_twin.realtime_pipeline`

Real-time inference pipeline for digital twins.

`pinneapple_systems/digital_twin/realtime_pipeline.py` · 470 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LatencyBudgetExceeded`	classe	`()`	Raised when inference time exceeds data_period and policy='raise'.
`LatencyStats`	classe	`(n_frames, n_skipped, n_budget_violations, mean_inference_ms, max_inference_ms, budget_ms…)`	Rolling latency statistics.
`RealtimePipelineConfig`	classe	`(data_period, latency_policy, buffer_maxlen, latency_alpha, throttle_min_points, throttle…)`	Configuration for RealtimePipeline.
`RealtimePipeline`	classe	`(model, data_period, field_names, coord_names=None, *, config=None, on_state=None, on_lat…)`	Real-time inference pipeline wrapping a surrogate model.

`pinneapple_systems.digital_twin.recession`

A river digital twin from baseflow recession: calibrate on the recent past of a gauge, forecast the present, flag a sustained departure.

pinneapple_systems/digital_twin/recession.py · 195 linhas · testes: 8

nome	tipo	assinatura	o que faz
recession	função	(q0, k_per_hour, hours) -> float | np.ndarray	Q0 exp(-k t), t in hours since t0.
fit_recession	função	(hours, values) -> dict[str, float]	Least-squares Q0, k on the log-linearised recession; hours is re-based to its first sample (t0_hours).
forecast	função	(calibration, hours) -> np.ndarray	The calibrated curve at hours (same axis as the fit).
detect_divergence	função	(hours, actual, predicted, residual_mean, residual_std, n_sigma=N_SIGMA, sustained_minute…)	First sustained excursion of actual - predicted beyond residual_mean +- n_sigma * residual_std.
run_twin	função	(timestamps, values, *, stage_timestamps=None, stage_values=None, calibration_hours=CALIB…)	Calibrate, forecast and compare one discharge series (and, optionally, fit the gage height the same way for the water level now vs forecast). Raises ValueError when the series is too short to calibrate.
synthetic_recession	função	(hours=96.0, step_minutes=15.0, q0=2000.0, k_per_hour=0.004, noise=0.005, bump_at_hour=No…)	A synthetic gauge record for tests and offline demos: an exponential recession with multiplicative noise and an optional step increase bump (fraction) from bump_at_hour on. Timestamps start 2026-01-01 UTC.

pinneapple_systems.digital_twin.signal_reconstruction

Signal reconstruction from incoming data streams.

pinneapple_systems/digital_twin/signal_reconstruction.py · 587 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ReconstructionResult	classe	(sensor_id, field_name, method, timestamps, signal, raw_timestamps, raw_values, quality, …)	Output of a signal reconstruction pass.
SignalReconstructor	classe	(twin_or_observations, *, default_method='ssa', default_n_out=512) -> None	Convenience wrapper that reconstructs signals from the observation history held by a live DigitalTwin (or from any list of Observation objects).
reconstruct_spectral	função	(timestamps, values, *, n_harmonics=10, out_times=None, min_points=8) -> Tuple[np.ndarray…)	FFT-based reconstruction: fit the dominant harmonics and evaluate on a dense grid (or out_times if provided).
reconstruct_ssa	função	(timestamps, values, *, window=0, n_components=5, out_times=None) -> Tuple[np.ndarray, np…)	SSA-based reconstruction: decompose the signal and reconstruct from the dominant components (noise suppression).
reconstruct_wavelet	função	(timestamps, values, *, wavelet='db4', level=4, threshold_mode='soft', threshold_sigma_mu…)	Wavelet soft-thresholding denoising + reconstruction.
reconstruct_interpolate	função	(timestamps, values, *, kind='cubic', out_times=None, n_out=512) -> Tuple[np.ndarray, np.…)	Scipy interpolation-based reconstruction (gap filling / upsampling).

nome	tipo	assinatura	o que faz
`reconstruct_model_based`	função	`(timestamps, values, *, model, sensor_coord, coord_names, field_name, field_index=0, out_…)`	Use a trained physics model to reconstruct the time series at a given spatial location by querying model(x, y, t).

pinneapple_systems.digital_twin.standards

Digital-twin interoperability: describe a PINNeAPPle twin once and publish it in the open standards industry platforms read.

`pinneapple_systems/digital_twin/standards.py` · 322 linhas · testes: 31

nome	tipo	assinatura	o que faz
`TwinProperty`	classe	`(name, value, unit, kind, description, writable)`	
`TwinModel`	classe	`(name, family, status, compatibility, violated, probable_cause)`	A model behind the twin and its physical validity on the current operating point.
`TwinDescription`	classe	`(id, name, asset_type, description, manufacturer, properties, models, relationships)`	
`OpcUaTwinServer`	classe	`(desc, endpoint=…, namespace='urn:pinneapple:twins')`	Serve a twin over OPC UA (asyncua) in a background thread: one object per twin with a variable per property (and the validity status of each model). :meth:`update` writes new values; OPC UA clients read or subscribe.
`to_aas`	função	`(desc, as_json=True)`	Asset Administration Shell (metamodel v3) with submodels Identification, TechnicalData (parameters), OperationalData (telemetry and estimated state) and SimulationModels (models and physical validity), built with the Eclipse BaSyx Python SDK. Returns the JSON text (`as_json`) or the BaSyx object …
`to_dtdl`	função	`(desc, version=1) -> dict[str, Any]`	A DTDL v3 Interface: parameters and estimated state as Properties, measurements as Telemetry, model validity as Properties, links to other twins as Relationships.
`to_ngsi_ld`	função	`(desc, context=…, observed_at=None) -> dict[str, Any]`	An NGSI-LD entity: every property as a `Property` (unit as `unitText`, measurements with `observedAt`), model validity as properties, links as `Relationship`s.
`to_ditto_thing`	função	`(desc, namespace='org.pinneapple') -> dict[str, Any]`	An Eclipse Ditto Thing: identification and parameters as attributes; telemetry, state and model validity as features.

pinneapple_systems.digital_twin.state

System state representation for digital twins.

`pinneapple_systems/digital_twin/state.py` · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Observation`	classe	`(timestamp, sensor_id, values, coords, metadata)`	A single raw observation from a sensor stream.
`SystemState`	classe	`(timestamp, fields, coords, covariance, residuals, metadata, history)`	Current estimate of the physical system state.

`pinneapple_systems.digital_twin.twin`

Core DigitalTwin class.

`pinneapple_systems/digital_twin/twin.py` · 650 linhas · testes: 151

nome	tipo	assinatura	o que faz
`DigitalTwinConfig`	classe	`(device, batch_size, use_amp, update_interval, obs_queue_maxsize, max_history, assimilati…)`	Configuration for the DigitalTwin.
`DigitalTwin`	classe	`(model, field_names, coord_names=None, *, config=None, assimilation_filter=None) -> None`	Digital twin that fuses a trained surrogate model with live sensor data.
`build_digital_twin`	função	`(model, field_names, *, coord_names=None, device='cpu', update_interval=0.5, anomaly_z_th…)`	Convenience factory for DigitalTwin.

`pinneapple_systems.excitation`

Actuator excitation signals for system identification and surrogate training (numpy).

`pinneapple_systems/excitation.py` · 152 linhas · testes: 6

nome	tipo	assinatura	o que faz
`shape_step`	função	`(tau, t0=0.3, **_)`	Unit step at `tau = t0`.
`shape_ramp`	função	`(tau, t1=0.8, **_)`	Linear ramp from 0 at `tau = 0` to 1 at `tau = t1`, then held.
`shape_sinusoid`	função	`(tau, freq=2.0, phase=0.0, **_)`	`0.5 + 0.5 sin(2 pi freq tau + phase)` (`freq` in cycles per unit `tau`).
`shape_piecewise_constant`	função	`(tau, rng=None, n_seg=5, **_)`	Random staircase: `n_seg` levels ~ U(0, 1) between sorted random edges in `[0.05, 0.95]`.
`shape_prbs`	função	`(tau, rng=None, dwell=0.08, **_)`	Pseudo-random binary sequence in `{0, 1}`.
`shape_chirp`	função	`(tau, f0=0.5, f1=6.0, **_)`	Linear chirp `0.5 + 0.5 sin(phi)`, instantaneous frequency `f0 + (f1 - f0) tau` (cycles per unit tau).
`shape_sawtooth`	função	`(tau, freq=3.0, **_)`	Rising sawtooth `(freq * tau) mod 1`.
`shape_smoothed_random_walk`	função	`(tau, rng=None, **_)`	Gaussian random walk smoothed by a 15-sample moving average, rescaled to `[0, 1]`.
`shape_gaussian_bumps`	função	`(tau, rng=None, n_bumps=4, width=0.05, **_)`	Sum of `n_bumps` Gaussians (centres ~ U(0.1, 0.9), amplitudes ~ U(0.5, 1), std `width`), peak-normalised.
`shape_am_two_tone`	função	`(tau, f_carrier=5.0, f_mod=0.7, **_)`	Amplitude modulation: `(0.5 + 0.5 sin 2 pi f_carrier tau) (0.5 + 0.5 sin 2 pi f_mod tau)`.

nome	tipo	assinatura	o que faz
`rate_limit`	função	`(series, tau_steps) -> np.ndarray`	First-order low-pass `y[i] = y[i-1] + (x[i] - y[i-1]) / tau_steps` (time constant `tau_steps` samples).
`actuator_series`	função	`(shape, n_steps, *, lo=0.0, hi=1.0, seed=0, rate_limit_steps=0, **kwargs) -> np.ndarray`	Sample one excitation on `n_steps` uniform points of `tau` in `[0, 1]`, scaled to `[lo, hi]`.

`pinneapple_systems.operations`

Operations: run validated physical models and surrogates next to live plant data.

`pinneapple_systems/operations/__init__.py` · 94 linhas · testes: 24

`pinneapple_systems.operations.anomaly`

Physics anomaly detection on sensor time series: statistics, time-series models, conservation balances and physical-model residuals, fused into events with severities and verifiable explanations.

`pinneapple_systems/operations/anomaly.py` · 573 linhas · testes: 14

nome	tipo	assinatura	o que faz
`BalanceRule`	classe	`(name, accumulation, inflow, outflow, units, description)`	A conservation law over the columns: `d(accumulation)/dt = inflow - outflow`.
`DetectorConfig`	classe	`(normal_until, sensors, methods, balances, constants, window, threshold, ar_order, cusum_…)`	What to run and how strict to be. `normal_until` (time, same units as `t`) ends the normal window.
`Channel`	classe	`(method, name, variables, residual, centre, spread, units, description, score, threshold)`	One detector output: a residual series, its normal-window centre/spread and score = (r - centre) / spread.
`Event`	classe	`(start, end, i0, i1, severity, peak_ratio, methods, channels, variables, explanations, ev…)`	
`DetectionResult`	classe	`(t, channels, flags, events, config, normal_samples)`	
`LabelledInterval`	classe	`(kind, start, end, target)`	A labelled abnormal interval `[start, end)` read from data (same interface as a plant `Fault`).

nome	tipo	assinatura	o que faz
tank_balance_rules	função	() -> list[BalanceRule]	Mass and energy balances of the heated tank (:mod:pinneapple_systems.operations.plant).
tank_constants	função	(p=None) -> dict[str, float]	
windowed_balance	função	(t, acc, net_in, w) -> np.ndarray	[acc(t_k) - acc(t_{k-w})] / (t_k - t_{k-w}) - (trapezoid mean of net_in over the same interval).
severity_of	função	(ratio) -> str	Severity from the peak score / threshold ratio: < 2 low, < 4 medium, < 8 high, else critical.
detect	função	(data, config, model=None, model_states=(), model_inputs=()) -> DetectionResult	Run the configured detectors on data (DataFrame or dict of columns with t).
benchmark_faults	função	(t0=7200.0) -> list[Fault]	The faults of :func:fault_benchmark, after a fault-free first t0 seconds (the calibration window). Magnitudes are small on purpose (a few percent of the nominal flows and powers).
fault_benchmark	função	(seed=0, t0=7200.0, dt=10.0) -> tuple[dict[str, np.ndarray], list[Fault]]	One labelled run: t0 s of normal operation, then the six faults of :func:benchmark_faults separated by normal periods. Different seed = different operating profile and noise (a held-out run).
intervals_from_labels	função	(t, labels) -> list[LabelledInterval]	Contiguous runs of a label column: empty, 0, normal or false mean normal, anything else names the anomaly. Each run ends at the next sample's time.
score_events	função	(result, faults, grace=600.0, evaluate_from=None) -> dict	Point-wise and event-wise scores against the true fault intervals.
run_benchmark	função	(seeds=(101, 102, 103), methods_sets=None, window=12, threshold=5.0) -> dict	Score each detector family alone and all together on held-out labelled runs (one per seed). The calibration window of every run is its own fault-free first two hours; thresholds are never tuned on the faulty part. Returns per-method aggregate scores (sums of TP/FP/FN over runs) and per-run details.

pinneapple_systems.operations.contracts

The operations tools speak pinneapple_data.contracts: models travel as ModelRef (with their held-out validation metrics and validity ranges) and every twin, anomaly or control study ends as a StudyResult whose checks record what actually ran.

pinneapple_systems/operations/contracts.py · 192 linhas · testes: 14

nome	tipo	assinatura	o que faz
save_model	função	(model, path, name, *, app='', dataset=None, trust_decision='UNCHECKED') -> ModelRef	Write model (a twin model or a :class:ControlledOpInf) as JSON at path and return its ModelRef (saved next to it as <path>.modelref.json). Held-out metrics come from the model's own validation record; a model without one carries no metrics.
load_model	função	(ref, root='.')	Open the model a ModelRef points to (a local JSON file, checked against its SHA-256).
check	função	(name, passed, value=None, threshold=None, evidence='', detail='', warn=False) -> CheckRe…)	A CheckRecord: passed=None means it did not run (NOT_RUN).
new_id	função	(prefix) -> str	
twin_study	função	(runtime, *, app='twin', model_ref=None, title='') -> StudyResult	The twin's run as a StudyResult: estimates table, alerts, validity checks and revalidations.
anomaly_study	função	(result, *, app='anomaly', benchmark=None, title='') -> StudyResult	A detection run (and optionally a benchmark) as a StudyResult.
control_study	função	(comparison, *, app='control', title='') -> StudyResult	A controller comparison as a StudyResult (one row per controller).

pinneapple_systems.operations.control

Physics-based control of the heated tank: model predictive control against PID and rule-based baselines, in closed loop on the same simulated plant, in simulation / recommendation mode only.

pinneapple_systems/operations/control.py · 658 linhas · testes: 14

nome	tipo	assinatura	o que faz
ControlSpec	classe	(dt, horizon, setpoints, h_bounds, T_max, q_in_bounds, Q_bounds, dq_max, dQ_max, h_scale,…)	Objectives, constraints and actuator limits.
ControlScenario	classe	(duration, x0, T_in_steps, T_amb, faults, noise, seed)	The plant side of a closed-loop study: duration, disturbances and faults (shared by every controller).
PIDBaseline	classe	(spec, params=None, kp_h=0.02, ki_h=0.0001, kp_T=60000.0, ki_T=400.0)	Level loop on the pump, temperature loop on the heater, both with feed-forward bias and anti-windup.
RuleBasedBaseline	classe	(spec, band_h=0.05, band_T=0.5, q_low=0.004, q_high=0.01, Q_on=600000.0)	Hysteresis rules: pump at a high rate below h_ref - band_h and a low rate above h_ref + band_h; heater at Q_on below T_ref - band_T and off above T_ref + band_T.
LinearMPC	classe	()	Successively linearised MPC: a QP over the horizon with SLSQP.

nome	tipo	assinatura	o que faz
`NonlinearMPC`	classe	`(spec, params=None, model=None, backend='auto')`	Nonlinear MPC on the tank balances (CasADi/IPOPT if available) or on any `model` (SciPy).
`make_estimator`	função	`(spec, params=None, seed=0) -> TwinRuntime`	The state estimator the MPCs read: an EnKF twin of the tank with unaccounted outflow and heater efficiency.
`run_closed_loop`	função	`(controller, spec, scenario, params=None) -> dict`	Simulate `controller` on `scenario`. Returns the trajectories, the controller's plans (every 10th step, MPC only) and :func:`performance` metrics.
`performance`	função	`(tr, spec) -> dict`	Integrated absolute tracking errors (true states), constraint violations, energy, actuator travel and solve time of a closed-loop trajectory.
`compare_controllers`	função	`(spec=None, scenario=None, params=None, controllers=…, surrogate=None) -> dict`	Run the named controllers on the same scenario. Names: `pid`, `rules`, `lmpc`, `nmpc` and `smpc` (nonlinear MPC on `surrogate`, a model with `step(x, v)`).
`recommend`	função	`(estimate, reading, spec=None, params=None, t=0.0, method='nmpc') -> dict`	Proposed next actions for a person to review. `estimate`: state (`h`, `T` and optionally `q_leak`, `eta`); `reading`: current `q_in`, `Q`, `T_in`, `T_amb`. Nothing is sent anywhere.

`pinneapple_systems.operations.exchanger`

A counterflow water/water heat exchanger with fouling — the plant a twin is built for.

`pinneapple_systems/operations/exchanger.py` · 158 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`OperatingPoint`	classe	`(t_hi, t_ci, m_h, m_c)`	Where the exchanger is being run.
`PlantConfig`	classe	`(ua_clean, flow_sensitivity, tau_fouling_s, rf_final)`	The as-built exchanger, including the model error the twin has to live with.
`exchanger_model`	função	`(X, theta, *, ua_clean) -> dict[str, np.ndarray]`	`model(X, theta)` for the twin: outlet temperatures from the operating point.
`plant_outlets`	função	`(op, rf, cfg) -> tuple[float, float]`	The real exchanger: the model plus a flow-dependent UA the model does not know.
`fouling_at`	função	`(t_s, cfg) -> float`	Asymptotic fouling build-up: fast at first, then a plateau.
`generate_history`	função	`(hours=72.0, sampling_s=300.0, *, seed=7, cfg=None, op=None, t_start=1767225600.0) -> lis…)`	A plant history with fouling, flow wobble and instrument noise.

`pinneapple_systems.operations.live`

Run a :class:`~pinneapple_systems.operations.twin.TwinRuntime` as a service: on a live stream (MQTT, REST pushes, an experimental OPC UA poller) or over a historical table.

`pinneapple_systems/operations/live.py` · 251 linhas · testes: 11 · sinais no código: experimental

nome	tipo	assinatura	o que faz
LiveTwin	classe	`(twin, *, source='simulated', topic=None, broker_host='127.0.0.1', broker_port=0, embedde…)`	One asset's live twin. `source`: `"simulated"` (built-in plant over MQTT), `"mqtt"` (readings from an external publisher on `topic`), `"rest"` (only :meth:`push`) or `"opcua"` (experimental).
make_twin	função	`(model, *, asset='tank-1', store=None, rules=None, sigma_obs=None, sigma_process=None, se…)`	A twin of the heated tank with the noise levels of its sensors (override for another asset).
replay	função	`(table, model, *, asset='tank-1', store=None, rules=None, forecast_steps=60, revalidate=T…)`	Run a twin over a recorded table (DataFrame or dict of columns with `t` and the model's inputs and observed states). Returns the twin and a JSON-ready result with the history, alerts, final forecast and, when `revalidate`, an open-loop re-score of the model on the last part of the record.
demo_record	função	`(duration=7200.0, dt=10.0, seed=4, faults=None) -> dict`	A recorded run of the simulated tank with a 0.5 L/s leak over its second half (the twin app's offline example).
default_model	função	`(kind='lumped', dt=10.0)`	

pinneapple_systems.operations.plant

Heated buffer tank: a lumped (0-D) plant with a mass and an energy balance, its sensors and its faults.

`pinneapple_systems/operations/plant.py` · 290 linhas · testes: 25

nome	tipo	assinatura	o que faz
TankParams	classe	`(area, k_out, ua, rho, cp, h_min, h_max, q_in_max, q_max_heater)`	Physical parameters of the heated tank (SI units, temperatures in degC).
Fault	classe	`(kind, start, end, magnitude, target)`	A fault active on `[start, end)` seconds. `target` names the sensor for sensor faults.
Scenario	classe	`(duration, dt, x0, T_in, T_amb, q_in, Q)`	Operating scenario: how the inputs and disturbances evolve. Callables take the time [s] and return a value.
SensorNoise	classe	`(sigma)`	Standard deviation of each sensor's white measurement noise (SI units).
TankSimulator	classe	`(params=None, x0=(1.2, 25.0), faults=(), noise=None, seed=0)`	Step-by-step simulator of the heated tank with sensors and faults. Used open loop (scenario inputs), closed loop (a controller sets the inputs each step) or as the live source of a digital twin demo.
tank_rhs	função	`(x, u, d, p, eta=1.0, valve=1.0, q_leak=0.0) -> np.ndarray`	Time derivative `[dh/dt, dT/dt]` at state `x = [h, T]`, inputs `u = [q_in, Q]` and disturbances `d = [T_in, T_amb]`. Works on arrays of states with the leading axis being the state component.

nome	tipo	assinatura	o que faz
`tank_step`	função	`(x, u, d, p, dt, substeps=4, **fault_kw) -> np.ndarray`	Advance the tank `dt` seconds with classical RK4 (inputs held constant over the step).
`steady_state`	função	`(p, q_in, Q, T_in, T_amb) -> np.ndarray`	Equilibrium `[h, T]` for constant inputs (closed form).
`inputs_for_steady_state`	função	`(p, h, T, T_in, T_amb) -> np.ndarray`	Inputs `[q_in, Q]` that hold the tank at `[h, T]` (closed form).
`measure`	função	`(truth, t, faults, noise, rng, frozen) -> dict[str, float]`	Sensor readings for one time step: true value + noise, then sensor faults.
`simulate`	função	`(scenario, params=None, faults=(), noise=None, seed=0) -> dict[str, np.ndarray]`	Open-loop run of `scenario` (its `q_in` and `Q` must be set). Returns columns: `t`, the measured sensors, `true_<state>`, `true_q_leak`, `true_eta` and `fault` (name of the active fault or "").
`demo_scenario`	função	`(duration=3600.0, dt=10.0, seed=0) -> Scenario`	A realistic day-shift profile: pump flow steps between production rates, heater follows a schedule, inlet water temperature wanders slowly. Deterministic for a given `seed`.
`mass_balance_residual`	função	`(t, h, q_in, q_out, area) -> np.ndarray`	`A dh/dt - (q_in - q_out)` [m3/s] from measured signals (central differences in time, inputs averaged over each interval). Zero for a fault-free tank up to noise; a leak makes it equal to minus the leak flow.
`energy_balance_residual`	função	`(t, h, T, q_in, q_out, Q, T_in, T_amb, p) -> np.ndarray`	Energy balance in conservative form [W]::
`with_params`	função	`(p, **changes) -> TankParams`	

`pinneapple_systems.operations.rom`

Operator inference with inputs: a discrete-time polynomial reduced model fitted by ridge regression.

`pinneapple_systems/operations/rom.py` · 178 linhas · testes: 11

nome	tipo	assinatura	o que faz
`ControlledOpInf`	classe	`(state_names, input_names, dt, W, x_mean, x_scale, v_mean, v_scale, quadratic, bilinear, …)`	A fitted model; build it with :func:`fit_controlled_opinf` or :meth:`from_dict`.
`fit_controlled_opinf`	função	`(trajectories, state_names, input_names, dt, *, holdout=1, ridge=1e-06, quadratic=True, b…)`	Fit on all but the last `holdout` trajectories and validate on those.
`residual_sigma`	função	`(model, trajectories) -> dict[str, float]`	One-step residual standard deviation per state on `trajectories` (use held-out ones): the process noise a filter running this model should assume.

pinneapple_systems.operations.series

The days 13-31 series end to end on one asset: Campaign Planner -> (runs) -> Surrogate Studio -> Uncertainty -> Optimizer -> Digital Twin, every hand-over a `pinneapple_data.contracts` file.

`pinneapple_systems/operations/series.py` · 339 linhas · testes: 14

nome	tipo	assinatura	o que faz
`operating_space`	função	`() -> ParameterSpace`	
`app_available`	função	`(pattern, apps_dir) -> str | None`	Name of the `apps/` directory of a series app (matched by its whole name), if this checkout has it.
`run_series_flow`	função	`(workdir, *, seed=0, n_runs=12, steps_per_run=12, hold=300.0, dt=20.0, T_min=28.0, q_min=…)`	Run the chain and write every contract under `workdir`. Returns a JSON-ready summary.

pinneapple_systems.operations.twin

Digital twin runtime: run a validated model next to live data, estimate its state, forecast, alert and watch the model's own validity.

`pinneapple_systems/operations/twin.py` · 602 linhas · testes: 25

nome	tipo	assinatura	o que faz
`LumpedTankModel`	classe	`(params=None, dt=10.0, estimate=('q_leak', 'eta'))`	The heated tank's mass and energy balances as a twin model, optionally with parameters in the state.
`SurrogateModel`	classe	`(rom)`	A :class:`ControlledOpInf` reduced model as a twin model (no parameters in the state).
`AlertRules`	classe	`(limits, param_limits, param_sigmas, nis_probability, nis_consecutive, forecast_steps, dr…)`	Alert thresholds. `limits` and `param_limits` map a state name to `(low, high)` (`None` = open).
`Alert`	classe	`(t, kind, severity, message, data, cleared)`	
`TwinStore`	classe	`(url=':memory:')`	History of readings, estimates, alerts and revalidations.
`TwinRuntime`	classe	`(model, obs_names, sigma_obs, sigma_process, *, x0=None, sigma0=None, n_ens=48, inflation…)`	Assimilate readings one at a time into `model` and keep the twin's state, alerts and validity.
`model_from_dict`	função	`(d)`	Rebuild a twin model from its JSON form (`kind` = `lumped_tank` or `controlled_opinf`).

pinneapple_systems.process_components

pinneapple_systems.process_components — physical unit-operation models for process/plant digital twins: real-gas thermodynamic properties, turbomachinery (polytropic compression/expansion + nondimensional similarity maps), control valves, heat exchangers, and 1D pipe networks.

`pinneapple_systems/process_components/__init__.py` · 272 linhas · testes: 58

pinneapple_systems.process_components.beam_nonlinear_fem

pinneapple_systems.process_components.beam_nonlinear_fem -- static, geometrically nonlinear (Von Karman) Euler-Bernoulli beam FEM via load-stepped Newton-Raphson.

`pinneapple_systems/process_components/beam_nonlinear_fem.py` · 172 linhas · testes: 6

nome	tipo	assinatura	o que faz
`StaticNonlinearBeamResult`	classe	`(z, u_m, w_m, theta_rad, bending_moment, bending_stress, load_step_deflections, iteration…)`	
`solve_nonlinear_beam_static`	função	`(boundary, L_m, A_m2, I_m4, E_Pa, fiber_distance_m, axial_dist_load_N_per_m=0.0, transver…)`	Load-stepped Newton-Raphson static solve of the geometrically nonlinear (Von Karman) Euler-Bernoulli beam, at the static limit of `NonlinearBeamNewmarkFEM` (see module docstring). `fiber_distance_m` is the distance from the neutral axis to the outer fiber used for bending-stress recovery (height/2 ...

pinneapple_systems.process_components.beam_statics

pinneapple_systems.process_components.beam_statics -- closed-form static, linear-elastic Euler-Bernoulli beam solutions for the standard cantilever and simply-supported load cases.

`pinneapple_systems/process_components/beam_statics.py` · 196 linhas · testes: 19

nome	tipo	assinatura	o que faz
`BeamResult`	classe	`(x, deflection, slope, bending_moment, shear_force)`	
`rectangular_section_properties`	função	`(width_m, height_m) -> tuple[float, float]`	Returns (I, Q) for a rectangular cross-section: second moment of area I = b*h^3/12, first moment of area at the neutral axis Q = b*h^2/8 (used for shear stress).
`von_mises_stress_rectangular_section`	função	`(bending_moment, shear_force, I, Q, width_m, height_m) -> np.ndarray`	
`solve_beam`	função	`(case, x, L, E, I, q0, load_extent=None) -> BeamResult`	Dispatches to the closed-form solution for `case`. `load_extent` (the partial-load length, measured from the load's own reference end per case) is required for the two partial-uniform-load cases and ignored otherwise.

pinneapple_systems.process_components.control_valve

pinneapple_systems.process_components.control_valve — compressible and incompressible control-valve flow sizing per IEC 60534-2-1 (harmonized with ANSI/ISA-75.01.01), plus actuator response and closed-seat leakage.

`pinneapple_systems/process_components/control_valve.py` · 115 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ValveSpec`	classe	`(name, Cv_max_us, xT, characteristic, equal_percentage_R, actuator_time_constant_s, leaka…)`	
`ValveFlowResult`	classe	`(mass_flow_kg_s, choked, x, Y, Cv_effective_us)`	
`installed_cv`	função	`(travel_fraction, spec) -> float`	Cv at a given travel fraction (0=closed, 1=full open), per the inherent characteristic -- excludes leakage, see `effective_cv`.
`effective_cv`	função	`(travel_fraction, spec, leakage_cv_us_override=None) -> float`	Cv actually passing flow, including the closed-seat leakage floor -- pass `leakage_cv_us_override` to represent a TRUE (e.g. degraded) leakage different from the spec's nameplate value.
`compressible_mass_flow`	função	`(Cv_us, P1_Pa, P2_Pa, rho1_kg_m3, k_isentropic, xT) -> ValveFlowResult`	
`incompressible_mass_flow`	função	`(Cv_us, P1_Pa, P2_Pa, rho_kg_m3) -> float`	Liquid-service sizing (no expansion factor / choking model beyond simple flashing checks are out of scope here) -- w = Cv_SI * sqrt(dP * rho).
`actuator_response_rhs`	função	`(travel_actual, travel_command, spec) -> float`	d(travel_actual)/dt for the first-order actuator lag.

`pinneapple_systems.process_components.curved_path_geometry`

pinneapple_systems.process_components.curved_path_geometry -- the "build-and-hold" curved-conduit path generator: a straight (vertical) section, a constant-curvature build section transitioning to a target inclination, and a straight tangent/hold section at that inclination.

`pinneapple_systems/process_components/curved_path_geometry.py` · 113 linhas · testes: 7 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`CurvedPathProfile`	classe	`(s_m, inclination_deg, tvd_m, hd_m, n_bins, vertical_length_m, total_length_m, bin_width_…)`	
`inclination_at_depth`	função	`(s_m, vertical_length_m, build_length_m, final_inclination_deg) -> np.ndarray`	Inclination [deg, from horizontal] at each measured depth `s_m`: 90 up to `vertical_length_m`, linear (constant curvature) through the following `build_length_m`, held at `final_inclination_deg` after.
`build_and_hold_profile`	função	`(total_length_m, build_length_m, final_inclination_deg, hold_length_m, bin_width_m) -> Cu…)`	Discretize the build-and-hold path into `bin_width_m`-wide bins (midpoint-evaluated) and integrate TVD/HD along it. `vertical_length_m = total_length_m - build_length_m - hold_length_m` is derived, not an independent input, so the three lengths are always consistent.

nome	tipo	assinatura	o que faz
`circular_arc_tvd_hd`	função	`(s_within_build_m, build_length_m, final_inclination_deg) -> tuple`	Closed-form TVD/HD accumulated over the build section only, for a circular arc of constant curvature `kappa = delta_theta_rad / build_length_m` (delta_theta = 90 - final_inclination_deg, in rad): `tvd(s) = sin(kappa*s)/kappa`, `hd(s) = (1 - cos(kappa*s))/kappa`. Independent closed-form cross-check ...

`pinneapple_systems.process_components.explicit_equation_system`

pinneapple_systems.process_components.explicit_equation_system -- a generic, sandboxed "define your model as named equations" engine: given a set of scalar/array symbols (inputs, parameters, computed intermediates, and outputs) and a set of equations relating them, safely parse and evaluate the ...

`pinneapple_systems/process_components/explicit_equation_system.py` · 393 linhas · testes: 171

nome	tipo	assinatura	o que faz
`ExplicitEquationError`	classe	`()`	
`Definition`	classe	`(name, lhs, rhs, rhs_symbols)`	
`AnalysisResult`	classe	`(valid, errors, evaluation_order)`	
`ParameterSpec`	classe	`(symbol, value, fit, min, max)`	
`CalibrationResult`	classe	`(fitted_parameters, start, bounds, n_evaluations, final_scaled_residual_ssr, termination_…)`	
`safe_eval`	função	`(expr, env) -> Any`	
`build_definitions`	função	`(inputs, parameters, intermediates, outputs, equations) -> tuple[list[Definition], list[s…)`	Validates and builds Definitions from raw {"name","equation"} dicts. Returns (definitions, errors) -- errors accumulate rather than failing on the first one, so a caller can report everything wrong with a submitted model at once.
`evaluation_order`	função	`(definitions) -> list[Definition]`	Topological sort by LHS symbol, DFS with cycle detection.
`evaluate`	função	`(input_values, parameter_values, order) -> dict[str, Any]`	Evaluates every definition in the given (topological) order, returning the full symbol -> value environment (leaves + every computed symbol).

nome	tipo	assinatura	o que faz
analyze	função	(inputs, parameters, intermediates, outputs, equations) -> AnalysisResult	Pure validation/dry-run -- builds and orders the system, reports errors, does NOT evaluate any numbers.
calibrate	função	(input_columns, parameters, order, measured_rows, target_symbols) -> CalibrationResult	Bounded nonlinear least-squares calibration (Trust Region Reflective) of every parameter with fit=True against measured_rows's target-symbol columns, re-evaluating the FULL equation system (not a linearization) at every trial parameter vector -- see module docstring's NUMERICAL METHOD.

pinneapple_systems.process_components.fatigue_analysis

pinneapple_systems.process_components.fatigue_analysis -- S-N (Wohler) curve fatigue life, Goodman mean-stress correction, and Miner's rule cumulative damage accumulation for cyclic structural loading.

pinneapple_systems/process_components/fatigue_analysis.py · 108 linhas · testes: 8

nome	tipo	assinatura	o que faz
MinerDamageResult	classe	(N_f, damage_this_period, cumulative_damage, remaining_life_fraction)	
sn_curve_cycles_to_failure	função	(sigma_a, A, m) -> ArrayLike	N_f = 10**(A - m*log10(sigma_a)) for a fully-reversed amplitude sigma_a (floored at 1.0 to keep log10 finite).
goodman_equivalent_amplitude	função	(sigma_alt, sigma_mean, S_ut) -> ArrayLike	Equivalent fully-reversed amplitude per the modified-Goodman line (see module docstring): sigma_alt / (1 - sigma_mean/S_ut). A mean stress approaching (or exceeding) S_ut drives this to infinity/ negative, correctly signaling the Goodman boundary has already been crossed by static overload alone.
goodman_safety_ratio	função	(sigma_alt, sigma_mean, S_e, S_ut) -> ArrayLike	sigma_alt/S_e + sigma_mean/S_ut -- the Goodman-line safety ratio itself (< 1 inside the safe region, independent of any S-N life estimate); S_e is the endurance limit / target-life fully-reversed strength the diagram is drawn against.
miners_rule_damage	função	(sigma_alt, sigma_mean, n_cycles, A_sn, m_sn, S_ut, prior_damage_fraction=0.0) -> MinerDa…)	Goodman-corrected S-N life N_f, this-period damage n_cycles/N_f, and Miner's-rule cumulative damage (clipped to [0, 1]) combining it with prior_damage_fraction.

pinneapple_systems.process_components.heat_exchanger

pinneapple_systems.process_components.heat_exchanger -- effectiveness-NTU heat exchanger model (counter-flow and parallel-flow closed-form effectiveness), plus a lumped-capacitance transient response.

pinneapple_systems/process_components/heat_exchanger.py · 113 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HeatExchangerSpec`	classe	`(name, UA_design_W_K, design_dP_Pa, design_m_dot_hot_kg_s, design_rho_hot_kg_m3, thermal_…)`	
`HeatExchangerResult`	classe	`(Q_W, T_hot_out_K, T_cold_out_K, effectiveness, NTU, dP_Pa, UA_effective_W_K)`	
`steady_state`	função	`(spec, m_dot_hot_kg_s, T_hot_in_K, cp_hot_J_kgK, T_cold_in_K, rho_hot_kg_m3, UA_multiplie…)`	
`transient_rhs`	função	`(T_hot_out_current_K, spec, m_dot_hot_kg_s, T_hot_in_K, cp_hot_J_kgK, T_cold_in_K, rho_ho…)`	d(T_hot_out)/dt -- relaxes toward the instantaneous steady-state effectiveness-NTU outlet temperature with a time constant set by thermal_mass_J_K / (m_dot * cp).

`pinneapple_systems.process_components.non_newtonian_pipe_flow`

pinneapple_systems.process_components.non_newtonian_pipe_flow -- Herschel-Bulkley rheology, the generalized (Metzner-Reed) Reynolds number and friction factor for power-law/yield-stress fluids, and steady-state 1D pressure-drop integration along an arbitrarily inclined conduit (straight pipe, ...

`pinneapple_systems/process_components/non_newtonian_pipe_flow.py` · 155 linhas · testes: 10 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`PressureProfile`	classe	`(s_m, tvd_m, P_Pa, rho_eq_kg_m3, rho_kg_m3, friction_factor, reynolds_number)`	
`herschel_bulkley_stress`	função	`(tau_y, K, n, gamma_dot) -> ArrayLike`	tau = tau_y + K*gamma_dot**n. `gamma_dot` is floored at 1e-9 s^-1.
`effective_viscosity`	função	`(tau_y, K, n, gamma_dot) -> ArrayLike`	mu_eff = tau/gamma_dot = tau_y/gamma_dot + K*gamma_dot**(n-1).
`generalized_reynolds_number`	função	`(rho, v, D, K, n) -> ArrayLike`	Re_gen = rho*|v|**(2-n)*D**n / (K*8**(n-1)); laminar if Re_gen < 2100.
`metzner_reed_friction_factor`	função	`(Re_gen) -> ArrayLike`	f = 16/Re_gen (laminar, Re_gen < 2100) or 0.0791*Re_gen**-0.25 (turbulent, Blasius/Metzner-Reed).
`pressure_gradient`	função	`(rho, v, D_h, f, inclination_rad=np.pi / 2, g=9.81) -> ArrayLike`	dP/ds = rho*g*sin(inclination) + f*rho*v**2/(2*D_h). `inclination_rad` is measured from horizontal (pi/2 = vertical, the default).

nome	tipo	assinatura	o que faz
integrate_pressure_profile	função	(length_m, v_m_s, D_h_m, K_Pa_sn, n_flow_index, P_inlet_Pa, rho_profile=1000.0, inclinati…)	Steady-state 1D pressure integration along path length s in [0, length_m], by explicit forward Euler on dP/ds from pressure_gradient. rho_profile/inclination_deg_profile may be a constant or a callable f(s_m) -> value. tvd_profile(s_m) supplies true vertical depth for the …

pinneapple_systems.process_components.pipe_network_1d

pinneapple_systems.process_components.pipe_network_1d — 1D real-gas pipe flow: a rapid steady-state scenario mode and a transient finite-volume mode, covering a single pipe segment.

pinneapple_systems/process_components/pipe_network_1d.py · 290 linhas · testes: 151

nome	tipo	assinatura	o que faz
PipeSpec	classe	(name, length_m, diameter_m, roughness_m, elevation_change_m, ambient_temperature_K, wall…)	
SteadyProfilePoint	classe	(x_m, P_Pa, T_K, rho_kg_m3, G_kg_m2s, mach)	
PipeState	classe	(P_Pa, T_K)	
TransientPipe	classe	(spec, gas)	Finite-volume transient solver -- see module docstring's DISCRETIZATION NOTE for why initialize_from_steady_state averages node values into cell-center values rather than reusing them directly.
colebrook_white_f	função	(Re, relative_roughness) -> float	Darcy friction factor -- laminar closed form below Re=2300, else Colebrook-White, solved via Newton iteration on 1/sqrt(f).
face_reynolds_number	função	(G_kg_m2s, D_m, viscosity_Pa_s) -> float	
rapid_steady_state_profile	função	(spec, gas, m_dot_kg_s, P_in_Pa, T_in_K) -> list[SteadyProfilePoint]	Marches downstream cell-by-cell from the known inlet, solving each face's momentum closure explicitly (no time integration).

pinneapple_systems.process_components.pipe_stress_mechanics

pinneapple_systems.process_components.pipe_stress_mechanics -- combined-load stress analysis for a slender tube (axial + torsional + bending + internal/external pressure), Euler column buckling, and constrained-rod (Paslay-Dawson) buckling of a slender member confined within a surrounding …

pinneapple_systems/process_components/pipe_stress_mechanics.py · 209 linhas · testes: 16

nome	tipo	assinatura	o que faz
ConstrainedRodBucklingResult	classe	(F_critical_sinusoidal, F_critical_helical, mode)	
RotatingBendingCycleResult	classe	(theta_rad, sigma_von_mises, sigma_alternating, sigma_mean)	
lame_hoop_stress_outer	função	(p_internal, r_outer, r_inner) -> ArrayLike	sigma_theta at r=r_outer: 2*p_i*ri**2 / (ro**2-ri**2) (the general Lame hoop-stress formula's (1 + ro**2/r**2) factor evaluates to exactly 2 at r=r_outer -- see module docstring).
lame_hoop_stress_inner	função	(p_internal, r_outer, r_inner) -> ArrayLike	sigma_theta at r=r_inner (the maximum): p_i*(ri**2+ro**2) / (ro**2-ri**2).
torsional_shear_stress	função	(torque, r_outer, J) -> ArrayLike	tau = T*c/J, c = r_outer (maximum shear, at the outer fiber).
bending_stress_from_curvature	função	(curvature, E, r_outer) -> ArrayLike	sigma_bend = E*kappa*r_outer (outer-fiber bending stress for a circular cross-section bent to curvature kappa [1/length]).
von_mises_triaxial	função	(sigma_axial, sigma_hoop, sigma_radial=0.0, tau=0.0) -> ArrayLike	Full triaxial Von Mises equivalent stress for three orthogonal principal-direction normal stresses plus a shear stress acting in the (axial, hoop) plane. See module docstring for the derivation and its biaxial (sigma_radial=0) reduction.
euler_critical_buckling_load	função	(E, I, length, end_condition_factor=1.0) -> float	P_cr = pi**2*E*I / (K*L)**2. end_condition_factor K: 0.5 fixed- fixed, 0.7 fixed-pinned, 1.0 pinned-pinned (default), 2.0 fixed-free (cantilever).
constrained_rod_buckling_load	função	(E, I, lateral_load_per_length, radial_clearance) -> ConstrainedRodBucklingResult	Paslay-Dawson sinusoidal/helical critical compressive loads for a slender rod confined within a surrounding cylindrical clearance (see module docstring). lateral_load_per_length is the distributed load normal to the rod axis (e.g. buoyed self-weight component); returns "not_applicable" mode with …
classify_buckling_mode	função	(compressive_force, result) -> ArrayLike	0 = straight, 1 = sinusoidal, 2 = helical, per Paslay-Dawson critical loads.
beam_column_moment_amplification_factor	função	(compressive_force, E, I, length, end_condition_factor=1.0, max_ratio=0.98) -> ArrayLike	AF = 1 / (1 - P/P_cr), the standard beam-column second-order (P-Delta) moment-amplification approximation (see module docstring). P/P_cr is clipped to max_ratio before dividing, since the approximation (and physically, the member) diverges as the axial load approaches the critical load -- the …
rotating_bending_stress_cycle	função	(sigma_axial, sigma_bending_amplitude, sigma_hoop=0.0, tau=0.0, n_angles=36) -> RotatingB…)	Von Mises stress envelope over one full rotation of a shaft/rod under constant axial + hoop + torsional stress plus a bending stress that varies as sigma_bending_amplitude*cos(theta) (see module docstring), and the alternating/mean stress a fatigue analysis needs from that envelope.

pinneapple_systems.process_components.polytropic_path

pinneapple_systems.process_components.polytropic_path — real-gas polytropic compression/expansion path integration for turbomachinery, consistent with ASME PTC 10 ("Performance Test Code on Compressors

and Exhausters").

`pinneapple_systems/process_components/polytropic_path.py` · 196 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PolytropicPathResult`	classe	`(inlet, outlet, polytropic_work_J_kg, polytropic_efficiency, isentropic_work_J_kg, isentr…)`	
`solve_path_from_pressure_ratio`	função	`(inlet, gas, P2_Pa, polytropic_efficiency) -> PolytropicPathResult`	Integrate equation (1) from the inlet state to a known discharge/ exit pressure P2 -- works for compression (P2 > P1) or expansion (P2 < P1).
`solve_path_from_work`	função	`(inlet, gas, polytropic_work_J_kg, polytropic_efficiency, *, max_pressure_ratio=6.0) -> P…)`	Integrate equation (1) until the accumulated work integral reaches a target (a map's polytropic head, in energy-per-mass units) -- solves for the discharge/exit pressure self-consistently. Positive `polytropic_work_J_kg` compresses; negative expands.

`pinneapple_systems.process_components.reaction_kinetics`

pinneapple_systems.process_components.reaction_kinetics -- a generic multi-species mass-action reaction-network engine, plus a generic 1D advection-dispersion-reaction transport solver built on top of it.

`pinneapple_systems/process_components/reaction_kinetics.py` · 257 linhas · testes: 164

nome	tipo	assinatura	o que faz
`Reaction`	classe	`(name, rate_fn, stoichiometry)`	
`ReactionNetwork`	classe	`(species, reactions)`	
`IntegrationResult`	classe	`(t, C, success)`	
`AdvectionDispersionReactionSolver`	classe	`(network, n_grid, length_m, velocity_m_s, dispersion_m2_s)`	1D transport (equation 3) on a periodic grid, driven by a ReactionNetwork for R_s(C) -- see module docstring's TRANSPORT section for the discretization and the periodic-only scope note.
`mass_action_rate`	função	`(k, orders) -> RateFn`	Builds equation (2): rho(C) = k * prod(C[s]^order[s]). `k` may be a plain float or a callable(T_K=.., pH=.., C=..) -> float for a temperature/pH/catalyst-dependent rate constant (see `arrhenius_rate_constant`, `linear_combination_rate_constant`).
`arrhenius_rate_constant`	função	`(A, Ea_over_R) -> Callable[..., float]`	k(T_K) = A * exp(-Ea_over_R / T_K). Raises if called without a temperature (a temperature-dependent rate constant used where no T is supplied is a caller bug, not something to silently default).

nome	tipo	assinatura	o que faz
quadratic_in_T	função	(a, b, c) -> Callable[[float], float]	f(T_K) = a*T_K^2 + b*T_K + c -- e.g. for a polynomial-fit pKa(T) or equilibrium constant, when a literature source gives that form instead of an Arrhenius one.
linear_combination_rate_constant	função	(terms) -> Callable[..., float]	k_eff(C) = sum(coefficient_i * C[species_i]) -- for a rate constant that is itself a linear combination of catalyst-species concentrations (e.g. an acid/base/buffer-catalyzed reaction).
acid_fraction	função	(pH, pKa) -> float	Henderson-Hasselbalch: fraction of a monoprotic acid-base pair present in the PROTONATED (acid) form.
base_fraction	função	(pH, pKa) -> float	Fraction present in the DEPROTONATED (base/conjugate-base) form -- exactly 1 - acid_fraction(pH, pKa).
diprotic_fractions	função	(pH, pKa1, pKa2) -> tuple[float, float, float]	Bjerrum fractions (alpha_H2A, alpha_HA, alpha_A) for a diprotic acid-base system (H2A / HA- / A2-), generalizing the carbonate system (H2CO3/HCO3-/CO3^2-) to any two-pKa diprotic pair.
integrate_network	função	(network, C0, t_eval, T_K=None, pH=None, *, method='Radau', rtol=1e-09, atol=1e-14) -> In…	Integrates dC/dt = network.rhs(C) from C0 over t_eval, clamping concentrations non-negative before every RHS evaluation and raising RuntimeError (never silently returning a failed/partial solve) if the integrator does not report success.

pinneapple_systems.process_components.real_gas_eos

pinneapple_systems.process_components.real_gas_eos — multi-component real-gas mixture properties via a Helmholtz-energy equation of state.

pinneapple_systems/process_components/real_gas_eos.py · 259 linhas · testes: 151

nome	tipo	assinatura	o que faz
OutOfEnvelopeError	classe	()	Raised when a requested state falls outside a caller-declared validity envelope.
GasComposition	classe	(components, mole_fractions)	A fixed-composition mixture, mole fractions summing to 1.
ValidityEnvelope	classe	(P_min_Pa, P_max_Pa, T_min_K, T_max_K)	Optional, caller-declared (P, T) bounds -- states outside it raise OutOfEnvelopeError instead of silently extrapolating. Pass None to state_from_*'s envelope argument to skip this check entirely.
GasState	classe	(P_Pa, T_K, rho_kg_m3, Z, h_J_kg, s_J_kgK, cp_J_kgK, cv_J_kgK, k_isentropic, speed_of_sou…)	A fully-resolved thermodynamic state point, SI units throughout.
StandardConditions	classe	(P_Pa, T_K)	
state_from_PT	função	(gas, P_Pa, T_K, *, backend='HEOS', specify_phase='gas', envelope=None) -> GasState	

nome	tipo	assinatura	o que faz
`state_from_Ph`	função	`(gas, P_Pa, h_J_kg, *, backend='HEOS', specify_phase='gas', envelope=None) -> GasState`	Pressure + specific enthalpy -> full state (temperature recovered by EOS flash). Used wherever an energy balance produces h directly.
`state_from_Ps`	função	`(gas, P_Pa, s_J_kgK, *, backend='HEOS', specify_phase='gas', envelope=None) -> GasState`	Pressure + specific entropy -> full state (an isentropic endpoint).
`standard_volumetric_flow_to_mass_flow`	função	`(q_std_m3_s, gas, std) -> float`	
`mass_flow_to_standard_volumetric_flow`	função	`(m_dot_kg_s, gas, std) -> float`	
`central_difference`	função	`(gas, P_Pa, T_K, attr, wrt, *, rel_step=0.0001) -> float`	d(attr)/d(wrt) via central differences -- for use OUTSIDE an autograd graph (CoolProp's compiled solve is not autodiff-transparent; any training loop that needs a gradient through this EOS should call this instead of tracing through `state_from_*` directly).

`pinneapple_systems.process_components.similarity_map`

pinneapple_systems.process_components.similarity_map — nondimensional turbomachinery performance-map utilities (flow/head coefficients, tip Mach and machine Reynolds corrections, surge/choke margins).

`pinneapple_systems/process_components/similarity_map.py` · 194 linhas · testes: 8

nome	tipo	assinatura	o que faz
`MapCoefficients`	classe	`(name, D2_m, n_stages, phi_ref_bep, psi_ref_bep, psi_shape_a, psi0, eta_p_max, eta_p_shap…)`	
`MapEvaluation`	classe	`(phi, psi, eta_p, Mu2, Re, surge_margin_frac, choke_margin_frac)`	
`derive_psi_shape_a`	função	`(psi0, phi_ref_bep, psi_ref_bep) -> float`	Curvature 'a' in psi(phi) = psi0 - a*phi^2 that makes the curve pass exactly through (phi_ref_bep, psi_ref_bep).
`make_map`	função	`(name, D2_m, phi_ref_bep, psi_ref_bep, psi0, eta_p_max, eta_p_shape_b, phi_surge, phi_cho…)`	
`flow_coefficient`	função	`(Q1_m3_s, N_rpm, coeffs) -> float`	
`tip_speed_m_s`	função	`(N_rpm, coeffs) -> float`	
`tip_mach_number`	função	`(N_rpm, coeffs, speed_of_sound_m_s) -> float`	

nome	tipo	assinatura	o que faz
machine_reynolds_number	função	(N_rpm, coeffs, kinematic_viscosity_m2_s) -> float	
evaluate_map	função	(coeffs, Q1_m3_s, N_rpm, speed_of_sound_m_s, kinematic_viscosity_m2_s, Re_ref=None) -> Ma…	
polytropic_head_from_psi	função	(psi, N_rpm, coeffs) -> float	Total machine polytropic head (J/kg) = n_stages * psi * U2^2.
required_speed_for_head	função	(target_head_J_kg, Q1_m3_s, coeffs, speed_of_sound_m_s, kinematic_viscosity_m2_s, *, N_mi…)	Solves for the shaft speed producing target_head_J_kg at a fixed actual inlet volumetric flow, via bisection on the monotonic head(N) relation at fixed Q1.

pinneapple_systems.process_components.torsional_stickslip

pinneapple_systems.process_components.torsional_stickslip -- 1D torsional-wave finite-difference simulation of a compliant rotating shaft, driven at one end at a prescribed angular velocity and loaded at the other end by Stribeck (velocity-weakening) friction acting on a lumped end inertia -- the …

pinneapple_systems/process_components/torsional_stickslip.py · 179 linhas · testes: 6

nome	tipo	assinatura	o que faz
StickSlipResult	classe	(t_s, z_m, omega_field_rad_s, end_omega_ts_rad_s, surface_torque_ts_N_m, stick_slip_index)	
stribeck_friction_torque	função	(omega, T_static, T_kinetic, omega_breakout) -> float	T_friction = sign(omega)*[T_kinetic + (T_static-T_kinetic)*exp(-|omega|/omega_breakout)].
simulate_torsional_stickslip	função	(length_m, G_Pa, J_m4, rho_kg_m3, omega_set_rad_s, T_static_N_m, T_kinetic_N_m, omega_bre…)	Explicit finite-difference solve of the 1D torsional wave equation described in the module docstring, from a driven end (node 0, held at omega_set_rad_s) to a friction-loaded end (node -1, lumped inertia end_inertia_kg_m2 under Stribeck friction). Uses a fixed uniform shaft cross-section …

pinneapple_systems.process_components.water_hammer

Water hammer in a pumped pipeline: steady operating point and the valve-closure transient by the method of characteristics (MOC).

pinneapple_systems/process_components/water_hammer.py · 191 linhas · testes: 39

nome	tipo	assinatura	o que faz
Pipeline	classe	(length, diameter, wave_speed, roughness, z_in, z_out, nu, rho)	

nome	tipo	assinatura	o que faz
PumpCurve	classe	(shutoff_head, k, sump_head)	H = shutoff_head - k Q|Q| (fixed speed); sump_head: suction-side head [m].
OperatingPoint	classe	(flow, velocity, reynolds, friction_factor, pump_head, valve_head, friction_loss, valve_c…)	
TransientResult	classe	(t, x, head, flow, tau, dt, courant, op, info)	
operating_point	função	(line, pump, *, valve_k, tank_head=0.0) -> OperatingPoint	Flow where the pump curve meets friction (Colebrook) + the open valve's loss valve_k * V^2 / (2 g).
linear_closure	função	(closure_time, start=0.0)	tau(t): 1 until start, linearly to 0 over closure_time (0 = instantaneous).
moc_valve_closure	função	(line, pump, op, *, closure_time, t_end, n_reaches=40, friction=True, closure_start=0.0, …)	Valve closure transient from the steady operating point op (pump upstream, valve + tank downstream).
joukowsky_head_rise	função	(wave_speed, dv) -> float	a dV / g [m]: head rise of an instantaneous velocity change dv (Joukowsky).
wave_period	função	(length, wave_speed) -> float	4 L / a [s]: period of the pressure oscillation after a closure (pipe from a reservoir).

pinneapple_systems.sports

Sports physics: physics-informed analytics on public sports tracking and event data (football first).

pinneapple_systems/sports/__init__.py · 2 linhas · **sem teste direto**

pinneapple_systems.sports.football

Football: physics-informed analytics on public tracking and event data (ball flight, pitch control, player kinematics, expected goals). See docs/guides/football_physics.md.

pinneapple_systems/sports/football/__init__.py · 3 linhas · testes: 16

pinneapple_systems.sports.football.ball

Ball flight physics: gravity, quadratic drag and Magnus lift, the inverse problem from tracked positions, and a rolling-ball model for ground passes.

pinneapple_systems/sports/football/ball.py · 464 linhas · testes: 16

nome	tipo	assinatura	o que faz
Ball	classe	(mass, diameter)	
FlightFit	classe	(model, x0, v0, cd, magnus, g, t0, params, names, cov, rmse, n, success, extra)	Result of :func:fit_flight: parameters, their Laplace (Gauss-Newton) standard errors and fit quality.
Flight	classe	(match_id, t, xyz, detected)	An aerial ball flight cut from a tracking record (times in s, positions in m).

nome	tipo	assinatura	o que faz
cd_crisis	função	(re, cd_sub=0.45, cd_super=0.18, re_crit=260000.0, width=25000.0)	A smooth drag-crisis curve: cd_sub below re_crit, cd_super above (logistic of width width).
simulate	função	(x0, v0, t, *, cd=0.0, magnus=(0.0, 0.0, 0.0), g=G, ball=BALL, cd_fn=None, dt=0.005) -> n…	Positions (len(t), 3) of a ball launched at x0 with velocity v0 at t[0] (RK4, step dt).
fit_flight	função	(t, xyz, model='drag', *, sigma=None, ball=BALL, g=G, observe=(0, 1, 2), cd_prior=None, l…)	Fit a flight model to tracked positions xyz ((n, 3); NaN rows are skipped) at times t.
predict	função	(fit, t, ball=BALL) -> np.ndarray	Positions of a fitted flight at times t.
kinematic_extrapolation	função	(t, xyz, t_new, degree=2) -> np.ndarray	Data-only baseline: a polynomial per axis fitted to the observed positions, evaluated at t_new.
extract_flights	função	(track, *, match_id=None, z_on=0.6, min_apex=2.5, min_duration=0.8, max_duration=4.5, max…)	Aerial flights in a ball track (frame, x, y, z, detected, e.g. :func:data.skillcorner_ball).
fisher_identifiability	função	(x0, v0, t, *, cd=0.25, magnus=(0.0, 0.0, 0.0), sigma=0.3, observe=(0, 1, 2), ball=BALL, …)	Cramer-Rao standard deviations of the flight parameters for positions observed with noise sigma (m).
rolling_fit	função	(t, xy, *, ball=BALL) -> dict[str, float]	Fit dv/dt = -mu g - k_d v^2 (rolling resistance plus drag) to a ground pass observed in 2-D.
rolling_travel_time	função	(distance, v0, decel, cd=0.0, ball=BALL)	Time for a rolling ball launched at v0 to cover distance (inf if it stops first).
travel_time	função	(distance, *, speed=None, v0=None, decel=None)	Ball travel time to a target: constant speed (the usual pitch-control assumption) or a rolling ball.

pinneapple_systems.sports.football.ball_study

Ball-flight studies: does flight physics help on real tracking data, and when is the inverse problem posed?

pinneapple_systems/sports/football/ball_study.py · 235 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
synthetic_flights	função	(n=40, *, noise=0.2, fps=10.0, seed=0, g=B.G) -> list[tuple[B.Flight, dict]]	Simulated kicks (speed 15-30 m/s, elevation 15-40 deg, drag-crisis C_D, random spin) observed at fps with Gaussian noise noise (m) on each coordinate. Returns (Flight, truth) pairs.
synthetic_recovery	função	(n=40, *, noise=0.2, seed=0) -> dict[str, Any]	Fit drag and drag_free_g to :func:synthetic_flights: error of launch speed, C_D and g, and the coverage of the Laplace 95 % intervals (should be close to 0.95 when the model is right).
identifiability	função	(speed=22.0, elevation_deg=25.0, *, fps=25.0, sigma=0.2, duration=1.6) -> dict[str, Any]	Cramer-Rao standard deviations for one lofted pass seen in 3-D versus 2-D (ball height not observed).

nome	tipo	assinatura	o que faz
`flights_from_skillcorner`	função	`(match_ids, root=None, **kw) -> list[B.Flight]`	Aerial flights of SkillCorner matches (:func:`ball.extract_flights` on :func:`data.skillcorner_ball`).
`gravity_audit`	função	`(flights, *, max_rmse=0.6, workers=1) -> dict[str, Any]`	Gravity fitted freely to each flight (vacuum: quadratic in time; drag: ODE with free `g`).
`holdout_prediction`	função	`(train, test, *, frac=0.5, train_fracs=(0.3, 0.5, 0.7), seed=0, workers=1) -> dict[str, A…)`	Observe the first `frac` of each test flight, predict the remaining positions; errors per model.
`summarise`	função	`(res) -> list[dict[str, Any]]`	Rows `model, rmse_xy, rmse_z, landing_err_median` of :func:`holdout_prediction`, best landing first.

pinneapple_systems.sports.football.data

Public football data, downloaded at run time, cached outside git with the SHA-256 of every file.

`pinneapple_systems/sports/football/data.py` · 282 linhas · testes: 16

nome	tipo	assinatura	o que faz
`MetricaMatch`	classe	`(game, t, period, players, names, ball, events, fps, gk)`	One Metrica sample match in metres (origin at the centre spot, x along the long side).
`cache_dir`	função	`(root=None) -> Path`	The football cache folder (created).
`manifest`	função	`(root=None) -> dict[str, dict[str, Any]]`	URL -> {"path", "sha256", "bytes"} of every file fetched into the cache.
`fetch`	função	`(url, rel, root=None, *, timeout=120.0) -> Path`	Download `url` once to `<cache>/rel` and record its SHA-256; return the local path.
`metrica_match`	função	`(game, root=None) -> MetricaMatch`	Metrica sample game 1 or 2: tracking of both teams, ball and events in metres (downloaded once, ~65 MB).
`skillcorner_matches`	função	`(root=None) -> list[dict]`	The ten SkillCorner open-data matches (`id`, teams, date).
`skillcorner_ball`	função	`(match_id, root=None) -> dict[str, np.ndarray]`	Ball track of a SkillCorner match at 10 Hz: `frame, t, period, x, y, z, detected` (NaN when missing).
`statsbomb_shots`	função	`(seasons=None, root=None, *, max_matches=None) -> list[dict]`	Every open-play and set-piece shot with a 360 freeze frame in the given StatsBomb seasons.

pinneapple_systems.sports.football.kin_study

Player-kinematics studies: smoothing with known truth, processing uncertainty of metabolic power on real tracking, and trajectory prediction with and without physical limits at equal data.

`pinneapple_systems/sports/football/kin_study.py` · 175 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`synthetic_runs`	função	`(n=20, T=60.0, fps=25.0, *, seed=0, v_cap=9.0, a_cap=6.0, tau=1.0, sigma_a=3.0)`	True positions, velocities and accelerations of `n` players: Ornstein-Uhlenbeck acceleration (time constant `tau`, stationary std `sigma_a`) clipped at `a_cap`, speed clipped at `v_cap`.
`synthetic_smoothing`	função	`(noise=(0.05, 0.2), fps=(25.0, 10.0), *, n=20, seed=0) -> list[dict[str, Any]]`	RMSE of speed, acceleration and metabolic power per smoothing method against the known truth.
`match_load`	função	`(match, *, team='Home', max_frames=None) -> dict[str, Any]`	Metabolic power of one team of a Metrica match per player and per method, and plausibility per method (the first `max_frames` frames only, if given). `gated_mean_mp_per_player` drops the samples above the human limits of :mod:`.kinematics` (tracking jumps), which otherwise dominate a match mean.
`prediction_samples`	função	`(match, horizon, *, every=1.0, history=1.0) -> dict[str, np.ndarray]`	Samples `(features, pos, vel, acc, target)` for every outfield player every `every` s with a complete past `history` and a tracked position `horizon` s later. States are causal Kalman estimates.
`prediction_study`	função	`(train, test, horizon, *, sizes=(100, 300, 1000, 3000, None), seeds=(0, 1, 2)) -> dict[st…)`	Final displacement error on `test` of each predictor trained on random subsets of `train`.

`pinneapple_systems.sports.football.kinematics`

Player kinematics from tracking: smoothing with uncertainty, physical plausibility, metabolic power and short-horizon trajectory prediction with and without physical limits.

`pinneapple_systems/sports/football/kinematics.py` · 300 linhas · testes: 16

nome	tipo	assinatura	o que faz
`MLPredictor`	classe	`(residual_of, project, max_iter, seed, _models)`	Data-only displacement predictor: gradient-boosted trees (scikit-learn) on :func:`history_features`.
`kalman_smooth`	função	`(xy, dt, *, sigma_pos=0.1, jerk=30.0, causal=False, max_gap=1.0) -> dict[str, np.ndarray]`	RTS-smoothed position, velocity and acceleration of tracks `xy` of shape `(T, ..., 2)`.
`fit_noise`	função	`(xy, dt, sigmas=(0.01, 0.02, 0.05, 0.1, 0.2), jerks=(5.0, 10.0, 20.0, 40.0, 80.0))`	Measurement noise and jerk density by maximum marginal likelihood of :func:`kalman_smooth` (grid search on the innovations): the data, not a guess, set how much the smoother trusts the samples.
`finite_differences`	função	`(xy, dt) -> dict[str, np.ndarray]`	Central differences of raw positions: `vel`, `acc` (the data-only baseline with no smoothing).
`savgol_derivatives`	função	`(xy, dt, window=0.5, order=2) -> dict[str, np.ndarray]`	Savitzky-Golay smoothed position, velocity and acceleration (`window` in seconds; NaN filled linearly).

nome	tipo	assinatura	o que faz
plausibility	função	(vel, acc, *, v_max=V_MAX, a_max=A_MAX) -> dict[str, float]	Share of finite samples whose speed exceeds v_max or whose acceleration magnitude exceeds a_max.
minetti_cost	função	(i)	Energy cost of running on slope i (Minetti et al. 2002), J/(kg m); i clipped to [-0.45, 0.45].
metabolic_power	função	(speed, accel_fwd, *, terrain=1.0) -> np.ndarray	Metabolic power in W/kg from speed (m/s) and forward acceleration (m/s^2), equivalent-slope model.
forward_acceleration	função	(vel, acc) -> np.ndarray	Component of the acceleration along the velocity (tangential), the input of :func:metabolic_power.
reachable_radius	função	(speed0, horizon, *, v_max=V_MAX, a_max=A_MAX)	Largest distance a player moving at speed0 can cover in horizon s (accelerate at a_max up to v_max): the physical bound on any displacement prediction.
predict_constant_velocity	função	(pos, vel, horizon)	Position after horizon s at constant velocity.
predict_limited	função	(pos, vel, acc, horizon, *, v_max=V_MAX, a_max=A_MAX, tau=0.5, dt=0.04)	Physics prediction: the current acceleration (capped at a_max) decays with time constant tau and the speed is capped at v_max (players cannot keep accelerating; a simple first-order motor model).
project_reachable	função	(pos, vel, pred, horizon, *, v_max=V_MAX, a_max=A_MAX)	Project predictions onto the reachable set: the disc around the coasting point pos + vel * horizon of radius a_max horizon^2 / 2 intersected with the disc of radius :func:reachable_radius around pos.
history_features	função	(pos_hist, ball_rel=None) -> np.ndarray	Features for the ML predictor: past positions relative to the current one ((n, k, 2) -> (n, 2k)), optionally the ball position relative to the player.
displacement_errors	função	(pred, true, pos=None, vel=None, horizon=None) -> dict[str, float]	Mean and 90th-percentile final displacement error (m); with pos, vel, horizon also the share of predictions outside the physically reachable set.

pinneapple_systems.sports.football.metrics

Scores for probabilistic binary predictions: log-loss, Brier, ROC AUC, calibration (reliability bins, ECE) and bootstrap intervals of score differences between two models on the same events.

pinneapple_systems/sports/football/metrics.py · 78 linhas · testes: 16

nome	tipo	assinatura	o que faz
log_loss	função	(p, y) -> float	
brier	função	(p, y) -> float	
auc	função	(p, y) -> float	ROC AUC by the rank-sum (Mann-Whitney) formula, ties averaged.
reliability	função	(p, y, bins=10, strategy='quantile') -> dict[str, np.ndarray]	Mean predicted probability, observed frequency and count per bin (quantile or uniform bins).

nome	tipo	assinatura	o que faz
ece	função	(p, y, bins=10) -> float	Expected calibration error with quantile bins.
score	função	(p, y) -> dict[str, float]	All scores of one model.
paired_bootstrap	função	(p_a, p_b, y, metric='log_loss', n=2000, seed=0) -> dict[str, Any]	Difference metric(a) - metric(b) on the same events with a 95 % bootstrap interval (negative: a better for losses).

pinneapple_systems.sports.football.pc_study

Pass outcome: physics-based pitch control against data-only classifiers at equal data.

pinneapple_systems/sports/football/pc_study.py · 131 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
physics_features	função	(ps, p) -> np.ndarray	Pitch-control probability, earliest attacker and defender intercept times minus the ball travel time.
fit_models	função	(train, *, seed=0, which=…) -> dict[str, Any]	Fitted models (parameters or estimators) on train.
predict_models	função	(models, ps) -> dict[str, np.ndarray]	
compare	função	(train, test, *, seed=0) -> dict[str, Any]	Scores of every model on test after fitting on train, paired bootstrap of log-loss differences against the best data-only model, and the fitted physical parameters.
data_efficiency	função	(train, test, sizes=(25, 50, 100, 200, 400, None), seeds=(0, 1, 2), which=…) -> dict[str,…)	Test log-loss of each model trained on random subsets of train (None = all passes).
with_params	função	(p, **kw) -> PC.PCParams	

pinneapple_systems.sports.football.pitch_control

Pitch control: who wins the ball at a target point, from player kinematics and ball travel time.

pinneapple_systems/sports/football/pitch_control.py · 275 linhas · testes: 16

nome	tipo	assinatura	o que faz
PCParams	classe	(tti, v_max, a_max, reaction, sigma, lam, kappa_def, gk_factor, ball, ball_speed, roll, d…)	
PassSet	classe	(att_pos, att_vel, def_pos, def_vel, att_gk, def_gk, ball, target, y, duration, meta)	Passes with the state at the kick: (n, 11, 2) attacker/defender positions and velocities (causal Kalman estimates), ball position, target (pass end), label (1 = completed to a teammate, 0 = intercepted).

nome	tipo	assinatura	o que faz
`time_to_intercept`	função	`(pos, vel, target, p) -> np.ndarray`	Time for players at `pos` with velocity `vel` (`(..., 2)`) to reach `target` (broadcastable).
`control_probability`	função	`(att_pos, att_vel, def_pos, def_vel, ball_pos, target, p=DEFAULT, att_gk=None, def_gk=Non…)`	Probability that the attacking team controls the ball at `target` (vectorised over events).
`passes_from_metrica`	função	`(match, *, sigma_pos=0.1, jerk=30.0) -> PassSet`	Completed (`PASS`) and intercepted (`BALL LOST` with an interception subtype) passes of a :class:`data.MetricaMatch`, with player states from the causal Kalman filter of :mod:`.kinematics`.
`fit_ball_travel`	função	`(ps, *, min_len=5.0) -> dict[str, Any]`	Fit the rolling-ball travel time `t(d)` (kick speed `v0 = a + b d`, constant deceleration) and a constant speed to the observed durations of the completed passes; returns both fits and their RMSE (s).
`fit_params`	função	`(ps, base, names=…, *, max_iter=200) -> tuple[PCParams, float]`	Maximum-likelihood fit of the named parameters on the pass outcomes (Nelder-Mead on log-parameters).
`geometric_features`	função	`(ps) -> tuple[np.ndarray, list[str]]`	Data-only features of a pass (no kinematics, no physics): lengths, positions, distances of the nearest attackers and defenders to the target and to the pass line, defender counts near the target.
`pitch_surface`	função	`(att_pos, att_vel, def_pos, def_vel, ball_pos, p=DEFAULT, *, nx=53, ny=34, pitch=(105.0, …)`	Attacking-team control probability on an `nx x ny` grid for one frame (for figures).

`pinneapple_systems.sports.football.plots`

Figures of the football studies (matplotlib, static PNG): pitch, pitch-control surface, ball flights in 3-D, gravity audit, reliability curves, data-efficiency curves, smoothing errors.

`pinneapple_systems/sports/football/plots.py` · 224 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`color_of`	função	`(name) -> str`	The fixed colour of a model name (grey for names outside :data:`MODEL_COLORS`).
`draw_pitch`	função	`(ax, length=105.0, width=68.0, color='#b9b7b0')`	Pitch lines in metres, origin at the centre spot.
`pitch_control_figure`	função	`(xs, ys, q, att_pos, att_vel, def_pos, def_vel, ball, target, path, title='')`	
`flights_3d`	função	`(flights, path, title='')`	`(observed xyz, fitted xyz or None, label)` per flight, in a 3-D view over the pitch plane.
`gravity_audit_figure`	função	`(audit, synthetic_g, path)`	
`reliability_figure`	função	`(rel, path, title='', models=None, lim=1.0)`	

nome	tipo	assinatura	o que faz
efficiency_figure	função	(rows, metric, path, title='', ylabel=None, models=None, flat=None)	Mean test metric against training-set size per model (seeds averaged, min-max band).
smoothing_figure	função	(rows, path, metric='mp_rmse', ylabel='metabolic power RMSE [W/kg]')	
holdout_figure	função	(res, path, models=None)	Distribution of landing errors per model (box plot).

pinneapple_systems.sports.football.xg

Expected goals with and without physics: a standard shot-geometry baseline against intercept-time features computed from the StatsBomb 360 freeze frame.

pinneapple_systems/sports/football/xg.py · 239 linhas · testes: 16

nome	tipo	assinatura	o que faz
XGParams	classe	(v_shot, v_gk, reach_gk, reaction_gk, v_def, reach_def, reaction_def, n_mouth, n_path)	
shots_arrays	função	(shots) -> dict[str, Any]	Shot table to arrays: positions in metres, label, freeze frame padded to (n, 22, 4) (x, y, mate, gk).
standard_features	função	(A) -> tuple[np.ndarray, list[str]]	
physics_features	função	(A, p=None, *, chunk=200) -> tuple[np.ndarray, list[str]]	
freeze_geometry_features	função	(A) -> tuple[np.ndarray, list[str]]	Data-only freeze-frame features (no physics): opponents inside the shot triangle, keeper distance to the goal centre and his lateral offset from the shot line, nearest opponent distance to the shooter.
select_params	função	(A_train, grid_v_shot=(12.0, 16.0, 20.0, 25.0, 30.0), grid_v_gk=(3.0, 5.0, 7.0), folds=5)…)	Choose v_shot and v_gk by grouped cross-validated log-loss of a logistic model on the standard plus physics features, on the training shots only.
compare	função	(A_train, A_test, *, params=None, seed=0) -> dict[str, Any]	Test scores of: standard logistic / GBM, physics-only logistic, standard+physics logistic / GBM, standard plus data-only freeze-frame geometry (with and without physics); StatsBomb's own xG on the same shots as an external reference (a proprietary model trained on far more shots).
data_efficiency	função	(A_train, A_test, params, sizes=(100, 200, 400, 800, None), seeds=(0, 1, 2, 3, 4))	Test log-loss of the standard and standard+physics logistic models trained on random subsets.

pinneapple_systems.time_series

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/__init__.py` · 108 linhas · testes: 90

pinneapple_systems.time_series.adapters

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/adapters/__init__.py` · 1 linhas · **sem teste direto**

pinneapple_systems.time_series.adapters.shape

Shape adapters for time series models (e.g. LastStepToHorizon).

`pinneapple_systems/time_series/adapters/shape.py` · 14 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LastStepToHorizon`	classe	`(model, horizon)`	

pinneapple_systems.time_series.adaptive

Adaptive forecasting: switch between and combine forecasting models online, using only past out-of-sample errors.

`pinneapple_systems/time_series/adaptive.py` · 447 linhas · testes: 9

nome	tipo	assinatura	o que faz
`SimpleExpSmoothing`	classe	`(alpha, level_)`	
`HoltDamped`	classe	`(alpha, beta, phi, level_, trend_)`	Holt's linear trend with damping phi (Gardner & McKenzie 1985).
`HoltWintersAdditive`	classe	`(season_length, alpha, beta, gamma, level_, trend_, seasonal_, n_)`	
`Theta`	classe	`(alpha, level_, slope_, n_)`	Theta method as SES with drift equal to half the regression slope (Assimakopoulos & Nikolopoulos 2000; Hyndman & Billah 2003, Int. J. Forecasting 19:287).
`MovingAverage`	classe	`(window, value_)`	
`RidgeAR`	classe	`(p, window, ridge, coef_, tail_)`	Autoregression of order p with an intercept, ridge-regularised, fitted on the last `window` points; multi-step forecasts are recursive.
`LagRegressorExpert`	classe	`(estimator, n_lags=12, refit_every=24, window=500)`	Any tabular regressor (scikit-learn style `fit(X, y)` / `predict(X)`, including the ML forecasters of `pinneapple_systems.time_series.models`) as an expert: lagged values as features, recursive multi-step forecasts. The estimator is refitted every `refit_every` calls (on the last `window` ...
`AdaptiveRun`	classe	`(origins, forecast, expert_forecasts, lower, upper, weights, active, expert_names, y)`	Result of :meth:`AdaptiveForecaster.run`: one row per forecast origin t (forecasts for t+1..t+H).

nome	tipo	assinatura	o que faz
`AdaptiveForecaster`	classe	`(experts, horizon=1, mode='combine', etas=(0.5, 2.0, 8.0), alphas=(0.0, 0.01, 0.05), loss…)`	Online selection and combination of forecasting models.
`default_experts`	função	`(season_length=1) -> dict[str, Any]`	A diverse, cheap pool. Hyper-parameter variants are separate experts, so choosing a smoothing constant is part of the online selection instead of an in-sample fit.

pinneapple_systems.time_series.audit

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/audit/__init__.py` · 4 linhas · **sem teste direto**

pinneapple_systems.time_series.audit.report

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/audit/report.py` · 28 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`AuditSection`	classe	`(name, results)`	
`AuditReport`	classe	`(meta, sections)`	Serializable container for time-series diagnostic outputs.

pinneapple_systems.time_series.audit.tests

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/audit/tests.py` · 315 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TSAuditor`	classe	`(nlags, arch_lags)`	
`adf_test`	função	`(y, *, autolag='AIC', min_n=20) -> Dict[str, Any]`	
`kpss_test`	função	`(y, *, regression='c', nlags='auto', min_n=20) -> Dict[str, Any]`	
`pp_test`	função	`(y, min_n=20) -> Dict[str, Any]`	Phillips–Perron test.
`arch_test`	função	`(y, *, lags=12, min_n=30) -> Dict[str, Any]`	
`acf_pacf`	função	`(y, *, nlags=40, min_n=50) -> Dict[str, Any]`	
`chow_test_mean_shift`	função	`(y, break_idx, min_n=20) -> Dict[str, Any]`	Simple Chow test for a mean shift using an intercept-only model.
`changepoints_ruptures`	função	`(y, *, model='l2', max_breaks=5, min_n=50) -> Dict[str, Any]`	Multi-break approximation via `ruptures` (if installed).

pinneapple_systems.time_series.base

Base TSOutput and TSModelBase for time series models.

`pinneapple_systems/time_series/base.py` · 26 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`TSOutput`	classe	`(y_hat, extras)`	Standard output for time series model predictions.
`TSModelBase`	classe	`()`	Base opcional para modelos TS.

`pinneapple_systems.time_series.baselines`

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/baselines/__init__.py` · 2 linhas · **sem teste direto**

`pinneapple_systems.time_series.baselines.naive`

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/baselines/naive.py` · 71 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`NaiveForecaster`	classe	`(last_value_)`	Predicts the last observed value for all future steps.
`SeasonalNaiveForecaster`	classe	`(season_length, history_)`	Repeats the value from the same season in the last cycle.
`DriftForecaster`	classe	`(first_, last_, n_)`	Linearly extrapolates from the first to the last training point.

`pinneapple_systems.time_series.classical_wrappers`

Real wrappers around the classical statistical forecasting libraries.

`pinneapple_systems/time_series/classical_wrappers.py` · 244 linhas · testes: 10

nome	tipo	assinatura	o que faz
`StatsmodelsARIMAForecaster`	classe	`(order, seasonal_order, trend, enforce_stationarity, enforce_invertibility, fit_kwargs)`	Real ARIMA/SARIMA forecaster wrapping `statsmodels`.
`AutoARIMAForecaster`	classe	`(seasonal, m, max_p, max_q, max_d, max_P, max_Q, max_D, stepwise, suppress_warnings, auto…)`	Real forecaster wrapping `pmdarima.auto_arima`.

`pinneapple_systems.time_series.csv_loader`

CSV → tensor loader for time series datasets.

`pinneapple_systems/time_series/csv_loader.py` · 322 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TimeSeriesScaler`	classe	`() -> None`	Channel-wise z-score scaler: fit on training data, apply to all splits.
`load_time_series_csv`	função	`(path, *, target_cols=None, feature_cols=None, time_col=None, fill_method='ffill', normal…)`	Load a CSV file and return train / val / test tensors ready for PINNeAPPle.
`generate_synthetic_csv`	função	`(path, T=1000, seed=42, freq='D') -> Path`	Generate a synthetic multivariate time series CSV and save it to *path*.

pinneapple_systems.time_series.datamodule

Time series DataModule with deterministic, time-ordered splits and index-based loaders.

`pinneapple_systems/time_series/datamodule.py` · 80 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TSDataModule`	class e	`(series, spec, batch_size, val_ratio, num_workers, pin_memory)`	

pinneapple_systems.time_series.datasets

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/datasets/__init__.py` · 1 linhas · **sem teste direto**

pinneapple_systems.time_series.datasets.public

Openly licensed real series, downloaded from a pinned commit and checked against a SHA-256.

`pinneapple_systems/time_series/datasets/public.py` · 183 linhas · testes: 8

nome	tipo	assinatura	o que faz
`PublicSeries`	classe	`(name, time, values, source)`	
`YouTubeDaily`	classe	`(dates, n_entries, total_views, total_likes, category_counts, source)`	One row per trending day: the date, the number of trending entries, their total views and likes, and the number of entries per YouTube category id.
`fetch_pinned`	função	`(key, cache_dir=None, *, local=None) -> bytes`	Bytes of the pinned file `key` (from `local`, the cache, or the network), checksum verified.
`load_usd_eur_daily`	função	`(cache_dir=None) -> PublicSeries`	U.S. dollars per euro on business days (days without a quote are dropped).
`load_sunspots_yearly`	função	`(cache_dir=None) -> PublicSeries`	
`log_returns`	função	`(values) -> np.ndarray`	
`youtube_daily_aggregates`	função	`(csv_text) -> YouTubeDaily`	Aggregate the rows of a trending-videos CSV (columns `trending_date` as yy.dd.mm, `category_id`, `views`, `likes`) to one row per trending day, in date order.
`load_youtube_trending_us`	função	`(cache_dir=None) -> YouTubeDaily`	

pinneapple_systems.time_series.datasets.windowed

Windowed time series dataset for input/horizon pairs.

`pinneapple_systems/time_series/datasets/windowed.py` · 41 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WindowedTimeSeriesDataset`	classe	`(series, spec)`	series: (T, F) x: (L_in, F) y: (H, C) (aqui C = F por padrão)

pinneapple_systems.time_series.decomposition

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/decomposition/__init__.py` · 53 linhas · **sem teste direto**

pinneapple_systems.time_series.decomposition.fft_forecaster

Pure FFT forecaster: fits dominant sinusoids and extrapolates.

`pinneapple_systems/time_series/decomposition/fft_forecaster.py` · 86 linhas · testes: 151

nome	tipo	assinatura	o que faz
`FFTForecaster`	classe	`(n_harmonics=10, detrend=True)`	Fits a signal using its top-K dominant frequency components (via FFT) and extrapolates them forward.

pinneapple_systems.time_series.decomposition.fft_lstm

Hybrid FFT decomposition + LSTM residual forecaster (multivariate-ready).

`pinneapple_systems/time_series/decomposition/fft_lstm.py` · 335 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FFTDecomposer`	classe	`(n_harmonics=10, detrend=True, auto_harmonics=True, auto_threshold=0.05) -> None`	Fits one FFTForecaster per channel; computes residuals and extrapolations.
`ResidualLSTMConfig`	classe	`(input_len, horizon, n_features, n_targets, hidden_size, num_layers, dropout, layer_norm)`	Hyperparameters for ResidualLSTMForecaster.
`ResidualLSTMForecaster`	classe	`(cfg=None, **kw)`	LSTM forecaster trained on FFT residuals.
`FFTLSTMPipeline`	classe	`(*, n_harmonics=10, detrend=True, auto_harmonics=True, auto_threshold=0.05, input_len=64,…)`	Orchestrates the two-stage hybrid FFT + LSTM pipeline.

pinneapple_systems.time_series.decomposition.fft_nn

FFT + Neural Network on residual decomposition forecaster.

`pinneapple_systems/time_series/decomposition/fft_nn.py` · 100 linhas · testes: 151

nome	tipo	assinatura	o que faz
`FFTNNForecaster`	classe	`(n_harmonics=10, detrend=True, input_len=32, horizon=16, nn_model=None, nn_epochs=50, nn_…)`	Two-stage forecaster: 1. FFT component captures global periodicity / trend. 2. A neural network (default: MLP) models the residual.

pinneapple_systems.time_series.decomposition.hht_lstm

Hilbert-Huang Transform (EMD-based) + LSTM residual forecaster (multivariate-ready).

`pinneapple_systems/time_series/decomposition/hht_lstm.py` · 341 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HHTDecomposer`	classe	`(n_imfs=6, dominant_energy_ratio=0.8, detrend=True) -> None`	Fits one EMD decomposer per channel; computes residuals and extrapolations.
`HHTLSTMPipeline`	classe	`(n_imfs=6, dominant_energy_ratio=0.8, detrend=True, input_len=64, horizon=16, n_features=…)`	Orchestrates HHTDecomposer + LSTM residual forecasting.

pinneapple_systems.time_series.decomposition.hht_nn

Hilbert-Huang Transform + Neural Network on residual decomposition forecaster.

`pinneapple_systems/time_series/decomposition/hht_nn.py` · 172 linhas · testes: 151

nome	tipo	assinatura	o que faz
`HHTNNForecaster`	classe	`(n_imfs=5, input_len=32, horizon=16, nn_model=None, nn_epochs=50, nn_lr=0.001)`	Two-stage forecaster: 1. EMD (Empirical Mode Decomposition) decomposes the signal into IMFs. Dominant IMFs are extrapolated via Hilbert instantaneous frequency. 2. A neural network models the residual (sum of discarded IMFs + trend).

pinneapple_systems.time_series.eda

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/eda/__init__.py` · 26 linhas · **sem teste direto**

pinneapple_systems.time_series.eda.plots

10 EDA tools for time series analysis.

`pinneapple_systems/time_series/eda/plots.py` · 463 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_trend`	função	`(y, *, window=24, title='Trend Analysis', figsize=(12, 6), show=False) -> Figure`	Raw series + rolling mean + rolling std band.
`track_setpoint`	função	`(actual, setpoint, *, tolerance=0.05, title='Setpoint Tracking', figsize=(12, 5), show=Fa…)`	Actual vs setpoint with deviation shading and tolerance band.
`rolling_statistics`	função	`(y, windows=(8, 24, 96), *, stats=('mean', 'std'), title='Rolling Statistics', figsize=(1…)`	Multi-window rolling statistics overlay.
`step_response`	função	`(y, step_idx=None, *, title='Step Response', figsize=(12, 5), show=False) -> Figure`	Visualize system response to a detected or specified step. Estimates: steady-state gain, rise time (10%→90%), settling time (±2%).

nome	tipo	assinatura	o que faz
`power_spectrum`	função	`(y, *, fs=1.0, n_top=5, title='FFT Power Spectrum', figsize=(12, 5), show=False) -> Figure`	Log-scale FFT power spectrum with top-N dominant frequencies annotated.
`plot_acf_pacf`	função	`(y, nlags=40, *, alpha=0.05, title='ACF / PACF', figsize=(12, 5), show=False) -> Figure`	Side-by-side ACF and PACF with confidence bounds.
`stationarity_report`	função	`(y, *, title='Stationarity Tests', figsize=(10, 4), show=False) -> Figure`	Runs ADF, KPSS, and PP tests and renders a summary table.
`changepoint_plot`	função	`(y, n_bkps=3, *, model='rbf', title='Change Point Detection', figsize=(12, 5), show=False…)`	Visualize change points detected via the ruptures library.
`cross_correlation`	função	`(df, cols=None, max_lag=20, *, title='Cross-Correlation', figsize=(12, 6), show=False) ->…)`	Heatmap of peak cross-correlation values and lag at peak between all column pairs.
`rga_matrix`	função	`(K, mv_labels=None, cv_labels=None, *, title='Relative Gain Array (RGA)', figsize=(8, 6),…)`	Compute and visualize the Bristol Relative Gain Array.

`pinneapple_systems.time_series.enso`

ENSO prediction: data, physical models, physics-informed and data-driven forecasters, evaluation.

`pinneapple_systems/time_series/enso/__init__.py` · 2 linhas · testes: 22

`pinneapple_systems.time_series.enso.data`

Openly licensed ENSO observations, downloaded once, reduced to monthly tropical-Pacific products and cached with the SHA-256 of every source file.

`pinneapple_systems/time_series/enso/data.py` · 674 linhas · testes: 22

nome	tipo	assinatura	o que faz
`Manifest`	classe	`(path, expected=None)`	URL -> {sha256, bytes} of every source file of a product. `expected` (an earlier manifest) pins hashes.
`Gridded`	classe	`(time, lat, lon, values, source)`	Monthly fields (nt, ny, nx) on a regular lat/lon grid (cell centres, lon in 0-360).
`manifests`	função	`(cache_dir=None) -> dict[str, Path]`	The manifests written so far in the cache: product -> path of `manifest_<product>.json`.
`fetch`	função	`(url, *, timeout=300, retries=4) -> bytes`	Bytes at `url` (through the environment's proxy settings), retried with exponential backoff.
`month_range`	função	`(start, end) -> list[str]`	'YYYY-MM' strings from `start` to `end` inclusive.

nome	tipo	assinatura	o que faz
oisst_monthly	função	(start='1981-09', end=None, *, lat=(-30.0, 30.0), lon=(100.0, 300.0), res=1.0, cache_dir=…)	Monthly mean OISST v2.1 SST [deg C] in res-degree cells (a multiple of 0.25) over lat x lon.
box_mean	função	(g, var, box) -> np.ndarray	cos(latitude)-weighted mean of g.values[var] over cells whose centres lie in box.
nino_indices	função	(g, var='sst') -> dict[str, np.ndarray]	Nino 3.4, 3, 4 and 1+2 box means (not anomalies) of g.
coarsen	função	(g, ky, kx) -> Gridded	Block means of ky x kx cells (NaN-aware; trailing cells that do not fill a block are dropped).
anomalies	função	(x, time, base) -> tuple[np.ndarray, np.ndarray]	Anomalies from the calendar-month climatology of the period base = ('YYYY-MM', 'YYYY-MM') inclusive.
d20_from_profile	função	(z, T, *, iso=20.0, max_gap=60.0) -> float	Depth [m] where the profile first crosses iso going down (linear interpolation between the bracketing levels), or NaN when the profile does not start above iso within the top 30 m, does not reach below it, or the bracketing levels are more than max_gap m apart.
fetch_to_file	função	(url, path, *, timeout=1800, retries=4, chunk=1 << 22) -> str	Stream url to path and return its SHA-256 (large files never sit in memory).
wod_d20_casts	função	(years, kinds=('mrb',), *, lat=(-8.0, 8.0), lon=(120.0, 280.0), cache_dir=None, manifest=…)	D20 of every quality-controlled WOD cast (flags 0) in the box, from the yearly files wod_<kind>_<year>.nc (kind in mrb = moored buoys, xbt, pfl = profiling floats, ctd, ...).
d20_monthly	função	(casts, months, *, lat=(-5.0, 5.0), lon_edges=tuple(range(130, 281, 10)), min_casts=1, da…)	Monthly mean D20 [m] per longitude bin (len(lon_edges) - 1 bins) for casts with latitude in lat; NaN where a bin has fewer than min_casts casts. Each cast counts once (a mooring reporting daily weighs as much as 30 XBT drops); daily_per_platform is kept for the record (WOD moored …
d20_climatology	função	(casts, lon_edges, lat_edges, base) -> np.ndarray	Mean D20 [m] of the casts in each (lat, lon) bin over the months of base (each calendar month weighted equally); NaN where a bin has no cast. Shape (len(lat_edges) - 1, len(lon_edges) - 1).
era5_monthly	função	(variables=…, *, cache_dir=None, manifest=None, workers=8, derived=True, log=print) -> Gr…)	Monthly means of 6-hourly ERA5 fields on the 64 x 32 WeatherBench 2 grid, 1959-01 to 2022-12, global.
wind_stress	função	(u, v, rho_air=1.2, cd=0.0013)	Bulk wind stress tau = rho_air C_D |U| U [N m-2] from 10 m wind components [m/s] (constant C_D; Large & Pond 1981 give 1.1e-3 to 1.5e-3 for 4-11 m/s).

nome	tipo	assinatura	o que faz
soi_from_mslp	função	(g, var='mean_sea_level_pressure', base=None) -> np.ndarray	Tahiti (17.5S, 149.6W) minus Darwin (12.4S, 130.9E) sea-level pressure, each standardised per calendar month over base (default: the whole record), the difference standardised again (the CPC/BoM construction) -- taken at the nearest cells of g, so on a coarse grid it is a proxy of the …
ncei_heat_content	função	(box=(-5.0, 5.0, 120.0, 280.0), *, cache_dir=None, manifest=None) -> dict[str, Any]	Sum of the NCEI 0-700 m heat-content anomaly [1e18 J] over the 1-deg cells in box, 3-month means (JFM, AMJ, JAS, OND) 1955-2020; mid_month is the centre month 'YYYY-MM' of each season.
statsmodels_nino12	função	() -> dict[str, Any]	Monthly Nino 1+2 SST [deg C] 1950-2010 as shipped with statsmodels (NOAA CPC); time as 'YYYY-MM'.

pinneapple_systems.time_series.enso.dataset

The ENSO study data set: monthly indices and maps built from the cached sources of :mod:.data.

pinneapple_systems/time_series/enso/dataset.py · 226 linhas · testes: 22

nome	tipo	assinatura	o que faz
EnsoDataset	classe	(months, indices, sst_maps, grid, d20_band, d20_eq, d20_lon, d20_observed, taux, base, cl…)	
eof_fill	função	(X, base_rows, n_eofs=4, ridge=0.1) -> np.ndarray	Fill NaNs of each row of X (nt, nb) from its observed entries: least-squares amplitudes of the leading n_eofs EOFs (computed on the complete rows among base_rows), ridge-regularised in units of the EOF variances. Observed entries are kept as they are. Rows with fewer observed entries than …
build_dataset	função	(start='1982-01', end='2024-12', base=('1982-01', '2011-12'), grid=None, *, cache_dir=Non…)	
to_series	função	(ds, columns=COLUMNS, name='enso')	:class:~pinneapple_systems.time_series.forecast_lab.SeriesData with columns (target = first) and the maps in meta (month_of_year, sst_maps, d20_band, d20_eq, d20_lon, taux, grid, months).

pinneapple_systems.time_series.enso.evaluation

ENSO-specific scores on top of the forecast_lab ones: anomaly correlation per lead, skill stratified by target calendar month (the spring predictability barrier), categorical El Nino / neutral / La Nina verification, and a blocked cross-validated hindcast that gives every year an out-of-sample …

pinneapple_systems/time_series/enso/evaluation.py · 96 linhas · testes: 22

nome	tipo	assinatura	o que faz
acc	função	(mean, y) -> list[float]	Anomaly correlation (Pearson, over origins) per horizon column.
by_target_month	função	(mean, y, origins, month_of_year, horizons, min_n=6) -> dict[str, Any]	ACC and RMSE for each (target calendar month, horizon); NaN where fewer than min_n forecasts.
category	função	(x, threshold=0.5) -> np.ndarray	-1 La Nina (x <= -threshold), 0 neutral, +1 El Nino (x >= threshold).
categorical	função	(mean, y, threshold=0.5) -> dict[str, Any]	Per horizon: 3 x 3 contingency table (rows observed La Nina / neutral / El Nino, columns forecast), hit rate (probability of detection) of each category, false-alarm ratio of the warm and cold categories, accuracy, and the Heidke skill score (accuracy relative to random forecasts with the same ...
blocked_hindcast	função	(make, data, horizons, *, block=48, first=36, last=None, val_frac=0.15) -> dict[str, Any]	Out-of-sample forecasts for every origin in [first, last]: the origins are cut into blocks of block months; each block is forecast by a model fitted on all other origins whose targets end more than max(h) before the block or start after it (purged on both sides), with its last val_frac (in ...

pinneapple_systems.time_series.enso.forecasters

ENSO forecasters with the :class:~pinneapple_systems.time_series.forecast_lab.Forecaster interface, so the forecast_lab protocol (purged walk-forward tuning, untouched test period, DM tests) applies to them unchanged.

pinneapple_systems/time_series/enso/forecasters.py · 683 linhas · testes: 22 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ColumnSubset	classe	(inner, cols, name=None, family=None)	Wrap inner so that it sees only cols (names in data.meta["columns"]; the target must be one).
RechargeOscillatorForecaster	classe	(h_col='wwv', seasonal=(), multiplicative=False, stochastic=True, n_members=200, sub=4, s…)	Jin's recharge oscillator on (T, h) = (target column, h_col), fitted on the training segment.
LIMForecaster	classe	(n_eofs=10, extra_cols=('wwv',), tau0=1, name=None)	Linear inverse model dx/dt = L x + noise on x = (leading n_eofs PCs of the SST anomaly maps, the extra_cols indices), each standardised; L = log(C(tau0) C(0)^-1) / tau0 from the training segment; the target is read out by least squares on x. Forecast mean a . exp(L h) x(t), variance a . ...
MapCNNForecaster	classe	(n_lags=3, pool=2, scalar_cols=('wwv', 'h_west', 'h_east'), channels=(16, 32), hidden=64,…)	CNN on the last n_lags monthly SST anomaly maps (coarsened by pool) plus scalars (scalar_cols at the origin and the sine / cosine of the origin month); outputs the standardised change of the target at every horizon (persistence = zero output). n_members seeds are averaged.

nome	tipo	assinatura	o que faz
IntermediateModelForecaster	classe	(init='nudged', n_members=1, enkf_members=24, coupling=1.0, eps_a=0.1, c=2.7, r_days=900.…)	The coupled intermediate model fitted on the training segment (:func:fit_intermediate).
EnsembleForecaster	classe	(members, n_draw=100, seed=0, name=None, family=None)	Equal-weight mixture of fitted-together members: mean = mean of the members' means; samples = n_draw draws from each member's distribution (its samples, or its Gaussian).
box_index	função	(maps, grid, box=(-5.0, 5.0, 190.0, 240.0)) -> np.ndarray	cos-latitude weighted box mean of maps (..., ny, nx) on grid (default the Nino 3.4 box).
ro_rhs	função	(x, R=-0.05, F1=0.02, F2=1.5, eps=0.03)	Constant-coefficient recharge oscillator vector field on (T, h) for the forecast_lab ODE forecasters.
fit_intermediate	função	(meta, end, *, eps_a=0.1, c=2.7, r_days=900.0, coupling=1.0, window=4, ridge=0.001, L_T=6…)	Fit the coupled model to the rows [0, end) of the study data (meta of :func:.dataset.to_series).

pinneapple_systems.time_series.enso.intermediate

An intermediate coupled model of the tropical Pacific in the spirit of Zebiak & Cane (1987, Mon. Wea. Rev. 115:2262), simplified, and the checks that every component is solved correctly.

pinneapple_systems/time_series/enso/intermediate.py · 607 linhas · testes: 22

nome	tipo	assinatura	o que faz
OceanGrid	classe	(lon0, lon1, dlon, lat_max, dlat)	C-grid: h and T at cell centres (ny, nx); u on the nx + 1 west/east faces; v on the ny + 1 south/north faces.
ShallowWaterOcean	classe	(grid=None, c=2.7, H=150.0, r_days=900.0, sponge_deg=5.0, sponge_days=5.0, dtype='float64…)	Linear reduced-gravity shallow water on the equatorial beta-plane (dimensional, SI units).
GillAtmosphere	classe	(eps=0.1, c_a=60.0, n_modes=24, dlon=2.5, lon_offset=0.0)	Gill (1980) long-wave response in Matsuno units (y in L_a = sqrt(c_a / beta), x in L_a, Q per unit heating).
CoupledParams	classe	(eps_T, gamma, kappa, mu, mu_y, L_T, noise_tau, noise_months, noise_lon, extra)	Parameters of :class:CoupledModel (SST coefficients per ocean-grid longitude, in months).
CoupledModel	classe	(params, ocean=None, atmosphere=None, dt_hours=8.0, atm_every=3, closure=None, atm_lat=40…)	Linear coupled ocean-atmosphere anomaly model on :class:OceanGrid; time in months for the SST equation and seconds inside the ocean step. closure(T, h) (optional) adds a learned SST tendency [K/month] -- the hybrid models train it.

nome	tipo	assinatura	o que faz
`gill_analytic`	função	`(x, y, L, eps, period) -> dict[str, np.ndarray]`	Closed-form long-wave Gill response on a periodic circle of length `period` (Matsuno units) to Q = F(x) exp(-y^2 / 2) with F = cos(pi x / (2 L)) for |x| < L, else 0. Kelvin part q0 = -int G_K(x - s) F(s) ds, Rossby part q2 = -1/2 int G_R(s - x) F(s) ds with the periodic Green's functions G(d) = ...
`sst_tendency`	função	`(T, h, taux_c, eps_T, gamma, kappa, S)`	dT/dt [K/month] = -eps_T T + gamma S h + kappa S tau_x (T, h, tau_x at cell centres; gamma, kappa per longitude; S the meridional structure (ny, 1)).
`kelvin_wave_test`	função	`(grid=None, c=2.7, dt_hours=6.0, days=40.0, sigma_km=1500.0) -> dict`	Release an equatorial Kelvin wave (h = exp(-y^2 / 2 L^2 - (x - x0)^2 / 2 s^2), u = g' h / c, v = 0) in the western basin without damping; its speed is the slope of the tracked equatorial h peak. Theory: c; basin crossing time = basin width / c.
`rossby_wave_test`	função	`(grid=None, c=2.7, dt_hours=6.0, days=150.0, sigma_km=2500.0) -> dict`	Release a long n = 1 Rossby wave (Matsuno structure u ~ (2 y^2 - 3), h ~ (2 y^2 + 1) times exp(-y^2 / 2) in units of L = sqrt(c / beta)) in the eastern basin; its h centroid moves west at c / 3 in the long-wave limit (finite width makes it slightly slower).
`gill_test`	função	`(nlons=(72, 144, 288, 576), eps=0.1, L=2.0, n_modes=12) -> dict`	Numerical Gill response to Q = cos(pi x / 2L) exp(-y^2 / 2) on |x| < L against :func:`gill_analytic`, for several longitude resolutions (relative L2 error of u, v, p over the band |y| < 3).
`convergence_study`	função	`(factors=(1, 2, 4), c=2.7) -> dict`	Kelvin-wave speed error under joint refinement of dx, dy and dt (factor f: dlon = 5/f deg, dlat = 2/f deg, dt = 12/f h).
`forced_ocean_run`	função	`(ocean, taux, *, steps_per_month=91, tauy=None, state=None) -> tuple[np.ndarray, dict]`	Integrate the ocean alone under prescribed monthly stress `taux` (months, ny, nx + 1) [N m-2] (and `tauy` at v points); returns the monthly mean h (months, ny, nx) and the final state. NaN stress months are zero.

`pinneapple_systems.time_series.enso.oscillators`

Conceptual ENSO models: the recharge oscillator and the delayed oscillator.

`pinneapple_systems/time_series/enso/oscillators.py` · 264 linhas · testes: 22

nome	tipo	assinatura	o que faz
`ROParams`	classe	`(coef, sigma_T, sigma_h, B, info)`	Recharge-oscillator coefficients per month: `coef[name] = (c0, c1, c2)` (annual harmonics; c1 = c2 = 0 for a constant coefficient), noise amplitudes per sqrt(month), multiplicative noise B [1/K].
`ro_eigen`	função	`(R, F1, F2, eps) -> dict[str, float]`	Eigenvalues of the constant-coefficient recharge oscillator: growth rate [1/month], period [months] (inf when the eigenvalues are real) and the two eigenvalues.
`ro_coefficients`	função	`(p, month) -> dict[str, np.ndarray]`	Coefficients at (fractional) calendar month(s) `month` (0 = mid-January... any real number, period 12).
`ro_tendency`	função	`(p, X, month) -> np.ndarray`	Deterministic tendency (dT/dt, dh/dt) of states X (..., 2) at calendar month(s) `month`.

nome	tipo	assinatura	o que faz
simulate_recharge	função	(p, x0, n_months, *, month0=0.0, sub=10, n_members=1, stochastic=True, seed=0, method='rk…)	Integrate from x0 ((2,) or (n, 2) states) for n_months; returns (n_members, n, n_months + 1, 2) samples at month ends. Deterministic part: RK4 (method="rk4") or Euler; noise: Euler-Maruyama with sub steps per month (Ito; the multiplicative factor is clipped at zero). month_offsets …
fit_recharge	função	(T, h, month=None, *, seasonal=(), multiplicative=False, lead=1, sub=4) -> ROParams	Recharge oscillator fitted to monthly series T, h (pairs with a NaN are skipped).
do_rhs	função	(x, xt, a=1.0, b=0.75, c=1.0)	Battisti-Hirst / Suarez-Schopf delayed oscillator dT/dt = a T - b T(t - tau) - c T^3.
do_roots	função	(a, b, tau, branches=(0, -1, 1)) -> list[complex]	Roots of the linear characteristic equation lambda = a - b exp(-lambda tau) on Lambert-W branches, sorted by decreasing real part.
simulate_delayed	função	(a, b, c, tau, history, n_steps, dt, method='rk4') -> np.ndarray	Method of steps on a uniform grid (tau a multiple of dt): history holds the last tau / dt + 1 values (oldest first); returns the n_steps new values. RK4 uses the mean of the delayed values at the half step (second order in the delayed argument) unless the grid is refined.
oscillation_stats	função	(x, dt, *, skip=0) -> dict[str, float]	Period (mean spacing of upward zero crossings, linearly interpolated) and growth rate (slope of log |peak| over successive extrema) of a linear oscillation sampled every dt.

pinneapple_systems.time_series.enso.pinn

Physics-informed networks for ENSO, trained with :class:pinneapple_neural.trainer.inverse_pinn.InversePINN (network weights and unknown physical parameters optimised together; data misfit plus ODE/PDE residual at random collocation points, adaptive weighting).

pinneapple_systems/time_series/enso/pinn.py · 626 linhas · testes: 22

nome	tipo	assinatura	o que faz
FourierMLP	classe	(in_dim, out_dim, hidden=64, layers=3, n_freq=8, out_scale=None, freq_max=None)	MLP on [x, sin(2 pi k x_j), cos(2 pi k x_j)] (k = 1..n_freq for each input j, inputs scaled to [0, 1]), tanh hidden layers; out_scale multiplies the outputs.
GroupedMLP	classe	(groups, in_dim, out_dim, hidden=32, layers=2, n_freq=4, freq_max=2.0, out_scale=None, se…)	G independent Fourier MLPs evaluated on the same inputs: (B, in) -> (B, G * out) (group-major).

nome	tipo	assinatura	o que faz
PINNRechargeForecaster	classe	(h_col='wwv', seasonal=(), closure=False, stochastic=False, n_members=200, epochs=4000, s…)	Forecasts by integrating the recharge oscillator identified by :func:identify_recharge on the training segment (closure: with the learned thermocline-feedback network). stochastic: ensemble with the noise of the one-month residuals; otherwise deterministic with a calibrated Gaussian …
PINNExtrapolationForecaster	classe	(h_col='wwv', window=24, epochs=3000, n_col=512, hidden=32, chunk=256, seed=0, name=None)	For each origin t: a PINN X(tau), tau = (s - t) / window in [-1, max(h) / window], fitted to the observed (T, h) of the last window months and to the recharge-oscillator residual (coefficients fitted by least squares on the training segment, held fixed) on the whole interval; the forecast at t …
EquatorialWavePINN	classe	(lon_obs, h_obs, T_obs, lon_f, X0, X2, tau_eq, *, lon_w=120.0, lon_e=280.0, H=150.0, sst=…)	PINN for the two-mode long-wave equatorial ocean plus an equatorial SST equation (see the module docstring).
identify_recharge	função	(T, h, month=None, *, seasonal=(), closure=False, epochs=4000, n_col=2048, hidden=64, lay…)	PINN identification of the recharge oscillator on monthly series (T, h) of length n.
project_forcing	função	(taux, lat, c=2.7, H=150.0, rho=1025.0) -> tuple[np.ndarray, np.ndarray]	Projections X0, X2 of X = tau_x / (rho H) [m s-2] (..., nlat, nx) on phi_0 = exp(-y^2/2) and phi_2 = (4 y^2 - 2) exp(-y^2/2), y = lat / L with L = sqrt(c / beta) (norms sqrt(pi) and 8 sqrt(pi)).

pinneapple_systems.time_series.enso.pipeline

Study-level steps of the ENSO study, each returning JSON-able results (the example script only calls these).

pinneapple_systems/time_series/enso/pipeline.py · 419 linhas · testes: 22

nome	tipo	assinatura	o que faz
solver_checks	função	() -> dict[str, Any]	
enso_scores	função	(merged, month_of_year, threshold=0.5) -> dict[str, Any]	ACC per lead, target-month stratified ACC / RMSE and categorical scores of every model in merged.
case_panels	função	(parts, merged, data, *, families=…, events=None, months_before=18, leads=(1, 3, 6, 9, 12…)	Plume panels: for every case-study origin, the representative of each family (from merged) with its quantiles at the evaluated leads. parts[name]["_cases"] holds forecasts from origins after the development period; events[label][name] (from :func:.study.event_hindcast) those of …
skill_against	função	(merged, reference='climatology', alpha=0.05) -> dict[str, Any]	RMSE skill 1 - RMSE / RMSE(reference) on the test period for every model of merged (with its kept forecasts), and one-sided Diebold-Mariano tests (squared errors, HLN, long-run variance over the h-step overlap) of each non-baseline model against the reference, Holm-adjusted across those models …

nome	tipo	assinatura	o que faz
`spring_barrier_hindcasts`	função	`(data, prot, makers, *, block=48) -> dict[str, Any]`	Blocked, purged cross-validated hindcasts over the development period for each maker; returns, per model, the target-month stratified ACC / RMSE and the ACC per lead.
`phase_portrait`	função	`(data, prot, *, seasonal=('R', 'F1', 'F2', 'eps'), seed=0, onsets=…, n_path=48) -> dict[s…)`	Observed (h, T) over the record (and its 3-month running mean), the deterministic paths of the fitted recharge oscillators (constant and seasonal) for `n_path` months from the observed state at each El Nino onset in `onsets`, and one stochastic realisation of each with the record's length; …
`shallow_water_pinn_study`	função	`(ds, window=('1996-01', '1999-12'), *, epochs=6000, n_col=4096, pde_weight=100.0, spin_mo…)`	The equatorial-wave PINN (:class:`.pinn.EquatorialWavePINN`) on (a) synthetic equatorial h from the 2-D shallow-water model forced by ERA5 stress (known c = 2.7 m/s, r = 1/900 d, closed west wall) and (b) observed 2S-2N D20 and SST anomalies, both over `window`; plus the Hovmoller fields …
`slab_frames`	função	`(ds, oisst, casts, start, end, *, spin_months=36, lon=(121.0, 279.0), lat=(-19.5, 19.5)) …)`	Monthly frames for :func:`.render.render_frames`: full SST [deg C] from OISST (1 deg) and thermocline depth = the D20 climatology of the casts (2 deg lat x 10 deg lon bins, base period, smoothed and interpolated to the SST grid) plus the anomaly of the ERA5-forced 2-D ocean model, scaled to D20 by …
`markdown_tables`	função	`(res, leads=(1, 3, 6, 9, 12, 18, 24)) -> dict[str, str]`	Markdown tables of the study from the stored results (`res`: the study JSON).

`pinneapple_systems.time_series.enso.plots`

Figures of the ENSO study, in the style of :mod:`pinneapple_systems.time_series.forecast_lab.plots` (same family colours, grid and spines): forecast plumes, skill against lead, the spring-barrier heatmap, the recharge-oscillator phase portrait and Hovmoller diagrams.

`pinneapple_systems/time_series/enso/plots.py` · 217 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_plumes`	função	`(panels, path, *, ncols=3) -> str`	Forecast distributions at the evaluated leads from one origin per panel, one family per colour (median marker, 50 % thick and 90 % thin interval bars, slightly offset per family), observed Nino 3.4 in black. Each panel: `title`, `t_obs`/`y_obs` (months as fractional years), `origin_t`, and …
`plot_skill_vs_lead`	função	`(curves, path, *, metrics=…, labels=None) -> str`	One panel per metric: each curve (`family`, `horizons`, values, optional `significant` mask) against lead; baselines dashed grey.
`plot_spring_barrier`	função	`(maps, path, *, ncols=3, key='acc') -> str`	Heatmaps of `key` (ACC) by target month (rows) and lead (columns), one per model; cells with too few forecasts are blank, the count range is in each title.
`plot_phase_portrait`	função	`(traj, path) -> str`	(h, T) phase portraits: `traj[name] = {"h", "T", "family" or "color", "lw", "alpha"}`; arrows show the direction of time on the first (observed) curve.

nome	tipo	assinatura	o que faz
`plot_hovmoller`	função	`(panels, path, *, ncols=3) -> str`	Hovmoller diagrams (longitude x time): each panel `title`, `lon`, `t` (fractional years), `z` (nt, nlon), `vmax`, `units`.
`plot_validation`	função	`(kelvin, gill, conv, ro, path) -> str`	Solver checks: Kelvin-wave speed error under refinement, Gill error against resolution, recharge-oscillator period and growth (numerical vs analytic) against the time step.

`pinneapple_systems.time_series.enso.render`

3-D renders of the tropical Pacific during an ENSO event with the library's Blender Cycles renderer (:func:`pinneapple_tools.visualization.studio.render`).

`pinneapple_systems/time_series/enso/render.py` · 131 linhas · testes: 22

nome	tipo	assinatura	o que faz
`slab_scene`	função	`(lon, lat, sst, d20, *, d20_anom=None, band_deg=2.0, z_per_100m=1.0, x_per_deg=0.06, y_pe…)`	Scene of SST [deg C] (ny, nx) and thermocline depth `d20` [m, positive down] (ny, nx) on the same (lat, lon) grid (lon 0-360). The sea surface north of `band_deg` N and the equatorial band (`band_deg` S to N) carry `sst`; the band's southern wall drops from the surface to the thermocline ...
`render_frames`	função	`(frames, out_dir, *, sst_range=(22.0, 31.0), size=(960, 540), samples=24, view=(-0.4, -1.…)`	Render `frames` (dicts with `lon`, `lat`, `sst`, `d20`, optionally `d20_anom`, and a `label`) to PNG files in `out_dir`; `interpolate` > 1 inserts linearly interpolated frames between consecutive months. Needs `bpy` (`pip install bpy`) or a run inside Blender.
`encode_video`	função	`(paths, out, fps=8, crf=30) -> str`	H.264 MP4 (yuv420p, even frame size) from PNG frames with ffmpeg.

`pinneapple_systems.time_series.enso.study`

The ENSO forecasting study: the protocol, the model zoo, and one call per model (so the zoo runs in parallel processes and is merged by :func:`~pinneapple_systems.time_series.forecast_lab.study.merge_zoo`).

`pinneapple_systems/time_series/enso/study.py` · 173 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`protocol`	função	`(months, test_start='2012-01', horizons=HORIZONS, min_history=36) -> Protocol`	
`zoo`	função	`(quick=False) -> list[ModelSpec]`	The model zoo. `quick`: tiny epochs / members for smoke runs.
`spec`	função	`(name, quick=False) -> ModelSpec`	

nome	tipo	assinatura	o que faz
`run_model`	função	`(name, data, prot, *, budget=FP.BUDGET, quick=False, case_origins=None, log=print, seed=0…)`	Tune, fit and score one spec under `prot` (forecasts kept for :func:`merge_zoo`), plus its forecasts from the case-study origins that lie after the development period (`_cases`: label -> origin -> Forecast).
`event_hindcast`	função	`(name, data, event_start, origins, *, quick=False, horizons=HORIZONS, min_history=36, val…)`	Refit `name` (its first grid configuration) on data whose labels end before `event_start` and forecast from `origins` (for an event inside the development period, e.g. 1997/98, so no forecast uses a model that saw it). Returns the forecasts and the fitted model's description.

pinneapple_systems.time_series.features

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/features/__init__.py` · 2 linhas · **sem teste direto**

pinneapple_systems.time_series.features.engineering

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/features/engineering.py` · 110 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TSFeatureEngineer`	classe	`(lags, rolling_windows, rolling_stats_set, fourier_periods, fourier_K)`	Convenience wrapper to generate a feature dictionary.
`make_lags`	função	`(y, lags) -> Dict[str, np.ndarray]`	Create lag features for a 1D target series.
`rolling_stats`	função	`(y, windows, stats=('mean', 'std', 'min', 'max')) -> Dict[str, np.ndarray]`	Compute rolling statistics over the target series.
`fourier_features`	função	`(t, periods, K=1) -> Dict[str, np.ndarray]`	Fourier seasonality features: sin/cos harmonics for each period.
`rate_of_change`	função	`(y, orders=(1, 2)) -> Dict[str, np.ndarray]`	First and higher-order differences (velocity, acceleration).
`window_features`	função	`(y, input_len, horizon) -> tuple`	Build sliding-window (X, y_target) arrays for supervised ML. Returns: X : (n_windows, input_len) y_out : (n_windows, horizon)

pinneapple_systems.time_series.forecast_lab

Forecasting near-random series without fooling yourself: one interface for baselines, data-driven, data + physics, physical and hybrid forecasters, and the protocol that compares them (purged walk-forward tuning, an untouched test period, point and probabilistic scores, Diebold-Mariano tests with …

`pinneapple_systems/time_series/forecast_lab/__init__.py` · 44 linhas · testes: 50

pinneapple_systems.time_series.forecast_lab.base

Common interface of the forecasters in :mod:`pinneapple_systems.time_series.forecast_lab`.

`pinneapple_systems/time_series/forecast_lab/base.py` · 168 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`SeriesData`	classe	`(obs, target, dt, name, meta)`	Observed series (T, d), the forecast target column, the sampling step and free-form metadata.
`Forecast`	classe	`(mean, std, samples)`	Predictive distribution at (origin, horizon): a mean, and either a Gaussian `std` or `samples` (members, origins, horizons).
`Forecaster`	classe	`()`	Base class: subclasses implement :meth:`_fit` and :meth:`_predict_mean` (or override :meth:`predict`).
`train_end`	função	`(train_origins, horizons) -> int`	Exclusive end index of the data that a fit on these origins may use.
`forecast_origins`	função	`(lo, hi, horizons, *, stride=1, min_history=0) -> np.ndarray`	Origins t with lo <= t, t >= min_history - 1 and t + max(h) < hi, every `stride` samples.
`targets_at`	função	`(data, origins, horizons) -> np.ndarray`	

pinneapple_systems.time_series.forecast_lab.baselines

Baselines every other forecaster must beat: persistence, climatology, AR(p), ARIMA, exponential smoothing, and for (near-)periodic series the seasonal-naive forecast and a phase-aligned mean cycle.

`pinneapple_systems/time_series/forecast_lab/baselines.py` · 278 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Persistence`	classe	`()`	y_{t+h} = y_t (the random-walk forecast).
`Climatology`	classe	`()`	y_{t+h} = mean of the training segment, spread = its standard deviation.
`ARForecaster`	classe	`(max_order=20, criterion='aic', difference=False)`	Least-squares AR(p) on the target, order by AIC up to `max_order`, iterated to each horizon.
`ARIMAForecaster`	classe	`(p=(0, 1, 2), d=(0, 1), q=(0, 1), max_fit_length=2000)`	ARIMA(p, d, q) (statsmodels SARIMAX), order by AIC over a small grid on the training segment; forecasts from every origin with the fitted parameters fixed, through the Kalman filter state at that origin (so a forecast uses data up to its origin only). The Gaussian predictive variance comes from the …
`ExponentialSmoothing`	classe	`()`	Simple exponential smoothing l_t = alpha y_t + (1 - alpha) l_{t-1}, flat forecast l_t; alpha minimises the one-step squared error on the training segment.

nome	tipo	assinatura	o que faz
SeasonalNaive	classe	(period=None)	y_{t+h} = y_{t+h-kP} with k = ceil(h / P): the value one period (or the fewest whole periods) before the target. period in samples; by default the dominant period of the training segment (rounded to an integer number of samples).
PhaseClimatology	classe	(period=None, fit_cycles=1.0)	The mean cycle, aligned to the phase of each origin.
dominant_period	função	(y, min_period=2.0, max_frac=0.25) -> float	Period (in samples) of the highest periodogram peak of y after removing the mean and a linear trend, searched between min_period samples and max_frac of the series length.

pinneapple_systems.time_series.forecast_lab.hybrid

Hybrid forecasters: a physical model plus something learned.

pinneapple_systems/time_series/forecast_lab/hybrid.py · 181 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ResidualCorrectionForecaster	classe	(physical, corrector='gbm', n_lags=8, lag_step=1, max_iter=200, learning_rate=0.05, max_l…)	physical forecast + corrector(lags, physical forecast). corrector is "gbm" or "mlp".
UDEForecaster	classe	(physical, hidden=64, layers=2, window=4, epochs=200, patience=20, lr=0.001, weight_decay…)	dx/dt = f_known(x; theta_hat) + s * g_NN((x - mu) / sd), trained through RK4 over window samples.

pinneapple_systems.time_series.forecast_lab.learned

Data-driven direct forecasters on lagged windows: gradient boosting and neural networks (MLP, LSTM, TCN, transformer), with optional physics-derived input features and, for the MLP, a physics residual penalty.

pinneapple_systems/time_series/forecast_lab/learned.py · 355 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LagFeatures	classe	(n_lags=8, lag_step=1, cols=None, extra=None)	Lagged windows of the observed columns, plus optional extra features extra(data, origins) -> (n, k).
GBMForecaster	classe	(n_lags=8, lag_step=1, max_iter=200, learning_rate=0.05, max_leaf_nodes=15, min_samples_l…)	Histogram gradient boosting (scikit-learn), one regressor per horizon, on lag features.

nome	tipo	assinatura	o que faz
`NeuralForecaster`	classe	`(arch='mlp', n_lags=16, lag_step=1, hidden=64, layers=2, dropout=0.0, weight_decay=0.0, l…)`	Direct multi-horizon neural forecaster: `arch` in {"mlp", "lstm", "tcn", "transformer"}.
`PhysicsLossMLP`	classe	`(physical, lam=0.1, **kw)`	MLP that predicts the whole observed state at every step 1..max(h) and is penalised for violating a known vector field: loss = MSE + `lam` * mean |(X_{k+1} - X_k) / dt - f((X_{k+1} + X_k) / 2)|^2 / scale^2 along the predicted path (implicit-midpoint residual, second order in dt; X_0 is the …
`train_torch_regressor`	função	`(build, X_tr, Y_tr, X_va=None, Y_va=None, *, epochs=100, lr=0.001, weight_decay=0.0, batc…)`	Train `build()` on (X, Y) with :class:`pinneapple_neural.trainer.Trainer`.

`pinneapple_systems.time_series.forecast_lab.physical`

Physical forecasters: the governing equations with parameters estimated from the training data only.

`pinneapple_systems/time_series/forecast_lab/physical.py` · 564 linhas · testes: 22

nome	tipo	assinatura	o que faz
`ODEForecaster`	classe	`(rhs, params, fit_params=None, sub=5, fit_steps=2, max_fit_origins=1500, divergence_facto…)`	dx/dt = rhs(x, **params) with every state variable observed (the columns of `data.obs`).
`DelayODEForecaster`	classe	`(rhs, params, tau, fit_params=None, dt_int=0.1, fit_steps=2, max_fit_origins=1500, diverg…)`	dx/dt = rhs(x, x(t - tau), **params) for a scalar series sampled every dt; integrated with RK4 at `dt_int` (dt and tau must be multiples of it), the history taken from the observations by linear interpolation.
`LinearSSMForecaster`	classe	`(kind='ou', max_fit_length=4000, name=None, family=None)`	Linear Gaussian state-space model of a scalar observation y = mu + x_0 + noise.
`SDEDriftForecaster`	classe	`(degree=3, n_samples=200, sub=10, closure=None, seed=0, name=None, family=None)`	dx = f(x) dt + sigma dW for a scalar series, f(x) = sum_k c_k x^k (k <= `degree`) fitted to the increments, optionally plus a neural closure g(x) fitted to what the polynomial leaves (`closure=dict(hidden=.., epochs=..)`). Forecast: `n_samples` Euler-Maruyama paths with `sub` substeps per …

nome	tipo	assinatura	o que faz
`EnKFForecaster`	classe	`(physical, n_ens=50, inflation=1.02, q_frac=0.1, spinup=200, seed=0, name=None, family=No…)`	Stochastic EnKF over the observations with a fitted :class:`ODEForecaster` as the model.
`SINDyForecaster`	classe	`(threshold=0.05, l2=0.0001, library='poly', sub=5, divergence_factor=5.0, name=None, fami…)`	Equations discovered by SINDy (STLSQ, polynomial library of degree 2) on the training trajectory of a fully observed state; forecasts integrate them (RK4, `sub` substeps per sample) from the observed state.
`PhysicsFeatures`	classe	`(physical)`	Input features for a data-driven model computed by a physical forecaster (fitted on the training origins when the data model is fitted): its `physics_features(data, origins)` -- the fitted vector field or drift at the origin state, or the filtered state and energy of a linear model.

`pinneapple_systems.time_series.forecast_lab.pipeline`

One call per study case, and the tables of the guide, built from the stored results.

`pinneapple_systems/time_series/forecast_lab/pipeline.py` · 270 linhas · testes: 28

nome	tipo	assinatura	o que faz
`fan_panels`	função	`(zoo, data, title, n_origins=2, families=('physical', 'hybrid', 'data')) -> list[dict]`	Fan-chart data for the representatives of `families` at `n_origins` test origins (spread over the test period).
`surprise_record`	função	`(zoo, data, horizon_index=0) -> dict[str, Any] | None`	Per test origin: the absolute error at one horizon of the best non-baseline model (by validation score) and of the best baseline at that horizon (by validation RMSE), with the target dates when `data.meta["dates"]` exists. Feeds :func:`~.plots.plot_surprise_calendar`.
`run_case`	função	`(name, *, n=6000, budget=BUDGET, seed=0, cache_dir=None, n_surrogates=99, log=print) -> d…)`	
`run_overfitting`	função	`(name, *, n=6000, seed=0, cache_dir=None, schemes=…, log=print) -> dict[str, Any]`	
`lyapunov_study`	função	`(dt_ints=(0.1, 0.05), total_times=(10000.0, 50000.0, 200000.0), seeds=(0, 1, 2), log=prin…)`	Convergence of the Benettin estimate of the Mackey-Glass (tau = 17) largest Lyapunov exponent with the integration step, the length of the run and the seed (initial history and perturbation direction).

nome	tipo	assinatura	o que faz
`memorise_vs_learn`	função	`(name, variants=…, *, horizon_index=0, window=200, n=6000, seed=0, cache_dir=None) -> dic…)`	Refit two training variants of :func:`~.presets.overfitting_variants` on the final split of case `name` and return, for one horizon, the truth and both forecasts over the last `window` training origins and the first `window` test origins, with each variant's RMSE on the whole training and …
`markdown_tables`	função	`(results, overfit=()) -> dict[str, str]`	Markdown tables: predictability, skill horizon per family, scores of the representatives, the zoo (wall time, parameters), surrogate-trained twins, and the overfitting study.

`pinneapple_systems.time_series.forecast_lab.plots`

Figures of a forecasting study (matplotlib, static PNG).

`pinneapple_systems/time_series/forecast_lab/plots.py` · 268 linhas · testes: 8

nome	tipo	assinatura	o que faz
`plot_skill_vs_horizon`	função	`(results, path, *, lyapunov=None, ncols=4, floor=-0.5, titles=None, reference='best_basel…)`	One panel per series: RMSE skill of each family's representative (chosen on the validation block) against the horizon, relative to `reference` -- `"best_baseline"` (at each horizon the baseline with the lowest validation RMSE) or `"persistence"`; filled markers where the Holm-adjusted …
`plot_overfitting`	função	`(studies, path) -> str`	Per series: train, validation and test error of every training variant (bars), and the cross-validation estimates of each scheme (markers) for the variants where they were computed.
`plot_surrogate_tests`	função	`(reports, path, *, ncols=4) -> str`	Nonlinear prediction error of each series (vertical line) against its shuffled and IAAFT surrogates.
`plot_pit`	função	`(entries, path, *, ncols=4) -> str`	PIT histograms. `entries`: dicts with `title`, `freq` (relative frequencies) and `deviation`.
`plot_fan_charts`	função	`(panels, path, *, ncols=2) -> str`	Forecast distributions on the test period, drawn only at the evaluated horizons: a marker at the median and error bars for the central 50 % (thick) and 90 % (thin) intervals, with no line joining the horizons (the forecasts are direct, made for those horizons only). Each panel: `title`, …
`plot_memorise_vs_learn`	função	`(records, path) -> str`	For each record of :func:`~.pipeline.memorise_vs_learn`: truth and the two variants' forecasts over the end of the training period (left) and the start of the untouched test period (right). One shared legend; each panel title carries the variants' RMSE on the whole training or test set.
`calendar_grid`	função	`(dates, values) -> dict`	Arrange daily `values` on a week (column) x weekday (row) grid; days without a value are NaN. Returns the grid (7, n_weeks), the Monday of each week and the day range.
`plot_surprise_calendar`	função	`(records, path, *, vmax=1.0) -> str`	Calendar heatmaps, one per record (`title`, `dates`, `abs_error_model`, `abs_error_baseline`, `model`, `baseline`, `horizon`): each day coloured by log2(|error of the model| / |error of the best baseline|) -- blue where the model was closer, red where the baseline was, grey midpoint …

pinneapple_systems.time_series.forecast_lab.presets

The series and model zoos of the near-random forecasting study (examples/near_random_forecasting).

pinneapple_systems/time_series/forecast_lab/presets.py · 345 linhas · testes: 20

nome	tipo	assinatura	o que faz
Case	classe	(name, data, protocol, specs, truth, title)	
lorenz63_missing_xz_rhs	função	(x, sigma=10.0, rho=28.0, beta=8.0 / 3.0)	Lorenz-63 without the -x z term of dy/dt: a deliberately misspecified physical model.
baseline_specs	função	(arima=True) -> list[ModelSpec]	
data_specs	função	(lag_steps=(1,), lags=(8, 16), seq_lags=32, epochs=60, patience=8, max_train=3000, cols=N…)	GBM, MLP, LSTM, TCN and transformer on lagged windows, each with a grid sampled to BUDGET configs.
case	função	(name, *, n=6000, seed=0, cache_dir=None) -> Case	Build the study case name (one of :data:CASES); n is the length of the synthetic series.
overfitting_variants	função	(c, physics=False) -> dict[str, Any]	Training variants of an MLP (and of gradient boosting) for :func:~.study.overfitting_study: from a deliberately overfit network (wide, deep, long window, no early stopping, no weight decay, 200 epochs) to small, regularised, early-stopped and ensembled ones. physics adds the physics-loss MLP …

pinneapple_systems.time_series.forecast_lab.renders

3-D renders of a forecasting study (Blender Cycles through :mod:pinneapple_blender.cycles).

pinneapple_systems/time_series/forecast_lab/renders.py · 314 linhas · testes: 8

nome	tipo	assinatura	o que faz
landscape_tiles	função	(results, *, floor=-0.5, titles=None) -> list[dict[str, Any]]	One tile per case result (from pipeline.run_case): the families present, the horizon axis (log10 of Lyapunov times when the series is chaotic, else log10 of samples) and the skill against the best baseline of each family's representative at every horizon, clipped below at floor.
render_landscape	função	(tiles, path, *, ncols=4, resolution=(1600, 1000), samples=64, height_scale=1.2) -> str	Draw the tiles as surfaces (x: horizon, y: family, z: skill) on a grid of ncols columns over a dark studio floor, each with a translucent sea at skill 0 bounded by a glowing waterline. The families share one labelled axis per row (a tile with a single family is a narrow strip on that family's …

nome	tipo	assinatura	o que faz
dense_forecast	função	(data, protocol, make, max_horizon, *, quantiles=(0.05, 0.25, 0.5, 0.75, 0.95)) -> dict[s…)	Fit make() on the protocol's final split with horizons 1..max_horizon (every step) and forecast every test origin. Returns the quantiles at all test origins and the empirical coverage of the 50 % and 90 % bands per horizon (measured on the test period).
render_cones	função	(lanes, path, *, resolution=(1280, 720), samples=64, frames_dir=None, n_frames=0, history…)	Draw one lane per entry (title, dense from :func:dense_forecast, origin_pos index into its test origins, std for scaling), stacked vertically: the observed history as an emissive tube, the 90 % and 50 % bands as translucent ribbons, the median as a thin tube and what happened as a …
embedding_views	função	(x, *, delay=None, n_points=3000, seed=0) -> dict[str, Any]	Delay embeddings (dimension 3) of x, of an IAAFT surrogate and of a shuffled surrogate, each standardised, plus the nonlinear prediction error (h = 1, NN predictor) of the three series.
render_attractors	função	(views, path, *, resolution=(1280, 480), samples=64, titles=…) -> str	
ensemble_fibres	função	(step, x0, *, n_members=50, rel_perturbation=0.01, n_steps=300, scale=None, seed=0, compo…)	Integrate n_members copies of a state, each perturbed by rel_perturbation x scale (per component, default 1), with step(states) -> states acting on a batch (m, d). Returns the two chosen components over time for every member and the ensemble spread (std across members of the first …
render_ensembles	função	(panels, path, *, resolution=(1280, 900), samples=48) -> str	One lane per panel (title, fibres from :func:ensemble_fibres, times (n,), lt_time or None, family colour key), stacked vertically: members as thin tubes with time along x, the first component as height and the second as depth, and a translucent wall at one Lyapunov time.

pinneapple_systems.time_series.forecast_lab.study

The evaluation protocol: fix the test period first, tune on purged walk-forward folds of the rest, fit once, score once on the untouched test period, and test the differences.

pinneapple_systems/time_series/forecast_lab/study.py · 450 linhas · testes: 50

nome	tipo	assinatura	o que faz
Protocol	classe	(horizons, test_frac, val_frac, n_folds, fold_frac, inner_val_frac, min_history, test_str…)	Split of a series of length n into development (train + validation) and an untouched test period.
ModelSpec	classe	(name, family, factory, grid)	A forecaster in the zoo: factory(**config) builds an unfitted model; grid lists the configurations to try (one entry means no tuning).
tune	função	(spec, data, protocol, *, budget=None, seed=0) -> dict[str, Any]	Score every configuration (or budget of them, drawn at random) by purged walk-forward CV.

nome	tipo	assinatura	o que faz
evaluate	função	`(model, data, origins, *, levels=LEVELS, keep=False) -> dict[str, Any]`	Point and probabilistic scores per horizon on `origins`.
skill_horizon	função	`(horizons, skill, p_values=None, alpha=0.05) -> int | None`	Largest horizon h such that the skill is positive (and, with `p_values`, significant) at every horizon up to h; None when there is no skill at the first horizon.
run_zoo	função	`(data, specs, protocol, *, budget=None, keep_forecasts=(), keep_models=False, log=print, …)`	Tune, fit and score every model of the zoo on one series. Returns a JSON-able dict (`_forecasts` holds the test forecasts of `keep_forecasts` for figures, `_models` the fitted models when `keep_models`; both are dropped by `to_jsonable(drop_private=True)`). Zoos run in parts (e.g. one …
merge_zoo	função	`(parts) -> dict[str, Any]`	Combine :func:`run_zoo` results of the same series and protocol run in parts (each with its models' test forecasts kept): the models and forecasts are pooled and the comparisons (skill, DM tests against persistence and the best baseline, Holm and BH adjustments, family representatives) are …
overfitting_study	função	`(data, protocol, variants, *, schemes=('purged_walk_forward',), schemes_for=None, log=pri…)`	For each training variant: its error on its training origins, on the validation block and on the untouched test period, the gap test - train, and the cross-validation estimate under each scheme with its optimism (test - CV). The error is the mean over horizons of RMSE / RMSE of persistence *on the …
surrogate_trained	função	`(make, data, protocol, *, kind='iaaft', seed=0) -> dict[str, Any]`	Skill (vs persistence, mean over horizons and per horizon) on the real test period of a univariate model trained on the real development data and of the same model trained on a surrogate of it.

pinneapple_systems.time_series.foundation_models

Pretrained time-series *foundation model* forecasters.

`pinneapple_systems/time_series/foundation_models.py` · 379 linhas · testes: 14 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
ChronosForecaster	classe	`(model_id='amazon/chronos-t5-tiny', device_map='cpu', dtype=None, **pipeline_kwargs)`	Wraps Amazon's real, pretrained Chronos forecasting pipeline.
TimesFMForecaster	classe	`(model_id=…, max_context=512, max_horizon=128, torch_compile=False, **forecast_config_kwa…)`	Wraps Google's real, pretrained TimesFM forecasting model.

pinneapple_systems.time_series.losses

Time series forecast loss functions.

`pinneapple_systems/time_series/losses.py` · 9 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
mse_forecast	função	`(model, y_hat, batch)`	

pinneapple_systems.time_series.metrics

Time series metrics for forecasting evaluation.

`pinneapple_systems/time_series/metrics.py` · 104 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mae`	função	`(y_hat, batch)`	Mean Absolute Error.
`mse`	função	`(y_hat, batch)`	Mean Squared Error.
`rmse`	função	`(y_hat, batch)`	Root Mean Squared Error.
`mape`	função	`(y_hat, batch)`	Mean Absolute Percentage Error (%).
`smape`	função	`(y_hat, batch)`	Symmetric Mean Absolute Percentage Error (%). Bounded in [0, 200].
`r2`	função	`(y_hat, batch)`	Coefficient of determination R^2. $R^2=1$ → perfect; $R^2<0$ → worse than mean.
`mae_fn`	função	`(y_hat, y) -> torch.Tensor`	
`mse_fn`	função	`(y_hat, y) -> torch.Tensor`	
`rmse_fn`	função	`(y_hat, y) -> torch.Tensor`	
`mape_fn`	função	`(y_hat, y) -> torch.Tensor`	Mean Absolute Percentage Error (%).
`smape_fn`	função	`(y_hat, y) -> torch.Tensor`	
`r2_fn`	função	`(y_hat, y) -> torch.Tensor`	
`default_ts_metrics`	função	`()`	

`pinneapple_systems.time_series.metrics_ext`

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/metrics_ext/__init__.py` · 28 linhas · testes: 20

`pinneapple_systems.time_series.metrics_ext.comparison`

Comparing forecasts: skill scores, the Diebold-Mariano test and multiple-comparison control.

`pinneapple_systems/time_series/metrics_ext/comparison.py` · 127 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DMResult`	classe	`(statistic, p_value, p_value_better, mean_loss_difference, n, max_lag)`	
`skill_score`	função	`(score, reference) -> np.ndarray`	1 - score / reference, elementwise (for errors and other negatively oriented scores).
`diebold_mariano`	função	`(e1, e2, *, h=1, loss='se', max_lag=None, hln=True) -> DMResult`	Test E[L(e1)] = E[L(e2)] for two error series on the same targets (in time order).
`holm`	função	`(p_values)`	Holm step-down adjusted p-values (same keys / order as given); NaNs are left out of the family.
`benjamini_hochberg`	função	`(p_values)`	Benjamini-Hochberg adjusted p-values (q-values); NaNs are left out of the family.

pinneapple_systems.time_series.metrics_ext.point

Módulo sem docstring e sem texto.

pinneapple_systems/time_series/metrics_ext/point.py · 81 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PointMetrics	classe	(season_length)	
mae	função	(y_pred, y_true) -> float	
rmse	função	(y_pred, y_true) -> float	
mape	função	(y_pred, y_true, eps=1e-08) -> float	
smape	função	(y_pred, y_true, eps=1e-08) -> float	
mase	função	(y_pred, y_true, y_train, season_length=1, eps=1e-08) -> float	Mean Absolute Scaled Error (MASE).
multi_horizon_mae	função	(y_pred, y_true) -> Dict[str, float]	Overall MAE and MAE per horizon step for (N,H) or (N,H,C).

pinneapple_systems.time_series.metrics_ext.probabilistic

Módulo sem docstring e sem texto.

pinneapple_systems/time_series/metrics_ext/probabilistic.py · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ProbabilisticMetrics	classe	(quantiles)	
pinball_loss	função	(y_pred_q, y_true, q) -> float	
coverage	função	(y_lo, y_hi, y_true) -> float	
mean_interval_width	função	(y_lo, y_hi) -> float	
pit_gaussian	função	(mu, sigma, y) -> np.ndarray	
pit_ensemble	função	(samples, y, rng=None) -> np.ndarray	Randomised PIT of an ensemble (members on axis 0): ties and the discreteness of the empirical CDF are broken uniformly at random, so a calibrated ensemble gives exactly uniform values.
pit_histogram	função	(pit, n_bins=10) -> Dict[str, object]	Relative frequencies of the PIT in n_bins equal bins and their mean absolute deviation from 1 / n_bins (0 for a perfectly flat histogram), divided by 1 / n_bins.
gaussian_interval_coverage	função	(mu, sigma, y, level=0.9) -> float	

nome	tipo	assinatura	o que faz
ensemble_interval_coverage	função	(samples, y, level=0.9) -> float	

pinneapple_systems.time_series.models

Módulo sem docstring e sem texto.

pinneapple_systems/time_series/models/__init__.py · 29 linhas · **sem teste direto**

pinneapple_systems.time_series.models.classical

Classical ML forecasting models: XGBoost, LightGBM, RandomForest, CatBoost, GPR.

pinneapple_systems/time_series/models/classical.py · 315 linhas · testes: 151 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
ForecastModel	classe	()	
XGBoostForecaster	classe	(n_estimators=300, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, …)	XGBoost multi-output regressor (wraps MultiOutputRegressor for horizon>1).
LightGBMForecaster	classe	(n_estimators=300, num_leaves=31, learning_rate=0.05, subsample=0.8, **kwargs)	
RandomForestForecaster	classe	(n_estimators=200, max_depth=None, min_samples_leaf=1, **kwargs)	
CatBoostForecaster	classe	(iterations=300, depth=6, learning_rate=0.05, **kwargs)	
GPRForecaster	classe	(kernel=None, alpha=0.0001, normalize_y=True, **kwargs)	Gaussian Process Regressor — natively probabilistic. Best for small datasets (< ~5000 samples). Multi-output: one GP per output dimension.
MLPForecaster	classe	(hidden_layer_sizes=(128, 64), activation='relu', max_iter=1000, learning_rate_init=0.001…)	

pinneapple_systems.time_series.models.fno_forecaster

FNO-1D forecaster for time series multi-horizon prediction.

pinneapple_systems/time_series/models/fno_forecaster.py · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FNOForecastConfig	classe	(input_len, horizon, n_features, n_targets, width, modes, layers)	

nome	tipo	assinatura	o que faz
FNOForecaster	classe	(cfg)	Forecaster TS padrão usando FNO-1D. Convenção: x: (B, L_in, F) y_hat: (B, H, C)

pinneapple_systems.time_series.models.nbeats

N-BEATS: Neural Basis Expansion Analysis for Time Series (Oreshkin et al., 2019).

pinneapple_systems/time_series/models/nbeats.py · 127 linhas · testes: 151

nome	tipo	assinatura	o que faz
NBeatsConfig	classe	(input_len, horizon, n_features, stack_types, n_blocks, n_layers, layer_width, degree_of_…)	
NBeats	classe	(cfg=None, **kw)	N-BEATS with trend + seasonality + generic stacks. Input : (B, L, F) Output : TSOutput with y_hat (B, H, 1)

pinneapple_systems.time_series.models.recurrent

LSTM and GRU forecasters (PyTorch, GPU-ready).

pinneapple_systems/time_series/models/recurrent.py · 64 linhas · testes: 151

nome	tipo	assinatura	o que faz
RecurrentConfig	classe	(input_len, horizon, n_features, n_targets, hidden_size, num_layers, dropout, bidirection…)	
LSTMForecaster	classe	(cfg=None, **kw)	LSTM-based multi-step forecaster.
GRUForecaster	classe	(cfg=None, **kw)	GRU-based multi-step forecaster.

pinneapple_systems.time_series.models.tcn

Temporal Convolutional Network (TCN) forecaster.

pinneapple_systems/time_series/models/tcn.py · 74 linhas · testes: 151

nome	tipo	assinatura	o que faz
TCNConfig	classe	(input_len, horizon, n_features, n_targets, n_channels, n_layers, kernel_size, dropout)	
TCNForecaster	classe	(cfg=None, **kw)	TCN with exponentially increasing dilations (1,2,4,8,...). Input : (B, L, F) Output : TSOutput with y_hat (B, H, n_targets)

pinneapple_systems.time_series.models.tft

Temporal Fusion Transformer (Lim et al., 2021) — simplified, GPU-ready.

pinneapple_systems/time_series/models/tft.py · 197 linhas · testes: 151

nome	tipo	assinatura	o que faz
TFTConfig	classe	(input_len, horizon, n_features, n_targets, hidden_size, num_heads, num_lstm_layers, drop…)	
TFTForecaster	classe	(cfg=None, **kw)	Temporal Fusion Transformer. Input : (B, L, F) Output : TSOutput with y_hat (B, H, n_targets)

pinneapple_systems.time_series.predictability

Is there anything to predict? Measures of predictability to compute before fitting any forecaster.

pinneapple_systems/time_series/predictability.py · 350 linhas · testes: 20

nome	tipo	assinatura	o que faz
ARFit	classe	(order, intercept, coef, sigma, r2_one_step, criterion)	
SurrogateTest	classe	(statistic, kind, observed, surrogates, p_value, alternative, reject_at_0_05)	
fit_ar	função	(x, max_order=20, criterion='aic') -> ARFit	Least-squares AR(p), p = 0..max_order, all fitted on the same sample (the first max_order values are dropped for every p) so the information criteria are comparable.
permutation_entropy	função	(x, order=5, delay=1, normalize=True) -> float	Shannon entropy of the ordinal patterns of length order (ties broken by position).
sample_entropy	função	(x, m=2, r=None) -> float	-log(A / B): B = template pairs of length m within tolerance r (Chebyshev), A = the same for m + 1; self matches excluded and the same N - m templates used for both lengths. r defaults to 0.2 std(x).
delay_embedding	função	(x, dim, delay) -> np.ndarray	
first_acf_zero	função	(x, max_lag=None) -> int	First lag at which the autocorrelation drops below 1/e (a common embedding-delay rule).
rosenstein_lyapunov	função	(x, *, dim=5, delay=None, dt=1.0, min_tsep=None, n_steps=40, fit=(5, 30), max_points=3000…)	Largest Lyapunov exponent per unit time from a scalar series.
shuffle_surrogate	função	(x, rng) -> np.ndarray	Random permutation: keeps the amplitude distribution, destroys all temporal structure.
phase_randomized_surrogate	função	(x, rng) -> np.ndarray	Fourier-transform surrogate: same power spectrum, random phases (a linear Gaussian process).
iaaft_surrogate	função	(x, rng, n_iter=200, tol=1e-08) -> np.ndarray	Iterative amplitude-adjusted Fourier transform surrogate (Schreiber & Schmitz 1996): the same amplitude distribution as x exactly and nearly the same power spectrum.
nonlinear_prediction_error	função	(x, *, dim=4, delay=1, horizon=1, k=10, train_frac=0.5) -> float	Out-of-sample error of a local-constant (nearest-neighbour) predictor, divided by std(x).

nome	tipo	assinatura	o que faz
`time_reversal_asymmetry`	função	`(x, lag=1) -> float`	Normalised third moment of the lag-`lag` increments; 0 in expectation for any time-reversible process (every linear Gaussian process is).
`surrogate_test`	função	`(x, statistic, *, kind='iaaft', n_surrogates=99, alternative='less', seed=0, name=None) -…)`	Rank test of `statistic(x)` against `n_surrogates` surrogates of the given `kind`.
`baseline_errors`	função	`(x, horizons, train_frac=0.8) -> dict[str, dict[int, float]]`	RMSE of persistence (x_{t+h} = x_t) and climatology (x_{t+h} = mean of the first `train_frac`) on the last 1 - `train_frac` of the series, per horizon, both divided by std(x).
`predictability_report`	função	`(x, *, horizons=(1, 2, 4, 8, 16, 32), dt=1.0, max_ar_order=20, pe_order=5, embed_dim=4, n…)`	Predictability of a scalar series before any modelling.

pinneapple_systems.time_series.preparation

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/preparation/__init__.py` · 6 linhas · **sem teste direto**

pinneapple_systems.time_series.preparation.imputer

Missing value treatment with layered interpolation strategies.

`pinneapple_systems/time_series/preparation/imputer.py` · 126 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TimeSeriesImputer`	classe	`(strategy='layered', max_gap=None, verbose=False)`	Layered missing-value imputer for time series.

pinneapple_systems.time_series.preparation.outliers

Outlier detection and treatment for time series.

`pinneapple_systems/time_series/preparation/outliers.py` · 155 linhas · testes: 151

nome	tipo	assinatura	o que faz
`OutlierDetector`	classe	`(method='rolling_iqr', window=20, iqr_factor=1.5, z_threshold=3.0, treatment='interpolate…)`	Outlier detection (Rolling IQR / Z-Score) with 6 treatment options.

pinneapple_systems.time_series.preparation.resampler

Automatic upsampling / downsampling for numeric time-indexed DataFrames.

`pinneapple_systems/time_series/preparation/resampler.py` · 122 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TimeSeriesResampler`	classe	`(target_freq, time_col=None, agg_method='mean', upsample_method='linear', numeric_only=Tr…)`	Resample a time-indexed DataFrame to a target frequency.

`pinneapple_systems.time_series.problem`

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/problem.py` · 54 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ForecastProblemSpec`	classe	`(freq, time_col, target_cols, feature_cols, exog_past_cols, exog_future_cols, input_len, …)`	Formal forecasting problem definition.

`pinneapple_systems.time_series.registry`

Time series model catalog and default FNO forecaster registration.

`pinneapple_systems/time_series/registry.py` · 82 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TSModelCatalog`	classe	`(families, default_name)`	

`pinneapple_systems.time_series.solar_cycle`

Solar-cycle envelope from the daily sunspot number: (E)EMD, the IMF nearest 11 years, its Hilbert amplitude, and a fit of that amplitude by an extreme learning machine (ELM) with the end-point equality f(0) = f(1) built into the basis (a constrained expression of the Theory of Functional ...

`pinneapple_systems/time_series/solar_cycle.py` · 274 linhas · testes: 16

nome	tipo	assinatura	o que faz
`Decomposition`	classe	`(imfs, method, backend, trials, noise_width)`	
`HilbertResult`	classe	`(amplitude, phase, instantaneous_frequency)`	
`PeriodicFit`	classe	`(rmse, fitted, forecast_t, forecast, hidden_units, backend, endpoint_error)`	
`parse_silso_csv`	função	`(source)`	A SILSO daily CSV (path, file-like or bytes; ';'-separated, no header, -1 = missing) to a DataFrame with a DatetimeIndex and the non-missing `Sunspot` values.
`fetch_silso_daily`	função	`(url=SILSO_DAILY_URL, timeout=30.0)`	Download and parse the current SILSO daily series (network; raises on failure, never substitutes data).
`synthetic_daily`	função	`(start='2005-01-01', end='2025-06-30', *, period_years=11.0, peak=180.0, seed=0)`	A SILSO-shaped synthetic daily series: a sin^2 cycle of `period_years` (minima 3.9 years after `start`, as 2008.9 is after 2005), ~27-day rotational modulation and multiplicative noise, clipped at 0. Not solar data; for tests and offline demos.

nome	tipo	assinatura	o que faz
`preprocess`	função	`(df)`	`(t, y_scaled, (y_min, y_max))`: t = linspace(0, 1, N), y min-max scaled to [0, 1].
`decompose`	função	`(y, method='emd', *, max_imf=6, trials=16, noise_width=0.05, seed=0, backend='auto') -> D…)`	EMD (deterministic) or EEMD (noise-assisted ensemble of `trials` runs) of a 1-D series.
`imf_statistics`	função	`(imfs, dt_days=1.0) -> list[dict[str, Any]]`	Per IMF: dominant period (days and years, from the median positive Hilbert instantaneous frequency), mean instantaneous amplitude and share of the variance of the summed signal.
`closest_to_schwabe_cycle`	função	`(stats) -> tuple[dict | None, str | None]`	The IMF whose period is nearest 11 years, and a warning when it is more than 40 % off.
`hilbert_analysis`	função	`(imf, t) -> HilbertResult`	Instantaneous amplitude, unwrapped phase and frequency of one IMF.
`periodic_elm_fit`	função	`(t, y, *, hidden_units=50, forecast_horizon_frac=1.0, n_forecast_points=None, seed=0, bac…)`	Least-squares ELM fit of `y(t)` on t in [0, 1] with f(0) = f(1) exact, extended to [1, 1 + horizon].
`run_pipeline`	função	`(df, *, decomposition_method='emd', max_imf=6, eemd_trials=16, eemd_noise_width=0.05, imf…)`	Steps 1-4 of the module docstring on a daily DataFrame (`Sunspot` column, DatetimeIndex).

`pinneapple_systems.time_series.spec`

Time series specification (input_len, horizon, stride, target_offset).

`pinneapple_systems/time_series/spec.py` · 13 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TimeSeriesSpec`	classe	`(input_len, horizon, stride, target_offset)`	Specification for time series input length, horizon, stride, and target offset.

`pinneapple_systems.time_series.synthetic`

Synthetic time series with known ground truth, for forecasting studies.

`pinneapple_systems/time_series/synthetic.py` · 364 linhas · testes: 50

nome	tipo	assinatura	o que faz
`SyntheticSeries`	classe	`(name, t, state, obs, dt, target, observed, params, noise_std, noise_level, lyapunov, kin…)`	A synthetic series and its ground truth.
`rk4_step`	função	`(f, x, dt)`	One classical Runge-Kutta step of `dx/dt = f(x)` (NumPy or torch, batched over leading axes).

nome	tipo	assinatura	o que faz
lorenz63_rhs	função	(x, sigma=10.0, rho=28.0, beta=8.0 / 3.0)	
lorenz96_rhs	função	(x, F=8.0)	
mackey_glass_rhs	função	(x, x_tau, beta=0.2, gamma=0.1, n=10.0)	
double_well_drift	função	(x, a=1.0, b=1.0)	Drift of a particle in the quartic potential V(x) = -a x^2 / 2 + b x^4 / 4 (minima at +-sqrt(a / b)).
mackey_glass_step	função	(hist, dt, beta=0.2, gamma=0.1, n=10.0) -> np.ndarray	Advance Mackey-Glass histories by one RK4 step of size dt (method of steps).
largest_lyapunov_benettin	função	(step, x0, *, step_time, n_steps=20000, transient=2000, d0=1e-08, seed=0) -> float	Largest Lyapunov exponent (per unit time) of a deterministic map x -> step(x) by two-trajectory renormalisation (Benettin, Galgani, Giorgilli & Strelcyn 1980, Meccanica 15:9).
mackey_glass_lyapunov	função	(*, dt_int=0.1, total_time=50000.0, transient_time=2000.0, tau=17.0, beta=0.2, gamma=0.1,…)	Largest Lyapunov exponent (per unit time, natural log) of the Mackey-Glass delay equation.
lorenz63	função	(n=6000, *, dt=0.05, sub=5, noise=0.1, seed=0, sigma=10.0, rho=28.0, beta=8.0 / 3.0, obse…)	Lorenz-63 sampled every dt (RK4 with sub substeps); observation noise noise x std per column.
lorenz96	função	(n=6000, *, K=40, F=8.0, dt=0.05, sub=5, noise=0.1, seed=0, lyapunov_steps=20000) -> Synt…)	Lorenz-96 with K variables, every variable observed, target variable 0.
mackey_glass	função	(n=6000, *, dt=3.0, dt_int=0.1, tau=17.0, beta=0.2, gamma=0.1, n_exp=10.0, noise=0.1, see…)	Mackey-Glass sampled every dt, integrated with RK4 at dt_int (tau must be a multiple of dt_int).
ornstein_uhlenbeck	função	(n=6000, *, dt=0.1, theta=1.0, sigma=1.0, noise=0.1, seed=0) -> SyntheticSeries	Exact sampling: x_{k+1} = e^{-theta dt} x_k + sqrt(sigma^2 (1 - e^{-2 theta dt}) / (2 theta)) eps.
oscillator_transition	função	(omega, zeta, sigma, dt)	Exact discretisation of d[x, v] = A [x, v] dt + [0, sigma] dW with A = [[0, 1], [-omega^2, -2 zeta omega]].
stochastic_oscillator	função	(n=6000, *, dt=0.25, omega=1.0, zeta=0.05, sigma=0.3, noise=0.1, seed=0) -> SyntheticSeri…)	Damped oscillator driven by white noise, position observed with noise (velocity hidden).
double_well	função	(n=6000, *, dt=0.1, sigma=0.7, sub=100, a=1.0, b=1.0, noise=0.1, seed=0) -> SyntheticSeri…)	Double-well Langevin dynamics (Euler-Maruyama with sub substeps per sample): two regimes at +-1.
white_noise	função	(n=6000, *, seed=0) -> SyntheticSeries	

nome	tipo	assinatura	o que faz
`random_walk`	função	`(n=6000, *, seed=0) -> SyntheticSeries`	
`ode_stepper`	função	`(rhs, dt, sub=1) -> Callable[[np.ndarray], np.ndarray]`	A map advancing a batch of states (m, d) of dx/dt = rhs(x) by `dt` (`sub` RK4 substeps).
`mackey_glass_stepper`	função	`(dt_int=0.1, sub=10, **params) -> Callable[[np.ndarray], np.ndarray]`	A map advancing Mackey-Glass histories (m, tau / dt_int + 1) by `sub` RK4 steps of `dt_int`.
`linear_sde_stepper`	função	`(Phi, Q, seed=0) -> Callable[[np.ndarray], np.ndarray]`	A map x -> Phi x + w for a batch of states, with one noise draw w ~ N(0, Q) per call shared by the whole batch: members differ only in their initial state (sensitivity to initial conditi- ons, same forcing).

pinneapple_systems.time_series.tuning

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/tuning/__init__.py` · 4 linhas · **sem teste direto**

pinneapple_systems.time_series.tuning.optuna_tuner

Optuna-based HPO for all timeseries forecasters (neural + classical).

`pinneapple_systems/time_series/tuning/optuna_tuner.py` · 265 linhas · testes: 151

nome	tipo	assinatura	o que faz
`ClassicalTuner`	classe	`(model_type='xgboost', n_trials=30, direction='minimize', metric='mae', timeout=None)`	Optuna HPO wrapper for classical ML forecasters.
`NeuralTuner`	classe	`(model_type='lstm', n_trials=20, epochs_per_trial=20, lr=0.001, batch_size=64, input_len=…)`	Optuna HPO for PyTorch forecasters.
`temporal_split`	função	`(X, y, val_frac=0.15, test_frac=0.15)`	Chronological 3-way split → (X_tr, y_tr, X_val, y_val, X_te, y_te).

pinneapple_systems.time_series.uncertainty

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/uncertainty/__init__.py` · 2 linhas · **sem teste direto**

pinneapple_systems.time_series.uncertainty.quantile

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/uncertainty/quantile.py` · 8 linhas · **sem teste direto**

pinneapple_systems.time_series.validation

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/validation/__init__.py` · 16 linhas · testes: 20

pinneapple_systems.time_series.validation.backtest

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/validation/backtest.py` · 111 linhas · testes: 151

nome	tipo	assinatura	o que faz
`BacktestConfig`	classe	`(reset_model_each_fold, shuffle_train, val_batch_size)`	Backtesting settings.
`BacktestResult`	classe	`(folds, agg)`	
`BacktestRunner`	classe	`(*, trainer_factory, model_factory, datamodule, splits, train_cfg, preprocess_factory=Non…)`	Train/evaluate a model across a sequence of time-series splits.

pinneapple_systems.time_series.validation.splitters

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/validation/splitters.py` · 163 linhas · testes: 151

nome	tipo	assinatura	o que faz
`Split`	classe	`(fold, train_idx, val_idx, test_idx)`	A single backtesting fold expressed as indices of the windowed dataset.
`ExpandingWindowSplitter`	classe	`(initial_train_size, val_size, step_size, gap, max_folds)`	Walk-forward validation with an expanding training window.
`RollingWindowSplitter`	classe	`(train_size, val_size, step_size, gap, max_folds)`	Walk-forward validation with a fixed-size rolling training window.
`PurgedWalkForwardSplitter`	classe	`(n_folds, val_size, purge, embargo, window, min_train)`	Walk-forward (rolling-origin) validation over forecast origins with purge and embargo.
`PurgedKFoldSplitter`	classe	`(n_folds, purge, embargo)`	K contiguous folds; training on both sides of the validation fold with purge before and embargo after.
`ShuffledKFoldSplitter`	classe	`(n_folds, seed)`	Ordinary shuffled k-fold, which ignores time: kept to measure how optimistic it is on serial data.

pinneapple_systems.time_series.viz

Módulo sem docstring e sem texto.

`pinneapple_systems/time_series/viz/__init__.py` · 18 linhas · **sem teste direto**

pinneapple_systems.time_series.viz.animation

Animated rolling forecast visualization with optional MP4 export.

`pinneapple_systems/time_series/viz/animation.py` · 114 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`animate_rolling_forecast`	função	`(y_true, y_pred_seq, y_std_seq=None, horizon=16, context_len=64, interval=80, sigmas=(1.0…)`	Create an animated rolling forecast.

`pinneapple_systems.time_series.viz.forecast`

Forecast visualization: rolling predictions, confidence bands, parity plots.

`pinneapple_systems/time_series/viz/forecast.py` · 209 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_rolling_forecast`	função	`(y_true, y_pred, y_std=None, sigmas=(1.0, 2.0, 3.0), title='Rolling Forecast', xlabel='Ti…)`	Plot actual vs. predicted with ±1σ/2σ/3σ confidence bands.
`plot_forecast_horizon`	função	`(y_context, y_true, y_pred, y_std=None, sigmas=(1.0, 2.0), title='Forecast', figsize=(12,…)`	Plot context + future ground truth + forecast with confidence bands.
`plot_parity`	função	`(y_true, y_pred, title='Parity Plot', figsize=(5, 5), ax=None)`	Scatter plot of actual vs. predicted with diagonal reference line.
`plot_residuals`	função	`(y_true, y_pred, title='Residual Analysis', figsize=(14, 4))`	Three-panel: residuals vs time, histogram, Q-Q plot.
`plot_backtest`	função	`(y_true, predictions, title='Backtest Comparison', figsize=(14, 5))`	Overlay multiple model predictions on ground truth.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 104 de 182.
- Funções de teste em arquivos que importam algum módulo do pacote: 579.

Itens do catálogo de métodos (`pinneapple_catalog/method_status.json`) cujo código está neste pacote:

id	método	status	testes contra referência	outros testes
E22	Euler-Bernoulli beam	validated	1	11
E23	von Karman beam / plate	validated	2	6
E44	Reaction network / Arrhenius / mass action	tested	0	10
E47	Colebrook-White	tested	0	8
E49	Herschel-Bulkley + Metzner-Reed (non-Newtonian pipe flow)	tested	0	4
E51	Lame thick-walled cylinder	tested	0	3
E52	von Mises equivalent stress	tested	0	11
E53	Euler buckling + constrained rod buckling	validated	1	6
E54	Beam-column moment amplification factor	tested	0	3
E55	S-N curve + Goodman + Miner	validated	1	14

id	método	status	testes contra referência	outros testes
E56	Torsional stick-slip with Stribeck friction	tested	0	6
E57	Heat exchanger (effectiveness-NTU)	tested	0	8
E59	Control valve (Cv, compressible / incompressible)	tested	0	1
E60	Polytropic compression path	tested	0	1
E61	Real gas equation of state (CoolProp)	tested	0	2
E62	Compressor similarity map	validated	2	0
S20	Co-simulation (graph + engine, algebraic loops by Tarjan SCC)	tested	0	11
S21	Digital twin state estimation: KF/EKF/UKF/EnKF, prognostics, signal reconstruction	tested	2	39
S23	Explicit equation-system solver (evaluation order)	tested	0	1
S25	Causal telemetry conditioning (Hampel one-step, rate limit, first-order filter, gaps)	validated	1	6
T13	Uncertainty: heteroscedastic, B-PINN, MC Dropout, deep ensembles	tested	0	20

Limitações conhecidas

- *Nenhuma registrada.*

3. `pinneapple_twin3d`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
19	2.820	19	67	74 (7 arquivos)	4

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`demo`		1	1	6
`gallery`		1	11	16
`library`		1	0	0
`openfoam`		1	6	6
`primitives`		1	12	16
`scan`		1	4	4
`scene`		1	5	9
`serve`		1	1	6
`usd`		1	1	5
`wear`		9	45	30

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_twin3d`

3D digital-twin visualization: build a scene in Python, open it in the bundled web viewer.

`pinneapple_twin3d/__init__.py` · 26 linhas · testes: 34

`pinneapple_twin3d.demo`

Demo scene: a 90-degree pipe bend with erosion wear growing over time, plus two sensors.

`pinneapple_twin3d/demo.py` · 68 linhas · testes: 6

nome	tipo	assinatura	o que faz
`pipe_bend_scene`	função	`(years=(0, 1, 2, 3, 4, 5)) -> Scene`	

`pinneapple_twin3d.gallery`

Gallery: 3D twins of very different parts built from primitives, to exercise the generator beyond liners.

`pinneapple_twin3d/gallery.py` · 438 linhas · testes: 16

nome	tipo	assinatura	o que faz
`get`	função	`(key) -> Scene`	
`runner_channel`	função	`() -> Scene`	Blast-furnace cast-house runner: refractory-lined U trough from the tap hole to the skimmer.
`mixing_tank`	função	`() -> Scene`	Agitated tank: dished bottom, baffles, two-stage pitched-blade impeller; tracer blending over time.
`oil_pipeline`	função	`() -> Scene`	Buried-style crude line with two bends and a sag: wall loss from corrosion/erosion over 20 years.
`airliner`	função	`() -> Scene`	Narrow-body airliner: fuselage, swept wing, tail, two engines; skin temperature and wing-root load over a flight.
`rocket`	função	`() -> Scene`	Launch vehicle: ogive nose, tank stack, bell nozzle, four fins; aero heating and nozzle ablation during ascent.
`drone`	função	`() -> Scene`	Quadcopter: body, arms, motors, rotor discs, battery; motor and battery heating over a 20-minute flight.
`lunar_rover`	função	`() -> Scene`	Six-wheel lunar rover over one lunar day-night cycle: chassis temperature and regolith wheel wear.
`wind_turbine`	função	`() -> Scene`	Three-blade turbine: tapered tower, nacelle, twisted blades; leading-edge erosion and fatigue over 20 years.
`satellite`	função	`() -> Scene`	Small satellite in LEO: bus, two solar wings, dish; thermal cycling through sunlight and eclipse.
`export_all`	função	`(folder) -> str`	Export every gallery part to `folder/<key>/` and write `folder/index.html` linking them.

`pinneapple_twin3d.library`

The geometry library from the twin side: `pinneapple_twin3d.library.build("viper_rover").scene()`.

`pinneapple_twin3d/library.py` · 8 linhas · **sem teste direto**

pinneapple_twin3d.openfoam

OpenFOAM case -> Twin3D scene: wall patches as surfaces, per-face values over time.

`pinneapple_twin3d/openfoam.py` · 215 linhas · testes: 6

nome	tipo	assinatura	o que faz
`read_points`	função	`(case) -> np.ndarray`	
`read_faces`	função	`(case) -> List[np.ndarray]`	
`read_boundary`	função	`(case) -> Dict[str, Dict[str, object]]`	`{patch: {"type": str, "startFace": int, "nFaces": int}}` in file order.
`time_directories`	função	`(case) -> List[Tuple[float, str]]`	
`read_boundary_field`	função	`(path, patch, n_faces) -> Optional[np.ndarray]`	Per-face values of `patch` in a volScalarField/volVectorField file (vectors -> magnitude).
`scene_from_case`	função	`(case, *, title=None, patches=None, face_values=None, units=None, boundary_fields=(), tim…)`	Build a Twin3D scene from an OpenFOAM case.

pinneapple_twin3d.primitives

Mesh primitives for building twin parts without a CAD file: revolve, sweep, loft, box, cylinder.

`pinneapple_twin3d/primitives.py` · 183 linhas · testes: 16

nome	tipo	assinatura	o que faz
`revolve`	função	`(profile, n_theta=48, *, cap=False) -> Mesh`	Surface of revolution of `(r, y)` points around the y axis (`cap` closes the ends with a fan).
`sweep`	função	`(section, path, *, closed=True, up=(0, 1, 0)) -> Mesh`	Extrude a 2D section `(u, v)` along a 3D path. `u` runs along the frame normal (closest to `up`), `v` along the binormal. `closed=False` leaves an open profile (e.g. a U-channel).
`airfoil`	função	`(chord=1.0, thickness=0.12, n=24) -> np.ndarray`	Closed symmetric NACA 00xx outline, `(2n, 2)`: x along the chord from the leading edge, y thickness.
`loft`	função	`(rings) -> Mesh`	Skin through rings of equal point count `(K, 3)` (closed rings): wings, blades, fuselage sections.
`wing`	função	`(span, root_chord, tip_chord, *, sweep_deg=0.0, dihedral_deg=0.0, twist_deg=0.0, thicknes…)`	Half wing / blade along +z from the root (z=0): chord along x (leading edge at -x), thickness along y.
`box`	função	`(size, center=(0, 0, 0)) -> Mesh`	
`cylinder`	função	`(p0, p1, radius, n_theta=32, *, cap=True) -> Mesh`	Cylinder between two points (`radius` may be a pair for a frustum).
`rotation`	função	`(axis, angle_deg) -> np.ndarray`	

nome	tipo	assinatura	o que faz
transform	função	`(mesh, *, R=None, t=(0, 0, 0), scale=1.0) -> Mesh`	
mirror_z	função	`(mesh) -> Mesh`	
merge	função	`(meshes) -> Mesh`	
mesh_quality	função	`(mesh) -> dict`	Sanity numbers used by the tests: finite coordinates, indices in range, degenerate-triangle share, bounds.

pinneapple_twin3d.scan

Twin geometry from scans: cleaned scan meshes and MuJoCo scenes (e.g. LiteReality output).

`pinneapple_twin3d/scan.py` · 199 linhas · testes: 4

nome	tipo	assinatura	o que faz
ScanReport	classe	`(vertices_in, faces_in, vertices_out, faces_out, components_dropped, holes_filled, watert…)`	
clean_scan	função	`(path_or_mesh, *, unit='m', min_component_fraction=0.01, fill_holes=True, target_faces=No…)`	Clean a scan mesh. Returns `(trimesh.Trimesh, ScanReport)` in metres.
add_scan	função	`(scene, name, path_or_mesh, *, group='scan', **kw) -> ScanReport`	
scene_from_mjcf	função	`(path, *, title=None, include_primitives=True) -> Scene`	One Twin3D part per geom of a MuJoCo scene, in world coordinates.

pinneapple_twin3d.scene

Build and export a 3D digital-twin scene: parts, fields over time, sensors.

`pinneapple_twin3d/scene.py` · 360 linhas · testes: 9

nome	tipo	assinatura	o que faz
Part	classe	`(name, vertices, faces, group, color, fields, motion, visibility, physics)`	
PhysicsMaterial	classe	`(name, static_friction, dynamic_friction, restitution, density)`	Contact material (UsdPhysics `PhysicsMaterialAPI`).
PhysicsProps	classe	`(rigid_body, collider, kinematic, mass, density, center_of_mass, approximation, material,…)`	How a part takes part in a physics simulation (OpenUSD `UsdPhysics`).
Sensor	classe	`(id, position, label, unit, quantity, series, envelope)`	
Scene	classe	`(title, *, length_unit='m', times=None, time_unit='s', source='')`	A digital-twin scene. Add parts, fields and sensors, then `export` to a folder.

pinneapple_twin3d.serve

Serve an exported scene folder over HTTP (the viewer needs HTTP, not file://, to fetch its data).

`pinneapple_twin3d/serve.py` · 26 linhas · testes: 6

nome	tipo	assinatura	o que faz
serve	função	`(folder, port=8765, *, host='127.0.0.1', block=True)`	Serve `folder` at `http://host:port/`. With `block=False` returns the running server.

pinneapple_twin3d.usd

Export a Twin3D scene as OpenUSD (`.usda` text), for Omniverse / Isaac Sim / usdview.

`pinneapple_twin3d/usd.py` · 209 linhas · testes: 5

nome	tipo	assinatura	o que faz
export_usd	função	`(scene, path) -> str`	

pinneapple_twin3d.wear

Generator of 3D wear (refractory / liner / pipe) digital twins from a spec.

`pinneapple_twin3d/wear/__init__.py` · 21 linhas · testes: 30

pinneapple_twin3d.wear.build

WearDataset -> Scene (3D viewer) + hub contract + forecast/optimizer report.

`pinneapple_twin3d/wear/build.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
remaining_life_fields	função	`(ds, cap, **kw) -> dict`	Per-cell remaining life at every reading, capped at `cap` (no trend -> cap; reading 0 -> cap).
scene_from_wear	função	`(ds, *, life_cap=None, **fit_kw) -> Scene`	
report	função	`(ds, cfg=None, **fit_kw) -> dict`	Observed / predicted / recommended, kept apart on purpose.
export_wear_twin	função	`(ds, folder, *, cfg=None, usd=False, **fit_kw) -> str`	Write the 3D viewer scene, `wear_dataset.json` (hub contract) and `report.json` to `folder`.

pinneapple_twin3d.wear.calibrate

Calibrate the optimizer's assumptions from the plant's own records.

`pinneapple_twin3d/wear/calibrate.py` · 138 linhas · testes: 30

nome	tipo	assinatura	o que faz
StopRecord	classe	`(zones, duration_h, cost, x)`	
ParamReport	classe	`(name, value, source, n, se, note)`	
Calibration	classe	`(cfg, report)`	
estimate_recovery	função	`(ds, repair_drop=0.08, min_cells=20)`	Median fraction of the usable thickness restored per repaired cell, and the number of cells used.

nome	tipo	assinatura	o que faz
`calibrate`	função	`(stops=(), ds=None, base=None) -> Calibration`	

`pinneapple_twin3d.wear.forecast`

Remaining-life forecast per cell: the model of the reference twins, plus confidence.

`pinneapple_twin3d/wear/forecast.py` · 185 linhas · testes: 30

nome	tipo	assinatura	o que faz
`CellFit`	classe	`(remaining, remaining_lo, remaining_hi, slope, r2, n, confidence)`	
`ZoneForecast`	classe	`(zone, worst_fraction, n_over, min_remaining, rate, usable, critical_cell, confidence, r2…)`	
`t90`	função	`(df)`	Student-t 0.95 quantile (90 % two-sided interval) for `df` degrees of freedom (array ok).
`confidence_index`	função	`(n, r2)`	
`fit_zone`	função	`(ds, zone, s, *, win_x=DEFAULT_WIN_X, min_pts=8, repair_drop=0.08) -> CellFit`	
`forecast`	função	`(ds, s=None, **kw) -> Dict[str, ZoneForecast]`	
`forecast_history`	função	`(ds, **kw) -> List[dict]`	Minimum predicted remaining life recomputed at every reading (how the forecast evolved).
`backtest`	função	`(ds, **kw) -> dict`	Compare past predictions of 'x when the first cell hits the limit' with what happened.
`status`	função	`(fraction) -> str`	

`pinneapple_twin3d.wear.geometry`

Cell meshes for surfaces of revolution.

`pinneapple_twin3d/wear/geometry.py` · 62 linhas · testes: 30

nome	tipo	assinatura	o que faz
`sample_profile`	função	`(profile, n_rows) -> np.ndarray`	(n_rows + 1, 2) points (r, y) equally spaced in arc length along the polyline.
`cell_mesh`	função	`(zone) -> Tuple[np.ndarray, np.ndarray]`	Vertices (N*M*4, 3) and faces (N*M*2, 3) of the zone, one quad per cell.
`cell_to_vertex`	função	`(values) -> np.ndarray`	Per-cell values (..., N, M) -> per-vertex (..., N*M*4) following `cell_mesh`'s ordering.
`cell_centers`	função	`(zone) -> np.ndarray`	(N, M, 3) world position of each cell centre (used to place sensors / hotspots).

pinneapple_twin3d.wear.model

Data model of a wear twin: vessel spec (geometry + zones) and the wear dataset.

`pinneapple_twin3d/wear/model.py` · 165 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ZoneSpec`	classe	`(name, label, profile, n_rows, n_sectors, e0, emin, center, theta0, group)`	
`VesselSpec`	classe	`(key, title, zones, x_name, wear_unit, notes)`	
`WearDataset`	classe	`(spec, x, wear, origin, labels, note)`	Wear depth [mm] per zone: `wear[zone]` has shape (T, rows, sectors); NaN = no reading.

pinneapple_twin3d.wear.optimize

Maintenance (gunning / repair) scheduling over the forecast. Decision support only.

`pinneapple_twin3d/wear/optimize.py` · 166 linhas · testes: 30

nome	tipo	assinatura	o que faz
`OptimizerConfig`	classe	`(horizon, heats_per_day, recovery_frac, setup_hours, hours_per_zone, cost_per_hour, cost_…)`	
`ZoneState`	classe	`(zone, deadline, life_full, ext)`	
`Policy`	classe	`(id, name, schedule, stops, zones_served, downtime_hours, breach, cost, window)`	
`OptimizationResult`	classe	`(cfg, zones, policies, recommended, baseline, notes)`	
`prepare`	função	`(zones, cfg) -> List[ZoneState]`	
`simulate`	função	`(zs, stops, cfg) -> dict`	
`fixed_policy`	função	`(zs, cfg) -> List[dict]`	
`predictive_policy`	função	`(zs, cfg, window) -> List[dict]`	
`optimize`	função	`(forecasts, cfg=None) -> OptimizationResult`	
`explain`	função	`(res) -> List[dict]`	

pinneapple_twin3d.wear.presets

The four reference use cases as specs: steel ladle, pig-iron ladle, BOF converter, RH degasser.

`pinneapple_twin3d/wear/presets.py` · 176 linhas · testes: 30

nome	tipo	assinatura	o que faz
`get`	função	`(key)`	
`fill_level`	função	`(rin, y0, y1, rho, mass, n=300) -> float`	Bath level for `mass` [t] in a vessel of inner radius `rin(y)` (frustum integration).

nome	tipo	assinatura	o que faz
`steel_ladle`	função	`()`	
`pig_iron_ladle`	função	`()`	
`bof_converter`	função	`()`	
`rh_degasser`	função	`()`	
`smelting_reduction_reactor`	função	`()`	Generic smelting-reduction shaft reactor: shaft, tuyere zone, crucible, hearth.

`pinneapple_twin3d.wear.synthetic`

Synthetic wear campaigns: explicit hot spots + growth law + repairs. Always labelled SYNTHETIC.

`pinneapple_twin3d/wear/synthetic.py` · 80 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HotSpot`	classe	`(zone, s, theta_deg, ds, dtheta_deg, amp)`	
`base_pattern`	função	`(spec, hotspots, base) -> Dict[str, np.ndarray]`	
`synthetic_campaign`	função	`(spec, hotspots, *, n_readings=41, x_max=120.0, growth_power=1.0, base=0.2, noise=0.01, s…)`	`noise` is the sigma of additive measurement noise as a fraction of the usable thickness. Wear grows as `(x/x_max)**growth_power` towards `pattern * usable`; at each `repairs` x the wear of `repair_zones` drops by `repair_gain` of its value (gunning), then resumes.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 15 de 19.
- Funções de teste em arquivos que importam algum módulo do pacote: 74.

Limitações conhecidas

- *Nenhuma registrada.*

Volume 7 — Ferramentas, orquestração e produtos

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Ferramentas (benchmarks, exportação, visualização, automação de GUIs de engenharia, NVIDIA), o Flow, o Hub, o Lab, os LLMs, o servidor MCP, o laboratório autônomo e o app web.

Sumário

1. pinneapple_tools
2. pinneapple_blender
3. pinneapple_flow
4. pinneapple_hub
5. pinneapple_lab
6. pinneapple_llm
7. pinneapple_mcp
8. pinneapple_autolab
9. pinneapple_app

1. `pinneapple_tools`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
177	34.350	155	384	465 (35 arquivos)	107

Pasta de testes dedicada: `tests/pinneapple_tools` (30 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`aero_coefficients`		1	4	7
`automation`		22	62	32
`benchmark_suite`		52	118	155
`compute_backends`		3	8	156
`dataset_quality`		3	16	16
`hpo_experiments`		27	44	0
`model_evaluation`		3	12	5
`model_export`		1	4	0
`model_services`		6	17	39
`nvidia`		11	49	39

submódulo	o que faz	módulos	classes + funções	testes*
rea_bridge		6	20	22
reporting		3	14	14
sandbox		2	1	13
surrogate_policy		1	2	5
visualization		35	165	46

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_tools`

pinneapple_tools — Visualization, model export, HPO, benchmarking, and compute backends.

pinneapple_tools/__init__.py · 237 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot	função	`(model, x, *, field_name='u', dim=None, **kwargs)`	Quick visualization of a model prediction over points `x`.
export_model	função	`(model, path, fmt='torchscript', **kwargs) -> str`	Export a trained model to the requested format.
run_benchmark	função	`(models, tasks=None, **bench_kwargs) -> 'BenchmarkResult'`	Run a quick benchmark comparison across models.

`pinneapple_tools.aero_coefficients`

Shared aerodynamic-coefficient utilities (Cp, surface-force integration, CL/CD/CM).

pinneapple_tools/aero_coefficients.py · 293 linhas · testes: 7

nome	tipo	assinatura	o que faz
AeroCoefficients	classe	`(CL, CD, CM, CY)`	Non-dimensional aerodynamic coefficients about the wind axes.
compute_cp	função	`(p, p_inf, q_inf) -> np.ndarray`	Pressure coefficient `Cp = (p - p_inf) / q_inf`.
integrate_surface_forces	função	`(vertices, faces, face_normals, face_pressure, face_shear=None, moment_center=None) -> Tu…)`	Integrate per-face pressure (and optional shear) into a net force and moment over a triangulated surface.
force_to_coefficients	função	`(force, moment, q_inf, ref_area, ref_length, alpha_deg, *, drag_axis='x', lift_axis='z', …)`	Convert a net body-axis force/moment into wind-axis CL/CD/CM.

`pinneapple_tools.automation`

Engineering GUI agent: run a task in engineering software and deliver the files plus the evidence of what happened ("Selenium for computational engineering").

pinneapple_tools/automation/__init__.py · 84 linhas · testes: 32

nome	tipo	assinatura	o que faz
example_workflow	função	`(name) -> Path`	Path of a committed example workflow (`examples/<name>/workflow.yaml`).

nome	tipo	assinatura	o que faz
`catalog`	função	`(folder=None) -> dict[str, Workflow]`	Every workflow under `folder` (default: the committed examples), by id.

`pinneapple_tools.automation.adapters`

Adapters: one per application. Importing this package registers all of them.

`pinneapple_tools/automation/adapters/__init__.py` · 16 linhas · testes: 16

`pinneapple_tools.automation.adapters.base`

What every adapter implements, and the context a step runs in.

`pinneapple_tools/automation/adapters/base.py` · 135 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ActionFailed`	classe	`()`	The action ran and failed; the message says why, the log says where.
`ActionResult`	classe	`(outputs, artifacts, evidence, checks, actions)`	
`StepContext`	classe	`(run_dir, step_dir, toolchain, display, workflow_dir, criteria, previous, timeout_s, comm…)`	
`Adapter`	classe	`(requires, actions, needs_display)`	
`register`	função	`(adapter) -> Adapter`	
`get_adapter`	função	`(name) -> Adapter`	
`list_adapters`	função	`() -> dict[str, Adapter]`	
`known_actions`	função	`() -> set[str]`	

`pinneapple_tools.automation.adapters.checks`

The `verify` adapter: evaluate the workflow's acceptance criteria (`verify.criteria`) on the run's files, and the `web` adapter: open a web UI with Playwright and capture it (`web.capture`).

`pinneapple_tools/automation/adapters/checks.py` · 71 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VerifyAdapter`	classe	`()`	
`WebAdapter`	classe	`()`	

`pinneapple_tools.automation.adapters.freecad`

FreeCAD and the CfdOF workbench.

`pinneapple_tools/automation/adapters/freecad.py` · 195 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FreeCADAdapter`	classe	`()`	

nome	tipo	assinatura	o que faz
`CfdOFAdapter`	classe	`()`	
`run_in_freecad_gui`	função	`(script, args, ctx, name, view=None, hold_s=60.0) -> tuple[dict[str, Any], ActionResult]`	Run `script` inside the FreeCAD GUI with `ARGS = args`; returns (RESULT, evidence/commands).

`pinneapple_tools.automation.adapters.openfoam`

OpenFOAM through its command line: run the mesh script and the solver script of a case (CfdOF's `Allmesh` and `Allrun`, or any command list), then read what happened from the logs with :func:`pinneapple_data.simulation_metadata.openfoam.parse_log` (end reached, fatal errors, residual history, ...

`pinneapple_tools/automation/adapters/openfoam.py` · 223 linhas · testes: 16

nome	tipo	assinatura	o que faz
`OpenFOAMAdapter`	classe	`()`	
`patch_cfdof_allmesh`	função	`(mesh_case) -> str | None`	Make CfdOF's `Allmesh` use the `-scale` syntax on v-numbered (ESI) releases that do not set FOAM_API. Returns a description of the edit, or None when the script needed none.
`patch_cfdof_inlet`	função	`(case) -> list[str]`	Replace CfdOF's `fixedNormalInletOutletVelocity` inlets in `0/U` by `fixedValue` with the same normal velocity. On OpenFOAM v1912 the mixed condition is not assignable, so with the zeroGradient inlet pressure the pressure equation changed the inlet flux (6 % below U*A on the channel example). ...
`solver_log`	função	`(case) -> tuple[Path | None, dict[str, Any]]`	The solver log of a case (log.simpleFoam, log.foamRun, ...) and its parsed content.
`residual_plot`	função	`(parsed, path, title) -> Path | None`	

`pinneapple_tools.automation.adapters.paraview`

ParaView through `pvpython`: open an OpenFOAM case at its last time, export a cell field to CSV (cell centres and values) and render it on a slice to PNG (`render` evidence).

`pinneapple_tools/automation/adapters/paraview.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ParaViewAdapter`	classe	`()`	

`pinneapple_tools.automation.adapters.pipeline`

Adapters of the FreeCAD -> OpenFOAM pipeline (:mod:`pinneapple_simulation.pipelines.freecad_openfoam`):

`pinneapple_tools/automation/adapters/pipeline.py` · 57 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FreeCADHeadlessAdapter`	classe	`()`	
`pipeline_call`	função	`(fn, params, ctx) -> ActionResult`	

pinneapple_tools.automation.display

The screen GUI applications run on: a virtual X display (Xvfb) on a Linux server or container, screenshots of it, and the three ways of driving a GUI when an application has no usable API.

`pinneapple_tools/automation/display.py` · 278 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GuiUnavailable`	classe	`()`	The GUI driver cannot run here (missing tool, wrong platform, no display).
`VirtualDisplay`	classe	`(width=1600, height=1000, depth=24, reuse=False)`	An Xvfb display for the duration of a `with` block (or until :meth:`stop`).
`XdotoolDriver`	classe	`(display)`	Find and act on X11 windows. Every call is recorded in :attr:`log` (the commands, for the evidence report).
`PlaywrightDriver`	classe	`(headless=True, executable_path=None)`	Drive a web UI: open a page, click, fill, wait for text, take screenshots. Every action is logged.
`PywinautoDriver`	classe	`(backend='uia')`	Windows desktop GUIs through `pywinauto` (UI Automation backend).

pinneapple_tools.automation.examples.channel_obstacle.build_project

Módulo sem docstring e sem texto.

`pinneapple_tools/automation/examples/channel_obstacle/build_project.py` · 33 linhas · **sem teste direto**

pinneapple_tools.automation.interpret

From a task to a structured workflow.

`pinneapple_tools/automation/interpret.py` · 155 linhas · testes: 16

nome	tipo	assinatura	o que faz
`Interpretation`	classe	`(workflow, inputs, method, notes, score)`	
`parse_quantity`	função	`(text, unit) -> float | None`	The number in `text` converted to `unit` ("4 cm" -> 40 for mm); None if there is no number.
`interpret`	função	`(task, catalog, llm=None, min_score=0.15) -> Interpretation`	Turn `task` into a workflow of `catalog` and its inputs (see the module docstring).

pinneapple_tools.automation.record

What a run leaves behind: one :class:`StepRecord` per step (state, commands, logs, artifacts with their SHA-256, screenshots, checks, outputs, error) and the :class:`RunRecord` around them, saved as `run.json` in the run folder.

`pinneapple_tools/automation/record.py` · 171 linhas · testes: 14

nome	tipo	assinatura	o que faz
Check	classe	`(name, passed, detail, value, threshold, source)`	
Evidence	classe	`(path, kind, caption, sha256)`	An image of what happened. `kind` says what it is, so a report never passes a plot off as a screenshot: `screen` (capture of the whole virtual display), `window` (one application window), `render` (the application's own image export), `plot` (a chart drawn from the step's log or data).
FileRecord	classe	`(path, bytes, sha256, kind)`	
StepRecord	classe	`(id, title, action, mode, state, started, ended, commands, logs, artifacts, evidence, che…)`	
RunRecord	classe	`(run_id, workflow, inputs, folder, state, created, finished, steps, environment, criteria…)`	
file_record	função	`(path, root) -> FileRecord`	Size and SHA-256 of a file; for a folder, the total size and a hash over its sorted file hashes.

pinneapple_tools.automation.report

The evidence of a run, for people and for other tools.

`pinneapple_tools/automation/report.py` · 234 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
write_report	função	`(run, path) -> Path`	
to_study_result	função	`(run)`	The run as a :class:`pinneapple_data.contracts.StudyResult` (`study_type="runs"`, `app="guiagent"`).
write_study_result	função	`(run, path) -> Path`	

pinneapple_tools.automation.runner

Run a :class:`~pinneapple_tools.automation.workflow.Workflow` step by step, observe each step and stop on the first failure or on the first step that needs a human decision.

`pinneapple_tools/automation/runner.py` · 290 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Runner`	classe	`(workflow, run_dir, inputs=None, toolchain=None, on_event=None, report=True)`	
`plan`	função	`(workflow, toolchain=None) -> list[dict[str, Any]]`	What running the workflow would need, without running anything: per step its adapter, mode, whether the application is installed here, and the confirmations it will ask for.

`pinneapple_tools.automation.runner_step`

The GUI agent as a step of the Simulation Workflow Runner (`pinneapple_core.workflows`).

`pinneapple_tools/automation/runner_step.py` · 95 linhas · testes: 16

nome	tipo	assinatura	o que faz
`GuiAgentStep`	classe	`(toolchain=None)`	
`register`	função	`(toolchain=None) -> GuiAgentStep`	Register the `guiagent` step kind with the workflow runner; returns the adapter.

`pinneapple_tools.automation.scripts.cfdof_setup`

Módulo sem docstring e sem texto.

`pinneapple_tools/automation/scripts/cfdof_setup.py` · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`on_plane`	função	`(face, plane)`	

`pinneapple_tools.automation.scripts.freecad_gui_job`

Módulo sem docstring e sem texto.

`pinneapple_tools/automation/scripts/freecad_gui_job.py` · 68 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pump`	função	`(seconds)`	
`run_job`	função	`()`	

`pinneapple_tools.automation.scripts.freecad_inspect`

Módulo sem docstring e sem texto.

`pinneapple_tools/automation/scripts/freecad_inspect.py` · 27 linhas · **sem teste direto**

`pinneapple_tools.automation.scripts.paraview_export`

Módulo sem docstring e sem texto.

`pinneapple_tools/automation/scripts/paraview_export.py` · 92 linhas · **sem teste direto**

pinneapple_tools.automation.toolchain

Where the engineering applications are installed and which versions they are.

`pinneapple_tools/automation/toolchain.py` · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Toolchain`	class e	`(freecad, freecadcmd, cfdof, foam_bashrc, gmsh, pvpython)`	

pinneapple_tools.automation.verify

Acceptance criteria on a finished run, computed from the produced files with PINNeAPPle's checks.

`pinneapple_tools/automation/verify.py` · 176 linhas · testes: 30

nome	tipo	assinatura	o que faz
`patch_fluxes`	função	`(case) -> dict[str, Any]`	Volume flux [m³/s] out of every boundary patch of the latest time, plus net and total (sum of |flux|).
`patch_mean`	função	`(case, field, patches) -> dict[str, float]`	Area-weighted mean of a scalar field on each named patch at the latest time.
`patch_area`	função	`(case, patch) -> float`	Area [m²] of a boundary patch of the mesh.
`inflow`	função	`(case, inlet, velocity, tol=0.01) -> Check`	Volume flow through `inlet` against `velocity` [m/s] x its area.
`mass_balance`	função	`(case, inlet, outlet, tol=0.01) -> Check`	
`pressure_drop`	função	`(case, inlet, outlet, density=None, min_pa=None, max_pa=None) -> tuple[Check, float]`	
`evaluate_criterion`	função	`(kind, params, run_dir, previous) -> Check`	One criterion of a workflow (see the module docstring). `previous` maps step ids to their records.

pinneapple_tools.automation.workflow

The structured workflow an automation run executes: inputs with bounds, steps with preconditions, the artifacts each step must produce, the confirmations it needs, and the acceptance criteria of the result.

`pinneapple_tools/automation/workflow.py` · 274 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`WorkflowError`	classe	`(errors)`	The workflow file or the run inputs are invalid; `errors` lists every problem found.
`ArtifactSpec`	classe	`(path, kind, min_bytes, description)`	A file or folder a step must leave behind. `path` is relative to the run directory.
`InputSpec`	classe	`(default, min, max, unit, description, choices)`	

nome	tipo	assinatura	o que faz
`Step`	classe	`(id, action, title, description, params, after, requires, expects, confirm, timeout_s)`	
`Criterion`	classe	`(id, kind, title, params)`	An acceptance criterion on the result, evaluated by the `verify.criteria` action.
`Workflow`	classe	`(id, title, steps, description, application, inputs, criteria, limits, source)`	
`render`	função	`(value, ctx) -> Any`	Fill the templates of `value` (str, list or dict, recursively) from `ctx` = {"inputs", "steps", "workdir", "example"}. A string that is exactly one template takes the referenced value as is.

pinneapple_tools.benchmark_suite

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/__init__.py` · 108 linhas · testes: 2

pinneapple_tools.benchmark_suite.api

Arena — high-level API for running PINNeAPPle resolution pipelines.

`pinneapple_tools/benchmark_suite/api.py` · 591 linhas · testes: 153

nome	tipo	assinatura	o que faz
`ArenaResult`	classe	`(spec, model, history, metrics, artifacts, model_id, elapsed_s)`	Container returned by Arena.run() and Arena.compare().
`ArenaCompareResult`	classe	`(spec, results)`	Container returned by Arena.compare() with multiple models.
`Arena`	classe	`(spec) -> None`	High-level runner that connects `ProblemSpec` to the training pipeline.

pinneapple_tools.benchmark_suite.backends

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/backends/__init__.py` · 17 linhas · **sem teste direto**

pinneapple_tools.benchmark_suite.backends.base

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/backends/base.py` · 20 linhas · testes: 151

nome	tipo	assinatura	o que faz
`Backend`	classe	`(name)`	Training backend protocol.

pinneapple_tools.benchmark_suite.backends.deepxde_backend

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/backends/deepxde_backend.py` · 113 linhas · **sem teste direto**

nome	tipo	assina-tura	o que faz
`DeepXDEBackend`	classe	`(name)`	DeepXDE backend (baseline supervisionado em sensores).

`pinneapple_tools.benchmark_suite.backends.jax_pinn`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/backends/jax_pinn.py` · 335 linhas · **sem teste direto**

nome	tipo	assina-tura	o que faz
`JAXPINNBackend`	classe	`(name)`	Full PINN backend in JAX for FlowObstacle2D (steady 2D Navier–Stokes).

`pinneapple_tools.benchmark_suite.backends.native_pinn`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/backends/native_pinn.py` · 287 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`MLP`	classe	`(in_dim, out_dim, hidden, activation='tanh')`	Fully connected multilayer perceptron (MLP).
`NativePINNBackend`	classe	`()`	Native PINN training backend for steady 2D Navier-Stokes problems.

`pinneapple_tools.benchmark_suite.backends.physicsnemo_sym`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/backends/physicsnemo_sym.py` · 240 linhas · testes: 2

nome	tipo	assina-tura	o que faz
`PhysicsNeMoSymBackend`	classe	`()`	Backend integration for PhysicsNeMo Sym (formerly Modulus Sym).

`pinneapple_tools.benchmark_suite.backends.pinneapple_models_backend`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/backends/pinneapple_models_backend.py` · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PinneappleModelsBackend`	classe	`()`	Train any model from ModelRegistry.

`pinneapple_tools.benchmark_suite.benchmark`

PINN Arena — multi-architecture, multi-problem benchmark suite.

`pinneapple_tools/benchmark_suite/benchmark.py` · 1.309 linhas · testes: 155 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`BenchmarkConfig`	classe	`(n_col, n_bc, n_ic, epochs, lr, weight_pde, weight_bc, weight_ic, device, seed, n_eval, c…)`	
`ModelSpec`	classe	`(name, factory, description, input_kind, family)`	
`BenchmarkResult`	classe	`(problem_id, model_id, metrics, history, elapsed_s, n_params, rank)`	
`BenchmarkTaskBase`	classe	`(task_id, in_dim, out_dim)`	Abstract base for physics benchmark tasks.
`NotApplicableError`	classe	`()`	Raised (by callers, not by this module) when a task can't build a batch for a given input_kind. `run()` doesn't raise this itself — it relies on `task.build_batch()` returning `None` — but callers that pre-check applicability (e.g. BiaML's `run_benchmark()`) can use this to signal "not …
`PINNArenaBenchmark`	classe	`(tasks, model_specs=None, config=None) -> None`	Multi-problem, multi-architecture PINN benchmark suite.
`all_model_specs`	função	`() -> List[ModelSpec]`	Return a ModelSpec for every model in `pinneapple_models`, tagged with its real `input_kind` (pointwise_coords / sequence / graph / grid* / dynamics / operator_branch_trunk / autoencoder — see `pinneapple_neural.architectures.registry.ModelSpec.input_kind`, now accurately declared per family …

`pinneapple_tools.benchmark_suite.bundle`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/bundle/__init__.py` · 7 linhas · **sem teste direto**

`pinneapple_tools.benchmark_suite.bundle.loader`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/bundle/loader.py` · 149 linhas · testes: 2

nome	tipo	assinatura	o que faz
`BundleData`	classe	`(root, manifest, conditions, points_collocation, points_boundary, sensors)`	Immutable container for a validated PINNeAPPle bundle.
`load_bundle`	função	`(bundle_root, *, schema, require_sensors=False) -> BundleData`	Load and validate a PINNeAPPle bundle from disk.

pinneapple_tools.benchmark_suite.bundle.schema

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/bundle/schema.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BundleSchema`	classe	`(schema_version, required_files, required_regions, manifest_required_keys, sensors_requir…)`	Schema definition for validating a PINNeAPPle bundle.
`load_bundle_schema`	função	`(path) -> BundleSchema`	Load a bundle schema definition from a YAML configuration file.

pinneapple_tools.benchmark_suite.crossframework

Neutral parts of the cross-framework benchmark (PINNeAPPle vs PhysicsNeMo / PhysicsNeMo-Sym, DeepXDE, neuraloperator): problem definitions and references, the shared point sets and initial weights, the one evaluator, statistics over seeds, run records and the operator-learning data and training …

`pinneapple_tools/benchmark_suite/crossframework/__init__.py` · 17 linhas · testes: 20

pinneapple_tools.benchmark_suite.crossframework.darcy

Track B of the cross-framework benchmark: the Darcy data, its normalisation and the one training loop.

`pinneapple_tools/benchmark_suite/crossframework/darcy.py` · 182 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DarcyData`	classe	`(a, u, a_mean, a_std, u_mean, u_std)`	The three splits and the training-split normalisation.
`load_split`	função	`(split, resolution=64) -> tuple[np.ndarray, np.ndarray]`	`(a, u)` of `split` in `("train", "val", "test")` at `resolution` (float32, (n, r, r)).
`load_darcy`	função	`(resolution=64, n_train=N_TRAIN) -> DarcyData`	
`train_operator`	função	`(model, data, *, epochs, seed, batch_size=20, lr=0.001, weight_decay=0.0001, log=None) ->…)`	Train `model` ((B, 1, r, r) -> (B, 1, r, r), normalised) with the protocol's loop. Returns the history (loss per epoch, wall and CPU time per epoch) and the decoded predictions on the validation and test splits.
`run_track_b`	função	`(argv, *, framework, source, build, model_config) -> None`	Shared body of the three track-B runners: the runner only says how its library builds the FNO.
`inference_latency`	função	`(model, data, batch, repeats=20, warmup=3) -> float`	Median wall time [s] of one forward pass on `batch` test inputs (no gradients).

pinneapple_tools.benchmark_suite.crossframework.evaluate

The one evaluator of the cross-framework benchmark, and the statistics over seeds.

pinneapple_tools/benchmark_suite/crossframework/evaluate.py · 141 linhas · testes: 20

nome	tipo	assinatura	o que faz
evaluate_pinn	função	(problem, u_test, u_stencil=None, *, reference=None) -> dict[str, float]	Metrics of a forward run from its exported predictions.
evaluate_inverse	função	(nu_hat, nu_true, u_pred, u_ref) -> dict[str, float]	Inverse run: relative error of the identified coefficient and the field error on the test points.
per_sample_rel_l2	função	(pred, ref) -> np.ndarray	Per-sample relative L2 over all axes but the first (operator learning).
evaluate_operator	função	(pred, ref) -> dict[str, float]	Operator run: mean, median and max of the per-sample relative L2 on the test split.
seed_stats	função	(values) -> dict[str, object]	Median, min, max and n of the finite values; n_failed counts NaN / inf (failures are not dropped silently: they are counted, and the median is over the finite values only).
paired_ratio	função	(a, b) -> dict[str, object]	Per-seed ratio a / b over the seeds both have (finite): median, min, max, and how many seeds a < b.
sign_test_p	função	(k, n) -> float	Two-sided exact sign test p-value for k successes in n paired trials (ties excluded beforehand). With n = 5 the smallest attainable p is 0.0625: five seeds cannot show significance at 0.05 by this test.
summarize	função	(rows, keys, by) -> list[dict[str, object]]	Group rows by the by fields and give :func:seed_stats of each metric in keys.

pinneapple_tools.benchmark_suite.crossframework.inverse

Track C of the cross-framework benchmark: identify the viscosity of Burgers from noisy samples.

pinneapple_tools/benchmark_suite/crossframework/inverse.py · 28 linhas · testes: 20

nome	tipo	assinatura	o que faz
observations	função	(n=C['n_obs'], noise_rel=C['noise_rel'], seed=C['obs_seed']) -> dict[str, np.ndarray]	{"X", "u", "u_clean", "sigma"}: points (n, 2) as (x, t), noisy and clean values (n, 1), float32.

pinneapple_tools.benchmark_suite.crossframework.problems

The forward and inverse PINN problems of the cross-framework benchmark, with their references.

pinneapple_tools/benchmark_suite/crossframework/problems.py · 314 linhas · testes: 20

nome	tipo	assinatura	o que faz
PINNProblem	classe	(name, coords, bounds, params, time_dependent, fd_step, citation, test_grid, extra)	One benchmark problem. coords are ("x", "t") or ("x", "y"); bounds one (lo, hi) each.
get_problem	função	(name) -> PINNProblem	
make_point_set	função	(problem, *, n_interior=10000, n_boundary=400, seed=20260) -> dict[str, np.ndarray]	The training points every framework loads (protocol A1): X_f interior collocation points (uniform), X_b/u_b boundary points and values, and for time-dependent problems X_0/u_0 initial points.
point_set_digest	função	(points) -> str	SHA-256 over the arrays (sorted by name) so every run can record which point set it trained on.
save_point_set	função	(points, path) -> str	Write the point set to path (.npz) and return its digest.
load_point_set	função	(path) -> dict[str, np.ndarray]	
test_points	função	(problem) -> np.ndarray	The structured test grid (float64, N x 2): the same points for every framework.
residual_points	função	(problem, n=2000, seed=777) -> np.ndarray	Fresh interior points (never used in training; a different generator stream) for the residual, kept one stencil step away from the boundary.
stencil	função	(problem, X) -> np.ndarray	The 5-point stencil around each residual point: (N, 5, 2) = center, a+h, a-h, b+h, b-h. A runner predicts on stencil(...).reshape(-1, 2) and exports the values; :func:residual_from_stencil turns them into the residual.
residual_from_stencil	função	(problem, X, values) -> np.ndarray	Residual at X from the stencil values (N, 5) by second-order central differences (float64).
allen_cahn_reference	função	(d=0.0001, n=2048, dt=0.0001, t_end=1.0, n_save=1000) -> tuple[np.ndarray, np.ndarray, np…)	(x, t, u) of u_t = d u_xx + 5 u - 5 u^3, periodic on [-1, 1), u(x, 0) = x^2 cos(pi x).
callable_exact	função	(problem) -> Callable[[np.ndarray], np.ndarray]	X -> u column vector, for APIs that take a reference callable (pp.compare(..., reference="exact")).
describe	função	(problem) -> dict[str, object]	JSON-ready description (stored in every result file).

pinneapple_tools.benchmark_suite.crossframework.protocol

Fixed settings of the cross-framework benchmark protocol and the export every runner produces.

pinneapple_tools/benchmark_suite/crossframework/protocol.py · 97 linhas · testes: 20

nome	tipo	assinatura	o que faz
`export_predictions`	função	`(problem, predict, batch=20000) -> dict[str, np.ndarray]`	`{"u_test", "u_stencil"}` from `predict` ((N, 2) float32 array -> (N,) or (N, 1)), evaluated in batches on the shared test grid and on the residual stencil. Every runner exports exactly this.
`ensure_point_set`	função	`(problem, root) -> Path`	The A1 point set of `problem` under `root/points/` (created once; later calls load the same file).
`runner_parser`	função	`(description)`	The command line every runner accepts (the driver calls runners only through it).
`base_meta`	função	`(args, *, track, framework, source, config) -> dict`	Metadata every result starts from (the runner fills status, iterations and times).

`pinneapple_tools.benchmark_suite.crossframework.records`

Run records of the cross-framework benchmark: environment, threads, load, time caps, the result file schema.

`pinneapple_tools/benchmark_suite/crossframework/records.py` · 166 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Clock`	classe	`(wall_s, cpu_s, loadavg_before, loadavg_after)`	Wall and process CPU time of a block, with the load average before and after.
`TimeCap`	classe	`(seconds, kind='cpu')`	A per-run time budget, measured as process CPU time (`kind="cpu"`) or wall time (`kind="wall"`).
`setup_threads`	função	`(n) -> None`	Pin the thread count of this process (call before importing torch / a framework): OMP / MKL / OpenBLAS environment variables and `torch.set_num_threads`.
`loadavg`	função	`() -> list`	
`cpu_model`	função	`() -> str`	
`package_versions`	função	`(names) -> dict[str, str | None]`	
`environment`	função	`(threads) -> dict[str, Any]`	Machine and library record stored next to the results.
`write_run`	função	`(out_dir, run_id, meta, arrays) -> Path`	Write `<run_id>.json` (metadata) and `<run_id>.npz` (exported predictions) into `out_dir`.
`read_run`	função	`(out_dir, run_id)`	
`validate_result`	função	`(meta) -> list`	Missing schema fields of a result (empty when valid).

`pinneapple_tools.benchmark_suite.crossframework.weights`

Shared initial weights for the identical-configuration track (A1).

`pinneapple_tools/benchmark_suite/crossframework/weights.py` · 48 linhas · testes: 20

nome	tipo	assinatura	o que faz
`layer_sizes`	função	`(in_dim=2, out_dim=1, width=50, depth=4) -> list[int]`	
`xavier_uniform_init`	função	`(sizes, seed) -> list[tuple[np.ndarray, np.ndarray]]`	`[(W, b), ...]` with `W` of shape `(fan_out, fan_in)` (the `torch.nn.Linear` layout) drawn from `U(-a, a)`, `a = sqrt(6 / (fan_in + fan_out))` (Glorot & Bengio 2010), and `b = 0`; float32.
`load_into_linears`	função	`(linears, params) -> None`	Copy `params` into a sequence of `torch.nn.Linear` modules (shapes must match exactly).
`count_parameters`	função	`(module) -> int`	

pinneapple_tools.benchmark_suite.deeponet_ground_truth

The four canonical operator-learning ground-truth solvers from Lu, Jin & Karniadakis 2019 (DeepONet), arXiv:1910.03193, Section 4: given a GRF-sampled input function u(x) (`pinneapple_data.grf_sampler.GRFDraw`), each solver computes the TRUE solution of a different dynamical system driven by u. ...

`pinneapple_tools/benchmark_suite/deeponet_ground_truth.py` · 155 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`solve_ode_linear`	função	`(draw, x_query) -> np.ndarray`	Exact via the GRF spline's own antiderivative (piecewise-cubic, so integration is exact for the spline itself, not an ODE-solver approximation of it).
`solve_ode_nonlinear`	função	`(draw, x_query) -> np.ndarray`	
`solve_pendulum`	função	`(draw, x_query, k=1.0) -> np.ndarray`	Returns s1 (angle) at each query point.
`pendulum_rk4_reference`	função	`(draw, x_max, k, n_steps=20000) -> float`	Independent hand-written fixed-step RK4 integrator, useful ONLY to cross-check solve_ivp's adaptive integration in a validation script -- not meant for production use (solve_pendulum's adaptive RK45 is both faster and, by default tolerance, at least as accurate).
`solve_diffusion_reaction`	função	`(draw, nx=100, nt=100, D=0.01, k=0.01, x_max=1.0, t_max=1.0) -> 'tuple[np.ndarray, np.nda…)`	Crank-Nicolson for the diffusion term + one Picard correction per step for the k*s^2 reaction term, tridiagonal (Thomas algorithm) solves. Returns (x_grid, t_grid, s) with s.shape == (nt, nx).

pinneapple_tools.benchmark_suite.io

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/io/__init__.py` · 6 linhas · **sem teste direto**

pinneapple_tools.benchmark_suite.io.yamlx

Módulo sem docstring e sem texto.

pinneapple_tools/benchmark_suite/io/yamlx.py · 56 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
load_yaml	função	(path) -> Dict[str, Any]	Load a YAML file and return its content as a dictionary.
dump_yaml	função	(obj) -> str	Serialize a Python object into a YAML-formatted string.

pinneapple_tools.benchmark_suite.meta_benchmark

Meta-Learning Benchmark for PINNs.

pinneapple_tools/benchmark_suite/meta_benchmark.py · 937 linhas · testes: 151

nome	tipo	assinatura	o que faz
MetaBenchmarkConfig	classe	(n_meta_epochs, n_inner_steps, inner_lr, outer_lr, n_tasks_per_batch, k_shots, algorithms…)	
MetaBenchmarkFamily	classe	(name, train_task_factory, eval_task_factory, param_ranges, eval_params)	A parametric PDE family for meta-learning.
MetaBenchmarkResult	classe	(family_name, eval_params, algorithm, k_shot, metrics, elapsed_s)	
MetaBenchmarkPipeline	classe	(families, config=None) -> None	Meta-learning benchmark pipeline.
cfg_str	função	(results) -> str	

pinneapple_tools.benchmark_suite.physics_pipeline

PhysicsPipeline — standard 6-step PINNeAPPle physics pipeline.

pinneapple_tools/benchmark_suite/physics_pipeline.py · 1.332 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
PhysicsPipeline	classe	(problem, geometry=None, load_generate_data='generate', source=None, metrics='auto', coll…)	Standard 6-step PINNeAPPle physics pipeline.

pinneapple_tools.benchmark_suite.pipeline.dataset_builder

Arena dataset builder — thin re-export from pinneapple_data.adapters.pinn_batch_builders.

pinneapple_tools/benchmark_suite/pipeline/dataset_builder.py · 22 linhas · **sem teste direto**

pinneapple_tools.benchmark_suite.pipeline.model_selector

Model selection helpers.

pinneapple_tools/benchmark_suite/pipeline/model_selector.py · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
select_models	função	(*, input_kind, expects=None, supports_physics_loss=None, tags_any=None) -> List[str]	

pinneapple_tools.benchmark_suite.registry

Módulo sem docstring e sem texto.

pinneapple_tools/benchmark_suite/registry.py · 77 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
register_task	função	(task_cls) -> Type['ArenaTask']	Decorator to register a task by its task_id.
get_task	função	(task_id) -> Type['ArenaTask']	
list_tasks	função	() -> List[str]	
register_backend	função	(backend_cls) -> Type['Backend']	Decorator to register a backend by its name.
get_backend	função	(name) -> Type['Backend']	
list_backends	função	() -> List[str]	

pinneapple_tools.benchmark_suite.report

Benchmark report: structured output for PhysicsBenchmarkSpec and TimeSeriesBenchmarkSpec.

pinneapple_tools/benchmark_suite/report.py · 148 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ModelRunResult	classe	(model_id, n_params, training_time_s, metrics, history, param_estimates, rank, error_mess…)	
BenchmarkReport	classe	(benchmark_type, created_at, version, problem_info, config, model_results, leaderboard, b…)	

pinneapple_tools.benchmark_suite.runner

Módulo sem docstring e sem texto.

pinneapple_tools/benchmark_suite/runner/__init__.py · 25 linhas · **sem teste direto**

pinneapple_tools.benchmark_suite.runner.compare

Compare multiple trained model runs.

pinneapple_tools/benchmark_suite/runner/compare.py · 151 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
compare_runs	função	(run_dirs, *, out_dir='runs/arena_compare', title='arena_compare', make_plots=True) -> Di…)	Compare completed run directories.

pinneapple_tools.benchmark_suite.runner.leaderboard

Módulo sem docstring e sem texto.

pinneapple_tools/benchmark_suite/runner/leaderboard.py · 43 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`load_leaderboard`	função	`(path) -> List[Dict[str, Any]]`	Load a leaderboard JSON file.
`update_leaderboard`	função	`(path, row) -> None`	Append a new entry to the leaderboard JSON file.

`pinneapple_tools.benchmark_suite.runner.metrics`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/runner/metrics.py` · 30 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ensure_float_dict`	função	`(d) -> Dict[str, float]`	Convert dictionary values to floats when possible.

`pinneapple_tools.benchmark_suite.runner.report`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/runner/report.py` · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`write_run_artifacts`	função	`(*, artifacts_dir, run_id, report, metrics, summary) -> Path`	Persist run artifacts (report, metrics, summary) to disk.

`pinneapple_tools.benchmark_suite.runner.run_arena_yaml`

Run a complete Arena experiment from a unified YAML config.

`pinneapple_tools/benchmark_suite/runner/run_arena_yaml.py` · 563 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_arena_experiment`	função	`(yaml_path, *, out_dir=None) -> Dict[str, Any]`	Run a complete Arena experiment from a YAML config file.
`main`	função	`() -> None`	

`pinneapple_tools.benchmark_suite.runner.run_benchmark`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/runner/run_benchmark.py` · 132 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_benchmark`	função	`(*, artifacts_dir, task_cfg_path, run_cfg_path, bundle_schema_path) -> Dict[str, Any]`	

`pinneapple_tools.benchmark_suite.runner.run_models_from_cfg`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/runner/run_models_from_cfg.py` · 265 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_one`	função	`(*, run_cfg, run_dir, bundle_root=None) -> Dict[str, Any]`	
`main`	função	`()`	

`pinneapple_tools.benchmark_suite.runner.run_pipeline`

End-to-end pipeline: geometry → solver → dataset → train → predict → report.

`pinneapple_tools/benchmark_suite/runner/run_pipeline.py` · 1.444 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_full_pipeline`	função	`(config_path_or_dict, *, out_dir=None) -> Dict[str, Any]`	Run the complete end-to-end pipeline.
`main`	função	`() -> None`	

`pinneapple_tools.benchmark_suite.runner.run_sweep`

Run a benchmark sweep over multiple models.

`pinneapple_tools/benchmark_suite/runner/run_sweep.py` · 208 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SweepJob`	classe	`(model_name, run_cfg)`	
`run_sweep`	função	`(*, models, base_run_cfg, bundle_root, out_dir='runs/arena_sweep', parallelism='process',…)`	Run a sweep.

`pinneapple_tools.benchmark_suite.tasks`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/tasks/__init__.py` · 29 linhas · testes: 2

`pinneapple_tools.benchmark_suite.tasks.allen_cahn_1d`

Allen-Cahn 1D benchmark task.

`pinneapple_tools/benchmark_suite/tasks/allen_cahn_1d.py` · 135 linhas · testes: 7

nome	tipo	assinatura	o que faz
`AllenCahn1DTask`	classe	`(eps=0.01, build_reference=True) -> None`	

`pinneapple_tools.benchmark_suite.tasks.base`

Módulo sem docstring e sem texto.

`pinneapple_tools/benchmark_suite/tasks/base.py` · 53 linhas · testes: 2

nome	tipo	assinatura	o que faz
`TaskResult`	classe	`(metrics, artifacts)`	Standard output for a task evaluation.
`ArenaTask`	classe	`(task_id)`	Base class for Arena tasks.

pinneapple_tools.benchmark_suite.tasks.burgers_1d

Burgers 1D benchmark task.

pinneapple_tools/benchmark_suite/tasks/burgers_1d.py · 152 linhas · testes: 7

nome	tipo	assinatura	o que faz
Burgers1DTask	classe	(nu=0.01 / np.pi, build_reference=True) -> None	

pinneapple_tools.benchmark_suite.tasks.flow_obstacle_2d

Módulo sem docstring e sem texto.

pinneapple_tools/benchmark_suite/tasks/flow_obstacle_2d.py · 263 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
FlowObstacle2DTask	classe	(task_id, n_collocation_eval, n_boundary_eval, n_sensors_eval)	Task definition for the 2D steady flow around obstacle benchmark.

pinneapple_tools.benchmark_suite.tasks.heat_1d

Heat equation 1D benchmark task.

pinneapple_tools/benchmark_suite/tasks/heat_1d.py · 85 linhas · testes: 153

nome	tipo	assinatura	o que faz
Heat1DTask	classe	(alpha=0.01) -> None	

pinneapple_tools.benchmark_suite.tasks.lid_driven_cavity_3d

Módulo sem docstring e sem texto.

pinneapple_tools/benchmark_suite/tasks/lid_driven_cavity_3d.py · 394 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LidDrivenCavity3DTask	classe	(Re=100.0, size=1.0, lid_velocity=1.0) -> None	3D Lid-Driven Cavity Flow benchmark.

pinneapple_tools.benchmark_suite.tasks.navier_stokes_tgv_2d

Navier-Stokes 2D Taylor-Green Vortex benchmark task.

pinneapple_tools/benchmark_suite/tasks/navier_stokes_tgv_2d.py · 183 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
NavierStokesTGV2DTask	classe	(nu=0.01) -> None	

pinneapple_tools.benchmark_suite.tasks.poisson_2d

Poisson 2D benchmark task.

pinneapple_tools/benchmark_suite/tasks/poisson_2d.py · 98 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Poisson2DTask`	classe	`()`	

`pinneapple_tools.benchmark_suite.tasks.wave_1d`

Wave equation 1D benchmark task.

`pinneapple_tools/benchmark_suite/tasks/wave_1d.py` · 103 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Wave1DTask`	classe	`(c=1.0) -> None`	

`pinneapple_tools.benchmark_suite.timeseries_pipeline`

TimeSeriesBenchmarkSpec — declarative time-series benchmark pipeline.

`pinneapple_tools/benchmark_suite/timeseries_pipeline.py` · 579 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TimeSeriesBenchmarkSpec`	classe	`(source, models=('lstm',), metrics=('mse', 'mae', 'rmse'), target_cols=None, horizon=50, …)`	Declarative time-series benchmark pipeline.

`pinneapple_tools.benchmark_suite.transfer_benchmark`

Transfer Learning Benchmark for PINNs.

`pinneapple_tools/benchmark_suite/transfer_benchmark.py` · 798 linhas · testes: 151

nome	tipo	assinatura	o que faz
`TransferBenchmarkConfig`	classe	`(n_source_epochs, lr_source, n_finetune_epochs, lr_finetune, lr_base, lr_scratch, strateg…)`	
`TransferScenario`	classe	`(name, source_task, target_tasks, source_label, target_labels)`	One transfer scenario: one source task and N target tasks.
`TransferBenchmarkResult`	classe	`(scenario_name, target_label, strategy, metrics, history, elapsed_s, n_params_trainable, …)`	
`TransferBenchmarkPipeline`	classe	`(scenarios, config=None) -> None`	Benchmark comparing transfer learning strategies against from-scratch baselines.

`pinneapple_tools.compute_backends`

Multi-backend support for PINNeAPPle: PyTorch (default) + JAX.

`pinneapple_tools/compute_backends/__init__.py` · 44 linhas · **sem teste direto**

`pinneapple_tools.compute_backends.backend`

Backend abstraction layer for PINNeAPPle.

`pinneapple_tools/compute_backends/backend.py` · 53 linhas · testes: 156

nome	tipo	assinatura	o que faz
`Backend`	classe	`(TORCH, JAX)`	Namespace for backend string constants.

nome	tipo	assinatura	o que faz
`set_backend`	função	`(name) -> None`	Set the active computation backend.
`get_backend`	função	`() -> str`	Return the name of the currently active backend.

`pinneapple_tools.compute_backends.jax_backend`

JAX backend utilities for PINNeAPPle PINNs.

`pinneapple_tools/compute_backends/jax_backend.py` · 268 linhas · testes: 151

nome	tipo	assinatura	o que faz
`JAXBackend`	classe	`()`	JAX-based PINN utilities wrapping Equinox or Flax models.
`jax_available`	função	`() -> bool`	Return `True` if JAX can be imported.
`jit_pinn`	função	`(model_fn, residual_fn) -> Callable`	Module-level alias for :meth:`JAXBackend.jit_pinn`.
`vmap_residual`	função	`(residual_fn, batch_size=64) -> Callable`	Module-level alias for :meth:`JAXBackend.vmap_residual`.
`jax_pinn`	função	`(model_fn, residual_fn, vectorise=True) -> Callable`	Convenience: JIT-compile and optionally vmap a PINN residual.

`pinneapple_tools.dataset_quality`

pinneapple_tools.dataset_quality — dataset quality assessment.

`pinneapple_tools/dataset_quality/__init__.py` · 64 linhas · **sem teste direto**

`pinneapple_tools.dataset_quality.checks`

Generic dataset quality-assessment checks.

`pinneapple_tools/dataset_quality/checks.py` · 214 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`analyze_completeness`	função	`(fields) -> Dict[str, Any]`	NaN/missing-value fraction per field, plus an overall completeness score.
`validate_consistency`	função	`(coords, fields, monotonic_name_hints=_MONOTONIC_NAME_HINTS) -> Dict[str, Any]`	Generic cross-field consistency checks: - shape consistency: every field array's leading dim must match coords.shape[0] - monotonicity heuristic: fields whose *name* suggests a cumulative/ time-like quantity are checked for (near) non-decreasing behavior. This is intentionally generic/heuristic — ...
`analyze_distribution`	função	`(fields) -> Dict[str, Any]`	scipy.stats.describe per field (mean/variance/skewness/kurtosis).
`detect_outliers`	função	`(fields, random_state=42) -> Dict[str, Any]`	IsolationForest fit jointly across all numeric fields (stacked as a feature matrix over points); report the fraction flagged as outliers.
`field_range_issues`	função	`(fields, ranges) -> list`	Generic per-field physical-range check, keyed by the field's literal name: {field_name: (lo, hi)}. Returns a list of human-readable issue strings (empty if everything is within bounds).

nome	tipo	assinatura	o que faz
`build_1d_linear_interpolant`	função	`(z, y) -> 'Any'`	Build a differentiable (torch) piecewise-linear interpolant from a set of 1D-coordinate samples `z` and corresponding output samples `y` (shape (N,) and (N, k)). Useful for evaluating a differentiable PDE-residual function against static point-cloud data when no trained model is available — the …

`pinneapple_tools.dataset_quality.vlm_curation`

Vision-language-model dataset curation.

`pinneapple_tools/dataset_quality/vlm_curation.py` · 504 linhas · testes: 16 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`CurationRecord`	classe	`(id, label, confidence, reason, raw_text)`	One sequence's curation judgment.
`build_consistency_prompt`	função	`(pde_description, *, labels=DEFAULT_LABELS, extra_instructions=None) -> str`	Build the text prompt asking a VLM to judge a clip's physical consistency against a stated governing-physics assumption.
`extract_json`	função	`(text) -> Optional[dict]`	Best-effort extraction of the first `{...}` JSON object found in a block of free-form generated text. Returns `None` (never raises) if no valid JSON object is present.
`normalize_label`	função	`(label, *, labels=DEFAULT_LABELS, fallback=None) -> str`	Normalize a (possibly malformed) label value onto the closed `labels` set: exact match (case/whitespace-insensitive) first, then substring containment against each candidate label, else `fallback` (defaults to the middle label of `labels`, matching `physcurator`'s original "suspect" -- …
`clamp_confidence`	função	`(x, *, lo=0.0, hi=1.0) -> float`	Coerce `x` to a float and clamp it to `[lo, hi]`; a value that can't be parsed as a float falls back to the midpoint.
`parse_curation_response`	função	`(seq_id, text, *, labels=DEFAULT_LABELS, reason_max_chars=300, raw_text_max_chars=1200) -…)`	Parse one VLM generation into a :class:`CurationRecord`. Never raises: a response that fails to parse as JSON yields the fallback label at confidence 0.5, with an empty reason -- the honest signal that the model's output was unusable, not a guessed judgment.
`load_vlm`	função	`(model_name, *, device=None, dtype=None, model_cls=None, attn_implementation='sdpa')`	Load a Hugging Face vision-language chat model + its processor.
`curate_sequence`	função	`(model, processor, clip_path, prompt, *, seq_id='', fps=4, max_new_tokens=512, labels=DEF…)`	Run one VLM generation over a single clip and parse the result.

nome	tipo	assinatura	o que faz
curate_dataset	função	(index, *, out_dir, model_name='nvidia/Cosmos-Reason2-2B', model=None, processor=None, mo…)	Curate every sequence in index with a VLM and write per-sequence + aggregate curation metadata to out_dir (mirrors physcurator's curator/run_cosmos.py orchestration, generalized).
filter_curated	função	(index, curation_records, *, keep_labels=('consistent',), min_confidence=0.55, id_field='…)	Filter index down to entries whose curation judgment is in keep_labels and meets min_confidence (mirrors physcurator's data/make_curated.py, generalized off the "npz" field name and off filesystem copying being mandatory).

pinneapple_tools.hpo_experiments

Módulo sem docstring e sem texto.

pinneapple_tools/hpo_experiments/__init__.py · 14 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.agents

Módulo sem docstring e sem texto.

pinneapple_tools/hpo_experiments/agents/__init__.py · 6 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.agents.extractor

ExtractorAgent for extracting problem/solution from KB artifacts.

pinneapple_tools/hpo_experiments/agents/extractor.py · 101 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ExtractorAgent	classe	(provider)	

pinneapple_tools.hpo_experiments.agents.prompts

Agent system prompts for extract, reproduce, and verify.

pinneapple_tools/hpo_experiments/agents/prompts.py · 125 linhas · **sem teste direto**

pinneapple_tools.hpo_experiments.agents.reproducer

ReproducerAgent for generating reproduction patches from repo/project.

pinneapple_tools/hpo_experiments/agents/reproducer.py · 98 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ReproduceResult	classe	(project_dir, run_instructions)	
ReproducerAgent	classe	(provider)	

pinneapple_tools.hpo_experiments.agents.verifier

VerifierAgent for verifying project runnability and suggesting patches.

`pinneapple_tools/hpo_experiments/agents/verifier.py` · 75 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VerifyResult`	classe	`(status, diagnosis, patches, notes)`	
`VerifierAgent`	classe	`(provider)`	

`pinneapple_tools.hpo_experiments.config`

ResearcherConfig for topic, k_papers, k_repos, ranking weights.

`pinneapple_tools/hpo_experiments/config.py` · 42 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ResearcherConfig`	classe	`(topic, k_papers, k_repos, w_lexical, w_signals, w_freshness, w_quality, github_min_stars…)`	Researcher configuration.

`pinneapple_tools.hpo_experiments.kb`

Módulo sem docstring e sem texto.

`pinneapple_tools/hpo_experiments/kb/__init__.py` · 5 linhas · **sem teste direto**

`pinneapple_tools.hpo_experiments.kb.chunking`

Text chunking for KB indexing with overlap.

`pinneapple_tools/hpo_experiments/kb/chunking.py` · 29 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TextChunk`	classe	`(chunk_id, text)`	
`chunk_text`	função	`(text, *, chunk_chars=1800, overlap=200) -> List[TextChunk]`	

`pinneapple_tools.hpo_experiments.kb.store`

KB store for artifacts, indexing, and persistence.

`pinneapple_tools/hpo_experiments/kb/store.py` · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`KBStore`	classe	`(*, out_dir)`	Persists a KB run: - manifest.json (ranked items + metadata) - artifacts (papers/, repos/) - chunks.jsonl (chunked text for RAG)
`slugify`	função	`(s) -> str`	
`ensure_dir`	função	`(p) -> str`	

`pinneapple_tools.hpo_experiments.models`

RankedItem, DiscoveryResult data models for researcher output.

`pinneapple_tools/hpo_experiments/models.py` · 75 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RankedItem`	classe	`(type, id, title, url, score, why, meta)`	
`DiscoveryResult`	classe	`(topic, papers, repos, meta)`	
`KBArtifact`	classe	`(type, id, title, url, path, meta)`	A persisted artifact (paper/repo) stored inside a KB run folder.
`KBIndex`	classe	`(topic, run_dir, chunks_path, manifest_path)`	A light-weight index pointing to chunked documents (JSONL).
`ExtractedProblemSolution`	classe	`(source_type, source_id, title, problem, solution, equations, data_requirements, training…)`	Output of ExtractorAgent. Keep it generic but structured enough to map into pinneapple_problemdesign later.

`pinneapple_tools.hpo_experiments.pipelines`

Módulo sem docstring e sem texto.

`pinneapple_tools/hpo_experiments/pipelines/__init__.py` · 8 linhas · **sem teste direto**

`pinneapple_tools.hpo_experiments.pipelines.build_kb`

KB build pipeline: papers and repos to KBStore.

`pinneapple_tools/hpo_experiments/pipelines/build_kb.py` · 141 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_kb`	função	`(cfg, discovery) -> KBIndex`	

`pinneapple_tools.hpo_experiments.pipelines.discover`

Discovery pipeline: papers and repos from topic query.

`pinneapple_tools/hpo_experiments/pipelines/discover.py` · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`discover`	função	`(cfg) -> DiscoveryResult`	

`pinneapple_tools.hpo_experiments.pipelines.export_problemdesign`

Export extracted solutions to ProblemDesign format.

`pinneapple_tools/hpo_experiments/pipelines/export_problemdesign.py` · 169 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`to_problemdesign_dict`	função	`(x) -> Dict[str, Any]`	ProblemDesign-like schema (stable, explicit): - source - problem - solution - benchmark - strengths - weaknesses - alternatives - future_improvements

nome	tipo	assinatura	o que faz
`export_problemdesign`	função	`(items, *, out_path) -> str`	Write researcher findings to *out_path* (JSON) and, when `pinneapple_problemdesign` is available, also write a companion `*_pinneapple_spec.json` with partial `ProblemSpec` mappings and generated Pinneapple API objects for any item that has enough physics info.

`pinneapple_tools.hpo_experiments.pipelines.extract_problem_solutions`

Extract problem/solution pairs from KB via ExtractorAgent.

`pinneapple_tools/hpo_experiments/pipelines/extract_problem_solutions.py` · 39 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`extract_problem_solutions`	função	`(*, kb_index, out_dir=None, max_items=20) -> List[ExtractedProblemSolution]`	

`pinneapple_tools.hpo_experiments.pipelines.reproduce`

Reproduce pipeline: apply patches and verify project runnability.

`pinneapple_tools/hpo_experiments/pipelines/reproduce.py` · 79 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`reproduce`	função	`(*, item, kb_index_dir, out_dir=None, max_fix_iters=2) -> str`	

`pinneapple_tools.hpo_experiments.providers`

Módulo sem docstring e sem texto.

`pinneapple_tools/hpo_experiments/providers/__init__.py` · 4 linhas · **sem teste direto**

`pinneapple_tools.hpo_experiments.providers.gemini_provider`

Gemini API provider for LLM calls.

`pinneapple_tools/hpo_experiments/providers/gemini_provider.py` · 240 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeminiProviderConfig`	classe	`(api_base, api_version, model, api_key, timeout_s, gateway_base_url, gateway_route)`	
`GeminiProvider`	classe	`(*, api_key=None, model=None, api_base=None, api_version=None, timeout_s=None, gateway_ba…)`	Default: calls Gemini Developer API directly (models.generateContent).

`pinneapple_tools.hpo_experiments.ranking`

Ranking heuristics: lexical overlap, freshness, quality scores.

`pinneapple_tools/hpo_experiments/ranking.py` · 71 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`tokenize`	função	`(text) -> List[str]`	
`lexical_overlap_score`	função	`(query, doc) -> float`	v0.1 lexical overlap score (0..1). Simple: fraction of unique query tokens present in doc tokens.
`safe_parse_iso8601`	função	`(dt) -> Optional[datetime]`	
`freshness_score`	função	`(dt, *, half_life_days=365.0) -> float`	Exponential decay freshness score (0..1). score=1 now, ~0.5 at half_life_days, decays afterwards.
`normalize`	função	`(xs) -> List[float]`	
`sort_ranked`	função	`(items) -> List[RankedItem]`	

`pinneapple_tools.hpo_experiments.sources`

Módulo sem docstring e sem texto.

`pinneapple_tools/hpo_experiments/sources/__init__.py` · 12 linhas · **sem teste direto**

`pinneapple_tools.hpo_experiments.sources.arxiv_pdf`

Arxiv PDF download and text extraction with section splitting.

`pinneapple_tools/hpo_experiments/sources/arxiv_pdf.py` · 79 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`arxiv_pdf_url`	função	`(arxiv_id, use_export_subdomain=True) -> str`	
`download_pdf`	função	`(url, out_path, timeout_s=60) -> str`	
`extract_pdf_text`	função	`(pdf_path) -> str`	
`split_sections`	função	`(text) -> Dict[str, str]`	Heuristic: find headings by keyword matches; slice between them. Works “ok” for many arXiv PDFs but not perfect (PDF layout can be messy).

`pinneapple_tools.hpo_experiments.sources.arxiv_source`

Arxiv API source for paper search and metadata.

`pinneapple_tools/hpo_experiments/sources/arxiv_source.py` · 90 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ArxivPaper`	classe	`(arxiv_id, title, summary, url, published, authors, categories)`	
`ArxivSource`	classe	`()`	Minimal arXiv Atom API search.

`pinneapple_tools.hpo_experiments.sources.github_fetcher`

GitHub API fetcher for repo file tree and content.

`pinneapple_tools/hpo_experiments/sources/github_fetcher.py` · 179 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RepoFile`	classe	`(path, sha, size, url)`	
`GithubFetcher`	classe	`(token=None, api_base='https://api.github.com')`	Fetches README, repo tree, and key files using GitHub REST API.

`pinneapple_tools.hpo_experiments.sources.github_source`

GitHub API source for repository search.

`pinneapple_tools/hpo_experiments/sources/github_source.py` · 90 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GithubRepo`	classe	`(full_name, title, url, description, stars, forks, language, updated_at, license)`	
`GithubSource`	classe	`(token=None)`	GitHub Search API wrapper (repos discovery). Token optional but recommended (GITHUB_TOKEN).

`pinneapple_tools.hpo_experiments.utils`

Módulo sem docstring e sem texto.

`pinneapple_tools/hpo_experiments/utils/__init__.py` · 10 linhas · **sem teste direto**

`pinneapple_tools.hpo_experiments.utils.verify_runtime`

Runtime verification: project tree, py_compile, smoke run.

`pinneapple_tools/hpo_experiments/utils/verify_runtime.py` · 74 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RunResult`	classe	`(ok, step, returncode, stdout, stderr)`	
`project_tree`	função	`(cwd, max_files=400) -> str`	
`py_compile_all`	função	`(cwd) -> RunResult`	
`smoke_run`	função	`(cwd, timeout_s=180) -> RunResult`	

`pinneapple_tools.model_evaluation`

Módulo sem docstring e sem texto.

`pinneapple_tools/model_evaluation/__init__.py` · 1 linhas · **sem teste direto**

`pinneapple_tools.model_evaluation.metrics`

Generic model-evaluation metrics: regression performance, bootstrap uncertainty, one-at-a-time sensitivity, and calibration — independent of any particular model storage/registry format. Callers supply plain arrays (`y_true`, `y_pred`, `coords`) or a plain callable (for sensitivity), not a ...

`pinneapple_tools/model_evaluation/metrics.py` · 262 linhas · testes: 5

nome	tipo	assinatura	o que faz
`regression_metrics`	função	`(y_true, y_pred, field_names=None) -> Dict[str, Any]`	RMSE/MAE/relative-error/R2, overall and per output column.
`axis_binned_profile`	função	`(coord, field, n_bins=40) -> Dict[str, np.ndarray]`	Bin `field` by one coordinate axis and average within each bin -- the standard way to compare a statistically-homogeneous-in-the-other- directions field (e.g. a channel/boundary-layer flow's velocity, which varies with wall-normal distance but is turbulent/scattered in the other two directions) ...
`mape`	função	`(y_true, y_pred, eps=1e-08) -> float`	Mean Absolute Percentage Error, as a fraction (multiply by 100 for %). `eps` guards division by near-zero targets — MAPE dominated by that guard rather than genuine relative error is a sign `y_true` has values near zero, not a bug in this function.
`worst_n_points`	função	`(coords, abs_residuals, field_names, n=20) -> List[Dict[str, Any]]`	Indices/coordinates of the n points with the largest total absolute error (summed across output fields) — useful for spotting where a model fits worst.
`bootstrap_confidence_interval`	função	`(residuals, n_boot=1000, ci=90.0, seed=42) -> Dict[str, Any]`	Bootstrap confidence interval on the mean residual — a simple, dependency-free uncertainty estimate when no dedicated UQ model is available.
`one_at_a_time_sensitivity`	função	`(model_fn, base_inputs, param_columns, perturb_frac=0.1, default_nominal=1.0) -> Dict[str…)`	First-order, one-at-a-time sensitivity: for each named input column, perturb it +/- `perturb_frac` around its mean value in `base_inputs` and measure |model(plus) - model(minus)| / |delta_input|.
`gradient_sensitivity`	função	`(model, x, n_out=None) -> Dict[str, Any]`	Exact autograd-based sensitivity: mean |d output_j / d input_i| over the rows of `x`, computed via a single vectorized backward pass per output channel — not a finite-difference approximation like `one_at_a_time_sensitivity` above. Requires `model` to be a differentiable `torch.nn.Module` (or any ...
`calibration_curve`	função	`(y_true, y_pred, residual_std, levels=(0.5, 0.7, 0.8, 0.9, 0.95)) -> Dict[str, Any]`	Empirical coverage of a Gaussian(0, residual_std) predictive interval at each nominal confidence level — compares against the ideal diagonal to show whether a model's uncertainty estimate is well-calibrated.

`pinneapple_tools.model_evaluation.visualization`

Generic model-evaluation plots: error/value histograms, real-vs-predicted scatter, calibration curve. Pure matplotlib, no framework/storage coupling — callers pass arrays and an output directory.

`pinneapple_tools/model_evaluation/visualization.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_value_histograms`	função	`(fields, out_dir, bins=40) -> List[str]`	One histogram per field of its raw values (use when there's no ground-truth comparison, e.g. profiling a dataset rather than scoring a model).
`plot_error_histograms`	função	`(abs_residuals, field_names, out_dir, bins=40) -> List[str]`	One histogram per output field of |prediction - ground truth|.

nome	tipo	assinatura	o que faz
`plot_real_vs_predicted`	função	`(y_true, y_pred, out_path) -> str`	Scatter of predicted vs. actual values with a y=x reference line.
`plot_calibration_curve`	função	`(levels, empirical_coverage, out_path) -> str`	Empirical vs. nominal confidence coverage, against the ideal diagonal.

pinneapple_tools.model_export

Módulo sem docstring e sem texto.

`pinneapple_tools/model_export/__init__.py` · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`export_torchscript`	função	`(model, path, example_input=None) -> str`	Export any nn.Module to TorchScript.
`export_onnx`	função	`(model, path, example_input, input_names=None, output_names=None, opset_version=17, dynam…)`	Export any nn.Module to ONNX.
`export_csv`	função	`(coords, fields, path, *, coord_names=None, field_names=None) -> str`	Export coordinate + field arrays to a CSV file.
`export_npz`	função	`(coords, fields, path, *, coord_names=None, field_names=None) -> str`	Export coordinate + field arrays to a compressed NumPy `.npz` archive.

pinneapple_tools.model_services

PINNeAPPle Model Services: ready-made PINNeAPPle models as local microservices with the shape of NVIDIA NIM.

`pinneapple_tools/model_services/__init__.py` · 27 linhas · testes: 39

pinneapple_tools.model_services.__main__

python -m pinneapple_tools.model_services serve [--host 0.0.0.0] [--port 8000] [--no-zoo] [--api-key KEY]

`pinneapple_tools/model_services/__main__.py` · 27 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	

pinneapple_tools.model_services.client

One client for PINNeAPPle Model Services and NVIDIA NIM (same routes): health, models, infer, chat.

`pinneapple_tools/model_services/client.py` · 77 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ServiceError`	classe	`(status, message)`	
`ModelServiceClient`	classe	`(base_url, api_key=None, timeout=60.0)`	`base_url` ends in `/v1`. For NVIDIA NIM on build.nvidia.com pass `NIM_URL` and an `api_key` (default `$NVIDIA_API_KEY`).

pinneapple_tools.model_services.grid

Grid-operator services: a trained field model (e.g. a PhysicsNeMo FNO) and PINNeAPPle's reference solver behind the same `/v1/infer` route, each answer checked with the residual-based Darcy checks (no reference needed).

`pinneapple_tools/model_services/grid.py` · 50 linhas · testes: 39

nome	tipo	assinatura	o que faz
`darcy_trust`	função	`(a, u, f=1.0, envelope=None) -> dict`	
`grid_operator_service`	função	`(predictor, name, *, envelope=None, f=1.0, card=None, task='Darcy-type field operator') -…)`	
`darcy_solver_service`	função	`(name='pinneapple-darcy-fv', f=1.0) -> ModelService`	

pinneapple_tools.model_services.registry

Model services: a callable with a card and a trust function, and the registry of the built-in ones.

`pinneapple_tools/model_services/registry.py` · 200 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelService`	classe	`(id, kind, task, predict, trust, card, version, calls)`	
`Registry`	classe	`(services=())`	
`jsonable`	função	`(o)`	
`zoo_service`	função	`(name) -> ModelService`	A model of the PINNeAPPle zoo: `predict(**inputs)`; trust from `pp.zoo.validate` on the same inputs (card metrics reproduced, physics checks, validity range of the query).
`trust_card_service`	função	`() -> ModelService`	Physics checks as a service: `{"axes": {"x": [...], "y": [...]}, "values": [[...]], "pde": "laplace", "dirichlet": {"x_min": 0.0, ...}, "train_bounds": {...}}` -> the trust card of that field.
`intent_service`	função	`() -> ModelService`	`pinneapple-intent`: the deterministic specification extractor of `pinneapple_design.intent` behind an OpenAI-style chat route. The last user message is the request; the answer is the specification as JSON (template, values with their units and quotes, questions, values it could not place). …
`default_registry`	função	`(zoo=True) -> Registry`	
`dumps`	função	`(o) -> bytes`	
`infer`	função	`(model, inputs_json) -> dict`	One request to a built-in service in this process (no server): `inputs_json` is the `inputs` object of `/v1/infer` as JSON text. Returns the same answer the HTTP route returns (outputs, trust, warnings, timing).

pinneapple_tools.model_services.server

HTTP front end of the model services (standard library only), with the routes of NVIDIA NIM.

`pinneapple_tools/model_services/server.py` · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`make_server`	função	`(registry, host='127.0.0.1', port=8000, api_key=None)`	A `ThreadingHTTPServer` serving `registry`. `api_key`: if set, requests need `Authorization: Bearer`.
`serve_in_thread`	função	`(registry, host='127.0.0.1', port=0, api_key=None)`	Start the server on a background thread; returns `(server, base_url)` (`base_url` ends in `/v1`). `server.shutdown()` stops it.

pinneapple_tools.nvidia

pinneapple_tools.nvidia -- PINNeAPPle with the NVIDIA stack (PhysicsNeMo, Omniverse / OpenUSD, Isaac Sim, Cosmos, NIM).

`pinneapple_tools/nvidia/__init__.py` · 24 linhas · **sem teste direto**

pinneapple_tools.nvidia._brep_worker

USD BRep worker: builds a part from a recipe and measures it. Standalone (usd_brep + numpy + stdlib only), so it runs in whichever interpreter has `usd-brep` (its wheel is CPython 3.12):

`pinneapple_tools/nvidia/_brep_worker.py` · 157 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(req_path, out_path)`	

pinneapple_tools.nvidia.brep

Exact geometry with NVIDIA USD BRep (the SMLib NURBS B-rep kernel, `pip install usd-brep`), measured for PINNeAPPle: mass properties of the exact solid, curvature of every face, tessellations at several chord tolerances with their measured deviation from the exact surfaces, and B-rep USD export.

`pinneapple_tools/nvidia/brep.py` · 247 linhas · testes: 39

nome	tipo	assinatura	o que faz
`PartRecipe`	classe	`(ops, result)`	
`BRepReport`	classe	`(raw, meshes, seconds)`	
`brep_python`	função	`() -> str | None`	The interpreter that has `usd_brep`: this one, else `$PINNEAPPLE_BREP_PYTHON` (if it exists).
`brep_available`	função	`() -> bool`	

nome	tipo	assinatura	o que faz
evaluate_part	função	(recipe, *, chord_tolerances=(0.001, 0.0001), density=1.0, angle_deg=25.0, max_edge_lengt…)	Build recipe with USD BRep and measure it (see the module docstring). vertex_curvature: the exact surface curvature at every vertex of the finest mesh (rep.vertex_curvature, 1/m) and the B-rep face each vertex lies on (rep.vertex_face).
geometry_checks	função	(rep, *, element_size=None, points_per_radius=4.0, volume_rtol=0.001) -> list[dict[str, A…)	Trust checks of the geometry handed to a solver or a surrogate (the finest tessellation): manifold solid, chord error below the tolerance asked, mesh volume and area against the exact solid, and (with element_size) the element size against the smallest radius of curvature.
planar_face_mesh	função	(V, F, *, axis=2, value, atol=1e-09) -> tuple[np.ndarray, np.ndarray]	The triangles of a tessellation lying in the plane x[axis] = value (e.g. the top face of a plate), as a 2-D mesh (V2 (n, 2), F2 (m, 3)) in the other two coordinates, renumbered.
vertex_normals	função	(V, F) -> np.ndarray	Area-weighted unit vertex normals (outward for a consistently oriented closed mesh).
mesh_thickness	função	(V, F, *, normals=None, eps=1e-07, chunk=256) -> np.ndarray	Wall thickness at every vertex of a closed mesh: distance along the inward normal to the first triangle hit (Moller-Trumbore ray casting); inf where the ray leaves without a hit.
point_mesh_distance	função	(P, V, F, chunk=128) -> np.ndarray	Distance from each point to the nearest triangle vertex or centroid (a cheap upper bound on the distance to the mesh, used to sample the deviation of a mesh from points of an exact surface).

pinneapple_tools.nvidia.diagram

Architecture diagrams of a PINNeAPPle + NVIDIA pipeline, drawn by code (matplotlib only, no graphviz).

pinneapple_tools/nvidia/diagram.py · 118 linhas · testes: 39

nome	tipo	assinatura	o que faz
Block	classe	(key, label, owner, col, row, ran)	
architecture_diagram	função	(title, blocks, edges, path, *, subtitle='', width=2.6, height=1.0, dpi=150) -> str	Draw blocks and edges ((src, dst) or (src, dst, label)) and save a PNG at path.

pinneapple_tools.nvidia.figures

Result figures shared by the NVIDIA examples: field panels, check tables, training curves.

pinneapple_tools/nvidia/figures.py · 140 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
field_panels	função	(fields, titles, path, *, cmaps=None, extent=(0, 1, 0, 1), suptitle='', units=None, share…)	fields[i][ix, iy] drawn with x to the right and y up; shared: groups of panel indices on one range; ncols: panels per row (default: one row).
check_rows	função	(checks) -> list[dict[str, Any]]	Checks of any supported kind as [{name, value, limit, passed, detail}].

nome	tipo	assinatura	o que faz
checks_figure	função	(checks, path, *, title='Physics checks', decision='', dpi=130) -> str	Bar of value / limit per check (limit / value for a lower bound, "kind": "min"; log scale; the dashed line is the limit). Checks without a limit are drawn as pass / fail markers.
curves_figure	função	(panels, path, *, suptitle='', dpi=130) -> str	panels: [{"title", "xlabel", "ylabel", "series": [(label, x, y, style?)], "logy"?, "hlines"?}].

pinneapple_tools.nvidia.heatsink

Plate-fin heat-sink designs evaluated two ways at once: the exact solid from NVIDIA USD BRep (volume, area, mass) and PINNeAPPle's closed-form thermal and hydraulic model (pinneapple_physics.closed_form.plate_fin_heatsink: Teertstra forced convection, fin efficiency, spreading resistance, ...

pinneapple_tools/nvidia/heatsink.py · 113 linhas · testes: 39

nome	tipo	assinatura	o que faz
Duty	classe	(power_w, t_ambient_c, air_velocity_m_s, source, base_t, length, width, material, root_fi…)	
evaluate_design	função	(n_fins, fin_height, fin_t, duty=None, *, brep=True, chord_tolerances=()) -> dict	One design: closed-form thermal / hydraulic result and (brep) the exact solid's volume, area, mass.
sample_designs	função	(n, *, n_range=(6, 20), h_range=(0.015, 0.045), t_range=(0.001, 0.003), min_gap=0.002, wi…)	Latin-hypercube designs inside the ranges with at least min_gap between fins.
design_table	função	(designs, duty=None, *, brep=True) -> list[dict]	
verify	função	(n_fins, fin_height, fin_t, duty=None, **kw) -> dict	Closed form + exact solid for a design a surrogate proposed (the evaluation that decides).
surrogate_service	função	(net, X_train, *, name=…, t_ambient=25.0)	A heat-sink surrogate (n_fins, fin_height, fin_t) -> (T rise, dp, mass) as a PINNeAPPle Model Service. Inputs {"designs": [[n, H, t], ...]}; the trust report flags designs outside the training box and outputs that break physics (a negative temperature rise, pressure drop or mass).

pinneapple_tools.nvidia.isaac

Isaac Sim-ready robots in OpenUSD (UsdPhysics), their checks, and the Isaac Sim script that replays a motion.

pinneapple_tools/nvidia/isaac.py · 239 linhas · testes: 39 · sinais no código: shim ou obsoleto

nome	tipo	assinatura	o que faz
`author_articulation`	função	`(stage, spec) -> str`	Write `spec` under `/World/<name>`; returns that prim path. `stage`: a `FieldStage` or `Usd.Stage`.
`validate_physics`	função	`(path, *, expect_bodies=None, expect_joints=None) -> dict`	:func:`validate_stage` plus the `UsdPhysics` parse and the mass / joint checks a simulator relies on.
`isaac_replay_script`	função	`(usd_path, robot_prim, traj_npz, out_npz, *, mode='effort', physics_dt=1.0 / 600.0, headl…)`	Standalone Isaac Sim script (run with Isaac Sim's `python.sh`). `traj_npz` holds `t`, `q` [rad], `tau` [N m] (joint order of the USD articulation); `mode="effort"` applies `tau` open loop, `"position"` tracks `q` with the drives. Writes `t`, `q` and `effort` ...
`compare_replay`	função	`(npz, t, q, tau) -> dict`	Max tracking error [rad] and RMS effort error relative to the peak torque, Isaac Sim replay vs PINNeAPPle.

`pinneapple_tools.nvidia.omniverse`

PINNeAPPle results as OpenUSD stages for NVIDIA Omniverse (authored and checked with `usd-core`).

`pinneapple_tools/nvidia/omniverse.py` · 646 linhas · testes: 39

nome	tipo	assinatura	o que faz
`FieldStage`	classe	`(path, title='PINNeAPPle result', *, meters_per_unit=1.0, fps=24.0)`	A USD stage holding PINNeAPPle fields. `save()` writes it (`.usda` text or `.usd` / `.usdc`).
`require_pxr`	função	`()`	`pxr` (OpenUSD), with an install hint when it is missing.
`field_colors`	função	`(values, cmap='inferno', vrange=None) -> np.ndarray`	`(n, 3)` linear RGB of `values` through a matplotlib colour map (sRGB -> linear, as USD expects).
`grid_surface`	função	`(X, Y, Z=None) -> tuple[np.ndarray, np.ndarray]`	Triangles of a structured `(nx, ny)` grid of points: `(V (nx*ny, 3), F (2(nx-1)(ny-1), 3))`.
`extrude_profile`	função	`(xy, length, n=2, axis=1) -> tuple[np.ndarray, np.ndarray]`	A closed 2-D profile `(m, 2)` (in the plane normal to `axis`) extruded over `[0, length]` in `n` stations: the side wall as triangles. Vertex `k * m + i` is profile point `i` at station `k`.
`tube`	função	`(path, radius, n_around=12) -> tuple[np.ndarray, np.ndarray]`	A tube of `radius` around a polyline `path (p, 3)`: vertex `k * n_around + j` is ring `k`.
`validate_stage`	função	`(path, *, required_primvars=None, compliance=True) -> dict[str, Any]`	Re-open a USD file and check it. Returns `{"ok", "problems", "prims", "meshes", "curves", "points", "time_samples", "up_axis", "meters_per_unit", "compliance"}`. `required_primvars`: prim path -> primvar names that must be there.

nome	tipo	assinatura	o que faz
`usd_validators`	função	`(stage) -> dict[str, Any]`	Every validator OpenUSD registers (`pxr.UsdValidation`: stage metadata, composition, shading, geometry subsets, and the `UsdPhysics` rigid-body, collider, joint and articulation checkers) run on `stage`, or the legacy `UsdUtils.ComplianceChecker` on builds without `UsdValidation`.
`read_stage_geometry`	função	`(path, time=None, exclude=()) -> list[dict[str, Any]]`	Meshes and curves of a USD file as arrays: `[{"path", "kind", "points", "faces" or "counts", "colors"}]` (colours from `displayColor` at `time`, a time code; default: the first sample). `exclude`: prim path prefixes to leave out (e.g. a ground plane).
`render_stage_png`	função	`(path, png, *, time=None, elev=28, azim=-60, title='', colorbar=None, size=(7.5, 5.5), dp…)`	Draw the meshes and curves of a USD file (read back from disk) in their `displayColor`; `colorbar`: `(label, lo, hi, cmap)`; `flat`: axes scaled to the data in x, y and a squashed z (long, low scenes such as a pipeline). A file check, not an Omniverse RTX render.
`kit_render_script`	função	`(usd_path, png_path, *, frames=60) -> str`	Python to run inside Omniverse Kit / USD Composer (Window > Script Editor, or `kit --exec`): opens the stage and writes an RTX viewport capture. Needs Omniverse with an RTX GPU; not runnable with `usd-core` alone. Save it next to the stage: the paths are resolved from the script's own folder.
`render_stage_gif`	função	`(path, gif, times, *, fps=6.0, label=None, **kw) -> str`	An animated GIF of a time-sampled stage: :func:`render_stage_png` at each time code in `times`; `label(t)` gives each frame's title.
`rasterize_topdown`	função	`(path, H, W, *, bounds=None, time=None, exclude=(), background=(0.0, 0.0, 0.0), view='top…)`	An orthographic view of the meshes of a USD file, `(H, W, 3)` sRGB in [0, 1]: every triangle is filled with its vertices' `displayColor` interpolated (barycentric), nearest on top. `view`: `"top"` (looking down -z: x right, y up) or `"side"` (looking along +y: x right, z up). `bounds`: …

`pinneapple_tools.nvidia.parts`

Parametric parts as :class:`~pinneapple_tools.nvidia.brep.PartRecipe` data, with their closed-form volumes so the exact solid can be checked against them.

`pinneapple_tools/nvidia/parts.py` · 68 linhas · testes: 39

nome	tipo	assinatura	o que faz
`l_bracket`	função	`(*, base=0.08, height=0.06, width=0.05, t=0.006, fillet=0.005, bolt_d=0.009, web_hole_d=0…)`	An L bracket: base plate and web of thickness `t`, an inner fillet, two bolt holes in the base and one hole through the web (none when `web_hole_d` is 0).
`l_bracket_volume`	função	`(*, base=0.08, height=0.06, width=0.05, t=0.006, fillet=0.005, bolt_d=0.009, web_hole_d=0…)`	

nome	tipo	assinatura	o que faz
heat_sink	função	(*, n_fins, fin_height, fin_t, base_t=0.005, length=0.1, width=0.1, root_fillet=0.0) -> P…)	A plate-fin heat sink: a base length (along the fins, x) by width (y) and n_fins equally spaced fins of thickness fin_t and height fin_height on top; root_fillet rounds the inner fin roots (an extrusion has them; the closed-form thermal model does not).
heat_sink_geometry	função	(*, n_fins, fin_height, fin_t, base_t=0.005, length=0.1, width=0.1, root_fillet=0.0) -> d…)	Closed-form volume and areas (wetted_area: fins and the exposed base top; total_area: every face). The root fillets add r^2 (1 - pi/4) of section each to the volume; the areas ignore them.

pinneapple_tools.nvidia.physicsnemo

Small PhysicsNeMo surrogates for tabular physics (a few inputs -> a few outputs), trained and checked the PINNeAPPle way. Grid operators (FNO, MeshGraphNet) are in :mod:pinneapple_neural.physicsnemo_bridge.

pinneapple_tools/nvidia/physicsnemo.py · 146 linhas · testes: 39

nome	tipo	assinatura	o que faz
MLPSurrogate	classe	(in_features, out_features, layer_size, num_layers, activation_fn, seed, x_lo, x_hi, y_lo…)	
sign_violations	função	(f, axes, axis, output, sign=1, *, tol=0.0) -> float	Fraction of cells of the tensor grid axes (one 1-D array per input) where the forward difference of f(X)[:, output] along input axis has the sign opposite to sign by more than tol.
relative_errors	função	(pred, ref) -> np.ndarray	Per-output RMS error relative to the RMS of the reference output.

pinneapple_tools.nvidia.runinfo

What a run used: versions of the NVIDIA and PINNeAPPle packages, the hardware, and step timings.

pinneapple_tools/nvidia/runinfo.py · 89 linhas · testes: 39

nome	tipo	assinatura	o que faz
RunInfo	classe	(name='')	
environment	função	() -> dict[str, Any]	Python, OS, CPU, GPU (CUDA through torch, if any) and the installed versions of the relevant packages.

pinneapple_tools.rea_bridge

rea_bridge -- study reference implementations with REA, under a license gate.

pinneapple_tools/rea_bridge/__init__.py · 33 linhas · testes: 22

`pinneapple_tools.rea_bridge.client`

Thin, honest wrapper around the REA command-line tool.

`pinneapple_tools/rea_bridge/client.py` · 216 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ReaUnavailable`	classe	`()`	No REA executable could be found (see module docstring).
`ReaError`	classe	`(message, *, returncode, stderr_tail)`	REA ran but exited non-zero or printed something that is not JSON.
`ReaResult`	classe	`(command, payload, progress, stderr_tail)`	One REA invocation: the command, the parsed JSON payload and the progress events REA streamed on stderr (kept for the audit trail).
`ReaClient`	classe	`(command=None, *, timeout_s=900.0, cwd=None)`	Run REA commands and parse their JSON output.
`resolve_rea_command`	função	`(env=None) -> List[str]`	Return the argv prefix that runs REA, or raise :class:`ReaUnavailable`.

`pinneapple_tools.rea_bridge.evidence`

Turn REA's JavaScript application Evidence into a compact feature map.

`pinneapple_tools/rea_bridge/evidence.py` · 222 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FunctionRef`	classe	`(node_id, name, module, start_line, end_line)`	
`FeatureMap`	classe	`(evidence_id, subject_sha256, root, functions, calls, callers, timers, storage, limitatio…)`	
`FeatureHit`	classe	`(function, matched, snippet, callees, callers)`	
`summarize_js_evidence`	função	`(evidence, *, vendor_globs=DEFAULT_VENDOR_GLOBS, include_vendor=False) -> FeatureMap`	Build a :class:`FeatureMap` from one `analyze_javascript_application` Evidence payload (the parsed JSON, not the path).
`locate`	função	`(fmap, keywords, *, root=None, max_hits=20, snippet_lines=40) -> List[FeatureHit]`	Functions whose name or body mentions any keyword (case-insensitive), most matches first. `root` defaults to the analysed directory.
`exported_functions_from_layout`	função	`(evidence) -> List[str]`	Exported functions recorded by `rea inspect-binary-layout` (ELF): entries of the `SHT_DYNSYM` table that are defined, global/weak, default-visibility FUNC/GNU_IFUNC symbols.

`pinneapple_tools.rea_bridge.harvest`

Harvest ledger: what was studied, under which license, and what came of it.

`pinneapple_tools/rea_bridge/harvest.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LicenseGateError`	classe	`()`	A finding claims more reuse than the project's license allows.
`Finding`	classe	`(title, summary, source_refs, status, method, pinneapple_refs, note)`	
`HarvestEntry`	classe	`(project, url, commit, licenses, policy, findings, rea_version, evidence_id, subject_sha2…)`	
`HarvestLedger`	classe	`(path)`	A JSON file of :class:`HarvestEntry` items keyed by project name.

`pinneapple_tools.rea_bridge.licenses`

License gate for anything harvested from another project into PINNeAPPle.

`pinneapple_tools/rea_bridge/licenses.py` · 179 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`LicensePolicy`	classe	`(policy, licenses, attribution_required, reason, notes)`	
`policy_for`	função	`(spdx_ids) -> LicensePolicy`	Most restrictive policy over the given SPDX identifiers.
`detect_licenses`	função	`(root, *, max_bytes=200000) -> List[str]`	SPDX ids recognised in the license files at the top of `root` (plus `package.json`/`pyproject.toml`/`setup.py` declarations).
`policy_for_path`	função	`(root) -> LicensePolicy`	

`pinneapple_tools.rea_bridge.tools`

Expose the REA bridge as tools of :class:`PhysicsToolRegistry`, so :func:`pinneapple_llm.agent_loop.run_agent_loop` can study a reference implementation the same way it calls a solver: one real tool call, a real result fed back to the next turn.

`pinneapple_tools/rea_bridge/tools.py` · 94 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`register_rea_tools`	função	`(registry) -> List[str]`	Register the bridge's tools on `registry`; returns the names added.

`pinneapple_tools.reporting`

Reporting: figure style and PDF technical-report builder.

`pinneapple_tools/reporting/__init__.py` · 27 linhas · testes: 5

`pinneapple_tools.reporting.datasheet`

A technical datasheet: the engineering record of a study, laid out from data -- no LLM anywhere.

`pinneapple_tools/reporting/datasheet.py` · 570 linhas · testes: 14

nome	tipo	assinatura	o que faz
KPI	classe	(label, value, unit, note, tone)	One cell of the headline strip: a label, its value, an optional unit and a note.
Table	classe	(rows, caption, header, widths, align, emphasis, note)	A two-column (or wider) table. rows are sequences of cells; header names the columns.
Figure	classe	(path, caption, wide, max_width)	An image with a caption. wide takes the full text width, otherwise it shares a row.
Note	classe	(text, kind, title)	A callout. kind: info (neutral), ok, warn, fail (a conclusion).
Step	classe	(title, detail, status, cost)	One step of the run plan: what it does, why it is in the plan, what it cost, where it stands.
Section	classe	(title, number, lede, blocks)	One numbered section: a lede, then blocks (paragraphs, callouts, tables, figures, rows).
Datasheet	classe	(title, subtitle, eyebrow, generated, verified_by, kpis_, sections, plan_, confirm_, limi…)	A technical datasheet: headline strip, numbered sections, run plan, items to confirm, footer.
find_chrome	função	() -> str | None	A Chromium/Chrome executable that can print HTML to PDF, or None.

pinneapple_tools.reporting.paperkit

Figure style and PDF paper builder for PINNeAPPle technical reports.

pinneapple_tools/reporting/paperkit.py · 429 linhas · testes: 5

nome	tipo	assinatura	o que faz
Paper	classe	(title, subtitle='', authors='PINNeAPPle Labs', affiliation=…, date=None, report_id='', f…)	A4 technical report built with reportlab/platypus (needs reportlab).
setup_mpl	função	() -> None	Apply the report style to matplotlib.rcParams and register SEQ / DIV.
savefig	função	(fig, path, **kwargs) -> str	Save fig to path (creating parent folders), close it and return the path as a string.
fmt	função	(x, nd=3) -> str	Compact number for a table cell.
sanitize	função	(text) -> str	Turn runs of Unicode super/subscript characters into reportlab <super>/<sub> markup.
para	função	(text, style, **kw)	A reportlab Paragraph of str(text) after :func:sanitize (needs reportlab).

pinneapple_tools.sandbox

pinneapple_tools.sandbox — running untrusted, user-supplied solver scripts out-of-process with best-effort resource limits.

pinneapple_tools/sandbox/__init__.py · 11 linhas · **sem teste direto**

pinneapple_tools.sandbox.custom_solver_runner

Run an untrusted, user-supplied Python solver script out-of-process, with best-effort resource limits.

`pinneapple_tools/sandbox/custom_solver_runner.py` · 318 linhas · testes: 13

nome	tipo	assinatura	o que faz
`run_custom_solver`	função	`(script_path, params=None, *, timeout=60.0, memory_limit_mb=512) -> Dict[str, Any]`	Run `script_path`'s `solve(params)` in a separate, resource- limited subprocess. See the module docstring's SECURITY WARNING for exactly what protection this does (and does not) provide.

pinneapple_tools.surrogate_policy

pinneapple_tools.surrogate_policy — measured-runtime "does this problem need a trained surrogate, or can we just call its solver directly?" policy.

`pinneapple_tools/surrogate_policy.py` · 92 linhas · testes: 5

nome	tipo	assinatura	o que faz
`SurrogatePolicyResult`	classe	`(problem_id, needs_model, reason, measured_s)`	
`SurrogatePolicy`	classe	`(cache_path, *, slow_threshold_s=1.0)`	

pinneapple_tools.visualization

pinneapple_viz — CFD-style visualisation for PINNeAPPle.

`pinneapple_tools/visualization/__init__.py` · 116 linhas · **sem teste direto**

pinneapple_tools.visualization.animation

Time-evolution animations for PDE solutions — CFD style.

`pinneapple_tools/visualization/animation.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`animate_scalar_field`	função	`(trajectory, x=None, y=None, *, t_vals=None, title='Field evolution', field_label='u', cm…)`	Animate a time series of 2-D scalar fields.
`animate_streamlines`	função	`(u_traj, v_traj, x=None, y=None, *, speed_traj=None, t_vals=None, title='Velocity evoluti…)`	Animated streamline plot for a time series of 2-D velocity fields.
`make_gif`	função	`(frames, x, y, save_path, *, t_vals=None, cmap=DEFAULT_CMAP, dpi=80, fps=10) -> None`	Convenience wrapper: save a list of 2-D field frames as a GIF.

pinneapple_tools.visualization.dashboard

Animation-centred dashboards for any simulation: an animation in the middle, charts synced to its time cursor, KPI tiles, a parameter panel and a colour bar; exported as interactive HTML (scrubbing a chart moves the animation), a static PNG and a dashboard GIF/MP4.

`pinneapple_tools/visualization/dashboard/__init__.py` · 18 linhas · testes: 16

pinneapple_tools.visualization.dashboard.html

Interactive HTML export of a :class:~.spec.Dashboard: one self-contained file (no server, no internet).

pinneapple_tools/visualization/dashboard/html.py · 429 linhas · testes: 5

nome	tipo	assinatura	o que faz
payload	função	(db) -> dict	Everything the page draws, as JSON-able data.
scene3d_payload	função	(bodies, poses, terrain=None, lighting=None, axes='z_up') -> dict	Data for the live 3D view.
write_html	função	(db, path, *, fps=12, max_width=1280, quality=26) -> str	Write the self-contained page; returns path.

pinneapple_tools.visualization.dashboard.spec

What an animation-centred dashboard shows: an animation (frames), charts synced to its time cursor, KPI tiles, a parameter panel and a colour bar. Pure data (numpy arrays and strings); the exporters are in :mod:.html and :mod:.static.

pinneapple_tools/visualization/dashboard/spec.py · 190 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Series	classe	(label, y, color, dash, x, width)	A line. y has one value per frame (series charts) or per x point (phase-chart curves).
Marker	classe	(label, x, y, color)	A point that moves with the frames (phase charts): x[k], y[k] at frame k.
Chart	classe	(title, ylabel, xlabel, series, markers, x, kind, slot, ylim, note)	
Tile	classe	(label, value, unit, fmt, note)	A KPI: a scalar, or one value per frame (the tile follows the cursor).
Param	classe	(label, value, unit)	
ColorBar	classe	(title, cmap, lo, hi, saturation)	Colour bar beside the animation; saturation < 1 shows the colour map muted the way a semi-transparent, desaturated field overlay draws it (studio.planetary.SurfaceView.field_saturation).
Dashboard	classe	(title, t, frames, charts, tiles, params, subtitle, colorbar, footer, time_label, time_un…)	frames: a list of RGB arrays (H, W, 3) or image paths, one per entry of t [s].

pinneapple_tools.visualization.dashboard.static

PNG and GIF/MP4 exports of a :class:~.spec.Dashboard, drawn with matplotlib in the studio's dark style.

pinneapple_tools/visualization/dashboard/static.py · 352 linhas · testes: 14

nome	tipo	assinatura	o que faz
write_png	função	(db, path, k=None, size=(1920, 1080), dpi=100, font_scale=1.0) -> str	The dashboard at frame k (default: the last) as a PNG.
write_gif	função	(path, frames, fps, colors=255, loop=0) -> str	Animated GIF of RGB frames with one shared adaptive palette (octree quantisation of a few frames spread over the clip, which keeps small areas of distinct colour -- chart lines, a colour bar -- that a median cut merges away) and frame differencing: after the first frame, pixels whose palette …
hstack_gifs	função	(paths_or_frames, out, fps=10, colors=128, background='white', mismatch='error', loop=0)	Several animations side by side in one GIF, frame by frame (e.g. two runs of the same case compared).
write_video	função	(db, gif=None, mp4=None, *, fps=12, size=(1920, 1080), gif_size=(1280, 720), gif_every=1,…)	GIF and/or MP4 (H.264, size) of the dashboard over frames (default all). The GIF keeps every gif_every-th frame at the same playback speed. With gif_font_scale the GIF is laid out natively at gif_size with the text scaled by that factor (readable on small screens and social …

pinneapple_tools.visualization.electromagnetics

Figures and animations for electromagnetic fields: field maps with Poynting arrows, wave-propagation movies (GIF with Pillow, MP4 with ffmpeg) and radiation patterns.

pinneapple_tools/visualization/electromagnetics.py · 134 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
field_map	função	(field, extent, path=None, *, outline=None, vectors=None, title='', label='', cmap='RdBu_…)	Draw one 2-D field ([ix, iy]) with an optional material outline and a vector field vectors = (vx, vy) (e.g. the time-averaged Poynting vector) as arrows every stride nodes. Returns the matplotlib figure; saves it when path is given.
field_movie	função	(frames, extent, path, *, times=None, outline=None, title='', label='', cmap='RdBu_r', fp…)	Animate a list of 2-D fields with a fixed symmetric colour scale (set from all frames) and the geometry outline; .gif is written with Pillow, .mp4 with ffmpeg.
radiation_pattern	função	(theta, values, path=None, *, reference=None, labels=('computed', 'analytic'), db=True, f…)	Polar plot of a far-field pattern (normalised to its maximum), optionally against a reference; db plots 20 log10 |F| clipped at floor_db.

pinneapple_tools.visualization.fields

2-D / 3-D scalar and vector field plots — CFD style.

pinneapple_tools/visualization/fields.py · 361 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_scalar`	função	`(x, y, field, *, title='Scalar field', label='', cmap=DEFAULT_CMAP, n_levels=64, show_con…)`	Filled contour plot of a 2-D scalar field.
`plot_scalar_3d`	função	`(x, y, z, field, *, slice_axis='z', slice_val=None, title='Scalar field (3-D slice)', cma…)`	Plot a 2-D slice of a 3-D scalar field.
`plot_vectors`	função	`(x, y, u, v, *, title='Vector field', scale=None, density=20, color_by_mag=True, cmap='ra…)`	Quiver plot with optional background scalar field.
`plot_streamlines`	função	`(x, y, u, v, *, title='Streamlines', density=1.5, color_by_mag=True, cmap='plasma', linew…)`	Streamline plot (requires structured or interpolated regular grid). Handles scattered points by interpolating onto a regular grid.
`compare_fields`	função	`(x, y, fields, *, cmap=DEFAULT_CMAP, n_cols=3, figsize_per_panel=(5, 4), suptitle='', sho…)`	Side-by-side contour comparison of several fields.
`plot_error`	função	`(x, y, pred, ref, *, relative=True, title='Prediction error', cmap='hot', show=False) -> …)`	Visualise pointwise |pred - ref| (or relative error) on the domain.

`pinneapple_tools.visualization.integrations`

Bridges from PINNeAPPle results to other 3-D viewers.

`pinneapple_tools/visualization/integrations/__init__.py` · 11 linhas · **sem teste direto**

`pinneapple_tools.visualization.integrations.plotly_export`

Self-contained interactive 3-D HTML with Plotly: one file that opens in any browser (rotate, zoom, hover values), to share a result without a server or the PINNeAPPle viewer.

`pinneapple_tools/visualization/integrations/plotly_export.py` · 94 linhas · testes: 7

nome	tipo	assinatura	o que faz
`scene_html`	função	`(scene, path, field=None, cmap='Turbo', title='', include_plotlyjs=True) -> str`	
`molecule_html`	função	`(mol, path, xyz=None, slab=None, metal='Au', title='', include_plotlyjs=True) -> str`	
`field_slices_html`	função	`(x, y, z, values, path, iso=None, cmap='Viridis', title='', include_plotlyjs=True) -> str`	`values[i, j, k]` on the grid `x[i]`, `y[j]`, `z[k]`: three mid-plane slices and, with `iso`, an isosurface.

`pinneapple_tools.visualization.integrations.rerun_bridge`

Rerun (rerun.io) bridge: stream PINNeAPPle results into the Rerun viewer, live or into an `.rrd` file.

`pinneapple_tools/visualization/integrations/rerun_bridge.py` · 197 linhas · testes: 7

nome	tipo	assinatura	o que faz
`RerunLogger`	classe	`(app_id='pinneapple', path=None, spawn=False, connect=None)`	`path`: write an `.rrd` file; `spawn`: open a live viewer; `connect`: a gRPC URL of a running viewer. With none of them the data stay in memory (useful in tests).

`pinneapple_tools.visualization.integrations.trame_app`

Trame (Kitware) web app around a PyVista plotter: the VTK scene rendered on the server or in the browser, with a small control panel, served from Python (no JavaScript to write). Good for heavy meshes and fields that are too big for a standalone HTML file, inside a digital-twin dashboard or on a …

`pinneapple_tools/visualization/integrations/trame_app.py` · 37 linhas · testes: 7

nome	tipo	assinatura	o que faz
`build_app`	função	`(plotter, title='PINNeAPPle', local_rendering=False, server_name=None)`	A trame server showing `plotter`'s render window (`local_rendering`: vtk.js in the browser instead of images streamed from the server). Returns the server; call `server.start()` to serve it.

`pinneapple_tools.visualization.mesh`

Mesh and geometry visualisation utilities.

`pinneapple_tools/visualization/mesh.py` · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_mesh`	função	`(nodes, elems, *, node_values=None, cmap=DEFAULT_CMAP, edge_color='#4c9be8', edge_lw=0.5,…)`	Plot a 2-D mesh (triangles or quads).
`plot_boundary`	função	`(nodes, boundary_groups, *, title='Boundary groups', show=False) -> Figure`	Visualise labelled boundary groups.
`plot_point_cloud`	função	`(points, values=None, *, cmap='plasma', s=3.0, title='Point cloud', show=False, dim3=Fals…)`	2-D (or 3-D) scatter of a point cloud, optionally coloured by values.

`pinneapple_tools.visualization.molecular`

Molecular visualisation: proteins and molecules, alone or adsorbed on metal surfaces, rendered as stills, MP4 trajectories, 3-D scenes (glTF for the PINNeAPPle web viewer) and MD dashboards.

`pinneapple_tools/visualization/molecular.py` · 503 linhas · testes: 7

nome	tipo	assinatura	o que faz
`Molecule`	classe	`(xyz, elements, names, resnames, resids, chains, frames)`	
`StepLog`	classe	`(title='', color=None, stream=None)`	Step-by-step console log of a pipeline: numbered steps, details indented under them, elapsed time, colour on a terminal (plain text otherwise or with `color=False`); every line is also kept in `lines`.

nome	tipo	assinatura	o que faz
`residue_class`	função	`(resname) -> str`	
`fcc_slab`	função	`(metal='Au', facet='111', nx=10, ny=10, layers=4, a=None) -> tuple[np.ndarray, np.ndarray]`	Atom positions (n, 3) of an fcc slab with the given top facet, top layer at z = 0, and the in-plane cell vectors (2, 3) of the periodic box it fills exactly (so the lattice is not strained by periodicity).
`slab_for_footprint`	função	`(xyz, metal='Au', facet='111', padding=8.0, layers=4) -> tuple[np.ndarray, np.ndarray, tu…)`	A slab under a molecule: whole lattice repeats covering its x-y footprint plus `padding` on each side, centred under it. Returns (positions, cell, (nx, ny)).
`orient_on_surface`	função	`(xyz, masses=None, active_site=None, gap=3.0, surface_z=0.0) -> np.ndarray`	Rotate a molecule so its thinnest principal direction is the surface normal (the broadest face lies on the surface), turn `active_site` atoms (indices) away from the surface (else the flatter side faces down), and place its lowest atom `gap` above `surface_z`.
`rmsd`	função	`(frames, ref, align=True) -> np.ndarray`	RMSD of each frame against `ref` (Kabsch-aligned when `align`).
`height_above`	função	`(frames, surface_z, masses=None) -> np.ndarray`	Height of the centre of mass above the surface plane, per frame.
`contacts`	função	`(frames, surface_z, cutoff=3.5) -> np.ndarray`	Number of atoms within `cutoff` (A) of the surface plane, per frame.
`render`	função	`(mol, path, *, xyz=None, slab=None, metal='Au', color_by='residue', style='spheres', size…)`	A still of `mol` (positions `xyz` or its own) on `slab`. `style`: "spheres" (space-filling) or "cartoon" (backbone tube and side-chain atoms); `color_by`: residue class, element, chain or height.
`render_frames`	função	`(mol, frames, out_dir, *, slab=None, metal='Au', color_by='residue', style='spheres', siz…)`	One PNG per frame, with a fixed camera (or a slow orbit of `orbit_deg` over the whole run) so motion is readable. Returns the paths.
`encode_mp4`	função	`(pngs, out, fps=12, crf=22) -> str`	H.264 MP4 from PNG frames with the ffmpeg binary bundled by `imageio-ffmpeg`.
`md_dashboard`	função	`(path, t_ps, *, potential=None, temperature=None, rmsd_A=None, height_A=None, contacts_n=…)`	A panel per observable given, over time: potential energy (kJ/mol), temperature (K, with the target), backbone RMSD (A), height of the centre of mass above the surface (A), atoms in contact with the surface.
`to_glb`	função	`(mol, path, *, xyz=None, slab=None, metal='Au', max_atoms=20000) -> str`	The scene as glTF binary: one low-poly sphere per atom, merged per residue class, and the slab as metal; opens in the PINNeAPPle web viewer and any glTF viewer.

`pinneapple_tools.visualization.pinn`

PINN-specific visualisations: training curves, residual fields, collocation points.

`pinneapple_tools/visualization/pinn.py` · 341 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_loss_history`	função	`(history, *, keys=None, log_scale=True, title='Training loss history', show=False) -> Fig…)`	Plot loss curves from a training history list.
`plot_multi_loss`	função	`(history, *, groups=None, log_scale=True, title='Loss components', show=False) -> Figure`	Multi-panel loss breakdown.
`plot_collocation`	função	`(interior_pts, boundary_pts=None, initial_pts=None, data_pts=None, *, title='Collocation …)`	Scatter plot of collocation point sets in 2-D.
`plot_pinn_prediction`	função	`(x, y, pred, ref=None, *, field_name='u', cmap=DEFAULT_CMAP, show=False) -> Figure`	Two- or three-panel comparison: [Reference] [Prediction] [Error]. If ref is None, shows only the prediction.
`plot_pde_residual`	função	`(x, y, residual, *, title='PDE residual', cmap='hot', log_scale=True, show=False) -> Figu…)`	Plot |PDE residual| at collocation points. Optionally log-scaled for better dynamic range.
`plot_gradient_magnitude`	função	`(x, y, grad_x, grad_y, *, title='Gradient magnitude |∇u|', cmap='plasma', show=False) -> …)`	Plot |∇u| from separate partial derivative arrays.
`predict_and_plot`	função	`(model, x_coords, y_coords, *, ref=None, field_name='u', device=None, cmap=DEFAULT_CMAP, …)`	Run model inference and immediately plot the prediction panel.

`pinneapple_tools.visualization.pyvista_bridge`

PyVista bridge: turn PINNeAPPle meshes and fields into publication-quality 3-D images.

`pinneapple_tools/visualization/pyvista_bridge.py` · 195 linhas · testes: 7

nome	tipo	assinatura	o que faz
`von_mises`	função	`(stress6) -> np.ndarray`	Von Mises equivalent stress from CalculiX components `[SXX, SYY, SZZ, SXY, SYZ, SZX]`.
`grid_from_arrays`	função	`(points, cells, vtk_cell_type, point_data=None)`	Unstructured grid from `points (N, 3)` and `cells (M, k)` (0-based point indices).
`calculix_grid`	função	`(model, frd=None, step=-1, warp=None)`	PyVista grid of a CalculiX model, with `DISP`, `|U|`, `STRESS` and `VON_MISES` if present.
`scene_to_multiblock`	função	`(scene, time_index=0, field_name=None)`	`pinneapple_twin3d.Scene` to a `pyvista.MultiBlock` (one surface per part).
`export_vtk`	função	`(mesh, path) -> str`	Write `.vtu` / `.vtp` / `.vtk` / `.vtm` (ParaView-ready); format chosen by extension.
`can_render`	função	`() -> bool`	True if an off-screen OpenGL context can be created here. Probed in a subprocess (VTK crashes the interpreter instead of raising when there is none).

nome	tipo	assinatura	o que faz
render_png	função	(mesh, path, *, scalars=None, cmap='turbo', title='', show_edges=False, window_size=(1400…)	Render mesh (grid, PolyData or MultiBlock) to a PNG. scalars is a point/cell array name.
render_gif	função	(mesh, path, *, scalars=None, cmap='turbo', title='', n_frames=60, elevation=25.0, window…)	Orbit camera around mesh and write an animated GIF (turntable).

pinneapple_tools.visualization.solver

Visualise SolverOutput objects from FDM / FEM / FVM / meshfree solvers.

pinneapple_tools/visualization/solver.py · 198 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
plot_solver_output	função	(output, *, x=None, y=None, title='Solver result', cmap=DEFAULT_CMAP, show=False) -> Figu…)	Auto-plot a SolverOutput.
plot_fem_result	função	(output, *, component=0, title='FEM solution', cmap=DEFAULT_CMAP, show_mesh=True, show=Fa…)	Plot FEM solution on its Q1 mesh.
plot_fvm_result	função	(output, *, frame=-1, title='FVM solution', cmap=DEFAULT_CMAP, show=False) -> Figure	Plot a single time-step from an FVM trajectory.
plot_residuals	função	(output, *, title='Solver residuals', show=False) -> Figure	Bar chart of named residual/loss values stored in output.losses.

pinneapple_tools.visualization.studio

PINNeAPPle studio: realistic 3D visualisation of any geometry and its physics.

pinneapple_tools/visualization/studio/__init__.py · 47 linhas · testes: 41

nome	tipo	assinatura	o que faz
render	função	(*a, **kw)	Blender Cycles render; see :func:pinneapple_tools.visualization.studio.blender.render.
web_viewer	função	(*a, **kw)	Interactive page; see :func:pinneapple_tools.visualization.studio.web.web_viewer.
serve	função	(*a, **kw)	Write the viewer to a temporary folder and serve it; see :func:.web.serve.

pinneapple_tools.visualization.studio.animation

Time animation of a :class:~.scene.Scene: rigid bodies moving along a trajectory, per-frame arrows, trails, per-vertex fields and head-up numbers, for the browser viewer (:mod:.web) and Blender (:mod:.blender).

pinneapple_tools/visualization/studio/animation.py · 434 linhas · testes: 27

nome	tipo	assinatura	o que faz
Animation	classe	(times, tracks, vectors, trails, surface_fields, hud, camera, follow_group, follow_offset…)	Per-frame data on top of a static :class:~.scene.Scene; see the module docstring.
matrix_from_quat	função	(q) -> np.ndarray	Rotation matrix (3, 3) (or (..., 3, 3)) of unit quaternion(s) [w, x, y, z] (normalised here).
quat_from_matrix	função	(R) -> np.ndarray	Unit quaternion [w, x, y, z] (w >= 0) of a rotation matrix (3, 3) or a stack (..., 3, 3) (Shepperd's method: the largest of the four squared components is taken first, so it is stable).
slerp	função	(q0, q1, f) -> np.ndarray	Spherical interpolation between unit quaternions [w, x, y, z] (shortest arc), f in [0, 1].

pinneapple_tools.visualization.studio.blender

Blender Cycles renders of any :class:~.scene.Scene (pip install bpy, or run inside Blender).

pinneapple_tools/visualization/studio/blender.py · 1.296 linhas · testes: 27

nome	tipo	assinatura	o que faz
render	função	(scene, out, *, field=None, field_range=None, lines=False, line_range=None, slice=None, s…)	Render scene (a Scene or a .glb path) to out (.png or .jpg). Returns out.
blender_axes	função	(axes) -> np.ndarray	Rotation matrix scene axes -> Blender coordinates (what the glTF export + import does to the surfaces).
render_animation	função	(scene, out_dir, *, frames=None, size=(960, 540), samples=16, camera='chase', camera_offs…)	Render scene.animation (an :class:~.animation.Animation) frame by frame with Cycles (CPU, denoised), burn the HUD into each PNG and encode an MP4 (and a GIF). Returns {"frames": [png, ...], "mp4": path | None, "gif": path | None}.
render_still	função	(scene, out_png, frame, *, size=(1920, 1080), samples=64, **kw) -> str	One high-quality frame of scene.animation (frame index frame) as a PNG at out_png; takes every option of :func:render_animation (camera, dof, motion_blur, haze, hud_keys, sparkline, trails, lines, ...). With motion blur the neighbouring frames are keyframed too, so the blur is that of …

pinneapple_tools.visualization.studio.colormap

The CFD colour scale (jet: blue = low, red = high) and a colour bar burnt into an image.

pinneapple_tools/visualization/studio/colormap.py · 93 linhas · testes: 14

nome	tipo	assinatura	o que faz
jet	função	(t) -> Tuple[float, float, float]	sRGB jet colour for t in [0, 1], as floats in [0, 1].

nome	tipo	assinatura	o que faz
`ramp`	função	`(t, stops=None) -> tuple[float, float, float]`	sRGB colour of a piecewise-linear scale (default :data:`JET`) at t in [0, 1], floats in [0, 1].
`decimals`	função	`(lo, hi) -> int`	Decimals that tell five ticks over [lo, hi] apart (same rule as studio-core.js).
`fmt`	função	`(v, nd) -> str`	A tick label: fixed with `nd` decimals, exponent form beyond 1e4 or below 1e-3 (as JavaScript prints it).
`ticks`	função	`(lo, hi, n=5, nd=None) -> List[str]`	Evenly spaced tick labels from lo to hi, identical to the browser viewer's.
`colorbar`	função	`(path, title, lo, hi, nd=None, lo_txt='', hi_txt='', corner='bottom-right', stops=None) -…)`	Draw a colour bar (jet, or the scale `stops` such as :data:`RDBU`) with 5 ticks and a title in a corner of the image at `path` (overwritten).

`pinneapple_tools.visualization.studio.compose`

Colour maps for animations (jet and diverging blue-white-red), colour bars burnt into images, and multi-panel pictures and videos (a grid of renders with labels and one common colour bar).

`pinneapple_tools/visualization/studio/compose.py` · 238 linhas · testes: 19

nome	tipo	assinatura	o que faz
`stops`	função	`(cmap)`	
`cmap_color`	função	`(cmap, t) -> tuple[float, float, float]`	sRGB colour (floats in [0, 1]) of `cmap` at t in [0, 1] (NaN -> 0).
`symmetric_range`	função	`(values) -> tuple[float, float]`	(-m, m) with m the largest finite |value| (a diverging map keeps zero at its neutral midpoint).
`default_range`	função	`(values, cmap='jet') -> tuple[float, float]`	min..max for jet, symmetric about zero for a diverging map.
`colorbar_image`	função	`(width, title, lo, hi, cmap='jet', scale=1.0, background=(255, 255, 255, 225))`	A colour bar (title, gradient, five ticks) as an RGBA PIL image `width` pixels wide.
`draw_colorbars`	função	`(path, bars, corner='bottom-right', width_frac=0.3) -> str`	Burn colour bars `(title, lo, hi, cmap)` into the image at `path` (overwritten), stacked from `corner`.
`compose_grid`	função	`(images, out_png, labels=None, ncols=2, colorbar=None, background=(233, 237, 242)) -> str`	One picture of `images` on a grid of `ncols` columns (every image scaled to the first one's size), a label in each cell's top-right corner and one common colour bar `(title, lo, hi, cmap)` under the grid.

nome	tipo	assinatura	o que faz
`compose_grid_video`	função	`(mp4_paths, out_mp4, labels=None, ncols=2, colorbar=None, crf=28, webm=False, fps=None, b…)`	A grid video of `mp4_paths` (ffmpeg `xstack`; every clip scaled to the first one's size, the shorter ones hold their last frame), with the labels and one colour bar strip `(title, lo, hi, cmap)` burnt in. H.264 `crf` (yuv420p, even size, faststart); `webm=True` also writes a VP9 .webm …

`pinneapple_tools.visualization.studio.fieldview`

Fields on a mesh for the apps' 3-D views: the boundary surface of a `pinneapple_data.cae` mesh with several fields (values and errors), optional deformed shapes and marked regions, as a glTF plus a small JSON description.

`pinneapple_tools/visualization/studio/fieldview.py` · 179 linhas · testes: 9

nome	tipo	assinatura	o que faz
`FieldView`	classe	`(scene, meta)`	
`colour_range`	função	`(values, kind='value') -> dict[str, Any]`	`{"lo", "hi", "max", "clipped"}`: values span min..max; errors span 0..99th percentile (`clipped` when the maximum is above it).
`length_factor`	função	`(unit, to='m') -> float | None`	Factor from `unit` to `to` when `unit` is a length (`mm` -> m: 1e-3), else None.
`displacement_factor`	função	`(name, unit, components, mesh_unit='m') -> float | None`	If a nodal vector field is a displacement, the factor that converts it to the mesh's length unit: its unit is a length, or it has no unit and is named like one (`D`, `U`, `disp`, `displacement`). Else None.
`subsample`	função	`(points, n=MAX_MARKER_POINTS) -> np.ndarray`	At most `n` of the points, evenly spread over the array (markers do not need every node).
`field_view`	função	`(mesh, fields, *, deform=None, markers=(), max_faces=VIEW_MAX_FACES, title='', axes='z_up…)`	Build the view. `fields`: dicts with `name`, `values` (one per point or per cell, vectors as magnitude, 6-component tensors as von Mises), optional `where` ("point" / "cell", from the length), `label`, `unit`, `kind` ("value" or "error"), `group` (shared colour range), `deform` (a …

`pinneapple_tools.visualization.studio.lighting`

Lighting and sky presets shared by the studio renderers (PyVista, Blender Cycles, three.js viewer).

`pinneapple_tools/visualization/studio/lighting.py` · 171 linhas · testes: 6

nome	tipo	assinatura	o que faz
`LightingPreset`	classe	`(name, sun_elevation_deg, sun_azimuth_deg, sun_intensity, sun_color, ambient, background_…)`	
`get_preset`	função	`(preset) -> LightingPreset`	

nome	tipo	assinatura	o que faz
`apply_pyvista`	função	`(plotter, preset='lunar_south_pole', target=(0.0, 0.0, 0.0), extent=10.0, earth=None, sha…)`	Set the background, a directional Sun (with shadows when the preset asks for them), a weak fill light and, optionally, Earth in the sky of a PyVista `plotter`.
`add_earth_pyvista`	função	`(plotter, preset, target, extent)`	Earth as a shaded sphere far away, with the right angular size, lit from the Sun's direction.
`threejs_config`	função	`(preset='lunar_south_pole') -> dict`	The preset as plain data for the browser viewers (background, sun direction and intensity, fill).
`blender_world`	função	`(preset='lunar_south_pole', bpy=None)`	Configure the current Blender scene: world colour, one Sun lamp with the preset's angular size and strength, and (for black-sky presets) no world light. Returns the Sun object.

`pinneapple_tools.visualization.studio.particles`

Particle animations: thousands of spheres coloured by a value (speed, temperature, species), inside glass or steel equipment that can move (a turning impeller), rendered frame by frame with Blender Cycles; then composed with live charts that draw themselves as time runs, a colour bar and a clock, ...

`pinneapple_tools/visualization/studio/particles.py` · 323 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`render_particle_frames`	função	`(frames, d, out_dir, *, equipment=None, field='speed', field_range=None, view=(-1.0, -1.2…)`	Render every `every`-th frame to `out_dir/frame_XXXX.png`; the camera frames the equipment (or the particles) once. Returns the PNG paths.
`compose_video`	função	`(images, times, out, *, charts=(), colorbar=None, cmap='viridis', clock='Time: {t:.1f}', …)`	One composed frame per image: the render on the left, the `charts` (label, t, y) on the right drawn up to the current time (full axes from the start, so the line grows), a colour bar over the render and a clock. Writes a GIF to `out`; with `mp4` also `out` with .mp4 through Blender's FFmpeg ...
`encode_mp4`	função	`(pngs, out, fps=10) -> str`	H.264 MP4 of a PNG sequence with Blender's sequencer (no ffmpeg binary needed).

`pinneapple_tools.visualization.studio.planetary`

Renders of a vehicle on a planetary surface with PyVista: heightfield terrain with baked photometry and hard Sun shadows (:mod:`.shadows`), vehicle parts with their studio materials, rocks, optional field colour maps on the terrain and on the parts (with a colour bar), Earth in the sky and a title.

`pinneapple_tools/visualization/studio/planetary.py` · 348 linhas · testes: 17

nome	tipo	assinatura	o que faz
`SurfaceView`	classe	`(X, Y, Z, parts, rocks, preset, terrain_field, field_alpha, field_glow, field_saturation, …)`	
`horizon_distance`	função	`(height, planet_radius=MOON_RADIUS) -> float`	Distance [m] to the horizon of a smooth sphere seen from `height` metres above it.
`ring_height`	função	`(view, x, y)`	Height of the horizon annulus [m] at (x, y): `far_fn` (or the grid's edge level) blended into the edge level over `horizon_blend` metres, a long-wavelength relief faded in with distance, and the curvature drop.
`terrain_height`	função	`(view, x, y)`	Height of the rendered ground at (x, y): the detailed patch, the far grid, then the horizon annulus.
`border_points`	função	`(view, n=720) -> np.ndarray`	Points (n, 3) on the outer border of the rendered ground (the annulus rim, else the far grid's edge).
`border_visibility`	função	`(view, camera, size=(1600, 900), view_angle=35.0, samples=400, margin=0.0) -> dict`	Is the outer border of the ground mesh visible from `camera`? Every border point is projected through the camera; a point inside the frame (grown by `margin` pixels) counts as visible unless the ground between the camera and the point hides it (`samples` steps along the sight line, geometric …
`render_surface`	função	`(view, path, camera, *, size=(1600, 900), view_angle=35.0, title='', subtitle='', plotter…)`	Render `view` with `camera = (position, focal_point[, up])`. Returns the image path or (with `return_image`) the RGB array.

`pinneapple_tools.visualization.studio.pyvista_render`

Off-screen renders of a studio :class:`~.scene.Scene` with PyVista (VTK): the Blender-free counterpart of :func:`pinneapple_tools.visualization.studio.render`, for machines without `bpy`.

`pinneapple_tools/visualization/studio/pyvista_render.py` · 157 linhas · testes: 4

nome	tipo	assinatura	o que faz
`render_scene`	função	`(scene, path, *, field=None, cmap='jet', clim=None, lines=False, slice_name=None, view='i…)`	`slice_crop`: (s0, s1, t0, t1), the part of the slice to show as fractions of its `u` and `v` edges. `wireframe`: a colour (e.g. "#3B82F6") draws every surface as its mesh edges in that colour instead. `bar_title`: a short colour-bar title with its units (e.g. "Cp"); by default the …

`pinneapple_tools.visualization.studio.scene`

A 3D scene for any geometry: surfaces with materials and per-vertex fields, streamlines, slice planes.

`pinneapple_tools/visualization/studio/scene.py` · 682 linhas · testes: 46

nome	tipo	assinatura	o que faz
Surface	classe	`(name, vertices, faces, material, fields, group)`	One triangulated surface. `fields`: name -> one value per vertex.
Lines	classe	`(points, values, name, label, cmap, alpha, radius, color_range)`	Polylines (e.g. streamlines), each with optional per-point values (e.g. speed). `cmap`: "jet" or the diverging "coolwarm" / "bwr"; `color_range` (default: 1st..99th percentile, symmetric for a diverging map); `alpha` < 1 and `radius` (tube radius; default from the scene size) for soft ...
Slice	classe	`(name, origin, u, v, grid, label, group)`	A rectangular plane with a value grid: `origin` + s*`u` + t*v for s, t in [0, 1]. `grid[j, i]`: row j along v, column i along u; NaN = no value (inside a body).
Scene	classe	`(surfaces, lines, slices, axes, title, labels, lighting, materials, animation)`	
decimate_surface	função	`(srf, target_faces) -> 'Surface'`	Vertex clustering on a uniform grid whose cell size is searched (bisection) to land near `target_faces`. Fast and robust for display; thin features below the cell size may close up.
scalar_of	função	`(a) -> np.ndarray`	One value per entry: itself, the magnitude of a vector, the von Mises stress of a 6-component tensor (xx, yy, zz, xy, yz, zx).
vertex_normals	função	`(V, F) -> np.ndarray`	
read_stl	função	`(path) -> List[Tuple[str, np.ndarray, np.ndarray]]`	(name, vertices, faces) per solid; duplicate vertices merged.
read_obj	função	`(path) -> Tuple[np.ndarray, np.ndarray]`	
read_gltf	função	`(path, axes='z_up') -> List[Surface]`	Triangle meshes of a .glb/.gltf (embedded, data-URI or external buffers), node transforms applied, converted from glTF's y-up to the scene `axes`. Float vertex attributes starting with _ become fields (their original names when the file was written by this module).
read_vtp	função	`(path, material='grey') -> Surface`	VTK PolyData (.vtp) with its point and cell data (vectors as magnitude). Needs `pip install vtk`.
write_glb	função	`(surfaces, T, materials, generator='PINNeAPPle', groups=None) -> bytes`	glTF 2.0 binary with PBR materials (clearcoat, transmission, emissive). Every per-vertex field becomes a float attribute `_NAME` (upper case; three.js reads it back as `_name`). `groups`: node name -> indices of surfaces to parent under one node (e.g. a propeller that spins).
write_usda	função	`(surfaces, materials, title='scene', root='Scene') -> str`	OpenUSD text: one Mesh per surface with UsdPreviewSurface materials; z up, metres (Omniverse, usdview). Per-vertex fields become `primvars:<name>` (float, vertex interpolation).
write_stl	função	`(surfaces, solid='scene') -> bytes`	Binary STL of all surfaces.

`pinneapple_tools.visualization.studio.shadows`

Sun shadows and planetary-surface shading for heightfield terrains, independent of the renderer.

pinneapple_tools/visualization/studio/shadows.py · 171 linhas · testes: 17

nome	tipo	assinatura	o que faz
terrain_self_shadow	função	(Z, cell, sun_dir, max_distance=None) -> np.ndarray	Boolean (ny, nx): True where the heightfield Z (rows along y, columns along x, spacing cell) hides the Sun (sun_dir points from the ground to the Sun).
points_in_terrain_shadow	função	(P, X, Y, Z, sun_dir, step=None) -> np.ndarray	Boolean per point (N, 3): the heightfield (X, Y, Z) blocks the Sun as seen from the point.
cast_shadow	função	(meshes, X, Y, Z, sun_dir, iterations=3, supersample=1) -> np.ndarray	Boolean (ny, nx): cells of the regular grid (X, Y, Z) in the shadow of the triangle meshes [(V, F), ...].
lunar_lambert	função	(normals, sun_dir, view_dir, L=1.0) -> np.ndarray	McEwen (1991) Lunar-Lambert reflectance (without the albedo) for unit normals (N, 3); view_dir (3,) or (N, 3) points from the surface to the camera.
relief_noise	função	(X, Y, rms, wavelengths=(0.6, 8.0), n=40, seed=7, slope=1.2)	World-anchored smooth random relief [m] at points (X, Y): a sum of n plane waves with random directions and phases, wavelengths log-spaced in wavelengths and amplitudes ~ wavelength^slope, scaled to rms. The same (x, y) always gives the same height, so a moving window of terrain does ...
grid_normals	função	(Z, cell) -> np.ndarray	
shade_terrain	função	(X, Y, Z, sun_dir, camera_position, albedo=(0.45, 0.44, 0.42), shadow=None, exposure=1.0,…)	(ny, nx, 3) colours in [0, 1]: Lunar-Lambert photometry, shadows from shadow (bool grid or 0-1 fraction), a weak fill light, a filmic shoulder. tint (ny, nx, 3) multiplies the albedo (e.g. a field colour map).

pinneapple_tools.visualization.studio.solver_view

The result of a solver run as a 3-D view, to check it before its numbers are used: the mesh, its fields, the deformed shape (CalculiX displacement, with a scale factor) and the regions where the case defines its boundary conditions and loads.

pinneapple_tools/visualization/studio/solver_view.py · 153 linhas · testes: 6

nome	tipo	assinatura	o que faz
find_result	função	(folder) -> tuple[str, Path] | None	("calculix", <job>.frd) or ("openfoam", <case dir>) for a step folder (or the case itself), else None.
solver_result_view	função	(folder, *, max_faces=VIEW_MAX_FACES) -> tuple[FieldView, dict[str, Any]]	View of the CalculiX or OpenFOAM result in folder (see the module docstring). Raises ValueError when there is no result to show.

pinneapple_tools.visualization.studio.web

Interactive browser view of a :class:~.scene.Scene (three.js, shipped with the library; no internet needed).

pinneapple_tools/visualization/studio/web.py · 197 linhas · testes: 27

nome	tipo	assinatura	o que faz
web_viewer	função	(scene, out_dir, title=None, max_faces=MAX_FACES, single_file=False, filename='index.html…)	Write the viewer for scene into out_dir; returns the path of its index.html. Surfaces above max_faces triangles in total are decimated for the page (the scene itself is not changed); None keeps all. single_file: write only out_dir/filename, with scene.json, scene.glb, the viewer …
serve	função	(scene, port=8000, open_browser=True, block=True, out_dir=None, max_faces=MAX_FACES)	Write the viewer (to out_dir or a temporary folder) and serve it on port. block=False returns the server (call .shutdown()); otherwise serves until Ctrl+C.
screenshot	função	(viewer_dir, out_png, frame=None, size=(1280, 720), setup_js=None, timeout_ms=60000) -> s…)	PNG of the viewer written by :func:web_viewer in viewer_dir (or of a single-file page: the path of the .html, opened from file://), taken in headless Chromium (needs pip install playwright; PLAYWRIGHT_CHROMIUM or /opt/pw-browsers/chromium is used when the bundled browser is missing). …

pinneapple_tools.visualization.style

CFD-style colour maps, themes and defaults for PINNeAPPle visualisation.

pinneapple_tools/visualization/style.py · 118 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
use_cfd_style	função	(dark=True) -> None	Apply CFD dark/light theme globally.
get_cmap	função	(name=DEFAULT_CMAP)	Return a matplotlib colormap by PINNeAPPle alias or standard name.
make_figure	função	(nrows=1, ncols=1, **kwargs)	Create a styled figure.

pinneapple_tools.visualization.trust_panels

Side-by-side field panels with a trust verdict under each model: the figure (and GIF) of "which prediction can I use, and where does it break the physics".

pinneapple_tools/visualization/trust_panels.py · 167 linhas · testes: 2

nome	tipo	assinatura	o que faz
Panel	classe	(title, values, kind, verdict, verdicts, note, inset)	One field. kind: "field" (shared linear scale per row), "residual" (log magnitude) or "error" (magnitude on a linear scale from 0 to error_max of :func:trust_panels). inset: draw a dashed square that many cells inside the border (e.g. the edge of an interior region).
Row	classe	(label, panels)	
trust_panels	função	(rows, *, title='', footer='', cell=2.4, residual_range=None, error_max=None, error_label…)	Figure with one row per :class:Row: its "field" panels, a colour bar, its "residual" / "error" panels and their colour bar (labelled error_label); every row has the same number of panels of each group.

nome	tipo	assinatura	o que faz
`trust_panels_gif`	função	`(frames, path, *, duration_ms=1600, **kwargs) -> Path`	One :func:`trust_panels` figure per frame, written as a looping GIF. `kwargs` go to `trust_panels` (pass the same `residual_range` to every frame for a fixed residual scale).

pinneapple_tools.visualization.vortex

Vortex identification and flow visualization — Q-criterion, λ₂, vorticity.

`pinneapple_tools/visualization/vortex.py` · 675 linhas · testes: 10

nome	tipo	assinatura	o que faz
`compute_vorticity_2d`	função	`(u, v, dx=1.0, dy=1.0) -> np.ndarray`	z-component of vorticity: ω_z = ∂v/∂x − ∂u/∂y.
`compute_q_criterion_2d`	função	`(u, v, dx=1.0, dy=1.0) -> np.ndarray`	Q-criterion for 2-D incompressible flow.
`compute_vorticity_3d`	função	`(u, v, w, dx=1.0, dy=1.0, dz=1.0) -> np.ndarray`	Full vorticity vector field ω = ∇×u for 3-D flow: ω_x = ∂w/∂y − ∂v/∂z ω_y = ∂u/∂z − ∂w/∂x ω_z = ∂v/∂x − ∂u/∂y
`compute_strain_rate_tensor_3d`	função	`(u, v, w, dx=1.0, dy=1.0, dz=1.0)`	Symmetric strain-rate tensor S_ij = 0.5(∂u_i/∂x_j + ∂u_j/∂x_i) for 3-D flow.
`compute_q_criterion_3d`	função	`(u, v, w, dx=1.0, dy=1.0, dz=1.0) -> np.ndarray`	Q-criterion for 3-D flow: Q = 0.5(‖Ω‖²_F − ‖S‖²_F)
`compute_enstrophy`	função	`(vorticity) -> float`	Enstrophy: 0.5 * sum(|ω|²).
`compute_dissipation_rate`	função	`(strain_rate_tensor_components, nu) -> np.ndarray`	Viscous/turbulent dissipation rate: ε = 2ν S_ij S_ij (double contraction of the strain-rate tensor).
`compute_lambda2_3d`	função	`(u, v, w, dx=1.0, dy=1.0, dz=1.0) -> np.ndarray`	λ₂ vortex criterion (Jeong & Hussain, 1995). Vortex core: λ₂ < 0.
`plot_vorticity`	função	`(x, y, omega, *, title='Vorticity ω_z', cmap='RdBu_r', symmetric=True, obstacle_mask=None…)`	Coloured vorticity map, symmetric diverging colormap centred on zero.
`plot_q_criterion_2d`	função	`(x, y, Q, vel_mag=None, *, title='Q-criterion', q_levels=30, highlight_positive=True, cma…)`	2-D Q-criterion field.
`plot_vortex_identification`	função	`(x, y, u, v, *, dx=1.0, dy=1.0, title=…, obstacle_mask=None, figsize=(11, 5), show=False)…)`	One-shot wrapper: compute Q from velocity field and plot. Also overlays streamlines on the Q background.
`plot_q_criterion_3d`	função	`(x, y, z, Q, *, level=0.01, vel_mag=None, title='Q-criterion iso-surface', cmap='rainbow'…)`	3-D Q-criterion iso-surface coloured by velocity magnitude.
`plot_lbm_flow`	função	`(ux, uy, rho=None, obstacle_mask=None, *, dx=1.0, dy=1.0, title='LBM Flow Field', figsize…)`	4-panel LBM dashboard: [0] Velocity magnitude [1] Vorticity ω_z [2] Q-criterion [3] Pressure (density proxy) Solid obstacles are masked in grey.

nome	tipo	assinatura	o que faz
`plot_flow_panel`	função	`(x, y, ux, uy, pressure=None, obstacle_mask=None, *, title='Flow Field Analysis', figsize…)`	4-panel flow analysis: pressure, |v| + streamlines, Q-criterion, vorticity. Mirrors the post-processing style of commercial CFD dashboards.

`pinneapple_tools.visualization.voxel`

Visualisation of VoxelGrid objects.

`pinneapple_tools/visualization/voxel.py` · 127 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot_voxel_slice`	função	`(grid, *, axis='z', index=None, cmap=DEFAULT_CMAP, title='', show=False) -> Figure`	Plot a single 2-D slice through a 3-D VoxelGrid.
`plot_voxel_3d`	função	`(grid, *, threshold=0.5, cmap=DEFAULT_CMAP, alpha=0.6, title='Voxel occupancy', show=Fals…)`	Scatter-based 3-D visualisation of occupied voxels.
`plot_voxel_histogram`	função	`(grid, *, bins=50, title='Voxel value distribution', show=False) -> Figure`	Histogram of non-zero voxel values.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 70 de 177.
- Funções de teste em arquivos que importam algum módulo do pacote: 465.

Limitações conhecidas

- *Nenhuma registrada.*

2. `pinneapple_blender`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
5	616	1	7	15 (2 arquivos)	2

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`blender_addon`		1	1	0
`cycles`		1	3	8
`export`		1	3	7
`render`		1	1	7

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_blender`

Blender bridge: export a PINNeAPPle field/trajectory as a `.ply` sequence (`export.py`, pure Python + numpy, no Blender needed), and optionally build/save a Blender scene from it via a real local Blender installation (`render.py`, subprocess bridge to Blender's own `bpy`, mirroring the …

`pinneapple_blender/__init__.py` · 15 linhas · **sem teste direto**

`pinneapple_blender.blender_addon.import_pinneapple_sequence`

Runs INSIDE Blender's own Python (`bpy`) -- never imported by regular PINNeAPPle code, same reasoning as `cfd_formats/abaqus_reader.py`'s `_abaqus_odb_export_script.py`: `bpy` is Blender's bundled interpreter, not pip-installable into a normal environment, so this is launched as a …

`pinneapple_blender/blender_addon/import_pinneapple_sequence.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`()`	

`pinneapple_blender.cycles`

In-process Cycles renderer: build a small scene from NumPy arrays and render it with Blender's `bpy` module.

`pinneapple_blender/cycles.py` · 292 linhas · testes: 8

nome	tipo	assinatura	o que faz
`CyclesScene`	classe	`(resolution=(960, 540), samples=64, background=(0.97, 0.97, 0.96), ambient=0.6, transpare…)`	A fresh Blender scene rendered with Cycles on the CPU.
`available`	função	`() -> bool`	True when the `bpy` module can be imported.
`frames_to_video`	função	`(frames, path, *, fps=24, crf=28) -> str`	Assemble PNG frames (named `frame_%04d.png` in one directory) into an H.264 MP4 with ffmpeg.

`pinneapple_blender.export`

Export a PINNeAPPle result (a point cloud + one or more scalar/vector fields, optionally over several time steps) to a sequence of `.ply` files Blender can import natively (built-in importer, no add-on required just to look at one frame) -- the PINNeAPPle-side half of the Blender bridge; …

`pinneapple_blender/export.py` · 153 linhas · testes: 7

nome	tipo	assinatura	o que faz
`write_ply`	função	`(path, points, colors=None, scalar_fields=None) -> None`	Write a point cloud (no faces -- a vertex-only PLY, which Blender's importer turns into a mesh with only vertices, ready for a Point Cloud Visualizer-style geometry-nodes setup, or straightforward as a particle/point render). `colors`: (N,3) uint8 RGB, optional. `scalar_fields`: extra named …
`export_scene`	função	`(points, field, out_dir, *, field_name='field', cmap='viridis', frame_index=0, extra_fiel…)`	Export one frame (one time instant / one realisation) as `<out_dir>/frame_<NNNN>.ply`. Call once per frame with increasing `frame_index` to build a sequence -- see :func:`export_trajectory` for the common "I already have every frame's arrays" case in one call.
`export_trajectory`	função	`(points, fields_over_time, out_dir, *, field_name='field', cmap='viridis') -> list`	Export a whole sequence at once: `points` fixed (N, 3), one `field` array (N,) per frame in `fields_over_time`. This is the natural shape for a PINN surrogate's own output -- evaluate the model at the same spatial points across a range of `t` values and pass the resulting list of (N,) …

`pinneapple_blender.render`

Subprocess bridge to a real local Blender installation -- same pattern as `cfd_formats.abaqus_reader.export_odb_fields`: never guess at Blender's internals from outside, shell out to Blender's own bundled Python (`blender --background --python <script>`) running genuine `bpy` code …

`pinneapple_blender/render.py` · 61 linhas · testes: 7

nome	tipo	assinatura	o que faz
`build_scene`	função	`(ply_dir, *, output_blend=None, blender_executable='blender', timeout=600) -> str`	Run Blender headless to import a `.ply` sequence (as written by `pinneapple_blender.export`) into a scene, optionally saving it as a `.blend` file.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 3 de 5.
- Funções de teste em arquivos que importam algum módulo do pacote: 15.

Limitações conhecidas

- *Nenhuma registrada.*

3. `pinneapple_flow`

Objetivo

Esta parte cobre o núcleo do motor de fluxos do `pinneapple_flow` e os seus nós: o documento de fluxo (`schema`), o registro de tipos de nó com portas tipadas e formulários de parâmetros (`registry`), a execução sobre o motor de workflows de `pinneapple_core.workflows` (`runtime`), os nós embutidos (`nodes`) e os nós que reproduzem os estudos de

examples/* (nodes_ext), os nós gerados a partir da API pública (libcalls, autocalls, wrappers), o planejador que transforma uma descrição em texto em propostas de fluxo (planner), a paleta do Flow Studio (palette, studio), os registros de decisão com referências checadas (decisions, references), a identificação de equações e o gêmeo digital automático (identify, twin), os surrogates treináveis (learn) e a ligação com o selo do PINNeAPPle Lab (lab_status, examples).

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
89	18.086	23	381	263 (26 arquivos)	58

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
__main__		1	10	10
autocalls	Gera nós auto.<módulo>.<função> a partir das funções públicas da biblioteca que podem ser chamadas com segurança por um formulário, gravados em autocalls.json.	1	6	10
config		1	5	0
coverage		1	6	10
decisions	Registros de decisão de um fluxo (quem decidiu, por quê, com quais referências e evidências, com que confiança) e a seção de relatório em Markdown/HTML.	1	11	22
examples	Inventário de examples/* como fluxos: configurações rápidas, execução, comparações medidas e o documento EXAMPLES_AS_FLOWS.md.	1	15	8
identify	Identifica as equações que governam um problema a partir do texto (heurística offline ou LLM) e as descobre a partir de dados por regressão esparsa.	1	4	7
lab_status	Lê o selo do PINNeAPPle Lab de um item validado e diz que nós inserir no fluxo para subir de letra.	1	5	9
learn	Surrogates para nós de fluxo (blocos MLP, DeepONet, FNO, POD + MLP): construção, prévia como grafo de camadas, treino e busca de hiperparâmetros.	1	14	0
libcalls	Lista curada de chamadas públicas da biblioteca expostas como nós, com formulários gerados por introspecção e tipos de nó definidos pelo usuário.	1	10	10
libcalls_catalog		1	0	0
nodes	Tipos de nó embutidos: problema, dados, solvers, PINN, surrogates, métodos, validação, sistemas, raciocínio, veículo, visualização.	20	91	27
nodes_ext	Tipos de nó que reproduzem os estudos de examples/* como fluxos, um módulo por domínio, e as capacidades do planejador para eles.	38	112	8
palette	A paleta de nós do Flow Studio: seções em ordem de fluxo de engenharia, títulos legíveis, resumos e ranking de busca.	1	12	29
planner	Planejador: de uma descrição do problema a propostas de fluxo, por heurística offline, por LLM validado ou pelo assistente guiado.	6	23	16

sub-módulo	o que faz	módulos	classes + funções	testes *
`references`	Registro curado de referências (`references.yaml`) e a checagem (registro, DOI/URL) por que passa toda referência citada.	1	10	13
`registry`	Tipos de nó: portas tipadas, esquemas de parâmetros, a chamada de biblioteca de cada um e o decorador `node` que os registra.	1	6	46
`rules`		1	3	13
`runtime`	Executa um fluxo sobre o motor de workflows de `pinneapple_core.workflows`, com cache, re-execução parcial, artefatos e ganchos de fim de execução.	1	15	9
`schema`	O documento de fluxo `pinneapple-flow/1` (nós, arestas, metadados): carregar, salvar, validar e JSON Schema.	1	9	27
`service`		1	1	29
`store`		1	3	23
`studio`	Sobe o Flow Studio (`apps/flow_studio`, FastAPI + frontend React) em localhost, exigindo login para outros endereços.	1	3	38
`templates`		1	2	0
`twin`	Gêmeo digital montado automaticamente a partir de um sistema de equações identificado, sobre o `TwinRuntime` (filtro de Kalman por ensemble).	1	3	7
`types`		1	2	7
`wrappers`	Pontos de entrada com argumentos JSON para capacidades da biblioteca cuja API recebe objetos, para virarem nós de chamada de biblioteca.	1	18	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_flow`

PINNeAPPle Flow: engineering flows as typed node graphs, planned from a description, run headless or in Flow Studio.

`pinneapple_flow/__init__.py` · 91 linhas · testes: 231

nome	tipo	assinatura	o que faz
`register_references`	função	`(entries)`	Add curated references for another package's rules (see :func:`pinneapple_flow.references.register_references`).
`register_rule`	função	`(r)`	Add a heuristic rule that cites registered references (see :func:`pinneapple_flow.rules.register_rule`).
`run`	função	`(flow, **kw)`	Run a flow headless (see :func:`pinneapple_flow.runtime.run`).
`plan`	função	`(text, **kw)`	Proposals for a problem description (see :func:`pinneapple_flow.planner.plan`).
`catalog`	função	`() -> list[dict]`	Every node type as JSON (ports, parameter schema, availability).

`pinneapple_flow.__main__`

Command line: `python -m pinneapple_flow <command>`.

`pinneapple_flow/__main__.py` · 212 linhas · testes: 10

nome	tipo	assinatura	o que faz
cmd_run	função	(a) -> int	
cmd_plan	função	(a) -> int	
cmd_validate	função	(a) -> int	
cmd_catalog	função	(a) -> int	
cmd_examples	função	(a) -> int	
cmd_coverage	função	(a) -> int	
cmd_autocalls	função	(a) -> int	
cmd_references	função	(a) -> int	
cmd_studio	função	(a) -> int	
main	função	(argv=None) -> int	

pinneapple_flow.autocalls

Varre os módulos públicos que nenhum nó curado cobre e gera nós `auto.*` só para funções com argumentos de tipo simples e sem caminhos, URLs, comandos, `eval` ou escrita de arquivo; o resultado fica em `autocalls.json` e é regenerado por `python -m pinneapple_flow autocalls`.

`pinneapple_flow/autocalls.py` · 236 linhas · testes: 10

nome	tipo	assinatura	o que faz
scan	função	(module) -> tuple[list[dict[str, Any]], str]	Gera as entradas de nó de um módulo, ou a razão de ele não ter nenhuma.
generate	função	(modules=None) -> dict[str, Any]	Varre os módulos (por padrão todos os que os nós curados não cobrem) e devolve os dados de `autocalls.json`.
smoke	função	(calls, *, timeout=30.0, workers=6) -> None	Chama cada função gerada com os padrões do formulário num interpretador novo em pasta vazia e grava o resultado em `entry["check"]`.
prune	função	(data) -> None	Descarta entradas cuja checagem mostra que devolvem uma função ou classe, e dá essa razão ao módulo que ficar vazio.
load	função	() -> dict[str, Any]	Lê `autocalls.json` (ou devolve listas vazias se o arquivo não existe).
write	função	(data) -> None	Grava os dados em `autocalls.json`.

pinneapple_flow.config

Settings from the environment (and an optional `.env` file): LLM provider, database URL, folders, server.

`pinneapple_flow/config.py` · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Settings	classe	(host, port, db_url, llm_provider, llm_model, llm_base_url, llm_api_key)	
load_dotenv	função	(path='.env', *, override=False) -> dict[str, str]	Read `KEY=value` lines (# comments, optional quotes) into `os.environ`; returns what was read.
home	função	() -> Path	

nome	tipo	assinatura	o que faz
`data_root`	função	`() -> Path`	
`safe_data_path`	função	`(name) -> Path`	A path inside the data folder (no absolute paths elsewhere, no `..` escapes).

pinneapple_flow.coverage

Which PINNeAPPle capabilities are flow nodes, and why the others are not.

`pinneapple_flow/coverage.py` · 183 linhas · testes: 10

nome	tipo	assinatura	o que faz
`packages`	função	`() -> list[str]`	
`modules`	função	`(pkg) -> list[str]`	
`node_targets`	função	`(auto=True) -> dict[str, set[str]]`	module path -> node types that call into it (`auto`: with the nodes generated from the public API).
`coverage`	função	`(auto=True) -> list[dict[str, str]]`	One row per package and first-level module: the node types that reach it, or why there are none (a curated reason, or the one the public-API scan of :mod:`pinneapple_flow.autocalls` recorded).
`counts`	função	`(rows=None) -> dict[str, int]`	
`markdown`	função	`() -> str`	

pinneapple_flow.decisions

Define `Decision` e as fábricas de decisões de regra, de usuário e de LLM; a confiança vem de quem decidiu e da qualidade das referências (registro > verificada > não verificada), limitada a 0.35 sem referência nem evidência.

`pinneapple_flow/decisions.py` · 236 linhas · testes: 22

nome	tipo	assinatura	o que faz
`Decision`	classe	`(topic, what, decided_by, rationale, value, references, evidence, node, notes, uncited_re…)`	Registro de uma decisão do fluxo: tema, o que foi decidido, por quem, justificativa, valor, referências, evidências, confiança e rótulo.
`score`	função	`(d) -> Decision`	Calcula a confiança de uma decisão a partir de quem decidiu, das referências e das evidências; sem nenhuma, fica no máximo 0.35.
`rule_decision`	função	`(rule_id, *, what, node='', value=None, rationale='', topic=None, extra=()) -> Decision`	Cria a decisão tomada por uma regra heurística, com as referências e evidências da regra.

nome	tipo	assinatura	o que faz
`user_decision`	função	`(node, topic, what, value=None, *, notes='', references=(), user='user') -> Decision`	Cria a decisão de uma edição do usuário, com notas e referências checadas no registro ou por DOI/URL.
`llm_decision`	função	`(provider_model, *, topic, what, node='', value=None, rationale='', references=()) -> Dec…)`	Cria a decisão proposta por um LLM; cada referência é checada e nada é aceito só pela palavra do modelo.
`edits_to_decisions`	função	`(edits) -> list[Decision]`	Converte as edições em `flow.meta["user_edits"]` em decisões de usuário.
`collect`	função	`(flow, reports=None) -> list[dict[str, Any]]`	Junta todas as decisões de um fluxo: do planejador/LLM, das edições do usuário e as tomadas pelos nós durante a execução.
`by_node`	função	`(decisions) -> dict[str, list[dict[str, Any]]]`	Agrupa decisões pelo nó a que se referem.
`summary`	função	`(decisions) -> dict[str, Any]`	Resumo das decisões: total, quantas sem referência, contagem por rótulo e referências citadas.
`report_markdown`	função	`(decisions, title='Decisions and references') -> str`	Gera a seção "Decisions and references" do relatório em Markdown.
`report_html`	função	`(decisions, title='Decisions and references') -> str`	Gera a mesma seção em HTML com referências clicáveis.

`pinneapple_flow.examples`

Liga cada pasta de `examples/*` aos seus fluxos em `flows/examples/`, roda os fluxos (completo ou rápido), grava os resultados em `measured.json` e gera `EXAMPLES_AS_FLOWS.md`; `problems` é o portão de CI.

`pinneapple_flow/examples.py` · 410 linhas · testes: 8

nome	tipo	assinatura	o que faz
`inventory`	função	`() -> dict[str, Any]`	Lê `flows/examples/inventory.json` (estado e razão de cada pasta de exemplo).
`measured`	função	`() -> dict[str, Any]`	Lê `flows/examples/measured.json` (resultados medidos das execuções dos fluxos de exemplo).
`example_folders`	função	`(root=None) -> list[str]`	Lista as pastas de `examples/*` que são exemplos (sem rascunhos, `_out`, `_runs` e pastas de resultados).
`flows`	função	`() -> list[dict[str, Any]]`	Lista todos os fluxos de exemplo com nome, caminho, título, domínio, nós e se têm configuração rápida.
`flow_path`	função	`(name) -> Path`	Caminho do JSON de um fluxo de exemplo pelo nome; `KeyError` se não existir.
`with_quick`	função	`(doc) -> dict[str, Any]`	Devolve o fluxo com a configuração rápida (`meta.quick`) aplicada.
`run_example`	função	`(name, *, quick=False, workdir=None, record=True, on_update=None) -> dict[str, Any]`	Roda um fluxo de exemplo (completo ou rápido) e, com `record`, grava resultado e comparações em `measured.json`.

nome	tipo	assinatura	o que faz
`register_example_run`	função	`(result, flow) -> str`	Gancho de fim de execução que grava a execução de um fluxo de exemplo como manifesto de experimento e o importa no Lab do Flow Studio.
`register_hook`	função	`() -> None`	Adiciona `register_example_run` a `RUN_FINISHED_HOOKS` (uma vez só).
`scan`	função	`(example) -> dict[str, Any]`	Por script de um exemplo, lista as chamadas de biblioteca que ele importa e quais um nó declara.
`status`	função	`(example, inv=None, meas=None) -> dict[str, Any]`	Linha do inventário de um exemplo: fluxos, estado, razão e comparação medida.
`problems`	função	`() -> list[str]`	Portão de CI: pastas de exemplo sem fluxo nem razão, estados inválidos e fluxos que citam exemplo inexistente.
`markdown`	função	`() -> str`	Gera o texto de `docs/flow_studio/EXAMPLES_AS_FLOWS.md`.
`planner_rows`	função	`(rows) -> list[str]`	Tabela do planejador: por exemplo com fluxo, se o planejador heurístico propõe a capacidade em primeiro lugar a partir do título.
`main`	função	`(argv) -> int`	CLI `python -m pinneapple_flow example-flows` (list, run, doc, check).

`pinneapple_flow.identify`

Classifica sistemas de equações candidatos para uma descrição (palavras-chave, unidades, valores explícitos e, opcionalmente, um LLM validado) e ajusta uma EDP a dados em grade por regressão esparsa (STRidge/SINDy).

`pinneapple_flow/identify.py` · 256 linhas · testes: 7

nome	tipo	assinatura	o que faz
`identify`	função	`(text, *, dimension=None, units=None, n=3, llm=None, call=None) -> dict[str, Any]`	Devolve sistemas de equações candidatos ordenados para uma descrição, com confiança, raciocínio e decisões; um LLM opcional acrescenta um candidato validado.
`discover_pde`	função	`(axes, values, coords, *, field='u', threshold=0.05, trim=3) -> dict[str, Any]`	Ajusta `u_t` (e `u_tt`) a termos candidatos por regressão esparsa sobre dados em grade e devolve a equação, coeficientes, R^2 e família.
`confirm`	função	`(candidate, found) -> dict[str, Any]`	Diz se a equação descoberta concorda com um candidato (mesma família e, quando possível, coeficientes).
`to_json`	função	`(res) -> str`	Serializa o resultado em JSON convertendo arrays numpy em listas.

`pinneapple_flow.lab_status`

Consulta `pinneapple_lab.curation` e `pinneapple_lab.seal` (detectados em tempo de execução) para mostrar a letra do selo e os próximos passos; o Flow Studio nunca dá nota sozinho.

`pinneapple_flow/lab_status.py` · 211 linhas · testes: 9

nome	tipo	assinatura	o que faz
seal_status	função	(item, lab=None) -> dict[str, Any]	Letra do selo de um item do Lab com sua razão; None quando o item nunca foi curado.
nodes_for	função	(action='', command='', params=None, target='') -> list[dict[str, Any]]	Tipos de nó registrados que executam uma ação do Lab (pelo tipo ou pela chamada de biblioteca que declaram), curados primeiro.
next_steps	função	(item, lab=None) -> dict[str, Any]	O que falta ao item para a próxima letra, com a ação do Lab e os nós que a executam.
lab_items	função	(reports) -> list[str]	Itens do Lab gravados pelos nós validate.lab de uma execução.
insert_nodes	função	(flow, anchor, items) -> tuple[dict[str, Any], list[str]]	Insere nós num fluxo e liga cada entrada obrigatória a partir de um nó âncora ou de nós compatíveis; devolve o fluxo e as entradas não ligadas.

pinneapple_flow.learn

Monta surrogates torch a partir das arquiteturas da biblioteca num formato único de dataset (A, Y, grid), normaliza, treina com Adam e mede erros em partições separadas. [sem teste]

pinneapple_flow/learn.py · 366 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
default_config	função	(kind, in_dim, out_dim) -> dict[str, Any]	Arquitetura sugerida para um tipo de surrogate e os tamanhos dos dados.
blocks_net	função	(in_dim, out_dim, blocks)	Monta um torch.nn.Sequential a partir de blocos editáveis (dense, residual, dropout, layernorm) com camada linear final.
build	função	(cfg, in_dim, out_dim, *, n_fields=1, grid=None)	Constrói o surrogate torch de uma configuração, mapeando (B, in_dim) para (B, out_dim) em unidades normalizadas.
count_parameters	função	(model) -> int	Conta os parâmetros treináveis de um modelo.
layer_graph	função	(model) -> dict[str, Any]	Grafo das camadas folha com contagem de parâmetros, agrupadas por bloco (ramo e tronco da DeepONet em grupos separados).
architecture	função	(cfg, in_dim, out_dim, *, n_fields=1, grid=None) -> dict[str, Any]	Prévia de uma arquitetura sem treinar: grafo de camadas, número de parâmetros e a configuração usada.
split	função	(n, val=0.15, test=0.15, seed=0)	Divide índices em treino, validação e teste de forma aleatória com semente.
rel_l2_rows	função	(pred, true) -> float	Média por amostra de ||pred - true|| / ||true||.
fit	função	(cfg, data, *, epochs=500, lr=0.002, batch=32, seed=0, n_fields=1, splits=None, callback=…)	Treina o surrogate com Adam e agenda cosseno, guardando os pesos de menor erro de validação; devolve modelo e métricas por partição.
save	função	(model, cfg, path, *, n_fields=1, meta=None) -> None	Salva configuração, tamanhos e pesos do surrogate com torch.save.
load	função	(obj)	Reconstrói um surrogate a partir do que save gravou.

nome	tipo	assinatura	o que faz
search_space	função	(spec) -> dict[str, tuple]	Converte uma especificação simples de espaço de busca em parâmetros de AdaptiveSweepConfig.
apply_trial	função	(cfg, trial) -> tuple[dict[str, Any], dict[str, Any]]	Separa os valores de uma tentativa em mudanças de arquitetura e opções de treino.
tune	função	(cfg, data, space, *, n_trials=12, epochs=200, seed=0, n_fields=1, log=print)	Busca de hiperparâmetros (Optuna TPE se instalado, senão busca aleatória) minimizando o erro de validação em partições fixas.

pinneapple_flow.libcalls

Mantém CALLS, a lista de chamadas permitidas; gera o esquema de parâmetros a partir da assinatura e só executa entradas da lista, inclusive para nós custom.* criados pelo usuário.

pinneapple_flow/libcalls.py · 409 linhas · testes: 10

nome	tipo	assinatura	o que faz
LibCall	classe	(key, target, title, category, description, refs, inputs, exclude, overrides, init, requi…)	Entrada da lista permitida: alvo módulo:atributo, título, categoria, referências, entradas vindas de portas, parâmetros excluídos ou sobrescritos e tipo de saída.
resolve	função	(target)	Devolve o objeto chamável de um alvo da lista permitida.
introspect	função	(call) -> tuple[list[Param], list[str]]	Lê os parâmetros de tipo simples de uma chamada e os nomes que não puderam ser expostos.
summarize	função	(ctx, result, depth=0) -> dict[str, Any]	Transforma o que uma chamada devolveu em entradas de relatório e um resultado JSON.
spec_for	função	(call, *, type_=None, title=None, fixed=None, defaults=None, hidden=()) -> NodeSpec	Monta o NodeSpec de uma LibCall, com valores fixos, padrões e parâmetros ocultos opcionais.
wrapped	função	(call) -> list[str]	Para um wrapper de pinneapple_flow.wrappers, lê no código-fonte os módulos da biblioteca que ele importa.
custom_spec	função	(doc) -> NodeSpec	Cria um tipo de nó custom.* do usuário sobre uma entrada de CALLS; ValueError se citar outra coisa ou parâmetro inexistente.
library_specs	função	() -> list[NodeSpec]	Os NodeSpec de todas as entradas de CALLS.
auto_specs	função	() -> list[NodeSpec]	Os NodeSpec dos nós gerados em autocalls.json; os módulos só são importados quando o nó roda.
add	função	(*calls) -> None	Acrescenta LibCalls a CALLS, recusando chaves duplicadas.

pinneapple_flow.libcalls_catalog

The curated allowlist of library calls exposed as nodes (lib.<key>). Parameter forms come from the signatures; this file only says which callable, where it sits in the palette, which inputs come from ports and which references back it. Add a capability by adding an entry.

pinneapple_flow/libcalls_catalog.py · 119 linhas · **sem teste direto**

pinneapple_flow.nodes

Importa os módulos de nós embutidos, registra os nós de chamada de biblioteca (`library_specs` e `auto_specs`) e depois os nós de exemplos de `nodes_ext`.

pinneapple_flow/nodes/__init__.py · 29 linhas · testes: 17

pinneapple_flow.nodes._common

Peças comuns dos nós embutidos: documentos de problema, artefatos de solução em grade, soluções exatas, interpolação e gráficos. [sem teste direto]

pinneapple_flow/nodes/_common.py · 186 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
coords_of	função	(geometry) -> list[str]	Lista de coordenadas de uma geometria.
bounds_of	função	(problem) -> dict[str, tuple[float, float]]	Limites por eixo de um problema, incluindo t quando há tempo.
system_of	função	(problem, *, coefficients=None)	O EquationSystem analisado de um documento de problema.
need_problem	função	(v) -> dict[str, Any]	Garante que o valor é um problema com geometria e equações, senão falha o passo.
grid_points	função	(axes, coords) -> np.ndarray	Pontos de uma grade produto dos eixos, como matriz (N, d).
save_solution	função	(ctx, name, axes, values, coords, fields, method, label='', extra=None) -> dict[str, Any]	Grava uma solução em grade como artefato npz com eixos, campos e método.
load_solution	função	(v) -> dict[str, Any]	Lê um artefato de solução em grade (eixos, valores, coordenadas, campos).
exact_function	função	(problem)	Função da solução exata do problema, ou None.
interpolate	função	(sol, X) -> np.ndarray	Interpola linearmente uma solução em grade em pontos dados.
eval_axes	função	(problem, n=None, n_t=None) -> dict[str, np.ndarray]	Grade de avaliação de um problema: pontos por eixo dados, a malha anexada ou um padrão por dimensão.
plot_solution	função	(sol, title='', field=0)	Figura de uma solução: curvas no tempo (1-D), mapa (2-D) ou cortes no plano médio (3-D).

pinneapple_flow.nodes.analysis

Nós de validação `validate.compare` (erros contra referência e checagens de física do trust card) e `validate.lab` (curadoria do PINNeAPPle Lab com letra A-F e selo na figura). [sem teste direto]

pinneapple_flow/nodes/analysis.py · 306 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
compare	função	(ctx, inputs, params)	Nó validate.compare: mede cada candidato contra a referência nos mesmos pontos (L2 relativo, erro máximo e RMS) e as checagens de física.
lab	função	(ctx, inputs, params)	Nó validate.lab: grava a validação como execução do Lab, dá a letra A-F e carimba o selo na figura só nos níveis A e B.

pinneapple_flow.nodes.autolab

Nós do Autonomous Physics Lab (pinneapple_autolab): resfriamento de placa, gêmeo de tubulação que sabe quando erra e o aerodinamicista com OpenFOAM como referência. [sem teste direto]

pinneapple_flow/nodes/autolab.py · 143 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
board_cooling	função	(ctx, inputs, params)	Nó autolab.board_cooling: acha o dissipador mais leve e a velocidade do ventilador de uma placa dentro dos limites, com verificação em malhas finas e relatório de confiança.
pipe_twin	função	(ctx, inputs, params)	Nó autolab.pipe_twin: gêmeo de tubulação que distingue mudança de processo, sensor errado e modelo fora do domínio, medido em casos controlados.
aerodynamicist	função	(ctx, inputs, params)	Nó autolab.aerodynamicist: reduz o arrasto de um carro de corrida paramétrico mantendo downforce e balanço, com OpenFOAM como referência e surrogates comparados.

pinneapple_flow.nodes.cfd_pipeline

Nós do pipeline FreeCAD -> OpenFOAM (pinneapple_simulation.pipelines.freecad_openfoam): estudo, geometria, execução, varredura e surrogate da varredura; precisam do OpenFOAM instalado. [sem teste direto]

pinneapple_flow/nodes/cfd_pipeline.py · 172 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
study	função	(ctx, inputs, params)	Nó cfd.study: modelo FreeCAD paramétrico com regras de patches, malha, física, seções e referências.
geometry	função	(ctx, inputs, params)	Nó cfd.geometry: ajusta a planilha, recalcula, exporta e nomeia os patches, sem solver.
run	função	(ctx, inputs, params)	Nó cfd.run: CAD, malha, solução, pós-processamento e checagens de um caso.
sweep	função	(ctx, inputs, params)	Nó cfd.sweep: varredura de parâmetros retomável, um processo por caso.
surrogate	função	(ctx, inputs, params)	Nó cfd.surrogate: todos os surrogates aplicáveis com o mesmo orçamento, erro medido em casos separados.

pinneapple_flow.nodes.condense

Nó de condensação estática hierárquica de um encapsulamento de chip (pinneapple_condense). [sem teste direto]

pinneapple_flow/nodes/condense.py · 64 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`package_thermal`	função	`(ctx, inputs, params)`	Nó `condense.package_thermal`: resolve um encapsulamento multicamada sintético condensando blocos repetidos e relata temperatura de junção e balanço de energia.

`pinneapple_flow.nodes.data`

Nós de dados: observações sintéticas de uma solução, leitura de CSV, conjuntos paramétricos e o benchmark Burgers com condições iniciais de campo aleatório gaussiano. [sem teste direto]

`pinneapple_flow/nodes/data.py` · 194 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`observe`	função	`(ctx, inputs, params)`	Nó `data.observe`: observações pontuais de uma solução (sensores sintéticos) com ruído, guardando os coeficientes verdadeiros.
`ingest`	função	`(ctx, inputs, params)`	Nó `data.ingest`: lê medições de CSV colado ou de arquivo da pasta de dados.
`parametric`	função	`(ctx, inputs, params)`	Nó `data.parametric`: conjunto de treino para surrogates com um solve por diferenças finitas por amostra.
`grf_burgers`	função	`(ctx, inputs, params)`	Nó `data.grf_burgers`: condições iniciais de campo aleatório gaussiano mapeadas para a solução de Burgers viscosa (solver pseudo-espectral).
`dataset_of`	função	`(v)`	Lê um dataset de operador (`A`, `Y`, `grid`) de um artefato, senão falha o passo.

`pinneapple_flow.nodes.methods`

Nós de métodos que envolvem implementações da biblioteca: convergência de malha, filtro de Kalman, propagação de incerteza, ROMs, MeshGraphNet, transferência, meta-aprendizado, arena de PINNs, NSGA-II e controle. [sem teste direto]

`pinneapple_flow/nodes/methods.py` · 657 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`convergence`	função	`(ctx, inputs, params)`	Nó `verify.convergence`: três malhas aninhadas, ordem observada, extrapolação de Richardson e GCI.
`kalman`	função	`(ctx, inputs, params)`	Nó `assimilate.filter`: estima estados de canais ruidosos com os filtros de Kalman da biblioteca.
`propagate`	função	`(ctx, inputs, params)`	Nó `uq.propagate`: propaga incerteza uniforme nas entradas do surrogate por amostragem ou caos polinomial.
`rom_fit`	função	`(ctx, inputs, params)`	Nó `rom.fit`: ajusta ROMs aos snapshots e compara os erros de previsão com o mesmo orçamento.
`meshgraphnet`	função	`(ctx, inputs, params)`	Nó `gnn.meshgraphnet`: treina uma MeshGraphNet num campo de difusão em malhas aleatórias e compara com o estado inicial congelado.
`transfer`	função	`(ctx, inputs, params)`	Nó `transfer.pinn`: fine-tuning de uma PINN treinada no problema alvo contra treino do zero com as mesmas épocas.
`meta_learn`	função	`(ctx, inputs, params)`	Nó `meta.learn`: meta-aprende uma inicialização de PINN numa família de problemas e compara a adaptação com inicialização aleatória.

nome	tipo	assinatura	o que faz
`arena`	função	`(ctx, inputs, params)`	Nó `benchmark.arena`: mesmos problemas, orçamento e épocas para todos os modelos, com L2 relativo e posição por problema.
`nsga2`	função	`(ctx, inputs, params)`	Nó `optimize.nsga2`: busca um problema de projeto com NSGA-II (ou um baseline) e relata a frente de Pareto.
`control`	função	`(ctx, inputs, params)`	Nó `control.compare`: compara controladores em malha fechada no tanque aquecido da biblioteca (só simulação, sem comando a equipamento).

`pinneapple_flow.nodes.physics_learning`

Nó que verifica se um modelo térmico aprendido conhece a física com um verificador independente da perda de treino (`pinneapple_analysis.validation.physics_learning`). [sem teste direto]

`pinneapple_flow/nodes/physics_learning.py` · 62 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`physics_learning`	função	`(ctx, inputs, params)`	Nó `analysis.physics_learning`: checa resíduo, fluxo, contorno, balanço de energia, simetria, escala e intervenções causais contra referência de volumes finitos.

`pinneapple_flow.nodes.presets`

Nós de presets industriais (`pinneapple_presets`): rodar um caso pronto com parâmetros próprios e validá-lo contra os casos de referência.

`pinneapple_flow/nodes/presets.py` · 60 linhas · testes: 27

nome	tipo	assinatura	o que faz
`run_preset`	função	`(ctx, inputs, params)`	Nó `preset.run`: resolve um caso industrial pronto com as entradas do usuário.
`validate_preset`	função	`(ctx, inputs, params)`	Nó `preset.validate`: roda os casos de referência do preset e mede o erro.

`pinneapple_flow.nodes.problem`

Nós de problema: domínio (1-D, 2-D, 3-D), malha, sistema de equações em texto validado por parser seguro e o problema genérico que identifica as equações. [sem teste direto]

`pinneapple_flow/nodes/problem.py` · 227 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`geometry_doc`	função	`(params) -> dict[str, Any]`	Monta o documento de geometria (dimensão, limites, tempo) a partir dos parâmetros, checando os intervalos.
`domain`	função	`(ctx, inputs, params)`	Nó `geometry.domain`: dimensão, limites por eixo e intervalo de tempo.
`mesh`	função	`(ctx, inputs, params)`	Nó `geometry.mesh`: grade estruturada com nós por eixo e níveis de tempo salvos.
`equations`	função	`(ctx, inputs, params)`	Nó `equations.system`: EDPs, condições e coeficientes em texto, analisados por parser seguro e validados.

nome	tipo	assinatura	o que faz
require_problem	função	(v)	Atalho para need_problem.
generic	função	(ctx, inputs, params)	Nó problem.generic: identifica as equações de qualquer problema descrito, com confiança e referências, e as descobre a partir de dados quando houver.

pinneapple_flow.nodes.query

Nó de consulta física de um encapsulamento de chip, respondida pelo método válido mais barato com incerteza e procedência (pinneapple_query). [sem teste direto]

pinneapple_flow/nodes/query.py · 46 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
package_queries	função	(ctx, inputs, params)	Nó query.package: responde cada consulta pelo método mais barato que atinge a meta e diz qual, com incerteza.

pinneapple_flow.nodes.reason

Nós de raciocínio de engenharia (pinneapple_reason): requisito, conceito, modo de falha, verificação, contramedida, cadeia de porquês e relatório.

pinneapple_flow/nodes/reason.py · 183 linhas · testes: 17

nome	tipo	assinatura	o que faz
requirement	função	(ctx, inputs, params)	Nó reason.requirement: o que o projeto deve fazer e quais modos de falha o impediriam.
concept	função	(ctx, inputs, params)	Nó reason.concept: uma versão do projeto com parâmetros, elementos e origem de cada valor.
failure_mode	função	(ctx, inputs, params)	Nó reason.failure_mode: um modo de falha no conceito com demanda, capacidade e mecanismo.
verify	função	(ctx, inputs, params)	Nó reason.verify: todos os modos de falha de todos os requisitos e o primeiro que falha.
countermeasure	função	(ctx, inputs, params)	Nó reason.countermeasure: tenta as contramedidas da primeira falha e aplica a escolhida.
chain	função	(ctx, inputs, params)	Nó reason.chain: ciclo checar-corrigir a partir do conceito ingênuo até todos os requisitos passarem.
report	função	(ctx, inputs, params)	Nó reason.report: a cadeia como relatório com linha do tempo, margens, figuras e relatório de confiança.

pinneapple_flow.nodes.science

Nós que rodam um experimento de pinneapple_science contra a referência citada e desenham o atlas das áreas cobertas.

pinneapple_flow/nodes/science.py · 65 linhas · testes: 19

nome	tipo	assinatura	o que faz
`run_experiment`	função	`(ctx, inputs, params)`	Nó `science.run`: roda um experimento científico, cada grandeza contra sua referência citada.
`atlas`	função	`(ctx, inputs, params)`	Nó `science.atlas`: mapa das áreas científicas disponíveis, parciais e planejadas.

`pinneapple_flow.nodes.solvers`

Nós de solver: clássico (diferenças finitas, elementos finitos), OpenFOAM, PINN, PINN inverso e calibração por mínimos quadrados. [sem teste direto]

`pinneapple_flow/nodes/solvers.py` · 362 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`classical`	função	`(ctx, inputs, params)`	Nó `solver.classical`: resolve o sistema editado com método clássico e relata custo e resíduo.
`openfoam`	função	`(ctx, inputs, params)`	Nó `solver.openfoam`: roda um caso OpenFOAM (blockMesh + solver) e lê a saúde do log; precisa do OpenFOAM no PATH.
`pinn`	função	`(ctx, inputs, params)`	Nó `train.pinn`: treina uma PINN nas equações editadas (Adam e L-BFGS) e a valida em pontos novos.
`discover`	função	`(ctx, inputs, params)`	Nó `inverse.discover`: identifica coeficientes desconhecidos junto com o campo a partir de observações (PINN inversa).
`calibrate`	função	`(ctx, inputs, params)`	Nó `inverse.calibrate`: calibração por mínimos quadrados com o solver de diferenças finitas como modelo direto.

`pinneapple_flow.nodes.surrogates`

Nós de surrogate sobre `pinneapple_flow.learn`: treino com arquitetura editável, HPO, fine-tuning e índices de Sobol. [sem teste direto]

`pinneapple_flow/nodes/surrogates.py` · 274 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`surrogate`	função	`(ctx, inputs, params)`	Nó `surrogate.train`: treina um surrogate num dataset e relata erros de validação e teste.
`hpo`	função	`(ctx, inputs, params)`	Nó `surrogate.hpo`: busca arquitetura e treino na validação e retreina o melhor, sem usar o teste na busca.
`finetune`	função	`(ctx, inputs, params)`	Nó `train.finetune`: continua o treino de um surrogate com dados novos e compara o erro antes e depois.
`sensitivity`	função	`(ctx, inputs, params)`	Nó `uq.sensitivity`: índices de Sobol de uma grandeza do campo em relação às entradas do surrogate.

`pinneapple_flow.nodes.systems`

Nós de sistemas: gêmeo digital do tanque aquecido, previsão do ENSO (baixa dados e precisa de statsmodels), laboratório de previsão de séries e decolagem com a biblioteca de voo. [sem teste direto]

`pinneapple_flow/nodes/systems.py` · 424 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`twin`	função	`(ctx, inputs, params)`	Nó `twin.connect`: alimenta o runtime do gêmeo (Kalman por ensemble no tanque aquecido) com leituras e devolve estimativas, alertas e previsão.
`enso`	função	`(ctx, inputs, params)`	Nó `forecast.enso`: previsão do índice Niño 3.4 com o protocolo da biblioteca de ENSO; baixa as observações uma vez.
`forecast`	função	`(ctx, inputs, params)`	Nó `forecast.lab`: testes de previsibilidade e depois um conjunto de previsores em validação walk-forward com período de teste intocado.
`takeoff`	função	`(ctx, inputs, params)`	Nó `flight.takeoff`: simula uma decolagem com a biblioteca de voo 6-DOF e compara a distância com a forma fechada de Raymer.

`pinneapple_flow.nodes.twin_nodes`

Gêmeo digital automático: nó de dados ao vivo (sensores simulados, arquivo, MQTT, OPC UA) e nó que monta o gêmeo a partir das equações identificadas. [sem teste direto]

`pinneapple_flow/nodes/twin_nodes.py` · 279 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`stream`	função	`(ctx, inputs, params)`	Nó `data.stream`: leituras de sensores simuladas, de arquivo ou ao vivo via MQTT / OPC UA.
`twin_auto`	função	`(ctx, inputs, params)`	Nó `twin.auto`: monta o gêmeo a partir das equações identificadas, com alertas e painel.

`pinneapple_flow.nodes.vehicle`

Nós do rover planetário de `pinneapple_simulation.vehicle`: veículo, solo, terreno, travessia multicorpo, modelo de roda Wong-Reece, PINN do model zoo, painel animado e a missão que aprende o solo.

`pinneapple_flow/nodes/vehicle.py` · 479 linhas · testes: 6

nome	tipo	assinatura	o que faz
`vehicle_presets`	função	`() -> dict[str, Any]`	Funções de `pinneapple_simulation.vehicle` que constroem um `VehicleSpec`.
`build_spec`	função	`(v)`	Constrói o veículo a partir do preset e da massa, com geometria de roda editável.
`build_soil`	função	`(v)`	Constrói o solo do banco de terramecânica com parâmetros sobrescritos.
`build_site`	função	`(v)`	Constrói o terreno pelo cenário escolhido.
`load_run`	função	`(v) -> dict[str, Any]`	Lê a travessia gravada por um nó `vehicle.simulate`, conferindo o checksum.
`virtual_display`	função	`(size=(1920, 1080))`	Display para pyvista: o atual ou um Xvfb iniciado durante o bloco.
`vehicle_spec`	função	`(ctx, inputs, params)`	Nó `vehicle.spec`: rover da biblioteca (NASA VIPER) com massa e roda editáveis.
`soil`	função	`(ctx, inputs, params)`	Nó `terramechanics.soil`: solo do banco de terramecânica (parâmetros de Bekker / Wong-Reece) com sobrescrita.

nome	tipo	assinatura	o que faz
`site`	função	`(ctx, inputs, params)`	Nó `scenario.site`: terreno deformável com cratera e rocha, ou rampa plana.
`simulate`	função	`(ctx, inputs, params)`	Nó `vehicle.simulate`: dinâmica multicorpo do rover sobre o terreno com rodas Wong-Reece.
`wheel_model`	função	`(ctx, inputs, params)`	Nó `terramechanics.wheel_model`: modelo de roda Wong-Reece e suas curvas de bancada na carga estática.
`build_wheel_model`	função	`(v)`	Constrói o `WongReeceModel` a partir do solo, do veículo e das opções.
`zoo_surrogate`	função	`(ctx, inputs, params)`	Nó `terramechanics.zoo_surrogate`: PINN de contato da roda do model zoo (baixada uma vez, checksum verificado) com seus erros do model card.
`dashboard`	função	`(ctx, inputs, params)`	Nó `vehicle.dashboard`: painel animado da travessia com gráficos sincronizados, em HTML, PNG, MP4/GIF.
`learn_planet`	função	`(ctx, inputs, params)`	Nó `rover.learn_planet`: missão em que o rover atola, aprende o solo pelas rodas (EKF iterado) e replaneja, num terreno sintético de solos conhecidos.
`ctx_metrics`	função	`(nums) -> dict[str, float]`	Métricas numéricas planas do resumo de uma missão do rover.

`pinneapple_flow.nodes.viz`

Nós de visualização e exportação: gráficos, visualizador three.js, render (Blender Cycles se `bpy` existe, senão matplotlib) e relatório autocontido. [sem teste direto]

`pinneapple_flow/nodes/viz.py` · 237 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`plot`	função	`(ctx, inputs, params)`	Nó `viz.plot`: uma figura por solução (curvas, mapa de cores ou cortes).
`field3d`	função	`(ctx, inputs, params)`	Nó `viz.field3d`: visualização interativa three.js do campo.
`render`	função	`(ctx, inputs, params)`	Nó `viz.render`: imagem e animação de um campo 2-D com Blender Cycles ou, sem `bpy`, matplotlib.
`report`	função	`(ctx, inputs, params)`	Nó `export.report`: relatório autocontido com métricas, parâmetros e figuras embutidas.

`pinneapple_flow.nodes_ext`

Node types that reproduce the `examples/*` studies as flows, one module per domain, registered through the public API (:func:`pinneapple_flow.registry.node`, :func:`pinneapple_flow.types.register_port_type`). Each node wraps library calls; the flows that use them are under …

`pinneapple_flow/nodes_ext/__init__.py` · 58 linhas · **sem teste direto**

pinneapple_flow.nodes_ext.aero

Aircraft aerodynamics from the geometry library (`examples/geometry_library/03_airliner_vlm`): the vortex lattice of the `airliner_a320` entry (:func:`pinneapple_design.aero.vlm.solve`) and the friction-drag build-up of its mesh (:func:`pinneapple_design.aero.buildup.friction_drag`).

`pinneapple_flow/nodes_ext/aero.py` · 71 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`airliner_vlm`	função	`(ctx, inputs, params)`	Lift slope, neutral point (wing + tail), trim at the cruise lift coefficient, induced drag; CD0 from the library mesh; L/D. Published A320 class: CL_alpha about 5.5-6 /rad, cruise L/D about 16-18.

pinneapple_flow.nodes_ext.architectures

The model registry examples (`examples/architectures`): the registry and catalog listing (00, 01, the Group-B check of 60), building registry models by name with a forward pass on dummy tensors (00, 60) and POD compression with a DMD rollout on a synthetic linear system (30).

`pinneapple_flow/nodes_ext/architectures.py` · 172 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`registry_listing`	função	`(ctx, inputs, params)`	The registry's families and models, a few model specs and the catalog by family. This is what the registry tour and the model list print.
`forward`	função	`(ctx, inputs, params)`	One registry model built by name and run on dummy tensors. Reports the output shape, the parameter count, the losses it returns and, on request, the input-gradient norm (gradient flow for PINN use).
`pod_dmd`	função	`(ctx, inputs, params)`	POD reconstruction and POD + DMD rollout errors on a synthetic linear system. Trajectories of a contractive random linear system are compressed by POD (rank r), advanced by DMD on the POD coefficients and decoded.

pinneapple_flow.nodes_ext.arena_pipelines

The Arena pipelines of `examples/arena_pipelines`: the dataset registry (00), the physics and time-series benchmark specs (01, 02, :mod:`pinneapple_tools.benchmark_suite`) and the multi-model Arena run from a config (04, 05, :class:`pinneapple_arena.Arena`). Each run writes its figures and tables ...

`pinneapple_flow/nodes_ext/arena_pipelines.py` · 205 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`datasets`	função	`(ctx, inputs, params)`	The dataset ids per category and, for each requested id, the arrays it loads (or the error).
`physics_spec`	função	`(ctx, inputs, params)`	Every model of the spec on the problem (forward or inverse), metrics against the reference, a leaderboard and, for an inverse run, the identified parameters.
`timeseries_spec`	função	`(ctx, inputs, params)`	Every forecasting model of the spec on the same lookback / horizon split, metrics on the test part.

nome	tipo	assinatura	o que faz
arena_run	função	(ctx, inputs, params)	Data from the problem's reference, every model trained and evaluated on the same points; per model the L2 and relative error of each field, the training time and (PINN family) the final physics residual.

pinneapple_flow.nodes_ext.astro

Black-hole accretion nodes (examples/black_hole_weather): one RIAF torus simulation around a Schwarzschild black hole (:func:pinneapple_physics.blackhole.runs.simulate_run) and the time-window statistics the resolution study uses ...

pinneapple_flow/nodes_ext/astro.py · 117 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
simulate	função	(ctx, inputs, params)	Evolve the viscous, radiatively inefficient flow that forms from an equilibrium torus (Paczynski-Wiita potential), saving the primitives every every M (examples/black_hole_weather/simulate.py).
statistics	função	(ctx, inputs, params)	Mean accretion rate, density and specific angular momentum profiles and the persistence error at leads of 20, 100 and 1000 M over the window (the quantities of the resolution study, results/convergence.json).

pinneapple_flow.nodes_ext.autolab_caps

Planner capabilities for the example flows of nodes that live elsewhere: the industrial presets (:mod:pinneapple_flow.nodes.presets), the science experiments (:mod:pinneapple_flow.nodes.science) and the autonomous-lab missions (:mod:pinneapple_flow.nodes.autolab, rover.learn_planet).

pinneapple_flow/nodes_ext/autolab_caps.py · 42 linhas · **sem teste direto**

pinneapple_flow.nodes_ext.benchmarks

Benchmark harnesses as flows: the equal-budget benchmark (examples/benchmark, :func:pinneapple_analysis. benchmark.run) and the benchmark suite of examples/benchmark_suite (:mod:pinneapple_tools.benchmark_suite): the Arena experiment from the example's YAML config (02), and the ...

pinneapple_flow/nodes_ext/benchmarks.py · 239 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
equal_budget	função	(ctx, inputs, params)	Every method on every problem under the same ComputeBudget, the same hyper-parameter search effort and seeds; the result table (mean over seeds at the full budget), the correctness gates and the advisor's diagnostics. The run folder is the node's (config.yaml, prepared/, runs/, table.md, ...
arena_yaml	função	(ctx, inputs, params)	The problem preset, its reference data from the built-in solver, every model of the config trained on the same points; per model the best validation loss, the metrics on the reference points and the time.
transfer	função	(ctx, inputs, params)	Pre-train on a source member of each family, fine-tune on targets with each freezing strategy (and from scratch); relative L2 per scenario, target and strategy.

nome	tipo	assinatura	o que faz
`meta`	função	`(ctx, inputs, params)`	Meta-train on each PDE family, adapt K steps to held-out members against training from scratch; relative L2 per family, algorithm and K.

`pinneapple_flow.nodes_ext.bricks`

Self-modifying interlocking-brick builds (`examples/bricks/self_modifying_bricks`): the closed-form and Newton checks that gate the loop (:func:`pinneapple_design.bricks.validation.validate`) and the two search strategies at the same evaluation budget and seeds …

`pinneapple_flow/nodes_ext/bricks.py` · 83 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`validate`	função	`(ctx, inputs, params)`	Every check's simulated value (`check.<i>`, in the library's order, the check named in the note), whether it passed, and the trust decision.
`search`	função	`(ctx, inputs, params)`	Both strategies on every seed at the same budget: best score per seed, median (a seed with no feasible build counts as the worst), mean, spread, success rate; and the best build found.

`pinneapple_flow.nodes_ext.cfd`

CFD nodes on OpenFOAM: the free-surface trough of `examples/free_surface` (a 2-D section with a liquid bath fed by a jet, interFoam VOF with k-omega SST, :class:`pinneapple_simulation.numerical_solvers.free_surface.FreeSurfaceFlow`) and the run campaign of `examples/openfoam_campaign` …

`pinneapple_flow/nodes_ext/cfd.py` · 210 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`trough_jet`	função	`(ctx, inputs, params)`	examples/free_surface/01_runner_slope_study.py for one slope: time-mean k a few cm above the floor in the impact zone, its peak, the depth under the jet and how settled the liquid volume is.
`ratio`	função	`(ctx, inputs, params)`	`<case>.<metric> / <baseline>.<metric>` for every upstream case (e.g. k relative to the 0 degree floor).
`campaign`	função	`(ctx, inputs, params)`	examples/openfoam_campaign: the base case written by the library, then each run of the plan (jet speed and angle) with its health (Courant, phase bounds, volume) and the sampled profiles.

`pinneapple_flow.nodes_ext.core_ts`

Nodes for `examples/geophysics` (shallow water on the sphere, 4D-Var on Lorenz-96, Richards infiltration), `examples/physics_optimizer` (:class:`pinneapple_core.optim.PhysicsOptimizer` on a constrained shape), `examples/physics_module` (:class:`pinneapple_core.module.Hybrid`: a coarse FEM …

`pinneapple_flow/nodes_ext/core_ts.py` · 270 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`earth_system`	função	`(ctx, inputs, params)`	examples/geophysics/01_sphere_and_assimilation.py: the Rossby-Haurwitz wave (Williamson case 6) and the drift of its conserved quantities, 4D-Var on Lorenz-96 (background and analysis RMSE), Richards infiltration (Celia et al. 1990) and its mass-balance ratio.
`torsion_optimizer`	função	`(ctx, inputs, params)`	Maximise the torsional rigidity of an a x b rectangle with a b = 1 by SLSQP, the adjoint, a penalty, an augmented Lagrangian, differential evolution and Bayesian optimisation (examples/physics_optimizer).
`hybrid_fem`	função	`(ctx, inputs, params)`	-(k(x) u')' = 1 with k = 1 + 0.8 sin(2 pi f x + phase): a 6-cell FEM and a dilated Conv1d corrector trained end to end against a 128-cell FEM; held-out relative error of the coarse FEM alone and of the hybrid (examples/physics_module/01_coarse_fem_plus_neural_correction.py).
`regime_series`	função	`(n=900, seed=0) -> np.ndarray`	The series of examples/time_series/adaptive_forecasting.py: seasonal, then a trend, then a random walk.
`adaptive`	função	`(ctx, inputs, params)`	Experts weighted by their recent errors on a seasonal -> trend -> random-walk series: the ensemble's MAE against every expert, the 90 % interval coverage and when the leading model changes (examples/time_series/adaptive_forecasting.py).
`naive_baselines`	função	`(ctx, inputs, params)`	examples/time_series/00_quickstart.py: a seasonal + trend + noise series, time-ordered train / validation windows, the naive, seasonal-naive and drift baselines and their errors.

`pinneapple_flow.nodes_ext.design`

Design nodes: topology optimization (`examples/topology_optimization`) on :mod:`pinneapple_design.topology` (SIMP compliance of the MBB beam and the cantilever, thermal compliance, Stokes dissipation of the double pipe, the iron of a reluctance rotor for torque, the sensitivity check) and design …

`pinneapple_flow/nodes_ext/design.py` · 321 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`topology2d`	função	`(ctx, inputs, params)`	examples/topology_optimization/01_mbb_beam.py and 02_heat_and_stokes_flow.py.
`topology_gradcheck`	função	`(ctx, inputs, params)`	The adjoint sensitivities of the filtered, projected MBB compliance against central differences on the largest components (examples/topology_optimization/01_mbb_beam.py).
`reluctance_rotor`	função	`(ctx, inputs, params)`	examples/topology_optimization/03_reluctance_rotor.py: 4-pole rotor core, 4-fold symmetry, adjoint sensitivities; the final design re-evaluated with nonlinear iron and with the co-energy torque.
`from_intent`	função	`(ctx, inputs, params)`	A goal in plain language to a verified, buildable design, the questions it needs, or a refusal saying what is missing (examples/design_from_intent/design_from_intent.py).
`ahmed_slant`	função	`(ctx, inputs, params)`	examples/qualitative/03_ahmed_slant.py: the preview's expected drag change per slant against the measured one, its direction and the rank correlation; low confidence near the separation angle.

pinneapple_flow.nodes_ext.electromagnetics

Electromagnetics nodes: the studies of `examples/motor_magnetics` and `examples/electrodynamics` on :mod:`pinneapple_simulation.electromagnetics` (the verification table, the surface-PM machine, the 2004 Prius interior-PM motor: no-load, torque against current angle and MTPA, flux map and drive …

`pinneapple_flow/nodes_ext/electromagnetics.py` · 512 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`em_verification`	função	`(ctx, inputs, params)`	Every electromagnetics solver and reference against an independent result: errors and observed orders (examples/electrodynamics/07_em_verification.py).
`spm_machine`	função	`(ctx, inputs, params)`	examples/motor_magnetics/01_spm_cogging_and_torque.py: 4 poles, 24 slots, nonlinear iron; torque by Maxwell stress (Arkkio) and by virtual work, side by side.
`prius_machine`	função	`(ctx, inputs, params)`	examples/electrodynamics/10_pmsm_prius.py: the 2004 Prius motor (48 slots, 8 poles, V-shaped NdFeB, M400-50A), torque against current angle compared with Yang et al. (IEMDC 2013).
`prius_flux_map`	função	`(ctx, inputs, params)`	psi_d, psi_q and torque on an i_d x i_q grid (and loss data on a coarser grid) of the Prius motor.
`prius_drive`	função	`(ctx, inputs, params)`	examples/electrodynamics/11_prius_drive.py: MTPA + flux weakening envelope, the efficiency map and the battery energy over the WLTC class 3b cycle.
`prius_skew`	função	`(ctx, inputs, params)`	examples/electrodynamics/15_prius_skew.py: cogging, torque and ripple against the step skew, and the skewed back-EMF harmonics against the closed-form skew factor (the 3-D VTU export is left to the script).
`prius_tolerances`	função	`(ctx, inputs, params)`	examples/electrodynamics/17_prius_tolerances.py: torque and ripple spread from the tolerances of the magnet remanence, thickness, width and the air gap; standardised regression coefficients; probability within spec.

pinneapple_flow.nodes_ext.em_antenna

Antenna measurement ranges (`examples/use_cases/antennas/compact_range_qz`): the quiet zone of a compact range by physical optics (:mod:`pinneapple_simulation.electromagnetics.reflector`), its validation against the closed form and geometrical optics, and the feed gain x reflector size x rolled …

`pinneapple_flow/nodes_ext/em_antenna.py` · 67 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`compact_range`	função	`(ctx, inputs, params)`	The generic example (3 GHz, F = 1.5 m, 1.2 m section, 0.6 m quiet zone at 3 m): the reference design's quiet zone (ripple, taper, usable diameter, cross-polar), the validation and the design sweep.

pinneapple_flow.nodes_ext.em_prius

The longer Prius-motor studies of `examples/electrodynamics` (12-14, 16) as nodes on :mod:`pinneapple_simulation.electromagnetics.machines`: rotor design (GP-surrogate NSGA-II against FEA-only), the thermal network over WLTC cycles with demagnetisation margins, the cross-check against gmsh + …

pinneapple_flow/nodes_ext/em_prius.py · 315 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
rotor_design	função	(ctx, inputs, params)	examples/electrodynamics/12_prius_rotor_design.py: hypervolume and FEA calls of the hybrid search against FEA-only, the hybrid front re-evaluated on a finer FEA, and the Prius rotor through the same evaluators.
prius_thermal	função	(ctx, inputs, params)	examples/electrodynamics/13_prius_thermal.py: magnet temperature sweep, a lumped thermal network driven by the WLTC losses (peak node temperatures), the time to 180 C at peak load, and demagnetisation margins per NdFeB grade at the hottest magnet temperature.
getdp_crosscheck	função	(ctx, inputs, params)	examples/electrodynamics/14_prius_getdp_crosscheck.py: torque against current angle, cogging, back-EMF and apparent inductances, PINNeAPPle's FEA against an independent gmsh + GetDP model of the same machine.
end_effects	função	(ctx, inputs, params)	examples/electrodynamics/16_prius_end_effects.py.

pinneapple_flow.nodes_ext.examples

The node that checks a flow against the committed results of the example it reproduces (`examples/<name>`), and the helpers the example-domain nodes share (object artifacts, the checkout's `examples` folder).

pinneapple_flow/nodes_ext/examples.py · 309 linhas · testes: 8

nome	tipo	assinatura	o que faz
Written	classe	(text)	A reference number with the text it was written as (`1.40` keeps its two decimals).
save_object	função	(ctx, name, obj) -> dict[str, Any]	A Python object (flight histories, wind fields, studio scenes) as a pickle artifact of this node.
load_object	função	(v) -> Any	The object of an artifact written by :func:`save_object` (only artifacts a node of this run wrote).
examples_root	função	() -> Path | None	The checkout's `examples` folder (`$PINNEAPPLE_REPO/examples` or next to the packages), None when this is an installed wheel without it.
example_dir	função	(name) -> Path	
doc_table_value	função	(ref) -> float	`doc:<path under docs/>|<row key>|<column header>`: the first number in that cell of a markdown table (the row whose first cell is the key; – and × are read as minus and nothing).
reference_value	função	(folder, ref) -> float	The number `ref` names in an example folder (see the module docstring).
rounding_half_unit	função	(x) -> float	Half a unit of the last digit of `x` as Python writes it (0.00147 -> 5e-6); 0 for a number written with 12 or more significant digits (not rounded).

nome	tipo	assinatura	o que faz
`metric_value`	função	`(reports, name) -> float`	`node.metric` from the reports of the upstream nodes (a metric with its `how` is `{"value": ...}`).
`compare`	função	`(ctx, inputs, params)`	Each comparison: the flow's measured number against the example's committed one, within the tolerance.
`flatten`	função	`(ctx, inputs, params)`	Every number of a nested result as a metric (`a.b.0.c`), so `examples.compare` can check the result of a node that returns its numbers in an output instead of recording them; strings and booleans are kept too.

`pinneapple_flow.nodes_ext.flight`

Flight-dynamics nodes: the studies of `examples/flight_dynamics` (verification, trim and stability, take-off with crosswind and gust cases, turbulence encounters, the Pratt check, the same 1-cosine gust for every aircraft) as nodes on :mod:`pinneapple_simulation.flight`. Each study node takes the …

`pinneapple_flow/nodes_ext/flight.py` · 588 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`aircraft`	função	`(ctx, inputs, params)`	The aircraft models (lattice wing and tail, fin, fuselage, gear, engines, mass properties with sources).
`verification`	função	`(ctx, inputs, params)`	The checks of examples/flight_dynamics/01_verification.py, metric names as in its manifest.
`trim_stability`	função	`(ctx, inputs, params)`	Trim at the cruise point, non-dimensional derivatives of the full model and the eigenvalues of the 12-state linearisation (examples/flight_dynamics/02_trim_and_stability.py).
`takeoff_cases`	função	`(ctx, inputs, params)`	examples/flight_dynamics/03_takeoff.py: ground roll, rotation, lift-off, climb over the 35 ft screen, Raymer's closed form, a crosswind and a down-gust case.
`turbulence_encounter`	função	`(ctx, inputs, params)`	examples/flight_dynamics/04_turbulence_encounters.py: each aircraft at its own speed through the same turbulence; dnz, roll and pitch excursions, cabin accelerations, as the mean over the realisations.
`pratt`	função	`(ctx, inputs, params)`	Simulated peak dn for a 12.5-MAC 1-cosine gust against the Pratt-Walker formula (and unalleviated).
`same_gust`	função	`(ctx, inputs, params)`	examples/flight_dynamics/07_same_gust.py: peak load factor at the CG and in the cabin, pitch and height excursions, the quasi-steady and Pratt estimates for this gust.
`scene`	função	`(ctx, inputs, params)`	Animated 3-D scenes of the kept flights (aircraft, lattice loading, n_z trail, the gust field), as three.js viewers (examples/flight_dynamics/08_viewers.py) and as the input of the Blender render nodes.

`pinneapple_flow.nodes_ext.football`

Football nodes: the ball-flight inverse problem and the player-tracking smoothing check of `examples/sports` (simulated data, no download), and pass-outcome pitch control on a Metrica sample match (downloads it once).

`pinneapple_flow/nodes_ext/football.py` · 76 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`football_ball_flight_inverse`	função	`(ctx, inputs, params)`	Simulated kicks (drag crisis, random spin) observed with noise are fitted back with the drag-Magnus ODE: errors of launch speed and drag coefficient, coverage of the Laplace 95 % intervals, and the Cramer-Rao comparison of 3-D against 2-D tracking (no ball height: launch elevation and spin are not ...
`football_tracking_smoothing`	função	`(ctx, inputs, params)`	Simulated runs with known speed and acceleration, observed with noise: speed and metabolic-power error of raw differences, Savitzky-Golay and the constant-acceleration Kalman-RTS smoother (noise fitted by maximum likelihood), and the coverage of the smoother's 95 % velocity intervals.
`football_pass_pitch_control`	função	`(ctx, inputs, params)`	Spearman-type pitch control (fitted on one Metrica sample match) against logistic regression and boosted trees on geometric features, scored on the passes of the other match (log-loss, Brier, AUC, calibration).

`pinneapple_flow.nodes_ext.forecasting`

Forecasting nodes: the ENSO solver checks of `examples/enso_forecasting` (no data needed) and the `forecast_lab` studies of `examples/near_random_forecasting` (one case: predictability, model zoo, surrogate-trained twins; the overfitting study; the Mackey-Glass Lyapunov convergence). The ENSO ...

`pinneapple_flow/nodes_ext/forecasting.py` · 201 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`enso_solver_checks`	função	`(ctx, inputs, params)`	The intermediate coupled model's shallow-water solver (Kelvin and Rossby wave speeds, convergence order, the Gill steady response against the analytic solution) and the recharge / delayed oscillators against their eigenvalues (examples/enso_forecasting/run_study.py, results/solver_checks.json). No ...
`forecast_case`	função	`(ctx, inputs, params)`	examples/near_random_forecasting/run_study.py for one case: predictability first, then every model under the purged walk-forward protocol with an untouched test period.
`lyapunov`	função	`(ctx, inputs, params)`	Benettin estimate of the largest Lyapunov exponent of Mackey-Glass (tau = 17) against the step, the length of the run and the seed (results/lyapunov_mackey_glass.json).
`overfitting`	função	`(ctx, inputs, params)`	Shuffled k-fold, walk-forward without purge, purged walk-forward and purged k-fold scores of deliberately overfitting and regularised variants, against the untouched test score.

`pinneapple_flow.nodes_ext.geomechanics`

Geomechanics, inspection and component-library nodes: the granular column collapse of `examples/geomechanics` (:class:`pinneapple_simulation.particle_dynamics.geo_mpm.GeoMPM`, MLS-MPM with Drucker-Prager sand, against the runout law of Lube et al. 2005), the eddy-current inspection PINN of ...

`pinneapple_flow/nodes_ext/geomechanics.py` · 169 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`lube_2005`	função	`(a) -> float`	Lube et al. (2005) two-dimensional collapse runout (L_inf - L0) / L0: 1.2 a below a = 1.8, 1.9 a^(2/3) above 2.8, the mean of both in the transition (as examples/geomechanics/01_granular_column_collapse.py).
`column_collapse`	função	`(ctx, inputs, params)`	Columns of dry cohesionless sand against a back wall on a rough floor, released at t = 0; the final runout against the law Lube et al. measured, the final height (examples/geomechanics/01_granular_column_collapse.py).
`eddy_current`	função	`(ctx, inputs, params)`	A synthetic probe scan from the axisymmetric complex-Helmholtz FDM solver, then an InspectionPINN trained on it (examples/inspection/01_eddy_current_pinn.py).
`component_fit`	função	`(ctx, inputs, params)`	examples/systems/01_component_library_toolbox.py: the registered components and toolboxes, one component built and fitted. The example fits random targets (a demonstration of the API, not of the physics); this node seeds them.

`pinneapple_flow.nodes_ext.lbm`

Lattice Boltzmann nodes (`examples/lbm/01_sphere_wind_tunnel.py`): drag on a sphere in the 3-D wind tunnel of :class:`pinneapple_simulation.numerical_solvers.lbm_tunnel.WindTunnelLBM`, against Stokes' law (:func:`~pinneapple_simulation.numerical_solvers.lbm_tunnel.stokes_sphere_field` far field ...

`pinneapple_flow/nodes_ext/lbm.py` · 153 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`sphere_stokes`	função	`(ctx, inputs, params)`	A sphere in a cube whose six sides hold the analytic Stokes far field, started from that field; the momentum-exchange drag against F = 6 pi rho nu R U.
`sphere_drag`	função	`(ctx, inputs, params)`	Inlet at x-, outlet at x+, open sides; Cd from the mean of the last force samples, against Schiller & Naumann (1933); with both storages, the relative Cd difference and the largest velocity difference.

`pinneapple_flow.nodes_ext.learned_sim`

Learned mesh simulators on public data (`examples/meshgraphnet/02_cylinder_flow_deepmind`): MeshGraphNet on a byte prefix of DeepMind's `cylinder_flow` TFRecords (:mod:`pinneapple_data.datasets.deepmind_mgn`), trained and rolled out with ...

`pinneapple_flow/nodes_ext/learned_sim.py` · 112 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`mgn_cylinder_flow`	função	`(ctx, inputs, params)`	Next-step velocity of vortex shedding behind a cylinder on irregular triangle meshes, rolled out autoregressively on the validation trajectories; against the velocity frozen at t = 0.

pinneapple_flow.nodes_ext.non_hermitian

Non-Hermitian physics nodes (`examples/non_hermitian/ep_tuning.py`): the no-go theorem for exceptional-point tuning in PT-symmetric dimers (Zenodo 23287622) checked symbolically (:func:`pinneapple_physics.non_hermitian.analyze_ep_tuning` on ...

`pinneapple_flow/nodes_ext/non_hermitian.py` · 177 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ep_laws`	função	`(ctx, inputs, params)`	Per coupling class: the EP law gamma_EP^2(lam), whether it equals the linear target J^2 + lam J G exactly, its degree in lam, the four assumptions (A PT symmetry, B dimension 2, C polynomial coupling, D reciprocity) and whether the theorem applies.
`ep_numeric`	função	`(ctx, inputs, params)`	gamma_EP^2 found from the Hamiltonian alone (route C: closed form, the matrix is not PT symmetric) against the target J^2 + lam J G, and the route-B feedback fixed point.
`hypothesis_map`	função	`(ctx, inputs, params)`	Three designs audited against the theorem's assumptions: all VALID means linear tuning is out of reach; a VIOLATED assumption names the route that opened up.
`ep_figure`	função	`(ctx, inputs, params)`	The four-panel figure (spectrum across the EP, EP laws by coupling class, coupling plane, chains) and, for uniform chains with |b| = 1, the PT-breaking threshold kappa_N^2 = gamma_EP^2 and the order of the EP there.

pinneapple_flow.nodes_ext.pde_env

The PDE environment examples (`examples/pde_environment`): a `selector_type="tag"` preset trained through `solve_pde` on its analytic real-geometry fixture, with the per-tag loss terms before and after training (`07_tagged_presets_real_geometry`).

`pinneapple_flow/nodes_ext/pde_env.py` · 76 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`tagged_preset`	função	`(ctx, inputs, params)`	Per-tag loss terms of a tagged preset before and after training on its real geometry. The boundary points per tag come from the preset's box / cylinder fixture, a modified MLP is trained on them with `solve_pde`, and every compiled loss term (`pde`, `bc_<tag>`, `total`) is reported before ...

pinneapple_flow.nodes_ext.pinn_examples

The one-call solver examples: `examples/first_example` (a Burgers PINN against the Cole-Hopf solution with `pp.solve`) and `examples/solver_api` (a classical FEM and a PINN on the manufactured Poisson problem with `pp.compare`).

`pinneapple_flow/nodes_ext/pinn_examples.py` · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`burgers_first`	função	`(ctx, inputs, params)`	examples/first_example/first_example.py: u_t + u u_x = nu u_xx on [-1, 1]; the PINN's profile at t = 0.75 (the steep front) against the Cole-Hopf closed form.
`poisson_fem_vs_pinn`	função	`(ctx, inputs, params)`	examples/solver_api/01_classical_vs_pinn.py: laplace(u) = f on the unit square with u = sin(pi x) sin(pi y), one call for both methods against the exact solution.

`pinneapple_flow.nodes_ext.pinn_solver`

The PINN-solver examples (`examples/pinn_solver`): the symbolic loss of `PINNFactory` evaluated on a field (`01_symbolic_loss_basic`). The compiler examples (Poisson 2-D, Burgers 1-D, wave 1-D) and the inverse heat problem are flows of the generic equations / train.pinn / inverse.discover ...

`pinneapple_flow/nodes_ext/pinn_solver.py` · 57 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`factory_loss`	função	`(ctx, inputs, params)`	A PINNFactory loss evaluated for the zero field. The residual strings are compiled by SymPy into a torch loss and evaluated at random collocation points: the total and each component.

`pinneapple_flow.nodes_ext.problemdesign`

Problem design without an LLM (`examples/problem_designer/03_offline_spec_to_report`): a partly known :class:`pinneapple_problemdesign.schema.ProblemSpec`, its validators (warnings and safe assumptions), the FNO-first plan and the Markdown / JSON design report.

`pinneapple_flow/nodes_ext/problemdesign.py` · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`offline_report`	função	`(ctx, inputs, params)`	Validators, plan and report of a partly known spec, as in the example (gaps empty).

`pinneapple_flow.nodes_ext.render`

Render nodes for animated studio scenes: Blender Cycles clips and stills (:func:`pinneapple_tools.visualization.studio.blender.render_animation` / `render_still`) and a labelled grid of clips with one colour bar (:mod:`pinneapple_tools.visualization.studio.compose`). They take the `anim_scene` ...

`pinneapple_flow/nodes_ext/render.py` · 161 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`blender_clip`	função	`(ctx, inputs, params)`	Cycles (CPU, denoised) clip of each scene with the HUD burnt in, encoded to MP4, plus a still.
`clip_grid`	função	`(ctx, inputs, params)`	The clips side by side (e.g. the four aircraft through the same gust) with labels and one colour bar; the still grid uses each clip's marked frame (the gust peak).

pinneapple_flow.nodes_ext.render_worker

Blender rendering in its own process, for :mod:`pinneapple_flow.nodes_ext.render`: `bpy` is never imported by the flow engine or the Flow Studio server (a `bpy` wheel built against another numpy can crash the process that imports it, and Cycles holds the interpreter for minutes).

`pinneapple_flow/nodes_ext/render_worker.py` · 74 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check`	função	`() -> int`	
`render`	função	`(scenes_pkl, options_json, out_dir) -> int`	
`main`	função	`(argv) -> int`	

pinneapple_flow.nodes_ext.security

Cyber-physical security (`examples/security/02_physics_residual_fdi`): physics residuals against false data injection on the three-tank water network, against three data-driven detectors (:func:`pinneapple_security.cps_bench.run_benchmark`).

`pinneapple_flow/nodes_ext/security.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`fdi_benchmark`	função	`(ctx, inputs, params)`	Per detector, regime and attack: episode AUC, detection rate, median delay (minutes, median over the seeds' medians) and localisation, averaged over the seeds as the example script prints them; false alarms per day on clean data. The isolation forest needs scikit-learn (skipped by the library ...

pinneapple_flow.nodes_ext.solver_registry

Nodes over the solver registry (`examples/solvers`): every solver built by name through :class:`pinneapple_simulation.numerical_solvers.SolverRegistry` after :func:`~pinneapple_simulation.numerical_solvers.register_all` -- the time-series decompositions (EEMD, VMD, wavelet, STL, SSA, ...

`pinneapple_flow/nodes_ext/solver_registry.py` · 211 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`decompose`	função	`(ctx, inputs, params)`	A sum of sines plus Gaussian noise decomposed by each registered solver (examples/solvers/ 01_time_series_decompositions.py, with a seeded noise draw). Per solver: the output shape and, for the additive decompositions (EEMD, CEEMDAN, VMD, SSA, STL), how far the sum of the components is from the ...
`sph_box`	função	`(ctx, inputs, params)`	examples/solvers/02_sph_box.py: the trajectory, the final bounding box and the fraction of particles still inside the box (a warning when particles leave it: the run is then not a valid fluid simulation).
`poisson_fdm_spectral`	função	`(ctx, inputs, params)`	examples/solvers/03_poisson_solvers.py: the source f with u = 0 on the boundary (SOR) and with periodic boundaries (FFT), their mean squared difference as the script prints it, and each against the exact solution of its own equation: the registry's FDM solver solves lap(u) = -f (u = f / (2 (k ...

pinneapple_flow.nodes_ext.solvers

Numerical-solver nodes from examples/numerical_solvers: the NVE Lennard-Jones gas (:class:pinneapple_simulation.particle_dynamics.molecular_dynamics.MolecularDynamicsSimulator, velocity Verlet, periodic box) and the shallow-water dam break with a gate and an obstacle ...

pinneapple_flow/nodes_ext/solvers.py · 172 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
lj_gas	função	(ctx, inputs, params)	Particles on a lattice at the LJ well separation with a small random kick; velocity-Verlet steps; total energy against its initial value (examples/numerical_solvers/12_molecular_dynamics_lj_gas.py).
dam_break	função	(ctx, inputs, params)	examples/numerical_solvers/13_shallow_water_twin3d_browser_demo.py without the Twin3D export and its blocking server: the run and its health report (volume conservation, positivity, finiteness).
crate_and_pendulum	função	(ctx, inputs, params)	examples/rigid_body/newton_quickstart.py: where the crate stops against Coulomb friction, v0^2 / (2 mu g).

pinneapple_flow.nodes_ext.structures

Structures nodes: CalculiX (examples/calculix_cantilever) through :mod:pinneapple_simulation.external_solvers.calculix: a deck (the example's committed one or a cantilever written by the library), ccx (native or its Docker image), and the results of an .frd file (the run's or the ...

pinneapple_flow/nodes_ext/structures.py · 146 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
deck	função	(ctx, inputs, params)	The input deck of the cantilever (mesh, material, fixed root, tip load).
solve	função	(ctx, inputs, params)	
results	função	(ctx, inputs, params)	Tip deflection (the most negative u_z at the free end), the largest |sigma_xx|, and the Euler-Bernoulli tip deflection P L^3 / (3 E I) of the same beam.

pinneapple_flow.nodes_ext.surrogate_inverse

Surrogate-driven inverse design (:mod:pinneapple_design.surrogate_inverse, the five design tools of apps/*_ai): sample the design space, label it with a closed form, train an MLP, descend through the frozen surrogate and re-check the final design against the closed form (verification card and ...

pinneapple_flow/nodes_ext/surrogate_inverse.py · 59 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
surrogate_inverse	função	(ctx, inputs, params)	Train the case's surrogate, optimise the design through it, verify it against the closed form.

pinneapple_flow.nodes_ext.use_cases

Use-case nodes (examples/use_cases): the pin-fin inverse problem of fin_convection_inverse -- the convection coefficient h from five noisy thermocouples, by a PINN with h as a trainable parameter (:class:pinneapple_physics.pinn_solver.factory.pinn_factory.PINNFactory) and by a ...

pinneapple_flow/nodes_ext/use_cases.py · 137 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
pin_fin	função	(ctx, inputs, params)	examples/use_cases/fin_convection_inverse/fin_convection_inverse.py: theta'' - m^2 theta = 0, theta(0) = 1, -k theta'(L) = h theta(L); h and the field learned together from the fin equation and the readings, against a least-squares fit of the closed form; the heat the fin dissipates from the ...

pinneapple_flow.nodes_ext.validation

Validation nodes for examples/physics_check, examples/vs_physicsnemo (07, the check replay), examples/model_zoo, examples/proof_graph and examples/validation_arena: the physics linter (:func:pinneapple_check.run) against the rules each fixture must break, the zoo's evaluate / ...

pinneapple_flow/nodes_ext/validation.py · 213 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
physics_check	função	(ctx, inputs, params)	Run every rule on the declared targets (OpenFOAM cases, fields, QoI tables, surrogates, models) and compare the rules each target breaks with the example's list (examples/physics_check, the CI gate's fixtures).
zoo_evaluate	função	(ctx, inputs, params)	examples/model_zoo/quickstart.py: the card's held-out metrics reproduced, the physics checks and validity range, the comparison with XFOIL, and a guarded fine-tune.
pt_chain	função	(ctx, inputs, params)	A graph of assumptions, lemmas and a theorem with exact (sympy) and numerical checks: does the no-go theorem for linear exceptional-point tuning extend to PT tetramers (examples/proof_graph/pt_chain_no_go.py)?
arena	função	(ctx, inputs, params)	The red team looks for wrong outputs that pass every check (escapes) and correct ones that fail (false alarms); the blue team calibrates a combined test and adds checks until every stored escape is caught (examples/validation_arena/run_arena.py).
audit	função	(ctx, inputs, params)	The reference solver itself judged by the shipped defence: it should have no escapes and no false alarms (examples/validation_arena/external_model.py).

pinneapple_flow.nodes_ext.validation_gallery

The validation gallery (examples/validation) as flows: a model of one family on one physics, validated with pp.validate, and a broken variant of it that one named check must catch

(:func:`pinneapple_analysis.validation.gallery.conclude`: report, trust card, SARIF and the failure record).

`pinneapple_flow/nodes_ext/validation_gallery.py` · 197 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`pinn_heat`	função	`(ctx, inputs, params)`	-lap T = q on the unit square, T = 0 on the walls (manufactured source): the PINN as trained and the same PINN trained with boundary weight 0, both through `pp.validate(net, {"pinn": problem})`; `pinn.boundary` must catch the second.
`pod_dmd_wake`	função	`(ctx, inputs, params)`	POD + DMD fitted on the saturated wake (good) and on the growing transient (broken), rolled out from the same frame against the LBM reference; `rollout.invariants` must catch the amplifying ROM.
`opinf_advection`	função	`(ctx, inputs, params)`	Periodic advection-diffusion of a tracer: ROMs with the full basis (good) and cut at `energy_bad` (broken), rolled out against the exact solution with mass conserved; `rollout.spectrum` must catch the smearing.
`zoo_range`	função	`(ctx, inputs, params)`	The zoo model on test designs of its dataset (good) and on two of them moved to a velocity and power outside the card's validity range (broken); `applicability.range` must catch the second.

`pinneapple_flow.nodes_ext.viz`

Any result in 3-D (`examples/studio/01_any_result_in_3d`): a file `pinneapple_data.cae` reads (OpenFOAM case or zip, CalculiX .frd, Abaqus .inp, Gmsh, VTK) or STL / OBJ geometry, as a :class:`pinneapple.viz.Scene`, exported to glTF and USD, an interactive three.js viewer and, when bpy is ...

`pinneapple_flow/nodes_ext/viz.py` · 92 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`any_result_3d`	função	`(ctx, inputs, params)`	examples/studio/01_any_result_in_3d.py: load, export, view, render.

`pinneapple_flow.nodes_ext.worldmodel`

Physics world model (`examples/worldmodel/01_generate_train_evaluate`): trajectory datasets from the built-in scenarios (:func:`pinneapple_worldmodel.generate_datasets`), one specialist FNO world model per scenario (:func:`pinneapple_worldmodel.train_specialist`) and the library's benchmark tasks ...

`pinneapple_flow/nodes_ext/worldmodel.py` · 69 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`specialists`	função	`(ctx, inputs, params)`	Per scenario: parameters of the specialist and, per benchmark task of that scenario, next-step and rollout RMSE, relative L2, the divergence step and the overall score.

pinneapple_flow.palette

Descreve a paleta do Flow Studio: seções, título e resumo de cada nó, dica de instalação e busca que coloca nós curados acima dos `auto.*`.

`pinneapple_flow/palette.py` · 299 linhas · testes: 29

nome	tipo	assinatura	o que faz
`place`	função	`(type_, section) -> None`	Coloca um tipo de nó registrado por outro módulo numa seção da paleta.
`sections`	função	`() -> list[dict[str, Any]]`	As seções da paleta em ordem, com título, ícone e descrição.
`is_auto`	função	`(spec) -> bool`	Diz se o tipo de nó é gerado (`auto.*`).
`section_of`	função	`(spec) -> str`	Seção de um tipo de nó: pela etiqueta `section:`, pelo mapa curado ou pela categoria.
`package_of`	função	`(spec) -> str`	Pacote da biblioteca que um nó envolve, pela primeira chamada declarada.
`humanize`	função	`(name) -> str`	Converte um nome em snake_case num título legível, mantendo siglas em maiúsculas.
`summary`	função	`(text, limit=110) -> str`	Primeira frase de uma docstring, sem marcação reST, cortada num limite de caracteres.
`install_hint`	função	`(reason) -> str`	O que fazer com um nó indisponível, a partir da razão.
`entry`	função	`(spec, avail=None) -> dict[str, Any]`	Campos de paleta de um tipo de nó (seção, título, resumo, etiquetas, pacote, curado, dica de instalação).
`score`	função	`(e, query) -> float`	Relevância de uma entrada do catálogo para uma busca; título pesa mais e nós curados ficam acima dos gerados.
`search`	função	`(query, catalog=None, limit=50) -> list[dict[str, Any]]`	Entradas do catálogo que casam com a busca, melhores primeiro.
`layout`	função	`(catalog=None) -> list[dict[str, Any]]`	A paleta inteira como seções em ordem com seus tipos de nó.

pinneapple_flow.planner

Ponto de entrada `plan`: escolhe o planejador heurístico, o LLM ou `auto` (LLM se houver provedor configurado) e volta para a heurística quando o LLM falha, registrando o motivo.

`pinneapple_flow/planner/__init__.py` · 57 linhas · testes: 16

nome	tipo	assinatura	o que faz
`plan`	função	`(text, *, backend='auto', provider='', model='', base_url='', api_key='', n=3, call=None)…)`	Gera propostas de fluxo para uma descrição com o planejador escolhido, caindo para a heurística quando o LLM falha.

pinneapple_flow.planner.build

`FlowBuilder` para montar fluxos em código (nós, arestas, decisões) e `layout`, que posiciona os nós da esquerda para a direita.

`pinneapple_flow/planner/build.py` · 66 linhas · testes: 5

nome	tipo	assinatura	o que faz
`FlowBuilder`	classe	`(name, description='')`	Construtor de fluxos em código: adiciona nós, liga portas, registra decisões e devolve o fluxo ou uma proposta.
`layout`	função	`(nodes, edges, dx=270.0, dy=150.0)`	Posiciona os nós em colunas pelo caminho mais longo desde uma fonte e em linhas pela ordem de aparição.

`pinneapple_flow.planner.extensions`

Gancho público de extensão do planejador: outros pacotes declaram `Capability` (palavras-chave, nós, modelo de fluxo) e o planejador heurístico as propõe para descrições que casam, com uma decisão por escolha.

`pinneapple_flow/planner/extensions.py` · 168 linhas · testes: 5

nome	tipo	assinatura	o que faz
`Capability`	classe	`(id, title, domain, keywords, nodes, description, inputs, downstream, references, uncited…)`	Capacidade declarada para o planejador: id, título, domínio, palavras-chave com pesos, nós, referências, modelo de fluxo e pontuação mínima.
`register_capability`	função	`(cap) -> Capability`	Adiciona ou substitui (mesmo id) uma capacidade e sua regra de decisão.
`capabilities`	função	`() -> list[Capability]`	Todas as capacidades registradas, em ordem de id.
`score`	função	`(cap, text) -> tuple[float, list[str]]`	Pontuação de uma capacidade para um texto (soma dos pesos das palavras-chave encontradas), zero se faltar um grupo obrigatório.
`match`	função	`(text) -> list[tuple[float, Capability, list[str]]]`	Capacidades cuja pontuação atinge `min_score`, melhores primeiro.
`template_doc`	função	`(name, quick=False) -> dict[str, Any]`	Lê o fluxo de exemplo usado como modelo, opcionalmente na configuração rápida.
`proposal`	função	`(cap, text, det, hits, s) -> dict[str, Any]`	Monta a proposta de fluxo de uma capacidade com as decisões que explicam a escolha.
`proposals`	função	`(text, det=None, *, limit=2) -> list[dict[str, Any]]`	Fluxos das capacidades que melhor casam com o texto (no máximo `limit`).
`describe`	função	`() -> list[dict[str, Any]]`	Todas as capacidades em JSON, para o assistente da interface e o prompt do planejador LLM.

`pinneapple_flow.planner.heuristic`

Planejador offline e determinístico: palavras-chave em inglês e português, grandezas com unidades e valores de coeficientes detectam a física e o pedido, e cada proposta é um fluxo do catálogo.

`pinneapple_flow/planner/heuristic.py` · 573 linhas · testes: 11

nome	tipo	assinatura	o que faz
`detect`	função	`(text) -> dict[str, Any]`	Tudo o que as heurísticas leem no texto (família de EDP, dimensão, regime, condições, pedidos), mostrado ao usuário.

nome	tipo	assinatura	o que faz
`family_length_ok`	função	`(family) -> bool`	Diz se a família aceita o comprimento do domínio dado (falso para `burgers`).
`plan_heuristic`	função	`(text, *, max_proposals=3) -> dict[str, Any]`	Plano heurístico completo: capacidades registradas primeiro, depois fluxos montados conforme a física e os pedidos detectados.
`sensor_count`	função	`(text) -> int | None`	Lê no texto o número de sensores (1 a 64), em inglês ou português, ou `None`.
`sensor_layout`	função	`(n, dim) -> list`	Posições de `n` sensores espalhadas no intervalo ou quadrado unitário.

`pinneapple_flow.planner.llm`

Planejador por LLM via `pinneapple_llm.call_llm` (Ollama, endpoint compatível com OpenAI, OpenAI ou Anthropic): catálogo e esquema vão no prompt, a resposta JSON é validada e reparada. [sem teste direto; os testes usam `plan` com `call` falso]

`pinneapple_flow/planner/llm.py` · 228 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`catalog_prompt`	função	`() -> str`	Texto do catálogo de nós (portas, parâmetros e descrição) para o prompt.
`build_prompt`	função	`(text, n=2) -> str`	Prompt completo com o catálogo, o esquema e um exemplo de resposta JSON.
`repair`	função	`(flow_doc) -> tuple[dict[str, Any], list[str]]`	Reconstrói um fluxo válido a partir da saída do LLM e lista os reparos feitos.
`plan_llm`	função	`(text, *, provider, model='', base_url='', api_key='', n=2, call=None) -> dict[str, Any]`	Propostas de um LLM, cada uma validada e reparada; levanta erro em falha do provedor ou saída inútil.

`pinneapple_flow.planner.wizard`

Assistente guiado de "New flow" (problema, física, geometria/dados, métodos, validação, saídas) que transforma respostas numa descrição para o planejador heurístico.

`pinneapple_flow/planner/wizard.py` · 89 linhas · testes: 5

nome	tipo	assinatura	o que faz
`describe`	função	`(answers) -> str`	A descrição em texto que o planejador lê para as respostas do assistente.
`plan_from_answers`	função	`(answers, *, max_proposals=3) -> dict[str, Any]`	Resultado do planejador para as respostas do assistente, com a capacidade escolhida em primeiro lugar.

`pinneapple_flow.references`

Carrega o registro curado `references.yaml`, aceita referências de outros pacotes, verifica DOIs online (doi.org e Crossref) e formata citações.

`pinneapple_flow/references.py` · 216 linhas · testes: 13

nome	tipo	assinatura	o que faz
`registry`	função	`() -> dict[str, dict[str, Any]]`	Lê `references.yaml` mais as referências registradas por outros pacotes.
`register_references`	função	`(entries) -> list[str]`	Acrescenta referências curadas (dict ou YAML) checando o formato; recusa chave existente com conteúdo diferente.
`ref`	função	`(key) -> dict[str, Any]`	Uma referência do registro pela chave; `KeyError` se não existir.
`link`	função	`(r) -> str`	Link clicável de uma referência: DOI, senão URL, senão vazio.
`problems`	função	`(entry) -> list[str]`	Lista o que está errado numa entrada do registro (vazia se bem formada).
`verify_doi`	função	`(doi, title='') -> dict[str, Any]`	Resolve um DOI online (doi.org e Crossref) e devolve `verified`, `not found` ou `unverified` (sem rede).
`verify_registry`	função	`() -> dict[str, dict[str, Any]]`	Verifica online todas as entradas do registro que têm DOI.
`check`	função	`(reference, *, online=None) -> dict[str, Any]`	Classifica uma referência citada como `registry`, `verified`, `not found` ou `unverified`.
`cite`	função	`(*keys, supports='') -> list[dict[str, Any]]`	Referências do registro para as chaves dadas.
`format_reference`	função	`(r) -> str`	Formata uma referência como texto (autores, ano, título, veículo, link, estado).

`pinneapple_flow.registry`

Define `Port`, `Param`, `NodeSpec` e `Registry` e o decorador `node`, que registra `fn(ctx, inputs, params)` como tipo de nó.

`pinneapple_flow/registry.py` · 256 linhas · testes: 46

nome	tipo	assinatura	o que faz
`Port`	classe	`(name, type, required, multiple, help)`	Porta de entrada ou saída de um nó: nome, tipo, se é obrigatória ou múltipla e ajuda.
`Param`	classe	`(name, kind, default, help, choices, minimum, maximum, max_length, advanced, label)`	Parâmetro de um nó com tipo, padrão, escolhas e limites; gera seu JSON Schema e checa valores.
`NodeSpec`	classe	`(type, title, category, fn, inputs, outputs, params, description, calls, requires, availa…)`	Especificação de um tipo de nó: função, portas, parâmetros, descrição, chamadas de biblioteca, requisitos e disponibilidade.
`Registry`	classe	`()`	Catálogo de tipos de nó: registrar, registrar nós do usuário, buscar e listar o catálogo.
`palette_entry`	função	`(spec, avail) -> dict[str, Any]`	Atalho para `palette.entry` de um `NodeSpec`.
`node`	função	`(type_, title, *, category, inputs=(), outputs=(), params=(), description='', calls='', r…)`	Decorador que registra `fn(ctx, inputs, params) -> {porta: valor}` como tipo de nó.

`pinneapple_flow.rules`

The heuristic rules behind every automatic decision of a flow, with the references they rest on.

`pinneapple_flow/rules.py` · 208 linhas · testes: 13

nome	tipo	assinatura	o que faz
`Rule`	classe	`(id, description, cites, uncited, evidence, topic)`	
`register_rule`	função	`(r) -> Rule`	Add a heuristic rule (another package's decisions). It must cite registered references (:func:`pinneapple_flow.references.register_references` first) or say why it has none; an existing id is refused unless the rule is the same.
`rule`	função	`(rule_id) -> Rule`	

`pinneapple_flow.runtime`

Converte cada nó num `Step` do motor de workflows (ordem, logs, retentativas, cancelamento e reaproveitamento de saídas inalteradas); valores grandes viram artefatos em arquivo e o fim de cada execução passa por `on_run_finished`.

`pinneapple_flow/runtime.py` · 523 linhas · testes: 9

nome	tipo	assinatura	o que faz
`NotRun`	classe	`(reason)`	Exceção levantada por um nó que não pode rodar aqui: ele termina `not_run` e os nós que dependem dele também.
`NodeContext`	classe	`(node_id, node_type, workdir, log, should_stop=lambda: False, meta=None)`	O que a função de um nó recebe além de entradas e parâmetros: pasta, log e ajudantes para métricas, tabelas, figuras, decisões e artefatos.
`FlowNodeAdapter`	classe	`()`	Adaptador do motor que roda um nó no processo: resolve entradas a partir das saídas anteriores e chama o tipo registrado.
`FlowRunResult`	classe	`(run_id, status, nodes, workdir, seconds, hooks)`	Resultado de uma execução: id, estado, nós, pasta, segundos e o que cada gancho fez; dá acesso a saídas e relatórios.
`default_workdir`	função	`() -> Path`	Pasta de trabalho padrão (`$PINNEAPPLE_FLOW_HOME/work` ou `~/.pinneapple_flow/work`).
`artifact`	função	`(path, kind) -> dict[str, Any]`	Cria o valor `{"$artifact": caminho, "kind": tipo}` que referencia um arquivo.
`is_artifact`	função	`(v) -> bool`	Diz se um valor é uma referência de artefato.
`load_artifact`	função	`(v)`	Lê o conteúdo de um artefato: npz como dict de arrays, json como objeto, pt como objeto torch, png como caminho.
`to_workflow`	função	`(flow, *, nodes=None, force=frozenset(), token='', max_parallel=1) -> Workflow`	Converte o fluxo no `Workflow` do motor; nós em `force` recebem um token novo para rodar de novo.
`node_states`	função	`(runner, run_id, *, with_logs=False) -> dict[str, dict[str, Any]]`	Estado, erro e saídas de cada nó de uma execução, marcando `not_run` e `degraded`.
`register_custom_nodes`	função	`(docs) -> list[str]`	Registra tipos de nó do usuário validados por `custom_spec`.

nome	tipo	assinatura	o que faz
`run`	função	`(flow, *, workdir=None, from_node=None, only=None, run_id=None, cache=True, max_parallel=…)`	Roda um fluxo sem interface e devolve um `FlowRunResult`, com cache, re-execução a partir de um nó e paralelismo opcional.
`on_run_finished`	função	`(result, flow) -> None`	Ponto único por onde passa toda execução terminada; chama a exportação para o Lab e os ganchos registrados sem nunca falhar a execução.
`lab_export_status`	função	`() -> str`	Diz se `pinneapple_lab.manifest.from_flow_run` está disponível.
`export_to_lab`	função	`(result, flow) -> str`	Grava o manifesto de experimento da execução (`experiment.json`) com o `from_flow_run` do Lab.

`pinneapple_flow.schema`

O documento de fluxo `pinneapple-flow/1` com `Flow`, `Node`, `Edge` e `Issue`; `validate_flow` acusa tipos e parâmetros desconhecidos, portas incompatíveis, entradas faltando e ciclos.

`pinneapple_flow/schema.py` · 307 linhas · testes: 27

nome	tipo	assinatura	o que faz
`FlowError`	classe	`(message, issues=None)`	Exceção para documento de fluxo malformado ou inválido, com a lista de problemas.
`Node`	classe	`(id, type, params, position, label)`	Um nó do fluxo: id, tipo, parâmetros, posição e rótulo.
`Edge`	classe	`(source, source_port, target, target_port, id)`	Uma aresta do fluxo: nó e porta de origem, nó e porta de destino.
`Flow`	classe	`(name, nodes, edges, id, description, meta, schema)`	O documento de fluxo com nós, arestas e metadados, com ordem topológica, ancestrais e leitura/gravação em JSON.
`Issue`	classe	`(level, message, node, edge, param)`	Um problema de validação com nível, mensagem e o nó, aresta ou parâmetro envolvido.
`as_flow`	função	`(flow) -> Flow`	Obtém um `Flow` a partir de um Flow, dict, string JSON ou caminho de arquivo.
`validate_flow`	função	`(flow, registry=None) -> list[Issue]`	Lista todos os problemas de um fluxo; lista vazia significa válido.
`check_flow`	função	`(flow, registry=None) -> Flow`	Devolve o fluxo ou levanta `FlowError` com os erros (avisos passam).
`flow_json_schema`	função	`() -> dict[str, Any]`	JSON Schema (draft 2020-12) do documento de fluxo.

`pinneapple_flow.service`

The operations Flow Studio (or any other front end, e.g. the PINNeAPPle-CFD app) exposes, as plain methods.

`pinneapple_flow/service.py` · 352 linhas · testes: 29

nome	tipo	assinatura	o que faz
`FlowService`	class	`(*, db_url=None, workdir=None, settings=None, max_runs=1)`	

pinneapple_flow.store

Where flows, their versions, runs and per-node outputs live: SQLite by default, any SQLAlchemy URL otherwise.

`pinneapple_flow/store.py` · 261 linhas · testes: 23

nome	tipo	assinatura	o que faz
`FlowStore`	classe	`(url=None, *, table_prefix=None)`	`table_prefix`: put the tables under a prefix (`flow_` -> `flow_flows`, `flow_runs`, ...) when they share an application's database; default `$PINNEAPPLE_FLOW_TABLE_PREFIX` or none.
`default_url`	função	`() -> str`	
`redact`	função	`(url) -> str`	The URL with any password replaced (for logs and the GUI).

pinneapple_flow.studio

Serve o app `apps/flow_studio` com uvicorn; recusa ouvir fora de localhost sem `FLOW_USER` e hash de senha, salvo `allow_insecure`.

`pinneapple_flow/studio.py` · 64 linhas · testes: 38

nome	tipo	assinatura	o que faz
`app_dir`	função	`() -> Path`	Pasta do app Flow Studio (`$PINNEAPPLE_FLOW_APP` ou `apps/flow_studio` do checkout).
`load_app`	função	`()`	Carrega o app FastAPI de `apps/flow_studio/backend/app.py` (precisa do checkout do repositório).
`serve`	função	`(host=None, port=None, *, open_browser=True, env_file='.env', allow_insecure=False) -> int`	Sobe o Flow Studio com uvicorn e abre o navegador; recusa endereço não local sem login.

pinneapple_flow.templates

Equation templates per PDE family and dimension: the PDEs, conditions, coefficients and (when one is known) the exact solution a planner proposes and a user then edits.

`pinneapple_flow/templates.py` · 114 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`template`	função	`(family, dimension=1) -> dict[str, Any]`	PDEs, conditions, coefficients, domain and exact solution for `family` in `dimension` (1-3) spatial dimensions. ValueError for an unknown family or a dimension the family does not have here.
`retarget`	função	`(equations, dimension) -> dict[str, Any]`	The equations of a template family moved to another dimension (the 2-D <-> 3-D switch). PDEs written with `lap()` keep their text; conditions on `boundary` stay; a known template is rebuilt for the new dimension.

pinneapple_flow.twin

`PDETwinModel` transforma o sistema identificado num modelo de gêmeo (campo em grade grossa, coeficientes como estados de passeio aleatório, um estado por sensor); `simulate_sensors` e `run_twin` simulam e as-

similam leituras. Só primeira ordem no tempo.

`pinneapple_flow/twin.py` · 281 linhas · testes: 7

nome	tipo	assinatura	o que faz
`PDETwinModel`	classe	`(system, bounds, *, n=21, dt=0.01, estimate=None, coefficients=None)`	O sistema identificado como modelo de gêmeo: estados do campo em grade grossa, coeficientes estimados e sensores interpolados.
`simulate_sensors`	função	`(plant, *, sensors, t_end, every=1, noise=0.0, coefficients=None, fault=None, seed=0) -> …)`	Leituras simuladas de um ativo (em geral uma grade mais fina), com ruído gaussiano e falha opcional num instante.
`run_twin`	função	`(model, readings, *, sensors, sigma_obs, sigma_process=0.0001, sigma_param=None, limits=N…)`	Assimila leituras no modelo com o `TwinRuntime` e devolve o runtime e um resumo (estimativas, alertas, coeficientes calibrados).

`pinneapple_flow.types`

Port types of the flow graph and the rule that says which output may feed which input.

`pinneapple_flow/types.py` · 71 linhas · testes: 7

nome	tipo	assinatura	o que faz
`compatible`	função	`(source, target) -> bool`	True when an output of type `source` may feed an input of type `target`.
`register_port_type`	função	`(name, description, *, converts_to=(), converts_from=()) -> None`	Add a port type (for another package's node types, e.g. a CFD case). `converts_to` / `converts_from`: existing types it may feed / be fed from although the names differ. Re-registering the same description is a no-op; a different one for an existing name is refused, so documents stay portable.

`pinneapple_flow.wrappers`

Funções com argumentos JSON que montam os objetos com os construtores da própria biblioteca e chamam a biblioteca; nada aqui calcula física. [sem teste direto]

`pinneapple_flow/wrappers.py` · 292 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`solve_preset`	função	`(preset='burgers_1d', method='analytic', architecture='modified_mlp', epochs=50, hidden_d…)`	`pp.solve` num preset da biblioteca; para `pinn`, a arquitetura é qualquer variante PINN do registro de modelos.
`compare_preset`	função	`(preset='burgers_1d', methods=('analytic', 'pinn'), reference='analytic', architecture='m…)`	`pp.compare` de métodos num preset contra uma referência nos mesmos pontos.
`lbm_channel`	função	`(nx=64, ny=32, Re=100.0, u_in=0.05, steps=200, save_every=50) -> dict`	Escoamento em canal por Lattice Boltzmann via `SolverRegistry.build('lbm')`.

nome	tipo	assinatura	o que faz
beam_deflection	função	(L_m=1.0, E_Pa=200000000000.0, I_m4=8.33e-06, q_N_per_m=1000.0, nx=200, bc_type=0) -> dict	Deflexão de viga de Euler-Bernoulli por diferenças finitas (beam_bvp_fdm).
xtfc_pde	função	(pde='burgers_1d', n_basis=60, n_query=40) -> dict	Resolve uma EDP com o solver X-TFC (xtfc_pde).
wheel_on_soil	função	(soil='GRC-1', radius_m=0.25, width_m=0.2, load_N=100.0, slip=0.2, gravity=1.62) -> dict	Modelo roda-solo de Wong-Reece em regime permanente (pinneapple_simulation.terramechanics).
meshless_compare	função	(kind='poisson', exact='sin(pi*x)*sin(pi*y)', domain='square', methods=…, k=4.0, alpha=1.…)	Compara métodos sem malha e com malha num problema (meshless_study), com checagem das expressões pela biblioteca.
rom_compare	função	(Y=None, A=None, methods=(), ranks=(), budget_s=5.0, seed=0) -> dict	Compara modelos de ordem reduzida em snapshots paramétricos (rom_study).
parametric_pod	função	(A=None, Y=None, r=16, regressor='rbf', test_fraction=0.2, seed=0) -> dict	Ajusta ParametricPOD a um dataset e mede o L2 relativo em amostras separadas.
surrogate_study	função	(X=None, Y=None, example='none', methods=('ridge', 'gp', 'gbt'), budget=10.0, seeds=1) ->…)	Compara surrogates clássicos (ridge, GP, árvores, rede) com pinneapple_analysis.surrogate.run_study.
calculix_cantilever	função	(length_m=1.0, b_m=0.1, h_m=0.1, nx=20, element_type='C3D20R', run=True) -> dict	Escreve o deck de uma viga em balanço com a biblioteca e roda o CalculiX se ccx (ou sua imagem Docker) existir; senão só escreve o deck.
fmu_simulate	função	(file='', stop_time=1.0, step_size=0.01) -> dict	Simula um FMU da pasta de dados (precisa de fmpy).
newton_validate	função	(solver='xpbd', cases=…) -> dict	Valida solvers de corpo rígido contra formas fechadas (precisa do NVIDIA Newton).
openfoam_available	função	() -> dict	Diz se o OpenFOAM está instalado (blockMesh no PATH).
model_architecture	função	(name='fno', in_dim=1, out_dim=1, hidden=64) -> dict	Constrói qualquer modelo do registro com argumentos padrão e tamanhos dados e conta seus parâmetros.
doe_design	função	(parameters=None, n=16, method='lhs', seed=0, out_dir='') -> dict	Plano de experimentos com pinneapple_data.doe.design sobre um espaço de parâmetros, gravado como CSV.
geometry_export	função	(name='caarc_building', fmt='stl', lod='light', out_dir='') -> dict	Constrói uma geometria paramétrica de pinneapple_design.geometry_library e a exporta (STL, OBJ, glTF, USD ou STEP).
design_intent	função	(text=…, seed=0, budget=200, n_screen=500, out_dir='') -> dict	pinneapple_design.intent.design: de um objetivo em linguagem natural a um projeto verificado, às perguntas que faltam ou à recusa.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 31 de 89.

- Funções de teste em arquivos que importam algum módulo do pacote: 263.

Limitações conhecidas

- *Nenhuma registrada.*

4. `pinneapple_hub`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
31	7.260	22	170	183 (18 arquivos)	8

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`hub`		1	2	0
`library`		10	64	22
`model_card`		1	1	21
`verified_card`		1	4	6
`zoo`		17	121	125

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_hub`

Model hub client: `push_to_hub`/`from_pretrained` + `ModelCard`.

`pinneapple_hub/__init__.py` · 12 linhas · testes: 122

`pinneapple_hub.hub`

`push_to_hub` / `from_pretrained` for PINNeAPPle models, built on the Hugging Face Hub's own storage/auth/versioning infrastructure rather than a new hosting service this repository would have to build, operate, and secure itself. This is a deliberate, common pattern (many domain-specific …

`pinneapple_hub/hub.py` · 151 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`push_to_hub`	função	`(model, repo_id, model_card, *, architecture=None, architecture_config=None, private=Fals…)`	Upload a trained model's weights + architecture config + model card to a Hugging Face Hub model repo.
`from_pretrained`	função	`(repo_id, *, revision='main', device='cpu', token=None)`	Download and reconstruct a model pushed with :func:`push_to_hub`.

pinneapple_hub.library

The PINNeAPPle Hub index: every ready-made thing in the repository -- tools (public functions, Flow Studio nodes, recipes, MCP tools), trained models, geometries, datasets, flows and examples -- as one generated list, each item with a runnable snippet whose recorded run is checked by a test.

`pinneapple_hub/library/__init__.py` · 303 linhas · testes: 22

nome	tipo	assinatura	o que faz
`collect`	função	`(only=None) -> list[dict[str, Any]]`	Items from every source (:data:`.sources.COLLECTORS`), ids unique, sorted by kind then title.
`run_file`	função	`(item_id, runs=RUNS) -> Path`	The file holding the recorded run of one item: `snippet_runs/<id with unsafe characters as _>.json`.
`read_runs`	função	`(path=RUNS) -> dict[str, Any]`	Every recorded run, by item id (each file under `snippet_runs/` carries its item's `id`).
`write_run`	função	`(item_id, rec, runs=RUNS) -> Path`	
`attach_runs`	função	`(items, runs) -> None`	Set `run` on each item: the recorded run of its current snippet (same code hash), else the module reference's own run of a recipe, else `{"status": "not run"}`.
`facets`	função	`(items) -> dict[str, Any]`	
`quality_key`	função	`(it) -> str`	The filter value of an item's quality: a Lab letter, a zoo decision, or "not graded".
`featured`	função	`(items, n=8) -> list[str]`	Highlights, by a fixed rule: approved zoo models, A-sealed examples with figures, geometries with a thumbnail, then flows; each with a passing recorded run.
`build`	função	`(only=None, runs=None) -> dict[str, Any]`	The whole index (not written; see :func:`write_index`).
`write_index`	função	`(index, path=INDEX) -> Path`	
`load_index`	função	`(path=INDEX, *, rebuild=False) -> dict[str, Any]`	The index at `path`. The default index is a cache: built from the sources (and written) when it is missing or `rebuild` is set; rebuild it after changing a source with `python -m pinneapple_hub.library build`.
`get`	função	`(index, item_id) -> dict[str, Any]`	
`score`	função	`(it, query) -> float`	Relevance of an item to a query; 0 = no match. Every query word must match: title weighs most, then name and tags, then summary, domain and description.
`search`	função	`(index, query='', *, kind=None, domain=None, quality=None, subkind=None, category=None, l…)`	Items matching `query` and the filters (`quality`: a letter, a comma list of letters, a zoo decision in lower case, or "not graded"), best first; without a query, in index order.

nome	tipo	assinatura	o que faz
record_runs	função	(items, *, stale_only=False, timeout=300.0, ids=None, workers=4, log=print) -> dict[str, …)	Run the snippets (all, the ones not recorded for their current code, or ids) in workers parallel interpreters, save their figures under data/web/hub/figures and write one snippet_runs/<id>.json each. Recipes the module reference already ran are reused.
check	função	(index) -> list[str]	Problems of an index: a snippet without a passing run of its current code, a file or link path that does not exist in the repository, an example id that is not an item.

pinneapple_hub.library.__main__

python -m pinneapple_hub.library build | run [--stale] [ids ...] | check (see the package docstring).

pinneapple_hub/library/__main__.py · 47 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
main	função	(argv=None) -> int	

pinneapple_hub.library.categories

The 20 task categories of the Hub's tools. Every tool gets exactly one, by the longest rule prefix that matches its key; the key names where the tool comes from (node:<flow node type>, lib:<module>:<function> for a generated library call, fn:<dotted name>, mcp:<agent tool>). Recipes …

pinneapple_hub/library/categories.py · 174 linhas · testes: 14

nome	tipo	assinatura	o que faz
key	função	(it) -> str	Where a tool comes from, as the prefix rules read it.
category_of	função	(it) -> str | None	The slug of a tool's category, or None when no rule matches (a test forbids that).
table	função	() -> list[dict[str, str]]	

pinneapple_hub.library.compose

"Create your own": a custom tool composed from existing blocks (Flow Studio node types, optionally a bundled flow to start from and a library geometry), returned as a flow document and the Python that runs it.

pinneapple_hub/library/compose.py · 191 linhas · testes: 22

nome	tipo	assinatura	o que faz
ComposeError	classe	()	
blocks	função	() -> list[dict[str, Any]]	The blocks the composer offers: every node type of Flow Studio's palette (curated nodes first, then the generated library calls), by palette section, with its ports and parameters.

nome	tipo	assinatura	o que faz
compose	função	(spec) -> dict[str, Any]	See the module doc. spec: name, description, blocks ([{type, params}]), optional start (a bundled flow name to append the blocks to) and geometry (a geometry library entry the code loads).
code_for	função	(doc, *, geometry='', workdir='hub_tool', expose=None) -> str	Python that rebuilds the flow, validates it, runs it headless and prints what each node measured; exposed parameters are variables set before the run.

pinneapple_hub.library.flowlink

The Hub and Flow Studio on one registry: a ready flow around any node type, and links between the two apps.

pinneapple_hub/library/flowlink.py · 173 linhas · testes: 8

nome	tipo	assinatura	o que faz
FlowLinkError	classe	()	
bundled_flows_using	função	(type_) -> list[str]	Names of the bundled flows that contain a node of type_.
flow_for_node	função	(type_, params=None, *, report=True) -> dict[str, Any]	See the module doc. Raises :class:FlowLinkError for an unknown type; parameter problems and unwired inputs come back in issues.
b64url	função	(obj) -> str	
from_b64url	função	(text) -> Any	
flow_link	função	(flow, *, base) -> str	Flow Studio URL that opens flow (?flow=).
add_link	função	(type_, params=None, *, base) -> str	Flow Studio URL that adds a node of type_ (with params) to a flow the user picks (?add=).

pinneapple_hub.library.flowruns

Recorded runs of the bundled flows (pinneapple_flow/flows), shown in the Hub on each flow's page and on the page of every node type the flow uses.

pinneapple_hub/library/flowruns.py · 109 linhas · testes: 8

nome	tipo	assinatura	o que faz
flow_hash	função	(name) -> str	
read_flow_runs	função	(path=FLOW_RUNS) -> dict[str, Any]	
current	função	(runs, name) -> dict[str, Any] | None	The recorded run of name if it was recorded for the flow file as it is now.
record	função	(names=None, *, timeout=900.0, workers=2, log=print) -> dict[str, Any]	Run the bundled flows (all, or names) and write flow_runs.json.

nome	tipo	assinatura	o que faz
`runs_for_node`	função	`(runs, type_) -> list[dict[str, Any]]`	Every node of `type_` in a current recorded flow run: flow, node id, status, metrics, summary.

`pinneapple_hub.library.myhub`

"My Hub": the tools a person composed and saved, as Hub items in a local folder (one JSON file each).

`pinneapple_hub/library/myhub.py` · 87 linhas · testes: 14

nome	tipo	assinatura	o que faz
`MyHubError`	classe	`()`	
`MyHub`	classe	`(root)`	

`pinneapple_hub.library.sandbox`

Run a code snippet in a short-lived, separate interpreter and keep what it printed and drew.

`pinneapple_hub/library/sandbox.py` · 104 linhas · testes: 22

nome	tipo	assinatura	o que faz
`code_hash`	função	`(code) -> str`	Short SHA-256 of a snippet: a recorded run is current only while its snippet has this hash.
`run_snippet`	função	`(code, *, timeout=60.0, memory_mb=None, cpu_s=None, max_figures=4, env=None) -> dict[str,…)`	Run `code` (see the module doc). Returns `ok`, `stdout`, `stderr` (last lines), `seconds`, `timed_out`, `code_hash` and `figures` (a list of PNG `bytes`, at most `max_figures`).

`pinneapple_hub.library.snippets`

The minimal, runnable code of each Hub item. Every snippet built here is run by `python -m pinneapple_hub.library run` (output recorded in `data/web/hub/snippet_runs/`) and again by `tests/test_hub_library.py`; keep them short, offline and fast.

`pinneapple_hub/library/snippets.py` · 121 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`model`	função	`(name, example) -> str`	
`dataset`	função	`(name, split) -> str`	
`geometry`	função	`(name) -> str`	
`flow`	função	`(name) -> str`	
`node`	função	`(type_) -> str`	
`auto_call`	função	`(target, node_type, defaults=None) -> str`	A generated library call that runs with its defaults (the node's parameter form): call it with them and show what it returns.
`api`	função	`(dotted) -> str`	
`mcp`	função	`(name, *, call) -> str`	

nome	tipo	assinatura	o que faz
example	função	(folder) -> str	
lab_item	função	(key) -> str	
symbols	função	(code) -> list[str]	pp.xxx names a snippet calls (for tags and links).

pinneapple_hub.library.sources

Where the Hub's items come from. Each collector reads one existing source of the repository and returns items in the Hub format (see :mod:pinneapple_hub.library); nothing here is written by hand.

pinneapple_hub/library/sources.py · 579 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
rel	função	(p) -> str	
domain	função	(name, tags=()) -> str	The physical domain, by the Lab's own rule (pinneapple_lab.manifest.domain_of): on the first word of name (the item's own name) when that decides it, else on the whole text and the tags.
used_by_examples	função	(*needles) -> list[str]	Example folders whose scripts contain one of needles as a quoted string or a call.
models	função	() -> list[dict[str, Any]]	
datasets	função	() -> list[dict[str, Any]]	
geometries	função	() -> list[dict[str, Any]]	
flows	função	() -> list[dict[str, Any]]	
nodes	função	() -> list[dict[str, Any]]	Every node type of Flow Studio's palette (pinneapple_flow.REGISTRY: the curated nodes and the generated library calls), so the palette and the Hub list the same tools from the same registry.
node_for_callable	função	() -> dict[str, str]	module:function -> the node type that wraps it (generated library calls by their target, curated nodes by the module:function or module.function they declare in calls).
recipes	função	() -> list[dict[str, Any]]	
apis	função	() -> list[dict[str, Any]]	One item per row of AGENTS.md's "Use these APIs first" table, for the functions that row names.
agent_tools	função	() -> list[dict[str, Any]]	The MCP server's tools, read from the source (@server.tool() functions).

nome	tipo	assina-tura	o que faz
examples	função	`() -> list[dict[str, Any]]`	

pinneapple_hub.model_card

`ModelCard`: the reproducibility contract every hub-pushed model carries.

`pinneapple_hub/model_card.py` · 180 linhas · testes: 21

nome	tipo	assinatura	o que faz
`ModelCard`	classe	`(name, description, task, version, inputs, outputs, validity_range, known_limitations, ar…)`	

pinneapple_hub.verified_card

Model cards whose `validation_metrics` come only from a guardrail run against the model, refused otherwise.

`pinneapple_hub/verified_card.py` · 122 linhas · testes: 6

nome	tipo	assinatura	o que faz
`CardRefused`	classe	`(problems)`	The card built from a guardrail run did not pass `ModelCard.validate()`; `.problems` lists why.
`validation_metrics_from_report`	função	`(report) -> dict[str, float]`	`{check name: value}` for numeric checks, `{<name>_passed: 1.0 | 0.0}` for pass/fail-only checks.
`card_from_guardrail`	função	`(spec, report, *, name, preset, preset_kwargs=None, architecture, architecture_config, tr…)`	A `ModelCard` for a model of preset `preset` from its guardrail `report` (not validated here).
`certify_model`	função	`(preset, name, *, preset_kwargs=None, architecture='modified_mlp', architecture_config=No…)`	Train (or load `state_dict`: a dict or `torch.save` bytes) a model of `preset`, check it, build its card.

pinneapple_hub.zoo

`pp.zoo`: pre-trained, verified PINNeAPPle models in one line -- load, predict, evaluate against the stored reference, validate (physics checks + validity range), benchmark and fine-tune with a held-out guard.

`pinneapple_hub/zoo/__init__.py` · 341 linhas · testes: 125

nome	tipo	assinatura	o que faz
`FineTuneRejectedError`	classe	`()`	The fine-tuned model is worse than the original on the original test set (pass `force=True` to save it).

nome	tipo	assinatura	o que faz
FineTuneResult	classe	(model, original, before, after, accepted, tolerance, history, new_data_metrics, saved_to)	
export_onnx	função	(name, path, *, checks=False, check_parity=True, offline=None) -> dict[str, Any]	Export a model to ONNX (pre/post-processing included; parity with PyTorch checked to 1e-5). See :func:pinneapple_hub.zoo.export.export_onnx.
onnx_parity	função	(name, onnx_model, X=None, *, offline=None) -> float	Largest scaled difference between an ONNX graph and the PyTorch model (:mod:pinneapple_hub.zoo.export).
query_checks	função	(name, inputs=None, *, offline=None, **named) -> builtins.list	Per-query trust checks and verdict for each input row (:mod:pinneapple_hub.zoo.checks).
check_spec	função	(name, *, offline=None) -> dict[str, Any]	The declarative spec of a model's query checks (:func:pinneapple_hub.zoo.checks.check_spec).
export_web	função	(name, directory, *, offline=None) -> dict[str, Any]	ONNX graph with query checks + JSON card/spec for the browser (:mod:pinneapple_hub.zoo.web).
list	função	(task=None) -> builtins.list	Names of the shipped models (optionally only those whose task or tags mention task).
info	função	(name, *, offline=None, version=None) -> ModelCard	The model card (ModelCard): task, inputs/outputs with units, validity range, metrics against the reference, training data, licence, citation and the stored trust report. print(card.to_markdown()).
load	função	(name, *, offline=None, version=None) -> ZooModel	A ready-to-use predictor (weights verified against the manifest's SHA-256). version loads a superseded release (see :func:versions; a SupersededModelWarning says why it was superseded).
versions	função	(name) -> builtins.list[dict[str, str]]	Every release of a model, newest first, with the reason each older one was superseded.
predict	função	(name, inputs=None, **named) -> dict[str, np.ndarray]	load(name).predict(...) in one call.
catalogue	função	() -> str	Markdown page of every model and dataset: metrics, trust decision, size, Hugging Face link (docs/guides/zoo_catalogue.md is this page).
status	função	(name) -> dict[str, str]	Where each file of a model resolves from right now (bundled | cached | remote), without downloading.
evaluate	função	(name, dataset=None, *, split=None, offline=None, rtol=None) -> dict[str, Any]	Metrics of a model against reference cases.

nome	tipo	assinatura	o que faz
validate	função	(name, inputs=None, *, offline=None)	Trust report (pinneapple_analysis.trust.TrustReport) of a model: its card metrics reproduced on the stored reference, the acceptance limits of the zoo gate, the model's physics checks and -- if inputs are given -- whether they are inside the validity range (an out-of-range warning is raised ...
finetune	função	(name, new_data, *, epochs=200, lr=0.0001, batch=256, weight_decay=0.0, tolerance=0.02, s…)	Fine-tune a copy of a zoo model on new_data ((inputs, outputs), a dataset split or a mapping of columns; NaN outputs are ignored) and check it on the model's ORIGINAL held-out test set.
benchmark	função	(name, *, budget_s=60.0, seeds=1, baselines=('retrain',), out_dir='zoo_benchmark', force=…)	Benchmark a zoo model with pp.benchmark: the shipped model against the same architecture retrained from scratch on the same training split under budget_s seconds (CPU), on the same held-out test set, with pp.benchmark's gates, report and trust card. Returns its BenchmarkResult. ...

pinneapple_hub.zoo._artifacts

Artifact store of the zoo: manifest, SHA-256 checks, local cache and lazy download from the Hugging Face Hub.

pinneapple_hub/zoo/_artifacts.py · 237 linhas · testes: 81

nome	tipo	assinatura	o que faz
ZooError	classe	()	Base class of the zoo's errors.
ArtifactNotPublishedError	classe	()	The artifact is in the manifest but not (yet) on the Hugging Face Hub.
ArtifactUnavailableError	classe	()	The artifact is neither bundled nor cached and cannot be downloaded (offline or network error).
ChecksumMismatchError	classe	()	A file's SHA-256 differs from the manifest: corrupt, truncated or tampered with.
sha256_file	função	(path, chunk=1 << 20) -> str	
manifest	função	() -> dict[str, Any]	The parsed manifest.json (re-read when the file changes).
cache_dir	função	() -> Path	
is_offline	função	(offline=None) -> bool	
entry	função	(kind, name, version=None) -> dict[str, Any]	The manifest entry of a model (kind="models") or dataset (kind="datasets"); with version, the entry of that release -- the current one or one listed under superseded (its files are on the Hub, under its own tag; the reason it was superseded is in superseded_reason).

nome	tipo	assinatura	o que faz
`versions`	função	`(kind, name) -> list[dict[str, str]]`	Every release of an item, newest first: version, revision and, for older ones, why they were superseded.
`revisions`	função	`(ent) -> list[str]`	Hub revisions holding an entry's files, in the order they are tried: its `revision` and, when that is a republication of the same files (`v1.0.0.post1`: new card, Parquet or safetensors copies), the tag of the version itself (`v1.0.0`). Every file is checked against the manifest's SHA-256 ...
`fetch`	função	`(kind, name, filename, *, offline=None, version=None) -> Path`	Local path of one file of a model or dataset (`version`: a superseded release instead of the current one), verified against the manifest's SHA-256.
`status`	função	`(kind, name) -> dict[str, str]`	Where each file of an item currently resolves from without downloading: bundled | cached | remote.

`pinneapple_hub.zoo._bench`

`pp.zoo.benchmark`: a zoo model inside `pp.benchmark` (`pinneapple_analysis.benchmark`).

`pinneapple_hub/zoo/_bench.py` · 224 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`run_benchmark`	função	`(model, *, budget_s, seeds, baselines, out_dir, force, log, **kwargs)`	

`pinneapple_hub.zoo._catalog`

Static description of every shipped model and dataset (the source of truth `scripts/zoo/build_assets.py` turns into `manifest.json` + `model_card.json` files). Only items that passed the zoo's gate are listed here; the candidates that did not, and why, are in ...

`pinneapple_hub/zoo/_catalog.py` · 715 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`hf_repo`	função	`(kind, name) -> str`	Hub repo of a model/dataset: PINNeAPPle-Labs/.

`pinneapple_hub.zoo._models`

Zoo model adapters: a ready-to-use predictor per shipped model, with its validity range, metrics against the stored reference, physics checks (feeding a `TrustReport`) and the hooks fine-tuning needs.

`pinneapple_hub/zoo/_models.py` · 657 linhas · testes: 7 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`OutOfRangeWarning`	classe	`()`	Inputs outside the model's validity range (the prediction is an extrapolation).

nome	tipo	assinatura	o que faz
`ZooModel`	classe	`(name, card, config, state, spec)`	Base class: a loaded zoo model. Subclasses implement `_build`, `_encode`/`_decode` and the metrics.
`AirfoilMLP`	classe	`()`	
`AirfoilGNN`	classe	`()`	MeshGraphNet on the near field of the campaign's O-grid: p, Ux, Uy, log(1 + nut/nu) at the graph nodes and Cl, Cd, Cm from a pooled wall readout. Coefficient metrics and checks are the MLP adapter's; the field metrics are relative L2 errors on the held-out shapes of the field dataset (steady runs).
`HeatsinkForced`	classe	`()`	
`HeatsinkNatural`	classe	`()`	Natural convection: no air velocity input and no pressure drop; trained with monotonicity constraints in conductivity and source width (`pinneapple_design.thermal.heatsink.monotone`).
`RoverContact`	classe	`()`	
`SupersededModelWarning`	classe	`()`	A superseded release of a zoo model was loaded.
`load_dir`	função	`(directory) -> ZooModel`	Load a model saved with :meth:`ZooModel.save` (e.g. a fine-tuned copy).
`load_model`	função	`(name, *, offline=None, version=None) -> ZooModel`	
`copy_model`	função	`(m) -> ZooModel`	
`fit`	função	`(m, X, Y, *, epochs=300, lr=0.0001, batch=256, weight_decay=0.0, seed=0, progress=None) -…)`	Adam on the model's own normalised targets (NaN targets are masked); returns the loss per epoch.

`pinneapple_hub.zoo.catalogue`

The zoo catalogue: one Markdown page listing every model and dataset with its metrics, size and Hub link.

`pinneapple_hub/zoo/catalogue.py` · 73 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`markdown`	função	`() -> str`	The catalogue page (Markdown).

`pinneapple_hub.zoo.checks`

Per-query trust checks of the zoo models, and the verdict they add up to.

`pinneapple_hub/zoo/checks.py` · 358 linhas · testes: 34

nome	tipo	assinatura	o que faz
`has_query_checks`	função	`(zm) -> bool`	Whether the model has per-query checks (:func:`query_checks`); otherwise only `in_range` applies.
`check_spec`	função	`(zm) -> dict[str, Any]`	The declarative description of a model's query checks: `checks` (one entry per column of the `checks` output: id, rule, name, op, limit, severity, kind, group, detail template), `aux` (names of the `aux` columns the templates use), `groups` (display name and rule of grouped checks) and …
`checks_module`	função	`(zm, predictor=None)`	`torch.nn.Module`: query rows (m, d) -> (values (m, C), aux (m, A)) of :func:`check_spec`.
`check_values`	função	`(zm, X) -> tuple[np.ndarray, np.ndarray]`	(values, aux) of the query checks for rows `X` (physical inputs, card order).
`compare`	função	`(value, op, limit) -> bool`	`value op limit`; NaN never passes.
`evaluate`	função	`(spec, values, aux) -> dict[str, Any]`	Statuses, details and the verdict for one row of check values (the browser does the same with the graph's outputs).
`verdict`	função	`(checks) -> dict[str, str]`	The worst status of the trust checks (design constraints excluded) with its title and explanation.
`query_checks`	função	`(model, inputs, *, offline=None) -> list[dict[str, Any]]`	The query checks and the verdict for each input row (a mapping of named columns or an (n, d) array).

`pinneapple_hub.zoo.datasets`

`pp.datasets`: ready-made, verified datasets in one line.

`pinneapple_hub/zoo/datasets.py` · 199 linhas · testes: 8

nome	tipo	assinatura	o que faz
`DatasetInfo`	classe	`(name, version, title, description, license, source, verification, splits, columns, citat…)`	
`DatasetSplit`	classe	`(data, name, split, ds_info)`	One split: `{column: np.ndarray}` (all columns have `n_rows` entries) plus `name`, `split`, `info`.
`audit_card`	função	`(name) -> dict[str, Any] | None`	The stored physics audit card of a dataset (`pinneapple_data.audit`), or None when it was not audited.

nome	tipo	assinatura	o que faz
audit	função	`(name, *, offline=None, save=False, **kw)`	Physics audit of a dataset (:func:`pinneapple_data.audit.audit_zoo_dataset`): residual of every sample against its PDE where one is stated, declared constraints, diversity, duplicates, train/test leakage and units. `save=True` stores the card with the package (shown in …
list	função	`() -> builtins.list[str]`	Names of the shipped datasets.
info	função	`(name) -> DatasetInfo`	
split_files	função	`(name, split=None) -> builtins.list[str]`	Files holding `split` (all files when None): `data.npz`, or `<split>.npz` / `<split>-NN.npz`.
load	função	`(name, split=None, *, offline=None)`	A split (:class:`DatasetSplit`), or {`split`: `DatasetSplit`} for all of them when `split` is None.
status	função	`(name) -> dict[str, str]`	Where the files resolve from right now (bundled | cached | remote), without downloading.

pinneapple_hub.zoo.demo

What the zoo's demo Space shows, computed by the library: each model's prediction along a sweep of each input, with its ensemble spread or conformal bound and where its query checks warn or fail, and the files of the static Space that shows them.

`pinneapple_hub/zoo/demo.py` · 189 linhas · testes: 21

nome	tipo	assinatura	o que faz
models	função	`() -> list[str]`	Names of the zoo models the demo offers.
trust	função	`(name) -> dict[str, Any]`	The model's trust report as a whole (`pp.zoo.validate`): decision and the reasons.
controls	função	`(name) -> list[dict[str, Any]]`	One entry per input of the model: name, unit, description, range (validity box, or the held-out span for inputs bounded only through derived quantities) and a starting value (the first held-out test case).
sweep	função	`(name, values, vary, *, n=61, margin=0.15) -> dict[str, Any]`	Predictions and query checks at `values` and along `vary` from `margin` below to `margin` above its range (so the sweep shows where the model stops being trusted).
bands	função	`(outputs) -> dict[str, dict[str, Any]]`	The main outputs of a model and how each one's uncertainty is drawn: `{"std": "<name>_std"}` (ensemble spread, drawn as +-2 std) or `{"upper": "<name with upper95>"}` (split-conformal 95 % upper bound).
precompute	função	`(name, *, n=SWEEP_POINTS) -> dict[str, Any]`	Everything the static Space shows for one model without running it: the inputs (:func:`controls`), the outputs with units and bands (:func:`bands`), the model's trust decision (:func:`trust`), the prediction and the query checks at the held-out case, and one :func:`sweep` per input (x, every …

nome	tipo	assinatura	o que faz
export_space	função	(directory, models_=None) -> dict[str, Any]	Write the data and runtime of the static Space into directory: models/<name>.{onnx,json,sweeps.json}, lib/{onnx,zoo}.js and index.json (the models, in order, with their task and trust decision).

pinneapple_hub.zoo.export

ONNX export of zoo models and their per-query trust checks.

pinneapple_hub/zoo/export.py · 317 linhas · testes: 55 · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
predictor_module	função	(zm)	torch.nn.Module mapping the card's inputs (n, d) to a tuple of outputs (see :func:output_names).
output_names	função	(zm) -> dict[str, list[str]]	ONNX output -> the predict keys of its columns.
as_graph_outputs	função	(zm, pred) -> dict[str, np.ndarray]	predict output laid out like the graph's outputs.
output_scale	função	(zm) -> dict[str, np.ndarray]	Per column, the scale of the parity tolerance: NaN = relative to the value (positive, log-modelled outputs: Cd, R, dP; L/D relative to max(|L/D|, y_std(Cl) / Cd), the error a Cl within tolerance implies), otherwise the network's standardisation scale.
parity_error	função	(zm, ref, got) -> float	Largest |got - ref| / scale over every output column (see :func:output_scale). Spreads (std, LD_std) are measured relative to the quantity they spread around.
export_bytes	função	(zm, *, checks=False) -> bytes	The ONNX graph of a loaded zoo model (see :func:export_onnx).
export_onnx	função	(name, path, *, checks=False, check_parity=True, offline=None) -> dict[str, Any]	Export a zoo model to ONNX at path. The graph maps the card's inputs (input x, shape (n, d)) to its outputs; with checks=True it also takes query (m, d) and returns the query-check values (checks, aux; see :func:pinneapple_hub.zoo.checks.check_spec). Unless …
parity_inputs	função	(zm, *, n_random=500, seed=7) -> np.ndarray	The model's held-out and independent-reference inputs (every evaluation split) plus random points inside the validity box (when the box describes every input).
run_onnx	função	(data, X, query=None) -> dict[str, np.ndarray]	Run a graph in onnxruntime (CPU, one thread) and return its outputs by name (float64).
onnx_parity	função	(name, onnx_model, X=None, *, offline=None) -> float	Largest scaled difference (:func:parity_error) between an exported graph's predictions and ZooModel.predict on X (default :func:parity_inputs).

nome	tipo	assinatura	o que faz
checks_parity	função	(name, onnx_model, X=None, *, offline=None) -> dict[str, float]	Compare the query checks of a graph exported with checks=True with :func:pinneapple_hub.zoo.checks.check_values: the largest value difference (relative to max(|value|, 1)) and the number of rows where a status differs although the value is farther than CHECK_TOL from its limit (float32 …
export_safetensors	função	(name, path, *, offline=None) -> dict[str, Any]	Write the model's weights (the tensors of weights.pt) as safetensors at path; the file's metadata names the model, its version and adapter. Returns {"path", "bytes", "sha256", "tensors"}.
load_safetensors	função	(name, path, *, offline=None) -> ZooModel	The zoo model name (config and card from the zoo) with its weights read from a safetensors file.

pinneapple_hub.zoo.hub_cards

The README cards of the zoo's Hugging Face repos: YAML front matter the Hub reads, and the text people read.

pinneapple_hub/zoo/hub_cards.py · 368 linhas · testes: 21

nome	tipo	assinatura	o que faz
space_url	função	() -> str	
front_matter	função	(meta) -> str	
parse_front_matter	função	(readme) -> dict[str, Any]	The YAML block at the top of a README ({} when there is none).
code_blocks	função	(readme, lang='python') -> list[str]	The fenced code blocks of a README in language lang.
split_order	função	(name) -> list[str]	Splits of a dataset with train / val / cal / test first, then the others in manifest order.
dataset_card	função	(name, layout) -> str	README of a dataset repo: front matter (configs, dataset_info, licence, task categories, sizes, tags) and the card. layout is what :func:pinneapple_hub.zoo.parquet.write_dataset returned for this dataset.
model_example	função	(zm) -> dict[str, Any]	A held-out reference case of the model (first row of its test split): the inputs (6 significant digits), the model's outputs for exactly those inputs, the stored reference outputs, the query-check verdict and the model's trust decision. The "How to use" snippet prints these numbers and the …
model_card	função	(name, *, example, onnx_info=None, files=()) -> str	README of a model repo. example: :func:model_example; onnx_info: what :func:pinneapple_hub.zoo.export.export_onnx returned (None: no ONNX file); files: the derived files published next to the manifest's (model.safetensors, model.onnx).
space_readme	função	() -> str	README of the demo Space: the configuration of a static Space (index.html, no server), the models and datasets it shows, and what the page does.
example_json	função	(example) -> str	

pinneapple_hub.zoo.parquet

Parquet copies of the zoo datasets, for the Hugging Face dataset viewer and `datasets.load_dataset`.

`pinneapple_hub/zoo/parquet.py` · 281 linhas · testes: 21

nome	tipo	assinatura	o que faz
`read_npz`	função	`(paths) -> dict[str, dict[str, np.ndarray]]`	`{split: {column: array}}` from npz files whose arrays are named `<split>__<column>`; a split spread over several files (shards `<split>-NN.npz`, in the given order) is concatenated along the rows.
`plan_split`	função	`(arrays, *, explode_row_bytes=EXPLODE_ROW_BYTES, max_shard_bytes=MAX_SHARD_BYTES, row_gro…)`	How one split is laid out: `samples` (npz rows), `steps` (0, or the length of the exploded axis), `exploded` columns, `rows` (Parquet rows), `row_bytes` and the rows per shard and per row group.
`write_dataset`	função	`(name, data, out_dir, *, split_order=(), column_order=(), explode_row_bytes=EXPLODE_ROW_B…)`	Write `{split: {column: array}}` as Parquet under `out_dir/data/<config>/` and return the layout: `{"configs": [{"name", "default", "columns", "features" (dataset_info YAML), "splits": [{"name", "samples", "steps", "exploded", "rows", "files" (relative to out_dir), "paths", "bytes"}]}]}`.
`read_parquet`	função	`(paths) -> dict[str, np.ndarray]`	The npz arrays of one split back from its Parquet shards (in order): the inverse of :func:`write_dataset`.
`parquet_files`	função	`(layout) -> list[str]`	Every Parquet file of a layout (paths relative to the dataset's root).
`features_yaml`	função	`(schema) -> list[dict[str, Any]]`	The `dataset_info.features` list of a Parquet schema, as `datasets.Features._to_yaml_list` writes it.

pinneapple_hub.zoo.publish

Publish the zoo to the Hugging Face Hub and check what the Hub serves (`scripts/zoo/publish_to_hub.py` is the command line, `.github/workflows/publish-zoo.yml` runs it).

`pinneapple_hub/zoo/publish.py` · 611 linhas · testes: 49

nome	tipo	assinatura	o que faz
`plan`	função	`(staging=None, *, need_files=True, hub_fallback=False, work=None, only=()) -> list[dict]`	Every item to publish: `kind`, `name`, `repo`, `repo_type`, `revision`, `files` (manifest files to upload: name -> local path), `hub_only` and `source` ("local", or "hub" for a Hub-only dataset converted from the Hub copy, prepared when it is published). Items with local files are …
`prepare`	função	`(it, work) -> dict`	Write the item's derived files into `work` and its README: Parquet copies and the dataset card, or the safetensors and ONNX copies, the example and the model card. Fills `it["derived"]` (path in repo -> local path) and `it["readme"]`.

nome	tipo	assinatura	o que faz
publish	função	(items, token, *, work=None, tags=True) -> list[str]	Upload every item (one commit each) and tag it with its revision; returns the items that failed (each failure is printed and the others are still published). A Hub-only dataset from the Hub copy is prepared here, one at a time, and its downloads deleted afterwards (the npz are already on the Hub, ...
build_space	função	(out, app_dir, playground_dir, models=None) -> list[str]	Write the static demo Space into `out`: the data and runtime of :func:`pinneapple_hub.zoo.demo.export_space`, the page of `app_dir` (`apps/zoo_space`), the Playground's charts, widgets, styles and icon (`playground_dir` = `apps/playground/site`) and the README (Space configuration). ...
publish_space	função	(token, app_dir, playground_dir, *, work=None, from_hub=True) -> list[str]	Build the static demo Space (:func:`build_space`) and upload it in one commit, deleting the files of an earlier version that the new one does not have. With `from_hub` the sweeps are computed from the models and data downloaded from the Hub (the bundled copies ignored), i.e. from what was ...
verify_files	função	(items, allow_unpublished=False) -> int	Download every manifest file anonymously from the item's revision (or the version tag it republishes) and compare its SHA-256 with the manifest; returns the number of failures. With `allow_unpublished`, a Hub-only item that is not on the Hub yet (HTTP 404) is reported but not counted; a wrong ...
viewer_status	função	(repo, *, timeout=2400, interval=30, get=_get_json, sleep=time.sleep) -> dict[str, Any]	The dataset viewer's state of one dataset repo: `is-valid` (polled until `preview` and `viewer` are true or `timeout`), the configs and splits it found and, per split, the `first-rows` answer (rows, truncated cells, error).
verify_viewer	função	(names=None, *, timeout=2400, interval=30, get=_get_json, sleep=time.sleep) -> int	Check the dataset viewer of every dataset (anonymous datasets-server API): `is-valid` must say preview and viewer, every split of the manifest must be listed, and `first-rows` of every split must return rows. Prints one block per dataset; returns the number of datasets that fail.
verify_model	função	(name, work) -> list[str]	Problems found with the published model `name` (empty when none): it is loaded from the Hub with the zoo's loader (bundled copy ignored, fresh cache); its card metrics must be reproduced on the stored reference (`pp.zoo.evaluate`); the card's example must be reproduced by the PyTorch weights, ...
verify_models	função	(names=None) -> int	:func:`verify_model` for every model, anonymously (no token), with the bundled copies ignored and an empty cache; returns the number of models with problems.
verify_space	função	(*, timeout=2400, interval=30, runtime=None, fetch=_fetch, sleep=time.sleep) -> int	Wait for the demo Space to run and serve its page; 0 when the stage is RUNNING and the page, `index.json` and every model's sweeps load (anonymously), 1 otherwise. Prints the stage, hardware and what was fetched.

pinneapple_hub.zoo.relations

Metamorphic relations the physics of each zoo model implies (#698), checked without any reference solution by :func:`check_zoo_relations` (`pinneapple_analysis.validation.metamorphic`).

`pinneapple_hub/zoo/relations.py` · 95 linhas · testes: 9

nome	tipo	assinatura	o que faz
`check_zoo_relations`	função	`(model, name=None, *, n=64, seed=0, search='random', relations=None) -> MetamorphicReport`	Run the relations of a zoo model (`model`: a loaded zoo model or anything with the same `predict`).

`pinneapple_hub.zoo.validation_reports`

Stored validation reports of the zoo models (#706): one `pp.validate` report per model, kept with the package, shown in the model's Hub card, and compared with a fresh report before a release.

`pinneapple_hub/zoo/validation_reports.py` · 59 linhas · testes: 7

nome	tipo	assinatura	o que faz
`build`	função	`(name)`	Validate the zoo model `name` and store its report.
`stored`	função	`(name) -> dict[str, Any] | None`	
`check_release`	função	`(name, model=None)`	`(diff, fresh report)` of the model as it is now (or `model`, e.g. a new version) against its stored report; `diff.regressed` blocks the release. Without a stored report the diff is against an empty one.
`card_section`	função	`(name) -> str`	The markdown block of the stored report for the model's Hub card ("" when none is stored).

`pinneapple_hub.zoo.web`

Zoo models for the browser: ONNX graphs with their query checks, model cards and the JS runtime.

`pinneapple_hub/zoo/web/__init__.py` · 160 linhas · testes: 13

nome	tipo	assinatura	o que faz
`runtime_dir`	função	`() -> Path`	Folder of the JS runtime (`onnx.js`, `zoo.js`).
`reference_data`	função	`(zm) -> dict[str, Any]`	Reference points to plot next to the predictions: for the airfoil, the held-out OpenFOAM runs and the XFOIL polar of each NACA preset; for the rover, the held-out Wong-Reece states.
`export_web`	função	`(name, directory, *, offline=None) -> dict[str, Any]`	Write `<name>.onnx` and `<name>.json` for the browser into `directory`; returns the JSON content.
`fixtures`	função	`(*, offline=None) -> dict[str, Any]`	Python answers for fixed inputs: predictions and query-check evaluations of each model (in range and out of range), and the geometry of a few CST shapes. The JS runtime tests and the Playground's browser test compare against these.
`export_bundle`	função	`(directory, models=MODELS, *, offline=None) -> dict[str, Any]`	Everything the Playground loads: each model (:func:`export_web`), the CST geometry graphs and fixtures.

`pinneapple_hub.zoo.web.__main__`

python -m pinneapple_hub.zoo.web [DIR]: write the browser bundle of the zoo (default data/web/zoo).

`pinneapple_hub/zoo/web/__main__.py` · 23 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 23 de 31.
- Funções de teste em arquivos que importam algum módulo do pacote: 183.

Limitações conhecidas

- *Nenhuma registrada.*

5. `pinneapple_lab`

Objetivo

O `pinneapple_lab` é o laboratório de experimentos do PINNeAPPle. Ele roda muitos experimentos de forma reprodutível e acumula um banco de resultados e de datasets. Um experimento é uma classe com parâmetros padrão e um método `run(ctx)`; o runner dá a cada execução um id derivado do nome, da versão e dos parâmetros, grava entradas, saídas, métricas, checagens de validação, figuras e datasets numa pasta própria e indexa tudo em SQLite (`LabStore`). Rodar de novo com os mesmos parâmetros devolve o resultado em cache.

Por cima desse banco existe uma camada de curadoria. `curation.curate` avalia cada execução em dimensões calculadas só a partir do que foi gravado (checagens, métricas, arquivos, código, datasets, documentação e uma revisão humana opcional) e dá uma nota de A (flagship) a F (não utilizável). As notas são portões, não média ponderada: uma checagem que falhou nunca é escondida por uma figura bonita. `next_steps` diz o que falta para a próxima letra e qual ação do PINNeAPPle fecharia cada lacuna; `seal` carimba a letra em figuras, renders e vídeos; `explorer`, `html`, `server`, `reports` e `showcase` publicam tudo como página, API, relatório ou painel animado.

O pacote se liga ao resto da biblioteca de várias formas. Os experimentos embutidos em `pinneapple_lab.experiments` chamam solvers e modelos de outros pacotes (`pinneapple_simulation`, `pinneapple_physics`, `pinneapple_neural`, `pinneapple_design`) e conferem os resultados contra soluções exatas ou referências publicadas. O módulo `manifest` define o formato `experiment.json`, lido pelo site, pela curadoria e pelo Flow Studio, e converte para manifesto resultados de `pp.solve` (`from_solution`), tabelas de `pp.compare` (`from_comparison`), workflows

de `pinneapple_core` e execuções do Flow Studio. Qualquer script de `examples/` vira item do Lab com `pinneapple_lab.experiment(...)` (`capture`).

A linha de comando `python -m pinneapple_lab` reúne tudo: `list`, `run`, `sweep`, `status`, `report`, `serve`, `curate`, `review`, `brief`, `seal`, `stamp`, `improve`, `register`, `scaffold`, `manifests --check` (o portão de CI dos manifestos), `showcase` e `export`.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
42	11.775	45	119	96 (9 arquivos)	27

Submódulos

submódulo	o que faz	módulos	classes + funções	testes *
`__main__`	Linha de comando `python -m pinneapple_lab`: rodar, varrer, curar, carimbar, servir e exportar.	1	1	59
`capture`	`experiment(...)`: embrulha um script para que ele deixe um manifesto, um item do Lab e uma nota.	1	3	0
`context`	`RunContext`, o objeto que o experimento recebe para gravar tudo o que produz.	1	1	0
`curation`	Curadoria: avalia execuções em dimensões de evidência e dá notas A-F com prontidão por uso.	1	17	65
`dataset`	Datasets gerados pelas execuções, gravados em shards `.npz` com um cartão descritivo.	1	2	0
`experiments`	Experimentos embutidos (ODE, PDE, LBM, FEM, CFD, descoberta de leis, ROM, importação de resultados do repositório).	22	39	20
`explorer`	Página de experimentos com selo A-F, cartão de confiança, métricas, mídia e filtros.	1	11	6
`html`	Catálogo navegável do Lab em uma única página HTML autocontida.	1	6	14
`improve`	Comando `improve`: mostra as ações para a próxima letra e, se pedido, roda as executáveis e recura.	1	2	0
`manifest`	Formato `experiment.json` (manifesto de experimento), conversões, validação e portão de CI.	1	19	29
`next_steps`	Lacunas entre a letra atual e a próxima, cada uma com as ações do PINNeAPPle que a fecham.	1	9	31
`reports`	Relatórios em HTML, Markdown ou PDF de um item ou de um conjunto filtrado.	1	6	20
`results_rules`	Regras que extraem métricas e checagens dos arquivos de resultados que um exemplo versiona.	1	6	14
`runner`	Execução de um experimento ou de varreduras (grade ou amostras), com cache e paralelismo.	1	3	0
`seal`	Selo de qualidade PINNeAPPle: badge com a letra da curadoria, carimbado em figuras e vídeos.	1	21	39
`server`	Servidor HTTP (só biblioteca padrão) do catálogo, arquivos das execuções, API JSON e downloads.	1	2	26
`showcase`	Painel animado (MP4, GIF, PNG) com os melhores resultados, montado a partir da curadoria.	1	5	3
`spec`	Classe base `Experiment` e registro de experimentos por nome.	1	4	0
`store`	`LabStore`: pastas de execução mais índice SQLite, consultas, exportação de datasets e catálogo.	1	2	0

sub-módulo	o que faz	módulos	classes + funções	testes *
uq	Checagens de incerteza para o cartão de confiança: cobertura de intervalos, GCI e espalhamento de ensemble.	1	5	20

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

pinneapple_lab

Ponto de entrada do pacote: reexporta `Experiment`, `register`, `run`, `sweep`, `LabStore`, `experiment` e afins.

`pinneapple_lab/__init__.py` · 43 linhas · testes: 71

pinneapple_lab.__main__

Linha de comando `python -m pinneapple_lab` com subcomandos para rodar, varrer, curar, revisar, carimbar, servir, gerar relatórios e exportar datasets.

`pinneapple_lab/__main__.py` · 366 linhas · testes: 59

nome	tipo	assinatura	o que faz
main	função	`(argv=None) -> int`	Monta o `argparse` com todos os subcomandos e despacha para as funções do pacote; devolve o código de saída.

pinneapple_lab.capture

Embrulha um script ou exemplo (`with experiment(...) as ex:` ou decorador) para gravar parâmetros, métricas, checagens e arquivos num manifesto e registrá-lo no Lab. [sem teste direto]

`pinneapple_lab/capture.py` · 168 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Capture	classe	`(name, *, title='', description='', domain='', methods=(), modules=(), params=None, out_d…)`	Contexto/decorador que acumula parâmetros, métricas (`log_metric`), checagens (`check`) e arquivos (`add_file`) e, ao sair, escreve o manifesto e registra o item no Lab.
experiment	função	`(name, **kw) -> Capture`	Cria um `Capture`, usável como `with experiment(...) as ex:` ou como `@experiment(...)`.
register_manifest	função	`(path, lab=None, *, curate=True) -> str`	Importa um manifesto válido no Lab como item `manifest/<id>`, recura e devolve a chave do item; recusa manifesto inválido.

pinneapple_lab.context

Define o `RunContext`, que o experimento recebe em `run(ctx)` e que grava tudo na pasta da execução e no índice. [sem teste direto]

`pinneapple_lab/context.py` · 217 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RunContext`	classe	`(run_dir, params, experiment, run_id, seed=None)`	Objeto entregue a `Experiment.run`: dá os parâmetros e grava log, estágios, entradas, saídas, figuras, imagens, GIFs, métricas, checagens e datasets na pasta da execução.

`pinneapple_lab.curation`

Curadoria das execuções: avalia dimensões de evidência (validação, referência, baseline, física, incerteza, reprodutibilidade etc.) e atribui notas A-F com prontidão para produto, artigo, marketing ou dados de treino. Usa o vocabulário de `pinneapple_veriphysics.applicability`.

`pinneapple_lab/curation.py` · 970 linhas · testes: 65

nome	tipo	assinatura	o que faz
`Dimension`	classe	`(name, score, status, tier, evidence)`	Uma dimensão avaliada: nota 0-1 (ou None = não rodou), status, nível de evidência e lista de evidências.
`Assessment`	classe	`(run_id, experiment, status, tier, score, coverage, dimensions, readiness, gaps, reviewed…)`	Avaliação completa de uma execução: letra, nota, cobertura, dimensões, prontidão por uso e lacunas.
`infer_check_kind`	função	`(check) -> str`	Deduz de que uma checagem é evidência (referência, benchmark, física etc.) pelo nome, pelo valor de referência ou pelo texto de detalhe.
`load_reviews`	função	`(root) -> dict[str, Any]`	Lê as revisões humanas gravadas em `<lab>/reviews.json`.
`save_review`	função	`(root, experiment, **fields) -> dict[str, Any]`	Grava (mesclando) a revisão humana de um experimento em `<lab>/reviews.json`.
`assess_run`	função	`(store, row, *, meta=None, reviews=None, siblings=None) -> Assessment`	Avalia uma execução (uma linha de `LabStore.runs()`) e devolve seu `Assessment`.
`path_to_next`	função	`(tier, issue, checks, dims, evidence, ok, is_ml, ctx, *, reviewed=False) -> list[Any]`	Lista as lacunas entre a letra atual e a próxima, cada uma com ações do PINNeAPPle, a partir das mesmas evidências da nota.
`next_steps`	função	`(item, lab=None) -> list[Any]`	Lê do registro de curadoria o caminho para a próxima letra de um item, como lista de `Gap`.
`problem`	função	`(status, checks, error=None) -> dict[str, Any] | None`	Diz qual regra E ou F a execução cai (travou, divergiu, física falhou, contradiz a referência...) ou None se tudo passou.
`problem_headline`	função	`(issue) -> dict[str, Any]`	Resumo de um item E ou F: a checagem principal que falhou, a regra e a dica do próximo passo.
`check_line`	função	`(check, max_chars=64) -> str`	Uma linha que diz contra o que a checagem comparou (rótulo dado pelo experimento ou nome da checagem mais detalhe).
`headline`	função	`(checks, tier, labels=None) -> dict[str, Any]`	Evidência que um selo ou bloco do showcase pode imprimir, tirada só das checagens que passaram.
`trust_stages`	função	`(d, checks, metrics, dims, tier, readiness, meta, is_ml, limitations) -> dict[str, dict[s…)`	Responde às seis perguntas do cartão de confiança (dados, modelo, física, benchmark, incerteza...) com base nas evidências da execução.

nome	tipo	assinatura	o que faz
evidence_stages	função	`(d, checks, metrics, dims, is_ml, limitations) -> dict[str, dict[str, Any]]`	As cinco primeiras perguntas do cartão de confiança, que não dependem da letra e alimentam a regra C/D.
plan_experiment	função	`(runs, meta, review) -> dict[str, Any]`	Plano de um experimento: limitações, ações que fecham lacunas (com esforço) e a execução mais próxima de estar pronta para cada uso.
assess_experiment	função	`(runs) -> dict[str, Any]`	Avalia um experimento inteiro: melhor execução, contagem por letra e consistência entre execuções.
curate	função	`(store, path_json=None, path_md=None) -> dict[str, Any]`	Avalia todas as execuções e experimentos e escreve `curation.json` e `CURATION.md` na pasta do Lab.

pinneapple_lab.dataset

Datasets das execuções: amostras gravadas em shards `.npz` com índice `samples.jsonl` e `card.json` (esquema, unidades, proveniência, checksum). [sem teste direto]

`pinneapple_lab/dataset.py` · 145 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
DatasetWriter	classe	`(folder, *, name, description='', units=None, shard_size=256, license='', provenance=None)`	Escreve amostras de um dataset em shards, com bufferização, e gera o cartão descritivo ao fechar.
read_dataset	função	`(folder) -> Iterator[dict[str, Any]]`	Itera sobre as amostras de uma pasta de dataset escrita por `DatasetWriter`.

pinneapple_lab.experiments

Pacote dos experimentos embutidos; cada módulo registra seus experimentos ao ser importado. [sem teste direto]

`pinneapple_lab/experiments/__init__.py` · 42 linhas · **sem teste direto**

pinneapple_lab.experiments.accretion

Experimentos de hidrodinâmica de buraco negro sobre `pinneapple_physics.blackhole`: acreção de Bondi e um toro acretando. [sem teste direto]

`pinneapple_lab/experiments/accretion.py` · 126 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
BondiAccretion	classe	`()`	Experimento `bondi_accretion`: o solver hidro, iniciado na solução exata de Bondi (potencial de Paczynski-Wiita), deve mantê-la estacionária.
AccretionFlowRun	classe	`()`	Experimento `accretion_flow`: toro quente acretando num buraco negro de Schwarzschild, com filme de densidade, taxa de acreção e balanços; os quadros viram dataset.

`pinneapple_lab.experiments.blackhole_forecast`

Experimento de previsão do "tempo" de buraco negro (Duarte, Nemmen & Navarro 2022) que embrulha `examples/black_hole_weather`; precisa do checkout do repositório. [sem teste direto]

`pinneapple_lab/experiments/blackhole_forecast.py` · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BlackHoleForecast`	classe	`()`	Experimento `bh_forecast`: U-Net que prevê a densidade do fluxo de acreção, avaliada por horizonte contra persistência.

`pinneapple_lab.experiments.bricks`

Construções de blocos encaixáveis (`pinneapple_design.bricks`) que se modificam até atingir uma meta (balanço, ponte, torre), verificadas por equilíbrio estático. [sem teste direto]

`pinneapple_lab/experiments/bricks.py` · 97 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BricksExperiment`	classe	`()`	Experimento `bricks`: verifica a física contra formas fechadas e depois muta a construção até cumprir a meta.
`make_goal`	função	`(case)`	Devolve a meta (`CantileverGoal`, `BridgeGoal` ou `TowerGoal`) de um caso pelo nome.

`pinneapple_lab.experiments.calculix_cases`

Casos de CalculiX via `pinneapple_simulation.external_solvers.calculix.study` (placa com furo, suporte em L, modal de viga...) contra referências em forma fechada. Depende de CalculiX/gmsh. [sem teste direto]

`pinneapple_lab/experiments/calculix_cases.py` · 283 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CalculixCase`	classe	`()`	Experimento `calculix_case`: roda um caso de CalculiX e compara tensão, convergência de malha e equilíbrio com fórmulas fechadas.

`pinneapple_lab.experiments.car`

Aerodinâmica de um perfil 2-D de carro paramétrico em túnel de vento virtual (LBM D2Q9 com LES) e um surrogate de IA física treinado nessas execuções.

`pinneapple_lab/experiments/car.py` · 438 linhas · testes: 20

nome	tipo	assinatura	o que faz
`CarLBM`	classe	`()`	Experimento `car_lbm`: um desenho de carro no túnel virtual, com arrasto e sustentação, campos médios e filme de vorticidade; gera dataset.

nome	tipo	assinatura	o que faz
CarSurrogate	classe	()	Experimento car_surrogate: treina um FNO (forma -> escoamento médio) e um ensemble de MLPs (parâmetros -> Cd, Cl) sobre as execuções car_lbm.
simulate_car	função	(shape, *, length_cells=64, Re=500.0, u_in=0.08, Cs=0.1, steps=8000, save_every=50, keep_…)	Roda o túnel de vento virtual para um perfil (ou máscara sólida) e devolve campos, históricos e coeficientes.

pinneapple_lab.experiments.cylinder

Escoamento em torno de cilindro por lattice-Boltzmann: regime, número de Strouhal e imagens de vorticidade rotuladas. [sem teste direto]

pinneapple_lab/experiments/cylinder.py · 61 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
CylinderLBM	classe	()	Experimento cylinder_lbm: canal 2-D com cilindro no solver D2Q9, medindo regime e Strouhal pelo espectro refinado.

pinneapple_lab.experiments.discovery

Descoberta de leis físicas a partir de dados: Kepler com dados orbitais reais, pêndulo filmado, Lorenz no estilo AI-Lorenz e oscilador, com SINDy e ELM. [sem teste direto]

pinneapple_lab/experiments/discovery.py · 548 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
KeplerLaw	classe	()	Experimento kepler_law: regressão esparsa sobre leis de potência redescobre a terceira lei de Kepler a partir de dados orbitais.
PendulumVideo	classe	()	Experimento pendulum_video: lei -> vídeo -> lei -> vídeo; mede o ângulo nos quadros, descobre a equação por SINDy fraco e resimula.
LorenzDiscovery	classe	()	Experimento lorenz_discovery: ajusta observações ruidosas de Lorenz por janelas com ELM e descobre as equações.
OscillatorDiscovery	classe	()	Experimento oscillator_discovery: usa trajetórias do oscilador já no banco e recupera ζ e ω por SINDy.
elm_fit	função	(t, y, n_features=300, ridge=1e-08, seed=0, scale=None)	Ajuste por mínimos quadrados com features tanh aleatórias fixas (extreme learning machine), com derivadas primeira e segunda.
windowed_elm	função	(t, y, window=200, overlap=40, **kw)	Ajustes ELM em janelas sobrepostas para registros longos e caóticos; devolve y e y' costurados.
weak_form_2nd_order	função	(t, u, du_hat, library, width, n_test, p=4)	Monta a forma fraca de u'' = f(u, u') com funções-teste, evitando derivar os dados (SINDy fraco).
sindy	função	(Theta, target, names, threshold=None, max_terms=3, min_contribution=0.03)	Seleção esparsa de lei: testa todos os subconjuntos de até max_terms termos e fica com o de menor BIC.

pinneapple_lab.experiments.em_pinn

PINNs nos presets de eletromagnetismo com solução fechada (`em_*`), resolvidos com `pp.solve(problem, "pinn")` e medidos com `pp.compare`. [sem teste direto]

`pinneapple_lab/experiments/em_pinn.py` · 109 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`EMPinn`	classe	`()`	Experimento `em_pinn`: treina a PINN de um caso `em_*` e mede o erro relativo contra o campo exato.

pinneapple_lab.experiments.examples

Transforma cada script de `examples/` em experimento do Lab: roda como um usuário e coleta figuras, JSON, arrays, saída de console e código.

`pinneapple_lab/experiments/examples.py` · 290 linhas · testes: 20

nome	tipo	assinatura	o que faz
`ExampleScript`	classe	`()`	Experimento `example_script`: roda um script de exemplo e guarda o que ele produziu como figuras, métricas, dataset `artifacts` e código.
`discover_examples`	função	`(repo=REPO) -> list[dict[str, str]]`	Lista os scripts executáveis de `examples/**/*.py`, sem helpers e pastas de rascunho, agrupados por pasta.

pinneapple_lab.experiments.landing

Importa como execuções do Lab os casos da landing page que vivem em repositórios companheiros públicos (PINNeAPPle-Benchmark, PINNeAPPle-Climate). [sem teste direto]

`pinneapple_lab/experiments/landing.py` · 292 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BenchmarkCase`	classe	`()`	Experimento `benchmark_case`: importa um experimento finalizado de repositório companheiro e reconfere a afirmação principal com o `results.json` dele.
`source_root`	função	`(key, lab_root=None) -> str | None`	Acha o checkout local de um repositório companheiro (variável de ambiente, pasta irmã ou clone raso).

pinneapple_lab.experiments.manifest_import

Experimento `manifest`: traz um `experiment.json` para o Lab como execução, para ser curado A-F como qualquer item. [sem teste direto]

`pinneapple_lab/experiments/manifest_import.py` · 45 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ManifestImport`	classe	`()`	Experimento `manifest`: grava parâmetros, métricas, checagens e cópias de figuras de um manifesto; falha se o manifesto registra erro.

pinneapple_lab.experiments.ode

Oscilador harmônico amortecido: integradores (Euler, Euler simplético, RK4) contra a solução exata. [sem teste direto]

`pinneapple_lab/experiments/ode.py` · 95 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Oscillator`	classe	`()`	Experimento `oscillator`: integra x'' + 2ζωx' + ω²x = 0 e compara com a solução exata; gera dataset de trajetórias.
`exact`	função	`(t, zeta, omega, x0=1.0, v0=0.0)`	Solução exata do oscilador amortecido nos casos sub, crítico e superamortecido.
`integrate`	função	`(method, zeta, omega, dt, t_end, x0=1.0, v0=0.0)`	Integra o oscilador com o método escolhido (`euler`, `symplectic`, `rk4`) e passo fixo.

pinneapple_lab.experiments.particles

Suspensão de sólidos em tanque agitado: partículas lagrangianas (DEM, arrasto, empuxo, dispersão) num escoamento analítico, renderizadas como vídeo de processo. [sem teste direto]

`pinneapple_lab/experiments/particles.py` · 156 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ParticleSuspension`	classe	`()`	Experimento `particle_suspension`: simula a suspensão e confere velocidade de sedimentação e rotação de suspensão contra Schiller-Naumann e Zwietering.

pinneapple_lab.experiments.pde

Experimentos de EDP resolvidos com os solvers da biblioteca e conferidos contra soluções exatas. [sem teste direto]

`pinneapple_lab/experiments/pde.py` · 49 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`HeatXTFC`	classe	`()`	Experimento `heat_xtfc`: equação do calor 1-D resolvida por X-TFC contra exp(-α π² t) sin(π x).

pinneapple_lab.experiments.pipes

Escoamento interno em tubos e misturadores estáticos com OpenFOAM (`internal_flow`): Hagen-Poiseuille, curva de 90 graus e misturador Kenics. Depende de OpenFOAM. [sem teste direto]

`pinneapple_lab/experiments/pipes.py` · 307 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PipeFlow`	classe	`()`	Experimento `pipe_flow`: roda um caso do solver de tubo e compara fator de atrito e perda com 64/Re, Colebrook e Ito.

`pinneapple_lab.experiments.repo_results`

Importa como execuções validadas os resultados já produzidos por scripts do repositório, reconferindo-os contra sua referência. [sem teste direto]

`pinneapple_lab/experiments/repo_results.py` · 317 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RepoResults`	classe	`()`	Experimento `repo_results`: lê os artefatos de uma fonte (`source`), reconfere e grava métricas, figuras e datasets.

`pinneapple_lab.experiments.repo_sources`

Mais fontes para `repo_results` (máquinas elétricas, aerofólio, CFD de avião, rover lunar, otimização topológica, modelo de clima etc.), cada uma transformando números versionados em checagens. [sem teste direto]

`pinneapple_lab/experiments/repo_sources.py` · 636 linhas · **sem teste direto**

`pinneapple_lab.experiments.rom`

Modelos de ordem reduzida (`pinneapple_neural.architectures.rom`) sobre os solvers do Lab: POD, DMD e Operator Inference na esteira de cilindro e ROM paramétrico de viga FEM. [sem teste direto]

`pinneapple_lab/experiments/rom.py` · 399 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ROMStudy`	classe	`()`	Experimento `rom_study`: roda um caso de ROM e confere energia dos modos POD e previsão contra persistência.
`cylinder_wake`	função	`(Re, steps, save_every, D=20)`	Gera snapshots de velocidade do escoamento em torno de cilindro (LBM D2Q9) em unidades de rede.
`cantilever`	função	`(H, W, angle_deg, *, L=2.0, mesh=(30, 3, 6), F=10000.0)`	Resolve por FEM sólido 3-D uma viga engastada de aço com carga na face livre e devolve o `FEMResult`.

`pinneapple_lab.experiments.structures`

Mecânica estrutural 3-D com o solver `solid_fem` (hexaedros de 8 nós com modos incompatíveis), conferida contra teoria de vigas e torção e contra CalculiX. [sem teste direto]

`pinneapple_lab/experiments/structures.py` · 225 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SolidFEMExperiment`	classe	`()`	Experimento `solid_fem`: roda um caso (balanço, biapoiada, torção...) e compara flecha, tensão e taxa de torção com fórmulas fechadas e CalculiX.
`build`	função	`(case, refine=1)`	Monta malha, modelo e referências em forma fechada de um caso.

pinneapple_lab.experiments.vehicles3d

Veículos 3-D (carro e foguete) em OpenFOAM com RANS, renders no Blender e visualizador three.js. Depende de OpenFOAM e Blender. [sem teste direto]

`pinneapple_lab/experiments/vehicles3d.py` · 162 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`VehicleCFD`	classe	`()`	Experimento `vehicle_cfd`: gera o veículo, roda RANS em meio modelo e confere coeficientes contra faixas de referência ou estabilidade de Barrowman.

pinneapple_lab.experiments.wear

Desgaste por deslizamento de barras de vários materiais (`pinneapple_physics.tribology`), com Archard e pressão redistribuída. [sem teste direto]

`pinneapple_lab/experiments/wear.py` · 238 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BarWear`	classe	`()`	Experimento `bar_wear`: desliza uma barra coroada até assentar e confere volume desgastado, contato inicial e taxa estacionária.
`BarWearRanking`	classe	`()`	Experimento `bar_wear_ranking`: compara todas as execuções `bar_wear` e confere o ranking dos materiais pelo coeficiente de Archard.

pinneapple_lab.explorer

Página de experimentos: cada item com selo A-F, cartão de confiança, métricas, próximos passos, figuras, vídeos e links, servida em `/experiments` ou exportada como HTML estático.

`pinneapple_lab/explorer.py` · 387 linhas · testes: 6

nome	tipo	assinatura	o que faz
`proper`	função	`(title) -> str`	Gera um título a partir de uma chave, com siglas em maiúsculas.
`collect`	função	`(store, examples=None) -> dict[str, Any]`	Reúne todos os experimentos da página (curadoria e manifestos), contagens por letra, facetas de filtro e lista de cobertura.
`coverage`	função	`(data) -> list[dict[str, Any]]`	Tabela experimento -> curado sim/não (e letra), uma linha por entrada.
`coverage_md`	função	`(data) -> str`	Gera em Markdown o relatório de cobertura da curadoria.
`filter_entries`	função	`(entries, *, letter=None, domain=None, method=None, ready=False, q=None, sort='letter') -…)`	Aplica os filtros da página (letra, domínio, método, pronto, busca, ordenação), também usados por `/api/experiments`.
`media_urls`	função	`(data, mode='server') -> dict[str, Any]`	Preenche as URLs de mídia, miniatura e selo, como rotas do servidor ou caminhos relativos do export estático.

nome	tipo	assinatura	o que faz
render	função	`(data) -> str`	Renderiza a página com os dados embutidos no template.
media_path	função	`(lab_root, entry, item) -> str | None`	Resolve o arquivo por trás de um item de mídia (arquivo de execução ou do repositório).
stamped_jpeg	função	`(path, key, cur, width=1280) -> bytes | None`	Devolve a figura carimbada com o selo do item como JPEG de largura limitada.
seal_png	função	`(key, cur, lang='en') -> bytes`	Renderiza o selo de um item como bytes PNG.
write_static	função	`(store, out_dir, *, examples=None, media=True) -> str`	Escreve a mesma página como arquivos estáticos (`index.html` e `media/`).

pinneapple_lab.html

Catálogo navegável do Lab como uma página HTML autocontida, com dados e miniaturas embutidos.

`pinneapple_lab/html.py` · 197 linhas · testes: 14

nome	tipo	assinatura	o que faz
thumbnail_bytes	função	`(path, width=360) -> bytes | None`	Miniatura JPEG de uma imagem (primeiro quadro de GIF), ou None se não der para ler.
showcase_score	função	`(path) -> int`	Pontua o nome de uma figura por palavras-chave para escolher a mais vistosa.
collect	função	`(store, *, thumbs_per_run=1, max_runs_per_experiment=400, thumbs='embed') -> dict[str, An…)`	Reúne tudo o que o catálogo mostra, com miniaturas embutidas ou apontando para a rota `thumb/` do servidor.
write_html	função	`(store, path=None, *, standalone=True, **kw) -> str`	Escreve a página do catálogo, como documento completo ou só o corpo.
render_page	função	`(data, *, standalone=True) -> str`	Renderiza a página do catálogo com os dados de `collect` embutidos.
write_site	função	`(store, out_dir) -> str`	Escreve um catálogo público autocontido para hospedagem estática (GitHub Pages), sem caminhos locais.

pinneapple_lab.improve

Comando `improve`: imprime lacunas e ações para a próxima letra e, com `--run`, executa só as ações registradas como executáveis e recura. [sem teste direto]

`pinneapple_lab/improve.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
runnable_actions	função	`(gaps) -> list[Any]`	Filtra as ações registradas e executáveis das lacunas, sem repetição.
improve	função	`(item, store, *, run=False, lang='en', log=print) -> int`	Imprime lacunas e ações de um item; com `run=True` roda os comandos do Lab e recura; devolve 0 ou 1.

pinneapple_lab.manifest

Manifestos de experimento (`experiment.json`, formato `pinneapple.experiment/1`): conversões de execuções, soluções, comparações e fluxos, validação, varredura do repositório e portão de CI. (A marca de placeholder é falso positivo: o docstring diz que métrica nunca é placeholder.)

`pinneapple_lab/manifest.py` · 650 linhas · testes: 29 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`Manifest`	classe	`(id, title, description, domain, methods, modules, params, scripts, metrics, checks, file…)`	Dataclass do manifesto: id, título, domínio, métodos, módulos, parâmetros, scripts, métricas, checagens, arquivos e proveniência.
`save`	função	`(m, path) -> str`	Grava o manifesto em um caminho (uma pasta recebe `experiment.json`).
`load`	função	`(path) -> Manifest`	Lê o manifesto de um `experiment.json`, de uma pasta que o contém ou de uma pasta de execução do Lab.
`validate`	função	`(m, base=None) -> list[str]`	Lista os problemas de um manifesto (esquema, id, título, métricas finitas com `how`, arquivos existentes).
`from_run_dir`	função	`(d) -> Manifest`	Converte uma pasta de execução do Lab em manifesto.
`write_for_run`	função	`(d) -> str | None`	Escreve `experiment.json` numa pasta de execução terminada; nunca levanta exceção.
`from_workflow_run`	função	`(runner, run_id) -> Manifest`	Converte uma execução de `pinneapple_core.workflows` em manifesto.
`from_solution`	função	`(sol, title=None) -> Manifest`	Converte um resultado de `pp.solve` em manifesto (usado por `Solution.to_experiment`).
`from_comparison`	função	`(cmp, title=None) -> Manifest`	Converte uma tabela de `pp.compare` em manifesto, com todas as métricas de cada método.
`from_flow_run`	função	`(run, out_dir=None) -> Manifest`	Gancho do Flow Studio: converte uma execução de fluxo terminada em manifesto e, opcionalmente, grava.
`domain_of`	função	`(name, tags=()) -> str`	Deduz o domínio (por padrões de texto) a partir do nome e das tags; padrão `general`.
`methods_of`	função	`(text) -> list[str]`	Lista rótulos de métodos cujos nomes aparecem no texto, como faceta de filtro.
`manifest_from_dir`	função	`(folder, lab_curation=None) -> Manifest`	Monta um manifesto com o que uma pasta de exemplo já contém (README, docstring, imports, figuras, resultados).
`lab_items_for`	função	`(rel, cur) -> list[str]`	Lista os itens do Lab que avaliam uma pasta de exemplo, melhor letra primeiro.
`from_results`	função	`(folder, rules=None) -> Manifest`	Manifesto da pasta acrescido das métricas e checagens extraídas dos resultados versionados por `results_rules`.
`example_dirs`	função	`(root=None) -> list[str]`	Lista as pastas de `examples/*` que precisam ter manifesto.

nome	tipo	assinatura	o que faz
`scan`	função	`(roots=None, *, lab=None) -> list[tuple[str, Manifest]]`	Encontra todos os manifestos sob as raízes dadas e todas as execuções do Lab.
`check_repo`	função	`(examples=None, lab=None) -> list[str]`	Portão de CI: uma linha por problema nos manifestos dos exemplos e nas execuções do Lab; vazio se tudo ok.
`scaffold`	função	`(name, root=None, *, title='', domain='') -> str`	Cria `examples/<name>/` com `run.py` ligado a `pinneapple_lab.experiment`, README e manifesto inicial; recusa pasta existente.

`pinneapple_lab.next_steps`

Registro de tipos de lacuna e de ações: diz o que falta para a próxima letra e qual ação do PINNeAPPle fecha cada lacuna.

`pinneapple_lab/next_steps.py` · 399 linhas · testes: 31

nome	tipo	assinatura	o que faz
`Action`	classe	`(id, kind, target, command, text, text_pt, params, runnable, argv)`	Ação sugerida: chamada Python ou comando CLI, com experimento e parâmetros preenchidos e textos em inglês e português.
`Gap`	classe	`(type, to, stage, why, why_pt, short, short_pt, check, measured, actions)`	Lacuna entre o item e a próxima letra: tipo, estágio do cartão de confiança, porquê, checagem relacionada e ações.
`measured`	função	`(check) -> dict[str, Any] | None`	Extrai o valor medido de uma checagem e seu limite, ou None se não houver número.
`measured_text`	função	`(m, lang='en') -> str`	Formata o valor medido contra o limite como texto em inglês ou português.
`context_for`	função	`(experiment, params, run_dir, tags, is_ml, cls_params) -> dict[str, Any]`	Monta o contexto (resolução, semente, se é ML etc.) usado para preencher as ações de um item.
`make_action`	função	`(aid, ctx) -> Action | None`	Preenche uma ação registrada para o item, ou None se a condição dela não vale.
`make_gap`	função	`(gtype, to, ctx, *, check=None, why=None, meas=None) -> Gap`	Cria uma lacuna de tipo registrado com as ações preenchidas; `KeyError` para tipo desconhecido.
`group`	função	`(gaps) -> list[dict[str, Any]]`	Agrupa lacunas que compartilham as mesmas ações, na ordem em que aparecem.
`explain`	função	`(gaps, lang='en', title='') -> str`	Escreve as lacunas como texto, com porquê, valor medido e ações agrupadas.

`pinneapple_lab.reports`

Relatórios do Lab (HTML, Markdown ou PDF) de um item ou de vários filtrados, tudo tirado do registro e da curadoria. PDF precisa de `reportlab`.

`pinneapple_lab/reports.py` · 428 linhas · testes: 20

nome	tipo	assinatura	o que faz
`select`	função	`(cur, *, items=None, experiment=None, tier=None, use=None, ready=None, tag=None, tags_of=…)`	Seleciona chaves de itens da curadoria pelos filtros; `use` ordena por distância a esse uso.

nome	tipo	assinatura	o que faz
`gather`	função	`(store, cur, key, n_figures=3) -> dict[str, Any]`	Junta tudo o que o relatório diz de um item.
`render_html`	função	`(store, groups, title, filters='') -> str`	Renderiza o relatório como HTML.
`render_md`	função	`(groups, title, filters='') -> str`	Renderiza o relatório como Markdown com tabela de itens.
`render_pdf`	função	`(groups, title, filters='') -> bytes`	Renderiza o relatório como PDF com `reportlab`.
`write_report`	função	`(store, out, *, keys=None, title=None, fmt=None, filters='', cur=None, **select_kw) -> str`	Seleciona os itens e escreve o relatório no formato indicado pela extensão ou por `fmt`.

`pinneapple_lab.results_rules`

Regras (`Metric`, `Flag`, `Bound`, `CheckList`) que transformam resultados versionados de um exemplo em métricas e checagens de manifesto, com limites citados como `arquivo:linha`. (A marca de placeholder é falso positivo.)

`pinneapple_lab/results_rules.py` · 219 linhas · testes: 14 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`Metric`	classe	`(file, path, name, unit, how)`	Regra que lê uma métrica de um arquivo de resultados por caminho, com nome, unidade e `how`.
`Flag`	classe	`(file, path, name, kind, detail, invert)`	Regra de checagem a partir de um veredito booleano que o próprio estudo gravou.
`Bound`	classe	`(file, path, name, kind, literal, source, op, absolute, unit)`	Regra que compara um valor medido com o limite de aceitação citado no teste do estudo (`arquivo:linha`).
`CheckList`	classe	`(file, path, name, kind, value_key, reference_key, tolerance_key, limit_key, passed_key)`	Regra que lê uma lista de registros de checagem gravados pelo estudo (valor, referência, tolerância ou limite).
`extract`	função	`(folder, rules) -> tuple[list[dict], list[dict]]`	Aplica as regras e devolve (métricas, checagens); regra sem arquivo ou caminho não gera nada.
`check_citations`	função	`(repo) -> list[str]`	Confere que cada literal de `Bound` aparece na linha citada do arquivo citado.

`pinneapple_lab.runner`

Executa um experimento ou varreduras por grade ou amostragem (hipercubo latino), com cache por id e paralelismo por processos. [sem teste direto; coberto pelos testes do pacote]

`pinneapple_lab/runner.py` · 243 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`RunResult`	classe	`(run_id, experiment, status, dir, metrics, seconds, error)`	Resultado de uma execução: id, experimento, status (inclusive `cached:<status>`), pasta, métricas, tempo e erro.

nome	tipo	assinatura	o que faz
`run`	função	`(experiment, params=None, *, root=None, force=False, tags=None, raise_errors=False) -> Ru…)`	Roda um experimento com parâmetros sobre os padrões; não repete execução já concluída, salvo `force`.
`sweep`	função	`(experiment, *, grid=None, samples=None, space=None, fixed=None, method='lhs', seed=0, n_…)`	Roda um experimento sobre uma grade e/ou amostras aleatórias do espaço, opcionalmente em paralelo (`n_jobs`).

`pinneapple_lab.seal`

Selo de qualidade PINNeAPPle: badge com a letra dada pela curadoria (nunca escolhida à mão), carimbado em figuras, renders, GIFs e vídeos. Vídeos usam ffmpeg.

`pinneapple_lab/seal.py` · 829 linhas · testes: 39

nome	tipo	assinatura	o que faz
`SealError`	classe	`()`	Erro quando o item não pode receber selo porque sua letra está fora das letras pedidas.
`Seal`	classe	`(item, tier, title, reason, reason_kind, curated, experiment, run_id, hint, reason_pt, hi…)`	Dados de um selo: item, letra, título, motivo (manchete da curadoria), tipo do motivo e execução.
`load_curation`	função	`(lab=None) -> dict[str, Any]`	Carrega o registro de curadoria de uma pasta, caminho, `LabStore`, dict ou do Lab padrão.
`parse_tiers`	função	`(tiers) -> tuple[str, ...]`	Converte `"A,B"`, lista ou None numa tupla de letras válidas.
`items`	função	`(lab=None, tiers=SEALABLE) -> list[str]`	Lista as chaves de itens curados com uma das letras pedidas.
`seal_for`	função	`(item, lab=None, tiers=None) -> Seal`	Monta o selo de um item curado; `KeyError` se desconhecido e `SealError` se a letra estiver fora do filtro.
`run_dir`	função	`(seal, lab=None) -> str`	Devolve a pasta da execução a que o selo se refere.
`draw_text`	função	`(draw, xy, text, font, fill, anchor='ls') -> None`	Desenha texto com uma marca inicial (✓, ◐, ✗ ou ↑) na fonte DejaVu Sans.
`render_seal`	função	`(seal, *, height=96, fmt='png', reason=True, compact=None, opacity=1.0, lang='en', next_s…)`	Renderiza o badge como imagem PIL RGBA ou SVG, com forma compacta em tamanhos pequenos e idioma en/pt.
`contrast_ratio`	função	`(fg, bg=BADGE_BG) -> float`	Razão de contraste WCAG 2 entre duas cores `#rrggbb`.
`save_seal`	função	`(seal, path, **kw) -> str`	Grava o badge como SVG ou imagem raster, conforme a extensão.
`pick_corner`	função	`(frames, w, h, margin, avoid=()) -> str`	Escolhe o canto onde o badge cobre menos conteúdo, evitando caixas proibidas.
`corner_scores`	função	`(frames, w, h, margin, avoid=()) -> dict[str, float]`	Mede o quanto cada canto utilizável está ocupado.

nome	tipo	assinatura	o que faz
`stamp_image`	função	`(image, seal, *, corner='auto', size=0.24, opacity=0.95, margin=0.02, reason=True, avoid=…)`	Carimba uma imagem PIL, array ou arquivo, no canto mais vazio ou numa faixa extra se todos estiverem ocupados.
`stamp_figure`	função	`(fig, seal, *, corner='auto', size=0.24, opacity=0.95, margin=0.015, reason=True, extend=…)`	Carimba uma figura matplotlib no lugar, evitando barras de cor.
`sealed_savefig`	função	`(seal, **stamp_kw)`	Context manager em que todo `Figure.savefig` carimba a figura antes de salvar.
`stamp_video`	função	`(src, seal, out, *, corner='auto', size=0.24, opacity=0.95, margin=0.02, reason=True, ext…)`	Carimba um GIF quadro a quadro ou um MP4/WebM/MOV via overlay do ffmpeg.
`stamp_file`	função	`(path, seal, out=None, **kw) -> str`	Carimba um arquivo de imagem, GIF ou vídeo e devolve o caminho de saída; recusa SVG e PDF.
`stamp_all`	função	`(lab=None, out_dir='export/sealed', *, tiers=SEALABLE, only=None, lang='en', **kw) -> dic…)`	Modo lote: carimba todas as figuras e vídeos dos itens nas letras pedidas, com selos SVG/PNG e manifesto.
`legend_seals`	função	`(lang='en') -> list[Seal]`	Um selo de exemplo por letra, com a linha que explica o significado da letra.
`render_legend`	função	`(lang='en', height=72)`	Renderiza a legenda dos seis selos, A a F, em fundo claro e escuro.

`pinneapple_lab.server`

Servidor HTTP só com biblioteca padrão: página do catálogo, arquivos das execuções, API JSON, miniaturas sob demanda e downloads de datasets.

`pinneapple_lab/server.py` · 382 linhas · testes: 26

nome	tipo	assinatura	o que faz
`LabServer`	classe	`(root=None, *, refresh=30.0, user=None, password=None)`	Catálogo e arquivos de uma pasta do Lab; `handler()` dá a classe que atende as requisições.
`serve`	função	`(root=None, *, host='127.0.0.1', port=8093, refresh=30.0) -> None`	Serve o catálogo até ser interrompido.

`pinneapple_lab.showcase`

Painel animado 1080 x 1350 com os melhores resultados do Lab (destaque, grade de itens A e fileira de B), gerado da curadoria como MP4, GIF e PNG.

`pinneapple_lab/showcase.py` · 589 linhas · testes: 3

nome	tipo	assinatura	o que faz
`Tile`	classe	`(item, tier, figure, title, evidence, evidence_kind, seal, frame, lang)`	Um bloco do painel: item, letra, figura, título, linha de evidência e selo.
`evidence_line`	função	`(rec, lang='en', short=False) -> str`	Escolhe a linha de evidência do bloco a partir da manchete da curadoria, no idioma pedido.

nome	tipo	assinatura	o que faz
choose_tiles	função	(lab=None, picks=None, lang='en', n_grid=9, n_b=2) -> dict[str, Any]	Escolhe destaque, grade (A) e fileira B a partir da curadoria e das escolhas manuais picks.
fit_crop	função	(img, aspect, *, photo=None) -> tuple[tuple[int, int, int, int], bool]	Calcula o recorte de uma imagem para caber no bloco e se precisa de tarjas.
panel	função	(lab=None, *, lang='en', out_dir='docs/assets/showcase', picks=None, seconds=8.0, fps=15,…)	Renderiza o painel e devolve os arquivos escritos e os blocos usados.

pinneapple_lab.spec

Classe base Experiment e registro por nome; Experiment.run da base levanta NotImplementedError (deve ser sobrescrito). [sem teste direto]

pinneapple_lab/spec.py · 92 linhas · **sem teste direto** · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
Experiment	classe	(name, version, description, tags, params, space, code_files, references, limitations, ca…)	Classe base de experimento: defina name, params e run(ctx); a versão entra no id da execução.
register	função	(cls) -> type[Experiment]	Decorador de classe que registra o experimento por nome para o runner e a CLI.
get	função	(name) -> type[Experiment]	Busca um experimento pelo nome, carregando os embutidos ou importando modulo:Classe.
availble	função	() -> list[str]	Lista os nomes de todos os experimentos registrados.

pinneapple_lab.store

Banco do Lab: pastas runs/<experimento>/<run_id> mais índice index.sqlite, consultas, tabelas, exportação de datasets e catálogos. [sem teste direto; coberto pelos testes do pacote]

pinneapple_lab/store.py · 285 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
LabStore	classe	(root=None)	Banco de experimentos numa raiz: indexa execuções, consulta, monta tabelas, lista e exporta datasets e escreve catálogos.
default_root	função	() -> str	Raiz padrão do Lab: $PINNEAPPLE_LAB ou ./lab.

pinneapple_lab.uq

Checagens de incerteza para o cartão de confiança: cobertura de intervalos preditivos, índice de convergência de malha (GCI) e espalhamento de ensemble.

pinneapple_lab/uq.py · 95 linhas · testes: 20

nome	tipo	assinatura	o que faz
coverage	função	(y_true, mean, std, level=0.9) -> float	Fração dos valores verdadeiros dentro do intervalo gaussiano central de nível dado.

nome	tipo	assinatura	o que faz
coverage_check	função	`(ctx, name, y_true, mean, std, level=0.9, tol=0.07) -> float`	Grava a cobertura empírica em dados não vistos e checa se fica perto do nível nominal.
gci	função	`(values, h, order=None, safety=None) -> dict`	Índice de convergência de malha do valor mais fino, com ordem observada para três malhas (Celik et al. 2008).
gci_check	função	`(ctx, name, values, h, order=None, max_band=0.05) -> dict`	Grava a faixa GCI e o valor extrapolado de Richardson e checa se a faixa fica abaixo de `max_band`.
ensemble_spread	função	`(values) -> float`	Desvio padrão relativo de resultados repetidos.

Exemplos de uso reais do repositório

Executar, reaproveitar o cache e varrer parâmetros — `tests/test_lab.py`

```
        def test_run_records_everything_and_is_cached(tmp_path):
            root = str(tmp_path)
            r = run("_test_square", {"a": 2.0}, root=root)
            assert r.status == "completed" and r.metrics["y_max"] == 2.0
            ...
            again = run("_test_square", {"a": 2.0}, root=root)
            assert again.status == "cached:completed" and again.run_id ==
r.run_id
```

Embrulhar um script de exemplo com `pinneapple_lab.experiment` — `examples/chain_of_whys/chain_of_whys.py`

```
            with pinneapple_lab.experiment(
                    "examples/chain_of_whys", title="Chain of whys",
                    description="Engineering reasoning from physics: naive
concept -> computed failure mode -> countermeasure "
                                "-> check again, for a roof, a footbridge, a
power board and a pumped main.",
                    domain="design and geometry", methods=["closed-form
checks", "FD beam", "3-D board FV", "MOC"],
                    modules=["pinneapple_reason"], out_dir=HERE,
register=False,
                    links={"guide": "docs/guides/chain_of_whys.md",
"script": "examples/chain_of_whys/chain_of_whys.py"}) as ex:
                for name in a.case or list(cases.CASES):
                    ...
                    ex.check(f"{name}: every requirement passes on the final
version", chain.resolved, kind="physics",
                             detail=" -> ".join(f"{m}:{c}" for _, m, c in
chain.path()))
```

Varredura paralela e linha de comando — `tests/test_lab.py`

```
            res = sweep("oscillator", grid={"zeta": [0.1, 0.4], "method":
["rk4", "symplectic"]}, n_jobs=2, root=root)
            assert all(r.ok for r in res)
            assert cli(["--root", root, "status"]) == 0
            assert cli(["--root", root, "run", "oscillator", "-p",
"zeta=0.3"]) == 0
            assert cli(["--root", root, "report"]) == 0
```

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 15 de 42.
- Funções de teste em arquivos que importam algum módulo do pacote: 96.

Limitações conhecidas

- Vários experimentos embutidos dependem de ferramentas externas: CalculiX e gmsh (`calculix_case`), OpenFOAM (`pipe_flow`, `vehicle_cfd`), Blender (renders de `vehicle_cfd`) e ffmpeg (carimbo de vídeos e `showcase`).
- `bh_forecast`, `example_script` e `repo_results` leem arquivos do repositório e só funcionam num checkout do código-fonte; `benchmark_case` precisa de repositórios companheiros (variável de ambiente, pasta irmã ou clone raso).
- A maioria dos módulos de `pinneapple_lab.experiments` não tem teste direto; `runner`, `store`, `context` e `dataset` são exercitados pelos testes que importam o pacote (`tests/test_lab.py`), não por testes próprios.
- A curadoria só classifica a evidência que a execução gravou: um experimento sem checagens fica no máximo em D, e as letras A exigem revisão humana para virar produto, artigo ou marketing.
- `Experiment.run` da classe base levanta `NotImplementedError`; cada experimento precisa sobrescrevê-lo.
- Relatórios em PDF precisam de `reportlab`; selos e miniaturas usam Pillow e matplotlib.
- A curadoria reutiliza o vocabulário de `pinneapple_veriphysics.applicability`, importado de forma opcional.

6. `pinneapple_llm`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
49	10.750	66	99	242 (20 arquivos)	28

Pasta de testes dedicada: `tests/pinneapple_llm` (4 funções de teste).

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`_dispatch`		1	1	0
`agent_loop`		1	3	15
`cad_draft`		1	7	22
`catalog_match`		1	2	4
`conversation_store`		1	2	0
`draft`		1	5	23
`finetune`		1	3	0

submódulo	o que faz	módulos	classes + funções	testes*
geometry_draft		1	2	0
guardrail		1	3	60
local_llm		1	3	0
org_knowledge		1	2	4
qa		23	80	48
research		1	2	0
twin_draft		1	3	0
workbench		12	47	37

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

pinneapple_llm

LLM-assisted physics-AI pipeline drafting, gated by a physics-grounded verification layer.

pinneapple_llm/__init__.py · 61 linhas · testes: 128

pinneapple_llm._dispatch

Provider-agnostic LLM call dispatch, shared by `draft.py`, `geometry_draft.py`, `research.py` and `twin_draft.py` -- one place that knows how to reach Anthropic, OpenAI, or a local Ollama server, and one place that logs every call to a :class:`ConversationStore` when one is configured (see ...

pinneapple_llm/_dispatch.py · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
call_llm	função	(prompt, *, provider='anthropic', model=None, api_key=None, system='', json_mode=False, m…)	Dispatch a single prompt to whichever provider is requested and return the raw text response. See individual provider modules for what **provider_kwargs accepts (host/port/timeout for "ollama"; base_url for "openai" / "openai_compatible", i.e. any server speaking the ...

pinneapple_llm.agent_loop

A genuine multi-step LLM agent loop: the LLM itself decides, turn by turn, which real tool to call next, observes the result, and decides the next step -- grounded in :func:`pinneapple_llm._dispatch.call_llm` (a real, single-call LLM dispatch across anthropic/openai/ollama) plus a real tool ...

pinneapple_llm/agent_loop.py · 331 linhas · testes: 15

nome	tipo	assinatura	o que faz
AgentStep	classe	(tool_name, tool_args, tool_result, error, llm_reasoning)	One turn of the loop: the tool the LLM chose (or None for a final-answer / error turn that called no tool), the arguments it supplied, the real result of invoking that tool (or error set instead when invocation failed), and the LLM's stated reasoning for the turn, if the provider/prompt ...

nome	tipo	assinatura	o que faz
`AgentLoopResult`	classe	`(steps, final_answer, stopped_reason)`	Full trace of a :func:`run_agent_loop` run.
`run_agent_loop`	função	`(goal, tool_registry=None, *, llm_provider='anthropic', model=None, max_steps=8, llm_call…)`	Run a genuine multi-step agent loop toward *goal*.

`pinneapple_llm.cad_draft`

LLM-assisted CAD/geometry generation -- the same constrained pattern as `draft.py`'s `draft_problem` and `geometry_draft.py`'s `draft_geometry`, extended to genuinely NEW (not just named-catalog) geometry: instead of picking one fixed preset name, the LLM composes a recipe out of …

`pinneapple_llm/cad_draft.py` · 633 linhas · testes: 22

nome	tipo	assinatura	o que faz
`CadRecipeResult`	classe	`(kind, recipe, name, params, reasoning, raw_response)`	
`draft_mesh_recipe`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Ask an LLM to compose a (possibly CSG-combined) geometry recipe for a natural-language description, e.g. "a box with a cylindrical hole through it" or "two overlapping spheres fused together".
`draft_cadquery_template`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Ask an LLM to pick + parametrise a genuinely parametric CAD template (real B-rep solid, true STEP-exportable) for a natural-language description, e.g. "a cold plate with an internal cooling channel" or "a heat sink plate with 12 fins".
`build_recipe`	função	`(result)`	Actually build the geometry a :class:`CadRecipeResult` describes.
`export_recipe`	função	`(result, out_path, *, fmt=None) -> str`	Export a built recipe to a real CAD/mesh file on disk.
`measure_part`	função	`(result) -> Dict[str, Any]`	Build a `"mesh_recipe"` and measure it: volume, bounding-box extents, watertightness, and whether a boolean engine was available. Without one (`manifold3d`, extra `geometry`) `build_mesh` replaces every cut/union by a concatenation of the parts (a UserWarning only), so a washer comes out as …
`draft_part`	função	`(description, *, tier='quick', provider='ollama', model=None, api_key=None, max_attempts=…)`	Draft a buildable part, retrying refused drafts: `tier="quick"` asks :func:`draft_mesh_recipe` (CSG of primitives), `tier="engineering"` asks :func:`draft_cadquery_template` (parametric CadQuery template).

pinneapple_llm.catalog_match

pinneapple_llm.catalog_match — `resolve_problem()`: a two-stage, never-raises front end to `draft_problem()`.

`pinneapple_llm/catalog_match.py` · 128 linhas · testes: 4

nome	tipo	assinatura	o que faz
`ResolveResult`	classe	`(preset, kwargs, reasoning, used_llm, shortlist)`	
`resolve_problem`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None,…)`	Resolve `description` to a registered PDE preset — the shortlist step always runs; the LLM step is attempted next and its result used if it succeeds, else the top keyword match is returned instead. Never raises.

pinneapple_llm.conversation_store

SQLite-backed conversation log for every `pinneapple_llm` LLM call.

`pinneapple_llm/conversation_store.py` · 120 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ConversationRecord`	classe	`(id, timestamp, module, provider, model, system_prompt, user_prompt, response, metadata)`	
`ConversationStore`	classe	`(db_path=_DEFAULT_DB_PATH)`	Thin SQLite wrapper. Safe to construct repeatedly (opens/creates the DB file each time); not held open across calls, so it's safe to share a path across processes/threads without extra locking beyond what SQLite itself provides.

pinneapple_llm.draft

LLM-assisted `ProblemSpec` drafting -- deliberately constrained to selecting and parametrising one of PINNeAPPle's own already-implemented, already-testable presets, never to inventing new PDE machinery from scratch.

`pinneapple_llm/draft.py` · 231 linhas · testes: 23

nome	tipo	assinatura	o que faz
`DraftResult`	classe	`(preset, kwargs, reasoning, raw_response)`	
`DraftRejected`	classe	`(message, attempts)`	No acceptable draft after every attempt; `.attempts` lists why each one was refused.

nome	tipo	assinatura	o que faz
`draft_problem`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Ask an LLM to pick + parametrise a `ProblemSpec` preset for a natural-language problem description.
`check_kwargs_in_bounds`	função	`(kwargs, bounds) -> Dict[str, float]`	Coerce drafted kwargs to finite floats and check each one against `bounds` ({name: (lo, hi)}).
`draft_preset_kwargs`	função	`(description, expected_preset, *, bounds=None, provider='ollama', model=None, api_key=Non…)`	Draft kwargs for ONE known preset: retry `draft_problem` until it picks `expected_preset` with kwargs that pass :func:`check_kwargs_in_bounds`. Returns `(kwargs, reasoning, attempts_used)`; raises :class:`DraftRejected` after `max_attempts` refusals (a wrong preset, no fit, a hallucinated …

pinneapple_llm.finetune

LoRA fine-tuning of a local causal LM on data logged via `conversation_store.ConversationStore`.

`pinneapple_llm/finetune.py` · 116 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`FineTuneConfig`	classe	`(base_model, output_dir, lora_r, lora_alpha, lora_dropout, learning_rate, num_train_epoch…)`	
`prepare_dataset`	função	`(store, out_jsonl, *, module=None) -> str`	Export logged (prompt, response) pairs to a JSONL file, ready for :func:`finetune_lora`. Thin wrapper over `ConversationStore .export_jsonl` -- kept as a separate step (not folded into `finetune_lora`) so the dataset can be inspected/filtered/versioned before spending any GPU time on it.
`finetune_lora`	função	`(dataset_jsonl, cfg)`	Run a real LoRA fine-tune (`transformers.Trainer` + `peft .LoraConfig`) of `cfg.base_model` on the (prompt, response) pairs in `dataset_jsonl` (as written by :func:`prepare_dataset`).

pinneapple_llm.geometry_draft

LLM-assisted geometry drafting -- same constrained pattern as `draft.py`'s `draft_problem`, applied to CAD/domain generation instead of PDE problem specs: the LLM selects and parametrises one of `pinneapple_design.geometry`'s own already-implemented generators …

`pinneapple_llm/geometry_draft.py` · 121 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeometryDraftResult`	classe	`(generator, kind, kwargs, reasoning, raw_response)`	

nome	tipo	assinatura	o que faz
`draft_geometry`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Ask an LLM to pick + parametrise a geometry generator for a natural-language description (e.g. "a NACA 2412 airfoil" or "a lid driven cavity in 3D").

`pinneapple_llm.guardrail`

`PhysicsGuardrail`: the actual anti-hallucination mechanism.

`pinneapple_llm/guardrail.py` · 1.069 linhas · testes: 60

nome	tipo	assinatura	o que faz
`CheckResult`	classe	`(name, passed, detail, value, threshold)`	
`GuardrailReport`	classe	`(checks)`	
`PhysicsGuardrail`	classe	`(spec, *, residual_threshold=0.01, n_check_points=4096, conservation_n_points_per_face=20…)`	Runs a fixed sequence of independent checks against a trained model + the `ProblemSpec` it was trained on.

`pinneapple_llm.local_llm`

Local LLM provider (Ollama) -- so `pinneapple_llm` doesn't require a paid API/external network call at all: run a model entirely on your own machine, no data leaves it.

`pinneapple_llm/local_llm.py` · 119 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`start_server`	função	`(*, host='127.0.0.1', port=11434, wait_s=10.0) -> subprocess.Popen`	Launch `ollama serve` as a background subprocess (no-op / raises if `ollama` is not installed -- there is no bundled fallback, same reasoning as every other "requires the real local tool" bridge in this package).
`ensure_model_pulled`	função	`(model, *, host='127.0.0.1', port=11434, timeout=3600) -> None`	`ollama pull <model>` if it isn't already present locally. Blocks until the pull completes (can be slow for a large model the first time -- `timeout` bounds it).
`call_ollama`	função	`(prompt, model, api_key=None, system='', *, host='127.0.0.1', port=11434, timeout=300, js…)`	Chat-completion call against a local Ollama server. Raises a clear error (not a generic connection-refused traceback) if no server is reachable -- call :func:`start_server` first, or run `ollama serve` yourself.

`pinneapple_llm.org_knowledge`

Grounded LLM answers over a real local :class:`pinneapple_analysis.retrieval.RetrievalIndex` -- the "local LLM that knows about the organisation's own code" building block, built for (and by) `org-management`'s knowledge-base requirement, but placed here rather than in that downstream repo for …

`pinneapple_llm/org_knowledge.py` · 133 linhas · testes: 4

nome	tipo	assinatura	o que faz
`OrgAnswer`	classe	`(question, context_used, answer)`	
`answer_with_context`	função	`(question, retrieval_index, *, top_k=5, provider='ollama', model='llama3.2:3b', api_key=N…)`	Answer *question* grounded in *retrieval_index*'s real search results (any object exposing a `search(query, top_k=...) -> List[SearchResult]` method -- i.e. a real `pinneapple_analysis.retrieval.RetrievalIndex`, matching that class's own interface).

pinneapple_llm.qa

Scientific and industrial quality assurance for `pinneapple_llm` artifacts.

`pinneapple_llm/qa/__init__.py` · 51 linhas · testes: 48

pinneapple_llm.qa.examples

Runnable reference examples for the quality pipeline.

`pinneapple_llm/qa/examples/__init__.py` · 10 linhas · testes: 39

pinneapple_llm.qa.examples.pipe_wear_twin

End-to-end reference example: a 3D digital twin of a pipe under wall wear.

`pinneapple_llm/qa/examples/pipe_wear_twin.py` · 342 linhas · testes: 39

nome	tipo	assinatura	o que faz
`hollow_pipe_mesh`	função	`(*, radius=OUTER_DIAMETER_M / 2, thickness=WALL_THICKNESS_M, length=LENGTH_M, n_theta=64)…)`	A genuinely hollow pipe: outer wall, inner wall and two end annuli.
`solid_pipe_mesh`	função	`(*, radius=OUTER_DIAMETER_M / 2, length=LENGTH_M, n_theta=64) -> Tuple[np.ndarray, np.nda…)`	A filled cylinder of the same outer dimensions.
`build_spec`	função	`(*, anomaly=False) -> PhysicalSpecification`	
`build_twin`	função	`(*, anomaly=False) -> Dict[str, Any]`	
`export_twin`	função	`(bundle, output_dir=None) -> str`	Write the real viewer files and the hub contract; returns `scene.json`.
`analytical_reference`	função	`(bundle) -> ReferenceData`	Closed-form uniform-wear residual thickness, per cell and per time.
`run_example`	função	`(*, output_dir=None, anomaly=False, write_report=True) -> Tuple[QualityReport, Dict[str, …)`	Run stages A-G over the synthetic pipe twin and return the report.

pinneapple_llm.qa.examples.pipe_wear_twin_main

Run the reference example and print its quality report::

`pinneapple_llm/qa/examples/pipe_wear_twin_main.py` · 37 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`main`	função	`(argv=None) -> int`	

pinneapple_llm.qa.generation

Stage B informed: the LLM's structured claims, and why they are not evidence.

`pinneapple_llm/qa/generation.py` · 240 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`GeneratedArtifact`	classe	`(code, assumptions, dependencies, units, equations, limitations, provider, model, elapsed…)`	The LLM's proposal plus its structured claims. Untrusted by construction.
`build_prompt`	função	`(spec) -> str`	A prompt carrying the specification, not a paraphrase of it.
`parse_generation_response`	função	`(raw, *, provider='', model='', elapsed_s=0.0) -> Tuple[GeneratedArtifact, StaticResult]`	Parse and schema-check an LLM response. Never raises on malformed JSON.
`check_claims`	função	`(artifact, spec) -> List[Finding]`	Confront the model's declarations with the specification and the unit table.
`validate_generated`	função	`(artifact, spec, *, filename='artifact.py') -> Tuple[List[Finding], Any]`	Claim checks + the full static validation of the returned code.
`generate_artifact`	função	`(spec, *, provider='anthropic', model=None, api_key=None, conversation_store=None) -> Gen…)`	Ask a provider for an artifact + envelope, through the existing dispatch.

pinneapple_llm.qa.levels

Quality levels Q0-Q4, awarded from what actually ran -- never from an LLM.

`pinneapple_llm/qa/levels.py` · 182 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`evaluate_level`	função	`(report, spec=None) -> Tuple[str, List[str]]`	Return `(level, reasons)` -- the reasons explain why it stopped there.
`blocking_reasons`	função	`(report) -> List[str]`	Human-readable reasons the report cannot be treated as approved.
`level_description`	função	`(level) -> str`	

pinneapple_llm.qa.pipeline

The quality pipeline: stages A-G wired into one reproducible run.

`pinneapple_llm/qa/pipeline.py` · 455 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PipelineConfig`	classe	`(validators, run_artifacts, source_name, static_policy, repair_attempts)`	
`QualityPipeline`	classe	`(spec, *, validators=None, sandbox=None, run_artifacts=False, static_policy=None, referen…)`	

pinneapple_llm.qa.references

Stage F: comparison against independent reference data.

`pinneapple_llm/qa/references.py` · 202 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
ReferenceData	classe	(name, origin, version, values, unit, limitations, used_for_calibration)	A reference the artifact is compared against -- never invented here.
compare	função	(predictions, reference, criteria, *, subset='validation', groups=None, baseline=None) ->…)	Compare predictions with reference.values under criteria.
split_mask	função	(n, holdout_fraction=0.3, seed=0) -> np.ndarray	A deterministic boolean mask marking the validation (holdout) subset.

pinneapple_llm.qa.repair

Stage G: structured diagnosis and bounded automatic repair.

pinneapple_llm/qa/repair.py · 296 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Diagnosis	classe	(failed_steps, findings, confirmed_cause, probable_cause, constraints_to_preserve, tests_…)	Why an attempt was rejected, in the form a repair step can consume.
RepairAttempt	classe	(attempt, accepted, status, diagnosis, diff, notes, duration_s)	
RepairOutcome	classe	(accepted, artifact, attempts, results, stopped_because)	
RepairLoop	classe	(*, evaluate, repair, spec=None, required=(), max_attempts=DEFAULT_MAX_ATTEMPTS, on_attem…)	Bounded generate -> validate -> diagnose -> repair loop.
diagnose	função	(results, *, required=()) -> Diagnosis	Turn failing validator results into one structured diagnosis.

pinneapple_llm.qa.report

Reproducible quality report -- the machine-readable output of every run.

pinneapple_llm/qa/report.py · 327 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Status	classe	()	
Finding	classe	(id, status, message, severity, evidence, justification, limitations)	One observed fact, with the evidence that produced it.
StepOutcome	classe	(step, status, findings, duration_s, metadata)	The result of one pipeline stage.
ArtifactRecord	classe	(name, kind, path, sha256, size_bytes, metadata)	
QualityReport	classe	(run_id, created_at, spec_id, spec_hash, criteria_hash, provider, model, steps, artifacts…)	Everything one pipeline run observed, and what it is allowed to claim.
environment_snapshot	função	() -> Dict[str, Any]	Versions and platform, so a report can be traced to an environment.

pinneapple_llm.qa.sandbox

Stage D: controlled execution of generated artifacts.

`pinneapple_llm/qa/sandbox.py` · 389 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`IsolationLevel`	classe	`()`	
`SandboxLimits`	classe	`(timeout_s, cpu_s, memory_mb, file_size_mb, open_files, max_processes, write_bytes)`	Resource ceilings. `None` disables the corresponding limit.
`SandboxResult`	classe	`(status, isolation, workdir, exit_code, stdout, stderr, duration_s, timed_out, violations…)`	
`SandboxExecutor`	classe	`(*, limits=None, allow_execution=False, allow_network=False, base_dir=None, keep_workdir=…)`	Run a generated Python artifact in a separate, limited process.
`hash_bytes`	função	`(blob) -> str`	

pinneapple_llm.qa.spec

Typed, serializable physical specification -- stage A of the quality pipeline.

`pinneapple_llm/qa/spec.py` · 496 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Provenance`	classe	`()`	
`SpecError`	classe	`()`	Raised when a specification is structurally invalid.
`Quantity`	classe	`(value, unit, provenance, source, uncertainty)`	A number with a unit, a provenance tag, and (optionally) an uncertainty.
`Invariant`	classe	`(name, kind, target, params, severity, description)`	A physical constraint the artifact must satisfy. Declarative, not code.
`Observation`	classe	`(sensor_id, quantity, unit, position, timestamps, time_unit, values, provenance, holdout)`	A sensor reading: where, what, when, and whether it may be used for validation.
`GeometrySpec`	classe	`(kind, coordinate_system, dimensions, hollow, wall_thickness, notes)`	Geometry, domain and coordinate system, with units made explicit.
`PhysicalSpecification`	classe	`(objective, phenomenon, geometry, independent_variables, dependent_variables, parameters,…)`	Stage A output: what the user asked for, expressed so it can be checked.
`quantity_map`	função	`(**kwargs) -> Dict[str, Quantity]`	`quantity_map(nu=(1e-6, "m^2/s"))` -- small construction helper.

pinneapple_llm.qa.static

Stage C: static and structural validation of generated artifacts.

`pinneapple_llm/qa/static.py` · 596 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
StaticPolicy	classe	(allowed_roots, forbidden_roots, forbidden_calls, forbidden_attr_calls, allowed_write_roo…)	
StaticResult	classe	(findings, resolved_imports)	
check_syntax	função	(source, filename='<artifact>') -> List[Finding]	Compile-only check. A SyntaxError is reported, never raised.
check_imports	função	(source, policy) -> Tuple[List[Finding], Dict[str, Any]]	Refuse forbidden imports; report unknown/unresolvable ones with evidence.
check_api_contracts	função	(source, resolved) -> List[Finding]	Resolve every pinneapple_* attribute the artifact touches, for real.
check_signatures	função	(source, resolved) -> List[Finding]	Compare literal call sites against the real inspect.signature.
check_security	função	(source, policy, write_roots=()) -> List[Finding]	Refuse process spawning, code execution, network and out-of-scope writes.
check_json_artifact	função	(data, schema=None, name='artifact') -> List[Finding]	Validate a JSON artifact against a small, explicit schema subset.
validate_python_artifact	função	(source, *, policy=None, filename='<artifact>', write_roots=(), json_schema=None) -> Stat…)	Run every static check and return one :class:StaticResult.
validate_json_artifact	função	(payload, schema=None, name='artifact') -> StaticResult	
load_json_file	função	(path) -> Any	

pinneapple_llm.qa.units

Dimensional analysis for the quality pipeline -- dependency-free, deliberately small.

pinneapple_llm/qa/units.py · 327 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
UnitError	classe	()	Raised for an unparsable unit string or an incompatible conversion.
Unit	classe	(name, dim, scale, offset)	An SI-scaled unit: si_value = value * scale + offset.
parse_unit	função	(text) -> Unit	Parse a unit string into a :class:Unit. Raises :class:UnitError.
are_compatible	função	(a, b) -> bool	True if two unit strings have the same physical dimension.
convert	função	(value, from_unit, to_unit) -> float	Convert value between two compatible units. Raises on incompatibility.
dimension_of	função	(unit) -> Dim	
dimension_str	função	(unit) -> str	

pinneapple_llm.qa.validators

Scientific validators and their registry (stage E).

`pinneapple_llm/qa/validators/__init__.py` · 19 linhas · **sem teste direto**

`pinneapple_llm.qa.validators.base`

Validator protocol, shared context and registry.

`pinneapple_llm/qa/validators/base.py` · 164 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`SensorReading`	classe	`(id, position, unit, quantity, series, holdout, position_unit)`	One sensor: where, what unit, and the series over the artifact's time axis.
`ValidationContext`	classe	`(spec, meshes, fields, field_units, times, time_unit, sensors, payload, model, extra)`	Everything a validator may look at. Absent data means "not supplied".
`ValidatorResult`	classe	`(name, status, findings, evidence, limitations, duration_s)`	
`register`	função	`(name) -> Callable[[Validator], Validator]`	
`available`	função	`() -> List[str]`	
`get`	função	`(name) -> Validator`	
`run`	função	`(name, ctx) -> ValidatorResult`	
`not_applicable`	função	`(name, reason, **evidence) -> ValidatorResult`	
`blocked`	função	`(name, reason, **evidence) -> ValidatorResult`	

`pinneapple_llm.qa.validators.geometry`

Geometry validation, including the hollow-body check the spec calls out.

`pinneapple_llm/qa/validators/geometry.py` · 206 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_geometry`	função	`(ctx) -> ValidatorResult`	

`pinneapple_llm.qa.validators.guardrail`

Bridge to the existing :class:`pinneapple_llm.guardrail.PhysicsGuardrail`.

`pinneapple_llm/qa/validators/guardrail.py` · 70 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_guardrail`	função	`(ctx) -> ValidatorResult`	

`pinneapple_llm.qa.validators.numeric`

Numeric integrity: finite values, shapes, degenerate fields.

`pinneapple_llm/qa/validators/numeric.py` · 108 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_numeric`	função	`(ctx) -> ValidatorResult`	

`pinneapple_llm.qa.validators.physics`

Physical invariants declared by the specification (stage E, part 3).

`pinneapple_llm/qa/validators/physics.py` · 203 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_physics`	função	`(ctx) -> ValidatorResult`	

`pinneapple_llm.qa.validators.sensors`

Sensor / observation consistency.

`pinneapple_llm/qa/validators/sensors.py` · 123 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_sensors`	função	`(ctx) -> ValidatorResult`	

`pinneapple_llm.qa.validators.temporal`

Temporal consistency: ordering, sample counts, validity window.

`pinneapple_llm/qa/validators/temporal.py` · 99 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_temporal`	função	`(ctx) -> ValidatorResult`	

`pinneapple_llm.qa.validators.units`

Dimensional consistency across geometry, parameters, fields and sensors.

`pinneapple_llm/qa/validators/units.py` · 142 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`check_units`	função	`(ctx) -> ValidatorResult`	

`pinneapple_llm.research`

Literature/repository search integrated with an LLM -- built entirely on PINNeAPPle's own already-existing, real API-backed search (`pinneapple_tools.hpo_experiments.sources.ArxivSource`/ `GithubSource`, which call the real arXiv and GitHub search APIs), never an LLM asked to "recall" papers or ...

`pinneapple_llm/research.py` · 85 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ResearchReport`	classe	`(query, papers, repos, llm_summary)`	
`search_literature`	função	`(query, *, k_papers=5, k_repos=5, summarize=False, provider='anthropic', model=None, api_…)`	Search arXiv + GitHub for real papers/repositories matching `query`, optionally with an LLM summary of the (real) results.

`pinneapple_llm.twin_draft`

LLM-assisted digital twin drafting -- same constrained pattern as `draft.py`/`geometry_draft.py`: the LLM proposes values for `pinneapple_systems.digital_twin.DigitalTwinConfig`'s own real, in-

trospected fields (update interval, assimilation method, anomaly threshold, ...), never a free-form ...

`pinneapple_llm/twin_draft.py` · 156 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`TwinDraftResult`	classe	`(config_kwargs, reasoning, raw_response)`	
`draft_digital_twin`	função	`(description, *, provider='anthropic', model=None, api_key=None, conversation_store=None)…)`	Ask an LLM to propose `DigitalTwinConfig` kwargs for a natural-language description (e.g. "a twin that re-infers every 100ms and flags anomalies aggressively").
`build_3d_live_twin`	função	`(model, field_names, *, export_dir, points, config_kwargs=None, **build_kwargs)`	Build a real `DigitalTwin` (`pinneapple_systems.digital_twin.build_digital_twin`) plus a bound "3D export" hook: every time the twin's own state updates, write the current field prediction at `points` as the next frame in a `pinneapple_blender` `.ply` sequence in `export_dir` -- so ...

`pinneapple_llm.workbench`

PINNeAPPle Agent Workbench: a team of coding and engineering agents running on a local model.

`pinneapple_llm/workbench/__init__.py` · 22 linhas · testes: 20

`pinneapple_llm.workbench.__main__`

`python -m pinneapple_llm.workbench <command>` (also `pinneapple-workbench`).

`pinneapple_llm/workbench/__main__.py` · 274 linhas · testes: 20

nome	tipo	assinatura	o que faz
`cmd_serve`	função	`(args)`	
`cmd_doctor`	função	`(args)`	
`cmd_pull`	função	`(args)`	
`cmd_run`	função	`(args)`	
`cmd_eval`	função	`(args)`	
`cmd_finetune`	função	`(args)`	
`cmd_vscode`	função	`(args)`	
`main`	função	`(argv=None)`	

`pinneapple_llm.workbench.agents`

The agent definitions: one Markdown file per agent, with a small front matter (name, title, description, tools) and the system prompt as the body. `prompts/_common.md` is appended to every agent.

`pinneapple_llm/workbench/agents.py` · 65 linhas · testes: 20

nome	tipo	assinatura	o que faz
`Agent`	classe	`(name, title, description, tools, prompt, source, extra)`	

nome	tipo	assinatura	o que faz
`parse_agent`	função	`(text, source='') -> Agent`	
`load_agents`	função	`(workspace=None, user_dir=USER_DIR) -> tuple[dict[str, Agent], str]`	All agents by name (built-in, then the user folder, then the workspace folder), and the common rules.

`pinneapple_llm.workbench.backend`

Chat-with-tools backends: Ollama, any OpenAI-compatible server (LM Studio, llama.cpp, vLLM, MLX), and a scripted backend for tests.

`pinneapple_llm/workbench/backend.py` · 262 linhas · testes: 20 · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`ToolCall`	classe	`(name, arguments, id)`	
`Reply`	classe	`(content, tool_calls, usage)`	
`Backend`	classe	`(model, tool_mode)`	
`OllamaBackend`	classe	`(model, host='http://127.0.0.1:11434', *, num_ctx=32768, temperature=0.2, tool_mode='nati…)`	`/api/chat` of a local Ollama server (`ollama serve`).
`OpenAIBackend`	classe	`(model, base_url='http://127.0.0.1:1234/v1', api_key='local', *, temperature=0.2, tool_mo…)`	`/v1/chat/completions` of any OpenAI-compatible server: LM Studio, llama.cpp `llama-server`, vLLM, `mlx_lm.server`, or a hosted endpoint.
`ScriptedBackend`	classe	`(script, model='scripted')`	Replays a list of replies, or calls `fn(messages, tools) -> Reply`: for tests and demos without a model.
`parse_json_tool_call`	função	`(text, tool_names) -> ToolCall | None`	Read `{"tool": ..., "arguments": {...}}` (or `name`/`args`) from a reply; None when there is none.
`make_backend`	função	`(kind='ollama', model='', **kw) -> Backend`	

`pinneapple_llm.workbench.evaluate`

A small, checkable benchmark of the workbench on your machine and model.

`pinneapple_llm/workbench/evaluate.py` · 128 linhas · testes: 20

nome	tipo	assinatura	o que faz
`TaskResult`	classe	`(name, passed, seconds, model_calls, prompt_tokens, completion_tokens, detail)`	
`run_eval`	função	`(backend, names=None, on_event=None) -> list[TaskResult]`	
`format_results`	função	`(results, model) -> str`	

nome	tipo	assinatura	o que faz
as_dicts	função	(results) -> list[dict]	

pinneapple_llm.workbench.finetune

Fine-tune the local model on your own workbench sessions.

pinneapple_llm/workbench/finetune.py · 178 linhas · testes: 20

nome	tipo	assinatura	o que faz
TrainConfig	classe	(model, adapter_path, iters, batch_size, num_layers, learning_rate, max_seq_length, lora_…)	
redact	função	(text) -> str	
export_dataset	função	(store, out_dir, *, min_rating=0, mode='trajectories', agents=None, val_fraction=0.1, see…)	Write train.jsonl / valid.jsonl; returns the counts.
mlx_train_command	função	(data_dir, cfg) -> list[str]	
train_mlx	função	(data_dir, cfg=None, *, dry_run=False) -> str	LoRA fine-tune with mlx_lm on Apple Silicon; returns the adapter folder. --mask-prompt trains on the final assistant turn of each example.
serve_mlx_command	função	(cfg=None, port=8080) -> list[str]	
train_peft	função	(data_dir, cfg) -> str	LoRA fine-tune with Hugging Face TRL + PEFT (CUDA). cfg.model is a Hugging Face model id; returns the adapter folder.

pinneapple_llm.workbench.models

Which local model to run, from the machine's memory.

pinneapple_llm/workbench/models.py · 75 linhas · testes: 20

nome	tipo	assinatura	o que faz
Machine	classe	(system, arch, memory_gb, apple_silicon)	
Recommendation	classe	(min_memory_gb, model, approx_size_gb, context, note)	
detect_machine	função	() -> Machine	
recommend	função	(memory_gb=None) -> Recommendation	The default model for a machine with memory_gb of memory (detected when omitted).

pinneapple_llm.workbench.runtime

The agent loop: the lead talks to you, calls tools, keeps a plan and delegates parts of the work to the specialist agents, each running its own loop with its own prompt and tools.

`pinneapple_llm/workbench/runtime.py` · 301 linhas · testes: 20 · sinais no código: placeholder/stub citado no docstring

nome	tipo	assinatura	o que faz
`Cancelled`	classe	`()`	
`SessionSettings`	classe	`(max_steps, max_depth, context_chars, keep_recent, language)`	
`Session`	classe	`(backend, workspace, *, policy=None, settings=None, on_event=None, agents=None, common=No…)`	

`pinneapple_llm.workbench.server`

The workbench web app: FastAPI routes over :class:`~.runtime.Session` and the :class:`~.store.Store`, plus the static single-page frontend in `static/` (no build step).

`pinneapple_llm/workbench/server.py` · 318 linhas · testes: 20

nome	tipo	assinatura	o que faz
`Manager`	classe	`(store, backend_factory, default_workspace, language='')`	Live sessions (one worker thread per running turn) over the store.
`create_app`	função	`(*, backend='ollama', model='', host='', base_url='', api_key='', workspace='.', db=DEFAU…)`	

`pinneapple_llm.workbench.store`

The workbench database: a local SQLite file, or a PostgreSQL server you already run.

`pinneapple_llm/workbench/store.py` · 158 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`Store`	classe	`(url=DEFAULT_DB)`	

`pinneapple_llm.workbench.tools`

The tools the agents use, confined to one workspace folder.

`pinneapple_llm/workbench/tools.py` · 355 linhas · testes: 37

nome	tipo	assinatura	o que faz
`ToolError`	classe	`()`	
`ApprovalPolicy`	classe	`(mode)`	
`Workspace`	classe	`(root, policy=None, approve=None, run_timeout=600, web=True)`	A folder the agents may read and change. `approve(tool, args) -> bool` is asked before guarded actions.

`pinneapple_llm.workbench.web`

Internet search and page fetching for the agents.

`pinneapple_llm/workbench/web.py` · 142 linhas · testes: 20

nome	tipo	assinatura	o que faz
`parse_duckduckgo`	função	`(page) -> list[dict]`	
`web_search`	função	`(query, max_results=8) -> list[dict]`	`[{title, url, snippet}]` from the first configured provider.
`html_to_text`	função	`(page) -> str`	
`fetch_text`	função	`(url, max_chars=12000) -> str`	The readable text of a page (HTML stripped) or a raw text file; GitHub `blob` links are read raw.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 21 de 49.
- Funções de teste em arquivos que importam algum módulo do pacote: 242.

Limitações conhecidas

- *Nenhuma registrada.*

7. `pinneapple_mcp`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
8	1.541	2	31	30 (3 arquivos)	3

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
`__main__`		1	0	0
`config`		1	3	10
`expressions`		1	2	10
`jobs`		1	5	0
`problems`		1	2	0
`server`		1	2	10
`tools`		1	17	30

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_mcp`

PINNeAPPle as an MCP server: solve, benchmark and validate physics problems from any coding agent.

`pinneapple_mcp/__init__.py` · 34 linhas · testes: 30

nome	tipo	assinatura	o que faz
`build_server`	função	`(*args, **kwargs)`	
`main`	função	`(argv=None)`	

`pinneapple_mcp.__main__`

Módulo sem docstring e sem texto.

`pinneapple_mcp/__main__.py` · 5 linhas · **sem teste direto**

`pinneapple_mcp.config`

Limits and locations of the MCP server. Every value has a safe default and an environment variable.

`pinneapple_mcp/config.py` · 91 linhas · testes: 10

nome	tipo	assinatura	o que faz
`Config`	classe	`(workdir, max_seconds, max_memory_mb, max_jobs, max_epochs, max_points, max_file_mb, allo…)`	
`get_config`	função	`() -> Config`	
`set_config`	função	`(cfg) -> Config`	

`pinneapple_mcp.expressions`

Safe arithmetic expressions of the coordinates, e.g. `"sin(pi*x)*sinh(pi*y)/sinh(pi)"`.

`pinneapple_mcp/expressions.py` · 105 linhas · testes: 10

nome	tipo	assinatura	o que faz
`compile_expression`	função	`(text, coords) -> Callable[[np.ndarray], np.ndarray]`	`f(X) -> (N,)` for points `X` of shape `(N, len(coords))`. Raises `ValueError` on anything that is not plain arithmetic of the coordinates.
`value`	função	`(v, coords)`	A number stays a number; a string becomes `compile_expression`.

`pinneapple_mcp.jobs`

Long work as jobs: a separate process per job, with a wall-time limit, a memory limit and no network.

`pinneapple_mcp/jobs.py` · 199 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
running	função	`() -> list[str]`	
submit	função	`(kind, args, cfg, max_seconds=None) -> str`	Start job `kind` (a key of `tools.JOB_KINDS`) in a new process; returns its id.
status	função	`(job_id, cfg, wait_seconds=0.0) -> dict[str, Any]`	Status of a job (`queued`, `running`, `done`, `failed`, `timeout`, `cancelled`, `lost`), its result when done, and the files in its directory. `wait_seconds` waits up to that long for it to finish.
cancel	função	`(job_id, cfg) -> dict[str, Any]`	
list_jobs	função	`(cfg, limit=20) -> list[dict[str, Any]]`	

pinneapple_mcp.problems

A physics problem from JSON: a preset, or a box problem described field by field.

`pinneapple_mcp/problems.py` · 156 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
BuiltProblem	classe	`(problem, ctx, exact, dirichlet, source, spec)`	
build_problem	função	`(spec) -> BuiltProblem`	`BuiltProblem` from the JSON description above. Raises `ValueError` with the reason on bad input.

pinneapple_mcp.server

The stdio MCP server: `pinneapple-mcp` or `python -m pinneapple_mcp`.

`pinneapple_mcp/server.py` · 284 linhas · testes: 10

nome	tipo	assinatura	o que faz
build_server	função	`(config=None)`	The MCP server object (SDK 2.x `MCPServer` or 1.x `FastMCP`) with every PINNeAPPle tool.
main	função	`(argv=None) -> None`	

pinneapple_mcp.tools

The MCP tools as plain functions: JSON-like arguments in, JSON-like dictionaries out.

`pinneapple_mcp/tools.py` · 667 linhas · testes: 30

nome	tipo	assinatura	o que faz
capabilities	função	`() -> dict[str, Any]`	What this installation offers: version, optional APIs present, device, limits and the working directory.

nome	tipo	assinatura	o que faz
`list_methods`	função	`(category='all', validated_only=False) -> dict[str, Any]`	Solve methods usable by `solve` (grouped by kind) and the method catalog (solvers, training methods, equations, problems) with the validation status of each item.
`describe_method`	função	`(method) -> dict[str, Any]`	A solve method (e.g. `pinn`, `fem`) with its options, or a catalog item (e.g. `S2`) with its code, references, validation status and the tests that give it.
`list_presets`	função	`(contains=None) -> dict[str, Any]`	Problem presets usable as `{"preset": name}` in `solve` and `benchmark`.
`trust_card`	função	`(file=None, field=None, grid=None, values=None, pde=None, source=None, alpha=1.0, dirichl…)`	Physics checks of a field on a grid and its trust card (decision APPROVED / REVIEW / REJECT).
`solve`	função	`(problem, method='pinn', options=None, reference=None, reference_options=None, grid_point…)`	Solve `problem` with `method` in a job; the result carries metrics against the reference, the physics checks and the trust card, and the paths of the saved prediction and figures. Returns the job status (`wait_seconds` > 0 waits for the result up to that long; otherwise poll `job_status`).
`benchmark`	função	`(problem, methods=('pinn', 'fem'), budget=None, seeds=1, reference=None, options=None, gr…)`	Run several methods on one problem with the same training budget (epochs, collocation points, seeds for every neural method) and score them on the same grid against one reference; each run gets a trust card. Returns the job status (see `solve`).
`zoo_list`	função	`(task=None) -> dict[str, Any]`	Pre-trained, verified models of the zoo (with what they predict, their validity range and held-out errors) and the datasets of the dataset hub.
`zoo_predict`	função	`(name, inputs) -> dict[str, Any]`	Predictions of zoo model `name` for `inputs` ({input name: value or list}), with the mask of rows inside the model's validity range (rows outside it are reported, never silently extrapolated).
`zoo_validate`	função	`(name, inputs=None) -> dict[str, Any]`	Trust report of zoo model `name`: its card metrics reproduced on the stored reference, the zoo acceptance limits, the model's physics checks and, with `inputs`, whether they are inside its validity range.
`job_status`	função	`(job_id, wait_seconds=0.0) -> dict[str, Any]`	Status of a job and, when it is done, its result with the paths of its files.
`list_jobs`	função	`(limit=20) -> dict[str, Any]`	The most recent jobs with their status.
`cancel_job`	função	`(job_id) -> dict[str, Any]`	Stop a running job (its process is killed).

nome	tipo	assinatura	o que faz
design	função	(text, answers=None, verify=True, seed=0, max_seconds=None, wait_seconds=0.0) -> dict[str…)	Design from a plain-language goal. Out-of-scope requests and missing values are answered at once (status out_of_scope with what is needed, or needs_input with the questions); an in-scope request starts a job whose result is the design package (verdict, design, margins, verification, files).
reason	função	(case=None, requirements=None, concept=None, lang='pt', max_steps=12, max_seconds=None, w…)	Chain of whys (pinneapple_reason): from a naive concept, check every requirement's failure modes, apply the best countermeasure for the first failure, repeat. Starts a job; its result is the path (version, failure found, countermeasure), the explanation and the report files.
reason_library	função	(domain=None, lang='pt') -> dict[str, Any]	The failure modes (by physical domain) and countermeasures pinneapple_reason knows, and the worked cases.
reason_check	função	(mode, params, elements=(), limits=None, lang='pt') -> dict[str, Any]	Run one failure mode of the library on the given parameters (SI units): demand, capacity, margin, the mechanism in words, every measured number, sources, applicability and self-checks.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 5 de 8.
- Funções de teste em arquivos que importam algum módulo do pacote: 30.

Limitações conhecidas

- *Nenhuma registrada.*

8. pinneapple_autolab

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
16	3.596	38	39	37 (5 arquivos)	10

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
agent		1	9	6
core		1	13	0
dashboard		1	5	5

submódulo	o que faz	módulos	classes + funções	testes*
domain		1	2	0
domains		2	11	11
lab		1	2	10
ledger		1	4	0
mission		1	4	0
qualification		1	1	0
replay		1	1	6
report		1	8	0
simulator		1	3	0
surrogates		1	9	0
trust		1	5	0

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_autolab`

pinneapple_autolab -- the Autonomous Physics Lab: an AI that manages evidence to make engineering decisions.

`pinneapple_autolab/__init__.py` · 70 linhas · testes: 31

`pinneapple_autolab.agent`

The lab driven by an agent: a mission in words -> a Mission -> tool calls on the lab -> an audited answer.

`pinneapple_autolab/agent.py` · 488 linhas · testes: 6

nome	tipo	assinatura	o que faz
MissionRefused	classe	`()`	The request cannot be run as written (with the reason and what is needed).
MissionDomain	classe	`(title, variables, baseline, output_words, units, operating_point, acceptance, conditions…)`	What a simulator can be asked: its variables and baseline, outputs by the words that name them, the fixed operating point, default acceptance tolerances. `output_words`: output -> words (e.g. "drag_N": ["drag"]); `operating_point`: name -> (value, unit, regex capturing the value in a request).
MissionReading	classe	`(mission, readings, assumptions)`	
LabPlanner	classe	`(lab, n_doe=None)`	Deterministic step planner over :func:`lab_toolbox` (no network): read the mission, plan, baseline, space-filling designs, then (learn, assess, follow the lab's rule, simulate) until only the verification reserve is left, verify, read the status, answer. A stand-in for a language model, not a …
AgentResult	classe	`(reading, refused, answer, audit, run, summary, wall_s, planner)`	

nome	tipo	assinatura	o que faz
interpret_mission	função	`(text, dom, llm=None) -> MissionReading`	Read `text` into a Mission over `dom`. `llm` (optional) has `extract(text, schema) -> dict` (e.g. :class:`pinneapple_design.intent.CallLLM`); its answer is accepted only where it names closed options and quotes words present in the text.
lab_toolbox	função	`(lab, reading=None) -> Toolbox`	The closed tool set an agent drives `lab` with. Every tool returns what the lab measured or predicted (predictions are labelled), rounded to 5 significant digits; `answer` records the final answer.
audit_claims	função	`(answer, tool_log, lab, recommended_run=None, rtol=0.005) -> dict[str, Any]`	Every number in `answer` must match (within `rtol`, or as a rounding) a number the tools returned or a run id/variable/budget figure; the recommended run must exist, have succeeded and meet every limit.
run_agent	função	`(text, simulator, folder, *, domain, planner=None, provider=None, model=None, llm=None, m…)`	Interpret `text`, build the lab and let the agent drive it through :func:`lab_toolbox`. Planner: the given `planner`, else an :class:`~pinneapple_autolab.core.LLMStepPlanner` when `provider` or `llm` is given ("anthropic", "openai", "openai_compatible", "ollama"), else :class:`LabPlanner`.

`pinneapple_autolab.core`

Mission -> plan -> controlled tool calls -> evidence: the agent loop of the Autonomous Physics Lab.

`pinneapple_autolab/core.py` · 297 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
Tool	classe	`(name, fn, description, inputs, summary)`	A library call the agent may make: `fn(state, **args) -> dict` (the dict lands in `state[name]`). `inputs`: argument name -> description (the only arguments accepted); `summary`: result -> short dict for the evidence log (default: the scalar entries of the result).
Toolbox	classe	`(kind, tools=None)`	The controlled set of tools of one mission kind.
Step	classe	`(tool, args, why)`	
Plan	classe	`(mission, steps, planner, notes)`	
Planner	classe	`()`	
TemplatePlanner	classe	`(steps)`	The fixed sequence of steps of a mission kind.
LLMPlanner	classe	`(fallback, *, provider='anthropic', model=None, llm=None, required=())`	A language model orders the toolbox's tools into a plan; the template plan is the fallback.
Evidence	classe	`(step, tool, args, why, summary, seconds, ok, error)`	

nome	tipo	assinatura	o que faz
`Run`	classe	`(plan, state, evidence)`	
`StepPlanner`	classe	`(name)`	Chooses the next step after seeing every result so far (`None`: done).
`LLMStepPlanner`	classe	`(*, provider='anthropic', model=None, llm=None, rules='')`	A language model chooses each next step from the toolbox, seeing the mission, the tools and the evidence so far (summaries only: the numbers the tools returned). It answers `{"tool", "args", "why"}` or `{"done": true}`. An unknown tool or argument is not run: the refusal is recorded as evidence …
`execute`	função	`(plan, toolbox, state=None, *, stop_on_error=True, log=None) -> Run`	Run the plan's steps in order through the toolbox only. Each tool sees the shared `state` (earlier outputs by tool name, plus whatever the caller put there; `_plan` and `_evidence` hold the plan and the evidence so far) and returns a dict stored under its name.
`act`	função	`(mission, toolbox, planner, state=None, *, max_steps=200, max_refusals=5, log=None) -> Run`	Decide-then-act loop: ask `planner` for one step, check it against the toolbox, run it, record the evidence, repeat until the planner is done, `max_steps` or `max_refusals` consecutive refused / failed steps. Same :class:`Tool` / :class:`Evidence` contract as :func:`execute`; the returned …

`pinneapple_autolab.dashboard`

The lab's dashboard: four synchronised areas over the real state of a session.

`pinneapple_autolab/dashboard.py` · 233 linhas · testes: 5

nome	tipo	assinatura	o que faz
`live_state`	função	`(folder, out_dir=None, tail=120) -> dict[str, Any]`	The session's state from its files: `ledger.json` (runs), `events.jsonl` (what is happening), `snapshot.json` / `summary.json` when written (in `folder` or `out_dir`). `status` says what is going on now: `running` (the run being simulated), `training`, `phase` (last stage), …
`build_dashboard`	função	`(folder, out_dir, *, scenes=None, title='', state=None, live=False, replay=False, figures…)`	Write `out_dir/index.html`: the dashboard with the current state embedded (`live=True`: it also polls `state.json` next to it, see :func:`serve`; `replay=True`: a slider and a play button step through the recorded events, every number the recorded one). `scenes`: name -> studio Scene …
`serve`	função	`(folder, out_dir, port=8120, host='127.0.0.1', block=True, **kw)`	Serve the dashboard of `folder` live: `/` the page (rebuilt once), `/state.json` the state read from the session's files at every request. `block=False` returns the server (`.shutdown()` to stop).
`capture`	função	`(html, png, *, width=1600, height=1100, full_page=True, replay_to=None, wait_ms=5000) -> …)`	Screenshot of a dashboard page (`replay_to`: the replay at that event). Needs `playwright`.
`record_replay`	função	`(html, mp4, *, frames=60, fps=4.0, hold_s=3.0, width=1600, height=2040, out_width=1280, c…)`	An MP4 of the dashboard's replay (the recorded events, evenly sampled), via ffmpeg. Needs `playwright`.

pinneapple_autolab.domain

Where the surrogate has evidence: the validated domain of a set of simulated designs.

`pinneapple_autolab/domain.py` · 72 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`DomainVerdict`	classe	`(inside, ratio, nearest, reasons)`	
`ValidatedDomain`	classe	`(U, names=None, bounds=None, max_ratio=1.5, margin=0.05)`	

pinneapple_autolab.domains

Simulators and missions of the Autonomous Physics Lab, one module per physical domain.

`pinneapple_autolab/domains/__init__.py` · 2 linhas · **sem teste direto**

pinneapple_autolab.domains.racecar

The AI Aerodynamicist's simulator: a parametric open-wheel racing car in OpenFOAM, and its flagship mission.

`pinneapple_autolab/domains/racecar.py` · 464 linhas · testes: 11

nome	tipo	assinatura	o que faz
`RaceCarAero`	classe	`(resolution='campaign', iterations=800, procs=4, n_lines=40, keep_cases=False, conditions…)`	OpenFOAM external aerodynamics of :class:`FormulaCar`. `resolution`: a key of :data:`MESHES` ("coarse" 0.14 M cells, "medium" 0.21 M, "fine" 0.82 M cells at the baseline, "campaign": medium with the wings and the floor at the fine levels); `iterations` of simpleFoam. Forces are written every …
`make_car`	função	`(params)`	
`force_convergence`	função	`(history, window=2) -> dict[str, float]`	Relative change of drag and downforce over the last `window` written intervals (e.g. the last 200 iterations when forces are written every 100).
`solver_log_summary`	função	`(case) -> dict[str, Any]`	Last iteration, final initial residuals per field, whether checkMesh passed, snappy cells and timings.
`render_run`	função	`(params, flow_npz, out_prefix, title='', size=(1200, 675)) -> dict[str, str]`	PyVista renders of one simulated design from its saved flow scene: surface Cp (jet, from above and below) and the streamlines coloured by speed. Returns {name: path}.
`cycles_renders`	função	`(params, flow_npz, out_prefix, title='', samples=64, size=(1600, 900)) -> dict[str, str]`	Blender Cycles renders of a simulated design (needs `bpy`): the streamlines coloured by speed on black, and the surface Cp in the studio, each with its jet colour bar. Returns {} without `bpy`.

nome	tipo	assinatura	o que faz
`flow_scene`	função	`(params, flow_npz, title='')`	The studio Scene of a simulated design (car parts + skin Cp/Cf, streamlines, slices) from its flow file.
`build_dashboard`	função	`(work, out_dir, summary, *, figures=None, figure_runs=None, viewers=True, replay=True, re…)`	The lab dashboard of a race-car session: renders of the original and the recommended design (surface Cp, surface mesh, streamlines, mid-plane Cp, wake vorticity) from their saved flow files, their 3-D studio viewers, and the recorded session (`work`: the lab folder with ledger.json and …
`recorded_dashboard`	função	`(results_dir, out_dir) -> str`	The dashboard of a recorded session from its committed files only (`ledger.json`, `events.jsonl`, `summary.json`, `snapshot.json` and the renders in `figures/`): no OpenFOAM, no flow files, so no 3-D viewer; the replay steps through the recorded events.
`racecar_domain`	função	`()`	What the race-car simulator can be asked, for :func:`pinneapple_autolab.agent.interpret_mission`.
`flagship_mission`	função	`(budget=30, reserve_verify=3) -> Mission`	"Find a lower-drag aerodynamic design at 70 m/s. Preserve the required downforce, stay within the geometry constraints, and validate the final design using CFD."

`pinneapple_autolab.lab`

The autonomous physics lab: plan -> build -> simulate -> learn -> optimise -> verify -> deliver, on any simulator.

`pinneapple_autolab/lab.py` · 546 linhas · testes: 10

nome	tipo	assinatura	o que faz
`AutonomousLab`	classe	`(mission, simulator, folder, *, families=…, field_family='gp', seed=0, nsga_budget=3000, …)`	`critical_region`: name of a simulator region whose field error decides the trust in a constraint (e.g. the diffuser for downforce); `field_tol`: the tolerated predicted std of that field there. `probe(params, **options)` runs another fidelity (e.g. a finer mesh) that never joins the training …
`showcase`	função	`(decisions, baseline) -> dict[str, dict[str, Any]]`	Three simulated designs the lab judged differently before simulating them, by a fixed rule:

`pinneapple_autolab.ledger`

The evidence ledger: every simulation the lab ran (why, with what, what it cost, what came out) and every event of the session, in the order they happened.

`pinneapple_autolab/ledger.py` · 185 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`BudgetExhaustedError`	classe	`()`	The mission's simulation budget has been spent.

nome	tipo	assinatura	o que faz
`RunRecord`	classe	`(id, purpose, params, ok, outputs, message, cost_s, started, reason, predicted, meta, fie…)`	One reference simulation. `outputs` holds only what the solver produced (empty when it failed); `predicted` what the surrogate said before the run (for the held-out error), `reason` why it was run.
`Ledger`	classe	`(folder, max_runs=None, max_wall_s=None, listeners=None)`	Runs and events of one lab session, kept in `folder`. `listeners`: callables `f(event_dict)` called on every event (a live dashboard, a test).
`environment`	função	`(extra=None) -> dict[str, Any]`	Hardware and software of this machine: CPU, cores, memory, OS, Python and package versions, git commit.

`pinneapple_autolab.mission`

A mission as numbers: what to improve, what must hold, what may change, how much simulation it may spend.

`pinneapple_autolab/mission.py` · 161 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`UnresolvedLimitError`	classe	`()`	A limit relative to the baseline was used before the baseline was simulated.
`Budget`	classe	`(max_runs, reserve_verify, max_wall_s)`	Simulation budget. `max_runs` reference simulations in total, of which `reserve_verify` are kept for the final verification of the chosen designs; `max_wall_s` (optional) caps the summed solver wall time.
`Limit`	classe	`(name, lower, upper, lower_frac_of_baseline, upper_frac_of_baseline, unit, kind, label)`	`lower <= output <= upper`; a bound can instead be a fraction of the baseline's value (`lower_frac_of_baseline=1.0`: at least the baseline's). `kind`: "physical", "geometry", "validity".
`Mission`	classe	`(title, statement, variables, objectives, constraints, baseline, budget, acceptance, cond…)`	What the lab is asked to do. `acceptance`: relative tolerance per output for a prediction to count as confirmed by the reference solver (|prediction - reference| / |reference| <= tol). `conditions`: the fixed operating and boundary conditions (speed, ground, ...); the lab never changes them.

`pinneapple_autolab.qualification`

Before a campaign: how much of the reference solver's answer is the mesh and the iteration count?

`pinneapple_autolab/qualification.py` · 49 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`solver_qualification`	função	`(sim, params, variants, folder, reference, campaign, outputs=None) -> dict[str, Any]`	

`pinneapple_autolab.replay`

A simulator that answers from a ledger of measured runs when it holds the exact design, and runs the real simulator otherwise.

pinneapple_autolab/replay.py · 68 linhas · testes: 6

nome	tipo	assinatura	o que faz
ReplaySimulator	classe	(simulator, ledger_path, fields_dir=None, atol=1e-09)	

pinneapple_autolab.report

Figures and the engineering report of a lab session, drawn only from its snapshot (ledger runs, candidates, model board, decisions): nothing here computes a physical number.

pinneapple_autolab/report.py · 348 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
shrink_png	função	(path, colors=256) -> str	Re-save a PNG with an adaptive palette of colors (renders and dense scatter plots shrink 3-4x).
pareto_front	função	(F) -> np.ndarray	Indices of the non-dominated rows of F (all columns minimised), sorted by the first column.
fig_pareto	função	(snap, path, x='drag_N', y='downforce_N', scale=0.001, units='kN', annotate=None) -> str	
fig_models	função	(snap, path, outputs, labels=None, scale=None) -> str	
fig_learning	função	(snap, path, outputs, objective, acceptance) -> str	
fig_field_triptych	função	(points, ref, pred, path, title='', clim=(-1.5, 1.0), err_max=0.3, label='Cp', mirror=Tru…)	Reference | prediction | |error| from above and from below (half-model points mirrored), colormap jet.
fig_trust_cards	função	(cards, path, outputs, units=None) -> str	One panel per card: {"tag", "label", "reasons", "predicted", "std", "reference", "decision", "action"}; bars = prediction +- 1.96 std, diamond = reference (when simulated).
write_report	função	(summary, path, figures=None, qualification=None, extra='') -> str	The engineering report in Markdown: mission, plan, solver qualification, budget, models, decisions, recommendation (a simulated design), environment.

pinneapple_autolab.simulator

The reference simulator the lab calls: a solver run on one design, its checks before it runs, its cost after.

pinneapple_autolab/simulator.py · 91 linhas · **sem teste direto** · sinais no código: levanta NotImplementedError em algum caminho

nome	tipo	assinatura	o que faz
`SimOutcome`	classe	`(outputs, ok, message, cost_s, fields, meta)`	
`Simulator`	classe	`(name, fidelity, outputs, field_names, regions)`	Base class: set `name`, `fidelity`, `outputs` (and `field_names` / `field_points` / `regions` if it returns fields), implement :meth:`check` and :meth:`run`.
`FunctionSimulator`	classe	`(fn, outputs, *, name='function', checker=None, fail=None, field_names=(), regions=None, …)`	`fn(params) -> {output: value}` (optionally also `"__fields__": {name: array}`); `checker(params) -> list[str]`; `fail(params) -> str | None` makes a run fail with that message (to test failure paths).

`pinneapple_autolab.surrogates`

Surrogates of the reference solver, compared on data they did not see, each with an uncertainty.

`pinneapple_autolab/surrogates.py` · 260 linhas · **sem teste direto** · sinais no código: levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`GPModel`	classe	`(restarts=1, seed=0)`	
`MLPEnsemble`	classe	`(n=5, hidden=(32, 32), seed=0, max_iter=600)`	
`QuadraticRSM`	classe	`(alpha=1e-06)`	
`ModelBoard`	classe	`(table, chosen, models, cv_pred, n_train, cv_y)`	Cross-validated scores of every family on every output, the choice per output and the fitted models.
`FieldSurrogate`	classe	`(family='gp', energy=0.99, max_modes=12)`	POD + per-mode regression of fields sampled on fixed points. `energy`: fraction of the snapshot variance the retained modes keep (capped at `max_modes` and at n_snapshots - 1).
`make_model`	função	`(family, **kw) -> _Model`	
`cross_validate`	função	`(family, U, y, k=5, seed=0) -> dict[str, Any]`	k-fold cross-validation of one family on one output; returns the metrics and the held-out predictions.
`compare_models`	função	`(U, Y, families=…, k=5, seed=0, select='rmse') -> ModelBoard`	Cross-validate every family on every output, choose the lowest held-out `select` metric per output (a family whose 95 % interval holds fewer than half the held-out points is only chosen if nothing else is available), refit the chosen family on all data.
`field_errors`	função	`(pred, ref, regions=None, std=None) -> dict[str, dict[str, float]]`	RMSE, max error and (with `std`) the 95 % coverage of a predicted field, overall and per region.

`pinneapple_autolab.trust`

Can this prediction be used for a decision? Triage of surrogate candidates and the choice of the next simulation.

`pinneapple_autolab/trust.py` · 262 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CandidateAssessment`	classe	`(index, params, predicted, std, p_feasible, cei, domain, label, reasons, report, critical…)`	
`NextExperiment`	classe	`(candidate, rule, reason, alternatives)`	
`assess_candidates`	função	`(X, names, pred, objectives, constraints, domain_verdicts, best_value, acceptance, model_…)`	Assess candidates `X` (physical units, columns `names`). `pred`: output -> (mean, std) arrays; `best_value`: the best simulated feasible value of the first objective (None: no feasible simulation yet, then the EI is replaced by the predicted improvement over the candidates' worst); …
`choose_next`	função	`(cands, objective, exclude=()) -> NextExperiment`	The next simulation, by the written rule in the module docstring.
`verification_checks`	função	`(c, reference, ok, acceptance, constraints, message='') -> CandidateAssessment`	Update an assessment with the reference result: measured relative errors, the confirmation check and the feasibility under the reference values.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 6 de 16.
- Funções de teste em arquivos que importam algum módulo do pacote: 37.

Limitações conhecidas

- *Nenhuma registrada.*

9. `pinneapple_app`

Objetivo

Sem texto de objetivo.

Números do inventário

módulos	linhas	classes públicas	funções públicas	testes em arquivos que importam o pacote	módulos sem teste direto
18	3.672	29	53	48 (3 arquivos)	11

Submódulos

submódulo	o que faz	módulos	classes + funções	testes*
backend		18	82	41

* maior número de testes que importam um módulo do submódulo.

Módulos e API pública

`pinneapple_app.backend`

Módulo sem docstring e sem texto.

`pinneapple_app/backend/__init__.py` · 1 linhas · **sem teste direto**

`pinneapple_app.backend.core`

Módulo sem docstring e sem texto.

`pinneapple_app/backend/core/__init__.py` · 1 linhas · **sem teste direto**

`pinneapple_app.backend.core.admin_auth`

pinneapple_app.backend.core.admin_auth — a minimal, generic admin gate.

`pinneapple_app/backend/core/admin_auth.py` · 31 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`require_admin`	função	`(x_admin_token=Header(default='')) -> None`	FastAPI dependency: raises 401 unless `X-Admin-Token` matches `PINNEAPPLE_ADMIN_TOKEN`. If that env var is unset, admin routes are refused entirely (503) rather than silently open — there is no "admin token not configured, so let everyone in" fallback.

`pinneapple_app.backend.core.benchmark`

Benchmark report generator — produces plots and tables from ExperimentResult.

`pinneapple_app/backend/core/benchmark.py` · 208 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`build_benchmark_payload`	função	`(result) -> Dict[str, Any]`	Convert an ExperimentResult into a JSON-serialisable benchmark payload.

`pinneapple_app.backend.core.collocation`

Collocation point generation strategies for pinneapple_app.

`pinneapple_app/backend/core/collocation.py` · 208 linhas · testes: 7

nome	tipo	assinatura	o que faz
`CollocationConfig`	classe	`(strategy, n_interior, n_boundary, n_initial, seed, use_geometry, geometry_sdf_fn, domain…)`	Configuration for collocation point generation.
`generate_points`	função	`(config, bounds, *, include_time=False, t_range=(0.0, 1.0)) -> Dict[str, np.ndarray]`	Generate collocation points from a config and domain bounds.

`pinneapple_app.backend.core.data_pipeline`

Data generation pipeline — runs a solver to produce synthetic training data.

`pinneapple_app/backend/core/data_pipeline.py` · 163 linhas · testes: 7

nome	tipo	assinatura	o que faz
`DataConfig`	classe	`(solver_key, n_snapshots, grid_resolution, t_end, use_solver)`	Configuration for synthetic data generation.
`DataBundle`	classe	`(x_col, x_bnd, x_ic, u_ref, solver_outputs, meta)`	Output of the data pipeline, consumed by ExperimentRunner.
`run_data_pipeline`	função	`(problem, collocation_cfg, data_cfg, *, device='cpu', verbose=True) -> DataBundle`	Execute data generation: (optionally) run solver + sample collocation pts.

`pinneapple_app.backend.core.experiment`

Experiment runner: trains multiple models on the same problem and collects results.

`pinneapple_app/backend/core/experiment.py` · 475 linhas · testes: 7 · sinais no código: experimental

nome	tipo	assinatura	o que faz
`ModelRunConfig`	classe	`(name, extra_kwargs, weight_override)`	
`ExperimentConfig`	classe	`(experiment_id, problem_name, models, metrics, epochs, lr, device, batch_size, seed, grad…)`	Full configuration for one experimental run (one problem, N models).
`ModelResult`	classe	`(name, metrics, loss_history, physics_loss_history, bc_loss_history, train_time_s, n_para…)`	
`ExperimentResult`	classe	`(experiment_id, config, model_results, completed_at)`	
`ExperimentRunner`	classe	`(config, data_bundle, problem)`	Trains all configured models and streams progress via an async generator.

`pinneapple_app.backend.core.model_catalog`

Model catalog: lists available models, enriches with metadata, and recommends.

`pinneapple_app/backend/core/model_catalog.py` · 195 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelEntry`	classe	`(name, family, description, supports_physics_loss, tags, recommended_for)`	
`list_models`	função	`(family=None) -> List[ModelEntry]`	Return all available models, optionally filtered by family.
`recommend_for_problem`	função	`(problem_family, n=5) -> List[str]`	Return top-N recommended model names for a given problem family.
`get_model_info`	função	`(name) -> Optional[ModelEntry]`	

`pinneapple_app.backend.core.physics_adapter`

Physics adapters: turn a `ProblemDefinition` into a trainable physics loss.

`pinneapple_app/backend/core/physics_adapter.py` · 552 linhas · testes: 7 · sinais no código: placeholder/stub citado no docstring; levanta `NotImplementedError` em algum caminho

nome	tipo	assinatura	o que faz
`ConditionInfo`	classe	`(key, name, kind, fields, weight, n_points, how, target_rms)`	
`PhysicsAdapter`	classe	`()`	Shared interface used by the experiment runner and the report.
`PresetPhysics`	classe	`(spec, *, w_physics=None, w_bc=None)`	
`CustomPhysics`	classe	`(problem, *, w_physics=None, w_bc=None)`	
`unwrap`	função	`(out) -> torch.Tensor`	Model output -> plain (N, F) tensor (PINNOutput/ModelOutput carry `.y`).
`spatial_coords`	função	`(coords) -> List[str]`	
`all_faces`	função	`(coords) -> List[Face]`	
`faces_for_tag`	função	`(tag, coords) -> Optional[List[Face]]`	Box-face convention for the tags in `_BOX_TAGS`; `None` otherwise.
`sample_faces`	função	`(bounds, coords, faces, n, rng) -> Tuple[np.ndarray, np.ndarray]`	Uniform points on the given box faces, with outward unit normals.
`sample_initial`	função	`(bounds, coords, n, rng) -> np.ndarray`	
`sample_interior`	função	`(bounds, coords, n, rng) -> np.ndarray`	
`characteristic_length`	função	`(bounds, coords) -> float`	Mean spatial extent (time extent for ODE-only problems).
`parse_equation`	função	`(expr, coords, fields)`	`"u_t + u*u_x - 0.01*u_xx"` -> SymPy expression in u(x, t).
`parse_location`	função	`(location, bounds, coords) -> List[Face]`	`""/"all"/"boundary"`, `"x=0"`, `"x_min"`, `"left"`...
`build_physics`	função	`(problem, *, weight_override=None) -> Optional[PhysicsAdapter]`	Adapter for the problem, or `None` when it defines no physics at all.

`pinneapple_app.backend.core.problem`

Problem definition layer for pinneapple_app.

`pinneapple_app/backend/core/problem.py` · 184 linhas · testes: 7

nome	tipo	assinatura	o que faz
`BoundaryConditionSpec`	classe	`(kind, location, value, field)`	Human-readable BC for custom problems.
`InitialConditionSpec`	classe	`(expression, field)`	Human-readable IC for custom problems.
`EquationSpec`	classe	`(expression, label, field)`	A single PDE residual equation (string or SymPy expression).
`ProblemDefinition`	classe	`(kind, name, spec, dim, domain_bounds, pde_family, is_time_dependent, preset_name, equati…)`	Unified problem container used throughout pinneapple_app.
`load_preset`	função	`(preset_name) -> ProblemDefinition`	Load a ProblemDefinition from the environment preset registry.
`define_custom`	função	`(name, equations, bcs, ics=None, domain_bounds=None, dim=2, pde_family=FAMILY_GENERIC, is…)`	Build a ProblemDefinition from user-supplied equations, BCs, and ICs.

`pinneapple_app.backend.core.problem_registry`

Problem registry: metadata, solver recommendations, and model suggestions per preset.

`pinneapple_app/backend/core/problem_registry.py` · 469 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`get_problem_meta`	função	`(preset_name) -> Dict[str, Any]`	Return metadata dict for a preset, with sensible defaults for unknowns.
`recommend_models`	função	`(preset_name) -> list`	
`recommended_solver`	função	`(preset_name) -> str`	
`all_problems`	função	`() -> list`	

`pinneapple_app.backend.core.surrogate_report`

Surrogate quality report: the "can I trust this model?" section of a run.

`pinneapple_app/backend/core/surrogate_report.py` · 552 linhas · testes: 7

nome	tipo	assinatura	o que faz
`error_section`	função	`(model, fields, X_test, Y_test, reference_source) -> Dict[str, Any]`	
`convergence_section`	função	`(loss_history) -> Dict[str, Any]`	
`generalization_section`	função	`(model, X_train, Y_train, X_test, Y_test, res_train, res_unseen) -> Dict[str, Any]`	
`variables_section`	função	`(model, adapter, X_eval, supervised) -> Dict[str, Any]`	

nome	tipo	assinatura	o que faz
`weights_section`	função	`(model, adapter, final_raw, w_data, data_loss) -> Dict[str, Any]`	
`uncertainty_section`	função	`(model, snapshots, fields, X_eval, X_cal, Y_cal, X_test, Y_test, coverage=0.9) -> Dict[st…)`	
`field_scales`	função	`(adapter, pred) -> np.ndarray`	Per-field magnitude: max of the predicted spread, the declared range and the RMS of the Dirichlet/initial values imposed on that field.
`residual_scale`	função	`(model, adapter, X, pred) -> float`	Typical size of the PDE's individual terms, to normalise its residual.
`physics_checks_section`	função	`(model, adapter, X_eval, raw_unseen, raw_initial, pred_eval) -> List[Dict[str, Any]]`	
`verdict`	função	`(report) -> Dict[str, Any]`	
`build_report`	função	`(*, model, adapter, loss_history, final_raw_train, raw_initial, raw_unseen, res_train, X_…)`	Assemble all sections. `split` holds numpy/tensor train/cal/test arrays.

`pinneapple_app.backend.main`

FastAPI application entry point for PINNeAPPle App.

`pinneapple_app/backend/main.py` · 112 linhas · testes: 41

nome	tipo	assinatura	o que faz
`health`	função	`()`	
`metrics`	função	`() -> str`	Prometheus text-exposition format — deliberately hand-rolled (no new dependency) rather than pulling in `prometheus-client` for two gauges. See `pinneapple_app/deploy/monitoring/prometheus.yml` for the scrape config pointed at this endpoint.
`info`	função	`()`	Return library capabilities summary.

`pinneapple_app.backend.routers`

Módulo sem docstring e sem texto.

`pinneapple_app/backend/routers/__init__.py` · 1 linhas · **sem teste direto**

`pinneapple_app.backend.routers.admin`

pinneapple_app.backend.routers.admin — internal curation/status view: cross-references the problem catalog, the model registry (`pinneapple_registry.ArtifactRegistry`), and dataset scenarios into one dashboard-ready payload. Gated by `core.admin_auth.require_admin` (a generic bearer token, see …

`pinneapple_app/backend/routers/admin.py` · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelSummaryItem`	classe	`(problem_id, latest_model_version, stage, dataset_scenarios)`	
`AdminSummaryResponse`	classe	`(registry_root, n_presets, n_architectures, models)`	
`PromoteRequest`	classe	`(problem_id, version, stage)`	
`admin_summary`	função	`() -> AdminSummaryResponse`	
`promote_model`	função	`(req) -> Dict[str, Any]`	Promotes a stored model version to a new stage. Being behind `require_admin` already IS the confirmation gate here (unlike `pinneapple_orchestration`'s flow-level `confirm=True` flag, which guards an unattended pipeline step instead of a human clicking a button in an admin UI).

`pinneapple_app.backend.routers.experiments`

Experiments API router — launch, monitor, and retrieve experiments.

`pinneapple_app/backend/routers/experiments.py` · 266 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`CollocationRequest`	classe	`(strategy, n_interior, n_boundary, n_initial, seed, use_geometry, domain_name)`	
`DataRequest`	classe	`(solver_key, n_snapshots, grid_resolution, t_end, use_solver)`	
`ModelRequest`	classe	`(name, extra_kwargs)`	
`ExperimentRequest`	classe	`(problem_name, custom_problem, collocation, data, models, metrics, epochs, lr, device, se…)`	
`ExperimentStatusResponse`	classe	`(experiment_id, status, progress, message)`	
`launch_experiment`	função	`(req, background_tasks)`	Launch a new experiment asynchronously. Returns experiment_id immediately.
`get_status`	função	`(experiment_id)`	
`get_results`	função	`(experiment_id)`	
`list_experiments`	função	`()`	
`experiment_ws`	função	`(websocket, experiment_id)`	WebSocket endpoint — streams training progress events as JSON.

`pinneapple_app.backend.routers.models`

Models API router.

`pinneapple_app/backend/routers/models.py` · 84 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`ModelListItem`	classe	`(name, family, description, supports_physics_loss, tags, recommended_for)`	
`MetricItem`	classe	`(key, label)`	
`list_models`	função	`(family=None)`	Return all available models, optionally filtered by family.
`list_families`	função	`()`	Return all model families.
`recommend_models`	função	`(problem_family, n=5)`	Return top-N recommended model names for a problem family.
`list_metrics`	função	`()`	Return all available evaluation metrics.
`get_model`	função	`(name)`	Return metadata for a single model.

`pinneapple_app.backend.routers.problems`

Problems API router.

`pinneapple_app/backend/routers/problems.py` · 86 linhas · **sem teste direto**

nome	tipo	assinatura	o que faz
`PresetListItem`	classe	`(name, family, description, tags, time_dependent, recommended_models, recommended_solver)`	
`CustomProblemRequest`	classe	`(name, equations, boundary_conditions, initial_conditions, domain_bounds, dim, pde_family…)`	
`list_problems`	função	`()`	Return all available preset problems with metadata.
`get_problem`	função	`(name)`	Return metadata for a single preset problem.
`validate_custom_problem`	função	`(req)`	Validate a custom problem definition without running it.

Exemplos de uso reais do repositório

Nenhum exemplo encontrado no repositório para este pacote.

Testes e status de validação

- Módulos com pelo menos um teste que os importa: 7 de 18.
- Funções de teste em arquivos que importam algum módulo do pacote: 48.

Limitações conhecidas

- *Nenhuma registrada.*

Volume 8 — Apps, exemplos, reproduções, scripts e CI

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

O que cada app, exemplo, reprodução de paper, script, benchmark e workflow de CI faz. A coluna *testes* conta funções de teste em arquivos que citam o caminho.

1. Apps (`apps/`)

64 itens.

item	o que faz	arquivos.py	testes
`apps/_shared`	apps/_shared: what every app gets: `appkit.py` is the glue every app under `apps/` imports (`apps/_shared` is on `PYTHONPATH` in each Dockerfile). The logic behind it lives in the library, with tests:	1	23
`apps/aero_optimizer`	Aircraft Design Optimizer: **A single-aisle airliner (A320 / 737 class), searched by a genetic algorithm, checked in OpenFOAM 3D, shown with its flow fields.**	9	0
`apps/anomaly_detector`	Physics Anomaly Detector: **Detect abnormal behaviour with data *and* physical knowledge: which balance broke, by how much, and which sensors are involved.**	2	4
`apps/benchmark_arena`	Physics AI Benchmark Arena (day 22): **Live:** https://arena.pinneapple.org · **Port:** 8105 · **Prefix:** `ARN`	3	5
`apps/brakecool_ai`	BrakeCool AI: Inverse design of the radial vanes of a vented brake rotor (vane count, vent gap, vane thickness) under a sustained heat load, through a frozen neural surrogate re-checked against the physics, plus a Monte Carlo of machining tolerances. Imported from PINNeAPPle-apps (`brakecool_ai`); the logic lives in `pinneapple_design.surrogate_inverse.brake_rotor`, this folder holds routes and the page.	2	5
`apps/calibration_lab`	Model Calibration Lab (day 18): **Fit model parameters to real measurements, with intervals only where the data determine them, checked on measurements the fit never saw.** Live at https://calibration.pinneapple.org (port 8101).	2	4
`apps/control_lab`	Physics-Based Control Lab: **Use physical models to compute better control actions, and compare them fairly with what runs today.**	2	4
`apps/data_health`	Engineering Data Health: **Upload a dataset, get a physics-aware quality report.**	2	0

item	o que faz	arquivos.py	testes
apps/data_standardizer	Engineering Data Standardizer: **Many files, many conventions, one physical schema.**	2	0
apps/deploy	Deploy the apps on one server: HeatSink Sizer, PCB Hotspot, Engineering Data Health, Engineering Data Standardizer, Simulation Metadata, the Form Compiler, Inverse Heat Lab, Simulation Preflight, Mesh Quality, Simulation Comparator, Engineering Model Lineage and the Simulation Interoperability Hub, the Aircraft Design Optimizer, the PINNeAPPle Observatory, the Inverse Model Studio, the Parameter Identifiability Lab, the Model Calibration Lab, the Engineering Evidence Report, the Physics Validation Lab and the PINNeAPPle Lab …	0	9
apps/design_optimization	Design Optimization: **A wing section redesigned live: less profile drag at the same lift, every step re-checked on the physics.**	2	2
apps/design_requirements	Form Compiler: **Pick the document format, drop the datasheets and specifications, get the form or datasheet compiled, with the source of every value and a list of what is missing or contradictory.**	3	0
apps/engineering_optimizer	Engineering Design Optimizer: **Better designs within physical and manufacturing limits, found by black-box optimisation on a fast model and confirmed by a high-fidelity one.** Day 17 of the PINNeAPPle app series; live at `https://optimize.pinneapple.org`.	2	3
apps/evidence_report	Engineering Evidence Report: **Turn experiments and simulations into reproducible technical evidence. A test is passed only if it ran.**	2	5
apps/flow_studio	PINNeAPPle Flow Studio: A node-based canvas for engineering flows, in the style of n8n: describe a problem, get one or more proposed flows, edit the equations and the methods, run, and click any node to see what it produced (metrics, logs, figures, a three.js 3-D view, the network architecture, tables). Everything a node does is a call into the PINNeAPPle library (`pinneapple_flow` plus the existing packages); this folder is only the HTTP shell and the React frontend. The same flows run without the GUI: `python -m …	1	19
apps/flowtrace	FlowTrace: **Upload a video or an image sequence of a seeded flow; get a cross-correlation PIV velocity field back. Any camera.**	2	3
apps/governed_card	GovernedCard: **A model card that cannot be issued without a physics check of the model behind it.**	2	3
apps/guiagent	Engineering GUI Agent: **guiagent.pinneapple.org** · port 8115 · day 27 extra · library: `pinneapple_tools.automation` (guide)	2	2

item	o que faz	arquivos.py	testes
`apps/heatsink_inverse_ai`	HeatSink Design AI: Gradient-based inverse design of a straight-fin CPU heat sink (fin count, height, thickness on a fixed 80 x 80 mm aluminium base) through a frozen neural surrogate, with every answer re-checked against the physics it learned. Imported from PINNeAPPle-apps (`heatsink_design_ai`); the logic now lives in `pinneapple_design.surrogate_inverse.heatsink_fins` and this folder holds only routes and the page.	2	5
`apps/heatsink_sizer`	HeatSink Sizer: **Plate-fin heat sinks, sized and verified by physics — in seconds.**	7	0
`apps/helio_tfc`	HelioTFC: **Where is the solar cycle's envelope heading? EMD of the daily sunspot number, the Hilbert amplitude of the ~11-year component, and a fit whose end values are tied exactly.**	2	4
`apps/hub`	PINNeAPPle Hub: The catalogue of ready-made tools, trained models, geometries, datasets, flows and examples of PINNeAPPle: search, see an item with its recorded run, copy its code, download its files, open it in Flow Studio, run a zoo model in the browser or on the server, and compose your own tool from existing blocks. Docs: docs/apps/hub.md.	2	14
`apps/identifiability_lab`	Parameter Identifiability Lab (day 25): **Before estimating parameters, check that the measurements can determine them.** Live at	2	4
`apps/interop_hub`	Simulation Interoperability Hub: **Upload your simulation result. Get it in a neutral engineering representation.**	2	0
`apps/inverse_heat`	Inverse Heat Lab: **The convection coefficient h from a few thermocouples, with a physics-informed neural network, in 1D, 2D and 3D, and every answer checked against an independent solution.**	7	0
`apps/inverse_studio`	Inverse Model Studio (day 21): **From measurements back to the unknown parameters and fields: one problem, five methods, the same data, checked.** Live at https://inverse-studio.pinneapple.org (port 8104). This is a separate app from the Inverse Heat Lab (`apps/inverse_heat`), which is unchanged. The studio reuses that lab's physics (the fin and plate physics-informed networks and the finite-volume references) through `pinneapple_analysis.inverse_problems`, and links to it.	2	3
`apps/mesh_quality`	Mesh Quality: **Where your mesh is bad, how bad, and what to do about it, before the solver finds out.**	2	0
`apps/meshbridge`	MeshBridge: **Upload a CFD / FEM / CAD mesh file; get its points and fields as NPZ, or a repaired triangle surface.**	2	4
`apps/meshless_lab`	Meshless Method Lab: **Meshless or mesh-based: measure where each wins on your problem.** Day 26 of the PINNeAPPle apps program; live at	2	5

item	o que faz	arquivos .py	testes
`apps/model_benchmarking`	Model Benchmarking: **Four PINN architectures trained live on the same viscous Burgers problem, ranked by accuracy and by a check on the PDE residual.**	2	2
`apps/model_gateway`	Physics AI Model Gateway (day 31): **Live:** https://models.pinneapple.org · **Port:** 8114 · **Prefix:** `MDL`	2	6
`apps/model_lineage`	Engineering Model Lineage: **How was this result produced? The digital thread from CAD to prediction.**	2	0
`apps/nozzle_screen`	NozzleScreen: **Is this rocket-nozzle idea in the right ballpark? An ideal-nozzle estimate in microseconds, and a PINN cross-check that says when the two disagree.**	2	2
`apps/observatory`	PINNeAPPle Observatory: **The sky and the solar system as they are right now, from where you stand, in 3-D.**	3	0
`apps/orbitwatch`	OrbitWatch: **What changed at this place between two satellite images? Per-band reflectance change and NDVI change, as a self-contained HTML report.**	2	5
`apps/parameter_discovery`	Parameter Discovery: **Can a physics-informed network recover a hidden material property from a few noisy readings and the PDE alone?**	2	2
`apps/patchdose_ai`	PatchDose AI: Smallest membrane-controlled release patch (area, membrane thickness, reservoir concentration) for a target release rate and wear time, found through a frozen neural surrogate and re-checked against Fick's law, plus the release curve over the wear time. Imported from PINNeAPPle-apps (`patchdose_ai`); the logic lives in `pinneapple_design.surrogate_inverse.transdermal_patch`.	2	4
`apps/pcb_hotspot`	PCB Hotspot: **Board-level PCB thermal analysis: junction temperatures, hotspots and what fixes them, in about a second.**	4	0
`apps/physics_copilot`	PhysicsCopilot: **A physics problem in plain language -> a trained PINN and the guardrail's re-computed report of it.**	2	3
`apps/physics_design`	Physics Design Optimizer: **Optimise parametric physical designs with explicit objectives, constraints and physics models, gradient-aware: compare gradient, evolutionary and Bayesian methods on the same problem and budget, and see which candidates the high-fidelity model confirms.** Day 23 of the PINNeAPPle app series; live at `https://design.pinneapple.org`.	2	2
`apps/pinn_arena`	PINNeAPPle Arena: **Port:** 8130 · **Prefix:** `PINN_ARENA` · **Service:** `pinnarena` (apps/deploy)	21	25

item	o que faz	arquivos.py	testes
apps/playground	PINNeAPPle Playground: Draw an airfoil, size a heat sink or drive a lunar-rover wheel: a PINNeAPPle zoo model answers in the browser, and the physics checks and the trust verdict sit next to every number. It is a static site, so nothing is installed and no server runs any code. The state lives in the URL.	1	2
apps/predictive_twin	Predictive Twin: **A digital twin of a real river gauge: calibrated on its own recent past, forecasting the present, flagging when the river stops following the forecast.**	2	3
apps/ridetune_ai	RideTune AI: Lighter cross-section (width x thickness) of a suspension arm modelled as a rectangular cantilever, for a fatigue life target, found through a frozen neural surrogate of the bending stress and re-checked against the closed form, plus local sensitivities of life and mass. Imported from PINNeAPPle-apps (`ridetune_ai`); the logic lives in `pinneapple_design.surrogate_inverse.suspension_arm`.	2	4
apps/rom_lab	Reduced Order Model Lab: **Cheaper models of your simulations, with the error on held-out data in sight.** Day 19 of the PINNeAPPle apps program; live at https://rom.pinneapple.org (port 8102).	3	4
apps/rover_mobility	RoverMobility: **Rigid-wheel mobility screening: drawbar pull against slip from a Bekker-Wong PINN, with each physics constraint re-checked on its own.**	2	2
apps/satscreen	SatScreen: **Quick conjunction screening: closest approach and a collision-probability estimate in Clohessy-Wiltshire relative motion, plus two-body + J2 propagation.**	2	3
apps/sensitivity_lab	Engineering Sensitivity Lab: **Which parameters really control the result, and which can you freeze?**	2	7
apps/simulation_campaign	Simulation Campaign Planner: **Which simulations really need to run, in which order, and the case files to run them.**	2	8
apps/simulation_comparator	Simulation Comparator: **How far is this result from the reference, field by field, and where.**	2	0
apps/simulation_metadata	Simulation Metadata API: **Drop a simulation, get its solver, mesh, boundary conditions, parameters and convergence verdict.**	2	0
apps/simulation_preflight	Simulation Preflight: **Before spending hours of solver time, find out whether your simulation is set up correctly.**	2	5

item	o que faz	arquivos.py	testes
`apps/simulation_runner`	Simulation Workflow Runner (day 27): **runner.pinneapple.org** · port 8110 · `RUNNER_*` environment	2	4
`apps/soundshape_ai`	SoundShape AI: Bass-reflex port radius and length for a target tuning frequency, found through a frozen neural surrogate and re-checked against the Helmholtz formula, plus a sweep of the trade-off between a small port and an accurate tuning. Imported from PINNeAPPle-apps (`soundshape_ai`); the logic lives in `pinneapple_design.surrogate_inverse.reflex_port`.	2	4
`apps/surrogate_speed`	Surrogate Speed: **A trained surrogate answers in microseconds; how much faster is that than a numerical solve, and how far apart are they?**	2	2
`apps/surrogate_studio`	Surrogate Studio (day 14): **Live:** https://surrogate.pinneapple.org · **Port:** 8097 · **Prefix:** `SUR`	2	4
`apps/text2part`	Text2Part: **Describe a part in plain language; get a mesh (STL) or a parametric CAD file (STEP) and its measured size.**	2	4
`apps/twin_ops`	Twin Ops — a commissioned digital twin, end to end: **HX-201, a fouled shell-and-tube exchanger.** The example exists to show the whole product, not a plot: a signed contract, a preflight that can refuse, a held-out backtest, an evidence file, and a drift ladder for when the plant moves away from the model.	1	24
`apps/twin_runtime`	Digital Twin Runtime: **Put a validated physical model or surrogate to work next to operational data, and know when to stop trusting it.**	2	6
`apps/uncertainty_lab`	Engineering Uncertainty Lab: **Know when a prediction is reliable, and when the case must go to the high-fidelity solver.**	2	8
`apps/validation_lab`	Physics Validation Lab: **Is this model or result physically and numerically consistent? Every check, with the record that it ran.**	3	5
`apps/verification_gap`	Verification Gap: **A design that passed its nominal safety check, and what its manufacturing tolerances do to that pass.**	2	2
`apps/verified_physics`	VerifiedPhysics: **The LLM proposes the physics; PINNeAPPle executes it and re-checks it mechanically, with the evidence attached.**	2	5
`apps/zoo_space`		0	0

2. Exemplos (`examples/`)

88 itens.

item	o que faz	arquivos.py	testes
`examples/README.md`	# Pinneapple Examples	0	0
`examples/architectures`	Pinneapple Models — Showcase Examples: These examples are **curated demos** focused on the full breadth of `pinneapple_models`: registry/catalog usage, neural operators, PINNs (including inverse parameters), ROMs, GNNs, and time-series transformers.	12	0
`examples/arena_pipelines`	00_list_datasets.py — List and inspect all available PINNeAPPle datasets.	10	0
`examples/autolab`	The self-optimizing engineer (PINNeAPPle Autonomous Physics Lab, demo 2: thermal).	1	0
`examples/autolab_aero`	The AI Aerodynamicist (PINNeAPPle Autonomous Physics Lab, stage 1: the scientific cycle without an agent).	1	0
`examples/autolab_aero_agent`	The AI Aerodynamicist, stage 2: an agent reads the mission and runs the lab only through controlled tools.	1	0
`examples/autolab_digital_twin`	The digital twin that knows when it is wrong (PINNeAPPle Autonomous Physics Lab, demo 3).	1	0
`examples/autolab_rover`	PINNeAPPle Autonomous Physics Lab, demo 4: the rover that learns the planet.	1	0
`examples/benchmark`	An equal-budget benchmark in one call: PINN vs FNO vs DeepONet on 1-D Burgers.	1	0
`examples/benchmark_suite`	PINNeAPPle Arena — Examples: These examples are meant to **show the full potential of `pinneapple_arena`**:	20	0
`examples/black_hole_weather`	Black-hole weather: forecasting the gas around a black hole, and knowing for how long to trust it: This is a reproduction and extension of **Duarte, Nemmen & Navarro (2022)**, *Black hole weather forecasting with deep learning: a pilot study*: - MNRAS 512, 5848; arXiv:2102.06242; - original code (MIT): black-hole-group/DL_BH_fluids.	5	20
`examples/bricks`	Self-modifying interlocking-brick builds: simulate, judge by physics, redesign, repeat.	1	20
`examples/calculix_cantilever`	Viga em balanço (CalculiX): Execução de referência do solver CalculiX (`ccx`) de uma viga em balanço de 1,0 × 0,1 × 0,1 (m).	0	0

item	o que faz	arquivos.py	testes
examples/calculix_pyvista_cantilever.py	CalculiX cantilever -> PyVista image: von Mises stress on the deformed beam.	1	0
examples/chain_of_whys	Chain of whys: four projects, four scales, the same reasoning (pinneapple_reason).	1	0
examples/condensation_falsification	Condensation falsification campaign: the measured runs of docs/research/condensation_falsification.md.	1	0
examples/core_primitives	Poisson PINN on the core primitives (Domain, Geometry, FunctionField, Mesh, Field).	2	0
examples/data_api	A PINN, a neural operator and an inverse problem trained from the same loader API.	1	0
examples/data_pipeline	03_prefetch_streaming_gpu.py	8	0
examples/design_from_intent	Design from intent: five requests through the same pipeline.	1	0
examples/electrodynamics	Electrodynamics & Electromagnetics Examples: Physics-informed neural network (PINN) solutions to classical electrostatics, magnetostatics and electromagnetics problems.	17	20
examples/end_to_end		6	0
examples/enso_forecasting	El Nino (ENSO) prediction: data-driven, physical, physics-informed and hybrid models under one honest protocol.	1	20
examples/first_example	Your first PINNeAPPle result in about a minute on a laptop CPU: solve Burgers' equation with a physics-informed network, check it against the exact solution and plot both.	1	0
examples/flight_dynamics	Verification of the flight-dynamics library: turbulence spectra and variances against MIL-F-8785C, trim of the four aircraft held for 10 s stick fixed, energy / angular-momentum conservation of the integrator, the lattice-in-the-loop lift against the steady vortex lattice. Writes ...	8	20
examples/flow_studio	Write the example flows of Flow Studio into `pinneapple_flow/flows/` (the flows the GUI's "Open..." lists).	3	0

item	o que faz	arquivos .py	testes
examples/free_surface	Free-surface flow (interFoam, VOF): `01_runner_slope_study.py`: a tapping jet falling into a melt runner held by a weir; the floor slope (+2°, 0°, -2°) changes the depth of metal under the jet. 2-D sections with `pp.cfd.FreeSurfaceFlow`, k sampled 5 cm above the floor while the solver runs and averaged in time. Needs OpenFOAM. Results: `docs/core_concepts/free_surface.md`.	1	0
examples/freecad_openfoam	FreeCAD -> OpenFOAM pipelines: three validated worked studies, a parameter sweep and a surrogate.	1	0
examples/geomechanics	Geomechanics with the material point method: `01_granular_column_collapse.py`: sand columns of aspect ratio 0.5 to 3 collapse on a rough floor; runout against Lube et al. (2005); frames written in GeoTaichi's layout and read back through the GeoTaichi bridge. About 6 minutes on one core. Results and limits: `docs/core_concepts/geomechanics_mpm.md`.	1	0
examples/geometry	pinneapple_geom examples: This folder focuses on geometry + meshing workflows that typically appear in Physics-AI / PINN pipelines:	8	0
examples/geometry_library	Wind on the CAARC standard tall building with OpenFOAM, through the generic STL path of `pp.cfd`.	3	0
examples/geophysics	Earth-system blocks in one script (about one minute on a laptop CPU):	1	0
examples/getting_started	Getting Started Examples — Tier 1: Explorer: These examples are your **first step into Physics AI**. Each one:	10	0
examples/handoff_cases	Engineering hand-off cases: Two self-contained case packages for testing an engineering-simulation agent (or an engineer) from a cold start. Each folder holds the task (`brief.md`), the geometry, the simulation set-up and scripts, results that were really computed, and verified references; `<case>.zip` is the same folder as one archive.	13	0
examples/hpo_experiments	pinneapple_researcher examples: These examples are designed to show the full workflow:	8	0
examples/industrial_presets	Presets industriais: Problemas de engenharia já montados, resolvidos e validados com o PINNeAPPle. Cada pasta tem a ficha (entradas SI com a fonte de cada valor padrão), os resultados rodados aqui, a validação com o erro medido e as figuras. Regenere com `python -m pinneapple_presets report all`.	0	0
examples/inspection	pinneapple_analysis.inspection example 01: eddy-current NDE end to end.	1	0
examples/latent	Latent surrogate with a trust card: error, speed and calibration against the full solver.	1	0
examples/lbm	3-D lattice Boltzmann wind tunnel: drag on a sphere against Stokes' law and the Schiller-Naumann correlation, in 32-bit and 16-bit storage of the distributions.	1	0

item	o que faz	arquivos.py	testes
examples/meshgraphnet	MeshGraphNet in PINNeAPPle: Implementation: - `pinneapple_neural/architectures/graphnn/mesh_graph_net.py`: the encoder, processor and decoder network, registered as `mgn` / `meshgraphnet`. - `mgn_dynamics.py`: the transient training recipe of Pfaff et al. It covers normalization, training noise, the Δv target, and auto-regressive rollout with boundary nodes held fixed.	6	6
examples/model_zoo	Model zoo quickstart: load a pre-trained model, predict, compare with the reference, validate, benchmark and fine-tune -- every step on the shipped, verified airfoil surrogate and its dataset.	1	0
examples/modelica	Modelica models as Physics AI data: Models: `models/DrainingTank.mo`, `models/LadleCooling.mo`. OpenModelica is used from the PATH or from Docker (`docker pull openmodelica/openmodelica:v1.24.4-minimal`). Results and discussion: `docs/core_concepts/modelica.md`.	4	6
examples/molecular	A protein on a gold surface: orient, build the Au(111) slab, run a short OpenMM MD and visualise it.	1	0
examples/motor_magnetics	Surface-PM machine (4 poles, 24 slots, nonlinear iron): cogging torque, torque vs current angle, back-EMF and losses with the 2D A_z FEM of pinneapple_simulation.electromagnetics.	1	0
examples/near_random_forecasting	Visual pack of the near-random forecasting study: four Blender Cycles renders and two 2-D figures.	2	2 0
examples/non_hermitian	Exceptional-point tuning in PT-symmetric dimers: the no-go theorem (Zenodo 23287622) checked symbolically and numerically, the three routes around it, the hypothesis map as a physics audit, and the figure.	1	0
examples/numerical_solvers	pinneapple_solvers examples: This folder contains runnable scripts showcasing the **numerical-solvers layer**:	13	0
examples/nvidia	PINNeAPPle with the NVIDIA stack: Every example puts PINNeAPPle at the centre (the problem, the reference solvers, the geometry checks, the physics checks and the trust decision) and joins it with a different combination of stacks. The glue lives in `pinneapple_tools.nvidia`; each example is a thin `run.py` that writes `results/architecture.png` (PINNeAPPle's role and each stack's role, and what was *not* run here), result figures, `summary.json` and `run_info.json` (versions, hardware, timings, which components ran).	25	3 9
examples/openfoam_campaign	OpenFOAM parameter campaign: One base case, six runs that change the jet speed and elevation, split between two machines.	1	0
examples/pde_environment	End-to-end demonstration for the astrophysics specialization: train a PINN on the `kepler_two_body_orbit` preset and compare the trained network's predicted trajectory against the exact (Kepler's-equation) solution.	9	1 0
examples/physics_audit	Physics audit on pipe friction: models judged by their physics, not only by their past errors.	1	0
examples/physics_check	A stand-in for a drag surrogate Cd(Re, aoa) fitted on wind-tunnel points (Re 1e5 .. 1e6, aoa 0 .. 10 deg).	7	2 2

item	o que faz	arquivos.py	testes
examples/physics_condensation	Adaptive Physics Condensation, track 1: the measured benchmark of docs/research/physics_condensation.md.	1	0
examples/physics_db	Quickstart: discover a dataset via CMR/earthaccess and inspect variables.	7	0
examples/physics_ensemble	Side by side: the five-family ensemble reacting only to past errors (left) and with the current-case physics residual against the level each model has in its own training regime (right, `residual_lookahead` + `residual_baseline`), on the same stream of cases.	5	3
examples/physics_learning_verification	Physics learning verification, track 3: the measured study of docs/research/physics_condensation.md (section 6).	1	0
examples/physics_module	Hybrid model: a coarse finite-element solve plus a neural correction, trained end to end.	1	0
examples/physics_optimizer	One constrained design problem through one interface, six optimizers.	1	0
examples/physics_query	Physics query engine, track 2: the measured demonstration of docs/research/physics_condensation.md (section 7).	1	0
examples/pinn_solver	pinneapple_physics.pinn_solver examples: This folder focuses on the **PINN-specific** layer of PINNeAPPle.	10	0
examples/pineapple_and_physicsnemo	Wave equation 1D u_tt - (c^2 u_x)_x = 0	4	0
examples/pineapple_benchmark	01_pde_comparison.py — PINNeAPPle PDE Architecture Benchmark	2	0
examples/pineapple_cosim	01_spring_mass_pinn.py — 2-DOF Coupled Spring-Mass System with Two PINN Models	1	0
examples/problem_designer	pinneapple_problemdesign examples: This folder showcases **problem elicitation + specification + planning**.	5	22

item	o que faz	arquivos.py	testes
examples/proof_graph	Proof-graph experiment: does the no-go theorem for linear exceptional-point tuning (PT dimers, Zenodo 23287622) extend to PT tetramers whose bonds are modulated independently? An open question of the paper (Q2), attacked as a graph of assumptions, lemmas and a theorem, each with exact (sympy) and …	1	0
examples/qualitative	Qualitative preview of heat-sink variants, then the quantitative check.	3	0
examples/quickstart	01 — First look: what is installed, and what is already solved for you.	13	0
examples/reproductions	Reproduce Gated X-TFC (Dwivedi, Schiassi, Sigovan, Sixou, arXiv:2510.01039v1) with the protocol frozen in TARGETS.md.	3	0
examples/results_chassis		0	0
examples/rigid_body	Rigid bodies with NVIDIA Newton: check the solver, then run a small scene.	1	0
examples/science	PINNeAPPle science: experiments: Every row is a quantity computed by an experiment of `pinneapple_science` (`pp.science`) and compared with its reference; the values come from the committed `result.json` files. Regenerate with `python -m pinneapple_science report all`. The map of every area (available, partial, planned) is ATLAS.md and atlas.html.	0	0
examples/scratch	scratch: This directory holds working and experimental scripts that were used during development (benchmarking snippets, one-off result-generation and report-embedding utilities, etc.) but were never organized into the curated topic-folder examples (`examples/architectures/`, `examples/geometry/`, and so on). They may be less polished, less documented, or narrower in scope than the rest of `examples/`, and some assume they're run from the repository root with the project installed in editable mode — …	6	0
examples/security	Secure Physics AI pipeline, end to end, with pinneapple_security.	2	0
examples/solver_api	The same problem through one call: a classical FEM solve and a PINN, compared against the exact solution.	1	0
examples/solvers	Examples: time-series decomposition solvers.	3	0
examples/sports	Ball flight: drag, Magnus and gravity from tracked positions; where the inverse problem is posed, and whether the physics predicts real broadcast-tracking flights better than data-only extrapolation and boosted trees.	5	0
examples/studio	Any simulation result in 3D, with three calls: load, export, render.	7	6
examples/systems	Component registry + toolbox + physics-informed fit, end to end.	1	0

item	o que faz	arquivos.py	testes
examples/time_series	PINNeAPPle Time Series Examples: This folder contains runnable scripts that demonstrate **forecasting workflows** supported by `pinneapple_timeseries`.	10	0
examples/topology_optimization	Half MBB beam and cantilever: SIMP compliance minimisation with pinneapple_design.topology.	3	0
examples/trainer	pinneapple_train example 02: DataModule + robust preprocessing.	5	0
examples/use_cases	Compact range quiet zone by physical optics: validation, feed gain x reflector size (x rolled edge) sweep, a surrogate with held-out error and the trade-off map. Generic example parameters (3 GHz, F = 1.5 m, a 1.2 m square section with a superellipse junction, a 0.6 m quiet zone at 3 m).	34	17
examples/validation	PINN, steady heat conduction: -lap T = q on the unit square, T = 0 on the walls (manufactured source, exact T).	12	4
examples/validation_arena	Run an external model (for instance the FNO or MeshGraphNet of a PhysicsNeMo experiment) through the hardened defence of the validation arena (#707), and compare each of its outputs with the reference solver.	2	0
examples/visualizations	Run all PINNeAPPle visualization examples.	8	0
examples/vs_physicsnemo	PINNeAPPle vs PhysicsNeMo — Comparison & Integration Guide: > As linhas marcadas como medidas vêm do benchmark entre frameworks > (`benchmarks/vs_frameworks/REPORT.md`): só CPU (Intel Xeon, 1 thread por run), > 3 a 5 seeds, nvidia-physicsnemo 2.2.2 e nvidia-physicsnemo.sym 2.4.0. Nada foi medido em GPU, que é onde o > PhysicsNeMo é otimizado. As outras linhas descrevem funcionalidades e não foram medidas.	7	22
examples/weather_forecasting	Weather forecasting on ERA5: how many days a model is good for, and extreme events: A global forecast model trained with `pinneapple_physics.weather` on ERA5 (WeatherBench2, public, no account), scored by lead time against persistence, climatology and the published IFS HRES, Pangu-Weather, Keisler and NeuralGCM forecasts on the same 2020 cases, with the useful horizon (ACC >= 0.6) and the no-skill horizon (RMSE reaches climatology), a per-forecast horizon from an ensemble, and seven extreme events forecast against what happened.	2	6
examples/worldmodel	Physics World Model — end-to-end example.	1	0

3. Reproduções de papers

Sem texto.

4. Scripts (`scripts/`)

24 itens.

item	o que faz	ar-quivos .py	testes
scripts/README.md	# scripts/ — Runbook do MVP (Flow Obstacle 2D)	0	0
scripts/app_helpers_baseline.txt	# Existing duplicates of library helpers in apps/ and examples/ (scripts/check_app_helpers.py).	0	4
scripts/arena		5	0
scripts/astro	Build the compact star and constellation catalogues shipped in pinneapple_physics/astro/data.	3	0
scripts/build_method_status.py	Compute the validation status of every item in pinneapple_catalog.methods from the test suites.	1	2
scripts/check_app_helpers.py	Flag reusable logic that is written in apps/ or examples/ instead of being imported from the library.	1	0
scripts/check_dist.py	Check the built distributions: every wheel package is in both the sdist and the wheel, and imports when the sdist is installed in a clean virtual environment (roadmap I5, #92).	1	0
scripts/check_example_imports.py	Check that every pinneapple* import in examples/ and templates/ resolves.	1	0
scripts/check_experiment_manifests.py	CI gate: every examples/* folder has a valid experiment.json and every Lab run reads as a valid manifest.	1	0
scripts/check_release_version.py	Fail unless the git tag, pyproject.toml, CITATION.cff, pinneapple.**version** and CHANGELOG.md agree on the release version.	1	0
scripts/cosmos	scripts/cosmos — Cosmos (LLM / World Foundation Model) integration: Esses scripts assumem que você tem um endpoint HTTP para Cosmos (ou gateway interno) que aceita JSON e retorna JSON.	3	0
scripts/dev		1	0
scripts/example_imports_baseline.txt	examples/architectures/50_timeseries_transformer_forecast_toy.py	0	2
scripts/fault_injection_matrix.py	Regenerate the detection matrix of the validation checks (#697): every check against the fault library on Darcy 2-D and Burgers 1-D.	1	0
scripts/geometry_library.py	Generate, verify and publish the geometry library files.	1	21
scripts/issues	Snapshot the open issues of the repository into the contribute page (docs/org/contribute/index.html).	1	0
scripts/mvp	scripts/mvp — scripts locais “cola-tudo” do MVP: Aqui ficam scripts para: - gerar/copiar JSONs do bundle (conditions/manifest) - validar bundle contra schema + presença de arquivos - rodar uma matriz de runs da Arena (native + physicsnemo)	3	0
scripts/omniverse	scripts/omniverse — executar dentro do Omniverse Kit: Esses scripts rodam DENTRO do Omniverse Code / Isaac Sim.	2	0

item	o que faz	arquivos .py	testes
scripts/publish_issues.sh	#!/usr/bin/env bash	0	0
scripts/reference	Build the module reference at docs/org/reference/: every package, subpackage and module of the repository with its docstring and public API (read from the source with ast, nothing is imported), plus the curated explanations and the examples run by run_examples.py (code, printed output, ...	3	0
scripts/run_colab_notebooks.py	Execute the Colab notebooks headless, top to bottom, and check that the committed copies carry no outputs.	1	0
scripts/update_imports.py	Step 1: Update all Python import statements to use the new mega-module paths.	1	0
scripts/validate_model_card.py	Governance-as-code gate for the model hub (see ROADMAP_PHYSICS_AI_HUB.md, P1.5): reject a model-card push with no computed validation_metrics or no reference_source citation, the same schema check ModelCard.validate() already implements -- this script is just the CI-callable/CLI ...	1	8
scripts/zoo	Build the zoo's bundled assets, model cards and manifest.json from the committed sources.	10	28

5. Benchmarks (benchmarks/)

7 itens.

item	o que faz	arquivos .py	testes
benchmarks/data_io_bench.py		1	0
benchmarks/lbm_vortex_shedding.py	Vortex shedding with the D2Q9 LBM solver: Strouhal number of a cylinder and a NACA 4412.	1	0
benchmarks/lpm_ellipse_design_space.py	Large Physics Model on an exact design space: potential-flow Cp on ellipses.	1	0
benchmarks/shard_balance_bench.py		1	0
benchmarks/ssqn_helmholtz.py	Paper benchmark: 2D Helmholtz with BFGS vs SSBFGS vs SSBroyden vs Adam.	1	0
benchmarks/ssqn_paper_benchmarks.py	Paper benchmarks for the self-scaled quasi-Newton optimizers (BFGS / SSBFGS / SSBroyden vs Adam).	1	0
benchmarks/vs_frameworks	Cross-framework benchmark: runners, driver and report (see PROTOCOL.md).	15	0

6. Integração contínua (.github/workflows/)

workflow	nome	gatilhos	jobs	o que faz
docs.yml	Deploy Docs to GitHub Pages	push, workflow_dispatch, contents	deploy	
draft-pdf.yml	Draft PDF	push, pull_request, workflow_dispatch	paper	

work-flow	nome	gatilhos	jobs	o que faz
extras.yml	Install extras	push, pull_request, workflow_dispatch	extras	
observatory.yml	Observatory (browser tests)	push, pull_request, workflow_dispatch	browser	
physics-check.yml	Physics check (pinneapple check)	push, pull_request, workflow_dispatch, contents	passing-fixtures, failing-fixtures, darcy-trust, unit-tests	
playground.yml	Zoo web export and Playground (ONNX parity, JS runtime, browser smoke test)	push, pull_request, workflow_dispatch	onnx-parity, browser, docker	
publish-zoo.yml	Publish zoo to the Hugging Face Hub	workflow_dispatch, push, contents, group	publish, geometries	
release.yml	Release to PyPI	push	build, publish	
security.yml	Security	pull_request, push, schedule, workflow_dispatch, contents (cron 17 5 * * 1)	scan, tests, image	
tests.yml	Tests (audit + core suite)	push, pull_request, workflow_dispatch	test, generated, first-example, colab-notebooks	
training-callbacks.yml	Training callbacks (Lightning, PhysicsNeMo, Hugging Face)	push, pull_request, workflow_dispatch, contents	callbacks	
validation-gallery.yml	Validation gallery	push, pull_request, workflow_dispatch, contents	gallery	

7. Documentação existente (`docs/`)

190 arquivos Markdown. Lista com o título de cada um:

arquivo	título
docs/SUMMARY.md	SUMMARY
docs/apps/anomaly_detector.md	Physics Anomaly Detector (day 28)
docs/apps/benchmark_arena.md	Physics AI Benchmark Arena
docs/apps/calibration_lab.md	Model Calibration Lab (day 18)
docs/apps/control_lab.md	Physics-Based Control Lab (day 29)
docs/apps/evidence_report.md	Engineering Evidence Report (day 30)
docs/apps/hub.md	PINNeAPPle Hub
docs/apps/identifiability_lab.md	Parameter Identifiability Lab (day 25)
docs/apps/inverse_studio.md	Inverse Model Studio (day 21)
docs/apps/meshless_lab.md	Meshless Method Lab (day 26)
docs/apps/model_gateway.md	Physics AI Model Gateway
docs/apps/rom_lab.md	Reduced Order Model Lab (day 19)
docs/apps/surrogate_studio.md	Surrogate Studio

arquivo	título
docs/apps/twin_runtime.md	Digital Twin Runtime (day 20)
docs/apps/validation_lab.md	Physics Validation Lab (day 24)
docs/architecture/execution_model.md	Execution Model
docs/architecture/package_layers.md	Package Layers
docs/architecture/system_overview.md	System Overview
docs/archive/overview_en.md	PINNeAPPle — Library Overview
docs/archive/overview_pt.md	PINNeAPPle — Visão Geral da Biblioteca
docs/assets/apps/README.md	App images (30 apps with PINNeAPPle, days 1–14)
docs/assets/proof_graph/pt_chain_no_go_report.md	No linear exceptional-point tuning in PT-symmetric tetramers
docs/core_concepts/adaptive_forecasting.md	Adaptive forecasting: switching and combining models online
docs/core_concepts/astro.md	Positional astronomy and astrophysics (pinneapple_physics.astro)
docs/core_concepts/astroml.md	astroML: periodograms, deconvolved densities and fits with errors
docs/core_concepts/backend.md	Backend
docs/core_concepts/black_hole_weather.md	Black-hole weather: forecasting accretion flows with a trust horizon
docs/core_concepts/core_primitives.md	Core primitives: Domain, Geometry, Mesh, Field
docs/core_concepts/dashboard.md	Animation-centred dashboards
docs/core_concepts/earth_system.md	Earth-system building blocks and data assimilation
docs/core_concepts/fitmesh.md	From a problem description to checked CAD, fit-for-purpose meshes and one report
docs/core_concepts/free_surface.md	Free-surface flow: runners, channels, launders
docs/core_concepts/geomechanics_mpm.md	Geomechanics with the material point method
docs/core_concepts/geometry_domain.md	Geometry & Domain
docs/core_concepts/lab.md	PINNeAPPle Lab: experiments at scale, and a database of results and datasets
docs/core_concepts/lbm_wind_tunnel.md	Lattice Boltzmann wind tunnel and the FluidX3D bridge
docs/core_concepts/lunar_rover_terramechanics.md	Lunar rover terramechanics in 3D
docs/core_concepts/model.md	Model
docs/core_concepts/modelica.md	Modelica models as Physics AI data
docs/core_concepts/molecular_visualization.md	Molecular visualisation: proteins on metal surfaces
docs/core_concepts/motor_and_topopt_audit.md	Audit: electric-motor magnetics and topology optimization
docs/core_concepts/multiscale_ensembles.md	Multiscale ensembles: scales, filters and closures
docs/core_concepts/newton_rigid_body.md	Rigid bodies with NVIDIA Newton
docs/core_concepts/non_hermitian.md	Non-Hermitian physics: PT-symmetric dimers and exceptional-point tuning
docs/core_concepts/openfoam_acceleration.md	Faster implicit pressure solves in OpenFOAM
docs/core_concepts/openfoam_campaigns.md	OpenFOAM parameter campaigns on several machines

arquivo	título
docs/core_concepts/overview.md	Core Concepts
docs/core_concepts/physical_sample.md	PhysicalSample
docs/core_concepts/physics_audit.md	Physics audit: which model to use, when to trust it, when to revisit the physics
docs/core_concepts/physics_ensembles.md	Adaptive ensembles of physics models
docs/core_concepts/physics_optimizer.md	PhysicsOptimizer
docs/core_concepts/pinn.md	PINN / Physics
docs/core_concepts/problem_definition.md	ProblemDefinition
docs/core_concepts/proof_graphs.md	Proof graphs: mathematics with the same evidence discipline
docs/core_concepts/qualitative_preview.md	Qualitative preview: physical impact of geometry changes before the numbers
docs/core_concepts/reduced_order_models.md	Reduced-order models
docs/core_concepts/researcher_benchmarking.md	Researcher & Benchmarking
docs/core_concepts/security.md	Security for scientific and industrial Physics AI (pinneapple_security)
docs/core_concepts/solver.md	Solver
docs/core_concepts/studio.md	3D studio and external flow
docs/core_concepts/training_pipeline.md	Training Pipeline
docs/core_concepts/veriphysics.md	Verification and evidence (pinneapple_veriphysics)
docs/core_concepts/viewers_and_twin_standards.md	Other 3-D viewers and digital-twin standards
docs/dev/ANALISE_FEATURES_COMERCIAIS.md	O que as ferramentas comerciais e os outros CFDs têm que o PINNeAPPle ainda não tem
docs/dev/AUDIT_REPORT.md	Evidenced audit report
docs/dev/CATALOGO_METODOS.md	Catálogo de métodos do PINNeAPPle
docs/dev/LATENT_AND_CYBER_EVALUATION.md	Latent-space methods and cybersecurity applications: evaluation and roadmap
docs/dev/LIBRARY_CONSOLIDATION.md	Library consolidation: reusable logic outside the library
docs/dev/MARKET_SCAN_LOVABLE_PHYSICS_AI_2026-09-13.md	Pesquisa: Physics AI / Simulação / CAE / Digital Twin em lovable.app
docs/dev/PEDIDOS_2026-09-24.md	Pedidos e insights de 2026-09-24 — registro e decisões
docs/dev/REA_INTEGRATION.md	REA no PINNeAPPle — integração, fluxo de harvest e mapa do ecossistema
docs/dev/ROADMAP_PHYSICS_AI_HUB.md	PINNeAPPle → "the Hugging Face of Physics AI": full roadmap
docs/dev/issues/001-geometric-similarity-score.md	
docs/dev/issues/002-active-simulation-loop.md	
docs/dev/issues/003-raw-geometry-field-surrogate.md	
docs/dev/issues/004-rom-fmu-export.md	
docs/dev/issues/005-pinneapple-mcp-server.md	

arquivo	título
docs/dev/issues/006-topology-optimization-simp.md	
docs/dev/issues/007-solution-adaptive-amr.md	
docs/dev/issues/008-multi-fidelity-cokriging.md	
docs/dev/issues/009-hybrid-rom-online-calibration.md	
docs/dev/issues/010-pce-sobol.md	
docs/dev/issues/011-geometry-encoders.md	
docs/dev/issues/012-lbm-d3q27-cumulant-voxel.md	
docs/dev/issues/013-ifno-incremental-modes.md	
docs/dev/issues/014-multiadam.md	
docs/dev/issues/015-mapdl-rst-reader-pyfluent.md	
docs/dev/issues/016-browser-pde-preview.md	
docs/dev/issues/017-physicsnemo-missing-models.md	
docs/dev/issues/018-rea-native-ghidra.md	
docs/dev/issues/019-adjoint-mma-bspline.md	
docs/dev/issues/020-design-study-object.md	
docs/dev/issues/021-rbf-mesh-morphing.md	
docs/dev/issues/022-inference-cache-viewer.md	
docs/dev/issues/023-the-well-adapter.md	
docs/dev/issues/024-procedural-pde-datasets.md	
docs/dev/issues/025-neural-corrected-stepper.md	
docs/dev/issues/026-differentiable-linear-solves.md	
docs/dev/issues/027-nektar-bridge.md	
docs/dev/issues/028-jaxfluids-bridge.md	
docs/dev/issues/029-paraview-catalyst-insitu.md	
docs/dev/issues/030-nn-in-weak-forms.md	
docs/dev/issues/031-warp-backend.md	
docs/dev/issues/032-rbf-kriging-surrogates-outliers.md	
docs/dev/issues/033-onshape-configuration-sampler.md	

arquivo	título
docs/dev/issues/034-rescale-job-connector.md	
docs/dev/issues/035-comsol-mph-bridge.md	
docs/dev/model_dataset_inventory.md	Model and dataset inventory for the zoo (2026-10-13)
docs/dev/pr/feat-rea-bridge.md	Summary
docs/getting_started/first_pinn.md	First PINN
docs/getting_started/installation.md	Installation
docs/getting_started/project_layout.md	Project Layout
docs/getting_started/quickstart.md	Quickstart
docs/getting_started/start_here.md	Start here (docs)
docs/guides/adding_an_experiment.md	Adding an experiment / Como adicionar um experimento
docs/guides/agent_workbench.md	Agent Workbench: coding and engineering agents on a local model
docs/guides/antenna_design.md	Antenna design: an end-fire Vivaldi with an external FEM solver
docs/guides/autonomous_lab.md	Autonomous Physics Lab: an AI that manages evidence for engineering decisions
docs/guides/autonomous_physics_lab.md	Autonomous Physics Lab
docs/guides/campaign_planner.md	Simulation Campaign Planner: which simulations need to run
docs/guides/chain_of_whys.md	Chain of whys: why each element of a design exists, computed
docs/guides/check_power.md	Check power: how often each check catches a known defect
docs/guides/check_power_matrix.md	Check power: the detection matrix
docs/guides/choosing_a_method.md	Choosing a method: PINN, neural operator, graph network or the solver
docs/guides/colab_notebooks.md	Colab notebooks
docs/guides/compact_range.md	Compact range quiet zone: feed gain × reflector size
docs/guides/dataset_audit.md	Dataset physics audit: does the data solve its PDE?
docs/guides/design_from_intent.md	Design from intent
docs/guides/design_report.md	Design reports without an LLM: `Datasheet` and the self-planning study
docs/guides/design_studies.md	Design studies: one problem, many optimisers, verified candidates
docs/guides/electromagnetics.md	Electromagnetics and electrodynamics
docs/guides/engineering_design_optimizer.md	Engineering Design Optimizer (app, day 17)
docs/guides/engineering_gui_agent.md	Engineering GUI agent: run engineering software and keep the evidence
docs/guides/enso_forecasting.md	Predicting El Niño: physical, physics-informed, data-driven and hybrid models under one protocol
docs/guides/evidence_ledger.md	Evidence ledger: a test passed only if it ran

arquivo	título
docs/guides/flight_dynamics.md	Flight dynamics, turbulence and take-off
docs/guides/flow_schema.md	The flow schema (pinneapple-flow/1) and how another app reuses it
docs/guides/fmi_compatibility.md	FMI import compatibility
docs/guides/football_physics.md	Football: where physics helps on public tracking data
docs/guides/freecad_openfoam_pipelines.md	FreeCAD -> OpenFOAM pipelines
docs/guides/geometry_library.md	Geometry library
docs/guides/identifiability_calibration_inversion.md	Identifiability, calibration and inverse problems
docs/guides/industrial_presets.md	Industrial presets
docs/guides/interlocking_bricks.md	Self-modifying brick builds
docs/guides/mcp_server.md	PINNeAPPle from any coding agent: the MCP server
docs/guides/metamorphic_tests.md	Metamorphic tests: checking a surrogate without ground truth
docs/guides/model_zoo.md	Pre-trained models and datasets (pp.zoo, pp.datasets)
docs/guides/near_random_forecasting.md	Forecasting near-random series without overfitting
docs/guides/observatory.md	The PINNeAPPle Observatory
docs/guides/physics_checks_in_ci.md	Physics checks in CI
docs/guides/physics_design_optimizer.md	Physics Design Optimizer (app, day 23)
docs/guides/physics_residual_attack_detection.md	Detect false data injection with physics residuals
docs/guides/physicsnemo_trust.md	PhysicsNeMo trains, PINNeAPPle says when to trust
docs/guides/pinn_validation.md	PINN validation: what the training loss does not say
docs/guides/pinneapple_llm_quality_pipeline.md	Quality pipeline for LLM-generated physics and twin artifacts
docs/guides/playground.md	Playground: physics surrogates in the browser
docs/guides/pp_validate.md	pp.validate: one call, one validation report
docs/guides/recreate_a_wear_twin.md	Recreate a 3D wear twin (refractory, liner, pipe wall)
docs/guides/residual_error_bound.md	From residual to error bound: how wrong is this prediction, without the true answer?
docs/guides/rollout_validation.md	Rollout validation: when an autoregressive surrogate stops being trustworthy
docs/guides/rom_and_meshless_studies.md	Reduced-order models and meshless methods: measured, not assumed
docs/guides/rover_learns_the_planet.md	The rover that learns the planet
docs/guides/running_a_benchmark.md	Running a benchmark
docs/guides/scale_aware_ensembles.md	Choose a model by the scale of the question
docs/guides/science.md	Science by area (pp.science)
docs/guides/sensitivity_lab.md	Engineering Sensitivity Lab
docs/guides/spectral_fidelity.md	Spectral fidelity: what relative L2 does not see
docs/guides/surrogates_arena_gateway.md	Surrogates from simulation results: study, regime arena, gateway

arquivo	título
docs/guides/topology_optimization.md	Topology optimisation
docs/guides/training_callbacks.md	Physics checks during training (Lightning, PhysicsNeMo, Hugging Face)
docs/guides/trusted_latent_surrogate.md	A latent surrogate that says when not to trust it
docs/guides/uncertainty_and_sensitivity.md	Uncertainty and sensitivity studies on any model
docs/guides/uncertainty_lab.md	Engineering Uncertainty Lab
docs/guides/unit_cell_benchmark.md	Periodic unit cell: an FNO over frequency × scan, and an equal-budget benchmark against Bayesian optimisation
docs/guides/validation_arena.md	Red team / blue team for the validation checks
docs/guides/validation_gallery.md	Validation gallery: one worked example per model family and per physics
docs/guides/validation_methods_reference.md	Validation methods reference
docs/guides/workflow_runner.md	Simulation Workflow Runner: campaigns as traceable, resumable workflows
docs/guides/zoo_catalogue.md	Zoo catalogue
docs/index.md	PINNeAPPle
docs/lab/coverage.md	Experiments coverage
docs/philosophy.md	Philosophy
docs/research/condensation_falsification.md	Falsification campaign for physics condensation — pre-registration
docs/research/physics_condensation.md	Adaptive Physics Condensation: geometry-independent thermal models of chip packages
docs/security/audit.md	Security audit: the PINNeAPPle web apps and their deployment
docs/security/runbook.md	Operator runbook: PINNeAPPle apps
docs/security/server.md	Server hardening: the host that runs the PINNeAPPle apps
docs/strategy/STRATEGY_PHYSICS_AI.md	Physics AI Strategy — structure and first implementation
docs/validation_status.md	Validation status
docs/why.md	Why PINNeAPPle?

Volume 9 — Índice alfabético, glossário e apêndice numérico

> Parte do Mapa completo do PINNeAPPle. Gerado por `docs/pt/_mapa/gerar.py` a partir do inventário do commit `e53cdcfb04`.

Índice de todas as classes e funções públicas de nível de módulo, o glossário e os números do inventário.

1. Índice alfabético

8.887 entradas. **C** = classe, **f** = função. O caminho é o módulo onde o nome é definido (ver o capítulo do pacote no volume indicado).

A

- `a320` f — `pinneapple_simulation.flight.catalog` (v03)
- `abaqus_inp_to_upd` f — `pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader` (v03)
- `acc` f — `pinneapple_physics.weather.evaluate` (v02)
- `acc` f — `pinneapple_systems.time_series.enso.evaluation` (v06)
- `acceleration_from_displacement` f — `pinneapple_simulation.numerical_solvers.nonlinear_beam_fem` (v03)
- `accelerations` f — `pinneapple_physics.astro.nbody` (v02)
- `AcceptanceGates` C — `pinneapple_systems.digital_twin.product.spec` (v06)
- `accretion_scene` f — `pinneapple_physics.blackhole.twin` (v02)
- `accretion_view` f — `pinneapple_physics.astro.lensing` (v02)
- `AccretionFlow` C — `pinneapple_physics.blackhole.hydro` (v02)
- `AccretionFlowRun` C — `pinneapple_lab.experiments.accretion` (v07)
- `accuracy` f — `pinneapple_condense.figures` (v02)
- `accuracy_vs_modes` f — `pinneapple_condense.benchmark` (v02)
- `acf_pacf` f — `pinneapple_systems.time_series.audit.tests` (v06)
- `acid_fraction` f — `pinneapple_systems.process_components.reaction_kinetics` (v06)
- `acoustic_field` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `act` f — `pinneapple_autolab.core` (v07)
- `Action` C — `pinneapple_lab.next_steps` (v07)
- `ActionFailed` C — `pinneapple_tools.automation.adapters.base` (v07)
- `ActionResult` C — `pinneapple_tools.automation.adapters.base` (v07)
- `activation_derivative` f — `pinneapple_simulation.numerical_solvers.xtfc_general` (v03)
- `active_learning` f — `pinneapple_simulation.electromagnetics.antenna.design` (v03)

- `active_learning_retrain_loop` f — `pinneapple_neural.physicsnemo_bridge.active_learning_bridge` (v04)
- `ActiveLearningConfig` C — `pinneapple_data.active_learning` (v04)
- `ActiveLearningResult` C — `pinneapple_design.design_optimizer.verify` (v06)
- `ActiveSampler` C — `pinneapple_core.data` (v01)
- `Actuator` C — `pinneapple_simulation.vehicle.spec` (v03)
- `actuator` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `actuator_response_rhs` f — `pinneapple_systems.process_components.control_valve` (v06)
- `actuator_series` f — `pinneapple_systems.excitation` (v06)
- `AdaHedge` C — `pinneapple_physics.online_learning` (v02)
- `adapt_model` f — `pinneapple_adaptation` (v04)
- `adapt_model` f — `pinneapple_arena.pointwise.adapters` (v04)
- `adapt_output` f — `pinneapple_physics.multiscale.comparison` (v02)
- `adapt_problem` f — `pinneapple_decision.adapter` (v05)
- `AdaptedProblem` C — `pinneapple_decision.adapter` (v05)
- `AdaptedSequenceDataset` C — `pinneapple_neural.trainer.datamodule` (v04)
- `Adapter` C — `pinneapple_core.workflows.adapters` (v01)
- `Adapter` C — `pinneapple_tools.automation.adapters.base` (v07)
- `adaptive` f — `pinneapple_flow.nodes_ext.core_ts` (v07)
- `AdaptiveCollocationConfig` C — `pinneapple_neural.trainer.collocation` (v04)
- `AdaptiveCollocationSampler` C — `pinneapple_neural.trainer.collocation` (v04)
- `AdaptiveCollocationTrainer` C — `pinneapple_data.active_learning` (v04)
- `AdaptiveConformal` C — `pinneapple_physics.online_learning` (v02)
- `AdaptiveForecaster` C — `pinneapple_systems.time_series.adaptive` (v06)
- `AdaptivePrefetchConfig` C — `pinneapple_data.prefetch_adaptive` (v04)
- `AdaptivePrefetchZarrUPDIterable` C — `pinneapple_data.prefetch_adaptive` (v04)
- `AdaptiveResult` C — `pinneapple_condense.learned` (v02)
- `AdaptiveRun` C — `pinneapple_systems.time_series.adaptive` (v06)
- `AdaptiveSampler` C — `pinneapple_core.data` (v01)
- `AdaptiveSweepConfig` C — `pinneapple_neural.trainer.adaptive_sweep` (v04)
- `AdaptiveWeights` C — `pinneapple_physics.pinn_solver.compiler.loss` (v02)
- `add` f — `pinneapple_flow.libcalls` (v07)
- `add_earth_pyvista` f — `pinneapple_tools.visualization.studio.lighting` (v07)
- `add_gaussian_noise` f — `pinneapple_data.synth.operators.noise` (v04)
- `add_link` f — `pinneapple_hub.library.flowlink` (v07)
- `add_scan` f — `pinneapple_twin3d.scan` (v06)
- `add_vorticity_divergence` f — `pinneapple_pdb.derived` (v04)
- `adf_test` f — `pinneapple_systems.time_series.audit.tests` (v06)
- `admin_summary` f — `pinneapple_app.backend.routers.admin` (v07)
- `AdminSummaryResponse` C — `pinneapple_app.backend.routers.admin` (v07)
- `admissible` f — `pinneapple_check.checks.common` (v05)
- `advection_diffusion` f — `pinneapple_condense.meshless2d` (v02)

- advection_diffusion_2d f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- advection_dominated f — pinneapple_physics.physics_audit.assumptions (v02)
- AdvectionDiffusion1D C — pinneapple_arena.problems (v04)
- AdvectionDiffusion1D C — pinneapple_physics.advection_diffusion_1d (v02)
- AdvectionDispersionReactionSolver C — pinneapple_systems.process_components.reaction_kinetics (v06)
- adversarial_sensitivity f — pinneapple_security.model_security (v05)
- AdversarialReviewReport C — pinneapple_analysis.verification.architecture_critique (v05)
- Advisor C — pinneapple_reason.engine (v05)
- AEBase C — pinneapple_neural.architectures.autoencoders.base (v04)
- AEOutput C — pinneapple_neural.architectures.autoencoders.base (v04)
- AeroCoefficients C — pinneapple_tools.aero_coefficients (v07)
- aerodynamicist f — pinneapple_flow.nodes.autolab (v07)
- AeroGNN C — pinneapple_design.aero.surrogate (v06)
- AEROMHybrid C — pinneapple_neural.architectures.autoencoders.ae_rom_hybrid (v04)
- AeroModel C — pinneapple_simulation.flight.aero (v03)
- AFNO C — pinneapple_neural.architectures.afno (v04)
- AFNOLayer C — pinneapple_neural.architectures.afno (v04)
- Agent C — pinneapple_llm.workbench.agents (v07)
- agent_tools f — pinneapple_hub.library.sources (v07)
- AgentLoopResult C — pinneapple_llm.agent_loop (v07)
- AgentResult C — pinneapple_autolab.agent (v07)
- AgentStep C — pinneapple_llm.agent_loop (v07)
- aggregate f — pinneapple_arena.pointwise.leaderboard (v04)
- ahmed_body f — pinneapple_design.geometry.bodies (v06)
- ahmed_body_polygon f — pinneapple_physics.pde_environment.presets._curve_geometry (v02)
- ahmed_slant f — pinneapple_flow.nodes_ext.design (v07)
- air f — pinneapple_physics.closed_form.pipe_insulation (v02)
- air f — pinneapple_reason.props (v05)
- air_paths f — pinneapple_design.thermal.board_cooling.scene (v06)
- air_properties f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- Aircraft C — pinneapple_design.aero.aircraft (v06)
- Aircraft C — pinneapple_simulation.flight.aircraft (v03)
- aircraft f — pinneapple_flow.nodes_ext.flight (v07)
- Aircraft3D C — pinneapple_design.aero.aircraft3d (v06)
- aircraft_surfaces f — pinneapple_simulation.flight.visuals (v03)
- aircraft_wing_aerodynamics f — pinneapple_physics.pde_environment.presets.engineering (v02)
- aircraft_wing_structural f — pinneapple_physics.pde_environment.presets.engineering (v02)
- airfoil f — pinneapple_twin3d.primitives (v06)

- airfoil_baseline f — pinneapple_design.design_optimizer.presets (v06)
- airfoil_bodies f — pinneapple_design.design_optimizer.presets (v06)
- airfoil_existing f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- airfoil_feasible f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- airfoil_naca_mask f — pinneapple_simulation.numerical_solvers.lbm (v03)
- airfoil_naca_mask_3d f — pinneapple_simulation.numerical_solvers.lbm (v03)
- airfoil_nodes f — pinneapple_design.aero.panel (v06)
- airfoil_outline f — pinneapple_design.design_optimizer.airfoil_drag (v06)
- airfoil_problem f — pinneapple_design.design_optimizer.presets (v06)
- airfoil_rans f — pinneapple_analysis.surrogate.examples (v05)
- airfoil_space f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- AirfoilEvaluator C — pinneapple_design.design_optimizer.presets (v06)
- AirfoilGNN C — pinneapple_hub.zoo._models (v07)
- AirfoilMLP C — pinneapple_hub.zoo._models (v07)
- AirfoilProblem C — pinneapple_analysis.benchmark.builtin.airfoil (v05)
- AirfoilSection C — pinneapple_presets.cases.airfoil (v02)
- Airframe C — pinneapple_design.aero.airframe (v06)
- airframe_from f — pinneapple_design.aero.aircraft3d (v06)
- Airliner C — pinneapple_design.aero.airliner (v06)
- airliner f — pinneapple_twin3d.gallery (v06)
- airliner_from f — pinneapple_design.aero.airliner3d (v06)
- airliner_vlm f — pinneapple_flow.nodes_ext.aero (v07)
- AirlinerModel C — pinneapple_design.aero.airliner3d (v06)
- airmass f — pinneapple_physics.astro.astrometry (v02)
- aleatoric_nll_loss f — pinneapple_analysis.uncertainty.aleatoric (v05)
- AleatoricHead C — pinneapple_analysis.uncertainty.aleatoric (v05)
- Alert C — pinneapple_systems.operations.twin (v06)
- AlertRules C — pinneapple_systems.operations.twin (v06)
- all_faces f — pinneapple_app.backend.core.physics_adapter (v07)
- all_model_specs f — pinneapple_tools.benchmark_suite.benchmark (v07)
- all_problems f — pinneapple_app.backend.core.problem_registry (v07)
- allen_cahn_reference f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- AllenCahn C — pinneapple_arena.problems (v04)
- AllenCahn C — pinneapple_arena.problems (v04)
- AllenCahn1DTask C — pinneapple_tools.benchmark_suite.tasks.allen_cahn_1d (v07)
- allmesh_script f — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- allowable_stress_for_life_pa f — pinneapple_design.surrogate_inverse.suspension_arm (v06)

- `allrun_script` f — `pinneapple_simulation.pipelines.freecad_openfoam.mesh` (v03)
- `along_track` f — `pinneapple_design.intent.physics.loop_track` (v06)
- `ALRetrainResult` C — `pinneapple_neural.physicsnemo_bridge.active_learning_bridge` (v04)
- `ALRound` C — `pinneapple_simulation.electromagnetics.antenna.design` (v03)
- `ALRoundResult` C — `pinneapple_neural.physicsnemo_bridge.active_learning_bridge` (v04)
- `AmbiguityResult` C — `pinneapple_analysis.inverse_problems.counterfactual_attribution` (v05)
- `amg` f — `pinneapple_simulation.accelerated_foam.solvers` (v03)
- `Ammeter` f — `pinneapple_science.instruments` (v02)
- `AmortizedCost` C — `pinneapple_analysis.cost.equal_budget` (v05)
- `AMPContext` C — `pinneapple_neural.trainer.parallel` (v04)
- `ampere_maxwell` f — `pinneapple_physics.maxwell_checks` (v02)
- `AmpereLaw` C — `pinneapple_science.em.statics` (v02)
- `analyse` f — `pinneapple_design.bricks.statics` (v06)
- `analyse` f — `pinneapple_physics.astro.stars_ml` (v02)
- `analyse_assembly` f — `pinneapple_design.bricks.statics` (v06)
- `AnalysisResult` C — `pinneapple_analysis.verification.pipeline` (v05)
- `AnalysisResult` C — `pinneapple_systems.process_components.explicit_equation_system` (v06)
- `analytic_for` f — `pinneapple_physics.solving` (v02)
- `analytical_reference` f — `pinneapple_llm.qa.examples.pipe_wear_twin` (v07)
- `AnalyticalNode` C — `pinneapple_systems.cosimulation.node` (v06)
- `analyze` f — `pinneapple_analysis.verification.pipeline` (v05)
- `analyze` f — `pinneapple_data.data_health` (v04)
- `analyze` f — `pinneapple_systems.process_components.explicit_equation_system` (v06)
- `analyze_completeness` f — `pinneapple_tools.dataset_quality.checks` (v07)
- `analyze_distribution` f — `pinneapple_tools.dataset_quality.checks` (v07)
- `analyze_ep_tuning` f — `pinneapple_physics.non_hermitian.ep_tuning` (v02)
- `analyze_identifiability` f — `pinneapple_analysis.inverse_problems.identifiability` (v05)
- `analyze_model` f — `pinneapple_analysis` (v05)
- `analyze_pair` f — `pinneapple_perception.change_detection` (v04)
- `analyze_wheel_surrogate` f — `pinneapple_simulation.terramechanics.mobility` (v03)
- `angle_features` f — `pinneapple_worldmodel.robot_perception` (v04)
- `angular_diameter_deg` f — `pinneapple_physics.astro.journey` (v02)
- `angular_momentum` f — `pinneapple_physics.astro.nbody` (v02)
- `angular_momentum_ned` f — `pinneapple_simulation.flight.rigid_body` (v03)
- `angular_separation` f — `pinneapple_physics.astro.frames` (v02)
- `animate` f — `pinneapple_simulation.rover.viz` (v03)

- animate f — pinneapple_simulation.vehicle.viz (v03)
- animate_ensemble f — pinneapple_physics.ensemble_viz (v02)
- animate_multiscale_ensemble f — pinneapple_physics.multiscale.viz (v02)
- animate_rolling_forecast f — pinneapple_systems.time_series.viz.animation (v06)
- animate_scalar_field f — pinneapple_tools.visualization.animation (v07)
- animate_streamlines f — pinneapple_tools.visualization.animation (v07)
- Animation C — pinneapple_tools.visualization.studio.animation (v07)
- Anisotropic C — pinneapple_simulation.electromagnetics.media (v03)
- annular_current_source f — pinneapple_simulation.numerical_solvers.eddy_current_fdm (v03)
- AnnularDomain2D C — pinneapple_design.geometry.gen.domains (v06)
- annulus f — pinneapple_design.geometry.bodies (v06)
- annulus f — pinneapple_design.geometry.csg (v06)
- annulus f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- anomalies f — pinneapple_systems.time_series.enso.data (v06)
- anomaly_study f — pinneapple_systems.operations.contracts (v06)
- AnomalyEvent C — pinneapple_systems.digital_twin.monitoring.anomaly (v06)
- AnomalyMonitor C — pinneapple_systems.digital_twin.monitoring.anomaly (v06)
- Answer C — pinneapple_query.core (v02)
- answer_with_context f — pinneapple_llm.org_knowledge (v07)
- antenna_checks f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- antenna_surfaces f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- any_result_3d f — pinneapple_flow.nodes_ext.viz (v07)
- aperture_gain f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- aperture_gain_dbi f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- api f — pinneapple_hub.library.snippets (v07)
- APIKeyStore C — pinneapple_core.service.apikeys (v01)
- apis f — pinneapple_hub.library.sources (v07)
- aposteriori_test f — pinneapple_physics.multiscale.closures (v02)
- AposterioriReport C — pinneapple_physics.multiscale.closures (v02)
- app_available f — pinneapple_systems.operations.series (v06)
- app_dir f — pinneapple_flow.studio (v07)
- Apparent C — pinneapple_physics.astro.astrometry (v02)
- apparent f — pinneapple_physics.astro.astrometry (v02)
- apparent_magnitude_star f — pinneapple_physics.astro.journey (v02)
- ApplicabilityDomain C — pinneapple_analysis.uncertainty.applicability (v05)
- ApplicabilityMap C — pinneapple_veriphysics.applicability (v05)
- applicable f — pinneapple_analysis.surrogate.methods (v05)
- apply_baseline f — pinneapple_check.baseline (v05)

- apply_constraints f — pinneapple_decision.constraints (v05)
- apply_derived f — pinneapple_pdb.derived (v04)
- apply_dirichlet_bc f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- apply_filter f — pinneapple_physics.multiscale.filters (v02)
- apply_neumann_bc f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- apply_non_invention_policy f — pinneapple_problemdesign.policy (v02)
- apply_prior f — pinneapple_decision.debias (v05)
- apply_pyvista f — pinneapple_tools.visualization.studio.lighting (v07)
- apply_suggestions f — pinneapple_analysis.benchmark.advisor (v05)
- apply_trial f — pinneapple_flow.learn (v07)
- apply_tunables f — pinneapple_arena.autoresearch (v04)
- ApprovalDecision C — pinneapple_problemdesign.autonomy (v02)
- ApprovalGate C — pinneapple_problemdesign.autonomy (v02)
- ApprovalPolicy C — pinneapple_llm.workbench.tools (v07)
- ApprovalRequest C — pinneapple_problemdesign.autonomy (v02)
- apriori_test f — pinneapple_physics.multiscale.closures (v02)
- AprioriReport C — pinneapple_physics.multiscale.closures (v02)
- arc f — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- arch_test f — pinneapple_systems.time_series.audit.tests (v06)
- architecture f — pinneapple_flow.learn (v07)
- architecture_diagram f — pinneapple_tools.nvidia.diagram (v07)
- architecture_gates f — pinneapple_analysis.benchmark.checks (v05)
- ArchitectureCandidate C — pinneapple_analysis.verification.architecture_recommendation (v05)
- ArchitectureCandidate C — pinneapple_arena.nas (v04)
- ArchitectureFamily C — pinneapple_arena.pointwise.schema (v04)
- ArchitectureInfo C — pinneapple_arena.pointwise.schema (v04)
- ArchitectureNotApplicable C — pinneapple_arena.pointwise.adapters (v04)
- ArchitectureRecommendation C — pinneapple_analysis.verification.architecture_recommendation (v05)
- ArchitectureSearchSpace C — pinneapple_arena.nas (v04)
- ArchitectureUnavailable C — pinneapple_arena.pointwise.training (v04)
- ArchiveLimits C — pinneapple_security.archives (v05)
- ArchMeta C — pinneapple_arena.external.schema (v04)
- arco_states f — pinneapple_physics.weather.events (v02)
- ARDGP C — pinneapple_presets.surrogate (v02)
- are_compatible f — pinneapple_llm.qa.units (v07)
- Area C — pinneapple_science.atlas (v02)
- area f — pinneapple_design.geometry_library.meshops (v06)
- area_patch f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- area_ratio f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- Arena C — pinneapple_arena.arena (v04)

- `Arena` C — `pinneapple_tools.benchmark_suite.api` (v07)
- `arena` f — `pinneapple_flow.nodes.methods` (v07)
- `arena` f — `pinneapple_flow.nodes_ext.validation` (v07)
- `arena_choice` f — `pinneapple_arena.decision_mode` (v04)
- `arena_markdown` f — `pinneapple_arena.regimes` (v04)
- `arena_model_rules` f — `pinneapple_arena.decision_mode` (v04)
- `arena_problem_facts` f — `pinneapple_arena.decision_mode` (v04)
- `arena_run` f — `pinneapple_flow.nodes_ext.arena_pipelines` (v07)
- `arena_study_result` f — `pinneapple_analysis.surrogate.contracts` (v05)
- `arena_yaml` f — `pinneapple_flow.nodes_ext.benchmarks` (v07)
- `ArenaBudget` C — `pinneapple_analysis.validation.arena.loop` (v05)
- `ArenaCompareResult` C — `pinneapple_tools.benchmark_suite.api` (v07)
- `ArenaConfig` C — `pinneapple_arena.config` (v04)
- `ArenaDecisionReport` C — `pinneapple_arena.decision_mode` (v04)
- `ArenaExecutor` C — `pinneapple_arena.decision_mode` (v04)
- `ArenaProblem` C — `pinneapple_arena.problems` (v04)
- `ArenaReport` C — `pinneapple_analysis.validation.arena.loop` (v05)
- `ArenaResult` C — `pinneapple_arena.regimes` (v04)
- `ArenaResult` C — `pinneapple_tools.benchmark_suite.api` (v07)
- `ArenaTask` C — `pinneapple_tools.benchmark_suite.tasks.base` (v07)
- `ARFit` C — `pinneapple_systems.time_series.predictability` (v06)
- `ARForecaster` C — `pinneapple_systems.time_series.forecast_lab.baselines` (v06)
- `ARIMA` C — `pinneapple_neural.architectures.classical_ts.arima` (v04)
- `ARIMAForecaster` C — `pinneapple_systems.time_series.forecast_lab.baselines` (v06)
- `arm_camera_sample` f — `pinneapple_worldmodel.robot_perception` (v04)
- `arm_motions` f — `pinneapple_worldmodel.robot_perception` (v04)
- `arm_pick_and_place` f — `pinneapple_simulation.robot_sim.checks` (v03)
- `ArmDesign` C — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `ArrayProblem` C — `pinneapple_analysis.benchmark.builtin.fields` (v05)
- `ARResidualDetector` C — `pinneapple_security.cps_bench` (v05)
- `arrhenius_rate_constant` f — `pinneapple_systems.process_components.reaction_kinetics` (v06)
- `ArrheniusModel` C — `pinneapple_analysis.inverse_problems.models` (v05)
- `ArticulationSpec` C — `pinneapple_simulation.robot_sim.spec` (v03)
- `Artifact` C — `pinneapple_data.lineage.graph` (v04)
- `artifact` f — `pinneapple_flow.runtime` (v07)
- `ArtifactNotPublishedError` C — `pinneapple_hub.zoo._artifacts` (v07)
- `ArtifactRecord` C — `pinneapple_llm.qa.report` (v07)
- `ArtifactRegistry` C — `pinneapple_registry.registry` (v01)
- `ArtifactSpec` C — `pinneapple_tools.automation.workflow` (v07)
- `ArtifactUnavailableError` C — `pinneapple_hub.zoo._artifacts` (v07)

- `arxiv_pdf_url` f — `pinneapple_tools.hpo_experiments.sources.arxiv_pdf` (v07)
- `ArxivPaper` C — `pinneapple_tools.hpo_experiments.sources.arxiv_source` (v07)
- `ArxivSource` C — `pinneapple_tools.hpo_experiments.sources.arxiv_source` (v07)
- `as_array_dict` f — `pinneapple_systems.digital_twin.product.model` (v06)
- `as_dict` f — `pinneapple_analysis.validation.report_artifact` (v05)
- `as_dicts` f — `pinneapple_llm.workbench.evaluate` (v07)
- `as_field_model` f — `pinneapple_analysis.validation.validate` (v05)
- `as_flow` f — `pinneapple_flow.schema` (v07)
- `as_graph_outputs` f — `pinneapple_hub.zoo.export` (v07)
- `as_mesh` f — `pinneapple_design.qualitative.descriptors` (v06)
- `as_numpy_closure` f — `pinneapple_physics.multiscale.learned` (v02)
- `as_parameter` f — `pinneapple_analysis.uncertainty.distributions` (v05)
- `as_problem` f — `pinneapple_physics.solving` (v02)
- `as_space` f — `pinneapple_analysis.uncertainty.distributions` (v05)
- `as_tensor` f — `pinneapple_neural.utils` (v04)
- `assemble` f — `pinneapple_condense.fv` (v02)
- `assemble` f — `pinneapple_condense.meshless` (v02)
- `assemble_blocks` f — `pinneapple_condense.learned` (v02)
- `assemble_oseen` f — `pinneapple_condense.flow` (v02)
- `assemble_package` f — `pinneapple_condense.fv` (v02)
- `assemble_scalar` f — `pinneapple_condense.flow` (v02)
- `assembled_mesh` f — `pinneapple_simulation.rover.geometry` (v03)
- `Assembly` C — `pinneapple_design.bricks.assembly` (v06)
- `Assembly` C — `pinneapple_design.qualitative.descriptors` (v06)
- `assembly_meshes` f — `pinneapple_design.bricks.mesh` (v06)
- `assert_no_code` f — `pinneapple_veriphysics.validation_lab` (v05)
- `assert_valid_physical_sample` f — `pinneapple_data.validators` (v04)
- `assess` f — `pinneapple_data.simulation_metadata.calculix` (v04)
- `assess` f — `pinneapple_design.intent.pipeline` (v06)
- `assess` f — `pinneapple_security.compliance` (v05)
- `assess_candidates` f — `pinneapple_autolab.trust` (v07)
- `assess_drift` f — `pinneapple_analysis.uncertainty.drift` (v05)
- `assess_experiment` f — `pinneapple_lab.curation` (v07)
- `assess_mesh_quality` f — `pinneapple_analysis.verification.mesh_intelligence` (v05)
- `assess_run` f — `pinneapple_lab.curation` (v07)
- `assess_steady` f — `pinneapple_data.simulation_metadata.convergence` (v04)
- `assess_table` f — `pinneapple_data.simulation_metadata.convergence` (v04)
- `assess_transient` f — `pinneapple_data.simulation_metadata.convergence` (v04)
- `Assessment` C — `pinneapple_design.intent.pipeline` (v06)
- `Assessment` C — `pinneapple_lab.curation` (v07)
- `assign_patches` f — `pinneapple_simulation.pipelines.freecad_openfoam.cad` (v03)

- `Assumption` C — `pinneapple_physics.physics_audit.assumptions` (v02)
- `Assumption` C — `pinneapple_problemdesign.schema` (v02)
- `AssumptionAudit` C — `pinneapple_physics.physics_audit.assumptions` (v02)
- `AssumptionCheck` C — `pinneapple_physics.physics_audit.assumptions` (v02)
- `astro_constant_si` f — `pinneapple_physics.pde_environment.presets.astropy_bridge` (v02)
- `astroml_available` f — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `Atlas` C — `pinneapple_science.atlas` (v02)
- `atlas` f — `pinneapple_flow.nodes.science` (v07)
- `atmosphere` f — `pinneapple_physics.closed_form.pump_npsh` (v02)
- `atmospheric_pressure` f — `pinneapple_reason.props` (v05)
- `attach_geometry` f — `pinneapple_data.adapters.geom_adapter` (v04)
- `attach_runs` f — `pinneapple_hub.library` (v07)
- `attach_snippets` f — `pinneapple_data.preflight` (v04)
- `attach_upd_state` f — `pinneapple_data.adapters.upd_adapter` (v04)
- `attached_flow` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `attest_and_sign_experiment` f — `pinneapple_security.integrate` (v05)
- `attest_experiment` f — `pinneapple_security.attestation` (v05)
- `attest_files` f — `pinneapple_security.attestation` (v05)
- `attribute_failure` f — `pinneapple_physics.physics_audit.attribution` (v02)
- `Attribution` C — `pinneapple_physics.physics_audit.attribution` (v02)
- `auc` f — `pinneapple_analysis.validation.fault_injection.matrix` (v05)
- `auc` f — `pinneapple_systems.sports.football.metrics` (v06)
- `audit` f — `pinneapple_flow.nodes_ext.validation` (v07)
- `audit` f — `pinneapple_hub.zoo.datasets` (v07)
- `audit_assumptions` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `audit_card` f — `pinneapple_hub.zoo.datasets` (v07)
- `audit_claims` f — `pinneapple_autolab.agent` (v07)
- `audit_dataset` f — `pinneapple_data.audit` (v04)
- `audit_model` f — `pinneapple_analysis.validation.arena.loop` (v05)
- `audit_zoo_dataset` f — `pinneapple_data.audit` (v04)
- `AuditCard` C — `pinneapple_data.audit` (v04)
- `AuditCheck` C — `pinneapple_data.audit` (v04)
- `AuditLog` C — `pinneapple_security.audit` (v05)
- `AuditReport` C — `pinneapple_systems.time_series.audit.report` (v06)
- `AuditSection` C — `pinneapple_systems.time_series.audit.report` (v06)
- `augment_interpolation` f — `pinneapple_data.augmentation` (v04)
- `augment_noise` f — `pinneapple_data.augmentation` (v04)
- `augment_transform` f — `pinneapple_data.augmentation` (v04)
- `AugmentedLagrangian` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `aurora_probability` f — `pinneapple_physics.astro.spaceweather` (v02)

- `AuthAudit` C — `pinneapple_security.web` (v05)
- `author_articulation` f — `pinneapple_tools.nvidia.isaac` (v07)
- `auto_call` f — `pinneapple_hub.library.snippets` (v07)
- `auto_procs` f — `pinneapple_simulation.pipelines.freecad_openfoam.steps` (v03)
- `auto_residual` f — `pinneapple_physics.symbolic_pde.compiler` (v02)
- `auto_specs` f — `pinneapple_flow.libcalls` (v07)
- `AutoARIMAForecaster` C — `pinneapple_systems.time_series.classical_wrappers` (v06)
- `AutoBalancer` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `AutoBalancerConfig` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `AutoBatchSizeFinder` C — `pinneapple_neural.trainer.hpc` (v04)
- `AutoCorrelation` C — `pinneapple_neural.architectures.transformers.autoformer` (v04)
- `AutoCorrelationLayer` C — `pinneapple_neural.architectures.transformers.autoformer` (v04)
- `Autoencoder2D` C — `pinneapple_neural.architectures.autoencoders.ae_2d` (v04)
- `AutoEncoderAdapter` C — `pinneapple_neural.architectures.adapters.ae` (v04)
- `AutoencoderCatalog` C — `pinneapple_neural.architectures.autoencoders.registry` (v04)
- `Autoformer` C — `pinneapple_neural.architectures.transformers.autoformer` (v04)
- `AutonomousLab` C — `pinneapple_autolab.lab` (v07)
- `AutonomousResearchAgent` C — `pinneapple_problemdesign.research_loop` (v02)
- `AutonomyLevel` C — `pinneapple_problemdesign.autonomy` (v02)
- `AutoPDEWeightCalibrator` C — `pinneapple_neural.trainer.loss_balancer` (v04)
- `Autopilot` C — `pinneapple_simulation.flight.control` (v03)
- `AutoResearch` C — `pinneapple_arena.autoresearch` (v04)
- `ava_simulator` f — `pinneapple_simulation.electromagnetics.antenna.runner` (v03)
- `ava_transform` f — `pinneapple_simulation.electromagnetics.antenna.design` (v03)
- `available` f — `pinneapple_blender.cycles` (v07)
- `available` f — `pinneapple_lab.spec` (v07)
- `available` f — `pinneapple_llm.qa.validators.base` (v07)
- `available` f — `pinneapple_simulation.electromagnetics.machines.getdp` (v03)
- `available` f — `pinneapple_simulation.rover.autolab.render` (v03)
- `available_bodies` f — `pinneapple_physics.astro.smallbodies` (v02)
- `available_methods` f — `pinneapple_analysis.inverse_problems.studio` (v05)
- `available_methods` f — `pinneapple_design.design_optimizer.methods` (v06)
- `available_orchestrator_tools` f — `pinneapple_problemdesign.knowledge.mapping` (v02)
- `AVAParams` C — `pinneapple_simulation.electromagnetics.antenna.ava` (v03)
- `axial_compressor_cascade_2d` f — `pinneapple_physics.pde_environment.presets.turbomachinery` (v02)

- axial_compressor_meanline f — pinneapple_physics.pde_environment.presets.turbomachinery (v02)
- axial_compressor_stage_3d f — pinneapple_physics.pde_environment.presets.turbomachinery (v02)
- axial_fan f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- axial_flux_density f — pinneapple_simulation.numerical_solvers.eddy_current_fdm (v03)
- axis_angle f — pinneapple_simulation.multibody.tree (v03)
- axis_binned_profile f — pinneapple_tools.model_evaluation.metrics (v07)
- axis_time f — pinneapple_physics.astro.observing (v02)
- axisymmetric_linear_elasticity_2d_default f — pinneapple_physics.pde_environment.presets.solid_mechanics (v02)

B

- b64url f — pinneapple_hub.library.flowlink (v07)
- back_emf f — pinneapple_simulation.electromagnetics.postprocess (v03)
- Backend C — pinneapple_llm.workbench.backend (v07)
- Backend C — pinneapple_tools.benchmark_suite.backends.base (v07)
- Backend C — pinneapple_tools.compute_backends.backend (v07)
- backend_info f — pinneapple_quantum (v04)
- BackendConfig C — pinneapple_quantum.backends.backend (v04)
- backtest f — pinneapple_systems.digital_twin.product.commission (v06)
- backtest f — pinneapple_twin3d.wear.forecast (v06)
- BacktestConfig C — pinneapple_systems.time_series.validation.backtest (v06)
- BacktestResult C — pinneapple_systems.time_series.validation.backtest (v06)
- BacktestRunner C — pinneapple_systems.time_series.validation.backtest (v06)
- badge f — pinneapple_analysis.validation.report_artifact (v05)
- badge f — pinneapple_science.plots (v02)
- balance f — pinneapple_simulation.flight.mass (v03)
- balance_check f — pinneapple_simulation.rover.checks (v03)
- balance_residual f — pinneapple_security.anomaly (v05)
- Balancer C — pinneapple_core.loss (v01)
- BalanceRule C — pinneapple_systems.operations.anomaly (v06)
- Ball C — pinneapple_systems.sports.football.ball (v06)
- ballistic_from_bstar f — pinneapple_physics.astro.decay (v02)
- balloon f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- band_difference f — pinneapple_perception.change_detection (v04)
- band_metrics f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- band_ratios f — pinneapple_simulation.flight.verification (v03)
- BandChangeResult C — pinneapple_perception.change_detection (v04)
- banded f — pinneapple_design.surrogate_inverse.core (v06)

- bands f — pinneapple_hub.zoo.demo (v07)
- BarWear C — pinneapple_lab.experiments.wear (v07)
- BarWear C — pinneapple_physics.tribology.archard (v02)
- BarWearRanking C — pinneapple_lab.experiments.wear (v07)
- base_fraction f — pinneapple_systems.process_components.reaction_kinetics (v06)
- base_meta f — pinneapple_tools.benchmark_suite.crossframework.protocol (v07)
- base_name f — pinneapple_design.geometry_library.entries._adapt (v06)
- base_pattern f — pinneapple_twin3d.wear.synthetic (v06)
- base_theta f — pinneapple_design.thermal.heatsink.engine (v06)
- baseline f — pinneapple_design.aero.optimize (v06)
- baseline_errors f — pinneapple_systems.time_series.predictability (v06)
- baseline_specs f — pinneapple_systems.time_series.forecast_lab.presets (v06)
- baseline_x f — pinneapple_design.aero.aircraft3d (v06)
- baseline_x_al f — pinneapple_design.aero.airliner3d (v06)
- baselines f — pinneapple_analysis.validation.physics_learning (v05)
- BaseModel C — pinneapple_neural.architectures.base (v04)
- BaseStream C — pinneapple_systems.digital_twin.io.stream (v06)
- basquin_allowable_stress_pa f — pinneapple_physics.closed_form.cantilever_fatigue (v02)
- basquin_fatigue_life_cycles f — pinneapple_physics.closed_form.cantilever_fatigue (v02)
- Batch C — pinneapple_core.data (v01)
- Batch C — pinneapple_data.upd_types (v04)
- batch_means_ci f — pinneapple_analysis.verification.forecast_skill (v05)
- batched_inference f — pinneapple_neural.predictor.infer (v04)
- BatchPrior C — pinneapple_decision.debias (v05)
- BatteryCellVanillaPINN C — pinneapple_systems.component_library.components (v06)
- bayesian_block_edges f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- BayesianDesignOptimizer C — pinneapple_design.design_optimizer.optimizer (v06)
- BayesianLinear C — pinneapple_neural.architectures.pinns.bpinn (v04)
- BayesianPINN C — pinneapple_neural.architectures.pinns.bpinn (v04)
- BayesianRNN C — pinneapple_neural.architectures.continuous.bayesian_rnn (v04)
- bbox_size f — pinneapple_design.surrogate_inverse.meshes (v06)
- beam f — pinneapple_design.intent.physics.bracket (v06)
- beam_column_moment_amplification_factor f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- beam_deflection f — pinneapple_flow.wrappers (v07)
- beam_matrices f — pinneapple_simulation.numerical_solvers.beam_fe (v03)
- beam_midspan_deflection f — pinneapple_design.bricks.closed_form (v06)

- beam_modes f — pinneapple_simulation.numerical_solvers.beam_fe (v03)
- beam_moment f — pinneapple_design.bricks.closed_form (v06)
- beam_reactions f — pinneapple_design.bricks.closed_form (v06)
- beam_static f — pinneapple_simulation.numerical_solvers.beam_fe (v03)
- BeamBVPFDMSolver C — pinneapple_simulation.numerical_solvers.beam_bvp_fdm (v03)
- BeamResult C — pinneapple_systems.process_components.beam_statics (v06)
- bearing_capacity_factors f — pinneapple_simulation.terramechanics.wong_reece (v03)
- beauty f — pinneapple_simulation.vehicle.viz (v03)
- beauty_view f — pinneapple_simulation.vehicle.viz (v03)
- before_after_series f — pinneapple_data.unified (v04)
- bekker_rigid_wheel_sinkage f — pinneapple_simulation.rover.checks (v03)
- bekker_wong_forces_torch f — pinneapple_simulation.numerical_solvers.bekker_wong (v03)
- bekker_wong_surrogate_2d f — pinneapple_physics.pde_environment.presets.terramechanics (v02)
- BekkerWongContact C — pinneapple_simulation.rover.contact (v03)
- BekkerWongSolver C — pinneapple_simulation.numerical_solvers.bekker_wong (v03)
- belt_winding f — pinneapple_simulation.electromagnetics.machine (v03)
- Benchmark C — pinneapple_physics.compile_api (v02)
- benchmark f — pinneapple_hub.zoo (v07)
- benchmark f — pinneapple_mcp.tools (v07)
- benchmark_catalog f — pinneapple_pdb.benchmarks (v04)
- benchmark_closures f — pinneapple_physics.multiscale.turbulence (v02)
- benchmark_component_type f — pinneapple_systems.component_library.benchmark (v06)
- benchmark_component_type_task f — pinneapple_orchestration.tasks.benchmark_tasks (v01)
- benchmark_dataset f — pinneapple_arena.dataset_bench (v04)
- benchmark_faults f — pinneapple_systems.operations.anomaly (v06)
- BenchmarkCase C — pinneapple_lab.experiments.landing (v07)
- BenchmarkComparison C — pinneapple_data.physics_case (v04)
- BenchmarkConfig C — pinneapple_analysis.benchmark.config (v05)
- BenchmarkConfig C — pinneapple_tools.benchmark_suite.benchmark (v07)
- BenchmarkEntry C — pinneapple_pdb.benchmarks (v04)
- BenchmarkProblem C — pinneapple_analysis.validation.fault_injection.problems (v05)
- BenchmarkReport C — pinneapple_tools.benchmark_suite.report (v07)
- BenchmarkResult C — pinneapple_analysis.benchmark.api (v05)
- BenchmarkResult C — pinneapple_tools.benchmark_suite.benchmark (v07)
- BenchmarkResult C — pinneapple_worldmodel.benchmark (v04)
- BenchmarkTask C — pinneapple_worldmodel.benchmark (v04)

- BenchmarkTaskBase C — pinneapple_tools.benchmark_suite.benchmark (v07)
- bending_stress_at_position_pa f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- bending_stress_from_curvature f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- bending_stress_pa f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- benjamini_hochberg f — pinneapple_systems.time_series.metrics_ext.comparison (v06)
- best_device f — pinneapple_arena.pointwise.training (v04)
- best_device f — pinneapple_neural.trainer.parallel (v04)
- best_method f — pinneapple_core.onboarding.catalog (v01)
- best_period f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- beveled_box f — pinneapple_design.geometry.vehicle_cad (v06)
- BHCurve C — pinneapple_simulation.electromagnetics.materials (v03)
- BiasModel C — pinneapple_systems.digital_twin.product.model (v06)
- BiGRUModel C — pinneapple_neural.architectures.recurrent.gru (v04)
- Biharmonic2D C — pinneapple_arena.problems (v04)
- Bilinear C — pinneapple_analysis.benchmark.builtin.standard (v05)
- bilinear_mac f — pinneapple_condense.meshless2d (v02)
- BiLSTMModel C — pinneapple_neural.architectures.recurrent.lstm (v04)
- BinaryCheck C — pinneapple_simulation.external_solvers.modelica.fmu_inspect (v03)
- biot f — pinneapple_physics.physics_audit.regime (v02)
- biot_savart f — pinneapple_simulation.electromagnetics.statics (v03)
- BiotSavart C — pinneapple_science.em.statics (v02)
- black_scholes_1d f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- blackbody_rgb f — pinneapple_physics.astro.stars (v02)
- blackbody_rgb f — pinneapple_physics.blackhole.raytrace (v02)
- blackbody_spectrum f — pinneapple_physics.astro.stars (v02)
- BlackBoxNode C — pinneapple_systems.cosimulation.node (v06)
- BlackHoleForecast C — pinneapple_lab.experiments.blackhole_forecast (v07)
- BlackScholes1D C — pinneapple_arena.problems (v04)
- blasius f — pinneapple_physics.closed_form.pipe_flow (v02)
- blend_sections f — pinneapple_design.geometry_library.kit (v06)
- blender_available f — pinneapple_design.bricks.visuals (v06)
- blender_axes f — pinneapple_tools.visualization.studio.blender (v07)
- blender_clip f — pinneapple_flow.nodes_ext.render (v07)
- blender_world f — pinneapple_tools.visualization.studio.lighting (v07)
- Block C — pinneapple_data.cae.inp (v04)
- Block C — pinneapple_physics.weather.model (v02)

- Block C — pinneapple_tools.nvidia.diagram (v07)
- block_3d f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- block_average f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- block_bootstrap f — pinneapple_analysis.uncertainty.resampling (v05)
- block_bootstrap_indices f — pinneapple_analysis.verification.forecast_skill (v05)
- block_bootstrap_mean f — pinneapple_analysis.verification.forecast_skill (v05)
- block_bootstrap_ratio f — pinneapple_analysis.verification.forecast_skill (v05)
- block_functionals f — pinneapple_condense.substructure (v02)
- block_indicator f — pinneapple_condense.exceptions (v02)
- block_model f — pinneapple_condense.substructure (v02)
- block_observables f — pinneapple_condense.learned (v02)
- block_precomputed f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- blocked f — pinneapple_llm.qa.validators.base (v07)
- blocked_hindcast f — pinneapple_systems.time_series.enso.evaluation (v06)
- blocking_reasons f — pinneapple_llm.qa.levels (v07)
- BlockModel C — pinneapple_condense.substructure (v02)
- blocks f — pinneapple_hub.library.compose (v07)
- blocks_net f — pinneapple_flow.learn (v07)
- bloom f — pinneapple_physics.blackhole.raytrace (v02)
- Board C — pinneapple_physics.pcb.solver (v02)
- board_cooling f — pinneapple_flow.nodes.autolab (v07)
- board_scene f — pinneapple_design.thermal.board_cooling.scene (v06)
- bodies_from_items f — pinneapple_design.geometry_library.entries._adapt (v06)
- Body C — pinneapple_design.bricks.statics (v06)
- Body C — pinneapple_design.geometry_library.core (v06)
- Body C — pinneapple_simulation.external_solvers.newton_physics.scene (v03)
- Body C — pinneapple_simulation.multibody.tree (v03)
- body_of_sections f — pinneapple_design.geometry_library.kit (v06)
- body_position f — pinneapple_physics.astro.journey (v02)
- body_rotation f — pinneapple_simulation.rover.geometry (v03)
- body_vertices f — pinneapple_simulation.flight.visuals (v03)
- BodyLimit C — pinneapple_security.web (v05)
- BodyState C — pinneapple_analysis.uncertainty.reachability (v05)
- bof_converter f — pinneapple_twin3d.wear.presets (v06)
- bolometric_correction f — pinneapple_physics.astro.stars (v02)
- bondi_pw f — pinneapple_physics.blackhole.hydro (v02)
- BondiAccretion C — pinneapple_lab.experiments.accretion (v07)
- bootstrap_confidence_interval f — pinneapple_tools.model_evaluation.metrics (v07)
- bootstrap_peak_distribution f — pinneapple_physics.pde_environment.presets.astroml_bridge

(v02)
- border_points f — pinneapple_tools.visualization.studio.planetary (v07)
- border_visibility f — pinneapple_tools.visualization.studio.planetary (v07)
- bounce f — pinneapple_simulation.external_solvers.newton_physics.validation (v03)
- bouncing_ball_scene f — pinneapple_worldmodel.physical_world (v04)
- Bound C — pinneapple_condense.query (v02)
- Bound C — pinneapple_lab.results_rules (v07)
- boundaries f — pinneapple_physics.blackhole.hydro_numba (v02)
- Boundary C — pinneapple_simulation.geophysics.richards (v03)
- boundary f — pinneapple_core.loss (v01)
- boundary f — pinneapple_data.simulation_metadata.foam_dict (v04)
- boundary_check f — pinneapple_physics.physics_audit.monitor (v02)
- boundary_conditions f — pinneapple_data.cae.foam (v04)
- boundary_edges f — pinneapple_simulation.numerical_solvers.plane_stress_fem (v03)
- boundary_flux_balance f — pinneapple_physics.darcy_checks (v02)
- boundary_nodes f — pinneapple_simulation.electromagnetics.mesh (v03)
- boundary_rect f — pinneapple_simulation.numerical_solvers._bc (v03)
- boundary_values f — pinneapple_physics.darcy_checks (v02)
- boundary_vertices f — pinneapple_design.geometry.ops.curvature (v06)
- BoundaryCheck C — pinneapple_analysis.validation.boundary (v05)
- BoundaryConditionSpec C — pinneapple_app.backend.core.problem (v07)
- BoundaryLoop C — pinneapple_analysis.verification.geometry_intelligence (v05)
- BoundaryRegion C — pinneapple_design.geometry.gen.domains (v06)
- BoundaryRegion C — pinneapple_worldmodel.geometry (v04)
- BoundarySampler C — pinneapple_core.data (v01)
- bounds_check f — pinneapple_physics.physics_audit.monitor (v02)
- bounds_of f — pinneapple_flow.nodes._common (v07)
- boussinesq f — pinneapple_physics.physics_audit.assumptions (v02)
- box f — pinneapple_design.geometry.bodies (v06)
- box f — pinneapple_design.geometry.vehicle_cad (v06)
- box f — pinneapple_design.geometry_library.kit (v06)
- box f — pinneapple_presets.viz3d (v02)
- box f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- box f — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- box f — pinneapple_twin3d.primitives (v06)
- Box3D C — pinneapple_worldmodel.geometry (v04)
- box3d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- box_beam f — pinneapple_design.geometry.vehicle_cad (v06)
- box_face_normals f — pinneapple_physics.pde_environment.condition_sampling (v02)

- box_index f — pinneapple_systems.time_series.enso.forecasters (v06)
- box_inertia f — pinneapple_simulation.robot_sim.spec (v03)
- box_mean f — pinneapple_systems.time_series.enso.data (v06)
- box_mesh f — pinneapple_design.bricks.mesh (v06)
- box_mesh f — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- box_sampler f — pinneapple_analysis.validation.metamorphic (v05)
- BoxBoundary C — pinneapple_simulation.numerical_solvers.sph_boundaries (v03)
- BoxConstraint C — pinneapple_design.design_optimizer.constraints (v06)
- BPINNConfig C — pinneapple_neural.architectures.pinns.bpinn (v04)
- bracket_figures f — pinneapple_design.intent.figures (v06)
- BracketTemplate C — pinneapple_design.intent.templates.bracket (v06)
- BrauerCurve C — pinneapple_simulation.electromagnetics.materials (v03)
- BrazilDataCubeClient C — pinneapple_data.datasets.external_sources (v04)
- brep_available f — pinneapple_tools.nvidia.brep (v07)
- brep_python f — pinneapple_tools.nvidia.brep (v07)
- BRepReport C — pinneapple_tools.nvidia.brep (v07)
- brewster_angle f — pinneapple_simulation.electromagnetics.waves (v03)
- Brick C — pinneapple_design.bricks.assembly (v06)
- brick_mesh f — pinneapple_design.bricks.mesh (v06)
- BricksExperiment C — pinneapple_lab.experiments.bricks (v07)
- BridgeGoal C — pinneapple_design.bricks.goals (v06)
- brier f — pinneapple_systems.sports.football.metrics (v06)
- bright_stars_in_field f — pinneapple_physics.astro.lensing (v02)
- BrooksCorey C — pinneapple_simulation.geophysics.richards (v03)
- Buckle C — pinneapple_simulation.external_solvers.calculix.study (v03)
- Budget C — pinneapple_analysis.cost.profiler (v05)
- Budget C — pinneapple_autolab.mission (v07)
- BudgetExceeded C — pinneapple_analysis.cost.profiler (v05)
- BudgetExhaustedError C — pinneapple_autolab.ledger (v07)
- BudgetRun C — pinneapple_analysis.cost.equal_budget (v05)
- build f — pinneapple_data.cae.report (v04)
- build f — pinneapple_design.geometry_library.core (v06)
- build f — pinneapple_flow.learn (v07)
- build f — pinneapple_hub.library (v07)
- build f — pinneapple_hub.zoo.validation_reports (v07)
- build f — pinneapple_lab.experiments.structures (v07)
- build f — pinneapple_simulation.flight.catalog (v03)
- build_1d_linear_interpolant f — pinneapple_tools.dataset_quality.checks (v07)
- build_3d f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- build_3d_live_twin f — pinneapple_llm.twin_draft (v07)
- build_and_hold_profile f — pinneapple_systems.process_components.curved_path_geometry

(v06)
- `build_app` f — `pinneapple_tools.visualization.integrations.trame_app` (v07)
- `build_applicability_map` f — `pinneapple_veriphysics.applicability` (v05)
- `build_arrays` f — `pinneapple_physics.blackhole.dataset` (v02)
- `build_axisymmetric_helmholtz_system` f — `pinneapple_simulation.numerical_solvers.eddy_current_fdm` (v03)
- `build_benchmark_payload` f — `pinneapple_app.backend.core.benchmark` (v07)
- `build_circuit` f — `pinneapple_quantum` (v04)
- `build_classical_circuit` f — `pinneapple_quantum.circuits.base` (v04)
- `build_consistency_prompt` f — `pinneapple_tools.dataset_quality.vlm_curation` (v07)
- `build_corpus` f — `pinneapple_analysis.retrieval.corpus` (v05)
- `build_csp` f — `pinneapple_security.web` (v05)
- `build_dashboard` f — `pinneapple_autolab.dashboard` (v07)
- `build_dashboard` f — `pinneapple_autolab.domains.racecar` (v07)
- `build_dataset` f — `pinneapple_data.cae.dataset` (v04)
- `build_dataset` f — `pinneapple_systems.time_series.enso.dataset` (v06)
- `build_datasheet` f — `pinneapple_simulation.electromagnetics.reflector.study_report_text` (v03)
- `build_default_index` f — `pinneapple_analysis.retrieval` (v05)
- `build_default_mapping_atmosphere` f — `pinneapple_physics.pinn_solver.io.mappings` (v02)
- `build_definitions` f — `pinneapple_systems.process_components.explicit_equation_system` (v06)
- `build_digital_twin` f — `pinneapple_systems.digital_twin.twin` (v06)
- `build_digital_twin_for_model` f — `pinneapple_neural.physicsnemo_bridge.adapters` (v04)
- `build_document` f — `pinneapple_simulation.pipelines.freecad_openfoam.models` (v03)
- `build_domain` f — `pinneapple_design` (v06)
- `build_evidence_graph` f — `pinneapple_analysis.verification.evidence_graph` (v05)
- `build_figures` f — `pinneapple_simulation.electromagnetics.reflector.study_report_figures` (v03)
- `build_fmu` f — `pinneapple_simulation.external_solvers.modelica.omc` (v03)
- `build_frame` f — `pinneapple_design.bricks.visuals` (v06)
- `build_from_bundle` f — `pinneapple_data.adapters.pinn_batch_builders` (v04)
- `build_from_real_data` f — `pinneapple_data.adapters.pinn_batch_builders` (v04)
- `build_from_solver` f — `pinneapple_data.adapters.pinn_batch_builders` (v04)
- `build_gauss_legendre_tableau` f — `pinneapple_simulation.numerical_solvers.irk_gauss_legendre` (v03)
- `build_geometry_asset` f — `pinneapple_design.geometry.core.registry` (v06)

- `build_heated_pipe` f — `pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job` (v03)
- `build_journal` f — `pinneapple_simulation.external_solvers.ansys.journal_builder` (v03)
- `build_kb` f — `pinneapple_tools.hpo_experiments.pipelines.build_kb` (v07)
- `build_knowledge_graph` f — `pinneapple_analysis.verification.knowledge_graph` (v05)
- `build_lattice` f — `pinneapple_design.aero.vlm` (v06)
- `build_literature_corpus` f — `pinneapple_analysis.retrieval.corpus` (v05)
- `build_loss` f — `pinneapple_neural.trainer.losses` (v04)
- `build_manifest` f — `pinneapple_security.integrity` (v05)
- `build_mesh` f — `pinneapple_design.geometry.gen.primitives` (v06)
- `build_mesh_from_cadquery_object` f — `pinneapple_design.geometry.gen.cadquery_gen` (v06)
- `build_mesh_from_parametric_spec` f — `pinneapple_design.geometry.gen.cadquery_gen` (v06)
- `build_mesh_from_step` f — `pinneapple_design.geometry.gen.cadquery_gen` (v06)
- `build_metrics_from_cfg` f — `pinneapple_neural.trainer.metrics_cfg` (v04)
- `build_model` f — `pinneapple_arena.model_factory` (v04)
- `build_model` f — `pinneapple_neural` (v04)
- `build_neighbor_list` f — `pinneapple_simulation.numerical_solvers.neighbors` (v03)
- `build_package` f — `pinneapple_veriphysics.evidence_package` (v05)
- `build_pennylane_circuit` f — `pinneapple_quantum.circuits.base` (v04)
- `build_physical_sample_dataloader` f — `pinneapple_data.dataloaders` (v04)
- `build_physics` f — `pinneapple_app.backend.core.physics_adapter` (v07)
- `build_pinneapple_spec` f — `pinneapple_problemdesign.codegen` (v02)
- `build_pipe_bend` f — `pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job` (v03)
- `build_plan` f — `pinneapple_problemdesign.knowledge.mapping` (v02)
- `build_plan_cfd_first` f — `pinneapple_problemdesign.knowledge.mapping` (v02)
- `build_plan_fno_first` f — `pinneapple_problemdesign.knowledge.mapping` (v02)
- `build_plan_pinn_first` f — `pinneapple_problemdesign.knowledge.mapping` (v02)
- `build_plugin` f — `pinneapple_simulation.accelerated_foam.build` (v03)
- `build_primitive` f — `pinneapple_design.geometry.gen.primitives` (v06)
- `build_problem` f — `pinneapple_mcp.problems` (v07)
- `build_prompt` f — `pinneapple_flow.planner.llm` (v07)
- `build_prompt` f — `pinneapple_llm.qa.generation` (v07)
- `build_pt_chain_graph` f — `pinneapple_analysis.proof.pt_chains` (v05)
- `build_recipe` f — `pinneapple_llm.cad_draft` (v07)

- build_report f — pinneapple_app.backend.core.surrogate_report (v07)
- build_report f — pinneapple_design.fitmesh.report (v06)
- build_report f — pinneapple_veriphysics.evidence_package (v05)
- build_scene f — pinneapple_blender.render (v07)
- build_scene f — pinneapple_simulation.external_solvers.genesis.scene_builder (v03)
- build_server f — pinneapple_mcp (v07)
- build_server f — pinneapple_mcp.server (v07)
- build_site f — pinneapple_flow.nodes.vehicle (v07)
- build_slurm_script f — pinneapple_neural.trainer.hpc (v04)
- build_soil f — pinneapple_flow.nodes.vehicle (v07)
- build_space f — pinneapple_hub.zoo.publish (v07)
- build_spec f — pinneapple_flow.nodes.vehicle (v07)
- build_spec f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- build_spheroid f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- build_stage_gaps f — pinneapple_problemdesign.elicitation.questions (v02)
- build_surrogate f — pinneapple_presets.surrogate (v02)
- build_tables f — pinneapple_analysis.benchmark.report (v05)
- build_tag_batch f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- build_terrain f — pinneapple_simulation.rover.autolab.render (v03)
- build_thread_polygon f — pinneapple_design.geometry.gen.threaded_connection (v06)
- build_torchrun_cmd f — pinneapple_neural.trainer.hpc (v04)
- build_twin f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- build_upd_dataloader f — pinneapple_data.dataloaders (v04)
- build_variant f — pinneapple_design.fitmesh.cad (v06)
- build_vehicle f — pinneapple_simulation.vehicle.build (v03)
- build_wheel_model f — pinneapple_flow.nodes.vehicle (v07)
- build_xtfc f — pinneapple_neural.architectures.pinns.xtfc (v04)
- build_xy f — pinneapple_physics.astro.spaceweather (v02)
- building_steps f — pinneapple_design.bricks.visuals (v06)
- builtin f — pinneapple_analysis.study_models (v05)
- builtin f — pinneapple_design.intent.templates (v06)
- builtin_cases f — pinneapple_analysis.validation.arena.cases (v05)
- BuiltProblem C — pinneapple_mcp.problems (v07)
- bulldozing_resistance f — pinneapple_simulation.terramechanics.wong_reece (v03)
- bundled_flows_using f — pinneapple_hub.library.flowlink (v07)
- BundleData C — pinneapple_tools.benchmark_suite.bundle.loader (v07)
- BundleSchema C — pinneapple_tools.benchmark_suite.bundle.schema (v07)
- Burgers C — pinneapple_analysis.benchmark.builtin.standard (v05)
- Burgers1D C — pinneapple_arena.problems (v04)
- Burgers1DTask C — pinneapple_tools.benchmark_suite.tasks.burgers_1d (v07)
- burgers_1d f — pinneapple_analysis.validation.pinn (v05)

- burgers_1d f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- burgers_1d_default f — pinneapple_physics.pde_environment.presets.academics (v02)
- burgers_fdm_baseline f — pinneapple_analysis.surrogate.examples (v05)
- burgers_fields f — pinneapple_analysis.surrogate.examples (v05)
- burgers_first f — pinneapple_flow.nodes_ext.pinn_examples (v07)
- burgers_inputs f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- burgers_periodic_pairs f — pinneapple_data.datasets.physics (v04)
- burgers_periodic_solve f — pinneapple_data.datasets.physics (v04)
- burgers_sine_exact f — pinneapple_physics.closed_form.burgers (v02)
- burgers_sine_inviscid_exact f — pinneapple_physics.closed_form.burgers (v02)
- BurgersProblem C — pinneapple_analysis.validation.fault_injection.problems (v05)
- buy_vs_build_recommendation f — pinneapple_analysis.verification.tool_recommendation (v05)
- by_key f — pinneapple_physics.weather.events (v02)
- by_node f — pinneapple_flow.decisions (v07)
- by_target_month f — pinneapple_systems.time_series.enso.evaluation (v06)
- ByteCacheStats C — pinneapple_data.cache_bytes (v04)
- ByteLRUCache C — pinneapple_data.cache_bytes (v04)

C

- cache_dir f — pinneapple_design.geometry_library.store (v06)
- cache_dir f — pinneapple_hub.zoo._artifacts (v07)
- cache_dir f — pinneapple_systems.sports.football.data (v06)
- cache_path f — pinneapple_data.datasets.download (v04)
- Cached C — pinneapple_condense.query (v02)
- CachedUPDZarrStore C — pinneapple_data.zarr_cached_store (v04)
- CachedUPDZarrStoreBytes C — pinneapple_data.zarr_cached_store_bytes (v04)
- CacheStats C — pinneapple_data.cache (v04)
- cad_reference f — pinneapple_physics.astro.smallbodies (v02)
- cad_step f — pinneapple_simulation.pipelines.freecad_openfoam.steps (v03)
- CadGeometry C — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- CADPart C — pinneapple_design.geometry.vehicle_cad (v06)
- cadquery_available f — pinneapple_design.geometry.gen.cadquery_gen (v06)
- cadquery_solid_to_upd f — pinneapple_design.geometry.io.cadquery_bridge (v06)
- cadquery_to_trimesh f — pinneapple_design.geometry.gen.cadquery_gen (v06)
- CadQueryBuildOptions C — pinneapple_design.geometry.gen.cadquery_gen (v06)

- `CadQueryRegistry` C — `pinneapple_design.geometry.gen.cadquery_gen` (v06)
- `CadQueryTessellationOptions` C — `pinneapple_design.geometry.gen.cadquery_gen` (v06)
- `CadRecipeResult` C — `pinneapple_llm.cad_draft` (v07)
- `CADToCFDPipeline` C — `pinneapple_simulation.numerical_solvers.cfd_pipeline` (v03)
- `CadVariant` C — `pinneapple_design.fitmesh.cad` (v06)
- `cae_geometry` f — `pinneapple_check._reuse` (v05)
- `cae_io` f — `pinneapple_check._reuse` (v05)
- `calculix_cantilever` f — `pinneapple_flow.wrappers` (v07)
- `calculix_grid` f — `pinneapple_tools.visualization.pyvista_bridge` (v07)
- `calculix_model` f — `pinneapple_design.intent.physics.bracket` (v06)
- `CalculiXAdapter` C — `pinneapple_core.workflows.adapters` (v01)
- `CalculixCase` C — `pinneapple_lab.experiments.calculix_cases` (v07)
- `CalculixModel` C — `pinneapple_simulation.external_solvers.calculix.inp` (v03)
- `calendar_grid` f — `pinneapple_systems.time_series.forecast_lab.plots` (v06)
- `calibrate` f — `pinneapple_analysis.benchmark.builtin.inverse_heat_impl` (v05)
- `calibrate` f — `pinneapple_analysis.inverse_problems.calibration` (v05)
- `calibrate` f — `pinneapple_analysis.validation.arena.blue` (v05)
- `calibrate` f — `pinneapple_flow.nodes.solvers` (v07)
- `calibrate` f — `pinneapple_physics.astro.conditions` (v02)
- `calibrate` f — `pinneapple_physics.pcb.engine` (v02)
- `calibrate` f — `pinneapple_simulation.terramechanics.testbed` (v03)
- `calibrate` f — `pinneapple_systems.process_components.explicit_equation_system` (v06)
- `calibrate` f — `pinneapple_twin3d.wear.calibrate` (v06)
- `calibrate_holdout` f — `pinneapple_simulation.terramechanics.testbed` (v03)
- `calibrate_scores` f — `pinneapple_analysis.validation.arena.blue` (v05)
- `calibrate_tolerances` f — `pinneapple_physics.darcy_checks` (v02)
- `Calibration` C — `pinneapple_twin3d.wear.calibrate` (v06)
- `calibration_against_reference` f — `pinneapple_veriphysics.robustness` (v05)
- `calibration_curve` f — `pinneapple_tools.model_evaluation.metrics` (v07)
- `calibration_study` f — `pinneapple_analysis.inverse_problems.contracts` (v05)
- `CalibrationMetrics` C — `pinneapple_analysis.uncertainty.calibration` (v05)
- `CalibrationResult` C — `pinneapple_analysis.inverse_problems.calibration` (v05)
- `CalibrationResult` C — `pinneapple_systems.process_components.explicit_equation_system` (v06)
- `CalibrationSummary` C — `pinneapple_analysis.verification.physics_confidence_score`

(v05)
- `call_infer` f — `pinneapple_arena.external.runners.registry` (v04)
- `call_llm` f — `pinneapple_llm._dispatch` (v07)
- `call_ollama` f — `pinneapple_llm.local_llm` (v07)
- `call_quiet` f — `pinneapple_presets.core` (v02)
- `callable_exact` f — `pinneapple_tools.benchmark_suite.crossframework.problems` (v07)
- `CallableExecutor` C — `pinneapple_decision.loop` (v05)
- `CallableModel` C — `pinneapple_analysis.inverse_problems.forward` (v05)
- `CallablePostModel` C — `pinneapple_neural.workflows.hybrid_surrogate_physics` (v04)
- `CallLLM` C — `pinneapple_design.intent.parse` (v06)
- `CallLog` C — `pinneapple_design.intent.pipeline` (v06)
- `Camera` C — `pinneapple_physics.astro.observing` (v02)
- `Camera` C — `pinneapple_physics.blackhole.raytrace` (v02)
- `camera_frames` f — `pinneapple_worldmodel.robot_perception` (v04)
- `camera_head` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `camera_transform` f — `pinneapple_worldmodel.inspection_data` (v04)
- `CameraConfig` C — `pinneapple_worldmodel.camera_system` (v04)
- `CameraSystem` C — `pinneapple_worldmodel.camera_system` (v04)
- `CameraVariation` C — `pinneapple_worldmodel.robot_perception` (v04)
- `Campaign` C — `pinneapple_simulation.external_solvers.openfoam.campaign` (v03)
- `campaign` f — `pinneapple_flow.nodes_ext.cfd` (v07)
- `campaign` f — `pinneapple_simulation.electromagnetics.periodic.benchmark` (v03)
- `CampaignPlan` C — `pinneapple_data.doe.campaign` (v04)
- `CampaignRun` C — `pinneapple_simulation.external_solvers.openfoam.campaign` (v03)
- `can_render` f — `pinneapple_tools.visualization.pyvista_bridge` (v07)
- `cancel` f — `pinneapple_mcp.jobs` (v07)
- `cancel_job` f — `pinneapple_mcp.tools` (v07)
- `Cancelled` C — `pinneapple_llm.workbench.runtime` (v07)
- `Candidate` C — `pinneapple_analysis.surrogate.methods` (v05)
- `Candidate` C — `pinneapple_data.formfill.extract` (v04)
- `Candidate` C — `pinneapple_design.bricks.search` (v06)
- `Candidate` C — `pinneapple_design.intent.pipeline` (v06)
- `candidate_ids` f — `pinneapple_analysis.surrogate.methods` (v05)
- `CandidateAssessment` C — `pinneapple_autolab.trust` (v07)
- `CandidateCause` C — `pinneapple_analysis.inverse_problems.counterfactual_attribution` (v05)
- `CandidateDetail` C — `pinneapple_veriphysics.recommend` (v05)
- `CandidateLibrary` C — `pinneapple_analysis.inverse_problems.missing_term` (v05)
- `CandidateResult` C — `pinneapple_analysis.verification.pipeline` (v05)
- `candidates` f — `pinneapple_design.intent.materials` (v06)
- `canonical_json` f — `pinneapple_security.integrity` (v05)
- `canonical_json` f — `pinneapple_veriphysics.ledger` (v05)
- `canonical_name` f — `pinneapple_data.unified` (v04)
- `Cantilever` C — `pinneapple_design.qualitative.models` (v06)

- cantilever f — pinneapple_lab.experiments.rom (v07)
- cantilever_3d f — pinneapple_analysis.surrogate.examples (v05)
- cantilever_bending_stress_at_position_pa f — pinneapple_physics.closed_form.cantilever_fatigue (v02)
- cantilever_compare f — pinneapple_core.workflows.templates (v01)
- cantilever_deck f — pinneapple_core.workflows.templates (v01)
- cantilever_frequency_roots f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- cantilever_hex_mesh f — pinneapple_simulation.external_solvers.calculix.inp (v03)
- cantilever_root_bending_stress_pa f — pinneapple_physics.closed_form.cantilever_fatigue (v02)
- cantilever_space f — pinneapple_simulation.external_solvers.campaign_cases (v03)
- CantileverGoal C — pinneapple_design.bricks.goals (v06)
- capabilities f — pinneapple_flow.planner.extensions (v07)
- capabilities f — pinneapple_mcp.tools (v07)
- Capability C — pinneapple_flow.planner.extensions (v07)
- capacitance f — pinneapple_simulation.electromagnetics.electrostatics (v03)
- capacity f — pinneapple_design.bricks.statics (v06)
- capsule2d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- Capture C — pinneapple_lab.capture (v07)
- capture f — pinneapple_autolab.dashboard (v07)
- capture_twin f — pinneapple_physics.blackhole.capture (v02)
- car_brake_thermal f — pinneapple_physics.pde_environment.presets.engineering (v02)
- car_external_aero f — pinneapple_physics.pde_environment.presets.engineering (v02)
- car_mesh f — pinneapple_design.intent.physics.loop_track (v06)
- car_suspension_fatigue f — pinneapple_physics.pde_environment.presets.engineering (v02)
- Card C — pinneapple_core.onboarding.catalog (v01)
- card f — pinneapple_core.onboarding.catalog (v01)
- card f — pinneapple_physics.astro.facts (v02)
- card_from_guardrail f — pinneapple_hub.verified_card (v07)
- card_main f — pinneapple_core.onboarding.cli (v01)
- card_section f — pinneapple_hub.zoo.validation_reports (v07)
- CardRefused C — pinneapple_hub.verified_card (v07)
- CarLBM C — pinneapple_lab.experiments.car (v07)
- CarProfile C — pinneapple_design.geometry.gen.car2d (v06)
- CarSurrogate C — pinneapple_lab.experiments.car (v07)
- Case C — pinneapple_analysis.validation.fault_injection.problems (v05)
- Case C — pinneapple_systems.time_series.forecast_lab.presets (v06)
- case f — pinneapple_simulation.electromagnetics.machines.verification (v03)
- case f — pinneapple_simulation.electromagnetics.verification (v03)
- case f — pinneapple_systems.time_series.forecast_lab.presets (v06)
- case_id f — pinneapple_simulation.pipelines.freecad_openfoam.sweep (v03)

- case_panels f — pinneapple_systems.time_series.enso.pipeline (v06)
- case_zip f — pinneapple_design.aero.case (v06)
- CaseFields C — pinneapple_simulation.numerical_solvers.external_flow (v03)
- CaseResult C — pinneapple_simulation.electromagnetics.verification (v03)
- CaseResult C — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- cast_shadow f — pinneapple_tools.visualization.studio.shadows (v07)
- catalog f — pinneapple_flow (v07)
- catalog f — pinneapple_tools.automation (v07)
- catalog_family f — pinneapple_arena.decision_mode (v04)
- catalog_prompt f — pinneapple_flow.planner.llm (v07)
- catalog_sky f — pinneapple_physics.astro.lensing (v02)
- catalog_status f — pinneapple_arena.pointwise.architectures (v04)
- catalogue f — pinneapple_core.onboarding.catalog (v01)
- catalogue f — pinneapple_hub.zoo (v07)
- CatBoostForecaster C — pinneapple_systems.time_series.models.classical (v06)
- categorical f — pinneapple_systems.time_series.enso.evaluation (v06)
- categories f — pinneapple_design.geometry_library (v06)
- category f — pinneapple_systems.time_series.enso.evaluation (v06)
- category_of f — pinneapple_hub.library.categories (v07)
- category_of f — pinneapple_veriphysics.validation_lab (v05)
- CauchyMisfit C — pinneapple_analysis.inverse_problems.noise_models (v05)
- causal_weights f — pinneapple_core.loss (v01)
- CausalPINNTrainer C — pinneapple_neural.trainer.causal (v04)
- CausalWeightScheduler C — pinneapple_neural.trainer.causal (v04)
- Cause C — pinneapple_physics.physics_audit.attribution (v02)
- CavityResonance C — pinneapple_science.em.waves (v02)
- cb_block f — pinneapple_condense.transient (v02)
- CBBlock C — pinneapple_condense.transient (v02)
- CBTransient C — pinneapple_condense.transient (v02)
- ccx_backend f — pinneapple_simulation.external_solvers.calculix.runner (v03)
- CcxRun C — pinneapple_simulation.external_solvers.calculix.runner (v03)
- cd_crisis f — pinneapple_systems.sports.football.ball (v06)
- ceemdan_1d f — pinneapple_simulation.numerical_solvers.ceemdan (v03)
- CEEMDANSolver C — pinneapple_simulation.numerical_solvers.ceemdan (v03)
- cell f — pinneapple_analysis.benchmark.report (v05)
- cell_adjacency f — pinneapple_data.cae.geometry (v04)
- cell_centers f — pinneapple_twin3d.wear.geometry (v06)
- cell_centres f — pinneapple_design.aero.mesh (v06)
- cell_centres f — pinneapple_simulation.numerical_solvers.free_surface (v03)
- cell_centres_volumes f — pinneapple_data.cae.compare (v04)
- cell_geometry f — pinneapple_core.fem (v01)

- `cell_geometry` f — `pinneapple_data.cae.geometry` (v04)
- `cell_mesh` f — `pinneapple_twin3d.wear.geometry` (v06)
- `cell_simulator` f — `pinneapple_simulation.electromagnetics.periodic.benchmark` (v03)
- `cell_to_vertex` f — `pinneapple_twin3d.wear.geometry` (v06)
- `cell_volumes` f — `pinneapple_physics.blackhole.convergence` (v02)
- `CellBlock` C — `pinneapple_data.cae.model` (v04)
- `CellFit` C — `pinneapple_twin3d.wear.forecast` (v06)
- `cem` f — `pinneapple_simulation.robot_sim.learn` (v03)
- `CEMResult` C — `pinneapple_simulation.robot_sim.learn` (v03)
- `central_derivative` f — `pinneapple_science.core` (v02)
- `central_difference` f — `pinneapple_systems.process_components.real_gas_eos` (v06)
- `centre_of_pressure` f — `pinneapple_design.bricks.statics` (v06)
- `certify_model` f — `pinneapple_hub.verified_card` (v07)
- `cfd_eval` f — `pinneapple_design.aero.verify` (v06)
- `CFDMesh` C — `pinneapple_simulation.numerical_solvers.cfd_pipeline` (v03)
- `CfdOFAdapter` C — `pinneapple_tools.automation.adapters.freecad` (v07)
- `cfg_str` f — `pinneapple_tools.benchmark_suite.meta_benchmark` (v07)
- `cgns_to_upd` f — `pinneapple_simulation.external_solvers.cfd_formats.cgns_reader` (v03)
- `Chain` C — `pinneapple_reason.chain` (v05)
- `chain` f — `pinneapple_flow.nodes.reason` (v07)
- `chain_checks` f — `pinneapple_reason.trust` (v05)
- `chain_hamiltonian` f — `pinneapple_analysis.proof.pt_chains` (v05)
- `chamfer_edges` f — `pinneapple_design.geometry.ops.occ_features` (v06)
- `chamfered_box` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `chamfered_rect` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `changepoint_plot` f — `pinneapple_systems.time_series.eda.plots` (v06)
- `changepoints_ruptures` f — `pinneapple_systems.time_series.audit.tests` (v06)
- `Channel` C — `pinneapple_systems.operations.anomaly` (v06)
- `channel_flow_3d` f — `pinneapple_physics.pde_environment.presets.cfd` (v02)
- `channel_names` f — `pinneapple_physics.weather.data` (v02)
- `channel_with_hole` f — `pinneapple_design.geometry.csg` (v06)
- `ChannelConfig` C — `pinneapple_simulation.numerical_solvers.thermal_channel_2d` (v03)
- `ChannelDomain2D` C — `pinneapple_design.geometry.gen.domains` (v06)
- `ChannelDomain3D` C — `pinneapple_design.geometry.gen.domains3d` (v06)
- `ChannelFlowConfig3D` C — `pinneapple_simulation.numerical_solvers.fdm3d` (v03)
- `ChannelFlowSolver3D` C — `pinneapple_simulation.numerical_solvers.fdm3d` (v03)
- `ChannelWithObstacleDomain2D` C — `pinneapple_design.geometry.gen.domains` (v06)

- characteristic_length f — pinneapple_app.backend.core.physics_adapter (v07)
- characteristic_velocity f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- Chart C — pinneapple_tools.visualization.dashboard.spec (v07)
- chart_points f — pinneapple_design.design_optimizer.report (v06)
- chassis_rotation f — pinneapple_simulation.vehicle.viz (v03)
- chebyshev_gauss_lobatto f — pinneapple_simulation.numerical_solvers.xtfc_ivp (v03)
- checagem_booleana f — pinneapple_analysis.trust.trust_report (v05)
- checagem_erro_relativo f — pinneapple_analysis.trust.trust_report (v05)
- Check C — pinneapple_analysis.benchmark.registry (v05)
- Check C — pinneapple_analysis.proof.graph (v05)
- Check C — pinneapple_analysis.trust.trust_report (v05)
- Check C — pinneapple_core.onboarding.doctor (v01)
- Check C — pinneapple_tools.automation.record (v07)
- check f — pinneapple_data.expressions (v04)
- check f — pinneapple_data.preflight.calculix (v04)
- check f — pinneapple_data.preflight.openfoam (v04)
- check f — pinneapple_flow.nodes_ext.render_worker (v07)
- check f — pinneapple_flow.references (v07)
- check f — pinneapple_hub.library (v07)
- check f — pinneapple_presets.core (v02)
- check f — pinneapple_systems.operations.contracts (v06)
- check_answer f — pinneapple_data.formfill.extract (v04)
- check_api_contracts f — pinneapple_llm.qa.static (v07)
- check_applicable f — pinneapple_arena.pointwise.adapters (v04)
- check_balance f — pinneapple_check.checks.results (v05)
- check_block_step f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- check_citations f — pinneapple_lab.results_rules (v07)
- check_claims f — pinneapple_llm.qa.generation (v07)
- check_code f — pinneapple_design.fitmesh.llm (v06)
- check_config_entry f — pinneapple_physics.darcy_checks (v02)
- check_config_entry f — pinneapple_physics.maxwell_checks (v02)
- check_cors_origins f — pinneapple_security.web (v05)
- check_dataset f — pinneapple_check.checks.datasets (v05)
- check_directivity_limit f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_eclipses f — pinneapple_physics.astro.validation (v02)
- check_efficiency_le_one f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_efficiency_ratio f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_envelope f — pinneapple_simulation.external_solvers.calculix.virtual_ccx (v03)
- check_equations f — pinneapple_simulation.electromagnetics.units (v03)
- check_finite f — pinneapple_simulation.numerical_solvers.etdrk4 (v03)
- check_flow f — pinneapple_flow.schema (v07)

- check_fmu f — pinneapple_simulation.external_solvers.modelica.fmi_compat (v03)
- check_fmus f — pinneapple_simulation.external_solvers.modelica.fmi_compat (v03)
- check_frame_origins f — pinneapple_security.web (v05)
- check_geometry f — pinneapple_llm.qa.validators.geometry (v07)
- check_guardrail f — pinneapple_llm.qa.validators.guardrail (v07)
- check_horizons f — pinneapple_physics.astro.validation (v02)
- check_imports f — pinneapple_llm.qa.static (v07)
- check_integrators f — pinneapple_physics.astro.validation (v02)
- check_integrity f — pinneapple_check.checks.integrity (v05)
- check_iss_passes f — pinneapple_physics.astro.validation (v02)
- check_json_artifact f — pinneapple_llm.qa.static (v07)
- check_kwargs_in_bounds f — pinneapple_llm.draft (v07)
- check_library f — pinneapple_analysis.validation.fault_injection.checks (v05)
- check_line f — pinneapple_lab.curation (v07)
- check_line_fits f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- check_main f — pinneapple_check.cli (v05)
- check_member_name f — pinneapple_security.archives (v05)
- check_model f — pinneapple_check.checks.models (v05)
- check_nodal_darcy_batch f — pinneapple_physics.darcy_checks (v02)
- check_novas f — pinneapple_physics.astro.validation (v02)
- check_numeric f — pinneapple_llm.qa.validators.numeric (v07)
- check_outbound_url f — pinneapple_security.web (v05)
- check_overrides f — pinneapple_data.formfill.pipeline (v04)
- check_peak_ge_boresight f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_period_recovery f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- check_physics f — pinneapple_llm.qa.validators.physics (v07)
- check_pin f — pinneapple_check.checks.integrity (v05)
- check_power_closure f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_realized_gain f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_relations f — pinneapple_analysis.validation.metamorphic (v05)
- check_release f — pinneapple_hub.zoo.validation_reports (v07)
- check_repo f — pinneapple_lab.manifest (v07)
- check_report f — pinneapple_analysis.validation.report_artifact (v05)
- check_result f — pinneapple_check.checks.results (v05)
- check_rows f — pinneapple_tools.nvidia.figures (v07)
- check_s11_resolved f — pinneapple_simulation.electromagnetics.antenna.checks (v03)
- check_security f — pinneapple_llm.qa.static (v07)
- check_sensors f — pinneapple_llm.qa.validators.sensors (v07)
- check_signatures f — pinneapple_llm.qa.static (v07)

- check_skyfield_live f — pinneapple_physics.astro.validation (v02)
- check_spec f — pinneapple_hub.zoo (v07)
- check_spec f — pinneapple_hub.zoo.checks (v07)
- check_summary f — pinneapple_simulation.electromagnetics.antenna.study (v03)
- check_syntax f — pinneapple_llm.qa.static (v07)
- check_temporal f — pinneapple_llm.qa.validators.temporal (v07)
- check_units f — pinneapple_check.checks.common (v05)
- check_units f — pinneapple_llm.qa.validators.units (v07)
- check_units f — pinneapple_simulation.electromagnetics.units (v03)
- check_usno_almanac f — pinneapple_physics.astro.validation (v02)
- check_values f — pinneapple_check.checks.common (v05)
- check_values f — pinneapple_hub.zoo.checks (v07)
- check_xd_means f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- check_zip f — pinneapple_security.archives (v05)
- check_zoo_relations f — pinneapple_hub.zoo.relations (v07)
- CheckItem C — pinneapple_systems.digital_twin.product.evidence (v06)
- CheckList C — pinneapple_lab.results_rules (v07)
- ChecklistItem C — pinneapple_veriphysics.applicability (v05)
- CheckOutcome C — pinneapple_analysis.proof.graph (v05)
- Checkpoint C — pinneapple_neural.trainer.checkpoint (v04)
- CheckRecord C — pinneapple_data.contracts.models (v04)
- CheckReport C — pinneapple_systems.digital_twin.product.evidence (v06)
- CheckResult C — pinneapple_analysis.validation.core (v05)
- CheckResult C — pinneapple_design.surrogate_inverse.core (v06)
- CheckResult C — pinneapple_llm.guardrail (v07)
- CheckResult C — pinneapple_physics.physics_audit.monitor (v02)
- checks_figure f — pinneapple_tools.nvidia.figures (v07)
- checks_module f — pinneapple_hub.zoo.checks (v07)
- checks_parity f — pinneapple_hub.zoo.export (v07)
- CheckSpec C — pinneapple_analysis.validation.schema (v05)
- ChecksumMismatchError C — pinneapple_hub.zoo._artifacts (v07)
- chi f — pinneapple_reason.modes.structural (v05)
- choice_for_objective f — pinneapple_decision.loop (v05)
- ChoiceNode C — pinneapple_decision.tree (v05)
- choked_mass_flow f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- choose_next f — pinneapple_autolab.trust (v07)
- choose_tiles f — pinneapple_lab.showcase (v07)
- chow_test_mean_shift f — pinneapple_systems.time_series.audit.tests (v06)
- ChronosForecaster C — pinneapple_systems.time_series.foundation_models (v06)
- chunk_text f — pinneapple_analysis.retrieval.corpus (v05)
- chunk_text f — pinneapple_tools.hpo_experiments.kb.chunking (v07)
- ChunkSpec C — pinneapple_data.chunking (v04)
- Circle C — pinneapple_worldmodel.geometry (v04)
- circle f — pinneapple_design.geometry.gen.sdf_shapes (v06)

- `circle_segments` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `Circuits` C — `pinneapple_science.em.induction` (v02)
- `circular_arc_cascade_blade_points` f — `pinneapple_physics.pde_environment.presets._curve_geometry` (v02)
- `circular_arc_tvd_hd` f — `pinneapple_systems.process_components.curved_path_geometry` (v06)
- `circular_waveguide_cutoff` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `circular_waveguide_modes` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `cite` f — `pinneapple_flow.references` (v07)
- `clamp_confidence` f — `pinneapple_tools.dataset_quality.vlm_curation` (v07)
- `ClappHornberger` C — `pinneapple_simulation.geophysics.richards` (v03)
- `classical` f — `pinneapple_flow.nodes.solvers` (v07)
- `classical_eddy_coefficient` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `classical_job` f — `pinneapple_analysis.benchmark.builtin.inverse_heat_impl` (v05)
- `ClassicalBackend` C — `pinneapple_quantum.backends.backend` (v04)
- `ClassicalQNode` C — `pinneapple_quantum.backends.backend` (v04)
- `ClassicalTSBase` C — `pinneapple_neural.architectures.classical_ts.base` (v04)
- `ClassicalTSCatalog` C — `pinneapple_neural.architectures.classical_ts.registry` (v04)
- `ClassicalTSOutput` C — `pinneapple_neural.architectures.classical_ts.base` (v04)
- `ClassicalTuner` C — `pinneapple_systems.time_series.tuning.optuna_tuner` (v06)
- `ClassicalVQC` C — `pinneapple_quantum.circuits.base` (v04)
- `classify` f — `pinneapple_physics.symbolic_pde.text` (v02)
- `classify_ambiguity` f — `pinneapple_analysis.inverse_problems.counterfactual_attribution` (v05)
- `classify_buckling_mode` f — `pinneapple_systems.process_components.pipe_stress_mechanics` (v06)
- `classify_coupling` f — `pinneapple_physics.non_hermitian.ep_tuning` (v02)
- `classify_flow_regime` f — `pinneapple_analysis.verification.dimensional_analysis` (v05)
- `classify_geometry` f — `pinneapple_analysis.verification.geometry_intelligence` (v05)
- `classify_modes` f — `pinneapple_simulation.flight.trim` (v03)
- `classify_physics_shape` f — `pinneapple_physics.codegen.fenics_script_generator` (v02)
- `clean_for_modeling` f — `pinneapple_data.process_optimizer` (v04)
- `clean_mesh` f — `pinneapple_design.geometry.ops.clean` (v06)
- `clean_scan` f — `pinneapple_twin3d.scan` (v06)
- `CleanMeshConfig` C — `pinneapple_design.geometry.ops.clean` (v06)
- `cleanup` f — `pinneapple_worldmodel.inspection_data` (v04)
- `clear_cache` f — `pinneapple_condense.substructure` (v02)
- `client_ip` f — `pinneapple_security.web` (v05)

- clift_sphere_cd f — pinneapple_physics.closed_form.particle_drag (v02)
- climate_atmosphere_2d f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- climate_ocean_gyre f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- Climatology C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- climatology f — pinneapple_physics.weather.data (v02)
- climatology_at f — pinneapple_physics.weather.data (v02)
- climatology_forecast f — pinneapple_physics.weather.evaluate (v02)
- clip_grid f — pinneapple_flow.nodes_ext.render (v07)
- CLISimulator C — pinneapple_simulation.electromagnetics.antenna.runner (v03)
- Clock C — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- close f — pinneapple_science.plots (v02)
- close_approaches f — pinneapple_physics.astro.smallbodies (v02)
- close_sheets f — pinneapple_design.geometry_library.entries._adapt (v06)
- closed_form f — pinneapple_design.intent.physics.loop_track (v06)
- closed_multiscale_residual f — pinneapple_physics.multiscale.comparison (v02)
- closest_to_schwabe_cycle f — pinneapple_systems.time_series.solar_cycle (v06)
- ClosestApproach C — pinneapple_physics.astro.conjunction (v02)
- closure_flag_table f — pinneapple_physics.multiscale.comparison (v02)
- closure_health f — pinneapple_physics.multiscale.closures (v02)
- ClosureHealth C — pinneapple_physics.multiscale.closures (v02)
- Cloud C — pinneapple_condense.meshless (v02)
- cloud f — pinneapple_condense.meshless2d (v02)
- cloud_at f — pinneapple_physics.astro.conditions (v02)
- clusters f — pinneapple_design.bricks.dynamics (v06)
- cmaes f — pinneapple_analysis.validation.arena.search (v05)
- cmap_color f — pinneapple_tools.visualization.studio.compose (v07)
- cmd_autocalls f — pinneapple_flow.__main__ (v07)
- cmd_catalog f — pinneapple_flow.__main__ (v07)
- cmd_coverage f — pinneapple_flow.__main__ (v07)
- cmd_doctor f — pinneapple_llm.workbench.__main__ (v07)
- cmd_eval f — pinneapple_llm.workbench.__main__ (v07)
- cmd_examples f — pinneapple_flow.__main__ (v07)
- cmd_finetune f — pinneapple_llm.workbench.__main__ (v07)
- cmd_list f — pinneapple_arena.external.__main__ (v04)
- cmd_plan f — pinneapple_flow.__main__ (v07)
- cmd_pull f — pinneapple_llm.workbench.__main__ (v07)
- cmd_references f — pinneapple_flow.__main__ (v07)
- cmd_run f — pinneapple_arena.external.__main__ (v04)
- cmd_run f — pinneapple_flow.__main__ (v07)
- cmd_run f — pinneapple_llm.workbench.__main__ (v07)
- cmd_serve f — pinneapple_llm.workbench.__main__ (v07)
- cmd_studio f — pinneapple_flow.__main__ (v07)

- cmd_validate f — pinneapple_flow.__main__ (v07)
- cmd_vscode f — pinneapple_llm.workbench.__main__ (v07)
- coarse_power f — pinneapple_condense.homogenize (v02)
- coarsen f — pinneapple_physics.weather.events (v02)
- coarsen f — pinneapple_systems.time_series.enso.data (v06)
- coastlines f — pinneapple_physics.weather.viz (v02)
- coax_capacitance f — pinneapple_simulation.electromagnetics.statics (v03)
- coax_field f — pinneapple_simulation.electromagnetics.analytic (v03)
- coax_inductance f — pinneapple_simulation.electromagnetics.statics (v03)
- code_blocks f — pinneapple_hub.zoo.hub_cards (v07)
- code_for f — pinneapple_hub.library.compose (v07)
- code_hash f — pinneapple_hub.library.sandbox (v07)
- code_repo_documents f — pinneapple_analysis.retrieval.corpus (v05)
- CodeNotAllowed C — pinneapple_veriphysics.validation_lab (v05)
- coef_from_targets f — pinneapple_design.aero.surrogate (v06)
- coef_scores f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- coef_targets f — pinneapple_design.aero.surrogate (v06)
- coefficient_stats f — pinneapple_physics.darcy_checks (v02)
- CoefficientEnvelope C — pinneapple_physics.darcy_checks (v02)
- coefs f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- coefs f — pinneapple_design.aero.train (v06)
- coil_end_path f — pinneapple_simulation.electromagnetics.machines.endwinding (v03)
- coil_impedance f — pinneapple_physics.closed_form.eddy_coil (v02)
- CoilSide C — pinneapple_simulation.electromagnetics.postprocess (v03)
- CoilSpringTemplate C — pinneapple_design.intent.templates.example_spring (v06)
- cole_hopf f — pinneapple_analysis.validation.pinn (v05)
- colebrook f — pinneapple_physics.closed_form.pipe_flow (v02)
- colebrook f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- colebrook_white_f f — pinneapple_systems.process_components.pipe_network_1d (v06)
- collate_pinn_batches f — pinneapple_data.collate (v04)
- collate_upd_supervised f — pinneapple_data.collate (v04)
- collect f — pinneapple_analysis.benchmark.report (v05)
- collect f — pinneapple_flow.decisions (v07)
- collect f — pinneapple_hub.library (v07)
- collect f — pinneapple_lab.explorer (v07)
- collect f — pinneapple_lab.html (v07)
- collect_outputs f — pinneapple_core.workflows.templates (v01)
- collect_studies f — pinneapple_analysis.benchmark.report (v05)
- collision_probability_2d f — pinneapple_physics.astro.conjunction (v02)
- collision_probability_centered f — pinneapple_physics.astro.conjunction (v02)
- collocation_pool f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)

- `CollocationBatch3D` C — `pinneapple_design.geometry.mesh_collocator` (v06)
- `CollocationConfig` C — `pinneapple_app.backend.core.collocation` (v07)
- `CollocationConfig` C — `pinneapple_data.collocation` (v04)
- `CollocationRequest` C — `pinneapple_app.backend.routers.experiments` (v07)
- `CollocationSampler` C — `pinneapple_core.data` (v01)
- `CollocationSampler` C — `pinneapple_data.collocation` (v04)
- `color_of` f — `pinneapple_systems.sports.football.plots` (v06)
- `ColorBar` C — `pinneapple_tools.visualization.dashboard.spec` (v07)
- `colorbar` f — `pinneapple_tools.visualization.studio.colormap` (v07)
- `colorbar_image` f — `pinneapple_tools.visualization.studio.compose` (v07)
- `Colour` C — `pinneapple_design.bricks.parts` (v06)
- `colour` f — `pinneapple_design.bricks.parts` (v06)
- `colour_range` f — `pinneapple_tools.visualization.studio.fieldview` (v07)
- `colour_rgb` f — `pinneapple_design.bricks.assembly` (v06)
- `Column` C — `pinneapple_analysis.benchmark.registry` (v05)
- `Column` C — `pinneapple_data.contracts.models` (v04)
- `column_collapse` f — `pinneapple_flow.nodes_ext.geomechanics` (v07)
- `ColumnHints` C — `pinneapple_arena.pointwise.schema` (v04)
- `columns` f — `pinneapple_analysis.study_models` (v05)
- `columns` f — `pinneapple_simulation.electromagnetics.antenna.surrogates` (v03)
- `ColumnSubset` C — `pinneapple_systems.time_series.enso.forecasters` (v06)
- `combine` f — `pinneapple_core.loss` (v01)
- `combine` f — `pinneapple_science.instruments` (v02)
- `combine` f — `pinneapple_security.classification` (v05)
- `combine_mass` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `CombinedAL` C — `pinneapple_data.active_learning` (v04)
- `CombinedLoss` C — `pinneapple_neural.trainer.losses` (v04)
- `CommandAdapter` C — `pinneapple_core.workflows.adapters` (v01)
- `CommandEvaluator` C — `pinneapple_design.design_optimizer.evaluators` (v06)
- `commercial_mode` f — `pinneapple_neural._licencas` (v04)
- `commission` f — `pinneapple_systems.digital_twin.product.runner` (v06)
- `commutation_error` f — `pinneapple_physics.multiscale.filters` (v02)
- `compact` f — `pinneapple_design.geometry_library.meshops` (v06)
- `compact_range` f — `pinneapple_flow.nodes_ext.em_antenna` (v07)
- `CompactRange` C — `pinneapple_simulation.electromagnetics.reflector.compact_range` (v03)
- `compare` f — `pinneapple_condense.homogenize` (v02)
- `compare` f — `pinneapple_data.cae.compare` (v04)
- `compare` f — `pinneapple_design.bricks.search` (v06)
- `compare` f — `pinneapple_flow.nodes.analysis` (v07)
- `compare` f — `pinneapple_flow.nodes_ext.examples` (v07)
- `compare` f — `pinneapple_hub.zoo.checks` (v07)

- compare f — pinneapple_llm.qa.references (v07)
- compare f — pinneapple_physics.solving (v02)
- compare f — pinneapple_systems.sports.football.pc_study (v06)
- compare f — pinneapple_systems.sports.football.xg (v06)
- compare_contact_models f — pinneapple_simulation.rover.autolab.wheel_models (v03)
- compare_controllers f — pinneapple_systems.operations.control (v06)
- compare_design f — pinneapple_design.aero.verify (v06)
- compare_fields f — pinneapple_tools.visualization.fields (v07)
- compare_methods f — pinneapple_analysis.inverse_problems.studio (v05)
- compare_methods f — pinneapple_design.design_optimizer.compare (v06)
- compare_models f — pinneapple_autolab.surrogates (v07)
- compare_on_states f — pinneapple_simulation.terramechanics.surrogate (v03)
- compare_preset f — pinneapple_flow.wrappers (v07)
- compare_replay f — pinneapple_tools.nvidia.isaac (v07)
- compare_runs f — pinneapple_tools.benchmark_suite.runner.compare (v07)
- compare_solvers f — pinneapple_analysis.verification.solver_orchestration (v05)
- compare_to_analytical f — pinneapple_analysis.validation.analytical (v05)
- compare_variants f — pinneapple_presets.report (v02)
- compare_with_references f — pinneapple_design.fitmesh.cad (v06)
- compare_with_sgp4 f — pinneapple_physics.astro.decay (v02)
- Comparison C — pinneapple_physics.solving (v02)
- Comparison C — pinneapple_science.core (v02)
- comparison f — pinneapple_physics.astro.facts (v02)
- ComparisonReport C — pinneapple_analysis.verification.solver_orchestration (v05)
- compatible f — pinneapple_flow.types (v07)
- CompatRow C — pinneapple_simulation.external_solvers.modelica.fmi_compat (v03)
- Compilation C — pinneapple_data.formfill.extract (v04)
- compile f — pinneapple_data.formfill.extract (v04)
- compile f — pinneapple_physics.compile_api (v02)
- compile_documents f — pinneapple_data.formfill.pipeline (v04)
- compile_expr f — pinneapple_physics.symbolic_pde.text (v02)
- compile_expression f — pinneapple_mcp.expressions (v07)
- compile_flags f — pinneapple_simulation.accelerated_foam.build (v03)
- compile_physics f — pinneapple_physics (v02)
- compile_problem f — pinneapple_physics.pinn_solver.compiler.compile (v02)
- compile_requirements f — pinneapple_data.udr1 (v04)
- compile_system f — pinneapple_simulation.numerical_solvers.symbolic_fd (v03)
- CompiledEquation C — pinneapple_physics.pinn_solver.factory.sympy_backend (v02)
- CompiledProblem C — pinneapple_physics.compile_api (v02)
- Compliance2D C — pinneapple_design.topology.problems (v06)
- Component C — pinneapple_physics.pcb.solver (v02)
- Component C — pinneapple_simulation.flight.mass (v03)

- component_fit f — pinneapple_flow.nodes_ext.geomechanics (v07)
- ComponentBenchmarkResult C — pinneapple_systems.component_library.benchmark (v06)
- ComponentModel C — pinneapple_systems.component_library.base (v06)
- ComponentRegistry C — pinneapple_systems.component_library.registry (v06)
- components f — pinneapple_check.numerics (v05)
- components f — pinneapple_design.geometry_library.meshops (v06)
- ComponentSpec C — pinneapple_systems.component_library.registry (v06)
- Compose C — pinneapple_core.transforms (v01)
- compose f — pinneapple_hub.library.compose (v07)
- compose_grid f — pinneapple_tools.visualization.studio.compose (v07)
- compose_grid_video f — pinneapple_tools.visualization.studio.compose (v07)
- compose_video f — pinneapple_tools.visualization.studio.particles (v07)
- ComposedGuidance C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- ComposedObsOperator C — pinneapple_analysis.inverse_problems.obs_operator (v05)
- ComposeError C — pinneapple_hub.library.compose (v07)
- CompositeObjective C — pinneapple_design.design_optimizer.objective (v06)
- CompositeRegularizer C — pinneapple_analysis.inverse_problems.regularization (v05)
- compressible_mass_flow f — pinneapple_systems.process_components.control_valve (v06)
- CompressibleConfig C — pinneapple_simulation.numerical_solvers.compressible_fv (v03)
- CompressibleFV C — pinneapple_simulation.numerical_solvers.compressible_fv (v03)
- compute_aleatoric_epistemic_uq f — pinneapple_neural.trainer.heteroscedastic (v04)
- compute_cp f — pinneapple_tools.aero_coefficients (v07)
- compute_curvature_proxy f — pinneapple_design.geometry.ops.features (v06)
- compute_derivatives f — pinneapple_neural.architectures.neural_operators.pino (v04)
- compute_dimensionless_numbers f — pinneapple_analysis.verification.dimensional_analysis (v05)
- compute_dissipation_rate f — pinneapple_tools.visualization.vortex (v07)
- compute_enstrophy f — pinneapple_tools.visualization.vortex (v07)
- compute_face_areas f — pinneapple_design.geometry.ops.features (v06)
- compute_face_normals f — pinneapple_design.geometry.ops.features (v06)
- compute_isosurface f — pinneapple_neural.predictor.postprocess (v04)
- compute_lambda2_3d f — pinneapple_tools.visualization.vortex (v07)
- compute_ndvi f — pinneapple_perception.change_detection (v04)

- `compute_pareto_front` f — `pinneapple_design.design_optimizer.pareto` (v06)
- `compute_physics_confidence` f — `pinneapple_analysis.verification.physics_confidence_score` (v05)
- `compute_q_criterion_2d` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_q_criterion_3d` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_sdf_grid` f — `pinneapple_data.geometry_features` (v04)
- `compute_sensitivity` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `compute_strain_rate_tensor_3d` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_streamlines` f — `pinneapple_neural.predictor.postprocess` (v04)
- `compute_stresses` f — `pinneapple_simulation.numerical_solvers.beam_bvp_fdm` (v03)
- `compute_vertex_normals` f — `pinneapple_design.geometry.ops.features` (v06)
- `compute_vorticity_2d` f — `pinneapple_tools.visualization.vortex` (v07)
- `compute_vorticity_3d` f — `pinneapple_tools.visualization.vortex` (v07)
- `ComputeBudget` C — `pinneapple_analysis.cost.equal_budget` (v05)
- `concat` f — `pinneapple_design.geometry_library.meshops` (v06)
- `concentration_at_membrane_depth` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `concentration_at_membrane_depth` f — `pinneapple_physics.closed_form.membrane_diffusion` (v02)
- `concentration_field` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `concentric_reducer` f — `pinneapple_design.geometry.gen.cadquery_advanced` (v06)
- `Concept` C — `pinneapple_reason.chain` (v05)
- `concept` f — `pinneapple_flow.nodes.reason` (v07)
- `conclude` f — `pinneapple_analysis.validation.gallery` (v05)
- `Condensation` C — `pinneapple_condense.algebraic` (v02)
- `condensation_order` f — `pinneapple_systems.cosimulation.utils` (v06)
- `Condensed` C — `pinneapple_condense.query` (v02)
- `Condensed` C — `pinneapple_condense.substructure` (v02)
- `CondensedState` C — `pinneapple_condense.substructure` (v02)
- `Condition` C — `pinneapple_physics.physics_audit.assumptions` (v02)
- `condition_coverage` f — `pinneapple_veriphysics.robustness` (v05)
- `condition_key` f — `pinneapple_query.core` (v02)
- `ConditionedSample` C — `pinneapple_systems.digital_twin.conditioning` (v06)
- `ConditionInfo` C — `pinneapple_app.backend.core.physics_adapter` (v07)
- `ConditionSpec` C — `pinneapple_data.upd_types` (v04)
- `ConditionSpec` C — `pinneapple_physics.pde_environment.conditions` (v02)
- `conduction_ke` f — `pinneapple_design.topology.problems` (v06)
- `confidence_index` f — `pinneapple_twin3d.wear.forecast` (v06)

- ConfidenceComponent C — pinneapple_analysis.verification.physics_confidence_score (v05)
- Config C — pinneapple_check.config (v05)
- Config C — pinneapple_mcp.config (v07)
- config_from_spec f — pinneapple_veriphysics.validation_lab (v05)
- ConfigError C — pinneapple_check.config (v05)
- ConfigMismatch C — pinneapple_analysis.benchmark.runner (v05)
- configurations f — pinneapple_condense.benchmark (v02)
- confirm f — pinneapple_flow.identify (v07)
- conformal_interval f — pinneapple_analysis.uncertainty.conformal_study (v05)
- conformal_quantile f — pinneapple_analysis.uncertainty.conformal (v05)
- conformal_quantile f — pinneapple_analysis.uncertainty.conformal_study (v05)
- conformal_quantile_valid f — pinneapple_analysis.uncertainty.conformal (v05)
- conformal_scores f — pinneapple_analysis.uncertainty.conformal_study (v05)
- conformal_study f — pinneapple_analysis.uncertainty.conformal_study (v05)
- ConformalPredictor C — pinneapple_analysis.uncertainty.conformal (v05)
- ConformalStudy C — pinneapple_analysis.uncertainty.conformal_study (v05)
- connect f — pinneapple_core.service.db (v01)
- Connection C — pinneapple_design.bricks.assembly (v06)
- Connection C — pinneapple_systems.cosimulation.connection (v06)
- ConnectionSpec C — pinneapple_design.geometry.gen.threaded_connection (v06)
- cons_to_prim f — pinneapple_physics.blackhole.hydro_numba (v02)
- ConsequentialActionClassifier C — pinneapple_problemdesign.autonomy (v02)
- conservation f — pinneapple_core.loss (v01)
- conservation_check f — pinneapple_physics.physics_audit.monitor (v02)
- conservation_check f — pinneapple_simulation.flight.verification (v03)
- ConservationCheck C — pinneapple_analysis.validation.conservation (v05)
- ConservationProjection C — pinneapple_physics.conservation (v02)
- ConsistencyReport C — pinneapple_physics.multiscale.consistency (v02)
- constant_properties f — pinneapple_physics.physics_audit.assumptions (v02)
- constant_provenance f — pinneapple_physics.pde_environment.presets.astropy_bridge (v02)
- constants_of f — pinneapple_design.design_optimizer.study (v06)
- ConstantWind C — pinneapple_simulation.flight.turbulence (v03)
- constellation_names f — pinneapple_physics.astro.stars (v02)
- constitutive_check f — pinneapple_physics.physics_audit.monitor (v02)

- `constrained_rod_buckling_load` f — `pinneapple_systems.process_components.pipe_stress_mechanics` (v06)
- `ConstrainedRodBucklingResult` C — `pinneapple_systems.process_components.pipe_stress_mechanics` (v06)
- `Constraint` C — `pinneapple_core.optim` (v01)
- `Constraint` C — `pinneapple_decision.constraints` (v05)
- `Constraint` C — `pinneapple_design.design_optimizer.study` (v06)
- `Constraint` C — `pinneapple_simulation.numerical_solvers.xtfc_general` (v03)
- `constraint_checks` f — `pinneapple_design.surrogate_inverse.core` (v06)
- `constraint_map` f — `pinneapple_simulation.electromagnetics.fem` (v03)
- `constraint_mask` f — `pinneapple_data.audit` (v04)
- `constraint_violation` f — `pinneapple_design.thermal.board_cooling.optimize` (v06)
- `constraint_violations` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `constraint_violations` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `constraint_violations` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `constraint_violations` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `constraint_violations` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `ConstraintBase` C — `pinneapple_design.design_optimizer.constraints` (v06)
- `constraints_from` f — `pinneapple_design.design_optimizer.spec` (v06)
- `ConstraintSet` C — `pinneapple_design.design_optimizer.constraints` (v06)
- `ConstraintsSpec` C — `pinneapple_problemdesign.schema` (v02)
- `ConstraintViolationError` C — `pinneapple_decision.constraints` (v05)
- `contact_entry_angle` f — `pinneapple_simulation.terramechanics.mobility` (v03)
- `contact_net` f — `pinneapple_simulation.terramechanics.surrogate` (v03)
- `contact_sheet` f — `pinneapple_design.geometry_library.render` (v06)
- `ContactResult` C — `pinneapple_simulation.terramechanics.wong_reece` (v03)
- `contacts` f — `pinneapple_tools.visualization.molecular` (v07)
- `ContactSurrogate` C — `pinneapple_simulation.terramechanics.surrogate` (v03)
- `Context` C — `pinneapple_physics.symbolic_pde.text` (v02)
- `context_for` f — `pinneapple_lab.next_steps` (v07)
- `continuity` f — `pinneapple_physics.maxwell_checks` (v02)
- `ContinuousAdjointSolver` C — `pinneapple_design.design_optimizer.adjoint` (v06)
- `ContinuousCatalog` C — `pinneapple_neural.architectures.continuous.registry` (v04)
- `ContinuousModelBase` C — `pinneapple_neural.architectures.continuous.base` (v04)

- `continuum` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `ContOutput` C — `pinneapple_neural.architectures.continuous.base` (v04)
- `ContOutput` C — `pinneapple_neural.architectures.continuous.deep_state_space` (v04)
- `Contract` C — `pinneapple_data.contracts.models` (v04)
- `contrast_ratio` f — `pinneapple_lab.seal` (v07)
- `control` f — `pinneapple_flow.nodes.methods` (v07)
- `control_probability` f — `pinneapple_systems.sports.football.pitch_control` (v06)
- `control_room` f — `pinneapple_systems.digital_twin.pipe_loop.report` (v06)
- `control_study` f — `pinneapple_systems.operations.contracts` (v06)
- `ControlledOpInf` C — `pinneapple_systems.operations.rom` (v06)
- `Controls` C — `pinneapple_simulation.flight.aircraft` (v03)
- `controls` f — `pinneapple_hub.zoo.demo` (v07)
- `controls_for` f — `pinneapple_security.compliance` (v05)
- `ControlScenario` C — `pinneapple_systems.operations.control` (v06)
- `ControlSpec` C — `pinneapple_systems.operations.control` (v06)
- `ControlSurface` C — `pinneapple_simulation.flight.aero` (v03)
- `Conv1DModel` C — `pinneapple_neural.architectures.convolutions.conv1d` (v04)
- `Conv2DModel` C — `pinneapple_neural.architectures.convolutions.conv2d` (v04)
- `Conv3DModel` C — `pinneapple_neural.architectures.convolutions.conv3d` (v04)
- `ConvAdapter` C — `pinneapple_neural.architectures.adapters.conv` (v04)
- `convection_coefficients` f — `pinneapple_physics.closed_form.pcb_thermal` (v02)
- `convection_diffusion` f — `pinneapple_simulation.numerical_solvers.gated_xtfc` (v03)
- `convection_exact` f — `pinneapple_physics.paper_problems` (v02)
- `ConvectionDiffusion1D` C — `pinneapple_arena.problems` (v04)
- `ConvectiveCooling` C — `pinneapple_design.qualitative.models` (v06)
- `convergence` f — `pinneapple_flow.nodes.methods` (v07)
- `convergence` f — `pinneapple_simulation.numerical_solvers.heat_fv_reference` (v03)
- `convergence` f — `pinneapple_simulation.numerical_solvers.operator_datasets` (v03)
- `convergence_order` f — `pinneapple_simulation.robot_sim.sim` (v03)
- `convergence_section` f — `pinneapple_app.backend.core.surrogate_report` (v07)
- `convergence_study` f — `pinneapple_systems.time_series.enso.intermediate` (v06)
- `ConvergenceResult` C — `pinneapple_analysis.verification.convergence` (v05)
- `ConversationRecord` C — `pinneapple_llm.conversation_store` (v07)
- `ConversationStore` C — `pinneapple_llm.conversation_store` (v07)
- `conversion_text` f — `pinneapple_data.unified` (v04)
- `convert` f — `pinneapple_data.contracts.tables` (v04)

- convert f — pinneapple_data.physical_units (v04)
- convert f — pinneapple_llm.qa.units (v07)
- convert_surface f — pinneapple_simulation.external_solvers.cfd_formats.convert (v03)
- convex_hull f — pinneapple_design.bricks.statics (v06)
- ConvModelBase C — pinneapple_neural.architectures.convolutions.base (v04)
- ConvolutionCatalog C — pinneapple_neural.architectures.convolutions.registry (v04)
- ConvOutput C — pinneapple_neural.architectures.convolutions.base (v04)
- CoolingDesign C — pinneapple_design.thermal.board_cooling.mission (v06)
- CoolingMission C — pinneapple_design.thermal.board_cooling.mission (v06)
- CoolingSurrogate C — pinneapple_design.thermal.board_cooling.learn (v06)
- Coordinate C — pinneapple_core.transforms (v01)
- CoordMapping C — pinneapple_physics.pinn_solver.io.mappings (v02)
- coords_of f — pinneapple_flow.nodes._common (v07)
- CopernicusClient C — pinneapple_data.datasets.external_sources (v04)
- copper_resistance f — pinneapple_simulation.electromagnetics.machines.drive (v03)
- copy_model f — pinneapple_hub.zoo._models (v07)
- corner_scores f — pinneapple_lab.seal (v07)
- corner_stackup f — pinneapple_analysis.uncertainty.margin (v05)
- CornerModule C — pinneapple_simulation.vehicle.spec (v03)
- correct f — pinneapple_condense.exceptions (v02)
- correction_study f — pinneapple_condense.benchmark (v02)
- CorrelatedLatentField C — pinneapple_physics.pinn_solver.stochastic (v02)
- corrupt f — pinneapple_worldmodel.physical_world (v04)
- cosim_from_models f — pinneapple_systems (v06)
- CoSimEngine C — pinneapple_systems.cosimulation.engine (v06)
- CoSimGraph C — pinneapple_systems.cosimulation.graph (v06)
- CoSimLoss C — pinneapple_systems.cosimulation.losses (v06)
- CoSimNode C — pinneapple_systems.cosimulation.node (v06)
- CoSimTrainer C — pinneapple_systems.cosimulation.trainer (v06)
- cosine_lr f — pinneapple_analysis.cost.equal_budget (v05)
- CosmosAdapter C — pinneapple_worldmodel.physical_world (v04)
- CosmosEncoderDataset C — pinneapple_worldmodel.training_hooks (v04)
- CosmosProvider C — pinneapple_problemdesign.providers.cosmos_provider (v02)
- CosmosProviderConfig C — pinneapple_problemdesign.providers.cosmos_provider (v02)
- cosq_feed f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- cost_text f — pinneapple_reason.chain (v05)
- CostLedger C — pinneapple_analysis.cost.profiler (v05)
- CostModel C — pinneapple_design.design_optimizer.presets (v06)
- CostRecord C — pinneapple_analysis.cost.profiler (v05)
- cotan_laplacian f — pinneapple_design.geometry.ops.curvature (v06)

- coulomb_potential f — pinneapple_quantum.loss.schrodinger (v04)
- CoulombSuperposition C — pinneapple_science.em.statics (v02)
- count_evaluations f — pinneapple_analysis.inverse_problems.forward (v05)
- count_flops f — pinneapple_analysis.cost.equal_budget (v05)
- count_gpus f — pinneapple_neural.trainer.parallel (v04)
- count_parameters f — pinneapple_flow.learn (v07)
- count_parameters f — pinneapple_tools.benchmark_suite.crossframework.weights (v07)
- count_rows f — pinneapple_registry.gateway_api (v01)
- count_trainable f — pinneapple_adaptation.transfer_learning.freeze (v04)
- counterexample_search f — pinneapple_analysis.proof.graph (v05)
- Countermeasure C — pinneapple_reason.library (v05)
- countermeasure f — pinneapple_flow.nodes.reason (v07)
- counts f — pinneapple_flow.coverage (v07)
- CoupledModel C — pinneapple_systems.time_series.enso.intermediate (v06)
- CoupledParams C — pinneapple_systems.time_series.enso.intermediate (v06)
- coupling_classes f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- CouplingLoss C — pinneapple_systems.cosimulation.losses (v06)
- courant_limit f — pinneapple_simulation.electromagnetics.fdtd (v03)
- courant_number f — pinneapple_data.cae.metrics (v04)
- coverage f — pinneapple_data.doe.coverage (v04)
- coverage f — pinneapple_flow.coverage (v07)
- coverage f — pinneapple_lab.explorer (v07)
- coverage f — pinneapple_lab.uq (v07)
- coverage f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- coverage_by_group f — pinneapple_analysis.uncertainty.conformal_study (v05)
- coverage_check f — pinneapple_lab.uq (v07)
- coverage_md f — pinneapple_lab.explorer (v07)
- coverage_study f — pinneapple_analysis.uncertainty.residual_bound (v05)
- CoverageReport C — pinneapple_analysis.uncertainty.residual_bound (v05)
- cp_max_newtonian f — pinneapple_physics.closed_form.compressible (v02)
- CPTECClient C — pinneapple_data.datasets.external_sources (v04)
- cpu_heatsink_thermal f — pinneapple_physics.pde_environment.presets.engineering (v02)
- cpu_model f — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- cr3bp_collinear_lagrange_point f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- cr3bp_lagrange_points f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- cr3bp_omega_gradient f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- cr3bp_planar_synodic f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- crate_and_pendulum f — pinneapple_flow.nodes_ext.solvers (v07)

- Crater C — pinneapple_simulation.rover.terrain (v03)
- crater_site f — pinneapple_simulation.vehicle.scenarios (v03)
- crease_normals f — pinneapple_design.geometry_library.meshops (v06)
- create_app f — pinneapple_design.fitmesh.api (v06)
- create_app f — pinneapple_llm.workbench.server (v07)
- creeping_flow f — pinneapple_physics.physics_audit.assumptions (v02)
- crf f — pinneapple_physics.closed_form.pipe_insulation (v02)
- Criterion C — pinneapple_tools.automation.workflow (v07)
- critical_angle f — pinneapple_simulation.electromagnetics.waves (v03)
- CritiqueFinding C — pinneapple_analysis.verification.architecture_critique (v05)
- cross f — pinneapple_simulation.multibody.tree (v03)
- cross_check f — pinneapple_design.bricks.dynamics (v06)
- cross_correlation f — pinneapple_systems.time_series.eda.plots (v06)
- cross_scale_consistency f — pinneapple_physics.multiscale.consistency (v02)
- cross_section_polygons f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- cross_validate f — pinneapple_autolab.surrogates (v07)
- cross_validate f — pinneapple_physics.astro.stars_ml (v02)
- crossing_rate f — pinneapple_analysis.uncertainty.quantile (v05)
- crowd_load f — pinneapple_reason.modes.structural (v05)
- crowding_distance f — pinneapple_design.design_optimizer.pareto (v06)
- crps_ensemble f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- crps_gaussian f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- crystal_phonon f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- CSGCircle C — pinneapple_design.geometry.csg (v06)
- CSGDifference C — pinneapple_design.geometry.csg (v06)
- CSGEllipse C — pinneapple_design.geometry.csg (v06)
- CSGIntersection C — pinneapple_design.geometry.csg (v06)
- CSGPolygon C — pinneapple_design.geometry.csg (v06)
- CSGRectangle C — pinneapple_design.geometry.csg (v06)
- CSGUnion C — pinneapple_design.geometry.csg (v06)
- cst_surface f — pinneapple_design.aero.geometry (v06)
- Ctx C — pinneapple_check.checks.common (v05)
- ctx_metrics f — pinneapple_flow.nodes.vehicle (v07)
- CUDAGraphModule C — pinneapple_neural.trainer.hpc (v04)
- CUDAPrefetcher C — pinneapple_neural.trainer.parallel (v04)
- curate f — pinneapple_lab.curation (v07)
- curate_dataset f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- curate_sequence f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- CurationRecord C — pinneapple_tools.dataset_quality.vlm_curation (v07)
- curl f — pinneapple_core.operators (v01)
- current f — pinneapple_hub.library.flowruns (v07)

- `current_period` f — `pinneapple_core.service.metering` (v01)
- `CurriculumConfig` C — `pinneapple_worldmodel.curriculum` (v04)
- `CurriculumStage` C — `pinneapple_worldmodel.curriculum` (v04)
- `curve_fit` f — `pinneapple_science.instruments` (v02)
- `curved_pipe_ratio` f — `pinneapple_physics.closed_form.pipe_flow` (v02)
- `CurvedPathProfile` C — `pinneapple_systems.process_components.curved_path_geometry` (v06)
- `CurveFit` C — `pinneapple_science.instruments` (v02)
- `CurveFitSynthGenerator` C — `pinneapple_data.synth.curvefit` (v04)
- `curves_figure` f — `pinneapple_tools.nvidia.figures` (v07)
- `custom` f — `pinneapple_analysis.proof.graph` (v05)
- `custom_spec` f — `pinneapple_flow.libcalls` (v07)
- `CustomPhysics` C — `pinneapple_app.backend.core.physics_adapter` (v07)
- `CustomProblemRequest` C — `pinneapple_app.backend.routers.problems` (v07)
- `cusum` f — `pinneapple_security.cps` (v05)
- `CusumAlarm` C — `pinneapple_security.cps` (v05)
- `cut_metrics_at` f — `pinneapple_simulation.electromagnetics.reflector.study_report` (v03)
- `CutawayMesh` C — `pinneapple_physics.blackhole.twin` (v02)
- `cw_closest_approach` f — `pinneapple_physics.astro.conjunction` (v02)
- `cw_exact_velocity` f — `pinneapple_physics.pde_environment.presets.astrophysics` (v02)
- `cw_mean_motion` f — `pinneapple_physics.astro.conjunction` (v02)
- `cycle_energy` f — `pinneapple_simulation.electromagnetics.machines.drive` (v03)
- `cycles_renders` f — `pinneapple_autolab.domains.racecar` (v07)
- `CyclesScene` C — `pinneapple_blender.cycles` (v07)
- `cylinder` f — `pinneapple_design.geometry.bodies` (v06)
- `cylinder` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `cylinder` f — `pinneapple_design.geometry_library.kit` (v06)
- `cylinder` f — `pinneapple_simulation.flight.mass` (v03)
- `cylinder` f — `pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job` (v03)
- `cylinder` f — `pinneapple_twin3d.primitives` (v06)
- `cylinder3d` f — `pinneapple_design.geometry.gen.sdf_shapes` (v06)
- `cylinder_conductance` f — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `cylinder_flow_to_tensors` f — `pinneapple_data.datasets.deepmind_mgn` (v04)
- `cylinder_inertia` f — `pinneapple_simulation.robot_sim.spec` (v03)
- `cylinder_mask` f — `pinneapple_simulation.numerical_solvers.lbm` (v03)
- `cylinder_mesh` f — `pinneapple_design.bricks.mesh` (v06)
- `cylinder_scattering_tm` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `cylinder_tm_coefficients` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `cylinder_tm_total` f — `pinneapple_physics.closed_form.electromagnetics` (v02)

- cylinder_wake f — pinneapple_lab.experiments.rom (v07)
- CylinderLBM C — pinneapple_lab.experiments.cylinder (v07)
- cylindrical_cavity_resonance f — pinneapple_simulation.electromagnetics.waves (v03)

D

- d f — pinneapple_analysis.validation.pinn (v05)
- d20_climatology f — pinneapple_systems.time_series.enso.data (v06)
- d20_from_profile f — pinneapple_systems.time_series.enso.data (v06)
- d20_monthly f — pinneapple_systems.time_series.enso.data (v06)
- d_optimal_design f — pinneapple_analysis.inverse_problems.identifiability (v05)
- dam_break f — pinneapple_flow.nodes_ext.solvers (v07)
- damkohler f — pinneapple_physics.physics_audit.regime (v02)
- Darcy C — pinneapple_analysis.benchmark.builtin.standard (v05)
- darcy_2d f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- darcy_checks f — pinneapple_check._reuse (v05)
- darcy_checks f — pinneapple_physics.darcy_checks (v02)
- darcy_consistency f — pinneapple_data.audit (v04)
- darcy_inputs f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- darcy_matrix f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- darcy_operator_apply f — pinneapple_physics.darcy_checks (v02)
- darcy_pressure_drop f — pinneapple_physics.closed_form.pipe_flow (v02)
- darcy_pressure_only_3d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- darcy_problem f — pinneapple_analysis.uncertainty.residual_bound (v05)
- darcy_residual f — pinneapple_physics.darcy_checks (v02)
- darcy_set f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- darcy_solver_service f — pinneapple_tools.model_services.grid (v07)
- darcy_trust f — pinneapple_tools.model_services.grid (v07)
- darcy_trust_report f — pinneapple_physics.darcy_checks (v02)
- DarcyCheck C — pinneapple_physics.darcy_checks (v02)
- DarcyData C — pinneapple_tools.benchmark_suite.crossframework.darcy (v07)
- DarcyFlow2D C — pinneapple_arena.problems (v04)
- DarcyProblem C — pinneapple_analysis.validation.fault_injection.problems (v05)
- dark_sky_nl f — pinneapple_physics.astro.observing (v02)
- Dashboard C — pinneapple_tools.visualization.dashboard.spec (v07)
- dashboard f — pinneapple_flow.nodes.vehicle (v07)
- Data C — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)

- data_dir f — pinneapple_core.service (v01)
- data_efficiency f — pinneapple_systems.sports.football.pc_study (v06)
- data_efficiency f — pinneapple_systems.sports.football.xg (v06)
- data_faults f — pinneapple_analysis.validation.fault_injection.faults (v05)
- data_from_spec f — pinneapple_analysis.surrogate.contracts (v05)
- data_root f — pinneapple_flow.config (v07)
- data_specs f — pinneapple_systems.time_series.forecast_lab.presets (v06)
- DataBundle C — pinneapple_app.backend.core.data_pipeline (v07)
- datacenter_airflow_2d f — pinneapple_physics.pde_environment.presets.engineering (v02)
- datacenter_cfd_3d f — pinneapple_physics.pde_environment.presets.engineering (v02)
- datacenter_server_thermal f — pinneapple_physics.pde_environment.presets.engineering (v02)
- DataConfig C — pinneapple_app.backend.core.data_pipeline (v07)
- DataConsistencyGuidance C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- DataConstraint f — pinneapple_physics.pde_environment.conditions (v02)
- DataLabel C — pinneapple_security.classification (v05)
- DataLoader C — pinneapple_core.data (v01)
- DataLoaderSpec C — pinneapple_data.dataloaders (v04)
- DataLoss C — pinneapple_systems.cosimulation.losses (v06)
- DataMisfitBase C — pinneapple_analysis.inverse_problems.noise_models (v05)
- DataModule C — pinneapple_neural.trainer.datamodule (v04)
- DataRequest C — pinneapple_app.backend.routers.experiments (v07)
- Dataset C — pinneapple_analysis.validation.fault_injection.problems (v05)
- dataset f — pinneapple_hub.library.snippets (v07)
- dataset f — pinneapple_physics.astro.stars_ml (v02)
- dataset f — pinneapple_worldmodel.physical_world (v04)
- dataset_audit f — pinneapple_check._reuse (v05)
- dataset_card f — pinneapple_hub.zoo.hub_cards (v07)
- dataset_checks f — pinneapple_analysis.validation.fault_injection.checks (v05)
- dataset_from_run f — pinneapple_core.workflows.datasets (v01)
- dataset_ids f — pinneapple_data.datasets (v04)
- dataset_info f — pinneapple_data.datasets (v04)
- dataset_of f — pinneapple_flow.nodes.data (v07)
- dataset_spec f — pinneapple_analysis.surrogate.contracts (v05)
- DatasetBuilder C — pinneapple_worldmodel.dataset (v04)
- DatasetCatalog C — pinneapple_worldmodel.dataset_factory (v04)
- DatasetConfig C — pinneapple_arena.config (v04)
- DatasetConfig C — pinneapple_worldmodel.dataset (v04)
- DatasetEntry C — pinneapple_worldmodel.dataset_factory (v04)
- DatasetGenerator C — pinneapple_arena.pointwise.datasets (v04)
- DatasetInfo C — pinneapple_data.datasets.registry (v04)
- DatasetInfo C — pinneapple_hub.zoo.datasets (v07)
- DatasetManifest C — pinneapple_worldmodel.dataset_packager (v04)

- DatasetPackager C — pinneapple_worldmodel.dataset_packager (v04)
- DatasetPhysics C — pinneapple_data.audit (v04)
- DatasetProblem C — pinneapple_arena.dataset_problems (v04)
- DatasetRef C — pinneapple_data.contracts.models (v04)
- DatasetRegistry C — pinneapple_data.datasets.registry (v04)
- datasets f — pinneapple_flow.nodes_ext.arena_pipelines (v07)
- datasets f — pinneapple_hub.library.sources (v07)
- DatasetSpec C — pinneapple_arena.external.schema (v04)
- DatasetSpec C — pinneapple_data.contracts.models (v04)
- DatasetSplit C — pinneapple_hub.zoo.datasets (v07)
- DatasetStore C — pinneapple_registry.dataset_store (v01)
- DatasetTarget C — pinneapple_check.config (v05)
- DatasetWriter C — pinneapple_lab.dataset (v07)
- Datasheet C — pinneapple_tools.reporting.datasheet (v07)
- DataSpec C — pinneapple_problemdesign.schema (v02)
- datetime_from_jd f — pinneapple_physics.astro.timescales (v02)
- DDPPINNTrainer C — pinneapple_neural.trainer.distributed (v04)
- DDPTrainerConfig C — pinneapple_neural.trainer.distributed (v04)
- dean_number f — pinneapple_physics.closed_form.pipe_flow (v02)
- Debye C — pinneapple_simulation.electromagnetics.media (v03)
- decay_rate_constant_per_day f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- DecayingTurbulence2D C — pinneapple_physics.multiscale.turbulence (v02)
- decide f — pinneapple_decision.api (v05)
- decimals f — pinneapple_tools.visualization.studio.colormap (v07)
- decimate_mesh f — pinneapple_design.geometry.ops.clean (v06)
- decimate_surface f — pinneapple_tools.visualization.studio.scene (v07)
- Decision C — pinneapple_data.formfill.extract (v04)
- Decision C — pinneapple_decision.schema (v05)
- Decision C — pinneapple_flow.decisions (v07)
- Decision C — pinneapple_security.classification (v05)
- decision_engine f — pinneapple_decision.api (v05)
- DecisionConfig C — pinneapple_arena.config (v04)
- DecisionLevel C — pinneapple_decision.schema (v05)
- DecisionLoop C — pinneapple_decision.loop (v05)
- DecisionRecord C — pinneapple_veriphysics.decision (v05)
- DecisionState C — pinneapple_decision.schema (v05)
- DecisionTree C — pinneapple_decision.tree (v05)
- Deck C — pinneapple_data.cae.inp (v04)
- deck f — pinneapple_flow.nodes_ext.structures (v07)
- deck_conditions f — pinneapple_data.cae.inp (v04)
- deck_mesh f — pinneapple_data.cae.inp (v04)
- decode f — pinneapple_design.intent.pipeline (v06)
- decode f — pinneapple_physics.blackhole.dataset (v02)
- decode_image_frames f — pinneapple_perception.piv_tools (v04)
- decode_video f — pinneapple_perception.piv_tools (v04)
- DecoderLayer C — pinneapple_neural.architectures.transformers.autoformer (v04)
- decompose f — pinneapple_flow.nodes_ext.solver_registry (v07)
- decompose f — pinneapple_systems.time_series.solar_cycle (v06)
- decompose_dict f — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)

- decompose_uncertainty f — pinneapple_analysis.uncertainty.decomposition (v05)
- Decomposition C — pinneapple_systems.time_series.solar_cycle (v06)
- decrypt_array f — pinneapple_security.encryption (v05)
- decrypt_bytes f — pinneapple_security.encryption (v05)
- decrypt_file f — pinneapple_security.encryption (v05)
- DecryptionError C — pinneapple_security.encryption (v05)
- DeepEnsemble C — pinneapple_systems.component_modeling.ensemble (v06)
- DeepONet C — pinneapple_neural.architectures.neural_operators.deeponet (v04)
- DeepONetMethod C — pinneapple_analysis.benchmark.builtin.fields (v05)
- DeepStateSpaceModel C — pinneapple_neural.architectures.continuous.deep_state_space (v04)
- DeepUQROM C — pinneapple_neural.architectures.rom.deep_uq_rom (v04)
- DeepXDEBackend C — pinneapple_tools.benchmark_suite.backends.deepxde_backend (v07)
- DeepXDESolveResult C — pinneapple_physics.deepxde_bridge.adapter (v02)
- default_cache f — pinneapple_design.bricks.ldraw (v06)
- default_checks f — pinneapple_analysis.validation.fault_injection.checks (v05)
- default_config f — pinneapple_flow.learn (v07)
- default_decider f — pinneapple_arena.decision_mode (v04)
- default_ephemeris f — pinneapple_physics.astro.ephemeris (v02)
- default_eps f — pinneapple_simulation.meshless.methods (v03)
- default_experts f — pinneapple_systems.time_series.adaptive (v06)
- default_geometry f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- default_geometry_note f — pinneapple_core.onboarding.catalog (v01)
- default_geometry_report f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- default_graph f — pinneapple_physics.physics_audit.knowledge (v02)
- default_guesses f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- default_metrics f — pinneapple_neural.trainer.metrics (v04)
- default_mission f — pinneapple_design.thermal.board_cooling.mission (v06)
- default_model f — pinneapple_systems.operations.live (v06)
- default_models f — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- default_range f — pinneapple_tools.visualization.studio.compose (v07)
- default_ranks f — pinneapple_neural.architectures.rom.study (v04)
- default_reference_meshes f — pinneapple_analysis.verification.geometry_ood_guardrail (v05)

- `default_registry` f — `pinneapple_tools.model_services.registry` (v07)
- `default_root` f — `pinneapple_lab.store` (v07)
- `default_schedule` f — `pinneapple_simulation.flight.takeoff` (v03)
- `default_solver_specs` f — `pinneapple_simulation.accelerated_foam.benchmark` (v03)
- `default_ts_metrics` f — `pinneapple_systems.time_series.metrics` (v06)
- `default_url` f — `pinneapple_flow.store` (v07)
- `default_variant` f — `pinneapple_condense.learned` (v02)
- `default_weigher` f — `pinneapple_data.cache_bytes` (v04)
- `default_workdir` f — `pinneapple_flow.runtime` (v07)
- `defective_models` f — `pinneapple_analysis.validation.physics_learning_cases` (v05)
- `Defence` C — `pinneapple_analysis.validation.arena.blue` (v05)
- `define_custom` f — `pinneapple_app.backend.core.problem` (v07)
- `define_problem` f — `pinneapple_arena.custom_builder` (v04)
- `define_problem` f — `pinneapple_physics` (v02)
- `Definition` C — `pinneapple_systems.process_components.explicit_equation_system` (v06)
- `deflection_angle_rad` f — `pinneapple_physics.astro.lensing` (v02)
- `DeformableTerrain` C — `pinneapple_simulation.terramechanics.terrain` (v03)
- `delay_embedding` f — `pinneapple_systems.time_series.predictability` (v06)
- `DelayODEForecaster` C — `pinneapple_systems.time_series.forecast_lab.physical` (v06)
- `deliver` f — `pinneapple_design.thermal.board_cooling.report` (v06)
- `deliver` f — `pinneapple_systems.digital_twin.pipe_loop.report` (v06)
- `delta_t_model` f — `pinneapple_physics.astro.timescales` (v02)
- `DEM` C — `pinneapple_simulation.numerical_solvers.particles` (v03)
- `demagnetisation_margin` f — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `demagnetisation_refinement` f — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `demand_capacity_figure` f — `pinneapple_reason.report` (v05)
- `demo_record` f — `pinneapple_systems.operations.live` (v06)
- `demo_scenario` f — `pinneapple_systems.operations.plant` (v06)
- `demo_site` f — `pinneapple_simulation.rover.autolab.site` (v03)
- `dense_forecast` f — `pinneapple_systems.time_series.forecast_lab.renders` (v06)
- `DenseAutoencoder` C — `pinneapple_neural.architectures.autoencoders.dense_ae` (v04)
- `DenseVolumeOperatorConfig` C — `pinneapple_neural.workflows.dense_volume_operator` (v04)
- `densify` f — `pinneapple_simulation.rover.autolab.planner` (v03)
- `DensityCodec` C — `pinneapple_physics.blackhole.forecast` (v02)
- `DensityFilter` C — `pinneapple_design.topology.filters` (v06)
- `DensityTable` C — `pinneapple_physics.astro.decay` (v02)
- `depletion_time_days` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `depletion_time_days` f — `pinneapple_physics.closed_form.membrane_diffusion` (v02)

- DeploymentSpec C — pinneapple_problemdesign.schema (v02)
- derivative_cache f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- derivative_key f — pinneapple_physics.symbolic_pde.text (v02)
- derivative_keys f — pinneapple_physics.symbolic_pde.text (v02)
- DerivativeComputer C — pinneapple_physics.pinn_solver.factory.autodiff (v02)
- DerivativeKey C — pinneapple_physics.pinn_solver.factory.autodiff (v02)
- derive_key f — pinneapple_security.encryption (v05)
- derive_psi_shape_a f — pinneapple_systems.process_components.similarity_map (v06)
- DerivedSpec C — pinneapple_pdb.derived (v04)
- descend f — pinneapple_design.surrogate_inverse.core (v06)
- describe f — pinneapple_core.onboarding.catalog (v01)
- describe f — pinneapple_data.cae.dataset (v04)
- describe f — pinneapple_design.qualitative.descriptors (v06)
- describe f — pinneapple_flow.planner.extensions (v07)
- describe f — pinneapple_flow.planner.wizard (v07)
- describe f — pinneapple_systems.digital_twin.product.runner (v06)
- describe f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- describe_library f — pinneapple_reason (v05)
- describe_method f — pinneapple_mcp.tools (v07)
- describe_unit f — pinneapple_data.physical_units (v04)
- Descriptors C — pinneapple_design.qualitative.descriptors (v06)
- Design C — pinneapple_data.doe.designs (v04)
- Design C — pinneapple_neural.lpm (v04)
- design f — pinneapple_data.doe.designs (v04)
- design f — pinneapple_design.intent.api (v06)
- design f — pinneapple_mcp.tools (v07)
- design_by_name f — pinneapple_design.thermal.fin_array_speed (v06)
- design_features f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- design_file f — pinneapple_design.design_optimizer.presets (v06)
- design_intent f — pinneapple_flow.wrappers (v07)
- design_loop f — pinneapple_design.thermal.board_cooling.optimize (v06)
- design_of f — pinneapple_design.thermal.fin_array_speed (v06)
- design_points f — pinneapple_physics.physics_audit.improvement (v02)
- design_table f — pinneapple_tools.nvidia.heatsink (v07)
- design_to_dict f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- design_to_dict f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- design_to_dict f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- design_to_dict f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- design_to_dict f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- DesignAgent C — pinneapple_problemdesign.agent (v02)

- DesignChain C — pinneapple_design.topology.driver (v06)
- DesignInput C — pinneapple_design.thermal.heatsink.engine (v06)
- DesignOptConfig C — pinneapple_design.design_optimizer.pipeline (v06)
- DesignOptimizerConfig C — pinneapple_design.design_optimizer.optimizer (v06)
- DesignOptLoop C — pinneapple_design.design_optimizer.pipeline (v06)
- DesignOptResult C — pinneapple_design.design_optimizer.pipeline (v06)
- DesignPackage C — pinneapple_design.intent.package (v06)
- DesignProblem C — pinneapple_design.design_optimizer.study (v06)
- DesignReport C — pinneapple_problemdesign.schema (v02)
- designs_frame f — pinneapple_design.design_optimizer.report (v06)
- DesignSafeClient C — pinneapple_data.datasets.external_sources (v04)
- DesignSpace C — pinneapple_design.design_optimizer.study (v06)
- DesignSpace C — pinneapple_design.thermal.board_cooling.mission (v06)
- DesignState C — pinneapple_problemdesign.state (v02)
- DesignVariable C — pinneapple_design.intent.template (v06)
- detect f — pinneapple_flow.planner.heuristic (v07)
- detect f — pinneapple_systems.operations.anomaly (v06)
- detect_divergence f — pinneapple_systems.digital_twin.recession (v06)
- detect_format f — pinneapple_simulation.external_solvers.cfd_formats.convert (v03)
- detect_licenses f — pinneapple_tools.rea_bridge.licenses (v07)
- detect_machine f — pinneapple_llm.workbench.models (v07)
- detect_outliers f — pinneapple_tools.dataset_quality.checks (v07)
- detect_project f — pinneapple_data.lineage.detect (v04)
- detect_time_column f — pinneapple_data.data_health (v04)
- detection_study f — pinneapple_analysis.validation.physics_learning (v05)
- DetectionMatrix C — pinneapple_analysis.validation.fault_injection.matrix (v05)
- DetectionResult C — pinneapple_systems.operations.anomaly (v06)
- DetectorConfig C — pinneapple_systems.operations.anomaly (v06)
- determinism_check f — pinneapple_analysis.benchmark.checks (v05)
- deterministic_npz f — pinneapple_physics.blackhole.dataset (v02)
- device_info f — pinneapple_arena.pointwise.training (v04)
- dew_point f — pinneapple_reason.props (v05)
- DFSPHSolver C — pinneapple_simulation.numerical_solvers.dfsph (v03)
- DFTMonitor C — pinneapple_simulation.electromagnetics.fdtd (v03)
- diagnose f — pinneapple_analysis.benchmark.advisor (v05)
- diagnose f — pinneapple_llm.qa.repair (v07)
- diagnose f — pinneapple_physics.physics_audit (v02)

- `Diagnoser` C — `pinneapple_systems.digital_twin.pipe_loop.diagnose` (v06)
- `Diagnosis` C — `pinneapple_analysis.benchmark.advisor` (v05)
- `Diagnosis` C — `pinneapple_llm.qa.repair` (v07)
- `Diagnosis` C — `pinneapple_physics.physics_audit` (v02)
- `DiagnosticReport` C — `pinneapple_neural.trainer.advisor` (v04)
- `dic` f — `pinneapple_simulation.accelerated_foam.solvers` (v03)
- `dict_collate` f — `pinneapple_physics.pinn_solver.compiler.dataset` (v02)
- `did_you_know` f — `pinneapple_physics.astro.facts` (v02)
- `diebold_mariano` f — `pinneapple_systems.time_series.metrics_ext.comparison` (v06)
- `diff` f — `pinneapple_analysis.validation.report_artifact` (v05)
- `Difference` C — `pinneapple_worldmodel.geometry` (v04)
- `DifferencingStep` C — `pinneapple_neural.trainer.preprocess` (v04)
- `differential_evolution` f — `pinneapple_analysis.validation.arena.search` (v05)
- `DifficultyLevel` C — `pinneapple_arena.pointwise.schema` (v04)
- `diffusion_dominated` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `digest_array` f — `pinneapple_security.integrity` (v05)
- `digest_record` f — `pinneapple_security.integrity` (v05)
- `DigitalTwin` C — `pinneapple_systems.digital_twin.twin` (v06)
- `DigitalTwinConfig` C — `pinneapple_systems.digital_twin.twin` (v06)
- `dim_str` f — `pinneapple_data.physical_units` (v04)
- `Dimension` C — `pinneapple_arena.pointwise.schema` (v04)
- `Dimension` C — `pinneapple_lab.curation` (v07)
- `dimension` f — `pinneapple_simulation.electromagnetics.units` (v03)
- `dimension_of` f — `pinneapple_llm.qa.units` (v07)
- `dimension_str` f — `pinneapple_llm.qa.units` (v07)
- `DimensionlessNumbers` C — `pinneapple_analysis.verification.dimensional_analysis` (v05)
- `dims_unit` f — `pinneapple_data.cae.foam` (v04)
- `dipole_cartesian` f — `pinneapple_science.em.waves` (v02)
- `dipole_pattern` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `DipoleRadiation` C — `pinneapple_science.em.waves` (v02)
- `diprotic_fractions` f — `pinneapple_systems.process_components.reaction_kinetics` (v06)
- `directivity_vs_length` f — `pinneapple_simulation.electromagnetics.antenna.plots` (v03)
- `dirichlet_boundary_values` f — `pinneapple_simulation.numerical_solvers.nonlinear_beam_fem` (v03)
- `dirichlet_conditions` f — `pinneapple_simulation.numerical_solvers._bc` (v03)
- `DirichletBC` f — `pinneapple_physics.pde_environment.conditions` (v02)
- `DirichletBC` C — `pinneapple_physics.symbolic_pde.bc` (v02)
- `disc_capacitor` f — `pinneapple_science.em.statics` (v02)
- `disc_image` f — `pinneapple_physics.astro.lensing` (v02)
- `discover` f — `pinneapple_check.config` (v05)
- `discover` f — `pinneapple_flow.nodes.solvers` (v07)

- discover f — pinneapple_tools.hpo_experiments.pipelines.discover (v07)
- discover_examples f — pinneapple_lab.experiments.examples (v07)
- discover_pde f — pinneapple_flow.identify (v07)
- DiscoveryResult C — pinneapple_tools.hpo_experiments.models (v07)
- DiscrepancyModel C — pinneapple_analysis.verification.causal_discrepancy (v05)
- discrete_gust f — pinneapple_simulation.flight.encounter (v03)
- discrete_time_rk_residual f — pinneapple_simulation.numerical_solvers.discrete_time_pinn (v03)
- DiscreteGust C — pinneapple_simulation.flight.turbulence (v03)
- DiscreteTimePINNSolver C — pinneapple_simulation.numerical_solvers.discrete_time_pinn (v03)
- DiscreteTimeResult C — pinneapple_simulation.numerical_solvers.discrete_time_pinn (v03)
- DiscreteTimeRKNet C — pinneapple_simulation.numerical_solvers.discrete_time_pinn (v03)
- Discretization C — pinneapple_simulation.meshless.methods (v03)
- discretize f — pinneapple_simulation.meshless.methods (v03)
- disk_grid f — pinneapple_simulation.electromagnetics.reflector.qz (v03)
- disk_on_axis f — pinneapple_simulation.electromagnetics.reflector.analytic (v03)
- displacement_errors f — pinneapple_systems.sports.football.kinematics (v06)
- displacement_factor f — pinneapple_tools.visualization.studio.fieldview (v07)
- display_units f — pinneapple_physics.weather.viz (v02)
- distill_to_symbolic f — pinneapple_neural.trainer.graybox (v04)
- DistributionSynthGenerator C — pinneapple_data.synth.distributions (v04)
- div f — pinneapple_core.operators (v01)
- divergence f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- divergence_time f — pinneapple_analysis.validation.rollout (v05)
- DMDBaseline C — pinneapple_physics.blackhole.skill (v02)
- DMDc C — pinneapple_neural.architectures.rom.dmd (v04)
- DMResult C — pinneapple_systems.time_series.metrics_ext.comparison (v06)
- do_rhs f — pinneapple_systems.time_series.enso.oscillators (v06)
- do_roots f — pinneapple_systems.time_series.enso.oscillators (v06)
- doc_table_value f — pinneapple_flow.nodes_ext.examples (v07)
- docker_available f — pinneapple_simulation.external_solvers.calculix.runner (v03)
- docker_available f — pinneapple_simulation.external_solvers.openradioss.runner (v03)

- DockerExecutor C — pinneapple_core.workflows.executors (v01)
- doctor f — pinneapple_core.onboarding.doctor (v01)
- doctor_main f — pinneapple_core.onboarding.cli (v01)
- DoctorReport C — pinneapple_core.onboarding.doctor (v01)
- Document C — pinneapple_analysis.retrieval.corpus (v05)
- Document C — pinneapple_data.formfill.extract (v04)
- document_text f — pinneapple_data.formfill.llm (v04)
- doe f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- doe f — pinneapple_simulation.pipelines.freecad_openfoam.sweep (v03)
- doe_design f — pinneapple_flow.wrappers (v07)
- dof_to_upd f — pinneapple_simulation.external_solvers.fenics.solver (v03)
- Domain C — pinneapple_core.domain (v01)
- domain f — pinneapple_flow.nodes.problem (v07)
- domain f — pinneapple_hub.library.sources (v07)
- domain f — pinneapple_simulation.meshless.domains (v03)
- Domain2D C — pinneapple_simulation.meshless.domains (v03)
- domain_of f — pinneapple_lab.manifest (v07)
- DomainMeta C — pinneapple_arena.external.schema (v04)
- domains f — pinneapple_core.onboarding.catalog (v01)
- DomainVerdict C — pinneapple_autolab.domain (v07)
- dominant_frequency f — pinneapple_physics.physics_audit.regime (v02)
- dominant_period f — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- DoMINO C — pinneapple_physics.pinn_solver.domino (v02)
- double_well f — pinneapple_systems.time_series.synthetic (v06)
- double_well_drift f — pinneapple_systems.time_series.synthetic (v06)
- double_well_potential f — pinneapple_quantum.loss.schrodinger (v04)
- DownBlockND C — pinneapple_neural.architectures.neural_operators.uno (v04)
- download f — pinneapple_physics.weather.data (v02)
- download_pdf f — pinneapple_tools.hpo_experiments.sources.arxiv_pdf (v07)
- download_prefix f — pinneapple_data.datasets.deepmind_mgn (v04)
- downsample f — pinneapple_data.simulation_metadata.convergence (v04)
- dp_grade_estimate f — pinneapple_simulation.rover.checks (v03)
- dp_histogram f — pinneapple_security.dp (v05)
- dp_mean f — pinneapple_security.dp (v05)
- DPSGD C — pinneapple_security.dp (v05)
- draft_cadquery_template f — pinneapple_llm.cad_draft (v07)
- draft_digital_twin f — pinneapple_llm.twin_draft (v07)
- draft_geometry f — pinneapple_llm.geometry_draft (v07)
- draft_mesh_recipe f — pinneapple_llm.cad_draft (v07)
- draft_part f — pinneapple_llm.cad_draft (v07)
- draft_preset f — pinneapple_problemdesign.preset_authoring (v02)
- draft_preset_kwargs f — pinneapple_llm.draft (v07)
- draft_problem f — pinneapple_llm.draft (v07)
- DraftedPreset C — pinneapple_problemdesign.preset_authoring (v02)
- DraftRejected C — pinneapple_llm.draft (v07)

- `DraftResult` C — `pinneapple_llm.draft` (v07)
- `drag_factor` f — `pinneapple_simulation.numerical_solvers.particles` (v03)
- `drag_sphere_schiller_naumann` f — `pinneapple_simulation.numerical_solvers.lbm_tunnel` (v03)
- `DragAdjointObjective` C — `pinneapple_design.design_optimizer.adjoint` (v06)
- `DragObjective` C — `pinneapple_design.design_optimizer.objective` (v06)
- `draw_colorbars` f — `pinneapple_tools.visualization.studio.compose` (v07)
- `draw_pitch` f — `pinneapple_systems.sports.football.plots` (v06)
- `draw_text` f — `pinneapple_lab.seal` (v07)
- `DriftForecaster` C — `pinneapple_systems.time_series.baselines.naive` (v06)
- `DriftMonitor` C — `pinneapple_systems.digital_twin.product.policy` (v06)
- `DriftPolicy` C — `pinneapple_systems.digital_twin.product.policy` (v06)
- `DriftReport` C — `pinneapple_analysis.uncertainty.drift` (v05)
- `DriftThresholds` C — `pinneapple_analysis.uncertainty.drift` (v05)
- `drill_assembly` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `Drive` C — `pinneapple_simulation.electromagnetics.machines.drive` (v03)
- `drone` f — `pinneapple_twin3d.gallery` (v06)
- `drop_degenerate` f — `pinneapple_design.geometry_library.entries._adapt` (v06)
- `drop_results` f — `pinneapple_data.preflight` (v04)
- `Drude` C — `pinneapple_simulation.electromagnetics.media` (v03)
- `drude_metal` f — `pinneapple_simulation.electromagnetics.media` (v03)
- `drug_diffusion_tissue` f — `pinneapple_physics.pde_environment.presets.multidisciplinary` (v02)
- `DrydenFilter` C — `pinneapple_simulation.flight.turbulence` (v03)
- `ds_airfoil` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_burgers` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_combustion` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_cosmology` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_crystal` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_em_grid` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_epidemic` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_graph_mesh` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_grid_2d` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_image_field` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_lorenz` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_molecular` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_option_pricing` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_pde_1d` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_pedestrians` f — `pinneapple_arena.external.demo_datasets` (v04)
- `ds_power_system` f — `pinneapple_arena.external.demo_datasets` (v04)

- ds_quantum_circuit f — pinneapple_arena.external.demo_datasets (v04)
- ds_rigidbody f — pinneapple_arena.external.demo_datasets (v04)
- ds_room f — pinneapple_arena.external.demo_datasets (v04)
- ds_seismic f — pinneapple_arena.external.demo_datasets (v04)
- ds_spin_lattice f — pinneapple_arena.external.demo_datasets (v04)
- ds_timeseries f — pinneapple_arena.external.demo_datasets (v04)
- ds_tle f — pinneapple_arena.external.demo_datasets (v04)
- ds_traffic f — pinneapple_arena.external.demo_datasets (v04)
- ds_weather_grid f — pinneapple_arena.external.demo_datasets (v04)
- DSMLoss C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- duarte_loss f — pinneapple_physics.blackhole.forecast (v02)
- duarte_regional_loss f — pinneapple_physics.blackhole.forecast (v02)
- DuarteUNet C — pinneapple_physics.blackhole.forecast (v02)
- dump_yaml f — pinneapple_tools.benchmark_suite.io.yamlx (v07)
- dumps f — pinneapple_decision.schema (v05)
- dumps f — pinneapple_tools.model_services.registry (v07)
- dut1 f — pinneapple_physics.astro.timescales (v02)
- Duty C — pinneapple_tools.nvidia.heatsink (v07)
- duty_point f — pinneapple_physics.closed_form.pump_npsh (v02)
- DynamicModeDecomposition C — pinneapple_neural.architectures.rom.dmd (v04)
- DynamicsAdapter C — pinneapple_neural.architectures.adapters.dynamics (v04)
- DynamicSmagorinsky C — pinneapple_physics.multiscale.closures (v02)

E

- e1_cfd f — pinneapple_condense.falsify (v02)
- ear_clip f — pinneapple_design.geometry_library.kit (v06)
- ear_clip_fast f — pinneapple_design.geometry_library.kit (v06)
- EarlyStopping C — pinneapple_neural.trainer.callbacks (v04)
- earth2_gif f — pinneapple_physics.weather.viz (v02)
- earth_system f — pinneapple_flow.nodes_ext.core_ts (v07)
- EarthEngineUnavailable C — pinneapple_perception.sentinel2 (v04)
- EasyArenaProblem C — pinneapple_arena.custom_builder (v04)
- ece f — pinneapple_systems.sports.football.metrics (v06)
- EchoStateNetwork C — pinneapple_neural.architectures.reservoir_computing.esn (v04)
- ecliptic_matrix f — pinneapple_physics.astro.frames (v02)
- economic_thickness f — pinneapple_physics.closed_form.pipe_insulation (v02)
- eddy_current f — pinneapple_flow.nodes_ext.geomechanics (v07)
- eddy_current_density f — pinneapple_simulation.numerical_solvers.eddy_current_fdm (v03)
- EddyCurrentFDMSolver C — pinneapple_simulation.numerical_solvers.eddy_current_fdm (v03)
- EddyCurrentInspection C — pinneapple_presets.cases.eddy_current (v02)
- EddyViscosity C — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)

- Edge C — pinneapple_flow.schema (v07)
- Edge C — pinneapple_physics.physics_audit.knowledge (v02)
- edge_features f — pinneapple_design.aero.surrogate (v06)
- edge_features f — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- edge_stats f — pinneapple_design.geometry_library.meshops (v06)
- EdgeRuntime C — pinneapple_systems.component_modeling.edge (v06)
- edit_file f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- edits_to_decisions f — pinneapple_flow.decisions (v07)
- EDMNoiseScheduler C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- ee_initialize f — pinneapple_perception.sentinel2 (v04)
- eemd_1d f — pinneapple_simulation.numerical_solvers.eemd (v03)
- EEMDSolver C — pinneapple_simulation.numerical_solvers.eemd (v03)
- effect_curves f — pinneapple_analysis.sensitivity.effects (v05)
- effective_conductivity f — pinneapple_physics.closed_form.pcb_thermal (v02)
- effective_cv f — pinneapple_systems.process_components.control_valve (v06)
- effective_layer f — pinneapple_condense.homogenize (v02)
- effective_properties f — pinneapple_condense.homogenize (v02)
- effective_sample_count f — pinneapple_data.audit (v04)
- effective_viscosity f — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- effectiveness f — pinneapple_physics.closed_form.heat_exchanger (v02)
- efficiency_figure f — pinneapple_systems.sports.football.plots (v06)
- eht_comparison f — pinneapple_physics.astro.lensing (v02)
- eigenvector_overlap f — pinneapple_physics.non_hermitian.pt_dimer (v02)
- einstein_radius_rad f — pinneapple_physics.astro.lensing (v02)
- EKIConfig C — pinneapple_analysis.inverse_problems.ensemble_kalman (v05)
- EKIHistory C — pinneapple_analysis.inverse_problems.ensemble_kalman (v05)
- Elastic C — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- Elasticity3DFDMSolver C — pinneapple_simulation.numerical_solvers.elasticity3d_fdm (v03)
- elasticity_ke f — pinneapple_design.topology.problems (v06)
- elasticity_matrix f — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- ElasticWave3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- ElasticWaveConfig3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- elbow_streamlines f — pinneapple_systems.digital_twin.pipe_loop.scene (v06)
- ElectrostaticSolution C — pinneapple_simulation.electromagnetics.electrostatics (v03)
- electrothermal f — pinneapple_reason.modes.thermal (v05)

- element_adjacency_by_nodes f — pinneapple_data.cae.geometry (v04)
- element_flux_density_history f — pinneapple_simulation.electromagnetics.postprocess (v03)
- element_label f — pinneapple_reason.chain (v05)
- element_metrics f — pinneapple_data.cae.quality (v04)
- elements_from_state f — pinneapple_physics.astro.orbits (v02)
- elf_exported_functions f — pinneapple_simulation.external_solvers.modelica.fmu_inspect (v03)
- ellipse f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- ellipsoid f — pinneapple_design.geometry_library.kit (v06)
- elm_fit f — pinneapple_lab.experiments.discovery (v07)
- elmer_case f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- em_cylinder_scattering f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_residuals f — pinneapple_physics.pinn_solver.compiler.em_kinds (v02)
- em_skin_effect f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_tm_cavity f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_tm_cavity_integral f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_two_dielectric_capacitor f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_two_dielectric_capacitor_differential f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- em_verification f — pinneapple_flow.nodes_ext.electromagnetics (v07)
- em_wire_az f — pinneapple_physics.pde_environment.presets.electromagnetics (v02)
- embedding_views f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- emd_1d f — pinneapple_simulation.numerical_solvers.eemd (v03)
- emit f — pinneapple_veriphysics.execution_log (v05)
- EMPinn C — pinneapple_lab.experiments.em_pinn (v07)
- enable_features f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- enable_gradient_checkpointing f — pinneapple_neural.trainer.parallel (v04)
- encode f — pinneapple_physics.blackhole.dataset (v02)
- encode_mp4 f — pinneapple_tools.visualization.molecular (v07)
- encode_mp4 f — pinneapple_tools.visualization.studio.particles (v07)
- encode_video f — pinneapple_systems.time_series.enso.render (v06)
- EncoderLayer C — pinneapple_neural.architectures.transformers.autoformer (v04)

- encounter_bars_figure f — pinneapple_simulation.flight.figures (v03)
- encounter_feasibility f — pinneapple_analysis.uncertainty.reachability (v05)
- encounter_metrics f — pinneapple_simulation.flight.encounter (v03)
- encounter_traces_figure f — pinneapple_simulation.flight.figures (v03)
- encrypt_array f — pinneapple_security.encryption (v05)
- encrypt_bytes f — pinneapple_security.encryption (v05)
- encrypt_file f — pinneapple_security.encryption (v05)
- end_effects f — pinneapple_flow.nodes_ext.em_prius (v07)
- end_leakage_3d f — pinneapple_simulation.electromagnetics.machines.getdp (v03)
- end_winding_inductance f — pinneapple_simulation.electromagnetics.machines.endwinding (v03)
- end_winding_paths f — pinneapple_simulation.electromagnetics.machines.endwinding (v03)
- endfire_directivity f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- endfire_directivity_dbi f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- energy f — pinneapple_core.loss (v01)
- energy f — pinneapple_physics.astro.nbody (v02)
- energy_balance f — pinneapple_simulation.multibody.arm (v03)
- energy_balance_residual f — pinneapple_systems.operations.plant (v06)
- energy_check f — pinneapple_simulation.flight.simulate (v03)
- energy_correct f — pinneapple_condense.exceptions (v02)
- energy_error_bound f — pinneapple_analysis.uncertainty.residual_bound (v05)
- energy_loss f — pinneapple_quantum.loss.schrodinger (v04)
- energy_norm f — pinneapple_condense.substructure (v02)
- energy_spectrum f — pinneapple_analysis.validation.spectral (v05)
- energy_spectrum f — pinneapple_physics.multiscale.ns2d (v02)
- Engine C — pinneapple_design.aero.optimize (v06)
- Engine C — pinneapple_query.core (v02)
- Engine3D C — pinneapple_design.aero.optimize3d (v06)
- EngineAL C — pinneapple_design.aero.optimize_airliner (v06)
- EnhancerConfig C — pinneapple_worldmodel.photorealistic_enhancer (v04)
- enkf_analysis f — pinneapple_analysis.state_estimation.kalman (v05)
- EnKFForecaster C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- ensemble_fibres f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- ensemble_forward f — pinneapple_physics.pinn_solver.stochastic (v02)
- ensemble_interval_coverage f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- ensemble_spread f — pinneapple_lab.uq (v07)
- ensemble_study f — pinneapple_veriphysics.robustness (v05)

- `EnsembleConfig` C — `pinneapple_analysis.uncertainty.ensemble` (v05)
- `EnsembleConfig` C — `pinneapple_physics.multiscale.comparison` (v02)
- `EnsembleForecaster` C — `pinneapple_systems.time_series.enso.forecasters` (v06)
- `EnsembleKalmanFilter` C — `pinneapple_analysis.state_estimation.kalman` (v05)
- `EnsembleKalmanFilter` C — `pinneapple_neural.architectures.classical_ts.enkf` (v04)
- `EnsembleKalmanInversion` C — `pinneapple_analysis.inverse_problems.ensemble_kalman` (v05)
- `EnsembleModel` C — `pinneapple_worldmodel.model_zoo` (v04)
- `EnsembleRun` C — `pinneapple_physics.ensemble` (v02)
- `EnsembleUQ` C — `pinneapple_analysis.uncertainty.ensemble` (v05)
- `enso` f — `pinneapple_flow.nodes.systems` (v07)
- `enso_scores` f — `pinneapple_systems.time_series.enso.pipeline` (v06)
- `enso_solver_checks` f — `pinneapple_flow.nodes_ext.forecasting` (v07)
- `EnsoDataset` C — `pinneapple_systems.time_series.enso.dataset` (v06)
- `ensure_dir` f — `pinneapple_tools.hpo_experiments.kb.store` (v07)
- `ensure_float_dict` f — `pinneapple_tools.benchmark_suite.runner.metrics` (v07)
- `ensure_image` f — `pinneapple_simulation.external_solvers.calculix.runner` (v03)
- `ensure_model_pulled` f — `pinneapple_llm.local_llm` (v07)
- `ensure_point_set` f — `pinneapple_tools.benchmark_suite.crossframework.protocol` (v07)
- `ensure_requires_grad` f — `pinneapple_physics.pinn_solver.factory.autodiff` (v02)
- `ensure_tensor` f — `pinneapple_physics.pinn_solver.compiler.autograd_ops` (v02)
- `ensure_twin_model` f — `pinneapple_systems.digital_twin.product.model` (v06)
- `EntitySpec` C — `pinneapple_simulation.external_solvers.genesis.scene_builder` (v03)
- `entry` f — `pinneapple_flow.palette` (v07)
- `entry` f — `pinneapple_hub.zoo._artifacts` (v07)
- `env_fingerprint` f — `pinneapple_neural.trainer.audit` (v04)
- `Envelope` C — `pinneapple_physics.physics_audit.assumptions` (v02)
- `envelope` f — `pinneapple_physics.astro.decay` (v02)
- `envelope_check` f — `pinneapple_physics.physics_audit.monitor` (v02)
- `envelope_checks` f — `pinneapple_analysis.benchmark.validation` (v05)
- `envelope_cls` f — `pinneapple_check._reuse` (v05)
- `Environment` C — `pinneapple_physics.closed_form.pcb_thermal` (v02)
- `environment` f — `pinneapple_autolab.ledger` (v07)
- `environment` f — `pinneapple_presets.core` (v02)
- `environment` f — `pinneapple_tools.benchmark_suite.crossframework.records` (v07)
- `environment` f — `pinneapple_tools.nvidia.runinfo` (v07)

- environment_snapshot f — pinneapple_llm.qa.report (v07)
- eof_fill f — pinneapple_systems.time_series.enso.dataset (v06)
- ep_figure f — pinneapple_flow.nodes_ext.non_hermitian (v07)
- ep_laws f — pinneapple_flow.nodes_ext.non_hermitian (v07)
- ep_numeric f — pinneapple_flow.nodes_ext.non_hermitian (v07)
- ep_tuning_figure f — pinneapple_physics.non_hermitian.viz (v02)
- Ephemeris C — pinneapple_physics.astro.ephemeris (v02)
- episode_metrics f — pinneapple_security.cps (v05)
- EPTuningReport C — pinneapple_physics.non_hermitian.ep_tuning (v02)
- eq f — pinneapple_core.optim (v01)
- equal_budget f — pinneapple_flow.nodes_ext.benchmarks (v07)
- equal_budget f — pinneapple_simulation.electromagnetics.periodic.benchmark (v03)
- equal_budget_search f — pinneapple_analysis.cost.equal_budget (v05)
- equal_flops_table f — pinneapple_analysis.benchmark.report (v05)
- Equation C — pinneapple_check.physics (v05)
- Equation C — pinneapple_physics.symbolic_pde.text (v02)
- equations f — pinneapple_check.physics (v05)
- equations f — pinneapple_flow.nodes.problem (v07)
- EquationSpec C — pinneapple_app.backend.core.problem (v07)
- EquationSyntaxError C — pinneapple_physics.symbolic_pde.text (v02)
- EquationSystem C — pinneapple_physics.symbolic_pde.text (v02)
- equatorial_to_horizontal f — pinneapple_physics.astro.frames (v02)
- EquatorialWavePINN C — pinneapple_systems.time_series.enso.pinn (v06)
- equilibrium_residual_rz f — pinneapple_physics.solid.axisymmetric (v02)
- EquivariantGNN C — pinneapple_neural.architectures.graphnn.equivariant_gnn (v04)
- era f — pinneapple_physics.astro.timescales (v02)
- era5_monthly f — pinneapple_systems.time_series.enso.data (v06)
- Era5Store C — pinneapple_physics.weather.data (v02)
- error_by_region f — pinneapple_analysis.validation.error_map (v05)
- error_hotspots f — pinneapple_analysis.validation.error_map (v05)
- error_map f — pinneapple_analysis.validation.error_map (v05)
- error_section f — pinneapple_app.backend.core.surrogate_report (v07)
- ErrorMap C — pinneapple_analysis.validation.error_map (v05)
- errors f — pinneapple_condense.substructure (v02)
- errors f — pinneapple_design.thermal.board_cooling.learn (v06)
- ESNRC C — pinneapple_neural.architectures.reservoir_computing.esn_rc (v04)
- estimate f — pinneapple_condense.hierarchy (v02)
- estimate_bounding_circle f — pinneapple_perception.image_geometry (v04)
- estimate_channel_scales f — pinneapple_neural.trainer.collocation (v04)
- estimate_mass f — pinneapple_design.bricks.parts (v06)

- estimate_recovery f — pinneapple_twin3d.wear.calibrate (v06)
- estimate_required_first_cell_height f — pinneapple_analysis.verification.mesh_intelligence (v05)
- estimate_rul f — pinneapple_systems.digital_twin.prognostics (v06)
- estimate_rul_from_twin f — pinneapple_systems.digital_twin.prognostics (v06)
- estimated_error f — pinneapple_physics.darcy_checks (v02)
- estimated_flux_error f — pinneapple_physics.darcy_checks (v02)
- ETDRK4 C — pinneapple_simulation.numerical_solvers.etdrk4 (v03)
- ETDRK4Result C — pinneapple_simulation.numerical_solvers.etdrk4 (v03)
- euler_characteristic f — pinneapple_design.geometry.ops.curvature (v06)
- euler_critical_buckling_load f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- euler_to_quat f — pinneapple_simulation.flight.rigid_body (v03)
- eval_act f — pinneapple_simulation.numerical_solvers.thermal_channel_datasets (v03)
- eval_axes f — pinneapple_flow.nodes._common (v07)
- eval_rollout f — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- eval_splits f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- evaluate f — pinneapple_analysis.reproduction (v05)
- evaluate f — pinneapple_analysis.surrogate.study (v05)
- evaluate f — pinneapple_condense.meshless (v02)
- evaluate f — pinneapple_data.expressions (v04)
- evaluate f — pinneapple_design.aero.aircraft (v06)
- evaluate f — pinneapple_hub.zoo (v07)
- evaluate f — pinneapple_hub.zoo.checks (v07)
- evaluate f — pinneapple_physics.astro.neural_propagator (v02)
- evaluate f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- evaluate f — pinneapple_physics.pcb.engine (v02)
- evaluate f — pinneapple_simulation.electromagnetics.reflector.compact_range (v03)
- evaluate f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- evaluate f — pinneapple_simulation.pipelines.freecad_openfoam.pipeline (v03)
- evaluate f — pinneapple_systems.process_components.explicit_equation_system (v06)
- evaluate f — pinneapple_systems.time_series.forecast_lab.study (v06)
- evaluate_criterion f — pinneapple_tools.automation.verify (v07)
- evaluate_design f — pinneapple_design.thermal.heatsink.engine (v06)
- evaluate_design f — pinneapple_tools.nvidia.heatsink (v07)
- evaluate_experts f — pinneapple_physics.multiscale.comparison (v02)

- `evaluate_expr` f — `pinneapple_check.checks.results` (v05)
- `evaluate_inverse` f — `pinneapple_tools.benchmark_suite.crossframework.evaluate` (v07)
- `evaluate_level` f — `pinneapple_llm.qa.levels` (v07)
- `evaluate_loss` f — `pinneapple_physics.blackhole.forecast` (v02)
- `evaluate_map` f — `pinneapple_systems.process_components.similarity_map` (v06)
- `evaluate_model` f — `pinneapple_arena.trainer` (v04)
- `evaluate_operator` f — `pinneapple_tools.benchmark_suite.crossframework.evaluate` (v07)
- `evaluate_part` f — `pinneapple_tools.nvidia.brep` (v07)
- `evaluate_pinn` f — `pinneapple_tools.benchmark_suite.crossframework.evaluate` (v07)
- `evaluate_posterior` f — `pinneapple_analysis.uncertainty.posterior_metrics` (v05)
- `evaluate_skill` f — `pinneapple_physics.blackhole.skill` (v02)
- `Evaluation` C — `pinneapple_design.bricks.goals` (v06)
- `Evaluation` C — `pinneapple_design.qualitative.models` (v06)
- `evaluation_order` f — `pinneapple_systems.process_components.explicit_equation_system` (v06)
- `Evaluations` C — `pinneapple_design.design_optimizer.study` (v06)
- `Evaluator` C — `pinneapple_design.design_optimizer.study` (v06)
- `evasion_search` f — `pinneapple_analysis.validation.arena.red` (v05)
- `Event` C — `pinneapple_physics.weather.events` (v02)
- `Event` C — `pinneapple_systems.operations.anomaly` (v06)
- `event_hindcast` f — `pinneapple_systems.time_series.enso.study` (v06)
- `event_states` f — `pinneapple_physics.weather.events` (v02)
- `Evidence` C — `pinneapple_autolab.core` (v07)
- `Evidence` C — `pinneapple_decision.schema` (v05)
- `Evidence` C — `pinneapple_tools.automation.record` (v07)
- `evidence_line` f — `pinneapple_lab.showcase` (v07)
- `evidence_stages` f — `pinneapple_lab.curation` (v07)
- `evidence_summary` f — `pinneapple_analysis.verification.evidence_graph` (v05)
- `EvidenceEntry` C — `pinneapple_veriphysics.ledger` (v05)
- `EvidenceLedger` C — `pinneapple_veriphysics.ledger` (v05)
- `EvidenceStore` C — `pinneapple_decision.loop` (v05)
- `evolution_timeline` f — `pinneapple_design.bricks.visuals` (v06)
- `EvolutionaryDesignOptimizer` C — `pinneapple_design.design_optimizer.optimizer` (v06)
- `evolve` f — `pinneapple_design.bricks.search` (v06)
- `exact` f — `pinneapple_lab.experiments.ode` (v07)
- `exact_advection` f — `pinneapple_physics.multiscale.ns2d` (v02)
- `exact_eigenstates_1d_harmonic` f — `pinneapple_quantum.data.quantum_dataset` (v04)
- `exact_function` f — `pinneapple_flow.nodes._common` (v07)
- `ExactShapleyAttributor` C — `pinneapple_analysis.inverse_problems.counterfactual_attribution` (v05)
- `example` f — `pinneapple_hub.library.snippets` (v07)
- `example_chiller_plant` f — `pinneapple_data.data_health` (v04)
- `example_dir` f — `pinneapple_flow.nodes_ext.examples` (v07)
- `example_dirs` f — `pinneapple_lab.manifest` (v07)
- `example_folders` f — `pinneapple_flow.examples` (v07)

- example_json f — pinneapple_hub.zoo.hub_cards (v07)
- example_plant_operations f — pinneapple_data.process_optimizer (v04)
- example_sources f — pinneapple_data.unified (v04)
- example_workflow f — pinneapple_tools.automation (v07)
- ExampleCase C — pinneapple_veriphysics.example_cases (v05)
- examples f — pinneapple_hub.library.sources (v07)
- examples_root f — pinneapple_flow.nodes_ext.examples (v07)
- ExampleScript C — pinneapple_lab.experiments.examples (v07)
- exception_patches f — pinneapple_condense.falsify (v02)
- exceptions_study f — pinneapple_condense.benchmark (v02)
- exchanger_model f — pinneapple_systems.operations.exchanger (v06)
- ExecResult C — pinneapple_core.workflows.executors (v01)
- execute f — pinneapple_autolab.core (v07)
- execute f — pinneapple_decision.api (v05)
- execute_jobs f — pinneapple_analysis.benchmark.runner (v05)
- ExecuteNode C — pinneapple_decision.tree (v05)
- ExecutionResult C — pinneapple_decision.schema (v05)
- Executor C — pinneapple_core.workflows.executors (v01)
- Executor C — pinneapple_decision.loop (v05)
- exhaustive f — pinneapple_analysis.proof.graph (v05)
- exodus_to_upd f — pinneapple_simulation.external_solvers.cfd_formats.exodus_reader (v03)
- expand f — pinneapple_data.cae.upload (v04)
- expand_target f — pinneapple_simulation.external_solvers.calculix.inp (v03)
- ExpandingWindowSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- expected_exponent f — pinneapple_analysis.cost.profiler (v05)
- Experiment C — pinneapple_lab.spec (v07)
- Experiment C — pinneapple_physics.experiment (v02)
- Experiment C — pinneapple_science.core (v02)
- experiment f — pinneapple_lab.capture (v07)
- experiment_ws f — pinneapple_app.backend.routers.experiments (v07)
- ExperimentConfig C — pinneapple_app.backend.core.experiment (v07)
- ExperimentMemory C — pinneapple_registry.experiment_memory (v01)
- ExperimentRecord C — pinneapple_registry.experiment_memory (v01)
- ExperimentRequest C — pinneapple_app.backend.routers.experiments (v07)
- ExperimentResult C — pinneapple_app.backend.core.experiment (v07)
- ExperimentResult C — pinneapple_physics.experiment (v02)
- ExperimentRunner C — pinneapple_app.backend.core.experiment (v07)
- experiments f — pinneapple_science.registry (v02)
- ExperimentStatusResponse C — pinneapple_app.backend.routers.experiments (v07)
- ExperimentStore C — pinneapple_registry.experiment_store (v01)
- Expert C — pinneapple_physics.ensemble (v02)
- explain f — pinneapple_lab.next_steps (v07)
- explain f — pinneapple_reason.narrative (v05)

- explain f — pinneapple_twin3d.wear.optimize (v06)
- explain_preset f — pinneapple_analysis.verification.knowledge_graph (v05)
- explain_trust f — pinneapple_analysis.verification.evidence_graph (v05)
- ExplicitAirframe C — pinneapple_simulation.flight.catalog (v03)
- ExplicitEquationError C — pinneapple_systems.process_components.explicit_equation_system (v06)
- exponential_density f — pinneapple_physics.astro.decay (v02)
- ExponentialSmoothing C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- export f — pinneapple_data.cae.dataset (v04)
- export f — pinneapple_design.geometry_library (v06)
- export f — pinneapple_design.geometry_library.exporters (v06)
- export_all f — pinneapple_twin3d.gallery (v06)
- export_bundle f — pinneapple_hub.zoo.web (v07)
- export_bundle f — pinneapple_simulation.external_solvers.openfoam.export_bundle (v03)
- export_bytes f — pinneapple_hub.zoo.export (v07)
- export_cad f — pinneapple_design.intent.physics.bracket (v06)
- export_cadquery f — pinneapple_design.geometry.gen.cadquery_gen (v06)
- export_csv f — pinneapple_tools.model_export (v07)
- export_dataset f — pinneapple_llm.workbench.finetune (v07)
- export_edge_package f — pinneapple_systems.component_modeling.edge (v06)
- export_model f — pinneapple_tools (v07)
- export_npz f — pinneapple_tools.model_export (v07)
- export_odb_fields f — pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader (v03)
- export_onnx f — pinneapple_design.aero.geometry_onnx (v06)
- export_onnx f — pinneapple_hub.zoo (v07)
- export_onnx f — pinneapple_hub.zoo.export (v07)
- export_onnx f — pinneapple_neural.architectures.neural_operators.fno_export (v04)
- export_onnx f — pinneapple_tools.model_export (v07)
- export_predictions f — pinneapple_tools.benchmark_suite.crossframework.protocol (v07)
- export_problemdesign f — pinneapple_tools.hpo_experiments.pipelines.export_problemdesign (v07)
- export_recipe f — pinneapple_llm.cad_draft (v07)
- export_rover f — pinneapple_simulation.rover.geometry (v03)
- export_safetensors f — pinneapple_hub.zoo.export (v07)
- export_scene f — pinneapple_blender.export (v07)
- export_scene f — pinneapple_systems.digital_twin.product.runner (v06)
- export_schemas f — pinneapple_data.contracts.models (v04)
- export_space f — pinneapple_hub.zoo.demo (v07)
- export_stl_bytes f — pinneapple_design.surrogate_inverse.brake_rotor (v06)

- export_stl_bytes f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- export_stl_bytes f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- export_stl_bytes f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- export_stl_bytes f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- export_to_lab f — pinneapple_flow.runtime (v07)
- export_torchscript f — pinneapple_tools.model_export (v07)
- export_trajectory f — pinneapple_blender.export (v07)
- export_twin f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- export_usd f — pinneapple_twin3d.usd (v06)
- export_vtk f — pinneapple_tools.visualization.pyvista_bridge (v07)
- export_wear_twin f — pinneapple_twin3d.wear.build (v06)
- export_web f — pinneapple_design.geometry_library (v06)
- export_web f — pinneapple_design.geometry_library.web (v06)
- export_web f — pinneapple_hub.zoo (v07)
- export_web f — pinneapple_hub.zoo.web (v07)
- exported_functions_from_layout f — pinneapple_tools.rea_bridge.evidence (v07)
- ExpressionError C — pinneapple_data.expressions (v04)
- ExpressionModel C — pinneapple_analysis.inverse_problems.forward (v05)
- ExtendedKalmanFilter C — pinneapple_analysis.state_estimation.kalman (v05)
- ExtendedKalmanFilter C — pinneapple_neural.architectures.classical_ts.ekf (v04)
- ExternalFlow C — pinneapple_design.qualitative.models (v06)
- ExternalFlow C — pinneapple_simulation.numerical_solvers.external_flow (v03)
- ExternalTool C — pinneapple_analysis.verification.tool_recommendation (v05)
- extinction f — pinneapple_physics.astro.observing (v02)
- extinction_mag f — pinneapple_physics.astro.astrometry (v02)
- extract f — pinneapple_data.simulation_metadata (v04)
- extract f — pinneapple_lab.results_rules (v07)
- extract_1d_signal f — pinneapple_simulation.numerical_solvers.adapters.upd_signal (v03)
- extract_boundary_points f — pinneapple_perception.image_geometry (v04)
- extract_case f — pinneapple_data.simulation_metadata.openfoam (v04)
- extract_dominant_frequencies f — pinneapple_perception.audio_modal (v04)
- extract_fields f — pinneapple_simulation.numerical_solvers.openfoam_bridge (v03)
- extract_flights f — pinneapple_systems.sports.football.ball (v06)
- extract_from_text f — pinneapple_problemdesign.extractor (v02)
- extract_geometry_features f — pinneapple_analysis.verification.geometry_ood_guardrail (v05)
- extract_json f — pinneapple_tools.dataset_quality.vlm_curation (v07)

- `extract_pdf_text` f — `pinneapple_tools.hpo_experiments.sources.arxiv_pdf` (v07)
- `extract_problem_solutions` f — `pinneapple_tools.hpo_experiments.pipelines.extract_problem_solutions` (v07)
- `extract_rules` f — `pinneapple_data.formfill.extract` (v04)
- `extract_rules` f — `pinneapple_data.udr1` (v04)
- `extract_with_llm` f — `pinneapple_data.formfill.llm` (v04)
- `ExtractedProblemSolution` C — `pinneapple_tools.hpo_experiments.models` (v07)
- `ExtractLLM` C — `pinneapple_reason.llm` (v05)
- `ExtractorAgent` C — `pinneapple_tools.hpo_experiments.agents.extractor` (v07)
- `extrapolation_study` f — `pinneapple_veriphysics.robustness` (v05)
- `ExtremeLearningMachine` C — `pinneapple_neural.architectures.reservoir_computing.elm` (v04)
- `extrude` f — `pinneapple_design.geometry_library.kit` (v06)
- `extrude` f — `pinneapple_design.qualitative.descriptors` (v06)
- `extrude` f — `pinneapple_presets.viz3d` (v02)
- `extrude_polygon` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `extrude_profile` f — `pinneapple_tools.nvidia.omniverse` (v07)
- `Eyepiece` C — `pinneapple_physics.astro.observing` (v02)

F

- `f0_validation` f — `pinneapple_condense.falsify` (v02)
- `f1_advection` f — `pinneapple_condense.falsify` (v02)
- `f2_stokes` f — `pinneapple_condense.falsify` (v02)
- `f3_cht` f — `pinneapple_condense.falsify` (v02)
- `Face` C — `pinneapple_simulation.pipelines.freecad_openfoam.cad` (v03)
- `face_geometry` f — `pinneapple_data.cae.geometry` (v04)
- `face_geometry` f — `pinneapple_design.qualitative.descriptors` (v06)
- `face_mesh` f — `pinneapple_data.cae.geometry` (v04)
- `face_modes` f — `pinneapple_condense.algebraic` (v02)
- `face_normals` f — `pinneapple_design.geometry_library.meshops` (v06)
- `face_reynolds_number` f — `pinneapple_systems.process_components.pipe_network_1d` (v06)
- `face_weights` f — `pinneapple_condense.substructure` (v02)
- `FaceMesh` C — `pinneapple_data.cae.geometry` (v04)
- `faces` f — `pinneapple_condense.algebraic` (v02)
- `faces_for_tag` f — `pinneapple_app.backend.core.physics_adapter` (v07)
- `faces_of` f — `pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job` (v03)
- `facets` f — `pinneapple_hub.library` (v07)
- `Fact` C — `pinneapple_design.fitmesh.context` (v06)
- `factor` f — `pinneapple_condense.fv` (v02)
- `factory_loss` f — `pinneapple_flow.nodes_ext.pinn_solver` (v07)
- `FactoryConfig` C — `pinneapple_worldmodel.dataset_factory` (v04)
- `FactoryResult` C — `pinneapple_worldmodel.synthetic_factory` (v04)
- `FactQuestion` C — `pinneapple_decision.selectors` (v05)

- failed f — pinneapple_arena.external.result (v04)
- failure_mode f — pinneapple_flow.nodes.reason (v07)
- FailureMode C — pinneapple_design.intent.template (v06)
- FailureMode C — pinneapple_reason.library (v05)
- FailureRound C — pinneapple_design.bricks.statics (v06)
- FakeFreeCAD C — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- false_alarm_search f — pinneapple_analysis.validation.arena.red (v05)
- family_length_ok f — pinneapple_flow.planner.heuristic (v07)
- fan_cooler_cfd f — pinneapple_physics.pde_environment.presets.engineering (v02)
- fan_panels f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- fan_positions f — pinneapple_design.thermal.board_cooling.scene (v06)
- far_surface f — pinneapple_simulation.vehicle.viz (v03)
- faraday f — pinneapple_physics.maxwell_checks (v02)
- faraday_rate f — pinneapple_reason.modes.corrosion (v05)
- FaradayInduction C — pinneapple_science.em.induction (v02)
- FarField C — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- farfield_metrics f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- fast_non_dominated_sort f — pinneapple_design.design_optimizer.pareto (v06)
- fatigue_life_cycles f — pinneapple_design.surrogate_inverse.suspension_arm (v06)
- Fault C — pinneapple_analysis.validation.fault_injection.faults (v05)
- Fault C — pinneapple_systems.operations.plant (v06)
- fault_benchmark f — pinneapple_systems.operations.anomaly (v06)
- fault_rng f — pinneapple_analysis.validation.fault_injection.matrix (v05)
- FaultInjectionReport C — pinneapple_analysis.validation.fault_injection.matrix (v05)
- Faults C — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- fcc_slab f — pinneapple_tools.visualization.molecular (v07)
- fd_derivative f — pinneapple_neural.architectures.neural_operators.pino (v04)
- fd_nodes f — pinneapple_simulation.meshless.methods (v03)
- FDError C — pinneapple_simulation.numerical_solvers.symbolic_fd (v03)
- fdi_benchmark f — pinneapple_flow.nodes_ext.security (v07)
- FDMScriptGenerator C — pinneapple_physics.codegen.fdm_script_generator (v02)
- FDMSolver C — pinneapple_simulation.numerical_solvers.fdm (v03)
- FDOperator C — pinneapple_simulation.numerical_solvers.symbolic_fd (v03)
- FDResult C — pinneapple_simulation.numerical_solvers.symbolic_fd (v03)
- FDTD C — pinneapple_simulation.electromagnetics.fdtd (v03)
- FDTDConfig C — pinneapple_science.em.fdtd_study (v02)
- FDTDRun C — pinneapple_science.em.fdtd_study (v02)
- FDTDStudy3D C — pinneapple_science.em.research (v02)

- fea_figure f — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- FEAResult C — pinneapple_design.bricks.fea (v06)
- feasibility_frequency f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- feasibility_report f — pinneapple_design.design_optimizer.study (v06)
- feature_angles f — pinneapple_worldmodel.robot_perception (v04)
- featured f — pinneapple_hub.library (v07)
- FeatureHit C — pinneapple_tools.rea_bridge.evidence (v07)
- FeatureMap C — pinneapple_tools.rea_bridge.evidence (v07)
- features f — pinneapple_design.thermal.heatsink.surrogate (v06)
- features f — pinneapple_physics.astro.neural_propagator (v02)
- features_yaml f — pinneapple_hub.zoo.parquet (v07)
- featurize_mesh f — pinneapple_data.geometry_features (v04)
- FEDformer C — pinneapple_neural.architectures.transformers.fedformer (v04)
- feedback_coupling f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- femm_lua f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- FEModel C — pinneapple_simulation.external_solvers.calculix.study (v03)
- FEMResult C — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- FEMSolver C — pinneapple_simulation.numerical_solvers.fem (v03)
- fenics_available f — pinneapple_simulation.numerical_solvers (v03)
- FEnicsBridge C — pinneapple_simulation.numerical_solvers.fenics_bridge (v03)
- FEniCSConfig C — pinneapple_simulation.external_solvers.fenics.solver (v03)
- FEniCSScriptGenerator C — pinneapple_physics.codegen.fenics_script_generator (v02)
- FEniCSWorkflow C — pinneapple_simulation.external_solvers.fenics.solver (v03)
- fetch f — pinneapple_catalog.resources (v01)
- fetch f — pinneapple_hub.zoo._artifacts (v07)
- fetch f — pinneapple_systems.sports.football.data (v06)
- fetch f — pinneapple_systems.time_series.enso.data (v06)
- fetch_cached f — pinneapple_data.datasets.download (v04)
- fetch_cloud_forecast f — pinneapple_physics.astro.conditions (v02)
- fetch_ensemble f — pinneapple_physics.astro.conditions (v02)
- fetch_iv f — pinneapple_systems.digital_twin.io.usgs (v06)
- fetch_literature_documents f — pinneapple_analysis.retrieval.corpus (v05)
- fetch_live_kp f — pinneapple_physics.astro.spaceweather (v02)
- fetch_parts_library f — pinneapple_design.bricks.ldraw (v06)
- fetch_pinned f — pinneapple_systems.time_series.datasets.public (v06)
- fetch_sentinel2_sequence f — pinneapple_perception.sentinel2 (v04)
- fetch_silso_daily f — pinneapple_systems.time_series.solar_cycle (v06)
- fetch_text f — pinneapple_llm.workbench.web (v07)

- `fetch_tles` f — `pinneapple_physics.astro.satellites` (v02)
- `fetch_to_file` f — `pinneapple_systems.time_series.enso.data` (v06)
- `FFmpegMissing` C — `pinneapple_perception.piv_tools` (v04)
- `FFTDecomposer` C — `pinneapple_systems.time_series.decomposition.fft_lstm` (v06)
- `FFTForecaster` C — `pinneapple_systems.time_series.decomposition.fft_forecaster` (v06)
- `FFTLSTMPipeline` C — `pinneapple_systems.time_series.decomposition.fft_lstm` (v06)
- `FFTNNForecaster` C — `pinneapple_systems.time_series.decomposition.fft_nn` (v06)
- `FFTSolver` C — `pinneapple_simulation.numerical_solvers.fft` (v03)
- `Fidelity` C — `pinneapple_design.intent.template` (v06)
- `Field` C — `pinneapple_core.field` (v01)
- `Field` C — `pinneapple_data.formfill.spec` (v04)
- `field3d` f — `pinneapple_flow.nodes.viz` (v07)
- `field_3d` f — `pinneapple_design.thermal.heatsink.engine` (v06)
- `field_3d` f — `pinneapple_physics.pcb.engine` (v02)
- `field_checks` f — `pinneapple_analysis.trust.field_card` (v05)
- `field_colors` f — `pinneapple_tools.nvidia.omniverse` (v07)
- `field_columns` f — `pinneapple_analysis.surrogate.data` (v05)
- `field_difference` f — `pinneapple_simulation.accelerated_foam.case` (v03)
- `field_divergence` f — `pinneapple_simulation.numerical_solvers.immersed_boundary_fdm` (v03)
- `field_energy` f — `pinneapple_science.numerics` (v02)
- `field_error_metrics` f — `pinneapple_analysis.validation.field_metrics` (v05)
- `field_errors` f — `pinneapple_autolab.surrogates` (v07)
- `field_features` f — `pinneapple_analysis.benchmark.validation` (v05)
- `field_fit_slice` f — `pinneapple_arena.showcase` (v04)
- `field_map` f — `pinneapple_tools.visualization.electromagnetics` (v07)
- `field_metrics` f — `pinneapple_arena.pointwise.training` (v04)
- `field_metrics` f — `pinneapple_check._reuse` (v05)
- `field_movie` f — `pinneapple_tools.visualization.electromagnetics` (v07)
- `field_on` f — `pinneapple_presets.viz3d` (v02)
- `field_on_points` f — `pinneapple_simulation.electromagnetics.reflector.study_report` (v03)
- `field_panels` f — `pinneapple_tools.nvidia.figures` (v07)
- `field_range_issues` f — `pinneapple_tools.dataset_quality.checks` (v07)
- `field_rotation_rate` f — `pinneapple_physics.astro.observing` (v02)
- `field_sampler` f — `pinneapple_analysis.validation.metamorphic` (v05)
- `field_scalar` f — `pinneapple_data.cae.dataset` (v04)
- `field_scales` f — `pinneapple_app.backend.core.surrogate_report` (v07)

- field_scales f — pinneapple_neural.architectures.rom.study (v04)
- field_scores f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- field_slices_html f — pinneapple_tools.visualization.integrations.plotly_export (v07)
- field_validation f — pinneapple_analysis.benchmark.validation (v05)
- field_value f — pinneapple_data.cae.compare (v04)
- field_view f — pinneapple_tools.visualization.studio.fieldview (v07)
- FieldCheck C — pinneapple_analysis.trust.field_card (v05)
- FieldData C — pinneapple_analysis.benchmark.builtin.fields (v05)
- FieldData C — pinneapple_check.fields (v05)
- FieldErrorReport C — pinneapple_analysis.validation.field_metrics (v05)
- FieldMetrics C — pinneapple_systems.digital_twin.product.evidence (v06)
- FieldOperatorProblem C — pinneapple_analysis.benchmark.builtin.fields (v05)
- FieldProbe f — pinneapple_science.instruments (v02)
- fields_of f — pinneapple_data.cae.compare (v04)
- FieldSet C — pinneapple_physics.maxwell_checks (v02)
- FieldSnapshot C — pinneapple_data.adapters.splash_archive_adapter (v04)
- FieldSolves C — pinneapple_simulation.accelerated_foam.case (v03)
- FieldSpec C — pinneapple_systems.digital_twin.product.spec (v06)
- FieldStage C — pinneapple_tools.nvidia.omniverse (v07)
- FieldSurrogate C — pinneapple_autolab.surrogates (v07)
- FieldTrustCard C — pinneapple_analysis.trust.field_card (v05)
- FieldView C — pinneapple_tools.visualization.studio.fieldview (v07)
- fig_field_triptych f — pinneapple_autolab.report (v07)
- fig_learning f — pinneapple_autolab.report (v07)
- fig_models f — pinneapple_autolab.report (v07)
- fig_pareto f — pinneapple_autolab.report (v07)
- fig_trust_cards f — pinneapple_autolab.report (v07)
- Figure C — pinneapple_tools.reporting.datasheet (v07)
- figure f — pinneapple_science.plots (v02)
- figure_confusion f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- figure_evolution f — pinneapple_design.thermal.board_cooling.report (v06)
- figure_holdout f — pinneapple_design.thermal.board_cooling.report (v06)
- figure_maps f — pinneapple_design.thermal.board_cooling.report (v06)
- figure_request f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- figures f — pinneapple_analysis.benchmark.report (v05)
- filament_inductance f — pinneapple_simulation.electromagnetics.statics (v03)
- file_fingerprint f — pinneapple_analysis.retrieval.corpus (v05)

- file_name f — pinneapple_design.geometry_library.store (v06)
- file_record f — pinneapple_tools.automation.record (v07)
- FileRecord C — pinneapple_tools.automation.record (v07)
- FileRef C — pinneapple_data.contracts.models (v04)
- FileWatchStream C — pinneapple_systems.digital_twin.io.stream (v06)
- fill f — pinneapple_core.workflows.dag (v01)
- fill_form f — pinneapple_data.udr1 (v04)
- fill_fraction f — pinneapple_simulation.electromagnetics.fdtd (v03)
- fill_gaps f — pinneapple_physics.weather.viz (v02)
- fill_level f — pinneapple_twin3d.wear.presets (v06)
- fill_pdf f — pinneapple_data.formfill.fill (v04)
- fillet_edges f — pinneapple_design.geometry.ops.occ_features (v06)
- FiLMLayer C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- Filter C — pinneapple_physics.multiscale.filters (v02)
- filter_curated f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- filter_entries f — pinneapple_lab.explorer (v07)
- filter_nan_rows f — pinneapple_neural.trainer.collocation (v04)
- filter_supported_kwargs f — pinneapple_neural.architectures.instantiate (v04)
- filter_variable_width f — pinneapple_physics.multiscale.filters (v02)
- filtering_consistency_loss f — pinneapple_physics.multiscale.learned (v02)
- Fin C — pinneapple_simulation.flight.aero (v03)
- fin_array f — pinneapple_design.surrogate_inverse.meshes (v06)
- fin_array_base_temperature_c f — pinneapple_physics.closed_form.fin_array_conduction (v02)
- fin_array_profile_params f — pinneapple_physics.closed_form.fin_array_conduction (v02)
- fin_array_temperature_at_position_c f — pinneapple_physics.closed_form.fin_array_conduction (v02)
- fin_cell_check f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- fin_exact f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- fin_plate f — pinneapple_simulation.flight.mass (v03)
- fin_q_exact f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- fin_synthetic_readings f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- fin_temperature_at_height_c f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- FinCellResult C — pinneapple_design.thermal.fin_cell (v06)
- find f — pinneapple_design.intent.materials (v06)
- find_bashrc f — pinneapple_simulation.accelerated_foam.case (v03)
- find_case f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- find_chrome f — pinneapple_tools.reporting.datasheet (v07)
- find_config f — pinneapple_check.config (v05)
- find_freecadcmd f — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)

- `find_kernel` f — `pinneapple_physics.astro.ephemeris` (v02)
- `find_minima` f — `pinneapple_physics.astro.phenomena` (v02)
- `find_passes` f — `pinneapple_physics.astro.satellites` (v02)
- `find_pii` f — `pinneapple_security.privacy` (v05)
- `find_plugin` f — `pinneapple_simulation.accelerated_foam.build` (v03)
- `find_presets_for_phenomenon` f — `pinneapple_analysis.verification.knowledge_graph` (v05)
- `find_quantities` f — `pinneapple_design.intent.parse` (v06)
- `find_result` f — `pinneapple_tools.visualization.studio.solver_view` (v07)
- `find_root` f — `pinneapple_data.simulation_metadata.openfoam` (v04)
- `find_roots` f — `pinneapple_physics.astro.phenomena` (v02)
- `find_verified_equations` f — `pinneapple_analysis.verification.knowledge_graph` (v05)
- `Finding` C — `pinneapple_analysis.validation.arena.red` (v05)
- `Finding` C — `pinneapple_analysis.validation.physics_learning` (v05)
- `Finding` C — `pinneapple_data.preflight.common` (v04)
- `Finding` C — `pinneapple_llm.qa.report` (v07)
- `Finding` C — `pinneapple_reason.chain` (v05)
- `Finding` C — `pinneapple_security.privacy` (v05)
- `Finding` C — `pinneapple_tools.rea_bridge.harvest` (v07)
- `fine_tune` f — `pinneapple_adaptation` (v04)
- `finetune` f — `pinneapple_flow.nodes.surrogates` (v07)
- `finetune` f — `pinneapple_hub.zoo` (v07)
- `finetune_brax` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_ceviche` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_chgnet` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_deepmd_kit` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_deepwave` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_deepxde` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_devito` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_diffrax` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_dsgp4` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_jax_cfd` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_jax_md` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_lora` f — `pinneapple_llm.finetune` (v07)
- `finetune_mace` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_ml4gw` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_mujoco_mjx` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_nequip` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_netket` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_neuralfoil` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_neuralgcm` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_neuraloperator` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_phiflow` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_photontorch` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_physicsnemo` f — `pinneapple_arena.external.finetune` (v04)
- `finetune_pina` f — `pinneapple_arena.external.finetune` (v04)

- finetune_pinnicle f — pinneapple_arena.external.finetune (v04)
- finetune_pybamm f — pinneapple_arena.external.finetune (v04)
- finetune_pyroomacoustics f — pinneapple_arena.external.finetune (v04)
- finetune_pysindy f — pinneapple_arena.external.finetune (v04)
- finetune_reservoirflow f — pinneapple_arena.external.finetune (v04)
- finetune_schnetpack f — pinneapple_arena.external.finetune (v04)
- finetune_sciann f — pinneapple_arena.external.finetune (v04)
- finetune_socialforce f — pinneapple_arena.external.finetune (v04)
- finetune_taichi f — pinneapple_arena.external.finetune (v04)
- finetune_tensorcircuit_ng f — pinneapple_arena.external.finetune (v04)
- finetune_torax f — pinneapple_arena.external.finetune (v04)
- finetune_warp f — pinneapple_arena.external.finetune (v04)
- FinetuneConfig C — pinneapple_llm.finetune (v07)
- FinetuneRejectedError C — pinneapple_hub.zoo (v07)
- FinetuneResult C — pinneapple_hub.zoo (v07)
- finite f — pinneapple_analysis.benchmark.builtin.fields (v05)
- finite_difference_check f — pinneapple_design.topology.driver (v06)
- finite_differences f — pinneapple_systems.sports.football.kinematics (v06)
- finite_solenoid_on_axis_B f — pinneapple_simulation.electromagnetics.statics (v03)
- first_acf_zero f — pinneapple_systems.time_series.predictability (v06)
- first_cell_height f — pinneapple_physics.closed_form.pipe_flow (v02)
- first_crossing f — pinneapple_physics.weather.evaluate (v02)
- first_order_given_data f — pinneapple_analysis.sensitivity.sobol (v05)
- fisher_identifiability f — pinneapple_systems.sports.football.ball (v06)
- FisherKPP1D C — pinneapple_arena.problems (v04)
- fit f — pinneapple_flow.learn (v07)
- fit f — pinneapple_hub.zoo._models (v07)
- fit f — pinneapple_neural.lpm (v04)
- fit_and_optimize f — pinneapple_data.process_optimizer (v04)
- fit_ar f — pinneapple_systems.time_series.predictability (v06)
- fit_ball_travel f — pinneapple_systems.sports.football.pitch_control (v06)
- fit_candidate f — pinneapple_analysis.surrogate.methods (v05)
- fit_controlled_opinf f — pinneapple_systems.operations.rom (v06)
- fit_crop f — pinneapple_lab.showcase (v07)
- fit_cst f — pinneapple_design.aero.geometry (v06)
- fit_discrepancy_model f — pinneapple_analysis.verification.causal_discrepancy (v05)
- fit_flight f — pinneapple_systems.sports.football.ball (v06)
- fit_fno f — pinneapple_simulation.electromagnetics.periodic.benchmark (v03)
- fit_intermediate f — pinneapple_systems.time_series.enso.forecasters (v06)

- `fit_inverse` f — `pinneapple_simulation.numerical_solvers.gated_xtfc` (v03)
- `fit_iron_loss` f — `pinneapple_simulation.electromagnetics.machines.steel` (v03)
- `fit_line_tls` f — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `fit_line_with_errors` f — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `fit_mass_model` f — `pinneapple_design.bricks.parts` (v06)
- `fit_models` f — `pinneapple_systems.sports.football.pc_study` (v06)
- `fit_noise` f — `pinneapple_systems.sports.football.kinematics` (v06)
- `fit_params` f — `pinneapple_systems.sports.football.pitch_control` (v06)
- `fit_power_law` f — `pinneapple_analysis.validation.spectral` (v05)
- `fit_quadratic` f — `pinneapple_physics.closed_form.pump_npsh` (v02)
- `fit_recession` f — `pinneapple_systems.digital_twin.recession` (v06)
- `fit_recharge` f — `pinneapple_systems.time_series.enso.oscillators` (v06)
- `fit_reduced_model` f — `pinneapple_neural.architectures.rom.study` (v04)
- `fit_scale_conditioned` f — `pinneapple_physics.multiscale.learned` (v02)
- `fit_static_weights` f — `pinneapple_physics.ensemble` (v02)
- `fit_supervised` f — `pinneapple_neural.utils` (v04)
- `fit_supervised` f — `pinneapple_physics.multiscale.learned` (v02)
- `fit_surrogate` f — `pinneapple_design.surrogate_inverse.core` (v06)
- `fit_surrogate` f — `pinneapple_simulation.pipelines.freecad_openfoam.sweep` (v03)
- `fit_surrogates` f — `pinneapple_simulation.electromagnetics.reflector.study` (v03)
- `fit_two_layer` f — `pinneapple_simulation.geophysics.energy_balance` (v03)
- `fit_zone` f — `pinneapple_twin3d.wear.forecast` (v06)
- `FitMeshSession` C — `pinneapple_design.fitmesh.session` (v06)
- `FitResult` C — `pinneapple_analysis.inverse_problems.estimation` (v05)
- `Fitted` C — `pinneapple_analysis.surrogate.methods` (v05)
- `FitzHughNagumo1D` C — `pinneapple_arena.problems` (v04)
- `fix_winding` f — `pinneapple_design.geometry_library.meshops` (v06)
- `fixed_base_forward` f — `pinneapple_simulation.multibody.arm` (v03)
- `fixed_base_inverse_dynamics` f — `pinneapple_simulation.multibody.arm` (v03)
- `fixed_policy` f — `pinneapple_twin3d.wear.optimize` (v06)
- `FixedShare` C — `pinneapple_physics.online_learning` (v02)
- `FixedShareGrid` C — `pinneapple_physics.online_learning` (v02)
- `fixtures` f — `pinneapple_hub.zoo.web` (v07)
- `Flag` C — `pinneapple_lab.results_rules` (v07)

- `flags` f — `pinneapple_analysis.validation.fault_injection.checks` (v05)
- `flagship_mission` f — `pinneapple_autolab.domains.racecar` (v07)
- `flanged_tube` f — `pinneapple_design.surrogate_inverse.meshes` (v06)
- `flap_effectiveness` f — `pinneapple_simulation.flight.aero` (v03)
- `flat_disk` f — `pinneapple_simulation.electromagnetics.reflector.geometry` (v03)
- `flat_metrics` f — `pinneapple_physics.tracking` (v02)
- `flat_params` f — `pinneapple_physics.tracking` (v02)
- `flat_site` f — `pinneapple_simulation.vehicle.scenarios` (v03)
- `flatten` f — `pinneapple_flow.nodes_ext.examples` (v07)
- `flatten_graph_batch` f — `pinneapple_neural.architectures.graphnn.utils` (v04)
- `Flight` C — `pinneapple_systems.sports.football.ball` (v06)
- `flight_scene` f — `pinneapple_simulation.flight.visuals` (v03)
- `FlightFit` C — `pinneapple_systems.sports.football.ball` (v06)
- `FlightHistory` C — `pinneapple_simulation.flight.simulate` (v03)
- `flights_3d` f — `pinneapple_systems.sports.football.plots` (v06)
- `flights_from_skillcorner` f — `pinneapple_systems.sports.football.ball_study` (v06)
- `Flow` C — `pinneapple_flow.schema` (v07)
- `flow` f — `pinneapple_hub.library.snippets` (v07)
- `flow_benchmark` f — `pinneapple_orchestration.flows.flow_03_benchmark` (v01)
- `flow_coefficient` f — `pinneapple_systems.process_components.similarity_map` (v06)
- `flow_data_generation` f — `pinneapple_orchestration.flows.flow_01_data_generation` (v01)
- `flow_digital_twin_prep` f — `pinneapple_orchestration.flows.flow_05_digital_twin_prep` (v01)
- `flow_directions` f — `pinneapple_simulation.numerical_solvers.external_flow` (v03)
- `flow_for_node` f — `pinneapple_hub.library.flowlink` (v07)
- `flow_hash` f — `pinneapple_hub.library.flowruns` (v07)
- `flow_json_schema` f — `pinneapple_flow.schema` (v07)
- `flow_link` f — `pinneapple_hub.library.flowlink` (v07)
- `flow_path` f — `pinneapple_flow.examples` (v07)
- `flow_registry_publish` f — `pinneapple_orchestration.flows.flow_04_registry_publish` (v01)
- `flow_run_manifest` f — `pinneapple_orchestration.common.provenance` (v01)
- `flow_scene` f — `pinneapple_autolab.domains.racecar` (v07)
- `flow_training` f — `pinneapple_orchestration.flows.flow_02_training` (v01)
- `FlowBuilder` C — `pinneapple_flow.planner.build` (v07)
- `FlowError` C — `pinneapple_flow.schema` (v07)
- `FlowLaw` C — `pinneapple_security.cps` (v05)
- `FlowLinkError` C — `pinneapple_hub.library.flowlink` (v07)
- `FlowNodeAdapter` C — `pinneapple_flow.runtime` (v07)
- `FlowObstacle2DTask` C — `pinneapple_tools.benchmark_suite.tasks.flow_obstacle_2d` (v07)
- `FlowResult` C — `pinneapple_simulation.numerical_solvers.external_flow` (v03)
- `FlowRunManifest` C — `pinneapple_orchestration.common.provenance` (v01)

- `FlowRunResult` C — `pinneapple_flow.runtime` (v07)
- `flows` f — `pinneapple_flow.examples` (v07)
- `flows` f — `pinneapple_hub.library.sources` (v07)
- `FlowService` C — `pinneapple_flow.service` (v07)
- `FlowStore` C — `pinneapple_flow.store` (v07)
- `FlowVisualizer` C — `pinneapple_neural.predictor.postprocess` (v04)
- `fluent_mesh_to_upd` f — `pinneapple_simulation.external_solvers.cfd_formats.fluent_mesh_reader` (v03)
- `FluentCaseConfig` C — `pinneapple_simulation.external_solvers.ansys.journal_builder` (v03)
- `FluentRunConfig` C — `pinneapple_simulation.external_solvers.ansys.runner` (v03)
- `FluentRunResult` C — `pinneapple_simulation.external_solvers.ansys.runner` (v03)
- `Fluid` C — `pinneapple_simulation.particle_dynamics.geo_mpm` (v03)
- `fluidx3d_available` f — `pinneapple_simulation.external_solvers.fluidx3d` (v03)
- `fluidx3d_home` f — `pinneapple_simulation.external_solvers.fluidx3d` (v03)
- `flux` f — `pinneapple_core.operators` (v01)
- `flux_density` f — `pinneapple_simulation.electromagnetics.fem` (v03)
- `flux_density_ellipses` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `flux_linkage` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `flux_magnitude` f — `pinneapple_physics.darcy_checks` (v02)
- `flux_through_path` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `FluxMap` C — `pinneapple_simulation.electromagnetics.machines.drive` (v03)
- `fluxmap_dataset` f — `pinneapple_simulation.electromagnetics.machines.interop` (v03)
- `fmt` f — `pinneapple_check.checks.common` (v05)
- `fmt` f — `pinneapple_reason.modes._util` (v05)
- `fmt` f — `pinneapple_tools.reporting.paperkit` (v07)
- `fmt` f — `pinneapple_tools.visualization.studio.colormap` (v07)
- `fmt_s` f — `pinneapple_analysis.benchmark.report` (v05)
- `fmu_simulate` f — `pinneapple_flow.wrappers` (v07)
- `fmu_sweep` f — `pinneapple_simulation.external_solvers.modelica.sweep` (v03)
- `fmu_to_upd` f — `pinneapple_simulation.external_solvers.modelica.fmi_runner` (v03)
- `FMUDataset` C — `pinneapple_simulation.external_solvers.modelica.sweep` (v03)
- `FMUReport` C — `pinneapple_simulation.external_solvers.modelica.fmu_inspect` (v03)
- `FMUSimConfig` C — `pinneapple_simulation.external_solvers.modelica.fmi_runner` (v03)
- `FnAdapter` C — `pinneapple_neural.trainer.datamodule` (v04)

- FNO2d C — pinneapple_neural.architectures.neural_operators.fno (v04)
- FNO3d C — pinneapple_neural.architectures.neural_operators.fno (v04)
- FNOForecastConfig C — pinneapple_systems.time_series.models.fno_forecaster (v06)
- FNOForecaster C — pinneapple_systems.time_series.models.fno_forecaster (v06)
- FNOMethod C — pinneapple_analysis.benchmark.builtin.fields (v05)
- foam f — pinneapple_check._reuse (v05)
- FoamFileHeader C — pinneapple_simulation.external_solvers.openfoam.binary_reader (v03)
- folder_of f — pinneapple_science.report (v02)
- follow_camera f — pinneapple_simulation.vehicle.viz (v03)
- football_ball_flight_inverse f — pinneapple_flow.nodes_ext.football (v07)
- football_pass_pitch_control f — pinneapple_flow.nodes_ext.football (v07)
- football_tracking_smoothing f — pinneapple_flow.nodes_ext.football (v07)
- footbridge f — pinneapple_reason.cases (v05)
- footprint f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- footprint_area f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- force_convergence f — pinneapple_autolab.domains.racecar (v07)
- force_to_coefficients f — pinneapple_tools.aero_coefficients (v07)
- forced_convection f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- forced_convection f — pinneapple_physics.physics_audit.assumptions (v02)
- forced_ocean_run f — pinneapple_systems.time_series.enso.intermediate (v06)
- forces f — pinneapple_design.aero.case (v06)
- Forecast C — pinneapple_systems.time_series.forecast_lab.base (v06)
- forecast f — pinneapple_flow.nodes.systems (v07)
- forecast f — pinneapple_physics.astro.spaceweather (v02)
- forecast f — pinneapple_systems (v06)
- forecast f — pinneapple_systems.digital_twin.recession (v06)
- forecast f — pinneapple_twin3d.wear.forecast (v06)
- forecast_case f — pinneapple_flow.nodes_ext.forecasting (v07)
- forecast_gif f — pinneapple_physics.weather.viz (v02)
- forecast_history f — pinneapple_twin3d.wear.forecast (v06)
- forecast_origins f — pinneapple_systems.time_series.forecast_lab.base (v06)
- Forecaster C — pinneapple_physics.weather.train (v02)
- Forecaster C — pinneapple_systems.time_series.forecast_lab.base (v06)
- ForecastModel C — pinneapple_systems.time_series.models.classical (v06)

- ForecastProblemSpec C — pinneapple_systems.time_series.problem (v06)
- form_values f — pinneapple_data.formfill.fill (v04)
- format_card f — pinneapple_core.onboarding.catalog (v01)
- format_reference f — pinneapple_flow.references (v07)
- format_results f — pinneapple_llm.workbench.evaluate (v07)
- format_template f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- FormatError C — pinneapple_data.cae.io (v04)
- FormSpec C — pinneapple_data.formfill.spec (v04)
- FormulaCar C — pinneapple_design.geometry.vehicles3d (v06)
- formulate f — pinneapple_design.intent.parse (v06)
- formulate_and_recommend f — pinneapple_veriphysics.recommend (v05)
- forward f — pinneapple_flow.nodes_ext.architectures (v07)
- forward_acceleration f — pinneapple_systems.sports.football.kinematics (v06)
- forward_points f — pinneapple_worldmodel.robot_perception (v04)
- ForwardModel C — pinneapple_analysis.inverse_problems.forward (v05)
- fouling_at f — pinneapple_systems.operations.exchanger (v06)
- FoundationConfig C — pinneapple_worldmodel.mega_model (v04)
- FoundationFNOBlock C — pinneapple_worldmodel.mega_model (v04)
- fourier_basis f — pinneapple_physics.multiscale.filters (v02)
- fourier_coefficients f — pinneapple_physics.multiscale.filters (v02)
- fourier_features f — pinneapple_systems.time_series.features.engineering (v06)
- fourier_pad f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- fourier_wavenumbers f — pinneapple_simulation.numerical_solvers.etdrk4 (v03)
- FourierEncoding C — pinneapple_neural.lpm (v04)
- FourierFeatureEmbedding C — pinneapple_neural.architectures.modified_mlp (v04)
- FourierFeatures C — pinneapple_core.transforms (v01)
- FourierFeatures C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- FourierGalerkinPINN C — pinneapple_physics.multiscale.learned (v02)
- FourierInterpolant C — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- FourierMLP C — pinneapple_systems.time_series.enso.pinn (v06)
- FourierNeuralOperator C — pinneapple_neural.architectures.neural_operators.fno (v04)
- FourierTimeFeatures C — pinneapple_neural.architectures.graphnn.gnn_ode (v04)
- fractal_relief f — pinneapple_simulation.terramechanics.terrain (v03)
- fractional_factorial_unit f — pinneapple_data.doe.designs (v04)
- Frame C — pinneapple_design.bricks.visuals (v06)
- frame_bounds f — pinneapple_design.bricks.visuals (v06)
- frame_geometry f — pinneapple_design.bricks.visuals (v06)
- frame_indices f — pinneapple_simulation.flight.visuals (v03)

- `frame_view` f — `pinneapple_simulation.vehicle.viz` (v03)
- `frames_to_samples` f — `pinneapple_simulation.external_solvers.geotaichi` (v03)
- `frames_to_video` f — `pinneapple_blender.cycles` (v07)
- `FrdResults` C — `pinneapple_simulation.external_solvers.calculix.frd` (v03)
- `free_fall` f — `pinneapple_simulation.external_solvers.newton_physics.validation` (v03)
- `free_geometry` f — `pinneapple_design.fitmesh.llm` (v06)
- `free_space_wavelength` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `FreeCADAdapter` C — `pinneapple_tools.automation.adapters.freecad` (v07)
- `FreeCADBackend` C — `pinneapple_simulation.pipelines.freecad_openfoam.cad` (v03)
- `FreeCADHeadlessAdapter` C — `pinneapple_tools.automation.adapters.pipeline` (v07)
- `FreeSurfaceFlow` C — `pinneapple_simulation.numerical_solvers.free_surface` (v03)
- `freeze` f — `pinneapple_design.surrogate_inverse.core` (v06)
- `freeze_all_except` f — `pinneapple_adaptation.transfer_learning.freeze` (v04)
- `freeze_geometry_features` f — `pinneapple_systems.sports.football.xg` (v06)
- `freeze_layers` f — `pinneapple_adaptation.transfer_learning.freeze` (v04)
- `freeze_recommendation` f — `pinneapple_analysis.sensitivity.effects` (v05)
- `Frequency` C — `pinneapple_simulation.external_solvers.calculix.study` (v03)
- `frequency` f — `pinneapple_reason.modes.structural` (v05)
- `frequency_grid` f — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `frequency_scene` f — `pinneapple_simulation.electromagnetics.antenna.scene3d` (v03)
- `FrequencyMean` C — `pinneapple_simulation.electromagnetics.antenna.surrogates` (v03)
- `freshness_score` f — `pinneapple_tools.hpo_experiments.ranking` (v07)
- `fresnel` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `FresnelSnell` C — `pinneapple_science.em.waves` (v02)
- `friction_creep` f — `pinneapple_design.bricks.dynamics` (v06)
- `friction_drag` f — `pinneapple_design.aero.buildup` (v06)
- `friction_factor` f — `pinneapple_physics.closed_form.pipe_flow` (v02)
- `FrohlichKennelly` C — `pinneapple_simulation.electromagnetics.hysteresis` (v03)
- `from_arrays` f — `pinneapple_analysis.surrogate.data` (v05)
- `from_assembly` f — `pinneapple_design.bricks.statics` (v06)
- `from_b64url` f — `pinneapple_hub.library.flowlink` (v07)
- `from_callable` f — `pinneapple_physics.ensemble` (v02)
- `from_comparison` f — `pinneapple_lab.manifest` (v07)
- `from_csr` f — `pinneapple_simulation.accelerated_foam.ldu` (v03)

- from_dbi f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- from_dict f — pinneapple_data.contracts.models (v04)
- from_flow_run f — pinneapple_lab.manifest (v07)
- from_intent f — pinneapple_flow.nodes_ext.design (v07)
- from_ldr f — pinneapple_design.bricks.ldraw (v06)
- from_mat f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- from_model_ref f — pinneapple_design.design_optimizer.evaluators (v06)
- from_pretrained f — pinneapple_hub.hub (v07)
- from_pyleecan f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- from_radec f — pinneapple_physics.astro.frames (v02)
- from_results f — pinneapple_lab.manifest (v07)
- from_run_dir f — pinneapple_lab.manifest (v07)
- from_si f — pinneapple_design.intent.spec (v06)
- from_snapshots f — pinneapple_analysis.surrogate.data (v05)
- from_solution f — pinneapple_lab.manifest (v07)
- from_solution f — pinneapple_physics.ensemble (v02)
- from_table f — pinneapple_analysis.surrogate.data (v05)
- from_text f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- from_torch f — pinneapple_physics.ensemble (v02)
- from_trust_score f — pinneapple_analysis.trust.trust_report (v05)
- from_workflow_run f — pinneapple_lab.manifest (v07)
- front_matter f — pinneapple_hub.zoo.hub_cards (v07)
- frontier_figure f — pinneapple_physics.multiscale.comparison (v02)
- FrontierPoint C — pinneapple_design.surrogate_inverse.reflex_port (v06)
- froude f — pinneapple_physics.physics_audit.regime (v02)
- frozen_ic_rmse f — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- FrozenField3D C — pinneapple_simulation.flight.turbulence (v03)
- FrozenLine C — pinneapple_simulation.flight.turbulence (v03)
- FSDPConfig C — pinneapple_neural.trainer.hpc (v04)
- FullResult C — pinneapple_design.intent.template (v06)
- FunctionEvaluator C — pinneapple_design.design_optimizer.study (v06)
- FunctionField C — pinneapple_core.field (v01)
- FunctionRef C — pinneapple_tools.rea_bridge.evidence (v07)
- FunctionSimulator C — pinneapple_autolab.simulator (v07)
- fundamental_args f — pinneapple_physics.astro.frames (v02)
- furnace_combustion_zone f — pinneapple_physics.pde_environment.presets.engineering (v02)
- fuselage_cd0 f — pinneapple_design.aero.aircraft3d (v06)
- fuselage_moments f — pinneapple_simulation.flight.aero (v03)
- fuselage_munk_factor f — pinneapple_simulation.flight.aero (v03)
- fuselage_wetted f — pinneapple_design.aero.aircraft3d (v06)
- FV C — pinneapple_check.fv (v05)
- fv_conduction f — pinneapple_condense.meshless2d (v02)

- fv_field f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- fv_geometry f — pinneapple_check.fields (v05)
- fv_heat_balance f — pinneapple_check.fv (v05)
- fv_mass_balance f — pinneapple_check.fv (v05)
- fv_metrics f — pinneapple_data.cae.quality (v04)
- fv_residual f — pinneapple_analysis.validation.physics_learning (v05)
- fv_residuals f — pinneapple_check.fv (v05)
- FVMSolver C — pinneapple_simulation.numerical_solvers.fvm (v03)

G

- gain_from_q f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- galactic f — pinneapple_physics.astro.journey (v02)
- galerkin f — pinneapple_simulation.accelerated_foam.warm_start (v03)
- GalerkinNeuralOperator C — pinneapple_neural.architectures.neural_operators.gno (v04)
- GalleryResult C — pinneapple_analysis.validation.gallery (v05)
- Gap C — pinneapple_lab.next_steps (v07)
- Gap C — pinneapple_problemdesign.schema (v02)
- Gardner C — pinneapple_simulation.geophysics.richards (v03)
- gardner_steady_infiltration f — pinneapple_simulation.geophysics.richards (v03)
- GasComposition C — pinneapple_systems.process_components.real_gas_eos (v06)
- gaspari_cohn f — pinneapple_analysis.state_estimation.kalman (v05)
- GasState C — pinneapple_systems.process_components.real_gas_eos (v06)
- gast_hours f — pinneapple_physics.astro.timescales (v02)
- gast_rad f — pinneapple_physics.astro.timescales (v02)
- gated_layout f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- GatedLayout C — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- GatedSolution C — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- gateway_router f — pinneapple_registry.gateway_api (v01)
- GatewayClient C — pinneapple_registry.gateway_client (v01)
- GatewayError C — pinneapple_registry.gateway_client (v01)
- gather f — pinneapple_lab.reports (v07)
- gauss_electric f — pinneapple_physics.maxwell_checks (v02)
- gauss_legendre_quadrature f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- gauss_lobatto_legendre_points f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- gauss_magnetic f — pinneapple_physics.maxwell_checks (v02)
- gaussian_curvature f — pinneapple_design.geometry.ops.curvature (v06)
- gaussian_interval_coverage f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)

- gaussian_mechanism f — pinneapple_security.dp (v05)
- gaussian_nll_diag f — pinneapple_neural.architectures.continuous.deep_state_space (v04)
- gaussian_pulse f — pinneapple_simulation.electromagnetics.fdtd (v03)
- gaussian_sigma f — pinneapple_security.dp (v05)
- GaussianMisfit C — pinneapple_analysis.inverse_problems.noise_models (v05)
- GaussianProcessField C — pinneapple_analysis.data_assimilation.sensor_placement (v05)
- GaussianProcessSurrogate C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- GaussLaw C — pinneapple_science.em.statics (v02)
- GBMForecaster C — pinneapple_systems.time_series.forecast_lab.learned (v06)
- gci f — pinneapple_design.fitmesh.report (v06)
- gci f — pinneapple_lab.uq (v07)
- gci_check f — pinneapple_analysis.benchmark.validation (v05)
- gci_check f — pinneapple_lab.uq (v07)
- gear_forces f — pinneapple_simulation.flight.gear (v03)
- GearLeg C — pinneapple_simulation.flight.gear (v03)
- GeminiProvider C — pinneapple_problemdesign.protocol (v02)
- GeminiProvider C — pinneapple_tools.hpo_experiments.providers.gemini_provider (v07)
- GeminiProviderConfig C — pinneapple_tools.hpo_experiments.providers.gemini_provider (v07)
- generalization_section f — pinneapple_app.backend.core.surrogate_report (v07)
- generalized_reynolds_number f — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- generate f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- generate f — pinneapple_design.fitmesh.mesh_gen (v06)
- generate f — pinneapple_flow.autocalls (v07)
- generate f — pinneapple_simulation.numerical_solvers.cylinder_wake (v03)
- generate f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- generate f — pinneapple_simulation.numerical_solvers.thermal_channel_datasets (v03)
- generate f — pinneapple_worldmodel.generate_datasets (v04)
- generate_1d_mesh f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- generate_acoustic_synthetic f — pinneapple_analysis.inspection.acoustic (v05)
- generate_artifact f — pinneapple_llm.qa.generation (v07)
- generate_case f — pinneapple_simulation.numerical_solvers.openfoam_bridge (v03)
- generate_chemical_mixing f — pinneapple_worldmodel.synthetic_factory (v04)

- `generate_cylinder_flow` f — `pinneapple_worldmodel.synthetic_factory` (v04)
- `generate_data` f — `pinneapple_simulation` (v03)
- `generate_dataset` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `generate_dataset` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `generate_dataset` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `generate_dataset` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `generate_dataset` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `generate_dataset` f — `pinneapple_design.thermal.heatsink.surrogate` (v06)
- `generate_eddy_current_synthetic` f — `pinneapple_analysis.inspection.eddy_current` (v05)
- `generate_emat_synthetic` f — `pinneapple_analysis.inspection.emat` (v05)
- `generate_guided_wave_synthetic` f — `pinneapple_analysis.inspection.guided_wave_ultrasonic` (v05)
- `generate_heat_transfer` f — `pinneapple_worldmodel.synthetic_factory` (v04)
- `generate_history` f — `pinneapple_systems.operations.exchanger` (v06)
- `generate_key` f — `pinneapple_security.signing` (v05)
- `generate_magnetic_flux_leakage_synthetic` f — `pinneapple_analysis.inspection.magnetic_flux_leakage` (v05)
- `generate_phased_array_synthetic` f — `pinneapple_analysis.inspection.phased_array_ultrasonic` (v05)
- `generate_pinn_dataset` f — `pinneapple_simulation.numerical_solvers.problem_runner` (v03)
- `generate_pipe_flow` f — `pinneapple_worldmodel.synthetic_factory` (v04)
- `generate_points` f — `pinneapple_app.backend.core.collocation` (v07)
- `generate_scenarios_task` f — `pinneapple_orchestration.tasks.data_generation_tasks` (v01)
- `generate_sequences` f — `pinneapple_neural.architectures.rom.toy_systems` (v04)
- `generate_synthetic_csv` f — `pinneapple_systems.time_series.csv_loader` (v06)
- `generate_via_factory` f — `pinneapple_worldmodel.generate_datasets` (v04)
- `GeneratedArtifact` C — `pinneapple_llm.qa.generation` (v07)
- `Generator` C — `pinneapple_analysis.validation.arena.generators` (v05)
- `generators_for` f — `pinneapple_analysis.validation.arena.generators` (v05)
- `generic` f — `pinneapple_flow.nodes.problem` (v07)
- `GenericFourierMLP` C — `pinneapple_neural.architectures.benchmarks.generic_pinn_models` (v04)
- `GenericLinear` C — `pinneapple_neural.architectures.benchmarks.generic_pinn_models` (v04)

- GenericMLP C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- GenericResMLP C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- GenericSIREN C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- genesis_traj_to_upd f — pinneapple_simulation.external_solvers.genesis.adapter (v03)
- genesis_trajs_to_upd f — pinneapple_simulation.external_solvers.genesis.adapter (v03)
- GenesisConfig C — pinneapple_simulation.external_solvers.genesis.scene_builder (v03)
- GenesisRunner C — pinneapple_simulation.external_solvers.genesis.runner (v03)
- GeneticSDF C — pinneapple_design.geometry.gen.genetic_sdf (v06)
- geo_l_bracket f — pinneapple_simulation.external_solvers.calculix.study (v03)
- geo_plate_with_hole f — pinneapple_simulation.external_solvers.calculix.study (v03)
- geo_shelf_bracket f — pinneapple_design.intent.physics.bracket (v06)
- geodetic_to_itrs f — pinneapple_physics.astro.frames (v02)
- geomagnetic_latitude f — pinneapple_physics.astro.spaceweather (v02)
- geometric_features f — pinneapple_systems.sports.football.pitch_control (v06)
- GeometricConstraint C — pinneapple_design.design_optimizer.constraints (v06)
- geometries f — pinneapple_hub.library.sources (v07)
- Geometry C — pinneapple_condense.meshless (v02)
- Geometry C — pinneapple_core.geometry (v01)
- Geometry C — pinneapple_design.geometry_library.core (v06)
- Geometry C — pinneapple_simulation.electromagnetics.fem (v03)
- geometry f — pinneapple_flow.nodes.cfd_pipeline (v07)
- geometry f — pinneapple_hub.library.snippets (v07)
- geometry_checks f — pinneapple_tools.nvidia.brep (v07)
- geometry_doc f — pinneapple_flow.nodes.problem (v07)
- geometry_errors f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- geometry_errors f — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- geometry_export f — pinneapple_flow.wrappers (v07)
- geometry_figure f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- GeometryAsset C — pinneapple_design.geometry.core.geometry (v06)
- GeometryBase C — pinneapple_worldmodel.geometry (v04)
- GeometryClassification C — pinneapple_analysis.verification.geometry_intelligence (v05)
- GeometryDraftResult C — pinneapple_llm.geometry_draft (v07)
- GeometryEncoder C — pinneapple_neural.lpm (v04)
- GeometryIndex C — pinneapple_design.geometry.retrieval (v06)

- GeometryLoadOptions C — pinneapple_data.adapters.geom_adapter (v04)
- GeometryOODGuardrail C — pinneapple_analysis.verification.geometry_ood_guardrail (v05)
- GeometryOODResult C — pinneapple_analysis.verification.geometry_ood_guardrail (v05)
- GeometryOptimizer C — pinneapple_design.geometry.optimize.loop (v06)
- GeometryPreset C — pinneapple_arena.pointwise.schema (v04)
- GeometrySpec C — pinneapple_design.geometry.core.geometry (v06)
- GeometrySpec C — pinneapple_design.geometry_library.core (v06)
- GeometrySpec C — pinneapple_llm.qa.spec (v07)
- GeometrySpec C — pinneapple_problemdesign.schema (v02)
- GeometrySynthGenerator C — pinneapple_data.synth.geometry (v04)
- GeometryType C — pinneapple_arena.pointwise.schema (v04)
- GeoMPM C — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- geotaichi_available f — pinneapple_simulation.external_solvers.geotaichi (v03)
- geotaichi_home f — pinneapple_simulation.external_solvers.geotaichi (v03)
- germano_error f — pinneapple_physics.multiscale.closures (v02)
- get f — pinneapple_design.intent.materials (v06)
- get f — pinneapple_hub.library (v07)
- get f — pinneapple_lab.spec (v07)
- get f — pinneapple_llm.qa.validators.base (v07)
- get f — pinneapple_reason.cases (v05)
- get f — pinneapple_science.registry (v02)
- get f — pinneapple_twin3d.gallery (v06)
- get f — pinneapple_twin3d.wear.presets (v06)
- get_adapter f — pinneapple_core.workflows.adapters (v01)
- get_adapter f — pinneapple_tools.automation.adapters.base (v07)
- get_assumption f — pinneapple_physics.physics_audit.assumptions (v02)
- get_backend f — pinneapple_quantum.backends.backend (v04)
- get_backend f — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- get_backend f — pinneapple_tools.benchmark_suite.registry (v07)
- get_backend f — pinneapple_tools.compute_backends.backend (v07)
- get_benchmark f — pinneapple_pdb.benchmarks (v04)
- get_benchmark_preset f — pinneapple_arena.dataset_bench (v04)
- get_cmap f — pinneapple_tools.visualization.style (v07)
- get_config f — pinneapple_mcp.config (v07)
- get_connection_geometry f — pinneapple_design.geometry.gen.threaded_connection (v06)
- get_dataset_preview f — pinneapple_arena.external.runners.registry (v04)
- get_domain f — pinneapple_design.geometry.gen.domains (v06)
- get_domain_3d f — pinneapple_design.geometry.gen.domains3d (v06)
- get_entry f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- get_example_case f — pinneapple_veriphysics.example_cases (v05)
- get_executor f — pinneapple_core.workflows.executors (v01)

- get_fault f — pinneapple_analysis.validation.fault_injection.faults (v05)
- get_generator f — pinneapple_analysis.validation.arena.generators (v05)
- get_geometry_mesh_config f — pinneapple_arena.pointwise.geometries (v04)
- get_irk_tableau f — pinneapple_simulation.numerical_solvers.irk_gauss_legendre (v03)
- get_method f — pinneapple_analysis.benchmark.registry (v05)
- get_method f — pinneapple_catalog.methods (v01)
- get_model f — pinneapple_app.backend.routers.models (v07)
- get_model_info f — pinneapple_app.backend.core.model_catalog (v07)
- get_pde_family f — pinneapple_physics.pde_environment.capabilities (v02)
- get_physics_backend f — pinneapple_core.backend (v01)
- get_player_for_tool f — pinneapple_analysis.verification.tool_recommendation (v05)
- get_potential f — pinneapple_quantum.loss.schrodinger (v04)
- get_preset f — pinneapple_analysis.inverse_problems.presets (v05)
- get_preset f — pinneapple_physics.pde_environment.presets.registry (v02)
- get_preset f — pinneapple_tools.visualization.studio.lighting (v07)
- get_problem f — pinneapple_analysis.benchmark.registry (v05)
- get_problem f — pinneapple_analysis.validation.fault_injection.problems (v05)
- get_problem f — pinneapple_app.backend.routers.problems (v07)
- get_problem f — pinneapple_arena.problems (v04)
- get_problem f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- get_problem_meta f — pinneapple_app.backend.core.problem_registry (v07)
- get_rank f — pinneapple_neural.trainer.distributed (v04)
- get_rans_preset f — pinneapple_physics.pde_environment.turbulence_presets (v02)
- get_resource f — pinneapple_catalog.resources (v01)
- get_results f — pinneapple_app.backend.routers.experiments (v07)
- get_rule f — pinneapple_check.rules (v05)
- get_soil f — pinneapple_simulation.terramechanics.soils (v03)
- get_spec f — pinneapple_data.formfill.specs (v04)
- get_spec f — pinneapple_design.geometry_library.core (v06)
- get_status f — pinneapple_app.backend.routers.experiments (v07)
- get_task f — pinneapple_tools.benchmark_suite.registry (v07)
- get_template f — pinneapple_design.intent.parse (v06)
- get_tools_by_category f — pinneapple_analysis.verification.tool_recommendation (v05)
- get_turbulence_closure f — pinneapple_physics.pde_environment.turbulence_selector (v02)
- get_world_size f — pinneapple_neural.trainer.distributed (v04)
- getdp_crosscheck f — pinneapple_flow.nodes_ext.em_prius (v07)
- gill_analytic f — pinneapple_systems.time_series.enso.intermediate (v06)

- gill_test f — pinneapple_systems.time_series.enso.intermediate (v06)
- GillAtmosphere C — pinneapple_systems.time_series.enso.intermediate (v06)
- girder_loads f — pinneapple_reason.modes.structural (v05)
- git_commit f — pinneapple_security.attestation (v05)
- github f — pinneapple_check.report (v05)
- GithubFetcher C — pinneapple_tools.hpo_experiments.sources.github_fetcher (v07)
- GithubRepo C — pinneapple_tools.hpo_experiments.sources.github_source (v07)
- GithubSource C — pinneapple_tools.hpo_experiments.sources.github_source (v07)
- GitPublisher C — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- GliderWingTemplate C — pinneapple_design.intent.templates.glider_wing (v06)
- GlobalSensitivity C — pinneapple_analysis.inverse_problems.sensitivity (v05)
- GLU C — pinneapple_neural.architectures.transformers.tft (v04)
- gmm_fit f — pinneapple_physics.astro.stars_ml (v02)
- gmsh_exe f — pinneapple_design.intent.physics.bracket (v06)
- GmshGetDPError C — pinneapple_simulation.electromagnetics.machines.getdp (v03)
- gmst82 f — pinneapple_physics.astro.satellites (v02)
- gmst_rad f — pinneapple_physics.astro.timescales (v02)
- gnielinski f — pinneapple_physics.closed_form.heat_exchanger (v02)
- GNNAdapter C — pinneapple_neural.architectures.adapters.gnn (v04)
- go_aperture_taper_db f — pinneapple_simulation.electromagnetics.reflector.analytic (v03)
- go_check f — pinneapple_simulation.electromagnetics.reflector.compact_range (v03)
- Goal C — pinneapple_design.bricks.goals (v06)
- good_models f — pinneapple_analysis.validation.physics_learning_cases (v05)
- goodman_equivalent_amplitude f — pinneapple_systems.process_components.fatigue_analysis (v06)
- goodman_safety_ratio f — pinneapple_systems.process_components.fatigue_analysis (v06)
- gp_loo_scale f — pinneapple_science.numerics (v02)
- GPModel C — pinneapple_autolab.surrogates (v07)
- GPModel C — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- GPRForecaster C — pinneapple_systems.time_series.models.classical (v06)
- GPSurrogate C — pinneapple_design.design_optimizer.verify (v06)
- gpu_info f — pinneapple_neural.trainer.parallel (v04)
- grad f — pinneapple_core.func (v01)
- grad f — pinneapple_core.operators (v01)
- grad f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)

- GradAccumConfig C — pinneapple_neural.trainer.parallel (v04)
- GradAccumTrainer C — pinneapple_neural.trainer.parallel (v04)
- grade_sweep f — pinneapple_simulation.vehicle.scenarios (v03)
- grade_table f — pinneapple_simulation.rover.autolab.wheel_models (v03)
- graded_axis f — pinneapple_science.numerics (v02)
- GradeTable C — pinneapple_simulation.rover.autolab.wheel_models (v03)
- gradient_check f — pinneapple_analysis.data_assimilation.var4d (v05)
- gradient_fit f — pinneapple_analysis.inverse_problems.estimation (v05)
- gradient_metrics f — pinneapple_analysis.validation.spectral (v05)
- gradient_sensitivity f — pinneapple_tools.model_evaluation.metrics (v07)
- GradientDesignOptimizer C — pinneapple_design.design_optimizer.optimizer (v06)
- GradientModel C — pinneapple_physics.multiscale.closures (v02)
- GradNormBalancer C — pinneapple_neural.trainer.weight_scheduler (v04)
- graph_data f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- graph_data f — pinneapple_design.aero.train (v06)
- graph_stats f — pinneapple_analysis.verification.knowledge_graph (v05)
- GraphBatch C — pinneapple_neural.architectures.graphnn.base (v04)
- GraphCast C — pinneapple_neural.architectures.graphnn.graphcast (v04)
- GraphCatalog C — pinneapple_neural.architectures.graphnn.registry (v04)
- GraphGuessNet C — pinneapple_simulation.accelerated_foam.learned (v03)
- GraphModelBase C — pinneapple_neural.architectures.graphnn.base (v04)
- GraphNeuralNetwork C — pinneapple_neural.architectures.graphnn.gnn (v04)
- GraphNeuralODE C — pinneapple_neural.architectures.graphnn.gnn_ode (v04)
- GraphOutput C — pinneapple_neural.architectures.graphnn.base (v04)
- grashof f — pinneapple_physics.physics_audit.regime (v02)
- gravity_audit f — pinneapple_systems.sports.football.ball_study (v06)
- gravity_audit_figure f — pinneapple_systems.sports.football.plots (v06)
- gravity_body f — pinneapple_simulation.flight.rigid_body (v03)
- GrayBoxConfig C — pinneapple_neural.trainer.graybox (v04)
- GrayBoxNet C — pinneapple_neural.trainer.graybox (v04)
- greedy_maximin f — pinneapple_data.doe.designs (v04)
- greedy_sensor_placement f — pinneapple_analysis.data_assimilation.sensor_placement (v05)
- grf_burgers f — pinneapple_flow.nodes.data (v07)
- grf_direction f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- GRFDraw C — pinneapple_data.grf_sampler (v04)

- GRFDraw2D C — pinneapple_data.grf_sampler (v04)
- Grid C — pinneapple_condense.flow (v02)
- Grid C — pinneapple_core.field (v01)
- grid f — pinneapple_simulation.numerical_solvers.xtfc_general (v03)
- grid_coords f — pinneapple_analysis.surrogate.data (v05)
- grid_derivative f — pinneapple_physics.symbolic_pde.gradient_backends (v02)
- grid_force f — pinneapple_check.balances (v05)
- grid_from_arrays f — pinneapple_tools.visualization.pyvista_bridge (v07)
- grid_graph f — pinneapple_neural.physicsnemo_bridge.field_operators (v04)
- grid_heat f — pinneapple_check.balances (v05)
- grid_layout f — pinneapple_arena.trainer (v04)
- grid_mass f — pinneapple_check.balances (v05)
- grid_normals f — pinneapple_tools.visualization.studio.shadows (v07)
- grid_operator_service f — pinneapple_tools.model_services.grid (v07)
- grid_points f — pinneapple_design.aero.mesh (v06)
- grid_points f — pinneapple_flow.nodes._common (v07)
- grid_surface f — pinneapple_tools.nvidia.omniverse (v07)
- GridAxisSpec C — pinneapple_physics.symbolic_pde.gradient_backends (v02)
- GridCase C — pinneapple_physics.paper_problems (v02)
- GridD C — pinneapple_check.numerics (v05)
- Gridded C — pinneapple_systems.time_series.enso.data (v06)
- GridFieldPredictor C — pinneapple_neural.physicsnemo_bridge.field_operators (v04)
- GridSpec C — pinneapple_design.aero.mesh (v06)
- GridSpec C — pinneapple_neural.architectures.neural_operators.pino (v04)
- GRN C — pinneapple_neural.architectures.transformers.tft (v04)
- Ground C — pinneapple_design.bricks.assembly (v06)
- ground_equilibrium f — pinneapple_simulation.flight.trim (v03)
- GroundTruthPackager C — pinneapple_worldmodel.physics_solver_bridge (v04)
- group f — pinneapple_lab.next_steps (v07)
- group_bootstrap f — pinneapple_analysis.uncertainty.resampling (v05)
- GroupBCatalog C — pinneapple_neural.architectures.group_b_registry (v04)
- GroupedMLP C — pinneapple_systems.time_series.enso.pinn (v06)
- GroupVSN C — pinneapple_neural.architectures.transformers.tft (v04)
- Grousers C — pinneapple_simulation.terramechanics.wheel (v03)
- grow_region f — pinneapple_analysis.validation.error_map (v05)
- GRUForecaster C — pinneapple_systems.time_series.models.recurrent (v06)
- GRUModel C — pinneapple_neural.architectures.recurrent.gru (v04)
- guardrail_checks f — pinneapple_check.evidence (v05)
- guardrail_checks_as_dicts f — pinneapple_analysis.verification.pipeline (v05)
- GuardrailReport C — pinneapple_llm.guardrail (v07)

- GuessServer C — pinneapple_simulation.accelerated_foam.guess_server (v03)
- GuiAgentStep C — pinneapple_tools.automation.runner_step (v07)
- GuidanceBase C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- GuiUnavailable C — pinneapple_tools.automation.display (v07)
- gunzip f — pinneapple_security.archives (v05)
- gust_sensitivity f — pinneapple_simulation.flight.encounter (v03)
- gust_sensitivity_figure f — pinneapple_simulation.flight.figures (v03)
- gust_wisps f — pinneapple_simulation.flight.visuals (v03)

H

- haaland f — pinneapple_physics.closed_form.pipe_flow (v02)
- half_wave_dipole_directivity f — pinneapple_simulation.electromagnetics.waves (v03)
- half_wave_dipole_radiation_resistance f — pinneapple_simulation.electromagnetics.waves (v03)
- hall_bar f — pinneapple_science.em.statics (v02)
- HallEffect C — pinneapple_science.em.statics (v02)
- HallSurrogate C — pinneapple_science.em.research (v02)
- hamiltonian_expectation f — pinneapple_quantum.loss.schrodinger (v04)
- HamiltonianNeuralNetwork C — pinneapple_neural.architectures.continuous.hamiltonian (v04)
- handles f — pinneapple_design.aero.geometry (v06)
- HandlingPolicy C — pinneapple_security.classification (v05)
- HardBC C — pinneapple_physics.symbolic_pde.bc (v02)
- harden f — pinneapple_analysis.validation.arena.blue (v05)
- HardNet f — pinneapple_analysis.benchmark.builtin.standard (v05)
- hardware_record f — pinneapple_analysis.cost.equal_budget (v05)
- harmonic f — pinneapple_design.bricks.closed_form (v06)
- harmonic_limit_lp f — pinneapple_design.bricks.validation (v06)
- harmonic_overhang f — pinneapple_design.bricks.closed_form (v06)
- harmonic_potential f — pinneapple_quantum.loss.schrodinger (v04)
- harmonic_stack f — pinneapple_design.bricks.closed_form (v06)
- harmonic_threshold f — pinneapple_design.bricks.dynamics (v06)
- HarvestEntry C — pinneapple_tools.rea_bridge.harvest (v07)
- HarvestLedger C — pinneapple_tools.rea_bridge.harvest (v07)
- has_cfmesh f — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- has_default_geometry f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- has_field f — pinneapple_physics.symbolic_pde.pinn (v02)
- has_pinnego_physical_sample f — pinneapple_data.synth.sample_adapter (v04)
- has_query_checks f — pinneapple_hub.zoo.checks (v07)
- hash_bytes f — pinneapple_llm.qa.sandbox (v07)
- hash_data f — pinneapple_analysis.validation.report_artifact (v05)
- hash_key f — pinneapple_core.service.apikeys (v01)

- hash_model f — pinneapple_analysis.validation.report_artifact (v05)
- hash_password f — pinneapple_security.passwords (v05)
- HashGridEncoding C — pinneapple_neural.architectures.hash_grid (v04)
- HashGridMLP C — pinneapple_neural.architectures.hash_grid (v04)
- hausen_mean_nusselt f — pinneapple_physics.closed_form.heat_exchanger (v02)
- HAVOK C — pinneapple_neural.architectures.rom.havok (v04)
- header f — pinneapple_data.cae.foam (v04)
- header f — pinneapple_data.simulation_metadata.foam_dict (v04)
- headline f — pinneapple_lab.curation (v07)
- health f — pinneapple_app.backend.main (v07)
- HealthIndicatorHistory C — pinneapple_systems.digital_twin.prognostics (v06)
- Heat C — pinneapple_simulation.external_solvers.calculix.study (v03)
- Heat1D C — pinneapple_arena.problems (v04)
- Heat1DTask C — pinneapple_tools.benchmark_suite.tasks.heat_1d (v07)
- Heat2D C — pinneapple_arena.problems (v04)
- heat_conduction_problem f — pinneapple_analysis.uncertainty.residual_bound (v05)
- heat_conduction_residual f — pinneapple_systems.component_modeling.physics_residuals (v06)
- heat_loss f — pinneapple_physics.closed_form.pipe_insulation (v02)
- heat_sink f — pinneapple_tools.nvidia.parts (v07)
- heat_sink_geometry f — pinneapple_tools.nvidia.parts (v07)
- heat_sink_material f — pinneapple_design.intent.materials (v06)
- heat_sink_mesh f — pinneapple_design.thermal.board_cooling.scene (v06)
- HeatConduction3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- HeatConfig3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- heated_pipe f — pinneapple_simulation.pipelines.freecad_openfoam.studies (v03)
- HeatExchangerResult C — pinneapple_systems.process_components.heat_exchanger (v06)
- HeatExchangerSpec C — pinneapple_systems.process_components.heat_exchanger (v06)
- HeatExchangerVanillaPINN C — pinneapple_systems.component_library.components (v06)
- heatsink_baseline f — pinneapple_design.design_optimizer.presets (v06)
- heatsink_figures f — pinneapple_design.intent.figures (v06)
- heatsink_problem f — pinneapple_design.design_optimizer.presets (v06)
- HeatsinkDesign C — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- HeatsinkEvaluator C — pinneapple_design.design_optimizer.presets (v06)
- HeatsinkForced C — pinneapple_hub.zoo._models (v07)
- HeatSinkGeometry C — pinneapple_physics.closed_form.plate_fin_heatsink (v02)

- `HeatsinkLimits` C — `pinneapple_design.design_optimizer.presets` (v06)
- `HeatsinkNatural` C — `pinneapple_hub.zoo._models` (v07)
- `HeatsinkOperating` C — `pinneapple_design.design_optimizer.presets` (v06)
- `HeatSinkTemplate` C — `pinneapple_design.intent.templates.heatsink` (v06)
- `HeatXTFC` C — `pinneapple_lab.experiments.pde` (v07)
- `heaviside` f — `pinneapple_design.topology.filters` (v06)
- `heaviside_derivative` f — `pinneapple_design.topology.filters` (v06)
- `height_above` f — `pinneapple_tools.visualization.molecular` (v07)
- `height_errors` f — `pinneapple_simulation.geophysics.sphere_shallow_water` (v03)
- `heldout` f — `pinneapple_simulation.electromagnetics.periodic.benchmark` (v03)
- `heldout_report` f — `pinneapple_analysis.surrogate.heldout` (v05)
- `helical_coil` f — `pinneapple_design.geometry.gen.cadquery_advanced` (v06)
- `heliocentric_state` f — `pinneapple_physics.astro.orbits` (v02)
- `helix_segments` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `helix_sweep` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `helmbold_lift_slope` f — `pinneapple_simulation.flight.aero` (v03)
- `Helmholtz2D` C — `pinneapple_arena.problems` (v04)
- `helmholtz_acoustics_3d_default` f — `pinneapple_physics.pde_environment.presets.industry` (v02)
- `helmholtz_coil_center_B` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `helmholtz_eigs` f — `pinneapple_science.numerics` (v02)
- `helmholtz_resonant_frequency_hz` f — `pinneapple_physics.closed_form.helmholtz_resonator` (v02)
- `hermite_shape_functions_1d` f — `pinneapple_simulation.numerical_solvers.nonlinear_beam_fem` (v03)
- `herschel_bulkley_stress` f — `pinneapple_systems.process_components.non_newtonian_pipe_flow` (v06)
- `hertzian_dipole_fields` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `hertzian_dipole_power` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `hertzian_dipole_radiation_resistance` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `hessian` f — `pinneapple_core.func` (v01)
- `hessian` f — `pinneapple_core.operators` (v01)
- `heston_pde_2d` f — `pinneapple_physics.pde_environment.presets.multidisciplinary` (v02)
- `heteroscedastic_nll_loss` f — `pinneapple_neural.trainer.heteroscedastic` (v04)
- `HeteroscedasticMisfit` C — `pinneapple_analysis.inverse_problems.noise_models` (v05)
- `HeteroscedasticWrapper` C — `pinneapple_neural.trainer.heteroscedastic` (v04)

- hex8_incompatible_stiffness f — pinneapple_design.qualitative.quantitative (v06)
- hex_model f — pinneapple_design.intent.physics.bracket (v06)
- HexMesh C — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- heywood_kt_net f — pinneapple_simulation.numerical_solvers.plane_stress_fem (v03)
- hf_repo f — pinneapple_hub.zoo._catalog (v07)
- HHTDecomposer C — pinneapple_systems.time_series.decomposition.hht_lstm (v06)
- HHTLSTMPipeline C — pinneapple_systems.time_series.decomposition.hht_lstm (v06)
- HHTNNForecaster C — pinneapple_systems.time_series.decomposition.hht_nn (v06)
- Hierarchy C — pinneapple_condense.hierarchy (v02)
- high_wavenumber_bias f — pinneapple_analysis.validation.spectral (v05)
- hilbert_analysis f — pinneapple_systems.time_series.solar_cycle (v06)
- HilbertHuangSolver C — pinneapple_simulation.numerical_solvers.hilbert_huang (v03)
- HilbertResult C — pinneapple_systems.time_series.solar_cycle (v06)
- history_features f — pinneapple_systems.sports.football.kinematics (v06)
- hmac_key f — pinneapple_security.signing (v05)
- hoerner_profile_cd f — pinneapple_physics.closed_form.thin_airfoil (v02)
- holdout f — pinneapple_design.thermal.board_cooling.learn (v06)
- holdout f — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- holdout_figure f — pinneapple_systems.sports.football.plots (v06)
- holdout_prediction f — pinneapple_systems.sports.football.ball_study (v06)
- holdout_split f — pinneapple_simulation.terramechanics.testbed (v03)
- hollow_pipe_mesh f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- holm f — pinneapple_systems.time_series.metrics_ext.comparison (v06)
- HoltDamped C — pinneapple_systems.time_series.adaptive (v06)
- HoltWintersAdditive C — pinneapple_systems.time_series.adaptive (v06)
- home f — pinneapple_flow.config (v07)
- homogenization_study f — pinneapple_condense.benchmark (v02)
- homogenized_observables f — pinneapple_condense.homogenize (v02)
- homogenized_system f — pinneapple_condense.homogenize (v02)
- horizon_altitude f — pinneapple_physics.astro.phenomena (v02)
- horizon_distance f — pinneapple_tools.visualization.studio.planetary (v07)
- horizons f — pinneapple_physics.weather.evaluate (v02)
- horizons_vectors f — pinneapple_physics.astro.smallbodies (v02)
- horizontal_to_equatorial f — pinneapple_physics.astro.frames (v02)

- horseshoe_induced f — pinneapple_design.aero.vlm (v06)
- HotSpot C — pinneapple_twin3d.wear.synthetic (v06)
- hotspot_power f — pinneapple_condense.package (v02)
- hour_angle_dec f — pinneapple_physics.astro.observing (v02)
- hpo f — pinneapple_flow.nodes.surrogates (v07)
- hr_diagram f — pinneapple_physics.astro.stars (v02)
- hstack_gifs f — pinneapple_tools.visualization.dashboard.static (v07)
- htail_weight f — pinneapple_design.aero.aircraft3d (v06)
- html_to_text f — pinneapple_llm.workbench.web (v07)
- HTTPPollStream C — pinneapple_systems.digital_twin.io.stream (v06)
- hub_url f — pinneapple_design.geometry_library.store (v06)
- HuberMisfit C — pinneapple_analysis.inverse_problems.noise_models (v05)
- HubQuery C — pinneapple_pdb.builder (v04)
- humanize f — pinneapple_flow.palette (v07)
- HWOffset C — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- Hybrid C — pinneapple_core.module (v01)
- hybrid_conduction f — pinneapple_condense.meshless2d (v02)
- hybrid_fem f — pinneapple_flow.nodes_ext.core_ts (v07)
- hybrid_pinn f — pinneapple_quantum.pipeline.presets (v04)
- hybrid_vs_fea_only f — pinneapple_simulation.electromagnetics.machines.design (v03)
- HybridModel C — pinneapple_quantum.models.hybrid_model (v04)
- HybridPrediction C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- HybridRBFNetwork C — pinneapple_neural.architectures.reservoir_computing.hybrid_rbf (v04)
- HybridSurrogatePhysics C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- hydraulically_smooth f — pinneapple_physics.physics_audit.assumptions (v02)
- hypervolume f — pinneapple_design.design_optimizer.study (v06)
- hypothesis_map f — pinneapple_flow.nodes_ext.non_hermitian (v07)
- HysteresisLoop C — pinneapple_simulation.electromagnetics.hysteresis (v03)

I

- i_section f — pinneapple_reason.modes.structural (v05)
- iaaft_surrogate f — pinneapple_systems.time_series.predictability (v06)
- iapws97_psat f — pinneapple_physics.closed_form.pump_npsh (v02)
- iban_ok f — pinneapple_security.privacy (v05)
- id_column_of f — pinneapple_core.workflows.datasets (v01)
- identifiability f — pinneapple_systems.sports.football.ball_study (v06)
- identifiability_report f — pinneapple_analysis.inverse_problems.heat_diffusivity (v05)
- identifiability_study f — pinneapple_analysis.inverse_problems.contracts (v05)

- IdentifiabilityAnalyzer C — pinneapple_analysis.inverse_problems.sensitivity (v05)
- IdentifiabilityReport C — pinneapple_analysis.inverse_problems.identifiability (v05)
- IdentifiabilityResult C — pinneapple_analysis.inverse_problems.sensitivity (v05)
- identify f — pinneapple_flow.identify (v07)
- identify f — pinneapple_physics (v02)
- identify_pde f — pinneapple_physics.pde_environment.capabilities (v02)
- identify_recharge f — pinneapple_systems.time_series.enso.pinn (v06)
- ids f — pinneapple_physics.astro.facts (v02)
- iges_to_mesh f — pinneapple_design.geometry.io.iges (v06)
- IgesImportConfig C — pinneapple_design.geometry.io.iges (v06)
- illuminated_fraction f — pinneapple_physics.astro.phenomena (v02)
- illustrative_steel f — pinneapple_simulation.electromagnetics.materials (v03)
- ImageReconstructionSynthGenerator C — pinneapple_data.synth.images (v04)
- ImageRegressor C — pinneapple_worldmodel.robot_perception (v04)
- imf_statistics f — pinneapple_systems.time_series.solar_cycle (v06)
- ImmersedBoundaryFDMSolver C — pinneapple_simulation.numerical_solvers.immersed_boundary_fdm (v03)
- impact_gif f — pinneapple_design.thermal.board_cooling.report (v06)
- impact_gif f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- implicit_solve f — pinneapple_core.func (v01)
- ImplicitSurfaceRBF C — pinneapple_design.geometry.ops.meshfree (v06)
- implied_inflow f — pinneapple_physics.blackhole.skill (v02)
- import_viper f — pinneapple_simulation.rover.autolab.render (v03)
- improve f — pinneapple_lab.improve (v07)
- improvement_plan f — pinneapple_physics.physics_audit.improvement (v02)
- inclination_at_depth f — pinneapple_systems.process_components.curved_path_geometry (v06)
- include_dirs f — pinneapple_simulation.accelerated_foam.build (v03)
- incompressible f — pinneapple_physics.physics_audit.assumptions (v02)
- incompressible_continuity_residual f — pinneapple_systems.component_modeling.physics_residuals (v06)
- incompressible_mass_flow f — pinneapple_systems.process_components.control_valve (v06)
- index f — pinneapple_design.geometry_library.web (v06)
- index f — pinneapple_presets.registry (v02)
- industrial_furnace_thermal f — pinneapple_physics.pde_environment.presets.engineering (v02)
- ineq f — pinneapple_core.optim (v01)

- `infeasibility_reasons` f — `pinneapple_design.aero.optimize` (v06)
- `Infeasible` C — `pinneapple_reason.library` (v05)
- `infer` f — `pinneapple_neural.predictor.infer` (v04)
- `infer` f — `pinneapple_tools.model_services.registry` (v07)
- `infer_allegro` f — `pinneapple_arena.external.runners.materials_md` (v04)
- `infer_astronomix` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_bpinn_wildfire` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_brax` f — `pinneapple_arena.external.runners.rigidbody_robotics` (v04)
- `infer_camels` f — `pinneapple_arena.external.runners.quantum_cosmology` (v04)
- `infer_ceviche` f — `pinneapple_arena.external.runners.em_photonics` (v04)
- `infer_check_kind` f — `pinneapple_lab.curation` (v07)
- `infer_chgnet` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_crnn` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_deepflame` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_deepmd_kit` f — `pinneapple_arena.external.runners.materials_md` (v04)
- `infer_deeponet_fno_benchmark` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_deeponet_original` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_deepwave` f — `pinneapple_arena.external.runners.geophysics_energy` (v04)
- `infer_deepxde` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_devito` f — `pinneapple_arena.external.runners.geophysics_energy` (v04)
- `infer_diffpd` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_diffrax` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_diffsph` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_difftaichi` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_dl4to` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_dojo_jl` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_dsgp4` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)

- `infer_epipinn` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_fairchem` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_ferminet` f — `pinneapple_arena.external.runners.quantum_cosmology` (v04)
- `infer_flowpm_jaxpm` f — `pinneapple_arena.external.runners.quantum_cosmology` (v04)
- `infer_fluidx3d` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_fourcastnet` f — `pinneapple_arena.external.runners.weather_climate` (v04)
- `infer_genesis` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_gns` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_graingnn` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_grainlearning` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_graphcast` f — `pinneapple_arena.external.runners.weather_climate` (v04)
- `infer_hpinn` f — `pinneapple_arena.external.runners.em_photonics` (v04)
- `infer_jax_cfd` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_jax_fluids` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_jax_md` f — `pinneapple_arena.external.runners.materials_md` (v04)
- `infer_lesnets` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_mace` f — `pinneapple_arena.external.runners.materials_md` (v04)
- `infer_mattersim` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_meshgraphnets_original` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_meshgraphnets_pytorch` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_ml4gw` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_mujoco_mjx` f — `pinneapple_arena.external.runners.rigidbody_robotics` (v04)
- `infer_nequip` f — `pinneapple_arena.external.runners.materials_md` (v04)
- `infer_netket` f — `pinneapple_arena.external.runners.quantum_cosmology` (v04)
- `infer_neuralfoil` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_neuralgcm` f — `pinneapple_arena.external.runners.weather_climate` (v04)
- `infer_neuraloperator` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_neuralpde_jl` f — `pinneapple_arena.external.runners.pinn_general` (v04)

- `infer_neuralsolvers` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_nimble_physics` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_on_grid_1d` f — `pinneapple_neural.predictor.infer` (v04)
- `infer_on_grid_2d` f — `pinneapple_neural.predictor.infer` (v04)
- `infer_openfold` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_orb_models` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_pangu_weather` f — `pinneapple_arena.external.runners.weather_climate` (v04)
- `infer_phiflow` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_photontorch` f — `pinneapple_arena.external.runners.em_photonics` (v04)
- `infer_physics` f — `pinneapple_check.physics` (v05)
- `infer_physics_informed_am` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_physicsnemo` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_physnet_thermal` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_piml_converter` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pina` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_pinn_ac_opf` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinn_corrosion_fatigue` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinn_multivp` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinn_option_pricing` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinn_swe_open` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinn_tfi_hss` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_pinn_wind_bearing` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinnicle` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pinns_original` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_pinns_torch` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_pino` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_pise` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_pybamm` f — `pinneapple_arena.external.runners.geophysics_energy` (v04)
- `infer_pyroomacoustics` f — `pinneapple_arena.external.runners.applied_domains` (v04)

- `infer_pysindy` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_quantity` f — `pinneapple_data.physical_units` (v04)
- `infer_reservoirflow` f — `pinneapple_arena.external.runners.geophysics_energy` (v04)
- `infer_rirpinn` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_schnetpack` f — `pinneapple_arena.external.runners.materials_md` (v04)
- `infer_sciann` f — `pinneapple_arena.external.runners.pinn_general` (v04)
- `infer_simvascular` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_socialforce` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_spdiff` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_stiff_pinn` f — `pinneapple_arena.external.runners.applied_domains` (v04)
- `infer_taichi` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_tensorcircuit_ng` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_torax` f — `pinneapple_arena.external.runners.geophysics_energy` (v04)
- `infer_torchmd` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_transolver` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_twophasepinn` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `infer_u_no` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_veros` f — `pinneapple_arena.external.runners.dynamics_robotics_astro` (v04)
- `infer_wakenet` f — `pinneapple_arena.external.runners.materials2_and_misc` (v04)
- `infer_warp` f — `pinneapple_arena.external.runners.fluids_structural` (v04)
- `inference_latency` f — `pinneapple_tools.benchmark_suite.crossframework.darcy` (v07)
- `InferenceResult` C — `pinneapple_neural.predictor.infer` (v04)
- `infinite_well_potential` f — `pinneapple_quantum.loss.schrodinger` (v04)
- `infinite_wire_B` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `inflow` f — `pinneapple_tools.automation.verify` (v07)
- `info` f — `pinneapple` (v01)
- `info` f — `pinneapple_app.backend.main` (v07)
- `info` f — `pinneapple_design.geometry_library.core` (v06)
- `info` f — `pinneapple_hub.zoo` (v07)
- `info` f — `pinneapple_hub.zoo.datasets` (v07)
- `Informer` C — `pinneapple_neural.architectures.transformers.informer` (v04)
- `ingest` f — `pinneapple_flow.nodes.data` (v07)

- `init_logvar_bias` f — `pinneapple_neural.trainer.heteroscedastic` (v04)
- `init_workdir` f — `pinneapple_arena.autoresearch` (v04)
- `initial_state` f — `pinneapple_physics.astro.nbody` (v02)
- `initial_state` f — `pinneapple_simulation.numerical_solvers.symbolic_fd` (v03)
- `InitialCondition` f — `pinneapple_physics.pde_environment.conditions` (v02)
- `InitialConditionSpec` C — `pinneapple_app.backend.core.problem` (v07)
- `inject` f — `pinneapple_security.cps_bench` (v05)
- `inline_hashes` f — `pinneapple_security.web` (v05)
- `InnerLoops` C — `pinneapple_simulation.flight.control` (v03)
- `InputError` C — `pinneapple_analysis.inverse_problems.heat_transfer` (v05)
- `inputs_for_steady_state` f — `pinneapple_systems.operations.plant` (v06)
- `InputSpec` C — `pinneapple_tools.automation.workflow` (v07)
- `insert_nodes` f — `pinneapple_flow.lab_status` (v07)
- `inspect_fmu` f — `pinneapple_simulation.external_solvers.modelica.fmu_inspect` (v03)
- `inspect_table` f — `pinneapple_analysis.surrogate.data` (v05)
- `inspection_dataset` f — `pinneapple_worldmodel.inspection_data` (v04)
- `inspection_sample` f — `pinneapple_worldmodel.inspection_data` (v04)
- `InspectionPINN` C — `pinneapple_analysis.inspection.base` (v05)
- `install_hint` f — `pinneapple_flow.palette` (v07)
- `install_state` f — `pinneapple_arena.external.status` (v04)
- `installed_cv` f — `pinneapple_systems.process_components.control_valve` (v06)
- `instantiate` f — `pinneapple_neural.architectures.instantiate` (v04)
- `instantiate_model` f — `pinneapple_arena.pointwise.training` (v04)
- `InstantiateReport` C — `pinneapple_neural.architectures.instantiate` (v04)
- `Instrument` C — `pinneapple_science.instruments` (v02)
- `integer_ticks` f — `pinneapple_science.plots` (v02)
- `integral` f — `pinneapple_physics.conservation` (v02)
- `IntegralObsOperator` C — `pinneapple_analysis.inverse_problems.obs_operator` (v05)
- `integrate` f — `pinneapple_core.operators` (v01)
- `integrate` f — `pinneapple_lab.experiments.ode` (v07)
- `integrate` f — `pinneapple_physics.astro.nbody` (v02)
- `integrate_network` f — `pinneapple_systems.process_components.reaction_kinetics` (v06)
- `integrate_p1` f — `pinneapple_core.fem` (v01)
- `integrate_pressure_profile` f — `pinneapple_systems.process_components.non_newtonian_pipe_flow` (v06)
- `integrate_surface_forces` f — `pinneapple_tools.aero_coefficients` (v07)
- `IntegrationResult` C — `pinneapple_systems.process_components.reaction_kinetics` (v06)
- `Integrity` C — `pinneapple_check.config` (v05)

- intent_service f — pinneapple_tools.model_services.registry (v07)
- IntentLLM C — pinneapple_design.intent.parse (v06)
- Interface C — pinneapple_design.bricks.statics (v06)
- interface_modes f — pinneapple_condense.substructure (v02)
- InterfaceBC f — pinneapple_physics.pde_environment.conditions (v02)
- IntermediateModelForecaster C — pinneapple_systems.time_series.enso.forecasters (v06)
- InternalFlow C — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- InternalResult C — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- interp_grid f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- interpolate f — pinneapple_flow.nodes._common (v07)
- interpolate_on_triangles f — pinneapple_design.geometry.sample.barycentric (v06)
- interpolate_to_points f — pinneapple_simulation.numerical_solvers.immersed_boundary_fdm (v03)
- Interpolator C — pinneapple_data.cae.compare (v04)
- interpret f — pinneapple_tools.automation.interpret (v07)
- interpret_mission f — pinneapple_autolab.agent (v07)
- Interpretation C — pinneapple_tools.automation.interpret (v07)
- Intersection C — pinneapple_worldmodel.geometry (v04)
- intervals_from_labels f — pinneapple_systems.operations.anomaly (v06)
- intervention_checks f — pinneapple_analysis.validation.physics_learning (v05)
- InterventionReport C — pinneapple_analysis.verification.causal_discrepancy (v05)
- InterventionRunResult C — pinneapple_analysis.verification.causal_discrepancy (v05)
- intrinsic_coercivity f — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- introspect f — pinneapple_flow.libcalls (v07)
- InvalidCell C — pinneapple_simulation.electromagnetics.periodic.tcda (v03)
- InvalidGeometry C — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- Invariance C — pinneapple_analysis.validation.metamorphic (v05)
- Invariant C — pinneapple_llm.qa.spec (v07)
- invariant_check f — pinneapple_physics.physics_audit.monitor (v02)
- invariant_drift f — pinneapple_analysis.validation.rollout (v05)
- invariant_series f — pinneapple_analysis.validation.rollout (v05)
- inventory f — pinneapple_flow.examples (v07)
- inverse f — pinneapple_core.loss (v01)
- inverse_study f — pinneapple_analysis.inverse_problems.contracts (v05)
- InverseConfig C — pinneapple_arena.config (v04)

- InverseDirichlet C — pinneapple_neural.trainer.loss_balancer (v04)
- InverseFit C — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- InverseHeatProblem C — pinneapple_analysis.benchmark.builtin.inverse_heat (v05)
- InverseParam C — pinneapple_neural.trainer.inverse_pinn (v04)
- InverseParamAdapter C — pinneapple_analysis.verification.uncertainty_inverse (v05)
- InversePermittivity C — pinneapple_science.em.research (v02)
- InversePINN C — pinneapple_neural.architectures.pinns.inverse (v04)
- InversePINN C — pinneapple_neural.trainer.inverse_pinn (v04)
- InversePINNDataset C — pinneapple_worldmodel.training_hooks (v04)
- InverseProblem C — pinneapple_analysis.inverse_problems.studio (v05)
- InverseProblemResult C — pinneapple_analysis.verification.uncertainty_inverse (v05)
- InverseProblemSolver C — pinneapple_analysis.inverse_problems.solver (v05)
- InverseSolverConfig C — pinneapple_analysis.inverse_problems.solver (v05)
- InverseSolverResult C — pinneapple_analysis.inverse_problems.solver (v05)
- invert f — pinneapple_analysis (v05)
- involute_tooth_profile f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- IPMMachine C — pinneapple_simulation.electromagnetics.machines.pmsm (v03)
- IPMTraction C — pinneapple_presets.cases.ipm_traction (v02)
- iron_loss f — pinneapple_simulation.electromagnetics.postprocess (v03)
- iron_loss_components f — pinneapple_simulation.electromagnetics.postprocess (v03)
- iron_split f — pinneapple_simulation.electromagnetics.machines.thermal (v03)
- IronLossModel C — pinneapple_simulation.electromagnetics.postprocess (v03)
- is_artifact f — pinneapple_flow.runtime (v07)
- is_auto f — pinneapple_flow.palette (v07)
- is_buildable f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- is_closed_volume f — pinneapple_design.geometry_library.kit (v06)
- is_deepxde_available f — pinneapple_physics.deepxde_bridge.adapter (v02)
- is_distributed f — pinneapple_neural.trainer.distributed (v04)
- is_graph_model f — pinneapple_arena.model_factory (v04)
- is_hash f — pinneapple_security.passwords (v05)
- is_installed f — pinneapple_arena.external.installer (v04)
- is_inverse_model f — pinneapple_arena.model_factory (v04)
- is_offline f — pinneapple_hub.zoo._artifacts (v07)
- is_operator_model f — pinneapple_arena.model_factory (v04)
- is_parameter_space f — pinneapple_analysis.study_contracts (v05)

- is_passive f — pinneapple_simulation.electromagnetics.media (v03)
- is_physicsnemo_available f — pinneapple_neural.physicsnemo_bridge.adapters (v04)
- is_pinn_model f — pinneapple_arena.model_factory (v04)
- is_result_file f — pinneapple_data.preflight (v04)
- isa f — pinneapple_design.aero.airliner3d (v06)
- isa_density f — pinneapple_design.aero.aircraft (v06)
- isaac_replay_script f — pinneapple_tools.nvidia.isaac (v07)
- IsaacSimBackend C — pinneapple_simulation.robot_sim.sim (v03)
- isentropic_ratios f — pinneapple_physics.closed_form.compressible (v02)
- ishigami f — pinneapple_analysis.sensitivity.testfunctions (v05)
- ishigami_indices f — pinneapple_analysis.sensitivity.testfunctions (v05)
- ishigami_space f — pinneapple_analysis.sensitivity.testfunctions (v05)
- isolated f — pinneapple_condense.benchmark (v02)
- IsolationForestDetector C — pinneapple_security.cps_bench (v05)
- IsolationLevel C — pinneapple_llm.qa.sandbox (v07)
- isotropic_line_spectra f — pinneapple_simulation.flight.turbulence (v03)
- ISPHSolver C — pinneapple_simulation.numerical_solvers.isph (v03)
- Issue C — pinneapple_data.data_health (v04)
- Issue C — pinneapple_flow.schema (v07)
- Item C — pinneapple_design.bricks.visuals (v06)
- ItemAdapter C — pinneapple_neural.trainer.datamodule (v04)
- items f — pinneapple_lab.seal (v07)
- iter_chunks f — pinneapple_data.chunking (v04)
- iter_known_answers f — pinneapple_reason.library (v05)
- iter_repo_files f — pinneapple_analysis.retrieval.corpus (v05)
- iter_tiles f — pinneapple_pdb.shard (v04)
- iter_time_windows f — pinneapple_pdb.shard (v04)
- iterate_refinement f — pinneapple_condense.substructure (v02)
- IteratedEKI C — pinneapple_analysis.inverse_problems.ensemble_kalman (v05)
- IterationRecord C — pinneapple_problemdesign.research_loop (v02)
- ito_bend_loss f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- itrs_to_geodetic f — pinneapple_physics.astro.frames (v02)

J

- jacfwd f — pinneapple_core.func (v01)
- jackknife f — pinneapple_analysis.uncertainty.resampling (v05)
- jacobi f — pinneapple_simulation.accelerated_foam.solvers (v03)
- jacobian f — pinneapple_analysis.inverse_problems.forward (v05)
- jacobian f — pinneapple_core.func (v01)
- jacobian f — pinneapple_core.operators (v01)
- jacobian f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- jacrev f — pinneapple_core.func (v01)

- jax_available f — pinneapple_tools.compute_backends.jax_backend (v07)
- jax_cfd_available f — pinneapple_simulation.numerical_solvers.jax_cfd_solver (v03)
- jax_pinn f — pinneapple_tools.compute_backends.jax_backend (v07)
- JAXBackend C — pinneapple_tools.compute_backends.jax_backend (v07)
- JAXCFDSolver C — pinneapple_simulation.numerical_solvers.jax_cfd_solver (v03)
- JaxPhysicsBackend C — pinneapple_core.backend (v01)
- JAXPINNBackend C — pinneapple_tools.benchmark_suite.backends.jax_pinn (v07)
- jd_from_datetime f — pinneapple_physics.astro.timescales (v02)
- jet f — pinneapple_tools.visualization.studio.colormap (v07)
- JilesAtherton C — pinneapple_simulation.electromagnetics.hysteresis (v03)
- jit_pinn f — pinneapple_tools.compute_backends.jax_backend (v07)
- job_checks f — pinneapple_analysis.benchmark.checks (v05)
- job_status f — pinneapple_mcp.tools (v07)
- JobCancelled C — pinneapple_core.service.jobs (v01)
- JobContext C — pinneapple_core.service.jobs (v01)
- JobQueue C — pinneapple_core.service.jobs (v01)
- JobTimeout C — pinneapple_core.service.jobs (v01)
- Joint C — pinneapple_simulation.external_solvers.newton_physics.scene (v03)
- Joint C — pinneapple_simulation.robot_sim.spec (v03)
- JointAdaptiveWeights C — pinneapple_neural.trainer.loss_balancer (v04)
- JointModel C — pinneapple_design.bricks.parts (v06)
- joukowsky_head f — pinneapple_simulation.numerical_solvers.water_hammer_moc (v03)
- joukowsky_head_rise f — pinneapple_systems.process_components.water_hammer (v06)
- json_schemas f — pinneapple_data.contracts.models (v04)
- jsonable f — pinneapple_science.core (v02)
- jsonable f — pinneapple_tools.model_services.registry (v07)
- julian_centuries f — pinneapple_physics.astro.timescales (v02)
- junction_temperature_c f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)
- junit f — pinneapple_check.report (v05)

K

- k_anonymity f — pinneapple_security.privacy (v05)
- KAEAutoencoder C — pinneapple_neural.architectures.autoencoders.kae (v04)
- KafkaStream C — pinneapple_systems.digital_twin.io.stream (v06)
- kalman f — pinneapple_flow.nodes.methods (v07)
- kalman_smooth f — pinneapple_systems.sports.football.kinematics (v06)
- KalmanFilter C — pinneapple_neural.architectures.classical_ts.kalman (v04)
- KBArtifact C — pinneapple_tools.hpo_experiments.models (v07)

- KBIndex C — pinneapple_tools.hpo_experiments.models (v07)
- KBStore C — pinneapple_tools.hpo_experiments.kb.store (v07)
- KdV1D C — pinneapple_arena.problems (v04)
- keep_largest_component f — pinneapple_design.geometry.ops.clean (v06)
- keff_maps f — pinneapple_condense.figures (v02)
- kelvin_wave_test f — pinneapple_systems.time_series.enso.intermediate (v06)
- kenics_elements f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- kepler_exact_state f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- kepler_positions f — pinneapple_physics.astro.neural_propagator (v02)
- kepler_two_body_orbit f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- KeplerLaw C — pinneapple_lab.experiments.discovery (v07)
- kern_shell f — pinneapple_physics.closed_form.heat_exchanger (v02)
- key f — pinneapple_hub.library.categories (v07)
- kind_of f — pinneapple_data.cae.io (v04)
- kinematic_extrapolation f — pinneapple_systems.sports.football.ball (v06)
- Kinematics C — pinneapple_simulation.multibody.tree (v03)
- kinetic_energy f — pinneapple_simulation.flight.rigid_body (v03)
- kirchhoff_disc_capacitance f — pinneapple_science.em.statics (v02)
- kit_render_script f — pinneapple_tools.nvidia.omniverse (v07)
- kl_diag_gaussians f — pinneapple_neural.architectures.continuous.deep_state_space (v04)
- KleinGordon1D C — pinneapple_arena.problems (v04)
- knn f — pinneapple_condense.meshless2d (v02)
- known_actions f — pinneapple_tools.automation.adapters.base (v07)
- known_answer_checks f — pinneapple_reason.trust (v05)
- knudsen f — pinneapple_physics.physics_audit.regime (v02)
- knuth_histogram f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- KOmegaSSTResiduals C — pinneapple_physics.pde_environment.turbulence_presets (v02)
- KoopmanAutoencoder C — pinneapple_neural.architectures.rom.koopman (v04)
- KoopmanOperator C — pinneapple_neural.architectures.reservoir_computing.koopman (v04)
- KoopmanOperator C — pinneapple_neural.architectures.rom.koopman (v04)
- kovasznay f — pinneapple_analysis.validation.pinn (v05)
- KovasznayNS C — pinneapple_arena.problems (v04)
- kp_needed f — pinneapple_physics.astro.spaceweather (v02)
- KPI C — pinneapple_tools.reporting.datasheet (v07)
- kpss_test f — pinneapple_systems.time_series.audit.tests (v06)
- KuramotoSivashinsky1D C — pinneapple_simulation.numerical_solvers.kuramoto_sivashinsky

(v03)

L

- `l2` f — `pinneapple_physics.metrics` (v02)
- `l_bracket` f — `pinneapple_tools.nvidia.parts` (v07)
- `l_bracket_volume` f — `pinneapple_tools.nvidia.parts` (v07)
- `l_mesh` f — `pinneapple_design.intent.physics.bracket` (v06)
- `lab` f — `pinneapple_flow.nodes.analysis` (v07)
- `lab_export_status` f — `pinneapple_flow.runtime` (v07)
- `lab_item` f — `pinneapple_hub.library.snippets` (v07)
- `lab_items` f — `pinneapple_flow.lab_status` (v07)
- `lab_items_for` f — `pinneapple_lab.manifest` (v07)
- `lab_toolbox` f — `pinneapple_autolab.agent` (v07)
- `label_check` f — `pinneapple_worldmodel.inspection_data` (v04)
- `LabelledInterval` C — `pinneapple_systems.operations.anomaly` (v06)
- `LabPlanner` C — `pinneapple_autolab.agent` (v07)
- `LabReport` C — `pinneapple_veriphysics.validation_lab` (v05)
- `LabServer` C — `pinneapple_lab.server` (v07)
- `LabStore` C — `pinneapple_lab.store` (v07)
- `ladder_rail` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `ladder_values` f — `pinneapple_science.em.fdtd_study` (v02)
- `LagFeatures` C — `pinneapple_systems.time_series.forecast_lab.learned` (v06)
- `lagrange_shape_functions_1d` f — `pinneapple_simulation.numerical_solvers.nonlinear_beam_fem` (v03)
- `LagRegressorExpert` C — `pinneapple_systems.time_series.adaptive` (v06)
- `Lambda` C — `pinneapple_core.module` (v01)
- `lame` f — `pinneapple_presets.cases.pressure_vessel` (v02)
- `lame_analytical` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)
- `lame_hoop_stress_inner` f — `pinneapple_systems.process_components.pipe_stress_mechanics` (v06)
- `lame_hoop_stress_outer` f — `pinneapple_systems.process_components.pipe_stress_mechanics` (v06)
- `lame_parameters` f — `pinneapple_physics.solid.axisymmetric` (v02)
- `lame_parameters` f — `pinneapple_simulation.numerical_solvers.elasticity3d_fdm` (v03)
- `laminar` f — `pinneapple_physics.closed_form.pipe_flow` (v02)
- `laminar` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `landscape_tiles` f — `pinneapple_systems.time_series.forecast_lab.renders` (v06)
- `lane_emden_exact_theta` f — `pinneapple_physics.pde_environment.presets.astrophysics` (v02)
- `lane_emden_polytrope` f — `pinneapple_physics.pde_environment.presets.astrophysics` (v02)
- `langevin` f — `pinneapple_simulation.electromagnetics.hysteresis` (v03)
- `lap_heads` f — `pinneapple_reason.modes.water` (v05)
- `Laplace2D` C — `pinneapple_arena.problems` (v04)

- `laplace_2d_default` f — `pinneapple_physics.pde_environment.presets.academics` (v02)
- `laplace_beltrami` f — `pinneapple_design.geometry.ops.curvature` (v06)
- `laplace_mechanism` f — `pinneapple_security.dp` (v05)
- `laplacian` f — `pinneapple_core.operators` (v01)
- `laplacian` f — `pinneapple_physics.pinn_solver.compiler.autograd_ops` (v02)
- `LargePhysicsModel` C — `pinneapple_neural.lpm` (v04)
- `largest_lyapunov_benettin` f — `pinneapple_systems.time_series.synthetic` (v06)
- `LastStepToHorizon` C — `pinneapple_systems.time_series.adapters.shape` (v06)
- `LatencyBudgetExceeded` C — `pinneapple_systems.digital_twin.realtime_pipeline` (v06)
- `LatencyStats` C — `pinneapple_systems.digital_twin.realtime_pipeline` (v06)
- `LatentConditionedModel` C — `pinneapple_physics.pinn_solver.stochastic` (v02)
- `LatentODE` C — `pinneapple_neural.architectures.continuous.latent_ode` (v04)
- `LatentStateTwin` C — `pinneapple_systems.digital_twin.product.assimilation` (v06)
- `lateral_figure` f — `pinneapple_simulation.flight.figures` (v03)
- `latest_time` f — `pinneapple_simulation.numerical_solvers.external_flow` (v03)
- `latin_hypercube` f — `pinneapple_design.aero.geometry` (v06)
- `latin_hypercube` f — `pinneapple_neural.architectures.rom.parametric` (v04)
- `latin_hypercube` f — `pinneapple_simulation.external_solvers.modelica.sweep` (v03)
- `Lattice` C — `pinneapple_design.aero.vlm` (v06)
- `lattice_check` f — `pinneapple_simulation.flight.verification` (v03)
- `lattice_order` f — `pinneapple_analysis.surrogate.data` (v05)
- `lattice_surface` f — `pinneapple_simulation.flight.visuals` (v03)
- `launch_experiment` f — `pinneapple_app.backend.routers.experiments` (v07)
- `LaunchVehicle` C — `pinneapple_design.geometry.vehicles3d` (v06)
- `Layer` C — `pinneapple_condense.package` (v02)
- `Layer` C — `pinneapple_physics.closed_form.pcb_thermal` (v02)
- `Layer` C — `pinneapple_simulation.electromagnetics.analytic` (v03)
- `layer_graph` f — `pinneapple_flow.learn` (v07)
- `layer_lr_groups` f — `pinneapple_adaptation.transfer_learning.freeze` (v04)
- `layer_sizes` f — `pinneapple_tools.benchmark_suite.crossframework.weights` (v07)
- `layer_total` f — `pinneapple_design.fitmesh.layers` (v06)
- `layered_disc` f — `pinneapple_design.surrogate_inverse.meshes` (v06)
- `layered_modes` f — `pinneapple_condense.substructure` (v02)
- `Layout` C — `pinneapple_design.aero.surrogate` (v06)
- `Layout` C — `pinneapple_simulation.electromagnetics.antenna.ava` (v03)
- `layout` f — `pinneapple_flow.palette` (v07)
- `layout` f — `pinneapple_flow.planner.build` (v07)

- `layout` f — `pinneapple_simulation.electromagnetics.antenna.ava` (v03)
- `layout_plot` f — `pinneapple_simulation.electromagnetics.antenna.plots` (v03)
- `lbfgs` f — `pinneapple_analysis.validation.arena.search` (v05)
- `LBFGSFinetuneResult` C — `pinneapple_neural.trainer.lbfgs_finetune` (v04)
- `lbm_channel` f — `pinneapple_flow.wrappers` (v07)
- `LBMSolver` C — `pinneapple_simulation.numerical_solvers.lbm` (v03)
- `LBMSolver3D` C — `pinneapple_simulation.numerical_solvers.lbm` (v03)
- `LCurveResult` C — `pinneapple_analysis.inverse_problems.regularization` (v05)
- `LCurveSelector` C — `pinneapple_analysis.inverse_problems.regularization` (v05)
- `LduSystem` C — `pinneapple_simulation.accelerated_foam.ldu` (v03)
- `lead_time_scores` f — `pinneapple_physics.blackhole.forecast` (v02)
- `leaf` f — `pinneapple_check._reuse` (v05)
- `leakage` f — `pinneapple_data.audit` (v04)
- `learn_hamiltonian` f — `pinneapple_quantum.pipeline.presets` (v04)
- `learn_planet` f — `pinneapple_flow.nodes.vehicle` (v07)
- `learned_study` f — `pinneapple_condense.benchmark` (v02)
- `LearnedBlocks` C — `pinneapple_condense.learned` (v02)
- `LearnedClosure` C — `pinneapple_physics.multiscale.learned` (v02)
- `LearnedWarmStart` C — `pinneapple_simulation.accelerated_foam.learned` (v03)
- `least_squares_fit` f — `pinneapple_analysis.inverse_problems.estimation` (v05)
- `Ledger` C — `pinneapple_autolab.ledger` (v07)
- `legend_seals` f — `pinneapple_lab.seal` (v07)
- `legendre_design` f — `pinneapple_analysis.uncertainty.pce` (v05)
- `legendre_tables` f — `pinneapple_simulation.geophysics.sphere_shallow_water` (v03)
- `lego_brick` f — `pinneapple_design.geometry.gen.cadquery_advanced` (v06)
- `length_factor` f — `pinneapple_tools.visualization.studio.fieldview` (v07)
- `lensed_sky` f — `pinneapple_physics.astro.lensing` (v02)
- `lensing_view` f — `pinneapple_physics.astro.lensing` (v02)
- `level_description` f — `pinneapple_llm.qa.levels` (v07)
- `lexical_overlap_score` f — `pinneapple_tools.hpo_experiments.ranking` (v07)
- `lhs` f — `pinneapple_presets.core` (v02)
- `li_forcing` f — `pinneapple_simulation.numerical_solvers.operator_benchmarks` (v03)
- `LibCall` C — `pinneapple_flow.libcalls` (v07)
- `Library` C — `pinneapple_reason.library` (v05)
- `library_specs` f — `pinneapple_flow.libcalls` (v07)
- `licence_counts` f — `pinneapple_arena.external` (v04)
- `LicenseGateError` C — `pinneapple_tools.rea_bridge.harvest` (v07)
- `LicensePolicy` C — `pinneapple_tools.rea_bridge.licenses` (v07)
- `lid_driven_cavity_3d` f — `pinneapple_physics.pde_environment.presets.cfd` (v02)

- `LidDrivenCavity3DTask` C — `pinneapple_tools.benchmark_suite.tasks.lid_driven_cavity_3d` (v07)
- `LidDrivenCavityConfig3D` C — `pinneapple_simulation.numerical_solvers.fdm3d` (v03)
- `LidDrivenCavityDomain2D` C — `pinneapple_design.geometry.gen.domains` (v06)
- `LidDrivenCavityDomain3D` C — `pinneapple_design.geometry.gen.domains3d` (v06)
- `LidDrivenCavitySolver3D` C — `pinneapple_simulation.numerical_solvers.fdm3d` (v03)
- `light_curve_sample` f — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `light_ga` f — `pinneapple_simulation.flight.catalog` (v03)
- `light_time_and_doppler` f — `pinneapple_physics.astro.orbits` (v02)
- `LightGBMForecaster` C — `pinneapple_systems.time_series.models.classical` (v06)
- `LightingPreset` C — `pinneapple_tools.visualization.studio.lighting` (v07)
- `LIMForecaster` C — `pinneapple_systems.time_series.enso.forecasters` (v06)
- `Limit` C — `pinneapple_autolab.mission` (v07)
- `limitations` f — `pinneapple_analysis.surrogate.limits` (v05)
- `limits_from_env` f — `pinneapple_core.service.metering` (v01)
- `limits_report` f — `pinneapple_simulation.electromagnetics.antenna.study` (v03)
- `line_charge_field` f — `pinneapple_simulation.electromagnetics.statics` (v03)
- `line_of` f — `pinneapple_data.preflight.common` (v04)
- `Lineage` C — `pinneapple_data.lineage.graph` (v04)
- `linear_closure` f — `pinneapple_systems.process_components.water_hammer` (v06)
- `linear_combination_rate_constant` f — `pinneapple_systems.process_components.reaction_kinetics` (v06)
- `linear_elastic` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `linear_elasticity_3d` f — `pinneapple_physics.pde_environment.presets.structural` (v02)
- `linear_elasticity_3d_default` f — `pinneapple_physics.pde_environment.presets.industry` (v02)
- `linear_elasticity_residual` f — `pinneapple_systems.component_modeling.physics_residuals` (v06)
- `linear_fit` f — `pinneapple_science.instruments` (v02)
- `linear_sde_stepper` f — `pinneapple_systems.time_series.synthetic` (v06)
- `LinearBVP1D` C — `pinneapple_simulation.numerical_solvers.gated_xtfc` (v03)
- `LinearDQ` C — `pinneapple_simulation.electromagnetics.machines.drive` (v03)
- `LinearElasticity2D` C — `pinneapple_arena.problems` (v04)
- `LinearFit` C — `pinneapple_science.instruments` (v02)
- `LinearMaterial` C — `pinneapple_simulation.electromagnetics.materials` (v03)
- `LinearMPC` C — `pinneapple_systems.operations.control` (v06)

- `LinearObsOperator` C — `pinneapple_analysis.inverse_problems.obs_operator` (v05)
- `LinearPhysics` C — `pinneapple_simulation.electromagnetics.antenna.surrogates` (v03)
- `LinearSPDProblem` C — `pinneapple_analysis.uncertainty.residual_bound` (v05)
- `LinearSSMForecaster` C — `pinneapple_systems.time_series.forecast_lab.physical` (v06)
- `LineFit` C — `pinneapple_physics.pde_environment.presets.astroml_bridge` (v02)
- `Lines` C — `pinneapple_tools.visualization.studio.scene` (v07)
- `Link` C — `pinneapple_simulation.robot_sim.spec` (v03)
- `link` f — `pinneapple_flow.references` (v07)
- `link_frames` f — `pinneapple_simulation.rover.geometry` (v03)
- `link_root_checks` f — `pinneapple_simulation.multibody.arm` (v03)
- `link_world_points` f — `pinneapple_simulation.multibody.arm` (v03)
- `Liquid` C — `pinneapple_simulation.numerical_solvers.particles` (v03)
- `list` f — `pinneapple_design.geometry_library` (v06)
- `list` f — `pinneapple_hub.zoo` (v07)
- `list` f — `pinneapple_hub.zoo.datasets` (v07)
- `list_adapters` f — `pinneapple_tools.automation.adapters.base` (v07)
- `list_analytic` f — `pinneapple_physics.solving` (v02)
- `list_available_solver_families` f — `pinneapple_analysis.verification.solver_orchestration` (v05)
- `list_backends` f — `pinneapple_core.backend` (v01)
- `list_backends` f — `pinneapple_tools.benchmark_suite.registry` (v07)
- `list_benchmarks` f — `pinneapple_arena.dataset_bench` (v04)
- `list_benchmarks` f — `pinneapple_pdb.benchmarks` (v04)
- `list_builders` f — `pinneapple_design.geometry.gen.primitives` (v06)
- `list_buy_vs_build_needs` f — `pinneapple_analysis.verification.tool_recommendation` (v05)
- `list_cards` f — `pinneapple_physics.astro.facts` (v02)
- `list_datasets` f — `pinneapple_data.datasets` (v04)
- `list_datasets` f — `pinneapple_simulation.terramechanics.testbed` (v03)
- `list_domains` f — `pinneapple_design.geometry.gen.domains` (v06)
- `list_domains_3d` f — `pinneapple_design.geometry.gen.domains3d` (v06)
- `list_experiments` f — `pinneapple_app.backend.routers.experiments` (v07)
- `list_families` f — `pinneapple_app.backend.routers.models` (v07)
- `list_field_vars` f — `pinneapple_simulation.numerical_solvers.adapters.upd_signal` (v03)
- `list_jobs` f — `pinneapple_mcp.jobs` (v07)
- `list_jobs` f — `pinneapple_mcp.tools` (v07)
- `list_kinds` f — `pinneapple_physics.solving` (v02)
- `list_methods` f — `pinneapple_catalog.methods` (v01)
- `list_methods` f — `pinneapple_core.optim` (v01)
- `list_methods` f — `pinneapple_mcp.tools` (v07)
- `list_methods` f — `pinneapple_physics.solving` (v02)

- `list_metrics` f — `pinneapple_app.backend.routers.models` (v07)
- `list_models` f — `pinneapple_app.backend.core.model_catalog` (v07)
- `list_models` f — `pinneapple_app.backend.routers.models` (v07)
- `list_pde_families` f — `pinneapple_physics.pde_environment.capabilities` (v02)
- `list_pde_rules` f — `pinneapple_core.transforms` (v01)
- `list_presets` f — `pinneapple_analysis.inverse_problems.presets` (v05)
- `list_presets` f — `pinneapple_mcp.tools` (v07)
- `list_presets` f — `pinneapple_physics.pde_environment.presets.registry` (v02)
- `list_presets` f — `pinneapple_presets.registry` (v02)
- `list_problems` f — `pinneapple_app.backend.routers.problems` (v07)
- `list_problems` f — `pinneapple_arena.problems` (v04)
- `list_problems_by_domain` f — `pinneapple_arena.problems` (v04)
- `list_resources` f — `pinneapple_catalog.resources` (v01)
- `list_signal_axes` f — `pinneapple_simulation.numerical_solvers.adapters.upd_signal` (v03)
- `list_soils` f — `pinneapple_simulation.terramechanics.soils` (v03)
- `list_specs` f — `pinneapple_data.formfill.specs` (v04)
- `list_strategies` f — `pinneapple_core.loss` (v01)
- `list_tasks` f — `pinneapple_tools.benchmark_suite.registry` (v07)
- `live_state` f — `pinneapple_autolab.dashboard` (v07)
- `LiveLineSampler` C — `pinneapple_simulation.numerical_solvers.free_surface` (v03)
- `LiveTwin` C — `pinneapple_systems.operations.live` (v06)
- `lj_force_magnitude` f — `pinneapple_simulation.particle_dynamics.molecular_dynamics` (v03)
- `lj_gas` f — `pinneapple_flow.nodes_ext.solvers` (v07)
- `lj_potential` f — `pinneapple_simulation.particle_dynamics.molecular_dynamics` (v03)
- `llm_decision` f — `pinneapple_flow.decisions` (v07)
- `llm_schema` f — `pinneapple_design.intent.parse` (v06)
- `LLMAdvisor` C — `pinneapple_reason.llm` (v05)
- `LLMAssist` C — `pinneapple_design.fitmesh.llm` (v06)
- `LLMMessage` C — `pinneapple_problemdesign.protocol` (v02)
- `LLMPlanner` C — `pinneapple_autolab.core` (v07)
- `LLMProposer` C — `pinneapple_arena.autoresearch` (v04)
- `LLMProvider` C — `pinneapple_problemdesign.protocol` (v02)
- `LLMProvider` C — `pinneapple_problemdesign.providers` (v02)
- `LLMResponse` C — `pinneapple_problemdesign.protocol` (v02)
- `LLMStepPlanner` C — `pinneapple_autolab.core` (v07)
- `lmtd_correction_1_2` f — `pinneapple_physics.closed_form.heat_exchanger` (v02)
- `Load` C — `pinneapple_design.bricks.statics` (v06)
- `load` f — `pinneapple_analysis.benchmark.api` (v05)
- `load` f — `pinneapple_data.contracts.models` (v04)
- `load` f — `pinneapple_design.geometry_library.store` (v06)
- `load` f — `pinneapple_flow.autocalls` (v07)
- `load` f — `pinneapple_flow.learn` (v07)
- `load` f — `pinneapple_hub.zoo` (v07)
- `load` f — `pinneapple_hub.zoo.datasets` (v07)
- `load` f — `pinneapple_lab.manifest` (v07)
- `load` f — `pinneapple_presets.registry` (v02)

- load f — pinneapple_science.atlas (v02)
- load_agents f — pinneapple_llm.workbench.agents (v07)
- load_app f — pinneapple_flow.studio (v07)
- load_arrays f — pinneapple_check.fields (v05)
- load_artifact f — pinneapple_flow.runtime (v07)
- load_baseline f — pinneapple_check.baseline (v05)
- load_body f — pinneapple_physics.astro.smallbodies (v02)
- load_bright_stars f — pinneapple_physics.astro.stars (v02)
- load_bundle f — pinneapple_tools.benchmark_suite.bundle.loader (v07)
- load_bundle_schema f — pinneapple_tools.benchmark_suite.bundle.schema (v07)
- load_caption_index f — pinneapple_design.geometry.retrieval (v06)
- load_cases f — pinneapple_analysis.validation.arena.cases (v05)
- load_catalog f — pinneapple_worldmodel.generate_datasets (v04)
- load_checkpoint f — pinneapple_neural.trainer.checkpoint (v04)
- load_checkpoint f — pinneapple_neural.trainer.inference (v04)
- load_config f — pinneapple_check.config (v05)
- load_constellations f — pinneapple_physics.astro.stars (v02)
- load_curation f — pinneapple_lab.seal (v07)
- load_darcy f — pinneapple_tools.benchmark_suite.crossframework.darcy (v07)
- load_dataset f — pinneapple_data.datasets (v04)
- load_dataset f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- load_dataset f — pinneapple_simulation.terramechanics.testbed (v03)
- load_defence f — pinneapple_analysis.validation.arena.cases (v05)
- load_dense_volume_operator f — pinneapple_neural.workflows.dense_volume_operator (v04)
- load_dense_volumes f — pinneapple_data.adapters.splash_archive_adapter (v04)
- load_dir f — pinneapple_hub.zoo._models (v07)
- load_dotenv f — pinneapple_flow.config (v07)
- load_exported_odb_npz f — pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader (v03)
- load_forecaster f — pinneapple_physics.blackhole.forecast (v02)
- load_geometry_asset f — pinneapple_data.adapters.geom_adapter (v04)
- load_geometry_asset f — pinneapple_design.geometry.core.registry (v06)
- load_hdf5 f — pinneapple_data.serialization (v04)
- load_history f — pinneapple_physics.astro.spaceweather (v02)
- load_index f — pinneapple_hub.library (v07)
- load_into_linears f — pinneapple_tools.benchmark_suite.crossframework.weights (v07)
- load_json_file f — pinneapple_llm.qa.static (v07)
- load_leaderboard f — pinneapple_tools.benchmark_suite.runner.leaderboard (v07)
- load_manifest f — pinneapple_data.serialization (v04)
- load_mat f — pinneapple_simulation.external_solvers.matlab.mat_io (v03)

- load_mesh f — pinneapple_data.adapters.splash_archive_adapter (v04)
- load_mesh f — pinneapple_simulation.external_solvers.openfoam.mesh_reader (v03)
- load_mesh_cache f — pinneapple_data.adapters.splash_archive_adapter (v04)
- load_meshio f — pinneapple_check.fields (v05)
- load_meshio f — pinneapple_design.geometry.io.meshio_bridge (v06)
- load_model f — pinneapple_check.models (v05)
- load_model f — pinneapple_hub.zoo._models (v07)
- load_model f — pinneapple_simulation.external_solvers.mujoco.loader (v03)
- load_model f — pinneapple_systems.operations.contracts (v06)
- load_object f — pinneapple_flow.nodes_ext.examples (v07)
- load_openfoam f — pinneapple_check.fields (v05)
- load_package f — pinneapple_simulation.electromagnetics.antenna.package (v03)
- load_parametric_dense_volume_operator f — pinneapple_neural.workflows.parametric_dense_volume_operator (v04)
- load_pinneapple_dataset f — pinneapple_arena.trainer (v04)
- load_point_set f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- load_predictor f — pinneapple_simulation.external_solvers.calculix.virtual_ccx (v03)
- load_preset f — pinneapple_app.backend.core.problem (v07)
- load_pt f — pinneapple_data.serialization (v04)
- load_qoi f — pinneapple_check.fields (v05)
- load_real_dataset f — pinneapple_data.datasets.real_world (v04)
- load_result f — pinneapple_check.fields (v05)
- load_reviews f — pinneapple_lab.curation (v07)
- load_run f — pinneapple_flow.nodes.vehicle (v07)
- load_run f — pinneapple_physics.blackhole.runs (v02)
- load_runs f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- load_runs f — pinneapple_design.aero.train (v06)
- load_safetensors f — pinneapple_hub.zoo.export (v07)
- load_snapshots f — pinneapple_data.snapshots (v04)
- load_solution f — pinneapple_flow.nodes._common (v07)
- load_source f — pinneapple_veriphysics.evidence_package (v05)
- load_split f — pinneapple_tools.benchmark_suite.crossframework.darcy (v07)
- load_stl f — pinneapple_data.stl_import (v04)
- load_stl f — pinneapple_design.geometry.io.stl (v06)
- load_stl_bytes f — pinneapple_data.stl_import (v04)
- load_sunspots_yearly f — pinneapple_systems.time_series.datasets.public (v06)
- load_surrogate f — pinneapple_presets.surrogate (v02)
- load_surrogate f — pinneapple_simulation.terramechanics.surrogate (v03)
- load_surrogates f — pinneapple_design.thermal.heatsink.surrogate (v06)
- load_table f — pinneapple_data.data_health (v04)

- `load_timeseries_csv` f — `pinneapple_systems.time_series.csv_loader` (v06)
- `load_trajectories` f — `pinneapple_physics.blackhole.dataset` (v02)
- `load_upd_item` f — `pinneapple_data.adapters.upd_adapter` (v04)
- `load_usd_eur_daily` f — `pinneapple_systems.time_series.datasets.public` (v06)
- `load_vlm` f — `pinneapple_tools.dataset_quality.vlm_curation` (v07)
- `load_yaml` f — `pinneapple_tools.benchmark_suite.io.yamlx` (v07)
- `load_youtube_trending_us` f — `pinneapple_systems.time_series.datasets.public` (v06)
- `load_zarr` f — `pinneapple_data.serialization` (v04)
- `loadavg` f — `pinneapple_tools.benchmark_suite.crossframework.records` (v07)
- `LoadedModel` C — `pinneapple_check.models` (v05)
- `LoadError` C — `pinneapple_check.fields` (v05)
- `LoadInfo` C — `pinneapple_data.data_health` (v04)
- `local_command_for` f — `pinneapple_arena.external.local_command` (v04)
- `local_commands` f — `pinneapple_arena.external.dataset_probe` (v04)
- `local_derivatives` f — `pinneapple_analysis.sensitivity.effects` (v05)
- `LocalDB` C — `pinneapple_core.service.db` (v01)
- `LocalExecutor` C — `pinneapple_core.workflows.executors` (v01)
- `LocalSensitivity` C — `pinneapple_analysis.inverse_problems.sensitivity` (v05)
- `LocalSensitivityResult` C — `pinneapple_analysis.inverse_problems.sensitivity` (v05)
- `locate` f — `pinneapple_tools.rea_bridge.evidence` (v07)
- `Location` C — `pinneapple_check.result` (v05)
- `locations` f — `pinneapple_data.cae.compare` (v04)
- `LockedDropout` C — `pinneapple_neural.architectures.continuous.bayesian_rnn` (v04)
- `lodo` f — `pinneapple_simulation.electromagnetics.antenna.surrogates` (v03)
- `lodo_rmse_bars` f — `pinneapple_simulation.electromagnetics.antenna.plots` (v03)
- `loft` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `loft` f — `pinneapple_design.geometry_library.kit` (v06)
- `loft` f — `pinneapple_twin3d.primitives` (v06)
- `log_evidence` f — `pinneapple_simulation.numerical_solvers.gated_xtfc` (v03)
- `log_loss` f — `pinneapple_systems.sports.football.metrics` (v06)
- `log_result` f — `pinneapple_physics.tracking` (v02)
- `log_returns` f — `pinneapple_systems.time_series.datasets.public` (v06)
- `log_softmax` f — `pinneapple_decision.debias` (v05)
- `log_subdomain_boundaries` f — `pinneapple_simulation.numerical_solvers.xtfc_subdomain` (v03)
- `log_ticks_ok` f — `pinneapple_science.plots` (v02)
- `LogCollector` C — `pinneapple_veriphysics.execution_log` (v05)
- `LogEntry` C — `pinneapple_veriphysics.execution_log` (v05)
- `LoginThrottle` C — `pinneapple_security.web` (v05)
- `LogProbLLMBackend` C — `pinneapple_decision.backends` (v05)

- LogStep C — pinneapple_neural.trainer.preprocess (v04)
- lomb_scargle_periodogram f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- long_run_statistics f — pinneapple_analysis.validation.rollout (v05)
- look f — pinneapple_physics.astro.satellites (v02)
- lookup_phenomenon f — pinneapple_problemdesign.knowledge.physics_knowledge (v02)
- Loop C — pinneapple_design.aero.geometry (v06)
- Loop C — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- loop_field f — pinneapple_simulation.electromagnetics.statics (v03)
- loop_figures f — pinneapple_design.intent.figures (v06)
- loop_flux f — pinneapple_science.em.induction (v02)
- loop_gif f — pinneapple_design.intent.figures (v06)
- loop_on_axis_B f — pinneapple_simulation.electromagnetics.statics (v03)
- loop_scene f — pinneapple_systems.digital_twin.pipe_loop.scene (v06)
- loop_self_inductance f — pinneapple_simulation.electromagnetics.statics (v03)
- LoopCarTemplate C — pinneapple_design.intent.templates.loop_car (v06)
- LoopDesign C — pinneapple_simulation.flight.control (v03)
- LoopTrack C — pinneapple_design.intent.physics.loop_track (v06)
- LoRALinear C — pinneapple_worldmodel.mega_model (v04)
- Lorentz C — pinneapple_simulation.electromagnetics.media (v03)
- lorenz63 f — pinneapple_systems.time_series.synthetic (v06)
- lorenz63_missing_xz_rhs f — pinneapple_systems.time_series.forecast_lab.presets (v06)
- lorenz63_rhs f — pinneapple_systems.time_series.synthetic (v06)
- lorenz63_step f — pinneapple_analysis.data_assimilation.models (v05)
- lorenz96 f — pinneapple_systems.time_series.synthetic (v06)
- lorenz96_rhs f — pinneapple_systems.time_series.synthetic (v06)
- lorenz96_step f — pinneapple_analysis.data_assimilation.models (v05)
- lorenz96_tendency f — pinneapple_analysis.data_assimilation.models (v05)
- LorenzDiscovery C — pinneapple_lab.experiments.discovery (v07)
- LossFit C — pinneapple_simulation.electromagnetics.machines.steel (v03)
- LossRatioBalancer C — pinneapple_neural.trainer.weight_scheduler (v04)
- LossTracker C — pinneapple_systems.digital_twin.pipe_loop.diagnose (v06)
- LossWeights C — pinneapple_physics.pinn_solver.compiler.loss (v02)
- LossyPlaneWave C — pinneapple_science.em.waves (v02)
- low_modes f — pinneapple_physics.multiscale.filters (v02)
- LRAnnealing C — pinneapple_neural.trainer.loss_balancer (v04)
- LRUCache C — pinneapple_data.cache (v04)
- lshape f — pinneapple_design.geometry.csg (v06)
- LShapeDomain2D C — pinneapple_design.geometry.gen.domains (v06)

- `LSTMForecaster` C — `pinneapple_systems.time_series.models.recurrent` (v06)
- `LSTMModel` C — `pinneapple_neural.architectures.recurrent.lstm` (v04)
- `lube_2005` f — `pinneapple_flow.nodes_ext.geomechanics` (v07)
- `luhn_ok` f — `pinneapple_security.privacy` (v05)
- `luminosity_group` f — `pinneapple_physics.astro.stars_ml` (v02)
- `lumped_capacitance` f — `pinneapple_physics.physics_audit.assumptions` (v02)
- `lumped_link` f — `pinneapple_simulation.multibody.arm` (v03)
- `LumpedCoolingModel` C — `pinneapple_analysis.inverse_problems.models` (v05)
- `LumpedTankModel` C — `pinneapple_systems.operations.twin` (v06)
- `lunar_eclipses` f — `pinneapple_physics.astro.phenomena` (v02)
- `lunar_lambert` f — `pinneapple_tools.visualization.studio.shadows` (v07)
- `lunar_phase_events` f — `pinneapple_physics.astro.phenomena` (v02)
- `lunar_rover` f — `pinneapple_twin3d.gallery` (v06)
- `LunarTerrain` C — `pinneapple_simulation.rover.terrain` (v03)
- `lvl` f — `pinneapple_analysis.benchmark.runner` (v05)
- `lyapunov` f — `pinneapple_flow.nodes_ext.forecasting` (v07)
- `lyapunov_study` f — `pinneapple_systems.time_series.forecast_lab.pipeline` (v06)

M

- `m1_advection` f — `pinneapple_condense.falsify` (v02)
- `m2_stokes` f — `pinneapple_condense.falsify` (v02)
- `m3_hybrid` f — `pinneapple_condense.falsify` (v02)
- `m4_meshless_condensed` f — `pinneapple_condense.falsify` (v02)
- `m_cr` f — `pinneapple_reason.modes.structural` (v05)
- `mach` f — `pinneapple_physics.physics_audit.regime` (v02)
- `mach_from_area_ratio` f — `pinneapple_physics.closed_form.rocket_nozzle` (v02)
- `Machine` C — `pinneapple_llm.workbench.models` (v07)
- `machine` f — `pinneapple_presets.cases.ipm_traction` (v02)
- `machine_network` f — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `machine_reynolds_number` f — `pinneapple_systems.process_components.similarity_map` (v06)
- `MachineGeometry` C — `pinneapple_simulation.electromagnetics.machines.geometry` (v03)
- `MachineMesh` C — `pinneapple_simulation.electromagnetics.machines.mesher` (v03)
- `MachineSolution` C — `pinneapple_simulation.electromagnetics.machine` (v03)
- `MachineThermalData` C — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `mackey_glass` f — `pinneapple_systems.time_series.synthetic` (v06)
- `mackey_glass_lyapunov` f — `pinneapple_systems.time_series.synthetic` (v06)
- `mackey_glass_rhs` f — `pinneapple_systems.time_series.synthetic` (v06)
- `mackey_glass_step` f — `pinneapple_systems.time_series.synthetic` (v06)
- `mackey_glass_stepper` f — `pinneapple_systems.time_series.synthetic` (v06)

- `MAE` C — `pinneapple_neural.trainer.metrics` (v04)
- `mae` f — `pinneapple_physics.metrics` (v02)
- `mae` f — `pinneapple_systems.time_series.metrics` (v06)
- `mae` f — `pinneapple_systems.time_series.metrics_ext.point` (v06)
- `mae_fn` f — `pinneapple_systems.time_series.metrics` (v06)
- `mag_to_nl` f — `pinneapple_physics.astro.observing` (v02)
- `Magnet` C — `pinneapple_simulation.electromagnetics.materials` (v03)
- `magnet_eddy_loss_resistance_limited` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `magnet_rings` f — `pinneapple_science.em.induction` (v02)
- `magnet_temperature_sweep` f — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `MagnetGrade` C — `pinneapple_simulation.electromagnetics.machines.steel` (v03)
- `magnetized_cylinder` f — `pinneapple_simulation.electromagnetics.analytic` (v03)
- `magnetized_ring` f — `pinneapple_simulation.electromagnetics.analytic` (v03)
- `MagnetostaticSolution` C — `pinneapple_simulation.electromagnetics.fem` (v03)
- `magnitude` f — `pinneapple_physics.astro.phenomena` (v02)
- `magnitude` f — `pinneapple_physics.astro.satellites` (v02)
- `MahalanobisDetector` C — `pinneapple_systems.digital_twin.monitoring.anomaly` (v06)
- `main` f — `pinneapple_analysis.benchmark.cli` (v05)
- `main` f — `pinneapple_analysis.validation.arena.__main__` (v05)
- `main` f — `pinneapple_analysis.validation.fault_injection.__main__` (v05)
- `main` f — `pinneapple_arena.external.__main__` (v04)
- `main` f — `pinneapple_blender.blender_addon.import_pinneapple_sequence` (v07)
- `main` f — `pinneapple_check.cli` (v05)
- `main` f — `pinneapple_core.onboarding.cli` (v01)
- `main` f — `pinneapple_design.aero.render_blender` (v06)
- `main` f — `pinneapple_design.intent.cli` (v06)
- `main` f — `pinneapple_design.thermal.heatsink.surrogate` (v06)
- `main` f — `pinneapple_flow.__main__` (v07)
- `main` f — `pinneapple_flow.examples` (v07)
- `main` f — `pinneapple_flow.nodes_ext.render_worker` (v07)
- `main` f — `pinneapple_hub.library.__main__` (v07)
- `main` f — `pinneapple_hub.zoo.web.__main__` (v07)
- `main` f — `pinneapple_lab.__main__` (v07)
- `main` f — `pinneapple_llm.qa.examples.pipe_wear_twin_main` (v07)
- `main` f — `pinneapple_llm.workbench.__main__` (v07)
- `main` f — `pinneapple_mcp` (v07)
- `main` f — `pinneapple_mcp.server` (v07)
- `main` f — `pinneapple_presets.__main__` (v02)
- `main` f — `pinneapple_problemdesign.cli` (v02)
- `main` f — `pinneapple_reason.__main__` (v05)
- `main` f — `pinneapple_science.__main__` (v02)
- `main` f — `pinneapple_security.__main__` (v05)
- `main` f — `pinneapple_simulation.electromagnetics.antenna.ava` (v03)
- `main` f — `pinneapple_simulation.electromagnetics.periodic.tcda` (v03)

- `main` f — `pinneapple_simulation.external_solvers.calculix.virtual_ccx` (v03)
- `main` f — `pinneapple_simulation.external_solvers.cfd_formats._abaqus_odb_export_script` (v03)
- `main` f — `pinneapple_simulation.external_solvers.openfoam.campaign` (v03)
- `main` f — `pinneapple_simulation.pipelines.freecad_openfoam.__main__` (v03)
- `main` f — `pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job` (v03)
- `main` f — `pinneapple_tools.benchmark_suite.runner.run_arena_yaml` (v07)
- `main` f — `pinneapple_tools.benchmark_suite.runner.run_models_from_cfg` (v07)
- `main` f — `pinneapple_tools.benchmark_suite.runner.run_pipeline` (v07)
- `main` f — `pinneapple_tools.model_services.__main__` (v07)
- `main` f — `pinneapple_tools.nvidia._brep_worker` (v07)
- `main` f — `pinneapple_worldmodel.generate_datasets` (v04)
- `main` f — `pinneapple_worldmodel.train` (v04)
- `main_roll` f — `pinneapple_simulation.electromagnetics.reflector.study_report` (v03)
- `main_wave_speed` f — `pinneapple_reason.modes.fluids` (v05)
- `make_acoustic_physics_loss` f — `pinneapple_analysis.inspection.acoustic` (v05)
- `make_action` f — `pinneapple_lab.next_steps` (v07)
- `make_actuation` f — `pinneapple_simulation.numerical_solvers.thermal_channel_datasets` (v03)
- `make_annulus` f — `pinneapple_worldmodel.geometry` (v04)
- `make_backend` f — `pinneapple_llm.workbench.backend` (v07)
- `make_blocks` f — `pinneapple_physics.blackhole.forecast` (v02)
- `make_car` f — `pinneapple_autolab.domains.racecar` (v07)
- `make_cases` f — `pinneapple_systems.digital_twin.pipe_loop.benchmark` (v06)
- `make_cavity` f — `pinneapple_worldmodel.geometry` (v04)
- `make_channel` f — `pinneapple_worldmodel.geometry` (v04)
- `make_channel_with_cylinder` f — `pinneapple_worldmodel.geometry` (v04)
- `make_data` f — `pinneapple_simulation.external_solvers.mujoco.loader` (v03)
- `make_dataset` f — `pinneapple_physics.astro.neural_propagator` (v02)
- `make_eddy_current_physics_loss` f — `pinneapple_analysis.inspection.eddy_current` (v05)
- `make_emat_physics_loss` f — `pinneapple_analysis.inspection.emat` (v05)
- `make_estimator` f — `pinneapple_systems.operations.control` (v06)
- `make_fd_residual_fn` f — `pinneapple_data.synth.pde_symbolic` (v04)
- `make_figure` f — `pinneapple_tools.visualization.style` (v07)

- make_figures f — pinneapple_veriphysics.evidence_package (v05)
- make_gap f — pinneapple_lab.next_steps (v07)
- make_geometry f — pinneapple_design.intent.package (v06)
- make_gif f — pinneapple_tools.visualization.animation (v07)
- make_goal f — pinneapple_lab.experiments.bricks (v07)
- make_guided_wave_physics_loss f — pinneapple_analysis.inspection.guided_wave_ultrasonic (v05)
- make_l_shaped f — pinneapple_worldmodel.geometry (v04)
- make_lags f — pinneapple_systems.time_series.features.engineering (v06)
- make_linear_system f — pinneapple_neural.architectures.rom.toy_systems (v04)
- make_map f — pinneapple_systems.process_components.similarity_map (v06)
- make_mfl_physics_loss f — pinneapple_analysis.inspection.magnetic_flux_leakage (v05)
- make_model f — pinneapple_autolab.surrogates (v07)
- make_phased_array_physics_loss f — pinneapple_analysis.inspection.phased_array_ultrasonic (v05)
- make_point_set f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- make_recipe f — pinneapple_data.unified (v04)
- make_server f — pinneapple_tools.model_services.server (v07)
- make_splits f — pinneapple_analysis.surrogate.splits (v05)
- make_twin f — pinneapple_systems.operations.live (v06)
- make_union_circles_sdf f — pinneapple_design.geometry.gen.genetic_sdf (v06)
- make_unit_square f — pinneapple_worldmodel.geometry (v04)
- MAMLConfig C — pinneapple_adaptation.meta_learning.config (v04)
- MAMLTrainer C — pinneapple_adaptation.meta_learning.maml (v04)
- Manager C — pinneapple_llm.workbench.server (v07)
- Manifest C — pinneapple_lab.manifest (v07)
- Manifest C — pinneapple_security.integrity (v05)
- Manifest C — pinneapple_systems.time_series.enso.data (v06)
- manifest f — pinneapple_hub.zoo._artifacts (v07)
- manifest f — pinneapple_systems.sports.football.data (v06)
- manifest_from_dir f — pinneapple_lab.manifest (v07)
- ManifestImport C — pinneapple_lab.experiments.manifest_import (v07)
- manifests f — pinneapple_systems.time_series.enso.data (v06)
- manual_install f — pinneapple_arena.external.result (v04)
- ManufacturabilityConstraint C — pinneapple_design.design_optimizer.constraints (v06)
- ManufacturedSolutionCheck C — pinneapple_problemdesign.preset_authoring (v02)
- map_axes f — pinneapple_design.thermal.board_cooling.simulate (v06)
- MapCNNForecaster C — pinneapple_systems.time_series.enso.forecasters (v06)
- MapCoefficients C — pinneapple_systems.process_components.similarity_map (v06)
- mape f — pinneapple_systems.time_series.metrics (v06)
- mape f — pinneapple_systems.time_series.metrics_ext.point (v06)
- mape f — pinneapple_tools.model_evaluation.metrics (v07)
- mape_fn f — pinneapple_systems.time_series.metrics (v06)

- `MapEvaluation` C — `pinneapple_systems.process_components.similarity_map` (v06)
- `marching_squares_boundary` f — `pinneapple_design.geometry.sample.marching_squares` (v06)
- `marching_squares_contours` f — `pinneapple_design.geometry.io.sdf_meshing` (v06)
- `margin_study` f — `pinneapple_analysis.uncertainty.margin` (v05)
- `margins_figure` f — `pinneapple_reason.report` (v05)
- `markdown` f — `pinneapple_analysis.benchmark.report` (v05)
- `markdown` f — `pinneapple_flow.coverage` (v07)
- `markdown` f — `pinneapple_flow.examples` (v07)
- `markdown` f — `pinneapple_hub.zoo.catalogue` (v07)
- `markdown_table` f — `pinneapple_simulation.accelerated_foam.benchmark` (v03)
- `markdown_table` f — `pinneapple_simulation.external_solvers.modelica.fmi_compat` (v03)
- `markdown_tables` f — `pinneapple_physics.multiscale.comparison` (v02)
- `markdown_tables` f — `pinneapple_systems.time_series.enso.pipeline` (v06)
- `markdown_tables` f — `pinneapple_systems.time_series.forecast_lab.pipeline` (v06)
- `Marker` C — `pinneapple_design.bricks.visuals` (v06)
- `Marker` C — `pinneapple_tools.visualization.dashboard.spec` (v07)
- `mase` f — `pinneapple_systems.time_series.metrics_ext.point` (v06)
- `mass` f — `pinneapple_core.fem` (v01)
- `mass_action_rate` f — `pinneapple_systems.process_components.reaction_kinetics` (v06)
- `mass_balance` f — `pinneapple_tools.automation.verify` (v07)
- `mass_balance_residual` f — `pinneapple_systems.operations.plant` (v06)
- `mass_flow_to_standard_volumetric_flow` f — `pinneapple_systems.process_components.real_gas_eos` (v06)
- `mass_model_residuals` f — `pinneapple_design.bricks.parts` (v06)
- `mass_properties` f — `pinneapple_simulation.flight.mass` (v03)
- `MassBalance` C — `pinneapple_security.cps` (v05)
- `MassConservationConstraint` C — `pinneapple_design.design_optimizer.constraints` (v06)
- `MassEnvelope` C — `pinneapple_physics.blackhole.forecast` (v02)
- `MassProjection` C — `pinneapple_physics.blackhole.forecast` (v02)
- `mat_to_upd` f — `pinneapple_simulation.external_solvers.matlab.field_reader` (v03)
- `match` f — `pinneapple_flow.planner.extensions` (v07)
- `match_fields` f — `pinneapple_data.cae.compare` (v04)
- `match_load` f — `pinneapple_systems.sports.football.kin_study` (v06)
- `Material` C — `pinneapple_design.intent.materials` (v06)
- `Material` C — `pinneapple_simulation.external_solvers.calculix.study` (v03)
- `material_fracture_2d` f — `pinneapple_physics.pde_environment.presets.multidisciplinary` (v02)
- `MaterialsProjectClient` C — `pinneapple_data.datasets.external_sources` (v04)

- matern_kernel f — pinneapple_analysis.data_assimilation.sensor_placement (v05)
- MATLABEngine C — pinneapple_simulation.external_solvers.matlab.engine (v03)
- matrix_from_quat f — pinneapple_tools.visualization.studio.animation (v07)
- max_abs f — pinneapple_analysis.reproduction (v05)
- max_abs f — pinneapple_physics.metrics (v02)
- max_climbable_grade f — pinneapple_simulation.rover.autolab.wheel_models (v03)
- MaxError C — pinneapple_neural.trainer.metrics (v04)
- maxwell_checks f — pinneapple_check._reuse (v05)
- maxwell_checks f — pinneapple_physics.maxwell_checks (v02)
- maxwell_trust_report f — pinneapple_physics.maxwell_checks (v02)
- MaxwellCheck C — pinneapple_physics.maxwell_checks (v02)
- maybe_compile f — pinneapple_neural.trainer.parallel (v04)
- mc_dropout_uncertainty f — pinneapple_systems.component_modeling.mc_dropout (v06)
- MCDropout C — pinneapple_analysis.uncertainty.mc_dropout (v05)
- MCDropoutConfig C — pinneapple_analysis.uncertainty.mc_dropout (v05)
- MCDropoutWrapper C — pinneapple_analysis.uncertainty.mc_dropout (v05)
- mcp f — pinneapple_hub.library.snippets (v07)
- md_dashboard f — pinneapple_tools.visualization.molecular (v07)
- MDState C — pinneapple_simulation.particle_dynamics.molecular_dynamics (v03)
- mean_abs f — pinneapple_analysis.reproduction (v05)
- mean_covariance_loss f — pinneapple_physics.pinn_solver.stochastic (v02)
- mean_curvature f — pinneapple_design.geometry.ops.curvature (v06)
- mean_interval_width f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- mean_nusselt_from_temperatures f — pinneapple_physics.closed_form.heat_exchanger (v02)
- mean_obliquity f — pinneapple_physics.astro.frames (v02)
- measure f — pinneapple_analysis.cost.profiler (v05)
- measure f — pinneapple_simulation.rover.autolab.estimation (v03)
- measure f — pinneapple_systems.operations.plant (v06)
- measure_design f — pinneapple_design.thermal.fin_array_speed (v06)
- measure_part f — pinneapple_llm.cad_draft (v07)
- measured f — pinneapple_flow.examples (v07)
- measured f — pinneapple_lab.next_steps (v07)
- measured_text f — pinneapple_lab.next_steps (v07)
- media_path f — pinneapple_lab.explorer (v07)
- media_urls f — pinneapple_lab.explorer (v07)
- Medium C — pinneapple_simulation.electromagnetics.media (v03)
- members f — pinneapple_physics.astro.conditions (v02)
- memorise_vs_learn f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- Mention C — pinneapple_design.intent.parse (v06)
- merge f — pinneapple_design.bricks.mesh (v06)

- `merge` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `merge` f — `pinneapple_design.geometry_library.kit` (v06)
- `merge` f — `pinneapple_design.surrogate_inverse.meshes` (v06)
- `merge` f — `pinneapple_presets.viz3d` (v02)
- `merge` f — `pinneapple_twin3d.primitives` (v06)
- `merge_into_spec` f — `pinneapple_problemdesign.merge` (v02)
- `merge_zoo` f — `pinneapple_systems.time_series.forecast_lab.study` (v06)
- `merkle_root` f — `pinneapple_security.integrity` (v05)
- `Mesh` C — `pinneapple_core.mesh` (v01)
- `Mesh` C — `pinneapple_data.cae.model` (v04)
- `mesh` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `mesh` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `mesh` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `mesh` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `mesh` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `mesh` f — `pinneapple_flow.nodes.problem` (v07)
- `Mesh2D` C — `pinneapple_design.geometry.mesh.mesher_2d` (v06)
- `Mesh2D` C — `pinneapple_simulation.electromagnetics.mesh` (v03)
- `Mesh3D` C — `pinneapple_design.geometry.mesh.mesher_3d` (v06)
- `mesh_convergence` f — `pinneapple_condense.benchmark` (v02)
- `mesh_courant_number` f — `pinneapple_data.cae.metrics` (v04)
- `mesh_figure` f — `pinneapple_simulation.pipelines.freecad_openfoam.post` (v03)
- `mesh_from_gmsh` f — `pinneapple_simulation.electromagnetics.mesh` (v03)
- `mesh_from_sdf_3d` f — `pinneapple_design.geometry.mesh.mesher_3d` (v06)
- `mesh_from_surface` f — `pinneapple_design.geometry.mesh.mesher_3d` (v06)
- `mesh_independence_study` f — `pinneapple_analysis.verification.convergence` (v05)
- `mesh_kind` f — `pinneapple_design.fitmesh.mesh_plan` (v06)
- `mesh_mass_properties` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `mesh_polygon_2d` f — `pinneapple_design.geometry.mesh.mesher_2d` (v06)
- `mesh_quality` f — `pinneapple_twin3d.primitives` (v06)
- `mesh_quality_check` f — `pinneapple_analysis.benchmark.validation` (v05)
- `mesh_quality_report` f — `pinneapple_design.geometry.mesh.mesher_2d` (v06)
- `mesh_quality_report_3d` f — `pinneapple_design.geometry.ops.clean` (v06)
- `mesh_rectangle_structured` f — `pinneapple_design.geometry.mesh.mesher_2d` (v06)
- `mesh_reflector` f — `pinneapple_simulation.electromagnetics.reflector.geometry` (v03)
- `mesh_sdf_2d` f — `pinneapple_design.geometry.mesh.mesher_2d` (v06)
- `mesh_sector` f — `pinneapple_simulation.electromagnetics.machines.mesher` (v03)
- `mesh_sizes` f — `pinneapple_design.intent.physics.bracket` (v06)

- mesh_study f — pinneapple_simulation.pipelines.freecad_openfoam.report (v03)
- mesh_study_figure f — pinneapple_simulation.pipelines.freecad_openfoam.report (v03)
- mesh_thickness f — pinneapple_tools.nvidia.brep (v07)
- mesh_to_fem_assembly_inputs f — pinneapple_simulation.numerical_solvers.adapters.upd_mesh (v03)
- mesh_to_fvm_topology f — pinneapple_simulation.numerical_solvers.adapters.upd_mesh (v03)
- mesh_to_obstacle_mask f — pinneapple_design.geometry.ops.lbm_bridge (v06)
- mesh_to_pointcloud f — pinneapple_design.geometry.ops.pointcloud (v06)
- mesh_views f — pinneapple_veriphysics.mesh_views (v05)
- mesh_wall_distance f — pinneapple_data.cae.metrics (v04)
- MeshBuilder C — pinneapple_design.geometry.gen.primitives (v06)
- MeshCollocator C — pinneapple_design.geometry.mesh_collocator (v06)
- MeshCollocatorConfig C — pinneapple_design.geometry.mesh_collocator (v06)
- MeshData C — pinneapple_design.geometry.core.mesh (v06)
- MeshDynamicsMGN C — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- MeshGeometry C — pinneapple_simulation.external_solvers.openfoam.mesh_reader (v03)
- MeshGraph C — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- MeshGraphNet C — pinneapple_neural.architectures.graphnn.mesh_graph_net (v04)
- meshgraphnet f — pinneapple_flow.nodes.methods (v07)
- MeshIndependenceStudyResult C — pinneapple_analysis.verification.convergence (v05)
- MeshInfo C — pinneapple_arena.pointwise.schema (v04)
- meshio_to_upd f — pinneapple_design.geometry.io.meshio_bridge (v06)
- meshless_checks f — pinneapple_simulation.meshless.contracts (v03)
- meshless_compare f — pinneapple_flow.wrappers (v07)
- meshless_convergence f — pinneapple_condense.figures (v02)
- meshless_fields f — pinneapple_condense.figures (v02)
- meshless_study f — pinneapple_condense.benchmark (v02)
- meshless_study f — pinneapple_simulation.meshless.study (v03)
- MeshlessSolution C — pinneapple_condense.meshless (v02)
- MeshlessStudy C — pinneapple_simulation.meshless.study (v03)
- MeshPlan C — pinneapple_design.fitmesh.mesh_plan (v06)
- MeshQualityReport C — pinneapple_analysis.verification.mesh_intelligence (v05)
- MeshResult C — pinneapple_design.fitmesh.mesh_gen (v06)
- MeshSampler C — pinneapple_core.data (v01)
- MeshSpec C — pinneapple_simulation.pipelines.freecad_openfoam.mesh (v03)
- meta f — pinneapple_flow.nodes_ext.benchmarks (v07)
- meta_adapt f — pinneapple_adaptation.meta_learning (v04)

- meta_learn f — pinneapple_flow.nodes.methods (v07)
- meta_train f — pinneapple_adaptation.meta_learning (v04)
- MetaBenchmarkConfig C — pinneapple_tools.benchmark_suite.meta_benchmark (v07)
- MetaBenchmarkFamily C — pinneapple_tools.benchmark_suite.meta_benchmark (v07)
- MetaBenchmarkPipeline C — pinneapple_tools.benchmark_suite.meta_benchmark (v07)
- MetaBenchmarkResult C — pinneapple_tools.benchmark_suite.meta_benchmark (v07)
- metabolic_power f — pinneapple_systems.sports.football.kinematics (v06)
- MetaConfig C — pinneapple_worldmodel.meta_learning (v04)
- MetaLearner C — pinneapple_worldmodel.meta_learning (v04)
- MetaModel C — pinneapple_adaptation.meta_learning.meta_model (v04)
- metamorphic f — pinneapple_check._reuse (v05)
- MetamorphicReport C — pinneapple_analysis.validation.metamorphic (v05)
- Method C — pinneapple_analysis.benchmark.registry (v05)
- Method C — pinneapple_catalog.methods (v01)
- Method C — pinneapple_query.core (v02)
- method_catalog f — pinneapple_check._reuse (v05)
- method_status f — pinneapple_catalog.methods (v01)
- MethodComparison C — pinneapple_design.design_optimizer.compare (v06)
- MethodNotAvailable C — pinneapple_physics.solving (v02)
- MethodRecommendation C — pinneapple_problemdesign.method_selection (v02)
- methods f — pinneapple_analysis.benchmark.registry (v05)
- methods_of f — pinneapple_lab.manifest (v07)
- Metric C — pinneapple_lab.results_rules (v07)
- Metric C — pinneapple_neural.trainer.metrics (v04)
- metric_value f — pinneapple_flow.nodes_ext.examples (v07)
- metrica_match f — pinneapple_systems.sports.football.data (v06)
- MetricaMatch C — pinneapple_systems.sports.football.data (v06)
- MetricBundle C — pinneapple_neural.trainer.metrics (v04)
- MetricItem C — pinneapple_app.backend.routers.models (v07)
- Metrics C — pinneapple_neural.trainer.metrics (v04)
- metrics f — pinneapple_app.backend.main (v07)
- metrics f — pinneapple_design.aero.train (v06)
- metropolis f — pinneapple_analysis.inverse_problems.estimation (v05)
- metzner_reed_friction_factor f — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- mgn_cylinder_flow f — pinneapple_flow.nodes_ext.learned_sim (v07)
- MGNMethod C — pinneapple_analysis.benchmark.builtin.airfoil (v05)
- michaud_head f — pinneapple_simulation.numerical_solvers.water_hammer_moc (v03)
- microstrip_width f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- mie f — pinneapple_simulation.electromagnetics.mie (v03)
- mie_coefficients f — pinneapple_simulation.electromagnetics.mie (v03)

- mie_pec f — pinneapple_simulation.electromagnetics.mie (v03)
- MieResult C — pinneapple_simulation.electromagnetics.mie (v03)
- MieScattering C — pinneapple_science.em.waves (v02)
- min_calibration_size f — pinneapple_analysis.uncertainty.conformal_study (v05)
- min_distance f — pinneapple_data.doe.designs (v04)
- min_length f — pinneapple_simulation.electromagnetics.antenna.bounds (v03)
- MinerDamageResult C — pinneapple_systems.process_components.fatigue_analysis (v06)
- miners_rule_damage f — pinneapple_systems.process_components.fatigue_analysis (v06)
- minetti_cost f — pinneapple_systems.sports.football.kinematics (v06)
- minimize f — pinneapple_analysis.validation.arena.search (v05)
- minimum_slope f — pinneapple_reason.modes.water (v05)
- MinMaxScaler C — pinneapple_data.transforms (v04)
- mirror f — pinneapple_design.geometry_library.kit (v06)
- mirror_z f — pinneapple_twin3d.primitives (v06)
- MissingValueStep C — pinneapple_neural.trainer.preprocess (v04)
- Mission C — pinneapple_autolab.mission (v07)
- Mission C — pinneapple_design.aero.airliner3d (v06)
- mission_numbers f — pinneapple_simulation.rover.autolab.report (v03)
- MissionDomain C — pinneapple_autolab.agent (v07)
- MissionReading C — pinneapple_autolab.agent (v07)
- MissionRefused C — pinneapple_autolab.agent (v07)
- MissionResult C — pinneapple_simulation.rover.autolab.mission (v03)
- MissionSpec C — pinneapple_simulation.rover.autolab.mission (v03)
- mixed_areas f — pinneapple_design.geometry.ops.curvature (v06)
- mixing_tank f — pinneapple_twin3d.gallery (v06)
- MJXParallelRunner C — pinneapple_simulation.external_solvers.mujoco.mjx_backend (v03)
- mli_blanket f — pinneapple_design.geometry.vehicle_cad (v06)
- MLP C — pinneapple_tools.benchmark_suite.backends.native_pinn (v07)
- mlp f — pinneapple_analysis.validation.pinn (v05)
- mlp f — pinneapple_neural.architectures.neural_operators.deeponet (v04)
- MLPDenoiser C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- MLPEnsemble C — pinneapple_autolab.surrogates (v07)
- MLPEnsemble C — pinneapple_design.aero.surrogate (v06)
- MLPFieldsMethod C — pinneapple_analysis.benchmark.builtin.airfoil (v05)
- MLPFNOSurrogate C — pinneapple_neural.architectures.neural_operators.fno (v04)
- MLPForecaster C — pinneapple_systems.time_series.models.classical (v06)
- MLPMethod C — pinneapple_analysis.benchmark.builtin.airfoil (v05)

- MLPredictor C — pinneapple_systems.sports.football.kinematics (v06)
- MLPSurrogate C — pinneapple_design.surrogate_inverse.core (v06)
- MLPSurrogate C — pinneapple_tools.nvidia.physicsnemo (v07)
- MLSApproximation C — pinneapple_simulation.numerical_solvers.meshfree (v03)
- mlx_train_command f — pinneapple_llm.workbench.finetune (v07)
- MMA C — pinneapple_design.topology.optimizers (v06)
- mobility_analysis f — pinneapple_simulation.terramechanics.mobility (v03)
- moc_valve_closure f — pinneapple_systems.process_components.water_hammer (v06)
- MockStream C — pinneapple_systems.digital_twin.io.stream (v06)
- ModbusStream C — pinneapple_systems.digital_twin.io.stream (v06)
- Model C — pinneapple_analysis.validation.physics_learning_cases (v05)
- Model C — pinneapple_simulation.pipelines.freecad_openfoam.models (v03)
- model f — pinneapple_hub.library.snippets (v07)
- model_architecture f — pinneapple_flow.wrappers (v07)
- model_card f — pinneapple_hub.zoo.hub_cards (v07)
- model_card_section f — pinneapple_analysis.validation.report_artifact (v05)
- model_example f — pinneapple_hub.zoo.hub_cards (v07)
- model_faults f — pinneapple_analysis.validation.fault_injection.faults (v05)
- model_forward f — pinneapple_arena.pointwise.training (v04)
- model_from_dict f — pinneapple_systems.operations.twin (v06)
- model_from_ref f — pinneapple_analysis.study_contracts (v05)
- model_from_spec f — pinneapple_analysis.inverse_problems.presets (v05)
- model_ref f — pinneapple_analysis.surrogate.contracts (v05)
- model_ref f — pinneapple_neural.architectures.rom.contracts (v04)
- model_ref_for_surrogate f — pinneapple_design.design_optimizer.report (v06)
- model_ref_target f — pinneapple_veriphysics.validation_lab (v05)
- model_scope f — pinneapple_design.thermal.heatsink.engine (v06)
- model_scope f — pinneapple_physics.pcb.engine (v02)
- model_to_ref f — pinneapple_analysis.study_contracts (v05)
- ModelAdapter C — pinneapple_neural.architectures.adapters.base (v04)
- ModelAudit C — pinneapple_analysis.validation.arena.loop (v05)
- ModelBoard C — pinneapple_autolab.surrogates (v07)
- ModelCard C — pinneapple_hub.model_card (v07)
- ModelCatalog C — pinneapple_neural.architectures.catalog (v04)
- ModelCheckpoint C — pinneapple_neural.trainer.callbacks (v04)
- ModelConfig C — pinneapple_arena.config (v04)
- ModelD C — pinneapple_check.numerics (v05)
- ModelEntry C — pinneapple_app.backend.core.model_catalog (v07)
- ModelGateway C — pinneapple_registry.gateway (v01)
- modelica_result_to_upd f — pinneapple_simulation.external_solvers.modelica.result_reader (v03)

- ModelListItem C — pinneapple_app.backend.routers.models (v07)
- ModelLoadError C — pinneapple_check.models (v05)
- ModelOutput C — pinneapple_neural.architectures.base (v04)
- ModelProfile C — pinneapple_physics.physics_audit.assumptions (v02)
- ModelRef C — pinneapple_data.contracts.models (v04)
- ModelRegistry C — pinneapple_neural.architectures.registry (v04)
- ModelRequest C — pinneapple_app.backend.routers.experiments (v07)
- ModelResult C — pinneapple_app.backend.core.experiment (v07)
- ModelResult C — pinneapple_arena.external.result (v04)
- ModelRunConfig C — pinneapple_app.backend.core.experiment (v07)
- ModelRunResult C — pinneapple_tools.benchmark_suite.report (v07)
- models f — pinneapple_hub.library.sources (v07)
- models f — pinneapple_hub.zoo.demo (v07)
- ModelSelector C — pinneapple_decision.selectors (v05)
- ModelService C — pinneapple_tools.model_services.registry (v07)
- ModelServiceClient C — pinneapple_tools.model_services.client (v07)
- ModelSpec C — pinneapple_arena.external.schema (v04)
- ModelSpec C — pinneapple_neural.architectures.registry (v04)
- ModelSpec C — pinneapple_systems.digital_twin.product.spec (v06)
- ModelSpec C — pinneapple_systems.time_series.forecast_lab.study (v06)
- ModelSpec C — pinneapple_tools.benchmark_suite.benchmark (v07)
- ModelStore C — pinneapple_registry.model_store (v01)
- ModelSummaryItem C — pinneapple_app.backend.routers.admin (v07)
- ModelTarget C — pinneapple_check.config (v05)
- ModelZoo C — pinneapple_worldmodel.model_zoo (v04)
- modes f — pinneapple_simulation.flight.trim (v03)
- ModifiedMLP C — pinneapple_neural.architectures.modified_mlp (v04)
- modulated_gaussian f — pinneapple_simulation.electromagnetics.fdtd (v03)
- modules f — pinneapple_flow.coverage (v07)
- mohr_coulomb_traction_limit f — pinneapple_simulation.terramechanics.mobility (v03)
- MolecularDynamicsSimulator C — pinneapple_simulation.particle_dynamics.molecular_dynamics (v03)
- Molecule C — pinneapple_tools.visualization.molecular (v07)
- molecule_html f — pinneapple_tools.visualization.integrations.plotly_export (v07)
- monitor_slip f — pinneapple_simulation.rover.autolab.verify (v03)
- monitoring_url f — pinneapple_systems.digital_twin.io.usgs (v06)
- monomials f — pinneapple_simulation.meshless.stencils (v03)
- monotone_violation_share f — pinneapple_neural.trainer.monotonic (v04)

- Monotonicity C — pinneapple_analysis.validation.metamorphic (v05)
- monotonicity_penalty f — pinneapple_neural.trainer.monotonic (v04)
- monte_carlo_uncertainty f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- month_range f — pinneapple_systems.time_series.enso.data (v06)
- moon_brightness_nl f — pinneapple_physics.astro.observing (v02)
- moon_phase f — pinneapple_physics.astro.phenomena (v02)
- morris f — pinneapple_analysis.sensitivity.morris (v05)
- morris_design f — pinneapple_analysis.sensitivity.morris (v05)
- MorrisResult C — pinneapple_analysis.sensitivity.morris (v05)
- motor_glider f — pinneapple_simulation.flight.catalog (v03)
- Mound C — pinneapple_simulation.rover.autolab.site (v03)
- move_batch_to_device f — pinneapple_data.collate (v04)
- MovingAverage C — pinneapple_systems.time_series.adaptive (v06)
- MPMSimulator C — pinneapple_simulation.particle_dynamics.mpm (v03)
- MPMState C — pinneapple_simulation.particle_dynamics.mpm (v03)
- MQTTBroker C — pinneapple_systems.digital_twin.io.mqtt_broker (v06)
- MQTTStream C — pinneapple_systems.digital_twin.io.stream (v06)
- MSE C — pinneapple_neural.trainer.metrics (v04)
- mse f — pinneapple_analysis.reproduction (v05)
- mse f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- mse f — pinneapple_systems.time_series.metrics (v06)
- mse_fn f — pinneapple_systems.time_series.metrics (v06)
- mse_forecast f — pinneapple_systems.time_series.losses (v06)
- msis_available f — pinneapple_physics.astro.decay (v02)
- Mu0Solenoid C — pinneapple_science.em.bench (v02)
- MuJoCoConfig C — pinneapple_simulation.external_solvers.mujoco.loader (v03)
- MuJoCoRunner C — pinneapple_simulation.external_solvers.mujoco.runner (v03)
- multi_fidelity f — pinneapple_simulation.electromagnetics.antenna.design (v03)
- multi_horizon_mae f — pinneapple_systems.time_series.metrics_ext.point (v06)
- multi_indices f — pinneapple_analysis.uncertainty.pce (v05)
- multi_round_lbfgs f — pinneapple_neural.trainer.lbfgs_finetune (v04)
- MultiAdam C — pinneapple_neural.trainer.multiadam (v04)
- MultibodyTree C — pinneapple_simulation.multibody.tree (v03)
- MultiCameraArray C — pinneapple_worldmodel.camera_system (v04)
- multilayer f — pinneapple_simulation.electromagnetics.waves (v03)
- MultilayerResult C — pinneapple_simulation.electromagnetics.waves (v03)
- MultiObstacleDomain2D C — pinneapple_design.geometry.gen.domains (v06)
- MultiPeriodicBC C — pinneapple_physics.symbolic_pde.bc (v02)
- MultiRestartConfig C — pinneapple_neural.trainer.multistart (v04)
- MultiRestartResult C — pinneapple_neural.trainer.multistart (v04)

- MultiRestartTrainer C — pinneapple_neural.trainer.multistart (v04)
- multiscale_neighbourhood f — pinneapple_neural.lpm (v04)
- multiscale_residuals f — pinneapple_physics.multiscale.residuals (v02)
- MultiScaleDeepONet C — pinneapple_neural.architectures.neural_operators.ms_deeponet (v04)
- MultiscaleResidual C — pinneapple_physics.multiscale.residuals (v02)
- multiterm_periodogram f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- mutate f — pinneapple_design.bricks.search (v06)
- mutation_escapes f — pinneapple_analysis.validation.arena.red (v05)
- mutation_score f — pinneapple_analysis.validation.arena.red (v05)
- mutual_inductance_coaxial_loops f — pinneapple_simulation.electromagnetics.statics (v03)
- MyHub C — pinneapple_hub.library.myhub (v07)
- MyHubError C — pinneapple_hub.library.myhub (v07)

N

- n_cells f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- n_fins_for f — pinneapple_design.thermal.heatsink.surrogate (v06)
- n_terms f — pinneapple_analysis.uncertainty.pce (v05)
- naca4 f — pinneapple_design.aero.geometry (v06)
- naca4 f — pinneapple_design.geometry.vehicles3d (v06)
- naca4 f — pinneapple_design.geometry_library.kit (v06)
- naca4_camber_slope f — pinneapple_physics.closed_form.thin_airfoil (v02)
- naca4_mask f — pinneapple_simulation.numerical_solvers.lbm (v03)
- naca4_points f — pinneapple_design.fitmesh.catalog (v06)
- naca4_symmetric_polygon f — pinneapple_physics.pde_environment.presets._curve_geometry (v02)
- naca_parametric f — pinneapple_design.geometry.gen.airfoil (v06)
- naive_baselines f — pinneapple_flow.nodes_ext.core_ts (v07)
- NaiveForecaster C — pinneapple_systems.time_series.baselines.naive (v06)
- names f — pinneapple_design.geometry_library.core (v06)
- names f — pinneapple_physics.astro.facts (v02)
- names f — pinneapple_presets.registry (v02)
- names f — pinneapple_science.registry (v02)
- names f — pinneapple_simulation.flight.catalog (v03)
- names_in f — pinneapple_data.expressions (v04)
- names_in_tree f — pinneapple_data.expressions (v04)
- NASAClient C — pinneapple_data.datasets.external_sources (v04)
- NASAPowerClient C — pinneapple_data.datasets.external_sources (v04)
- NASResult C — pinneapple_arena.nas (v04)

- NativePINNBackend C — pinneapple_tools.benchmark_suite.backends.native_pinn (v07)
- natural_convection f — pinneapple_physics.closed_form.plate_fin_heatsink (v02)
- navier_stokes_2d_vorticity f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- NavierStokes C — pinneapple_analysis.benchmark.builtin.standard (v05)
- NavierStokes3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- NavierStokesConfig3D C — pinneapple_simulation.numerical_solvers.fdm3d (v03)
- NavierStokesTGV2DTask C — pinneapple_tools.benchmark_suite.tasks.navier_stokes_tgv_2d (v07)
- NBeats C — pinneapple_systems.time_series.models.nbeats (v06)
- NBeatsConfig C — pinneapple_systems.time_series.models.nbeats (v06)
- NBodySolver C — pinneapple_simulation.numerical_solvers.nbody_rebound (v03)
- ncei_heat_content f — pinneapple_systems.time_series.enso.data (v06)
- ndvi_change f — pinneapple_perception.change_detection (v04)
- NdviChangeResult C — pinneapple_perception.change_detection (v04)
- near_duplicates f — pinneapple_data.audit (v04)
- nearfield_planes f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- NearToFarField C — pinneapple_simulation.electromagnetics.fdtd (v03)
- Need C — pinneapple_reason.library (v05)
- need_problem f — pinneapple_flow.nodes._common (v07)
- needs_geometry_tags f — pinneapple_core.onboarding.catalog (v01)
- needs_rehash f — pinneapple_security.passwords (v05)
- NeedsInput C — pinneapple_design.intent.api (v06)
- NeighborList C — pinneapple_simulation.numerical_solvers.neighbors (v03)
- Network C — pinneapple_simulation.numerical_solvers.pipe_network (v03)
- NetworkConfig C — pinneapple_arena.config (v04)
- neumann_boundary_time_series f — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- NeumannBC f — pinneapple_physics.pde_environment.conditions (v02)
- NeumannBC C — pinneapple_physics.symbolic_pde.bc (v02)
- NeuralCDE C — pinneapple_neural.architectures.continuous.neural_cde (v04)
- NeuralForecaster C — pinneapple_systems.time_series.forecast_lab.learned (v06)
- NeuralGaussianProcess C — pinneapple_neural.architectures.continuous.neural_gp (v04)
- NeuralNetwork C — pinneapple_physics.pinn_solver.factory.pinn_factory (v02)

- NeuralODE C — pinneapple_neural.architectures.continuous.neural_ode (v04)
- NeuralOperatorBase C — pinneapple_neural.architectures.neural_operators.base (v04)
- NeuralOperatorCatalog C — pinneapple_neural.architectures.neural_operators.registry (v04)
- NeuralOperatorDataset C — pinneapple_worldmodel.training_hooks (v04)
- NeuralPropagator C — pinneapple_physics.astro.neural_propagator (v02)
- NeuralROM C — pinneapple_neural.architectures.rom.neural_rom (v04)
- NeuralSDE C — pinneapple_neural.architectures.continuous.neural_sde (v04)
- NeuralTermConfig C — pinneapple_analysis.inverse_problems.missing_term (v05)
- NeuralTermDiscovery C — pinneapple_analysis.inverse_problems.missing_term (v05)
- NeuralTuner C — pinneapple_systems.time_series.tuning.optuna_tuner (v06)
- new_id f — pinneapple_systems.operations.contracts (v06)
- new_key f — pinneapple_security.encryption (v05)
- new_main f — pinneapple_core.onboarding.cli (v01)
- new_project f — pinneapple_core.onboarding.newproject (v01)
- newton_available f — pinneapple_simulation.robot_sim.sim (v03)
- newton_check f — pinneapple_design.intent.physics.loop_track (v06)
- newton_scene f — pinneapple_design.intent.physics.loop_track (v06)
- newton_solve f — pinneapple_core.func (v01)
- newton_validate f — pinneapple_flow.wrappers (v07)
- NewtonLoopResult C — pinneapple_design.intent.physics.loop_track (v06)
- next_steps f — pinneapple_flow.lab_status (v07)
- next_steps f — pinneapple_lab.curation (v07)
- NextExperiment C — pinneapple_autolab.trust (v07)
- nfw_dark_matter_potential f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- nfw_potential_exact f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- nfw_source_fn f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- nino_indices f — pinneapple_systems.time_series.enso.data (v06)
- nl_to_mag f — pinneapple_physics.astro.observing (v02)
- NLS1D C — pinneapple_arena.problems (v04)
- NOAAClient C — pinneapple_data.datasets.external_sources (v04)
- nodal_boundary_values f — pinneapple_physics.darcy_checks (v02)
- nodal_darcy_residual f — pinneapple_physics.darcy_checks (v02)
- nodal_flux_balance f — pinneapple_physics.darcy_checks (v02)
- Node C — pinneapple_analysis.proof.graph (v05)
- Node C — pinneapple_flow.schema (v07)
- Node C — pinneapple_physics.physics_audit.knowledge (v02)
- node f — pinneapple_flow.registry (v07)
- node f — pinneapple_hub.library.snippets (v07)
- node_features f — pinneapple_design.aero.surrogate (v06)
- node_for_callable f — pinneapple_hub.library.sources (v07)

- node_states f — pinneapple_flow.runtime (v07)
- node_targets f — pinneapple_flow.coverage (v07)
- NodeContext C — pinneapple_flow.runtime (v07)
- nodes f — pinneapple_hub.library.sources (v07)
- nodes_for f — pinneapple_flow.lab_status (v07)
- NodeSet C — pinneapple_simulation.meshless.domains (v03)
- NodeSpec C — pinneapple_flow.registry (v07)
- NoetherABUPT C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherAeroABUPT C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherAeroTransformer C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherAeroTransolver C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherAeroUPT C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherDatasetBridge C — pinneapple_data.noether_dataset (v04)
- NoetherModelConfig C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherSurrogateTrainer C — pinneapple_neural.trainer.noether_trainer (v04)
- NoetherTrainConfig C — pinneapple_neural.trainer.noether_trainer (v04)
- NoetherTrainResult C — pinneapple_neural.trainer.noether_trainer (v04)
- NoetherTransformer C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherTransolver C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoetherUPT C — pinneapple_neural.architectures.neural_operators.noether_bridge (v04)
- NoFeasibleOptionError C — pinneapple_decision.constraints (v05)
- NoiseSchedulerBase C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- NOMADClient C — pinneapple_data.datasets.external_sources (v04)
- nominal_image f — pinneapple_worldmodel.inspection_data (v04)
- NoModel C — pinneapple_physics.multiscale.closures (v02)
- Nondimensionalize C — pinneapple_core.transforms (v01)
- nonlinear_prediction_error f — pinneapple_systems.time_series.predictability (v06)
- NonlinearBeamFEMSolver C — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem (v03)
- NonlinearBeamNewmarkFEM C — pinneapple_simulation.numerical_solvers.nonlinear_beam_fem

(v03)
- NonlinearMPC C — pinneapple_systems.operations.control (v06)
- NonNegative C — pinneapple_analysis.surrogate.methods (v05)
- nonnegative_mask f — pinneapple_analysis.surrogate.study (v05)
- norm_dot_grad f — pinneapple_physics.pinn_solver.compiler.autograd_ops (v02)
- norm_factor f — pinneapple_simulation.accelerated_foam.ldu (v03)
- normal_shock f — pinneapple_physics.closed_form.compressible (v02)
- normalise_format f — pinneapple_design.geometry_library.exporters (v06)
- normalise_front f — pinneapple_design.design_optimizer.methods (v06)
- normalised_residual f — pinneapple_simulation.accelerated_foam.ldu (v03)
- normalization_loss f — pinneapple_quantum.loss.schrodinger (v04)
- normalize f — pinneapple_decision.debias (v05)
- normalize f — pinneapple_tools.hpo_experiments.ranking (v07)
- normalize_01 f — pinneapple_data.synth.operators.transforms (v04)
- normalize_kwargs f — pinneapple_neural.architectures.instantiate (v04)
- normalize_label f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- normalize_weights f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- Normalizer C — pinneapple_data.transforms (v04)
- Normalizer C — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- NormalizeStep C — pinneapple_neural.trainer.preprocess (v04)
- not_applicable f — pinneapple_llm.qa.validators.base (v07)
- not_installed f — pinneapple_arena.external.result (v04)
- NotApplicableError C — pinneapple_tools.benchmark_suite.benchmark (v07)
- Note C — pinneapple_tools.reporting.datasheet (v07)
- NotInvertible C — pinneapple_core.transforms (v01)
- NotRun C — pinneapple_flow.runtime (v07)
- nozzle_performance f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- npb_matrix f — pinneapple_physics.astro.frames (v02)
- npsh f — pinneapple_reason.modes.fluids (v05)
- npsh_available f — pinneapple_physics.closed_form.pump_npsh (v02)
- NRELClient C — pinneapple_data.datasets.external_sources (v04)
- ns2d_multiscale_residuals f — pinneapple_physics.multiscale.residuals (v02)
- ns_incompressible_2d_default f — pinneapple_physics.pde_environment.presets.cfd (v02)
- ns_incompressible_3d_default f — pinneapple_physics.pde_environment.presets.cfd (v02)
- ns_inputs f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- NSFlowSolver C — pinneapple_simulation.numerical_solvers.cfd_pipeline (v03)

- nsga2 f — pinneapple_flow.nodes.methods (v07)
- NTKWeightBalancer C — pinneapple_neural.trainer.weight_scheduler (v04)
- ntu_from_effectiveness f — pinneapple_physics.closed_form.heat_exchanger (v02)
- num f — pinneapple_data.preflight.common (v04)
- NumbaStepper C — pinneapple_physics.blackhole.hydro_numba (v02)
- numbers f — pinneapple_physics.astro.facts (v02)
- numeric f — pinneapple_analysis.proof.graph (v05)
- numerical_dispersion f — pinneapple_simulation.electromagnetics.fdtd (v03)
- nutation f — pinneapple_physics.astro.frames (v02)
- nutation_matrix f — pinneapple_physics.astro.frames (v02)

O

- Objective C — pinneapple_design.design_optimizer.study (v06)
- Objective C — pinneapple_design.qualitative.preview (v06)
- ObjectiveBase C — pinneapple_design.design_optimizer.objective (v06)
- ObjectiveChoice C — pinneapple_design.intent.spec (v06)
- objectives_from f — pinneapple_design.design_optimizer.spec (v06)
- oblock_mesh_dict f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- observables f — pinneapple_condense.meshless (v02)
- Observation C — pinneapple_analysis.data_assimilation.var4d (v05)
- Observation C — pinneapple_llm.qa.spec (v07)
- Observation C — pinneapple_systems.digital_twin.state (v06)
- Observations C — pinneapple_analysis.inverse_problems.forward (v05)
- observations f — pinneapple_tools.benchmark_suite.crossframework.inverse (v07)
- observe f — pinneapple_flow.nodes.data (v07)
- observed_order f — pinneapple_science.core (v02)
- observed_orders f — pinneapple_simulation.electromagnetics.verification (v03)
- Observer C — pinneapple_physics.astro.astrometry (v02)
- observer_position f — pinneapple_physics.astro.journey (v02)
- ObsOperatorBase C — pinneapple_analysis.inverse_problems.obs_operator (v05)
- oc_update f — pinneapple_design.topology.optimizers (v06)
- occ_bottle f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- OceanGrid C — pinneapple_systems.time_series.enso.intermediate (v06)
- ocr_available f — pinneapple_data.formfill.ocr (v04)
- ocr_image f — pinneapple_data.formfill.ocr (v04)
- ocr_pdf_page f — pinneapple_data.formfill.ocr (v04)
- OcrPage C — pinneapple_data.formfill.ocr (v04)
- ode_stepper f — pinneapple_systems.time_series.synthetic (v06)
- ODEForecaster C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- ODERNN C — pinneapple_neural.architectures.continuous.ode_rnn (v04)

- `Offer` C — `pinneapple_query.core` (v02)
- `offline` f — `pinneapple_design.geometry_library.store` (v06)
- `offline_benchmark` f — `pinneapple_simulation.accelerated_foam.benchmark` (v03)
- `offline_report` f — `pinneapple_flow.nodes_ext.problemdesign` (v07)
- `offset_polygon` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `OffsetReflector` C — `pinneapple_simulation.electromagnetics.reflector.geometry` (v03)
- `oil_pipeline` f — `pinneapple_twin3d.gallery` (v06)
- `oisst_monthly` f — `pinneapple_systems.time_series.enso.data` (v06)
- `ok` f — `pinneapple_arena.external.result` (v04)
- `OllamaBackend` C — `pinneapple_llm.workbench.backend` (v07)
- `OllamaClient` C — `pinneapple_data.formfill.llm` (v04)
- `OllamaError` C — `pinneapple_data.formfill.llm` (v04)
- `om_to_upd` f — `pinneapple_simulation.external_solvers.modelica.om_runner` (v03)
- `omc_available` f — `pinneapple_simulation.external_solvers.modelica.omc` (v03)
- `OMSimConfig` C — `pinneapple_simulation.external_solvers.modelica.om_runner` (v03)
- `on_plane` f — `pinneapple_tools.automation.scripts.cfdof_setup` (v07)
- `on_run_finished` f — `pinneapple_flow.runtime` (v07)
- `one_at_a_time_sensitivity` f — `pinneapple_tools.model_evaluation.metrics` (v07)
- `one_two_exchanger_ode` f — `pinneapple_physics.closed_form.heat_exchanger` (v02)
- `onnx_parity` f — `pinneapple_hub.zoo` (v07)
- `onnx_parity` f — `pinneapple_hub.zoo.export` (v07)
- `ONSClient` C — `pinneapple_data.datasets.external_sources` (v04)
- `ood_verdict` f — `pinneapple_registry.gateway` (v01)
- `OODGate` C — `pinneapple_analysis.surrogate.limits` (v05)
- `OPCUAStream` C — `pinneapple_systems.digital_twin.io.stream` (v06)
- `OpcUaTwinServer` C — `pinneapple_systems.digital_twin.standards` (v06)
- `open_case` f — `pinneapple_data.adapters.splash_archive_adapter` (v04)
- `open_study` f — `pinneapple_simulation.electromagnetics.antenna.study` (v03)
- `OpenAIBackend` C — `pinneapple_llm.workbench.backend` (v07)
- `openfoam` f — `pinneapple_flow.nodes.solvers` (v07)
- `openfoam_available` f — `pinneapple_analysis.benchmark.builtin.airfoil` (v05)
- `openfoam_available` f — `pinneapple_flow.wrappers` (v07)
- `openfoam_available` f — `pinneapple_simulation.accelerated_foam.case` (v03)
- `openfoam_available` f — `pinneapple_simulation.numerical_solvers` (v03)
- `openfoam_available` f — `pinneapple_simulation.numerical_solvers.external_flow` (v03)

- `openfoam_case_to_upd` f — `pinneapple_simulation.external_solvers.openfoam.field_reader` (v03)
- `openfoam_comparison` f — `pinneapple_simulation.accelerated_foam.benchmark` (v03)
- `openfoam_to_dataset` f — `pinneapple_simulation.numerical_solvers.openfoam_bridge` (v03)
- `OpenFOAMAdapter` C — `pinneapple_core.workflows.adapters` (v01)
- `OpenFOAMAdapter` C — `pinneapple_tools.automation.adapters.openfoam` (v07)
- `OpenFOAMBridge` C — `pinneapple_simulation.numerical_solvers.openfoam_bridge` (v03)
- `OpenFOAMCaseTemplate` C — `pinneapple_simulation.external_solvers.openfoam.case_builder` (v03)
- `OpenFOAMClient` C — `pinneapple_data.datasets.external_sources` (v04)
- `OpenFOAMRunConfig` C — `pinneapple_simulation.external_solvers.openfoam.runner` (v03)
- `OpenMeteoClient` C — `pinneapple_data.datasets.external_sources` (v04)
- `OpenRadiossDockerConfig` C — `pinneapple_simulation.external_solvers.openradioss.runner` (v03)
- `operating_history` f — `pinneapple_systems.digital_twin.pipe_loop.plant` (v06)
- `operating_point` f — `pinneapple_systems.process_components.water_hammer` (v06)
- `operating_space` f — `pinneapple_systems.operations.series` (v06)
- `OperatingInput` C — `pinneapple_design.thermal.heatsink.engine` (v06)
- `OperatingPoint` C — `pinneapple_physics.closed_form.plate_fin_heatsink` (v02)
- `OperatingPoint` C — `pinneapple_systems.operations.exchanger` (v06)
- `OperatingPoint` C — `pinneapple_systems.process_components.water_hammer` (v06)
- `OperatorAdapter` C — `pinneapple_neural.architectures.adapters.operators` (v04)
- `OperatorBlockND` C — `pinneapple_neural.architectures.neural_operators.uno` (v04)
- `OperatorInference` C — `pinneapple_neural.architectures.rom.opinf` (v04)
- `OperatorOutput` C — `pinneapple_neural.architectures.neural_operators.base` (v04)
- `opinf_advection` f — `pinneapple_flow.nodes_ext.validation_gallery` (v07)
- `opinion_dynamics_2d` f — `pinneapple_physics.pde_environment.presets.multidisciplinary` (v02)
- `optics` f — `pinneapple_physics.astro.observing` (v02)
- `OptimisationResult` C — `pinneapple_design.intent.pipeline` (v06)
- `optimise` f — `pinneapple_design.intent.pipeline` (v06)
- `OptimizationOutcome` C — `pinneapple_simulation.electromagnetics.antenna.design` (v03)
- `OptimizationResult` C — `pinneapple_design.design_optimizer.methods` (v06)

- `OptimizationResult` C — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `OptimizationResult` C — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `OptimizationResult` C — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `OptimizationResult` C — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `OptimizationResult` C — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `OptimizationResult` C — `pinneapple_twin3d.wear.optimize` (v06)
- `optimize` f — `pinneapple_design` (v06)
- `optimize` f — `pinneapple_design.thermal.board_cooling.optimize` (v06)
- `optimize` f — `pinneapple_design.topology.driver` (v06)
- `optimize` f — `pinneapple_simulation.electromagnetics.antenna.design` (v03)
- `optimize` f — `pinneapple_twin3d.wear.optimize` (v06)
- `optimize_design` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `optimize_design` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `optimize_design` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `optimize_design` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `optimize_design` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `optimize_diameters` f — `pinneapple_simulation.numerical_solvers.pipe_network` (v03)
- `OptimizerConfig` C — `pinneapple_twin3d.wear.optimize` (v06)
- `OptimizeResult` C — `pinneapple_core.optim` (v01)
- `Option` C — `pinneapple_reason.chain` (v05)
- `option_line` f — `pinneapple_reason.narrative` (v05)
- `OptionInfo` C — `pinneapple_decision.schema` (v05)
- `OptState` C — `pinneapple_design.design_optimizer.optimizer` (v06)
- `orbit_mean_density` f — `pinneapple_physics.astro.decay` (v02)
- `orbit_polyline` f — `pinneapple_physics.astro.orbits` (v02)
- `OrchestratorResult` C — `pinneapple_worldmodel.orchestrator` (v04)
- `OrgAnswer` C — `pinneapple_llm.org_knowledge` (v07)
- `orient_on_surface` f — `pinneapple_tools.visualization.molecular` (v07)
- `orient_outward` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `orient_outward` f — `pinneapple_design.geometry_library.meshops` (v06)
- `oriented_blocks` f — `pinneapple_data.cae.geometry` (v04)
- `ornstein_uhlenbeck` f — `pinneapple_systems.time_series.synthetic` (v06)
- `OrthogonalityResult` C — `pinneapple_analysis.verification.mesh_intelligence` (v05)
- `orthographic` f — `pinneapple_physics.weather.viz` (v02)
- `oscillation_stats` f — `pinneapple_systems.time_series.enso.oscillators` (v06)
- `Oscillator` C — `pinneapple_lab.experiments.ode` (v07)

- `oscillator_transition` f — `pinneapple_systems.time_series.synthetic` (v06)
- `OscillatorDiscovery` C — `pinneapple_lab.experiments.discovery` (v07)
- `Oscilloscope` C — `pinneapple_science.instruments` (v02)
- `out_of_bounds` f — `pinneapple_design.aero.geometry` (v06)
- `out_of_scope` f — `pinneapple_design.intent.parse` (v06)
- `Outcome` C — `pinneapple_check.result` (v05)
- `OutlierDetector` C — `pinneapple_systems.time_series.preparation.outliers` (v06)
- `outline` f — `pinneapple_design.aero.geometry` (v06)
- `OutOfEnvelope` C — `pinneapple_simulation.external_solvers.calculix.virtual_ccx` (v03)
- `OutOfEnvelopeError` C — `pinneapple_systems.process_components.real_gas_eos` (v06)
- `OutOfRangeWarning` C — `pinneapple_hub.zoo._models` (v07)
- `OutOfScope` C — `pinneapple_design.intent.parse` (v06)
- `Output` C — `pinneapple_presets.core` (v02)
- `output_filename` f — `pinneapple_data.formfill.pipeline` (v04)
- `output_names` f — `pinneapple_hub.zoo.export` (v07)
- `output_scale` f — `pinneapple_hub.zoo.export` (v07)
- `OutputConfig` C — `pinneapple_arena.config` (v04)
- `oval_boundary` f — `pinneapple_physics.astro.spaceweather` (v02)
- `overfitting` f — `pinneapple_flow.nodes_ext.forecasting` (v07)
- `overfitting_study` f — `pinneapple_systems.time_series.forecast_lab.study` (v06)
- `overfitting_variants` f — `pinneapple_systems.time_series.forecast_lab.presets` (v06)
- `overlaps` f — `pinneapple_physics.pcb.solver` (v02)

P

- `p6_rhs` f — `pinneapple_physics.paper_problems_xtfc` (v02)
- `p7_rhs` f — `pinneapple_physics.paper_problems_xtfc` (v02)
- `pack_splash_archive` f — `pinneapple_simulation.external_solvers.openfoam.splash_packer` (v03)
- `Package` C — `pinneapple_condense.package` (v02)
- `Package` C — `pinneapple_simulation.electromagnetics.antenna.package` (v03)
- `package_engine` f — `pinneapple_condense.query` (v02)
- `package_of` f — `pinneapple_flow.palette` (v07)
- `package_queries` f — `pinneapple_flow.nodes.query` (v07)
- `package_study` f — `pinneapple_simulation.electromagnetics.antenna.study` (v03)
- `package_thermal` f — `pinneapple_flow.nodes.condense` (v07)
- `package_versions` f — `pinneapple_tools.benchmark_suite.crossframework.records` (v07)
- `PackagedSample` C — `pinneapple_worldmodel.dataset_packager` (v04)
- `PackageModel` C — `pinneapple_condense.query` (v02)
- `PackagerConfig` C — `pinneapple_worldmodel.dataset_packager` (v04)
- `packages` f — `pinneapple_flow.coverage` (v07)
- `packed_bed` f — `pinneapple_simulation.numerical_solvers.particles` (v03)

- pae f — pinneapple_security.signing (v05)
- PairAnalysis C — pinneapple_perception.change_detection (v04)
- paired_bootstrap f — pinneapple_systems.sports.football.metrics (v06)
- paired_ratio f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- palette_entry f — pinneapple_flow.registry (v07)
- palmer_strip_capacitance f — pinneapple_science.em.statics (v02)
- pan_tilt_head f — pinneapple_design.geometry.vehicle_cad (v06)
- Panel C — pinneapple_tools.visualization.trust_panels (v07)
- panel f — pinneapple_lab.showcase (v07)
- panel_solve f — pinneapple_design.aero.panel (v06)
- PanelResult C — pinneapple_design.aero.panel (v06)
- Paper C — pinneapple_tools.reporting.paperkit (v07)
- para f — pinneapple_tools.reporting.paperkit (v07)
- paraboloid_dish f — pinneapple_design.geometry.vehicle_cad (v06)
- parallactic_angle f — pinneapple_physics.astro.observing (v02)
- parallel_plate_capacitance f — pinneapple_simulation.electromagnetics.statics (v03)
- parallel_plates_pressure_drop f — pinneapple_physics.closed_form.pipe_flow (v02)
- ParallelPlateCapacitor C — pinneapple_science.em.statics (v02)
- Param C — pinneapple_design.fitmesh.catalog (v06)
- Param C — pinneapple_design.geometry_library.core (v06)
- Param C — pinneapple_flow.registry (v07)
- Param C — pinneapple_presets.core (v02)
- Param C — pinneapple_tools.visualization.dashboard.spec (v07)
- param f — pinneapple_reason.library (v05)
- param_text f — pinneapple_reason.chain (v05)
- Parameter C — pinneapple_analysis.inverse_problems.forward (v05)
- Parameter C — pinneapple_data.contracts.models (v04)
- Parameter C — pinneapple_physics.physical_problem (v02)
- parameter_shift_gradient f — pinneapple_quantum.training.trainer (v04)
- parameter_shift_study f — pinneapple_veriphysics.robustness (v05)
- parameters_from_space f — pinneapple_analysis.inverse_problems.contracts (v05)
- ParameterSet C — pinneapple_analysis.inverse_problems.forward (v05)
- ParameterSpace C — pinneapple_data.contracts.models (v04)
- ParameterSpec C — pinneapple_systems.process_components.explicit_equation_system (v06)
- parametric f — pinneapple_flow.nodes.data (v07)
- parametric_pod f — pinneapple_flow.wrappers (v07)
- ParametricCadQuerySynthGenerator C — pinneapple_data.synth.geometry_cadquery (v04)
- ParametricCadSpec C — pinneapple_design.geometry.gen.cadquery_gen (v06)
- ParametricCase C — pinneapple_neural.workflows.parametric_dense_volume_operator (v04)

- ParametricDenseVolumeOperatorConfig C — pinneapple_neural.workflows.parametric_dense_volume_operator (v04)
- ParametricFamilyTransfer C — pinneapple_adaptation.transfer_learning.parametric (v04)
- ParametricPOD C — pinneapple_neural.architectures.rom.parametric (v04)
- ParamReport C — pinneapple_twin3d.wear.calibrate (v06)
- params_hash f — pinneapple_design.geometry_library.core (v06)
- ParamSpace C — pinneapple_design.design_optimizer.optimizer (v06)
- ParaViewAdapter C — pinneapple_tools.automation.adapters.paraview (v07)
- pareto f — pinneapple_condense.exceptions (v02)
- pareto f — pinneapple_condense.figures (v02)
- pareto_dominates f — pinneapple_design.design_optimizer.pareto (v06)
- pareto_figure f — pinneapple_design.intent.figures (v06)
- pareto_front f — pinneapple_autolab.report (v07)
- ParetoFront C — pinneapple_design.design_optimizer.pareto (v06)
- parity_error f — pinneapple_hub.zoo.export (v07)
- parity_inputs f — pinneapple_hub.zoo.export (v07)
- parity_plot f — pinneapple_simulation.electromagnetics.antenna.plots (v03)
- parquet_files f — pinneapple_hub.zoo.parquet (v07)
- parse f — pinneapple_data.simulation_metadata.foam_dict (v04)
- parse f — pinneapple_design.fitmesh.context (v06)
- parse_agent f — pinneapple_llm.workbench.agents (v07)
- parse_ccx_log f — pinneapple_data.simulation_metadata.calculix (v04)
- parse_curation_response f — pinneapple_tools.dataset_quality.vlm_curation (v07)
- parse_cvg f — pinneapple_data.simulation_metadata.calculix (v04)
- parse_dat f — pinneapple_data.simulation_metadata.calculix (v04)
- parse_dat f — pinneapple_data.simulation_metadata.openfoam (v04)
- parse_duckduckgo f — pinneapple_llm.workbench.web (v07)
- parse_equation f — pinneapple_app.backend.core.physics_adapter (v07)
- parse_equation f — pinneapple_physics.symbolic_pde.text (v02)
- parse_expression f — pinneapple_physics.symbolic_pde.text (v02)
- parse_forces f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- parse_front_matter f — pinneapple_hub.zoo.hub_cards (v07)
- parse_generation_response f — pinneapple_llm.qa.generation (v07)
- parse_inp f — pinneapple_data.simulation_metadata.calculix (v04)
- parse_iv f — pinneapple_systems.digital_twin.io.usgs (v06)
- parse_json_tool_call f — pinneapple_llm.workbench.backend (v07)
- parse_location f — pinneapple_app.backend.core.physics_adapter (v07)
- parse_log f — pinneapple_data.simulation_metadata.openfoam (v04)

- parse_log f — pinneapple_simulation.accelerated_foam.case (v03)
- parse_mission f — pinneapple_design.thermal.board_cooling.mission (v06)
- parse_mission f — pinneapple_simulation.rover.autolab.mission (v03)
- parse_numeric_table f — pinneapple_arena.pointwise.tables (v04)
- parse_objective f — pinneapple_design.qualitative.preview (v06)
- parse_parameters f — pinneapple_check.physics (v05)
- parse_pde f — pinneapple_analysis.surrogate.pde (v05)
- parse_quantity f — pinneapple_tools.automation.interpret (v07)
- parse_rate f — pinneapple_security.web (v05)
- parse_ref f — pinneapple_query.refs (v02)
- parse_residual_table f — pinneapple_data.simulation_metadata (v04)
- parse_silso_csv f — pinneapple_systems.time_series.solar_cycle (v06)
- parse_solver_log f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- parse_sta f — pinneapple_data.simulation_metadata.calculix (v04)
- parse_system f — pinneapple_physics.symbolic_pde.text (v02)
- parse_tiers f — pinneapple_lab.seal (v07)
- parse_tle f — pinneapple_physics.astro.satellites (v02)
- parse_unit f — pinneapple_data.physical_units (v04)
- parse_unit f — pinneapple_llm.qa.units (v07)
- parse_value f — pinneapple_data.formfill.extract (v04)
- Part C — pinneapple_design.aero.airframe (v06)
- Part C — pinneapple_twin3d.scene (v06)
- part f — pinneapple_design.bricks.parts (v06)
- part_sensitivity f — pinneapple_design.qualitative.preview (v06)
- ParticleSuspension C — pinneapple_lab.experiments.particles (v07)
- ParticleSystem C — pinneapple_simulation.particle_dynamics.particles (v03)
- Partition C — pinneapple_condense.algebraic (v02)
- partition f — pinneapple_condense.algebraic (v02)
- PartRecipe C — pinneapple_tools.nvidia.brep (v07)
- PartSensitivity C — pinneapple_design.qualitative.preview (v06)
- PartsLibrary C — pinneapple_design.bricks.ldraw (v06)
- PartType C — pinneapple_design.bricks.parts (v06)
- passes_from_metrica f — pinneapple_systems.sports.football.pitch_control (v06)
- passivity f — pinneapple_physics.maxwell_checks (v02)
- PassSet C — pinneapple_systems.sports.football.pitch_control (v06)
- Patch C — pinneapple_simulation.rover.autolab.site (v03)
- patch_area f — pinneapple_tools.automation.verify (v07)
- patch_cfdof_allmesh f — pinneapple_tools.automation.adapters.openfoam (v07)
- patch_cfdof_inlet f — pinneapple_tools.automation.adapters.openfoam (v07)
- patch_fluxes f — pinneapple_tools.automation.verify (v07)
- patch_mean f — pinneapple_tools.automation.verify (v07)
- patch_test f — pinneapple_simulation.numerical_solvers.plane_stress_fem (v03)

- PatchDesign C — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- PatchSpec C — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- path_axes f — pinneapple_simulation.flight.turbulence (v03)
- path_to_next f — pinneapple_lab.curation (v07)
- payload f — pinneapple_tools.visualization.dashboard.html (v07)
- payload_track_check f — pinneapple_worldmodel.robot_perception (v04)
- PCAResidualDetector C — pinneapple_security.cps_bench (v05)
- PCASurrogate C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- pcb_thermal f — pinneapple_physics.pde_environment.presets.engineering (v02)
- PCEResult C — pinneapple_analysis.uncertainty.pce (v05)
- pcg f — pinneapple_simulation.accelerated_foam.solvers (v03)
- PCGrad C — pinneapple_neural.trainer.loss_balancer (v04)
- PCParams C — pinneapple_systems.sports.football.pitch_control (v06)
- pd_tracking f — pinneapple_simulation.robot_sim.sim (v03)
- pde f — pinneapple_core.loss (v01)
- pde_from_sympy f — pinneapple_physics.symbolic_pde.compiler (v02)
- pde_residual_loss f — pinneapple_quantum.loss.schrodinger (v04)
- PDEDeclaration C — pinneapple_analysis.surrogate.pde (v05)
- PDEFamily C — pinneapple_physics.pde_environment.capabilities (v02)
- PDEParser C — pinneapple_physics.codegen.fdm_script_generator (v02)
- PDEResidualGuidance C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- PDESynthGenerator C — pinneapple_data.synth.pde (v04)
- PDETaskSampler C — pinneapple_adaptation.meta_learning.task_sampler (v04)
- PDETermSpec C — pinneapple_physics.pde_environment.spec (v02)
- PDETwinModel C — pinneapple_flow.twin (v07)
- pdf_widgets f — pinneapple_data.formfill.fill (v04)
- peak_rss_gb f — pinneapple_condense.benchmark (v02)
- pec_boundary f — pinneapple_physics.maxwell_checks (v02)
- peclet f — pinneapple_physics.physics_audit.regime (v02)
- pedestrian_response f — pinneapple_reason.modes.structural (v05)
- pendulum_period f — pinneapple_simulation.external_solvers.newton_physics.validation (v03)
- pendulum_rk4_reference f — pinneapple_tools.benchmark_suite.deeponet_ground_truth (v07)
- PendulumVideo C — pinneapple_lab.experiments.discovery (v07)
- PennyLaneBackend C — pinneapple_quantum.backends.backend (v04)
- per_field f — pinneapple_physics.metrics (v02)
- per_sample_error f — pinneapple_analysis.surrogate.limits (v05)
- per_sample_rel_l2 f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- per_tag_losses f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- perception_checks f — pinneapple_worldmodel.robot_perception (v04)

- performance f — pinneapple_design.aero.aircraft (v06)
- performance f — pinneapple_systems.operations.control (v06)
- Periodic C — pinneapple_core.transforms (v01)
- periodic_cnn2d f — pinneapple_physics.multiscale.learned (v02)
- periodic_elm_fit f — pinneapple_systems.time_series.solar_cycle (v06)
- periodic_grf f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- PeriodicBC C — pinneapple_physics.symbolic_pde.bc (v02)
- PeriodicFit C — pinneapple_systems.time_series.solar_cycle (v06)
- Periodogram C — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- permutation_entropy f — pinneapple_systems.time_series.predictability (v06)
- permutation_marginalized f — pinneapple_decision.debias (v05)
- PerQuantitySurrogate C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- Persistence C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- persistence f — pinneapple_physics.weather.evaluate (v02)
- petermann_factor f — pinneapple_physics.non_hermitian.pt_dimer (v02)
- Phase C — pinneapple_simulation.numerical_solvers.free_surface (v03)
- phase_angle f — pinneapple_physics.astro.phenomena (v02)
- phase_portrait f — pinneapple_systems.time_series.enso.pipeline (v06)
- phase_randomized_surrogate f — pinneapple_systems.time_series.predictability (v06)
- phase_scene f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- PhaseClimatology C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- PhenomenonEntry C — pinneapple_problemdesign.knowledge.physics_knowledge (v02)
- PhotorealisticEnhancer C — pinneapple_worldmodel.photorealistic_enhancer (v04)
- physical_features f — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- PhysicalDatasetBuilder C — pinneapple_pdb.builder (v04)
- PhysicalProblem C — pinneapple_physics.physical_problem (v02)
- PhysicalRef C — pinneapple_query.refs (v02)
- PhysicalSample C — pinneapple_data.physical_sample (v04)
- PhysicalSampleDataset C — pinneapple_data.noether_dataset (v04)
- PhysicalSampleLike C — pinneapple_data.synth.sample_adapter (v04)
- PhysicalSchema C — pinneapple_pdb.builder (v04)
- PhysicalSpecification C — pinneapple_llm.qa.spec (v07)
- Physics C — pinneapple_simulation.pipelines.freecad_openfoam.case (v03)
- Physics C — pinneapple_systems.component_library.physics (v06)
- physics_ai_tree f — pinneapple_decision.tree (v05)

- physics_audit_check f — pinneapple_analysis.benchmark.validation (v05)
- physics_aware_rank f — pinneapple_arena.arena (v04)
- physics_check f — pinneapple_flow.nodes_ext.validation (v07)
- physics_checks f — pinneapple_worldmodel.physical_world (v04)
- physics_checks_section f — pinneapple_app.backend.core.surrogate_report (v07)
- physics_detector f — pinneapple_security.cps_bench (v05)
- physics_features f — pinneapple_systems.sports.football.pc_study (v06)
- physics_features f — pinneapple_systems.sports.football.xg (v06)
- physics_learning f — pinneapple_flow.nodes.physics_learning (v07)
- physics_metrics f — pinneapple_physics.darcy_checks (v02)
- physics_mismatch f — pinneapple_simulation.terramechanics.wong_reece (v03)
- physics_resistance f — pinneapple_design.thermal.heatsink.engine (v06)
- physics_spec f — pinneapple_flow.nodes_ext.arena_pipelines (v07)
- PhysicsAdapter C — pinneapple_app.backend.core.physics_adapter (v07)
- PhysicsAIConfig C — pinneapple_worldmodel.pipeline (v04)
- PhysicsAIDataset C — pinneapple_worldmodel.training_hooks (v04)
- PhysicsAIPipeline C — pinneapple_worldmodel.pipeline (v04)
- PhysicsAIPipelineResult C — pinneapple_worldmodel.pipeline (v04)
- PhysicsArea C — pinneapple_arena.pointwise.schema (v04)
- PhysicsAttention C — pinneapple_neural.architectures.neural_operators.transolver (v04)
- PhysicsAwareBase C — pinneapple_neural.architectures.physics_aware.base (v04)
- PhysicsAwareCatalog C — pinneapple_neural.architectures.physics_aware.registry (v04)
- PhysicsAwareNeuralNetwork C — pinneapple_neural.architectures.physics_aware.pann (v04)
- PhysicsAwareOutput C — pinneapple_neural.architectures.physics_aware.base (v04)
- PhysicsAwareRouter C — pinneapple_physics.physics_audit.routing (v02)
- PhysicsBackend C — pinneapple_core.backend (v01)
- PhysicsBenchmark C — pinneapple_worldmodel.benchmark (v04)
- PhysicsCase C — pinneapple_data.physics_case (v04)
- PhysicsChoice C — pinneapple_decision.schema (v05)
- PhysicsConfidenceScore C — pinneapple_analysis.verification.physics_confidence_score (v05)
- PhysicsCurriculum C — pinneapple_worldmodel.curriculum (v04)
- PhysicsDataset C — pinneapple_core.data (v01)
- PhysicsDatasetFactory C — pinneapple_worldmodel.dataset_factory (v04)
- PhysicsDecider C — pinneapple_decision.decider (v05)
- PhysicsDecisionEngine C — pinneapple_decision.loop (v05)

- PhysicsDecoderDataset C — pinneapple_worldmodel.training_hooks (v04)
- PhysicsDescriptorEncoder C — pinneapple_worldmodel.mega_model (v04)
- PhysicsDomain C — pinneapple_worldmodel.geometry (v04)
- PhysicsDomain2D C — pinneapple_design.geometry.gen.domains (v06)
- PhysicsDomain3D C — pinneapple_design.geometry.gen.domains3d (v06)
- PhysicsEnsemble C — pinneapple_physics.ensemble (v02)
- PhysicsFeatures C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- PhysicsFoundationModel C — pinneapple_worldmodel.mega_model (v04)
- PhysicsGuardrail C — pinneapple_llm.guardrail (v07)
- PhysicsInformedDiffusion C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- PhysicsInformedKoopmanAutoencoder C — pinneapple_neural.architectures.autoencoders.koopman_pi_ae (v04)
- PhysicsInformedNeuralOperator C — pinneapple_neural.architectures.neural_operators.pino (v04)
- PhysicsKnowledgeGraph C — pinneapple_physics.physics_audit.knowledge (v02)
- PhysicsLoss C — pinneapple_systems.cosimulation.losses (v06)
- PhysicsLossGenerator C — pinneapple_arena.pointwise.losses (v04)
- PhysicsLossHook C — pinneapple_neural.trainer.losses (v04)
- PhysicsLossMLP C — pinneapple_systems.time_series.forecast_lab.learned (v06)
- PhysicsMaterial C — pinneapple_twin3d.scene (v06)
- PhysicsMismatchWarning C — pinneapple_simulation.terramechanics.surrogate (v03)
- PhysicsModule C — pinneapple_core.module (v01)
- PhysicsMonitor C — pinneapple_neural.callbacks (v04)
- physicsnemo_fno f — pinneapple_neural.physicsnemo_bridge.field_operators (v04)
- physicsnemo_meshgraphnet f — pinneapple_neural.physicsnemo_bridge.field_operators (v04)
- PhysicsNeMoHook C — pinneapple_neural.callbacks (v04)
- PhysicsNemoModelAdapter C — pinneapple_neural.physicsnemo_bridge.adapters (v04)
- PhysicsNeMoSymBackend C — pinneapple_tools.benchmark_suite.backends.physicsnemo_sym (v07)
- PhysicsOptimizer C — pinneapple_core.optim (v01)
- PhysicsOrchestrator C — pinneapple_worldmodel.orchestrator (v04)
- PhysicsPipeline C — pinneapple_physics (v02)
- PhysicsPipeline C — pinneapple_tools.benchmark_suite.physics_pipeline (v07)
- PhysicsPlan C — pinneapple_design.intent.pipeline (v06)
- PhysicsPointCloud3D C — pinneapple_design.geometry.mesh.mesher_3d (v06)
- PhysicsPostModel C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- PhysicsProblem C — pinneapple_arena.pointwise.schema (v04)

- PhysicsProps C — pinneapple_twin3d.scene (v06)
- PhysicsRenderer C — pinneapple_worldmodel.physics_renderer (v04)
- PhysicsResidualDetector C — pinneapple_security.cps (v05)
- PhysicsScenario C — pinneapple_worldmodel.scenario (v04)
- PhysicsSimulator C — pinneapple_worldmodel.simulator (v04)
- PhysicsSolverBridge C — pinneapple_worldmodel.physics_solver_bridge (v04)
- PhysicsSpec C — pinneapple_problemdesign.schema (v02)
- PhysicsSurrogate C — pinneapple_design.design_optimizer.surrogate (v06)
- PhysicsTool C — pinneapple_worldmodel.physics_tools (v04)
- PhysicsToolRegistry C — pinneapple_worldmodel.physics_tools (v04)
- PhysicsTransferAdapter C — pinneapple_adaptation.transfer_learning.adapter (v04)
- PhysicsValidator C — pinneapple_analysis.validation.validator (v05)
- PhysicsValidityMonitor C — pinneapple_physics.physics_audit.monitor (v02)
- PhysicsWarning C — pinneapple_neural.callbacks (v04)
- PhysicsWorldModel C — pinneapple_worldmodel.model (v04)
- picard f — pinneapple_condense.flow (v02)
- pick f — pinneapple_design.aero.verify (v06)
- pick_corner f — pinneapple_lab.seal (v07)
- PIDBaseline C — pinneapple_systems.operations.control (v06)
- PIDController C — pinneapple_systems.component_modeling.control (v06)
- PIDiffConfig C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- PIELM C — pinneapple_neural.architectures.pinns.pielm (v04)
- PIELMFactoryAdapter C — pinneapple_neural.architectures.pinns.pielm (v04)
- pig_iron_ladle f — pinneapple_twin3d.wear.presets (v06)
- pillow_block f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- pin_array f — pinneapple_condense.flow (v02)
- pin_fin f — pinneapple_flow.nodes_ext.use_cases (v07)
- pin_fin_heat f — pinneapple_physics.closed_form.heat (v02)
- pin_fin_heatsink f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- pin_fin_theta f — pinneapple_physics.closed_form.heat (v02)
- pin_sample f — pinneapple_data.device (v04)
- pin_sdf f — pinneapple_condense.meshless2d (v02)
- pinball_loss f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- pinball_loss_torch f — pinneapple_analysis.uncertainty.quantile (v05)
- PinFinModel C — pinneapple_analysis.inverse_problems.models (v05)
- PINN C — pinneapple_physics.pinn_solver.factory.pinn_factory (v02)
- pinn f — pinneapple_flow.nodes.solvers (v07)
- pinn_checks f — pinneapple_analysis.benchmark.builtin.fields (v05)
- pinn_factory_adapter f — pinneapple_neural.architectures.pinns.vpinn (v04)

- pinn_heat f — pinneapple_flow.nodes_ext.validation_gallery (v07)
- pinn_job f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- pinn_model f — pinneapple_analysis.validation.physics_learning_cases (v05)
- pinn_operation_costs f — pinneapple_analysis.cost.torch_ops (v05)
- pinn_problem f — pinneapple_physics.symbolic_pde.pinn (v02)
- PINNAdapter C — pinneapple_neural.architectures.adapters.pinn (v04)
- PINNArenaBenchmark C — pinneapple_tools.benchmark_suite.benchmark (v07)
- PINNBase C — pinneapple_neural.architectures.pinns.base (v04)
- PINNBatch C — pinneapple_data.adapters.pinn_batch_builders (v04)
- PINNCatalog C — pinneapple_neural.architectures.pinns.registry (v04)
- PINNeAPPleModelNode C — pinneapple_systems.cosimulation.adapters (v06)
- PinneappleModelsBackend C — pinneapple_tools.benchmark_suite.backends.pinneapple_models_backend (v07)
- PINNeAPPleProfiler C — pinneapple_neural.trainer.hpc (v04)
- PinneappleSpec C — pinneapple_problemdesign.schema (v02)
- PINNeProblemNode C — pinneapple_systems.cosimulation.adapters (v06)
- PINNExtrapolationForecaster C — pinneapple_systems.time_series.enso.pinn (v06)
- PINNFactory C — pinneapple_physics.pinn_solver.factory.pinn_factory (v02)
- PINNLSTM C — pinneapple_neural.architectures.pinns.pinn_lstm (v04)
- PINNMapping C — pinneapple_physics.pinn_solver.io.mappings (v02)
- PINNMethod C — pinneapple_analysis.benchmark.builtin.fields (v05)
- PINNMonitor C — pinneapple_analysis.validation.pinn (v05)
- PINNNode C — pinneapple_systems.cosimulation.node (v06)
- PINNOutput C — pinneapple_neural.architectures.pinns.base (v04)
- PINNProblem C — pinneapple_analysis.validation.pinn (v05)
- PINNProblem C — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- PINNProblemSpec C — pinneapple_physics.pinn_solver.factory.pinn_factory (v02)
- PINNRansMethod C — pinneapple_analysis.benchmark.builtin.airfoil (v05)
- PINNRechargeForecaster C — pinneapple_systems.time_series.enso.pinn (v06)
- PINNRefinement C — pinneapple_design.design_optimizer.refinement (v06)
- PINNReport C — pinneapple_analysis.validation.pinn (v05)
- PINNsFormer C — pinneapple_neural.architectures.pinns.pinnsformer (v04)
- pinnsformer_case f — pinneapple_physics.paper_problems (v02)

- PINNWithParameters C — pinneapple_physics.symbolic_pde.pinn (v02)
- pino_physics_fn f — pinneapple_neural.architectures.neural_operators.pino (v04)
- pip_install f — pinneapple_arena.external.installer (v04)
- pip_install_command f — pinneapple_arena.external.local_command (v04)
- Pipe C — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- pipe_bend f — pinneapple_simulation.pipelines.freecad_openfoam.studies (v03)
- pipe_bend_flanged f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- pipe_bend_scene f — pinneapple_twin3d.demo (v06)
- pipe_flow_3d f — pinneapple_physics.pde_environment.presets.cfd (v02)
- pipe_loss f — pinneapple_physics.closed_form.pump_npsh (v02)
- pipe_surface f — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- pipe_tee f — pinneapple_design.geometry.gen.cadquery_advanced (v06)
- pipe_twin f — pinneapple_flow.nodes.autolab (v07)
- PipeFlow C — pinneapple_lab.experiments.pipes (v07)
- PipeFlowDomain3D C — pinneapple_design.geometry.gen.domains3d (v06)
- Pipeline C — pinneapple_systems.process_components.water_hammer (v06)
- pipeline f — pinneapple_physics (v02)
- pipeline_call f — pinneapple_tools.automation.adapters.pipeline (v07)
- PipelineConfig C — pinneapple_llm.qa.pipeline (v07)
- PipelineSpec C — pinneapple_simulation.pipelines.freecad_openfoam.pipeline (v03)
- PipeSpec C — pinneapple_systems.process_components.pipe_network_1d (v06)
- PipeState C — pinneapple_systems.process_components.pipe_network_1d (v06)
- PipeVanillaPINN C — pinneapple_systems.component_library.components (v06)
- pit_ensemble f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- pit_gaussian f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- pit_histogram f — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- pitch_control_figure f — pinneapple_systems.sports.football.plots (v06)
- pitch_surface f — pinneapple_systems.sports.football.pitch_control (v06)
- pitot_pressure_ratio f — pinneapple_physics.closed_form.compressible (v02)
- piv_grid_count f — pinneapple_perception.piv_tools (v04)
- piv_parameter_guide f — pinneapple_perception.piv_tools (v04)
- piv_velocity_field f — pinneapple_perception.video_piv (v04)
- piv_velocity_sequence f — pinneapple_perception.video_piv (v04)
- pk_two_compartment f — pinneapple_physics.pde_environment.presets.multidisciplinary

(v02)
- place f — pinneapple_flow.palette (v07)
- place f — pinneapple_simulation.rover.autolab.render (v03)
- plain_solver f — pinneapple_simulation.accelerated_foam.case (v03)
- Plan C — pinneapple_analysis.benchmark.runner (v05)
- Plan C — pinneapple_autolab.core (v07)
- Plan C — pinneapple_problemdesign.schema (v02)
- plan f — pinneapple_core.onboarding.newproject (v01)
- plan f — pinneapple_design.aero.dataset (v06)
- plan f — pinneapple_design.intent.pipeline (v06)
- plan f — pinneapple_flow (v07)
- plan f — pinneapple_flow.planner (v07)
- plan f — pinneapple_hub.zoo.publish (v07)
- plan f — pinneapple_physics.astro.journey (v02)
- plan f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- plan f — pinneapple_tools.automation.runner (v07)
- plan_campaign f — pinneapple_data.doe.campaign (v04)
- plan_experiment f — pinneapple_lab.curation (v07)
- plan_from_answers f — pinneapple_flow.planner.wizard (v07)
- plan_heuristic f — pinneapple_flow.planner.heuristic (v07)
- plan_llm f — pinneapple_flow.planner.llm (v07)
- plan_mesh f — pinneapple_design.fitmesh.mesh_plan (v06)
- plan_split f — pinneapple_hub.zoo.parquet (v07)
- planar_face_mesh f — pinneapple_tools.nvidia.brep (v07)
- planar_two_link_torques f — pinneapple_simulation.multibody.arm (v03)
- planck_lambda f — pinneapple_physics.astro.stars (v02)
- plane_figure f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- plane_points f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- plane_strain_2d_default f — pinneapple_physics.pde_environment.presets.structural (v02)
- plane_stress_2d_default f — pinneapple_physics.pde_environment.presets.structural (v02)
- plane_surface f — pinneapple_simulation.electromagnetics.antenna.scene3d (v03)
- plane_wave f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- PlaneStressSolution C — pinneapple_simulation.numerical_solvers.plane_stress_fem (v03)
- planet_elements f — pinneapple_physics.astro.orbits (v02)
- planet_events f — pinneapple_physics.astro.phenomena (v02)
- planetary_transits f — pinneapple_physics.astro.phenomena (v02)
- PlaneWaveTFSF C — pinneapple_simulation.electromagnetics.fdtd (v03)
- PlannedStep C — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- Planner C — pinneapple_autolab.core (v07)
- Planner C — pinneapple_query.core (v02)
- planner_rows f — pinneapple_flow.examples (v07)
- PlanStep C — pinneapple_problemdesign.schema (v02)
- plant_outlets f — pinneapple_systems.operations.exchanger (v06)

- plant_physics f — pinneapple_data.process_optimizer (v04)
- PlantConfig C — pinneapple_systems.operations.exchanger (v06)
- plasma_dispersion_omega f — pinneapple_simulation.electromagnetics.waves (v03)
- plate f — pinneapple_simulation.flight.mass (v03)
- plate_2d f — pinneapple_simulation.numerical_solvers.heat_fv_reference (v03)
- plate_fin_bodies f — pinneapple_design.design_optimizer.presets (v06)
- plate_precomputed f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- PlateFinHeatsink C — pinneapple_presets.cases.heatsink (v02)
- PlateModel C — pinneapple_analysis.inverse_problems.models (v05)
- platform_tag f — pinneapple_simulation.external_solvers.modelica.fmi_compat (v03)
- plausibility f — pinneapple_systems.sports.football.kinematics (v06)
- Player C — pinneapple_analysis.verification.tool_recommendation (v05)
- PlaywrightDriver C — pinneapple_tools.automation.display (v07)
- plot f — pinneapple_flow.nodes.viz (v07)
- plot f — pinneapple_tools (v07)
- plot_acf_pacf f — pinneapple_systems.time_series.eda.plots (v06)
- plot_backtest f — pinneapple_systems.time_series.viz.forecast (v06)
- plot_boundary f — pinneapple_tools.visualization.mesh (v07)
- plot_calibration_curve f — pinneapple_tools.model_evaluation.visualization (v07)
- plot_centerline_velocity f — pinneapple_neural.predictor.visualize (v04)
- plot_chain_threshold f — pinneapple_physics.non_hermitian.viz (v02)
- plot_collocation f — pinneapple_tools.visualization.pinn (v07)
- plot_comparison f — pinneapple_design.bricks.report (v06)
- plot_convergence f — pinneapple_physics.blackhole.figures (v02)
- plot_coupling_plane f — pinneapple_physics.non_hermitian.viz (v02)
- plot_design_opt_convergence f — pinneapple_neural.predictor.visualize (v04)
- plot_ep_laws f — pinneapple_physics.non_hermitian.viz (v02)
- plot_error f — pinneapple_tools.visualization.fields (v07)
- plot_error_histograms f — pinneapple_tools.model_evaluation.visualization (v07)
- plot_error_map_1d f — pinneapple_neural.predictor.visualize (v04)
- plot_error_map_2d f — pinneapple_neural.predictor.visualize (v04)
- plot_fan_charts f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_fem_result f — pinneapple_tools.visualization.solver (v07)
- plot_field_1d f — pinneapple_neural.predictor.visualize (v04)
- plot_field_2d f — pinneapple_neural.predictor.visualize (v04)
- plot_field_comparison f — pinneapple_arena.visualizer (v04)

- `plot_flow_panel` f — `pinneapple_tools.visualization.vortex` (v07)
- `plot_forecast_horizon` f — `pinneapple_systems.time_series.viz.forecast` (v06)
- `plot_fvm_result` f — `pinneapple_tools.visualization.solver` (v07)
- `plot_gradient_magnitude` f — `pinneapple_tools.visualization.pinn` (v07)
- `plot_history` f — `pinneapple_neural.trainer.viz` (v04)
- `plot_hovmoller` f — `pinneapple_systems.time_series.enso.plots` (v06)
- `plot_internal_flow_summary` f — `pinneapple_neural.predictor.visualize` (v04)
- `plot_isosurface_3d` f — `pinneapple_neural.predictor.postprocess` (v04)
- `plot_lbm_flow` f — `pinneapple_tools.visualization.vortex` (v07)
- `plot_learning` f — `pinneapple_simulation.rover.autolab.visuals` (v03)
- `plot_loss_curve` f — `pinneapple_neural.predictor.visualize` (v04)
- `plot_loss_curves` f — `pinneapple_arena.visualizer` (v04)
- `plot_loss_history` f — `pinneapple_tools.visualization.pinn` (v07)
- `plot_memorise_vs_learn` f — `pinneapple_systems.time_series.forecast_lab.plots` (v06)
- `plot_mesh` f — `pinneapple_tools.visualization.mesh` (v07)
- `plot_metrics_table` f — `pinneapple_arena.visualizer` (v04)
- `plot_model_comparison` f — `pinneapple_simulation.rover.autolab.visuals` (v03)
- `plot_model_comparison_1d` f — `pinneapple_neural.predictor.visualize` (v04)
- `plot_model_comparison_2d` f — `pinneapple_neural.predictor.visualize` (v04)
- `plot_multi_loss` f — `pinneapple_tools.visualization.pinn` (v07)
- `plot_overfitting` f — `pinneapple_systems.time_series.forecast_lab.plots` (v06)
- `plot_pareto_front_2d` f — `pinneapple_neural.predictor.visualize` (v04)
- `plot_parity` f — `pinneapple_systems.time_series.viz.forecast` (v06)
- `plot_pde_residual` f — `pinneapple_tools.visualization.pinn` (v07)
- `plot_phase_portrait` f — `pinneapple_systems.time_series.enso.plots` (v06)
- `plot_pinn_prediction` f — `pinneapple_tools.visualization.pinn` (v07)
- `plot_pit` f — `pinneapple_systems.time_series.forecast_lab.plots` (v06)
- `plot_plumes` f — `pinneapple_systems.time_series.enso.plots` (v06)
- `plot_point_cloud` f — `pinneapple_tools.visualization.mesh` (v07)
- `plot_q_criterion_2d` f — `pinneapple_tools.visualization.vortex` (v07)
- `plot_q_criterion_3d` f — `pinneapple_tools.visualization.vortex` (v07)
- `plot_real_vs_predicted` f — `pinneapple_tools.model_evaluation.visualization` (v07)
- `plot_residuals` f — `pinneapple_systems.time_series.viz.forecast` (v06)
- `plot_residuals` f — `pinneapple_tools.visualization.solver` (v07)
- `plot_rolling_forecast` f — `pinneapple_systems.time_series.viz.forecast` (v06)

- plot_route_risk f — pinneapple_simulation.rover.autolab.visuals (v03)
- plot_rover_series f — pinneapple_simulation.rover.viz (v03)
- plot_run f — pinneapple_simulation.vehicle.viz (v03)
- plot_scalar f — pinneapple_tools.visualization.fields (v07)
- plot_scalar_3d f — pinneapple_tools.visualization.fields (v07)
- plot_site_map f — pinneapple_simulation.rover.autolab.visuals (v03)
- plot_skill f — pinneapple_physics.blackhole.figures (v02)
- plot_skill_vs_horizon f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_skill_vs_lead f — pinneapple_systems.time_series.enso.plots (v06)
- plot_solution f — pinneapple_flow.nodes._common (v07)
- plot_solver_output f — pinneapple_tools.visualization.solver (v07)
- plot_spectrum f — pinneapple_physics.non_hermitian.viz (v02)
- plot_spring_barrier f — pinneapple_systems.time_series.enso.plots (v06)
- plot_streamlines f — pinneapple_arena.visualizer (v04)
- plot_streamlines f — pinneapple_tools.visualization.fields (v07)
- plot_streamlines_2d f — pinneapple_neural.predictor.visualize (v04)
- plot_streamlines_2d_from_model f — pinneapple_neural.predictor.postprocess (v04)
- plot_surprise_calendar f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_surrogate_tests f — pinneapple_systems.time_series.forecast_lab.plots (v06)
- plot_testbed f — pinneapple_simulation.vehicle.viz (v03)
- plot_traverse f — pinneapple_simulation.rover.autolab.visuals (v03)
- plot_trend f — pinneapple_systems.time_series.eda.plots (v06)
- plot_uq f — pinneapple_arena.visualizer (v04)
- plot_validation f — pinneapple_simulation.terramechanics.testbed (v03)
- plot_validation f — pinneapple_systems.time_series.enso.plots (v06)
- plot_value_histograms f — pinneapple_tools.model_evaluation.visualization (v07)
- plot_vectors f — pinneapple_tools.visualization.fields (v07)
- plot_velocity_magnitude_slice f — pinneapple_neural.predictor.visualize (v04)
- plot_velocity_slice f — pinneapple_neural.predictor.visualize (v04)
- plot_volume_slice f — pinneapple_neural.predictor.postprocess (v04)
- plot_vortex_identification f — pinneapple_tools.visualization.vortex (v07)
- plot_vorticity f — pinneapple_tools.visualization.vortex (v07)
- plot_vorticity_slice f — pinneapple_neural.predictor.visualize (v04)
- plot_voxel_3d f — pinneapple_tools.visualization.voxel (v07)
- plot_voxel_histogram f — pinneapple_tools.visualization.voxel (v07)
- plot_voxel_slice f — pinneapple_tools.visualization.voxel (v07)

- plot_wheel_series f — pinneapple_simulation.rover.viz (v03)
- po_currents f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- POD C — pinneapple_neural.architectures.rom.pod (v04)
- pod_dmd f — pinneapple_flow.nodes_ext.architectures (v07)
- pod_dmd_stepper f — pinneapple_neural.architectures.rom.steppers (v04)
- pod_dmd_wake f — pinneapple_flow.nodes_ext.validation_gallery (v07)
- pod_opinf_stepper f — pinneapple_neural.architectures.rom.steppers (v04)
- PODWarmStart C — pinneapple_simulation.accelerated_foam.warm_start (v03)
- point f — pinneapple_simulation.flight.mass (v03)
- point_charge_field f — pinneapple_simulation.electromagnetics.statics (v03)
- point_cloud f — pinneapple_condense.meshless (v02)
- point_cloud_domain f — pinneapple_simulation.meshless.domains (v03)
- point_inside f — pinneapple_simulation.pipelines.freecad_openfoam.steps (v03)
- point_mesh_distance f — pinneapple_tools.nvidia.brep (v07)
- point_set_digest f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- PointCloud C — pinneapple_core.field (v01)
- PointCloud C — pinneapple_design.geometry.ops.pointcloud (v06)
- PointD C — pinneapple_check.numerics (v05)
- PointKernelOperator C — pinneapple_neural.architectures.neural_operators.uno (v04)
- PointMetrics C — pinneapple_systems.time_series.metrics_ext.point (v06)
- PointObsOperator C — pinneapple_analysis.inverse_problems.obs_operator (v05)
- PointOperatorBlock C — pinneapple_neural.architectures.neural_operators.uno (v04)
- points_in_terrain_shadow f — pinneapple_tools.visualization.studio.shadows (v07)
- pointwise_error f — pinneapple_analysis.validation.error_map (v05)
- Poisson2D C — pinneapple_arena.problems (v04)
- Poisson2DTask C — pinneapple_tools.benchmark_suite.tasks.poisson_2d (v07)
- poisson_2d f — pinneapple_analysis.validation.pinn (v05)
- poisson_2d_default f — pinneapple_physics.pde_environment.presets.academics (v02)
- poisson_fdm_spectral f — pinneapple_flow.nodes_ext.solver_registry (v07)
- poisson_fem_vs_pinn f — pinneapple_flow.nodes_ext.pinn_examples (v07)
- poisson_periodic_fft f — pinneapple_simulation.numerical_solvers.spectral (v03)
- Polar C — pinneapple_design.aero.aircraft (v06)
- polar_mesh f — pinneapple_simulation.electromagnetics.mesh (v03)
- Policy C — pinneapple_twin3d.wear.optimize (v06)
- policy_for f — pinneapple_tools.rea_bridge.licenses (v07)
- policy_for_path f — pinneapple_tools.rea_bridge.licenses (v07)

- poly_cells f — pinneapple_data.cae.dataset (v04)
- Polygon C — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- Polygon C — pinneapple_worldmodel.geometry (v04)
- polygon_area f — pinneapple_design.fitmesh.catalog (v06)
- polygon_area f — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- polygon_contains f — pinneapple_physics.pde_environment.presets._curve_geometry (v02)
- polygon_domain f — pinneapple_simulation.meshless.domains (v03)
- polygon_perimeter_sample f — pinneapple_physics.pde_environment.presets._curve_geometry (v02)
- polyline_segments f — pinneapple_simulation.electromagnetics.statics (v03)
- PolyMesh C — pinneapple_data.cae.model (v04)
- polynomial_chaos f — pinneapple_analysis.uncertainty.pce (v05)
- polytropic_head_from_psi f — pinneapple_systems.process_components.similarity_map (v06)
- PolytropicPathResult C — pinneapple_systems.process_components.polytropic_path (v06)
- pooled f — pinneapple_physics.metrics (v02)
- Port C — pinneapple_flow.registry (v07)
- port_volume_m3 f — pinneapple_physics.closed_form.helmholtz_resonator (v02)
- PortDesign C — pinneapple_design.surrogate_inverse.reflex_port (v06)
- Pose C — pinneapple_simulation.rover.geometry (v03)
- pose_at f — pinneapple_simulation.rover.autolab.render (v03)
- posed_parts f — pinneapple_simulation.vehicle.viz (v03)
- position f — pinneapple_physics.astro.facts (v02)
- position_angle f — pinneapple_physics.astro.frames (v02)
- post_depletion_decay_rate_constant_per_day f — pinneapple_physics.closed_form.membrane_diffusion (v02)
- post_filter_transfer f — pinneapple_physics.multiscale.filters (v02)
- post_step f — pinneapple_simulation.pipelines.freecad_openfoam.steps (v03)
- PosteriorSamples C — pinneapple_analysis.inverse_problems.estimation (v05)
- PotentialNet C — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- power_board f — pinneapple_reason.cases (v05)
- power_spectrum f — pinneapple_systems.time_series.eda.plots (v06)
- PowerPCB C — pinneapple_presets.cases.pcb (v02)
- poynting_balance f — pinneapple_physics.maxwell_checks (v02)
- poynting_flux_dft f — pinneapple_simulation.electromagnetics.fdtd (v03)
- pp_test f — pinneapple_systems.time_series.audit.tests (v06)
- prandtl f — pinneapple_physics.physics_audit.regime (v02)
- pratt f — pinneapple_flow.nodes_ext.flight (v07)
- pratt_check f — pinneapple_simulation.flight.encounter (v03)
- pratt_gust_load f — pinneapple_simulation.flight.turbulence (v03)
- precession_matrix f — pinneapple_physics.astro.frames (v02)

- `precompute` f — `pinneapple_hub.zoo.demo` (v07)
- `predict` f — `pinneapple_arena.pointwise.training` (v04)
- `predict` f — `pinneapple_hub.zoo` (v07)
- `predict` f — `pinneapple_neural` (v04)
- `predict` f — `pinneapple_neural.trainer.inference` (v04)
- `predict` f — `pinneapple_systems.sports.football.ball` (v06)
- `predict_and_plot` f — `pinneapple_tools.visualization.pinn` (v07)
- `predict_constant_velocity` f — `pinneapple_systems.sports.football.kinematics` (v06)
- `predict_frequency_hz` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `predict_gnn` f — `pinneapple_design.aero.train` (v06)
- `predict_limited` f — `pinneapple_systems.sports.football.kinematics` (v06)
- `predict_mlp` f — `pinneapple_design.aero.train` (v06)
- `predict_models` f — `pinneapple_systems.sports.football.pc_study` (v06)
- `predict_rate_mg_day` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `predict_single_step` f — `pinneapple_neural.workflows.dense_volume_operator` (v04)
- `predict_single_step_parametric` f — `pinneapple_neural.workflows.parametric_dense_volume_operator` (v04)
- `predict_stress_pa` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `predict_temperature_c` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `predict_temperature_c` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `predict_to_frd` f — `pinneapple_simulation.external_solvers.calculix.virtual_ccx` (v03)
- `predictability_report` f — `pinneapple_systems.time_series.predictability` (v06)
- `prediction_card` f — `pinneapple_simulation.electromagnetics.antenna.design` (v03)
- `prediction_quality` f — `pinneapple_arena.pointwise.training` (v04)
- `prediction_samples` f — `pinneapple_systems.sports.football.kin_study` (v06)
- `prediction_study` f — `pinneapple_systems.sports.football.kin_study` (v06)
- `prediction_view` f — `pinneapple_neural.architectures.rom.views` (v04)
- `predictive_policy` f — `pinneapple_twin3d.wear.optimize` (v06)
- `predictor_module` f — `pinneapple_hub.zoo.export` (v07)
- `PrefetchConfig` C — `pinneapple_data.zarr_prefetch` (v04)
- `PrefetchZarrUPDIterable` C — `pinneapple_data.zarr_prefetch` (v04)
- `preflight` f — `pinneapple_data.preflight` (v04)
- `preflight` f — `pinneapple_systems.digital_twin.product.commission` (v06)
- `preflight_check` f — `pinneapple_analysis.benchmark.validation` (v05)
- `PreparationError` C — `pinneapple_analysis.benchmark.runner` (v05)

- prepare f — pinneapple_analysis.benchmark.api (v05)
- prepare f — pinneapple_hub.zoo.publish (v07)
- prepare f — pinneapple_twin3d.wear.optimize (v06)
- prepare_dataset f — pinneapple_llm.finetune (v07)
- prepare_digital_twin_task f — pinneapple_orchestration.tasks.digital_twin_tasks (v01)
- prepare_problem f — pinneapple_analysis.benchmark.runner (v05)
- prepare_step f — pinneapple_simulation.pipelines.freecad_openfoam.steps (v03)
- prepare_tutorial f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- preprocess f — pinneapple_systems.time_series.solar_cycle (v06)
- PreprocessPipeline C — pinneapple_neural.trainer.preprocess (v04)
- PreprocessStep C — pinneapple_neural.trainer.preprocess (v04)
- Preset C — pinneapple_analysis.inverse_problems.presets (v05)
- Preset C — pinneapple_presets.core (v02)
- preset_bodies f — pinneapple_design.design_optimizer.presets (v06)
- preset_info f — pinneapple_design.design_optimizer.presets (v06)
- preset_name f — pinneapple_physics.pde_environment.presets.tag_geometry (v02)
- PresetCache C — pinneapple_presets.cases.airfoil (v02)
- PresetDraftError C — pinneapple_problemdesign.preset_authoring (v02)
- PresetListItem C — pinneapple_app.backend.routers.problems (v07)
- PresetPhysics C — pinneapple_app.backend.core.physics_adapter (v07)
- PresetSurrogate C — pinneapple_presets.surrogate (v02)
- pressure_drop f — pinneapple_tools.automation.verify (v07)
- pressure_gradient f — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- pressure_only f — pinneapple_simulation.rover.checks (v03)
- pressure_ratio f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- PressureProfile C — pinneapple_systems.process_components.non_newtonian_pipe_flow (v06)
- PressureVessel C — pinneapple_presets.cases.pressure_vessel (v02)
- preview f — pinneapple_data.cae.dataset (v04)
- preview f — pinneapple_design.qualitative.preview (v06)
- prim_to_cons f — pinneapple_physics.blackhole.hydro_numba (v02)
- principal_curvatures f — pinneapple_design.geometry.ops.curvature (v06)
- print_result f — pinneapple_arena.external.__main__ (v04)
- Prior C — pinneapple_analysis.inverse_problems.estimation (v05)
- prior f — pinneapple_condense.exceptions (v02)
- prioritize f — pinneapple_data.doe.coverage (v04)
- prius_2004 f — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- prius_drive f — pinneapple_flow.nodes_ext.electromagnetics (v07)

- prius_flux_map f — pinneapple_flow.nodes_ext.electromagnetics (v07)
- prius_machine f — pinneapple_flow.nodes_ext.electromagnetics (v07)
- prius_phase_resistance f — pinneapple_simulation.electromagnetics.machines.drive (v03)
- prius_skew f — pinneapple_flow.nodes_ext.electromagnetics (v07)
- prius_thermal f — pinneapple_flow.nodes_ext.em_prius (v07)
- prius_tolerances f — pinneapple_flow.nodes_ext.electromagnetics (v07)
- prius_torque_curve f — pinneapple_simulation.electromagnetics.machines.verification (v03)
- ProbabilisticMetrics C — pinneapple_systems.time_series.metrics_ext.probabilistic (v06)
- Probe C — pinneapple_simulation.electromagnetics.fdtd (v03)
- probe f — pinneapple_core.onboarding.catalog (v01)
- probe_dataset f — pinneapple_arena.external.dataset_probe (v04)
- probe_strouhal f — pinneapple_simulation.numerical_solvers.cylinder_wake (v03)
- Problem C — pinneapple_analysis.benchmark.registry (v05)
- Problem C — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- Problem C — pinneapple_simulation.meshless.problems (v03)
- problem f — pinneapple_lab.curation (v07)
- problem_from_spec f — pinneapple_decision.schema (v05)
- problem_from_spec f — pinneapple_design.design_optimizer.spec (v06)
- problem_headline f — pinneapple_lab.curation (v07)
- problem_in_dim f — pinneapple_arena.pointwise.training (v04)
- problem_out_dim f — pinneapple_arena.pointwise.training (v04)
- ProblemBuilder C — pinneapple_physics.pde_environment.builder (v02)
- ProblemConfig C — pinneapple_arena.config (v04)
- ProblemContext C — pinneapple_design.fitmesh.context (v06)
- ProblemDefinition C — pinneapple_app.backend.core.problem (v07)
- ProblemParameter C — pinneapple_arena.pointwise.schema (v04)
- problems f — pinneapple_analysis.benchmark.registry (v05)
- problems f — pinneapple_flow.examples (v07)
- problems f — pinneapple_flow.references (v07)
- ProblemSpec C — pinneapple_physics.pde_environment.spec (v02)
- ProblemSpec C — pinneapple_problemdesign.schema (v02)
- ProblemStatement C — pinneapple_worldmodel.orchestrator (v04)
- ProblemStore C — pinneapple_registry.problem_store (v01)
- ProblemTemplate C — pinneapple_design.intent.template (v06)
- profile_cd0 f — pinneapple_design.intent.templates.glider_wing (v06)
- profile_cd0 f — pinneapple_simulation.flight.catalog (v03)
- profile_likelihood f — pinneapple_analysis.inverse_problems.identifiability (v05)
- profile_ops f — pinneapple_analysis.cost.torch_ops (v05)
- profile_params f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- profile_params f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)

- profile_verdicts f — pinneapple_physics.darcy_checks (v02)
- ProfilerConfig C — pinneapple_neural.trainer.hpc (v04)
- profiles f — pinneapple_core.onboarding.doctor (v01)
- progressive_failure f — pinneapple_design.bricks.statics (v06)
- project_forcing f — pinneapple_systems.time_series.enso.pinn (v06)
- project_integral f — pinneapple_physics.conservation (v02)
- project_lines f — pinneapple_physics.weather.viz (v02)
- project_points_to_surface f — pinneapple_design.geometry.gen.primitives (v06)
- project_reachable f — pinneapple_systems.sports.football.kinematics (v06)
- project_tree f — pinneapple_tools.hpo_experiments.utils.verify_runtime (v07)
- projection_inputs f — pinneapple_simulation.accelerated_foam.learned (v03)
- ProjectionWarmStart C — pinneapple_simulation.accelerated_foam.warm_start (v03)
- prolong f — pinneapple_physics.multiscale.filters (v02)
- promote_model f — pinneapple_app.backend.routers.admin (v07)
- PromoteRequest C — pinneapple_app.backend.routers.admin (v07)
- ProofGraph C — pinneapple_analysis.proof.graph (v05)
- Prop C — pinneapple_design.intent.materials (v06)
- propagate f — pinneapple_analysis.uncertainty.tolerance (v05)
- propagate f — pinneapple_flow.nodes.methods (v07)
- propagate f — pinneapple_physics.astro.smallbodies (v02)
- propagate_space f — pinneapple_analysis.uncertainty.tolerance (v05)
- propagate_two_body_j2 f — pinneapple_physics.astro.conjunction (v02)
- propagation_constant f — pinneapple_simulation.electromagnetics.waves (v03)
- Propeller C — pinneapple_simulation.flight.propulsion (v03)
- proper f — pinneapple_lab.explorer (v07)
- properties f — pinneapple_design.aero.geometry (v06)
- proposal f — pinneapple_flow.planner.extensions (v07)
- proposals f — pinneapple_flow.planner.extensions (v07)
- propose_mapping f — pinneapple_data.unified (v04)
- propose_parameters f — pinneapple_design.fitmesh.llm (v06)
- Protocol C — pinneapple_systems.time_series.forecast_lab.study (v06)
- protocol f — pinneapple_systems.time_series.enso.study (v06)
- Provenance C — pinneapple_data.contracts.models (v04)
- Provenance C — pinneapple_llm.qa.spec (v07)
- provenance f — pinneapple_security.attestation (v05)
- ProvenanceRecord C — pinneapple_analysis.verification.provenance (v05)
- prune f — pinneapple_flow.autocalls (v07)
- psd f — pinneapple_simulation.flight.turbulence (v03)
- psd_figure f — pinneapple_simulation.flight.figures (v03)
- psd_slope f — pinneapple_physics.blackhole.convergence (v02)
- psd_temporal f — pinneapple_simulation.flight.turbulence (v03)
- pseudonymize f — pinneapple_security.privacy (v05)
- pseudonymize_column f — pinneapple_security.privacy (v05)
- pt_chain f — pinneapple_flow.nodes_ext.validation (v07)
- pt_violation f — pinneapple_physics.non_hermitian.pt_dimer (v02)

- PTChain C — pinneapple_physics.non_hermitian.pt_dimer (v02)
- PTDimer C — pinneapple_physics.non_hermitian.pt_dimer (v02)
- PublicSeries C — pinneapple_systems.time_series.datasets.public (v06)
- publish f — pinneapple_hub.zoo.publish (v07)
- publish_model_task f — pinneapple_orchestration.tasks.registry_tasks (v01)
- publish_space f — pinneapple_hub.zoo.publish (v07)
- PublishedForecasts C — pinneapple_physics.weather.evaluate (v02)
- pump f — pinneapple_tools.automation.scripts.freecad_gui_job (v07)
- PumpCurve C — pinneapple_systems.process_components.water_hammer (v06)
- pumped_main f — pinneapple_reason.cases (v05)
- PumpModifiedMLP C — pinneapple_systems.component_library.components (v06)
- PumpNPSH C — pinneapple_presets.cases.pump_npsh (v02)
- PurgedKFoldSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- PurgedWalkForwardSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- push_to_hub f — pinneapple_hub.hub (v07)
- py_compile_all f — pinneapple_tools.hpo_experiments.utils.verify_runtime (v07)
- PythonAdapter C — pinneapple_core.workflows.adapters (v01)
- PywinautoDriver C — pinneapple_tools.automation.display (v07)

Q

- q f — pinneapple_reason.modes._util (v05)
- Q4Grid C — pinneapple_design.topology.problems (v06)
- q_from_gain f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- QiskitBackend C — pinneapple_quantum.backends.backend (v04)
- QTrainer C — pinneapple_quantum.training.trainer (v04)
- QTrainerConfig C — pinneapple_quantum.training.trainer (v04)
- QTrainHistory C — pinneapple_quantum.training.trainer (v04)
- quadratic_in_T f — pinneapple_systems.process_components.reaction_kinetics (v06)
- QuadraticRSM C — pinneapple_autolab.surrogates (v07)
- QualitativeModel C — pinneapple_design.qualitative.models (v06)
- QualitativePreview C — pinneapple_design.qualitative.preview (v06)
- quality f — pinneapple_design.fitmesh.mesh_gen (v06)
- quality f — pinneapple_design.geometry_library.meshops (v06)
- quality_key f — pinneapple_hub.library (v07)
- QualityPipeline C — pinneapple_llm.qa.pipeline (v07)
- QualityReport C — pinneapple_llm.qa.report (v07)
- quantify_uncertainty f — pinneapple_analysis.verification.uncertainty_inverse (v05)
- quantile_bins f — pinneapple_analysis.uncertainty.conformal_study (v05)
- QuantileConfig C — pinneapple_analysis.uncertainty.quantile (v05)

- `QuantileHead` C — `pinneapple_analysis.uncertainty.quantile` (v05)
- `QuantileLoss` C — `pinneapple_analysis.uncertainty.quantile` (v05)
- `QuantitativeCheck` C — `pinneapple_design.qualitative.preview` (v06)
- `Quantity` C — `pinneapple_data.physical_units` (v04)
- `Quantity` C — `pinneapple_design.intent.spec` (v06)
- `Quantity` C — `pinneapple_design.qualitative.models` (v06)
- `Quantity` C — `pinneapple_llm.qa.spec` (v07)
- `Quantity` C — `pinneapple_physics.physical_problem` (v02)
- `quantity_map` f — `pinneapple_llm.qa.spec` (v07)
- `quantity_of` f — `pinneapple_data.cae.dataset` (v04)
- `QuantityGuess` C — `pinneapple_data.physical_units` (v04)
- `quantum_kernel` f — `pinneapple_quantum.pipeline.presets` (v04)
- `QuantumBackend` C — `pinneapple_quantum.backends.backend` (v04)
- `QuantumCircuitConfig` C — `pinneapple_quantum.circuits.base` (v04)
- `QuantumCollocationDataset` C — `pinneapple_quantum.data.quantum_dataset` (v04)
- `QuantumModel` C — `pinneapple_quantum.models.quantum_model` (v04)
- `QuantumObservable` C — `pinneapple_quantum.data.quantum_dataset` (v04)
- `QuantumState` C — `pinneapple_quantum.data.quantum_dataset` (v04)
- `quasi_steady_valid` f — `pinneapple_systems.digital_twin.pipe_loop.plant` (v06)
- `quat_from_matrix` f — `pinneapple_simulation.multibody.tree` (v03)
- `quat_from_matrix` f — `pinneapple_simulation.rover.geometry` (v03)
- `quat_from_matrix` f — `pinneapple_tools.visualization.studio.animation` (v07)
- `quat_mul` f — `pinneapple_simulation.multibody.tree` (v03)
- `quat_multiply` f — `pinneapple_simulation.flight.rigid_body` (v03)
- `quat_to_dcm` f — `pinneapple_simulation.flight.rigid_body` (v03)
- `quat_to_euler` f — `pinneapple_simulation.flight.rigid_body` (v03)
- `quat_to_matrix` f — `pinneapple_simulation.multibody.tree` (v03)
- `Query` C — `pinneapple_query.core` (v02)
- `query_checks` f — `pinneapple_hub.zoo` (v07)
- `query_checks` f — `pinneapple_hub.zoo.checks` (v07)
- `query_demo` f — `pinneapple_condense.query_benchmark` (v02)
- `Question` C — `pinneapple_design.intent.spec` (v06)
- `quickstart` f — `pinneapple` (v01)
- `quintic` f — `pinneapple_simulation.multibody.arm` (v03)
- `quiz` f — `pinneapple_physics.astro.facts` (v02)
- `qz_field` f — `pinneapple_simulation.electromagnetics.reflector.compact_range` (v03)
- `qz_figure` f — `pinneapple_simulation.electromagnetics.reflector.study` (v03)
- `qz_metrics` f — `pinneapple_simulation.electromagnetics.reflector.qz` (v03)
- `qz_metrics_at` f — `pinneapple_simulation.electromagnetics.reflector.study_report` (v03)
- `qz_study` f — `pinneapple_simulation.electromagnetics.reflector.study` (v03)

R

- R2 C — pinneapple_neural.trainer.metrics (v04)
- r2 f — pinneapple_physics.metrics (v02)
- r2 f — pinneapple_systems.time_series.metrics (v06)
- r2_fn f — pinneapple_systems.time_series.metrics (v06)
- racecar_domain f — pinneapple_autolab.domains.racecar (v07)
- RaceCarAero C — pinneapple_autolab.domains.racecar (v07)
- radec f — pinneapple_physics.astro.frames (v02)
- radial_fd f — pinneapple_physics.closed_form.pipe_insulation (v02)
- radial_power_spectrum f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- RadialLayers C — pinneapple_simulation.electromagnetics.analytic (v03)
- radiate f — pinneapple_simulation.electromagnetics.reflector.po (v03)
- radiation_pattern f — pinneapple_tools.visualization.electromagnetics (v07)
- radiator f — pinneapple_design.geometry.vehicle_cad (v06)
- RadiossMesh C — pinneapple_simulation.external_solvers.openradioss.vtk_reader (v03)
- ramp f — pinneapple_tools.visualization.studio.colormap (v07)
- random_block_mask f — pinneapple_data.synth.operators.masks (v04)
- random_craters f — pinneapple_simulation.rover.terrain (v03)
- random_designs f — pinneapple_design.thermal.heatsink.monotone (v06)
- random_points f — pinneapple_analysis.validation.physics_learning (v05)
- random_rocks f — pinneapple_simulation.rover.terrain (v03)
- random_vorticity f — pinneapple_physics.multiscale.ns2d (v02)
- random_walk f — pinneapple_systems.time_series.synthetic (v06)
- RandomForestForecaster C — pinneapple_systems.time_series.models.classical (v06)
- RandomizerConfig C — pinneapple_worldmodel.reality_randomizer (v04)
- RandomSearchProposer C — pinneapple_arena.autoresearch (v04)
- RangeLog C — pinneapple_simulation.rover.contact (v03)
- rank_by_accuracy f — pinneapple_arena.arena (v04)
- rank_geometries f — pinneapple_design.fitmesh.llm (v06)
- rank_key f — pinneapple_analysis.verification.pipeline (v05)
- rank_models f — pinneapple_arena.pointwise.training (v04)
- RankedItem C — pinneapple_tools.hpo_experiments.models (v07)
- rans_residuals f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- RansNet C — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- rapid_steady_state_profile f — pinneapple_systems.process_components.pipe_network_1d (v06)
- rasterize f — pinneapple_design.geometry_library.render (v06)
- rasterize_topdown f — pinneapple_tools.nvidia.omniverse (v07)
- rate_limit f — pinneapple_systems.excitation (v06)
- rate_of_change f — pinneapple_systems.time_series.features.engineering (v06)
- RateLimiter C — pinneapple_security.web (v05)

- RateRule C — pinneapple_security.web (v05)
- ratio f — pinneapple_flow.nodes_ext.cfd (v07)
- raw_distribution f — pinneapple_decision.debias (v05)
- raw_residual f — pinneapple_physics.multiscale.comparison (v02)
- rayleigh f — pinneapple_physics.physics_audit.regime (v02)
- rayleigh_qsca f — pinneapple_simulation.electromagnetics.mie (v03)
- raymer_takeoff f — pinneapple_simulation.flight.takeoff (v03)
- raymer_variants f — pinneapple_simulation.flight.takeoff (v03)
- RayPaths C — pinneapple_physics.blackhole.raytrace (v02)
- rbf_fd_weights f — pinneapple_simulation.meshless.methods (v03)
- RBFCollocationSolver C — pinneapple_simulation.numerical_solvers.meshfree (v03)
- RBFInterpolator C — pinneapple_design.geometry.ops.meshfree (v06)
- RBFNetwork C — pinneapple_neural.architectures.reservoir_computing.rbf (v04)
- RCBase C — pinneapple_neural.architectures.reservoir_computing.base (v04)
- RCDischarge C — pinneapple_science.em.bench (v02)
- RCOutput C — pinneapple_neural.architectures.reservoir_computing.base (v04)
- RDPAccountant C — pinneapple_security.dp (v05)
- reachable_radius f — pinneapple_systems.sports.football.kinematics (v06)
- ReaClient C — pinneapple_tools.rea_bridge.client (v07)
- Reaction C — pinneapple_systems.process_components.reaction_kinetics (v06)
- reaction_diffusion_2d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- reaction_exact f — pinneapple_physics.paper_problems (v02)
- ReactionDiffusion1D C — pinneapple_simulation.numerical_solvers.reaction_diffusion (v03)
- ReactionNetwork C — pinneapple_systems.process_components.reaction_kinetics (v06)
- read_abaqus_inp_mesh f — pinneapple_simulation.external_solvers.cfd_formats.abaqus_reader (v03)
- read_any f — pinneapple_data.cae.io (v04)
- read_binary_block f — pinneapple_simulation.external_solvers.openfoam.binary_reader (v03)
- read_boundary f — pinneapple_data.cae.foam (v04)
- read_boundary f — pinneapple_twin3d.openfoam (v06)
- read_boundary_field f — pinneapple_twin3d.openfoam (v06)
- read_boundary_values f — pinneapple_data.cae.foam (v04)
- read_case f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- read_case_displacements f — pinneapple_simulation.external_solvers.openradioss.vtk_reader (v03)
- read_cases f — pinneapple_core.workflows.datasets (v01)
- read_cgns_mesh_and_fields f — pinneapple_simulation.external_solvers.cfd_formats.cgns_reader (v03)
- read_coefficients f — pinneapple_design.aero.case (v06)

- `read_dat` f — `pinneapple_data.simulation_metadata.openfoam` (v04)
- `read_dat_displacements` f — `pinneapple_simulation.external_solvers.calculix.frd` (v03)
- `read_dataset` f — `pinneapple_lab.dataset` (v07)
- `read_deck` f — `pinneapple_data.cae.inp` (v04)
- `read_document` f — `pinneapple_data.formfill.extract` (v04)
- `read_dump` f — `pinneapple_simulation.accelerated_foam.ldu` (v03)
- `read_dxf_polylines` f — `pinneapple_simulation.electromagnetics.machines.interop` (v03)
- `read_energy_error` f — `pinneapple_simulation.external_solvers.openradioss.log_reader` (v03)
- `read_energy_errors` f — `pinneapple_simulation.external_solvers.openradioss.log_reader` (v03)
- `read_exodus` f — `pinneapple_simulation.external_solvers.cfd_formats.exodus_reader` (v03)
- `read_face_compact_list` f — `pinneapple_simulation.external_solvers.openfoam.binary_reader` (v03)
- `read_faces` f — `pinneapple_data.cae.foam` (v04)
- `read_faces` f — `pinneapple_twin3d.openfoam` (v06)
- `read_farfield` f — `pinneapple_simulation.electromagnetics.antenna.scene3d` (v03)
- `read_femm_lua_polygons` f — `pinneapple_simulation.electromagnetics.machines.interop` (v03)
- `read_field` f — `pinneapple_data.cae.foam` (v04)
- `read_field` f — `pinneapple_simulation.accelerated_foam.case` (v03)
- `read_filled` f — `pinneapple_data.formfill.fill` (v04)
- `read_flow_runs` f — `pinneapple_hub.library.flowruns` (v07)
- `read_fluent_mesh_nodes` f — `pinneapple_simulation.external_solvers.cfd_formats.fluent_mesh_reader` (v03)
- `read_frd` f — `pinneapple_data.cae.frd` (v04)
- `read_frd` f — `pinneapple_simulation.external_solvers.calculix.frd` (v03)
- `read_geotaichi` f — `pinneapple_simulation.external_solvers.geotaichi` (v03)
- `read_gltf` f — `pinneapple_tools.visualization.studio.scene` (v07)
- `read_header` f — `pinneapple_simulation.external_solvers.openfoam.binary_reader` (v03)
- `read_inp` f — `pinneapple_simulation.external_solvers.calculix.inp` (v03)
- `read_internal` f — `pinneapple_design.aero.case` (v06)
- `read_internal_field` f — `pinneapple_simulation.external_solvers.openfoam.binary_reader` (v03)
- `read_label_list` f — `pinneapple_simulation.external_solvers.openfoam.binary_reader` (v03)
- `read_labels` f — `pinneapple_data.cae.foam` (v04)
- `read_ldr` f — `pinneapple_design.bricks.ldraw` (v06)
- `read_manifest` f — `pinneapple_design.geometry_library.store` (v06)

- `read_members` f — `pinneapple_security.archives` (v05)
- `read_meshio` f — `pinneapple_data.cae.io` (v04)
- `read_nearfield` f — `pinneapple_simulation.electromagnetics.antenna.scene3d` (v03)
- `read_npz` f — `pinneapple_hub.zoo.parquet` (v07)
- `read_obj` f — `pinneapple_tools.visualization.studio.scene` (v07)
- `read_parquet` f — `pinneapple_hub.zoo.parquet` (v07)
- `read_particle_frames` f — `pinneapple_simulation.particle_dynamics.geo_mpm` (v03)
- `read_point_bundle` f — `pinneapple_simulation.external_solvers.cfd_formats.convert` (v03)
- `read_points` f — `pinneapple_data.cae.foam` (v04)
- `read_points` f — `pinneapple_data.cae.io` (v04)
- `read_points` f — `pinneapple_twin3d.openfoam` (v06)
- `read_polymesh` f — `pinneapple_data.cae.foam` (v04)
- `read_problem` f — `pinneapple_design.fitmesh.llm` (v06)
- `read_profiles` f — `pinneapple_simulation.numerical_solvers.free_surface` (v03)
- `read_readings` f — `pinneapple_systems.digital_twin.product.commission` (v06)
- `read_result3d` f — `pinneapple_design.aero.case3d` (v06)
- `read_results` f — `pinneapple_simulation.external_solvers.fluidx3d` (v03)
- `read_run` f — `pinneapple_tools.benchmark_suite.crossframework.records` (v07)
- `read_runs` f — `pinneapple_hub.library` (v07)
- `read_sampled_scalar_field` f — `pinneapple_simulation.external_solvers.openfoam.sampling` (v03)
- `read_shell_thickness` f — `pinneapple_simulation.external_solvers.openradioss.deck` (v03)
- `read_sidecar` f — `pinneapple_data.contracts.tables` (v04)
- `read_solver_dict` f — `pinneapple_simulation.accelerated_foam.benchmark` (v03)
- `read_splash_manifest` f — `pinneapple_simulation.external_solvers.openfoam.splash_packer` (v03)
- `read_stage_geometry` f — `pinneapple_tools.nvidia.omniverse` (v07)
- `read_starccm_cgns` f — `pinneapple_simulation.external_solvers.cfd_formats.starccm_reader` (v03)
- `read_starccm_table` f — `pinneapple_simulation.external_solvers.cfd_formats.starccm_reader` (v03)
- `read_stl` f — `pinneapple_data.cae.io` (v04)
- `read_stl` f — `pinneapple_design.geometry_library.exporters` (v06)
- `read_stl` f — `pinneapple_tools.visualization.studio.scene` (v07)
- `read_surface_mesh` f — `pinneapple_simulation.external_solvers.cfd_formats.convert` (v03)
- `read_surface_scalar` f — `pinneapple_check.fields` (v05)
- `read_system` f — `pinneapple_simulation.accelerated_foam.ldu` (v03)
- `read_table` f — `pinneapple_analysis.surrogate.data` (v05)
- `read_table` f — `pinneapple_check.checks.models` (v05)
- `read_table` f — `pinneapple_data.contracts.tables` (v04)

- `read_table` f — `pinneapple_simulation.electromagnetics.antenna.runner` (v03)
- `read_table_with_roles` f — `pinneapple_analysis.surrogate.contracts` (v05)
- `read_time` f — `pinneapple_data.cae.foam` (v04)
- `read_timestamp` f — `pinneapple_systems.digital_twin.product.commission` (v06)
- `read_trajectories` f — `pinneapple_data.datasets.deepmind_mgn` (v04)
- `read_tunables` f — `pinneapple_arena.autoresearch` (v04)
- `read_upload` f — `pinneapple_data.simulation_metadata` (v04)
- `read_vector_field_points` f — `pinneapple_simulation.external_solvers.openfoam.binary_reader` (v03)
- `read_vtk` f — `pinneapple_simulation.numerical_solvers.lbm_tunnel` (v03)
- `read_vtk_mesh` f — `pinneapple_simulation.external_solvers.openradioss.vtk_reader` (v03)
- `read_vtp` f — `pinneapple_tools.visualization.studio.scene` (v07)
- `readings` f — `pinneapple_analysis.benchmark.builtin.inverse_heat_impl` (v05)
- `readings` f — `pinneapple_systems.digital_twin.pipe_loop.plant` (v06)
- `readme` f — `pinneapple_presets.report` (v02)
- `readme` f — `pinneapple_science.report` (v02)
- `ReadReport` C — `pinneapple_design.bricks.ldraw` (v06)
- `ReaError` C — `pinneapple_tools.rea_bridge.client` (v07)
- `RealityRandomizer` C — `pinneapple_worldmodel.reality_randomizer` (v04)
- `realizability_mismatch` f — `pinneapple_physics.multiscale.filters` (v02)
- `RealSpectralConv2d` C — `pinneapple_neural.architectures.neural_operators.fno_export` (v04)
- `RealtimePipeline` C — `pinneapple_systems.digital_twin.realtime_pipeline` (v06)
- `RealtimePipelineConfig` C — `pinneapple_systems.digital_twin.realtime_pipeline` (v06)
- `ReaResult` C — `pinneapple_tools.rea_bridge.client` (v07)
- `reason` f — `pinneapple_mcp.tools` (v07)
- `reason` f — `pinneapple_reason.engine` (v05)
- `reason_check` f — `pinneapple_mcp.tools` (v07)
- `reason_library` f — `pinneapple_mcp.tools` (v07)
- `Reasoner` C — `pinneapple_reason.engine` (v05)
- `ReaUnavailable` C — `pinneapple_tools.rea_bridge.client` (v07)
- `recession` f — `pinneapple_systems.digital_twin.recession` (v06)
- `RechargeOscillatorForecaster` C — `pinneapple_systems.time_series.enso.forecasters` (v06)
- `recipes` f — `pinneapple_hub.library.sources` (v07)
- `reciprocity` f — `pinneapple_physics.maxwell_checks` (v02)
- `recommend` f — `pinneapple_core.onboarding.recommend` (v01)
- `recommend` f — `pinneapple_llm.workbench.models` (v07)
- `recommend` f — `pinneapple_physics.physics_audit.improvement` (v02)
- `recommend` f — `pinneapple_systems.operations.control` (v06)

- recommend_architecture f — pinneapple_analysis.verification.architecture_recommendation (v05)
- recommend_for_problem f — pinneapple_app.backend.core.model_catalog (v07)
- recommend_main f — pinneapple_core.onboarding.cli (v01)
- recommend_method f — pinneapple_problemdesign.method_selection (v02)
- recommend_method_from_spec f — pinneapple_problemdesign.method_selection (v02)
- recommend_models f — pinneapple_app.backend.core.problem_registry (v07)
- recommend_models f — pinneapple_app.backend.routers.models (v07)
- recommend_tools f — pinneapple_analysis.verification.tool_recommendation (v05)
- Recommendation C — pinneapple_core.onboarding.recommend (v01)
- Recommendation C — pinneapple_llm.workbench.models (v07)
- Recommendation C — pinneapple_physics.physics_audit.improvement (v02)
- RecommendationResult C — pinneapple_veriphysics.recommend (v05)
- recommended_solver f — pinneapple_app.backend.core.problem_registry (v07)
- reconstruct_interpolate f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruct_model_based f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruct_spectral f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruct_ssa f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruct_wavelet f — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- reconstruction_view f — pinneapple_neural.architectures.rom.views (v04)
- ReconstructionResult C — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- record f — pinneapple_hub.library.flowruns (v07)
- record_checks f — pinneapple_veriphysics.ledger (v05)
- record_replay f — pinneapple_autolab.dashboard (v07)
- record_runs f — pinneapple_hub.library (v07)
- recorded_dashboard f — pinneapple_autolab.domains.racecar (v07)
- rect_cavity_modes f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_cavity_resonance f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_cavity_te101_q f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_mesh f — pinneapple_simulation.electromagnetics.mesh (v03)
- rect_te_mn_fields f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_waveguide_cutoff f — pinneapple_simulation.electromagnetics.waves (v03)
- rect_waveguide_modes f — pinneapple_simulation.electromagnetics.waves (v03)
- Rectangle C — pinneapple_worldmodel.geometry (v04)

- rectangle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- rectangle_mask f — pinneapple_simulation.numerical_solvers.lbm (v03)
- rectangular_beam f — pinneapple_design.surrogate_inverse.meshes (v06)
- rectangular_section_properties f — pinneapple_systems.process_components.beam_statics (v06)
- RecurrentCatalog C — pinneapple_neural.architectures.recurrent.registry (v04)
- RecurrentConfig C — pinneapple_systems.time_series.models.recurrent (v06)
- RecurrentModelBase C — pinneapple_neural.architectures.recurrent.base (v04)
- redact f — pinneapple_flow.store (v07)
- redact f — pinneapple_llm.workbench.finetune (v07)
- redact f — pinneapple_security.privacy (v05)
- Reduced C — pinneapple_condense.algebraic (v02)
- reduced_block f — pinneapple_condense.learned (v02)
- ReducedModel C — pinneapple_neural.architectures.rom.study (v04)
- redundancy f — pinneapple_data.doe.coverage (v04)
- ref f — pinneapple_flow.references (v07)
- refan_concave_caps f — pinneapple_design.geometry_library.meshops (v06)
- Reference C — pinneapple_check.config (v05)
- Reference C — pinneapple_condense.query (v02)
- Reference C — pinneapple_design.fitmesh.catalog (v06)
- Reference C — pinneapple_presets.core (v02)
- Reference C — pinneapple_science.core (v02)
- reference f — pinneapple_design.aero.geometry_onnx (v06)
- reference_cantilever f — pinneapple_design.bricks.goals (v06)
- reference_data f — pinneapple_hub.zoo.web (v07)
- reference_decay f — pinneapple_physics.astro.decay (v02)
- reference_for_spec f — pinneapple_design.design_optimizer.spec (v06)
- reference_observables f — pinneapple_condense.substructure (v02)
- reference_solve f — pinneapple_design.thermal.fin_array_speed (v06)
- reference_solve f — pinneapple_design.thermal.heatsink.surrogate (v06)
- reference_solve_batch f — pinneapple_design.thermal.heatsink.surrogate (v06)
- reference_stack f — pinneapple_condense.package (v02)
- reference_trajectory f — pinneapple_physics.astro.surrogate (v02)
- reference_transient f — pinneapple_condense.transient (v02)
- reference_value f — pinneapple_flow.nodes_ext.examples (v07)
- ReferenceData C — pinneapple_llm.qa.references (v07)
- refilter f — pinneapple_physics.multiscale.filters (v02)
- refine_ladder f — pinneapple_science.numerics (v02)
- refinement f — pinneapple_condense.figures (v02)
- refinement_study f — pinneapple_condense.benchmark (v02)
- RefinementConfig C — pinneapple_design.design_optimizer.refinement (v06)
- RefinementResult C — pinneapple_design.design_optimizer.refinement (v06)

- `reflect_box` f — `pinneapple_simulation.numerical_solvers.sph_boundaries` (v03)
- `reflect_field` f — `pinneapple_analysis.validation.metamorphic` (v05)
- `ReflectorMesh` C — `pinneapple_simulation.electromagnetics.reflector.geometry` (v03)
- `refraction_deg` f — `pinneapple_physics.astro.astrometry` (v02)
- `refractory_lining` f — `pinneapple_physics.pde_environment.presets.engineering` (v02)
- `regime_features` f — `pinneapple_physics.physics_audit.regime` (v02)
- `regime_series` f — `pinneapple_flow.nodes_ext.core_ts` (v07)
- `regime_tags_for` f — `pinneapple_pdb.shard` (v04)
- `region_integral_A` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `region_mask` f — `pinneapple_physics.weather.events` (v02)
- `region_masks` f — `pinneapple_physics.darcy_checks` (v02)
- `region_rel_l2` f — `pinneapple_analysis.surrogate.heldout` (v05)
- `region_stencils` f — `pinneapple_condense.meshless` (v02)
- `regional` f — `pinneapple_physics.weather.viz` (v02)
- `regional_errors` f — `pinneapple_physics.darcy_checks` (v02)
- `regional_lines` f — `pinneapple_physics.weather.viz` (v02)
- `regional_turboprop` f — `pinneapple_simulation.flight.catalog` (v03)
- `regions` f — `pinneapple_data.cae.geometry` (v04)
- `register` f — `pinneapple_design.geometry_library.core` (v06)
- `register` f — `pinneapple_design.intent.parse` (v06)
- `register` f — `pinneapple_lab.spec` (v07)
- `register` f — `pinneapple_llm.qa.validators.base` (v07)
- `register` f — `pinneapple_presets.registry` (v02)
- `register` f — `pinneapple_science.registry` (v02)
- `register` f — `pinneapple_tools.automation.adapters.base` (v07)
- `register` f — `pinneapple_tools.automation.runner_step` (v07)
- `register_adapter` f — `pinneapple_core.workflows.adapters` (v01)
- `register_advanced_templates` f — `pinneapple_design.geometry.gen.cadquery_advanced` (v06)
- `register_all` f — `pinneapple_neural.architectures.register_all` (v04)
- `register_all` f — `pinneapple_simulation.numerical_solvers.registry` (v03)
- `register_analytic` f — `pinneapple_physics.solving` (v02)
- `register_backend` f — `pinneapple_core.backend` (v01)
- `register_backend` f — `pinneapple_tools.benchmark_suite.registry` (v07)
- `register_benchmark` f — `pinneapple_pdb.benchmarks` (v04)
- `register_builder` f — `pinneapple_design.geometry.gen.primitives` (v06)
- `register_capability` f — `pinneapple_flow.planner.extensions` (v07)
- `register_component` f — `pinneapple_systems.component_library.registry` (v06)
- `register_countermeasure` f — `pinneapple_reason.library` (v05)
- `register_custom_nodes` f — `pinneapple_flow.runtime` (v07)
- `register_default_templates` f — `pinneapple_design.geometry.gen.cadquery_gen` (v06)

- register_enhancer f — pinneapple_worldmodel.photorealistic_enhancer (v04)
- register_example_run f — pinneapple_flow.examples (v07)
- register_executor f — pinneapple_core.workflows.executors (v01)
- register_family_registry f — pinneapple_neural.architectures._registry_bridge (v04)
- register_hook f — pinneapple_flow.examples (v07)
- register_into_global f — pinneapple_neural.architectures.autoencoders.registry (v04)
- register_into_global f — pinneapple_neural.architectures.benchmarks.registry (v04)
- register_into_global f — pinneapple_neural.architectures.classical_ts.registry (v04)
- register_into_global f — pinneapple_neural.architectures.continuous.registry (v04)
- register_into_global f — pinneapple_neural.architectures.convolutions.registry (v04)
- register_into_global f — pinneapple_neural.architectures.graphnn.registry (v04)
- register_into_global f — pinneapple_neural.architectures.group_b_registry (v04)
- register_into_global f — pinneapple_neural.architectures.neural_operators.registry (v04)
- register_into_global f — pinneapple_neural.architectures.physics_aware.registry (v04)
- register_into_global f — pinneapple_neural.architectures.pinns.registry (v04)
- register_into_global f — pinneapple_neural.architectures.recurrent.registry (v04)
- register_into_global f — pinneapple_neural.architectures.reservoir_computing.registry (v04)
- register_into_global f — pinneapple_neural.architectures.rom.registry (v04)
- register_into_global f — pinneapple_neural.architectures.transformers.registry (v04)
- register_manifest f — pinneapple_lab.capture (v07)
- register_method f — pinneapple_analysis.benchmark.registry (v05)
- register_method f — pinneapple_physics.solving (v02)
- register_mode f — pinneapple_reason.library (v05)
- register_port_type f — pinneapple_flow.types (v07)
- register_powersgd_hook f — pinneapple_neural.trainer.hpc (v04)
- register_preset f — pinneapple_physics.pde_environment.presets.registry (v02)
- register_problem f — pinneapple_analysis.benchmark.registry (v05)
- register_problem f — pinneapple_arena.problems (v04)
- register_rea_tools f — pinneapple_tools.rea_bridge.tools (v07)
- register_references f — pinneapple_flow (v07)
- register_references f — pinneapple_flow.references (v07)
- register_rule f — pinneapple_flow (v07)
- register_rule f — pinneapple_flow.rules (v07)
- register_soil f — pinneapple_simulation.terramechanics.soils (v03)

- register_task f — pinneapple_tools.benchmark_suite.registry (v07)
- register_topk_hook f — pinneapple_neural.trainer.hpc (v04)
- Registry C — pinneapple_flow.registry (v07)
- Registry C — pinneapple_tools.model_services.registry (v07)
- registry f — pinneapple_flow.references (v07)
- registry f — pinneapple_models (v01)
- registry_key f — pinneapple_arena.pointwise.architectures (v04)
- registry_listing f — pinneapple_flow.nodes_ext.architectures (v07)
- Regression C — pinneapple_analysis.cost.profiler (v05)
- regression_metrics f — pinneapple_tools.model_evaluation.metrics (v07)
- regression_metrics_bundle f — pinneapple_neural.trainer.metrics (v04)
- RegressionMetrics C — pinneapple_neural.trainer.metrics (v04)
- RegularizerBase C — pinneapple_analysis.inverse_problems.regularization (v05)
- rel f — pinneapple_hub.library.sources (v07)
- rel_l2 f — pinneapple_analysis.reproduction (v05)
- rel_l2 f — pinneapple_check.checks.common (v05)
- rel_l2 f — pinneapple_simulation.numerical_solvers.operator_datasets (v03)
- rel_l2_rows f — pinneapple_flow.learn (v07)
- rel_rms f — pinneapple_check.numerics (v05)
- Relation C — pinneapple_analysis.validation.metamorphic (v05)
- RelationResult C — pinneapple_analysis.validation.metamorphic (v05)
- relations_from_spec f — pinneapple_analysis.validation.metamorphic (v05)
- relative_error f — pinneapple_design.surrogate_inverse.core (v06)
- relative_errors f — pinneapple_tools.nvidia.physicsnemo (v07)
- relative_l2 f — pinneapple_analysis.cost.equal_budget (v05)
- relative_l2 f — pinneapple_physics.ensemble (v02)
- relative_l2 f — pinneapple_physics.metrics (v02)
- relative_linf f — pinneapple_physics.metrics (v02)
- relative_residual f — pinneapple_physics.maxwell_checks (v02)
- release_rate_mg_day f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- reliability f — pinneapple_systems.sports.football.metrics (v06)
- reliability_figure f — pinneapple_systems.sports.football.plots (v06)
- relief_noise f — pinneapple_tools.visualization.studio.shadows (v07)
- RelL2 C — pinneapple_neural.trainer.metrics (v04)
- ReLoBRaLo C — pinneapple_neural.trainer.loss_balancer (v04)
- reluctance_rotor f — pinneapple_flow.nodes_ext.design (v07)
- remaining_life_fields f — pinneapple_twin3d.wear.build (v06)
- remap f — pinneapple_physics.blackhole.convergence (v02)
- remap_r f — pinneapple_physics.blackhole.convergence (v02)
- remesh_surface f — pinneapple_design.geometry.ops.remesh (v06)
- remove_outlier_vertices f — pinneapple_design.geometry.ops.clean (v06)

- remove_self_intersections f — pinneapple_design.geometry.ops.clean (v06)
- render f — pinneapple_check.report (v05)
- render f — pinneapple_design.aero.render_blender (v06)
- render f — pinneapple_design.geometry_library (v06)
- render f — pinneapple_flow.nodes.viz (v07)
- render f — pinneapple_flow.nodes_ext.render_worker (v07)
- render f — pinneapple_lab.explorer (v07)
- render f — pinneapple_physics.blackhole.raytrace (v02)
- render f — pinneapple_tools.automation.workflow (v07)
- render f — pinneapple_tools.visualization.molecular (v07)
- render f — pinneapple_tools.visualization.studio (v07)
- render f — pinneapple_tools.visualization.studio.blender (v07)
- render_3d f — pinneapple_design.thermal.board_cooling.report (v06)
- render_3d f — pinneapple_simulation.electromagnetics.antenna.study (v03)
- render_3d f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- render_animation f — pinneapple_tools.visualization.studio.blender (v07)
- render_attractors f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- render_blender f — pinneapple_design.bricks.visuals (v06)
- render_comparison_figure f — pinneapple_perception.change_detection (v04)
- render_cones f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- render_datasheet f — pinneapple_data.formfill.report (v04)
- render_ensembles f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- render_evidence_report_pdf f — pinneapple_veriphysics.evidence_report (v05)
- render_frame f — pinneapple_simulation.vehicle.viz (v03)
- render_frames f — pinneapple_design.bricks.visuals (v06)
- render_frames f — pinneapple_simulation.vehicle.dashboard (v03)
- render_frames f — pinneapple_systems.time_series.enso.render (v06)
- render_frames f — pinneapple_tools.visualization.molecular (v07)
- render_geometry f — pinneapple_design.geometry_library.render (v06)
- render_gif f — pinneapple_tools.visualization.pyvista_bridge (v07)
- render_hero f — pinneapple_simulation.rover.autolab.render (v03)
- render_html f — pinneapple_design.fitmesh.report (v06)
- render_html f — pinneapple_lab.reports (v07)
- render_html f — pinneapple_veriphysics.evidence_package (v05)
- render_html f — pinneapple_veriphysics.validation_lab (v05)
- render_html_report f — pinneapple_perception.change_detection (v04)
- render_json_report f — pinneapple_problemdesign.renderers.report_json (v02)
- render_landscape f — pinneapple_systems.time_series.forecast_lab.renders (v06)
- render_legend f — pinneapple_lab.seal (v07)

- render_like f — pinneapple_worldmodel.physical_world (v04)
- render_markdown_report f — pinneapple_problemdesign.renderers.report_md (v02)
- render_md f — pinneapple_lab.reports (v07)
- render_overview f — pinneapple_simulation.rover.autolab.render (v03)
- render_page f — pinneapple_lab.html (v07)
- render_particle_frames f — pinneapple_tools.visualization.studio.particles (v07)
- render_pdf f — pinneapple_lab.reports (v07)
- render_pdf f — pinneapple_veriphysics.evidence_package (v05)
- render_png f — pinneapple_tools.visualization.pyvista_bridge (v07)
- render_run f — pinneapple_autolab.domains.racecar (v07)
- render_scene f — pinneapple_tools.visualization.studio.pyvista_render (v07)
- render_seal f — pinneapple_lab.seal (v07)
- render_stage_gif f — pinneapple_tools.nvidia.omniverse (v07)
- render_stage_png f — pinneapple_tools.nvidia.omniverse (v07)
- render_still f — pinneapple_tools.visualization.studio.blender (v07)
- render_surface f — pinneapple_tools.visualization.studio.planetary (v07)
- render_traverse f — pinneapple_simulation.rover.autolab.render (v03)
- render_visualizations f — pinneapple_neural.predictor.visualize (v04)
- RendererConfig C — pinneapple_worldmodel.physics_renderer (v04)
- RenderResult C — pinneapple_worldmodel.physics_renderer (v04)
- repair f — pinneapple_design.bricks.search (v06)
- repair f — pinneapple_flow.planner.llm (v07)
- repair_mesh f — pinneapple_design.geometry.ops.repair (v06)
- RepairAttempt C — pinneapple_llm.qa.repair (v07)
- RepairLoop C — pinneapple_llm.qa.repair (v07)
- RepairOutcome C — pinneapple_llm.qa.repair (v07)
- Replay C — pinneapple_design.bricks.dynamics (v06)
- replay f — pinneapple_analysis.validation.arena.cases (v05)
- replay f — pinneapple_design.bricks.dynamics (v06)
- replay f — pinneapple_systems.operations.live (v06)
- replay_frames f — pinneapple_design.bricks.visuals (v06)
- replay_score f — pinneapple_security.anomaly (v05)
- ReplayResult C — pinneapple_analysis.validation.arena.cases (v05)
- ReplaySimulator C — pinneapple_autolab.replay (v07)
- replicate_sector f — pinneapple_simulation.electromagnetics.machines.mesher (v03)
- Reply C — pinneapple_llm.workbench.backend (v07)
- RepoFile C — pinneapple_tools.hpo_experiments.sources.github_fetcher (v07)
- RepoResults C — pinneapple_lab.experiments.repo_results (v07)
- Report C — pinneapple_check.result (v05)
- Report C — pinneapple_data.preflight.common (v04)
- report f — pinneapple_analysis.benchmark.api (v05)
- report f — pinneapple_flow.nodes.reason (v07)
- report f — pinneapple_flow.nodes.viz (v07)

- `report` f — `pinneapple_simulation.electromagnetics.reflector.study_report` (v03)
- `report` f — `pinneapple_twin3d.wear.build` (v06)
- `report_equations` f — `pinneapple_check.checks.results` (v05)
- `report_html` f — `pinneapple_flow.decisions` (v07)
- `report_markdown` f — `pinneapple_flow.decisions` (v07)
- `report_markdown` f — `pinneapple_security.compliance` (v05)
- `report_markdown` f — `pinneapple_simulation.rover.autolab.report` (v03)
- `ReportDiff` C — `pinneapple_analysis.validation.report_artifact` (v05)
- `representatives` f — `pinneapple_design.design_optimizer.pareto` (v06)
- `reproduce` f — `pinneapple_tools.hpo_experiments.pipelines.reproduce` (v07)
- `ReproducerAgent` C — `pinneapple_tools.hpo_experiments.agents.reproducer` (v07)
- `ReproduceResult` C — `pinneapple_tools.hpo_experiments.agents.reproducer` (v07)
- `ReptileConfig` C — `pinneapple_adaptation.meta_learning.config` (v04)
- `ReptileTrainer` C — `pinneapple_adaptation.meta_learning.reptile` (v04)
- `require_admin` f — `pinneapple_app.backend.core.admin_auth` (v07)
- `require_allowed` f — `pinneapple_catalog.resources` (v01)
- `require_deepxde` f — `pinneapple_physics.deepxde_bridge.adapter` (v02)
- `require_physicsnemo_model` f — `pinneapple_neural.physicsnemo_bridge.adapters` (v04)
- `require_prepared` f — `pinneapple_analysis.benchmark.runner` (v05)
- `require_problem` f — `pinneapple_flow.nodes.problem` (v07)
- `require_pxr` f — `pinneapple_tools.nvidia.omniverse` (v07)
- `require_research_use` f — `pinneapple_neural._licencas` (v04)
- `required_speed_for_head` f — `pinneapple_systems.process_components.similarity_map` (v06)
- `Requirement` C — `pinneapple_reason.chain` (v05)
- `requirement` f — `pinneapple_flow.nodes.reason` (v07)
- `Requirements` C — `pinneapple_design.aero.aircraft` (v06)
- `Requirements3D` C — `pinneapple_design.aero.aircraft3d` (v06)
- `RequirementsAL` C — `pinneapple_design.aero.airliner3d` (v06)
- `RequirementSpec` C — `pinneapple_design.intent.spec` (v06)
- `RerunLogger` C — `pinneapple_tools.visualization.integrations.rerun_bridge` (v07)
- `ResearcherConfig` C — `pinneapple_tools.hpo_experiments.config` (v07)
- `ResearchLoopResult` C — `pinneapple_problemdesign.research_loop` (v02)
- `ResearchOnlyError` C — `pinneapple_neural._licencas` (v04)
- `ResearchOnlyWarning` C — `pinneapple_neural._licencas` (v04)
- `ResearchReport` C — `pinneapple_llm.research` (v07)
- `reservoir_mass_mg` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `ReservoirCatalog` C — `pinneapple_neural.architectures.reservoir_computing.registry` (v04)

- reset_nonuniform f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- residual_baseline f — pinneapple_physics.ensemble (v02)
- residual_check f — pinneapple_physics.physics_audit.monitor (v02)
- residual_error_bound f — pinneapple_physics.darcy_checks (v02)
- residual_from_stencil f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- residual_l2_bound f — pinneapple_physics.darcy_checks (v02)
- residual_metrics f — pinneapple_analysis.inverse_problems.calibration (v05)
- residual_plot f — pinneapple_tools.automation.adapters.openfoam (v07)
- residual_points f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- residual_scale f — pinneapple_app.backend.core.surrogate_report (v07)
- residual_sigma f — pinneapple_systems.operations.rom (v06)
- ResidualAnalysisResult C — pinneapple_analysis.inverse_problems.missing_term (v05)
- ResidualAnalyzer C — pinneapple_analysis.inverse_problems.missing_term (v05)
- ResidualBasedAL C — pinneapple_data.active_learning (v04)
- ResidualBlock3D C — pinneapple_neural.architectures.convolutions.conv3d (v04)
- ResidualComponent C — pinneapple_security.cps (v05)
- ResidualCorrectionForecaster C — pinneapple_systems.time_series.forecast_lab.hybrid (v06)
- ResidualCUSUM C — pinneapple_security.anomaly (v05)
- ResidualDetector C — pinneapple_systems.digital_twin.monitoring.anomaly (v06)
- ResidualErrorCalibrator C — pinneapple_analysis.uncertainty.residual_bound (v05)
- ResidualLoss C — pinneapple_analysis.uncertainty.residual_bound (v05)
- ResidualLSTMConfig C — pinneapple_systems.time_series.decomposition.fft_lstm (v06)
- ResidualLSTMForecaster C — pinneapple_systems.time_series.decomposition.fft_lstm (v06)
- Residuals C — pinneapple_systems.digital_twin.pipe_loop.diagnose (v06)
- ResidualSpec C — pinneapple_neural.architectures.neural_operators.pino (v04)
- residue_class f — pinneapple_tools.visualization.molecular (v07)
- resign_version f — pinneapple_security.integrate (v05)
- resolution_study f — pinneapple_physics.blackhole.convergence (v02)
- ResolutionConsistency C — pinneapple_analysis.validation.metamorphic (v05)
- resolve f — pinneapple_decision.constraints (v05)
- resolve f — pinneapple_flow.libcalls (v07)
- resolve f — pinneapple_physics.astro.facts (v02)
- resolve f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- resolve_function f — pinneapple_core.workflows.adapters (v01)

- `resolve_physics` f — `pinneapple_simulation.pipelines.freecad_openfoam.pipeline` (v03)
- `resolve_problem` f — `pinneapple_llm.catalog_match` (v07)
- `resolve_rea_command` f — `pinneapple_tools.rea_bridge.client` (v07)
- `resolve_sets` f — `pinneapple_data.cae.inp` (v04)
- `ResolveResult` C — `pinneapple_llm.catalog_match` (v07)
- `resonant_frequency_hz` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `Resource` C — `pinneapple_catalog.resources` (v01)
- `response` f — `pinneapple_design.thermal.board_cooling.simulate` (v06)
- `RestartResult` C — `pinneapple_neural.trainer.multistart` (v04)
- `resting_contact` f — `pinneapple_simulation.external_solvers.newton_physics.validation` (v03)
- `restrict` f — `pinneapple_physics.multiscale.filters` (v02)
- `Result` C — `pinneapple_presets.core` (v02)
- `Result` C — `pinneapple_science.core` (v02)
- `Result` C — `pinneapple_simulation.external_solvers.calculix.study` (v03)
- `results` f — `pinneapple_flow.nodes_ext.structures` (v07)
- `results_to_samples` f — `pinneapple_simulation.external_solvers.fluidx3d` (v03)
- `ResultTable` C — `pinneapple_simulation.electromagnetics.antenna.runner` (v03)
- `ResultTarget` C — `pinneapple_check.config` (v05)
- `retarget` f — `pinneapple_flow.templates` (v07)
- `RetrainConfig` C — `pinneapple_neural.physicsnemo_bridge.active_learning_bridge` (v04)
- `retreat_point` f — `pinneapple_simulation.rover.autolab.mission` (v03)
- `RetrievalIndex` C — `pinneapple_analysis.retrieval.index` (v05)
- `reverse_field` f — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `review_mesh_plan` f — `pinneapple_design.fitmesh.llm` (v06)
- `revisions` f — `pinneapple_hub.zoo._artifacts` (v07)
- `RevoluteJoint` C — `pinneapple_simulation.multibody.tree` (v03)
- `revolve` f — `pinneapple_design.geometry_library.kit` (v06)
- `revolve` f — `pinneapple_twin3d.primitives` (v06)
- `revolve_closed` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `reynolds` f — `pinneapple_physics.physics_audit.regime` (v02)
- `rga_matrix` f — `pinneapple_systems.time_series.eda.plots` (v06)
- `rgb_preview_stretch` f — `pinneapple_perception.change_detection` (v04)
- `rh_degasser` f — `pinneapple_twin3d.wear.presets` (v06)
- `rhs` f — `pinneapple_physics.blackhole.hydro_numba` (v02)
- `RIAFConfig` C — `pinneapple_physics.blackhole.hydro` (v02)
- `richardson` f — `pinneapple_physics.physics_audit.regime` (v02)
- `richardson_extrapolate` f — `pinneapple_analysis.verification.convergence` (v05)
- `RichardsResult` C — `pinneapple_simulation.geophysics.richards` (v03)
- `ricker` f — `pinneapple_simulation.electromagnetics.fdtd` (v03)
- `RidgeAR` C — `pinneapple_systems.time_series.adaptive` (v06)

- `rigid_body_derivative` f — `pinneapple_simulation.flight.rigid_body` (v03)
- `rigid_column_max_head` f — `pinneapple_presets.cases.water_hammer` (v02)
- `rigid_scene` f — `pinneapple_design.bricks.dynamics` (v06)
- `RigidBody` C — `pinneapple_simulation.particle_dynamics.rigid_body` (v03)
- `RigidBodyState` C — `pinneapple_simulation.particle_dynamics.rigid_body` (v03)
- `RigidBodySystem` C — `pinneapple_simulation.particle_dynamics.rigid_body` (v03)
- `RigidRock` C — `pinneapple_simulation.terramechanics.terrain` (v03)
- `RigidScene` C — `pinneapple_simulation.external_solvers.newton_physics.scene` (v03)
- `RigidTrajectory` C — `pinneapple_simulation.external_solvers.newton_physics.runner` (v03)
- `Ring` C — `pinneapple_simulation.electromagnetics.mesh` (v03)
- `ring_height` f — `pinneapple_tools.visualization.studio.planetary` (v07)
- `ring_mesh` f — `pinneapple_presets.cases.pressure_vessel` (v02)
- `ring_potential` f — `pinneapple_science.em.statics` (v02)
- `rise_transit_set` f — `pinneapple_physics.astro.observing` (v02)
- `rise_transit_set` f — `pinneapple_physics.astro.phenomena` (v02)
- `Risk` C — `pinneapple_problemdesign.schema` (v02)
- `RiskPlanner` C — `pinneapple_simulation.rover.autolab.planner` (v03)
- `ritter` f — `pinneapple_simulation.numerical_solvers.shallow_water_fv` (v03)
- `rk4` f — `pinneapple_analysis.data_assimilation.models` (v05)
- `rk4_step` f — `pinneapple_simulation.flight.rigid_body` (v03)
- `rk4_step` f — `pinneapple_systems.time_series.synthetic` (v06)
- `rmae` f — `pinneapple_analysis.reproduction` (v05)
- `rms` f — `pinneapple_simulation.flight.simulate` (v03)
- `rmsd` f — `pinneapple_tools.visualization.molecular` (v07)
- `RMSE` C — `pinneapple_neural.trainer.metrics` (v04)
- `rmse` f — `pinneapple_analysis.reproduction` (v05)
- `rmse` f — `pinneapple_physics.metrics` (v02)
- `rmse` f — `pinneapple_physics.weather.evaluate` (v02)
- `rmse` f — `pinneapple_systems.time_series.metrics` (v06)
- `rmse` f — `pinneapple_systems.time_series.metrics_ext.point` (v06)
- `rmse_fn` f — `pinneapple_systems.time_series.metrics` (v06)
- `RNNOutput` C — `pinneapple_neural.architectures.recurrent.base` (v04)
- `ro_coefficients` f — `pinneapple_systems.time_series.enso.oscillators` (v06)
- `ro_eigen` f — `pinneapple_systems.time_series.enso.oscillators` (v06)
- `ro_rhs` f — `pinneapple_systems.time_series.enso.forecasters` (v06)
- `ro_tendency` f — `pinneapple_systems.time_series.enso.oscillators` (v06)
- `road` f — `pinneapple_design.geometry.vehicles3d` (v06)
- `RoadCar` C — `pinneapple_design.geometry.vehicles3d` (v06)
- `RobinBC` f — `pinneapple_physics.pde_environment.conditions` (v02)

- RobotRun C — pinneapple_simulation.robot_sim.sim (v03)
- RobotSim C — pinneapple_simulation.robot_sim.sim (v03)
- Robustness C — pinneapple_design.intent.pipeline (v06)
- robustness f — pinneapple_design.intent.pipeline (v06)
- RobustScaleStep C — pinneapple_neural.trainer.preprocess (v04)
- roc_auc f — pinneapple_security.cps (v05)
- Rock C — pinneapple_simulation.rover.terrain (v03)
- rocket f — pinneapple_twin3d.gallery (v06)
- rocket_nozzle_cfd f — pinneapple_physics.pde_environment.presets.engineering (v02)
- rocket_structural f — pinneapple_physics.pde_environment.presets.engineering (v02)
- rod f — pinneapple_simulation.flight.mass (v03)
- RodModel C — pinneapple_analysis.inverse_problems.models (v05)
- rolling_fit f — pinneapple_systems.sports.football.ball (v06)
- rolling_statistics f — pinneapple_systems.time_series.eda.plots (v06)
- rolling_stats f — pinneapple_systems.time_series.features.engineering (v06)
- rolling_travel_time f — pinneapple_systems.sports.football.ball (v06)
- RollingWindowSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- rollout f — pinneapple_analysis.validation.rollout (v05)
- rollout f — pinneapple_physics.blackhole.forecast (v02)
- rollout_dense_volume_operator f — pinneapple_neural.workflows.dense_volume_operator (v04)
- rollout_parametric_dense_volume_operator f — pinneapple_neural.workflows.parametric_dense_volume_operator (v04)
- rollout_relative_l2 f — pinneapple_physics.metrics (v02)
- RolloutReport C — pinneapple_analysis.validation.rollout (v05)
- rom_checks f — pinneapple_neural.architectures.rom.contracts (v04)
- rom_compare f — pinneapple_flow.wrappers (v07)
- rom_fit f — pinneapple_flow.nodes.methods (v07)
- rom_study f — pinneapple_neural.architectures.rom.study (v04)
- ROMBase C — pinneapple_neural.architectures.rom.base (v04)
- ROMCatalog C — pinneapple_neural.architectures.rom.registry (v04)
- ROMHybrid C — pinneapple_neural.architectures.rom.rom_hybrid (v04)
- ROMOutput C — pinneapple_neural.architectures.rom.base (v04)
- ROMStudy C — pinneapple_lab.experiments.rom (v07)
- ROMStudyResult C — pinneapple_neural.architectures.rom.study (v04)
- roof f — pinneapple_reason.cases (v05)
- roof_network f — pinneapple_reason.modes.thermal (v05)
- root_compressibility f — pinneapple_condense.benchmark (v02)
- ROParams C — pinneapple_systems.time_series.enso.oscillators (v06)
- rosenstein_lyapunov f — pinneapple_systems.time_series.predictability (v06)
- rossby_haurwitz_speed f — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- rossby_wave_test f — pinneapple_systems.time_series.enso.intermediate (v06)

- rot f — pinneapple_design.geometry.vehicle_cad (v06)
- rot f — pinneapple_design.geometry_library.kit (v06)
- rot_x f — pinneapple_physics.astro.frames (v02)
- rot_x f — pinneapple_simulation.rover.geometry (v03)
- rot_y f — pinneapple_physics.astro.frames (v02)
- rot_y f — pinneapple_simulation.rover.geometry (v03)
- rot_z f — pinneapple_physics.astro.frames (v02)
- rot_z f — pinneapple_simulation.rover.geometry (v03)
- rotary_coupling_torsion_default f — pinneapple_physics.pde_environment.presets.structural (v02)
- rotate f — pinneapple_design.geometry.vehicle_cad (v06)
- rotate_field f — pinneapple_analysis.validation.metamorphic (v05)
- rotating_bending_stress_cycle f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- RotatingBendingCycleResult C — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- rotation f — pinneapple_simulation.electromagnetics.media (v03)
- rotation f — pinneapple_twin3d.primitives (v06)
- rotations f — pinneapple_decision.debias (v05)
- rotor_design f — pinneapple_flow.nodes_ext.em_prius (v07)
- rotor_design_problem f — pinneapple_simulation.electromagnetics.machines.design (v03)
- rotor_evaluator f — pinneapple_simulation.electromagnetics.machines.design (v03)
- rotor_parameter_space f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- rotor_temperature_c f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- RotorDesign C — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- RotorTorqueProblem C — pinneapple_design.topology.magnetic (v06)
- roughness_reynolds f — pinneapple_physics.physics_audit.regime (v02)
- round_conductor_eddy_loss_low_freq f — pinneapple_simulation.electromagnetics.analytic (v03)
- round_section f — pinneapple_design.geometry_library.kit (v06)
- round_sig f — pinneapple_core.jsonable (v01)
- RoundBottomSlots C — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- rounded_box f — pinneapple_design.geometry_library.kit (v06)
- rounding_half_unit f — pinneapple_flow.nodes_ext.examples (v07)
- RoundReport C — pinneapple_analysis.validation.arena.loop (v05)
- Route C — pinneapple_simulation.numerical_solvers.internal_flow (v03)
- Route C — pinneapple_simulation.rover.autolab.planner (v03)
- route_a_sqrt_coupling f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- route_c_asymmetric f — pinneapple_physics.non_hermitian.ep_tuning (v02)
- route_cases f — pinneapple_analysis.uncertainty.applicability (v05)

- route_lines f — pinneapple_simulation.rover.autolab.render (v03)
- RouteDecision C — pinneapple_physics.physics_audit.routing (v02)
- routing_summary f — pinneapple_analysis.uncertainty.applicability (v05)
- rover_meshes f — pinneapple_simulation.rover.geometry (v03)
- rover_wheel f — pinneapple_design.geometry.vehicle_cad (v06)
- RoverContact C — pinneapple_hub.zoo._models (v07)
- RoverMission C — pinneapple_simulation.rover.autolab.mission (v03)
- RoverParams C — pinneapple_simulation.rover.geometry (v03)
- RoverSimulator C — pinneapple_simulation.rover.dynamics (v03)
- Row C — pinneapple_tools.visualization.trust_panels (v07)
- row_status f — pinneapple_analysis.benchmark.checks (v05)
- rows_to_xy f — pinneapple_design.thermal.heatsink.surrogate (v06)
- rtn_basis f — pinneapple_physics.astro.neural_propagator (v02)
- Rule C — pinneapple_check.rules (v05)
- Rule C — pinneapple_flow.rules (v07)
- rule f — pinneapple_flow.rules (v07)
- rule_decision f — pinneapple_flow.decisions (v07)
- rule_eval f — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- RuleBasedBackend C — pinneapple_decision.backends (v05)
- RuleBasedBaseline C — pinneapple_systems.operations.control (v06)
- RuleParser C — pinneapple_simulation.rover.autolab.mission (v03)
- RULEstimate C — pinneapple_systems.digital_twin.prognostics (v06)
- Run C — pinneapple_autolab.core (v07)
- Run C — pinneapple_design.intent.physics.loop_track (v06)
- run f — pinneapple_analysis.benchmark.api (v05)
- run f — pinneapple_check.engine (v05)
- run f — pinneapple_condense.exceptions (v02)
- run f — pinneapple_design.design_optimizer.methods (v06)
- run f — pinneapple_flow (v07)
- run f — pinneapple_flow.nodes.cfd_pipeline (v07)
- run f — pinneapple_flow.runtime (v07)
- run f — pinneapple_lab.runner (v07)
- run f — pinneapple_llm.qa.validators.base (v07)
- run f — pinneapple_science.registry (v02)
- run f — pinneapple_simulation.electromagnetics.machines.verification (v03)
- run f — pinneapple_simulation.electromagnetics.reflector.study_report (v03)
- run f — pinneapple_simulation.electromagnetics.verification (v03)
- run f — pinneapple_simulation.flight.verification (v03)
- run_adaptive_sweep f — pinneapple_neural.trainer.adaptive_sweep (v04)
- run_adversarial_review f — pinneapple_analysis.verification.architecture_critique (v05)
- run_agent f — pinneapple_autolab.agent (v07)

- run_agent_loop f — pinneapple_llm.agent_loop (v07)
- run_all f — pinneapple_physics.astro.validation (v02)
- run_all f — pinneapple_physics.experiment (v02)
- run_arena f — pinneapple_analysis.validation.arena.loop (v05)
- run_arena_experiment f — pinneapple_tools.benchmark_suite.runner.run_arena_yaml (v07)
- run_benchmark f — pinneapple_arena.external.benchmark (v04)
- run_benchmark f — pinneapple_arena.showcase (v04)
- run_benchmark f — pinneapple_hub.zoo._bench (v07)
- run_benchmark f — pinneapple_security.cps_bench (v05)
- run_benchmark f — pinneapple_systems.digital_twin.pipe_loop.benchmark (v06)
- run_benchmark f — pinneapple_systems.operations.anomaly (v06)
- run_benchmark f — pinneapple_tools (v07)
- run_benchmark f — pinneapple_tools.benchmark_suite.runner.run_benchmark (v07)
- run_case f — pinneapple_design.aero.case (v06)
- run_case f — pinneapple_reason (v05)
- run_case f — pinneapple_simulation.accelerated_foam.case (v03)
- run_case f — pinneapple_simulation.numerical_solvers.external_flow (v03)
- run_case f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- run_case3d f — pinneapple_design.aero.case3d (v06)
- run_ccx f — pinneapple_simulation.external_solvers.calculix.runner (v03)
- run_closed_loop f — pinneapple_systems.component_modeling.control (v06)
- run_closed_loop f — pinneapple_systems.operations.control (v06)
- run_command f — pinneapple_simulation.accelerated_foam.case (v03)
- run_compare f — pinneapple_arena.external.benchmark (v04)
- run_custom_solver f — pinneapple_tools.sandbox.custom_solver_runner (v07)
- run_data_pipeline f — pinneapple_app.backend.core.data_pipeline (v07)
- run_decision_mode f — pinneapple_arena.decision_mode (v04)
- run_design_optimization f — pinneapple_design.design_optimizer.airfoil_drag (v06)
- run_dir f — pinneapple_lab.seal (v07)
- run_ensembles f — pinneapple_physics.multiscale.comparison (v02)
- run_eval f — pinneapple_llm.workbench.evaluate (v07)
- run_example f — pinneapple_flow.examples (v07)
- run_example f — pinneapple_llm.qa.examples.pipe_wear_twin (v07)
- run_experiment f — pinneapple_flow.nodes.science (v07)
- run_fault_injection f — pinneapple_analysis.validation.fault_injection.matrix (v05)
- run_fdtd f — pinneapple_science.em.fdtd_study (v02)
- run_file f — pinneapple_hub.library (v07)
- run_fin f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- run_finetune f — pinneapple_arena.external.finetune (v04)
- run_fluent_case f — pinneapple_simulation.external_solvers.ansys.runner (v03)
- run_fluidx3d f — pinneapple_simulation.external_solvers.fluidx3d (v03)

- run_free_geometry f — pinneapple_design.fitmesh.llm (v06)
- run_full_pipeline f — pinneapple_tools.benchmark_suite.runner.run_pipeline (v07)
- run_geotaichi f — pinneapple_simulation.external_solvers.geotaichi (v03)
- run_in_freecad_gui f — pinneapple_tools.automation.adapters.freecad (v07)
- run_info f — pinneapple_simulation.rover.autolab.report (v03)
- run_intervention f — pinneapple_analysis.verification.causal_discrepancy (v05)
- run_inverse f — pinneapple_arena.trainer (v04)
- run_inverse_discovery f — pinneapple_analysis.inverse_problems.heat_diffusivity (v05)
- run_inverse_solver f — pinneapple_arena.pointwise.training (v04)
- run_job f — pinneapple_analysis.benchmark.runner (v05)
- run_job f — pinneapple_design.aero.dataset (v06)
- run_job f — pinneapple_tools.automation.scripts.freecad_gui_job (v07)
- run_live f — pinneapple_systems.digital_twin.product.runner (v06)
- run_manufactured_solution_check f — pinneapple_problemdesign.preset_authoring (v02)
- run_matlab_function f — pinneapple_simulation.external_solvers.matlab.runner (v03)
- run_matlab_script f — pinneapple_simulation.external_solvers.matlab.runner (v03)
- run_mission f — pinneapple_design.thermal.board_cooling.agent (v06)
- run_mission f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- run_model f — pinneapple_systems.time_series.enso.study (v06)
- run_mpc f — pinneapple_systems.component_modeling.mpc (v06)
- run_omc_script f — pinneapple_simulation.external_solvers.modelica.omc (v03)
- run_one f — pinneapple_tools.benchmark_suite.runner.run_models_from_cfg (v07)
- run_onnx f — pinneapple_hub.zoo.export (v07)
- run_openfoam f — pinneapple_simulation.numerical_solvers.openfoam_bridge (v03)
- run_openfoam_case f — pinneapple_simulation.external_solvers.openfoam.runner (v03)
- run_openradioss_case f — pinneapple_simulation.external_solvers.openradioss.runner (v03)
- run_openradioss_cases f — pinneapple_simulation.external_solvers.openradioss.runner (v03)
- run_overfitting f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- run_parallel_sweep f — pinneapple_neural.trainer.parallel (v04)
- run_pipeline f — pinneapple_simulation.pipelines.freecad_openfoam.pipeline (v03)
- run_pipeline f — pinneapple_systems.time_series.solar_cycle (v06)

- run_pipeline f — pinneapple_worldmodel.train (v04)
- run_plant f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- run_plate f — pinneapple_analysis.inverse_problems.heat_transfer (v05)
- run_preset f — pinneapple_flow.nodes.presets (v07)
- run_regime_arena f — pinneapple_arena.regimes (v04)
- run_sampling f — pinneapple_simulation.external_solvers.openfoam.sampling (v03)
- run_series_flow f — pinneapple_systems.operations.series (v06)
- run_snippet f — pinneapple_hub.library.sandbox (v07)
- run_statistics f — pinneapple_physics.blackhole.convergence (v02)
- run_study f — pinneapple_analysis.surrogate.study (v05)
- run_sweep f — pinneapple_design.thermal.fin_array_speed (v06)
- run_sweep f — pinneapple_tools.benchmark_suite.runner.run_sweep (v07)
- run_testbed f — pinneapple_simulation.terramechanics.testbed (v03)
- run_track_b f — pinneapple_tools.benchmark_suite.crossframework.darcy (v07)
- run_tree f — pinneapple_decision.api (v05)
- run_tree f — pinneapple_decision.tree (v05)
- run_trial f — pinneapple_arena.autoresearch (v04)
- run_turbodesigner f — pinneapple_simulation.external_solvers.turbodesigner.bridge (v03)
- run_twin f — pinneapple_flow.twin (v07)
- run_twin f — pinneapple_systems.digital_twin.recession (v06)
- run_uq f — pinneapple_arena.trainer (v04)
- run_validation f — pinneapple_veriphysics.validation_lab (v05)
- run_zoo f — pinneapple_systems.time_series.forecast_lab.study (v06)
- RunContext C — pinneapple_lab.context (v07)
- RunInfo C — pinneapple_tools.nvidia.runinfo (v07)
- RunLogger C — pinneapple_neural.trainer.audit (v04)
- runnable f — pinneapple_core.onboarding.catalog (v01)
- runnable_actions f — pinneapple_lab.improve (v07)
- Runner C — pinneapple_tools.automation.runner (v07)
- runner_channel f — pinneapple_twin3d.gallery (v06)
- runner_parser f — pinneapple_tools.benchmark_suite.crossframework.protocol (v07)
- running f — pinneapple_mcp.jobs (v07)
- runout f — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- RunRecord C — pinneapple_autolab.ledger (v07)
- RunRecord C — pinneapple_simulation.electromagnetics.antenna.runner (v03)
- RunRecord C — pinneapple_tools.automation.record (v07)
- RunResult C — pinneapple_lab.runner (v07)
- RunResult C — pinneapple_simulation.accelerated_foam.case (v03)
- RunResult C — pinneapple_systems.digital_twin.product.runner (v06)
- RunResult C — pinneapple_tools.hpo_experiments.utils.verify_runtime (v07)
- runs_for_node f — pinneapple_hub.library.flowruns (v07)

- `runtime_dir` f — `pinneapple_hub.zoo.web` (v07)
- `runway_surfaces` f — `pinneapple_simulation.flight.visuals` (v03)

S

- `safe_data_path` f — `pinneapple_flow.config` (v07)
- `safe_eval` f — `pinneapple_systems.process_components.explicit_equation_system` (v06)
- `safe_extract_zip` f — `pinneapple_security.archives` (v05)
- `safe_load` f — `pinneapple_security.model_security` (v05)
- `safe_name` f — `pinneapple_core.service.uploads` (v01)
- `safe_parse_iso8601` f — `pinneapple_tools.hpo_experiments.ranking` (v07)
- `saltelli_design` f — `pinneapple_analysis.sensitivity.sobol` (v05)
- `saltelli_perturbation_sweep` f — `pinneapple_data.parameter_sampling` (v04)
- `same_dimension` f — `pinneapple_design.intent.spec` (v06)
- `same_gust` f — `pinneapple_flow.nodes_ext.flight` (v07)
- `same_origin_ok` f — `pinneapple_security.web` (v05)
- `sample` f — `pinneapple_design.fitmesh.suggest` (v06)
- `sample_annulus_tag_batch` f — `pinneapple_design.geometry.builders.analytic_domain_batch_builder` (v06)
- `sample_axisymmetric_wall_tag_batch` f — `pinneapple_design.geometry.builders.analytic_domain_batch_builder` (v06)
- `sample_barycentric_uv` f — `pinneapple_design.geometry.sample.barycentric` (v06)
- `sample_boundary_points` f — `pinneapple_design.geometry.sample.marching_squares` (v06)
- `sample_boundary_points_sdf2d` f — `pinneapple_design.geometry.io.sdf_meshing` (v06)
- `sample_box_particles` f — `pinneapple_design.geometry.sample.particles` (v06)
- `sample_box_tag_batch` f — `pinneapple_design.geometry.builders.analytic_domain_batch_builder` (v06)
- `sample_box_with_curve_tag_batch` f — `pinneapple_design.geometry.builders.analytic_domain_batch_builder` (v06)
- `sample_camera` f — `pinneapple_worldmodel.robot_perception` (v04)
- `sample_checks` f — `pinneapple_physics.darcy_checks` (v02)
- `sample_checks` f — `pinneapple_physics.maxwell_checks` (v02)
- `sample_circle_particles` f — `pinneapple_design.geometry.sample.particles` (v06)
- `sample_condition_points` f — `pinneapple_physics.pde_environment.condition_sampling` (v02)
- `sample_cylinder_tag_batch` f — `pinneapple_design.geometry.builders.analytic_domain_batch_builder` (v06)
- `sample_designs` f — `pinneapple_design.geometry.gen.car2d` (v06)
- `sample_designs` f — `pinneapple_tools.nvidia.heatsink` (v07)
- `sample_domain` f — `pinneapple_design` (v06)
- `sample_domain_1d` f — `pinneapple_quantum.data.quantum_dataset` (v04)

- sample_domain_2d f — pinneapple_quantum.data.quantum_dataset (v04)
- sample_entropy f — pinneapple_systems.time_series.predictability (v06)
- sample_faces f — pinneapple_app.backend.core.physics_adapter (v07)
- sample_grf_2d_batch f — pinneapple_data.grf_sampler (v04)
- sample_grf_batch f — pinneapple_data.grf_sampler (v04)
- sample_grf_paths f — pinneapple_data.grf_sampler (v04)
- sample_initial f — pinneapple_app.backend.core.physics_adapter (v07)
- sample_interior f — pinneapple_app.backend.core.physics_adapter (v07)
- sample_latent f — pinneapple_physics.pinn_solver.stochastic (v02)
- sample_latin_hypercube_box f — pinneapple_design.geometry.sample.grids (v06)
- sample_lines f — pinneapple_simulation.numerical_solvers.free_surface (v03)
- sample_parameters f — pinneapple_data.parameter_sampling (v04)
- sample_physics_points f — pinneapple_design.geometry.mesh.mesher_3d (v06)
- sample_point_cloud f — pinneapple_data.geometry_features (v04)
- sample_pointcloud_from_mesh f — pinneapple_design.geometry.ops.pointcloud (v06)
- sample_points_on_triangles f — pinneapple_design.geometry.sample.barycentric (v06)
- sample_population f — pinneapple_physics.astro.neural_propagator (v02)
- sample_profile f — pinneapple_twin3d.wear.geometry (v06)
- sample_space f — pinneapple_analysis.uncertainty.distributions (v05)
- sample_surface_points f — pinneapple_design.geometry.sample.points (v06)
- sample_surface_points_weighted f — pinneapple_design.geometry.sample.points (v06)
- sample_uniform_box f — pinneapple_design.geometry.sample.grids (v06)
- sample_variation f — pinneapple_worldmodel.inspection_data (v04)
- sample_verdict f — pinneapple_physics.darcy_checks (v02)
- sample_verdict f — pinneapple_physics.maxwell_checks (v02)
- sample_voxelgrid f — pinneapple_design.geometry.ops.voxelize (v06)
- SampleLine C — pinneapple_simulation.numerical_solvers.free_surface (v03)
- Sampler C — pinneapple_core.data (v01)
- SamplingSpec C — pinneapple_data.upd_types (v04)
- Sand C — pinneapple_simulation.particle_dynamics.geo_mpm (v03)
- SandboxExecutor C — pinneapple_llm.qa.sandbox (v07)
- SandboxLimits C — pinneapple_llm.qa.sandbox (v07)
- SandboxResult C — pinneapple_llm.qa.sandbox (v07)
- sanitize f — pinneapple_tools.reporting.paperkit (v07)
- sanitize_log f — pinneapple_security.privacy (v05)
- sarif f — pinneapple_check.report (v05)
- Satellite C — pinneapple_physics.astro.satellites (v02)
- satellite f — pinneapple_twin3d.gallery (v06)

- satellite_j2_perturbation f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- saturation_pressure f — pinneapple_reason.props (v05)
- save f — pinneapple_flow.learn (v07)
- save f — pinneapple_lab.manifest (v07)
- save_cases f — pinneapple_analysis.validation.arena.cases (v05)
- save_checkpoint f — pinneapple_neural.trainer.checkpoint (v04)
- save_darcy_npz f — pinneapple_physics.darcy_checks (v02)
- save_forecaster f — pinneapple_physics.blackhole.forecast (v02)
- save_hdf5 f — pinneapple_data.serialization (v04)
- save_hdf5 f — pinneapple_data.serialization (v04)
- save_manifest f — pinneapple_data.serialization (v04)
- save_mat f — pinneapple_simulation.external_solvers.matlab.mat_io (v03)
- save_maxwell_npz f — pinneapple_physics.maxwell_checks (v02)
- save_mesh_cache f — pinneapple_data.adapters.splash_archive_adapter (v04)
- save_meshio f — pinneapple_design.geometry.io.meshio_bridge (v06)
- save_model f — pinneapple_systems.operations.contracts (v06)
- save_object f — pinneapple_flow.nodes_ext.examples (v07)
- save_point_set f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- save_pt f — pinneapple_data.serialization (v04)
- save_review f — pinneapple_lab.curation (v07)
- save_seal f — pinneapple_lab.seal (v07)
- save_solution f — pinneapple_flow.nodes._common (v07)
- save_zarr f — pinneapple_data.serialization (v04)
- savefig f — pinneapple_tools.reporting.paperkit (v07)
- savgol_derivatives f — pinneapple_systems.sports.football.kinematics (v06)
- sbom f — pinneapple_security.attestation (v05)
- scaffold f — pinneapple_lab.manifest (v07)
- scalar f — pinneapple_check.fields (v05)
- scalar f — pinneapple_data.simulation_metadata.foam_dict (v04)
- scalar_coords f — pinneapple_condense.flow (v02)
- scalar_of f — pinneapple_tools.visualization.studio.scene (v07)
- ScalarIndex C — pinneapple_condense.flow (v02)
- ScalarSpec C — pinneapple_simulation.pipelines.freecad_openfoam.case (v03)
- Scale C — pinneapple_core.transforms (v01)
- scale_compatibility f — pinneapple_physics.multiscale.scales (v02)
- scale_conditioning_study f — pinneapple_physics.multiscale.turbulence (v02)
- scale_to_context f — pinneapple_design.fitmesh.suggest (v06)
- ScaleAwareRouter C — pinneapple_physics.multiscale.scales (v02)
- ScaleCheck C — pinneapple_physics.multiscale.scales (v02)
- ScaleConditionedFNO2d C — pinneapple_physics.multiscale.learned (v02)
- scaled f — pinneapple_simulation.electromagnetics.reflector.compact_range (v03)
- scaled_field_head f — pinneapple_analysis.verification.nozzle_crosscheck (v05)

- ScaledCurve C — pinneapple_simulation.electromagnetics.machine (v03)
- ScaleProfile C — pinneapple_physics.multiscale.scales (v02)
- ScaleQuery C — pinneapple_physics.multiscale.scales (v02)
- ScaleRoute C — pinneapple_physics.multiscale.scales (v02)
- ScaleSpec C — pinneapple_physics.pde_environment.scales (v02)
- scaling f — pinneapple_condense.figures (v02)
- scaling_point f — pinneapple_condense.benchmark (v02)
- scaling_study f — pinneapple_analysis.cost.profiler (v05)
- ScalingFit C — pinneapple_analysis.cost.profiler (v05)
- ScalingModel C — pinneapple_design.qualitative.models (v06)
- ScalingStudy C — pinneapple_analysis.cost.profiler (v05)
- scan f — pinneapple_check.models (v05)
- scan f — pinneapple_flow.autocalls (v07)
- scan f — pinneapple_flow.examples (v07)
- scan f — pinneapple_lab.manifest (v07)
- scan_checkpoint f — pinneapple_security.model_security (v05)
- scan_path f — pinneapple_security.secrets_scan (v05)
- scan_pickle_bytes f — pinneapple_security.model_security (v05)
- scan_text f — pinneapple_security.secrets_scan (v05)
- ScanReport C — pinneapple_security.model_security (v05)
- ScanReport C — pinneapple_twin3d.scan (v06)
- scatter_add f — pinneapple_neural.architectures.graphnn.utils (v04)
- scatter_add_flat f — pinneapple_neural.architectures.graphnn.utils (v04)
- Scenario C — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- Scenario C — pinneapple_systems.operations.plant (v06)
- ScenarioGenerator C — pinneapple_worldmodel.scenario_generator (v04)
- ScenarioSpec C — pinneapple_worldmodel.scenario_generator (v04)
- Scene C — pinneapple_tools.visualization.studio.scene (v07)
- Scene C — pinneapple_twin3d.scene (v06)
- scene f — pinneapple_flow.nodes_ext.flight (v07)
- scene f — pinneapple_physics.astro.journey (v02)
- scene3d_payload f — pinneapple_tools.visualization.dashboard.html (v07)
- scene_from_case f — pinneapple_twin3d.openfoam (v06)
- scene_from_mjcf f — pinneapple_twin3d.scan (v06)
- scene_from_wear f — pinneapple_twin3d.wear.build (v06)
- scene_html f — pinneapple_tools.visualization.integrations.plotly_export (v07)
- scene_summary f — pinneapple_simulation.flight.visuals (v03)
- scene_to_multiblock f — pinneapple_tools.visualization.pyvista_bridge (v07)
- scenery f — pinneapple_design.bricks.visuals (v06)
- schema_templates f — pinneapple_pdb.templates (v04)
- SchemaAdapter C — pinneapple_physics.pinn_solver.schemas.schema_adapter (v02)
- SchemaAdapterOptions C — pinneapple_physics.pinn_solver.schemas.schema_adapter (v02)
- schiller_naumann_cd f — pinneapple_physics.closed_form.particle_drag (v02)

- schwarzschild_deflection_angle_weak_field f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- schwarzschild_light_bending_u_exact f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- schwarzschild_light_bending_weak_field f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- SConv C — pinneapple_physics.weather.model (v02)
- scope_from_batch f — pinneapple_veriphysics.scope (v05)
- scope_from_verdicts f — pinneapple_veriphysics.scope (v05)
- ScopeOfUse C — pinneapple_veriphysics.scope (v05)
- score f — pinneapple_analysis.validation.fault_injection.checks (v05)
- score f — pinneapple_flow.decisions (v07)
- score f — pinneapple_flow.palette (v07)
- score f — pinneapple_flow.planner.extensions (v07)
- score f — pinneapple_hub.library (v07)
- score f — pinneapple_systems.sports.football.metrics (v06)
- score_against_reference f — pinneapple_analysis.verification.physics_confidence_score (v05)
- score_events f — pinneapple_systems.operations.anomaly (v06)
- score_forecasts f — pinneapple_physics.weather.evaluate (v02)
- score_matrix f — pinneapple_analysis.validation.arena.blue (v05)
- ScoringBackend C — pinneapple_decision.backends (v05)
- screen_cw_conjunction f — pinneapple_physics.astro.conjunction (v02)
- screen_nozzle f — pinneapple_analysis.verification.nozzle_crosscheck (v05)
- screenshot f — pinneapple_tools.visualization.studio.web (v07)
- ScriptedBackend C — pinneapple_llm.workbench.backend (v07)
- SDEDriftForecaster C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- SDF C — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_annulus f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_capsule f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_circle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_convex_polygon f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_ellipse f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_line_segment f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_rectangle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_rounded_rectangle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_sector f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf2d_to_pointcloud f — pinneapple_design.geometry.ops.pointcloud (v06)
- sdf2d_to_tri_mesh f — pinneapple_design.geometry.io.sdf_meshing (v06)

- sdf2d_triangle f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_box f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_capsule f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_cone f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_cylinder f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_rounded_box f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_sphere f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf3d_torus f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_difference f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_elongate_2d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_intersection f — pinneapple_design.geometry.gen.genetic_sdf (v06)
- sdf_intersection f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_mirror_x f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_mirror_y f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_onion f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_repeat_2d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_rotate_2d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_scale f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_smooth_difference f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_smooth_intersection f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_smooth_union f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_translate f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sdf_union f — pinneapple_design.geometry.gen.genetic_sdf (v06)
- sdf_union f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- SDFConfig C — pinneapple_design.geometry.builders.stl_domain_batch_builder (v06)
- SDFDomain2D C — pinneapple_design.geometry.gen.domains (v06)
- SDFGrid2D C — pinneapple_design.geometry.io.sdf_meshing (v06)
- SDFShape C — pinneapple_design.geometry.csg (v06)
- Seal C — pinneapple_lab.seal (v07)
- seal_for f — pinneapple_lab.seal (v07)
- seal_png f — pinneapple_lab.explorer (v07)
- seal_status f — pinneapple_flow.lab_status (v07)
- sealed_savefig f — pinneapple_lab.seal (v07)
- SealError C — pinneapple_lab.seal (v07)
- search f — pinneapple_core.onboarding.catalog (v01)
- search f — pinneapple_flow.nodes_ext.bricks (v07)
- search f — pinneapple_flow.palette (v07)
- search f — pinneapple_hub.library (v07)

- search f — pinneapple_physics.astro.facts (v02)
- search_architecture f — pinneapple_arena.nas (v04)
- search_gates_bo f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- search_gates_nested f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- search_literature f — pinneapple_llm.research (v07)
- search_space f — pinneapple_flow.learn (v07)
- SearchResult C — pinneapple_analysis.cost.equal_budget (v05)
- SearchResult C — pinneapple_analysis.retrieval.index (v05)
- SearchResult C — pinneapple_analysis.validation.arena.search (v05)
- SearchResult C — pinneapple_design.bricks.search (v06)
- SearchTrial C — pinneapple_analysis.cost.equal_budget (v05)
- SeasonalNaive C — pinneapple_systems.time_series.forecast_lab.baselines (v06)
- SeasonalNaiveForecaster C — pinneapple_systems.time_series.baselines.naive (v06)
- seasons f — pinneapple_physics.astro.phenomena (v02)
- second_derivative f — pinneapple_simulation.numerical_solvers.beam_bvp_fdm (v03)
- seconds_since f — pinneapple_data.upd_types (v04)
- SecretFinding C — pinneapple_security.secrets_scan (v05)
- Section C — pinneapple_analysis.validation.physics_learning_cases (v05)
- Section C — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- Section C — pinneapple_tools.reporting.datasheet (v07)
- section_aero f — pinneapple_design.design_optimizer.airfoil_drag (v06)
- section_average f — pinneapple_simulation.pipelines.freecad_openfoam.post (v03)
- section_class_ok f — pinneapple_presets.cases.footbridge (v02)
- section_of f — pinneapple_flow.palette (v07)
- section_profile f — pinneapple_design.qualitative.descriptors (v06)
- section_ring f — pinneapple_design.geometry_library.kit (v06)
- section_stress f — pinneapple_design.intent.physics.bracket (v06)
- SectionProps C — pinneapple_simulation.numerical_solvers.beam_bvp_fdm (v03)
- sections f — pinneapple_flow.palette (v07)
- SectorGeometry C — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- SectorMesh C — pinneapple_simulation.electromagnetics.machines.mesher (v03)
- secure_save f — pinneapple_security.integrate (v05)
- SecurityHeaders C — pinneapple_security.web (v05)
- seed_spread f — pinneapple_analysis.benchmark.checks (v05)
- seed_stats f — pinneapple_analysis.reproduction (v05)
- seed_stats f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- SeedStats C — pinneapple_analysis.reproduction (v05)
- Segment C — pinneapple_physics.astro.spk (v02)
- select f — pinneapple_lab.reports (v07)
- select f — pinneapple_simulation.numerical_solvers._bc (v03)

- `select_adapter` f — `pinneapple_neural.architectures.adapters.base` (v04)
- `select_candidates` f — `pinneapple_design.design_optimizer.verify` (v06)
- `select_models` f — `pinneapple_tools.benchmark_suite.pipeline.model_selector` (v07)
- `select_params` f — `pinneapple_systems.sports.football.xg` (v06)
- `select_points` f — `pinneapple_analysis.validation.physics_learning` (v05)
- `select_rules` f — `pinneapple_check.rules` (v05)
- `select_slugs` f — `pinneapple_arena.external.benchmark` (v04)
- `select_solver_family` f — `pinneapple_analysis.verification.solver_orchestration` (v05)
- `select_template` f — `pinneapple_design.intent.parse` (v06)
- `self_check` f — `pinneapple_reason.library` (v05)
- `self_intersections` f — `pinneapple_design.geometry_library.meshops` (v06)
- `self_scaled_parameters` f — `pinneapple_neural.trainer.self_scaled_qn` (v04)
- `SelfAdaptiveWeights` C — `pinneapple_neural.trainer.weight_scheduler` (v04)
- `SelfScaledQuasiNewton` C — `pinneapple_neural.trainer.self_scaled_qn` (v04)
- `sensitivity` f — `pinneapple_design.bricks.report` (v06)
- `sensitivity` f — `pinneapple_flow.nodes.surrogates` (v07)
- `sensitivity_result` f — `pinneapple_analysis.study_contracts` (v05)
- `sensitivity_study` f — `pinneapple_analysis.sensitivity.study` (v05)
- `sensitivity_to_dict` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `SensitivityResult` C — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `Sensor` C — `pinneapple_simulation.numerical_solvers.shallow_water_fv` (v03)
- `Sensor` C — `pinneapple_systems.digital_twin.io.sensors` (v06)
- `Sensor` C — `pinneapple_twin3d.scene` (v06)
- `sensor_count` f — `pinneapple_flow.planner.heuristic` (v07)
- `sensor_layout` f — `pinneapple_flow.planner.heuristic` (v07)
- `SensorNoise` C — `pinneapple_systems.operations.plant` (v06)
- `SensorPlacement` C — `pinneapple_analysis.data_assimilation.sensor_placement` (v05)
- `SensorReading` C — `pinneapple_llm.qa.validators.base` (v07)
- `SensorRegistry` C — `pinneapple_systems.digital_twin.io.sensors` (v06)
- `Sensors` C — `pinneapple_simulation.rover.autolab.estimation` (v03)
- `SensorSpec` C — `pinneapple_systems.digital_twin.product.spec` (v06)
- `Sentinel2Frame` C — `pinneapple_perception.sentinel2` (v04)
- `Seq2SeqRNN` C — `pinneapple_neural.architectures.recurrent.seq2seq` (v04)
- `Sequential` C — `pinneapple_core.module` (v01)
- `Series` C — `pinneapple_systems.digital_twin.io.usgs` (v06)
- `Series` C — `pinneapple_tools.visualization.dashboard.spec` (v07)
- `SeriesData` C — `pinneapple_systems.time_series.forecast_lab.base` (v06)

- SeriesDecomp C — pinneapple_neural.architectures.transformers.autoformer (v04)
- serve f — pinneapple_autolab.dashboard (v07)
- serve f — pinneapple_flow.studio (v07)
- serve f — pinneapple_lab.server (v07)
- serve f — pinneapple_tools.visualization.studio (v07)
- serve f — pinneapple_tools.visualization.studio.web (v07)
- serve f — pinneapple_twin3d.serve (v06)
- serve_in_thread f — pinneapple_tools.model_services.server (v07)
- serve_mlx_command f — pinneapple_llm.workbench.finetune (v07)
- serve_spec f — pinneapple_presets.surrogate (v02)
- ServiceError C — pinneapple_tools.model_services.client (v07)
- Session C — pinneapple_llm.workbench.runtime (v07)
- SessionSettings C — pinneapple_llm.workbench.runtime (v07)
- set_backend f — pinneapple_tools.compute_backends.backend (v07)
- set_config f — pinneapple_mcp.config (v07)
- set_data_dir f — pinneapple_analysis.benchmark.builtin.airfoil_impl (v05)
- set_data_dir f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- set_deterministic f — pinneapple_neural.trainer.audit (v04)
- set_entry f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- set_lr f — pinneapple_analysis.cost.equal_budget (v05)
- set_parameters f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- set_seed f — pinneapple_neural.trainer.audit (v04)
- set_shell_thickness f — pinneapple_simulation.external_solvers.openradioss.deck (v03)
- set_solver_entries f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- Settings C — pinneapple_flow.config (v07)
- setup_mpl f — pinneapple_tools.reporting.paperkit (v07)
- setup_threads f — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- severity_of f — pinneapple_systems.operations.anomaly (v06)
- sgp4_decay f — pinneapple_physics.astro.decay (v02)
- sha256 f — pinneapple_data.lineage.graph (v04)
- sha256 f — pinneapple_design.geometry_library.store (v06)
- sha256_bytes f — pinneapple_security.integrity (v05)
- sha256_bytes f — pinneapple_veriphysics.ledger (v05)
- sha256_file f — pinneapple_core.workflows.engine (v01)
- sha256_file f — pinneapple_data.contracts.models (v04)
- sha256_file f — pinneapple_hub.zoo._artifacts (v07)
- sha256_file f — pinneapple_security.integrity (v05)
- sha256_file f — pinneapple_veriphysics.ledger (v05)
- sha256_json f — pinneapple_veriphysics.ledger (v05)
- shade_terrain f — pinneapple_tools.visualization.studio.shadows (v07)
- shadow_diameter_rad f — pinneapple_physics.astro.lensing (v02)
- shakura_sunyaev_accretion_disk f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- shakura_sunyaev_flux_exact f — pinneapple_physics.pde_environment.presets.astrophysics (v02)

- shakura_sunyaev_teff_exact f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- shallow_water f — pinneapple_physics.physics_audit.assumptions (v02)
- shallow_water_pinn_study f — pinneapple_systems.time_series.enso.pipeline (v06)
- ShallowWater2D C — pinneapple_simulation.numerical_solvers.shallow_water_fv (v03)
- ShallowWaterOcean C — pinneapple_systems.time_series.enso.intermediate (v06)
- shannon_entropy f — pinneapple_security.secrets_scan (v05)
- Shape C — pinneapple_simulation.external_solvers.newton_physics.scene (v03)
- shape_am_two_tone f — pinneapple_systems.excitation (v06)
- shape_chirp f — pinneapple_systems.excitation (v06)
- shape_descriptor f — pinneapple_design.geometry.retrieval (v06)
- shape_from_handles f — pinneapple_design.aero.geometry (v06)
- shape_gaussian_bumps f — pinneapple_systems.excitation (v06)
- shape_piecewise_constant f — pinneapple_systems.excitation (v06)
- shape_prbs f — pinneapple_systems.excitation (v06)
- shape_ramp f — pinneapple_systems.excitation (v06)
- shape_sawtooth f — pinneapple_systems.excitation (v06)
- shape_sinusoid f — pinneapple_systems.excitation (v06)
- shape_smoothed_random_walk f — pinneapple_systems.excitation (v06)
- shape_step f — pinneapple_systems.excitation (v06)
- ShapeParametrization C — pinneapple_design.design_optimizer.adjoint (v06)
- ShapleyAttributionResult C — pinneapple_analysis.inverse_problems.counterfactual_attribution (v05)
- ShardAwareConfig C — pinneapple_data.zarr_shard_iterable (v04)
- ShardAwareZarrUPDIterable C — pinneapple_data.zarr_shard_iterable (v04)
- ShardSpec C — pinneapple_data.zarr_shards (v04)
- ShardSpec C — pinneapple_pdb.shard (v04)
- shear_layer_velocity_fns f — pinneapple_simulation.numerical_solvers.jax_cfd_solver (v03)
- ShearWind C — pinneapple_simulation.flight.turbulence (v03)
- sheet f — pinneapple_simulation.pipelines.freecad_openfoam.freecad_scripts.fc_job (v03)
- shell_solid f — pinneapple_design.geometry.ops.occ_features (v06)
- ShellTubeCooler C — pinneapple_presets.cases.shell_tube (v02)
- sheltered_faces f — pinneapple_design.qualitative.models (v06)
- shields_endpoint f — pinneapple_analysis.validation.report_artifact (v05)
- shots_arrays f — pinneapple_systems.sports.football.xg (v06)
- showcase f — pinneapple_autolab.lab (v07)
- showcase_score f — pinneapple_lab.html (v07)
- shrink_png f — pinneapple_autolab.report (v07)
- shuffle_surrogate f — pinneapple_systems.time_series.predictability (v06)

- ShuffledKFoldSplitter C — pinneapple_systems.time_series.validation.splitters (v06)
- si_unit_for f — pinneapple_data.physical_units (v04)
- sidecar_path f — pinneapple_data.contracts.tables (v04)
- sigma_is_default f — pinneapple_neural.architectures.reservoir_computing.rbf (v04)
- sign_bytes f — pinneapple_security.signing (v05)
- sign_file f — pinneapple_security.signing (v05)
- sign_json f — pinneapple_security.signing (v05)
- sign_model_card f — pinneapple_security.integrate (v05)
- sign_record f — pinneapple_security.integrate (v05)
- sign_report f — pinneapple_analysis.validation.report_artifact (v05)
- sign_test_p f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- sign_violations f — pinneapple_tools.nvidia.physicsnemo (v07)
- SignalProfile C — pinneapple_systems.digital_twin.conditioning (v06)
- SignalReconstructor C — pinneapple_systems.digital_twin.signal_reconstruction (v06)
- signed_distance_inside f — pinneapple_design.bricks.statics (v06)
- signed_volume f — pinneapple_design.geometry.vehicle_cad (v06)
- signed_volume_sign f — pinneapple_data.cae.geometry (v04)
- SigningKey C — pinneapple_security.signing (v05)
- Similarity C — pinneapple_analysis.validation.metamorphic (v05)
- SimOutcome C — pinneapple_autolab.simulator (v07)
- SimpleExpSmoothing C — pinneapple_systems.time_series.adaptive (v06)
- SimplePhysicalSample C — pinneapple_data.synth.pde (v04)
- simplify f — pinneapple_simulation.rover.autolab.planner (v03)
- simplify_mesh f — pinneapple_design.geometry.ops.simplify (v06)
- simulate f — pinneapple_design.thermal.board_cooling.simulate (v06)
- simulate f — pinneapple_flow.nodes.vehicle (v07)
- simulate f — pinneapple_flow.nodes_ext.astro (v07)
- simulate f — pinneapple_simulation (v03)
- simulate f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- simulate f — pinneapple_simulation.electromagnetics.periodic.tcda (v03)
- simulate f — pinneapple_simulation.external_solvers.newton_physics.runner (v03)
- simulate f — pinneapple_simulation.flight.simulate (v03)
- simulate f — pinneapple_simulation.numerical_solvers.cylinder_wake (v03)
- simulate f — pinneapple_simulation.numerical_solvers.thermal_channel_datasets (v03)
- simulate f — pinneapple_systems.operations.plant (v06)
- simulate f — pinneapple_systems.sports.football.ball (v06)
- simulate f — pinneapple_twin3d.wear.optimize (v06)
- simulate_car f — pinneapple_lab.experiments.car (v07)
- simulate_delayed f — pinneapple_systems.time_series.enso.oscillators (v06)

- simulate_fmu f — pinneapple_simulation.external_solvers.modelica.fmi_runner (v03)
- simulate_many f — pinneapple_design.thermal.board_cooling.simulate (v06)
- simulate_mo f — pinneapple_simulation.external_solvers.modelica.omc (v03)
- simulate_openmodelica f — pinneapple_simulation.external_solvers.modelica.om_runner (v03)
- simulate_recharge f — pinneapple_systems.time_series.enso.oscillators (v06)
- simulate_release_timeseries f — pinneapple_design.surrogate_inverse.transdermal_patch (v06)
- simulate_release_timeseries f — pinneapple_physics.closed_form.membrane_diffusion (v02)
- simulate_run f — pinneapple_physics.blackhole.runs (v02)
- simulate_sensors f — pinneapple_flow.twin (v07)
- simulate_takeoff f — pinneapple_simulation.flight.takeoff (v03)
- simulate_three_tank f — pinneapple_security.cps_bench (v05)
- simulate_torsional_stickslip f — pinneapple_systems.process_components.torsional_stickslip (v06)
- SimulationBlowUp C — pinneapple_simulation.numerical_solvers.etdrk4 (v03)
- Simulator C — pinneapple_autolab.simulator (v07)
- SINDy C — pinneapple_neural.architectures.rom.sindy (v04)
- sindy f — pinneapple_lab.experiments.discovery (v07)
- SINDyForecaster C — pinneapple_systems.time_series.forecast_lab.physical (v06)
- SINDyIdentifier C — pinneapple_analysis.inverse_problems.missing_term (v05)
- SINDyResult C — pinneapple_analysis.inverse_problems.missing_term (v05)
- Sine C — pinneapple_neural.architectures.benchmarks.generic_pinn_models (v04)
- SineLayer C — pinneapple_neural.architectures.siren (v04)
- single_stud_torsion f — pinneapple_design.bricks.statics (v06)
- single_thread f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- single_wheel_check f — pinneapple_simulation.rover.checks (v03)
- SingleBatchDataset C — pinneapple_physics.pinn_solver.compiler.dataset (v02)
- sink_mass_and_air f — pinneapple_design.thermal.board_cooling.simulate (v06)
- sinusoid f — pinneapple_simulation.electromagnetics.fdtd (v03)
- SinusoidalEmbedding C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- SinusoidalPositionalEncoding C — pinneapple_neural.architectures.transformers.informer (v04)
- sir_epidemic f — pinneapple_physics.pde_environment.presets.multidisciplinary (v02)
- SIREN C — pinneapple_neural.architectures.siren (v04)
- site f — pinneapple_flow.nodes.vehicle (v07)
- size f — pinneapple_design.thermal.heatsink.sizer (v06)

- `size_gear` f — `pinneapple_simulation.flight.gear` (v03)
- `size_layers` f — `pinneapple_design.fitmesh.layers` (v06)
- `size_to_pass` f — `pinneapple_reason.modes._util` (v05)
- `SizingRequest` C — `pinneapple_design.thermal.heatsink.sizer` (v06)
- `skew` f — `pinneapple_simulation.multibody.tree` (v03)
- `skewed_field_3d` f — `pinneapple_simulation.electromagnetics.machines.robustness` (v03)
- `skewed_no_load` f — `pinneapple_simulation.electromagnetics.machines.robustness` (v03)
- `skewed_torque` f — `pinneapple_simulation.electromagnetics.machines.robustness` (v03)
- `skill_against` f — `pinneapple_systems.time_series.enso.pipeline` (v06)
- `skill_figure` f — `pinneapple_physics.weather.viz` (v02)
- `skill_horizon` f — `pinneapple_systems.time_series.forecast_lab.study` (v06)
- `skill_score` f — `pinneapple_analysis.uncertainty.resampling` (v05)
- `skill_score` f — `pinneapple_systems.time_series.metrics_ext.comparison` (v06)
- `skill_vs_lead` f — `pinneapple_analysis.verification.forecast_skill` (v05)
- `skillcorner_ball` f — `pinneapple_systems.sports.football.data` (v06)
- `skillcorner_matches` f — `pinneapple_systems.sports.football.data` (v06)
- `SkillResult` C — `pinneapple_analysis.verification.forecast_skill` (v05)
- `skin_depth` f — `pinneapple_physics.closed_form.eddy_coil` (v02)
- `skin_depth` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `skin_effect_slab` f — `pinneapple_physics.closed_form.electromagnetics` (v02)
- `skin_effect_wire` f — `pinneapple_simulation.electromagnetics.analytic` (v03)
- `skipped_heavy` f — `pinneapple_arena.external.result` (v04)
- `sky_brightness` f — `pinneapple_physics.astro.observing` (v02)
- `sky_state` f — `pinneapple_physics.astro.phenomena` (v02)
- `sky_temperature` f — `pinneapple_reason.props` (v05)
- `sky_view` f — `pinneapple_physics.astro.conditions` (v02)
- `slab_for_footprint` f — `pinneapple_tools.visualization.molecular` (v07)
- `slab_frames` f — `pinneapple_systems.time_series.enso.pipeline` (v06)
- `slab_heat_solve` f — `pinneapple_core.workflows.templates` (v01)
- `slab_scene` f — `pinneapple_systems.time_series.enso.render` (v06)
- `slab_transient` f — `pinneapple_physics.closed_form.heat` (v02)
- `slerp` f — `pinneapple_tools.visualization.studio.animation` (v07)
- `slew` f — `pinneapple_physics.astro.observing` (v02)
- `Slice` C — `pinneapple_tools.visualization.studio.scene` (v07)
- `slice_viewer` f — `pinneapple_science.em.research` (v02)
- `sliced_wasserstein` f — `pinneapple_analysis.uncertainty.posterior_metrics` (v05)

- sliding_friction f — pinneapple_simulation.external_solvers.newton_physics.validation (v03)
- slip_angle f — pinneapple_simulation.terramechanics.wong_reece (v03)
- slip_ratio f — pinneapple_simulation.terramechanics.wong_reece (v03)
- Slot C — pinneapple_design.intent.spec (v06)
- slugify f — pinneapple_tools.hpo_experiments.kb.store (v07)
- SLURMConfig C — pinneapple_neural.trainer.hpc (v04)
- SlurmExecutor C — pinneapple_core.workflows.executors (v01)
- Smagorinsky C — pinneapple_physics.multiscale.closures (v02)
- small_strain f — pinneapple_physics.physics_audit.assumptions (v02)
- SmallBody C — pinneapple_physics.astro.smallbodies (v02)
- smape f — pinneapple_systems.time_series.metrics (v06)
- smape f — pinneapple_systems.time_series.metrics_ext.point (v06)
- smape_fn f — pinneapple_systems.time_series.metrics (v06)
- smelting_reduction_reactor f — pinneapple_twin3d.wear.presets (v06)
- smoke f — pinneapple_flow.autocalls (v07)
- smoke_run f — pinneapple_tools.hpo_experiments.utils.verify_runtime (v07)
- smooth_block f — pinneapple_design.bricks.parts (v06)
- smooth_mesh f — pinneapple_design.geometry.ops.clean (v06)
- smoothing_figure f — pinneapple_systems.sports.football.plots (v06)
- sn_curve_cycles_to_failure f — pinneapple_systems.process_components.fatigue_analysis (v06)
- snap_to_box f — pinneapple_condense.meshless2d (v02)
- snappy_dict f — pinneapple_simulation.numerical_solvers.external_flow (v03)
- snapshot_values f — pinneapple_analysis.surrogate.data (v05)
- snapshots_from_arrays f — pinneapple_data.snapshots (v04)
- SnapshotSet C — pinneapple_data.snapshots (v04)
- snippet f — pinneapple_core.onboarding.catalog (v01)
- sobol_from_evaluations f — pinneapple_analysis.sensitivity.sobol (v05)
- sobol_from_runs f — pinneapple_analysis.sensitivity.sobol (v05)
- sobol_g f — pinneapple_analysis.sensitivity.testfunctions (v05)
- sobol_g_indices f — pinneapple_analysis.sensitivity.testfunctions (v05)
- sobol_g_space f — pinneapple_analysis.sensitivity.testfunctions (v05)
- sobol_indices f — pinneapple_analysis.sensitivity.sobol (v05)
- SobolIndices C — pinneapple_analysis.sensitivity.sobol (v05)
- SobolResult C — pinneapple_analysis.inverse_problems.sensitivity (v05)
- sod_exact f — pinneapple_simulation.numerical_solvers.compressible_fv (v03)
- sod_exact_solution f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- sod_shock_tube_astro f — pinneapple_physics.pde_environment.presets.astrophysics (v02)
- soft_sand f — pinneapple_simulation.rover.autolab.site (v03)
- SoftAdapt C — pinneapple_neural.trainer.loss_balancer (v04)

- `softmax_dict` f — `pinneapple_decision.debias` (v05)
- `soi_from_mslp` f — `pinneapple_systems.time_series.enso.data` (v06)
- `Soil` C — `pinneapple_simulation.terramechanics.soils` (v03)
- `soil` f — `pinneapple_flow.nodes.vehicle` (v07)
- `soil_theta` f — `pinneapple_simulation.rover.autolab.estimation` (v03)
- `SoilBelief` C — `pinneapple_simulation.rover.autolab.estimation` (v03)
- `SoilParams` C — `pinneapple_simulation.particle_dynamics.terramechanics` (v03)
- `SoilRegion` C — `pinneapple_simulation.rover.terrain` (v03)
- `SoilUnit` C — `pinneapple_simulation.rover.autolab.site` (v03)
- `solar_eclipses` f — `pinneapple_physics.astro.phenomena` (v02)
- `solar_panel` f — `pinneapple_design.geometry.vehicle_cad` (v06)
- `solid_pipe_mesh` f — `pinneapple_llm.qa.examples.pipe_wear_twin` (v07)
- `SolidFEM` C — `pinneapple_simulation.numerical_solvers.solid_fem` (v03)
- `SolidFEMExperiment` C — `pinneapple_lab.experiments.structures` (v07)
- `solidify` f — `pinneapple_design.geometry_library.meshops` (v06)
- `Solution` C — `pinneapple_physics.solving` (v02)
- `solution_dirs` f — `pinneapple_data.cae.geometry` (v04)
- `solution_to_vtu` f — `pinneapple_simulation.electromagnetics.machines.export` (v03)
- `solve` f — `pinneapple_condense.flow` (v02)
- `solve` f — `pinneapple_condense.fv` (v02)
- `solve` f — `pinneapple_condense.meshless` (v02)
- `solve` f — `pinneapple_design.aero.vlm` (v06)
- `solve` f — `pinneapple_design.bricks.fea` (v06)
- `solve` f — `pinneapple_flow.nodes_ext.structures` (v07)
- `solve` f — `pinneapple_mcp.tools` (v07)
- `solve` f — `pinneapple_physics.pcb.solver` (v02)
- `solve` f — `pinneapple_physics.solving` (v02)
- `solve` f — `pinneapple_quantum.pipeline.presets` (v04)
- `solve` f — `pinneapple_simulation.electromagnetics.machines.getdp` (v03)
- `solve` f — `pinneapple_simulation.external_solvers.calculix.study` (v03)
- `solve` f — `pinneapple_simulation.meshless.methods` (v03)
- `solve_adaptive` f — `pinneapple_condense.learned` (v02)
- `solve_and_package` f — `pinneapple_simulation.external_solvers.fenics.solver` (v03)
- `solve_axisymmetric_eddy_current` f — `pinneapple_simulation.numerical_solvers.eddy_current_fdm` (v03)
- `solve_base_field` f — `pinneapple_design.thermal.heatsink.field` (v06)
- `solve_beam` f — `pinneapple_systems.process_components.beam_statics` (v06)
- `solve_blocks` f — `pinneapple_condense.learned` (v02)
- `solve_burgers` f — `pinneapple_simulation.numerical_solvers.operator_datasets` (v03)
- `solve_darcy` f — `pinneapple_simulation.numerical_solvers.operator_datasets` (v03)

- `solve_diffusion_reaction` f — `pinneapple_tools.benchmark_suite.deeponet_ground_truth` (v07)
- `solve_electrostatic` f — `pinneapple_simulation.electromagnetics.electrostatics` (v03)
- `solve_euler_bernoulli_beam_1d` f — `pinneapple_simulation.numerical_solvers.beam_bvp_fdm` (v03)
- `solve_exact` f — `pinneapple_condense.learned` (v02)
- `solve_fd` f — `pinneapple_simulation.numerical_solvers.symbolic_fd` (v03)
- `solve_fin_cell` f — `pinneapple_design.thermal.fin_cell` (v06)
- `solve_forward` f — `pinneapple_simulation.numerical_solvers.gated_xtfc` (v03)
- `solve_homogenized` f — `pinneapple_condense.homogenize` (v02)
- `solve_ibm_external_flow` f — `pinneapple_simulation.numerical_solvers.immersed_boundary_fdm` (v03)
- `solve_ibm_internal_flow` f — `pinneapple_simulation.numerical_solvers.immersed_boundary_fdm` (v03)
- `solve_ibm_navier_stokes` f — `pinneapple_simulation.numerical_solvers.immersed_boundary_fdm` (v03)
- `solve_inverse` f — `pinneapple_analysis.inverse_problems.studio` (v05)
- `solve_inverse_problem` f — `pinneapple_analysis.verification.uncertainty_inverse` (v05)
- `solve_kepler` f — `pinneapple_physics.astro.orbits` (v02)
- `solve_magnetostatic` f — `pinneapple_simulation.electromagnetics.fem` (v03)
- `solve_mfl_nonlinear` f — `pinneapple_analysis.inspection.magnetic_flux_leakage` (v05)
- `solve_network` f — `pinneapple_simulation.numerical_solvers.pipe_network` (v03)
- `solve_nodal_darcy` f — `pinneapple_physics.darcy_checks` (v02)
- `solve_nonlinear_beam_static` f — `pinneapple_systems.process_components.beam_nonlinear_fem` (v06)
- `solve_nonlinear_beam_transient` f — `pinneapple_simulation.numerical_solvers.nonlinear_beam_fem` (v03)
- `solve_ns` f — `pinneapple_simulation.numerical_solvers.operator_datasets` (v03)
- `solve_ode_linear` f — `pinneapple_tools.benchmark_suite.deeponet_ground_truth` (v07)
- `solve_ode_nonlinear` f — `pinneapple_tools.benchmark_suite.deeponet_ground_truth` (v07)
- `solve_package` f — `pinneapple_condense.fv` (v02)
- `solve_path_from_pressure_ratio` f — `pinneapple_systems.process_components.polytropic_path` (v06)
- `solve_path_from_work` f — `pinneapple_systems.process_components.polytropic_path` (v06)
- `solve_pde` f — `pinneapple_physics` (v02)
- `solve_pde_kwargs_from_batch` f — `pinneapple_physics.pde_environment.presets.tag_geometry` (v02)
- `solve_pendulum` f — `pinneapple_tools.benchmark_suite.deeponet_ground_truth` (v07)

- `solve_plane_stress` f — `pinneapple_simulation.numerical_solvers.plane_stress_fem` (v03)
- `solve_poisson` f — `pinneapple_core.fem` (v01)
- `solve_potential` f — `pinneapple_science.numerics` (v02)
- `solve_preset` f — `pinneapple_flow.wrappers` (v07)
- `solve_richards` f — `pinneapple_simulation.geophysics.richards` (v03)
- `solve_static_elasticity_3d` f — `pinneapple_simulation.numerical_solvers.elasticity3d_fdm` (v03)
- `solve_stiff_reaction_network` f — `pinneapple_simulation.numerical_solvers.stiff_kinetics_ode` (v03)
- `solve_time_harmonic` f — `pinneapple_simulation.electromagnetics.eddy` (v03)
- `solve_with_deepxde` f — `pinneapple_physics.deepxde_bridge.adapter` (v02)
- `solve_xtfc` f — `pinneapple_simulation.numerical_solvers.xtfc_general` (v03)
- `solve_xtfc_ode` f — `pinneapple_simulation.numerical_solvers.xtfc_ivp` (v03)
- `solve_xtfc_pde` f — `pinneapple_simulation.numerical_solvers.xtfc_pde` (v03)
- `solve_xtfc_subdomains` f — `pinneapple_simulation.numerical_solvers.xtfc_subdomain` (v03)
- `solved_files` f — `pinneapple_simulation.electromagnetics.antenna.scene3d` (v03)
- `SolveInfo` C — `pinneapple_condense.fv` (v02)
- `solver_available` f — `pinneapple_core.onboarding.catalog` (v01)
- `solver_checks` f — `pinneapple_systems.time_series.enso.pipeline` (v06)
- `solver_entries` f — `pinneapple_simulation.accelerated_foam.benchmark` (v03)
- `solver_from` f — `pinneapple_simulation.electromagnetics.periodic.benchmark` (v03)
- `solver_log` f — `pinneapple_tools.automation.adapters.openfoam` (v07)
- `solver_log_summary` f — `pinneapple_autolab.domains.racecar` (v07)
- `solver_qualification` f — `pinneapple_autolab.qualification` (v07)
- `solver_requirement` f — `pinneapple_core.onboarding.catalog` (v01)
- `solver_result_view` f — `pinneapple_tools.visualization.studio.solver_view` (v07)
- `SolverBase` C — `pinneapple_simulation.numerical_solvers.base` (v03)
- `SolverBridgeConfig` C — `pinneapple_worldmodel.physics_solver_bridge` (v04)
- `SolverCatalog` C — `pinneapple_simulation.numerical_solvers.registry` (v03)
- `SolveResult` C — `pinneapple_simulation.accelerated_foam.solvers` (v03)
- `SolverFeatureStep` C — `pinneapple_neural.trainer.preprocess` (v04)
- `SolverModule` C — `pinneapple_core.module` (v01)
- `SolverOutput` C — `pinneapple_simulation.numerical_solvers.base` (v03)
- `SolverOutput` C — `pinneapple_worldmodel.physics_solver_bridge` (v04)

- `SolverOutput3D` C — `pinneapple_simulation.numerical_solvers.fdm3d` (v03)
- `SolverRecommendation` C — `pinneapple_analysis.verification.solver_orchestration` (v05)
- `SolverRegistry` C — `pinneapple_simulation.numerical_solvers.registry` (v03)
- `SolverSpec` C — `pinneapple_simulation.numerical_solvers.registry` (v03)
- `sort_ranked` f — `pinneapple_tools.hpo_experiments.ranking` (v07)
- `Source` C — `pinneapple_design.geometry_library.core` (v06)
- `Source` C — `pinneapple_veriphysics.evidence_package` (v05)
- `source_root` f — `pinneapple_lab.experiments.landing` (v07)
- `source_vector` f — `pinneapple_condense.fv` (v02)
- `sources` f — `pinneapple_physics.astro.facts` (v02)
- `space_debris_cw_relative_motion` f — `pinneapple_physics.pde_environment.presets.astrophysics` (v02)
- `space_from_contract` f — `pinneapple_analysis.study_contracts` (v05)
- `space_readme` f — `pinneapple_hub.zoo.hub_cards` (v07)
- `space_to_contract` f — `pinneapple_analysis.study_contracts` (v05)
- `space_url` f — `pinneapple_hub.zoo.hub_cards` (v07)
- `spacecraft_attitude_euler_rotation` f — `pinneapple_physics.pde_environment.presets.astrophysics` (v02)
- `SpaceTime` C — `pinneapple_pdb.builder` (v04)
- `SpalartAllmarasResiduals` C — `pinneapple_physics.pde_environment.turbulence_presets` (v02)
- `SparsityRegularizer` C — `pinneapple_analysis.inverse_problems.regularization` (v05)
- `spatial_coords` f — `pinneapple_app.backend.core.physics_adapter` (v07)
- `SpatiotemporalGNN` C — `pinneapple_neural.architectures.graphnn.spatiotemporal_gnn` (v04)
- `spec` f — `pinneapple_systems.time_series.enso.study` (v06)
- `spec_for` f — `pinneapple_flow.libcalls` (v07)
- `spec_of` f — `pinneapple_analysis.validation.schema` (v05)
- `spec_to_urdf` f — `pinneapple_simulation.robot_sim.spec` (v03)
- `SpecError` C — `pinneapple_design.design_optimizer.spec` (v06)
- `SpecError` C — `pinneapple_design.intent.spec` (v06)
- `SpecError` C — `pinneapple_llm.qa.spec` (v07)
- `SpecError` C — `pinneapple_systems.digital_twin.product.spec` (v06)
- `SpecialistConfig` C — `pinneapple_worldmodel.specialist_trainer` (v04)
- `specialists` f — `pinneapple_flow.nodes_ext.worldmodel` (v07)
- `SpecialistTrainer` C — `pinneapple_worldmodel.specialist_trainer` (v04)
- `species_diffusion_residual` f — `pinneapple_systems.component_modeling.physics_residuals` (v06)
- `specific_impulse` f — `pinneapple_physics.closed_form.rocket_nozzle` (v02)
- `spectral` f — `pinneapple_check._reuse` (v05)
- `spectral_class` f — `pinneapple_physics.astro.stars_ml` (v02)
- `spectral_comparison` f — `pinneapple_analysis.validation.spectral` (v05)
- `spectral_derivative` f — `pinneapple_neural.architectures.neural_operators.pino` (v04)

- spectral_derivative f — pinneapple_physics.multiscale.filters (v02)
- spectral_downsample f — pinneapple_simulation.numerical_solvers.operator_benchmarks (v03)
- spectral_drift f — pinneapple_analysis.validation.rollout (v05)
- spectral_error_shells f — pinneapple_analysis.validation.field_metrics (v05)
- spectral_fidelity f — pinneapple_analysis.validation.spectral (v05)
- spectral_peaks f — pinneapple_science.numerics (v02)
- spectral_relative_geomean f — pinneapple_analysis.uncertainty.posterior_metrics (v05)
- SpectralComparison C — pinneapple_analysis.validation.spectral (v05)
- SpectralConvND C — pinneapple_neural.architectures.neural_operators.uno (v04)
- SpectralFidelityReport C — pinneapple_analysis.validation.spectral (v05)
- SpectralNS3D C — pinneapple_simulation.numerical_solvers.ns3d_spectral (v03)
- SpectralSolver C — pinneapple_simulation.numerical_solvers.spectral (v03)
- spectrum_error f — pinneapple_physics.multiscale.closures (v02)
- sph_box f — pinneapple_flow.nodes_ext.solver_registry (v07)
- Sphere C — pinneapple_worldmodel.geometry (v04)
- sphere f — pinneapple_design.geometry.bodies (v06)
- sphere f — pinneapple_physics.blackhole.twin (v02)
- sphere f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- sphere f — pinneapple_simulation.pipelines.freecad_openfoam.studies (v03)
- sphere3d f — pinneapple_design.geometry.gen.sdf_shapes (v06)
- sphere_drag f — pinneapple_flow.nodes_ext.lbm (v07)
- sphere_pad f — pinneapple_physics.weather.model (v02)
- sphere_stokes f — pinneapple_flow.nodes_ext.lbm (v07)
- SphereShallowWater C — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- SphereUNet C — pinneapple_physics.weather.model (v02)
- spherical_capacitance f — pinneapple_simulation.electromagnetics.statics (v03)
- SphericalHarmonics C — pinneapple_simulation.geophysics.sphere_shallow_water (v03)
- SPHParticles C — pinneapple_simulation.particle_dynamics.particles (v03)
- SPHSolver C — pinneapple_simulation.numerical_solvers.sph (v03)
- SPHState C — pinneapple_simulation.numerical_solvers.sph (v03)
- SpinChainDataset C — pinneapple_quantum.data.quantum_dataset (v04)
- SPK C — pinneapple_physics.astro.spk (v02)
- SplashCase C — pinneapple_data.adapters.splash_archive_adapter (v04)
- SplashMesh C — pinneapple_data.adapters.splash_archive_adapter (v04)
- Split C — pinneapple_neural.architectures.rom.study (v04)

- `Split` C — `pinneapple_systems.time_series.validation.splitters` (v06)
- `split` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `split` f — `pinneapple_condense.flow` (v02)
- `split` f — `pinneapple_design.aero.train` (v06)
- `split` f — `pinneapple_flow.learn` (v07)
- `split_files` f — `pinneapple_hub.zoo.datasets` (v07)
- `split_header` f — `pinneapple_data.physical_units` (v04)
- `split_indices` f — `pinneapple_analysis.uncertainty.conformal_study` (v05)
- `split_indices` f — `pinneapple_data.splits` (v04)
- `split_indices` f — `pinneapple_design.thermal.heatsink.monotone` (v06)
- `split_mask` f — `pinneapple_llm.qa.references` (v07)
- `split_order` f — `pinneapple_hub.zoo.hub_cards` (v07)
- `split_output_ports` f — `pinneapple_systems.cosimulation.node` (v06)
- `split_physical_samples` f — `pinneapple_data.noether_dataset` (v04)
- `split_procs` f — `pinneapple_simulation.pipelines.freecad_openfoam.mesh` (v03)
- `split_sections` f — `pinneapple_tools.hpo_experiments.sources.arxiv_pdf` (v07)
- `split_snapshots` f — `pinneapple_neural.architectures.rom.study` (v04)
- `split_surface` f — `pinneapple_simulation.numerical_solvers.free_surface` (v03)
- `Splits` C — `pinneapple_analysis.surrogate.splits` (v05)
- `Splits` C — `pinneapple_data.contracts.models` (v04)
- `splits` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `SplitSpec` C — `pinneapple_data.splits` (v04)
- `spm_machine` f — `pinneapple_flow.nodes_ext.electromagnetics` (v07)
- `SPMMachine` C — `pinneapple_simulation.electromagnetics.machine` (v03)
- `SPMMotor` C — `pinneapple_presets.cases.spm_motor` (v02)
- `spp_wavenumber` f — `pinneapple_simulation.electromagnetics.waves` (v03)
- `spreading_resistance` f — `pinneapple_physics.closed_form.plate_fin_heatsink` (v02)
- `spring_barrier_hindcasts` f — `pinneapple_systems.time_series.enso.pipeline` (v06)
- `spur_gear` f — `pinneapple_design.geometry.gen.cadquery_advanced` (v06)
- `src` f — `pinneapple_analysis.uncertainty.tolerance` (v05)
- `ssa_decompose` f — `pinneapple_simulation.numerical_solvers.ssa` (v03)
- `SSASolver` C — `pinneapple_simulation.numerical_solvers.ssa` (v03)
- `sst_stft_1d` f — `pinneapple_simulation.numerical_solvers.sst` (v03)
- `sst_tendency` f — `pinneapple_systems.time_series.enso.intermediate` (v06)
- `SSTSolver` C — `pinneapple_simulation.numerical_solvers.sst` (v03)

- `stability_derivatives` f — `pinneapple_simulation.flight.trim` (v03)
- `stable_dt` f — `pinneapple_simulation.geophysics.sphere_shallow_water` (v03)
- `stable_time_step` f — `pinneapple_simulation.numerical_solvers.stable_fluids` (v03)
- `StableFluids2D` C — `pinneapple_simulation.numerical_solvers.stable_fluids` (v03)
- `StableFluidsState` C — `pinneapple_simulation.numerical_solvers.stable_fluids` (v03)
- `Stack` C — `pinneapple_condense.package` (v02)
- `stack` f — `pinneapple_design.bricks.assembly` (v06)
- `stacked` f — `pinneapple_simulation.electromagnetics.machines.steel` (v03)
- `stackup` f — `pinneapple_physics.closed_form.pcb_thermal` (v02)
- `Stage` C — `pinneapple_problemdesign.elicitation.stages` (v02)
- `stage_case` f — `pinneapple_simulation.external_solvers.openradioss.deck` (v03)
- `stage_case_for_scenario` f — `pinneapple_simulation.external_solvers.openfoam.case_builder` (v03)
- `stage_derivatives` f — `pinneapple_simulation.numerical_solvers.discrete_time_pinn` (v03)
- `stall_angle` f — `pinneapple_simulation.flight.encounter` (v03)
- `stamp_all` f — `pinneapple_lab.seal` (v07)
- `stamp_figure` f — `pinneapple_lab.seal` (v07)
- `stamp_file` f — `pinneapple_lab.seal` (v07)
- `stamp_image` f — `pinneapple_lab.seal` (v07)
- `stamp_video` f — `pinneapple_lab.seal` (v07)
- `stamped_jpeg` f — `pinneapple_lab.explorer` (v07)
- `standalone_snippet` f — `pinneapple_core.onboarding.catalog` (v01)
- `standard_features` f — `pinneapple_systems.sports.football.xg` (v06)
- `standard_volumetric_flow_to_mass_flow` f — `pinneapple_systems.process_components.real_gas_eos` (v06)
- `StandardConditions` C — `pinneapple_systems.process_components.real_gas_eos` (v06)
- `standardize` f — `pinneapple_data.unified` (v04)
- `standardize_dims` f — `pinneapple_pdb.validate` (v04)
- `StandardScaler` C — `pinneapple_data.transforms` (v04)
- `star_apparent` f — `pinneapple_physics.astro.astrometry` (v02)
- `star_field` f — `pinneapple_physics.blackhole.raytrace` (v02)
- `star_position` f — `pinneapple_physics.astro.journey` (v02)
- `star_properties` f — `pinneapple_physics.astro.stars` (v02)
- `star_splat` f — `pinneapple_physics.astro.lensing` (v02)
- `StarCatalog` C — `pinneapple_physics.astro.stars` (v02)
- `starccm_cgns_to_upd` f — `pinneapple_simulation.external_solvers.cfd_formats.starccm_reader` (v03)
- `starccm_table_to_upd` f — `pinneapple_simulation.external_solvers.cfd_formats.starccm_reader` (v03)
- `start_server` f — `pinneapple_llm.local_llm` (v07)
- `State` C — `pinneapple_condense.query` (v02)
- `state_from_elements` f — `pinneapple_physics.astro.orbits` (v02)

- state_from_Ph f — pinneapple_systems.process_components.real_gas_eos (v06)
- state_from_Ps f — pinneapple_systems.process_components.real_gas_eos (v06)
- state_from_PT f — pinneapple_systems.process_components.real_gas_eos (v06)
- state_pairs f — pinneapple_worldmodel.physical_world (v04)
- state_vector f — pinneapple_simulation.flight.rigid_body (v03)
- StateEncoder C — pinneapple_worldmodel.physical_world (v04)
- statement f — pinneapple_security.attestation (v05)
- Static C — pinneapple_simulation.external_solvers.calculix.study (v03)
- static_fields f — pinneapple_physics.weather.model (v02)
- static_sinkage_check f — pinneapple_simulation.rover.checks (v03)
- StaticNonlinearBeamResult C — pinneapple_systems.process_components.beam_nonlinear_fem (v06)
- StaticPolicy C — pinneapple_llm.qa.static (v07)
- StaticResult C — pinneapple_llm.qa.static (v07)
- StaticsResult C — pinneapple_design.bricks.statics (v06)
- stationarity_report f — pinneapple_systems.time_series.eda.plots (v06)
- statistics f — pinneapple_flow.nodes_ext.astro (v07)
- statsbomb_shots f — pinneapple_systems.sports.football.data (v06)
- statsmodels_nino12 f — pinneapple_systems.time_series.enso.data (v06)
- StatsmodelsARIMAForecaster C — pinneapple_systems.time_series.classical_wrappers (v06)
- Status C — pinneapple_llm.qa.report (v07)
- status f — pinneapple_analysis.reproduction (v05)
- status f — pinneapple_flow.examples (v07)
- status f — pinneapple_hub.zoo (v07)
- status f — pinneapple_hub.zoo._artifacts (v07)
- status f — pinneapple_hub.zoo.datasets (v07)
- status f — pinneapple_mcp.jobs (v07)
- status f — pinneapple_twin3d.wear.forecast (v06)
- steady_heat_conduction_3d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- steady_state f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- steady_state f — pinneapple_systems.operations.plant (v06)
- steady_state f — pinneapple_systems.process_components.heat_exchanger (v06)
- steady_state_release_rate_mg_day f — pinneapple_physics.closed_form.membrane_diffusion (v02)
- steady_values f — pinneapple_design.aero.dataset (v06)
- SteadyProfilePoint C — pinneapple_systems.process_components.pipe_network_1d (v06)
- SteamLineInsulation C — pinneapple_presets.cases.insulation (v02)
- steel_ladle f — pinneapple_twin3d.wear.presets (v06)
- SteelFootbridge C — pinneapple_presets.cases.footbridge (v02)
- SteelGrade C — pinneapple_simulation.electromagnetics.machines.steel (v03)
- stencil f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)

- stencil_weights f — pinneapple_simulation.meshless.stencils (v03)
- Step C — pinneapple_autolab.core (v07)
- Step C — pinneapple_core.workflows.dag (v01)
- Step C — pinneapple_reason.chain (v05)
- Step C — pinneapple_tools.automation.workflow (v07)
- Step C — pinneapple_tools.reporting.datasheet (v07)
- step_check f — pinneapple_design.geometry_library.exporters (v06)
- step_hash f — pinneapple_core.workflows.engine (v01)
- step_irk f — pinneapple_simulation.numerical_solvers.irk_gauss_legendre (v03)
- step_offsets f — pinneapple_simulation.electromagnetics.machines.robustness (v03)
- step_response f — pinneapple_systems.time_series.eda.plots (v06)
- step_skew_factor f — pinneapple_simulation.electromagnetics.machines.robustness (v03)
- step_to_mesh f — pinneapple_design.geometry.io.step (v06)
- StepCancelled C — pinneapple_core.workflows.adapters (v01)
- StepContext C — pinneapple_core.workflows.adapters (v01)
- StepContext C — pinneapple_tools.automation.adapters.base (v07)
- StepFailed C — pinneapple_core.workflows.adapters (v01)
- StepImportConfig C — pinneapple_design.geometry.io.step (v06)
- StepLog C — pinneapple_simulation.rover.autolab.mission (v03)
- StepLog C — pinneapple_tools.visualization.molecular (v07)
- StepOutcome C — pinneapple_llm.qa.report (v07)
- StepPlanner C — pinneapple_autolab.core (v07)
- StepRecord C — pinneapple_tools.automation.record (v07)
- StepState C — pinneapple_simulation.rover.dynamics (v03)
- StickFixed C — pinneapple_simulation.flight.control (v03)
- StickSlipResult C — pinneapple_systems.process_components.torsional_stickslip (v06)
- stiffness f — pinneapple_core.fem (v01)
- stills f — pinneapple_presets.viz3d (v02)
- StirredTank C — pinneapple_simulation.numerical_solvers.particles (v03)
- stl_bytes f — pinneapple_design.geometry_library.exporters (v06)
- stl_to_upd f — pinneapple_data.adapters.geom_adapter (v04)
- STLDomainBatchBuilder C — pinneapple_design.geometry.builders.stl_domain_batch_builder (v06)
- STLDomainBatchConfig C — pinneapple_design.geometry.builders.stl_domain_batch_builder (v06)
- STLMesh C — pinneapple_data.stl_import (v04)
- STLSolver C — pinneapple_simulation.numerical_solvers.stl (v03)
- STLTemplateSynthGenerator C — pinneapple_data.synth.geometry_cadquery (v04)

- `stochastic_oscillator` f — `pinneapple_systems.time_series.synthetic` (v06)
- `stoker` f — `pinneapple_simulation.numerical_solvers.shallow_water_fv` (v03)
- `stokes` f — `pinneapple_condense.meshless2d` (v02)
- `Stokes2D` C — `pinneapple_arena.problems` (v04)
- `stokes_sphere_field` f — `pinneapple_simulation.numerical_solvers.lbm_tunnel` (v03)
- `StokesDissipation2D` C — `pinneapple_design.topology.problems` (v06)
- `StopRecord` C — `pinneapple_twin3d.wear.calibrate` (v06)
- `stops` f — `pinneapple_tools.visualization.studio.compose` (v07)
- `Store` C — `pinneapple_llm.workbench.store` (v07)
- `stored` f — `pinneapple_hub.zoo.validation_reports` (v07)
- `strain` f — `pinneapple_physics.multiscale.closures` (v02)
- `strains_rz` f — `pinneapple_physics.solid.axisymmetric` (v02)
- `stream` f — `pinneapple_flow.nodes.twin_nodes` (v07)
- `StreamingConditioner` C — `pinneapple_systems.digital_twin.conditioning` (v06)
- `streamlines` f — `pinneapple_design.aero.vlm` (v06)
- `stress_field` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `stress_from_moment_pa` f — `pinneapple_physics.closed_form.cantilever_fatigue` (v02)
- `stress_tendency` f — `pinneapple_physics.multiscale.closures` (v02)
- `stresses_rz` f — `pinneapple_physics.solid.axisymmetric` (v02)
- `stribeck_friction_torque` f — `pinneapple_systems.process_components.torsional_stickslip` (v06)
- `strip_capacitor_2d` f — `pinneapple_science.em.statics` (v02)
- `strouhal` f — `pinneapple_physics.physics_audit.regime` (v02)
- `structural_checks` f — `pinneapple_analysis.validation.physics_learning` (v05)
- `StructuralObjective` C — `pinneapple_design.design_optimizer.objective` (v06)
- `Structure` C — `pinneapple_design.bricks.statics` (v06)
- `structure_functions` f — `pinneapple_analysis.validation.spectral` (v05)
- `StructureFunctions` C — `pinneapple_analysis.validation.spectral` (v05)
- `StructurePreservingNetwork` C — `pinneapple_neural.architectures.physics_aware.spn` (v04)
- `stud_cantilever_tension` f — `pinneapple_design.bricks.closed_form` (v06)
- `StudentTMisfit` C — `pinneapple_analysis.inverse_problems.noise_models` (v05)
- `studies_figure` f — `pinneapple_analysis.benchmark.report` (v05)
- `studies_markdown` f — `pinneapple_analysis.benchmark.report` (v05)
- `studio_scene` f — `pinneapple_design.bricks.mesh` (v06)
- `study` f — `pinneapple_flow.nodes.cfd_pipeline` (v07)
- `study_cost` f — `pinneapple_analysis.sensitivity.study` (v05)
- `study_figure` f — `pinneapple_simulation.electromagnetics.antenna.study` (v03)
- `study_metric_key` f — `pinneapple_analysis.benchmark.report` (v05)
- `study_result` f — `pinneapple_analysis.surrogate.contracts` (v05)

- study_result f — pinneapple_design.design_optimizer.report (v06)
- study_result f — pinneapple_neural.architectures.rom.contracts (v04)
- study_result f — pinneapple_simulation.meshless.contracts (v03)
- StudyModel C — pinneapple_analysis.study_models (v05)
- StudyResult C — pinneapple_analysis.surrogate.study (v05)
- StudyResult C — pinneapple_data.contracts.models (v04)
- style f — pinneapple_science.plots (v02)
- SU2Client C — pinneapple_data.datasets.external_sources (v04)
- SubDataset C — pinneapple_arena.external.schema (v04)
- Subdomain C — pinneapple_physics.pinn_solver.domino (v02)
- SubdomainPINN C — pinneapple_physics.pinn_solver.domino (v02)
- subject f — pinneapple_security.attestation (v05)
- submit f — pinneapple_mcp.jobs (v07)
- SubnetWrapper C — pinneapple_neural.architectures.pinns.xpinn (v04)
- subsample f — pinneapple_tools.visualization.studio.fieldview (v07)
- subset_tile f — pinneapple_pdb.shard (v04)
- subset_time f — pinneapple_pdb.shard (v04)
- sufficiency f — pinneapple_analysis.validation.physics_learning (v05)
- suggest_geometries f — pinneapple_design.fitmesh.suggest (v06)
- suggest_parameters f — pinneapple_design.fitmesh.suggest (v06)
- suggest_problem_spec f — pinneapple_physics.pde_environment.capabilities (v02)
- suggest_roles f — pinneapple_data.process_optimizer (v04)
- Suggestion C — pinneapple_neural.trainer.advisor (v04)
- summarise f — pinneapple_analysis.benchmark.checks (v05)
- summarise f — pinneapple_data.simulation_metadata.openfoam (v04)
- summarise f — pinneapple_systems.sports.football.ball_study (v06)
- summarise_runs f — pinneapple_physics.multiscale.comparison (v02)
- summarize f — pinneapple_design.aero.optimize (v06)
- summarize f — pinneapple_design.thermal.fin_array_speed (v06)
- summarize f — pinneapple_flow.libcalls (v07)
- summarize f — pinneapple_physics.astro.conditions (v02)
- summarize f — pinneapple_tools.benchmark_suite.crossframework.evaluate (v07)
- summarize f — pinneapple_veriphysics.validation_lab (v05)
- summarize3d f — pinneapple_design.aero.optimize3d (v06)
- summarize_al f — pinneapple_design.aero.optimize_airliner (v06)
- summarize_array f — pinneapple_arena.external.result (v04)
- summarize_js_evidence f — pinneapple_tools.rea_bridge.evidence (v07)
- summarize_piv f — pinneapple_perception.piv_tools (v04)
- summarize_samples f — pinneapple_analysis.uncertainty.margin (v05)
- summary f — pinneapple_check.report (v05)
- summary f — pinneapple_flow.decisions (v07)
- summary f — pinneapple_flow.palette (v07)
- summary f — pinneapple_physics.metrics (v02)
- summary_figure f — pinneapple_physics.multiscale.comparison (v02)

- summary_metrics f — pinneapple_analysis.benchmark.report (v05)
- summary_table f — pinneapple_design.geometry_library.core (v06)
- SumWind C — pinneapple_simulation.flight.turbulence (v03)
- sun_deflection_arcsec f — pinneapple_physics.astro.lensing (v02)
- sun_itrs f — pinneapple_physics.astro.satellites (v02)
- superellipse f — pinneapple_design.geometry_library.kit (v06)
- superellipse_radius f — pinneapple_simulation.electromagnetics.reflector.geometry (v03)
- Superposition C — pinneapple_analysis.validation.metamorphic (v05)
- Superposition C — pinneapple_condense.query (v02)
- SupersededModelWarning C — pinneapple_hub.zoo._models (v07)
- supervised f — pinneapple_core.loss (v01)
- SupervisedLoss C — pinneapple_neural.trainer.losses (v04)
- Surface C — pinneapple_tools.visualization.studio.scene (v07)
- surface f — pinneapple_data.cae.compare (v04)
- surface_checks f — pinneapple_data.cae.quality (v04)
- surface_coefficient f — pinneapple_physics.closed_form.pipe_insulation (v02)
- surface_from_pointcloud f — pinneapple_design.geometry.mesh.mesher_3d (v06)
- surface_impedance f — pinneapple_simulation.electromagnetics.waves (v03)
- surface_scalars f — pinneapple_design.aero.case3d (v06)
- SurfaceRegion C — pinneapple_analysis.verification.geometry_intelligence (v05)
- surfaces f — pinneapple_design.aero.geometry (v06)
- SurfaceView C — pinneapple_tools.visualization.studio.planetary (v07)
- surprise_record f — pinneapple_systems.time_series.forecast_lab.pipeline (v06)
- Surrogate C — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- Surrogate C — pinneapple_design.thermal.heatsink.surrogate (v06)
- Surrogate C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- Surrogate C — pinneapple_systems.digital_twin.pipe_loop.diagnose (v06)
- surrogate f — pinneapple_flow.nodes.cfd_pipeline (v07)
- surrogate f — pinneapple_flow.nodes.surrogates (v07)
- surrogate_assisted f — pinneapple_design.design_optimizer.verify (v06)
- surrogate_from_table f — pinneapple_design.design_optimizer.report (v06)
- surrogate_inverse f — pinneapple_flow.nodes_ext.surrogate_inverse (v07)
- surrogate_job f — pinneapple_analysis.benchmark.builtin.inverse_heat_impl (v05)
- surrogate_model f — pinneapple_analysis.validation.physics_learning_cases (v05)
- surrogate_path f — pinneapple_presets.surrogate (v02)
- surrogate_report f — pinneapple_design.thermal.heatsink.trust (v06)
- surrogate_search f — pinneapple_design.bricks.search (v06)

- surrogate_service f — pinneapple_tools.nvidia.heatsink (v07)
- surrogate_steady_state f — pinneapple_simulation.rover.autolab.wheel_models (v03)
- surrogate_study f — pinneapple_flow.wrappers (v07)
- surrogate_test f — pinneapple_systems.time_series.predictability (v06)
- surrogate_trained f — pinneapple_systems.time_series.forecast_lab.study (v06)
- surrogate_vs_numerical f — pinneapple_simulation.rover.checks (v03)
- SurrogateConfig C — pinneapple_design.design_optimizer.surrogate (v06)
- SurrogateContact C — pinneapple_simulation.rover.contact (v03)
- SurrogateData C — pinneapple_analysis.surrogate.data (v05)
- SurrogateFit C — pinneapple_design.design_optimizer.report (v06)
- SurrogateFit C — pinneapple_design.surrogate_inverse.core (v06)
- SurrogateModel C — pinneapple_analysis.surrogate.model (v05)
- SurrogateModel C — pinneapple_systems.operations.twin (v06)
- SurrogatePolicy C — pinneapple_tools.surrogate_policy (v07)
- SurrogatePolicyResult C — pinneapple_tools.surrogate_policy (v07)
- SurrogatePrediction C — pinneapple_neural.workflows.hybrid_surrogate_physics (v04)
- SurrogateSpec C — pinneapple_presets.core (v02)
- SurrogateTest C — pinneapple_systems.time_series.predictability (v06)
- survey_cutout f — pinneapple_physics.astro.observing (v02)
- survey_url f — pinneapple_physics.astro.observing (v02)
- suspend f — pinneapple_simulation.numerical_solvers.particles (v03)
- sutherland_mu f — pinneapple_design.aero.buildup (v06)
- SWAGApproximation C — pinneapple_systems.component_modeling.bayesian (v06)
- Sweep C — pinneapple_simulation.pipelines.freecad_openfoam.sweep (v03)
- sweep f — pinneapple_flow.nodes.cfd_pipeline (v07)
- sweep f — pinneapple_hub.zoo.demo (v07)
- sweep f — pinneapple_lab.runner (v07)
- sweep f — pinneapple_simulation.electromagnetics.reflector.compact_range (v03)
- sweep f — pinneapple_twin3d.primitives (v06)
- sweep_figure f — pinneapple_simulation.pipelines.freecad_openfoam.report (v03)
- sweep_le_from_qc f — pinneapple_design.aero.airliner (v06)
- sweep_pareto_frontier f — pinneapple_design.surrogate_inverse.reflex_port (v06)
- sweep_profile f — pinneapple_design.geometry_library.kit (v06)
- SweepConfig C — pinneapple_neural.trainer.parallel (v04)
- SweepJob C — pinneapple_tools.benchmark_suite.runner.run_sweep (v07)
- switching_functions f — pinneapple_simulation.numerical_solvers.xtfc_general (v03)
- symbolic_identity f — pinneapple_analysis.proof.graph (v05)
- symbolic_true f — pinneapple_analysis.proof.graph (v05)

- SymbolicFDSynthGenerator C — pinneapple_data.synth.pde_symbolic (v04)
- SymbolicPDE C — pinneapple_physics.symbolic_pde.compiler (v02)
- SymbolicPDENode C — pinneapple_systems.cosimulation.adapters (v06)
- symbols f — pinneapple_hub.library.snippets (v07)
- symmetric_range f — pinneapple_tools.visualization.studio.compose (v07)
- Symmetry C — pinneapple_analysis.validation.metamorphic (v05)
- Symmetry C — pinneapple_core.transforms (v01)
- symmetry f — pinneapple_core.loss (v01)
- SymmetryCheck C — pinneapple_analysis.validation.symmetry (v05)
- SymmetryMap C — pinneapple_design.topology.filters (v06)
- SymplecticODENet C — pinneapple_neural.architectures.continuous.symplectic_ode (v04)
- SymplecticRNN C — pinneapple_neural.architectures.continuous.symplectic_rnn (v04)
- SympyBackend C — pinneapple_physics.pinn_solver.factory.sympy_backend (v02)
- SympyTorchCompiler C — pinneapple_physics.pinn_solver.factory.sympy_backend (v02)
- SynthCatalog C — pinneapple_data.synth.registry (v04)
- SynthConfig C — pinneapple_data.synth.base (v04)
- synthetic f — pinneapple_analysis.inverse_problems.presets (v05)
- synthetic_campaign f — pinneapple_twin3d.wear.synthetic (v06)
- synthetic_daily f — pinneapple_systems.time_series.solar_cycle (v06)
- synthetic_flights f — pinneapple_systems.sports.football.ball_study (v06)
- synthetic_recession f — pinneapple_systems.digital_twin.recession (v06)
- synthetic_recovery f — pinneapple_systems.sports.football.ball_study (v06)
- synthetic_runs f — pinneapple_systems.sports.football.kin_study (v06)
- synthetic_smoothing f — pinneapple_systems.sports.football.kin_study (v06)
- SyntheticDataFactory C — pinneapple_worldmodel.synthetic_factory (v04)
- SyntheticFactoryConfig C — pinneapple_worldmodel.synthetic_factory (v04)
- SyntheticSeries C — pinneapple_systems.time_series.synthetic (v06)
- SyntheticSite C — pinneapple_simulation.rover.autolab.site (v03)
- SynthGenerator C — pinneapple_data.synth.base (v04)
- SynthOutput C — pinneapple_data.synth.base (v04)
- System C — pinneapple_condense.fv (v02)
- system_of f — pinneapple_flow.nodes._common (v07)
- SystemState C — pinneapple_systems.digital_twin.state (v06)

T

- `t10_pitch` f — `pinneapple_condense.falsify` (v02)
- `t1_point` f — `pinneapple_condense.falsify` (v02)
- `t1_reference` f — `pinneapple_condense.falsify` (v02)
- `t2_contrast` f — `pinneapple_condense.falsify` (v02)
- `t3_anisotropy` f — `pinneapple_condense.falsify` (v02)
- `t4_no_repetition` f — `pinneapple_condense.falsify` (v02)
- `t5_variants` f — `pinneapple_condense.falsify` (v02)
- `t6_t7_nonlinear` f — `pinneapple_condense.falsify` (v02)
- `t8_adversarial` f — `pinneapple_condense.falsify` (v02)
- `t90` f — `pinneapple_twin3d.wear.forecast` (v06)
- `t9_thin` f — `pinneapple_condense.falsify` (v02)
- `t_baseline` f — `pinneapple_design.thermal.board_cooling.agent` (v06)
- `t_benchmark` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `t_build_twin` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `t_calibrate` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `t_deliver` f — `pinneapple_design.thermal.board_cooling.agent` (v06)
- `t_deliver` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `t_junction` f — `pinneapple_design.geometry.csg` (v06)
- `t_learn` f — `pinneapple_design.thermal.board_cooling.agent` (v06)
- `t_monitor` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `t_optimize` f — `pinneapple_design.thermal.board_cooling.agent` (v06)
- `t_read_mission` f — `pinneapple_design.thermal.board_cooling.agent` (v06)
- `t_read_mission` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `t_request_simulation` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `t_sample` f — `pinneapple_design.thermal.board_cooling.agent` (v06)
- `t_screen_natural` f — `pinneapple_design.thermal.board_cooling.agent` (v06)
- `t_trust` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `t_verify` f — `pinneapple_design.thermal.board_cooling.agent` (v06)
- `Table` C — `pinneapple_data.contracts.models` (v04)
- `Table` C — `pinneapple_tools.reporting.datasheet` (v07)
- `table` f — `pinneapple_hub.library.categories` (v07)
- `table` f — `pinneapple_science.report` (v02)
- `table_boxes` f — `pinneapple_design.bricks.dynamics` (v06)
- `table_evaluator` f — `pinneapple_design.design_optimizer.report` (v06)
- `table_format` f — `pinneapple_data.contracts.tables` (v04)
- `table_markdown` f — `pinneapple_simulation.flight.figures` (v03)
- `TableIronLoss` C — `pinneapple_simulation.electromagnetics.machines.steel` (v03)
- `tables_to_csv` f — `pinneapple_veriphysics.validation_lab` (v05)

- `TableSpec` C — `pinneapple_data.formfill.spec` (v04)
- `TabulatedModel` C — `pinneapple_analysis.inverse_problems.forward` (v05)
- `TabulatedSignal` C — `pinneapple_physics.pinn_solver.factory.signals` (v02)
- `tag_names` f — `pinneapple_core.onboarding.catalog` (v01)
- `tag_solve_kwargs` f — `pinneapple_physics.pde_environment.presets.tag_geometry` (v02)
- `TagConditionsUnresolved` C — `pinneapple_physics` (v02)
- `tagged_preset` f — `pinneapple_flow.nodes_ext.pde_env` (v07)
- `TagHeuristics` C — `pinneapple_design.geometry.builders.stl_domain_batch_builder` (v06)
- `tai_minus_utc` f — `pinneapple_physics.astro.timescales` (v02)
- `takeoff` f — `pinneapple_flow.nodes.systems` (v07)
- `takeoff_cases` f — `pinneapple_flow.nodes_ext.flight` (v07)
- `takeoff_derivatives` f — `pinneapple_simulation.flight.takeoff` (v03)
- `takeoff_figure` f — `pinneapple_simulation.flight.figures` (v03)
- `takeoff_metrics` f — `pinneapple_simulation.flight.takeoff` (v03)
- `TakeoffPilot` C — `pinneapple_simulation.flight.control` (v03)
- `TakeoffSchedule` C — `pinneapple_simulation.flight.control` (v03)
- `tangential_continuity` f — `pinneapple_physics.maxwell_checks` (v02)
- `tank_balance_rules` f — `pinneapple_systems.operations.anomaly` (v06)
- `tank_constants` f — `pinneapple_systems.operations.anomaly` (v06)
- `tank_rhs` f — `pinneapple_systems.operations.plant` (v06)
- `tank_step` f — `pinneapple_systems.operations.plant` (v06)
- `TankParams` C — `pinneapple_systems.operations.plant` (v06)
- `TankShellVanillaPINN` C — `pinneapple_systems.component_library.components` (v06)
- `TankSimulator` C — `pinneapple_systems.operations.plant` (v06)
- `target_altaz` f — `pinneapple_physics.astro.observing` (v02)
- `target_margin` f — `pinneapple_reason.modes._util` (v05)
- `targets_at` f — `pinneapple_systems.time_series.forecast_lab.base` (v06)
- `tarjan_scc` f — `pinneapple_systems.cosimulation.utils` (v06)
- `TaskDistribution` C — `pinneapple_worldmodel.meta_learning` (v04)
- `TaskResult` C — `pinneapple_llm.workbench.evaluate` (v07)
- `TaskResult` C — `pinneapple_tools.benchmark_suite.tasks.base` (v07)
- `taylor_green_ic` f — `pinneapple_simulation.numerical_solvers.ns3d_spectral` (v03)
- `taylor_green_velocity_fns` f — `pinneapple_simulation.numerical_solvers.jax_cfd_solver` (v03)
- `TC38Geometry` C — `pinneapple_design.geometry.gen.threaded_connection` (v06)
- `TC46Geometry` C — `pinneapple_design.geometry.gen.threaded_connection` (v06)
- `TC50Geometry` C — `pinneapple_design.geometry.gen.threaded_connection` (v06)
- `TCDACell` C — `pinneapple_simulation.electromagnetics.periodic.tcda` (v03)
- `TCN` C — `pinneapple_neural.architectures.classical_ts.tcn` (v04)
- `TCNConfig` C — `pinneapple_systems.time_series.models.tcn` (v06)

- TCNForecaster C — pinneapple_systems.time_series.models.tcn (v06)
- tdb_minus_tt f — pinneapple_physics.astro.timescales (v02)
- teff_from_bv f — pinneapple_physics.astro.stars (v02)
- TelemetryConditioner C — pinneapple_systems.digital_twin.conditioning (v06)
- Telescope C — pinneapple_physics.astro.observing (v02)
- temperature_ratio f — pinneapple_physics.closed_form.rocket_nozzle (v02)
- Template C — pinneapple_core.workflows.templates (v01)
- Template C — pinneapple_design.fitmesh.catalog (v06)
- template f — pinneapple_flow.templates (v07)
- template_doc f — pinneapple_flow.planner.extensions (v07)
- template_plan f — pinneapple_design.thermal.board_cooling.agent (v06)
- template_plan f — pinneapple_systems.digital_twin.pipe_loop.agent (v06)
- TemplatePlanner C — pinneapple_autolab.core (v07)
- templates f — pinneapple_design.intent.parse (v06)
- temporal_split f — pinneapple_systems.time_series.tuning.optuna_tuner (v06)
- TemporalFusionTransformer C — pinneapple_neural.architectures.transformers.tft (v04)
- tensor_grid f — pinneapple_physics.paper_problems (v02)
- TensorGrid C — pinneapple_science.numerics (v02)
- term_dimension f — pinneapple_simulation.electromagnetics.units (v03)
- terminal_velocity f — pinneapple_simulation.numerical_solvers.particles (v03)
- TerminalNode C — pinneapple_decision.tree (v05)
- terrain_at f — pinneapple_simulation.vehicle.viz (v03)
- terrain_colors f — pinneapple_simulation.rover.autolab.render (v03)
- terrain_grid f — pinneapple_simulation.rover.autolab.render (v03)
- terrain_height f — pinneapple_tools.visualization.studio.planetary (v07)
- terrain_self_shadow f — pinneapple_tools.visualization.studio.shadows (v07)
- TerramechanicsResiduals C — pinneapple_physics.pde_environment.presets.terramechanics (v02)
- test_points f — pinneapple_tools.benchmark_suite.crossframework.problems (v07)
- TestbedData C — pinneapple_simulation.terramechanics.testbed (v03)
- tet_boundary_faces f — pinneapple_simulation.external_solvers.cfd_formats.convert (v03)
- tetramer_threshold_branches f — pinneapple_analysis.proof.pt_chains (v05)
- text f — pinneapple_check.report (v05)
- text_of f — pinneapple_reason.chain (v05)
- TextChunk C — pinneapple_tools.hpo_experiments.kb.chunking (v07)
- tf f — pinneapple_design.geometry_library.kit (v06)

- `tfc_available` f — `pinneapple_neural.architectures.pinns.xtfc` (v04)
- `TFCConstrainedExpression` C — `pinneapple_neural.architectures.pinns.xtfc` (v04)
- `TFTConfig` C — `pinneapple_systems.time_series.models.tft` (v06)
- `TFTForecaster` C — `pinneapple_systems.time_series.models.tft` (v06)
- `theorem_profile` f — `pinneapple_physics.non_hermitian.ep_tuning` (v02)
- `thermal_case` f — `pinneapple_condense.falsify` (v02)
- `thermal_field` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `thermal_field` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `thermal_field` f — `pinneapple_design.thermal.fin_array_speed` (v06)
- `thermal_image` f — `pinneapple_worldmodel.inspection_data` (v04)
- `thermal_plate` f — `pinneapple_worldmodel.inspection_data` (v04)
- `thermal_resistance_k_w` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `thermal_resistance_k_w` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `ThermalChannel2D` C — `pinneapple_simulation.numerical_solvers.thermal_channel_2d` (v03)
- `ThermalCompliance2D` C — `pinneapple_design.topology.problems` (v06)
- `ThermalEfficiencyObjective` C — `pinneapple_design.design_optimizer.objective` (v06)
- `ThermalNetwork` C — `pinneapple_simulation.electromagnetics.machines.thermal` (v03)
- `thermoelasticity_2d_default` f — `pinneapple_physics.pde_environment.presets.structural` (v02)
- `Theta` C — `pinneapple_systems.time_series.adaptive` (v06)
- `theta_soil` f — `pinneapple_simulation.rover.autolab.estimation` (v03)
- `thick_te` f — `pinneapple_design.geometry_library.kit` (v06)
- `thick_walled_cylinder_lame_default` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)
- `thin_airfoil_cl` f — `pinneapple_physics.closed_form.thin_airfoil` (v02)
- `thin_airfoil_zero_lift` f — `pinneapple_presets.cases.airfoil` (v02)
- `threaded_coupling_tc50_box_default` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)
- `threaded_coupling_tc50_pin_default` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)
- `threaded_coupling_tc50_rotating_default` f — `pinneapple_physics.pde_environment.presets.solid_mechanics` (v02)
- `ThreadedConnectionMesher` C — `pinneapple_design.geometry.gen.threaded_connection` (v06)
- `ThreadProfile` C — `pinneapple_design.geometry.gen.threaded_connection` (v06)

- `threejs_config` f — `pinneapple_tools.visualization.studio.lighting` (v07)
- `threshold_coefficient` f — `pinneapple_simulation.numerical_solvers.operator_benchmarks` (v03)
- `threshold_numeric` f — `pinneapple_analysis.proof.pt_chains` (v05)
- `ThresholdDetector` C — `pinneapple_systems.digital_twin.monitoring.anomaly` (v06)
- `ThresholdVerifier` C — `pinneapple_decision.verifiers` (v05)
- `ThroughputMonitor` C — `pinneapple_neural.trainer.parallel` (v04)
- `thrust_coefficient` f — `pinneapple_physics.closed_form.rocket_nozzle` (v02)
- `thrust_direction_body` f — `pinneapple_simulation.flight.propulsion` (v03)
- `thumbnail` f — `pinneapple_design.geometry_library` (v06)
- `thumbnail_bytes` f — `pinneapple_lab.html` (v07)
- `thumbnail_path` f — `pinneapple_design.geometry_library.store` (v06)
- `ticks` f — `pinneapple_tools.visualization.studio.colormap` (v07)
- `TikhonovRegularizer` C — `pinneapple_analysis.inverse_problems.regularization` (v05)
- `Tile` C — `pinneapple_lab.showcase` (v07)
- `Tile` C — `pinneapple_tools.visualization.dashboard.spec` (v07)
- `Time` C — `pinneapple_physics.astro.timescales` (v02)
- `time_average` f — `pinneapple_simulation.numerical_solvers.free_surface` (v03)
- `time_dependent_loss` f — `pinneapple_quantum.loss.schrodinger` (v04)
- `time_derivative` f — `pinneapple_physics.pinn_solver.compiler.autograd_ops` (v02)
- `time_directories` f — `pinneapple_twin3d.openfoam` (v06)
- `time_dirs` f — `pinneapple_data.cae.foam` (v04)
- `time_dirs` f — `pinneapple_design.aero.case` (v06)
- `time_dirs` f — `pinneapple_simulation.accelerated_foam.case` (v03)
- `time_per_query` f — `pinneapple_analysis.cost.equal_budget` (v05)
- `time_reversal_asymmetry` f — `pinneapple_systems.time_series.predictability` (v06)
- `time_sequence` f — `pinneapple_physics.paper_problems` (v02)
- `time_series_cv` f — `pinneapple_data.process_optimizer` (v04)
- `time_step` f — `pinneapple_physics.blackhole.hydro_numba` (v02)
- `time_surrogate` f — `pinneapple_design.thermal.fin_array_speed` (v06)
- `time_to_intercept` f — `pinneapple_systems.sports.football.pitch_control` (v06)
- `time_to_reach_region` f — `pinneapple_analysis.uncertainty.reachability` (v05)
- `TimeCap` C — `pinneapple_tools.benchmark_suite.crossframework.records` (v07)
- `TimeHarmonicSolution` C — `pinneapple_simulation.electromagnetics.eddy` (v03)
- `timeline_figure` f — `pinneapple_reason.report` (v05)
- `timeline_scene` f — `pinneapple_design.bricks.visuals` (v06)
- `timeline_scene` f — `pinneapple_design.thermal.board_cooling.scene` (v06)
- `TimeMarchingTrainer` C — `pinneapple_neural.trainer.time_marching` (v04)
- `Timer` C — `pinneapple_arena.external.result` (v04)

- `timeseries_spec` f — `pinneapple_flow.nodes_ext.arena_pipelines` (v07)
- `TimeSeriesAdapter` C — `pinneapple_neural.architectures.adapters.ts` (v04)
- `TimeSeriesBenchmarkSpec` C — `pinneapple_tools.benchmark_suite.timeseries_pipeline` (v07)
- `TimeSeriesCoSimNode` C — `pinneapple_systems.cosimulation.adapters` (v06)
- `TimeSeriesImputer` C — `pinneapple_systems.time_series.preparation.imputer` (v06)
- `TimeSeriesModelBase` C — `pinneapple_neural.architectures.transformers.base` (v04)
- `TimeSeriesResampler` C — `pinneapple_systems.time_series.preparation.resampler` (v06)
- `TimeSeriesScaler` C — `pinneapple_systems.time_series.csv_loader` (v06)
- `TimeSeriesSpec` C — `pinneapple_systems.time_series.spec` (v06)
- `TimesFMForecaster` C — `pinneapple_systems.time_series.foundation_models` (v06)
- `TimesNet` C — `pinneapple_neural.architectures.transformers.timesnet` (v04)
- `timestep_check` f — `pinneapple_simulation.rover.autolab.verify` (v03)
- `tip_mach_number` f — `pinneapple_systems.process_components.similarity_map` (v06)
- `tip_over_offset` f — `pinneapple_design.bricks.closed_form` (v06)
- `tip_speed_m_s` f — `pinneapple_systems.process_components.similarity_map` (v06)
- `tipping_report` f — `pinneapple_design.bricks.statics` (v06)
- `tisserand` f — `pinneapple_physics.astro.neural_propagator` (v02)
- `TJunctionDomain2D` C — `pinneapple_design.geometry.gen.domains` (v06)
- `tm_cavity_mode` f — `pinneapple_physics.closed_form.electromagnetics` (v02)
- `tmax_cells` f — `pinneapple_condense.substructure` (v02)
- `to_aas` f — `pinneapple_systems.digital_twin.standards` (v06)
- `to_arrays` f — `pinneapple_simulation.electromagnetics.periodic.benchmark` (v03)
- `to_dataframe` f — `pinneapple_analysis.benchmark.report` (v05)
- `to_dbi` f — `pinneapple_simulation.electromagnetics.antenna.bounds` (v03)
- `to_device_sample` f — `pinneapple_data.device` (v04)
- `to_ditto_thing` f — `pinneapple_systems.digital_twin.standards` (v06)
- `to_dtdl` f — `pinneapple_systems.digital_twin.standards` (v06)
- `to_dxf` f — `pinneapple_simulation.electromagnetics.machines.interop` (v03)
- `to_exportable` f — `pinneapple_neural.architectures.neural_operators.fno_export` (v04)
- `to_glb` f — `pinneapple_design.aero.airframe` (v06)
- `to_glb` f — `pinneapple_design.geometry_library.exporters` (v06)
- `to_glb` f — `pinneapple_tools.visualization.molecular` (v07)
- `to_json` f — `pinneapple_check.report` (v05)
- `to_json` f — `pinneapple_flow.identify` (v07)
- `to_json` f — `pinneapple_simulation.rover.autolab.mission` (v03)

- to_jsonable f — pinneapple_core.jsonable (v01)
- to_ldr f — pinneapple_design.bricks.ldraw (v06)
- to_markdown f — pinneapple_simulation.electromagnetics.verification (v03)
- to_mat f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- to_mtl f — pinneapple_design.geometry_library.exporters (v06)
- to_ngsi_ld f — pinneapple_systems.digital_twin.standards (v06)
- to_obj f — pinneapple_design.geometry_library.exporters (v06)
- to_parameter_space f — pinneapple_analysis.inverse_problems.contracts (v05)
- to_parameter_space f — pinneapple_design.design_optimizer.report (v06)
- to_physical_sample f — pinneapple_data.synth.sample_adapter (v04)
- to_physical_sample f — pinneapple_data.unified (v04)
- to_problemdesign_dict f — pinneapple_tools.hpo_experiments.pipelines.export_problemdesign (v07)
- to_pyleecan f — pinneapple_simulation.electromagnetics.machines.interop (v03)
- to_rgb f — pinneapple_science.plots (v02)
- to_series f — pinneapple_systems.time_series.enso.dataset (v06)
- to_si f — pinneapple_design.intent.spec (v06)
- to_si f — pinneapple_physics.pde_environment.presets.astropy_bridge (v02)
- to_signal_batch f — pinneapple_simulation.numerical_solvers.adapters.upd_signal (v03)
- to_step f — pinneapple_design.geometry_library.exporters (v06)
- to_stl f — pinneapple_design.aero.airframe (v06)
- to_stl f — pinneapple_design.geometry_library.exporters (v06)
- to_stl f — pinneapple_design.surrogate_inverse.meshes (v06)
- to_study_result f — pinneapple_tools.automation.report (v07)
- to_study_result f — pinneapple_veriphysics.evidence_package (v05)
- to_toml f — pinneapple_veriphysics.validation_lab (v05)
- to_usda f — pinneapple_design.aero.airframe (v06)
- to_usda f — pinneapple_design.geometry_library.exporters (v06)
- to_values f — pinneapple_analysis.uncertainty.distributions (v05)
- to_workflow f — pinneapple_flow.runtime (v07)
- toa_insolation f — pinneapple_physics.weather.model (v02)
- tokenize f — pinneapple_tools.hpo_experiments.ranking (v07)
- tolerance_model f — pinneapple_simulation.electromagnetics.machines.robustness (v03)
- tolerance_verdicts f — pinneapple_data.cae.compare (v04)
- ToleranceResult C — pinneapple_analysis.uncertainty.tolerance (v05)
- Tolerances C — pinneapple_veriphysics.applicability (v05)
- tone_map f — pinneapple_physics.astro.lensing (v02)
- tonight f — pinneapple_physics.astro.facts (v02)
- Tool C — pinneapple_autolab.core (v07)
- Toolbox C — pinneapple_autolab.core (v07)
- Toolbox C — pinneapple_systems.component_library.toolbox (v06)

- `toolbox` f — `pinneapple_design.thermal.board_cooling.agent` (v06)
- `toolbox` f — `pinneapple_systems.digital_twin.pipe_loop.agent` (v06)
- `ToolboxRegistry` C — `pinneapple_systems.component_library.toolbox` (v06)
- `ToolCall` C — `pinneapple_llm.workbench.backend` (v07)
- `Toolchain` C — `pinneapple_tools.automation.toolchain` (v07)
- `ToolError` C — `pinneapple_llm.workbench.tools` (v07)
- `ToolRecommendation` C — `pinneapple_analysis.verification.tool_recommendation` (v05)
- `top_load` f — `pinneapple_design.bricks.statics` (v06)
- `topic_matches` f — `pinneapple_systems.digital_twin.io.mqtt_broker` (v06)
- `topological_sort` f — `pinneapple_systems.cosimulation.utils` (v06)
- `topology` f — `pinneapple_design.aero.mesh` (v06)
- `topology2d` f — `pinneapple_flow.nodes_ext.design` (v07)
- `topology_gradcheck` f — `pinneapple_flow.nodes_ext.design` (v07)
- `TopOptResult` C — `pinneapple_design.topology.driver` (v06)
- `torch_filter` f — `pinneapple_physics.multiscale.learned` (v02)
- `torch_from_handles` f — `pinneapple_design.aero.geometry_onnx` (v06)
- `torch_geometry` f — `pinneapple_design.aero.geometry_onnx` (v06)
- `torch_pcg` f — `pinneapple_simulation.accelerated_foam.solvers` (v03)
- `torch_refilter` f — `pinneapple_physics.multiscale.learned` (v02)
- `torch_transfer` f — `pinneapple_physics.multiscale.learned` (v02)
- `TorchModuleEvaluator` C — `pinneapple_design.design_optimizer.evaluators` (v06)
- `TorchNode` C — `pinneapple_systems.cosimulation.node` (v06)
- `TorchPhysicsBackend` C — `pinneapple_core.backend` (v01)
- `TorchRunConfig` C — `pinneapple_neural.trainer.hpc` (v04)
- `TorchVorticityLES2D` C — `pinneapple_physics.multiscale.learned` (v02)
- `torque_coenergy` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `torque_free_spin` f — `pinneapple_simulation.external_solvers.newton_physics.validation` (v03)
- `torque_maxwell_stress` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `torque_maxwell_stress_gradient` f — `pinneapple_simulation.electromagnetics.postprocess` (v03)
- `torque_trajectory` f — `pinneapple_simulation.robot_sim.sim` (v03)
- `torsion_optimizer` f — `pinneapple_flow.nodes_ext.core_ts` (v07)
- `torsional_shear_stress` f — `pinneapple_systems.process_components.pipe_stress_mechanics` (v06)
- `torus` f — `pinneapple_design.geometry_library.kit` (v06)
- `torus3d` f — `pinneapple_design.geometry.gen.sdf_shapes` (v06)
- `torus_graph` f — `pinneapple_physics.multiscale.learned` (v02)
- `torus_state` f — `pinneapple_physics.blackhole.hydro` (v02)
- `TotalVariationRegularizer` C — `pinneapple_analysis.inverse_problems.regularization` (v05)
- `tower_tip_force` f — `pinneapple_design.bricks.closed_form` (v06)
- `TowerGoal` C — `pinneapple_design.bricks.goals` (v06)
- `tr` f — `pinneapple_reason.chain` (v05)

- `trace_snapshots` f — `pinneapple_condense.substructure` (v02)
- `trace_thin_disc` f — `pinneapple_physics.astro.lensing` (v02)
- `track_meshes` f — `pinneapple_design.intent.physics.loop_track` (v06)
- `track_object` f — `pinneapple_worldmodel.physical_world` (v04)
- `track_setpoint` f — `pinneapple_systems.time_series.eda.plots` (v06)
- `TrackingReport` C — `pinneapple_systems.digital_twin.product.evidence` (v06)
- `traction_monitor` f — `pinneapple_simulation.rover.autolab.mission` (v03)
- `tradeoff_map` f — `pinneapple_simulation.electromagnetics.reflector.study` (v03)
- `train` f — `pinneapple_design.thermal.heatsink.surrogate` (v06)
- `train` f — `pinneapple_physics.weather.train` (v02)
- `train_acoustic` f — `pinneapple_analysis.inspection.acoustic` (v05)
- `train_and_predict` f — `pinneapple_neural` (v04)
- `train_closed_form_model` f — `pinneapple_arena.pointwise.training` (v04)
- `train_closure_aposteriori` f — `pinneapple_physics.multiscale.learned` (v02)
- `train_component_task` f — `pinneapple_orchestration.tasks.training_tasks` (v01)
- `train_contact_surrogate` f — `pinneapple_simulation.terramechanics.surrogate` (v03)
- `train_decay_pinn` f — `pinneapple_physics.astro.decay` (v02)
- `train_dense_volume_operator` f — `pinneapple_neural.workflows.dense_volume_operator` (v04)
- `train_discrete_time` f — `pinneapple_simulation.numerical_solvers.discrete_time_pinn` (v03)
- `train_dynamics` f — `pinneapple_neural.architectures.graphnn.mgn_dynamics` (v04)
- `train_eddy_current` f — `pinneapple_analysis.inspection.eddy_current` (v05)
- `train_emat` f — `pinneapple_analysis.inspection.emat` (v05)
- `train_end` f — `pinneapple_systems.time_series.forecast_lab.base` (v06)
- `train_field_pinn` f — `pinneapple_analysis.benchmark.builtin.fields` (v05)
- `train_forecaster` f — `pinneapple_physics.astro.spaceweather` (v02)
- `train_forecaster` f — `pinneapple_physics.blackhole.forecast` (v02)
- `train_foundation` f — `pinneapple_worldmodel.train` (v04)
- `train_generic_inspection_pinn` f — `pinneapple_analysis.inspection.base` (v05)
- `train_gnn` f — `pinneapple_design.aero.train` (v06)
- `train_graph` f — `pinneapple_arena.trainer` (v04)
- `train_graybox` f — `pinneapple_neural.trainer.graybox` (v04)
- `train_grid_operator` f — `pinneapple_arena.trainer` (v04)
- `train_grid_operator` f — `pinneapple_neural.physicsnemo_bridge.field_operators` (v04)
- `train_guided_wave_ultrasonic` f — `pinneapple_analysis.inspection.guided_wave_ultrasonic` (v05)

- `train_magnetic_flux_leakage` f — `pinneapple_analysis.inspection.magnetic_flux_leakage` (v05)
- `train_meta` f — `pinneapple_worldmodel.train` (v04)
- `train_mgn` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `train_mlp` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `train_mlp` f — `pinneapple_design.aero.train` (v06)
- `train_mlp_fields` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `train_mlx` f — `pinneapple_llm.workbench.finetune` (v07)
- `train_model` f — `pinneapple_neural` (v04)
- `train_monotone` f — `pinneapple_design.thermal.heatsink.monotone` (v06)
- `train_nozzle_pinn` f — `pinneapple_analysis.verification.nozzle_crosscheck` (v05)
- `train_on_trajectories` f — `pinneapple_physics.blackhole.forecast` (v02)
- `train_operator` f — `pinneapple_analysis.benchmark.builtin.fields` (v05)
- `train_operator` f — `pinneapple_tools.benchmark_suite.crossframework.darcy` (v07)
- `train_orbit_pinn` f — `pinneapple_physics.astro.surrogate` (v02)
- `train_parametric_dense_volume_operator` f — `pinneapple_neural.workflows.parametric_dense_volume_operator` (v04)
- `train_peft` f — `pinneapple_llm.workbench.finetune` (v07)
- `train_phased_array_ultrasonic` f — `pinneapple_analysis.inspection.phased_array_ultrasonic` (v05)
- `train_pinn` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `train_pinn` f — `pinneapple_analysis.validation.pinn` (v05)
- `train_pinn` f — `pinneapple_arena.trainer` (v04)
- `train_pinn` f — `pinneapple_simulation.meshless.pinn` (v03)
- `train_specialist` f — `pinneapple_worldmodel.train` (v04)
- `train_supervised` f — `pinneapple_arena.trainer` (v04)
- `train_surrogate` f — `pinneapple_analysis.benchmark.builtin.inverse_heat_impl` (v05)
- `train_surrogate` f — `pinneapple_design.design_optimizer.airfoil_drag` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.brake_rotor` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.heatsink_fins` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `train_surrogate` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `train_surrogate` f — `pinneapple_design.thermal.fin_array_speed` (v06)
- `train_surrogates` f — `pinneapple_design.aero.train` (v06)
- `train_torch_model` f — `pinneapple_arena.pointwise.training` (v04)
- `train_torch_regressor` f — `pinneapple_systems.time_series.forecast_lab.learned` (v06)

- `train_wheel_surrogate` f — `pinneapple_simulation.terramechanics.mobility` (v03)
- `train_within_budget` f — `pinneapple_analysis.cost.equal_budget` (v05)
- `trainable` f — `pinneapple_core.onboarding.catalog` (v01)
- `TrainConfig` C — `pinneapple_llm.workbench.finetune` (v07)
- `TrainConfig` C — `pinneapple_neural.trainer.trainer` (v04)
- `TrainConfig` C — `pinneapple_physics.blackhole.forecast` (v02)
- `TrainContext` C — `pinneapple_analysis.surrogate.methods` (v05)
- `Trainer` C — `pinneapple_neural.trainer.trainer` (v04)
- `training_box` f — `pinneapple_simulation.electromagnetics.antenna.design` (v03)
- `training_distribution` f — `pinneapple_analysis.validation.physics_learning_cases` (v05)
- `training_gates` f — `pinneapple_analysis.benchmark.checks` (v05)
- `TrainingAdvisor` C — `pinneapple_neural.trainer.advisor` (v04)
- `TrainingConfig` C — `pinneapple_arena.config` (v04)
- `TrainingHooks` C — `pinneapple_worldmodel.training_hooks` (v04)
- `TrainingResult` C — `pinneapple_neural.physicsnemo_bridge.field_operators` (v04)
- `TrainingStrategySelector` C — `pinneapple_decision.selectors` (v05)
- `TrainResult` C — `pinneapple_arena.trainer` (v04)
- `trajectories_to_upd` f — `pinneapple_simulation.external_solvers.mujoco.adapter` (v03)
- `Trajectory` C — `pinneapple_physics.astro.smallbodies` (v02)
- `Trajectory` C — `pinneapple_simulation.rover.dynamics` (v03)
- `Trajectory` C — `pinneapple_systems.cosimulation.recorder` (v06)
- `trajectory_to_upd` f — `pinneapple_simulation.external_solvers.mujoco.adapter` (v03)
- `TrajectoryData` C — `pinneapple_worldmodel.simulator` (v04)
- `TrajectoryRecorder` C — `pinneapple_systems.cosimulation.recorder` (v06)
- `TrajectorySampler` C — `pinneapple_core.data` (v01)
- `transaction` f — `pinneapple_core.service.db` (v01)
- `transfer` f — `pinneapple_flow.nodes.methods` (v07)
- `transfer` f — `pinneapple_flow.nodes_ext.benchmarks` (v07)
- `transfer` f — `pinneapple_physics.multiscale.filters` (v02)
- `TransferBenchmarkConfig` C — `pinneapple_tools.benchmark_suite.transfer_benchmark` (v07)
- `TransferBenchmarkPipeline` C — `pinneapple_tools.benchmark_suite.transfer_benchmark` (v07)
- `TransferBenchmarkResult` C — `pinneapple_tools.benchmark_suite.transfer_benchmark` (v07)
- `TransferConfig` C — `pinneapple_adaptation.transfer_learning.config` (v04)
- `TransferScenario` C — `pinneapple_tools.benchmark_suite.transfer_benchmark` (v07)
- `TransferTrainer` C — `pinneapple_adaptation.transfer_learning.trainer` (v04)
- `Transform` C — `pinneapple_core.transforms` (v01)
- `transform` f — `pinneapple_twin3d.primitives` (v06)
- `transform_coordinates` f — `pinneapple_physics.pde_environment.presets.astropy_bridge` (v02)
- `TransformedSurrogate` C — `pinneapple_neural.workflows.hybrid_surrogate_physics` (v04)

- TransformerCatalog C — pinneapple_neural.architectures.transformers.registry (v04)
- transient f — pinneapple_condense.figures (v02)
- transient f — pinneapple_reason.modes.fluids (v05)
- transient_heat_3d_default f — pinneapple_physics.pde_environment.presets.industry (v02)
- transient_rhs f — pinneapple_systems.process_components.heat_exchanger (v06)
- transient_study f — pinneapple_condense.benchmark (v02)
- TransientPipe C — pinneapple_systems.process_components.pipe_network_1d (v06)
- TransientResult C — pinneapple_systems.process_components.water_hammer (v06)
- translate f — pinneapple_design.geometry.vehicle_cad (v06)
- Transolver C — pinneapple_neural.architectures.neural_operators.transolver (v04)
- TransolverBlock C — pinneapple_neural.architectures.neural_operators.transolver (v04)
- TransolverLite C — pinneapple_neural.architectures.neural_operators.transolver (v04)
- travel_time f — pinneapple_systems.sports.football.ball (v06)
- travel_times f — pinneapple_physics.astro.journey (v02)
- TreeRun C — pinneapple_decision.tree (v05)
- TreeStep C — pinneapple_decision.tree (v05)
- Trial C — pinneapple_arena.autoresearch (v04)
- TriangleQualityStats C — pinneapple_analysis.verification.mesh_intelligence (v05)
- triangles_to_edges f — pinneapple_neural.architectures.graphnn.mgn_dynamics (v04)
- trim f — pinneapple_simulation.flight.trim (v03)
- trim_checks f — pinneapple_simulation.flight.verification (v03)
- trim_stability f — pinneapple_flow.nodes_ext.flight (v07)
- TrimeshBridge C — pinneapple_design.geometry.io.trimesh_bridge (v06)
- TrimResult C — pinneapple_simulation.flight.trim (v03)
- trough_jet f — pinneapple_flow.nodes_ext.cfd (v07)
- TroughSection C — pinneapple_design.geometry.trough (v06)
- true_saving f — pinneapple_data.process_optimizer (v04)
- true_subfilter f — pinneapple_physics.multiscale.closures (v02)
- trust f — pinneapple_hub.zoo.demo (v07)
- trust_card f — pinneapple_analysis.benchmark.validation (v05)
- trust_card f — pinneapple_analysis.trust.field_card (v05)
- trust_card f — pinneapple_check.evidence (v05)
- trust_card f — pinneapple_mcp.tools (v07)
- trust_card f — pinneapple_reason.trust (v05)
- trust_card_service f — pinneapple_tools.model_services.registry (v07)
- trust_horizon f — pinneapple_physics.blackhole.forecast (v02)
- trust_panels f — pinneapple_tools.visualization.trust_panels (v07)
- trust_panels_gif f — pinneapple_tools.visualization.trust_panels (v07)
- trust_report f — pinneapple_physics.astro.neural_propagator (v02)

- trust_report f — pinneapple_physics.darcy_checks (v02)
- trust_report f — pinneapple_reason (v05)
- trust_report f — pinneapple_reason.trust (v05)
- trust_report f — pinneapple_systems.digital_twin.pipe_loop.report (v06)
- trust_stages f — pinneapple_check._reuse (v05)
- trust_stages f — pinneapple_lab.curation (v07)
- TrustedLatentSurrogate C — pinneapple_neural.architectures.rom.trusted (v04)
- TrustGate C — pinneapple_analysis.trust.trust_gate (v05)
- TrustReport C — pinneapple_analysis.trust.trust_report (v05)
- TrustScore C — pinneapple_analysis.trust.trust_gate (v05)
- try_parse_unit f — pinneapple_data.physical_units (v04)
- TSAuditor C — pinneapple_systems.time_series.audit.tests (v06)
- TSDataModule C — pinneapple_systems.time_series.datamodule (v06)
- TSFeatureEngineer C — pinneapple_systems.time_series.features.engineering (v06)
- TSModelBase C — pinneapple_systems.time_series.base (v06)
- TSModelCatalog C — pinneapple_systems.time_series.registry (v06)
- TSOutput C — pinneapple_neural.architectures.transformers.base (v04)
- TSOutput C — pinneapple_systems.time_series.base (v06)
- tube f — pinneapple_design.geometry_library.kit (v06)
- tube f — pinneapple_presets.viz3d (v02)
- tube f — pinneapple_systems.digital_twin.pipe_loop.scene (v06)
- tube f — pinneapple_tools.nvidia.omniverse (v07)
- tube_loop f — pinneapple_design.geometry_library.kit (v06)
- tube_shell f — pinneapple_design.geometry.vehicle_cad (v06)
- tune f — pinneapple_flow.learn (v07)
- tune f — pinneapple_systems.time_series.forecast_lab.study (v06)
- turbodesigner_to_upd f — pinneapple_simulation.external_solvers.turbodesigner.bridge (v03)
- TurboDesignerConfig C — pinneapple_simulation.external_solvers.turbodesigner.bridge (v03)
- TurboDesignerWorkflow C — pinneapple_simulation.external_solvers.turbodesigner.bridge (v03)
- Turbofan C — pinneapple_simulation.flight.propulsion (v03)
- turbulence_checks f — pinneapple_simulation.flight.verification (v03)
- turbulence_encounter f — pinneapple_flow.nodes_ext.flight (v07)
- turbulence_encounter f — pinneapple_simulation.flight.encounter (v03)
- turbulence_spec f — pinneapple_simulation.flight.turbulence (v03)
- TurbulenceModel C — pinneapple_physics.pde_environment.turbulence_selector (v02)
- TurbulenceSpec C — pinneapple_simulation.flight.turbulence (v03)
- turbulent f — pinneapple_physics.physics_audit.assumptions (v02)

- turbulent_flat_plate_cf f — pinneapple_physics.closed_form.thin_airfoil (v02)
- Turn C — pinneapple_problemdesign.state (v02)
- turntable f — pinneapple_design.geometry_library.render (v06)
- turntable_blender f — pinneapple_design.bricks.visuals (v06)
- tutorial_comparison f — pinneapple_simulation.accelerated_foam.benchmark (v03)
- tvd_advection_rhs f — pinneapple_simulation.numerical_solvers.tvd_advection (v03)
- twilight_events f — pinneapple_physics.astro.phenomena (v02)
- twilight_mag f — pinneapple_physics.astro.observing (v02)
- twin f — pinneapple_flow.nodes.systems (v07)
- twin_auto f — pinneapple_flow.nodes.twin_nodes (v07)
- twin_boundary_layer f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- twin_experiment f — pinneapple_analysis.data_assimilation.var4d (v05)
- twin_scene f — pinneapple_simulation.rover.viz (v03)
- twin_study f — pinneapple_systems.operations.contracts (v06)
- TwinDescription C — pinneapple_systems.digital_twin.standards (v06)
- TwinDraftResult C — pinneapple_llm.twin_draft (v07)
- TwinEstimate C — pinneapple_systems.digital_twin.product.assimilation (v06)
- TwinEvidence C — pinneapple_systems.digital_twin.product.evidence (v06)
- TwinMode C — pinneapple_systems.digital_twin.product.policy (v06)
- TwinModel C — pinneapple_systems.digital_twin.standards (v06)
- TwinModelError C — pinneapple_systems.digital_twin.product.model (v06)
- TwinProperty C — pinneapple_systems.digital_twin.standards (v06)
- TwinRuntime C — pinneapple_systems.operations.twin (v06)
- TwinSpec C — pinneapple_systems.digital_twin.product.spec (v06)
- TwinStore C — pinneapple_systems.operations.twin (v06)
- two_body_j2_rhs f — pinneapple_physics.astro.conjunction (v02)
- two_dielectric_capacitor f — pinneapple_physics.closed_form.electromagnetics (v02)
- two_loop_network f — pinneapple_simulation.numerical_solvers.pipe_network (v03)
- TwoLayerEBM C — pinneapple_simulation.geophysics.energy_balance (v03)
- TwoLinkArm C — pinneapple_simulation.robot_sim.arms (v03)
- TwoPhaseConfig C — pinneapple_neural.trainer.two_phase (v04)
- TwoPhaseHistory C — pinneapple_neural.trainer.two_phase (v04)
- TwoPhaseTrainer C — pinneapple_neural.trainer.two_phase (v04)

U

- u_exact f — pinneapple_analysis.inverse_problems.heat_diffusivity (v05)
- UDEForecaster C — pinneapple_systems.time_series.forecast_lab.hybrid (v06)
- Uncertain C — pinneapple_design.intent.template (v06)

- UncertainParameter C — pinneapple_analysis.uncertainty.distributions (v05)
- uncertainty_checks f — pinneapple_reason.trust (v05)
- uncertainty_result f — pinneapple_analysis.study_contracts (v05)
- uncertainty_section f — pinneapple_app.backend.core.surrogate_report (v07)
- uncertainty_study f — pinneapple_analysis.uncertainty.study (v05)
- UncertaintyReport C — pinneapple_analysis.verification.uncertainty_inverse (v05)
- unfreeze_layers f — pinneapple_adaptation.transfer_learning.freeze (v04)
- UnifiedAgentResult C — pinneapple_problemdesign.unified_agent (v02)
- UnifiedPhysicsAgent C — pinneapple_problemdesign.unified_agent (v02)
- uniform_package f — pinneapple_condense.package (v02)
- Union C — pinneapple_worldmodel.geometry (v04)
- union f — pinneapple_design.geometry_library.meshops (v06)
- Unit C — pinneapple_data.physical_units (v04)
- Unit C — pinneapple_llm.qa.units (v07)
- unit_sample f — pinneapple_analysis.uncertainty.distributions (v05)
- UnitBelief C — pinneapple_simulation.rover.autolab.estimation (v03)
- UnitError C — pinneapple_data.physical_units (v04)
- UnitError C — pinneapple_llm.qa.units (v07)
- UnitProblem C — pinneapple_simulation.electromagnetics.units (v03)
- Units C — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- units f — pinneapple_check._reuse (v05)
- UnitTables C — pinneapple_simulation.rover.autolab.planner (v03)
- UniversalUNO C — pinneapple_neural.architectures.neural_operators.uno (v04)
- UnnormModel C — pinneapple_neural.trainer.two_phase (v04)
- UNONdGrid C — pinneapple_neural.architectures.neural_operators.uno (v04)
- UNONdPoints C — pinneapple_neural.architectures.neural_operators.uno (v04)
- UnresolvedLimitError C — pinneapple_autolab.mission (v07)
- UnsafeArchive C — pinneapple_security.archives (v05)
- UnsafeCheckpoint C — pinneapple_check.models (v05)
- UnsafeModelError C — pinneapple_security.model_security (v05)
- UnsafeURL C — pinneapple_security.web (v05)
- UnscentedKalmanFilter C — pinneapple_neural.architectures.classical_ts.ukf (v04)
- unwrap f — pinneapple_app.backend.core.physics_adapter (v07)
- unwrap_model f — pinneapple_neural.trainer.parallel (v04)
- UpBlockND C — pinneapple_neural.architectures.neural_operators.uno (v04)
- upd_to_physical_sample f — pinneapple_data.adapters.upd_adapter (v04)
- update_leaderboard f — pinneapple_tools.benchmark_suite.runner.leaderboard (v07)

- UPDDataset C — pinneapple_data.upd_dataset (v04)
- UPDDataset C — pinneapple_physics.pinn_solver.io.upd_dataset (v02)
- UPDItem C — pinneapple_data.upd_types (v04)
- UPDZarrShardedWriter C — pinneapple_data.zarr_shards (v04)
- UPDZarrStore C — pinneapple_data.zarr_store (v04)
- UploadRejected C — pinneapple_core.service.uploads (v01)
- UploadStore C — pinneapple_core.service.uploads (v01)
- UploadTooLarge C — pinneapple_core.service.uploads (v01)
- upsample_sphere f — pinneapple_physics.weather.viz (v02)
- uq_predict f — pinneapple_analysis.uncertainty.core (v05)
- UQConfig C — pinneapple_arena.config (v04)
- UQResult C — pinneapple_analysis.uncertainty.core (v05)
- UsageLimitExceeded C — pinneapple_core.service.metering (v01)
- UsageMeter C — pinneapple_core.service.metering (v01)
- usd_validators f — pinneapple_tools.nvidia.omniverse (v07)
- use_backend f — pinneapple_core.backend (v01)
- use_cfd_style f — pinneapple_tools.visualization.style (v07)
- use_superlu f — pinneapple_simulation.electromagnetics.antenna.ava (v03)
- use_verdicts f — pinneapple_physics.darcy_checks (v02)
- used_by_examples f — pinneapple_hub.library.sources (v07)
- UseProfile C — pinneapple_physics.darcy_checks (v02)
- user_decision f — pinneapple_flow.decisions (v07)
- uses_cfd_approach f — pinneapple_problemdesign.schema (v02)
- uses_pinn_approach f — pinneapple_problemdesign.schema (v02)
- UseScope C — pinneapple_veriphysics.scope (v05)
- USGSDataError C — pinneapple_systems.digital_twin.io.usgs (v06)
- utc_now f — pinneapple_veriphysics.ledger (v05)
- utilisation f — pinneapple_design.bricks.visuals (v06)

V

- v_max_svpwm f — pinneapple_simulation.electromagnetics.machines.drive (v03)
- valid f — pinneapple_design.aero.geometry (v06)
- valid_cnpj f — pinneapple_security.privacy (v05)
- valid_cpf f — pinneapple_security.privacy (v05)
- validate f — pinneapple_analysis.validation.validate (v05)
- validate f — pinneapple_data.contracts.models (v04)
- validate f — pinneapple_design.bricks.validation (v06)
- validate f — pinneapple_flow.nodes_ext.bricks (v07)
- validate f — pinneapple_hub.zoo (v07)
- validate f — pinneapple_lab.manifest (v07)
- validate f — pinneapple_physics.astro.facts (v02)
- validate f — pinneapple_physics.pcb.solver (v02)
- validate f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- validate f — pinneapple_science.atlas (v02)
- validate f — pinneapple_simulation.electromagnetics.reflector.study (v03)
- validate f — pinneapple_simulation.external_solvers.newton_physics.validation (v03)
- validate f — pinneapple_simulation.terramechanics.testbed (v03)

- validate_against_solver f — pinneapple_analysis.validation.analytical (v05)
- validate_and_suggest f — pinneapple_problemdesign.elicitation.validators (v02)
- validate_consistency f — pinneapple_tools.dataset_quality.checks (v07)
- validate_custom_problem f — pinneapple_app.backend.routers.problems (v07)
- validate_dataset f — pinneapple_pdb.validate (v04)
- validate_flow f — pinneapple_flow.schema (v07)
- validate_generated f — pinneapple_llm.qa.generation (v07)
- validate_json_artifact f — pinneapple_llm.qa.static (v07)
- validate_model f — pinneapple_analysis.validation.validator (v05)
- validate_physical_sample f — pinneapple_data.validators (v04)
- validate_physics f — pinneapple_tools.nvidia.isaac (v07)
- validate_pinn f — pinneapple_analysis.validation.pinn (v05)
- validate_preset f — pinneapple_flow.nodes.presets (v07)
- validate_python_artifact f — pinneapple_llm.qa.static (v07)
- validate_result f — pinneapple_tools.benchmark_suite.crossframework.records (v07)
- validate_rollout f — pinneapple_analysis.validation.rollout (v05)
- validate_stage f — pinneapple_tools.nvidia.omniverse (v07)
- validate_tableau f — pinneapple_simulation.numerical_solvers.irk_gauss_legendre (v03)
- validate_targets f — pinneapple_simulation.electromagnetics.antenna.study (v03)
- validated_methods f — pinneapple_catalog.methods (v01)
- ValidatedDomain C — pinneapple_autolab.domain (v07)
- ValidateNode C — pinneapple_decision.tree (v05)
- validation_energy f — pinneapple_simulation.numerical_solvers.gated_xtfc (v03)
- validation_markdown f — pinneapple_simulation.pipelines.freecad_openfoam.report (v03)
- validation_metrics_from_report f — pinneapple_hub.verified_card (v07)
- validation_summary f — pinneapple_physics.astro.smallbodies (v02)
- ValidationCheck C — pinneapple_analysis.validation.validate (v05)
- ValidationContext C — pinneapple_llm.qa.validators.base (v07)
- ValidationIssue C — pinneapple_data.validators (v04)
- ValidationReport C — pinneapple_analysis.validation.core (v05)
- ValidationReport C — pinneapple_analysis.validation.validate (v05)
- ValidationReport C — pinneapple_simulation.terramechanics.testbed (v03)
- ValidationRequiredError C — pinneapple_decision.loop (v05)
- ValidationResult C — pinneapple_simulation.electromagnetics.antenna.surrogates (v03)
- ValidationSpec C — pinneapple_pdb.validate (v04)
- ValidationSpec C — pinneapple_problemdesign.schema (v02)
- ValidationStrategySelector C — pinneapple_decision.selectors (v05)

- ValidatorResult C — pinneapple_llm.qa.validators.base (v07)
- Validity C — pinneapple_reason.chain (v05)
- validity f — pinneapple_reason.library (v05)
- ValidityCheck C — pinneapple_physics.physics_audit.monitor (v02)
- ValidityEnvelope C — pinneapple_systems.process_components.real_gas_eos (v06)
- ValidityReport C — pinneapple_physics.physics_audit.monitor (v02)
- value f — pinneapple_mcp.expressions (v07)
- values f — pinneapple_simulation.numerical_solvers._bc (v03)
- valve_closure_moc f — pinneapple_simulation.numerical_solvers.water_hammer_moc (v03)
- ValveFlowResult C — pinneapple_systems.process_components.control_valve (v06)
- ValveModifiedMLP C — pinneapple_systems.component_library.components (v06)
- ValveSpec C — pinneapple_systems.process_components.control_valve (v06)
- van_leer_limiter f — pinneapple_simulation.numerical_solvers.tvd_advection (v03)
- vane_temperature_at_radial_position_c f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- VanGenuchten C — pinneapple_simulation.geophysics.richards (v03)
- VanillaPINN C — pinneapple_neural.architectures.pinns.vanilla (v04)
- VanillaTransformer C — pinneapple_neural.architectures.transformers.transformer (v04)
- VAR C — pinneapple_neural.architectures.classical_ts.var (v04)
- Var4D C — pinneapple_analysis.data_assimilation.var4d (v05)
- Var4DResult C — pinneapple_analysis.data_assimilation.var4d (v05)
- Variable C — pinneapple_data.contracts.models (v04)
- Variable C — pinneapple_design.design_optimizer.study (v06)
- VariableEnvelope C — pinneapple_veriphysics.applicability (v05)
- variables_section f — pinneapple_app.backend.core.surrogate_report (v07)
- VariableSelection C — pinneapple_pdb.builder (v04)
- VarianceBasedAL C — pinneapple_data.active_learning (v04)
- Variant C — pinneapple_condense.package (v02)
- variant_field f — pinneapple_condense.learned (v02)
- VariantPreview C — pinneapple_design.qualitative.preview (v06)
- Variation C — pinneapple_worldmodel.inspection_data (v04)
- VariationalAutoencoder C — pinneapple_neural.architectures.autoencoders.vae (v04)
- VarMapping C — pinneapple_physics.pinn_solver.io.mappings (v02)
- vec f — pinneapple_data.preflight.common (v04)
- vector f — pinneapple_data.simulation_metadata.foam_dict (v04)
- vector_potential f — pinneapple_simulation.electromagnetics.statics (v03)
- Vehicle C — pinneapple_simulation.electromagnetics.machines.drive (v03)
- vehicle_dashboard f — pinneapple_simulation.vehicle.dashboard (v03)

- vehicle_presets f — pinneapple_flow.nodes.vehicle (v07)
- vehicle_scene3d f — pinneapple_simulation.vehicle.dashboard (v03)
- vehicle_spec f — pinneapple_flow.nodes.vehicle (v07)
- VehicleCAD C — pinneapple_design.geometry.vehicle_cad (v06)
- VehicleCFD C — pinneapple_lab.experiments.vehicles3d (v07)
- VehicleModel C — pinneapple_simulation.vehicle.build (v03)
- VehicleRun C — pinneapple_simulation.vehicle.sim (v03)
- VehicleSimulator C — pinneapple_simulation.vehicle.sim (v03)
- VehicleSpec C — pinneapple_simulation.vehicle.spec (v03)
- velocity_components f — pinneapple_check.fields (v05)
- velocity_from_vorticity f — pinneapple_physics.multiscale.ns2d (v02)
- velocity_name f — pinneapple_check.fields (v05)
- velocity_set f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- vented_rotor f — pinneapple_design.surrogate_inverse.meshes (v06)
- verdict f — pinneapple_app.backend.core.surrogate_report (v07)
- verdict f — pinneapple_hub.zoo.checks (v07)
- verdict f — pinneapple_reason.modes._util (v05)
- Verification C — pinneapple_decision.schema (v05)
- Verification C — pinneapple_design.intent.pipeline (v06)
- Verification C — pinneapple_reason.chain (v05)
- verification f — pinneapple_flow.nodes_ext.flight (v07)
- verification_checks f — pinneapple_autolab.trust (v07)
- verification_jobs f — pinneapple_design.aero.verify (v06)
- VerificationCard C — pinneapple_design.surrogate_inverse.core (v06)
- VerificationError C — pinneapple_security.signing (v05)
- VerificationReport C — pinneapple_analysis.validation.physics_learning (v05)
- VerificationReport C — pinneapple_design.design_optimizer.verify (v06)
- Verifier C — pinneapple_decision.verifiers (v05)
- VerifierAgent C — pinneapple_tools.hpo_experiments.agents.verifier (v07)
- verify f — pinneapple_analysis.validation.physics_learning (v05)
- verify f — pinneapple_decision.api (v05)
- verify f — pinneapple_design.intent.pipeline (v06)
- verify f — pinneapple_design.thermal.board_cooling.verify (v06)
- verify f — pinneapple_flow.nodes.reason (v07)
- verify f — pinneapple_reason (v05)
- verify f — pinneapple_simulation.numerical_solvers.cylinder_wake (v03)
- verify f — pinneapple_simulation.numerical_solvers.thermal_channel_datasets (v03)
- verify f — pinneapple_tools.nvidia.heatsink (v07)
- verify_bytes f — pinneapple_security.signing (v05)
- verify_candidates f — pinneapple_design.design_optimizer.verify (v06)
- verify_design f — pinneapple_design.surrogate_inverse.brake_rotor (v06)
- verify_design f — pinneapple_design.surrogate_inverse.heatsink_fins (v06)

- `verify_design` f — `pinneapple_design.surrogate_inverse.reflex_port` (v06)
- `verify_design` f — `pinneapple_design.surrogate_inverse.suspension_arm` (v06)
- `verify_design` f — `pinneapple_design.surrogate_inverse.transdermal_patch` (v06)
- `verify_doi` f — `pinneapple_flow.references` (v07)
- `verify_file` f — `pinneapple_security.signing` (v05)
- `verify_files` f — `pinneapple_hub.zoo.publish` (v07)
- `verify_freeze` f — `pinneapple_analysis.sensitivity.effects` (v05)
- `verify_front` f — `pinneapple_design.design_optimizer.report` (v06)
- `verify_json` f — `pinneapple_security.signing` (v05)
- `verify_mission` f — `pinneapple_simulation.rover.autolab.verify` (v03)
- `verify_model` f — `pinneapple_hub.zoo.publish` (v07)
- `verify_model_card` f — `pinneapple_security.integrate` (v05)
- `verify_models` f — `pinneapple_hub.zoo.publish` (v07)
- `verify_package` f — `pinneapple_veriphysics.evidence_package` (v05)
- `verify_password` f — `pinneapple_security.passwords` (v05)
- `verify_quote` f — `pinneapple_data.formfill.llm` (v04)
- `verify_record` f — `pinneapple_security.integrate` (v05)
- `verify_registry` f — `pinneapple_flow.references` (v07)
- `verify_report` f — `pinneapple_analysis.validation.report_artifact` (v05)
- `verify_space` f — `pinneapple_hub.zoo.publish` (v07)
- `verify_splash_archive` f — `pinneapple_simulation.external_solvers.openfoam.splash_packer` (v03)
- `verify_uploaded_results` f — `pinneapple_design.design_optimizer.report` (v06)
- `verify_version` f — `pinneapple_security.integrate` (v05)
- `verify_viewer` f — `pinneapple_hub.zoo.publish` (v07)
- `VerifyAdapter` C — `pinneapple_tools.automation.adapters.checks` (v07)
- `VerifyKey` C — `pinneapple_security.signing` (v05)
- `VerifyResult` C — `pinneapple_tools.hpo_experiments.agents.verifier` (v07)
- `VeriPhysicsVerifier` C — `pinneapple_decision.verifiers` (v05)
- `versions` f — `pinneapple_hub.zoo` (v07)
- `versions` f — `pinneapple_hub.zoo._artifacts` (v07)
- `vertex_normals` f — `pinneapple_design.aero.airframe` (v06)
- `vertex_normals` f — `pinneapple_tools.nvidia.brep` (v07)
- `vertex_normals` f — `pinneapple_tools.visualization.studio.scene` (v07)
- `VesselSpec` C — `pinneapple_twin3d.wear.model` (v06)
- `via_copper_fraction` f — `pinneapple_physics.closed_form.pcb_thermal` (v02)
- `Video` C — `pinneapple_worldmodel.physical_world` (v04)
- `video_check` f — `pinneapple_worldmodel.physical_world` (v04)
- `view_payload` f — `pinneapple_data.cae.report` (v04)
- `viewer_status` f — `pinneapple_hub.zoo.publish` (v07)
- `views` f — `pinneapple_data.cae.compare` (v04)
- `viper` f — `pinneapple_simulation.vehicle.viper` (v03)
- `viper_cad` f — `pinneapple_simulation.vehicle.viper` (v03)
- `virtual_display` f — `pinneapple_flow.nodes.vehicle` (v07)

- VirtualDisplay C — pinneapple_tools.automation.display (v07)
- visibility f — pinneapple_physics.astro.observing (v02)
- visibility f — pinneapple_worldmodel.robot_perception (v04)
- VLMSolution C — pinneapple_design.aero.vlm (v06)
- vmap f — pinneapple_core.func (v01)
- vmap_residual f — pinneapple_tools.compute_backends.jax_backend (v07)
- vmd_1d f — pinneapple_simulation.numerical_solvers.vmd (v03)
- VMDSolver C — pinneapple_simulation.numerical_solvers.vmd (v03)
- Voltmeter f — pinneapple_science.instruments (v02)
- volume f — pinneapple_design.geometry_library.meshops (v06)
- von_karman_energy f — pinneapple_simulation.flight.turbulence (v03)
- von_mises f — pinneapple_simulation.numerical_solvers.solid_fem (v03)
- von_mises f — pinneapple_tools.visualization.pyvista_bridge (v07)
- von_mises_2d_default f — pinneapple_physics.pde_environment.presets.structural (v02)
- von_mises_from_components f — pinneapple_physics.solid.axisymmetric (v02)
- von_mises_rz f — pinneapple_physics.solid.axisymmetric (v02)
- von_mises_stress_rectangular_section f — pinneapple_systems.process_components.beam_statics (v06)
- von_mises_triaxial f — pinneapple_systems.process_components.pipe_stress_mechanics (v06)
- vorticity_residual f — pinneapple_physics.multiscale.ns2d (v02)
- VorticityNS2D C — pinneapple_physics.multiscale.ns2d (v02)
- voxel_fem_cantilever f — pinneapple_design.qualitative.quantitative (v06)
- voxel_mesh f — pinneapple_design.bricks.fea (v06)
- VoxelGrid C — pinneapple_design.geometry.ops.voxelize (v06)
- voxelgrid_to_collocation f — pinneapple_design.geometry.ops.voxelize (v06)
- VoxelInsideConfig C — pinneapple_design.geometry.builders.stl_domain_batch_builder (v06)
- voxelize_domain f — pinneapple_design.geometry.ops.voxelize (v06)
- voxelize_pointcloud f — pinneapple_design.geometry.ops.voxelize (v06)
- voxelize_sdf f — pinneapple_design.geometry.ops.voxelize (v06)
- voxelize_stl f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- VoxelModel C — pinneapple_design.bricks.fea (v06)
- VPINN C — pinneapple_neural.architectures.pinns.vpinn (v04)
- VPNoiseScheduler C — pinneapple_neural.architectures.pinns.physics_diffusion (v04)
- vq_pinn f — pinneapple_quantum.pipeline.presets (v04)
- VRotor C — pinneapple_simulation.electromagnetics.machines.geometry (v03)
- vtail_cd0 f — pinneapple_design.aero.aircraft3d (v06)
- vtail_weight f — pinneapple_design.aero.aircraft3d (v06)
- vtk_type f — pinneapple_data.cae.inp (v04)

W

- `WALEResiduals` C — `pinneapple_physics.pde_environment.turbulence_presets` (v02)
- `wall_distance` f — `pinneapple_analysis.benchmark.builtin.airfoil_impl` (v05)
- `wall_distance` f — `pinneapple_data.cae.metrics` (v04)
- `wall_geometry` f — `pinneapple_design.aero.case` (v06)
- `wall_loads` f — `pinneapple_simulation.pipelines.freecad_openfoam.post` (v03)
- `wall_loop` f — `pinneapple_design.aero.geometry` (v06)
- `wall_repulsion_force` f — `pinneapple_simulation.numerical_solvers.sph_boundaries` (v03)
- `wall_unit_height` f — `pinneapple_design.fitmesh.context` (v06)
- `WaMaModel` C — `pinneapple_worldmodel.model_zoo` (v04)
- `warm_solver` f — `pinneapple_simulation.accelerated_foam.case` (v03)
- `water` f — `pinneapple_systems.digital_twin.pipe_loop.plant` (v06)
- `water_density` f — `pinneapple_physics.closed_form.pump_npsh` (v02)
- `WaterHammer` C — `pinneapple_presets.cases.water_hammer` (v02)
- `WaterNetwork` C — `pinneapple_presets.cases.pipe_network` (v02)
- `Wave1D` C — `pinneapple_arena.problems` (v04)
- `Wave1DTask` C — `pinneapple_tools.benchmark_suite.tasks.wave_1d` (v07)
- `wave_exact` f — `pinneapple_physics.paper_problems` (v02)
- `wave_period` f — `pinneapple_systems.process_components.water_hammer` (v06)
- `wave_speed` f — `pinneapple_simulation.numerical_solvers.water_hammer_moc` (v03)
- `wave_ultrasound_3d_default` f — `pinneapple_physics.pde_environment.presets.industry` (v02)
- `wavedec_features` f — `pinneapple_simulation.numerical_solvers.wavelet` (v03)
- `WaveguideModes` C — `pinneapple_science.em.waves` (v02)
- `WaveguideUQ` C — `pinneapple_science.em.research` (v02)
- `wavelength` f — `pinneapple_simulation.electromagnetics.antenna.bounds` (v03)
- `WaveletSolver` C — `pinneapple_simulation.numerical_solvers.wavelet` (v03)
- `wavenumber` f — `pinneapple_simulation.electromagnetics.reflector.po` (v03)
- `wavenumbers` f — `pinneapple_physics.multiscale.filters` (v02)
- `weak_form_2nd_order` f — `pinneapple_lab.experiments.discovery` (v07)
- `WearDataset` C — `pinneapple_twin3d.wear.model` (v06)
- `WearMaterial` C — `pinneapple_physics.tribology.archard` (v02)
- `web_search` f — `pinneapple_llm.workbench.web` (v07)
- `web_viewer` f — `pinneapple_tools.visualization.studio` (v07)
- `web_viewer` f — `pinneapple_tools.visualization.studio.web` (v07)
- `WebAdapter` C — `pinneapple_tools.automation.adapters.checks` (v07)
- `weight_sensitivity_study` f — `pinneapple_veriphysics.robustness` (v05)
- `weight_vectors` f — `pinneapple_design.design_optimizer.methods` (v06)

- `weighted_mmd` f — `pinneapple_analysis.uncertainty.posterior_metrics` (v05)
- `weighted_wasserstein_1d` f — `pinneapple_analysis.uncertainty.posterior_metrics` (v05)
- `WeightMinimizationObjective` C — `pinneapple_design.design_optimizer.objective` (v06)
- `weights_section` f — `pinneapple_app.backend.core.surrogate_report` (v07)
- `WeightScheduler` C — `pinneapple_neural.trainer.weight_scheduler` (v04)
- `WeightSchedulerConfig` C — `pinneapple_neural.trainer.weight_scheduler` (v04)
- `welch_psd` f — `pinneapple_simulation.flight.turbulence` (v03)
- `weld` f — `pinneapple_design.geometry_library.meshops` (v06)
- `what_if` f — `pinneapple_physics.pcb.engine` (v02)
- `Wheel` C — `pinneapple_simulation.terramechanics.wheel` (v03)
- `wheel_model` f — `pinneapple_flow.nodes.vehicle` (v07)
- `wheel_on_soil` f — `pinneapple_flow.wrappers` (v07)
- `WheelParams` C — `pinneapple_simulation.particle_dynamics.terramechanics` (v03)
- `white_noise` f — `pinneapple_systems.time_series.synthetic` (v06)
- `wien_peak_m` f — `pinneapple_physics.astro.stars` (v02)
- `williamson_case2` f — `pinneapple_simulation.geophysics.sphere_shallow_water` (v03)
- `williamson_case5` f — `pinneapple_simulation.geophysics.sphere_shallow_water` (v03)
- `williamson_case6` f — `pinneapple_simulation.geophysics.sphere_shallow_water` (v03)
- `wilson_interval` f — `pinneapple_analysis.uncertainty.tolerance` (v05)
- `wind_convection` f — `pinneapple_reason.props` (v05)
- `wind_stress` f — `pinneapple_systems.time_series.enso.data` (v06)
- `wind_turbine` f — `pinneapple_twin3d.gallery` (v06)
- `WindField` C — `pinneapple_simulation.flight.turbulence` (v03)
- `Winding` C — `pinneapple_simulation.electromagnetics.machines.pmsm` (v03)
- `window_auc` f — `pinneapple_security.cps` (v05)
- `window_features` f — `pinneapple_systems.time_series.features.engineering` (v06)
- `window_mask` f — `pinneapple_physics.blackhole.convergence` (v02)
- `window_mass` f — `pinneapple_physics.blackhole.forecast` (v02)
- `window_mass` f — `pinneapple_physics.blackhole.skill` (v02)
- `window_volumes` f — `pinneapple_physics.blackhole.dataset` (v02)
- `windowed_balance` f — `pinneapple_systems.operations.anomaly` (v06)
- `windowed_elm` f — `pinneapple_lab.experiments.discovery` (v07)
- `WindowedTimeSeriesDataset` C — `pinneapple_systems.time_series.datasets.windowed` (v06)
- `WindTunnelLBM` C — `pinneapple_simulation.numerical_solvers.lbm_tunnel` (v03)
- `WindTunnelSetup` C — `pinneapple_simulation.external_solvers.fluidx3d` (v03)
- `wing` f — `pinneapple_design.geometry_library.kit` (v06)
- `wing` f — `pinneapple_twin3d.primitives` (v06)
- `wing_figures` f — `pinneapple_design.intent.figures` (v06)
- `wing_weight` f — `pinneapple_design.aero.aircraft3d` (v06)

- wing_weight_transport f — pinneapple_design.aero.airliner3d (v06)
- winkler_parabolic_contact f — pinneapple_physics.tribology.archard (v02)
- WinsorizeStep C — pinneapple_neural.trainer.preprocess (v04)
- wire_az f — pinneapple_physics.closed_form.electromagnetics (v02)
- wire_field f — pinneapple_simulation.electromagnetics.analytic (v03)
- with_loop f — pinneapple_systems.digital_twin.pipe_loop.plant (v06)
- with_mirror f — pinneapple_design.geometry_library.kit (v06)
- with_params f — pinneapple_systems.operations.plant (v06)
- with_params f — pinneapple_systems.sports.football.pc_study (v06)
- with_quick f — pinneapple_flow.examples (v07)
- with_units f — pinneapple_physics.pde_environment.presets.astropy_bridge (v02)
- wltc_class3b f — pinneapple_simulation.electromagnetics.machines.drive (v03)
- wod_d20_casts f — pinneapple_systems.time_series.enso.data (v06)
- WongReeceModel C — pinneapple_simulation.terramechanics.wong_reece (v03)
- words f — pinneapple_data.simulation_metadata.foam_dict (v04)
- worker_commands f — pinneapple_simulation.external_solvers.openfoam.campaign (v03)
- Workflow C — pinneapple_core.workflows.dag (v01)
- Workflow C — pinneapple_tools.automation.workflow (v07)
- workflow_from_cases f — pinneapple_core.workflows.dag (v01)
- WorkflowError C — pinneapple_core.workflows.dag (v01)
- WorkflowError C — pinneapple_tools.automation.workflow (v07)
- WorkflowRunner C — pinneapple_core.workflows.engine (v01)
- working_method f — pinneapple_core.onboarding.catalog (v01)
- Workspace C — pinneapple_analysis.benchmark.runner (v05)
- Workspace C — pinneapple_llm.workbench.tools (v07)
- WorldModel C — pinneapple_worldmodel.physical_world (v04)
- WorldModelConfig C — pinneapple_worldmodel.model (v04)
- WorldModelDataset C — pinneapple_worldmodel.dataset (v04)
- WorldModelLoss C — pinneapple_worldmodel.trainer (v04)
- WorldModelTrainConfig C — pinneapple_worldmodel.trainer (v04)
- WorldModelTrainer C — pinneapple_worldmodel.trainer (v04)
- worst_case f — pinneapple_simulation.electromagnetics.periodic.benchmark (v03)
- worst_cells f — pinneapple_data.cae.report (v04)
- worst_n_points f — pinneapple_tools.model_evaluation.metrics (v07)
- wrap f — pinneapple_simulation.flight.control (v03)
- wrap_data_parallel f — pinneapple_neural.trainer.parallel (v04)
- wrap_for_uq f — pinneapple_neural.physicsnemo_bridge.adapters (v04)
- wrap_fsdp f — pinneapple_neural.trainer.hpc (v04)
- wrap_zero_optimizer f — pinneapple_neural.trainer.hpc (v04)
- wrapped f — pinneapple_flow.libcalls (v07)
- write f — pinneapple_data.unified (v04)

- `write` f — `pinneapple_flow.autocalls` (v07)
- `write` f — `pinneapple_science.report` (v02)
- `write_all` f — `pinneapple_science.report` (v02)
- `write_baseline` f — `pinneapple_check.baseline` (v05)
- `write_calculix_cantilever` f — `pinneapple_simulation.external_solvers.campaign_cases` (v03)
- `write_calculix_from_template` f — `pinneapple_simulation.external_solvers.campaign_cases` (v03)
- `write_campaign` f — `pinneapple_simulation.external_solvers.campaign_cases` (v03)
- `write_case` f — `pinneapple_design.aero.case` (v06)
- `write_case3d` f — `pinneapple_design.aero.case3d` (v06)
- `write_case_tables` f — `pinneapple_simulation.external_solvers.campaign_cases` (v03)
- `write_catalogue` f — `pinneapple_presets.report` (v02)
- `write_catalogue` f — `pinneapple_science.report` (v02)
- `write_cavity` f — `pinneapple_simulation.accelerated_foam.case` (v03)
- `write_csv` f — `pinneapple_analysis.benchmark.report` (v05)
- `write_csv` f — `pinneapple_data.cae.dataset` (v04)
- `write_dataset` f — `pinneapple_hub.zoo.parquet` (v07)
- `write_decompose` f — `pinneapple_simulation.external_solvers.openfoam.campaign` (v03)
- `write_design_files` f — `pinneapple_design.design_optimizer.report` (v06)
- `write_engine_anim` f — `pinneapple_simulation.external_solvers.openradioss.deck` (v03)
- `write_field_dataset` f — `pinneapple_design.aero.field_dataset` (v06)
- `write_flow_fields` f — `pinneapple_simulation.numerical_solvers.external_flow` (v03)
- `write_for_run` f — `pinneapple_lab.manifest` (v07)
- `write_frd` f — `pinneapple_simulation.external_solvers.calculix.frd` (v03)
- `write_geo` f — `pinneapple_simulation.electromagnetics.machines.getdp` (v03)
- `write_geo_3d` f — `pinneapple_simulation.electromagnetics.machines.getdp` (v03)
- `write_gif` f — `pinneapple_design.bricks.visuals` (v06)
- `write_gif` f — `pinneapple_simulation.vehicle.viz` (v03)
- `write_gif` f — `pinneapple_tools.visualization.dashboard.static` (v07)
- `write_glb` f — `pinneapple_tools.visualization.studio.scene` (v07)
- `write_gltf` f — `pinneapple_design.bricks.mesh` (v06)
- `write_hdf5` f — `pinneapple_data.cae.dataset` (v04)
- `write_html` f — `pinneapple_design.intent.package` (v06)
- `write_html` f — `pinneapple_lab.html` (v07)
- `write_html` f — `pinneapple_tools.visualization.dashboard.html` (v07)
- `write_index` f — `pinneapple_hub.library` (v07)
- `write_index` f — `pinneapple_presets.registry` (v02)
- `write_inp` f — `pinneapple_simulation.external_solvers.calculix.inp` (v03)
- `write_inputs` f — `pinneapple_veriphysics.validation_lab` (v05)
- `write_instructions` f — `pinneapple_design.bricks.visuals` (v06)
- `write_internal_case` f — `pinneapple_simulation.pipelines.freecad_openfoam.case` (v03)

- `write_internal_mesh` f — `pinneapple_simulation.pipelines.freecad_openfoam.mesh` (v03)
- `write_json` f — `pinneapple_data.cae.dataset` (v04)
- `write_ldr` f — `pinneapple_design.bricks.ldraw` (v06)
- `write_manifest` f — `pinneapple_design.geometry_library.store` (v06)
- `write_markdown_block` f — `pinneapple_simulation.electromagnetics.verification` (v03)
- `write_motion_stage` f — `pinneapple_worldmodel.robot_perception` (v04)
- `write_mp4` f — `pinneapple_design.bricks.visuals` (v06)
- `write_mpd` f — `pinneapple_design.bricks.ldraw` (v06)
- `write_npz` f — `pinneapple_data.cae.dataset` (v04)
- `write_npz` f — `pinneapple_physics.blackhole.dataset` (v02)
- `write_npz` f — `pinneapple_simulation.numerical_solvers.operator_datasets` (v03)
- `write_openfoam_airfoil` f — `pinneapple_simulation.external_solvers.campaign_cases` (v03)
- `write_openfoam_from_base` f — `pinneapple_simulation.external_solvers.campaign_cases` (v03)
- `write_parquet` f — `pinneapple_data.cae.dataset` (v04)
- `write_particle_frame` f — `pinneapple_simulation.particle_dynamics.geo_mpm` (v03)
- `write_pdf` f — `pinneapple_design.intent.package` (v06)
- `write_pinneapple` f — `pinneapple_data.cae.dataset` (v04)
- `write_ply` f — `pinneapple_blender.export` (v07)
- `write_png` f — `pinneapple_tools.visualization.dashboard.static` (v07)
- `write_polymesh` f — `pinneapple_design.aero.mesh` (v06)
- `write_pro` f — `pinneapple_simulation.electromagnetics.machines.getdp` (v03)
- `write_pro_3d` f — `pinneapple_simulation.electromagnetics.machines.getdp` (v03)
- `write_profile` f — `pinneapple_simulation.numerical_solvers.free_surface` (v03)
- `write_python_runner` f — `pinneapple_simulation.external_solvers.campaign_cases` (v03)
- `write_report` f — `pinneapple_autolab.report` (v07)
- `write_report` f — `pinneapple_design.bricks.report` (v06)
- `write_report` f — `pinneapple_design.fitmesh.report` (v06)
- `write_report` f — `pinneapple_design.thermal.board_cooling.report` (v06)
- `write_report` f — `pinneapple_lab.reports` (v07)
- `write_report` f — `pinneapple_presets.report` (v02)
- `write_report` f — `pinneapple_reason` (v05)
- `write_report` f — `pinneapple_reason.report` (v05)
- `write_report` f — `pinneapple_simulation.rover.autolab.report` (v03)
- `write_report` f — `pinneapple_systems.digital_twin.pipe_loop.report` (v06)
- `write_report` f — `pinneapple_tools.automation.report` (v07)
- `write_run` f — `pinneapple_hub.library` (v07)
- `write_run` f — `pinneapple_tools.benchmark_suite.crossframework.records` (v07)
- `write_run_artifacts` f — `pinneapple_tools.benchmark_suite.runner.report` (v07)

- write_sample_dict_cloud f — pinneapple_simulation.external_solvers.openfoam.sampling (v03)
- write_setup f — pinneapple_simulation.external_solvers.fluidx3d (v03)
- write_sidecar f — pinneapple_data.contracts.tables (v04)
- write_site f — pinneapple_lab.html (v07)
- write_split_files f — pinneapple_physics.blackhole.dataset (v02)
- write_static f — pinneapple_lab.explorer (v07)
- write_stl f — pinneapple_design.bricks.mesh (v06)
- write_stl f — pinneapple_simulation.pipelines.freecad_openfoam.cad (v03)
- write_stl f — pinneapple_tools.visualization.studio.scene (v07)
- write_study_result f — pinneapple_tools.automation.report (v07)
- write_table f — pinneapple_data.contracts.tables (v04)
- write_table f — pinneapple_simulation.electromagnetics.antenna.runner (v03)
- write_usd f — pinneapple_design.bricks.mesh (v06)
- write_usda f — pinneapple_tools.visualization.studio.scene (v07)
- write_variation_stage f — pinneapple_worldmodel.inspection_data (v04)
- write_video f — pinneapple_tools.visualization.dashboard.static (v07)
- write_vtk f — pinneapple_simulation.numerical_solvers.lbm_tunnel (v03)
- write_vtu f — pinneapple_data.cae.dataset (v04)
- write_vtu f — pinneapple_simulation.electromagnetics.machines.export (v03)
- Written C — pinneapple_flow.nodes_ext.examples (v07)

X

- xavier_uniform_init f — pinneapple_tools.benchmark_suite.crossframework.weights (v07)
- xd_fit f — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- XdotoolDriver C — pinneapple_tools.automation.display (v07)
- XDResult C — pinneapple_physics.pde_environment.presets.astroml_bridge (v02)
- XGBoostForecaster C — pinneapple_systems.time_series.models.classical (v06)
- XGParams C — pinneapple_systems.sports.football.xg (v06)
- XPINN C — pinneapple_neural.architectures.pinns.xpinn (v04)
- XTFC C — pinneapple_neural.architectures.pinns.xtfc (v04)
- xtfc_case f — pinneapple_physics.paper_problems_xtfc (v02)
- xtfc_pde f — pinneapple_flow.wrappers (v07)
- xtfc_pde_predict f — pinneapple_simulation.numerical_solvers.xtfc_pde (v03)
- xtfc_predict f — pinneapple_simulation.numerical_solvers.xtfc_ivp (v03)
- XTFCCase C — pinneapple_physics.paper_problems_xtfc (v02)
- XTFCConfig C — pinneapple_neural.architectures.pinns.xtfc (v04)

- `XTFCFactoryModel` C — `pinneapple_neural.architectures.pinns.xtfc` (v04)
- `XTFCIVPSolver` C — `pinneapple_simulation.numerical_solvers.xtfc_ivp` (v03)
- `XTFCPDESolver` C — `pinneapple_simulation.numerical_solvers.xtfc_pde` (v03)
- `XTFCProblem` C — `pinneapple_simulation.numerical_solvers.xtfc_general` (v03)
- `XTFCResult` C — `pinneapple_simulation.numerical_solvers.xtfc_general` (v03)
- `XTFCSubdomainSolver` C — `pinneapple_simulation.numerical_solvers.xtfc_subdomain` (v03)

Y

- `yang_comparison` f — `pinneapple_simulation.electromagnetics.machines.verification` (v03)
- `YeeGrid` C — `pinneapple_simulation.electromagnetics.fdtd` (v03)
- `youtube_daily_aggregates` f — `pinneapple_systems.time_series.datasets.public` (v06)
- `YouTubeDaily` C — `pinneapple_systems.time_series.datasets.public` (v06)
- `yplus_check` f — `pinneapple_analysis.benchmark.validation` (v05)
- `YPlusEstimate` C — `pinneapple_analysis.verification.mesh_intelligence` (v05)

Z

- `ZarrByteCacheConfig` C — `pinneapple_data.zarr_cached_store_bytes` (v04)
- `ZarrCacheConfig` C — `pinneapple_data.zarr_cached_store` (v04)
- `ZarrUPDIterable` C — `pinneapple_data.zarr_iterable` (v04)
- `zipper` f — `pinneapple_simulation.electromagnetics.mesh` (v03)
- `ZoneForecast` C — `pinneapple_twin3d.wear.forecast` (v06)
- `zones` f — `pinneapple_neural.architectures.rom.views` (v04)
- `ZoneSpec` C — `pinneapple_twin3d.wear.model` (v06)
- `ZoneState` C — `pinneapple_twin3d.wear.optimize` (v06)
- `zoo` f — `pinneapple_systems.time_series.enso.study` (v06)
- `zoo_evaluate` f — `pinneapple_flow.nodes_ext.validation` (v07)
- `zoo_list` f — `pinneapple_mcp.tools` (v07)
- `zoo_predict` f — `pinneapple_mcp.tools` (v07)
- `zoo_problem` f — `pinneapple_design.design_optimizer.spec` (v06)
- `zoo_range` f — `pinneapple_flow.nodes_ext.validation_gallery` (v07)
- `zoo_service` f — `pinneapple_tools.model_services.registry` (v07)
- `zoo_surrogate` f — `pinneapple_flow.nodes.vehicle` (v07)
- `zoo_validate` f — `pinneapple_mcp.tools` (v07)
- `ZooContactSurrogate` C — `pinneapple_simulation.terramechanics.surrogate` (v03)
- `ZooEntry` C — `pinneapple_worldmodel.model_zoo` (v04)
- `ZooError` C — `pinneapple_hub.zoo._artifacts` (v07)
- `ZooEvaluator` C — `pinneapple_design.design_optimizer.evaluators` (v06)
- `ZooModel` C — `pinneapple_hub.zoo._models` (v07)
- `ZScoreDetector` C — `pinneapple_systems.digital_twin.monitoring.anomaly` (v06)

- `zwietering_njs` f — `pinneapple_simulation.numerical_solvers.particles` (v03)

2. Glossário

termo	significado
Adjunto	Método que calcula o gradiente de um objetivo em relação a muitos parâmetros com custo de uma simulação extra.
Assimilação de dados	Combinar um modelo com medições ao longo do tempo (por exemplo, filtro de Kalman, EnKF).
Autolab	Laboratório autônomo: planeja e executa missões com ferramentas controladas, compara surrogates e escolhe a próxima simulação.
Co-simulação	Acoplar simuladores diferentes que trocam dados a cada passo de tempo (por exemplo, via FMI/FMU).
Condensação (Schur)	Eliminar os graus de liberdade internos de um bloco, ficando só com a interface; reaproveitado entre blocos idênticos.
Condição de contorno (BC)	Valor ou fluxo imposto na fronteira do domínio. Dirichlet fixa o valor; Neumann fixa a derivada normal; Robin combina os dois.
Condição inicial (IC)	Estado do sistema no instante inicial de um problema transiente.
Craig-Bampton	Redução de modelos para dinâmica: modos de interface fixa mais modos de restrição.
Credibilidade	Nível atribuído por pp.validate a cada fator verificado, com decisão final pass / warn / fail.
DeepONet	Operador neural com uma rede 'branch' (entrada funcional) e uma 'trunk' (coordenadas).
EKI	Ensemble Kalman Inversion: método de inversão sem gradiente baseado em ensembles.
Extra (pyproject)	Grupo opcional de dependências instalado com pip install "pinneapple[extra]".
FDM / FEM / FVM	Diferenças finitas, elementos finitos e volumes finitos: famílias clássicas de discretização de equações diferenciais.
Flow	Grafo de nós tipados (JSON pinneapple-flow/1) que encadeia chamadas da biblioteca; executado por pinneapple_flow.
FMI / FMU	Functional Mock-up Interface: padrão para trocar modelos de simulação empacotados (Functional Mock-up Units).
FNO	Fourier Neural Operator: operador neural que mistura informação no espaço de Fourier.
Frente de Pareto	Conjunto de soluções em que nenhum objetivo melhora sem piorar outro.
GNN / MeshGraphNet	Rede de grafos que opera sobre os nós e arestas de uma malha.
Guardrail físico	Verificação que bloqueia propostas (por exemplo, de um LLM) que violam a física ou o escopo.
Gêmeo digital (digital twin)	Modelo de um ativo real que é atualizado por medições e usado para monitorar, prever ou decidir.
Hard BC	Condição de contorno imposta pela própria forma da rede (por construção), e não por um termo de perda.
Hub / zoo	Repositório de modelos treinados e verificados, datasets e biblioteca pesquisável de ferramentas e exemplos.
Lab / selo	pinneapple_lab registra experimentos com manifesto, métricas, verificações e uma nota (A-F); o selo resume essa avaliação.
LBM	Lattice Boltzmann Method: método de fluidos baseado em distribuições de partículas numa rede regular.
MCP	Model Context Protocol: protocolo para expor ferramentas a agentes de IA; pinneapple_mcp é o servidor do PINNeAPPle.
NSGA-II	Algoritmo genético multiobjetivo que encontra uma frente de Pareto.
OOD (fora do domínio)	Entrada fora da região onde o modelo foi treinado ou validado; os resultados ali não são confiáveis.

termo	significado
OpenUSD / Omniverse	Formato de cena 3-D da Pixar e plataforma da NVIDIA que o usa; o PINNeAPPle exporta campos e decisões de confiança para ele.
Operador neural	Rede que aprende um mapa entre funções (por exemplo, condição de contorno → campo solução). Exemplos: FNO, DeepONet, AFNO.
Otimização bayesiana	Otimização com um modelo probabilístico (em geral processo gaussiano) que escolhe o próximo ponto a avaliar.
PhysicalProblem	Classe única do PINNeAPPle que descreve um problema físico para todas as partes da biblioteca.
PhysicsNeMo	Framework da NVIDIA para IA física; o PINNeAPPle confere surrogates feitos com ele.
PINN	Physics-Informed Neural Network: rede neural treinada para satisfazer a equação diferencial (resíduo na perda), além de dados e condições de contorno.
Ponto de colocação (collocation point)	Ponto do domínio onde o resíduo da equação é avaliado durante o treino de uma PINN.
Predição conformal	Método que produz intervalos com cobertura garantida a partir de um conjunto de calibração.
Preset	Problema pronto, com nome, equações, domínio e parâmetros padrão; pode ser carregado e resolvido diretamente.
Problema inverso	Estimar parâmetros ou entradas desconhecidas a partir de observações da saída.
ProblemSpec	Definição executável de uma EDP (coordenadas, campos, termo da EDP, contornos, domínio) consumida pelo compilador de PINN.
Referência independente	Valor de comparação que não vem do próprio código testado: forma fechada, solução manufaturada ou valor publicado/tabelado.
Resíduo	Quanto a solução deixa de satisfazer a equação num ponto. Resíduo zero = equação satisfeita.
ROM	Reduced-Order Model: modelo de ordem reduzida (por exemplo, POD) que aproxima o modelo completo com poucos graus de liberdade.
SDF / CSG	Signed Distance Function (distância com sinal até a fronteira) e Constructive Solid Geometry (união, interseção e diferença de sólidos).
Shim de compatibilidade	Pacote que só reexporta outro, mantido para que imports antigos funcionem.
Solution	Objeto devolvido por pp.solve: valores, pontos, método e metadados (info).
Solução manufaturada	Solução escolhida de antemão, inserida na equação para gerar o termo fonte; serve para verificar a ordem de convergência de um solver.
SPH	Smoothed Particle Hydrodynamics: método sem malha baseado em partículas.
Surrogate (modelo substituto)	Modelo rápido, em geral aprendido, que aproxima um simulador caro.
Teste de Diebold-Mariano (DM)	Teste estatístico que compara a precisão de duas previsões.
Trust card	Cartão de confiança: conjunto de verificações físicas de um campo (resíduo, conservação, contornos, referência, domínio) com decisão APPROVED / REVIEW / REJECT.
UQ	Uncertainty Quantification: quantificação de incerteza (ensembles, conformal, propagação de tolerâncias, Monte Carlo).
validated / tested / untested	Status do catálogo de métodos: validated = algum teste compara contra referência independente; tested = há testes, mas nenhum contra referência; untested = sem testes.
Walk-forward purgado	Protocolo de validação de séries temporais que treina no passado, testa no futuro e remove amostras vizinhas para evitar vazamento.

termo	significado
X-TFC / TFC	Theory of Functional Connections: expressões que satisfazem as restrições exatamente; X-TFC combina isso com redes de camada única (extreme learning machine).

3. Apêndice: números do inventário

item	valor
pacotes `pinneapple*`	41
módulos Python nesses pacotes	2.142
linhas de código nesses módulos	494.775
classes públicas	2.757
funções públicas de nível de módulo	6.130
métodos públicos de classes	6.220
módulos importados por pelo menos um arquivo de teste	1.073
arquivos de teste (`tests/**/test*.py`)	513
funções de teste (`test*`, inclusive em classes `Test*`)	4.611
nomes expostos em `pp.*`	297
extras do `pyproject.toml`	54
apps em `apps/`	64
entradas em `examples/`	88
entradas em `scripts/`	24
arquivos Markdown em `docs/`	190
workflows de CI	12
métodos no catálogo	201

Status dos métodos do catálogo (regra: validated = a test exercising the item compares against an independent reference (closed form, manufactured solution, published/tabulated value); arquivo gerado em 2026-10-17):

status	métodos
tested	47
untested	42
validated	112

3.1 Por pacote

pacote	módulos	linhas	classes	funções	testes*	módulos sem teste
`pinneapple`	3	866	0	2	431	2
`pinneapple_adaptation`	13	2.946	10	9	154	2
`pinneapple_analysis`	159	42.004	249	450	939	56
`pinneapple_app`	18	3.672	29	53	48	11
`pinneapple_arena`	54	21.853	69	243	99	39
`pinneapple_autolab`	16	3.596	38	39	37	10
`pinneapple_blender`	5	616	1	7	15	2
`pinneapple_catalog`	3	1.024	2	8	16	1
`pinneapple_check`	23	4.547	22	71	144	11
`pinneapple_condense`	18	6.006	29	115	27	3
`pinneapple_core`	34	8.747	71	92	233	15
`pinneapple_data`	115	29.019	129	256	507	70

pacote	módulos	linhas	classes	funções	testes*	módulos sem teste
pinneapple_decision	12	2.499	37	20	44	9
pinneapple_design	196	48.153	269	740	744	86
pinneapple_flow	89	18.086	23	381	263	58
pinneapple_hub	31	7.260	22	170	183	8
pinneapple_lab	42	11.775	45	119	96	27
pinneapple_llm	49	10.750	66	99	242	28
pinneapple_mcp	8	1.541	2	31	30	3
pinneapple_models	2	74	0	1	1	1
pinneapple_neural	198	36.217	334	150	438	152
pinneapple_orchestration	16	591	1	11	6	14
pinneapple_pdb	7	1.929	9	14	100	5
pinneapple_perception	7	1.037	6	19	29	2
pinneapple_physics	166	43.959	169	763	1.075	52
pinneapple_presets	21	5.247	22	31	50	16
pinneapple_problemdesign	28	5.642	40	21	123	19
pinneapple_quantum	16	2.636	17	24	0	16
pinneapple_query	3	252	7	2	7	2
pinneapple_reason	18	3.757	17	60	17	8
pinneapple_registry	10	1.699	10	3	92	3
pinneapple_science	16	4.253	33	49	19	10
pinneapple_security	20	3.398	33	93	105	0
pinneapple_simulation	298	66.085	364	931	1.119	113
pinneapple_solvers	2	71	0	0	1	1
pinneapple_systems	182	37.262	301	515	579	78
pinneapple_tools	177	34.350	155	384	465	107
pinneapple_train	4	109	0	0	1	1
pinneapple_twin3d	19	2.820	19	67	74	4
pinneapple_veriphysics	13	3.411	17	38	98	1
pinneapple_worldmodel	31	15.016	90	49	94	23

* funções de teste em arquivos que importam algum módulo do pacote (um teste pode contar em mais de um pacote).

3.2 Extras do `pyproject.toml`

Projeto `pinneapple` versão `0.7.0`, Python `>=3.10`. Dependências obrigatórias: `numpy>=1.24`, `scipy>=1.10`, `pandas>=2.0`, `pyyaml>=6.0`, `tqdm>=4.66`, `typing-extensions>=4.8`, `pydantic>=2.0`, `matplotlib>=3.8`, `torch>=2.2`, `sympy>=1.12`, `xarray`, `zarr`, `tomli>=2.0; python_version < '3.11'`.

extra	pacotes
`cfd`	manifold3d>=2.4, huggingface_hub>=0.20, cadquery>=2.4, gmsh>=4.11, trimesh>=4.0, pyvista>=0.43, plotly>=5.18, meshio>=5.3, h5py>=3.8
`enterprise`	fastapi>=0.111, uvicorn[standard]>=0.29, cryptography>=41, argon2-cffi>=23.1, meshio>=5.3, h5py>=3.8, huggingface_hub>=0.24, pyarrow>=14, safetensors>=0.4, mcp>=1.10
`mcp`	mcp>=1.10
`security`	cryptography>=41, argon2-cffi>=23.1
`check`	meshio>=5.3, h5py>=3.8, cryptography>=41
`sports`	scikit-learn>=1.3
`em`	miepython>=3.0
`veriphysics`	reportlab>=4.0
`viz3d`	rerun-sdk>=0.20, plotly>=5.18, trame>=3.0, trame-vtk>=2.5, trame-vuetify>=2.4
`twin`	basyx-python-sdk>=1.0, asyncua>=1.1
`geometry`	manifold3d>=2.4, huggingface_hub>=0.20
`fitmesh`	cadquery>=2.4, gmsh>=4.11, trimesh>=4.0
`newton`	newton>=1.6
`sky`	sgp4>=2.22, pymsis>=0.10, skyfield-data>=6.0, skyfield>=1.48
`astroml`	astroML>=1.0
`fmi`	fmpy>=0.3.20, pythonfmu>=0.6
`tracking`	mlflow>=2.9, wandb>=0.16
`geom`	trimesh>=4.0.0, meshio>=5.3.0, gmsh>=4.11
`geom_simplify`	fast-simplification>=0.1.0, open3d>=0.18.0
`cad`	cadquery>=2.4.0
`solvers`	numba>=0.59
`process`	CoolProp>=6.6
`astro`	astropy>=6.0
`jax_cfd`	jax-cfd>=0.2.0
`fenics`	fenics-dolfinx>=0.7.0, mpi4py>=3.1, petsc4py>=3.20

extra	pacotes
`mujoco`	mujoco>=3.0
`mujoco_mjx`	mujoco>=3.0, jax[cuda12]>=0.4.25
`genesis`	genesis-world>=0.2.0
`noether`	emmiai-noether>=2026.4.0
`pinn`	sympy>=1.12
`uq`	scipy>=1.10
`serve`	fastapi>=0.111, uvicorn[standard]>=0.29
`export`	onnx>=1.16, onnxruntime>=1.18
`hub`	huggingface_hub>=0.24, pyarrow>=14, safetensors>=0.4, onnx>=1.16, onnxruntime>=1.18
`flow`	sqlalchemy>=2.0, sympy>=1.12, fastapi>=0.111, uvicorn[standard]>=0.29, httpx>=0.27, pillow>=10
`llm`	anthropic>=0.34, openai>=1.40
`orchestration`	prefect>=2.19
`tuning`	optuna>=3.6
`workbench`	fastapi>=0.110, uvicorn>=0.29, requests>=2.31
`finetune`	peft>=0.12, datasets>=2.20, transformers>=4.44
`vlm_curation`	transformers>=4.45, accelerate>=0.30
`digital_twin`	paho-mqtt>=2.0, kafka-python>=2.0, asyncua>=1.1, pymodbus>=3.6
`timeseries`	statsmodels>=0.14, pmdarima>=2.0, scikit-learn>=1.4, PyWavelets>=1.6
`synth`	sympy>=1.12, trimesh>=4.0.0, meshio>=5.3.0
`earthdata`	requests>=2.31, earthaccess>=0.10.0
`research`	requests>=2.31, pdfminer.six>=20231228
`retrieval`	sentence-transformers>=3.0

extra	pacotes
`all`	cryptography>=41, trimesh>=4.0.0, meshio>=5.3.0, gmsh>=4.11, fast-simplification>=0.1.0, cadquery>=2.4.0, numba>=0.59, sympy>=1.12, fastapi>=0.111, uvicorn[standard]>=0.29, onnx>=1.16, onnxruntime>=1.18, paho-mqtt>=2.0, kafka-python>=2.0, sentence-transformers>=3.0, statsmodels>=0.14, requests>=2.31, mujoco>=3.0, fmpy>=0.3.20, genesis-world>=0.2.0, emmiai-noether>=2026.4.0, huggingface_hub>=0.24, anthropic>=0.34, openai>=1.40, transformers>=4.45, accelerate>=0.30
`viz`	matplotlib>=3.8, scipy>=1.10
`viz_interactive`	plotly>=5.18, kaleido>=0.2
`pyvista`	pyvista>=0.43, matplotlib>=3.8, imageio>=2.34
`cost`	psutil>=5.9
`surrogate`	scikit-learn>=1.4, pyarrow>=14, h5py>=3.8, onnx>=1.15, onnxruntime>=1.17
`dev`	ruff>=0.5, pytest>=8.0, pytest-cov>=4.1, mypy>=1.8, build>=1.2, twine>=5.0, ipykernel>=6.0, httpx>=0.27, jupyter>=1.0

3.3 Catálogo completo de métodos

id	método	status	testes contra referência	outros testes	código
E1	Laplace	validated	7	28	
E2	Poisson	validated	15	24	
E3	Helmholtz	validated	2	1	
E4	Heat (steady anisotropic / transient)	validated	4	13	
E5	Wave	validated	1	0	
E6	Advection-diffusion / convection	validated	1	0	

id	método	status	testes contra referência	outros testes	código
E7	Burgers (viscous / inviscid)	validated	10	50	
E8	Reaction-diffusion 1D/2D (Gray-Scott)	validated	2	1	
E9	Fisher-KPP	validated	1	3	
E10	Incompressible Navier-Stokes 2D/3D	tested	0	13	
E11	Navier-Stokes in a rotating frame	tested	0	2	
E12	Stokes	validated	1	1	
E13	Brinkman	validated	1	0	
E14	Darcy	tested	0	12	
E15	Buckley-Leverett two-phase	validated	1	0	
E16	Compressible Euler 1D/2D/axisymmetric/rotating 3D	validated	3	1	
E17	Shallow water 1D/2D	validated	1	6	
E18	Stommel gyre	validated	2	0	
E19	Plane-stress / plane-strain linear elasticity	validated	2	5	

id	método	status	testes contra referência	outros testes	código
E20	Axisymmetric elasticity (+ torsion) / Lame	validated	1	4	
E21	Neo-Hookean hyperelasticity	validated	1	0	
E22	Euler-Bernoulli beam	validated	1	11	pinneapple_systems/process_components/beam_statics.py
E23	von Karman beam / plate	validated	2	6	pinneapple_systems/process_components/beam_nonlinear_fem.py
E24	Thermoelasticity	validated	1	0	
E25	Phase-field fracture	untested	0	0	
E26	Biot poroelasticity	validated	1	0	
E27	Maxwell TE	validated	1	0	
E28	Bekker-Wong terramechanics	validated	1	2	
E29	Compressor meanline 1D	validated	2	3	
E30	Phonon Boltzmann transport 1D (gray)	validated	3	1	
E31	Black-Scholes	validated	1	2	

id	método	status	testes contra referência	outros testes	código
E32	Heston	validated	2	1	
E33	Opinion dynamics (continuum Hegselmann-Krause)	validated	1	0	
E34	Kepler two-body orbit	validated	1	6	
E35	Circular restricted three-body problem (planar, synodic)	validated	3	3	
E36	J2 perturbation	validated	1	1	
E37	Clohessy-Wiltshire relative motion	validated	1	1	
E38	Euler rigid-body attitude	validated	1	1	
E39	Lane-Emden polytrope	validated	5	19	
E40	Shakura-Sunyaev accretion disk	validated	6	4	
E41	Schwarzschild weak-field light bending	validated	2	3	
E42	SIR epidemic	validated	1	0	
E43	Pharmacokinetics two-compartment	validated	1	0	

id	método	status	testes contra referência	outros testes	código
E44	Reaction network / Arrhenius / mass action	tested	0	10	pinneapple_systems/process_components/reaction_kinetics.py
E45	Hagen-Poiseuille	tested	0	2	PINNeAPPle-CFD/pinneapple_cfd/etapas/e02_modelo_matematico/correlacoes.py
E46	Blasius friction factor	validated	2	10	PINNeAPPle-CFD/…/correlacoes.py
E47	Colebrook-White	tested	0	8	PINNeAPPle-CFD/…/correlacoes.py, pinneapple_systems/process_components/pipe_network_1d.py
E48	Darcy-Weisbach pressure drop	untested	0	0	PINNeAPPle-CFD/…/correlacoes.py
E49	Herschel-Bulkley + Metzner-Reed (non-Newtonian pipe flow)	tested	0	4	pinneapple_systems/process_components/non_newtonian_pipe_flow.py
E50	DNV-RP-O501 erosion (straight pipe and bend)	untested	0	0	PINNeAPPle-CFD/pinneapple_cfd/etapas/e02_modelo_matematico/erosao.py
E51	Lame thick-walled cylinder	tested	0	3	pinneapple_systems/process_components/pipe_stress_mechanics.py
E52	von Mises equivalent stress	tested	0	11	pinneapple_systems/process_components/pipe_stress_mechanics.py, pinneapple_systems/process_components/beam_statics.py
E53	Euler buckling + constrained rod buckling	validated	1	6	pinneapple_systems/process_components/pipe_stress_mechanics.py
E54	Beam-column moment amplification factor	tested	0	3	pinneapple_systems/process_components/pipe_stress_mechanics.py
E55	S-N curve + Goodman + Miner	validated	1	14	pinneapple_systems/process_components/fatigue_analysis.py, pinneapple_physics/closed_form/cantilever_fatigue.py
E56	Torsional stick-slip with Stribeck friction	tested	0	6	pinneapple_systems/process_components/torsional_stickslip.py
E57	Heat exchanger (effectiveness-NTU)	tested	0	8	pinneapple_systems/process_components/heat_exchanger.py
E58	Fin array conduction	tested	0	4	pinneapple_physics/closed_form/fin_array_conduction.py
E59	Control valve (Cv, compressible / incompressible)	tested	0	1	pinneapple_systems/process_components/control_valve.py

id	método	status	testes contra referência	outros testes	código
E60	Polytropic compression path	tested	0	1	pinneapple_systems/process_components/polytropic_path.py
E61	Real gas equation of state (CoolProp)	tested	0	2	pinneapple_systems/process_components/real_gas_eos.py
E62	Compressor similarity map	validated	2	0	pinneapple_systems/process_components/similarity_map.py
E63	Helmholtz resonator	tested	0	4	pinneapple_physics/closed_form/helmholtz_resonator.py
E64	Membrane diffusion	tested	0	5	pinneapple_physics/closed_form/membrane_diffusion.py
E65	Cantilever fatigue	validated	1	4	pinneapple_physics/closed_form/cantilever_fatigue.py
E66	Electrostatics (Gauss's law with permittivity), differential and integral form	validated	1	0	
E67	Magnetostatics in the vector potential A_z	validated	1	0	
E68	Magneto-quasistatic eddy currents (time-harmonic A_z)	validated	1	0	
E69	Maxwell TMz time domain, differential and integral (Ampere-Maxwell circulation, Faraday) form	validated	1	0	
E70	Maxwell's curl equations in 3-D with optional Gauss constraints	validated	1	0	
E71	Frequency-domain TMz wave equation with complex permittivity	validated	1	0	
P9	PINNeAPPle-CFD v1: pipes + particle erosion (CFD-04 pilot)	untested	0	0	PINNeAPPle-CFD/pinneapple_cfd/
P1.1		validated	9	43	

id	método	status	testes contra referência	outros testes	código
P1.2		tested	0	4	
P1.3		untested	0	0	
P1.4		untested	0	0	
P1.5		validated	6	15	
P1.6		untested	0	0	
P1.7		tested	0	7	
P1.8		untested	0	0	
P1.9		validated	4	15	
P2.1		untested	0	0	
P2.2		tested	0	5	
P2.3		tested	0	1	
P2.4		untested	0	0	
P2.5		untested	0	0	
P2.6		validated	1	0	
P2.7		untested	0	0	

id	método	status	testes contra referência	outros testes	código
P2.8		tested	1	0	
P2.9		tested	0	1	
P3.1		untested	0	0	
P3.2		untested	0	0	
P3.3		validated	1	3	
P3.4		untested	0	0	
P3.5		untested	0	0	
P3.6		untested	0	0	
P3.7		untested	0	0	
P4.1		untested	0	0	
P4.2		validated	1	2	
P4.3		validated	1	0	
P4.4		tested	0	1	
P5.1		validated	2	6	

id	método	status	testes contra referência	outros testes	código
P5.2		untested	0	0	
P5.3		untested	0	0	
P5.4		validated	1	0	
P5.5		untested	0	0	
P5.6		validated	1	2	
P5.7		validated	1	5	
P5.8		validated	1	0	
P6.1		validated	1	2	
P6.2		untested	0	0	
P6.3		untested	0	0	
P6.4		validated	1	0	
P6.5		untested	0	0	
P6.6		tested	2	1	

id	método	status	testes contra referência	outros testes	código
P6.7		validated	1	0	
P6.8		validated	1	0	
P6.9		untested	0	0	
P7.1		validated	3	2	
P7.2		validated	1	2	
P7.3		validated	2	8	
P7.4		validated	1	1	
P7.5		validated	1	1	
P7.6		validated	1	2	
P7.7		validated	1	3	
P7.8		untested	0	0	
P7.9		validated	1	1	

id	método	status	testes contra referência	outros testes	código
P8.1		validated	1	2	
P1.10		validated	2	1	
P1.11		tested	0	1	
P1.12		untested	0	0	
P1.13		untested	0	0	
P2.10		untested	0	0	
P2.11		untested	0	0	
P2.12		untested	0	0	
P2.13		untested	0	0	
P7.10		validated	1	1	
S1	Finite differences 1D/2D/3D	validated	2	11	pinneapple_simulation/numerical_solvers/fdm.py, pinneapple_simulation/numerical_solvers/fdm3d.py
S2	Finite elements (linear) + nonlinear beam FEM	validated	6	15	pinneapple_simulation/numerical_solvers/fem.py, pinneapple_simulation/numerical_solvers/nonlinear_beam_fem.py
S3	Finite volumes	validated	2	2	pinneapple_simulation/numerical_solvers/fvm.py

id	método	status	testes contra referência	outros testes	código
S4	Spectral (Fourier pseudo-spectral)	validated	3	2	pinneapple_simulation/numerical_solvers/spectral.py
S5	Lattice Boltzmann D2Q9 / D3Q19 (+ Smagorinsky LES)	validated	2	21	pinneapple_simulation/numerical_solvers/lbm.py
S6	SPH / ISPH / DFSPH	untested	0	0	pinneapple_simulation/numerical_solvers/sph.py, pinneapple_simulation/numerical_solvers/isph.py, pinneapple_simulation/numerical_solvers/dfsph.py
S7	Meshfree RBF collocation (Kansa)	validated	1	1	pinneapple_simulation/numerical_solvers/meshfree.py
S8	X-TFC (IVP, PDE, subdomains)	validated	4	8	pinneapple_simulation/numerical_solvers/xtfc_ivp.py, pinneapple_simulation/numerical_solvers/xtfc_pde.py, pinneapple_simulation/numerical_solvers/xtfc_subdomain.py
S9	Discrete-time PINN / implicit Runge-Kutta Gauss-Legendre	tested	0	2	pinneapple_simulation/numerical_solvers/discrete_time_pinn.py, pinneapple_simulation/numerical_solvers/irk_gauss_legendre.py
S10	Stiff kinetics ODE	validated	1	0	pinneapple_simulation/numerical_solvers/stiff_kinetics_ode.py
S11	TVD advection	validated	1	0	pinneapple_simulation/numerical_solvers/tvd_advection.py
S12	Immersed boundary (FDM)	validated	1	1	pinneapple_simulation/numerical_solvers/immersed_boundary_fdm.py
S13	Beam BVP FDM / 3D elasticity FDM	validated	2	0	pinneapple_simulation/numerical_solvers/beam_bvp_fdm.py, pinneapple_simulation/numerical_solvers/elasticity3d_fdm.py
S14	Eddy current FDM	validated	3	3	pinneapple_simulation/numerical_solvers/eddy_current_fdm.py
S15	Bekker-Wong terramechanics	validated	1	19	pinneapple_simulation/numerical_solvers/bekker_wong.py, pinneapple_simulation/particle_dynamics/terramechanics.py

id	método	status	testes contra referência	outros testes	código
S16	N-body (REBOUND)	validated	1	2	pinneapple_simulation/numerical_solvers/nbody_rebound.py
S17	JAX-CFD	tested	0	3	pinneapple_simulation/numerical_solvers/jax_cfd_solver.py
S18	Particle dynamics: rigid body, MPM, molecular dynamics	tested	0	13	pinneapple_simulation/particle_dynamics/
S19	Signal decomposition: CEEMDAN, EEMD, VMD, HHT, SSA, SST, STL, FFT, wavelet	tested	0	1	
S20	Co-simulation (graph + engine, algebraic loops by Tarjan SCC)	tested	0	11	pinneapple_systems/cosimulation/
S21	Digital twin state estimation: KF/EKF/UKF/EnKF, prognostics, signal reconstruction	tested	2	39	pinneapple_systems/digital_twin/, pinneapple_neural/architectures/classical_ts/
S22	1D process components (pipe network, transient pipe, heat exchanger, valve, reaction, polytropic path, compressor map)	tested	0	3	
S23	Explicit equation-system solver (evaluation order)	tested	0	1	pinneapple_systems/process_components/explicit_equation_system.py
S24	External solvers: OpenFOAM, FEniCS, ANSYS/CFD formats, MATLAB, Modelica/FMI, MuJoCo, Genesis, TurboDesigner	validated	1	80	pinneapple_simulation/external_solvers/
S25	Causal telemetry conditioning (Hampel one-step, rate limit, first-order filter, gaps)	validated	1	6	pinneapple_systems/digital_twin/conditioning.py
S26	2D shallow-water finite volumes (MUSCL + HLL, wet/dry, walls, gates, virtual sensors, health checks)	validated	1	3	pinneapple_simulation/numerical_solvers/shallow_water_fv.py
S27	ETDRK4 exponential integrator for stiff semilinear PDEs (contour phi-functions, blow-up guard)	validated	5	1	pinneapple_simulation/numerical_solvers/etdrk4.py
S28	Stable fluids 2D (semi-Lagrangian advection, spectral projection, vorticity confinement)	validated	2	4	pinneapple_simulation/numerical_solvers/stable_fluids.py
S29	2-D A_z magnetostatics, P1 FEM with Newton (nonlinear B-H, magnets, (anti-)periodic constraints)	validated	1	6	pinneapple_simulation/electromagnetics/fem.py, pinneapple_simulation/electromagnetics/mesh.py, pinneapple_simulation/electromagnetics/materials.py

id	método	status	testes contra referência	outros testes	código
S30	Time-harmonic 2-D eddy currents (A_z, solid conductors with imposed net currents)	validated	1	1	pinneapple_simulation/electromagnetics/eddy.py
S31	Surface-PM machine model: Maxwell-stress and co-energy torque, cogging, back-EMF, losses	validated	1	8	pinneapple_simulation/electromagnetics/machine.py, pinneapple_simulation/electromagnetics/postprocess.py
S32	2-D electrostatics, P1 FEM (capacitance from field energy)	validated	2	0	pinneapple_simulation/electromagnetics/electrostatics.py
S33	Biot-Savart integration of straight filament segments, loop and solenoid fields	validated	3	0	pinneapple_simulation/electromagnetics/statics.py
S34	Mie series for dielectric and perfectly conducting spheres	validated	2	0	pinneapple_simulation/electromagnetics/mie.py
S35	Fresnel coefficients and characteristic matrices of planar multilayers	validated	2	0	pinneapple_simulation/electromagnetics/waves.py
S36	Jiles-Atherton hysteresis model (RK4 along an applied field path)	validated	1	0	pinneapple_simulation/electromagnetics/hysteresis.py
S37	Dispersive media (Drude, Lorentz, Debye) and their auxiliary-differential-equation coefficients	validated	1	0	pinneapple_simulation/electromagnetics/media.py
S38	Maxwell physics checks of predicted fields (Gauss, Faraday, Ampere-Maxwell, continuity, Poynting, interfaces, reciprocity, passivity)	validated	4	2	pinneapple_physics/maxwell_checks.py
S39	Density-based topology optimisation (SIMP, filter, projection, OC, MMA; compliance, thermal, Stokes-Brinkman, machine rotor)	validated	1	2	pinneapple_design/topology/
S40	Yee FDTD in 1-D/2-D/3-D (CPML, Courant check, Drude/Lorentz/Debye ADE, PEC/PMC/periodic/Bloch, TF/SF plane waves, DFT monitors, sub-cell averaging)	validated	4	6	pinneapple_simulation/electromagnetics/fdtd.py
S41	Near-to-far-field transform on a closed box (FDTD radiation patterns)	validated	1	0	pinneapple_simulation/electromagnetics/fdtd.py

id	método	status	testes contra referência	outros testes	código
T1	Base trainer: AMP, gradient accumulation, auto batch size, CUDA graphs, profiler	tested	0	9	pinneapple_neural/trainer/trainer.py, pinneapple_neural/trainer/hpc.py
T2	Distributed: DDP, FSDP, SLURM, torchrun, PowerSGD	untested	0	0	pinneapple_neural/trainer/distributed.py, pinneapple_neural/trainer/hpc.py
T3	Causal training	untested	0	0	pinneapple_neural/trainer/causal.py
T4	Time marching	untested	0	0	pinneapple_neural/trainer/time_marching.py
T5	Two phases Adam -> L-BFGS + L-BFGS finetune	tested	0	6	pinneapple_neural/trainer/two_phase.py, pinneapple_neural/trainer/lbfgs_finetune.py
T6	Multi-restart	untested	0	0	pinneapple_neural/trainer/multistart.py
T7	Adaptive residual-based collocation	untested	0	0	pinneapple_neural/trainer/collocation.py
T8	Loss balancing: ReLoBRaLo, SoftAdapt, PCGrad, Augmented Lagrangian, Inverse Dirichlet, LR Annealing, AutoBalancer	validated	1	6	pinneapple_neural/trainer/loss_balancer.py
T9	Weight schedulers: SA-PINN, GradNorm, LossRatio, NTK	tested	0	2	pinneapple_neural/trainer/weight_scheduler.py
T10	Domain decomposition: XPINN, DoMINO/D3M	tested	0	2	pinneapple_neural/architectures/pinns/xpinn.py, pinneapple_physics/pinn_solver/domino.py
T11	Inverse PINN (learnable physical parameters)	tested	0	3	pinneapple_neural/trainer/inverse_pinn.py
T12	Gray-box + symbolic regression (PySR)	untested	0	0	pinneapple_neural/trainer/graybox.py
T13	Uncertainty: heteroscedastic, B-PINN, MC Dropout, deep ensembles	tested	0	20	pinneapple_neural/trainer/heteroscedastic.py, pinneapple_neural/architectures/pinns/bpinn.py, pinneapple_systems/component_modeling/
T14	SINDy	tested	0	2	pinneapple_neural/architectures/rom/sindy.py
T15	Adaptive hyperparameter sweep / architecture search (arena NAS)	untested	0	0	pinneapple_neural/trainer/adaptive_sweep.py, pinneapple_arena/nas.py

id	método	status	testes contra referência	outros testes	código
T16	Noether trainer (research only)	tested	0	1	pinneapple_neural/trainer/noether_trainer.py
T17	Post-training advisor / audit	untested	0	0	pinneapple_neural/trainer/advisor.py, pinneapple_neural/trainer/audit.py
T18	Meta-learning: MAML, Reptile	tested	0	6	pinneapple_adaptation/meta_learning/
T19	Transfer learning (freeze, adapters, parametric)	tested	0	3	pinneapple_adaptation/transfer_learning/
T20	Self-scaled quasi-Newton: BFGS / SSBFGS / SSBroyden (new, 2026-09-24)	validated	1	3	pinneapple_neural/trainer/self_scaled_qn.py
T21	Autoresearch loop (fixed-budget trials, keep/revert, random or LLM proposer)	tested	0	2	pinneapple_arena/autoresearch.py
T22	Large Physics Model blocks (Fourier encoding, multi-scale neighbourhoods, geometry code, ensemble UQ, OOD score, fine-tuning)	tested	0	1	pinneapple_neural/lpm.py
T23	MultiAdam: per-loss-term Adam moments for multi-term PINN losses	validated	1	3	pinneapple_neural/trainer/multiadam.py

3.4 Como os números foram obtidos

- Tudo vem de `docs/pt/_mapa/inventario.py`, que lê o código com `ast` (nada é importado ou executado).
- *Público* = nome de nível de módulo que não começa com _. Métodos de classes são contados à parte.
- *Testes de um módulo* = funções `test*` em arquivos de `tests/` que importam aquele módulo (`import`, `from … import`, `importlib.import_module`, `pytest.importorskip`). É uma medida de alcance, não de cobertura de linhas.
- O status de validação dos métodos é o de `pinneapple_catalog/method_status.json`, gerado por `scripts/build_method_status.py`.
- Itens ligados a clientes foram excluídos do inventário e do texto.